DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY · THE LAZY PROGRAMMER

PyTorch: Deep Learning and Artificial Intelligence

ANNs, CNNs, RNNs, NLP, recommenders, transfer learning, GANs and deep RL in PyTorch

Volume 1 of 3 · Sections 1–38

38 SECTIONS

528 FIGURES

484 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 GANs (Generative Adversarial Networks) 4
LECTURE 2 Course Outline 16
LECTURE 3 Where to Get the Code 34
LECTURE 4 Git 41
LECTURE 5 Connect Apps to Drive 49
LECTURE 6 File and Google Drive 71
LECTURE 7 The Steps of a Machine Learning Script 89
LECTURE 8 All Data Is the Same 107
LECTURE 9 Finding the Slope and Intercept 126
LECTURE 10 Train the Model (Gradient Descent Loop) 148
LECTURE 11 Synthetic Data and Gaussian Noise 163
LECTURE 12 Exponential Growth of Computing 167
LECTURE 13 Log and Rate of Growth 178
LECTURE 14 Difference: New Loss 181
LECTURE 15 Build the Model 197
LECTURE 16 Test Loss and Train 206
LECTURE 17 State Dict 209
LECTURE 18 Linear Regression 211
LECTURE 19 Gradient Descent 225
LECTURE 20 Binary Cross Entropy and Cross Entropy Loss 239
LECTURE 21 Multiple Validation Sets 241
LECTURE 22 So in this short lecture, I want to... 255
LECTURE 23 Artificial Neural Networks (ANNs) 256
LECTURE 24 Repeating the Single Neuron 264
LECTURE 25 Making the Line More Complicated 277
LECTURE 26 The Vanishing Gradient Problem 288
LECTURE 27 The Softmax Function 311
LECTURE 28 Images as Input to Neural Networks 322

LECTURE 29 Why Does It Work? 337
LECTURE 30 Data 354
LECTURE 31 Original Data Points and Synthetic Data 359
LECTURE 32 Common Sense Things to Try 362
LECTURE 33 The Mechanics of Convolution 372
LECTURE 34 Cosine Similarity 395
LECTURE 35 Alternative Perspectives on Convolution 406
LECTURE 36 Convolution on Color Images 416
LECTURE 37 Different Pool Sizes 432
LECTURE 38 Why Use Custom Models? 464

LECTURE 1

GANs (Generative Adversarial Networks)

Hey everyone and welcome to PyTorch. Deep learning and artificial intelligence. Now this is one of the most exciting courses I've ever made. I've been making these deep learning courses for a long time before PyTorch was even invented. So it's been quite a journey for me to see how this field has evolved in the past few years. And guys it's been moving quick. The best practices and guidelines you'll learn in this course are the latest and greatest.

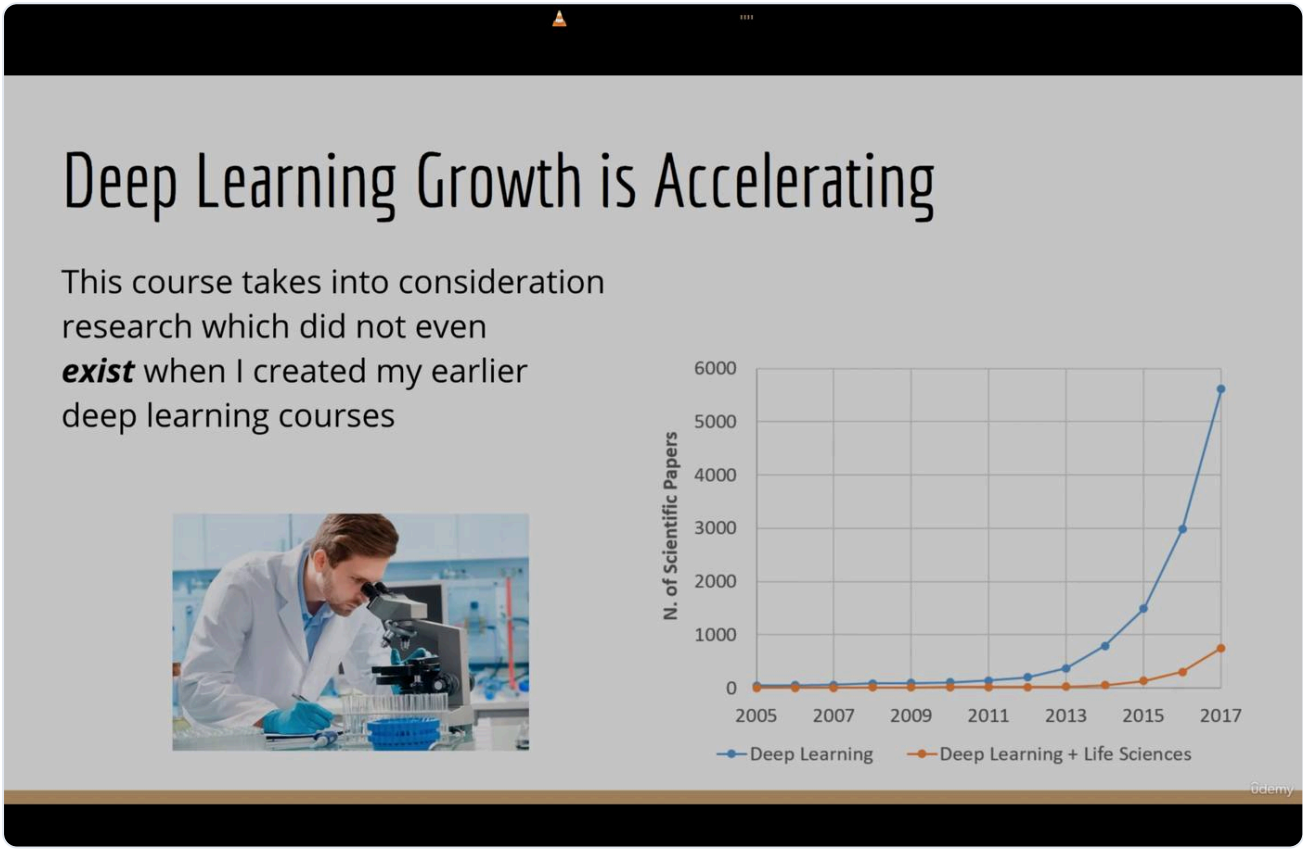


FIGURE 1.1

This stuff did not even exist when some of my previous deep learning courses came out. Things like the best way to train your neural network to architecture decisions. Thanks to the hard work of deep learning researchers all around the world. I've been able to take what they've learned and bring that information to you.

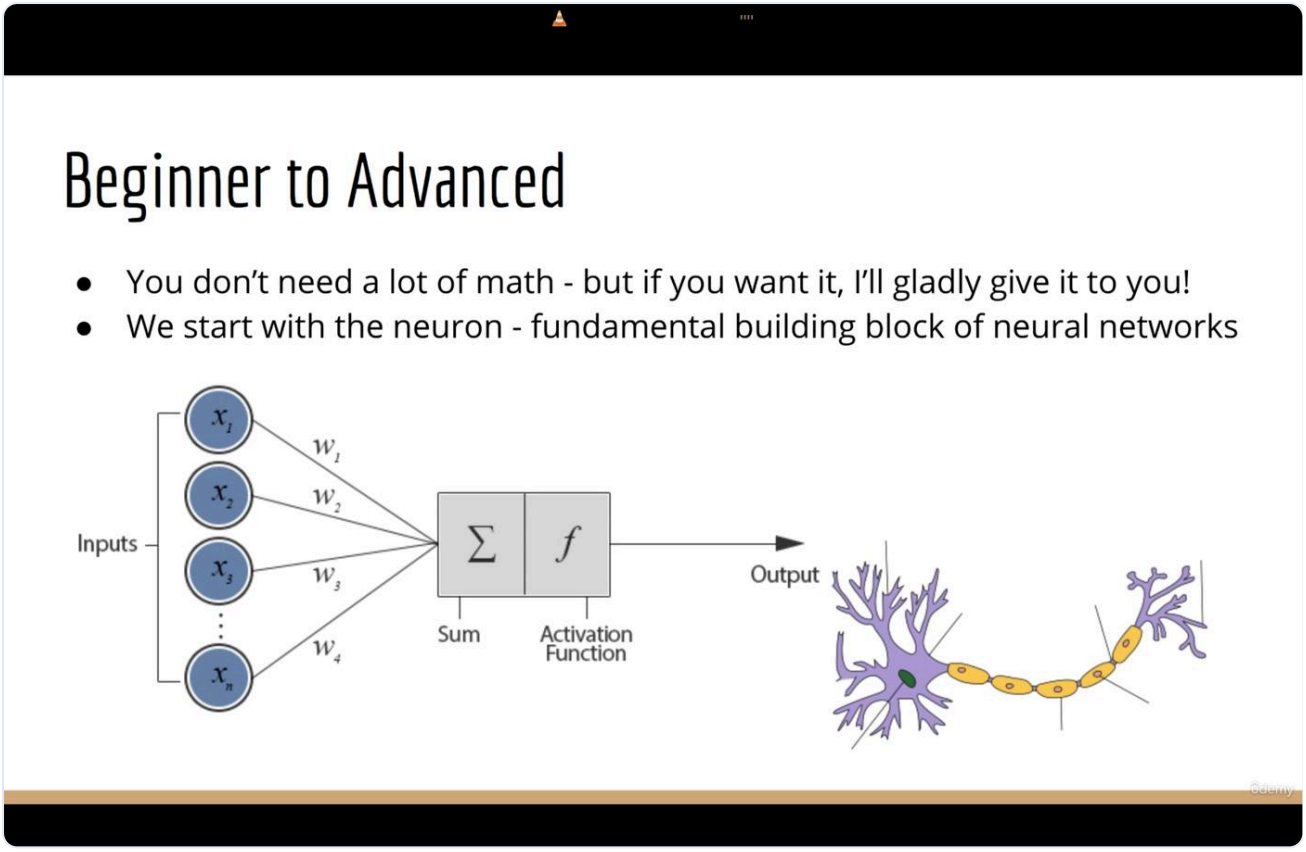


FIGURE 1.2 | Beginner to Advanced

This course is designed to be a beginner to advanced course so you don't need a lot of math and background knowledge but of course if you're into that sort of thing I certainly have resources for you. We're going to start by looking at basic machine learning in the form of a neuron.

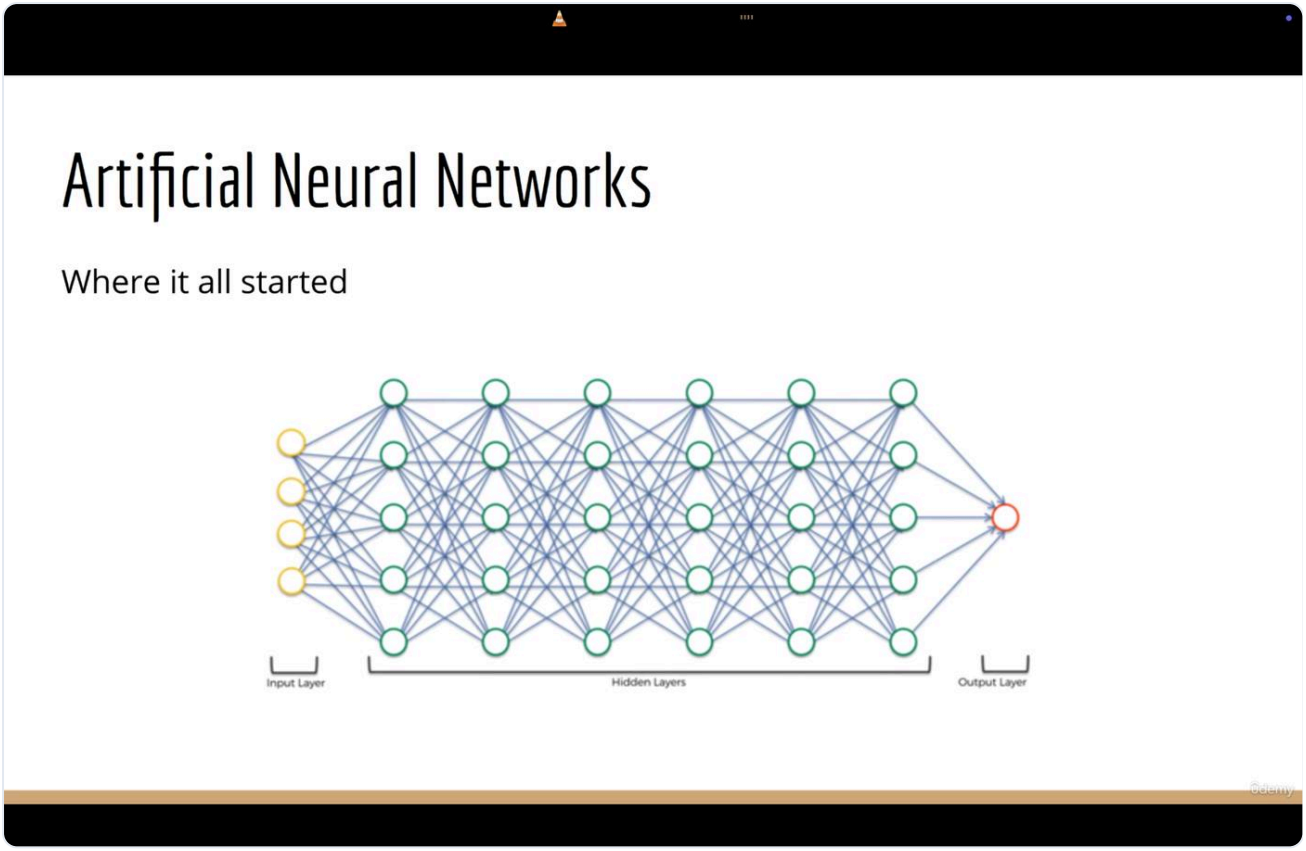


FIGURE 1.3 | Artificial Neural Networks

The fundamental building block of neural networks from there we are going to jump right into neural networks. The thing that started it all.

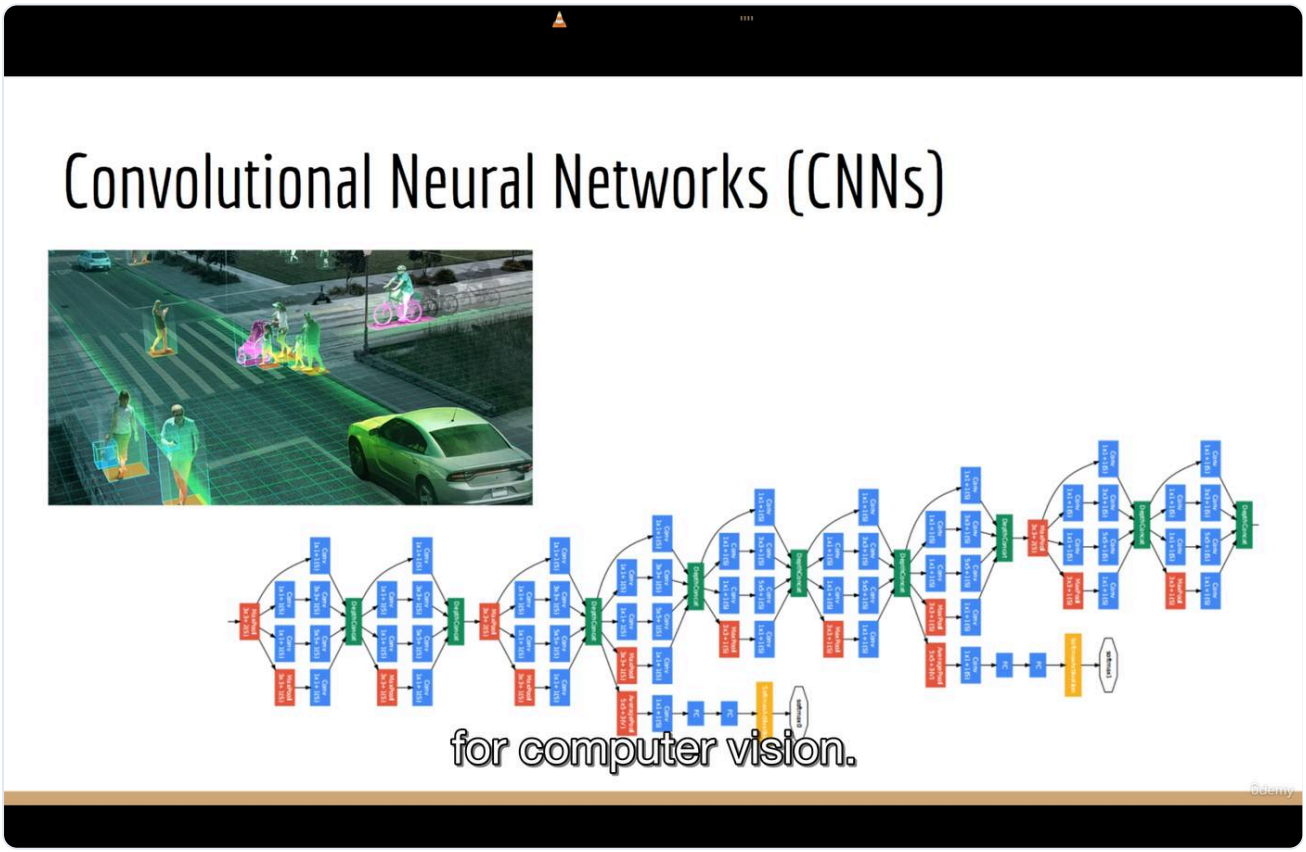


FIGURE 1.4

Luckily with PI talk you don't need a lot of heavy theory unless you wanted the PI talk API already implements a set of composed all building blocks so you can focus on building cool things rather than debugging and mathematical equations. Next we're going to look at convolution on their own networks which are specialized neural networks for computer vision. Then we'll look at recurrent neural networks which are specialized neural networks for sequence data such as Time series text speech and DNA.

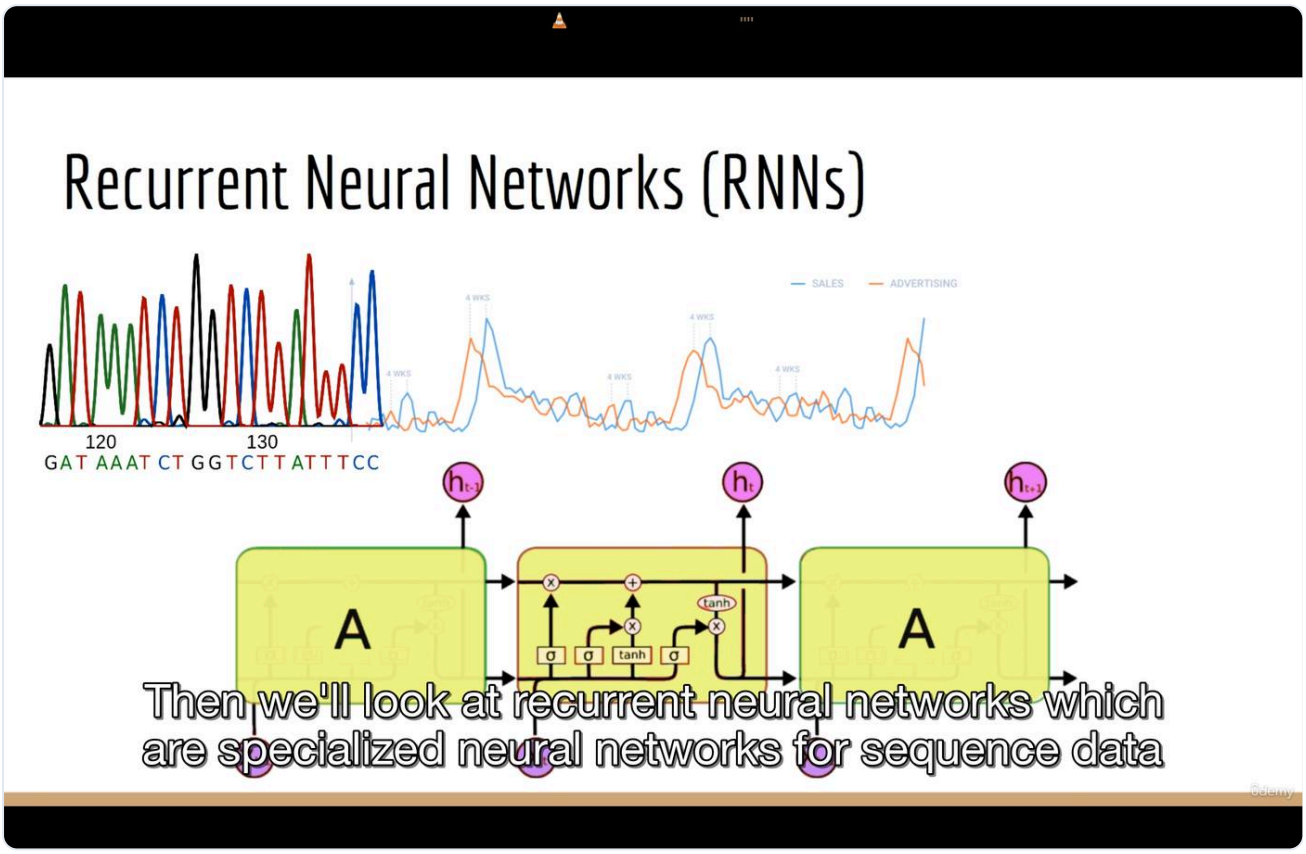


FIGURE 1.5 | Recurrent Neural Networks (RNNs)

We'll even apply our own ends to stock prediction. This is one of my favorite exercises from this course because it teaches you what 90 percent of other people are doing wrong.

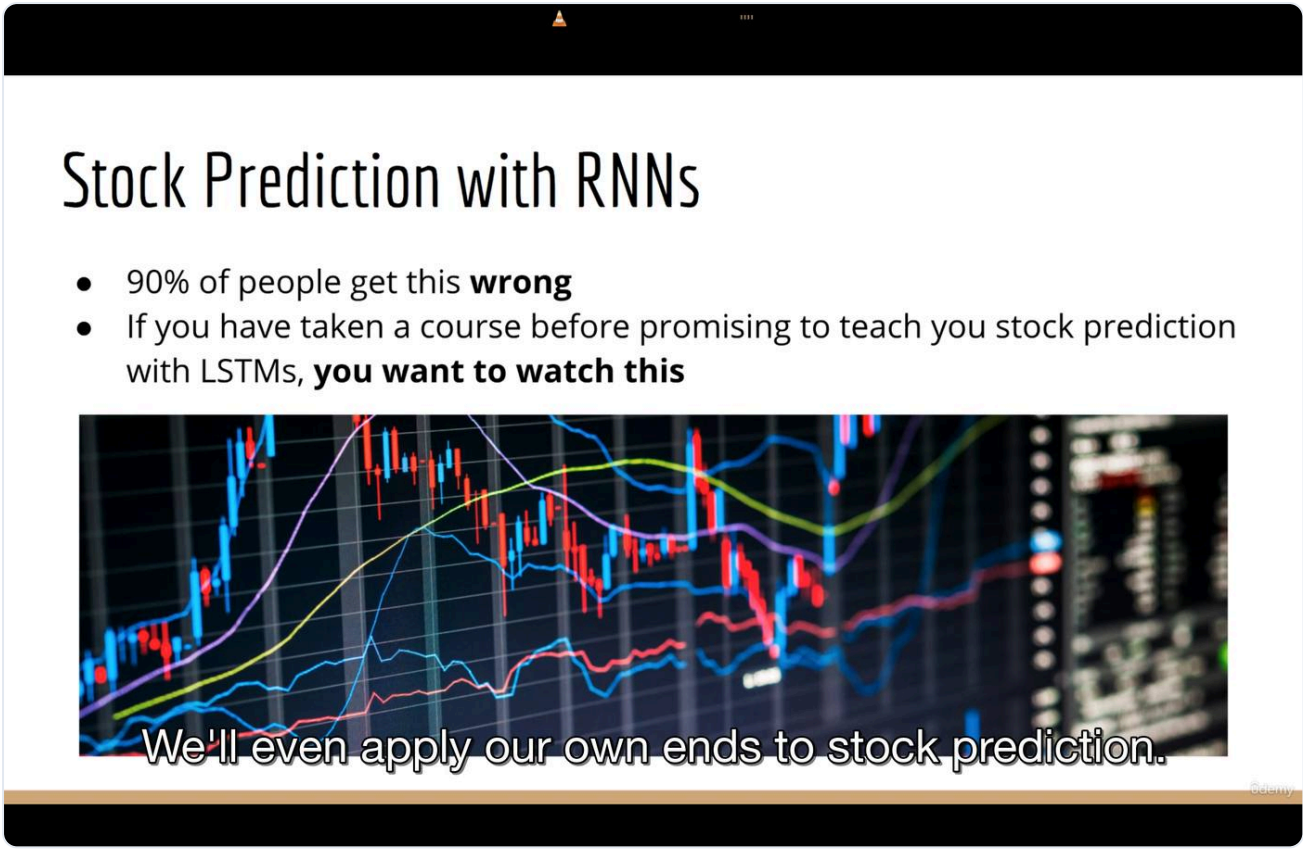


FIGURE 1.6 | Stock Prediction with RNNs

If you've ever used an LSTM for stock prediction before or taking a class that promised to teach you you'll definitely want to watch this because pi talk has a modern and easy to use interface. It makes building extremely complex things much easier than they used to be.

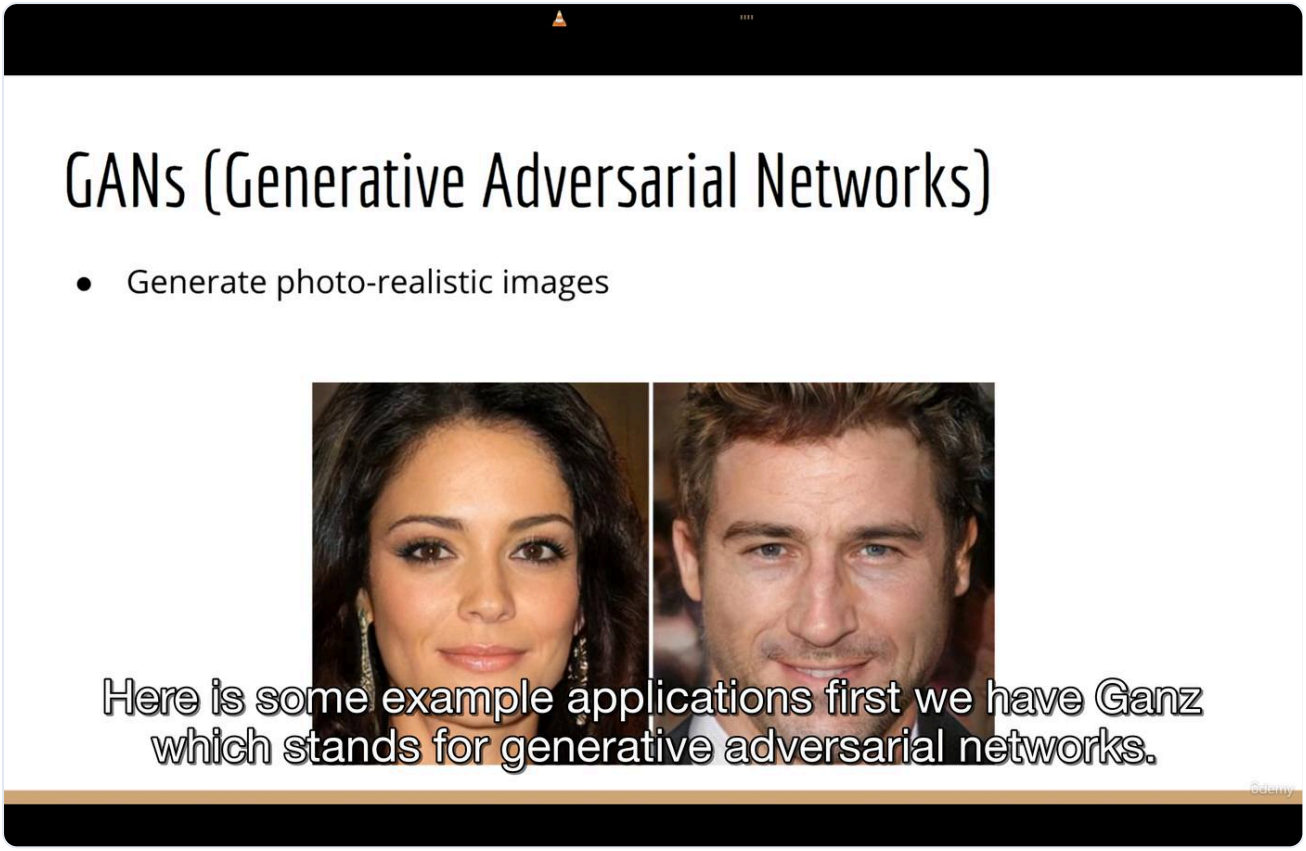


FIGURE 1.7 | GANs (Generative Adversarial Networks)

Here is some example applications first we have GANs which stands for generative adversarial networks. These are a new training paradigm which allow you to use deep learning to generate beautiful photo realistic images.

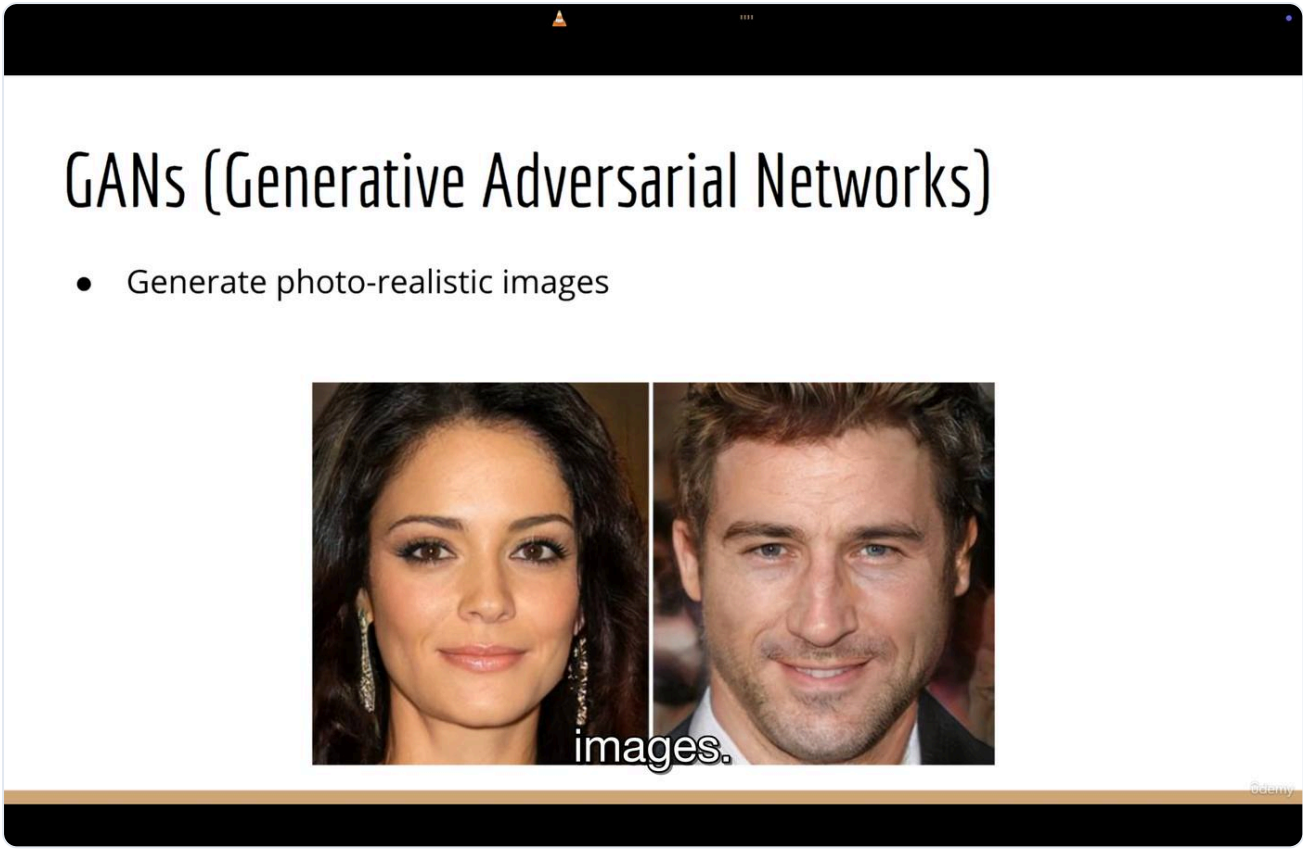


FIGURE 1.8 | GANs (Generative Adversarial Networks)

What's absolutely crazy about this is that the images you are looking at now are people that do not actually exist but it doesn't stop there.

FIGURE 1.9 | Deep Reinforcement Learning

Deep Learning has been applied in the field of reinforcement learning which excels at tasks that take multiple steps to complete. Like playing a video game. Deep reinforcement learning agents have beat world champions in the strategy game go. And in modern video games such as CSO and Dota 2 These are things we wouldn't have dreamed of just 10 years ago.

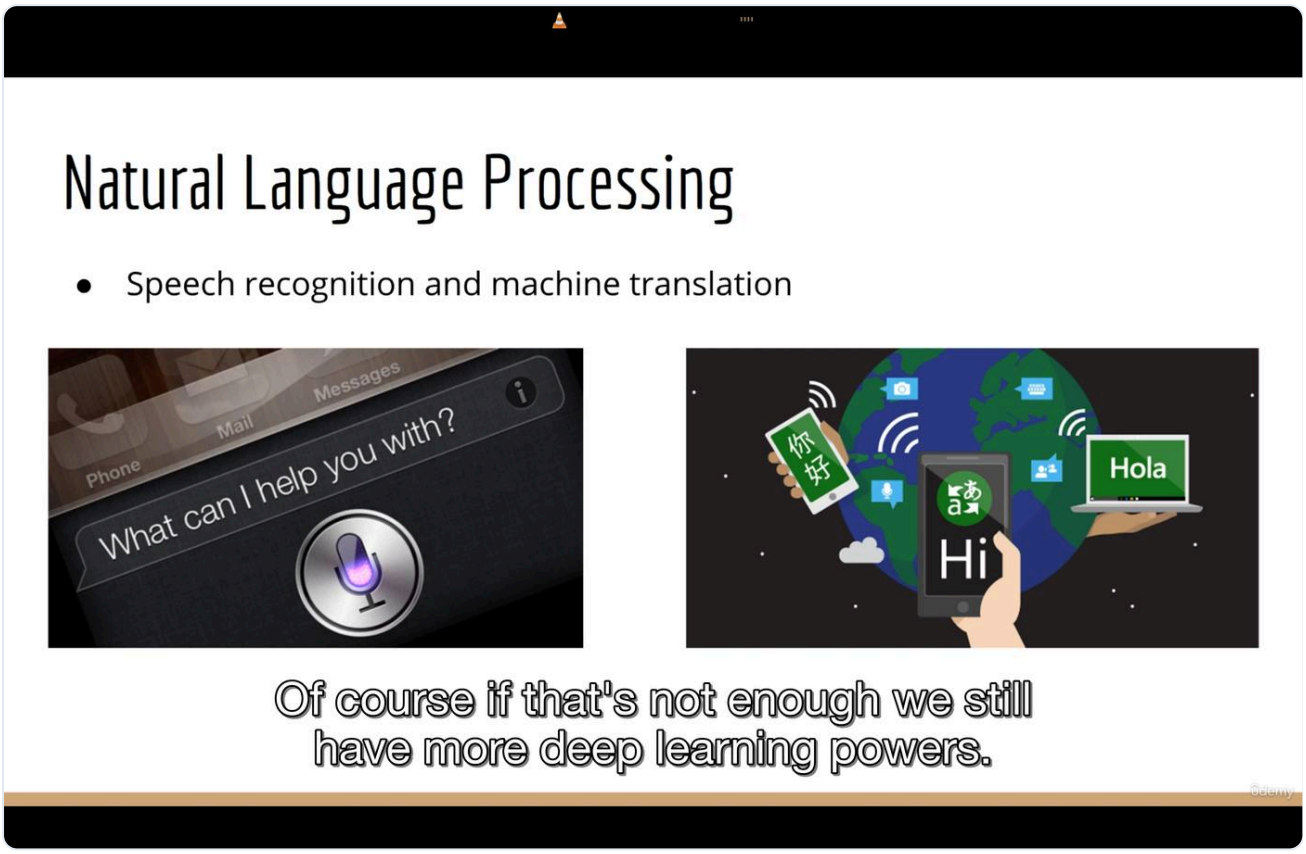


FIGURE 1.10 | Natural Language Processing

Of course if that's not enough we still have more deep learning powers. State of the art natural language processing applications such as speech recognition and machine translation. So whenever you talk to your phone and it figures out that you want to say order a pizza or send in a line tomorrow for 6 a.m. That's speech recognition using deep learning and action neural machine translation has significantly enhanced the capabilities of language translation in the recent years with students in 200 countries around the world. I've made use of this great technology more than once.

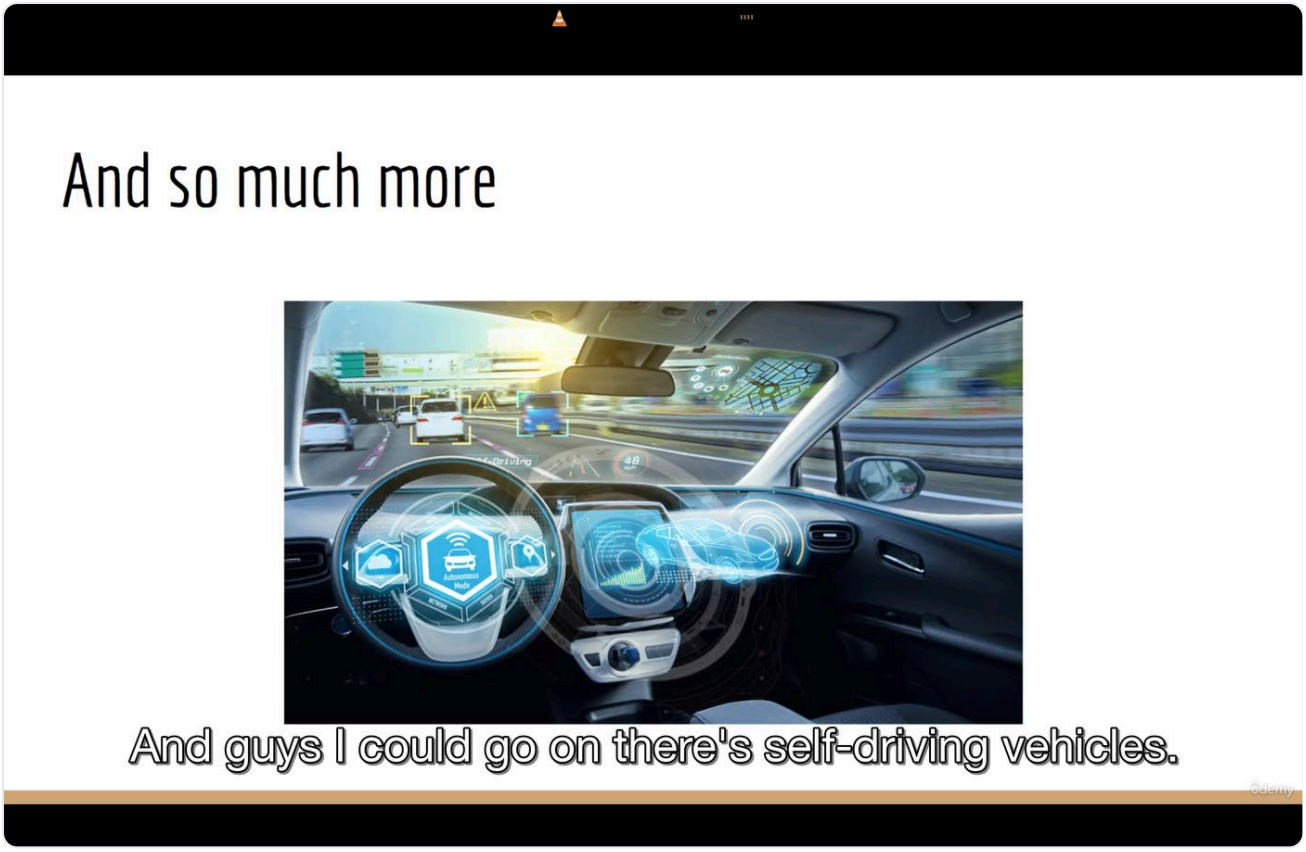


FIGURE 1.11 | And So Much More

And guys I could go on there's self-driving vehicles.

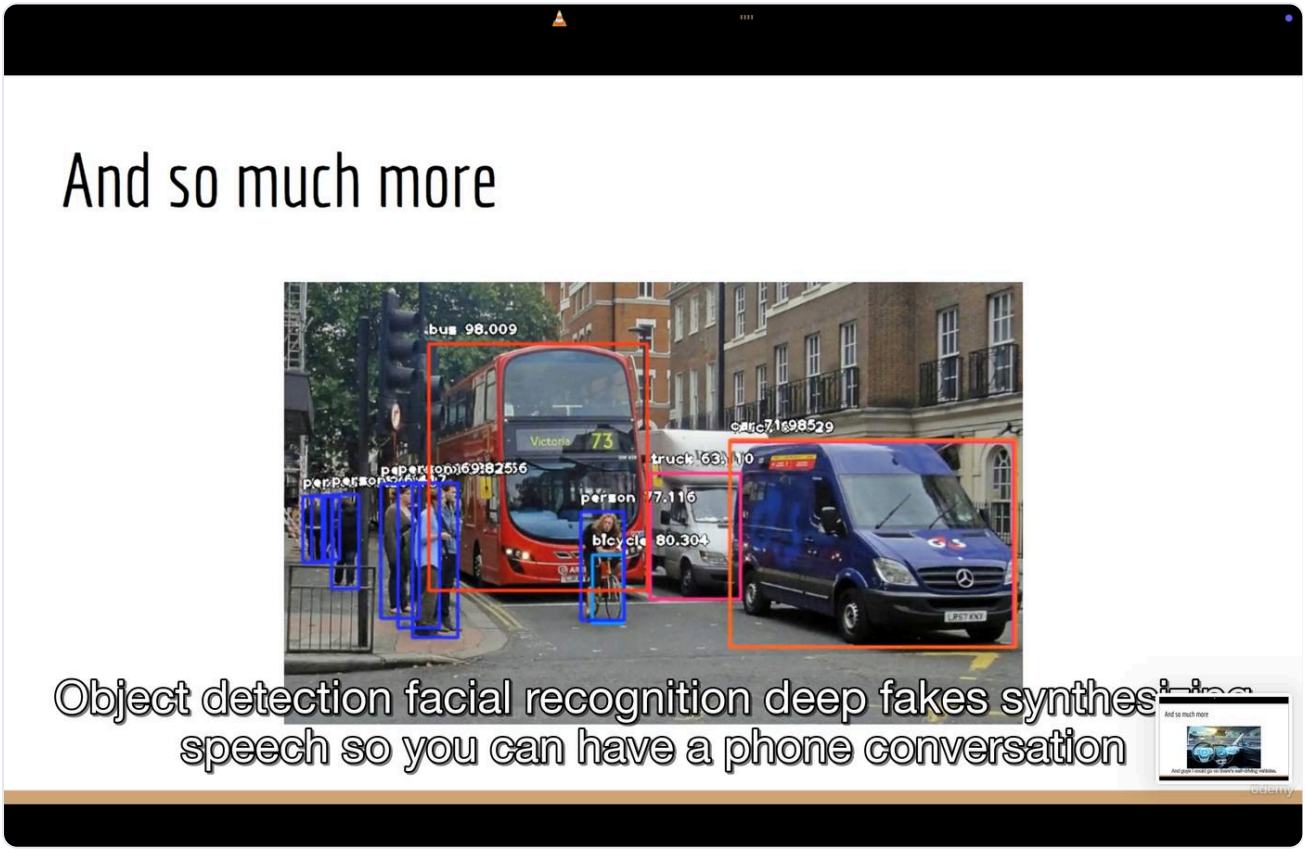


FIGURE 1.12 | And So Much More

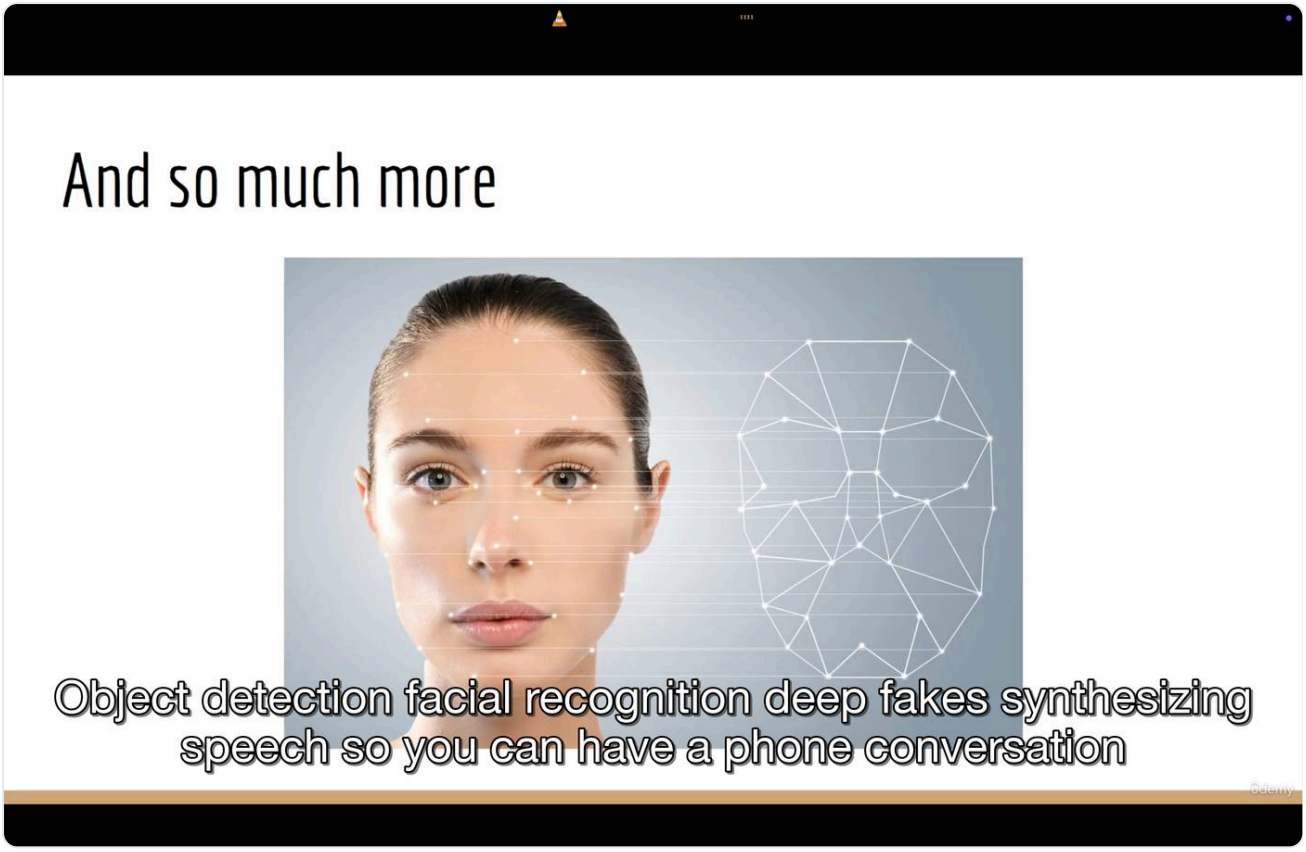


FIGURE 1.13 | And So Much More

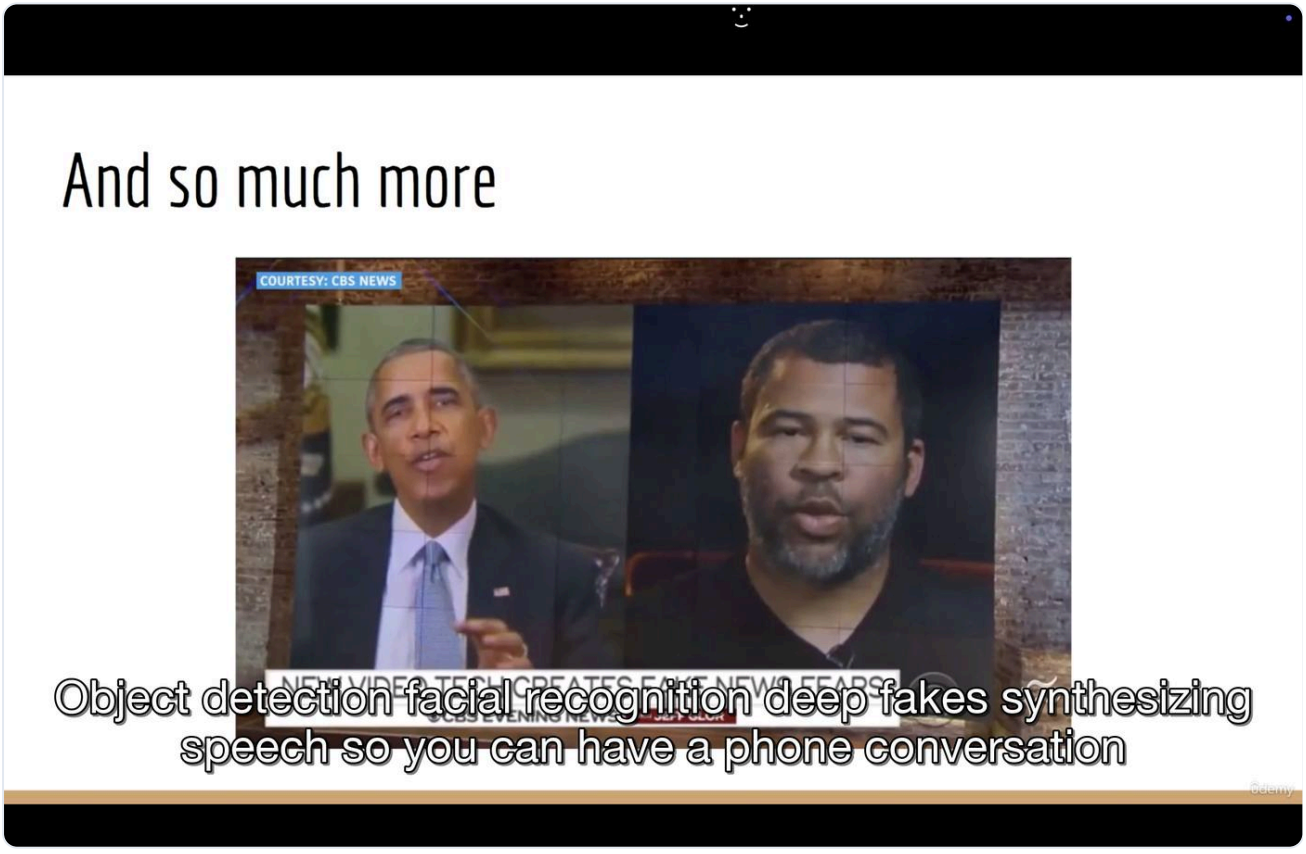


FIGURE 1.14 | And So Much More

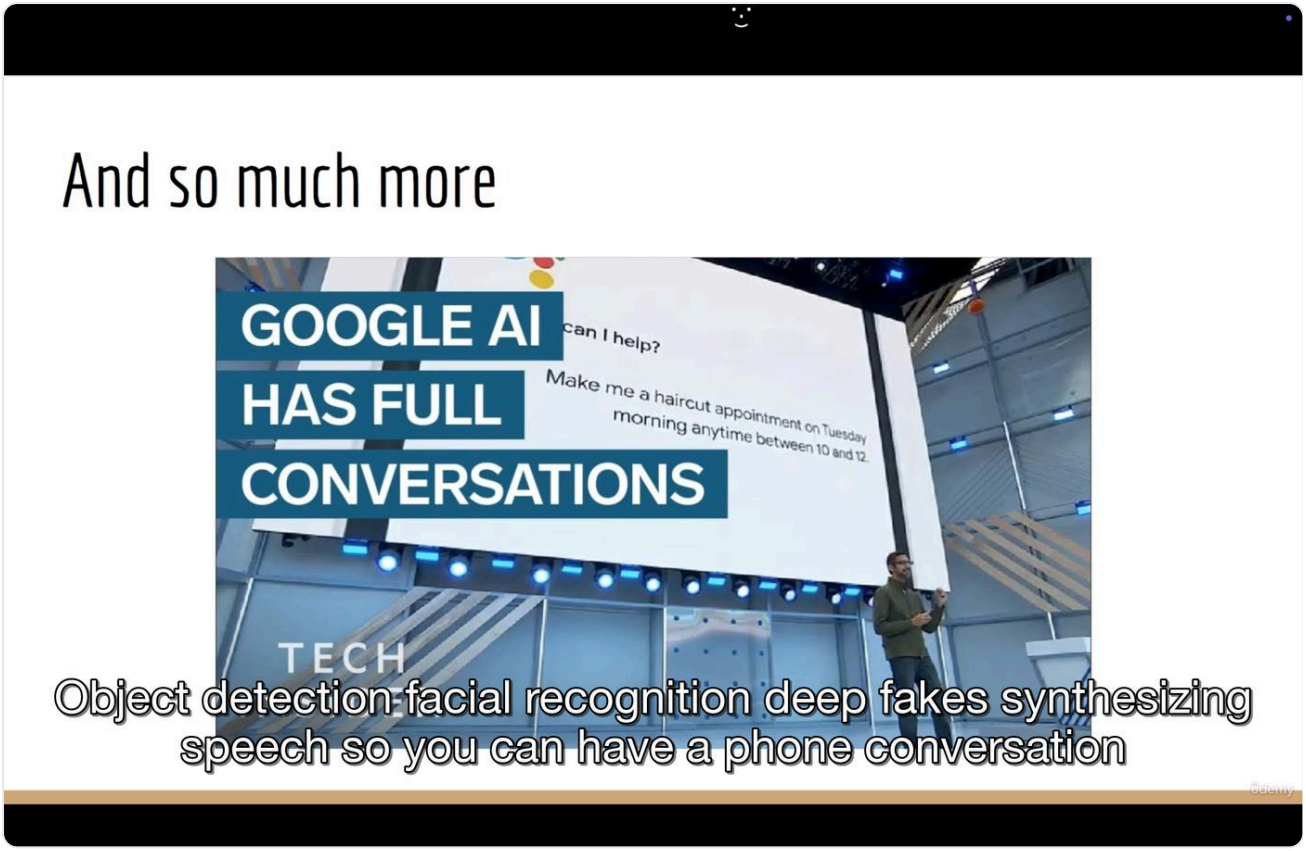


FIGURE 1.15 | And So Much More

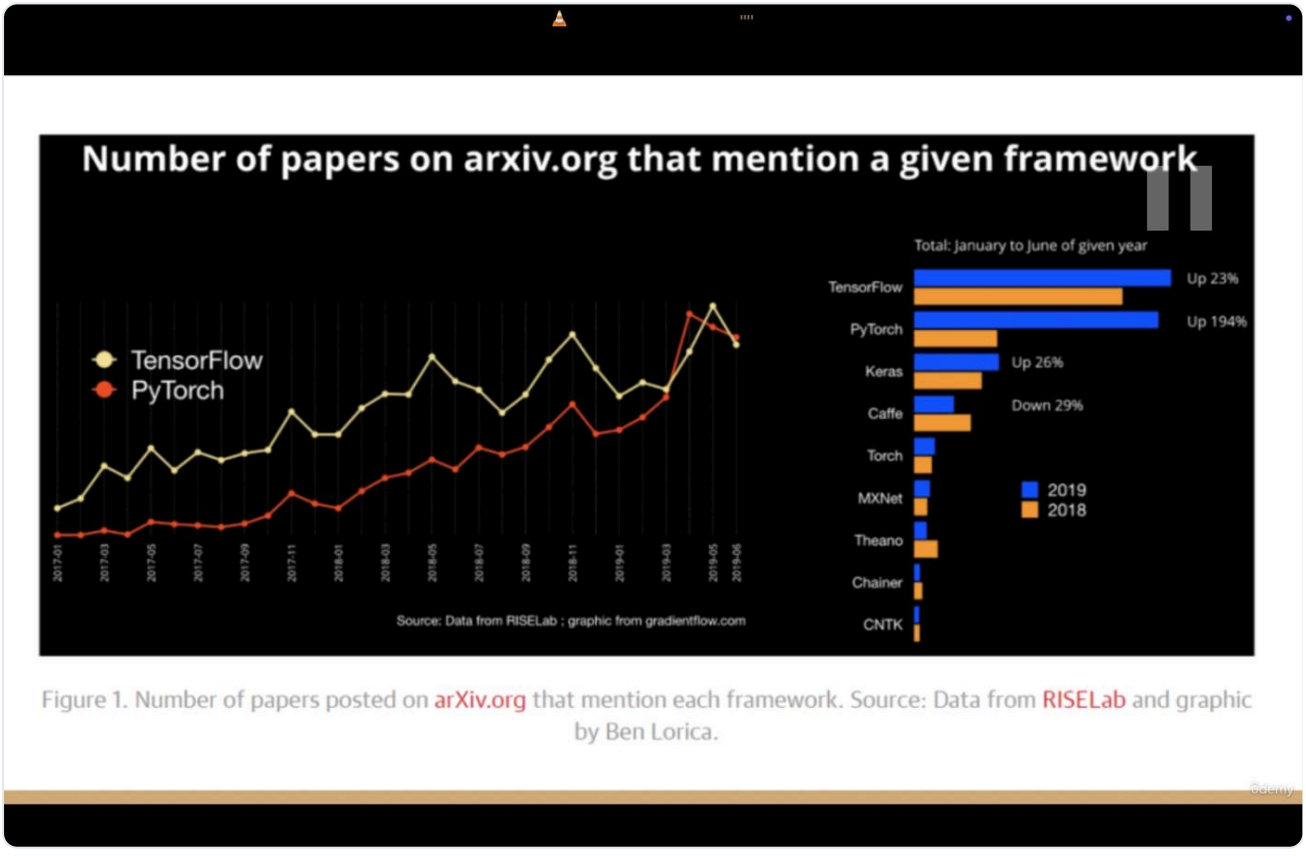


Figure 1. Number of papers posted on arXiv.org that mention each framework. Source: Data from RISELab and graphic by Ben Lorica.

FIGURE 1.16

Object detection facial recognition deep fakes synthesizing speech so you can have a phone conversation with a robot. So if you want to learn how to use the world's fastest rising Deep Learning library. This course is for you. Thanks for listening and I'll see you in the next lecture.

LECTURE 2

Course Outline

In this lecture I'm going to give you an outline for the rest of this course. First I want to give you a brief summary of what pytorch is all about. In case you've had previous experience with TensorFlow or other deep learning libraries such as the theano.. So in the beginning there was Theano. Theano was a significant improvement over what was being done previously for two major reasons.

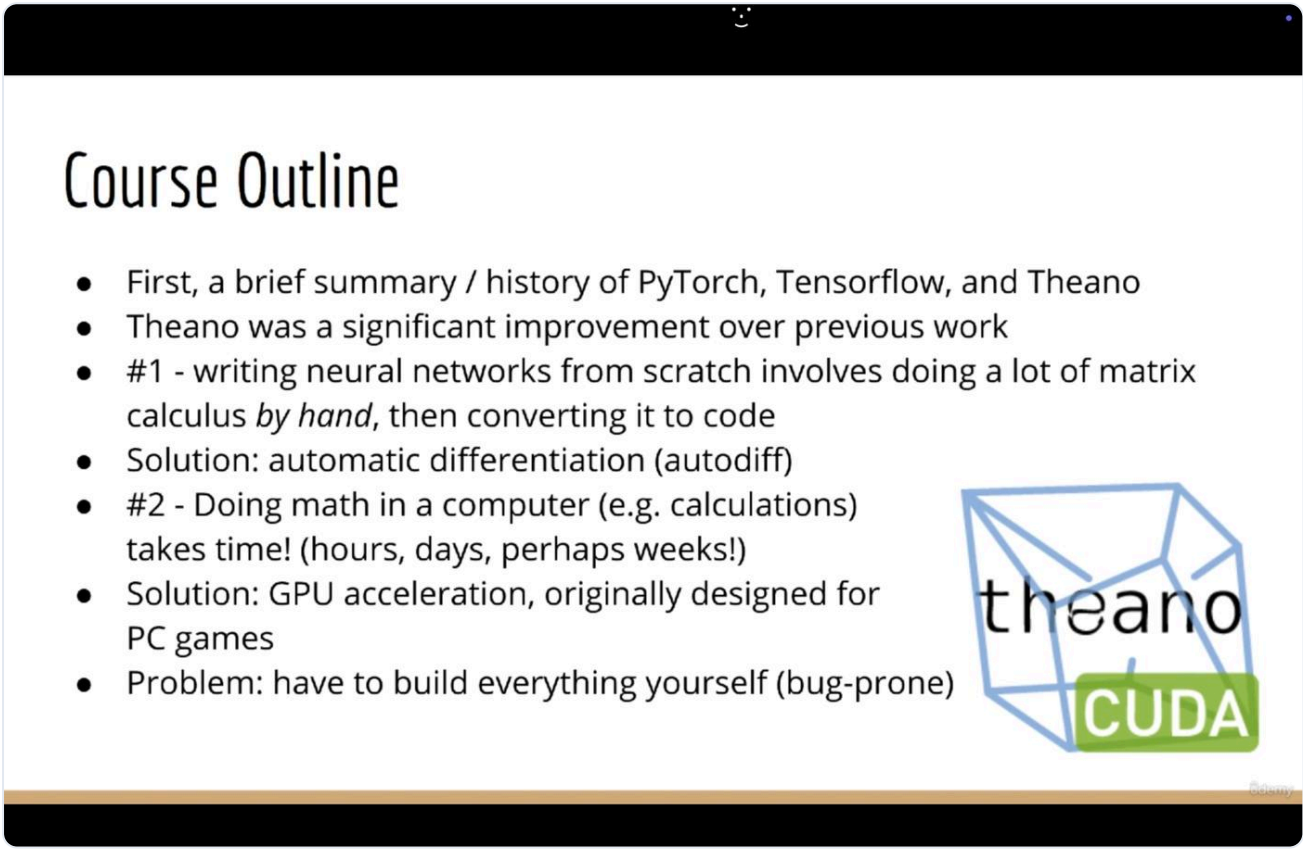


FIGURE 2.1 | Course Outline

Number one for reasons you'll learn about later writing neural network that works from scratch involves doing a lot of Matrix calculus by hand and then copying those equations into code for those of you who took my first deep learning course. You know how difficult this can be. The Theano library was the first to innovate in this area using automatic differentiation or auto diff for sure what that means is you don't have to write down Calculus Equations since the computer will do that for you. And number two there's a lot of math that has to happen in a neuron that work. And this takes a lot of time meaning you'd have to wait hours or even days or weeks to train your neural network.

The Theano Library was the first to innovate in this area by making use of GPUs which were originally designed to improve the performance of P.C. Games. One of the downsides of older libraries such as the Theano is that you have to build everything by yourself. This can really slow you down especially if it's your first time writing certain components. You not only have to write

each component on your own you'll also have to worry about them being right. If any single component you wrote is wrong then it could make your entire program fail. On top of that in the previous years it was announced that the Theano Library would no longer be maintained. What about TensorFlow ?

TensorFlow is a pretty popular library, thanks to the fact that it's backed by Google who at this point kind of run the Internet TensorFlow used to be really messy. In fact probably more so than Theano But now in version 2.0 it uses the Keras API which is the total opposite. It's very high level and very simple but there is a downside to very high level and very simple APIs.

FIGURE 2.2 | Popular Library Developed by Google

They make it very easy to do common things but hard to do uncommon things.

FIGURE 2.3

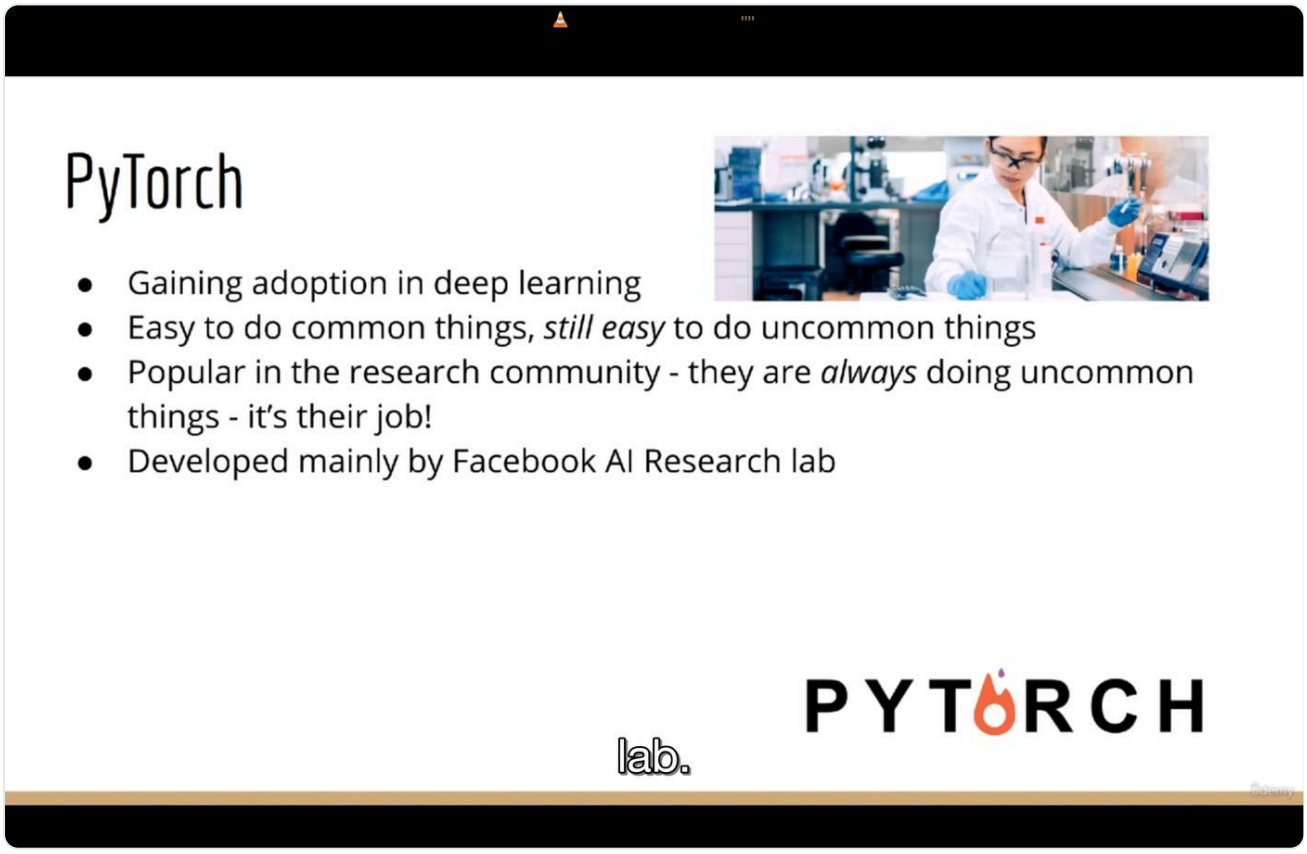


FIGURE 2.4

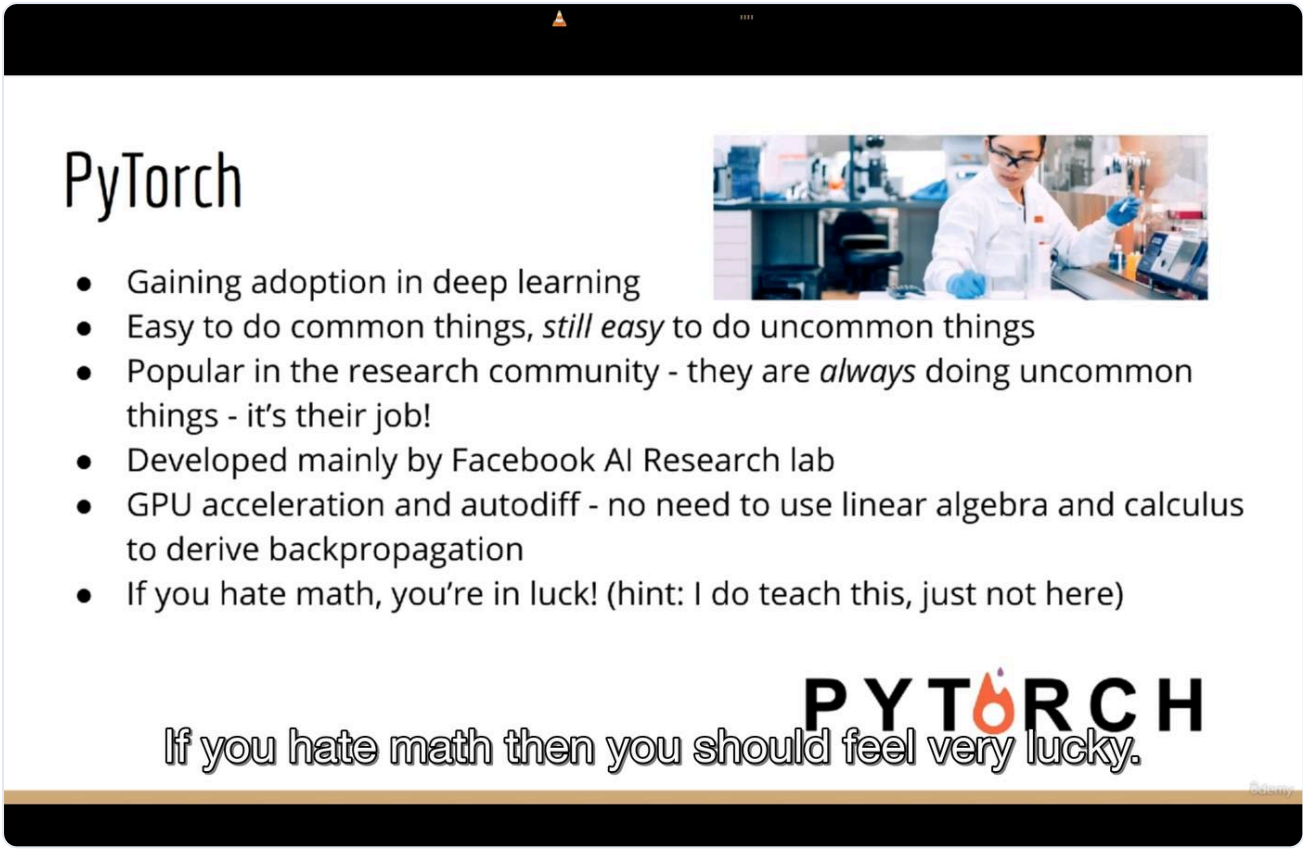


FIGURE 2.5

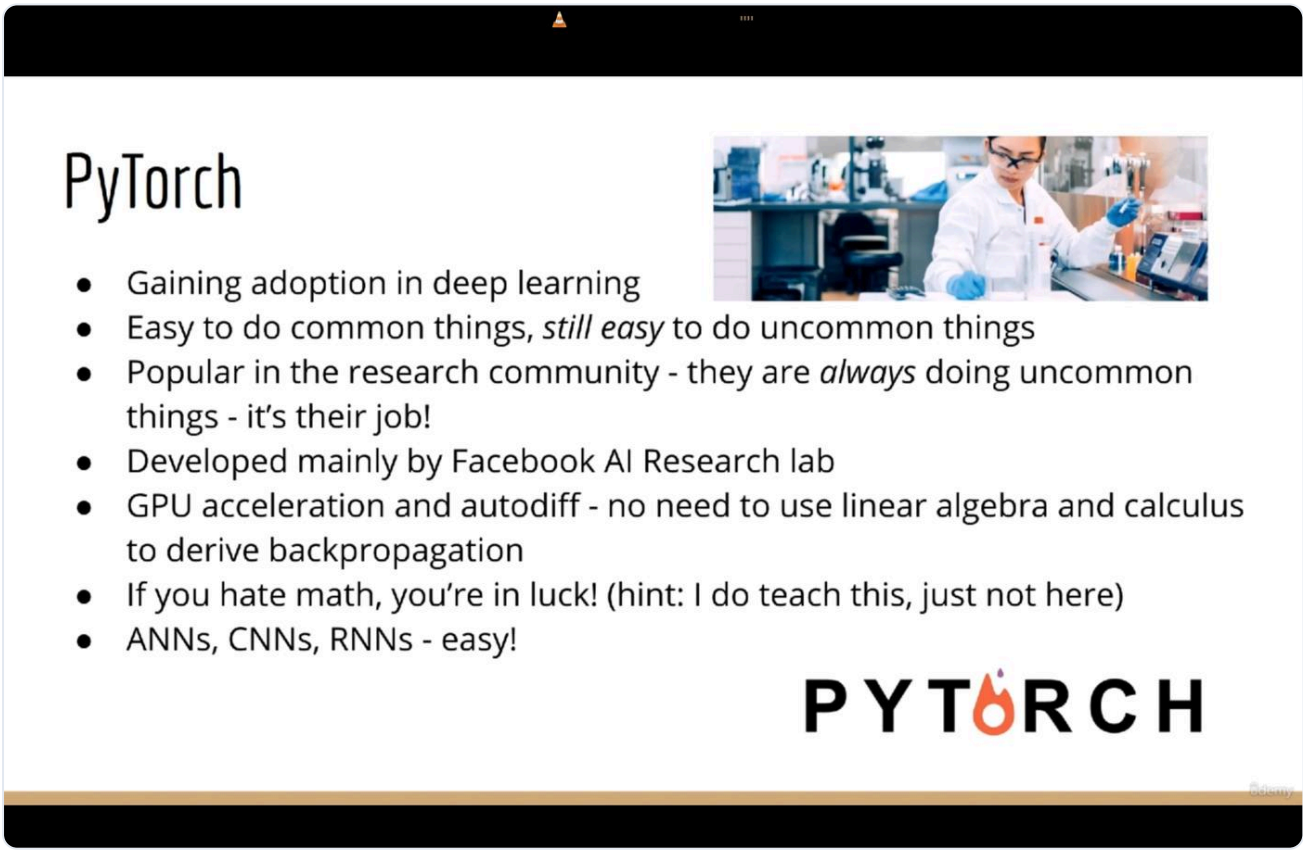


FIGURE 2.6

PyTorch on the other hand has been slowly gaining adoption in the field of deep learning thanks to the fact that it's relatively easy to do common things and still easy to do uncommon things. For this reason it's been extremely popular in the research community who by definition do lots of uncommon things. That's their job after all. PyTorch is developed mainly by another Internet giant Facebook, specifically the Facebook API research lab. As with all deep learning libraries PyTorch is supported by GPUs acceleration and has automatic differentiation so there's no need in this course to use tools like calculus and linear algebra to derive back propagation.

If you hate math then you should feel very lucky. In fact we won't discuss back propagation at all despite the fact that it's really the main ingredient that makes everything work. This combined with the fact that a lot of the basic building blocks are already built for you are some of the major advantages which are going to allow us to blast through each section very quickly. In the past it was necessary to focus on each architecture one at a time with RNN being the most complex to implement these days ANN, CNN and RNNs can be implemented in just a few lines of code. All right so how is this code structured. First before we even begin discussing deep learning or any type of statistical modeling we are going to look at a new coding environment which I really like called Google Colab.

Google Colab is basically Jupyter notebook hosted by Google but with a lot more bells and whistles. Personally I have never been a fan of Jupyter notebook since it seemed to have more disadvantages than advantages. However Google Colab is a different beast. It's hosted by Google so you don't have to use your own computing resources.

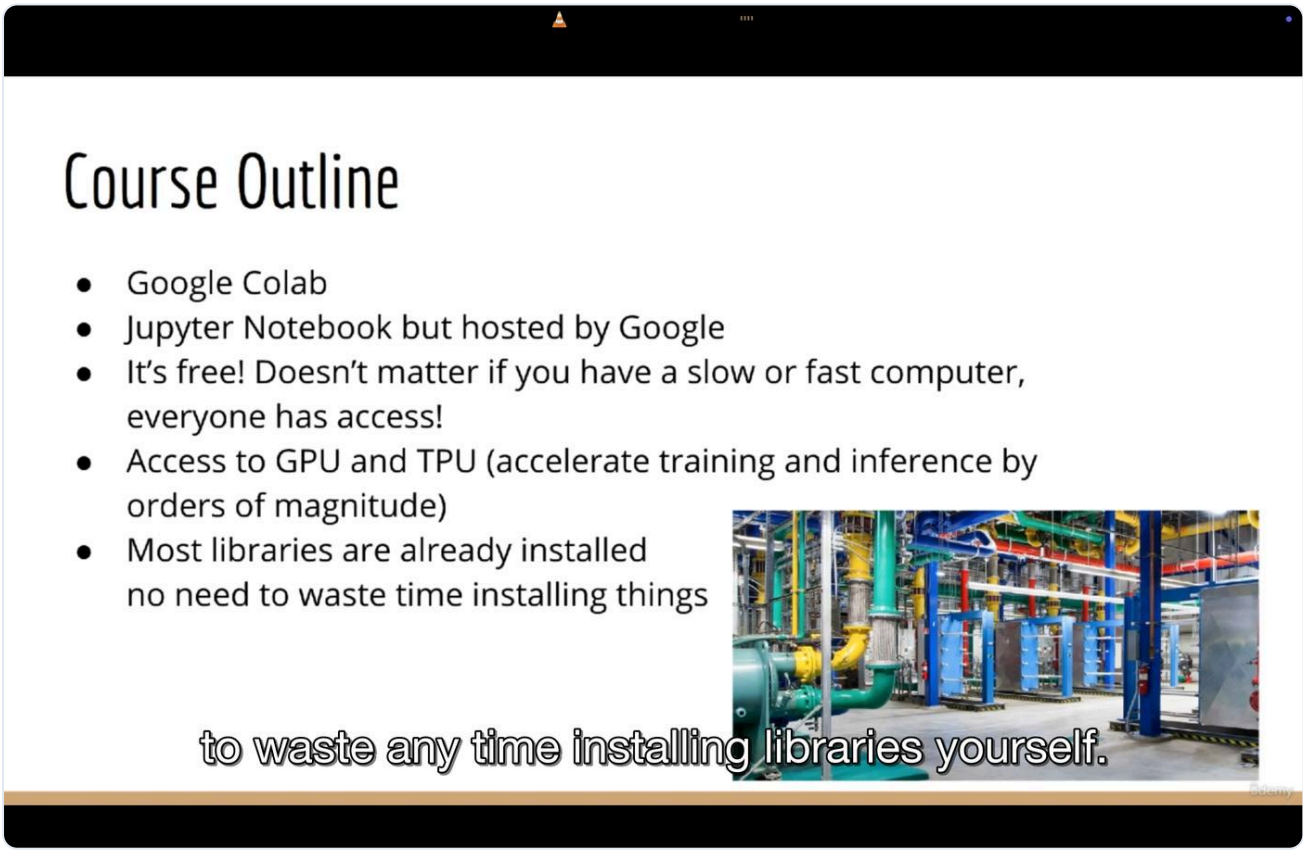


FIGURE 2.7 | Course Outline

It's free so it doesn't matter if you have a slow computer or a fast computer. Everyone has the same access it gives you free access to the GPUs and TPUs. You for orders of magnitude faster training and inference and finally most of what we need is already installed so you don't have to waste any time installing libraries yourself. Once you've done that we are going to go through the fundamental architectures involved in deep learning.

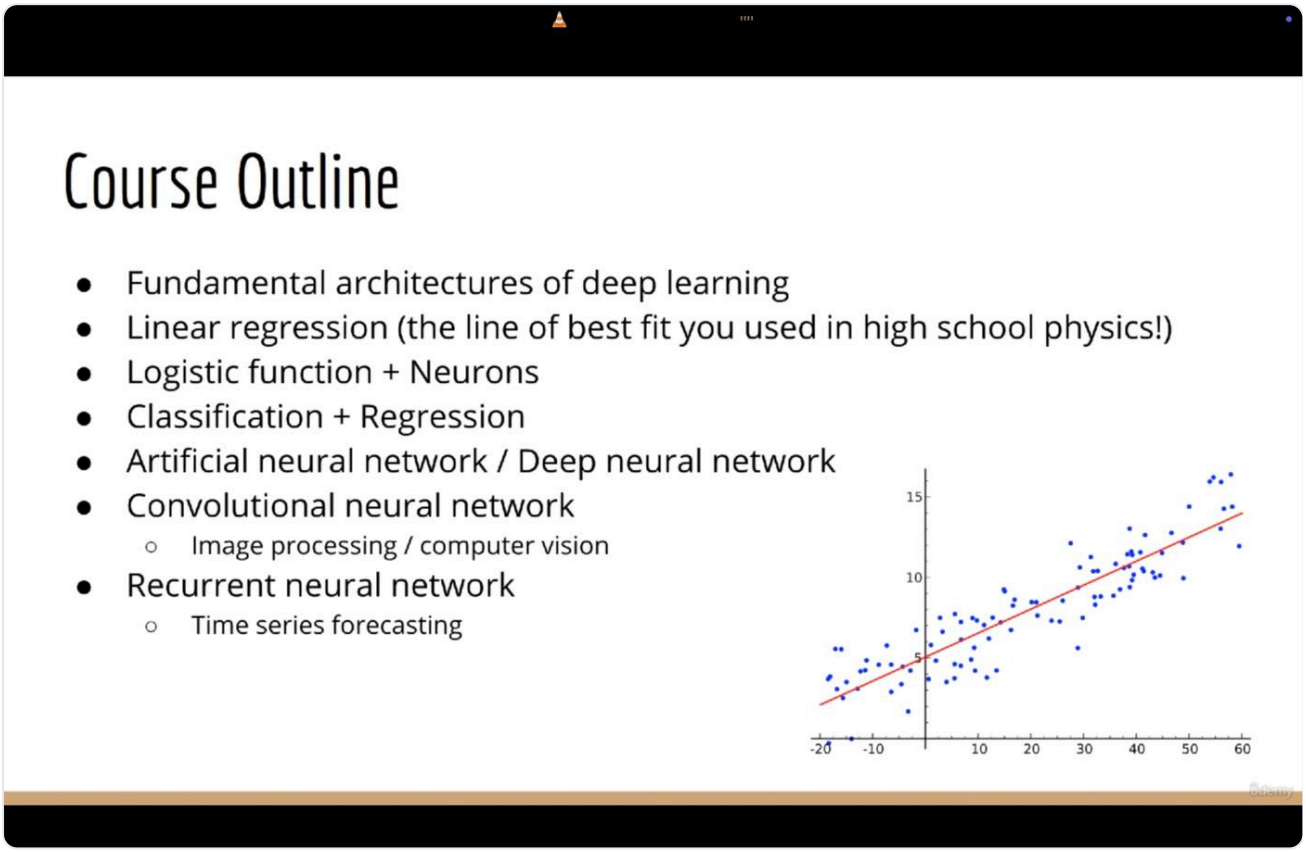


FIGURE 2.8 | Course Outline

Believe it or not this all starts with linear regression the line of best fit you learned about in high school physics. We'll see that with just one little change adding the logistic function on top of a linear model. We will get a neuron this covers the two major types of supervised learning classification and regression. Once you know the basics it's time to get started with deep learning. The first deep learning architecture you'll learn about is the artificial neural network also known these days as the deep neural network. Although neither of these names really do a good job of differentiating it with other architectures. Next we'll dive into image processing with convolution on their own that works.

You'll learn about convolution and how this works to create neural networks that are specially designed to achieve better performance on images and other physical signals next you'll learn about recurrent neural networks which specialize at working with sequence data. Unlike my previous courses where we used RNNs is mostly for natural language processing we are going to start with time series forecasting. This is going to let us cover a lot of ground that many other courses simply skip over entirely in particular.

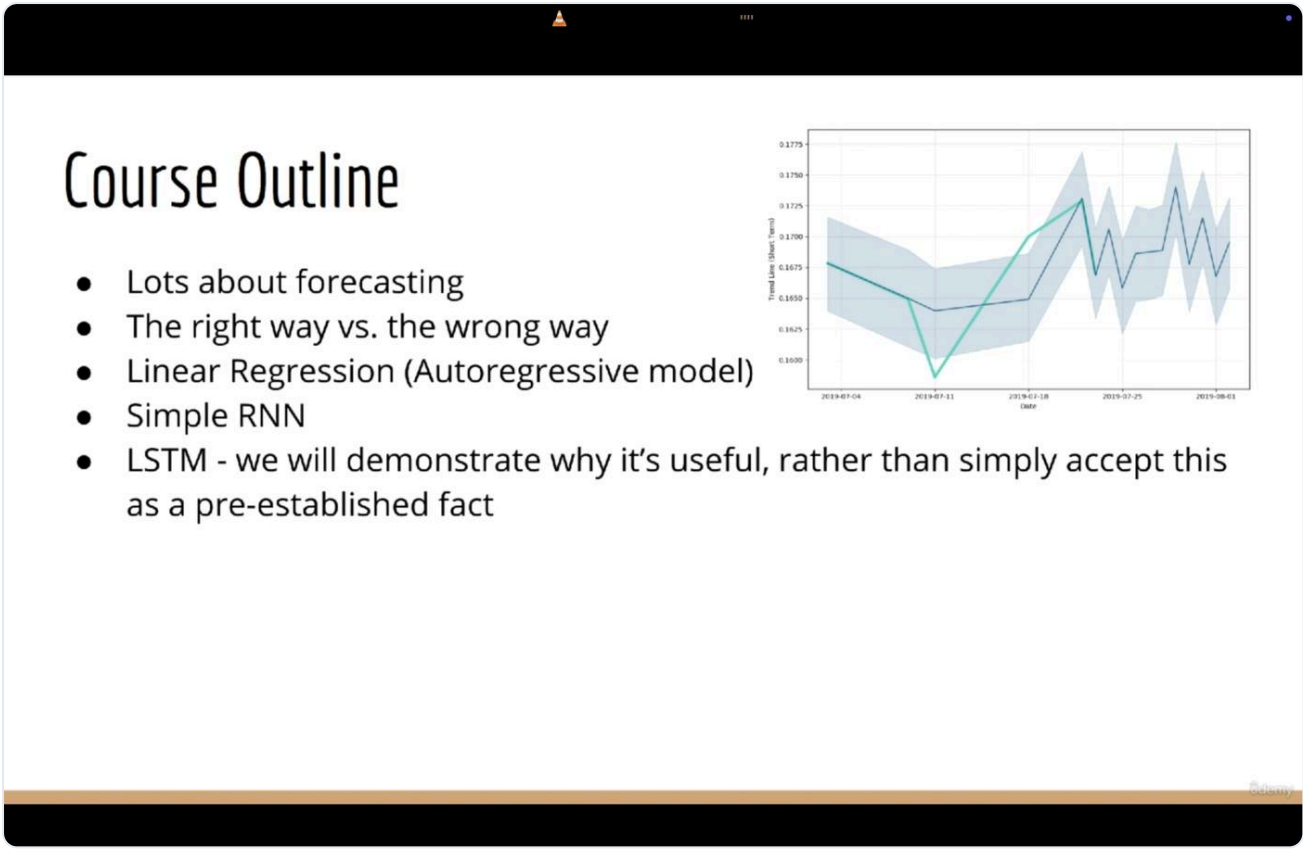


FIGURE 2.9 | Course Outline

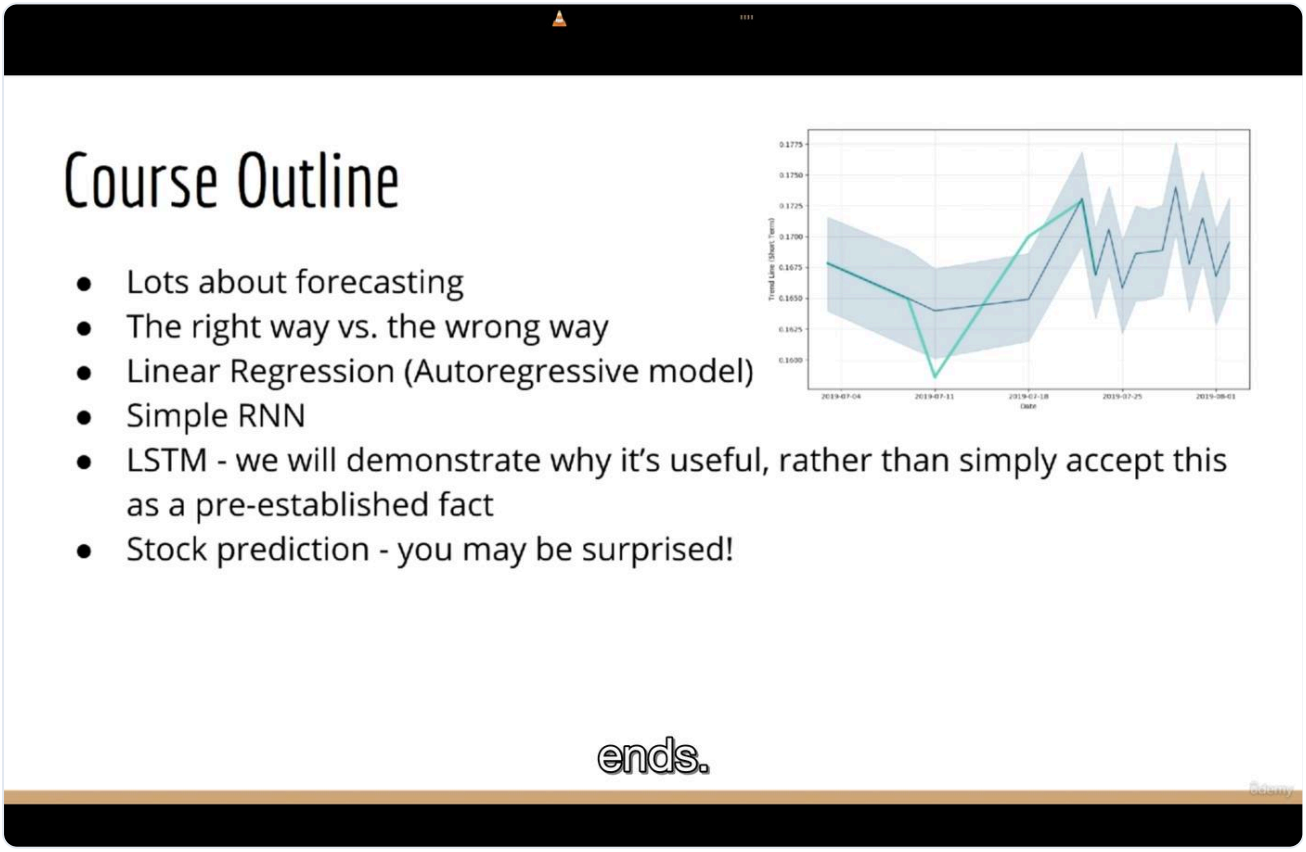


FIGURE 2.10 | Course Outline

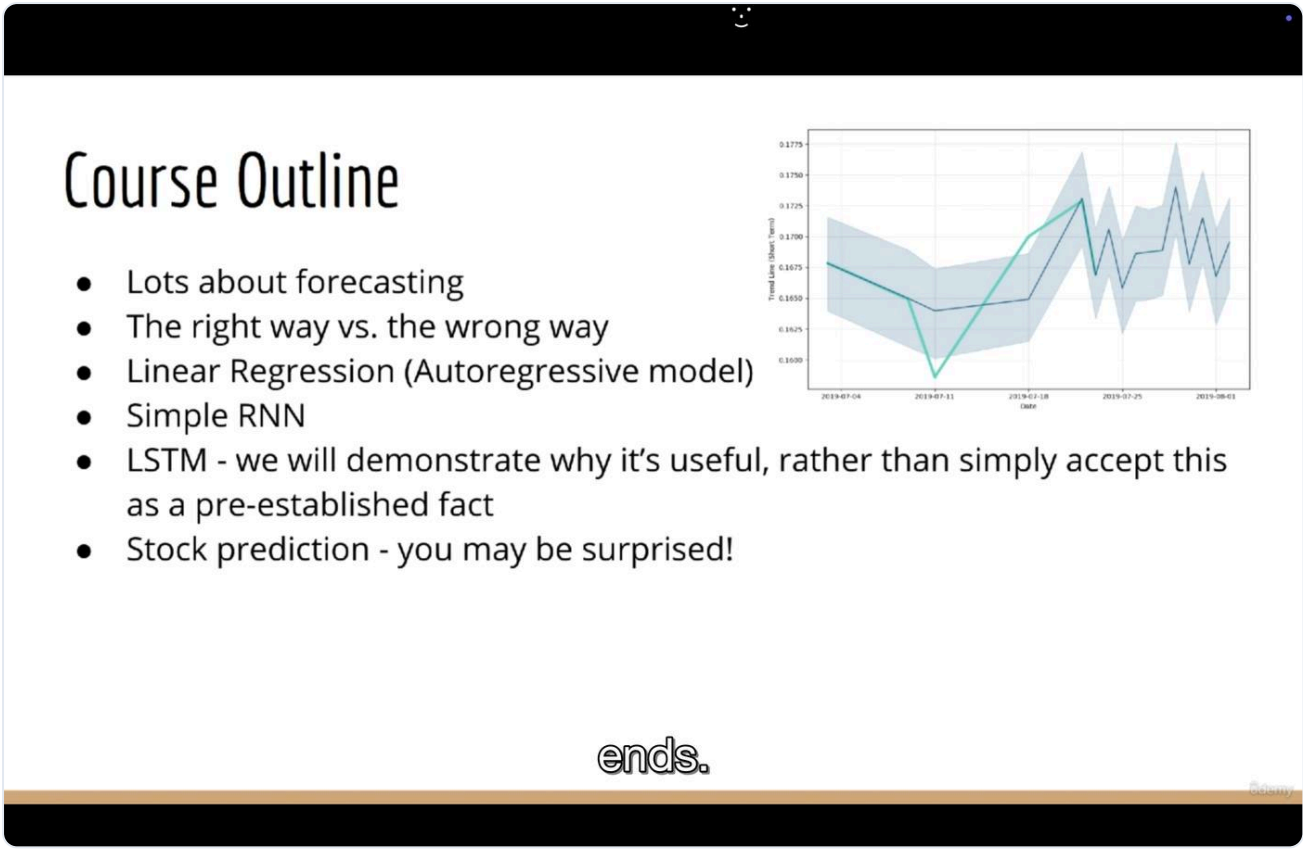


FIGURE 2.11 | Course Outline

We're going to cover the difference between the wrong way to do a time series forecast and the right way we're going to look at several time series that simple models like linear regression can and can't solve as well as several time series that slightly more complex models like the simple are in can and can't solve for the more difficult problems we'll see how the LSTM achieves superior performance and so you'll see first hand why LSTM are useful rather than me just telling you and you accepting it as fact without observing it for yourself. The next example is one of my favorites in the chorus which is on stock prediction with LSTMs and RNNs. I think most of you will find the contents of these lectures very surprising.

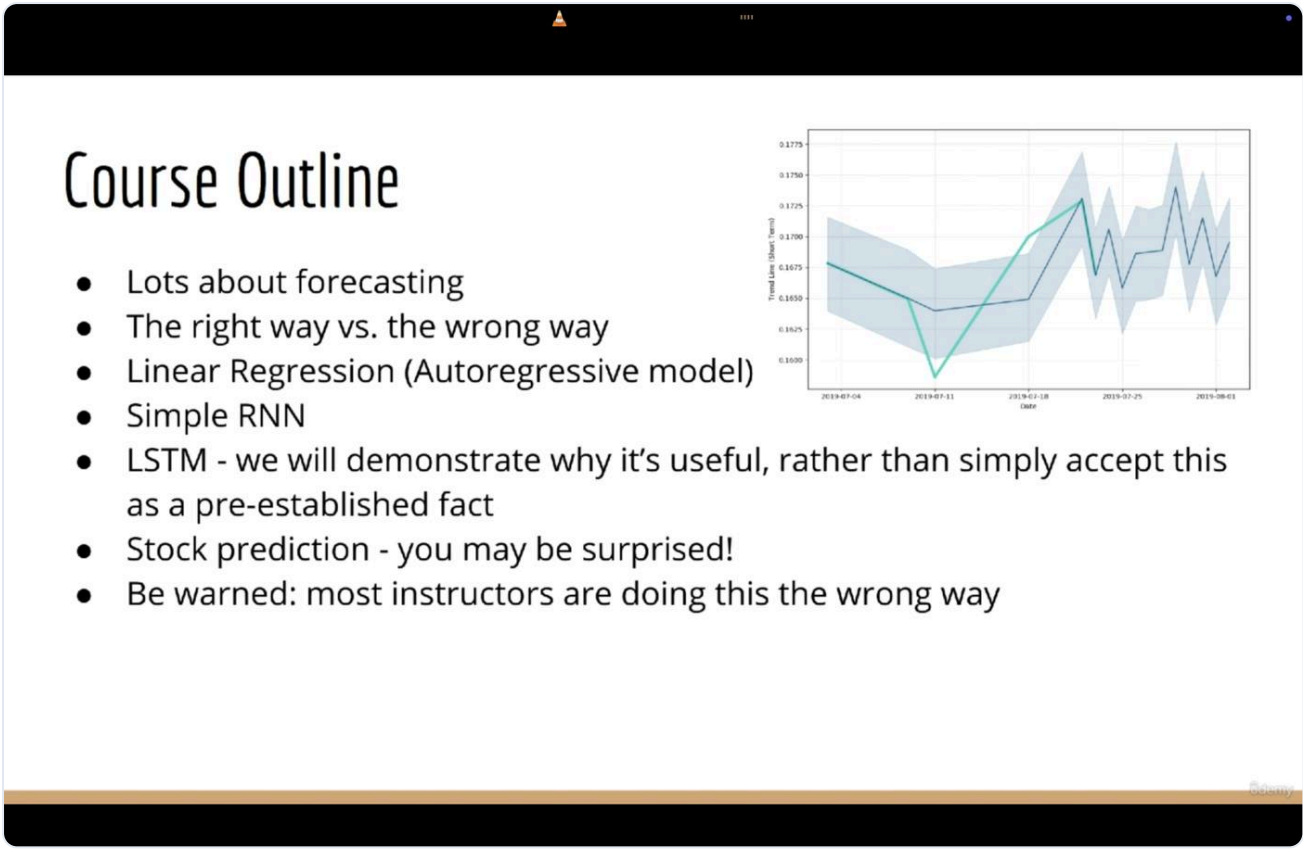


FIGURE 2.12 | Course Outline

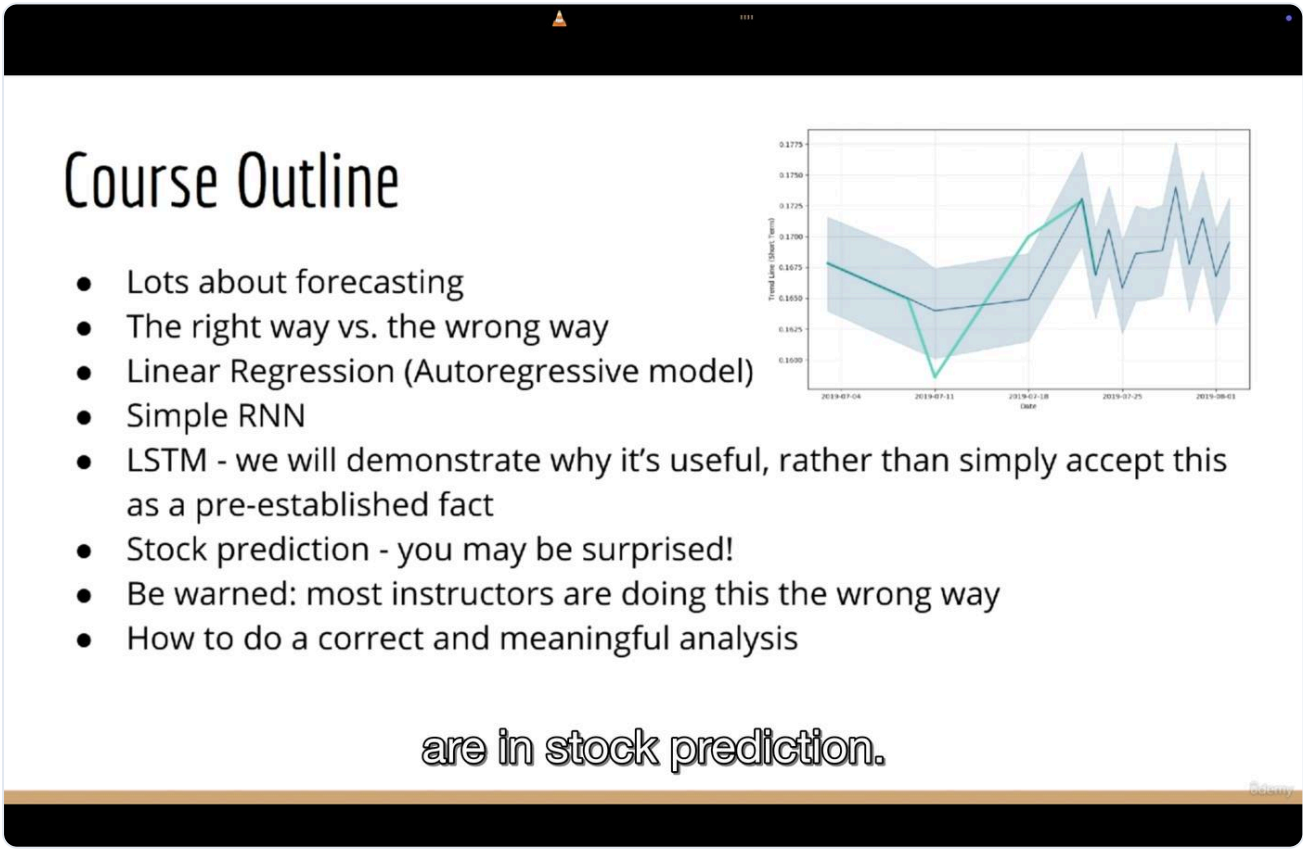


FIGURE 2.13 | Course Outline

If you've ever learned about stock prediction with Ellis teams in the past be warned most other instructors are doing this the wrong way. In this course I'm going to teach you why it's wrong how to correct it and what some of the real obstacles are in stock prediction.

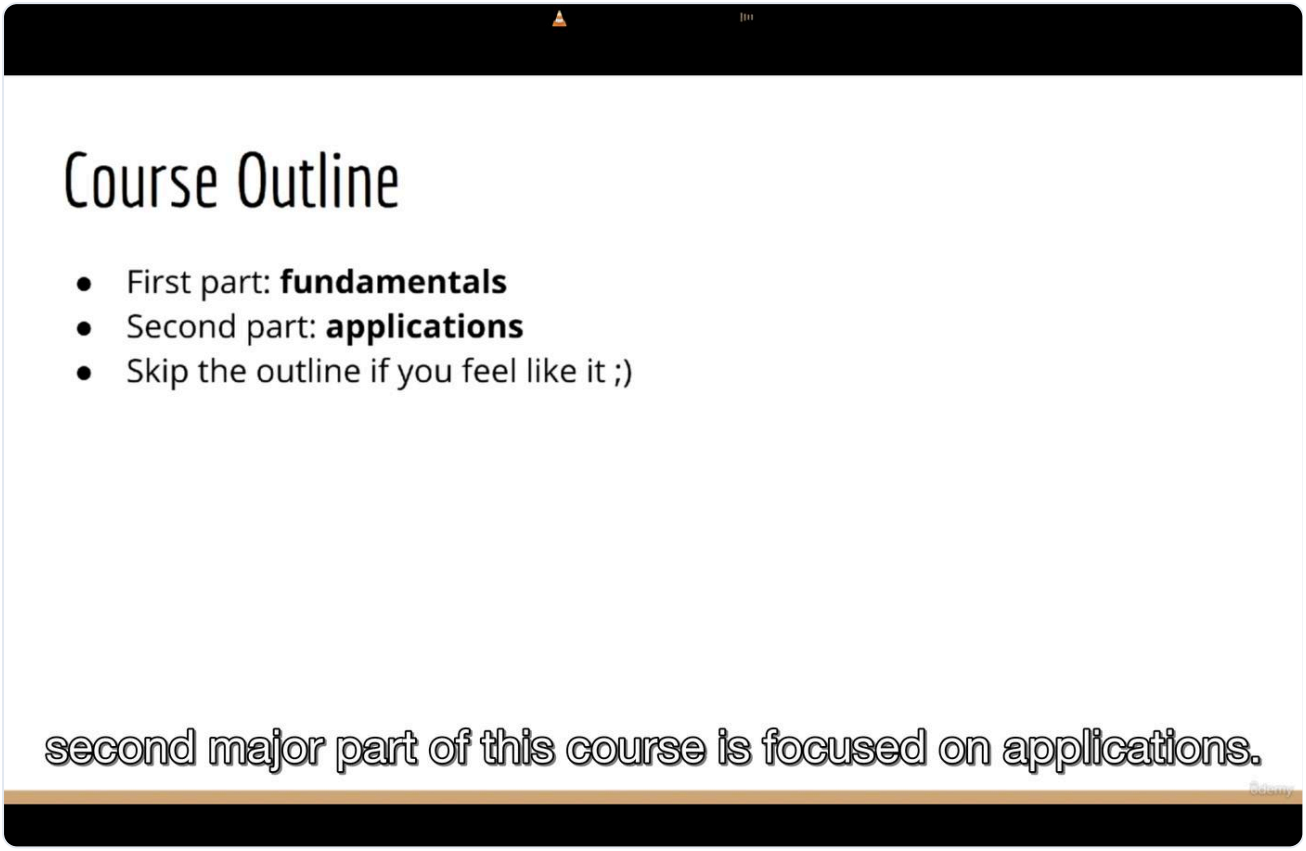


FIGURE 2.14 | Course Outline

The first major part of this course is about the fundamental architectures and deep learning while the second major part of this course is focused on applications. As a side note if you want to just skip the rest of this lecture and move on with the rest of the course that's fine too.

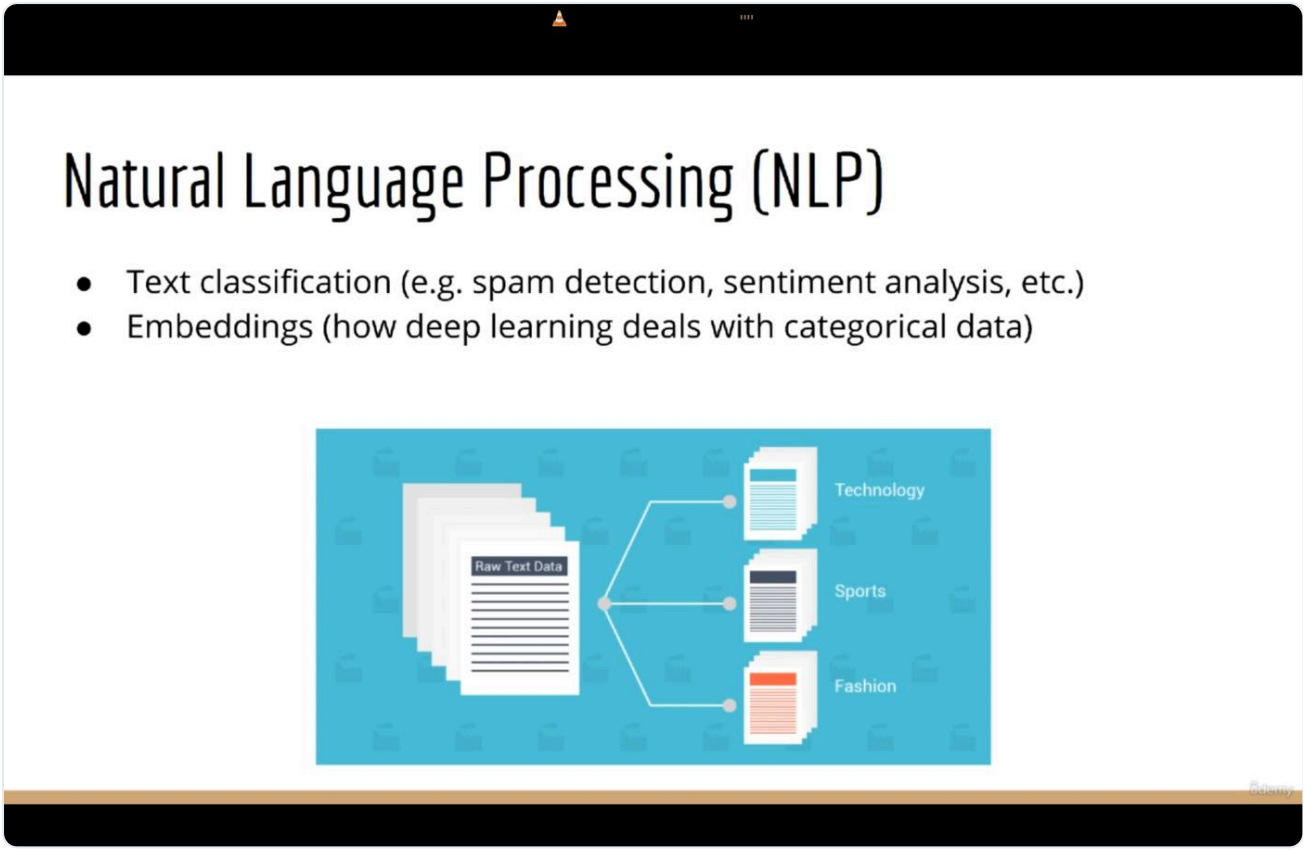


FIGURE 2.15 | Natural Language Processing (NLP)

The first application we will look at is natural language processing. Specifically with text documents you'll see how we can use deep learning for text classification which is the type of task you would use for spam detection sentiment analysis and so on.

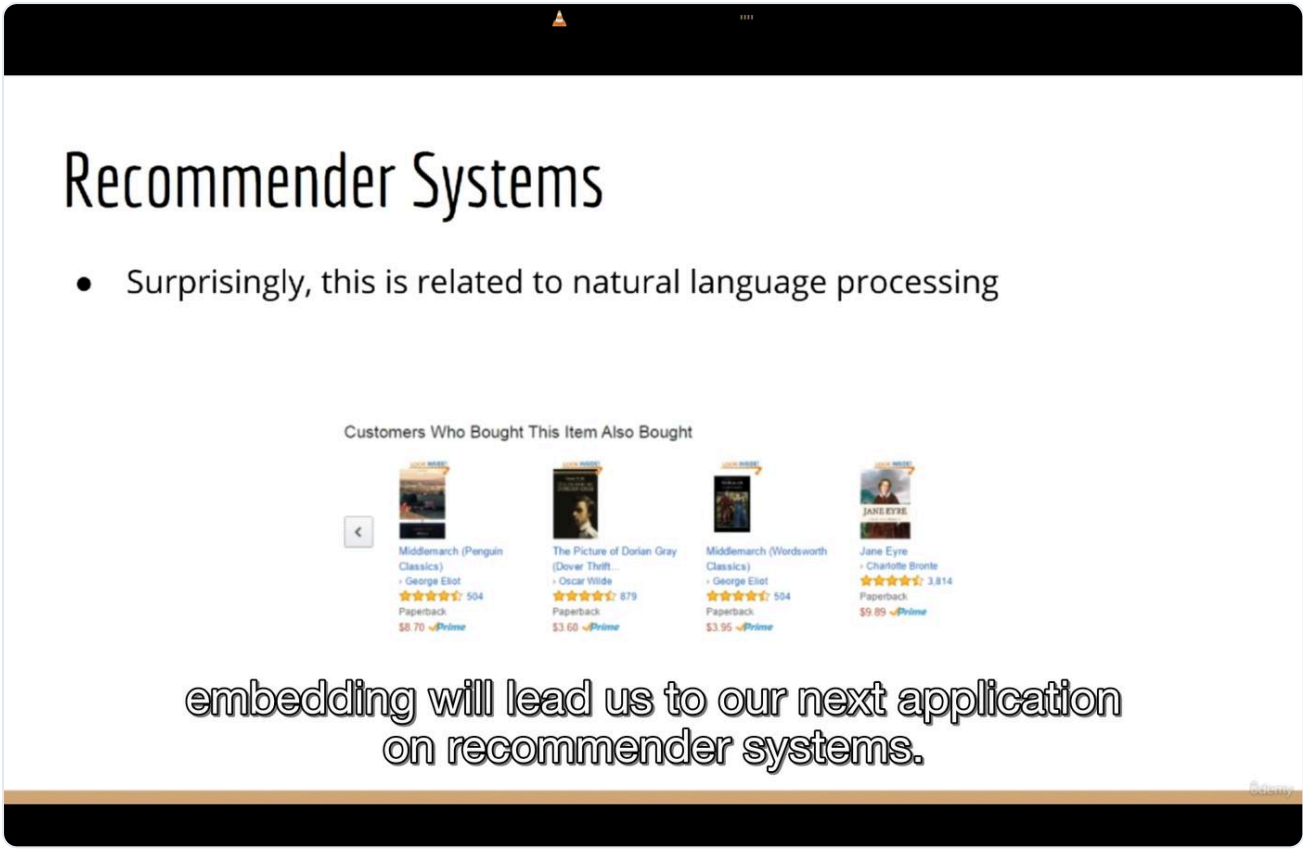


FIGURE 2.16 | Recommender Systems

We'll also look at embedding which are deep learnings method of dealing with categorical data embedding will lead us to our next application on recommender systems. Now it might seem weird to think that natural language processing would be somehow related to recommender systems but hopefully in this course you will learn about the hidden connection between these two different fields recommender systems is all about how to optimize the products or items that you recommend to your users. Facebook Amazon and Google have been using recommender systems to improve their profits by the billions. When you go to Amazon.com it is full of recommended products for you.

Facebook news feed and your Instagram feed are both recommendation lists. Google's search results advertisements and the YouTube sidebar are all examples of how Google uses recommendations. In short, recommender systems are one of the most practical business applications of deep learning and one that can be used in a pretty straightforward manner to improve your profits. This is in contrast to say natural language where it's not exactly clear how that would directly tie in to how much money your business makes.

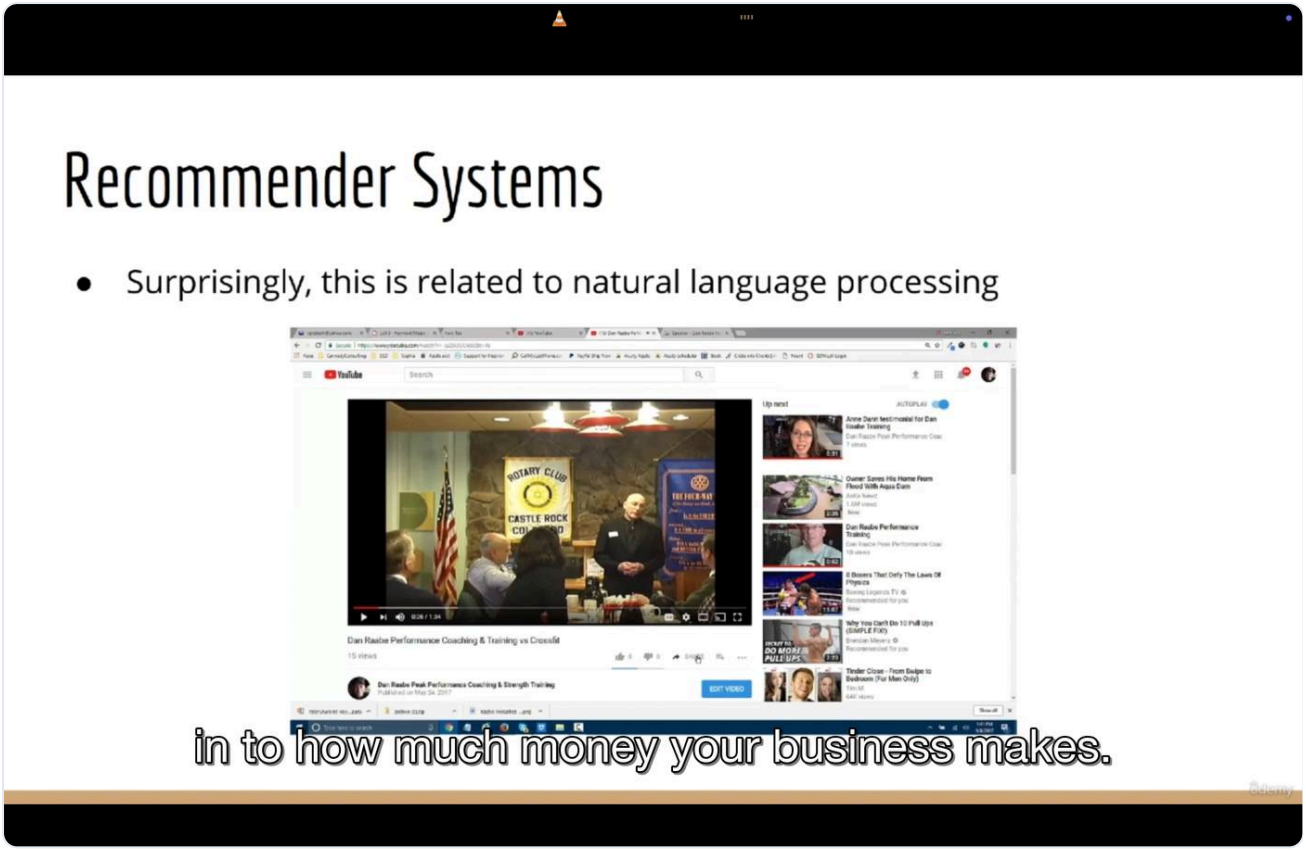


FIGURE 2.17 | Recommender Systems

Although it's already a fundamental part of our day to day lives. So don't discount it just yet. The next application we'll discuss was a game changer for deep learning. If you've taken any of my deep learning courses in the past you know that training a deep neural network takes time lots of time. You might end up waiting hours or even days or possibly even weeks. Luckily machine learning engineers have found new ways of allowing us to build on top of the work of others. The idea is companies like Google or Facebook or university research groups will train large neural networks on humongous datasets such as ImageNet which contains over a million images.

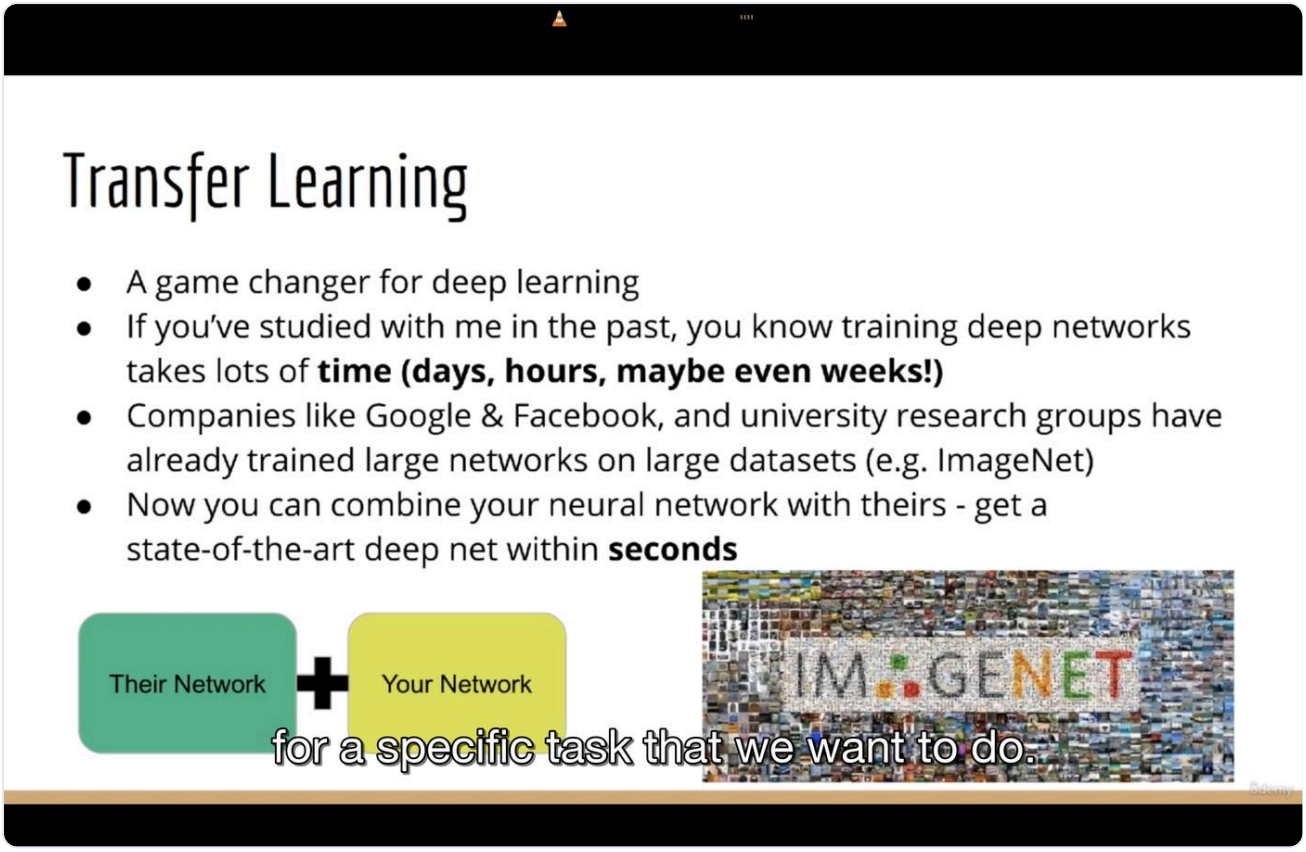


FIGURE 2.18 | Transfer Learning

That's not the kind of thing you can feasibly do on your home computer using transfer learning what we can do is take just a part of their neural network and combine it with our own neural network designed for a specific task that we want to do. Results have shown that this is an easy.

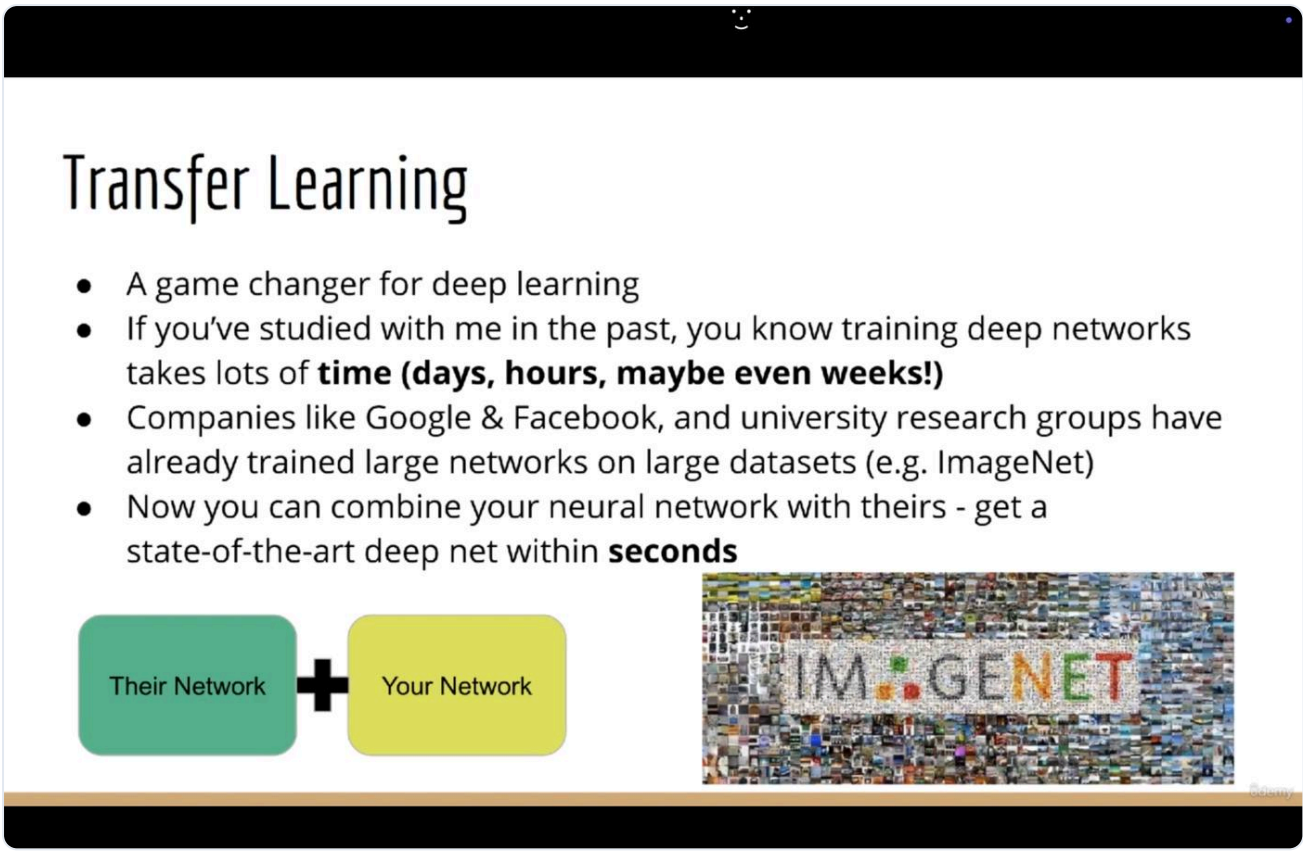


FIGURE 2.19 | Transfer Learning

And more importantly fast method of building state of the art deep learning models. In just a few seconds you can integrate the power of bleeding edge networks that works into your own applications. Next we have GANs or generative adversarial networks.

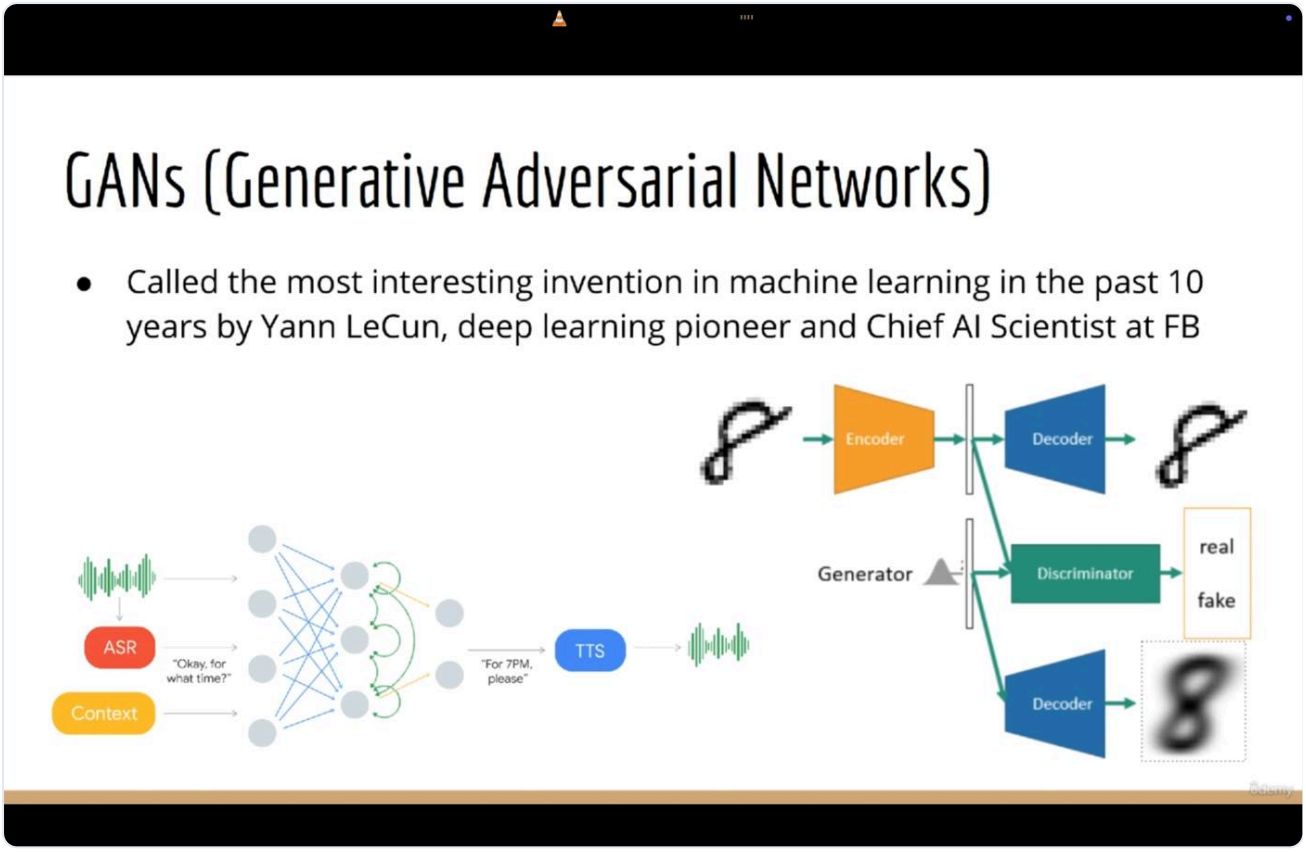


FIGURE 2.20 | GANs (Generative Adversarial Networks)

This was named as the most interesting invention in machine learning in the past 10 years. By Yann LeCun -- a deep learning pioneer who is now the chief A.I. Scientist at Facebook. GANs are all about generating images using a system of two neural networks one that generates the images and one whose job it is to discriminate between a real and fake images. Using this pair of neural networks they both help each other improve so the discriminator becomes better and better at recognizing fakes. While the generator becomes better and better at creating realistic images that cannot be differentiated from fakes when people think about deep learning they don't often think about how neural network can be used to generate things but rather just how neural networks can make predictions on things.

Predictions are nice but generating things opens up a whole new world. Recently Google announced a conversational A.I. Where you could have a personal assistant do something like call a restaurant and make a reservation for you. But that personal assistant is not a person. It is a robot.

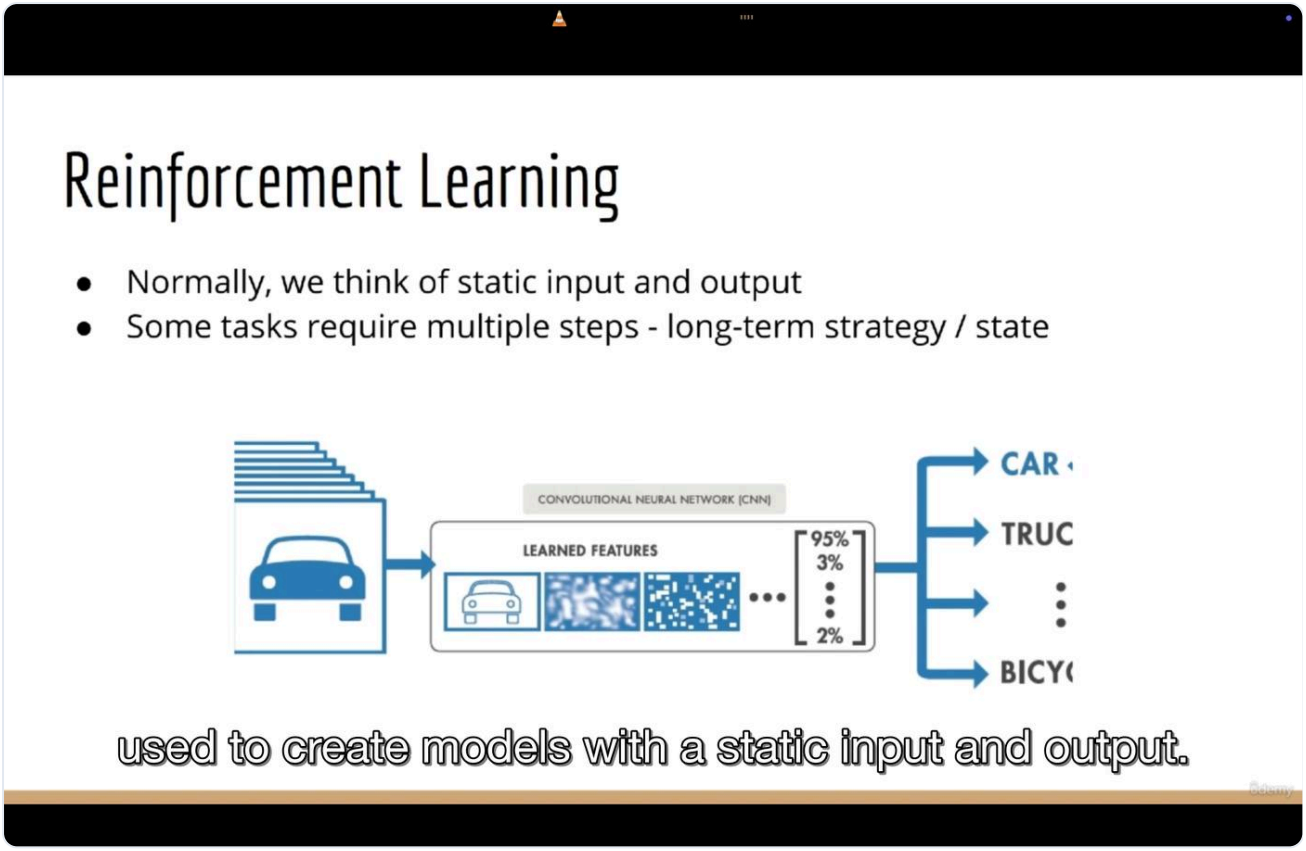


FIGURE 2.21 | Reinforcement Learning

All of its speech is generated by a computer another exciting application of deep learning is in a reinforcement learning usually deep learning is used to create models with a static input and output. For example you put in an image and it tells you what this image is of like a car or a truck or you put in an email and you're known that work tells you if it's spam or not spam.

FIGURE 2.22 | Reinforcement Learning

But some tasks require multiple steps which requires having some long term strategy and keeping track of state. For example if you are playing a game like Super Mario you can't just look at a still image of the game and decide what to do. Instead you need to have a long term strategy. You have to know that you should walk forward to reach the flag for that level. You have to know that if an enemy is walking towards you you have to avoid it or attack it. And also you have to have proper timing in doing so. This is what reinforcement learning is all about. To summarize the outline of the Course is like this. We can split it into two parts Part 1 architectures and part 2 applications.

In part one we discuss the fundamental architectures such as linear models, ANNs, CNNs, RNNs. In part two, we discuss applications such as NLP, recommender systems, GANs, transfer learning, reinforcement learning and possibly more. Thanks for listening and I'll see you in the next lecture.

LECTURE 3

Where to Get the Code

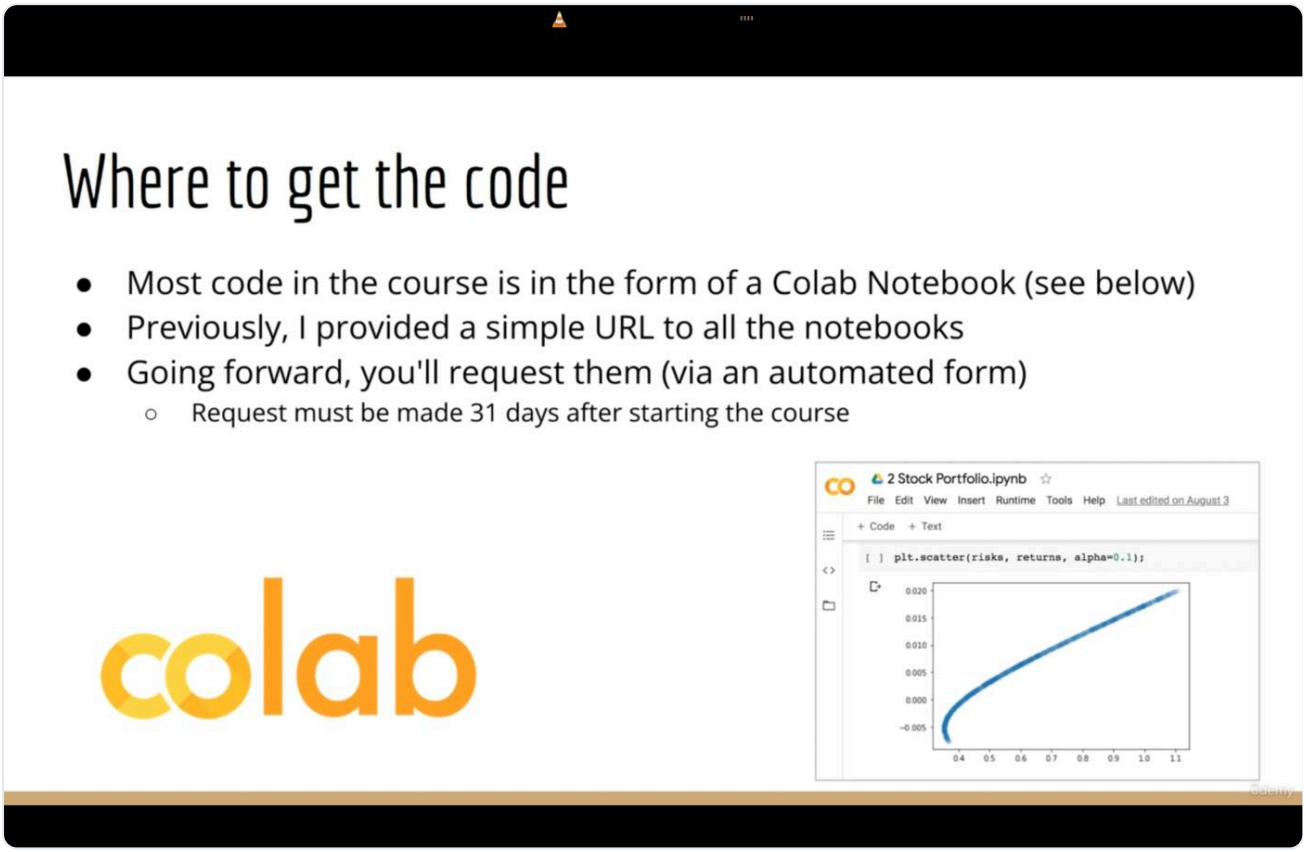


FIGURE 3.1 | Where to Get the Code

So in this lecture, we are going to discuss where to get the code for this course. The code in this course mostly comes in the form of Colab notebooks. In previous courses, I've provided a simple URL where you could download the code at your leisure. But in this course, I'm going to try something new. In order to get the notebooks for this course, you'll need to make a request for the notebooks by filling out a form. Furthermore, you'll only be able to make this request 31 days after you started the course. Okay. So specifically, how can you get the notebooks for this course? You'll need to go to deep learning courses dot com slash code request.

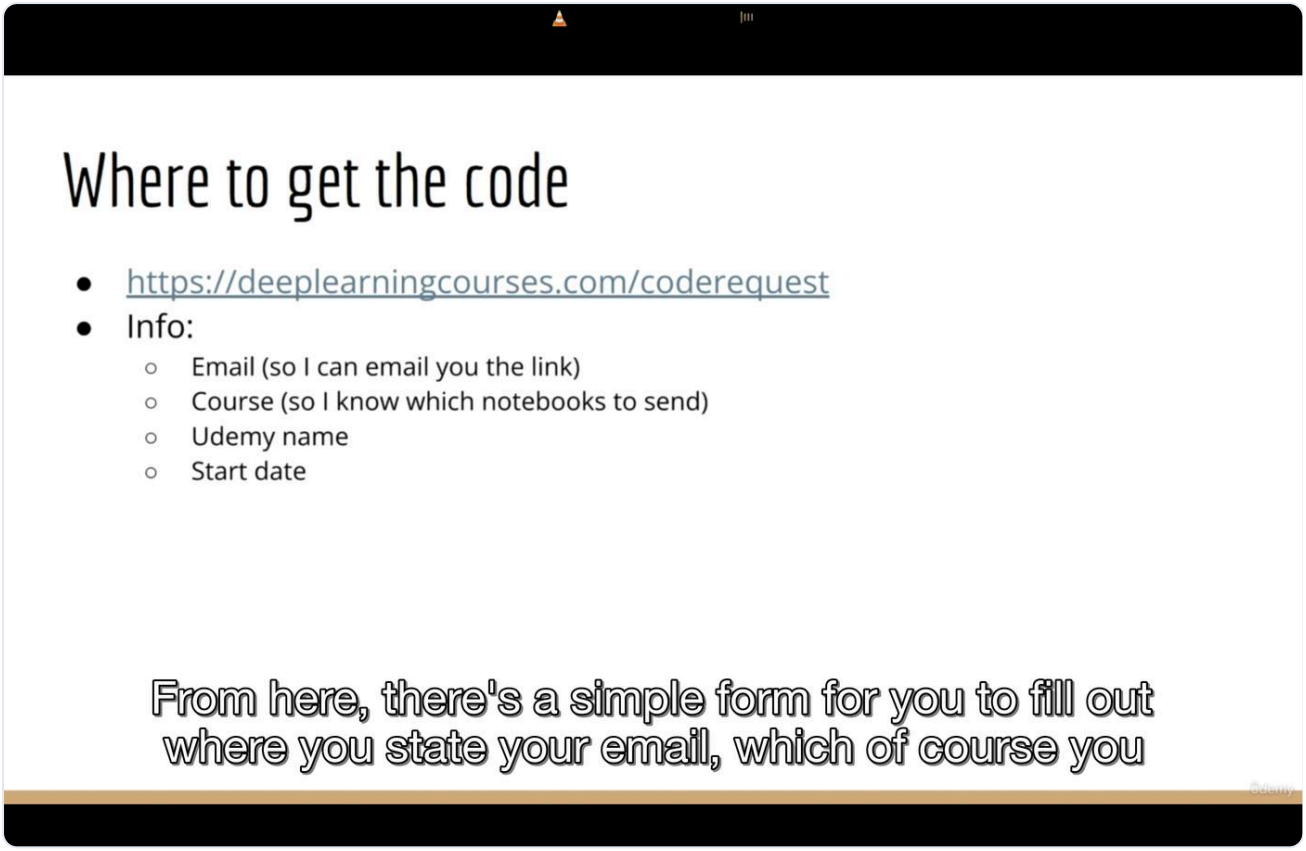


FIGURE 3.2 | Where to Get the Code

From here, there's a simple form for you to fill out where you state your email, which of course you are taking, that you would like the code for your name as it appears on Udemy and the date on which you purchase the course. The final two pieces of information are important because I won't be able to look you up by email. These will allow me to check when you signed up to confirm that it has in fact been 31 days since that time, although you still do have to provide your email so that I can email you back with a link to the notebooks. And please note that if you do email me directly, you'll still need to provide the above information. This form just makes it simpler by telling you exactly what information to submit.

Now you might be wondering, why am I doing this? The goal of this is to encourage students to code by themselves. Too often I see students who make it abundantly clear they have not attempted to write any code themselves. You may have noticed that coding by yourself is one of the most important activities in this course. In fact, there is a lecture all about it called How to Code by Yourself. I encourage you to watch this lecture. If this is confusing or you have doubts about why I would ask you to do this by making you wait 31 days.

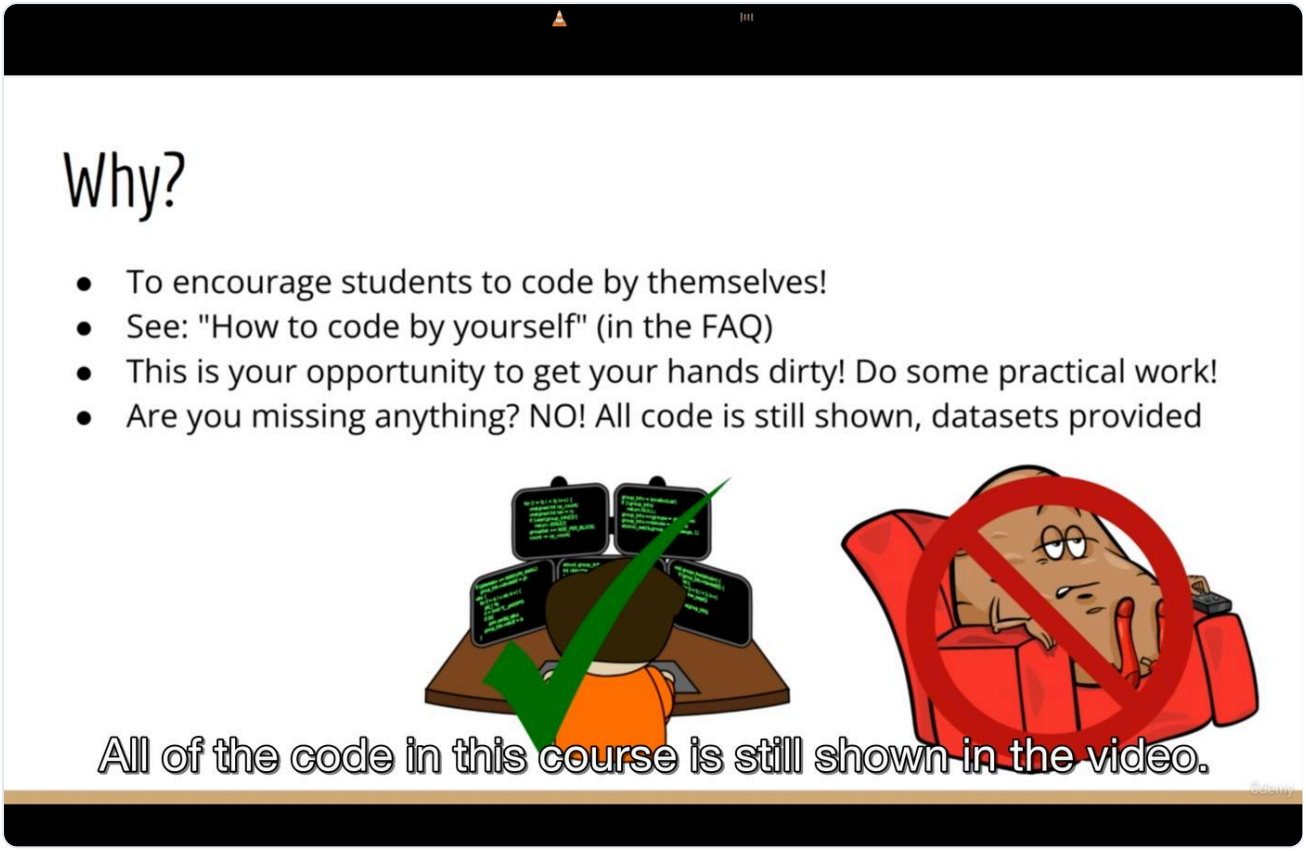


FIGURE 3.3

I'm giving you an opportunity to get your hands dirty and actually do some practical work. To be clear, you are not missing anything at all by not getting these notebooks right away. All of the code in this course is still shown in the video. You'll still be given the links to all the data sets you need. The only difference is that instead of me typing the whole thing for you, you're getting practice typing things by yourself. This builds muscle memory, which many people don't understand the importance of.

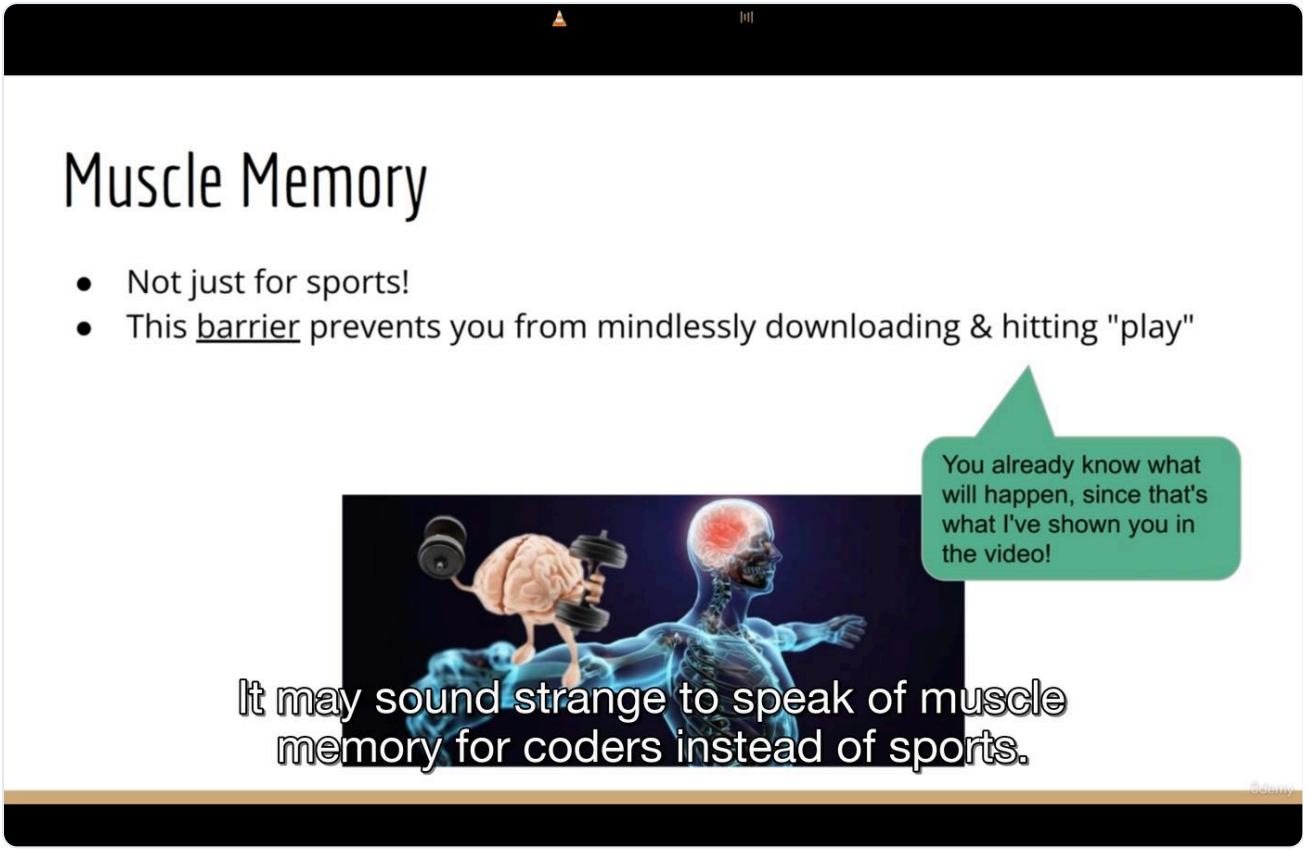


FIGURE 3.4 | Muscle Memory

It may sound strange to speak of muscle memory for coders instead of sports. But after trying it yourself, you will understand clearly how powerful it is. What I'm doing is I'm putting a barrier in between you and your desire to simply go to a URL. Hit the play button and watch things work. Okay. So your next question might be, well, where do I get the datasets? This is very simple. Just go to the resources for this lecture and you will see a link to the datasets. Okay. So aside from just notebooks, there is yet another place to get extra resources for this course. For example, reading materials, data processing scripts and other miscellaneous things. To get these, please go to my GitHub repo github dot com slash lazy programmer slash machine learning examples.

This contains materials for all my courses in a single repo and it's organized by folder. Essentially it's one folder per course, although there are some exceptions.

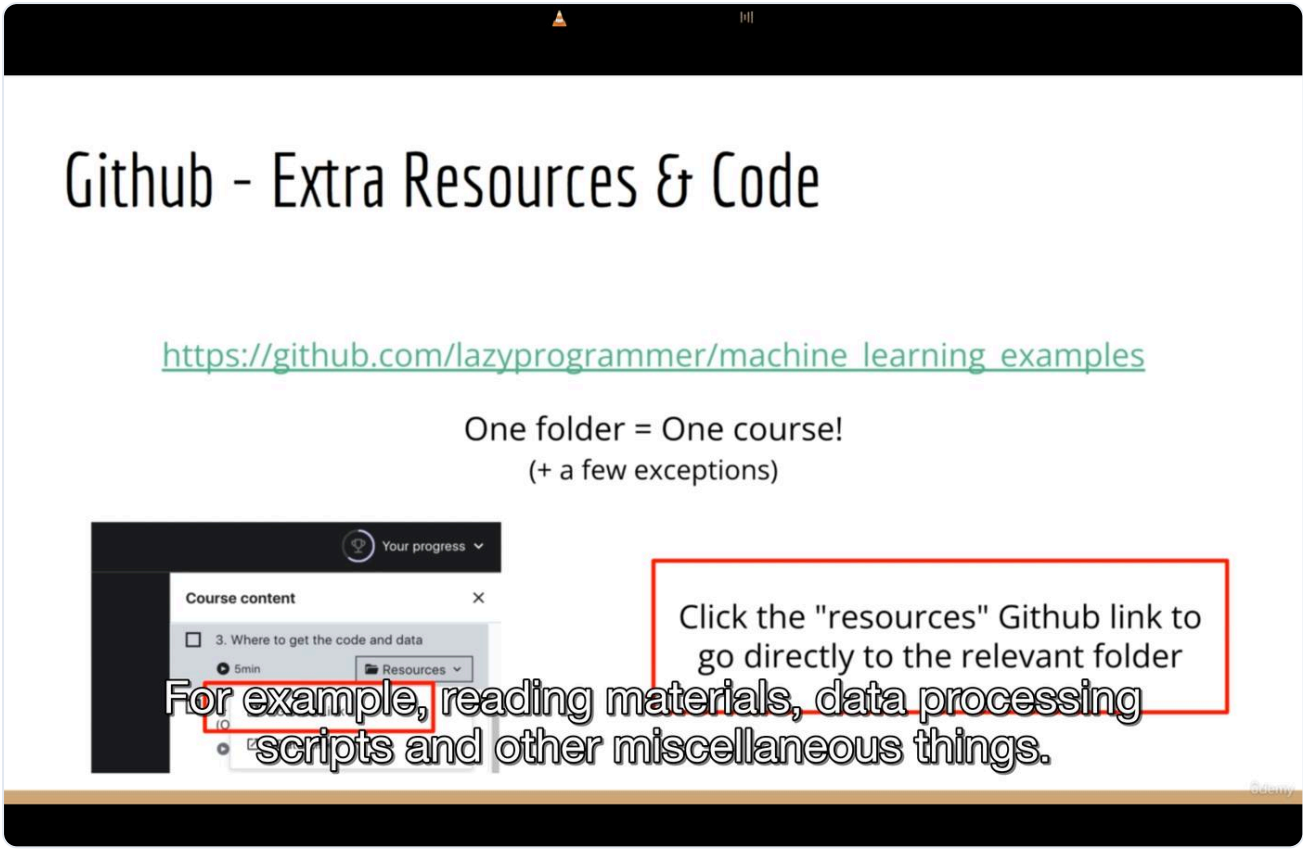


FIGURE 3.5 | Github - Extra Resources & Code

In order to go directly to the folder for this course, please click the resources link attached to this lecture. Now if you're new to Git or GitHub and you're a bit intimidated, please see the upcoming lecture called How to Use GitHub and Extra Coding Tips. Now there's one mistake that many students make, so it needs to be explicitly stated that the Colab notebooks are not on GitHub. I repeat, the Colab notebooks are not on GitHub. Let me repeat that one more time.

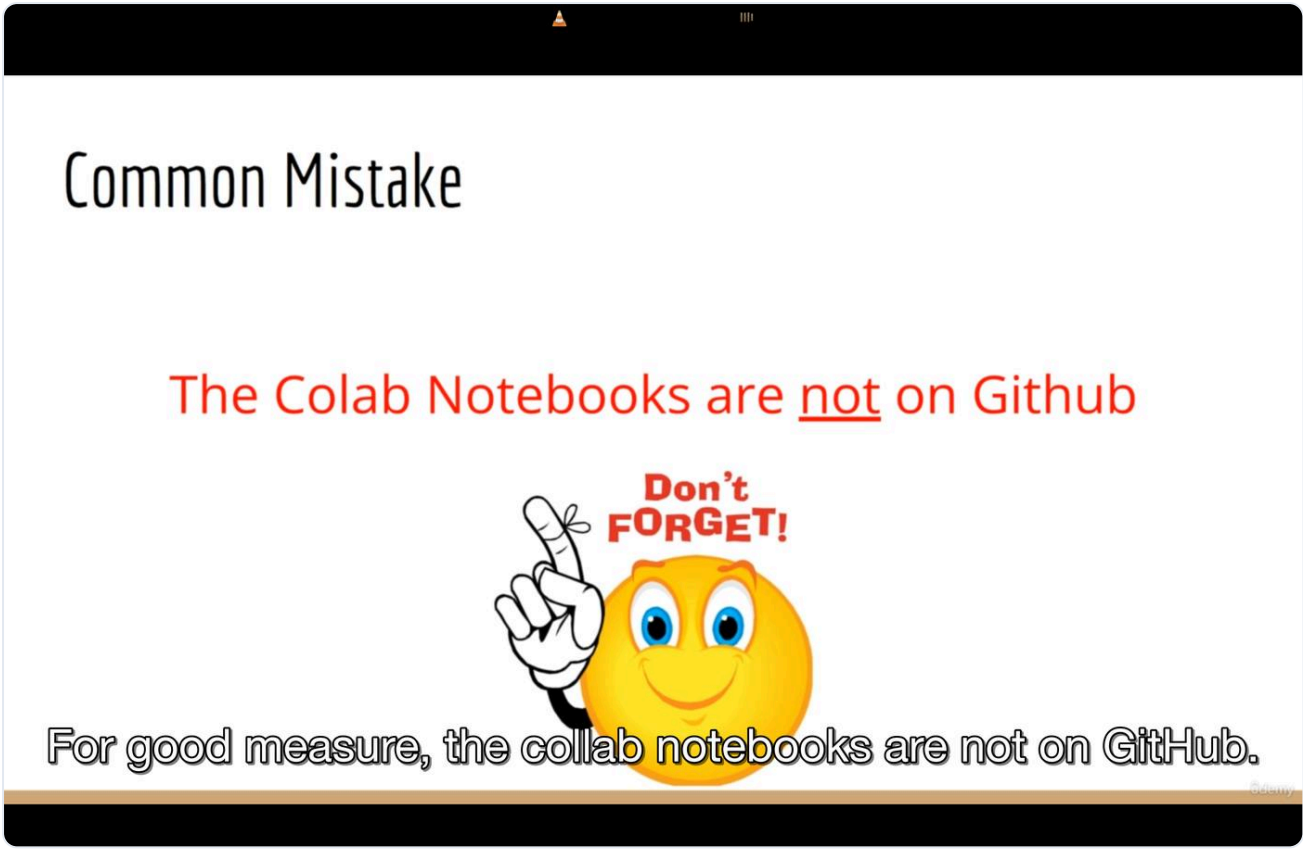


FIGURE 3.6 | Common Mistake

For good measure, the Colab notebooks are not on GitHub. Now there are a few more common mistakes and misconceptions that need to be discussed. Firstly, some students, for some reason, get the timeline completely backwards. They believe that they must request the code within the first 31 days of signing up for the course. Otherwise they will lose access to the code forever. This is not correct. Furthermore, this doesn't make any sense at all. The correct timeline is this for the first 31 days you are taking this course. You should be coding by yourself. This is good exercise. After 31 days, you can and should submit a code request to get access to the official bonus notebooks.

Remember, the goal is to stop you from just downloading notebooks and hitting play, which is a useless activity because the videos already show you what will happen when you do that. There will be no difference if you hit play or if I hit play. It's the same thing. The second thing students are getting wrong is that they think they can't access the data for this course because the URLs go off the screen in the videos. Again, this is not correct. You should not be typing URLs by hand, which is again, a useless activity. Also, do not try to search for the data links on Google or any other search engine because you'll just end up getting the wrong data set.

Instead, the data links have already been provided. As mentioned earlier in this lecture. The third thing students are getting wrong is that they believe they will never get access to the notebooks. I'm getting people emailing me saying I thought this course comes with notebooks. The answer is, yes, it does. You will get the notebooks. All you have to do is follow the instructions. So thinking you will not get access to the notebooks is completely incorrect. Of course, there are

caveats to this. For example, if you spam the website with 20 requests, enter the incorrect signup date or display any other bad behavior, then you may forfeit access. One more extreme comment I got once said that I have a contractual obligation to provide notebooks, which is not correct.

On the contrary, the notebooks should be considered as a bonus to this course. This platform is actually a video course platform, and the primary mode of instruction is video. Other, more dishonest students are saying things like, I purchased this course because, as said in the course description, that I will get the notebooks. Guys, this is very silly. You can't lie to me about what it says in the course description because I'm the person that wrote the course description. If you do this, you'll be rewarded accordingly. Finally, I just want to say that these misconceptions actually strengthen the case for implementing this policy because they show that some students simply are not listening to the lectures and gathering the information I have provided by listening more carefully and being a more diligent student.

These mistakes can be avoided, and moreover, your chance at success will increase.

LECTURE 4

Git

001 Data-Links https://docs.google.com/document/d/1awPuAtUj16nD5vC0nwyc3hJIN-7uSK_ANkllHyKRA7U/edit?usp=sharing

001 Github-Link https://github.com/lazyprogrammer/machine_learning_examples/tree/master/pytorch

Now, if you've never used GitHub before, then there are several ways for you to download the course repository. There are also several ways you should not download the repository. So first, let's start with some options for what you should do. Option number one use the git clone command. You can enter this in your console. As you can see here. Option number two if you're not comfortable using the console, then simply use the download button. Now, one thing to keep in mind is that the Internet is a constantly changing place, so do not freak out if GitHub has a new design by the time you visit this page. I am confident in your ability to figure this out.

Consider this your first exercise for the course. By the end of this lecture, you should have the entire course repository downloaded to your local machine using either of the two methods above. Now please note that if you're new to Git or GitHub and you find yourself a bit intimidated, please do not be. You do not have to know how to use Git for either of the previous two methods. Now, here are some things you should not do that I've seen several students do in the past. Number one, do not try to download each file one by one. As mentioned previously, this is not the way to use Git number two. On a similar note, do not go to file and say that this is not how to download files from GitHub.

Doing this will download the webpage to view the file, not the file itself. In fact, number one and number two are very silly because they actually take a lot more effort than simply using either of the methods I mentioned earlier. Number three, if you want to fork my repository, please feel free to do so. But try your best to keep it up to date. I make updates all the time, so please make sure to pull these updates. If you don't think you'll be able to do so, then please use the git clone command instead. And on that note, when you do download my repository, make sure it's actually my URL and not someone else's fork. Number four. This might sound silly, but your fellow students have proven to me that this needs to be said.

There is no need to type anything character by character. Simply use the copy button. So one extra note about the GitHub repo. Most courses also contain a file called Extra Reading Text. This file contains reference material for the course like journal articles, books and so forth. So if you hear me referring to extra reading text in the lectures, this is where it can be found. Reading these materials is optional, but they are there in case you want to further your studies. Okay. So I want to mention a couple of points pertaining to the code in this course. Firstly, for some of you, especially those who don't necessarily meet the prerequisites may be asking what is Git and how do I use it?

The short answer is Git is a way for developers to manage and track changes in their code.

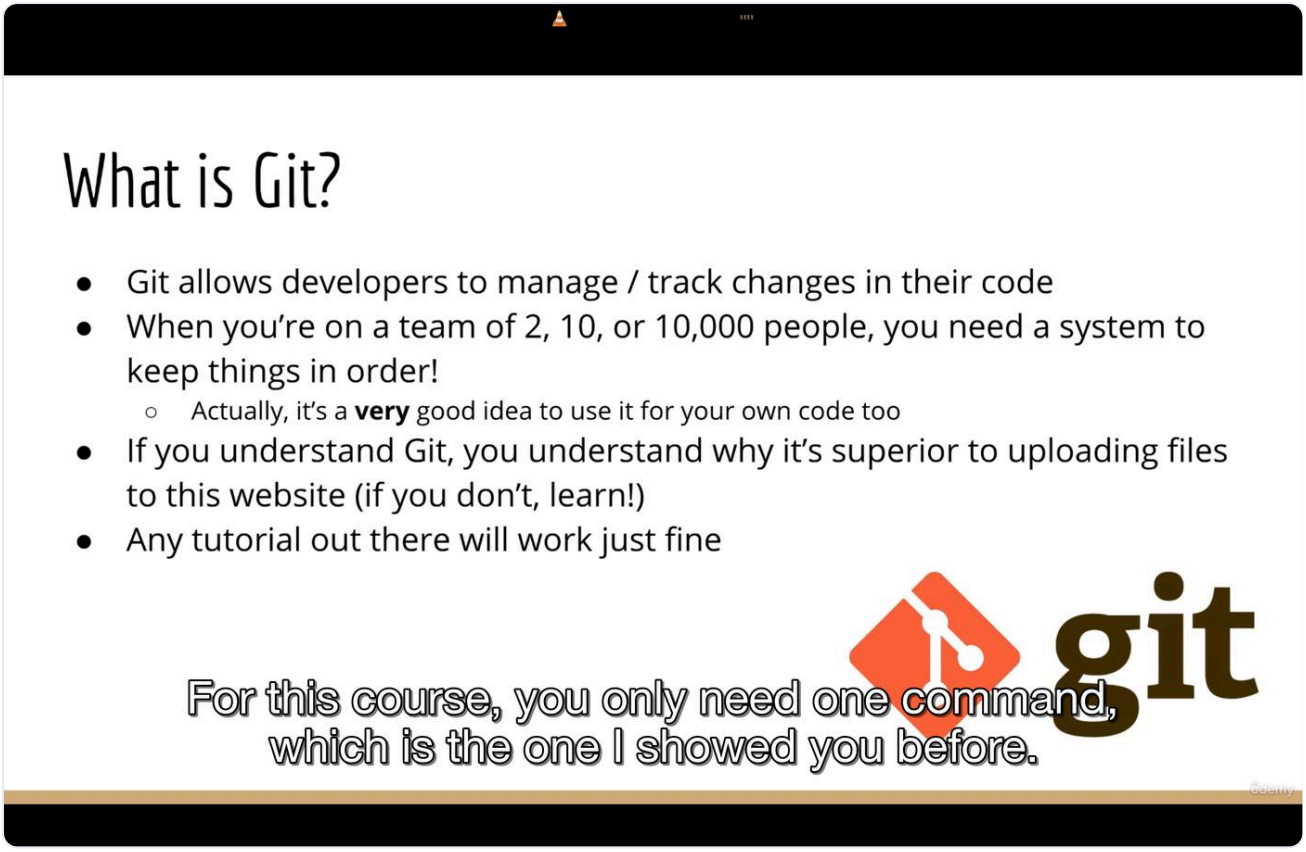


FIGURE 4.1 | What Is Git?

You might be used to writing code by yourself, but when you're working on a team of two or ten or even 10,000 people, you need a system to keep things in order. So this is why, for example, Git is superior to uploading files to this website. Learning how to use Git is very simple. There are many tutorials out there that will work just fine. For this course, you only need one command, which is the one I showed you before. So you don't even need to understand Git. You just need to copy the command I gave you. Therefore, I don't suspect this will be an issue unless you also don't know how to run commands. But if that is the case, then as mentioned, you can simply click the button that says download.

At this point, I'll assume you need no further resistance. And just as a side note, although you don't need to know how to use Git for this course, it is a valuable skill that will be necessary if you ever plan to work for any company where your job is to write code. Now, another common question I see from beginners is Should you code along? The short answer is no. Now, you may have taken online courses in the past that encouraged you to code along. But the reality is, this is going to give you a very false impression of how code is actually written. Coding is not a monkey see monkey do type of thing. If you're learning to play guitar or some yoga moves, yes, that makes sense.

But coding involves lots of thinking, not just typing. If you think you're going to write a complex algorithm in 20 minutes without actually paying attention to design of structure and without debugging and going back to fix things, think again. Real code in the real world is not written linearly from start to end.

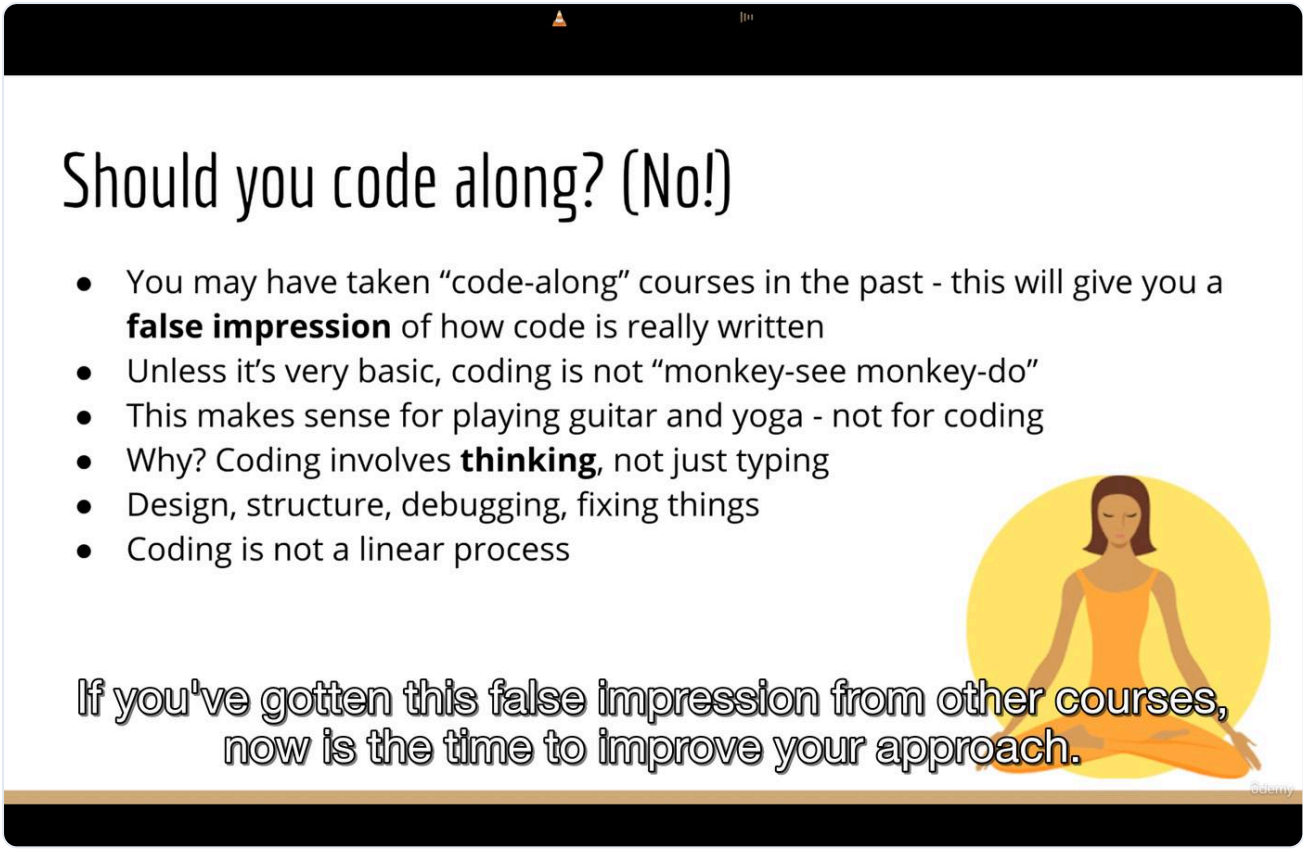


FIGURE 4.2

If you've gotten this false impression from other courses, now is the time to improve your approach.

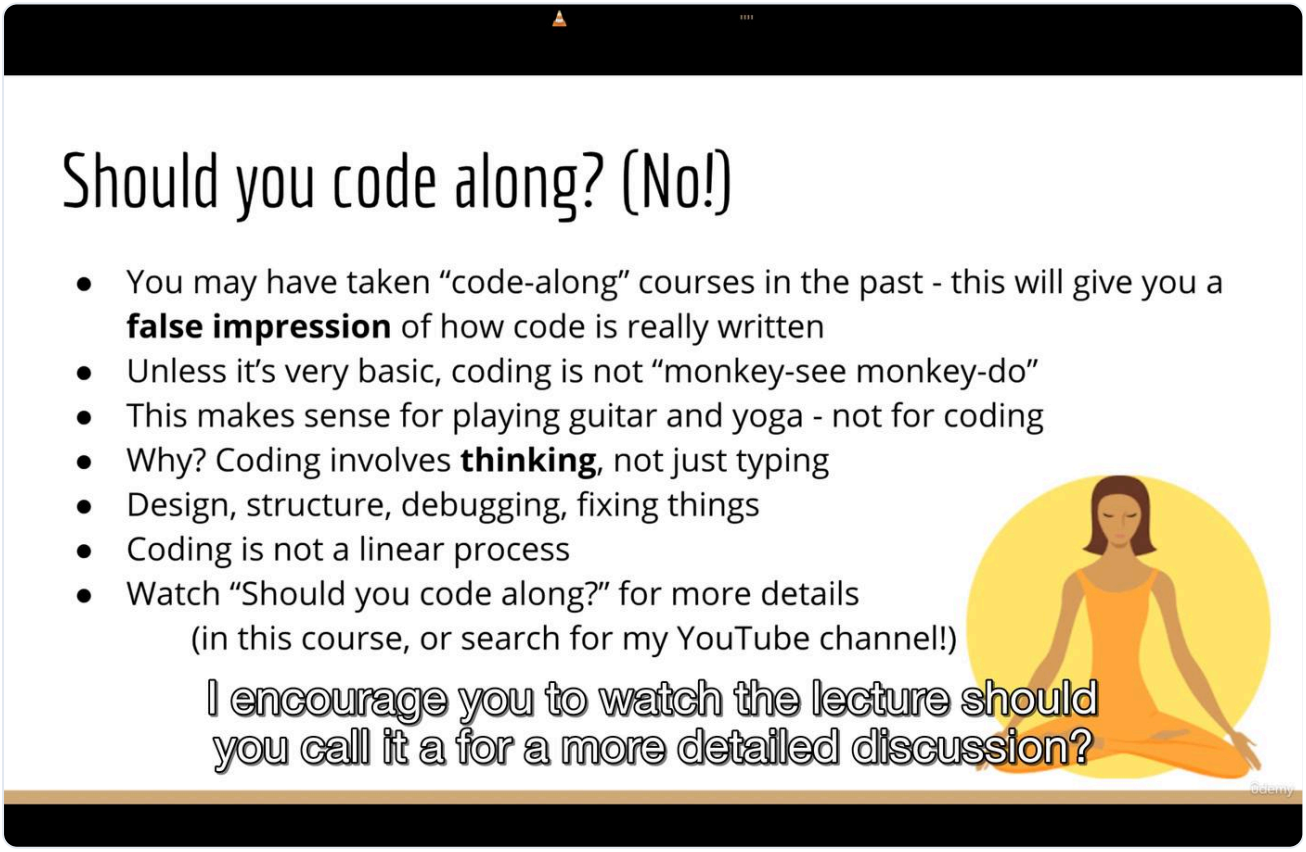


FIGURE 4.3

I encourage you to watch the lecture should you code along for a more detailed discussion? This brings us to our next point, which is what are the exercises for this course? Now, in some cases, this will be obvious because I'm going to say here is an exercise, please do it. But sometimes it's going to be less obvious. And I'm going to trust that you've watched this lecture and understood what I'm saying. Basically, any time that you see code, that is an exercise. There are two types of code you might see. Number one, I've just taught you an algorithm, in which case the next code lecture will involve an implementation of that algorithm.

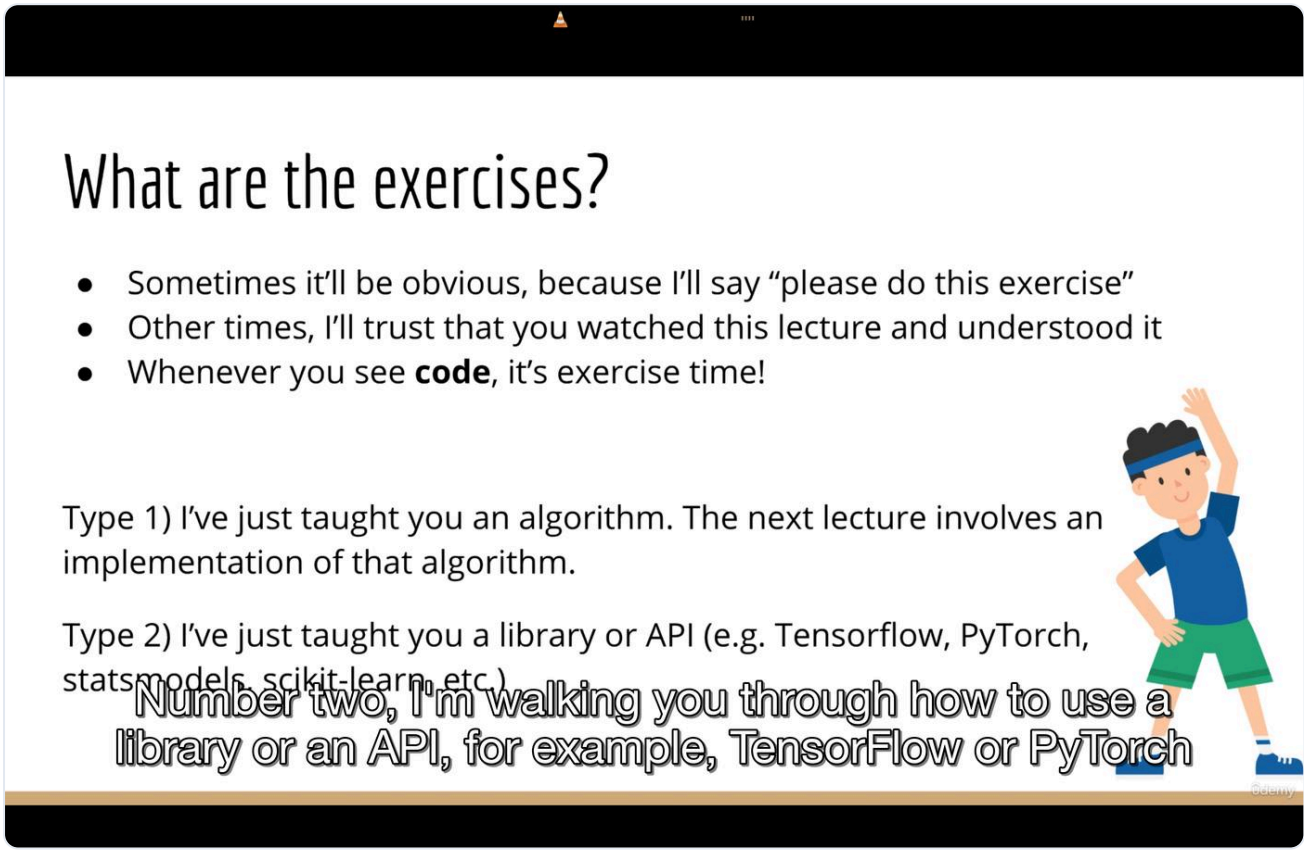


FIGURE 4.4 | What Are the Exercises?

Number two, I'm walking you through how to use a library or an API, for example, TensorFlow or PyTorch or stats models. So what is your exercise?

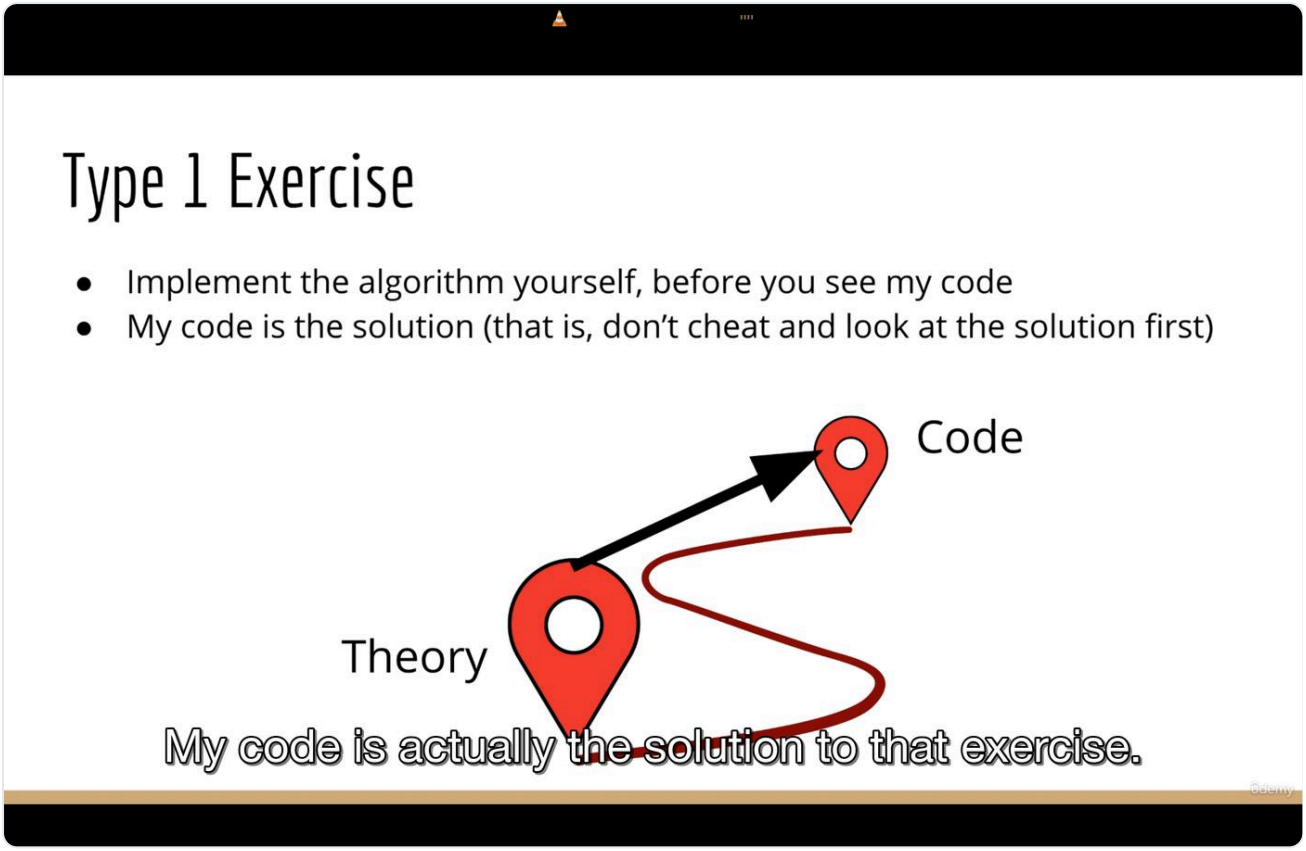


FIGURE 4.5 | Type 1 Exercise

For case number one, your exercise is to implement that algorithm yourself before you ever see my code. My code is actually the solution to that exercise. So don't take what I call the couch potato approach where you just sit there watching each lecture one by one, without doing any coding. That is not the intent of this course. Now, obviously what you do is up to you, but I hope that you'll do the exercises as intended. Now for case number two, it's a little bit different. Obviously, you're not going to know what the syntax is for a library or an API. Before I even show it to you. In this case, you should go ahead and watch the full code lecture without first doing any coding yourself.

After you've watched the lecture, you should be able to implement that same code yourself on another dataset that you care about.

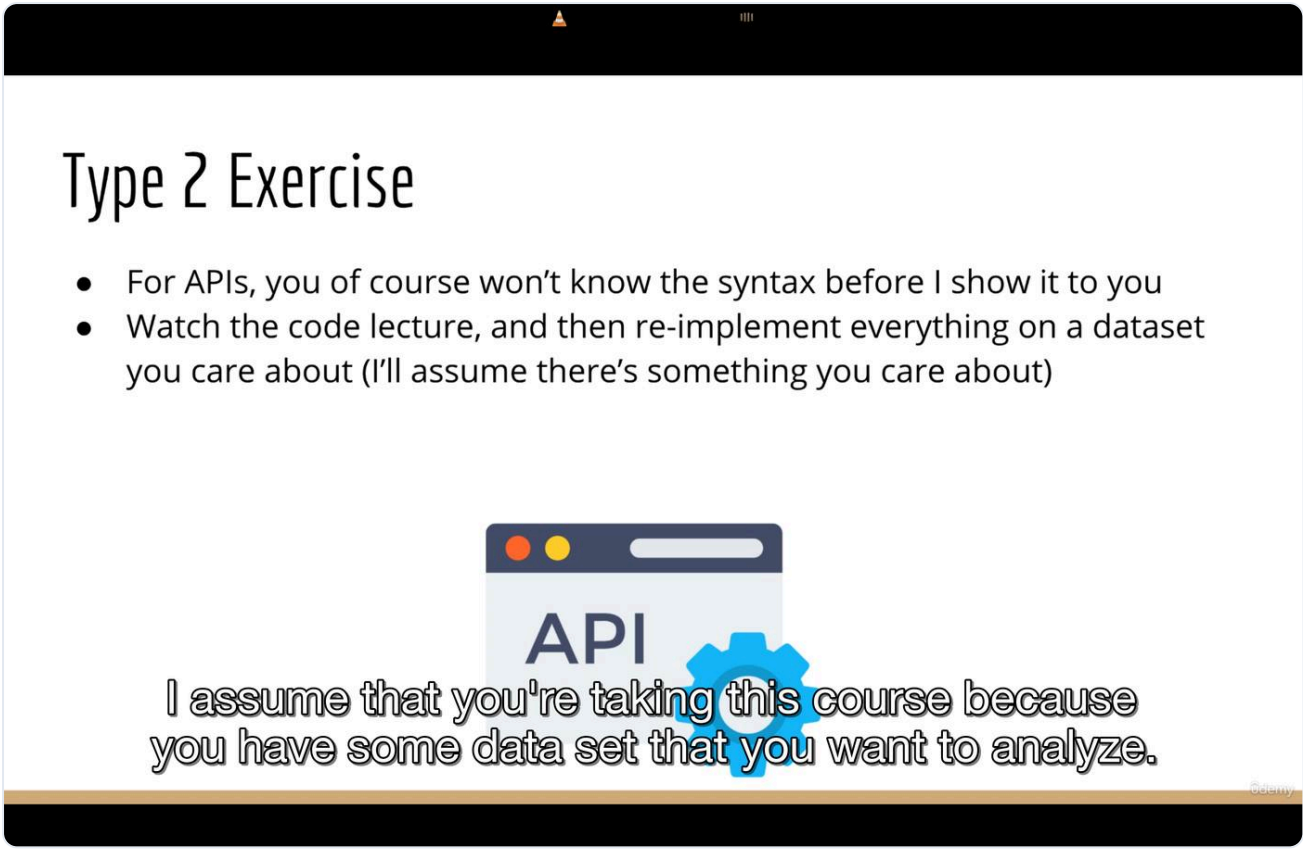


FIGURE 4.6 | Type 2 Exercise

I assume that you're taking this course because you have some data set that you want to analyze.

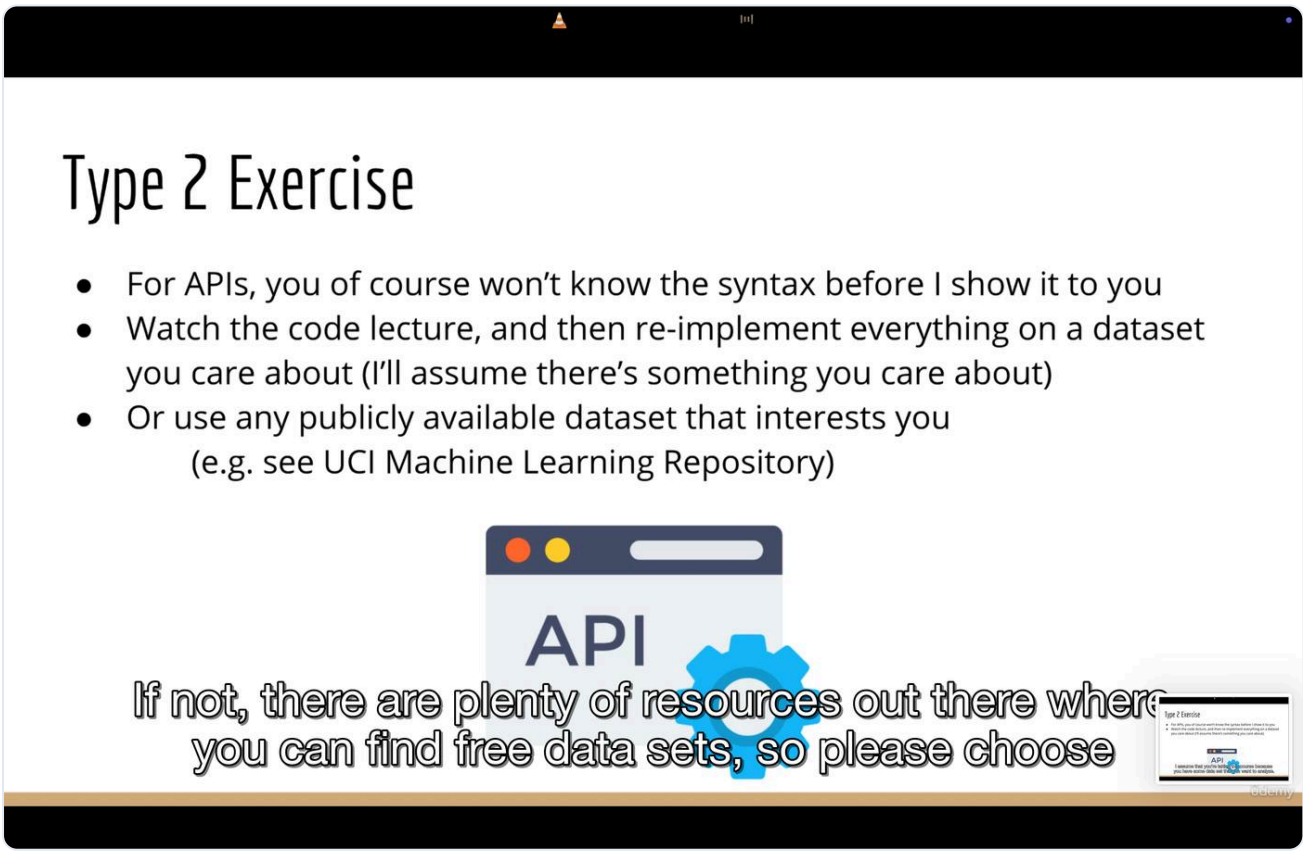


FIGURE 4.7 | Type 2 Exercise

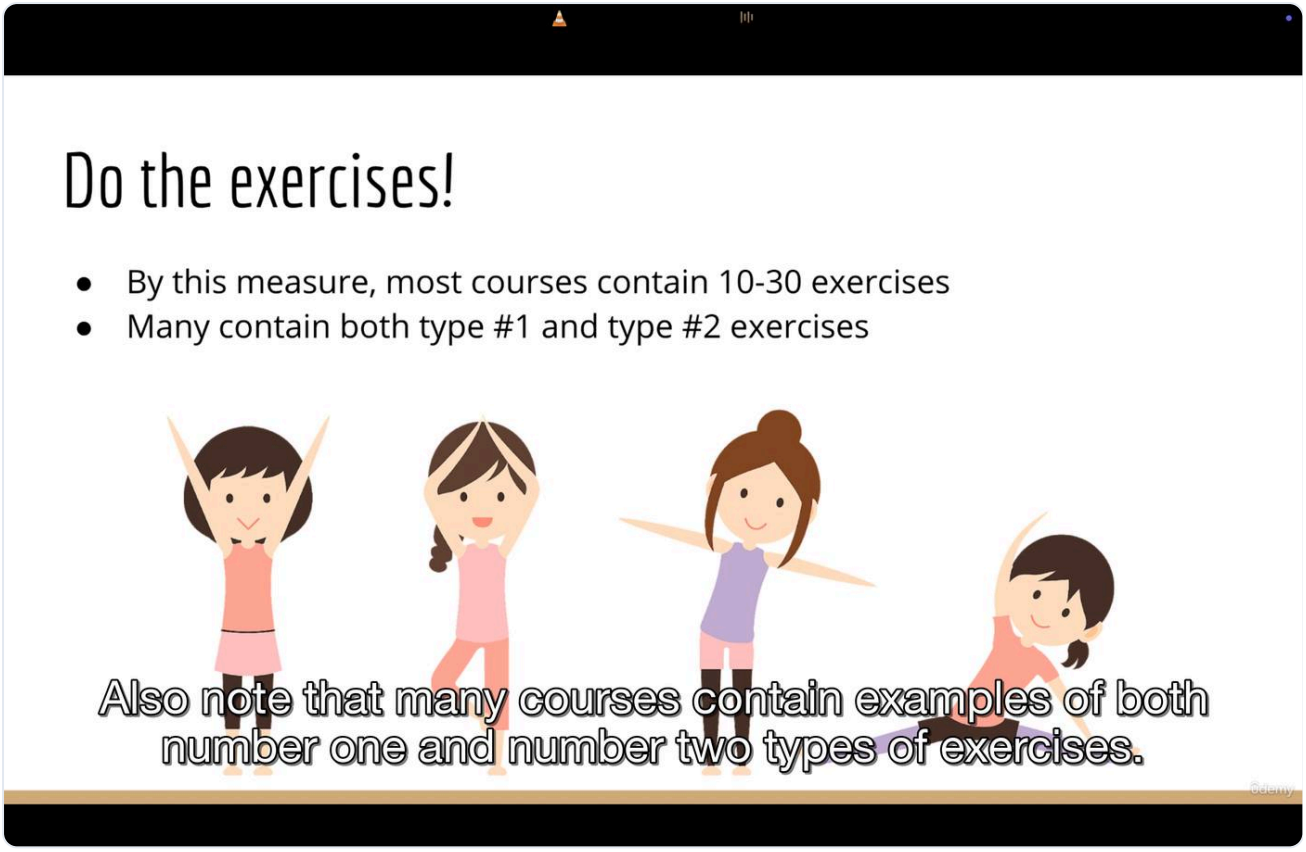


FIGURE 4.8

If not, there are plenty of resources out there where you can find free data sets, so please choose one that interests you. Note that by this measure, most of my courses have at least 10 to 30 exercises. So try to do as many as possible. Also note that many courses contain examples of both number one and number two types of exercises. You should be able to recognize it when you see it. Okay, so hopefully the exercises for this course are clear. And if not, please make a question on the Q&A. Now the next question I want to tackle is a funny one because I honestly didn't think this was a question I would need to answer, but here we are. So some students really like Jupyter Notebook very much, while other students dislike it.

Some students really like PyCharm. So you might ask, why am I not using whatever your favorite tool is? The answer is this question is totally irrelevant. Now, you might think I'm being dismissive of your concerns, but in fact, the reason people have this question is because they don't yet know how Python works. After this lecture, I hope you'll have a better understanding. So in this course I'm providing you with Python code, you'll get that python code either in a colab notebook or in a python file. But there's really no difference. Remember that in this course, this is a video course. You're just seeing code on a screen. What I use is irrelevant because you can't touch it nor interact with it.

I may as well be using Microsoft Word and it would make no difference. What you do with that python code is totally up to you. You can take my Python script and paste that into a Jupyter notebook or PyCharm or open it up in Visual Studio code. It's completely your choice. Python code works the same no matter what environment it's in. The same code will work in a python

file, in Jupyter Notebook, in CoLab notebook and so forth. And if you're having trouble understanding this, please use the Q&A so I can help you sort it out. Thanks for listening and I'll see you in the next lecture.

LECTURE 5

Connect Apps to Drive

In this lecture we are going to go over a very new and exciting environment for writing deep learning code in Python which is Google's Colab short for Colaboratory. For those of you who like to use Jupyter notebook this is an even better option. This is basically the same as Jupyter notebook with the following bonuses first. It's hosted by Google which means you don't have to use your own computing power. You'll notice that when you need to download data files it happens extremely quickly because while Google's network is extremely fast.

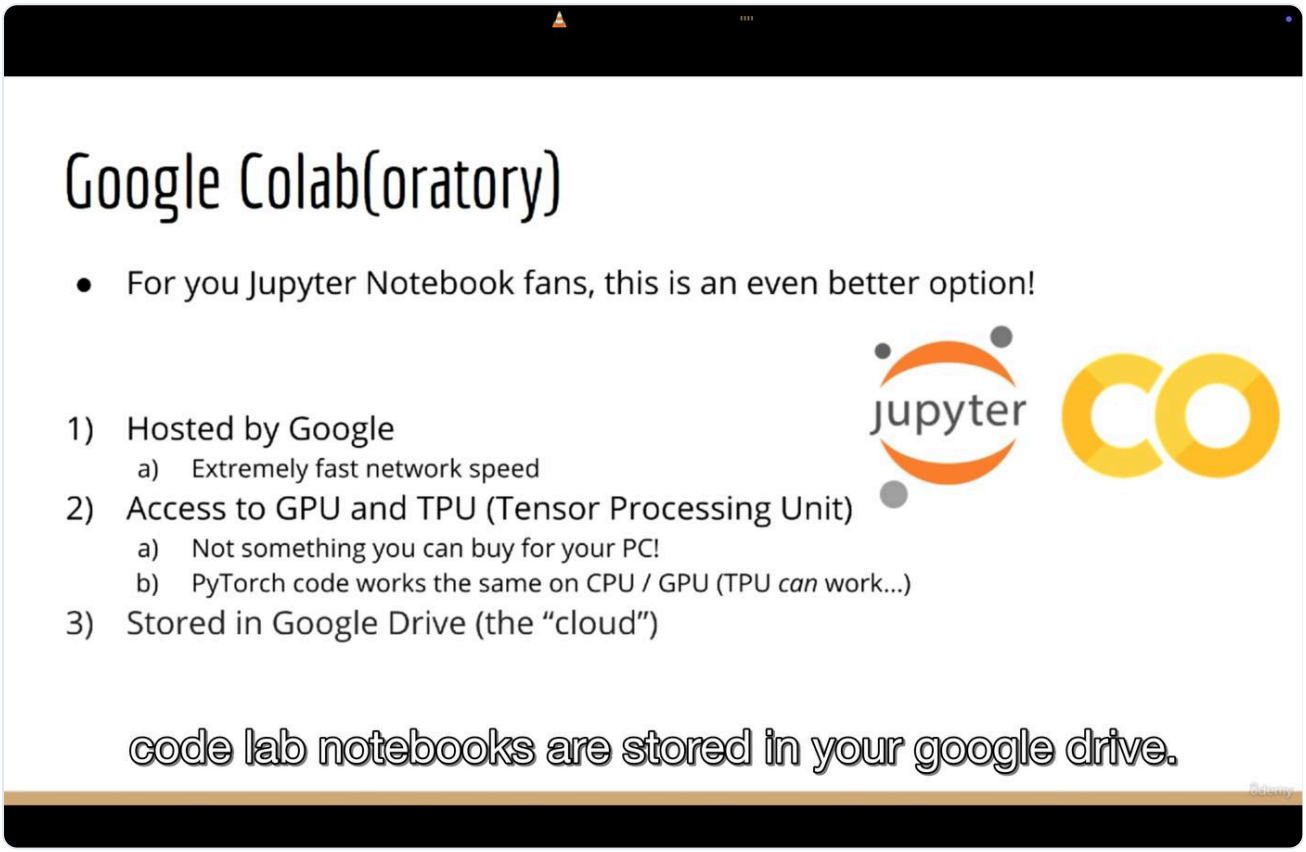


FIGURE 5.1

Second you get access to a GPU and even Google's new TPU which is pretty amazing a TPU is not something you can buy for your personal computer. So it's pretty nice to be able to make use of one. Remember that the way PyTorch code is written you don't have to worry about what kind of device you're using. Well for the most part TPU is another story. But generally speaking the same code will work whether you're using a CPU or a GPU third the Colab notebooks are stored in your google drive. So it's in the cloud.

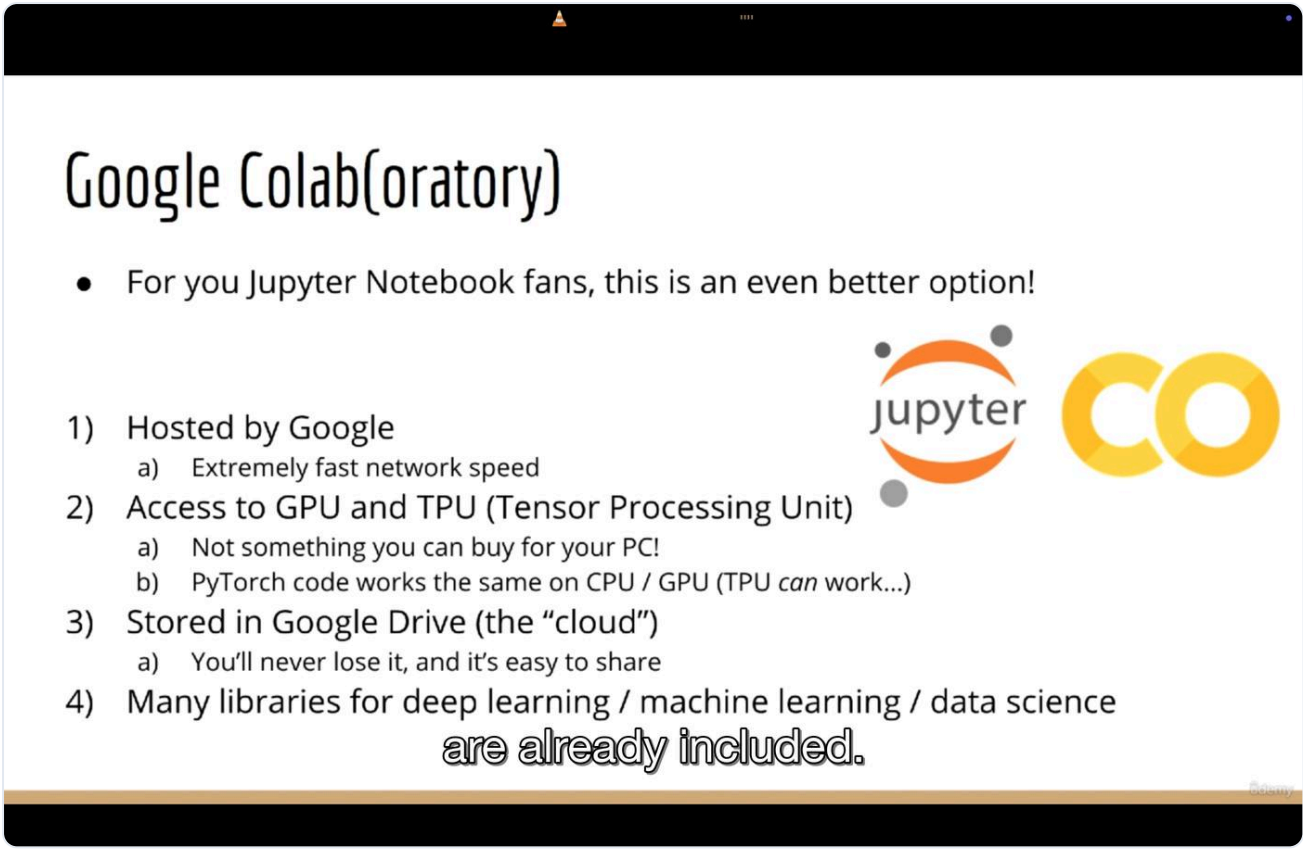


FIGURE 5.2

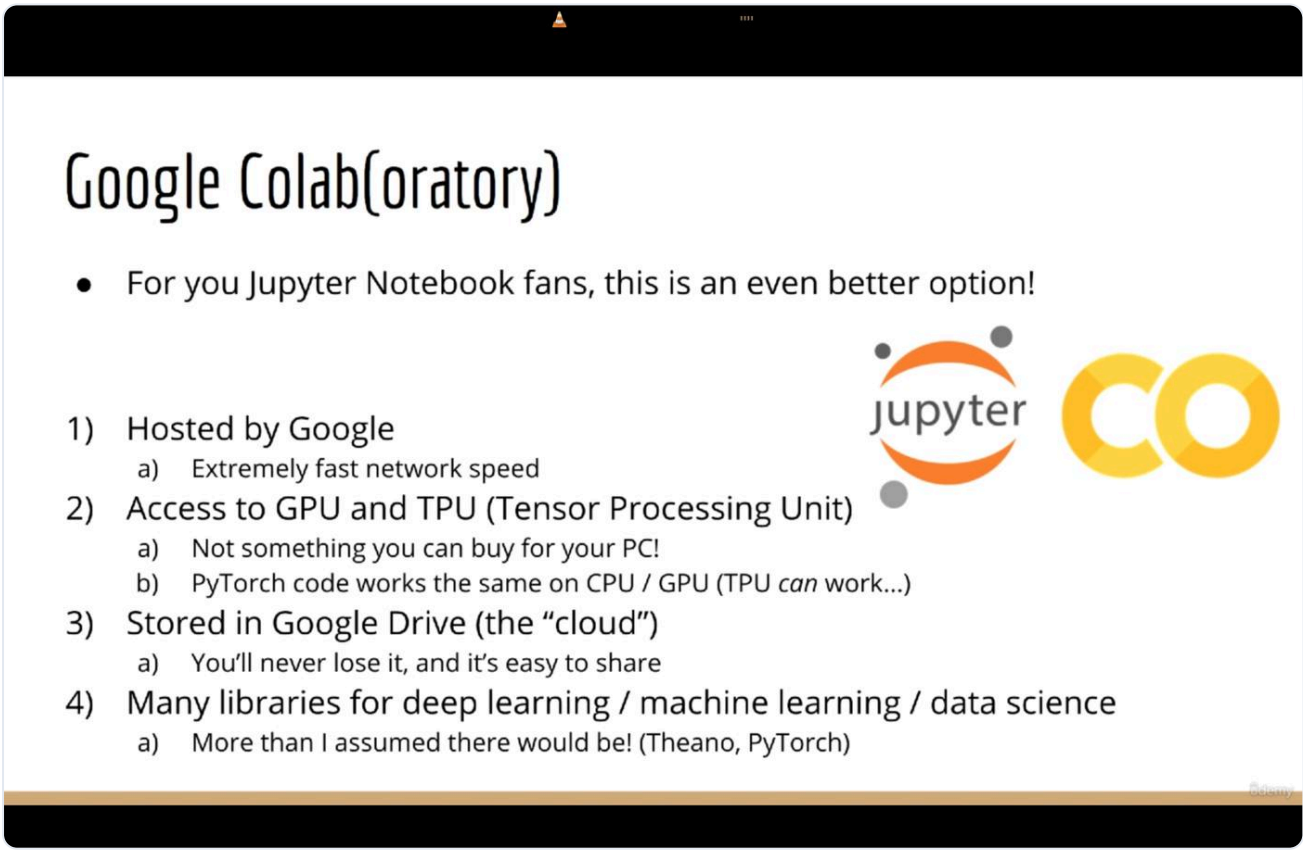


FIGURE 5.3

You'll never lose it and it's very easy to share with other people. Fourth is that many of the libraries you need for a deep learning machine learning and data science are already included. In fact I was surprised that there were many more than I assumed there would be. There are even competing Deep Learning libraries already included such as Theano and PyTorch. So for those of you who hate doing environments set up myself included this is really truly awesome. So in this lecture we're not going to do anything really technically complicated. Rather we're just going to talk about Google Colab and do some short demos.

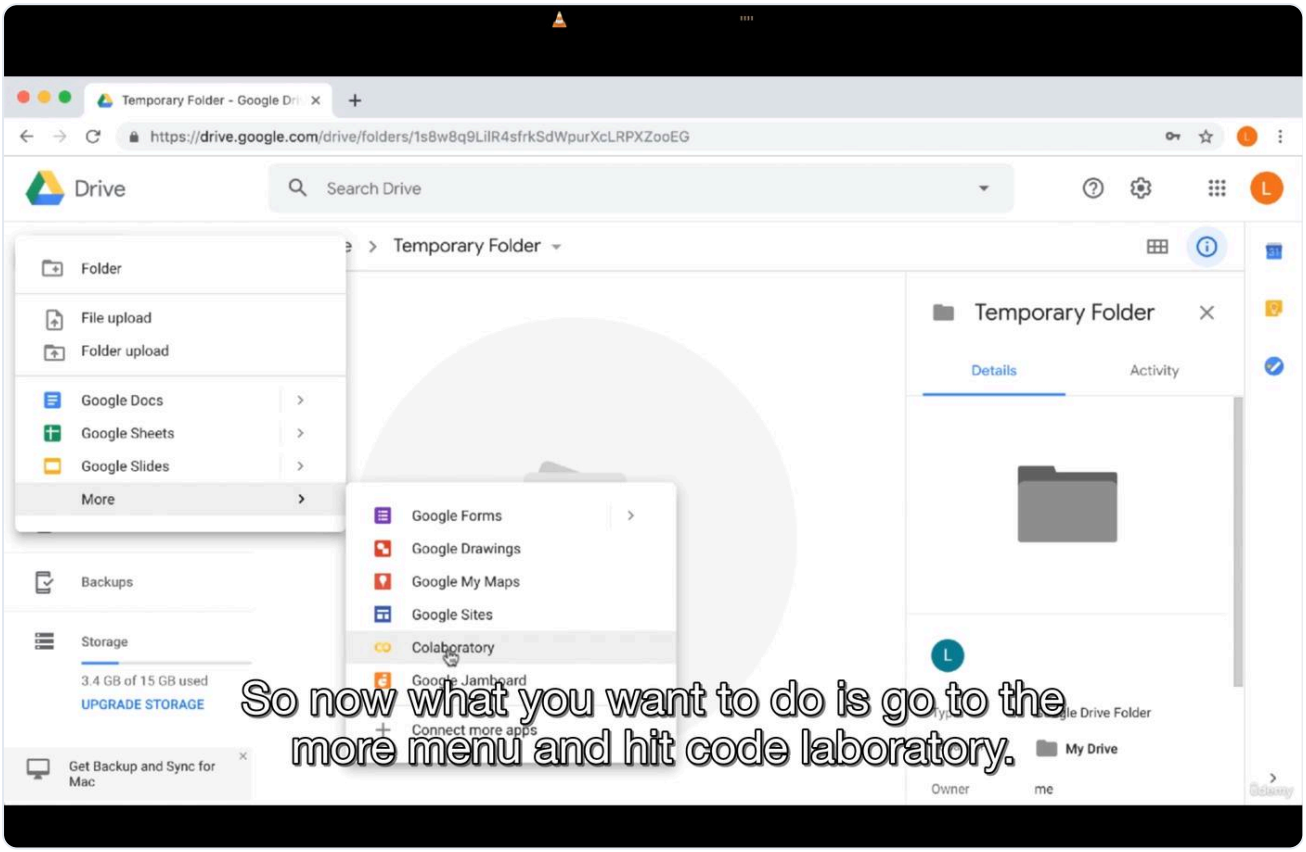


FIGURE 5.4

So you know how it works and can see for yourself that it's just like writing Python anywhere else to start. I'm going to assume you already know how to create a Google Drive account if you don't have one go to drive dot google dot com and sign up once you have your google drive account and you've logged in you'll see this interface from here. You can hit the new menu which allows you to create all different kinds of files such as Google Docs a spreadsheet a presentation and so forth. So let's do this. So now what you want to do is go to the more menu and hit Colaboratory. All right.

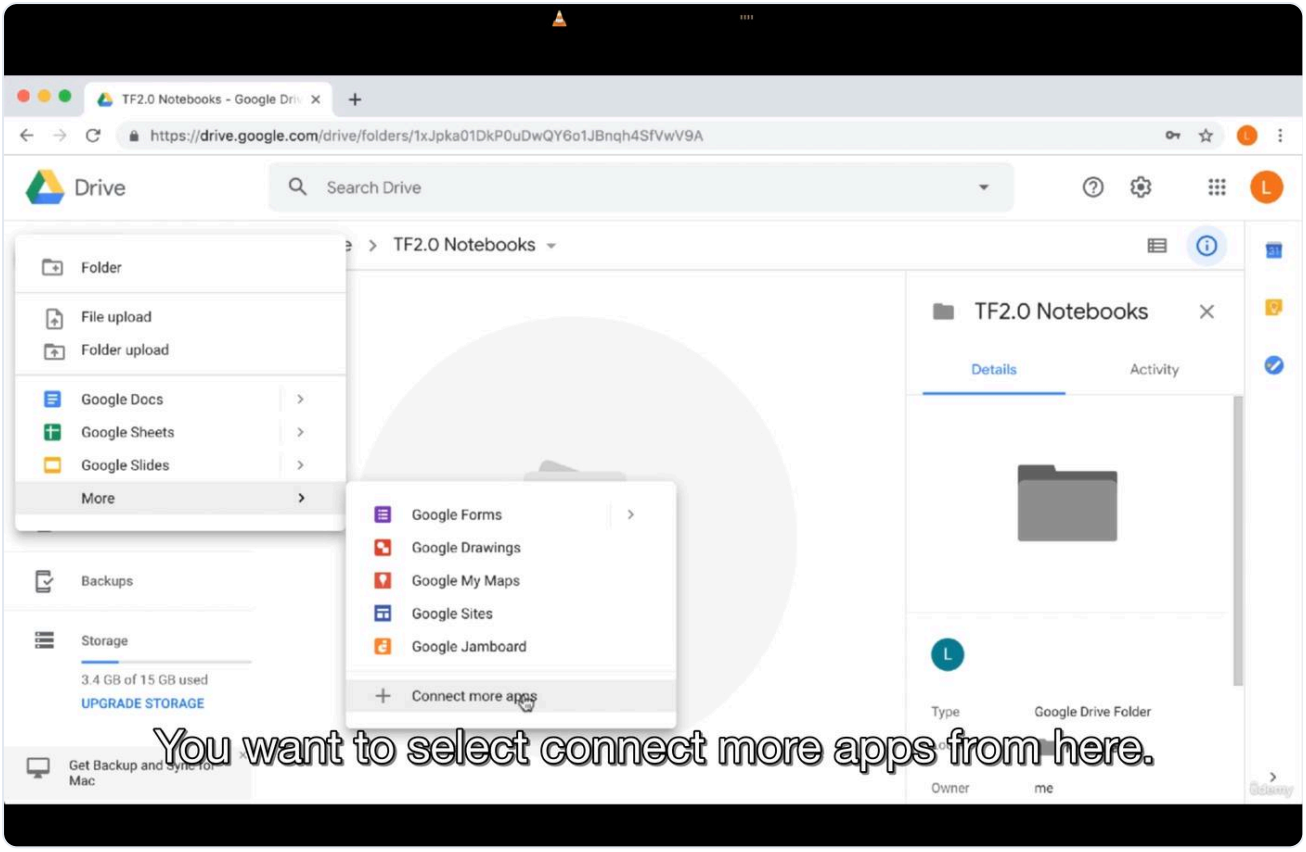


FIGURE 5.5

So as you can see this brings up a new notebook. And from here you can mostly use this as you would a normal notebook. Now one thing that might happen to you is that you might not see Colaboratory in the menu at all. So as you can see I've hit the new menu and I've hit more but I don't see Colab in this case. Here's what you can do. You want to select connect more apps from here. Just search for Colab.

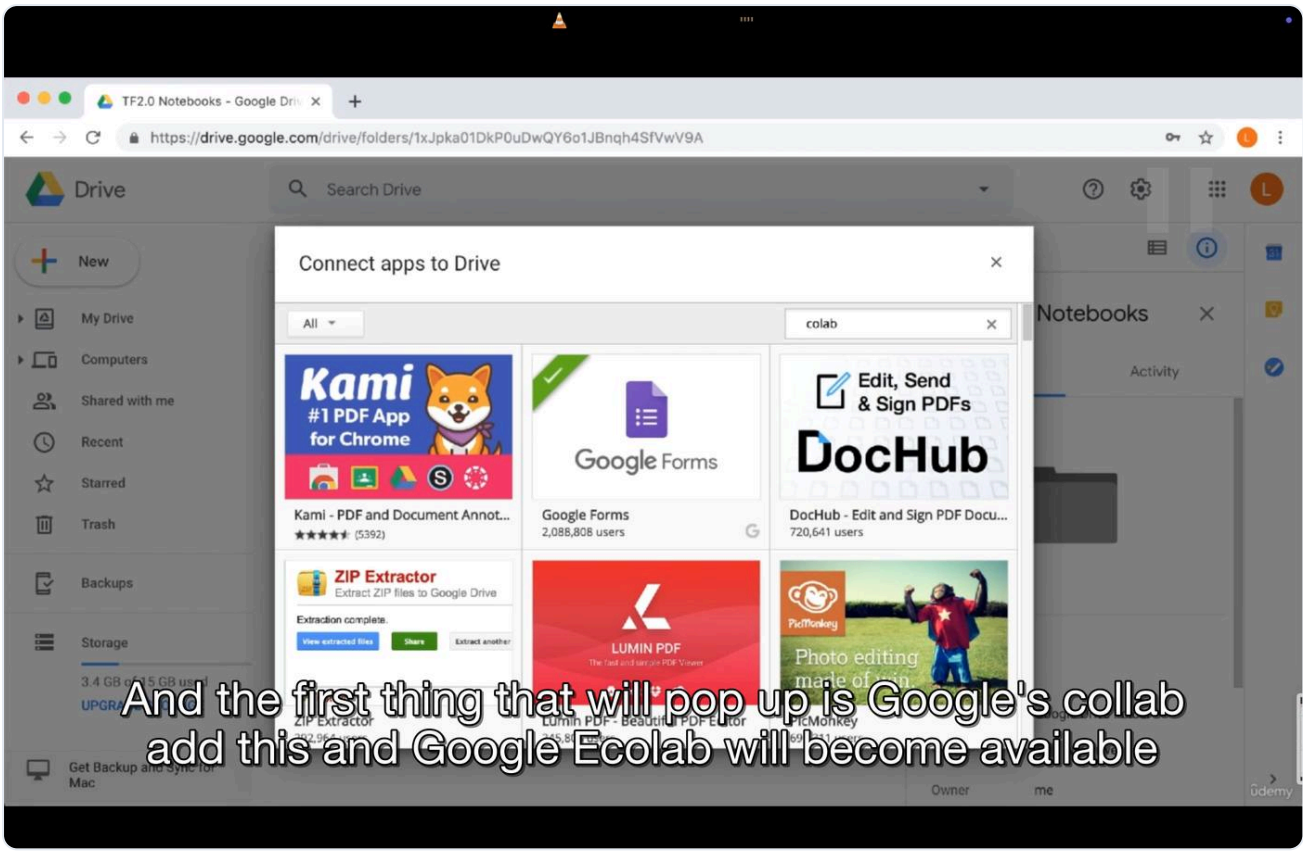


FIGURE 5.6 | Connect Apps to Drive

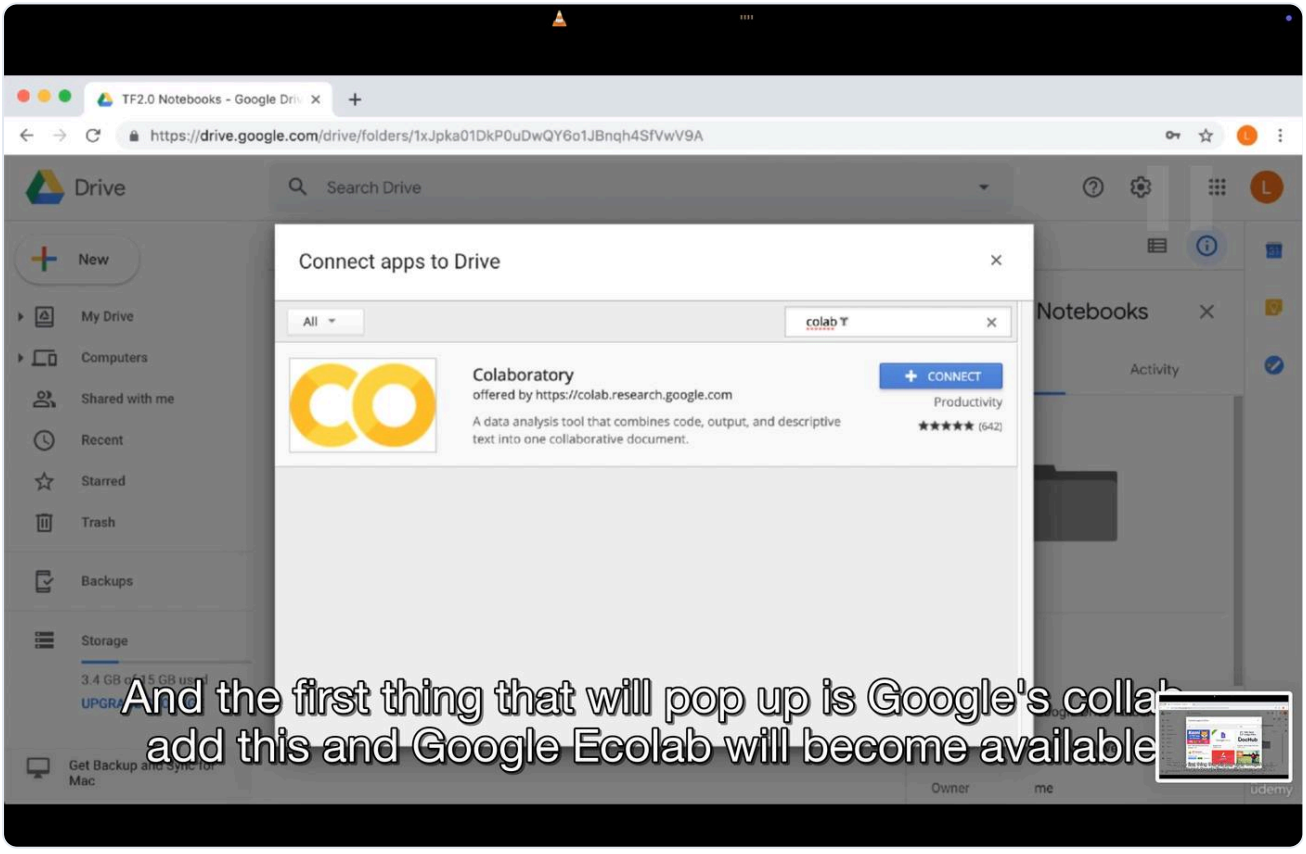


FIGURE 5.7 | Connect Apps to Drive

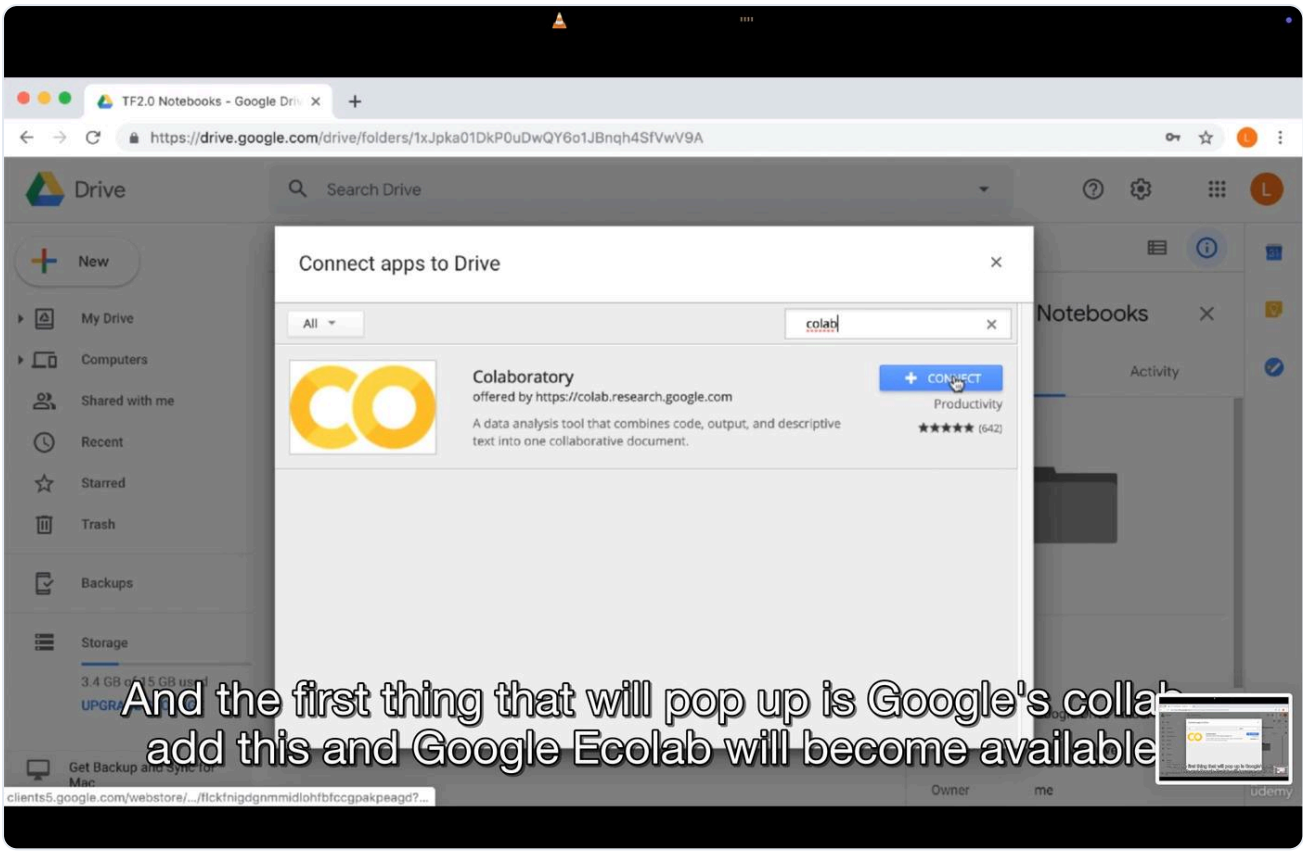


FIGURE 5.8 | Connect Apps to Drive

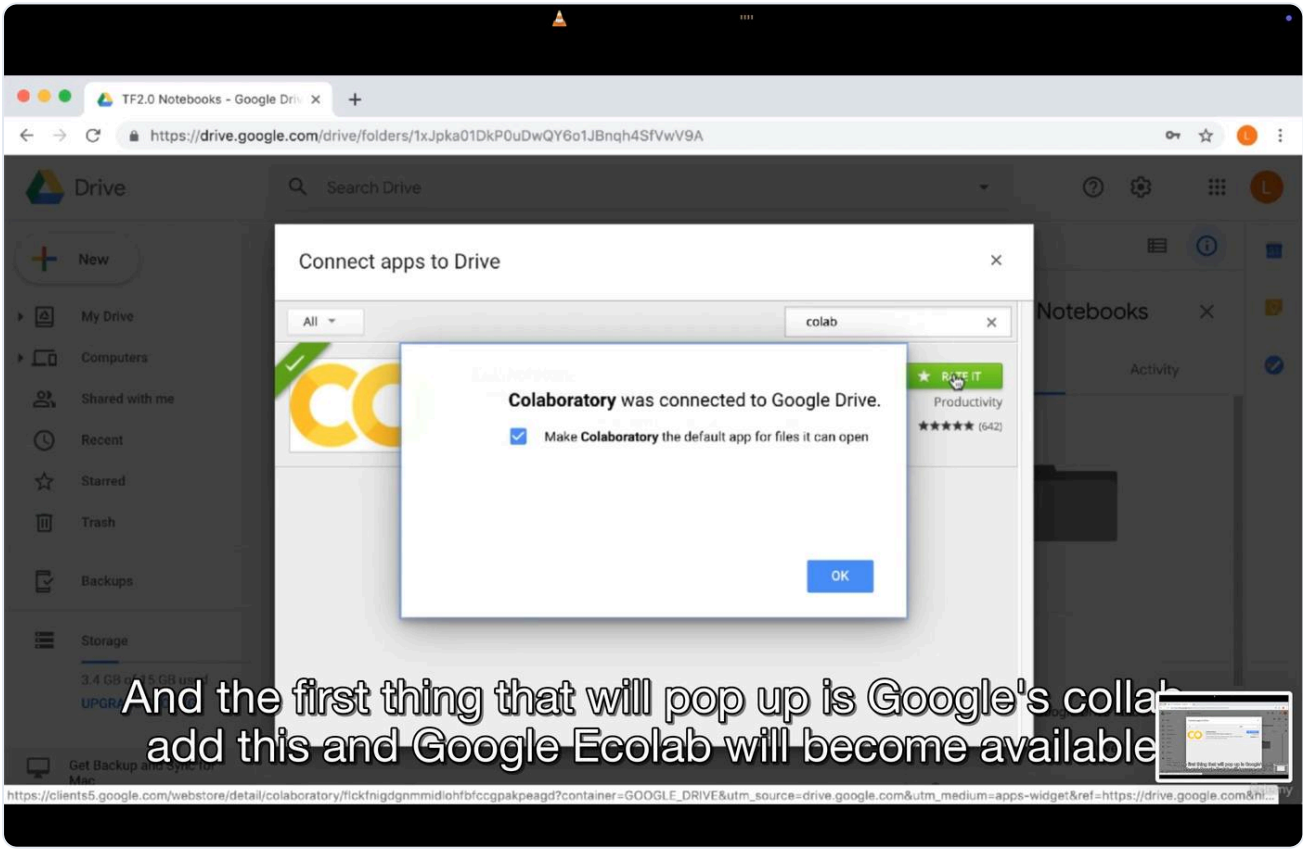


FIGURE 5.9 | Connect Apps to Drive

And the first thing that will pop up is Google's Colab add this and Google Colab will become available from the menu.

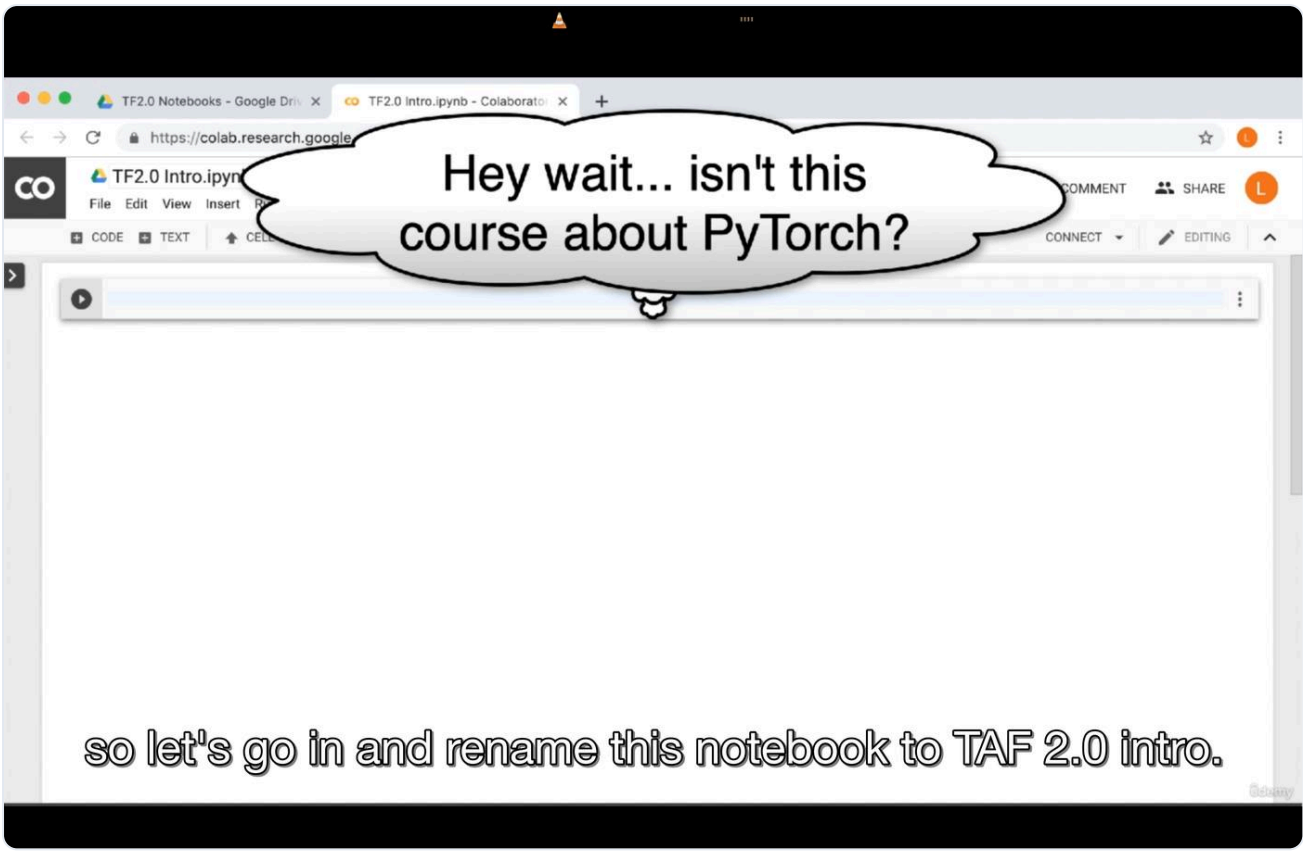


FIGURE 5.10

We just looked at so if we go there again we can see that Colab now appears where it should so let's go in and rename this notebook to TF 2.0 intro. So first we're gonna get right to the good stuff.

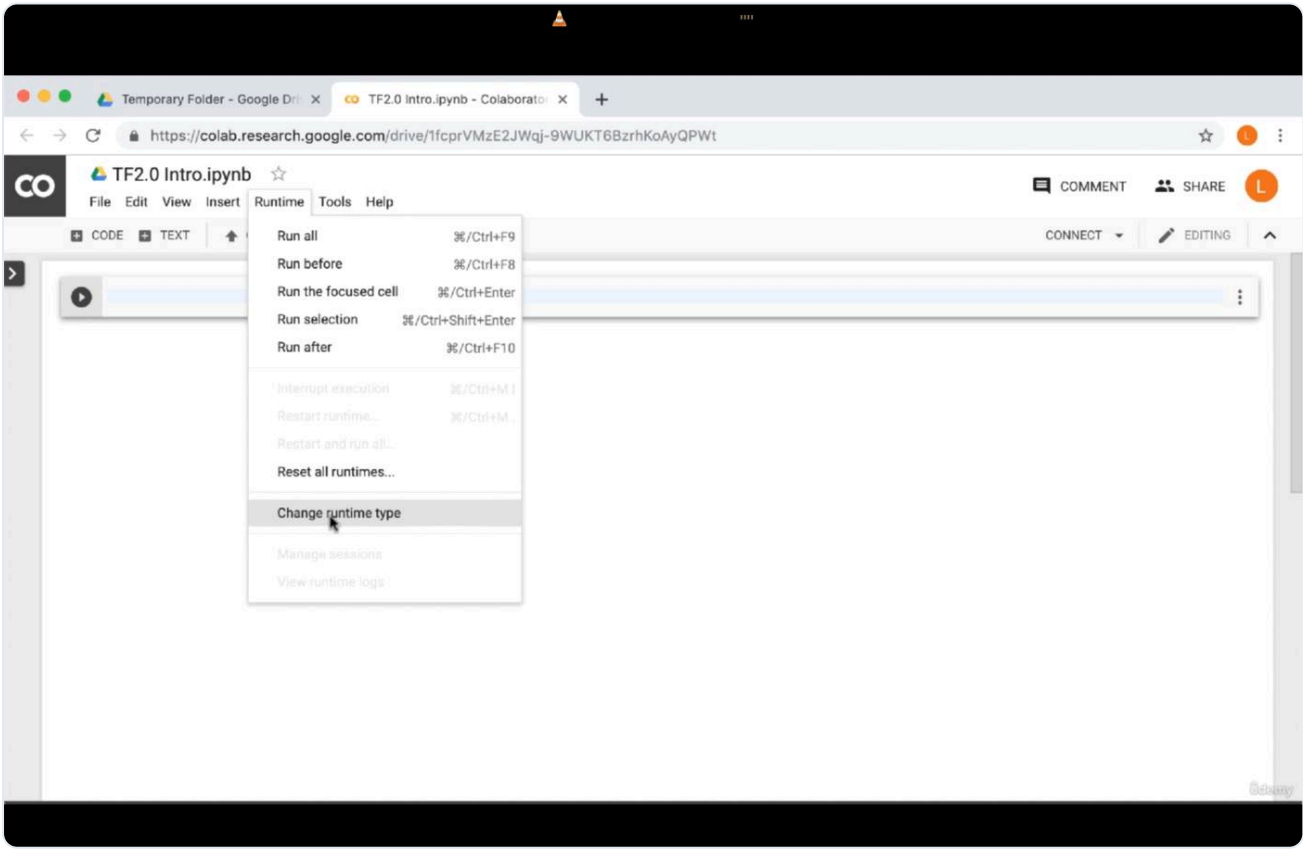


FIGURE 5.11

How can we make use of a GPU or TPU in order to do this. You want to go to the runtime menu and select change runtime type as you can see there are two select boxes here. The first one lets you select which python version you want to use.

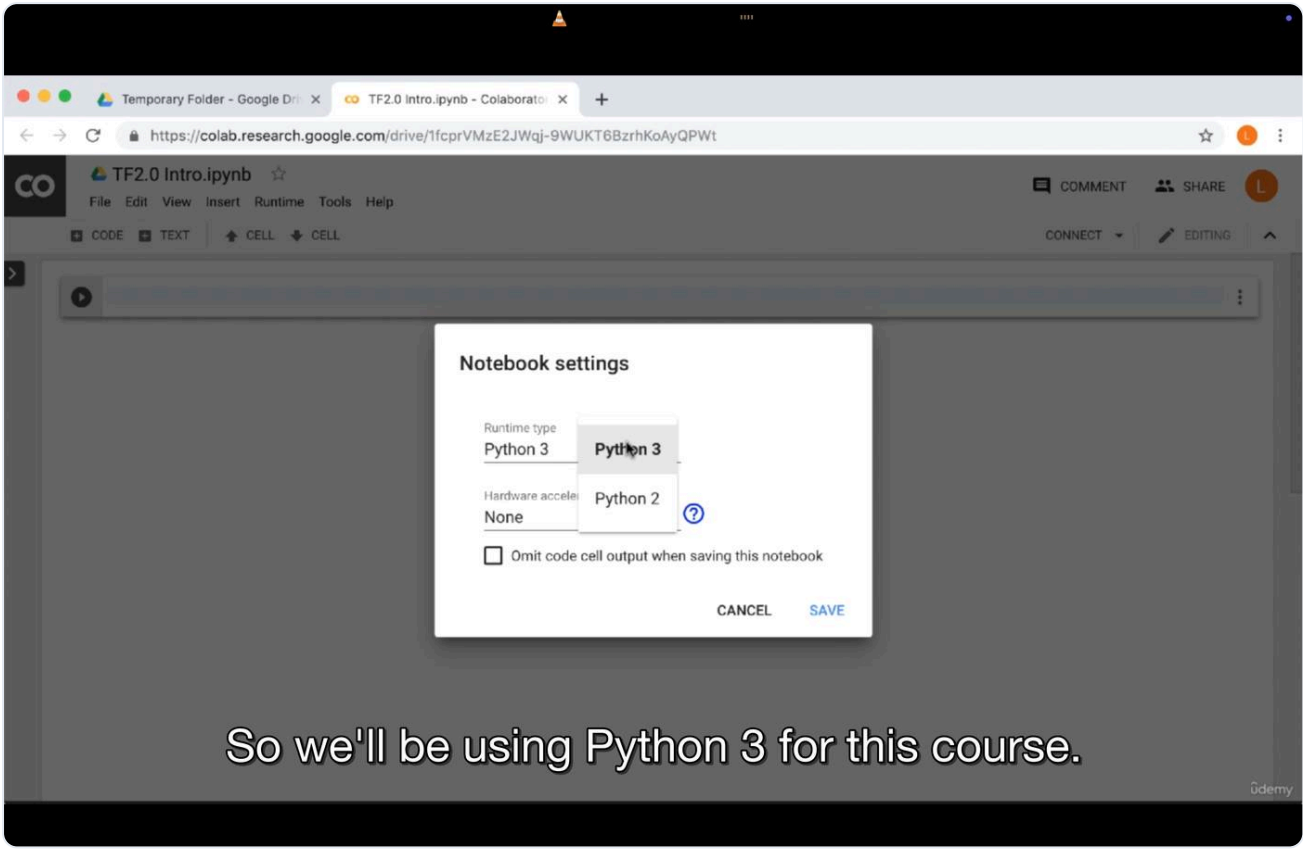


FIGURE 5.12

So we'll be using Python 3 for this course. The second lets you select what kind of device you want to use.

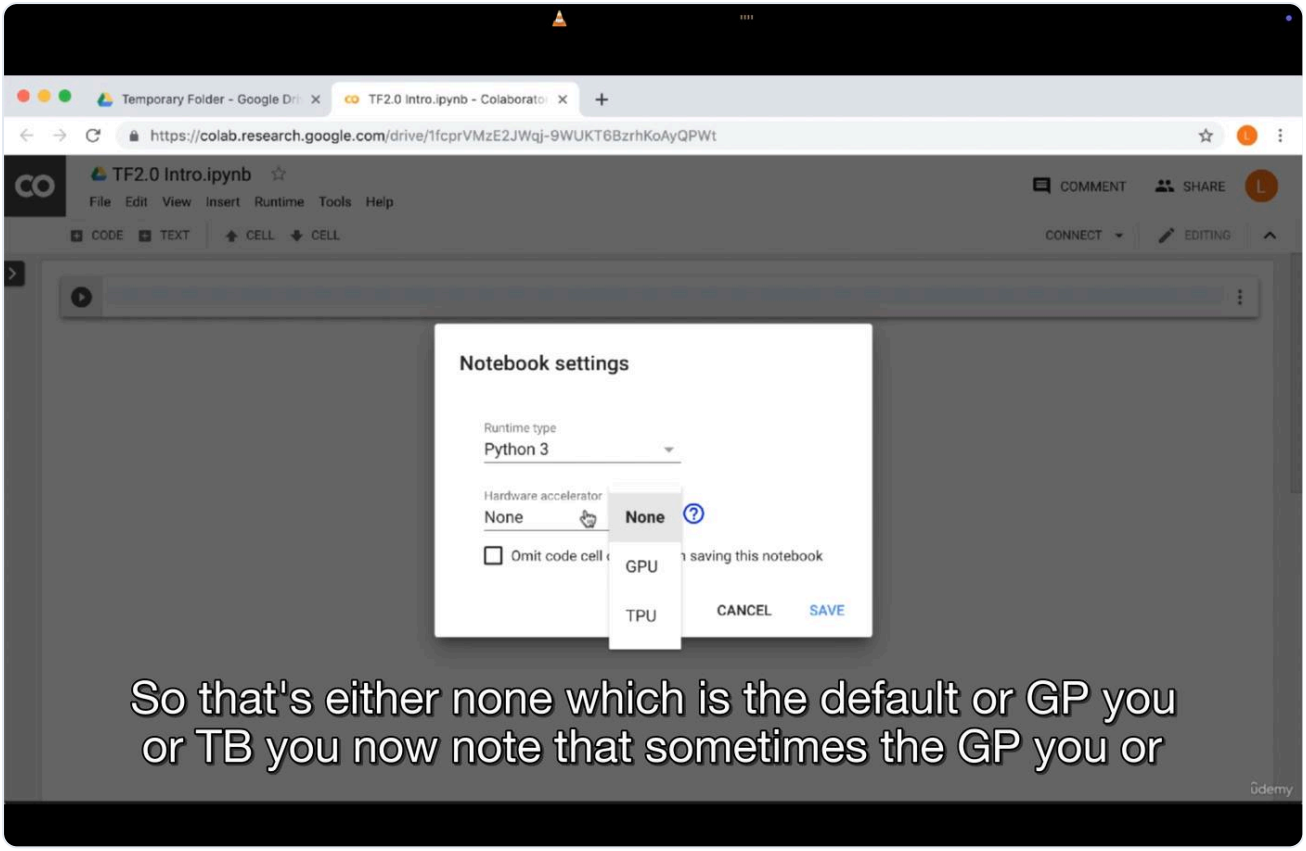


FIGURE 5.13

So that's either none which is the default or GPU or TPU now note that sometimes the GPU or TPU might not be available. This is because these are shared resources so your fellow peers taking this course and other machine learning students and researchers all around the world might be using Google Colab and we are all sharing these resources. So if our usage of these resources hits the limit of what's available then you might not have a GPU or TPU available when you need them. For this reason some of the code you'll see in this course may be done on my local machine as well. But remember Python code works the same anywhere so it does not make a difference next.

You can see that there are two main types of cells that we can create in the notebook code and text.

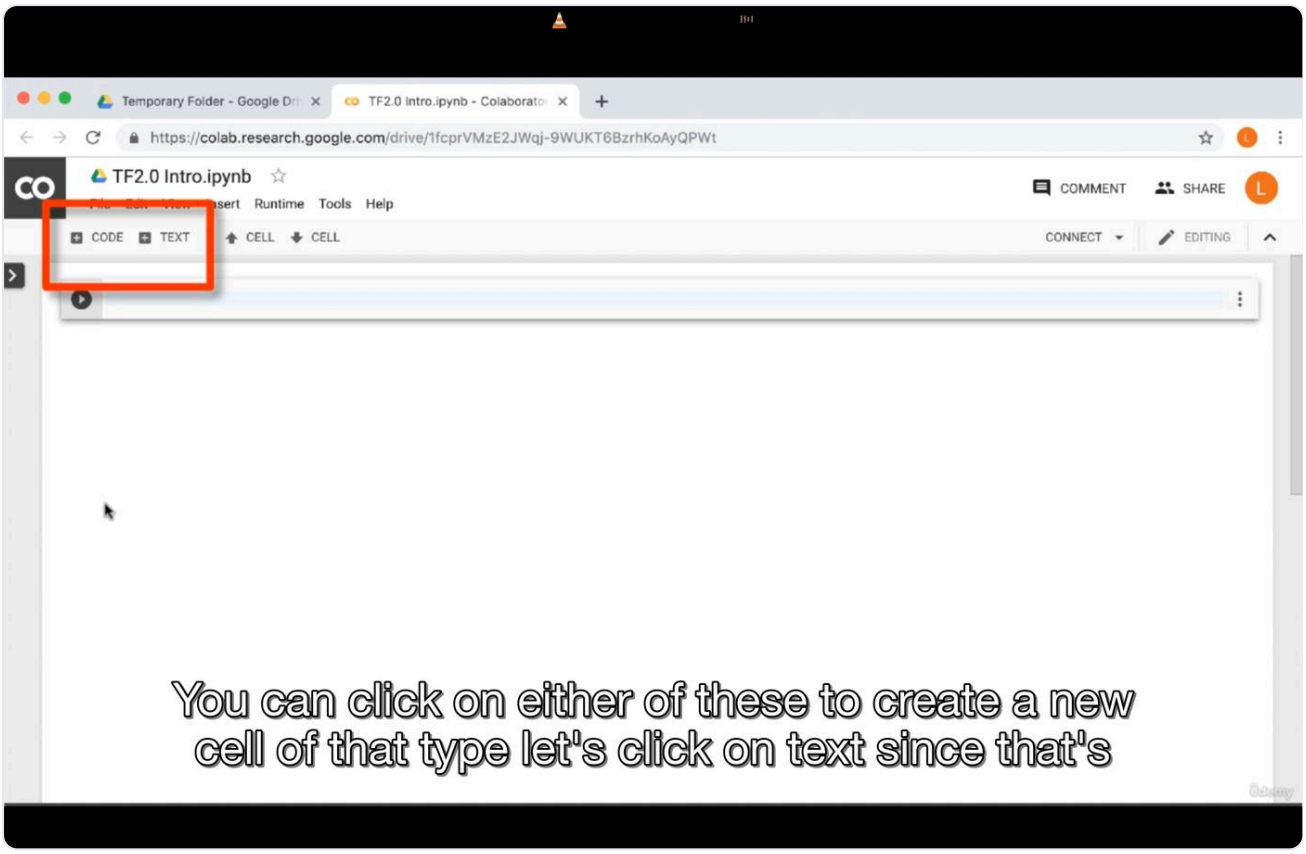


FIGURE 5.14

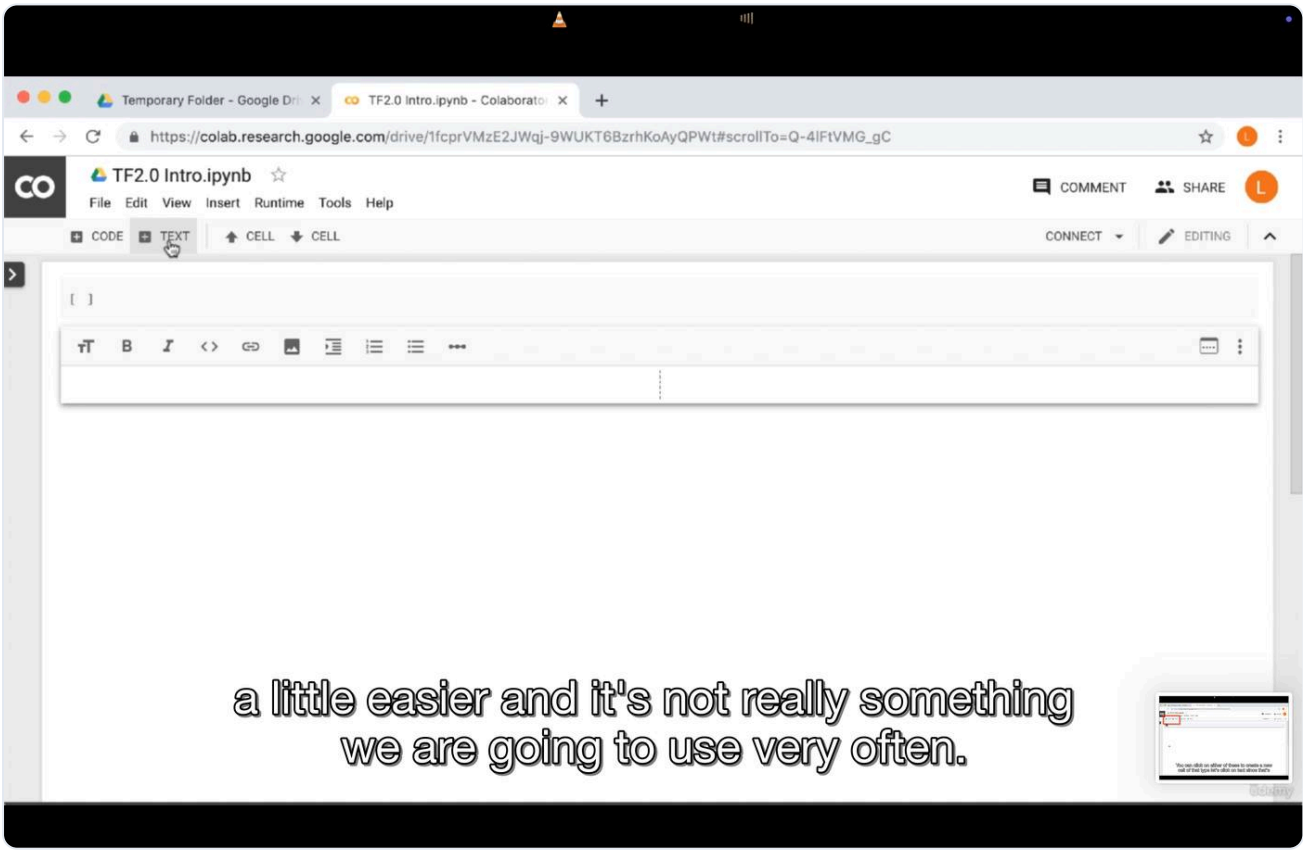


FIGURE 5.15

You can click on either of these to create a new cell of that type let's click on text since that's a little easier and it's not really something we are going to use very often. So let's just get it out of the way so I'm actually going to delete the very first cell all right. So as you can see when I click this it creates a new cell with what looks like a rich text editor. You'll notice that it's split into halves the left half is where you enter your text and the right half is a preview of what it will look like. So let's enter some text.

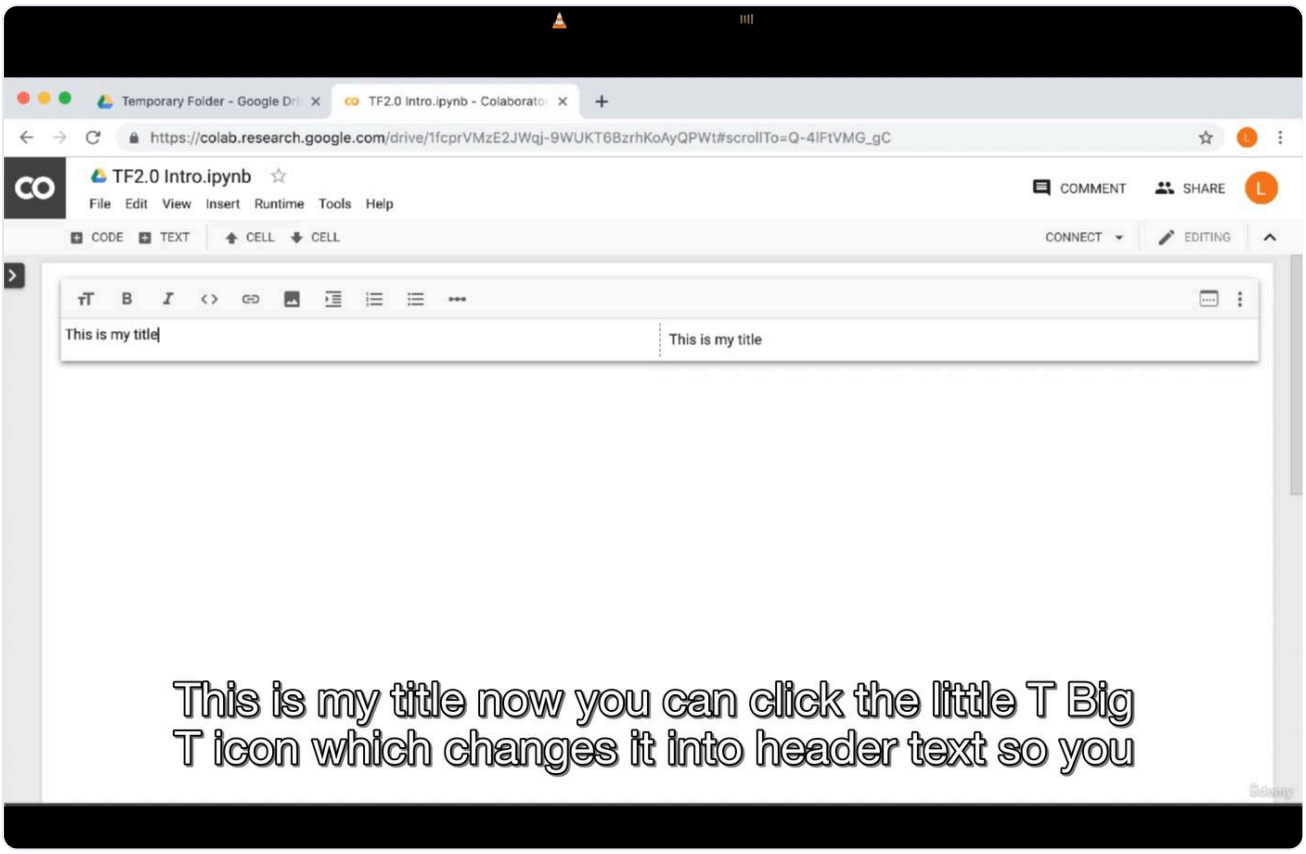


FIGURE 5.16

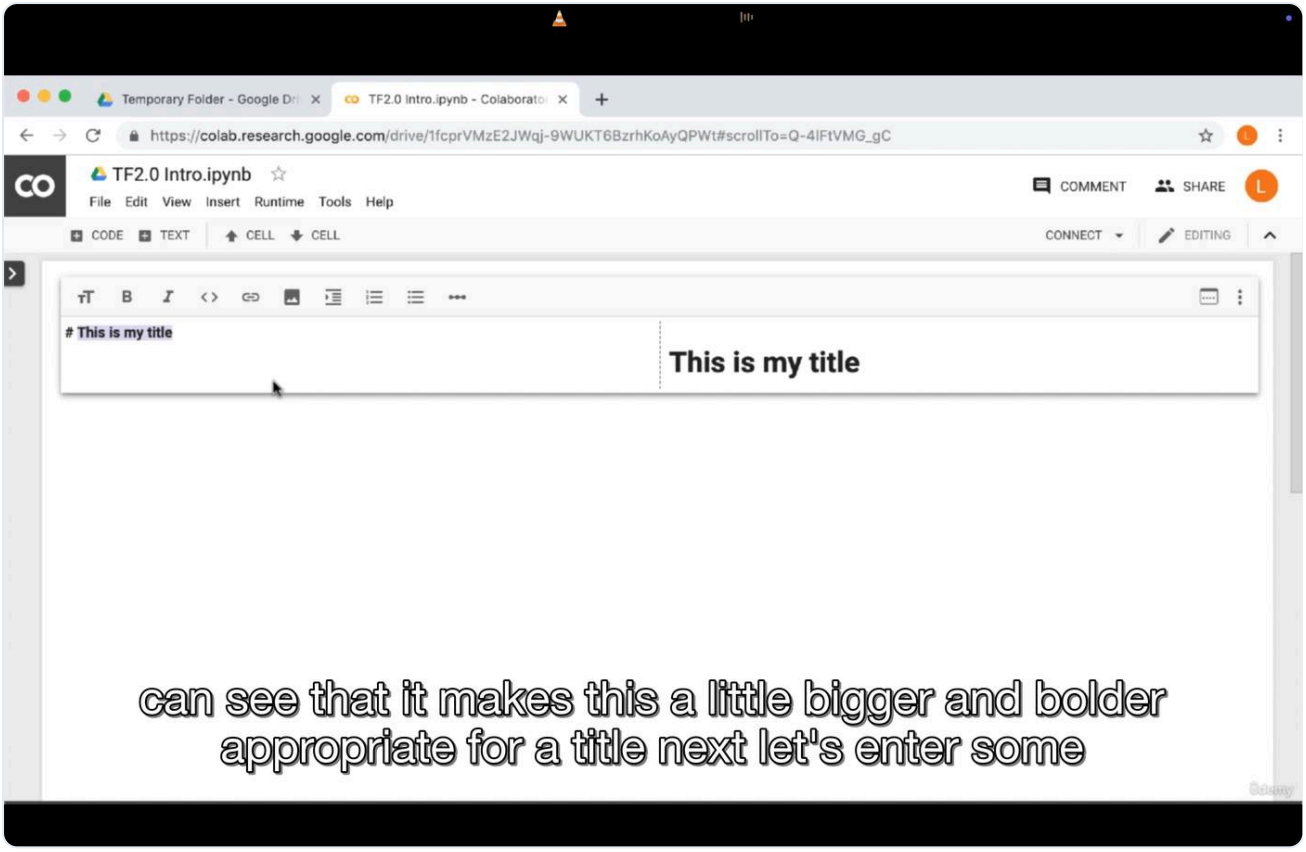


FIGURE 5.17

This is my title now you can click the little T Big T icon which changes it into header text so you can see that it makes this a little bigger and bolder appropriate for a title next let's enter some regular text.

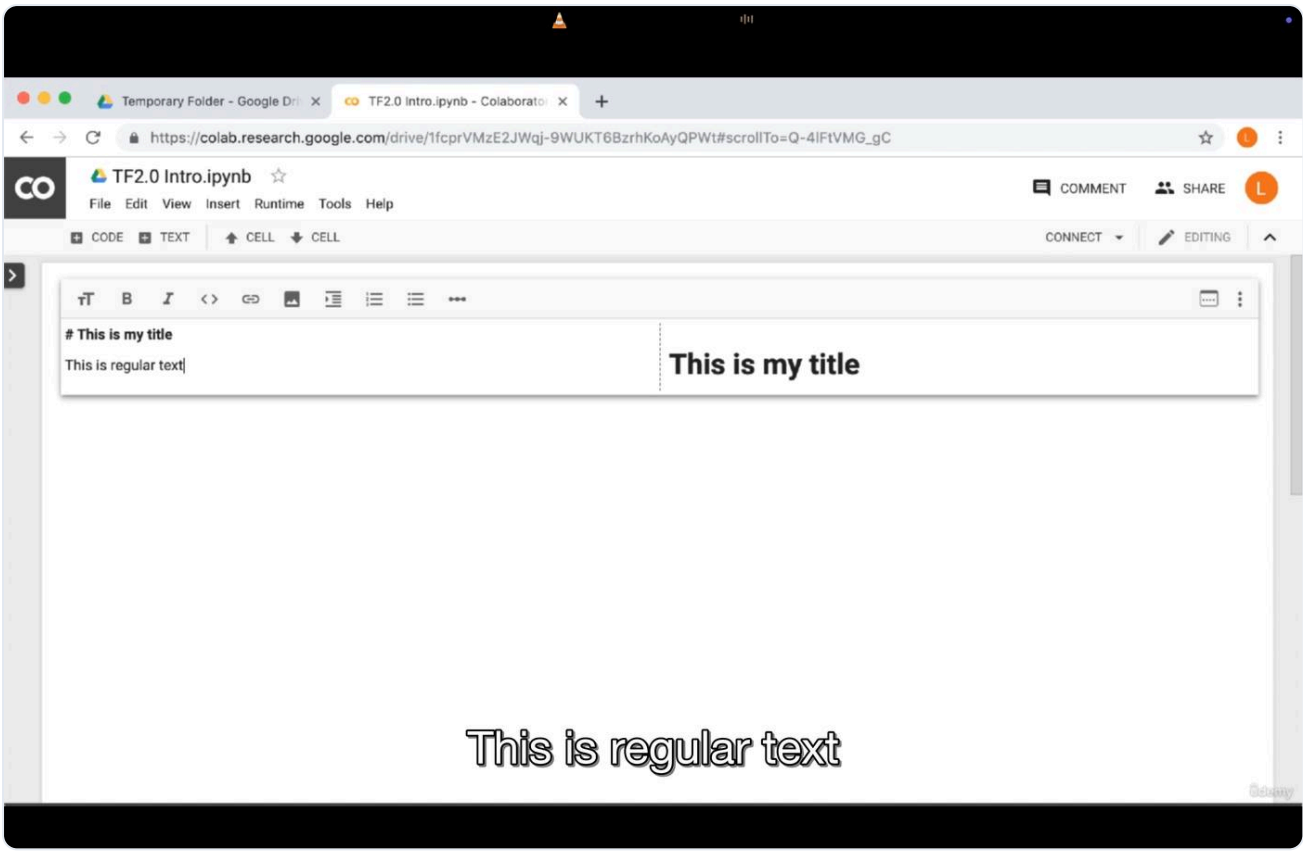


FIGURE 5.18

This is regular text and note that there are also these arrow brackets so it looks like it's going to let us enter code snippets.

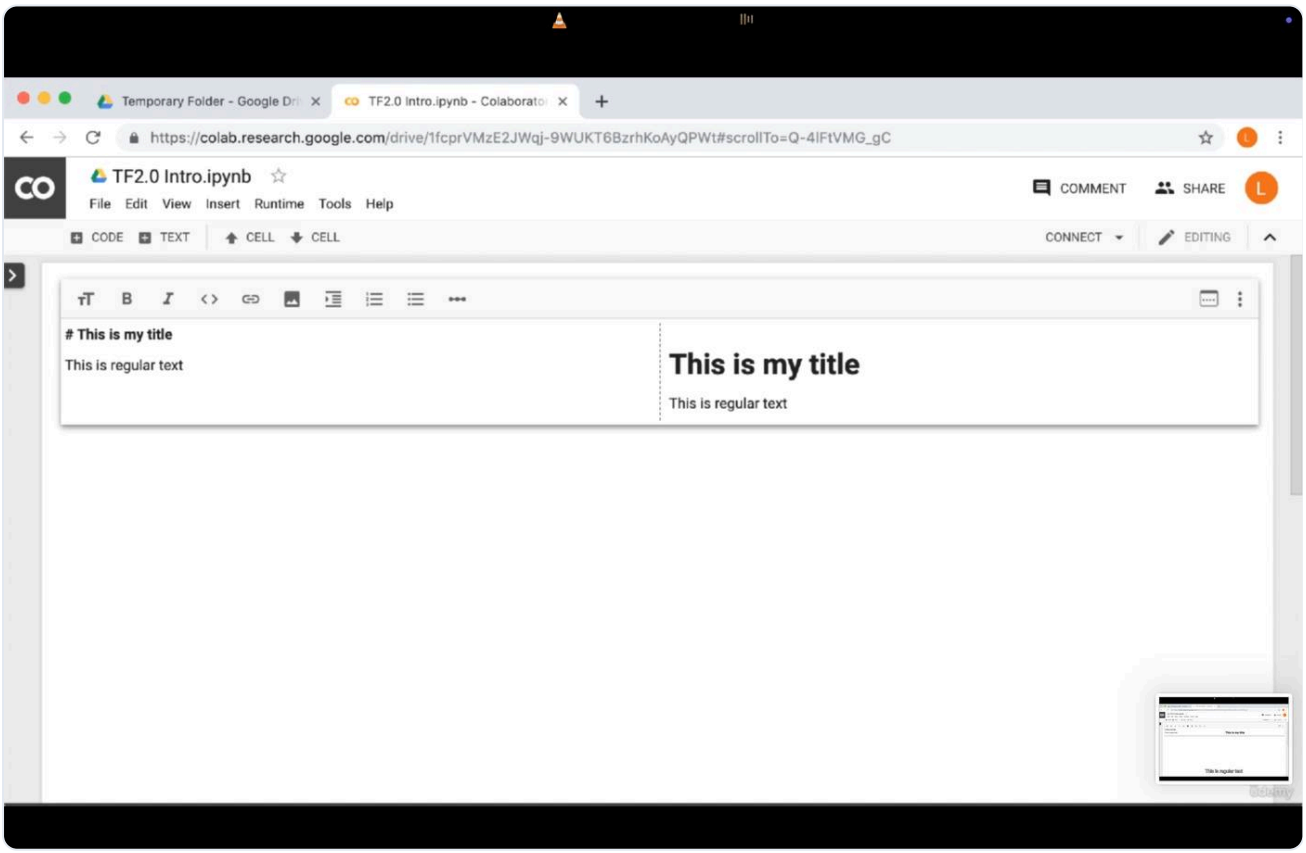


FIGURE 5.19

So let's try that.

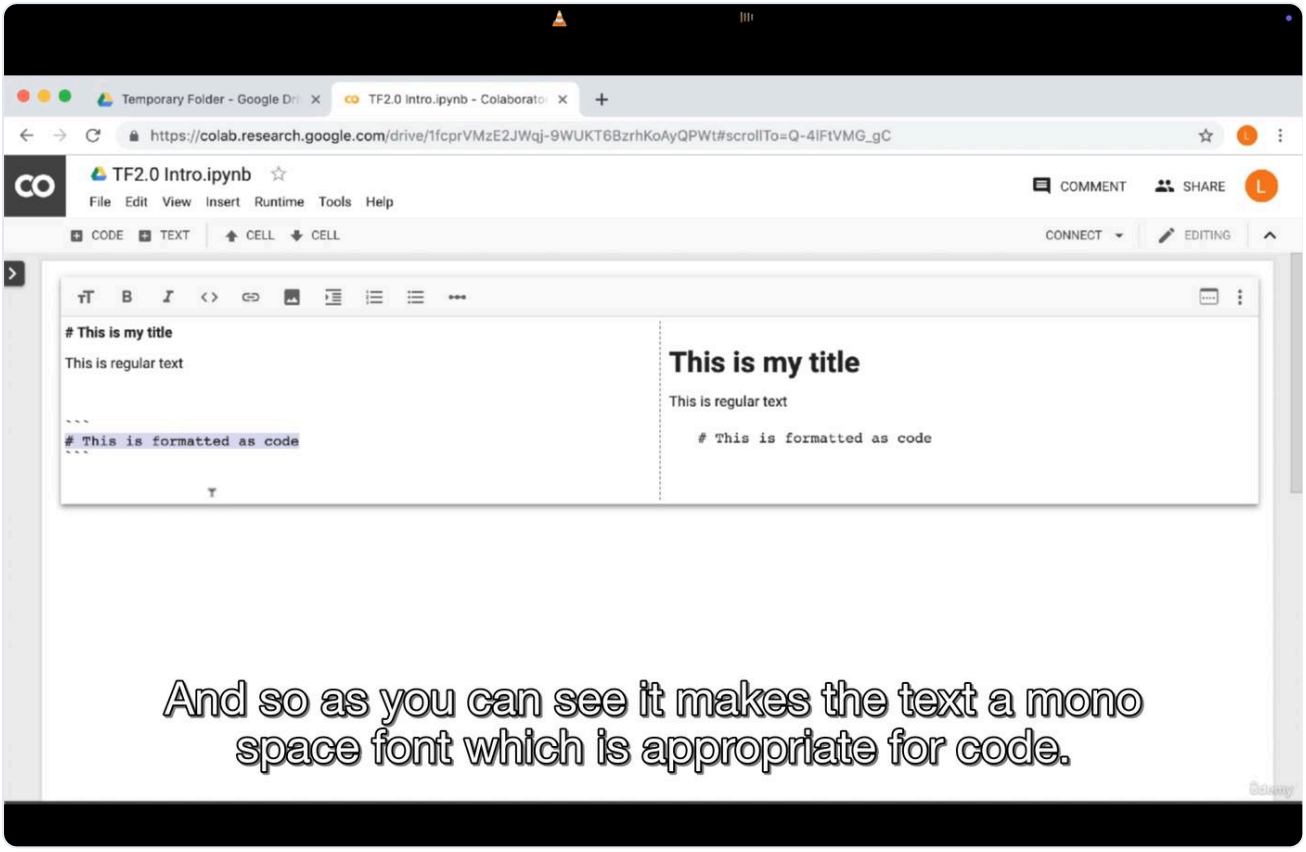


FIGURE 5.20

And so as you can see it makes the text a mono space font which is appropriate for code. Now there are some other options here. So you can make a link you can add images you can indent.

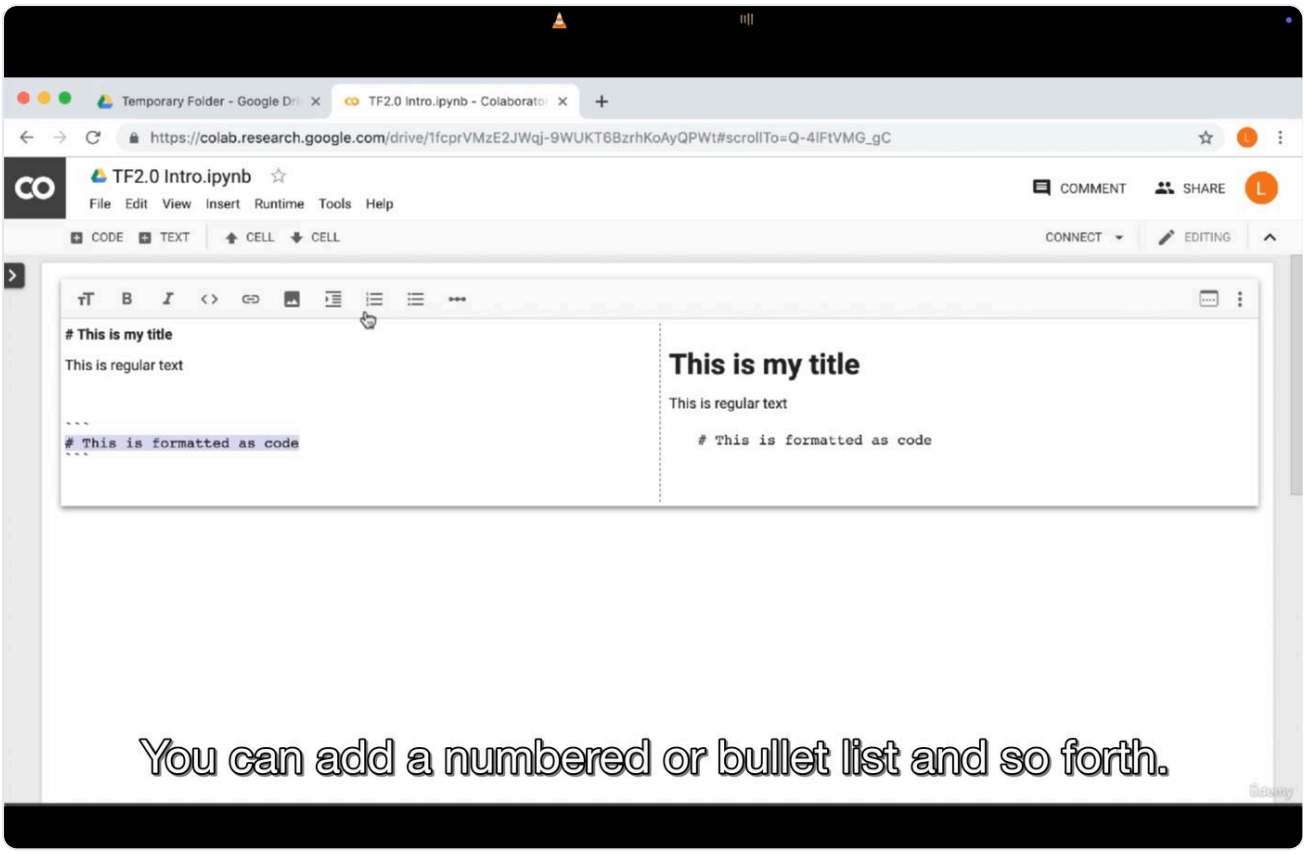


FIGURE 5.21

You can add a numbered or bullet list and so forth. So if you're interested play around with this.

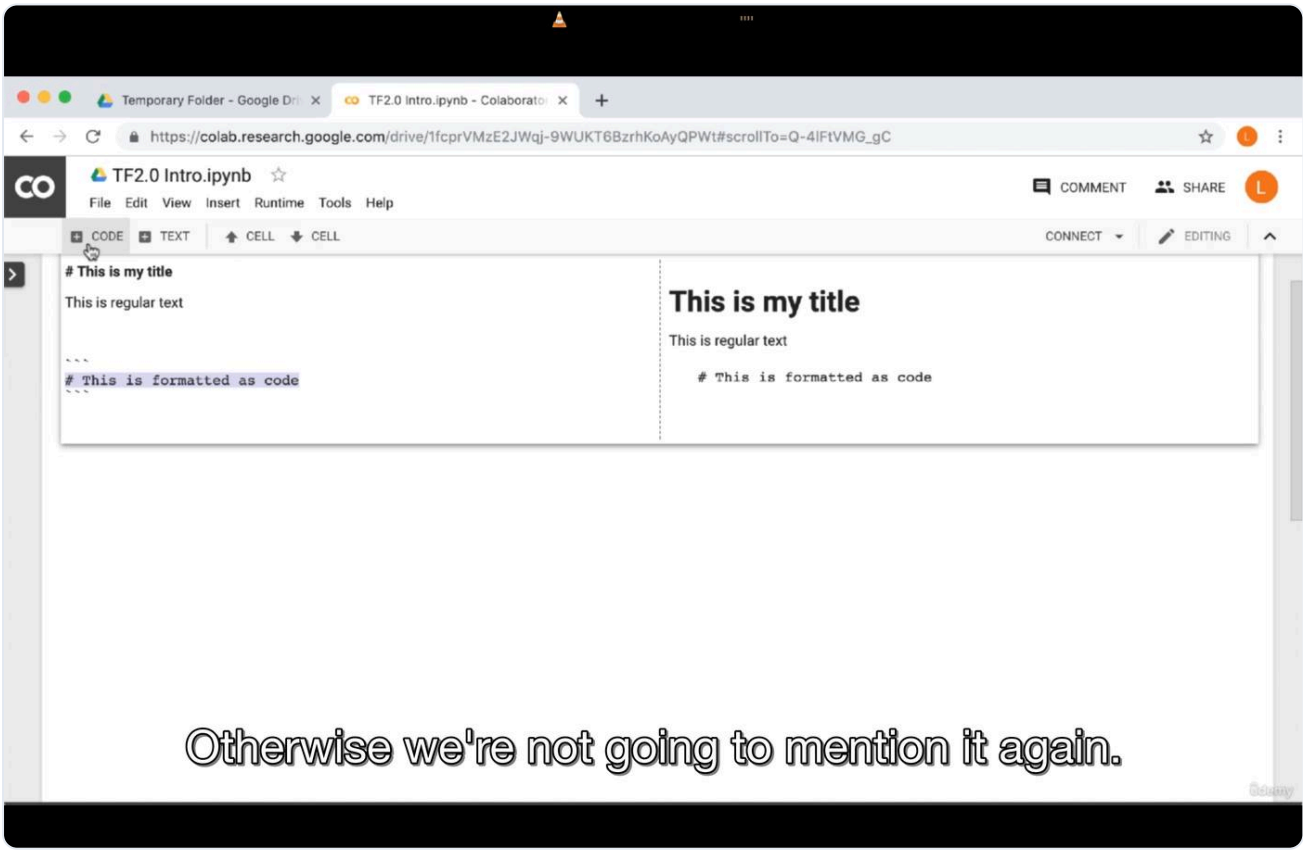


FIGURE 5.22

Otherwise we're not going to mention it again. Next we have the code cell.

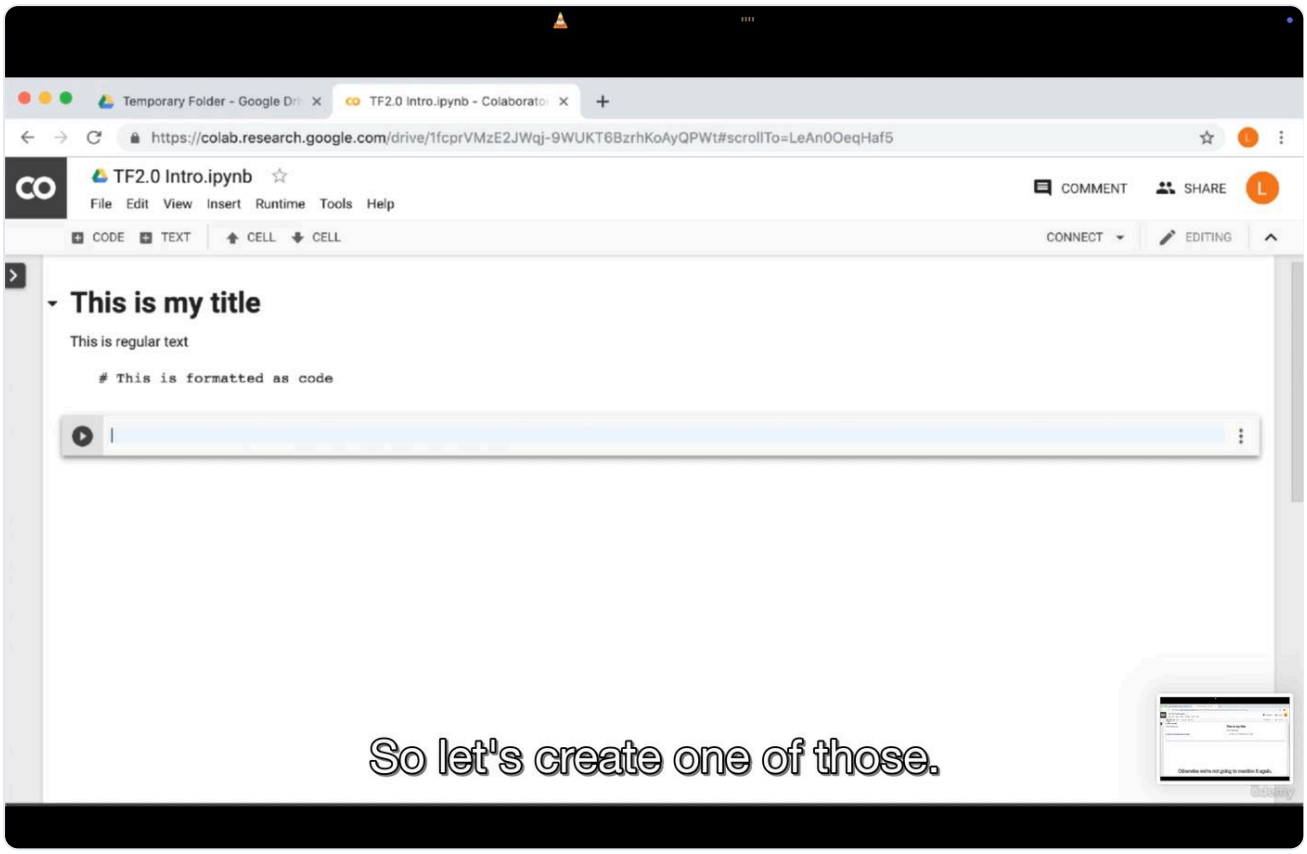


FIGURE 5.23

So let's create one of those. All right. And as mentioned we're not going to write any fancy code in this lecture. We just want to do something simple to make sure everything works as expected. So let's start by importing NumPy and matplotlib all right beautiful. As I mentioned earlier these already come pre installed next let's create a new code cell and make a sine wave. So first we need to create some x values so let's make X go from zero to 10 pi with 1000 points in between next let's make Y. The sine of X next let's create a new cell and plot what we just created so that's just plt dot plot x y now. Since this is a notebook there's no need to call plt dot show since the plot will just appear in the notebook itself.

All right.

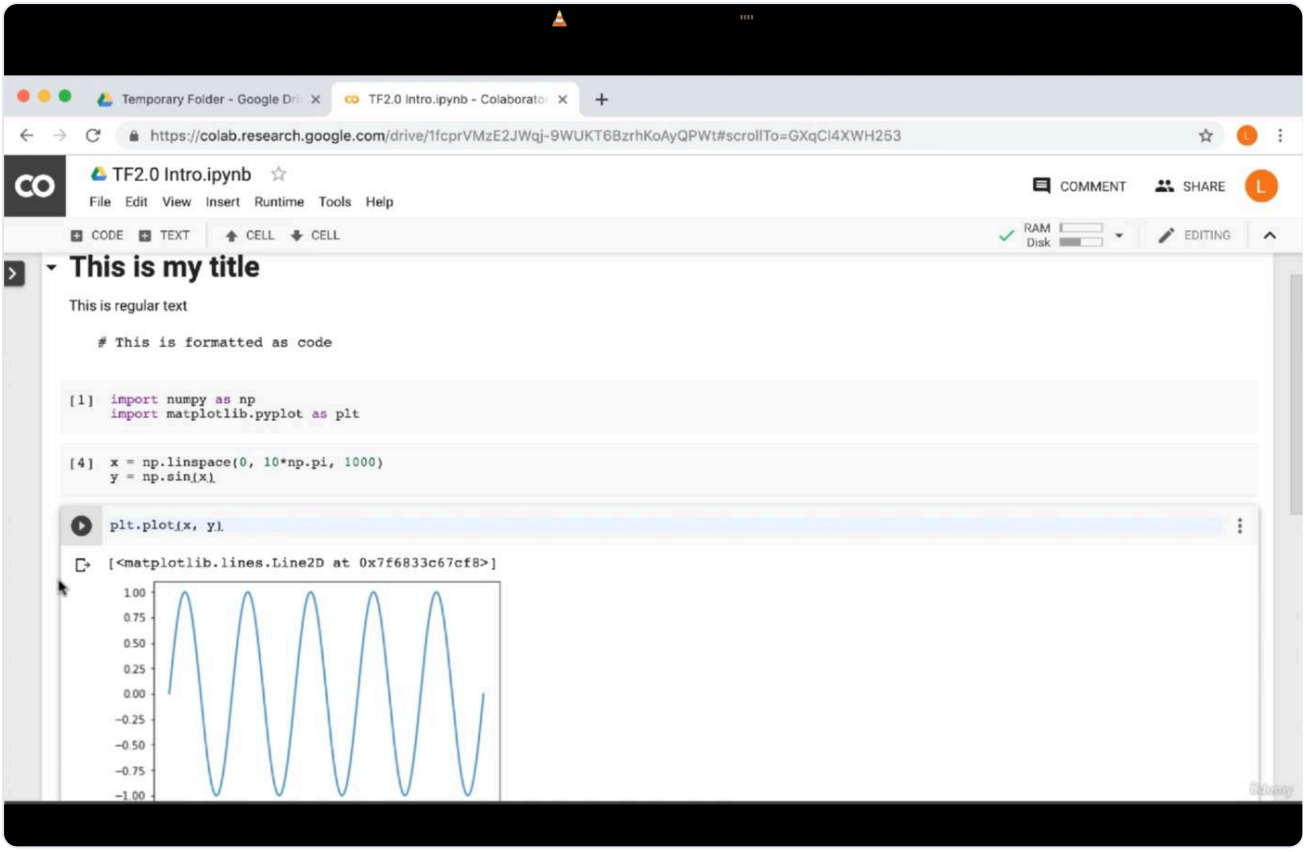


FIGURE 5.24

Very cool. Works just like a regular notebook at this point. We've convinced ourselves that Google Colab can do the usual things you'd expect from a Jupyter notebook. Now as I mentioned earlier one thing that's very nice about Colab is that it already comes with a bunch of useful libraries pre installed. In my opinion this makes Google Colab way better than Jupyter notebook. And if anyone ever asked me to write in a notebook environment I would choose Colab by default. I'm not a big fan of notebook but I am a big fan of Colab so here we can see that I've written some code to try and import a bunch of libraries specifically these libraries are libraries that have been using my courses some more than others.

Some are pretty rarely used so you might not expect that they would be included. Libraries like word cloud which we've only used once so far and yet if we look we see that everything I've tried to import here does not throw an error so this tells us that these libraries are indeed available.

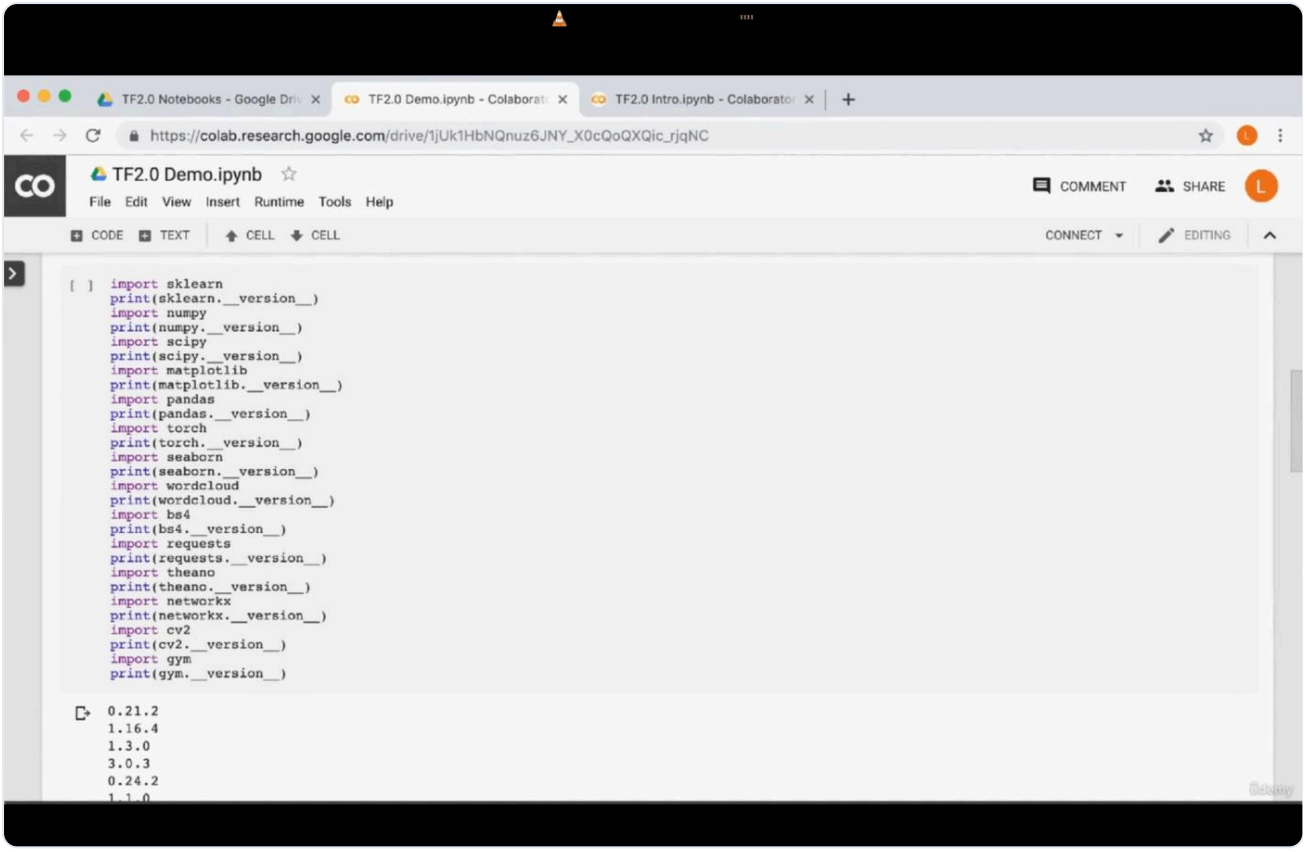


FIGURE 5.25

What's interesting to me is that some of these libraries are not machine learning related at all. Of course we've used them in my courses because they are generally useful as Python libraries but it's nice to see the folks at Google also make use of these same libraries. And so thought to include them. So here you can see the usual stuff such as scikit-learn NumPy SciPy matplotlib and pandas. We also have torch and Theano which is surprising because they are competing Deep Learning libraries and development for Theano has been stopped for a while now. We also have seaborn word cloud beautiful soup which is for XML and HTML parsing requests which is for making HTTP calls NetworkX which is for graph functionality cv2 which is for OpenCV and gym which is OpenAI Gym.

All in all very impressive and much more than I expected. So there's some final caveats to Colab that I want to mention first. The main thing you have to remember is that this is the cloud. So these are shared resources.

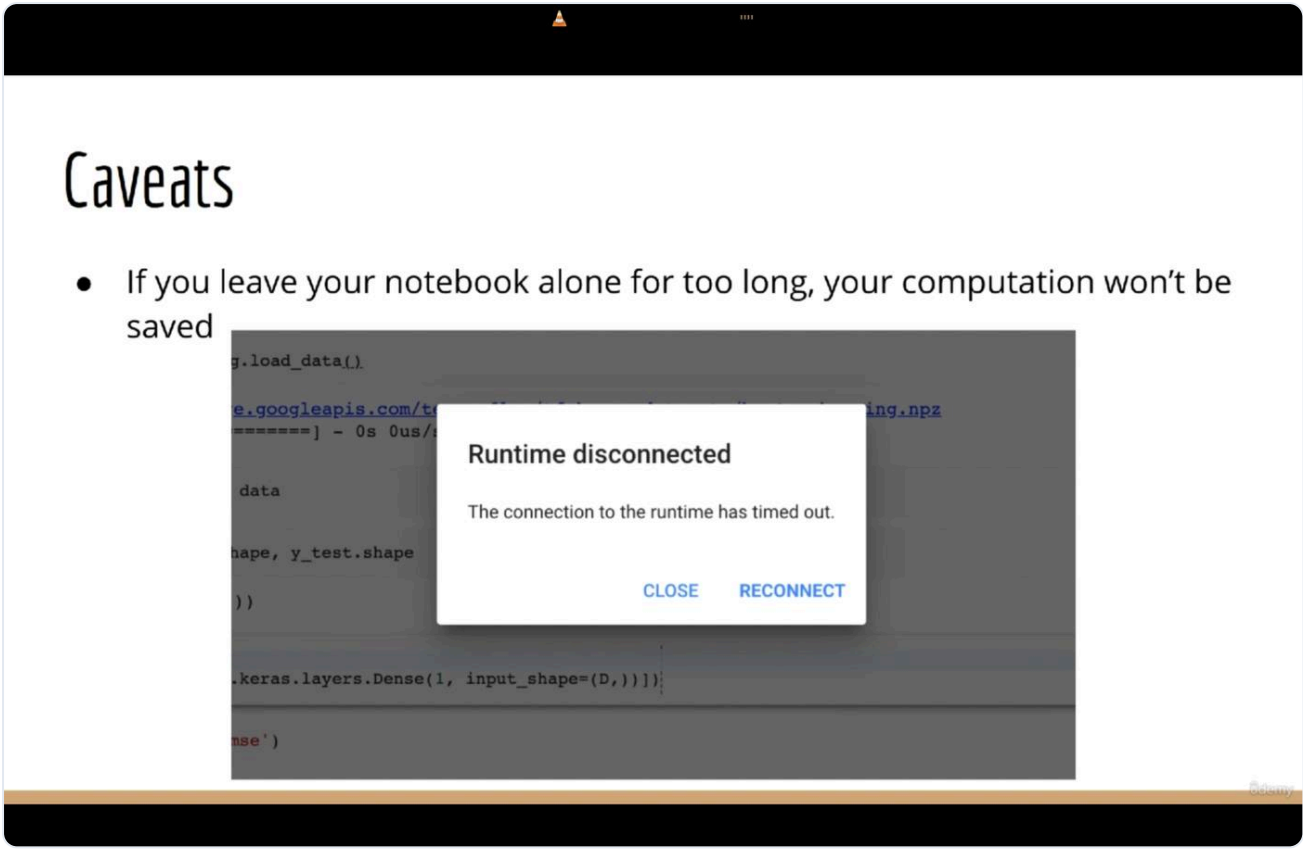


FIGURE 5.26 | Runtime Disconnected

So one way this affects you is if you leave your notebook alone for a long time it will become inactive and disconnect any computation that you may have run earlier won't be saved.

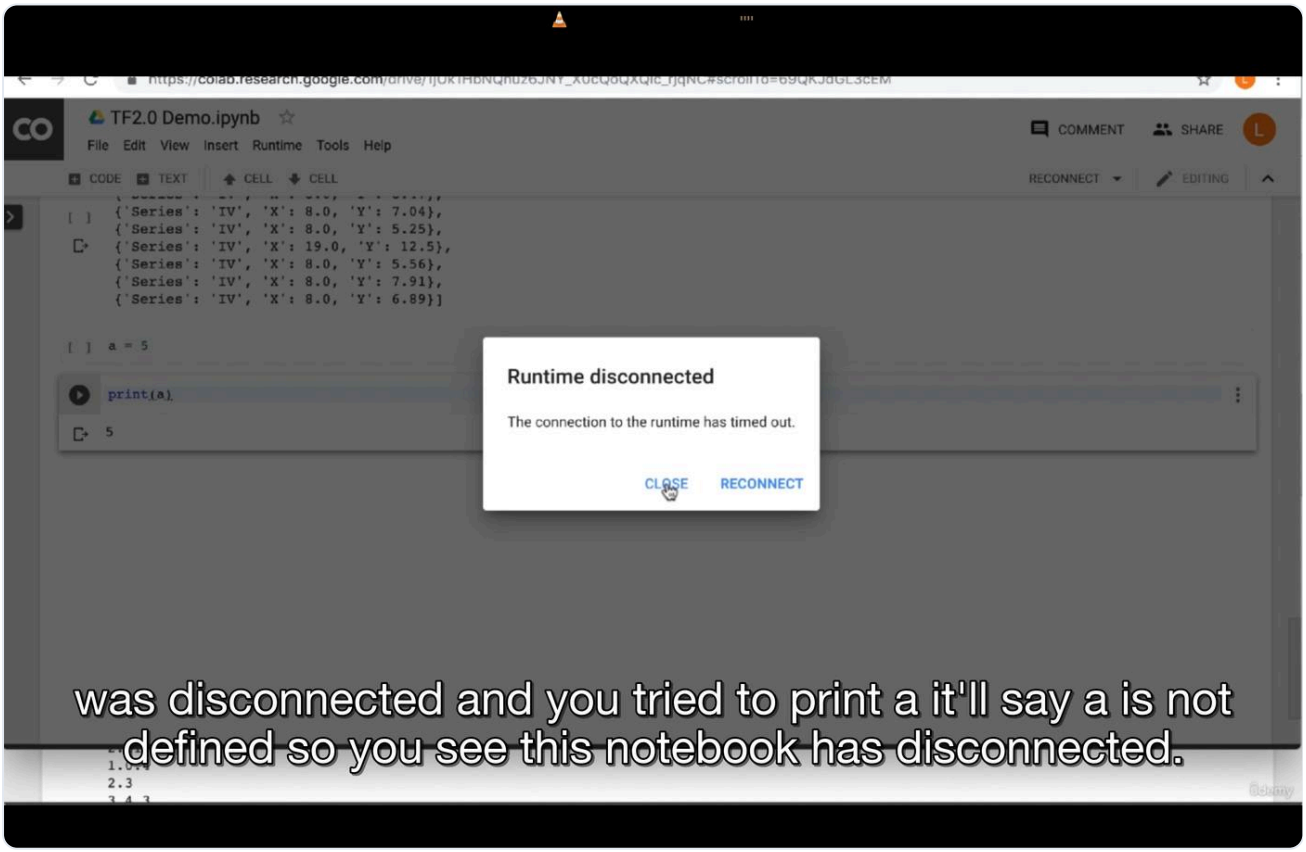


FIGURE 5.27 | Runtime Disconnected

So for example if you define a variable A equals five AND THEN YOU COME BACK LATER AFTER your notebook was disconnected and you tried to print a it'll say a is not defined so you see this notebook has disconnected.

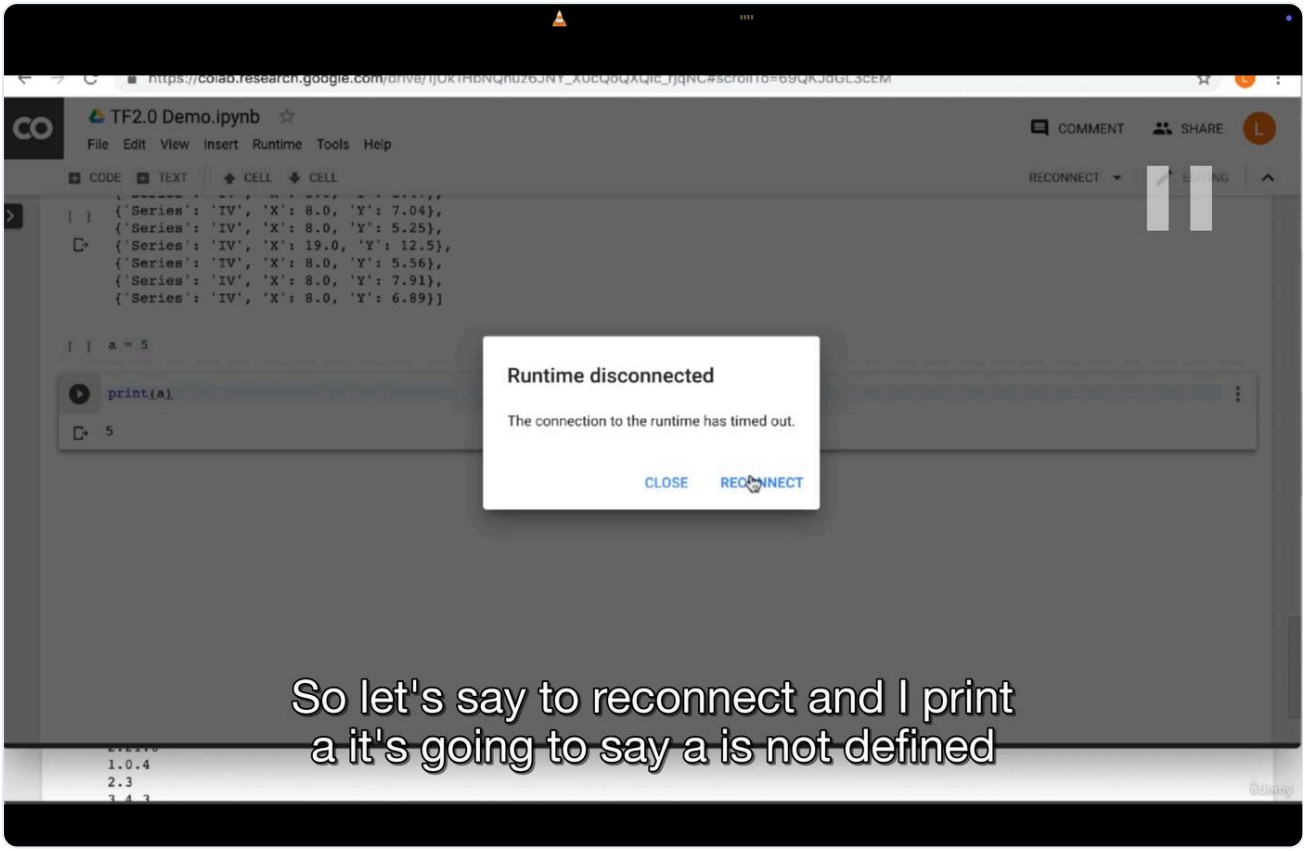


FIGURE 5.28 | Runtime Disconnected

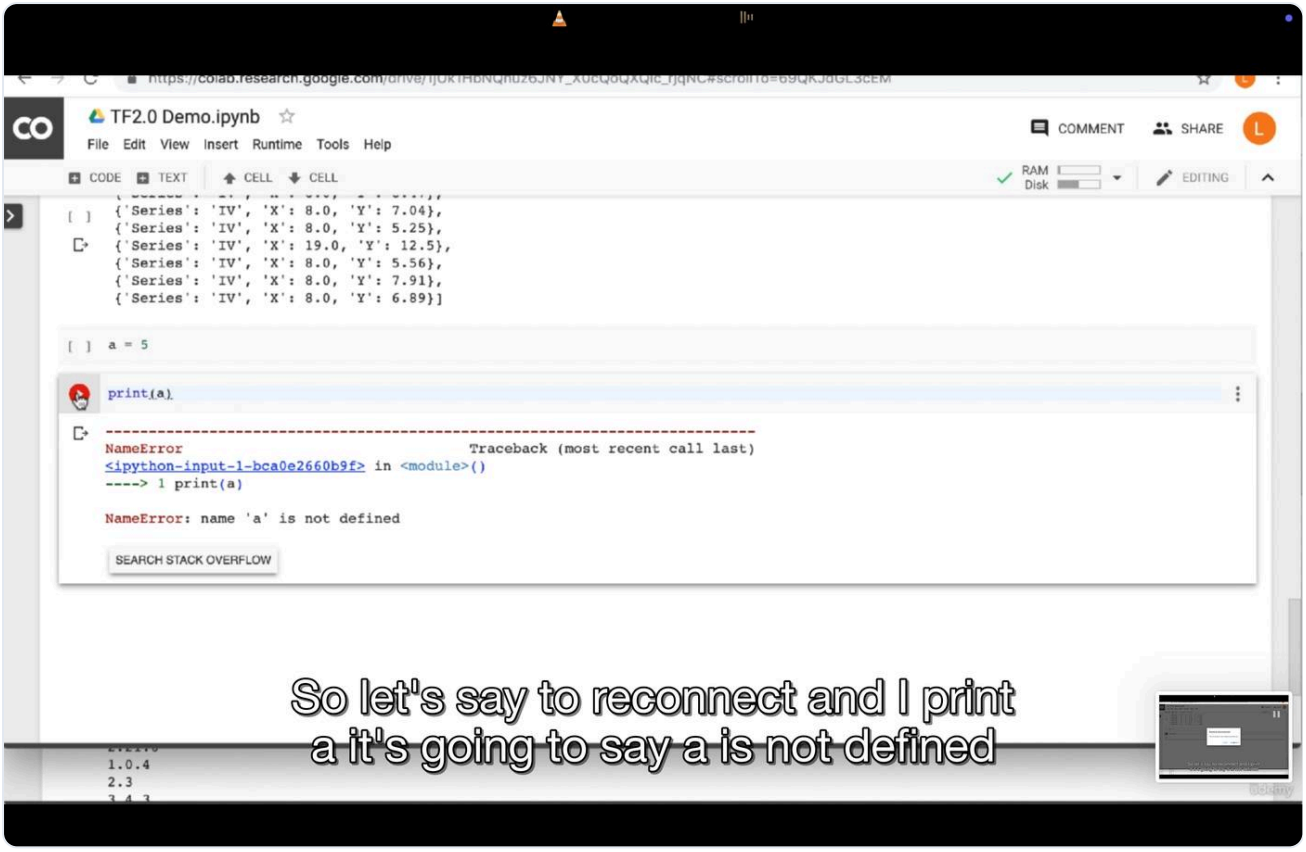


FIGURE 5.29

So let's say to reconnect and I print a it's going to say a is not defined another way this affects you is that you might run out of memory.

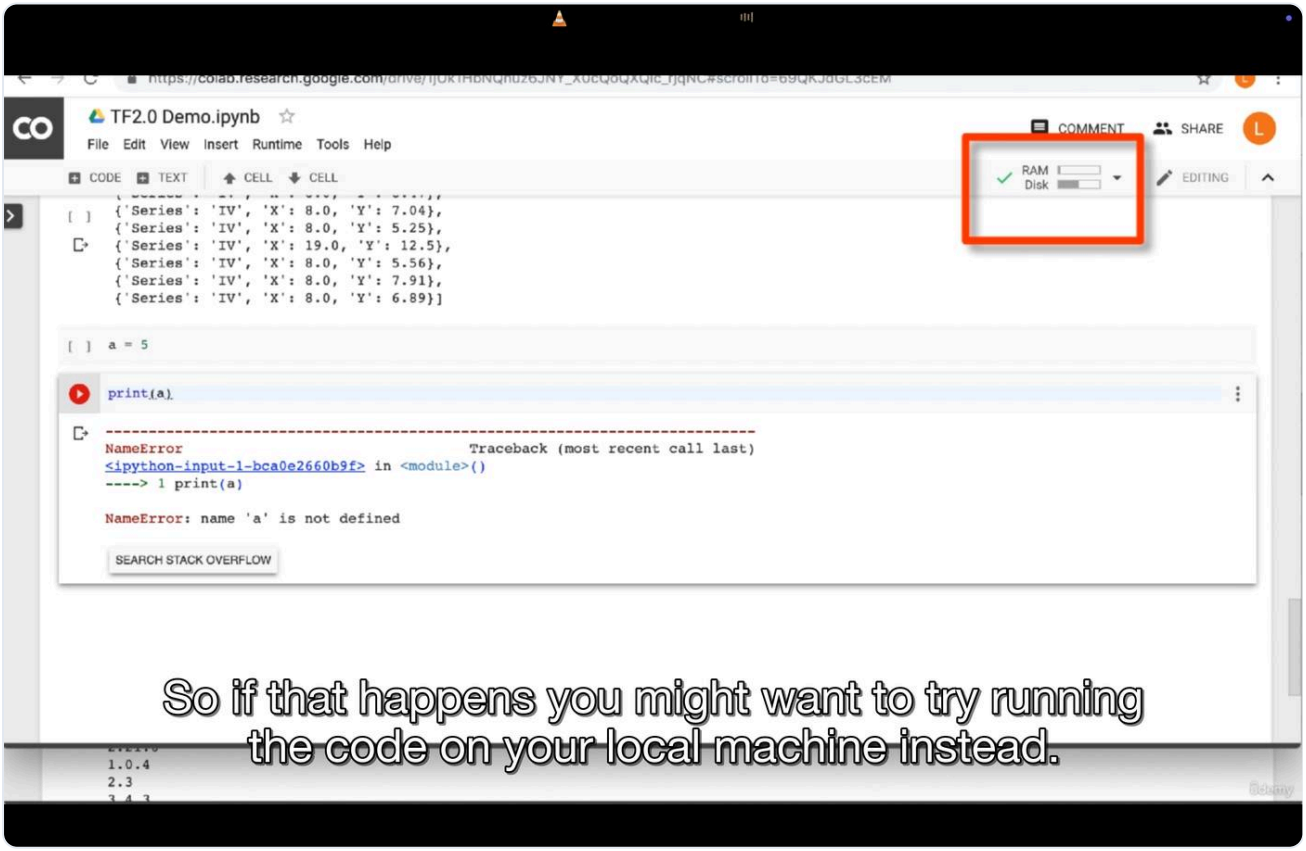


FIGURE 5.30

So if that happens you might want to try running the code on your local machine instead. And as mentioned earlier the GPU or TPU might be unavailable. So either you can run your code without the GPU or TPU or you can run the same code locally as always options you had previously are still available for example you can provision a GPU instance on AWS Which if you choose the correct AMI or Amazon machine image will come with the usual libraries pre installed also.

LECTURE 6

File and Google Drive

In this lecture we are going to do a few more tests in Colab. Specifically we're going to look at some ways to upload your own data set to Colab. Let's say for example you're a client or employer gives you a CSV file or you downloaded a CSV from Kaggle. How can we then make this file accessible from our Colab notebook. In this lecture we are going to discuss a few different ways of doing this.

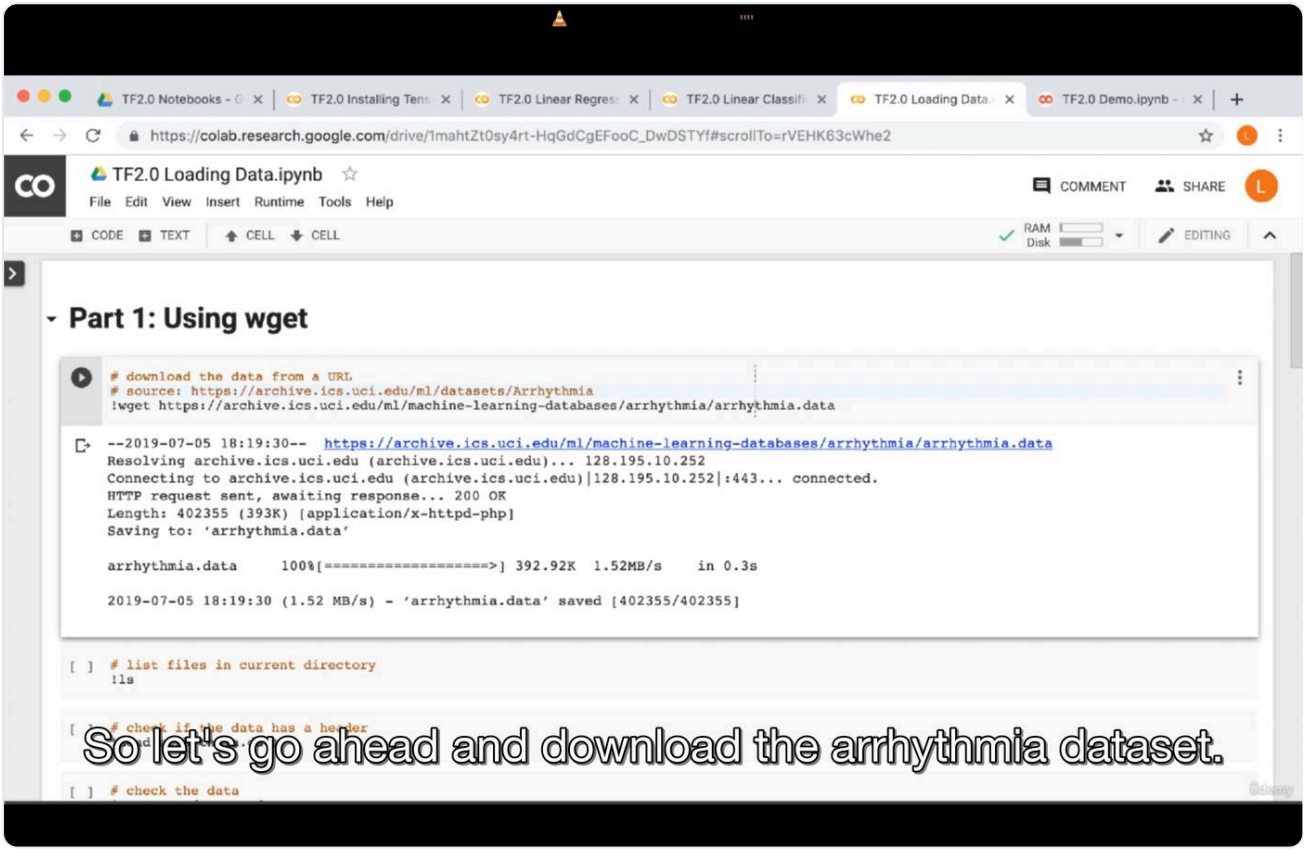


FIGURE 6.1

The first method we're going to look at is just to use the classic Linux command w get as mentioned previously you can run command line commands by preceding the command with the bang symbol or exclamation mark. So let's go ahead and download the arrhythmia dataset. Now we want to check where the data went so let's use bang ls to see if the data is in our current directory OK. It looks like it is now let's use the head command to see the first few lines of the data file and also to check whether or not the file has a header row. So it looks like it does not have a header row next let's try to load in the data using pandas.

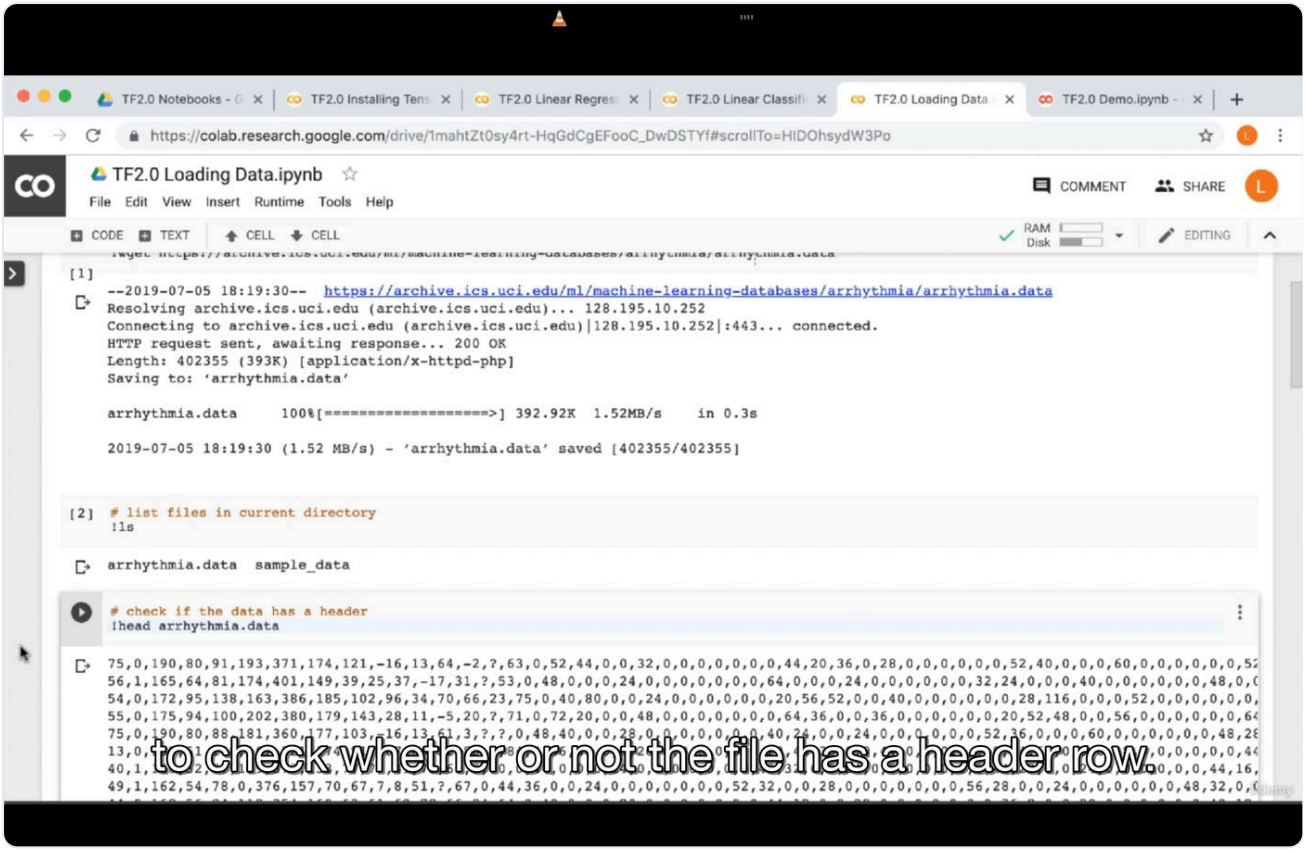


FIGURE 6.2

We're going to pass in a header equals none since we know that the data does not have a header next since the data has many columns. We're just going to take the first few. We're also going to rename the columns because they're currently just integer values as usual since this data is from the UCI machine learning repository. You can just check the documentation if you want to know more about the data like what each column is so let's run this next let's create a histogram of these data columns since notebook by default makes the plot pretty small. We're going to import matplotlib and change the figure size once we've done that.

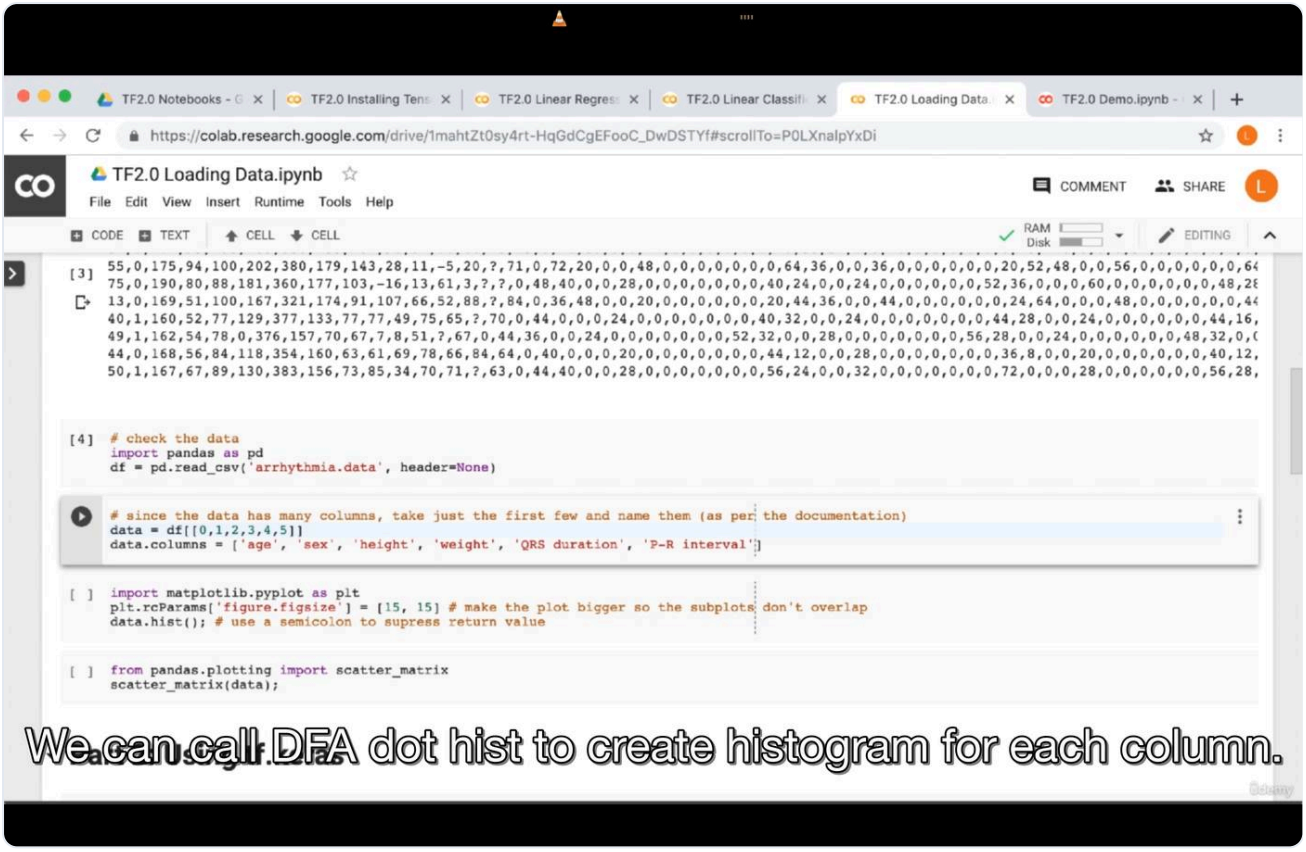


FIGURE 6.3

We can call df dot hist to create histogram for each column. Note that I've added a semicolon to the end of the df dot hist because if you don't then notebook will print out the last returned value like it usually does which we don't want right now.

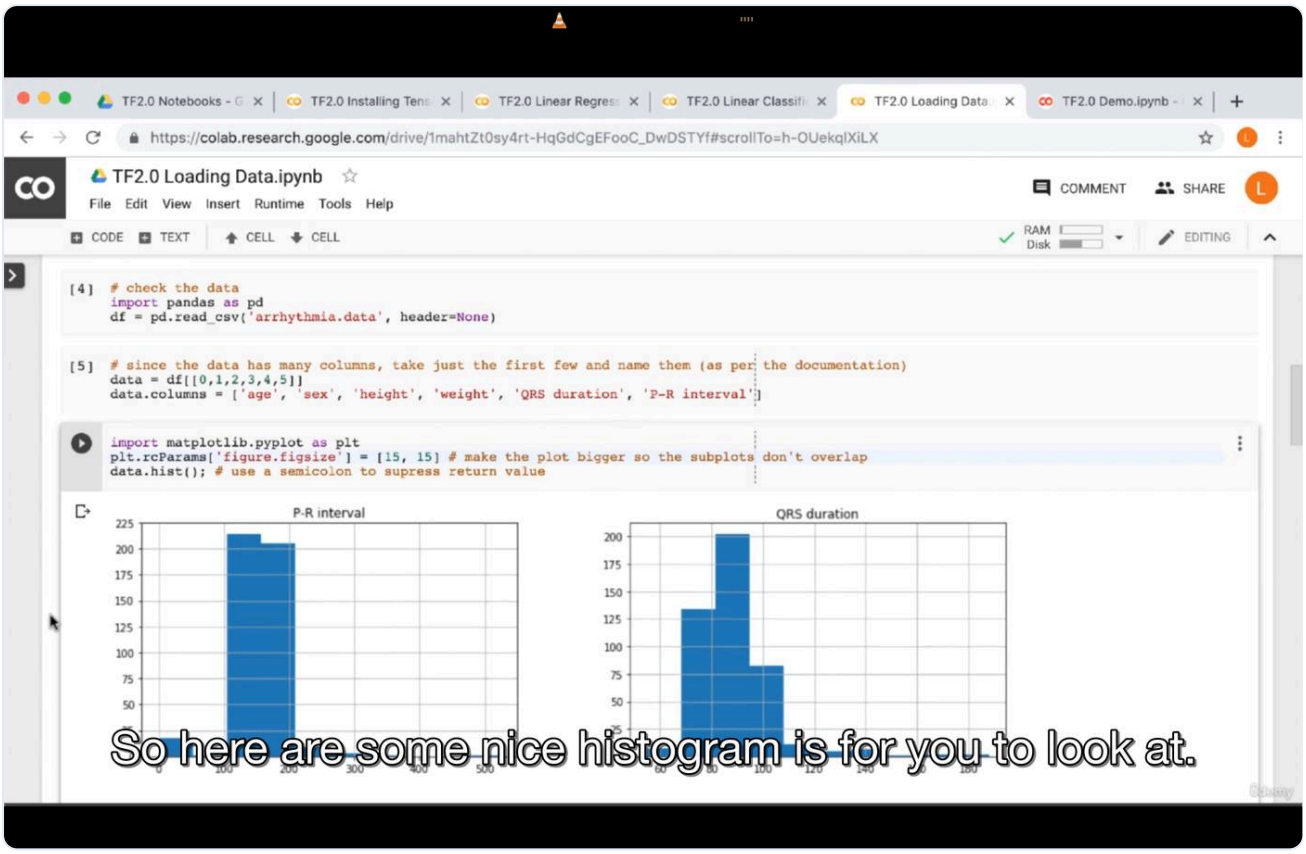


FIGURE 6.4

So here are some nice histograms for you to look at. Next let's create a common plot for data analysis.

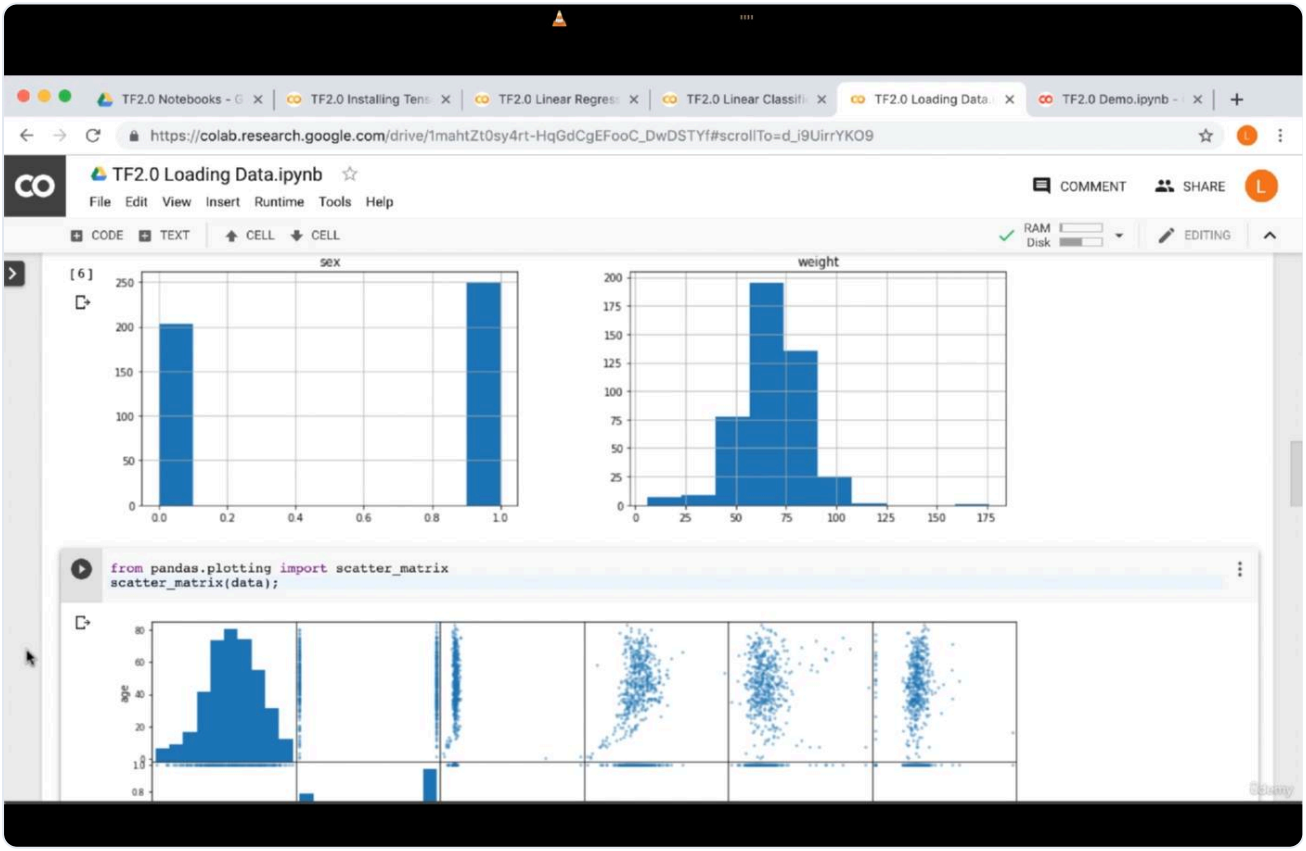


FIGURE 6.5

The scatter matrix. This does a scatter plot between each feature and every other feature along the diagonal it just plots a histogram of each feature which we've already seen all right.

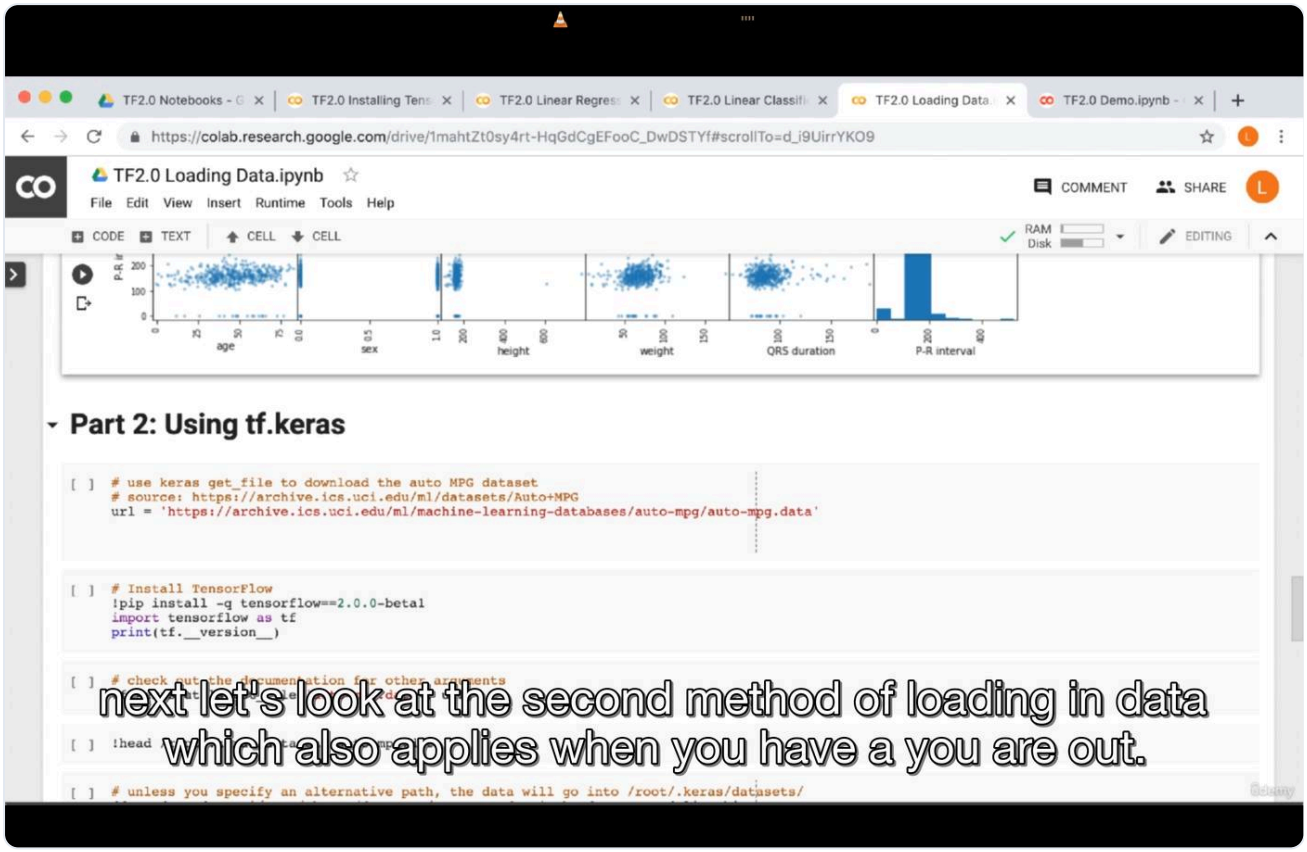


FIGURE 6.6

So pretty standard so far next let's look at the second method of loading in data which also applies when you have a URL. This is to use TensorFlow directly specifically the keras get file function. Now you might be thinking why would we use TensorFlow if this course is about PyTorch. That's a great question and I would understand if you were hesitant. However it's important to realize that these are all just Python libraries. You're not restricted to using only one or the other. That would be silly. As an example there are several libraries for making HTTP requests. One is called requests. Another is called urllib. I don't think anyone would hesitate to use both in their Python code if you need more convincing.

It's actually common. Even if you're working in PyTorch to make use of the text preprocessing functionality that comes with Keras. So in fact data scientists and machine learning engineers have already been doing this for years. Don't think just because I'm using PyTorch that I'm not allowed to use the utilities from any other deep learning library let's start by assigning the URL to a variable called url.

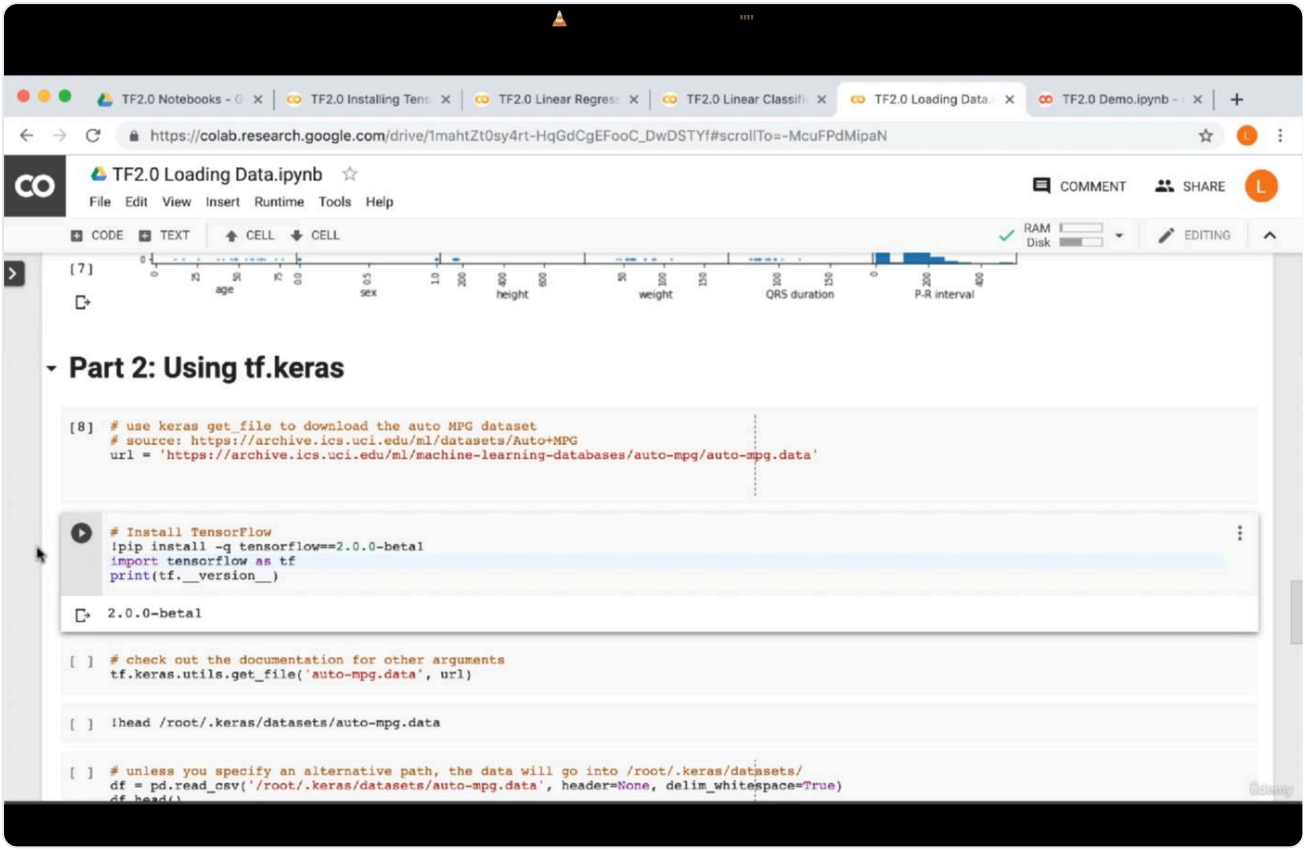


FIGURE 6.7

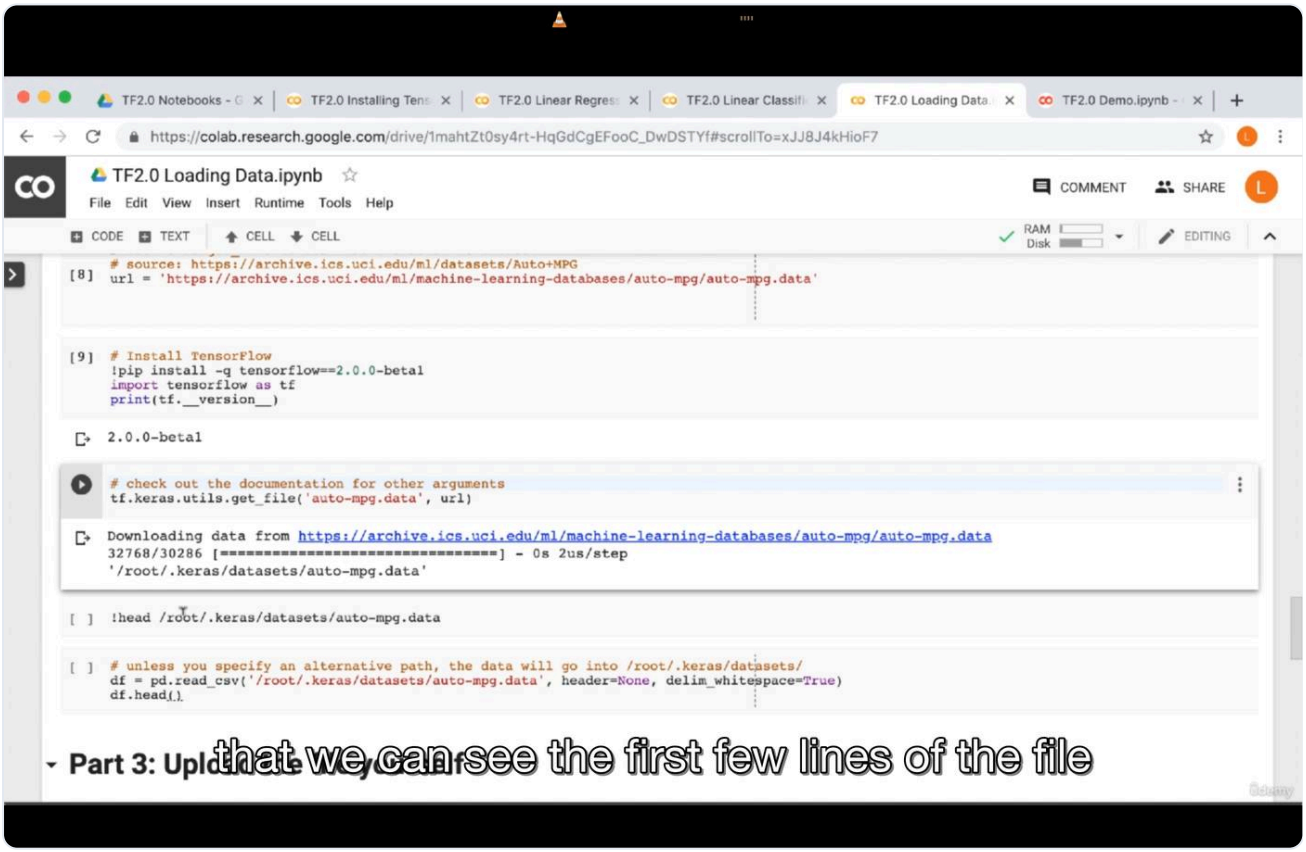


FIGURE 6.8

We're going to be using the auto MPG data set although it doesn't really matter what you use for this example. As long as you can access it directly via a URL let's run this next we're going to make sure we have TensorFlow 2.0 installed so we're going to run pip install TensorFlow and then prints out the version to make sure that we have the correct one note that when I first created this lecture TensorFlow to was not yet released today you could just do import tensorflow as tf and it'll automatically load the latest release which should be 2.0 or higher. In addition remember that this uses the Keras API so an equivalent alternative to this would be to just use Keras itself whenever you see tf dot keras you can replace that with just keras.

So tf dot keras dot get file becomes keras dot get file presuming you imported keras earlier although remember that Keras installation still require a backend.

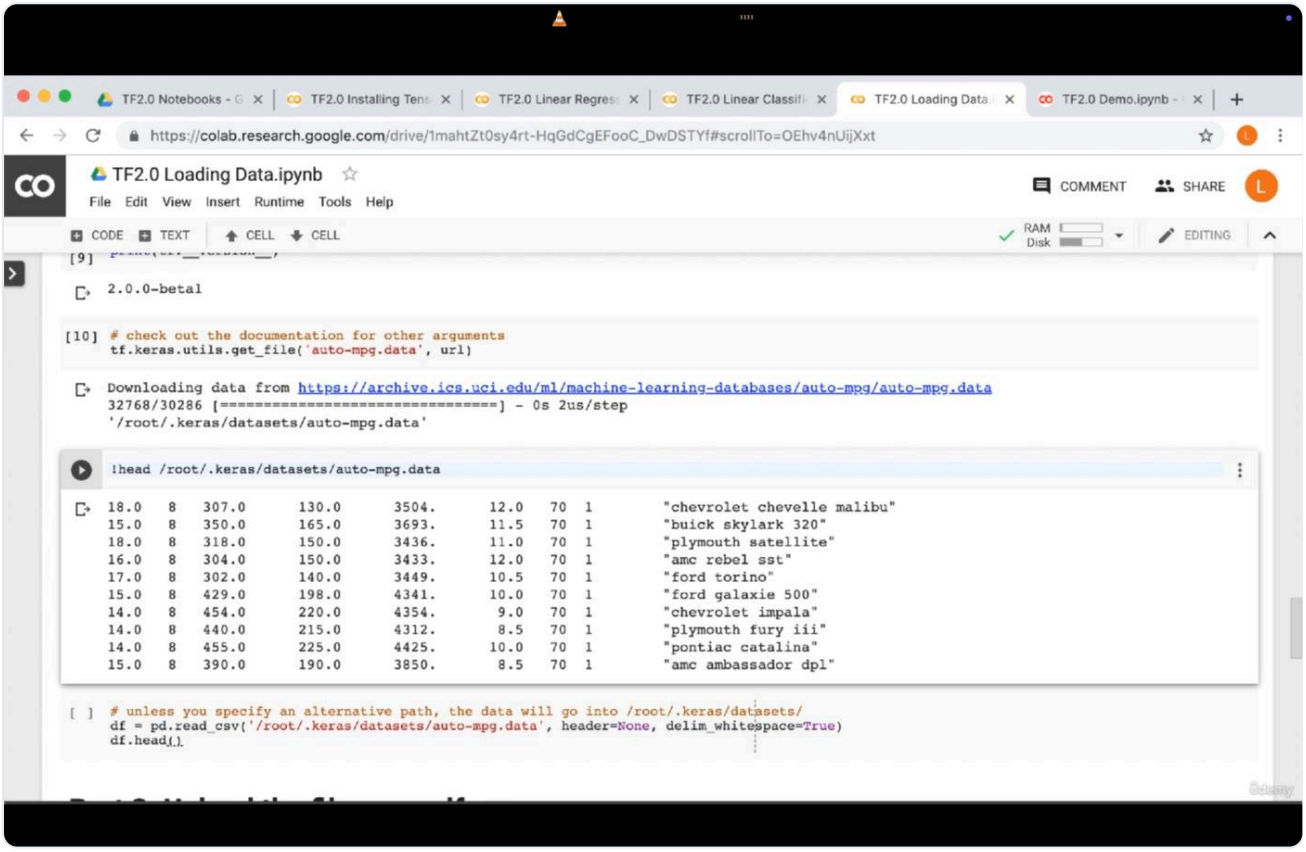


FIGURE 6.9

So ultimately you would probably already have TensorFlow installed next we're going to call the keras get file function. The first argument is the file path. We want to save too.

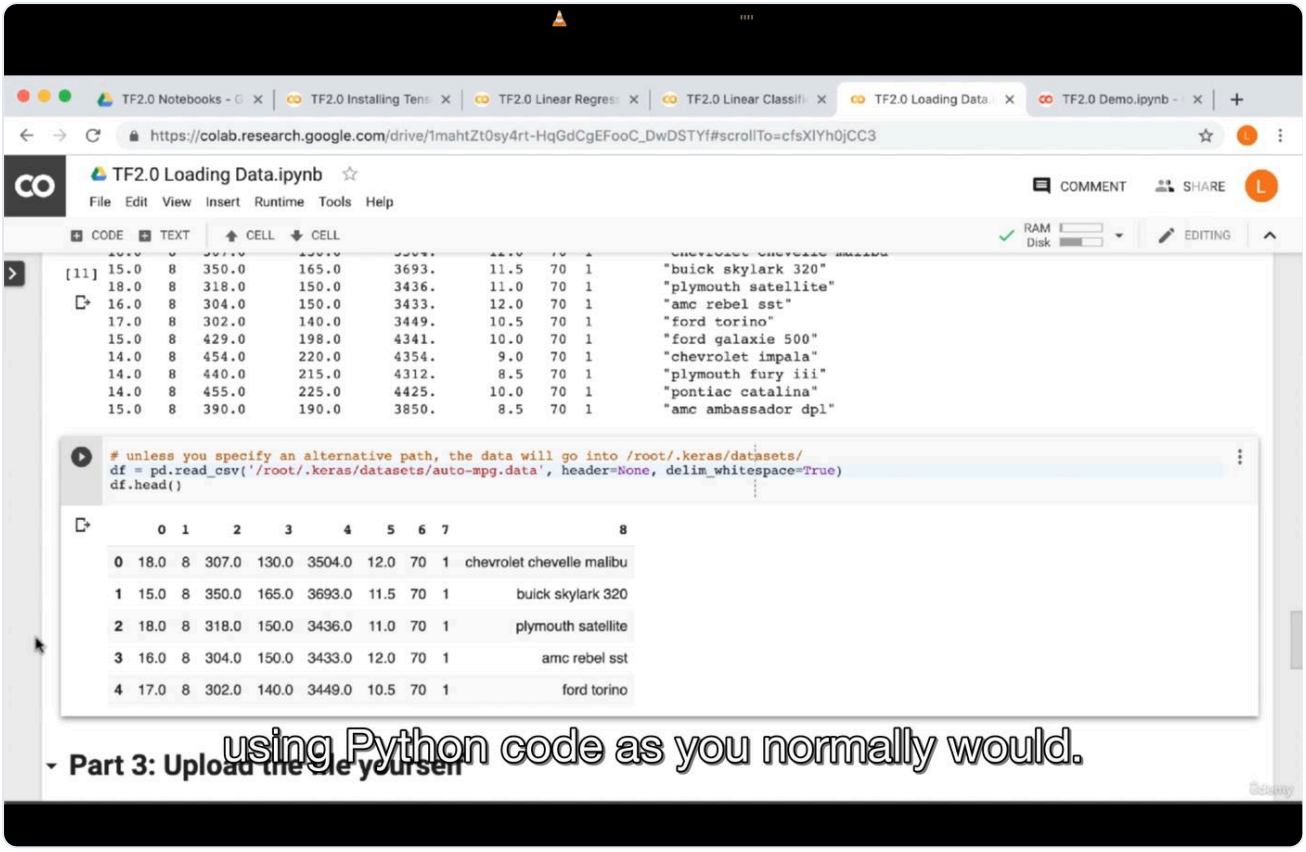


FIGURE 6.10

And the second argument is the far source so let's run this note that it's possible to save the file to a different directory but we'll be saving it to Keras's default folder so you can see from the print-out that the file ends up in slash root slash keras slash datasets next let's call the head command so that we can see the first few lines of the file as you can see it's not exactly a CSV. Instead each column is separated by whitespace and there is no header so in order to load this data we can still use the pandas read csv function but we have to pass in two extra arguments the first argument is to say that there's no header so header equals none. And the second extra argument is to tell pandas that the delimiter is whitespace.

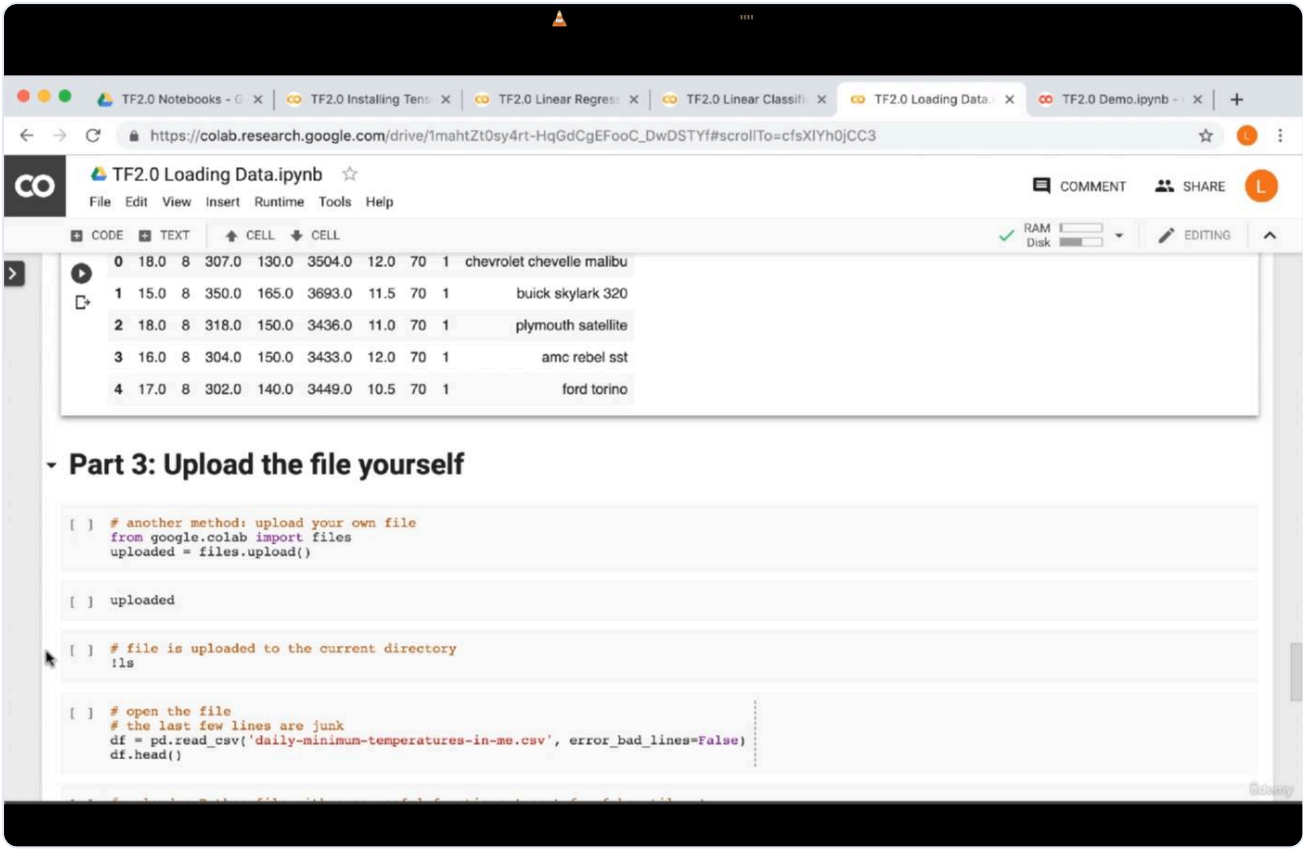


FIGURE 6.11

So we set delim whitespace equal to true X. We call df dot head is to make sure everything works as expected. So as you can see the result appears to be in the right format and from here you can process this data using Python code as you normally would. The third method we're going to look at in order to add your own files to Colab is to upload the file directly in order to do this.

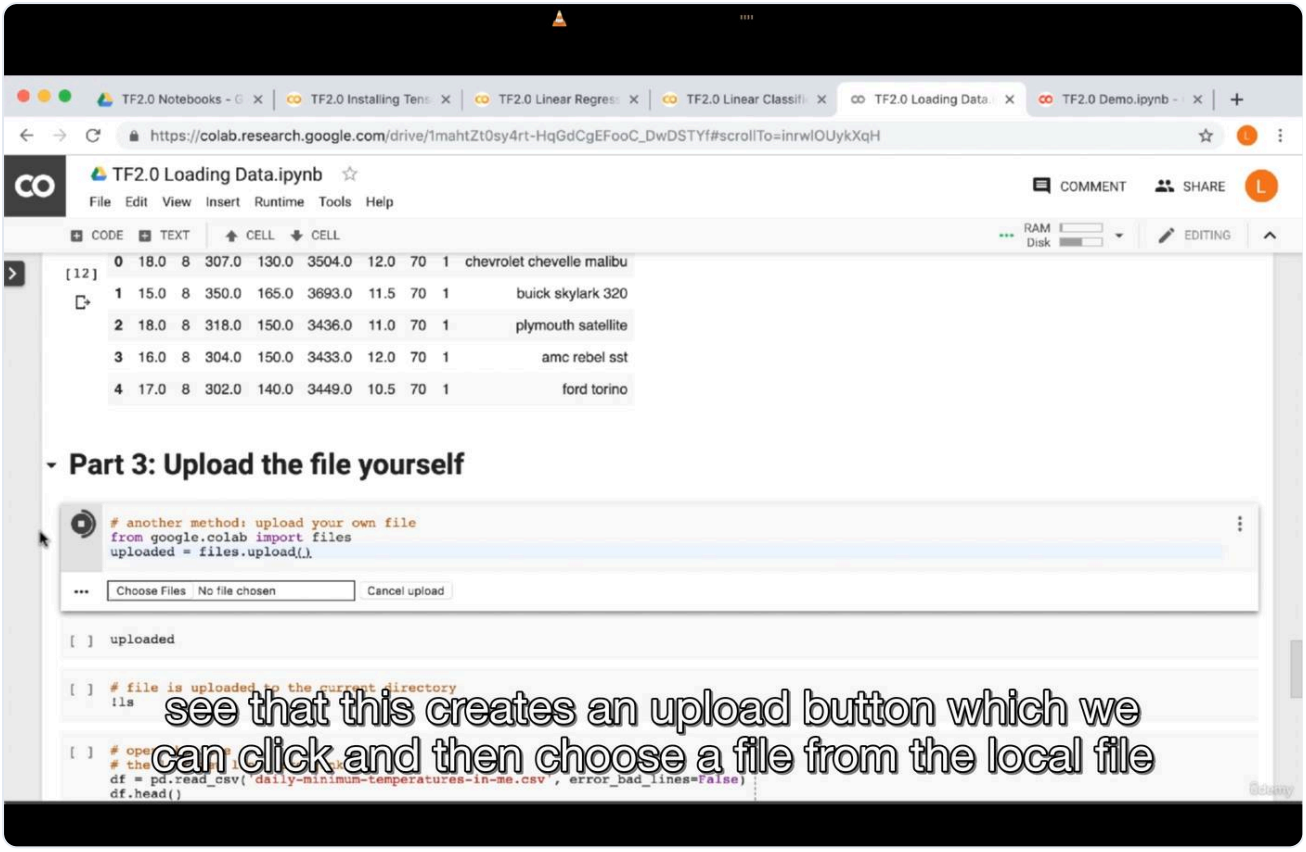


FIGURE 6.12

We have to run a special Colab function so we say from google dot colab import files then we call files dot upload so let's run this so you see that this creates an upload button which we can click and then choose a file from the local file system.

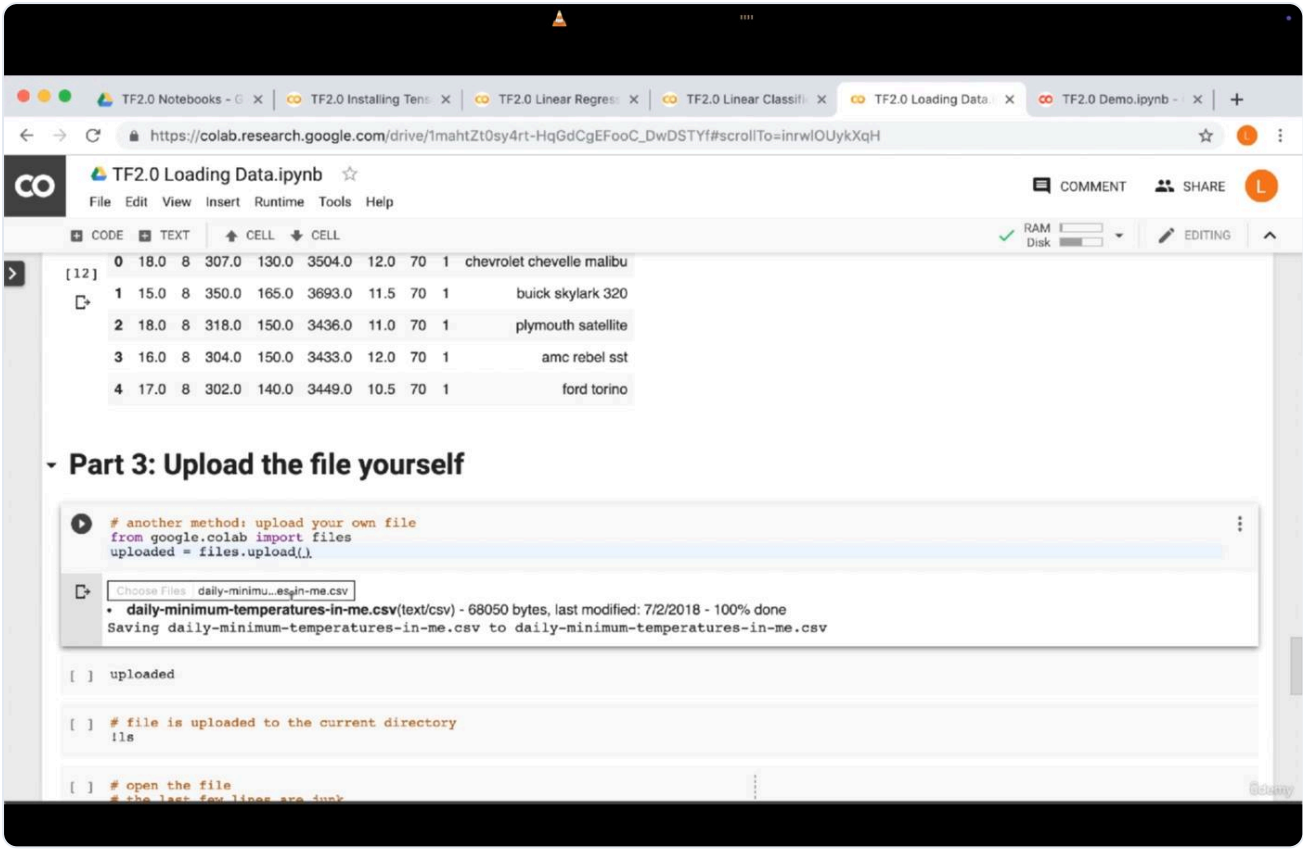


FIGURE 6.13

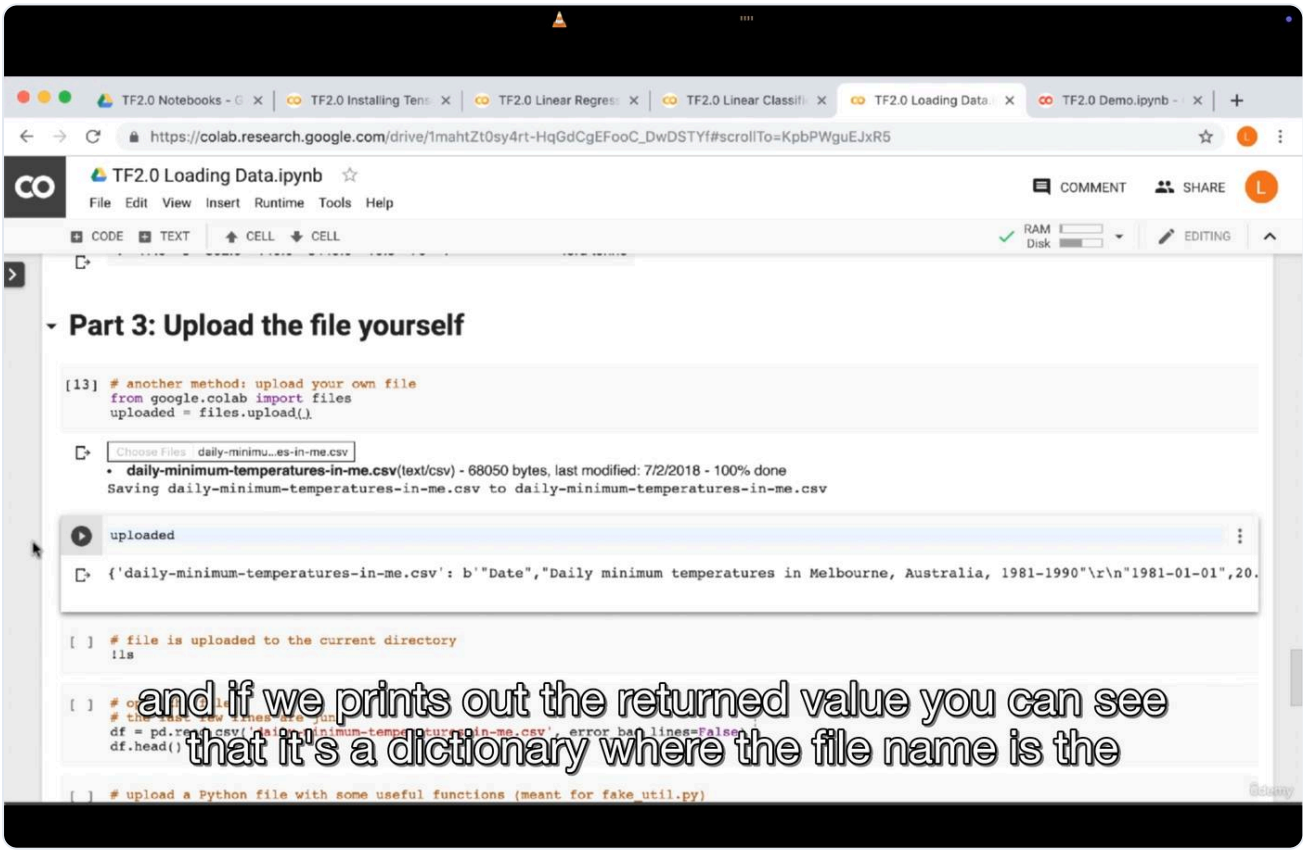


FIGURE 6.14

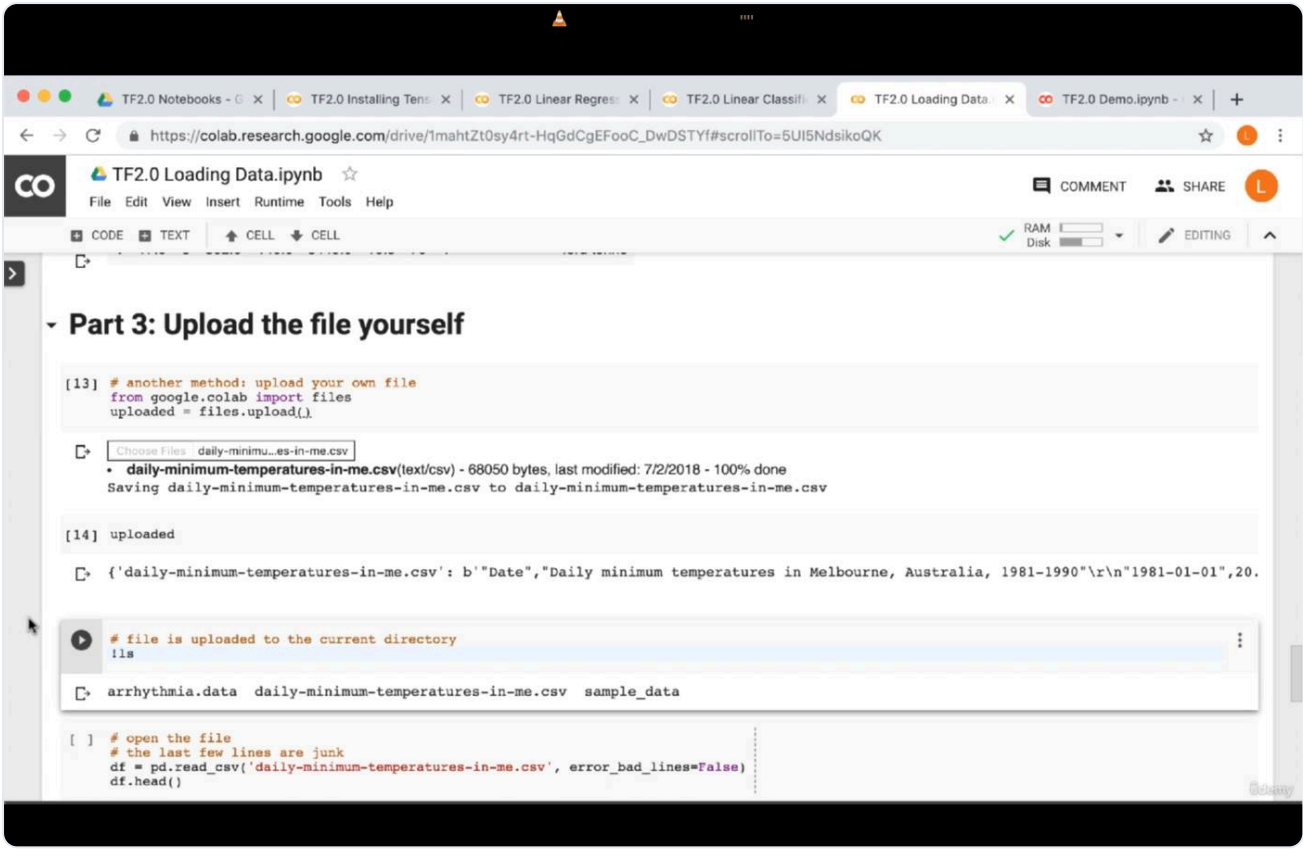


FIGURE 6.15

Some of the truce daily minimum temperatures and if we prints out the returned value you can see that it's a dictionary where the file name is the key and the value is the file contents if we use the command bang ls we can see that the file has been uploaded to the working directory next let's read in the file using pandas to make sure we get what we expect now this file has some garbage lines near the end. So I've accounted for that by setting the argument error bad lines equal to False. This ignores errors but prints them out as they are encountered as you can see the file is loaded in successfully to follow up this example we're going to look at a variation on what we just did.

You recall that when you're writing code in Python sometimes it's useful to split your code amongst several files. This helps to organize your code and keep similar things all in one place while keeping different things separate as a simple example. Sometimes we'll learn about multiple algorithms in one course but we'll test all those algorithms on the same dataset.

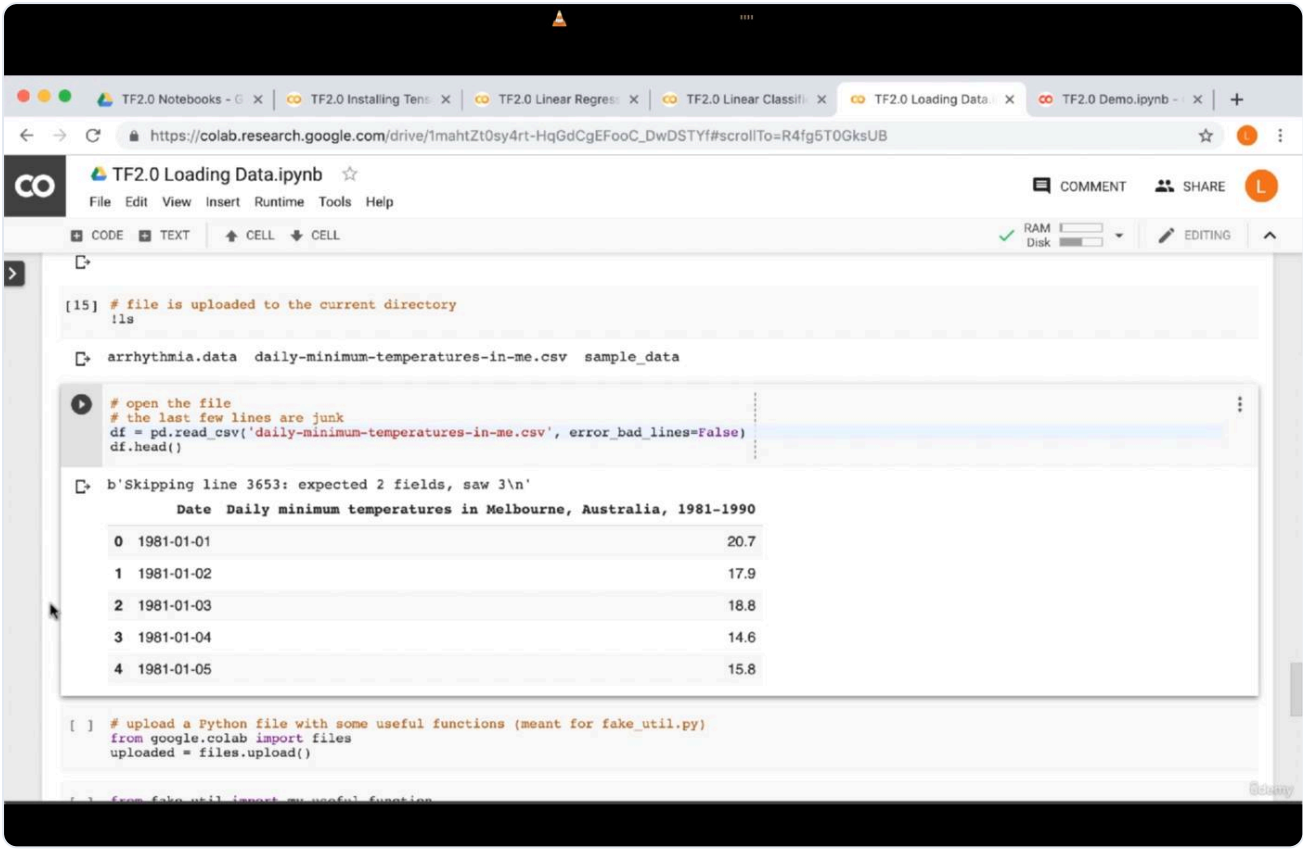


FIGURE 6.16

So there's no point in rewriting the code to load in the data set multiple different times. Instead we can write the data loading code once and then import it from each file. Now you might wonder is since we're working in Colab how can you import a function from a Python script. If that python script is on your local harddrive luckily we can take the same approach we already have been to upload that file to Google Colab. So here I'm going to call files dot upload again and this time I'm uploading the python script. fake util dot py so fake util dot py contains only one function called my useful function and all it does is print out hello world. So once you've uploaded the file you can see that we can import it just like we would if we were working locally.

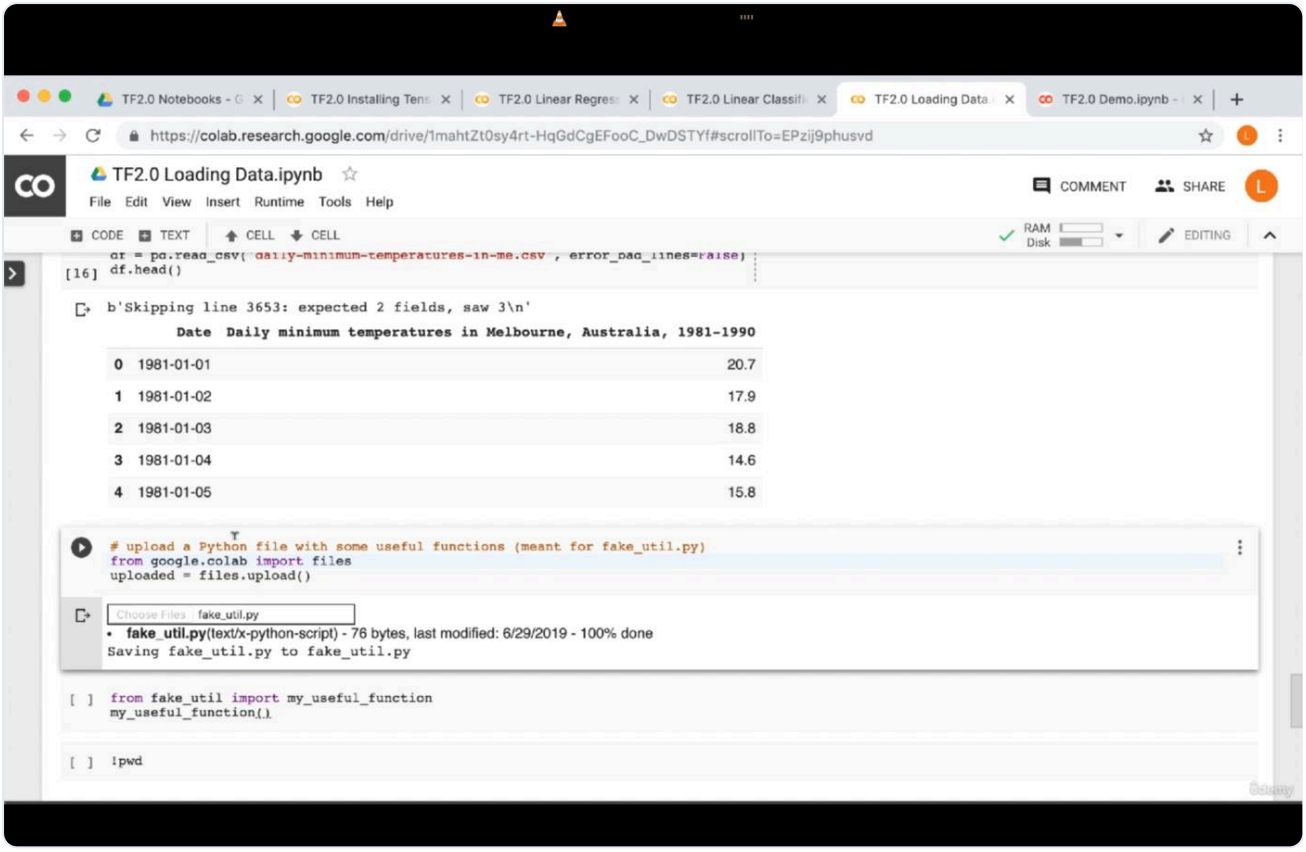


FIGURE 6.17

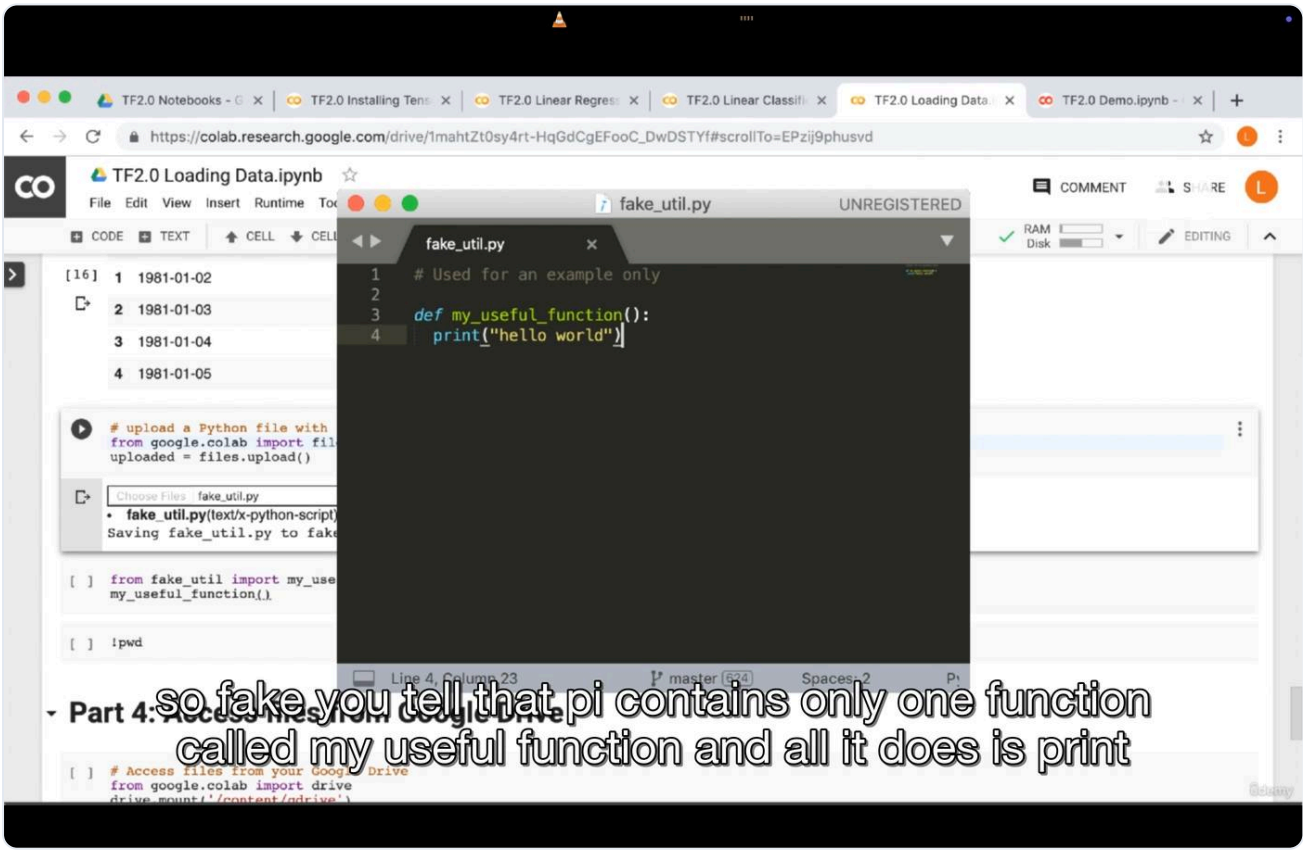


FIGURE 6.18

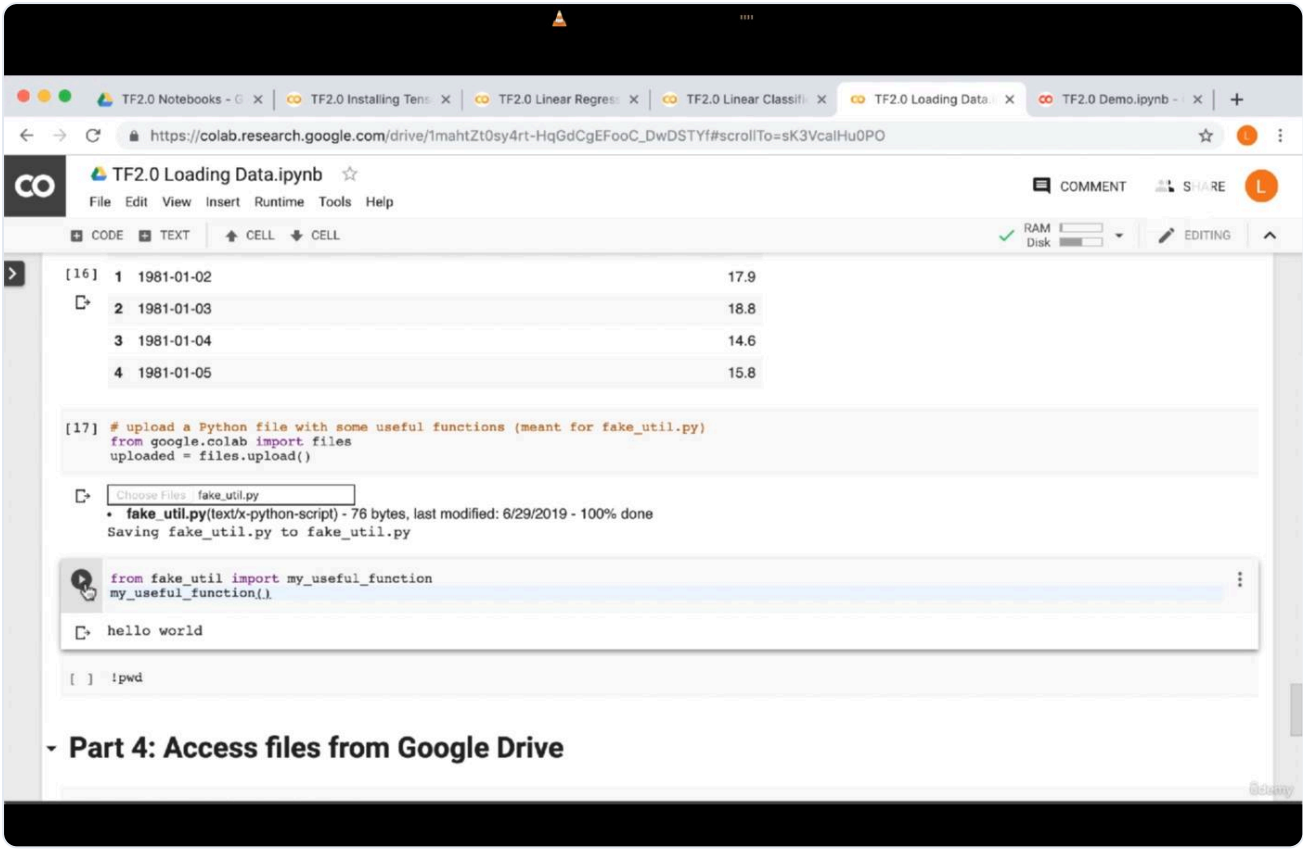


FIGURE 6.19

So I can say from fake util import my useful function. Then when I call my useful function you can see that hello world is printed out just like we expect and by the way you might be wondering as I did what the path of the current directory actually is to determine this. You can just run the usual Linux command pwd and that prints out slash content so slash content is our current working directory the last thing I want to cover is something you're probably all wondering. Google Drive is for storing files. So is it possible to access files on your google drive.

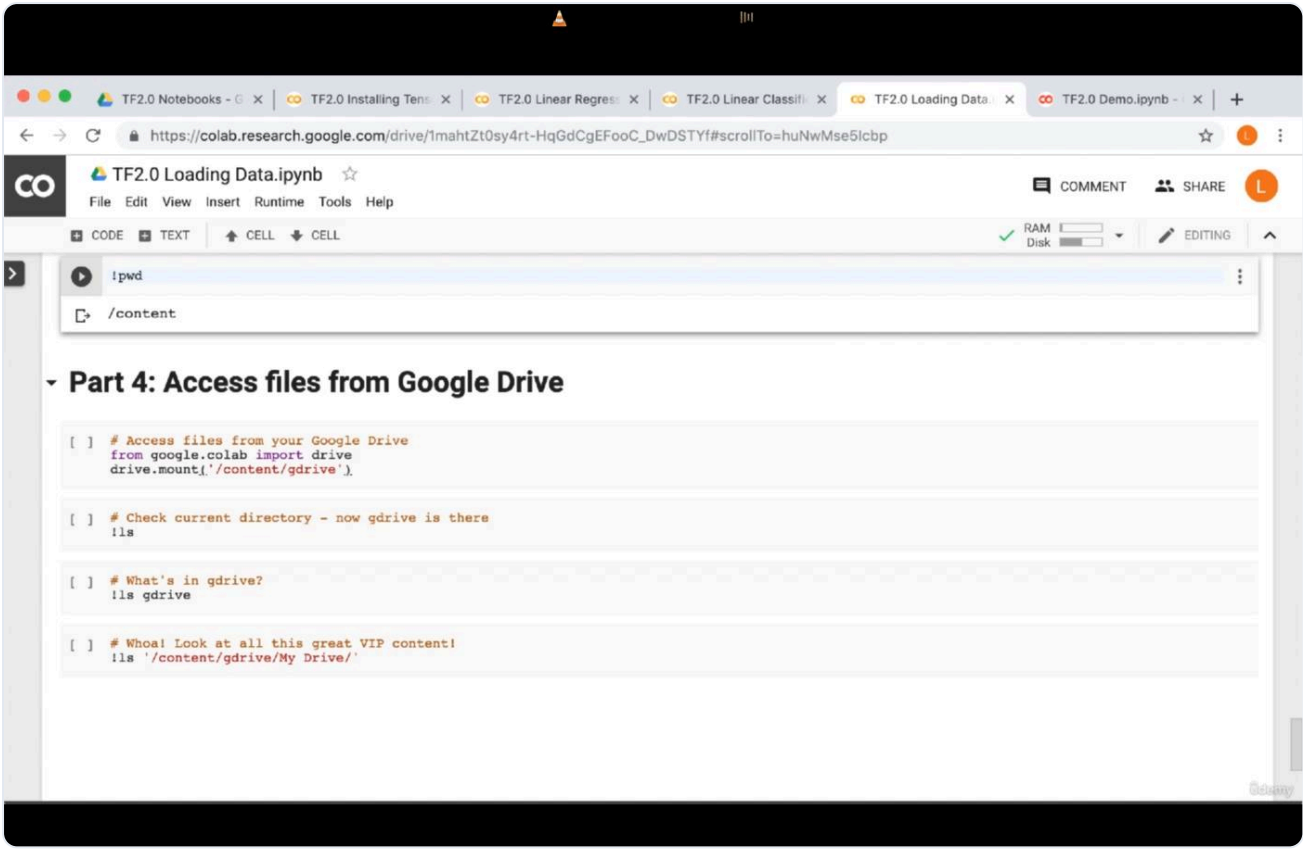


FIGURE 6.20

And of course the answer is yes. So in order to do this we have to import drive from google dot colab then we have to mount the drive by calling drive dot mount and specifying the path slash content slash gdrive. So this is going to give you an authorization code. So you go to the URL in your browser it asks you to sign in as a accepts terms and then it gives you a code. You copy this code and you put it back into this box. You hit Enter Okay so that works.

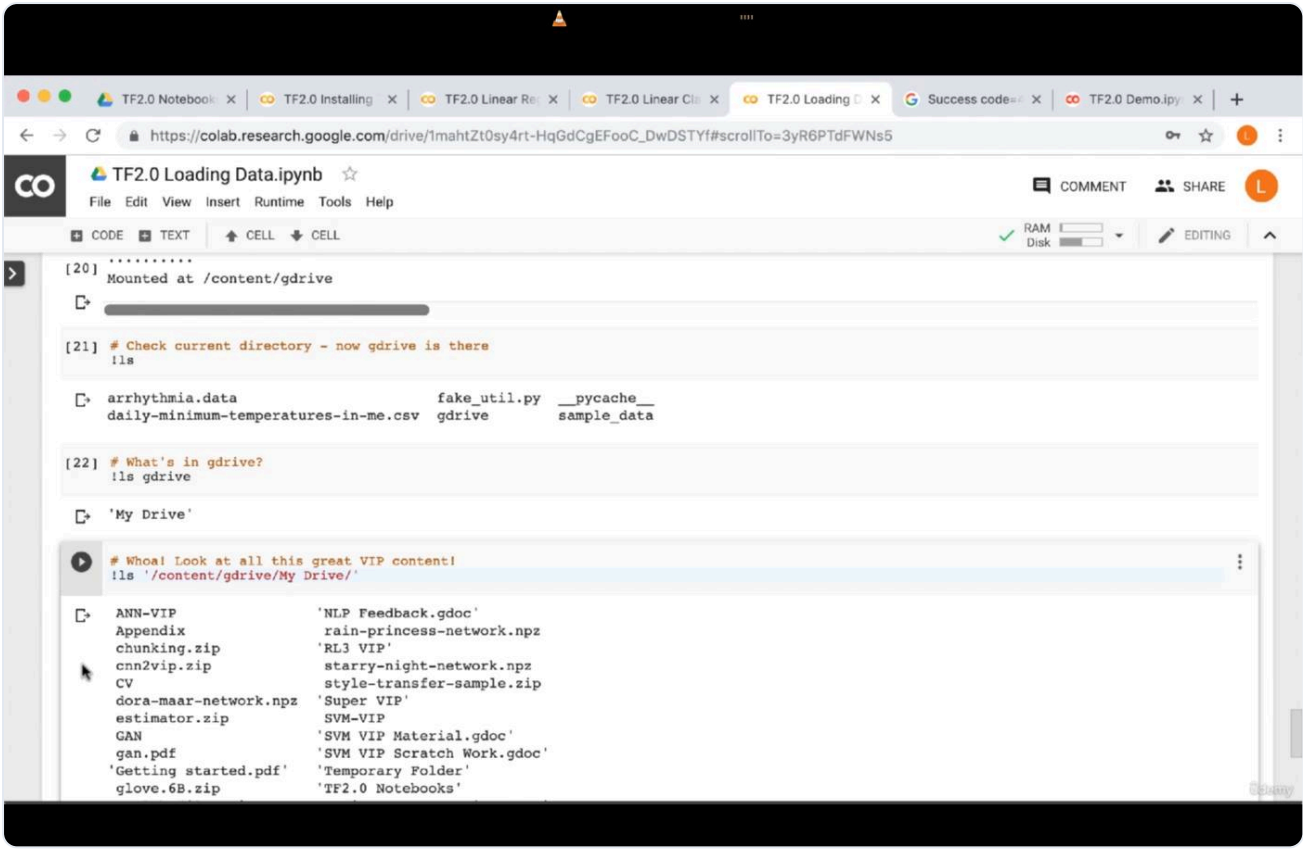


FIGURE 6.21

So after we've done this we can call ls again to check what's now in the current directory we can see that there is now an extra thing here. gdrive. So let's ls gdrive and see what that gives us. All right. So it looks like we now have a thing called Google Drive. Once again I'll ls this.

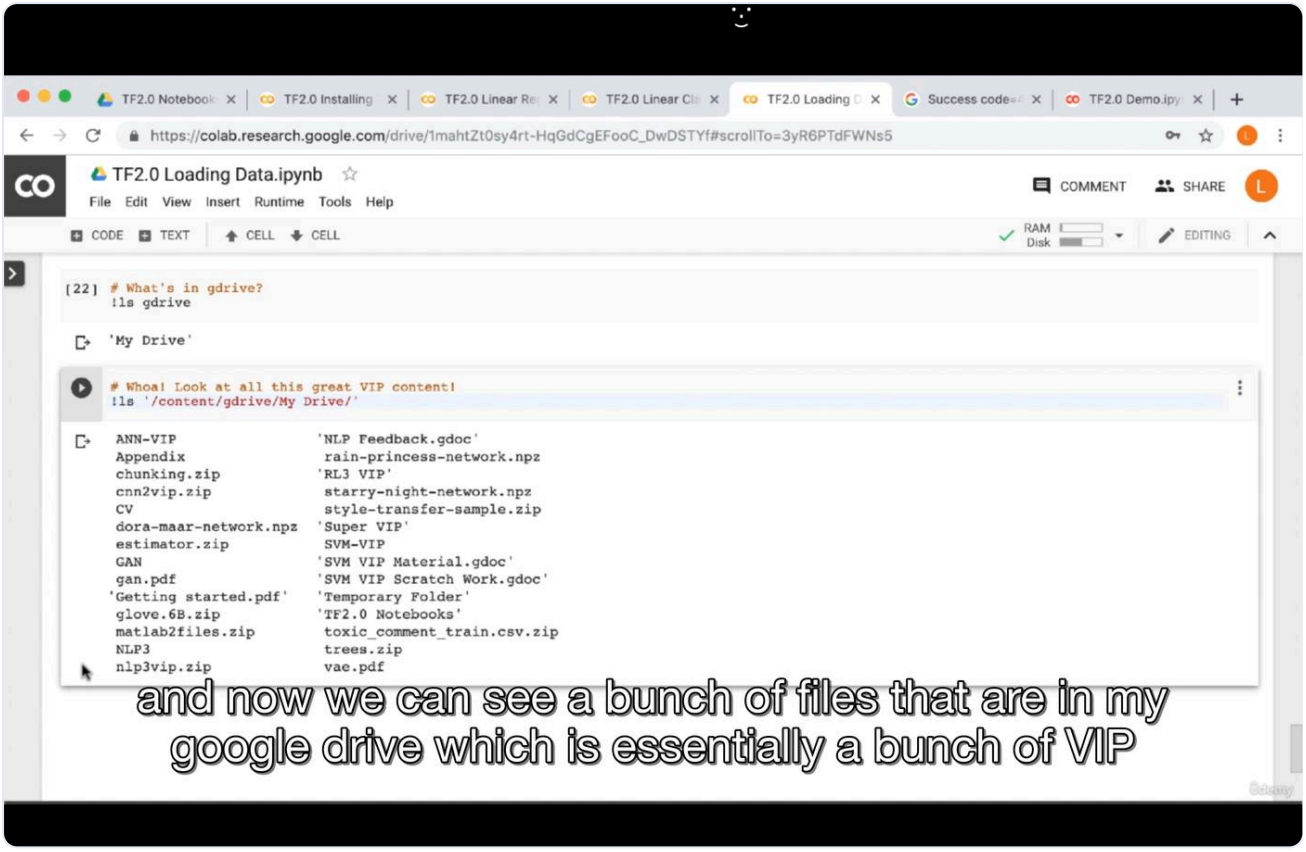


FIGURE 6.22

And remember that you have to add quotes if your path contains whitespace and now we can see a bunch of files that are in my google drive which is essentially a bunch of VIP content for the VIP versions of my courses.

LECTURE 7

The Steps of a Machine Learning Script

All right so if you saw the past few lectures and you had no idea what was going on you might be a bit alarmed. You might be wondering what the heck is NumPy and pandas. And where can I learn about them.

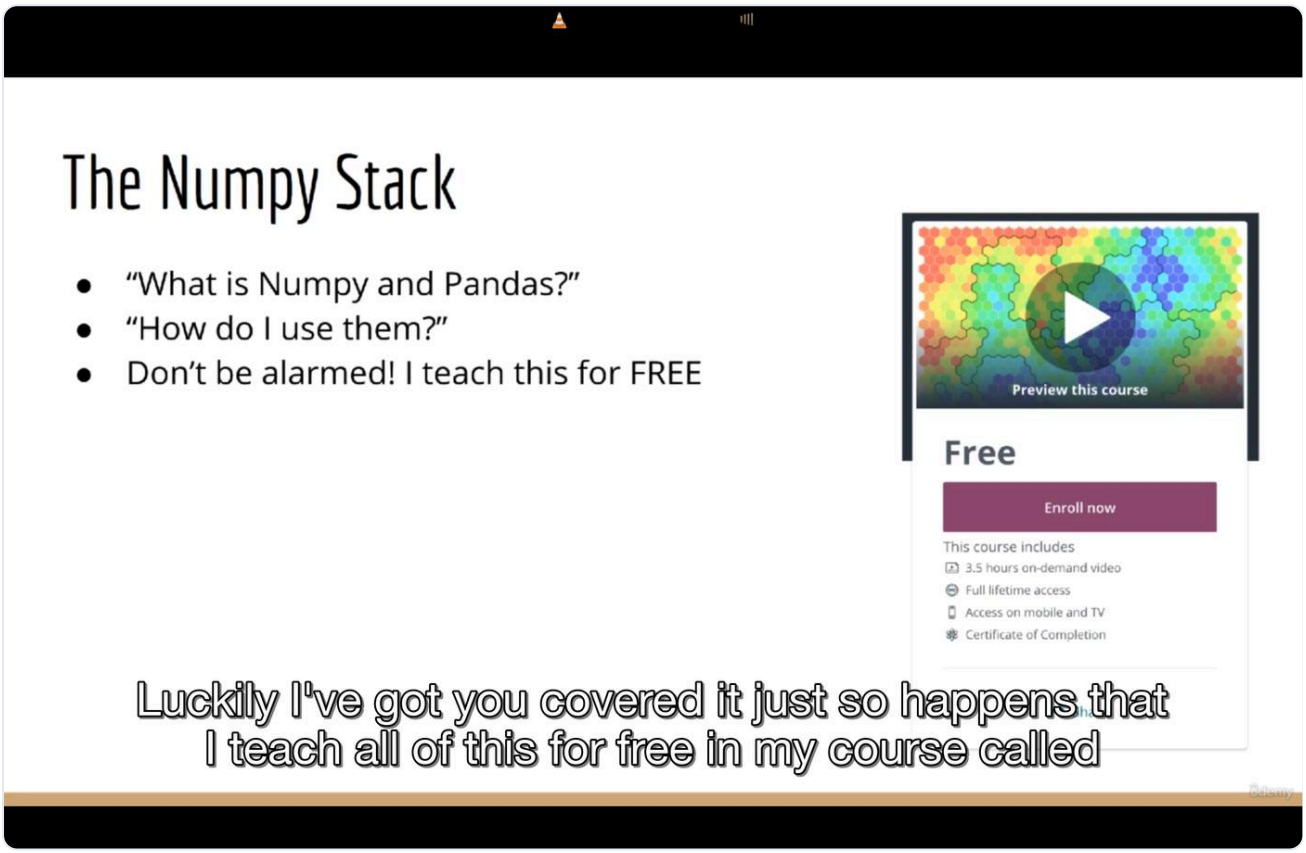


FIGURE 7.1 | The Numpy Stack

Luckily I've got you covered it just so happens that I teach all of this for free in my course called the NumPy stack in Python as usual.

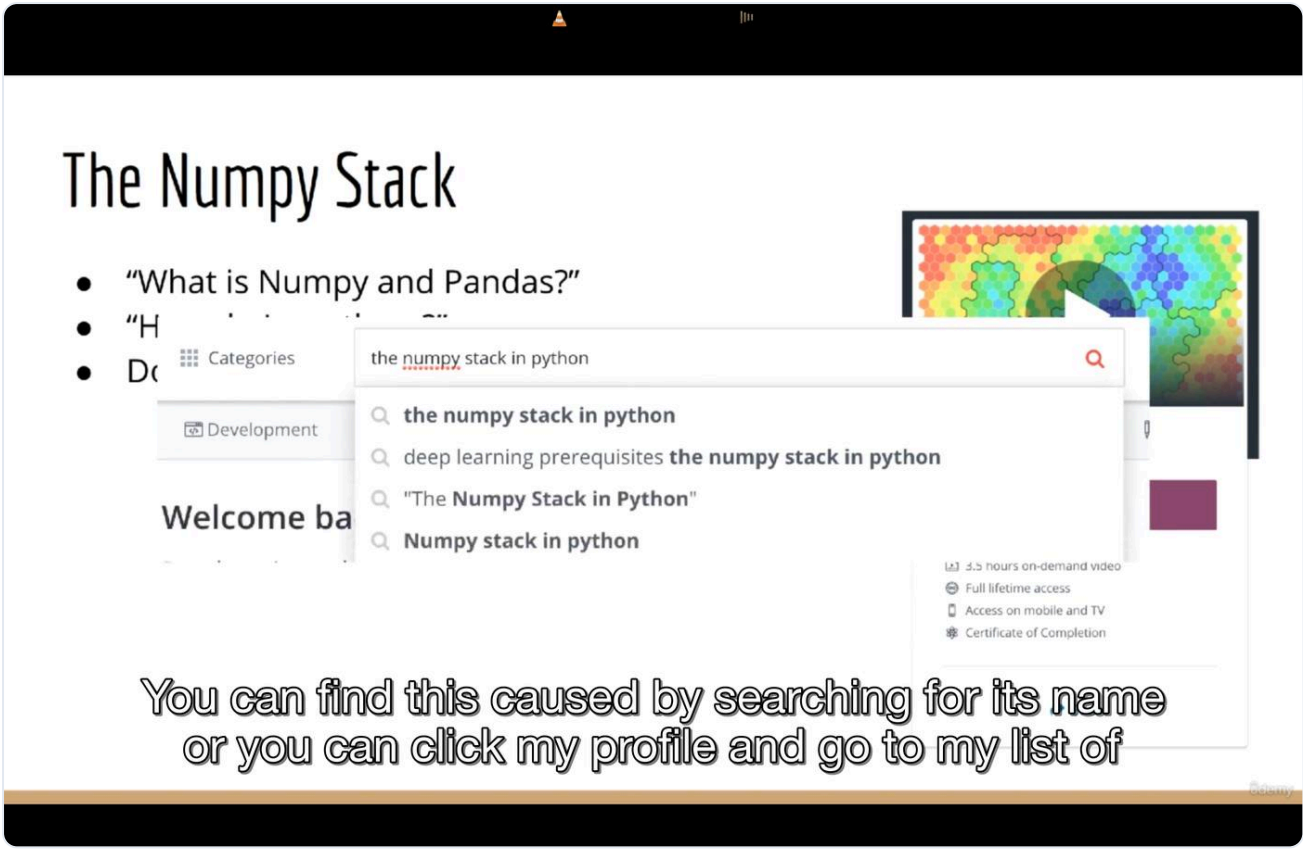


FIGURE 7.2 | The Numpy Stack

You can find this course by searching for its name or you can click my profile and go to my list of courses. If you still can't find it you can go to my Web site.

FIGURE 7.3 | The Numpy Stack

Lazy program or die me or list out all my courses and link to them there. Now because this course is free I'm going to assume that you've taken the time to do your best to prepare for this course. That means knowing the basics of NumPy matplotlib pandas SciPy and scikit-learn. Of course when I say basics that might be a little ambiguous what is basic to me may not be basic to you.

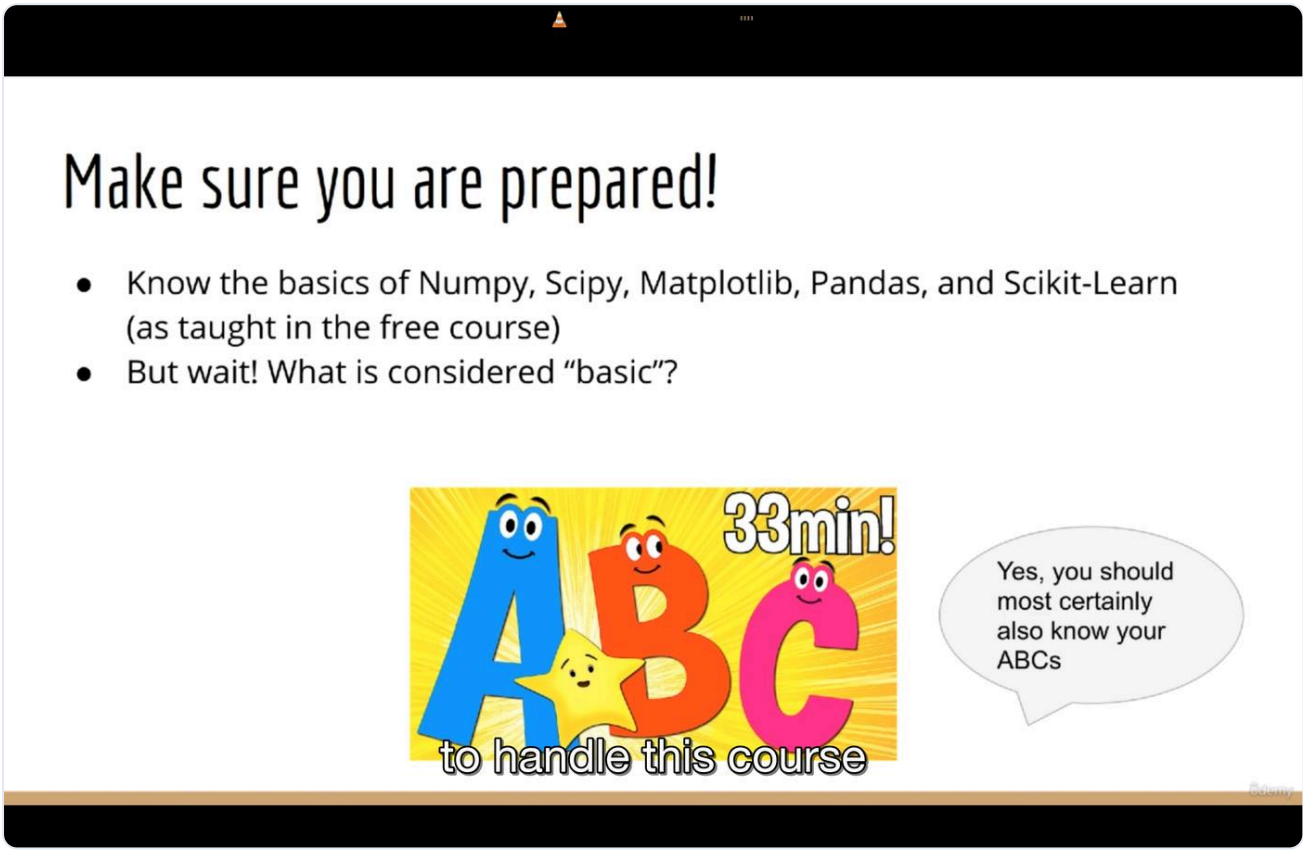


FIGURE 7.4

So let's enumerate and review what you should have learned so that you can make sure you are well equipped to handle this course for NumPy. I essentially want you to know matrix arithmetic in deep learning. We work with tensors which might sound scary but we actually just mean array like a regular old programming array.

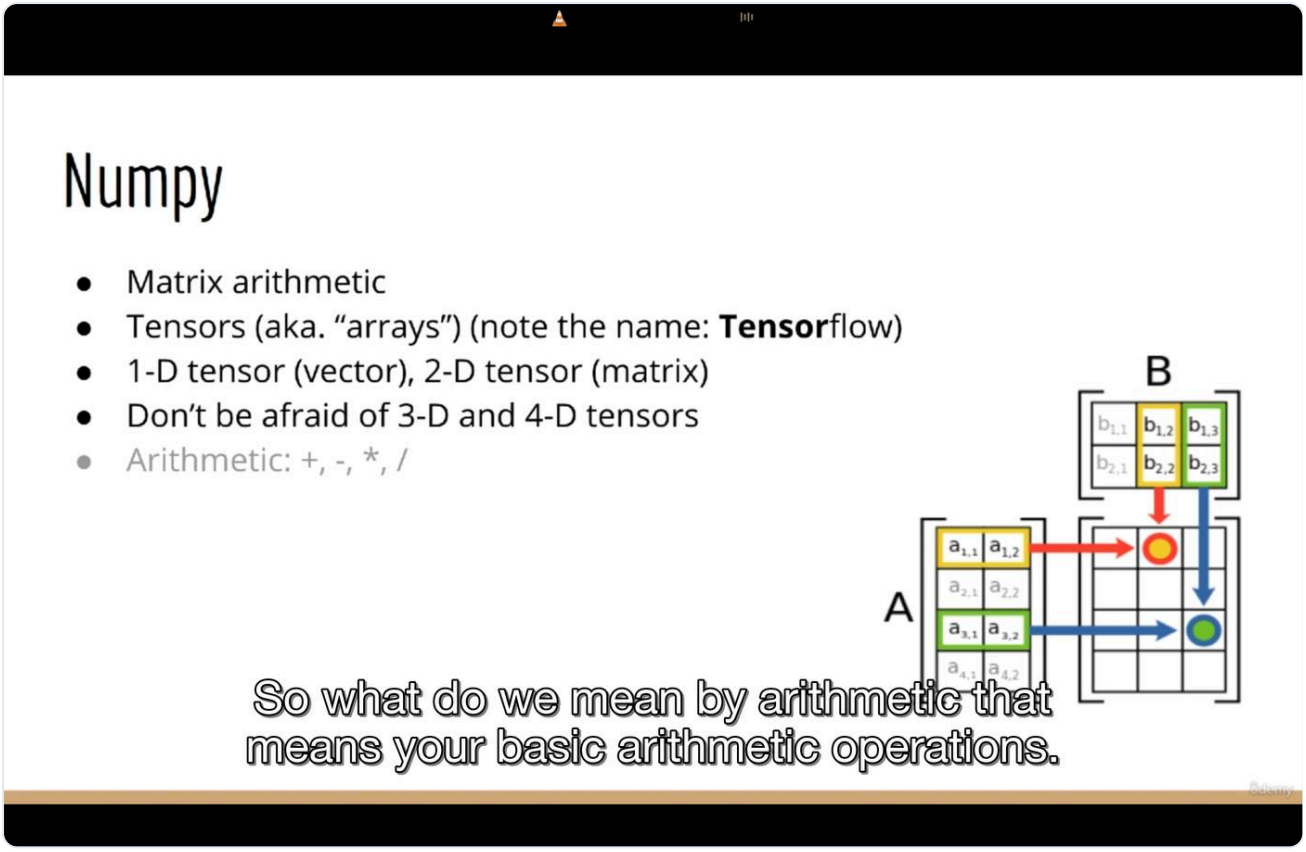


FIGURE 7.5 | Matrix Arithmetic

And if you've been paying attention the name TensorFlow actually includes the word tensor in it. So you know that it's going to come in handy tensors or if you prefer arrays can have any number of dimensions we have special names for one dimensional tensors and two dimensional tensors which we call vectors and matrices respectively. But you should not be afraid if you see a 3D tensor or a 4D tensor OK. So what do we mean by arithmetic that means your basic arithmetic operations.

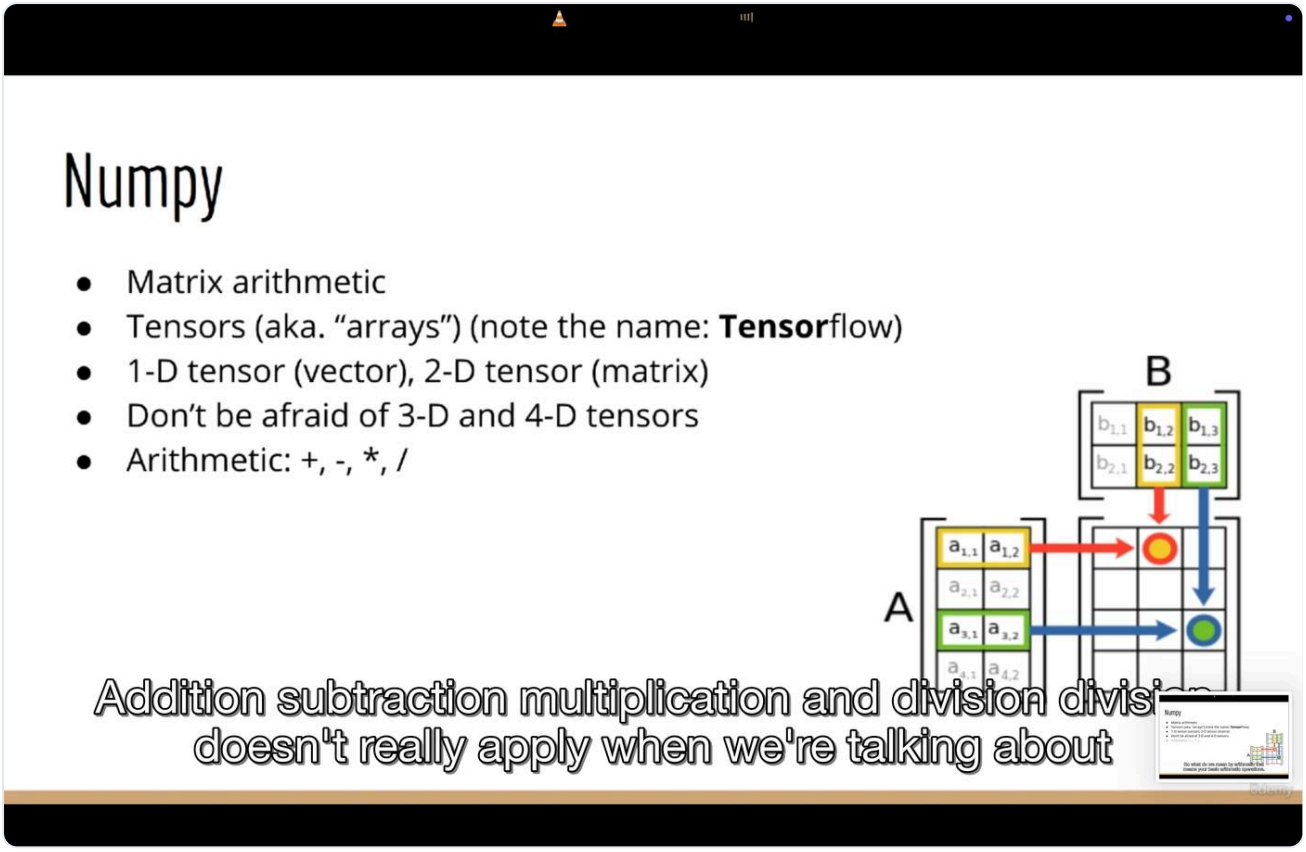


FIGURE 7.6 | Matrix Arithmetic

Addition subtraction multiplication and division division doesn't really apply when we're talking about matrices but there is an analogous concept of Matrix inversion which you should be familiar with. You should know when addition subtraction and multiplication are allowed. For example you should know that you can't add matrices of different sizes multiplication is the key operation however you should know that matrix multiplication is really just the dot product in disguise. In fact whether you're doing a dot product between vectors or matrix multiplication between matrices.

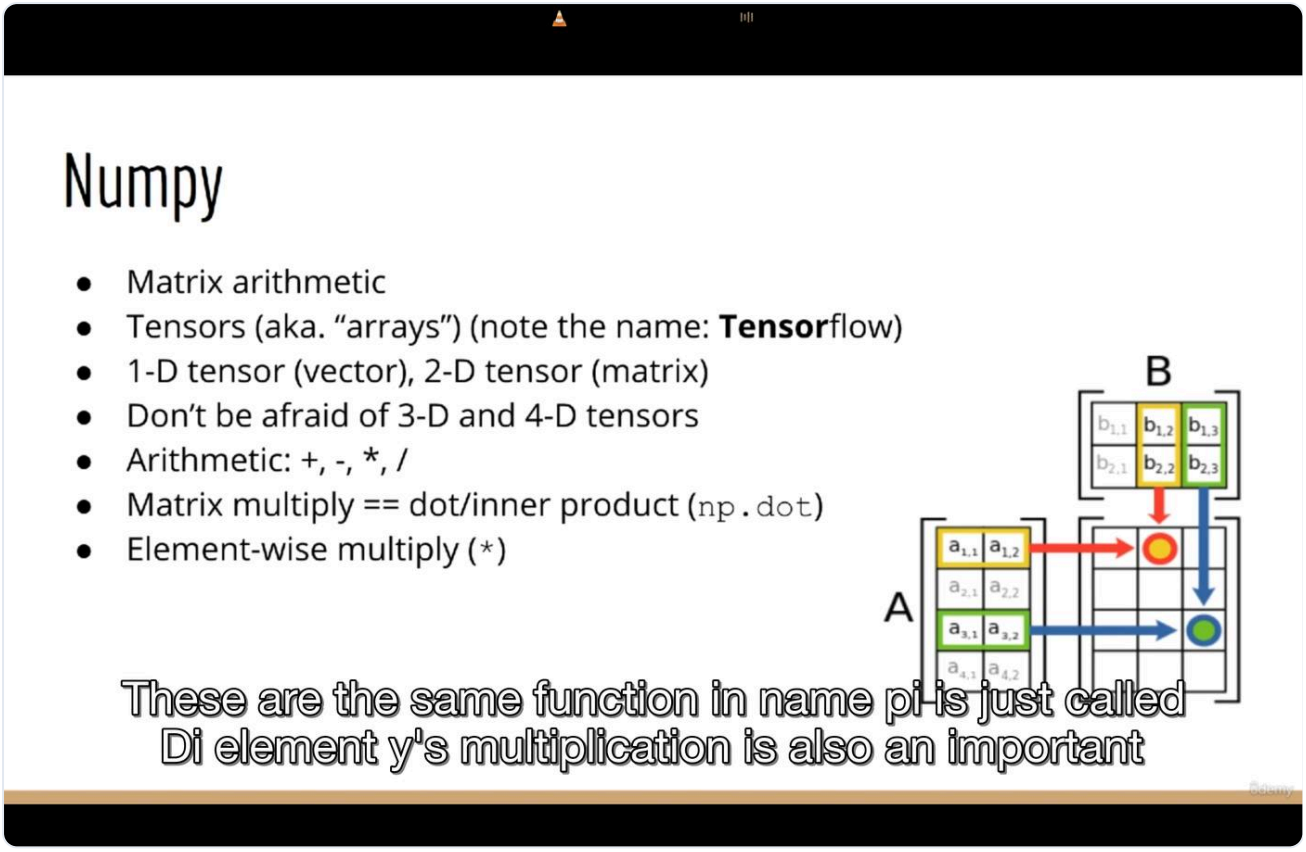


FIGURE 7.7 | Matrix Arithmetic

These are the same function in NumPy it's just called dot. Element wise multiplication is also an important operation and deep learning so you should know how to do that as well in terms of matplotlib.

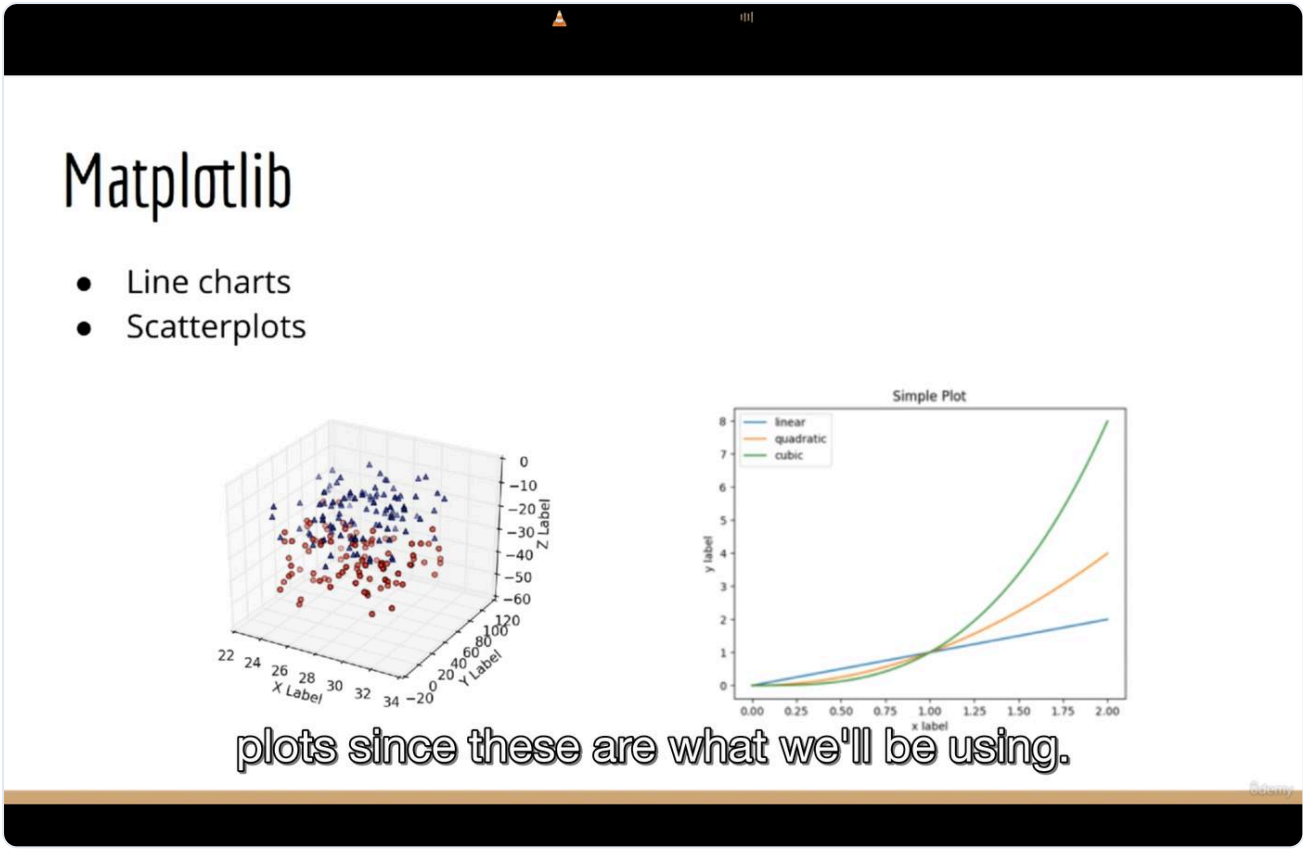


FIGURE 7.8

There isn't really anything fancy you need to know how to do just know about line charts and scatter plots since these are what we'll be using. Ninety nine percent of the time in terms of pandas. Also pretty simple.

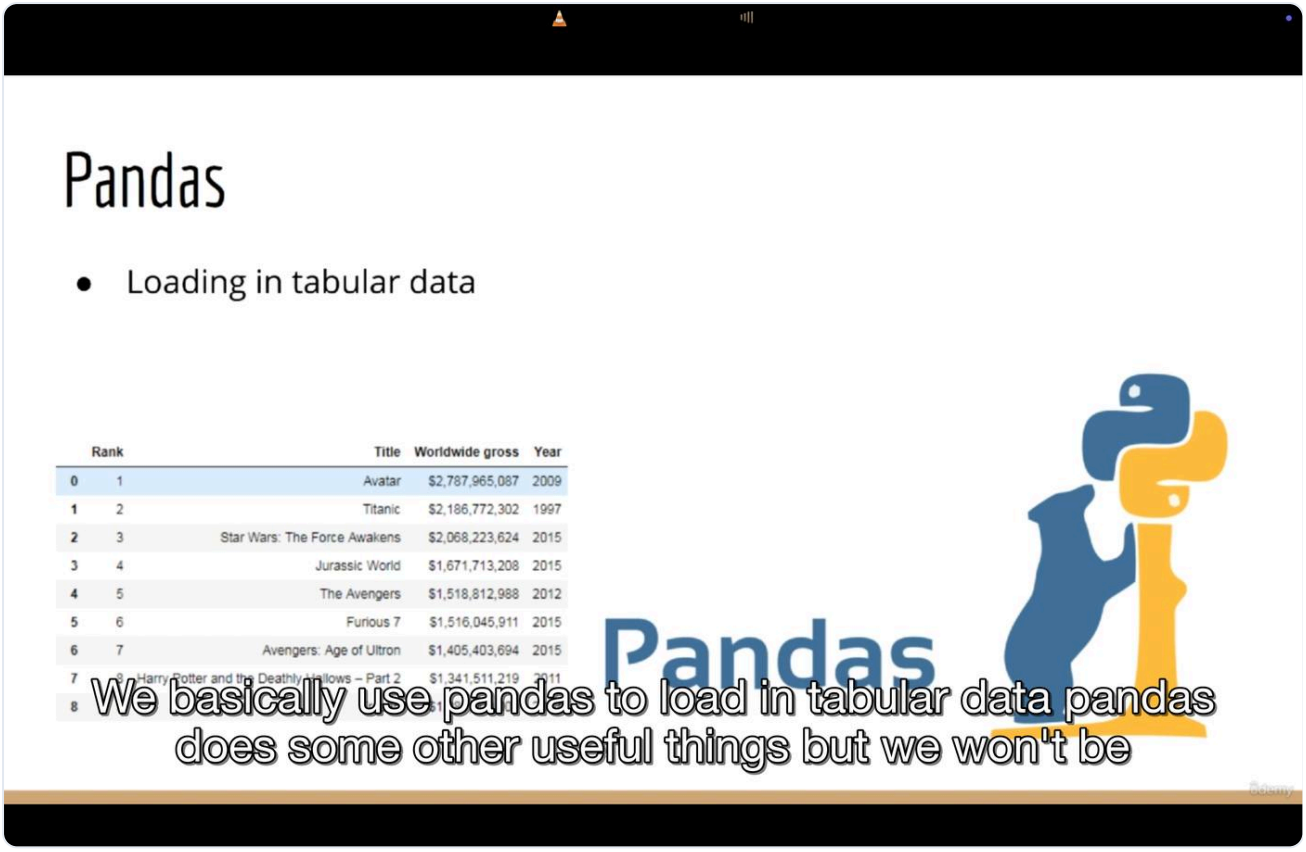


FIGURE 7.9

We basically use pandas to load in tabular data pandas does some other useful things but we won't be making full use of its capabilities. Most of the time it'll just be to loading CSV file. Next we have SciPy. Now I don't anticipate really using SciPy at all. However that could change.

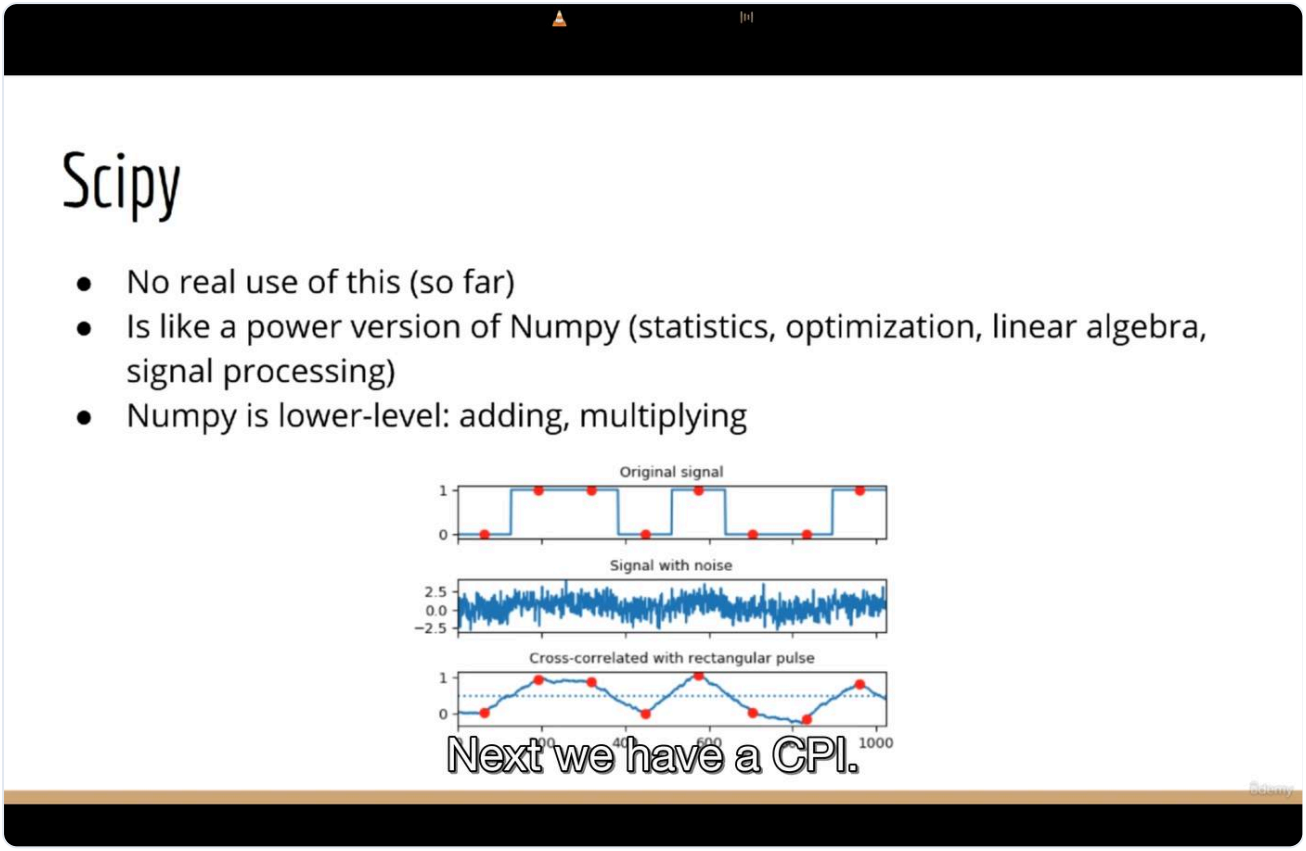


FIGURE 7.10 | No Real Use of This (So Far)

SciPy is kind of like a power version of NumPy so it has more useful statistics functions more useful optimization functions more useful linear algebra functions and so forth. NumPy is more for those lower level operations like adding and multiplying finally we have scikit-learn now at the point of knowing about scikit-learn isn't to know about scikit-learn per say.

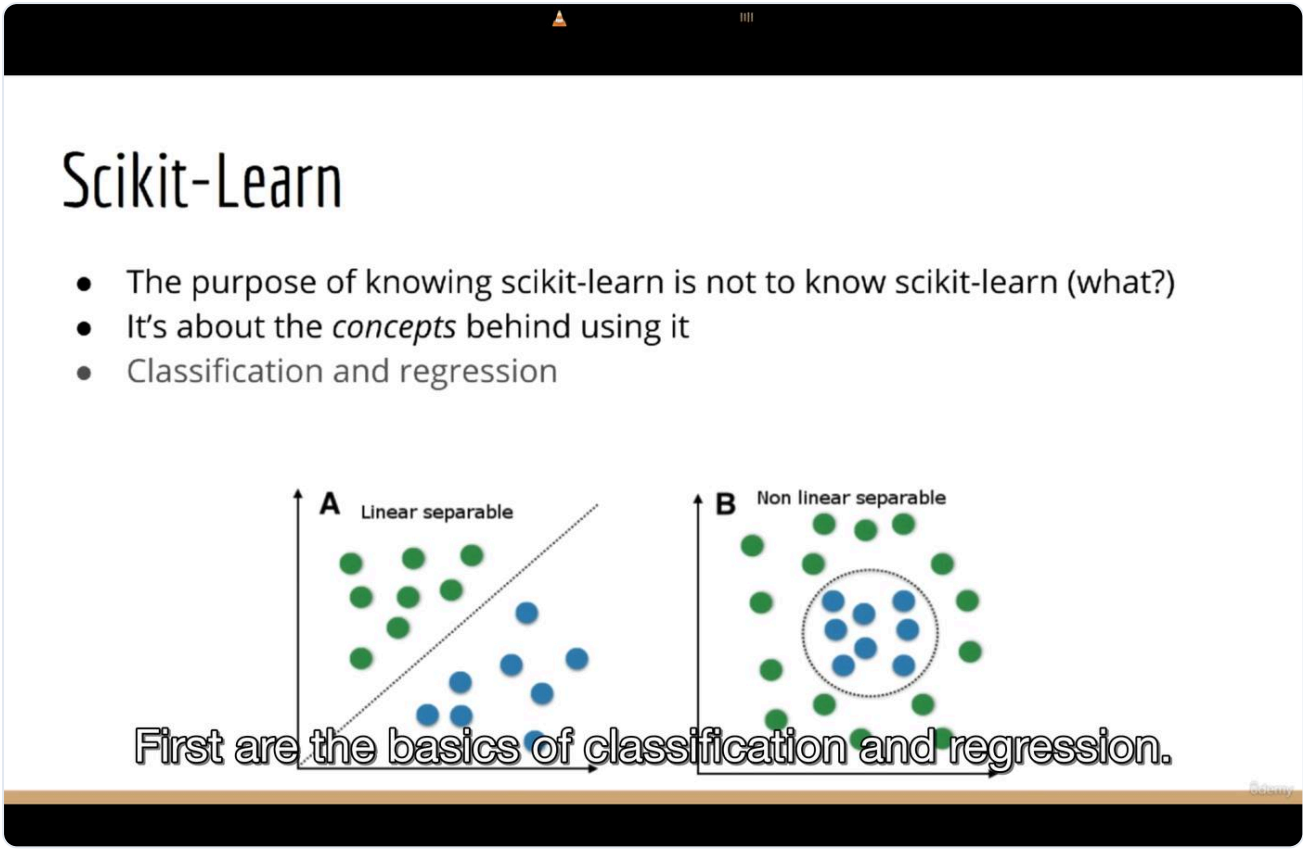


FIGURE 7.11

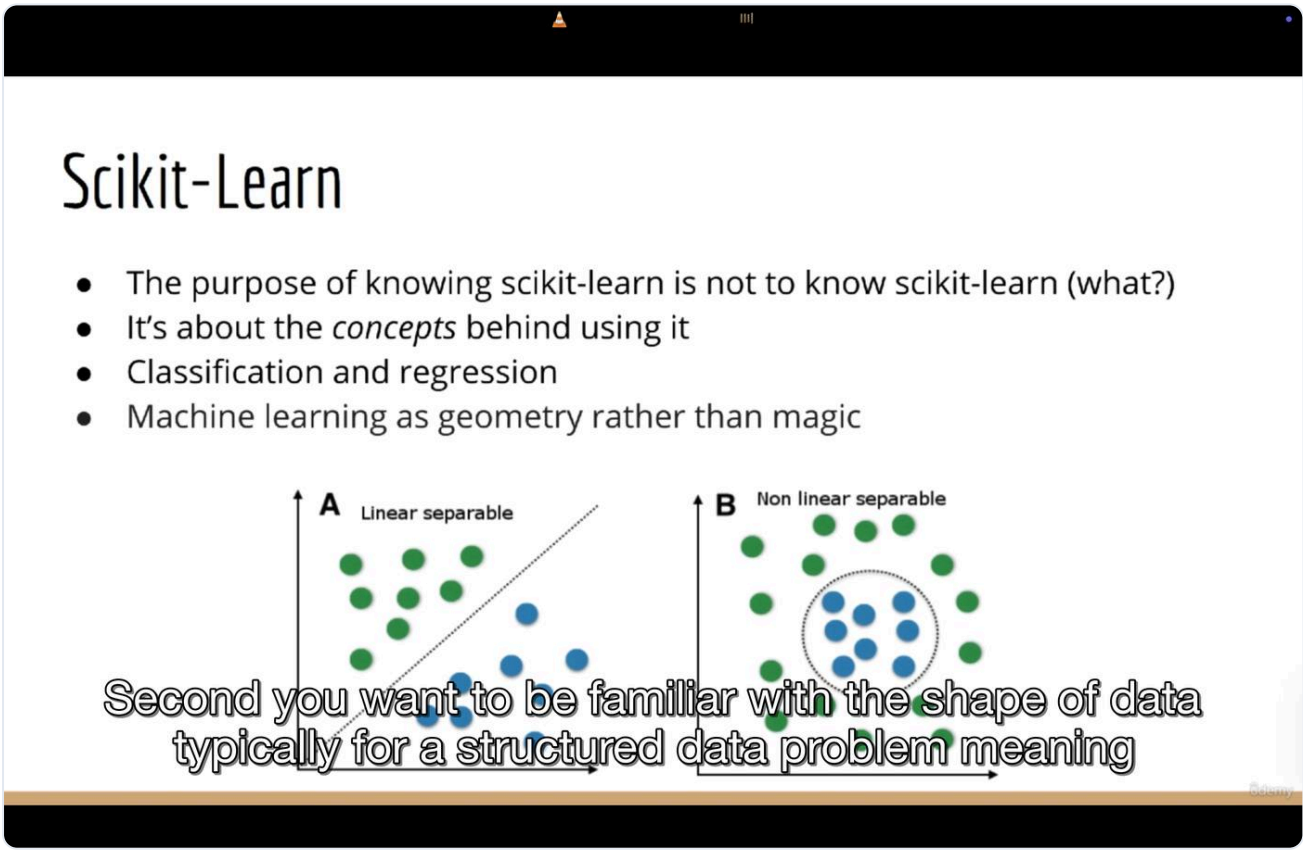


FIGURE 7.12 | It's About the Concepts Behind Using It

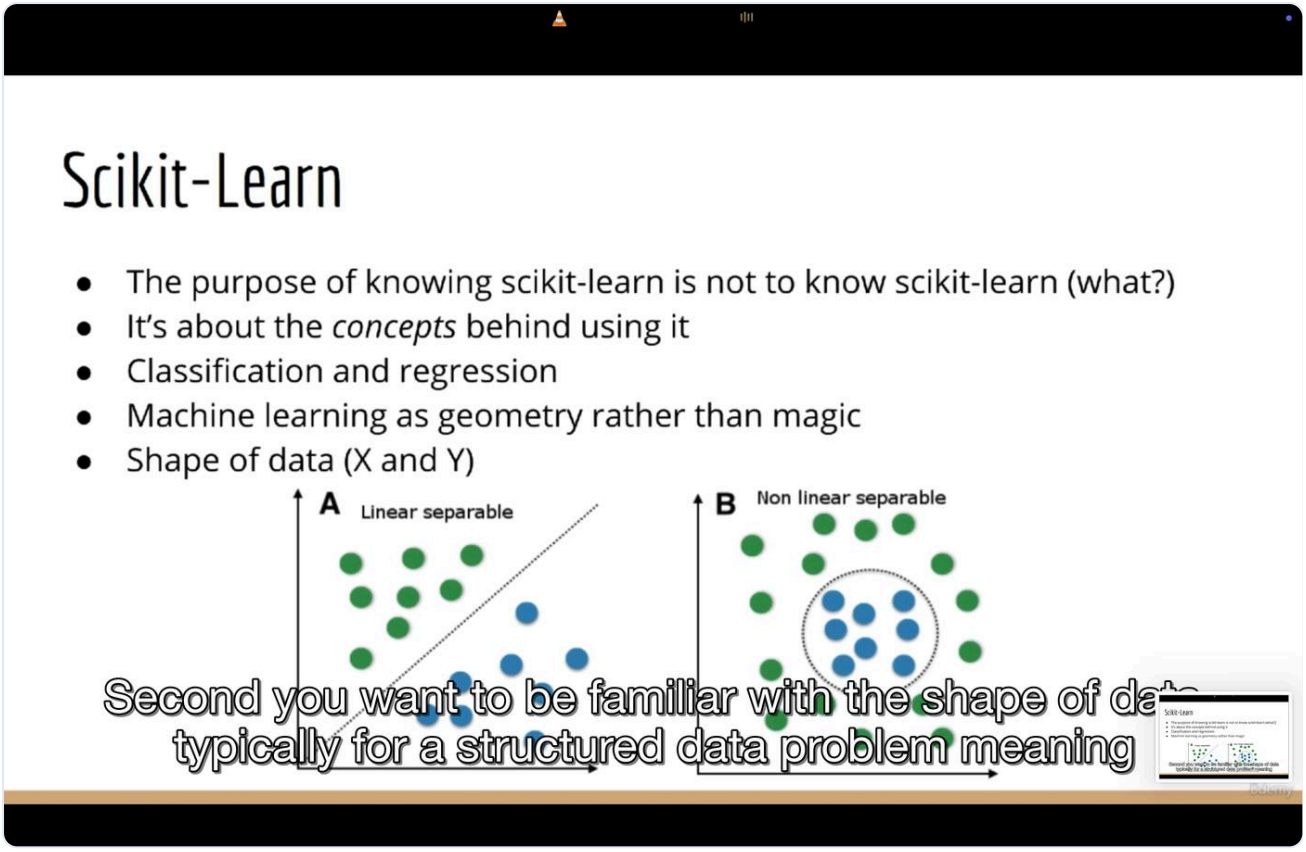


FIGURE 7.13 | It's About the Concepts Behind Using It

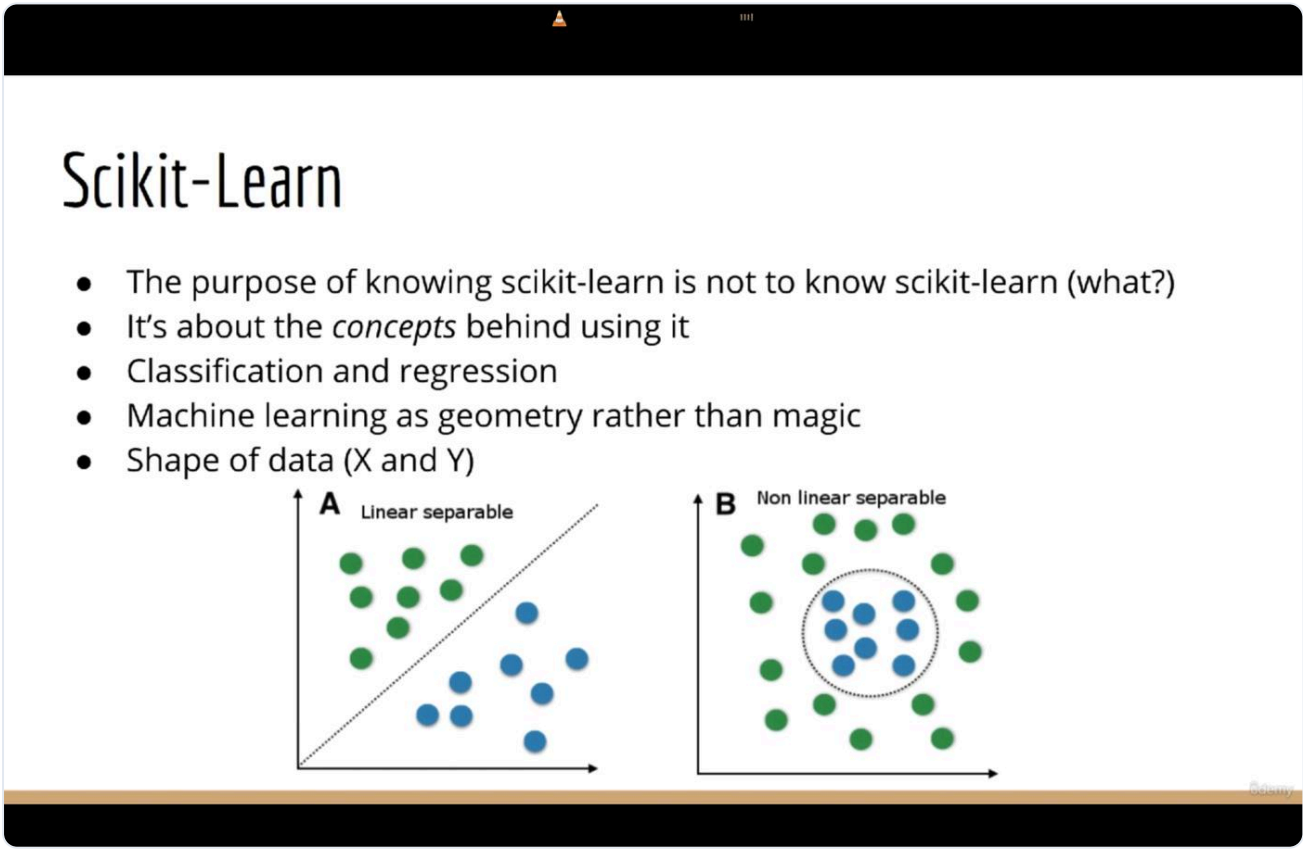


FIGURE 7.14 | It's About the Concepts Behind Using It

It's to know about the concepts behind using scikit-learn. In other words you're going to have to learn about the basics of machine learning in order to make proper use of scikit-learn. Again I re-iterate I teach this for free so I don't anticipate any problems catching up. So here are the things you want to remember and keep in mind. First are the basics of classification and regression. You want to recall how to think of these as geometry problems rather than magic machine learning voodoo. Second you want to be familiar with the shape of data typically for a structured data problem meaning not working with images sound or text but more traditional business type data.

We're going to have two objects which we call x and y x represents our input samples and Y represents the corresponding targets to give you a simple example suppose we want to predict whether or not a student will pass a math exam in order to build a model.

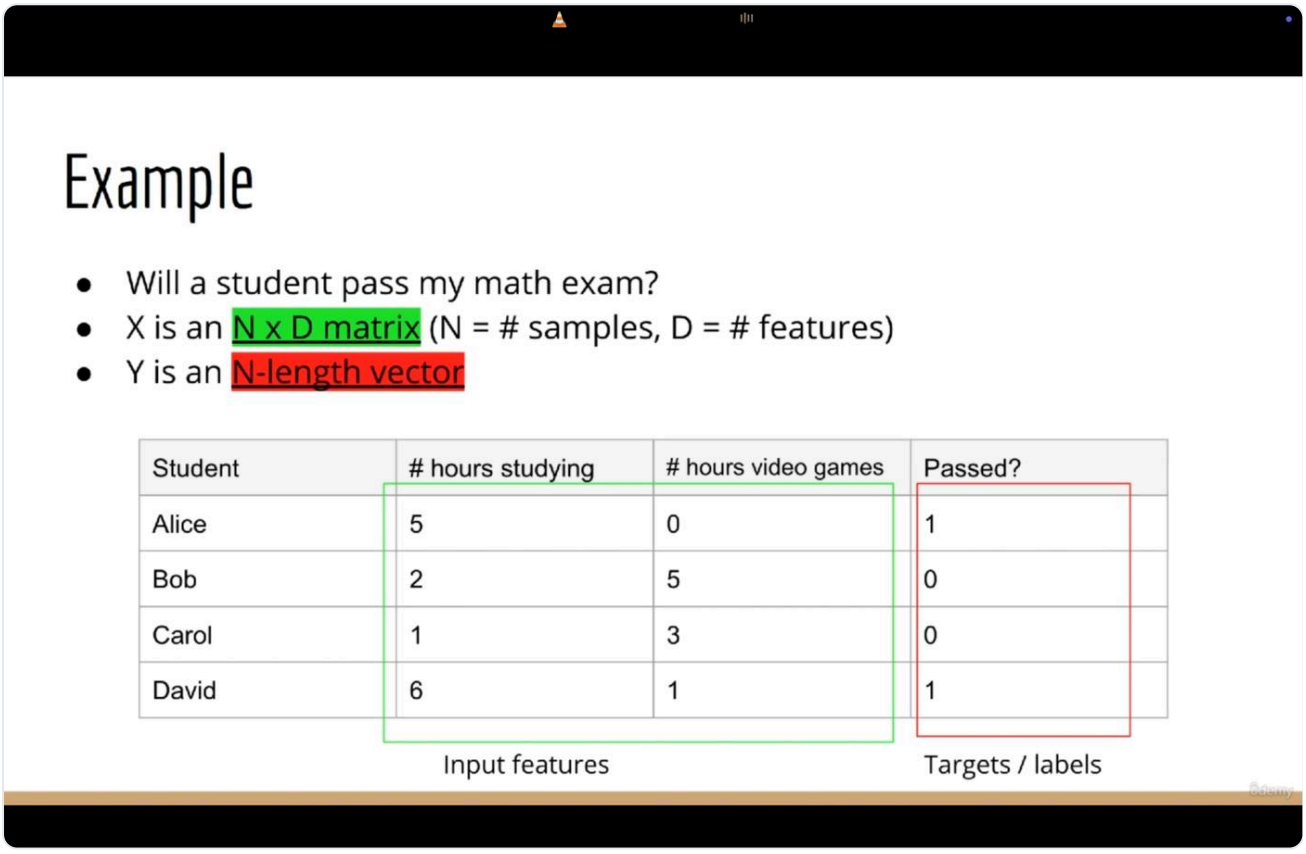


FIGURE 7.15

We've collected a bunch of data points for students who took this math exam in the past. We recorded several variables such as how many hours they spent studying and how many hours they spent playing video games. We would treat these as our X matrix our y matrix would be the target or label which is what we want to predict in the future for future students. So in the future we could ask a student Hey how many hours do you study. How many hours did you play video games. And then from that information alone we could make a prediction about whether or not they will pass the math exam. Why would be whether or not the student pastor failed and coded as 0 or 1.

In general we say that the X matrix is of size and by D.

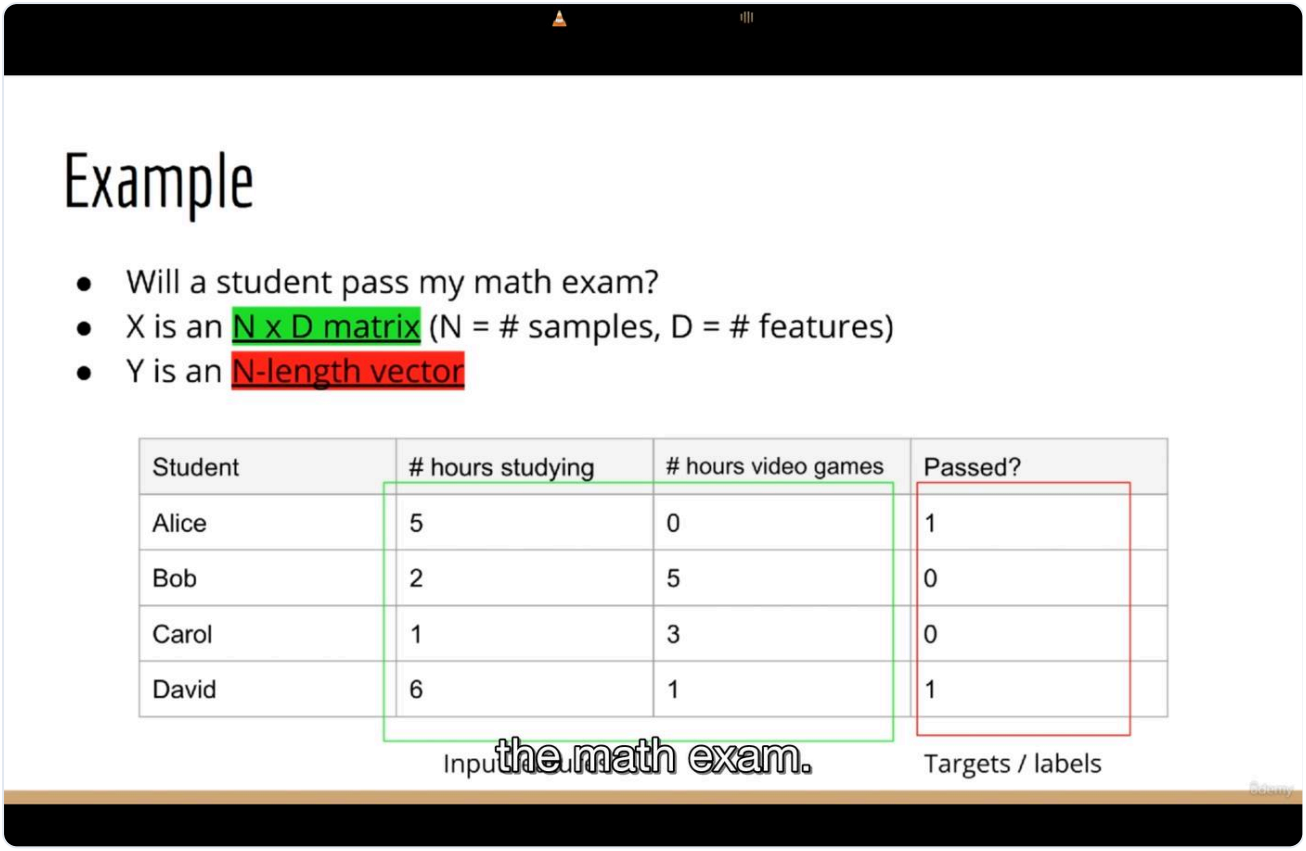


FIGURE 7.16

Where n is the number of samples. And D the number of features Y is just a one dimensional array of length n which you can also think of as a vector. You also want to know the steps we take in general when we are writing a machine learning script to test that a model we've just learned about. This is going to be pretty much the same every time. So if you don't know this you're going to be extremely confused. The steps are as follows. Number one we load in the data.

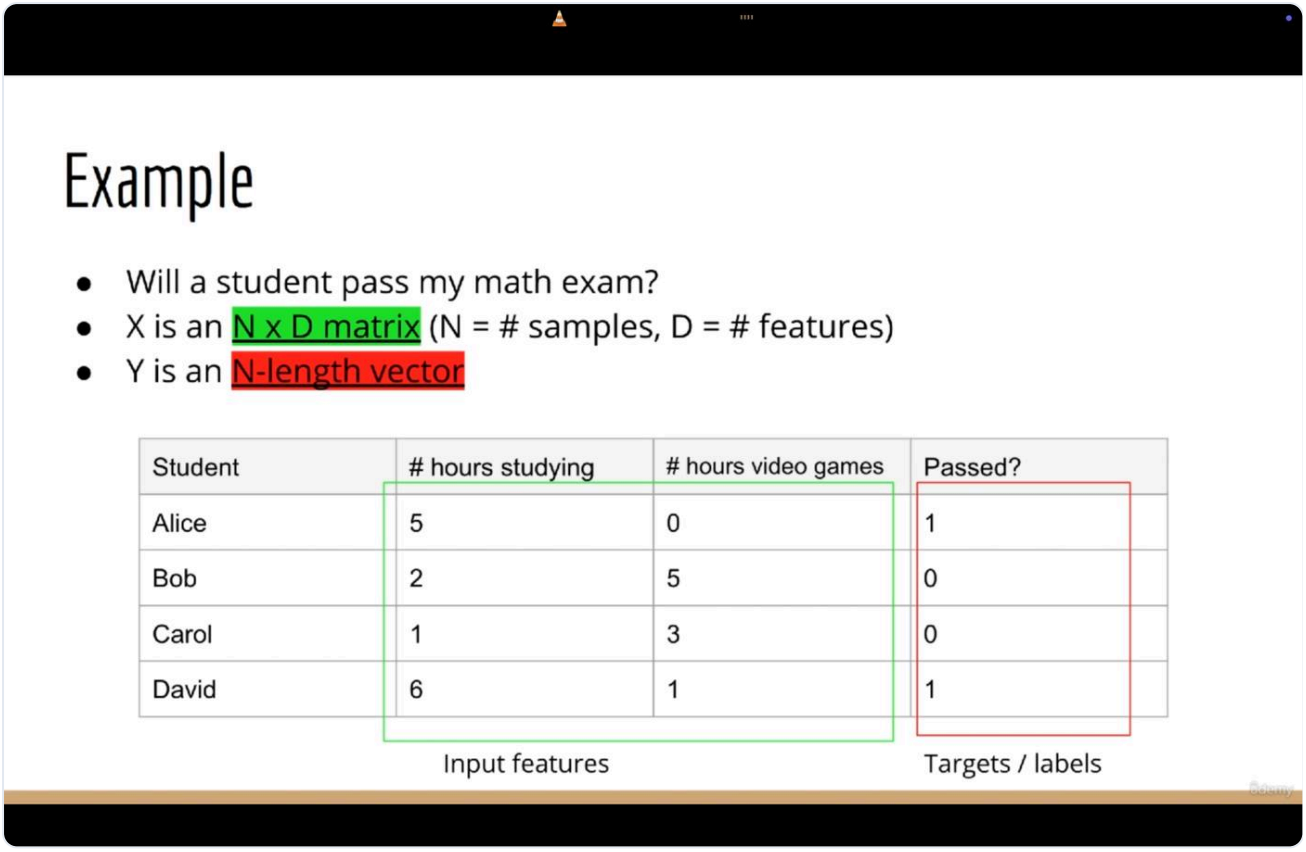


FIGURE 7.17

That's just the x and y we just talked about. We will use pandas for this when we have simple data sets or if we have something more complicated we'll write our own Python code number two we can optionally split the data into train and test sets depending on what we are doing. Note that sometimes the term test set is used interchangeably with validation set in which case the true test set would mean something else. Don't get too fussy over this. It is what it is. Number three we build a model now in Python. We usually use object oriented programming which means our model takes the form of a Python object in PyTorch we still use object oriented programming.

In fact probably more so than what you're used to.

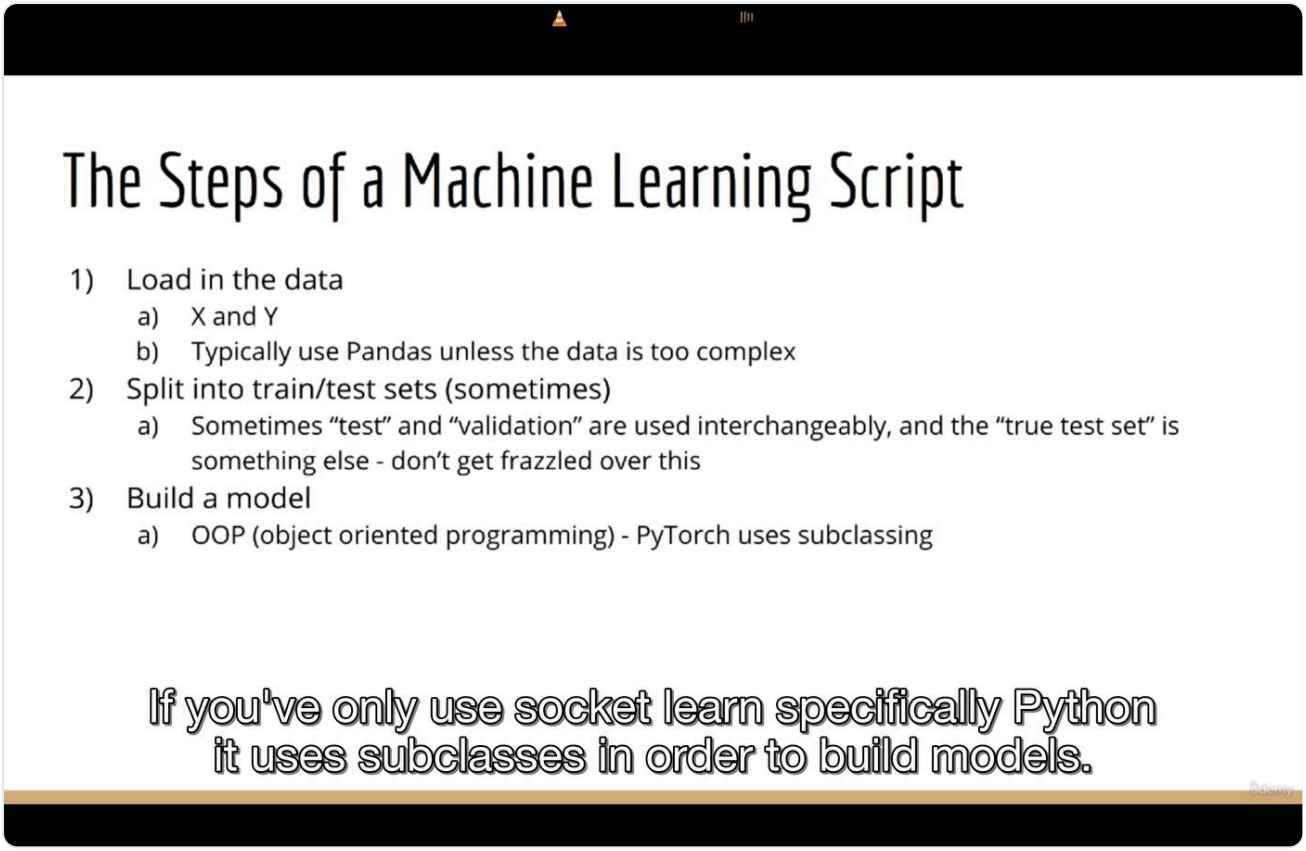


FIGURE 7.18 | The Steps of a Machine Learning Script

If you've only used scikit-learn specifically PyTorch it uses subclasses in order to build models. These models do not work like what you are used to. in scikit-learn but it's similar.

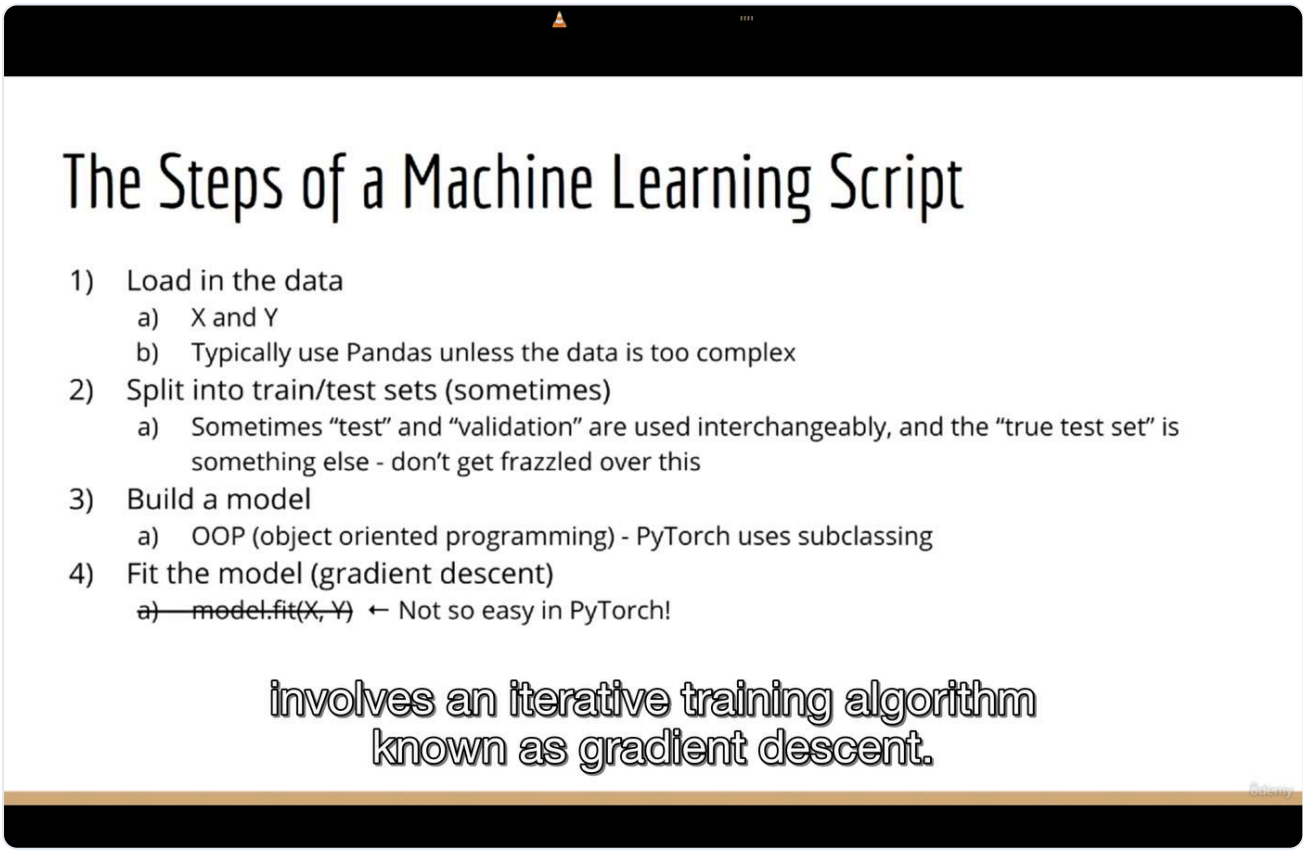


FIGURE 7.19 | The Steps of a Machine Learning Script

Ultimately you should still recognize the following steps number four. We fit the model in scikit-learn. This just means calling the fit function on the model and passing in your training data in deep learning this is a little more complicated because we have many more options since training in deep learning involves an iterative training algorithm known as gradient descent.

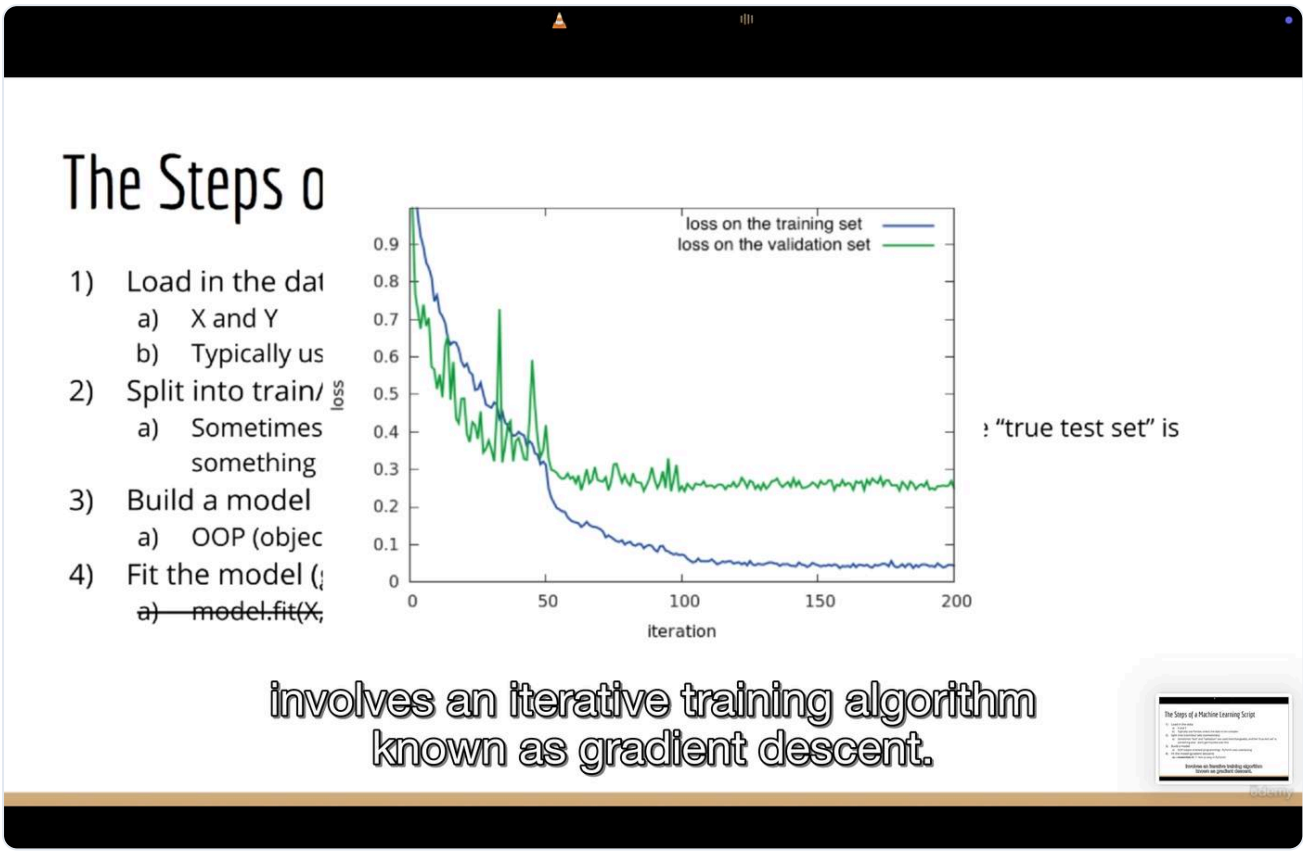


FIGURE 7.20

We typically plot a loss per iteration when we are finished training to confirm that our training process has converged as expected. If it didn't we have to go back and make changes.

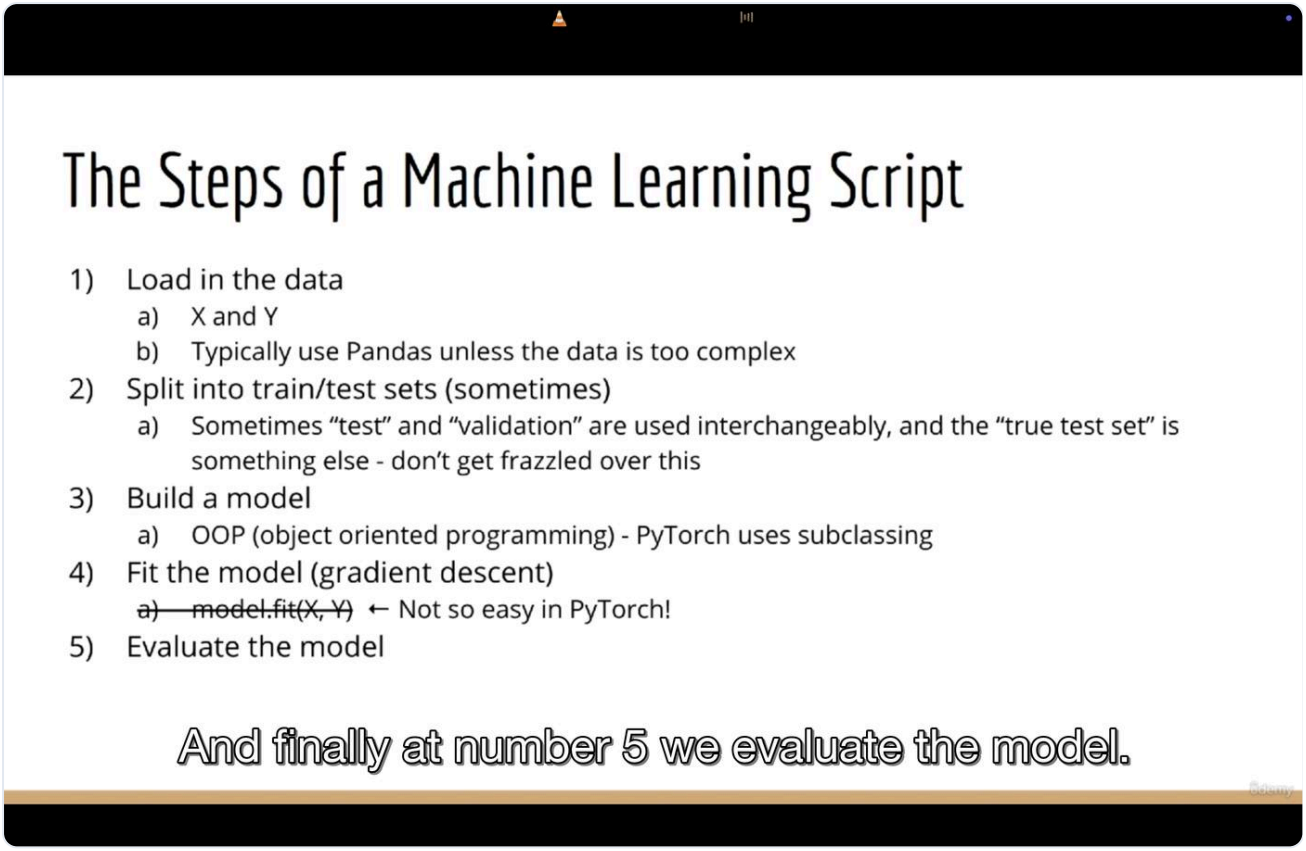


FIGURE 7.21 | The Steps of a Machine Learning Script

And finally at number 5 we evaluate the model. This involves checking the accuracy of our model on both the train and validation sets. At this time we can also check to see if the model is over fitting or under fitting and once this is all done we can make predictions with our model in PyTorch. This is a little more complicated than in scikit-learn because PyTorch doesn't really work with numpy arrays.

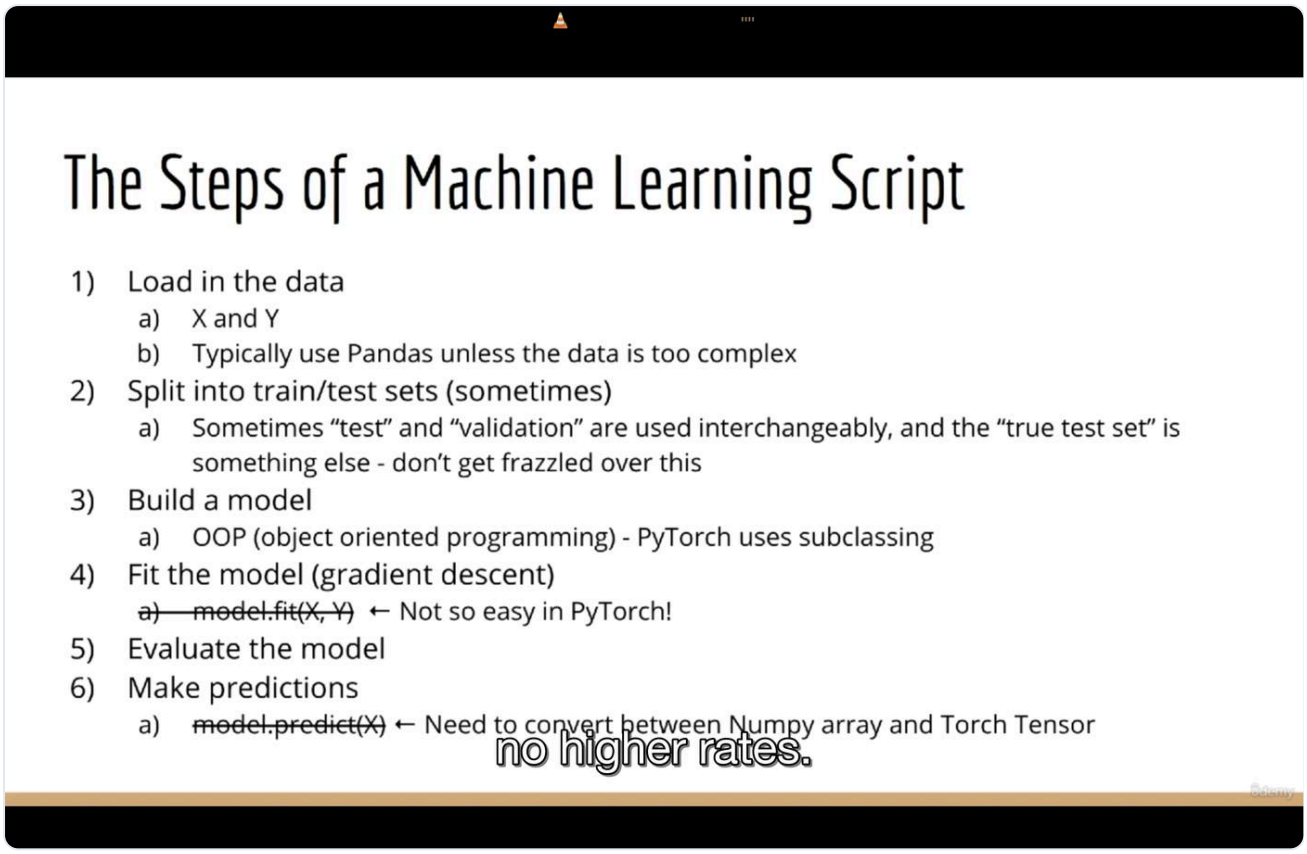


FIGURE 7.22 | The Steps of a Machine Learning Script

Instead PyTorch works with torch tensors so there will be a lot of times where we have to convert back and forth between a numpy array and torch tensor depending on what we need at the time. This might seem strange at first but you'll get used to it as you get more practice this process encapsulates the typical lifecycle of a machine learning model you load in your data trainer model evaluate it and then use it to make future predictions.

LECTURE 8

All Data Is the Same

In this lecture we are going to answer the question what is machine learning. I always like to discuss this first because they take a very different approach compared to most other resources you'll find in general I think the approach most other resources take is that they like to frame machine learning as something complex and something fancy and something futuristic and exciting. My approach instead is rather kind of depressing how I want you to think of machine learning is that it's something not fancy at all and something that only requires spatial reasoning.

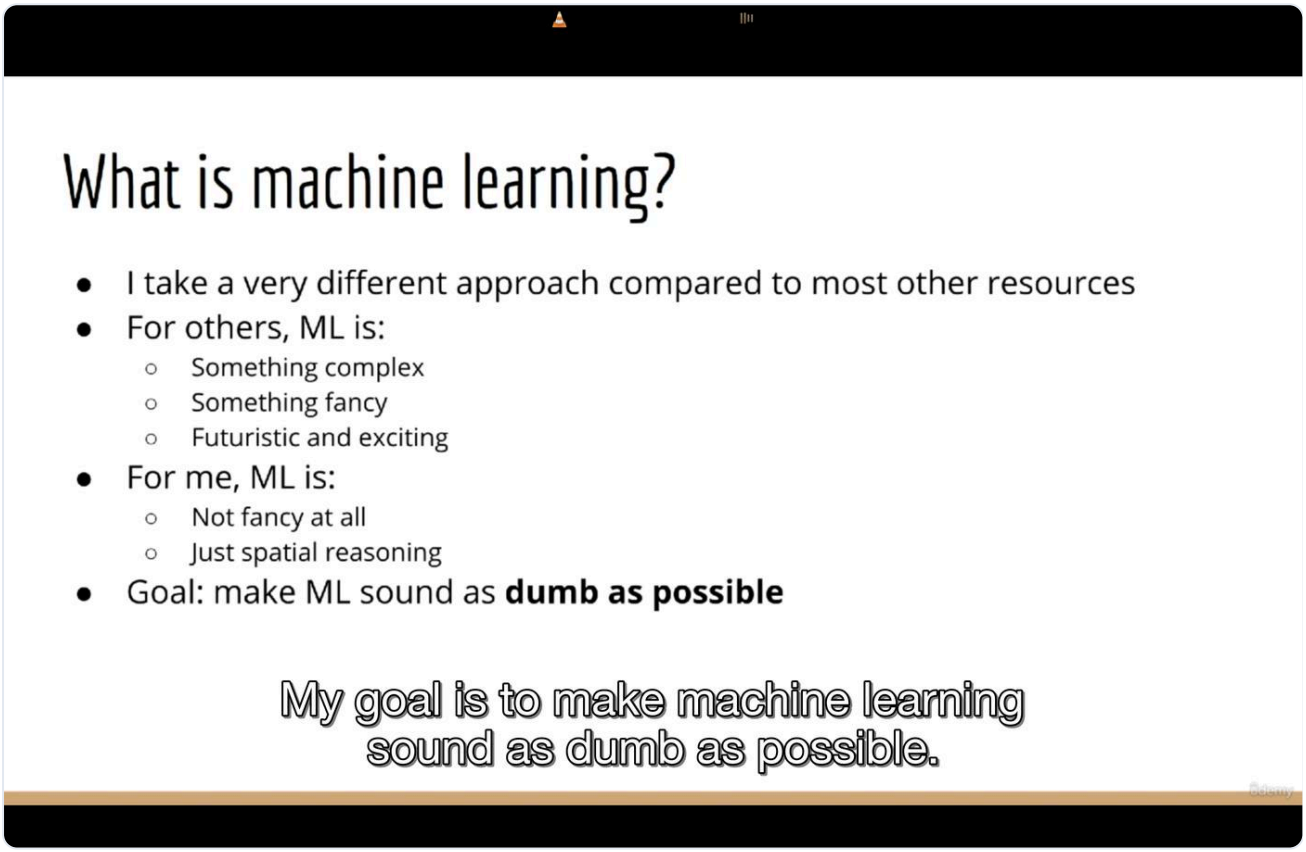


FIGURE 8.1 | What Is Machine Learning?

My goal is to make machine learning sound as dumb as possible. Here here's some anonymous definitions of machine learning that I found on the Internet. Let me read this one out to you. Machine learning is an application of artificial intelligence that provides systems the ability to automatically learn and improve from experience without being explicitly programmed. Machine learning focuses on the development of computer programs that can access data and use it to learn for themselves. The process of learning begins with observations or data such as examples direct experience or instruction.

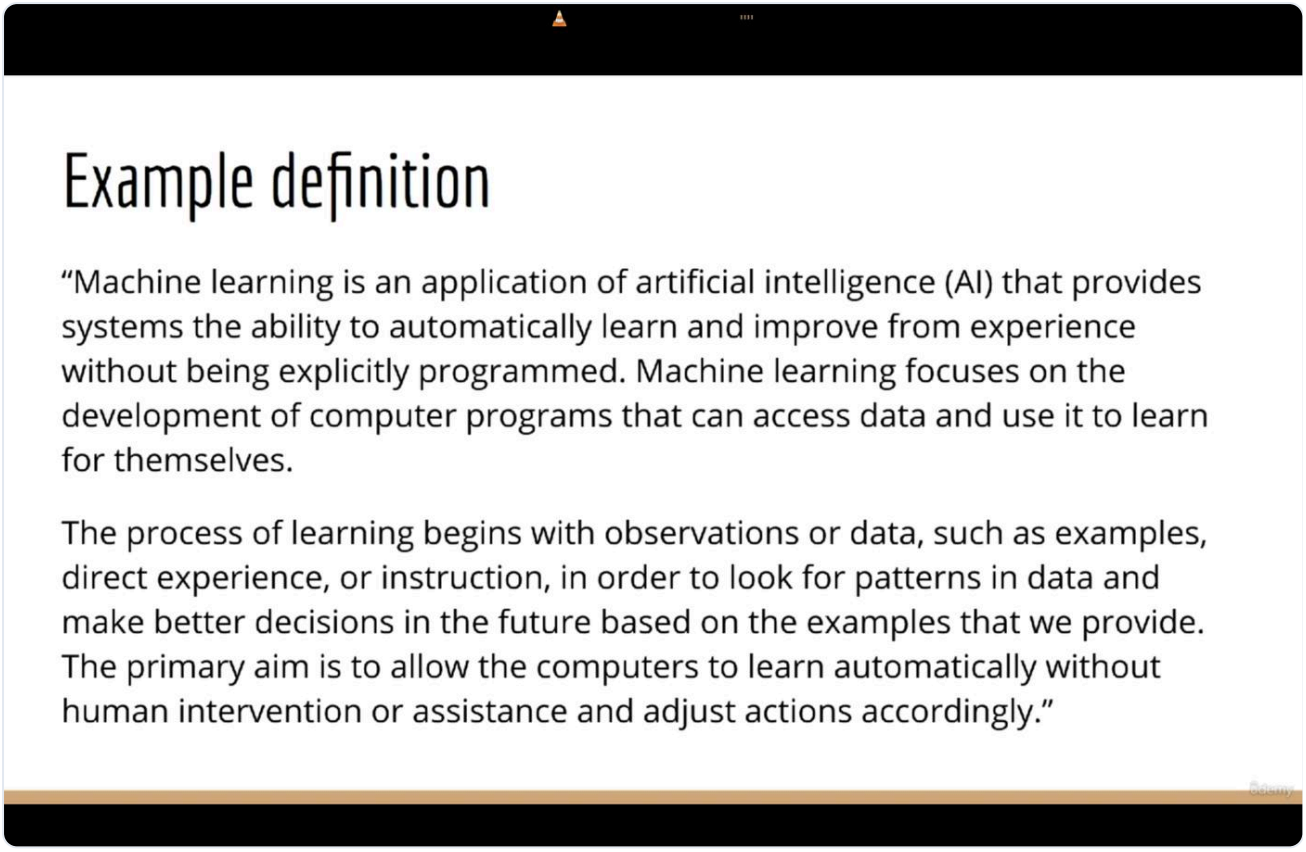


FIGURE 8.2

In order to look for patterns in data and make better decisions in the future based on the examples that we provide the primary aim is to allow the computers to learn automatically without human intervention or assistance and adjusts actions accordingly. Wow that seems pretty magical here's another one.

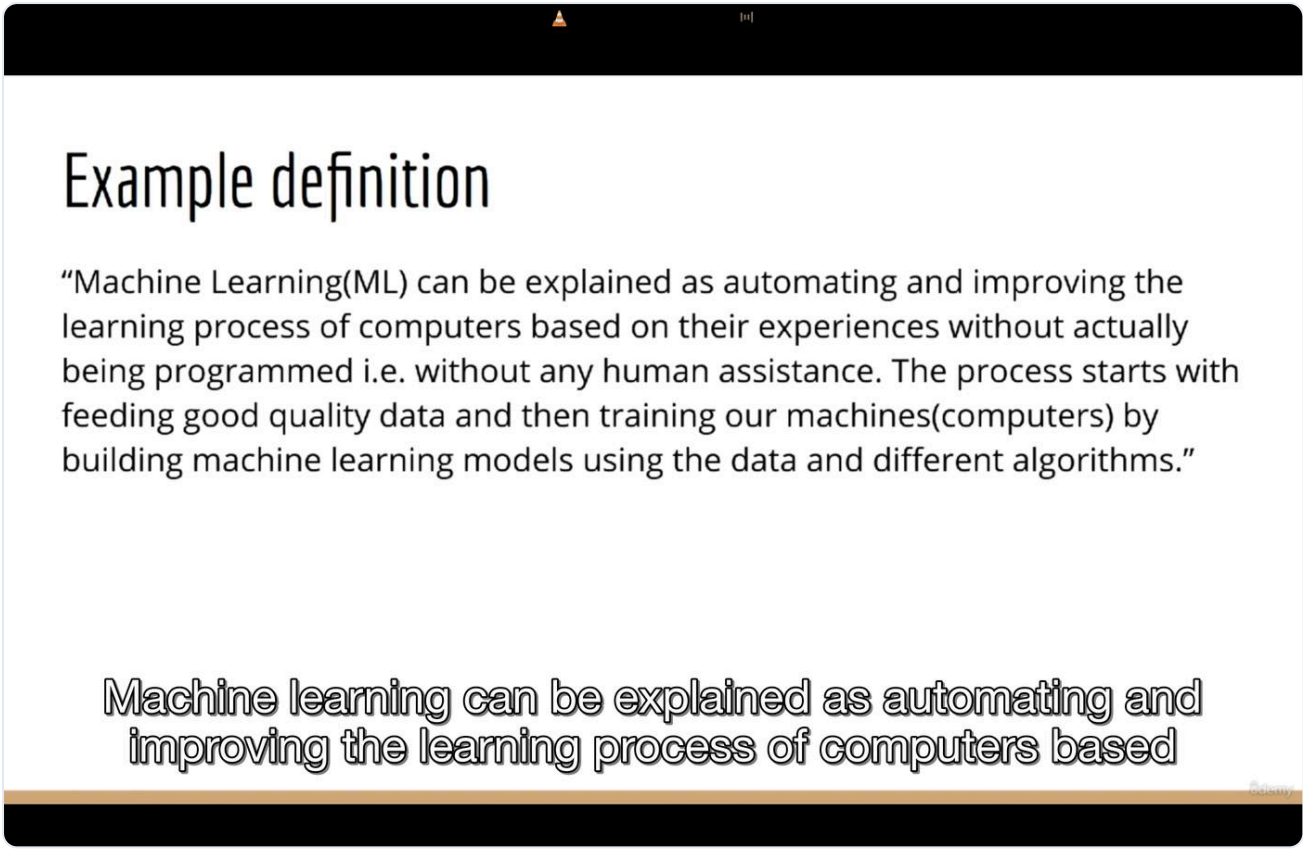


FIGURE 8.3

Machine learning can be explained as automating and improving the learning process of computers based on their experiences without actually being programmed i.e. Without any human assistance. The process starts with feeling good quality data and then training our machines computers by building machine learning models using the data and different algorithms. Again very generic but no substance. In fact these two definitions were so poorly written. I had to fix them up a little bit due to some obvious grammatical errors. I didn't edit them too heavily since I didn't want to lose the gist of what they were saying but you can tell that these were not written by people who actually know about machine learning.

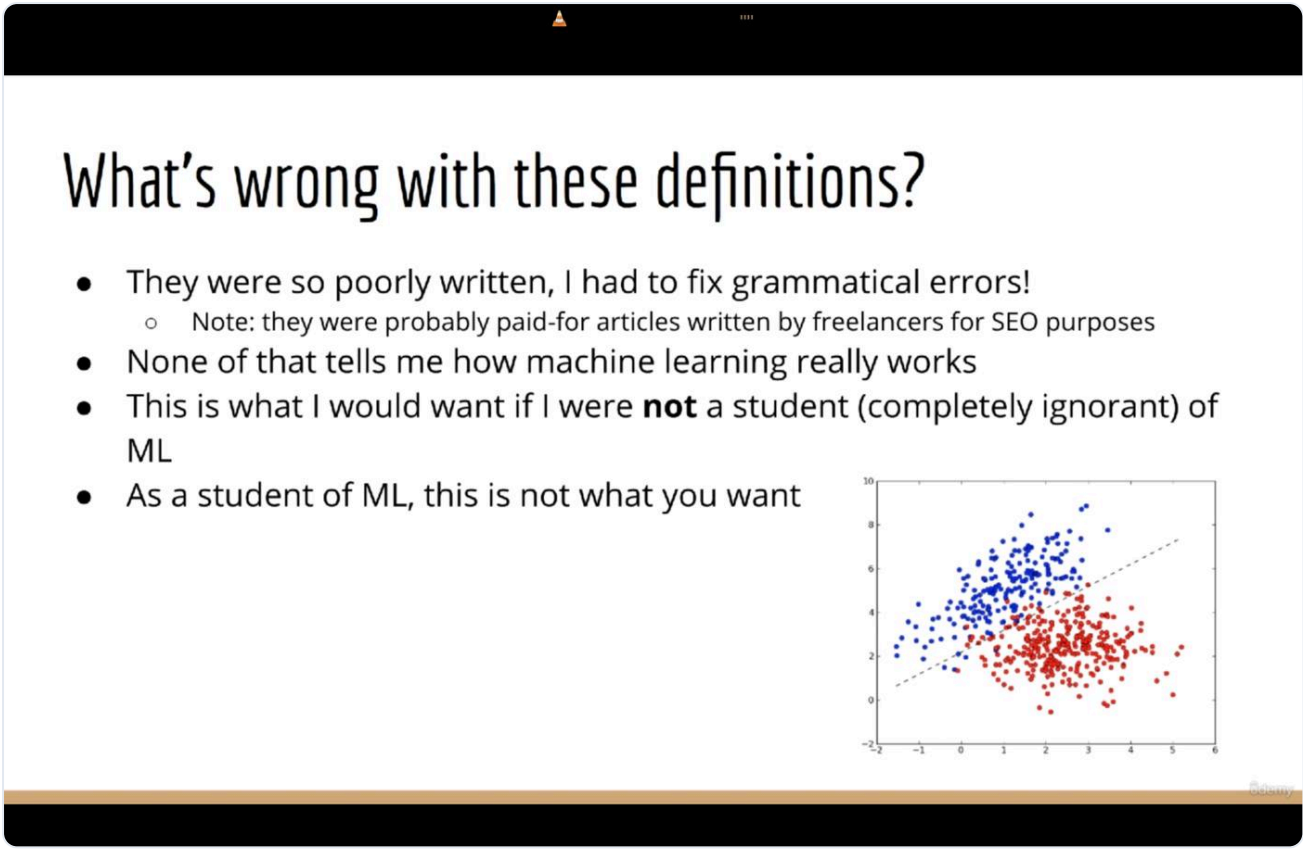


FIGURE 8.4

None of this tells me how machine learning actually works. This is what I would expect if I were not a student of machine learning. In other words just the layman and I wanted to know about how someone completely ignorant of machine learning would view it at a high level. In other words as students of machine learning this is not really how you want to think of what we are doing. So starting now you are no longer just a layman.

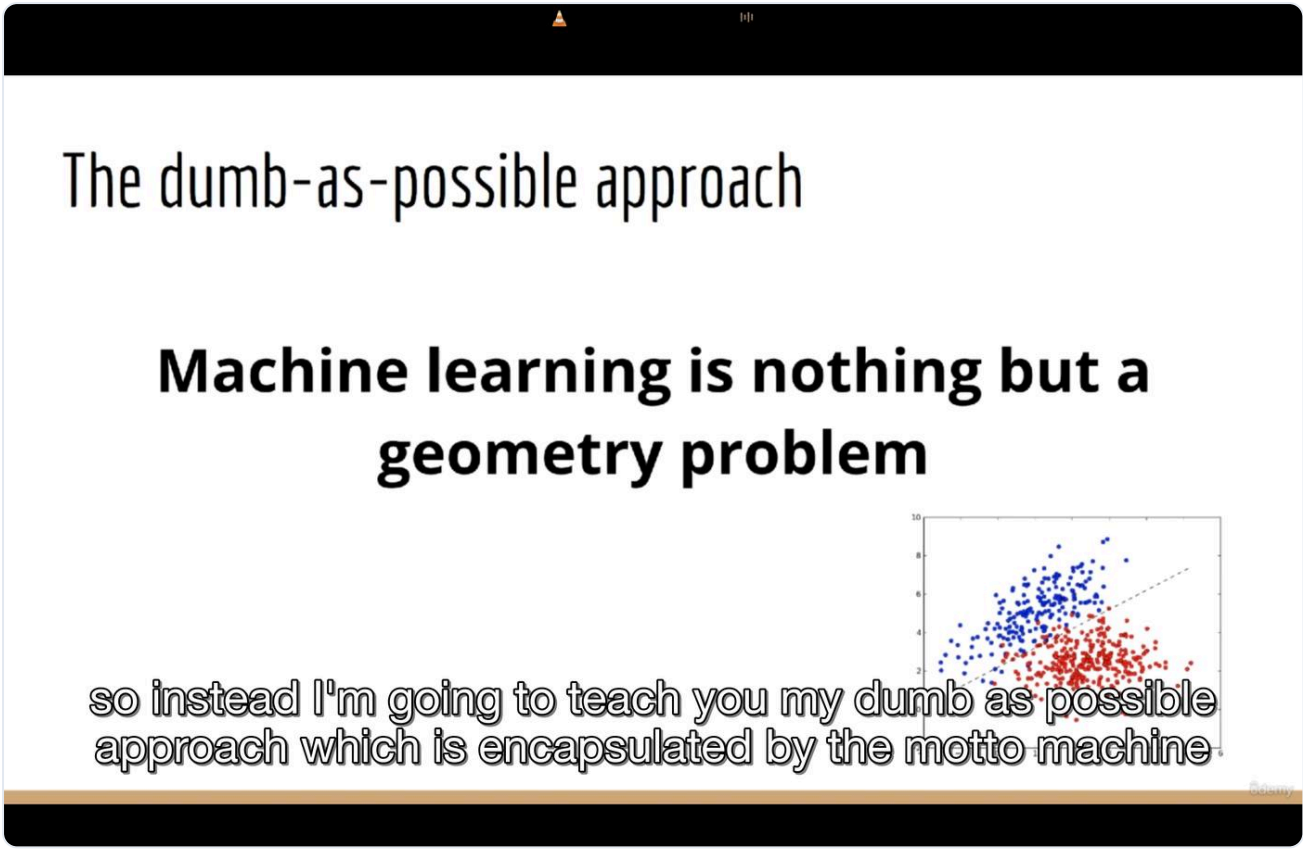


FIGURE 8.5 | Machine Learning Is Nothing but a

You are a machine learning student so instead I'm going to teach you my dumb as possible approach which is encapsulated by the motto machine learning is nothing but a geometry problem. Let's repeat that machine learning is nothing but a geometry problem.

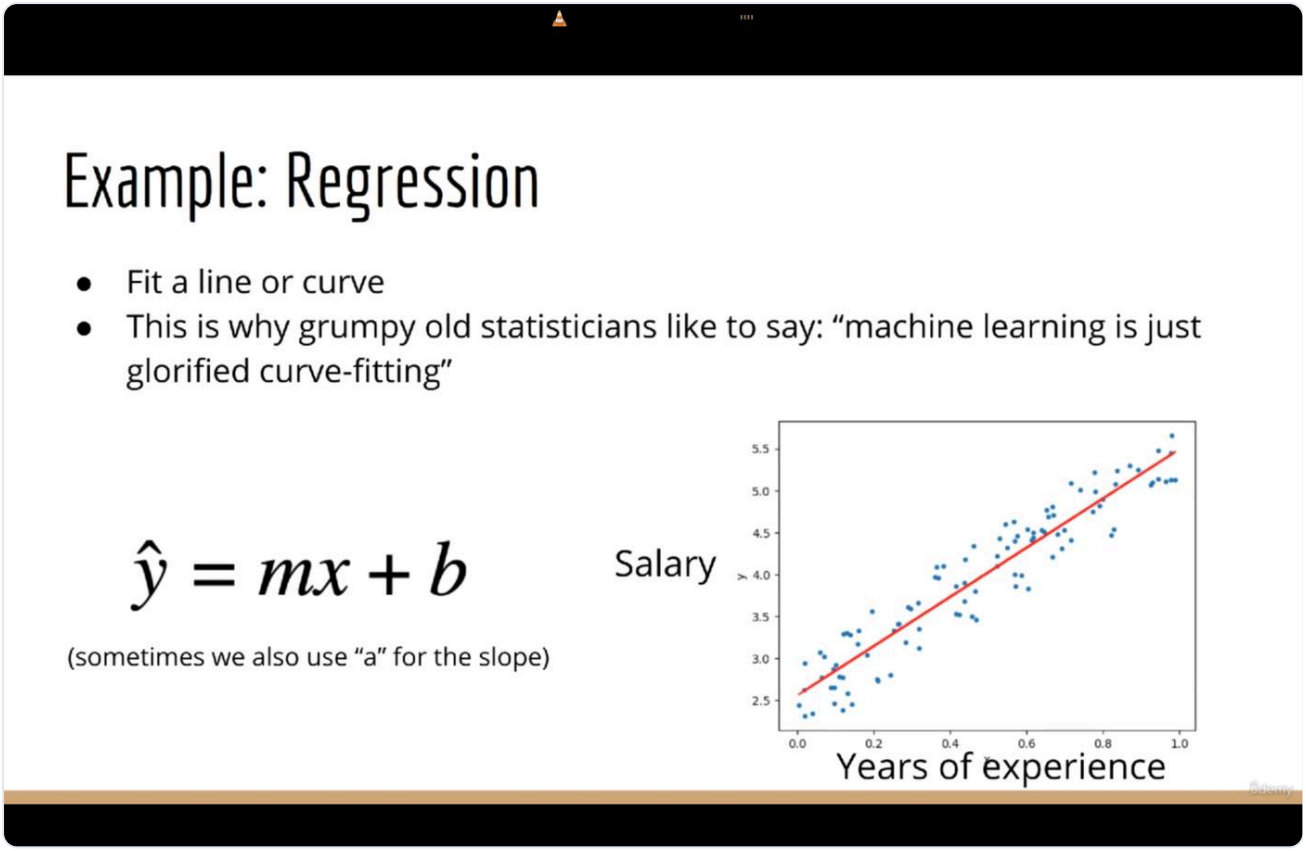


FIGURE 8.6 | Example: Regression

The best way to illustrate this is by example let's start with an example of regression regression basically means you're trying to fit a line or a curve so right away you understand that this geometry geometry is just lines planes curves circles and so forth.

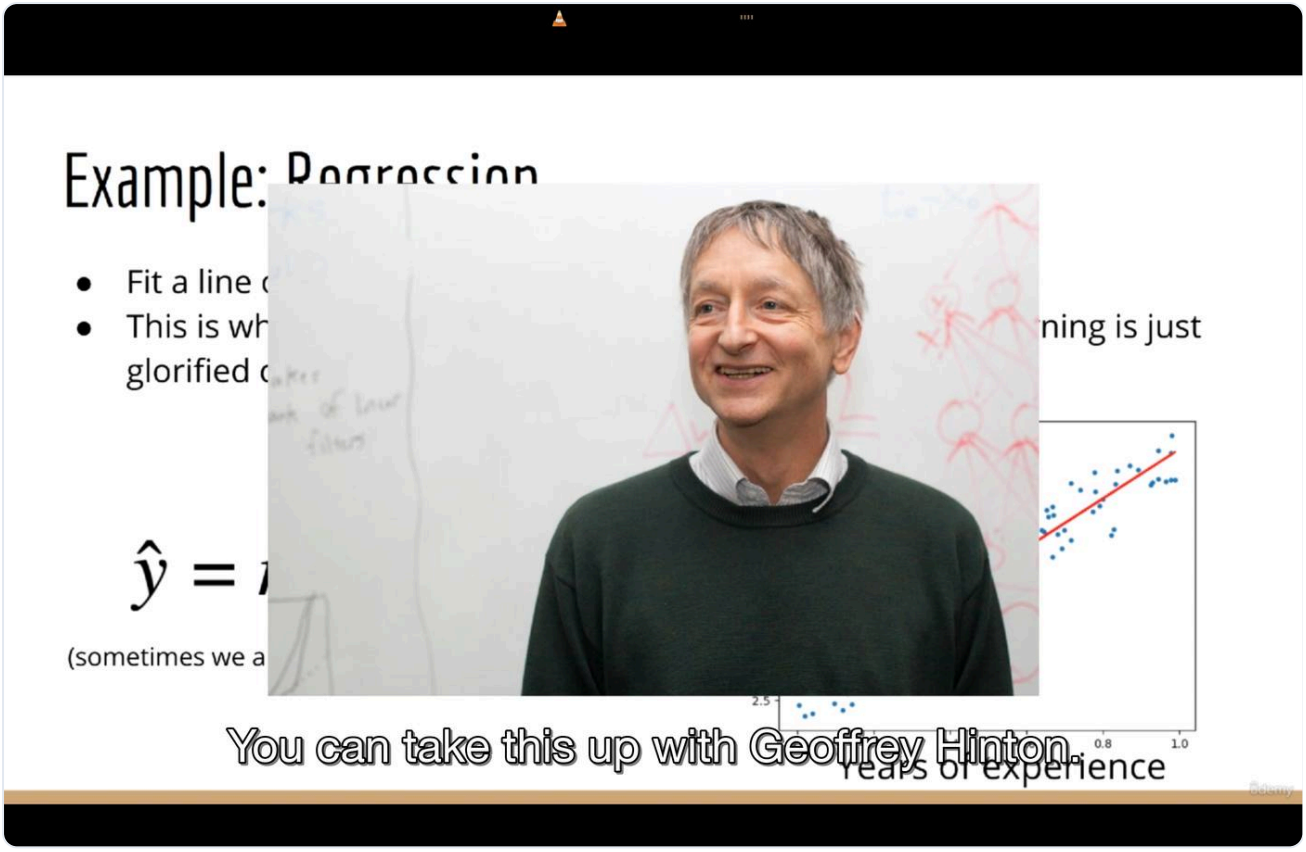


FIGURE 8.7

That's why you'll sometimes hear grumpy old statisticians say things like machine learning is nothing but glorified curve fitting by the way.

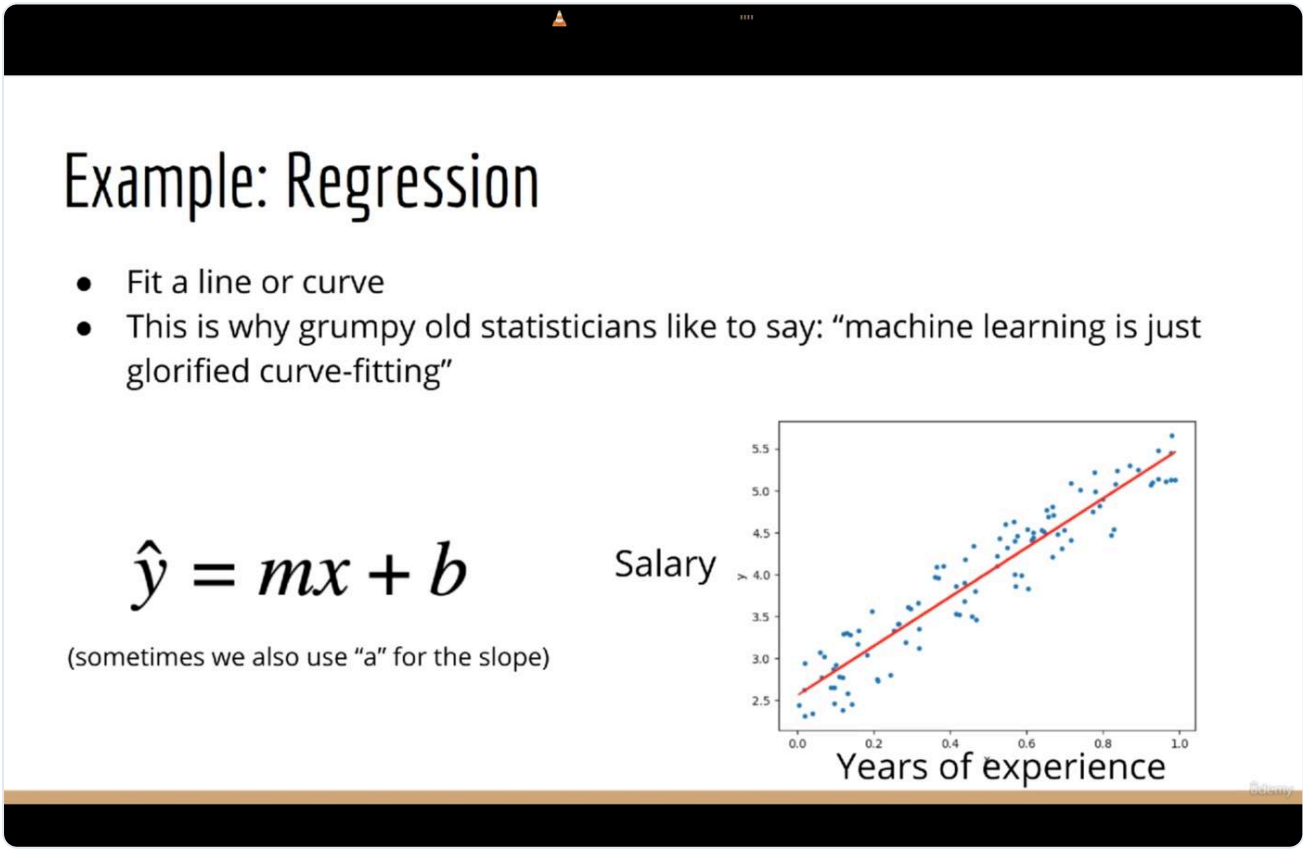


FIGURE 8.8 | Example: Regression

If you are a grumpy old statistician then you're probably going to be offended by everything we do. You can take this up with Geoffrey Hinton.

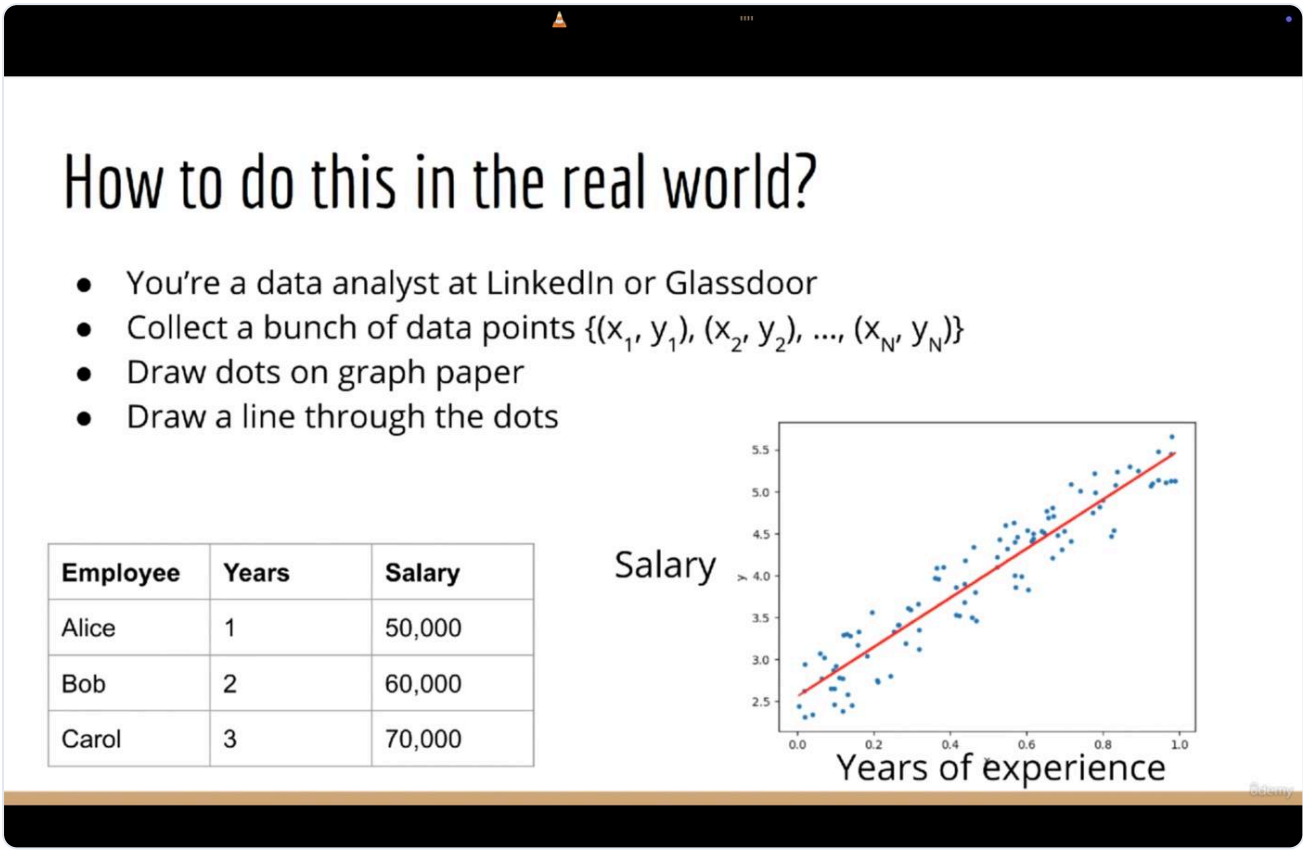


FIGURE 8.9 | How to Do This in the Real World?

So let's say you're a data analyst and you want to know how is salary related to years of experience. You might model this as a straight line where the input x is years of experience and the output y hat is the predicted salary as you may recall from your high school math studies. The equation of the line is y hat equals M X plus B. Here M is the slope and B is called the y intercept. Your job of course is to find these values of m and b to expand on this a little bit. Here's how you would do this in the quote unquote real world. Let's say for example you are a data scientist at LinkedIn or Glassdoor so you have access to their database and you can see for each user how many years of experience they have and what their current salary is.

Let's call these data points x 1 up to x n and y one up to why n. As mentioned previously X is the years of experience and Y represents the salary we use Y to represent the true salary whereas y hat represents the predicted salary.

FIGURE 8.10

Both X and Y are an index by the numbers 1 to N so there are N people in our database then what you're going to do is you're going to take a big piece of graph paper and plot each of these x y data points you're physically going to draw a dot for each data point then once you're finished drawing all your dots you're going to take a line that goes through all these data points. Pretty simple I think and believe it or not this is exactly what machine learning is so as mentioned the example we just discussed is called a regression. This is where you're trying to fit a line or a curve to some data points. Now there are two ways we can make this more complicated and more like the data sets we might encounter in the real world.

First we might have more than one input feature in the previous example. The number of years of experience was the only input feature but realistically you might measure more things. For example you might record things like what kind of degree the person has like a bachelors masters or PhD. You might record what school they went to what country they're from what their ages and their gender. All of these factors or features might affect the salary when you do this. The object you're trying to fit is no longer a line.

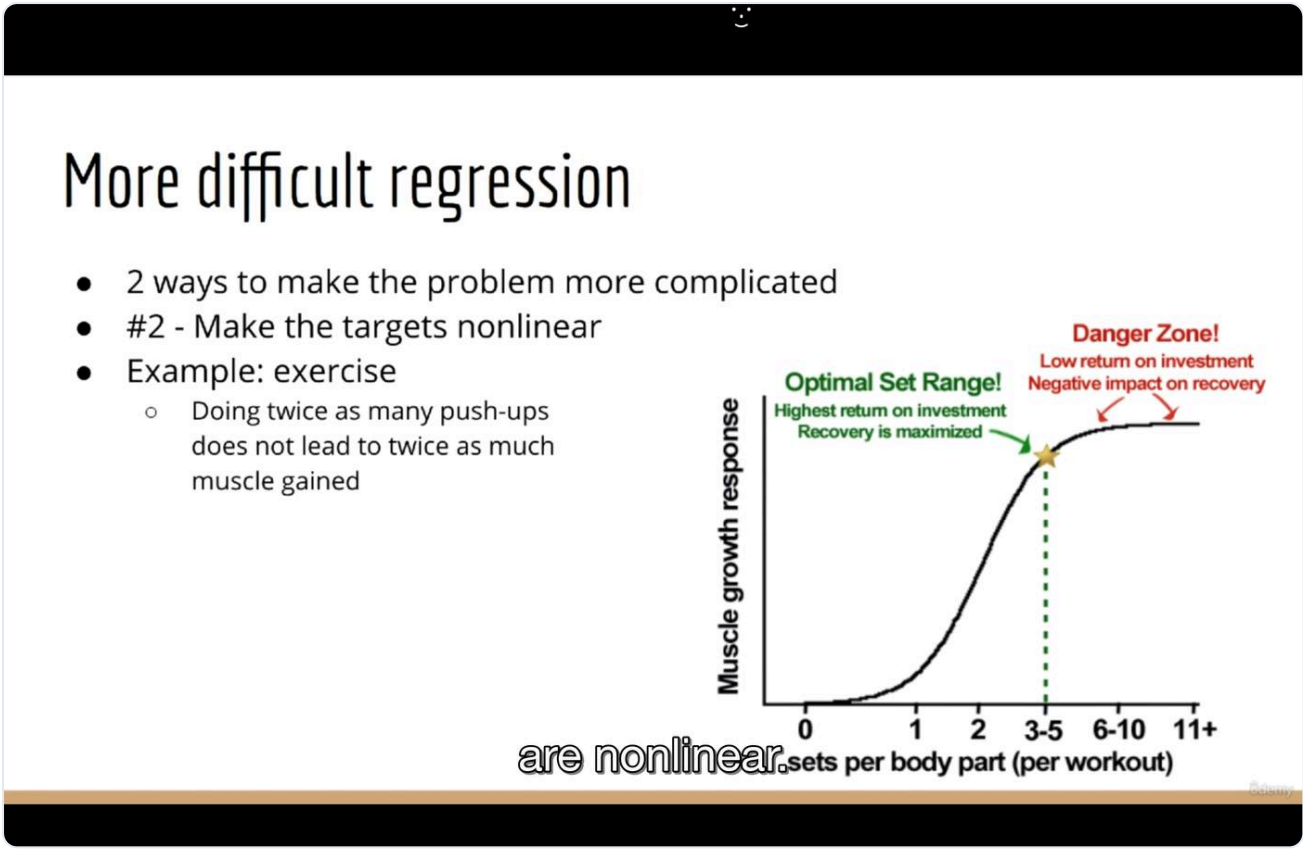


FIGURE 8.11

If you have two input features then you have three dimensions in total so you get a plane. If you have more than two input features then you get a hyperplane by the way in case it's not obvious you can't visualize anything beyond three dimensions because the physical world itself is three dimensions. The second way we can make the regression problem more complicated is that instead of trying to fit a straight or non curved object like a line or a plane we can fit a curve many real world data sets are nonlinear. Think of something simple like exercise. If I do 20 pushups Well I gain twice as much muscle as I would have if I did 10 pushups. If I do 30 pushups will I gain three times as much muscle.

Of course the answer is No. Otherwise we'd have people doing push ups all the time and becoming very large people. At some point the benefit of doing push ups is going to taper off.

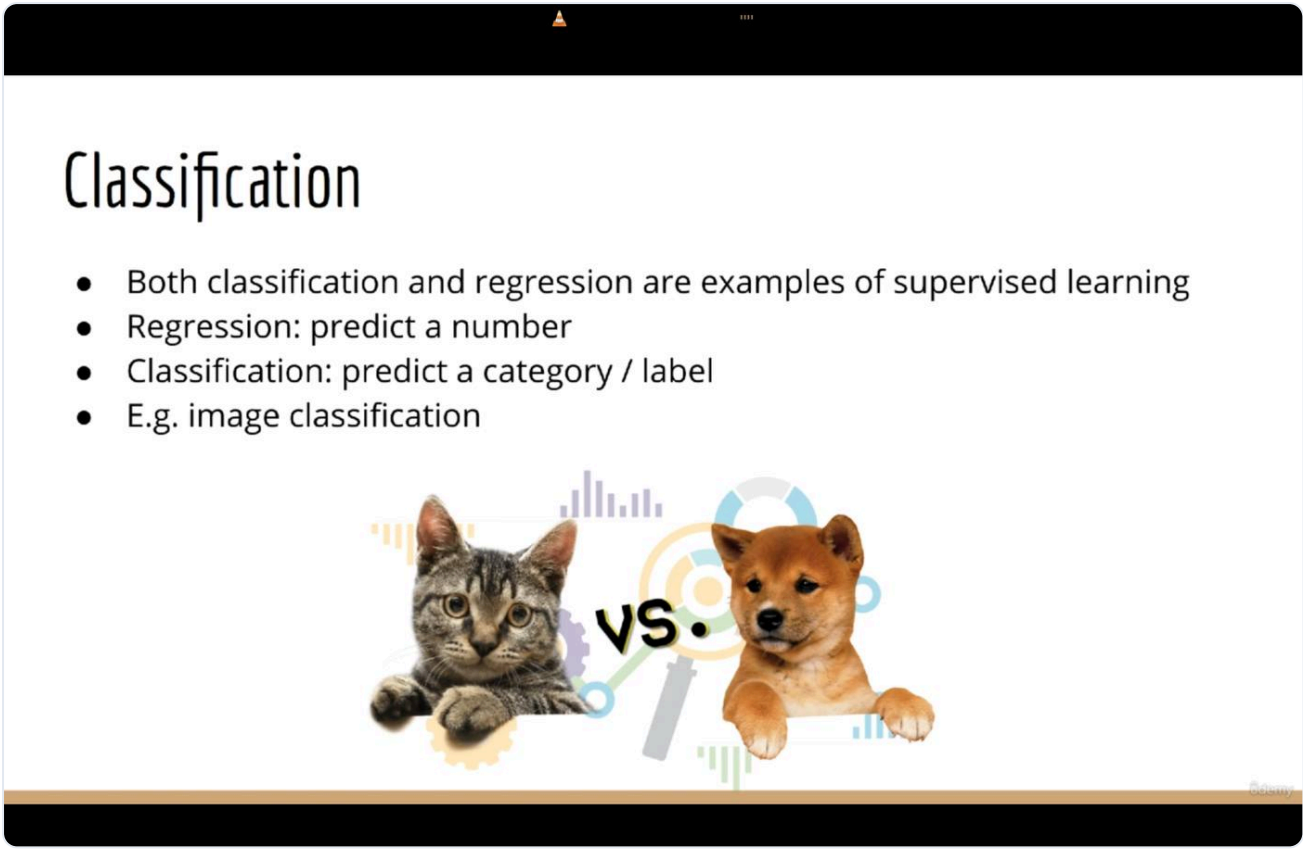


FIGURE 8.12 | Both Classification and Regression Are Examples of Supervised Learning

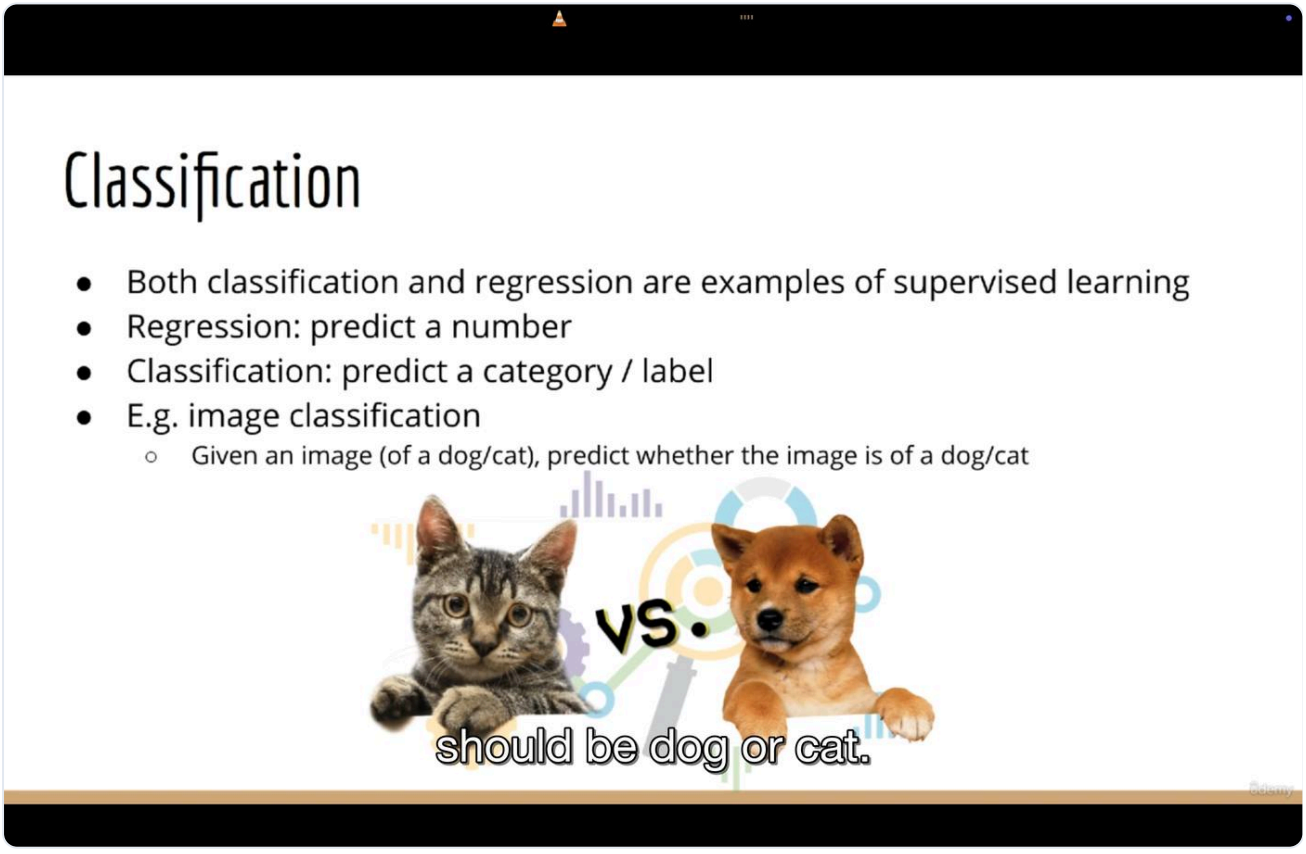


FIGURE 8.13 | Both Classification and Regression Are Examples of Supervised Learning

Now let's turn our attention to another kind of machine learning problem known as classification both classification and regression are examples of supervised learning which is going to be the main focus of this course we will sometimes discuss unsupervised learning as well but the main focus will be supervised learning so whereas regression is concerned with predicting a real value. No classification is concerned with predicting a category one popular example is image classification where your model accepts as input an image of a dog or a cat and tries to predict whether the label should be dog or cat.

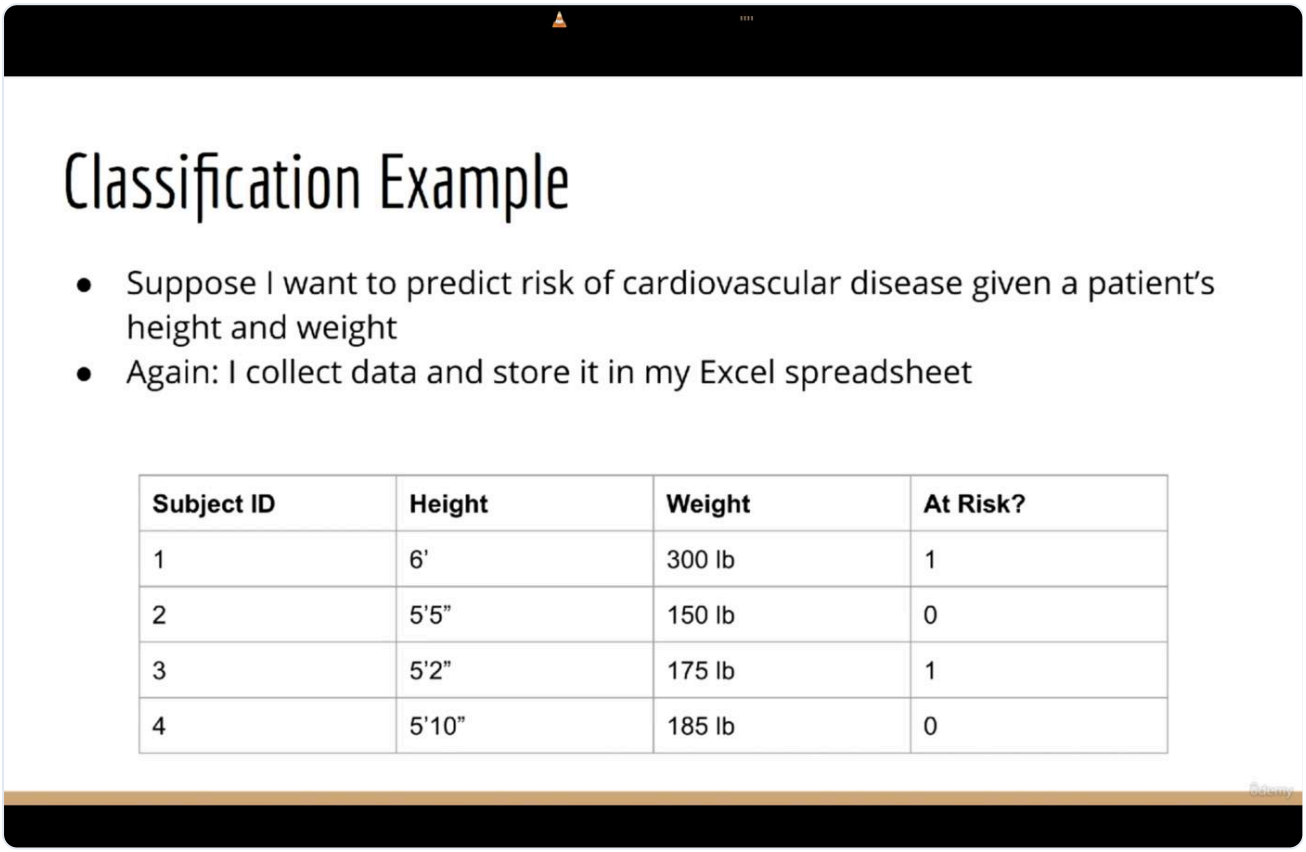


FIGURE 8.14 | Classification Example

However in this lecture we are going to look at an example that helps us build this geometrical intuition we've been talking about so let's say I want to predict the risk of cardiovascular disease given a patient's height and weight. This is a categorical problem because my prediction is either going to be at risk or not at risk. Again this comes down to a data collection experiment. I'm going to look at all my hospital records and I'm going to write down all of this information in an Excel spreadsheet. I'll have two columns to represent my X and one column to represent my target y by the way. Note that it's customary to represent binary targets as the integers 0 and 1.

So he would say not at risk is zero and at risk is 1. Note that there is no reason that at risk should be one. Just like the dogs and cats example it doesn't matter of dogs or one and cats are zero or dogs or zero and cats are one.

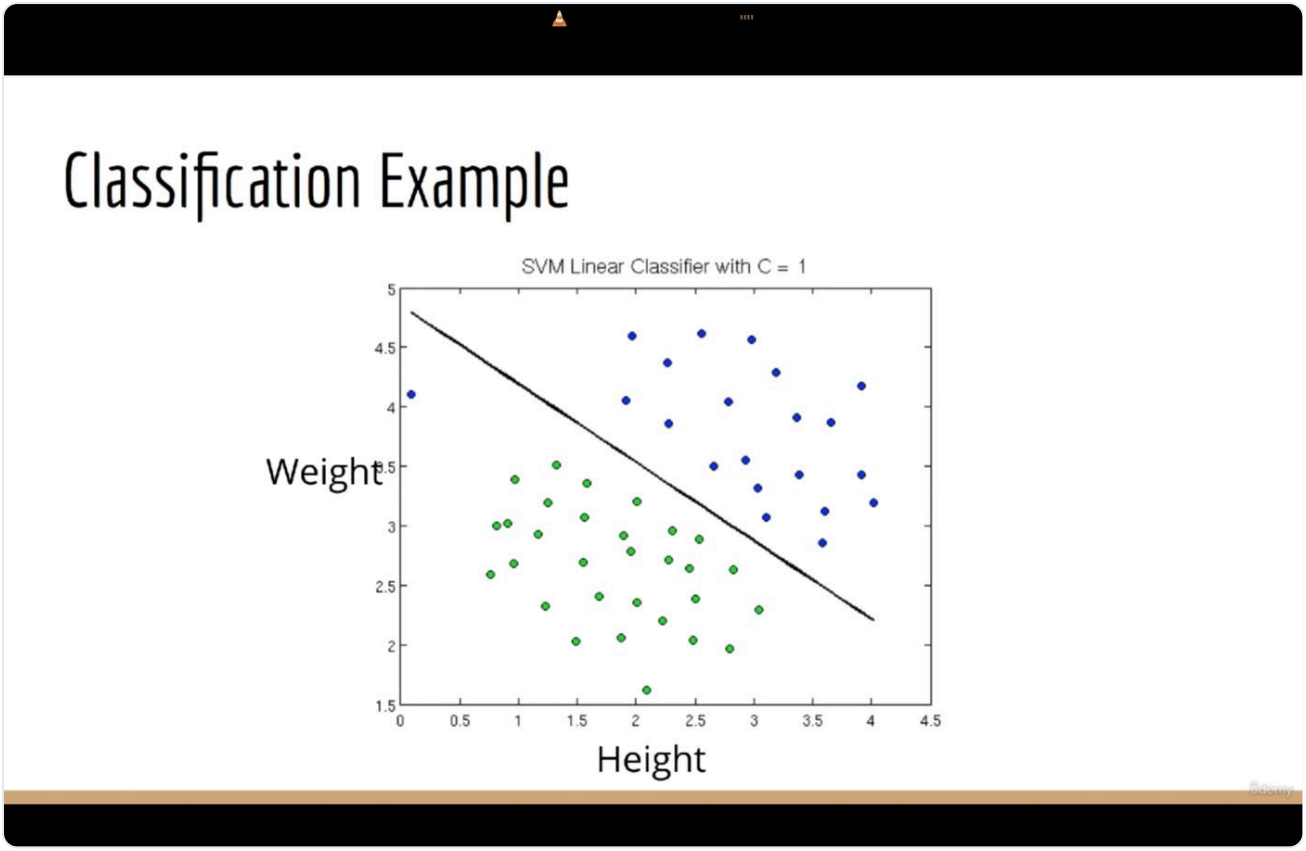


FIGURE 8.15 | Classification Example

This assignment is just arbitrary OK. So now that I've collected all my data what am I going to do. Well again I'm going to apply these on a grid on the horizontal axis. I'm going to have my first feature the patient's height on the vertical axis. I'm going to have my second feature the patient's weight now importantly unlike regression. The target does not get its own axis instead because it's categorical we would represent it with color so I might color the at risk patients as blue and then not at risk patients as green. So what is my model going to be for classification. Well the simplest model again is a line. However this line is a little bit different from our regression line from earlier.

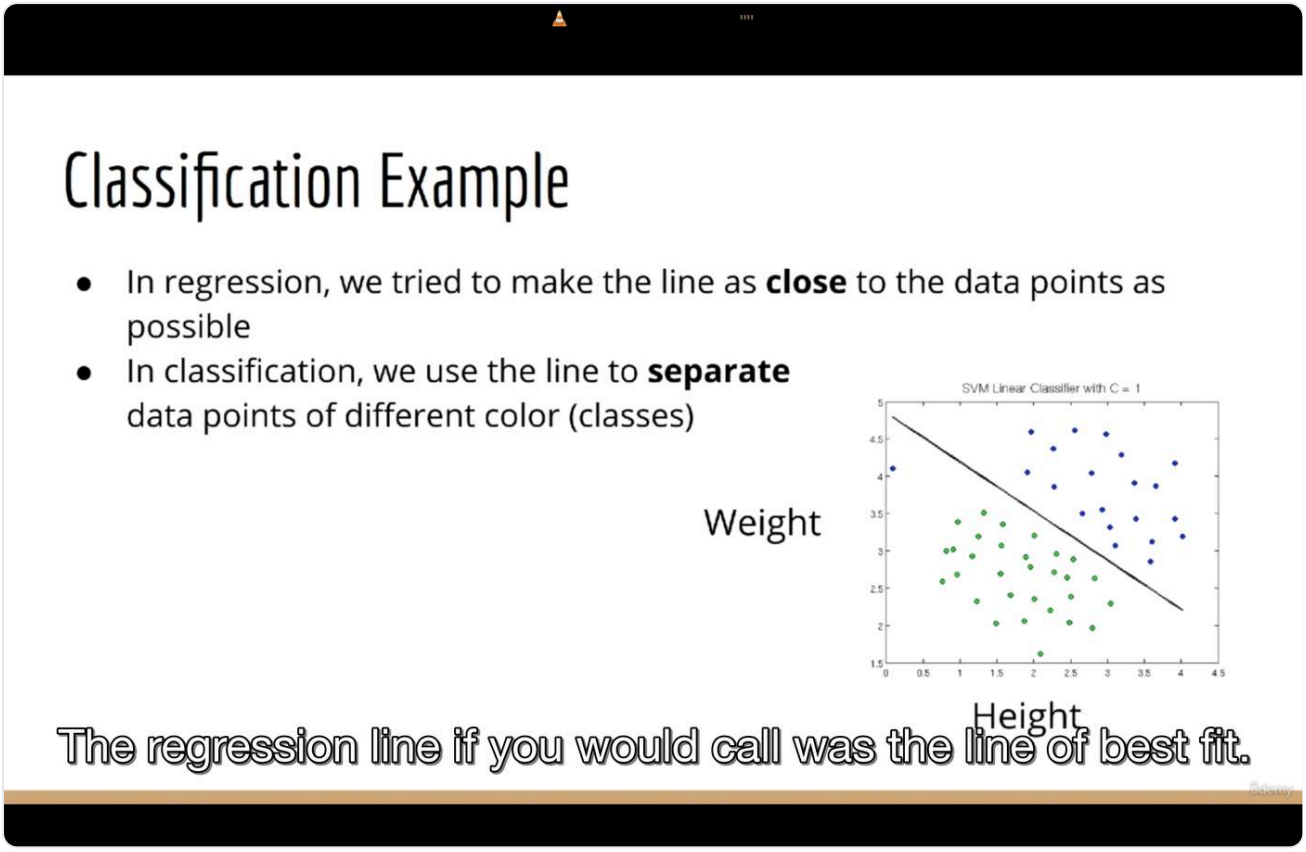


FIGURE 8.16 | Classification Example

The regression line if you would call was the line of best fit. We are trying to get the line to be close to all the data points. Now we are no longer trying to do that. Instead we are trying to get the line to separate the two groups of data points or in other words separate the different colors as you can see this again boils down to a geometry problem. How do we find this line that can indeed separate these categories one lesson that goes hand-in-hand with the geometrical perspective is another motto of mine. All data is the same.

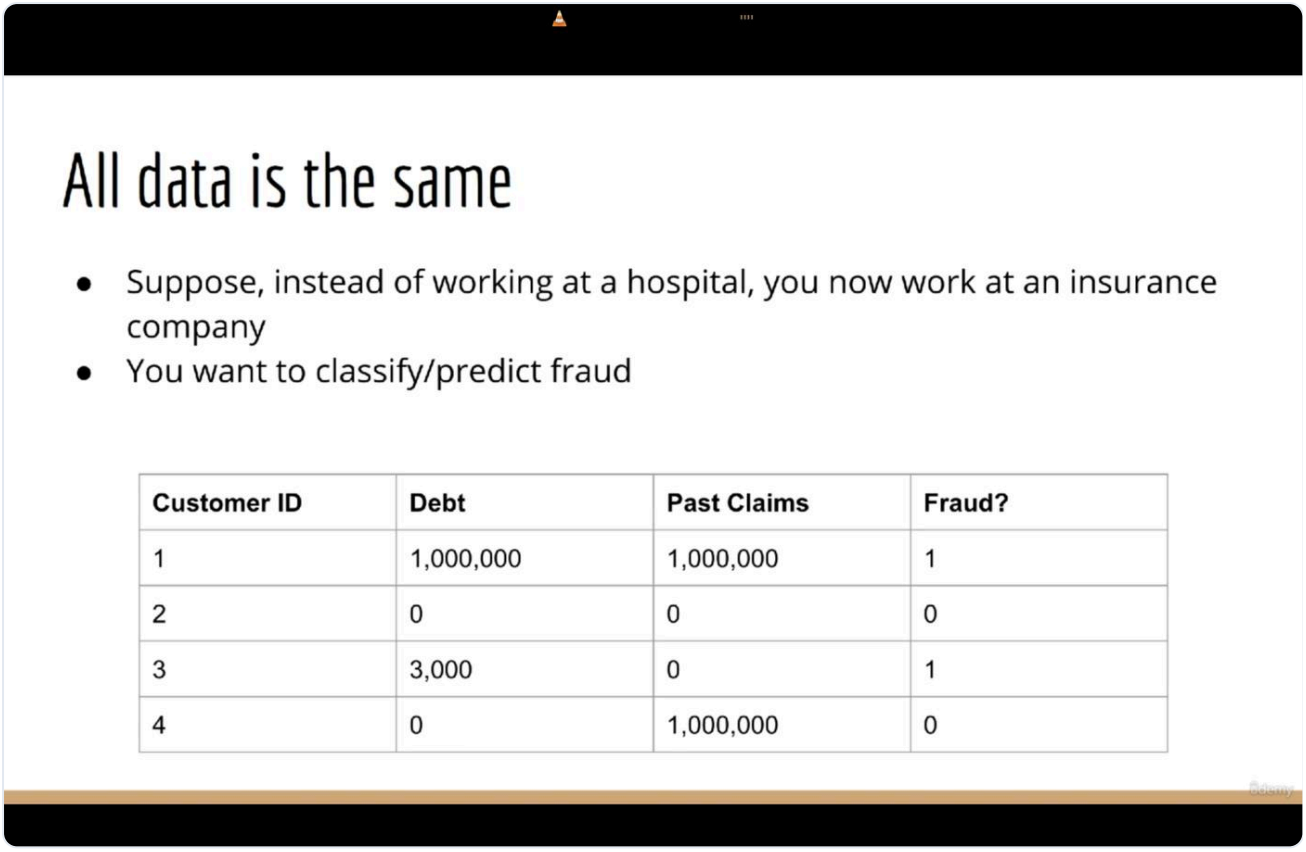


FIGURE 8.17 | All Data Is the Same

Let's use another example to illustrate the point suppose instead of working at a hospital you now work and an insurance company your job is to do fraud detection or in other words classify instances of fraud. So again you have two categories fraud or not fraud you're trying to predict whether or not a given claim is fraud. Let's say again you've collected some data points. Again with two input features. Suppose these are number one the amount of debt the claimant has. And number two the total amount of past insurance claims and again it's the same story.

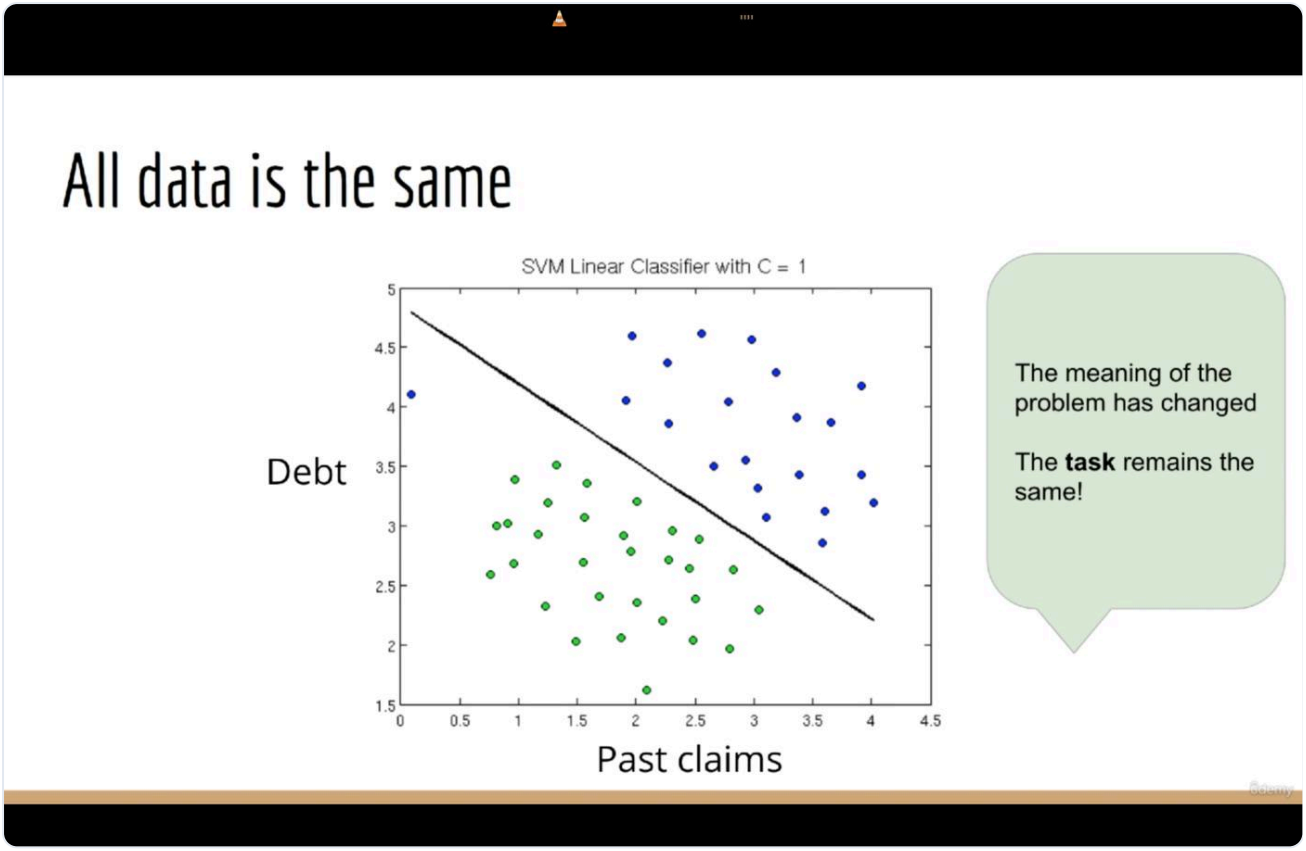


FIGURE 8.18 | All Data Is the Same

Once you've collected all your data points you're going to plot them and color them on a grid your job again is to separate these data points with a line or a curve. So you see that just because we change the meaning of the problem we haven't changed what our actual task is. It's still to plot these points and separate them with some kind of decision boundary a line is the simplest. But just like with regression there are two ways we can make this more complicated. The first way is that we can have more input features so that it's not possible to plot them on a grid that we can visualize. The second way is that the decision boundary may not be linear in which case the model will be an equation that's more complicated than a line or hyperplane.

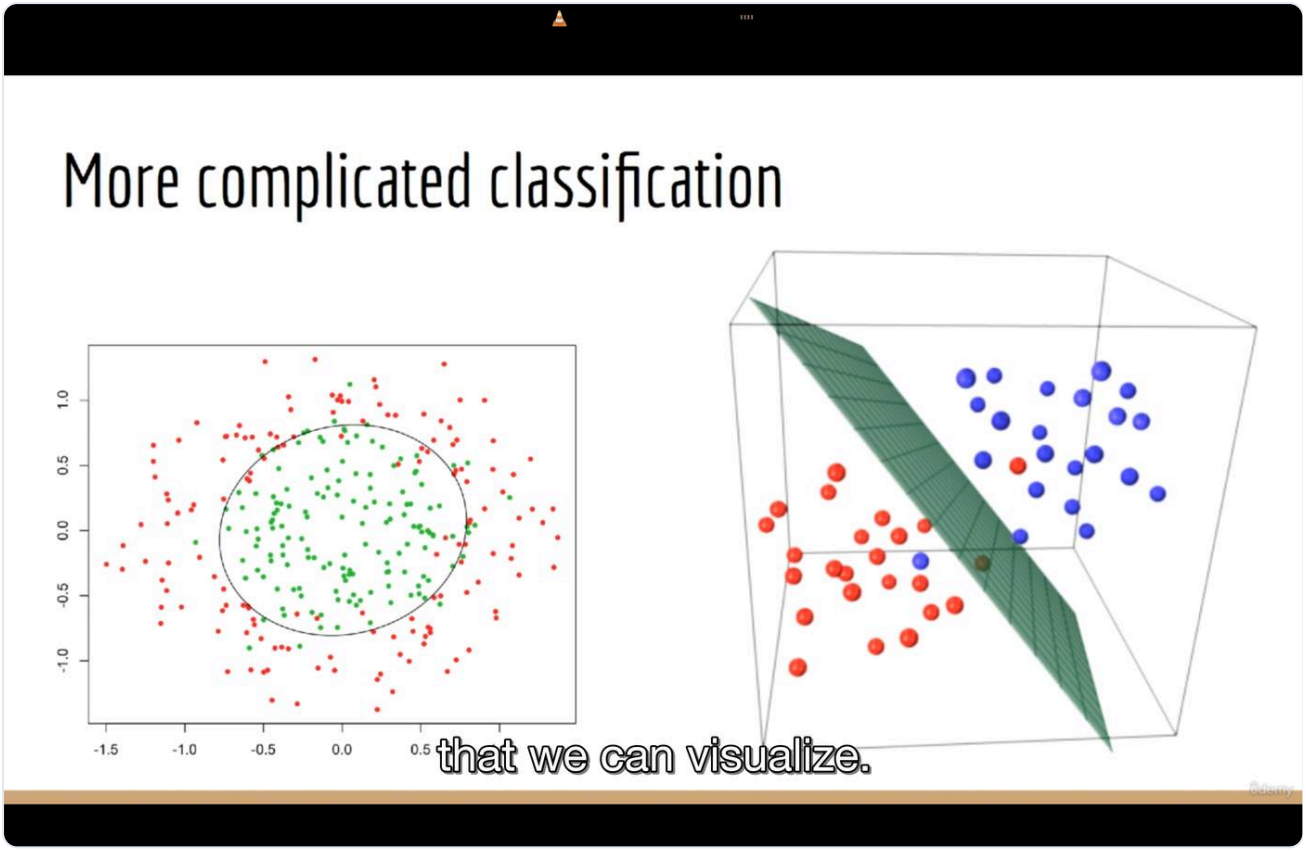


FIGURE 8.19 | More Complicated Classification

Importantly however the moral of the story remains the same. This is still a geometry problem and in fact for our two classification examples the geometry problem was the same. The only thing that changed was the meaning of the numbers of course machine learning algorithms don't care about these meanings and so they are essentially irrelevant in the eyes of the machine learning model. That's why we say all data is the same. This becomes a very powerful concept in the future. For example the same kind of model used for neural machine translation can also be used for a chatbot.

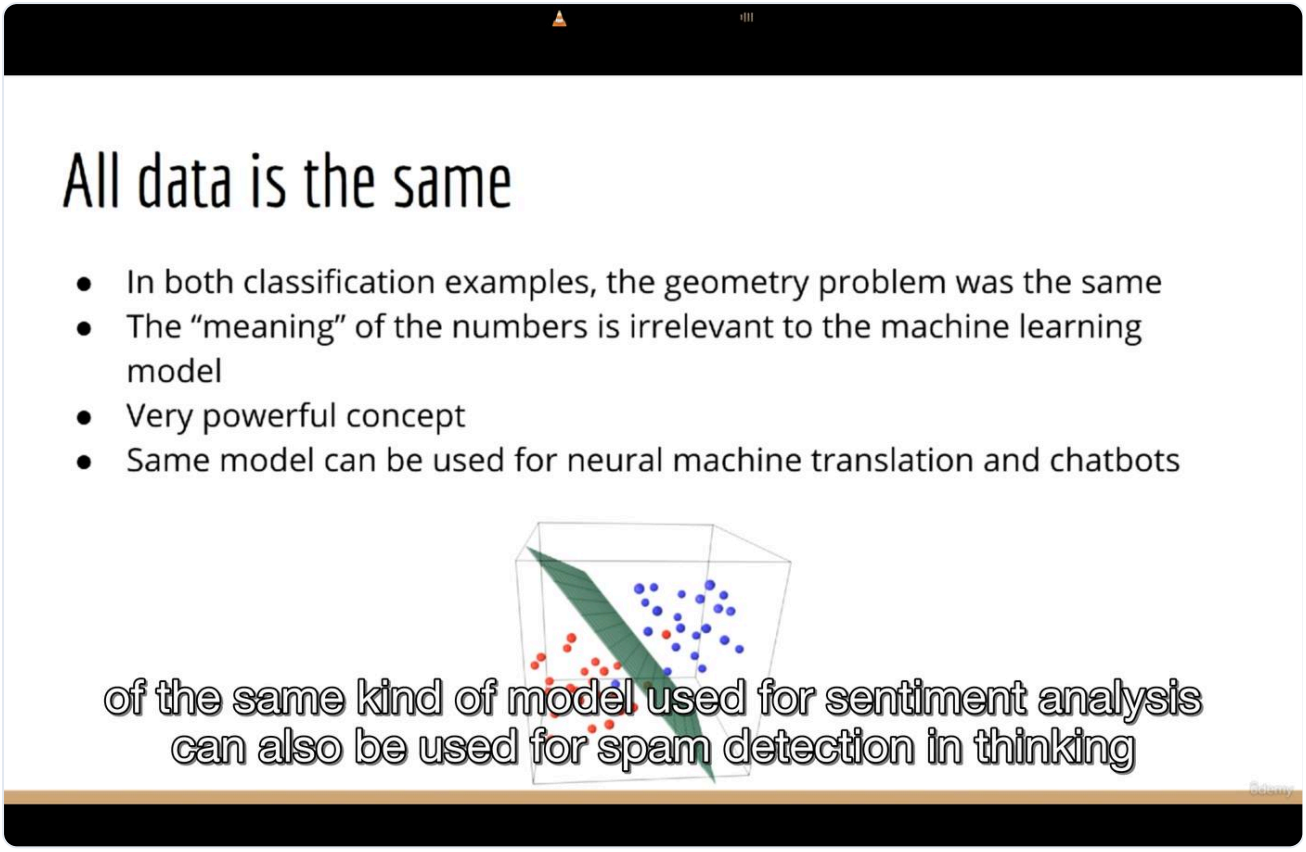


FIGURE 8.20 | All Data Is the Same

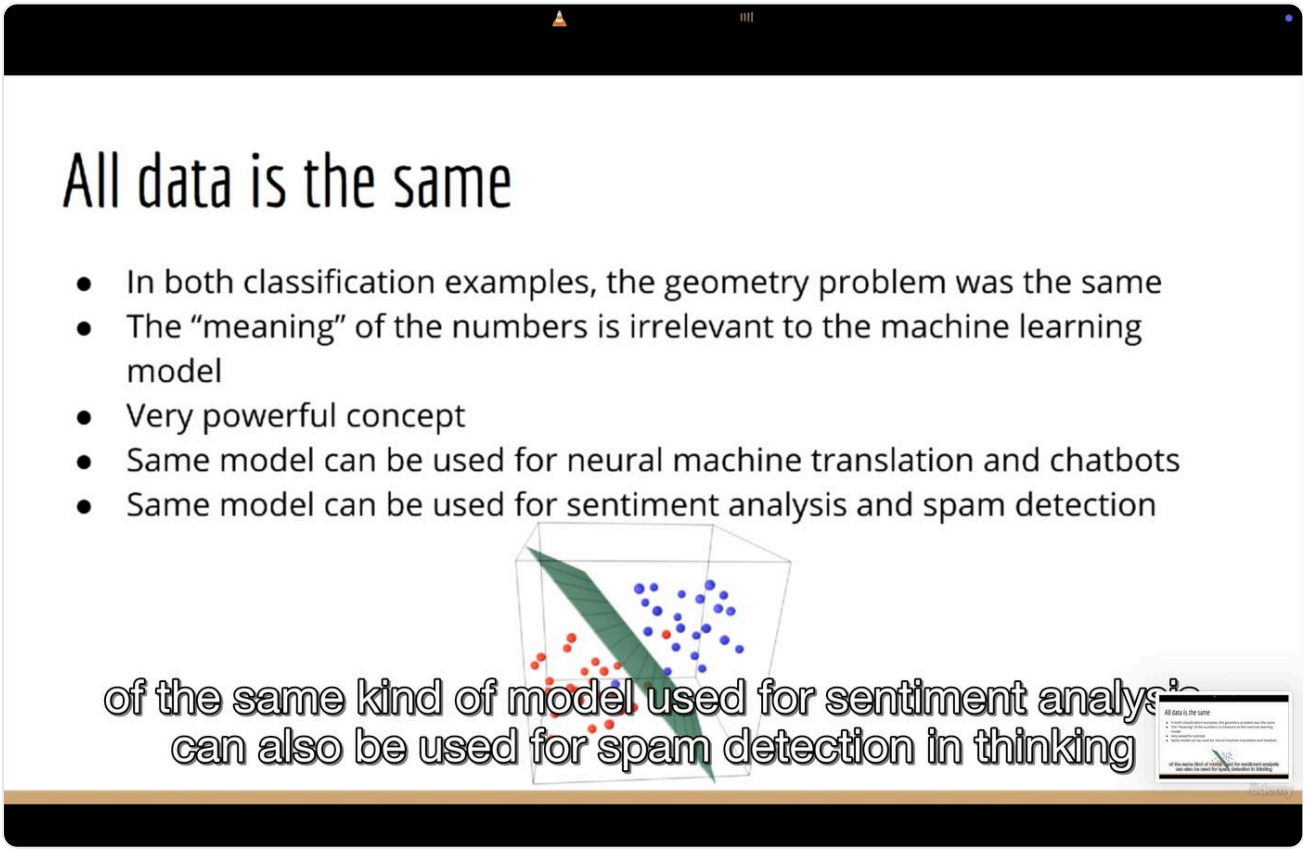


FIGURE 8.21 | All Data Is the Same

The same kind of model used for sentiment analysis can also be used for spam detection in thinking in this way essentially gives you machine learning superpowers. To summarize this lecture the goal was to take the magic away from machine learning you learned a very important lesson which is that machine learning is a magic. In fact it's just geometry. We learned about the two different kinds of geometry problems that supervised learning can solve. The first kind is regression where we try to get a line a plane a hyperplane or curve to be as close as possible to the data points from a given dataset.

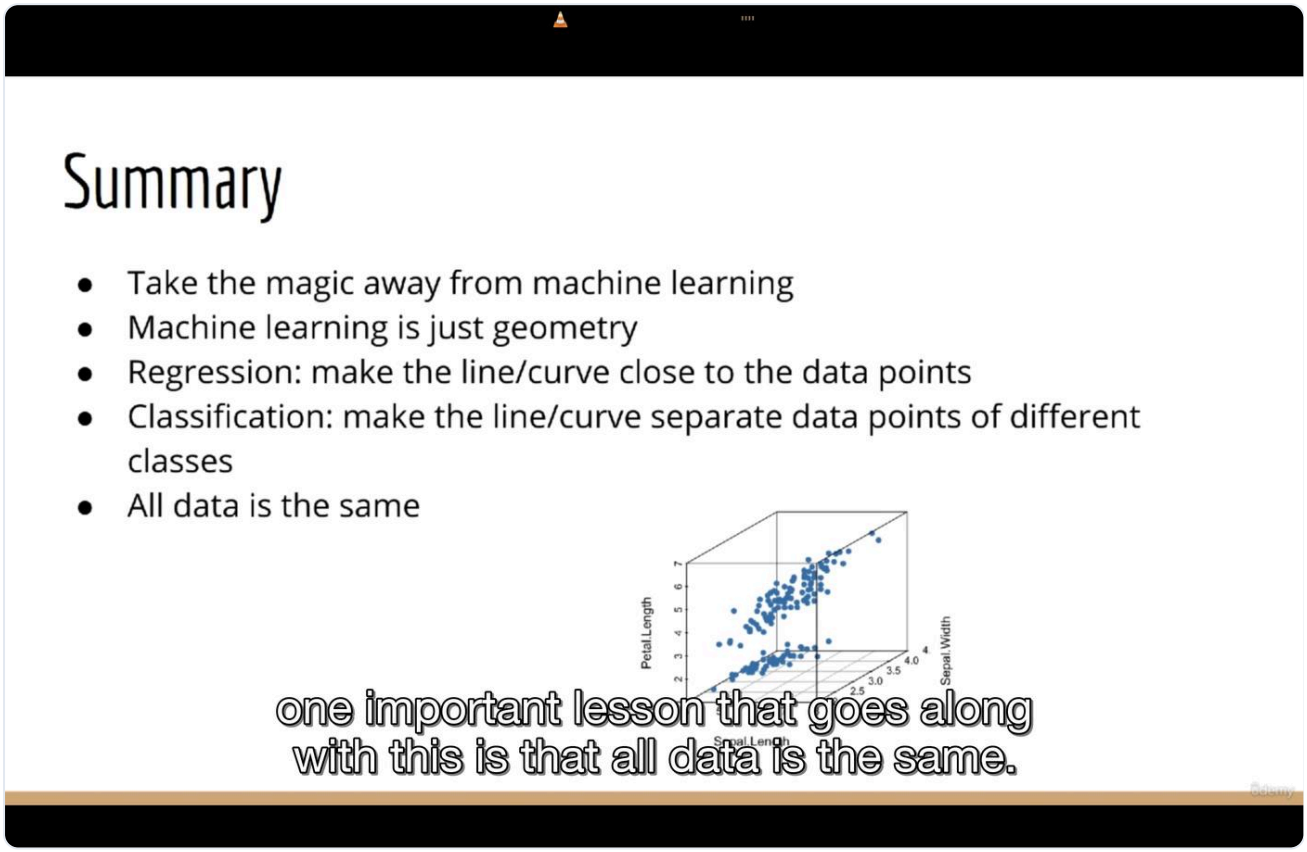


FIGURE 8.22 | Take the Magic Away from Machine Learning

The second kind is classification where instead of trying to get the curve as close as possible to the data points we try to separate data points belonging to different categories one important lesson that goes along with this is that all data is the same. It doesn't matter if we're trying to classify risk of cardiovascular disease or if we're trying to classify fraud in an insurance company. The general problem remains the same and it is a problem of geometry.

LECTURE 9

Finding the Slope and Intercept

In this lecture we are going to start looking at machine learning from a new perspective.

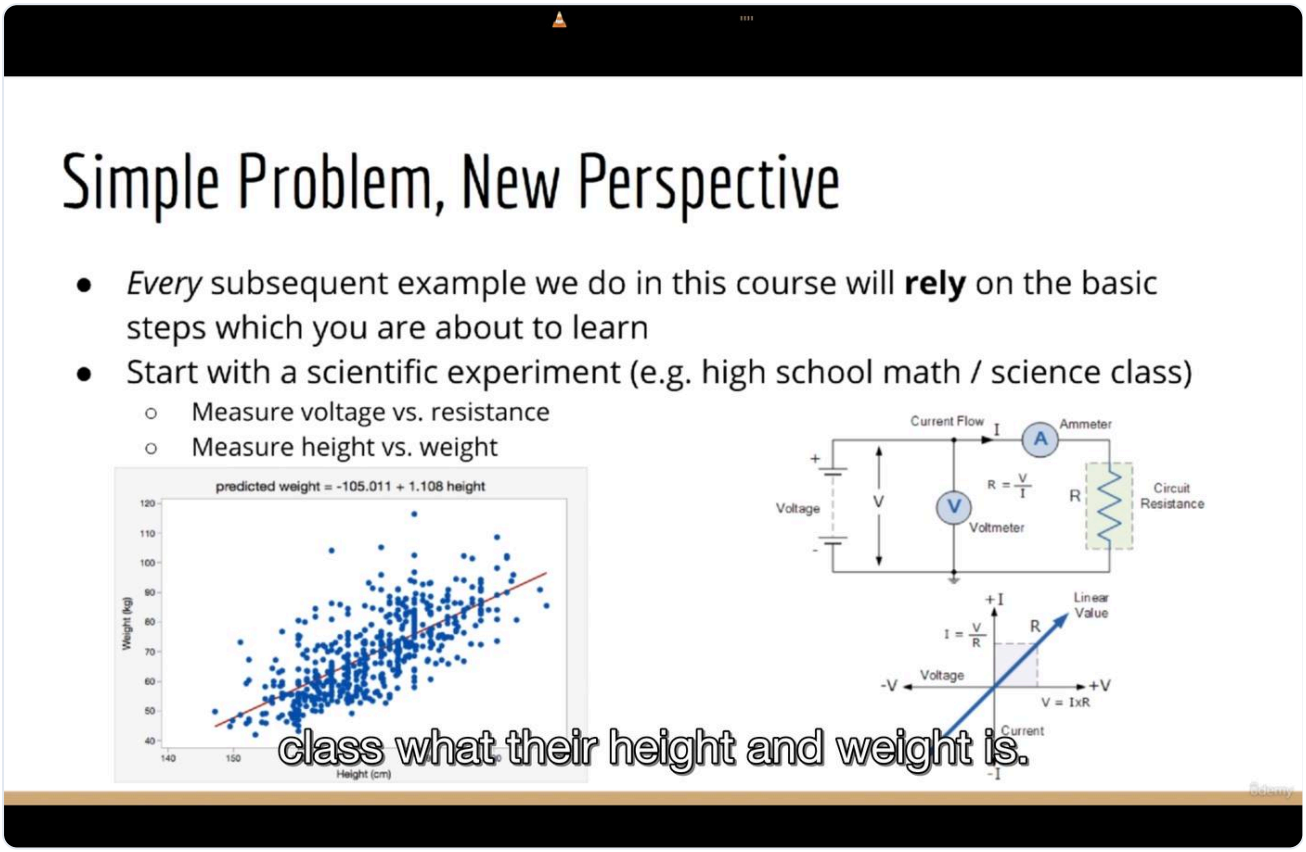


FIGURE 9.1 | Simple Problem, New Perspective

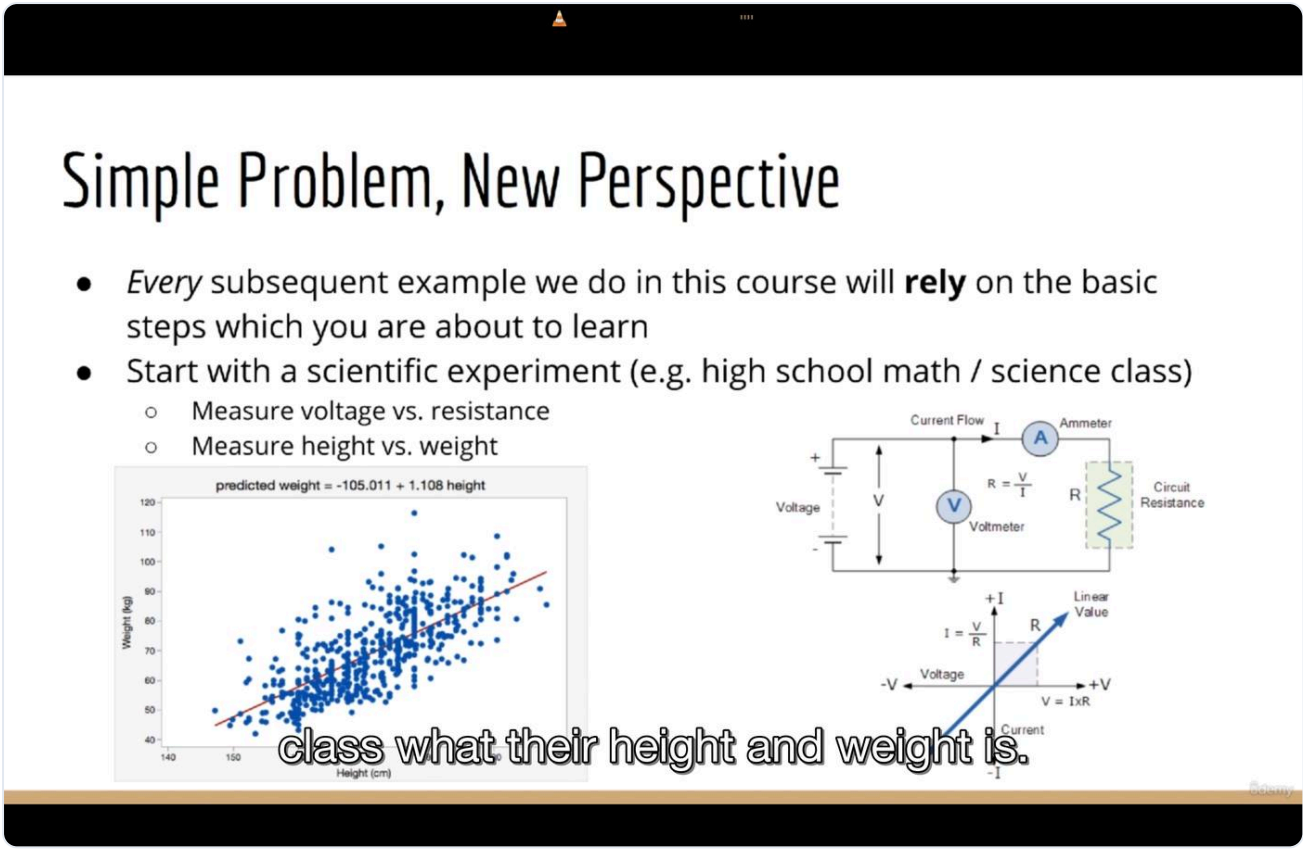


FIGURE 9.2 | Simple Problem, New Perspective

While this example might seem overly simplistic you might be surprised to learn that every example that we do in this course will rely on these basic steps which you are about to learn. First let's start by describing the basic task. I'm sure you're all familiar with this example from your grade school of science and math classes. Let's suppose we've done some kind of scientific experiment. For example we measured the voltage in a circuit with different resistors or we asked everyone in our class what their height and weight is. This gives us a dataset with two columns of numbers one that we can call X and the other that we can call y. Now imagine that we plot these points on a grid and we get something that looks pretty close to a line.

We would like to draw a line on this grid that closely passes through these points in your grade school class.

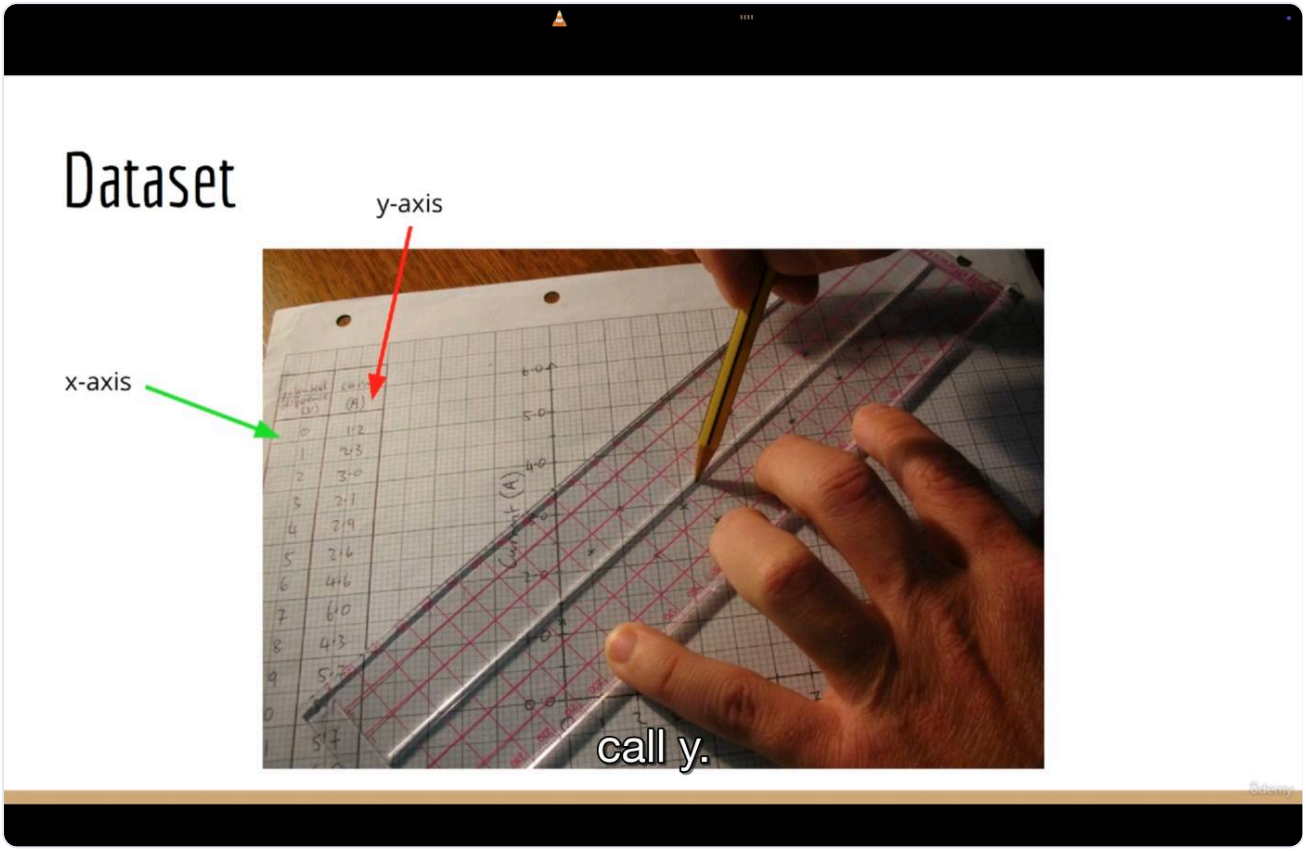


FIGURE 9.3

Maybe you did something like took a ruler and moved it around a little bit until you found the best possible line. But now that we're doing machine learning we need more sophisticated methods than just drawing dots on paper and using a ruler to estimate what the best line should be. The problem is what happens when we have thousands or millions of dots. What if our data is more than 2 dimensional then this approach is no longer possible. So perhaps you might say aha I know what to do. Let's use our trusty tool scikit-learn as you know by now. This is what a basic regression script might look like if we were to use scikit-learn. First we load in some data call that x and y.

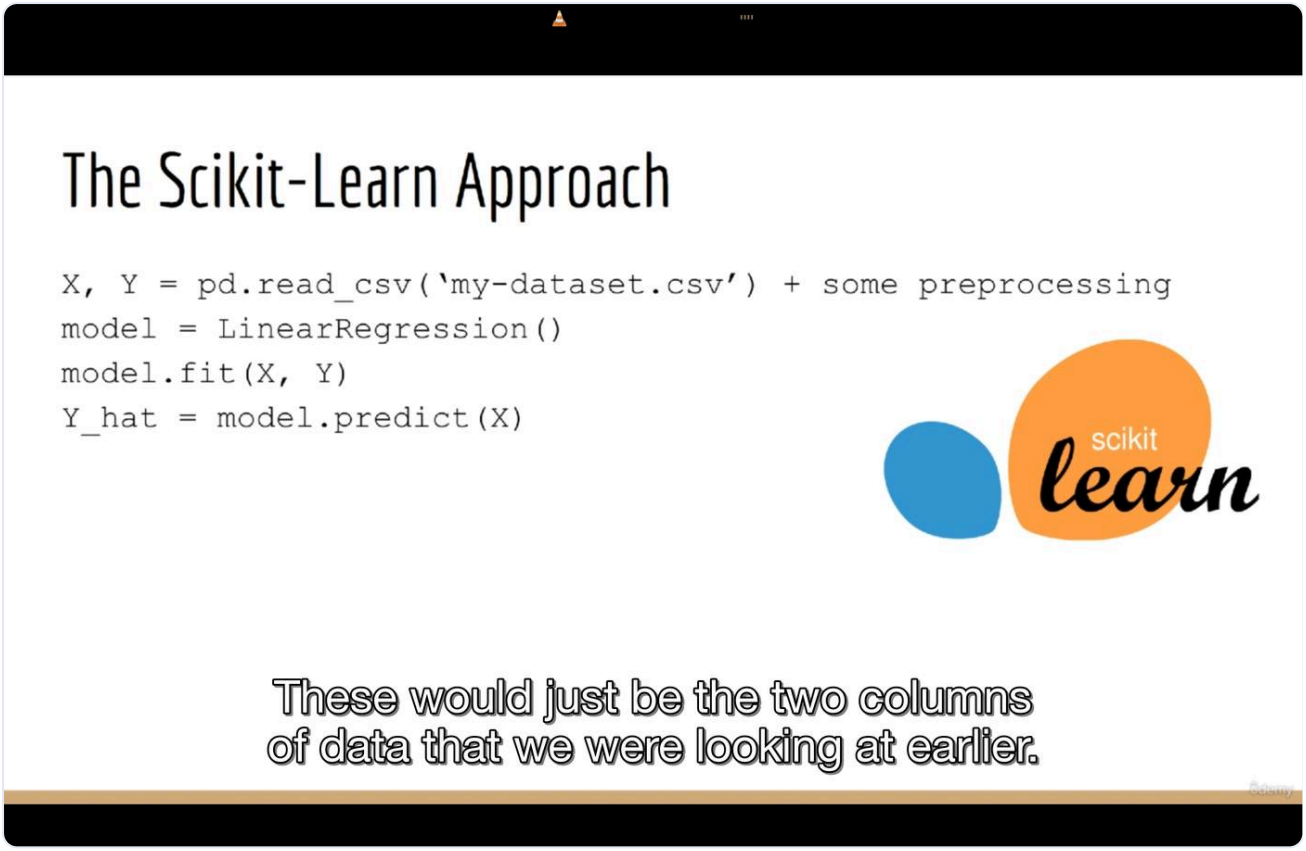


FIGURE 9.4

These would just be the two columns of data that we were looking at earlier. Next we create a model for the coming lectures we'll be interested in linear regression.

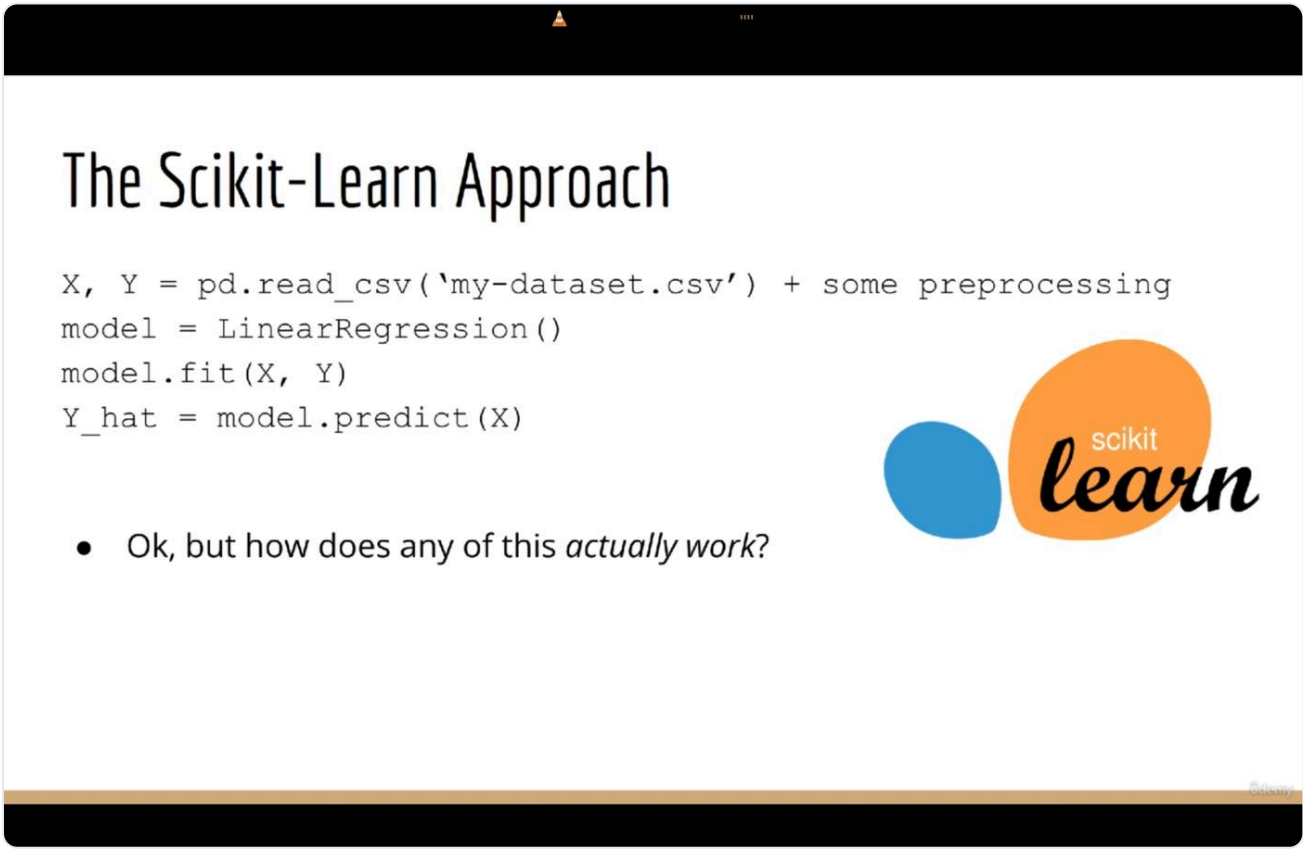


FIGURE 9.5

So let's create a linear regression model. Next we call the fit function to fit our model to the data.

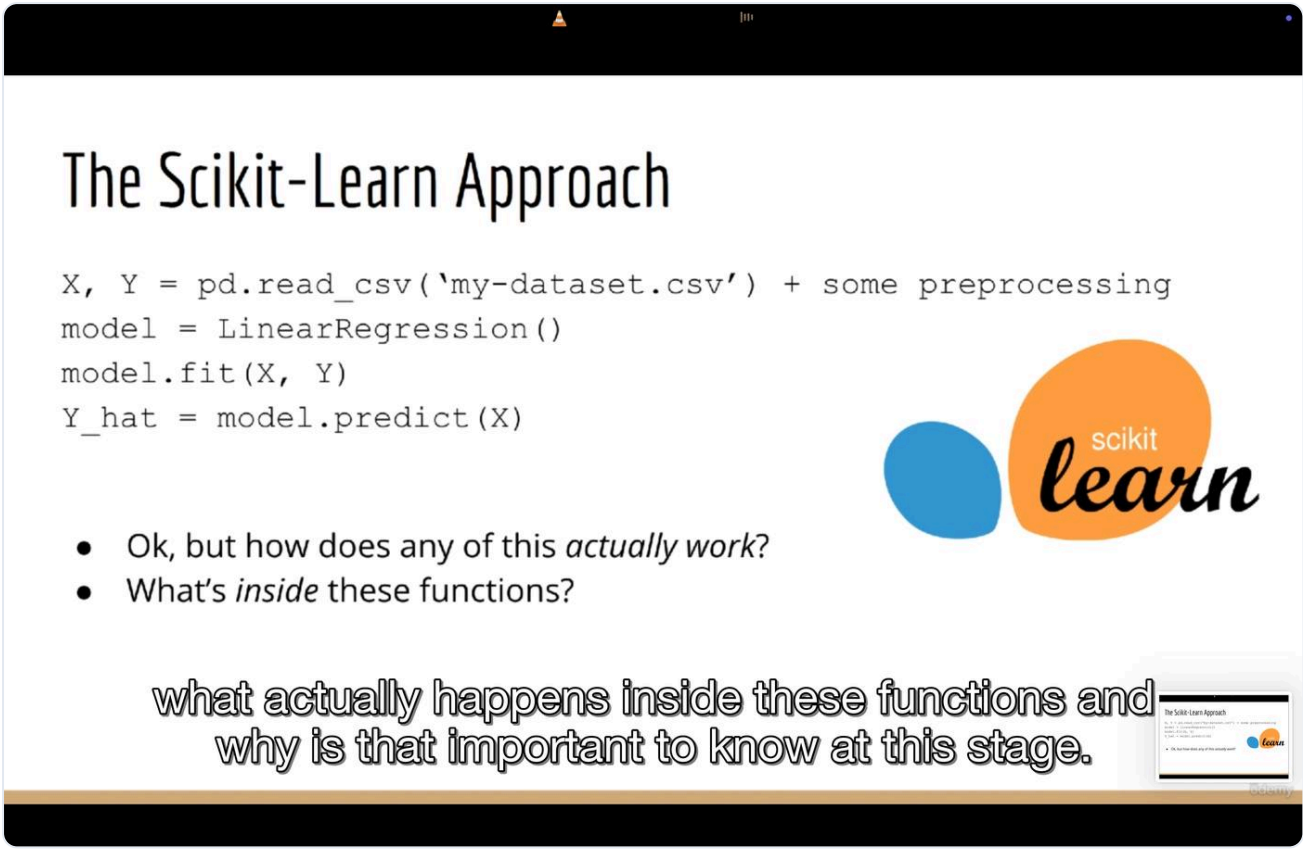


FIGURE 9.6

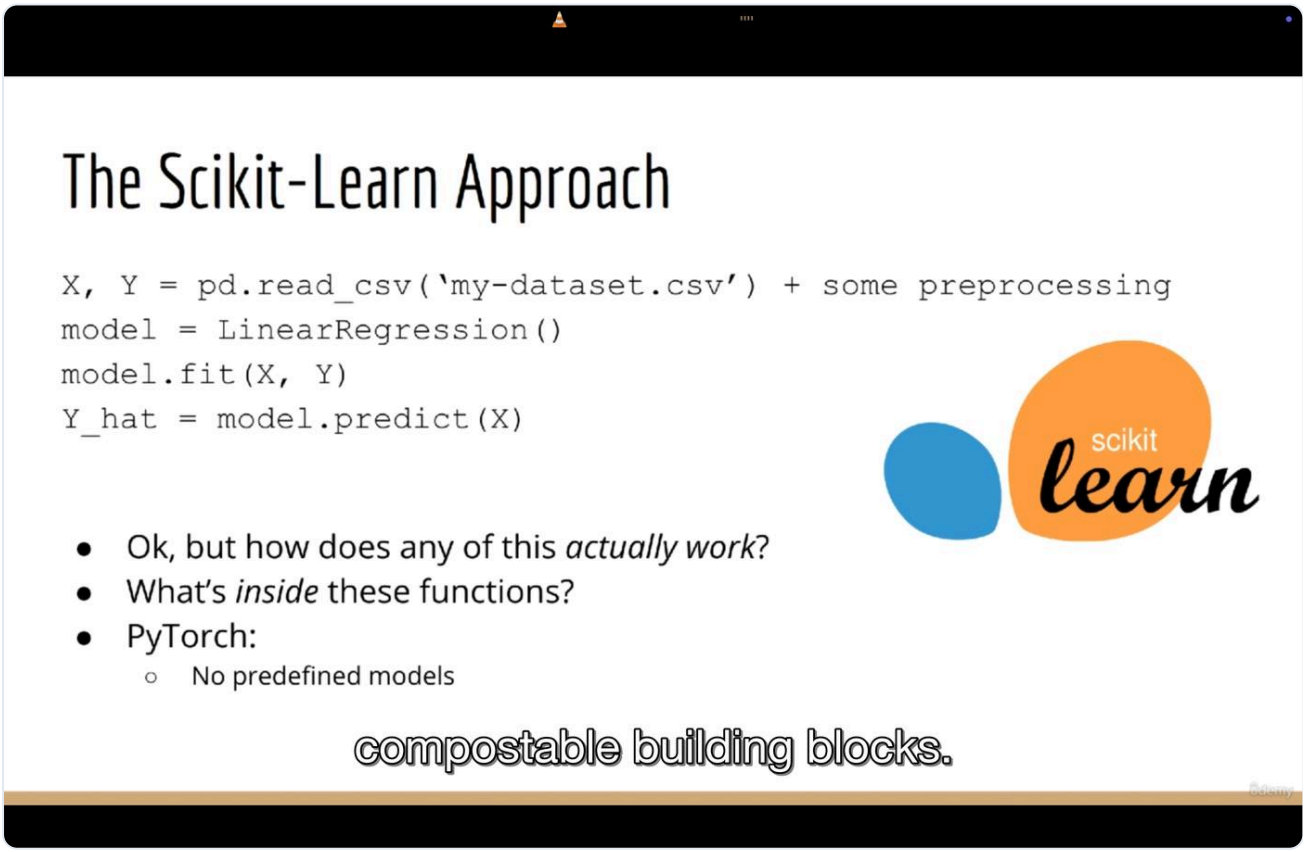


FIGURE 9.7

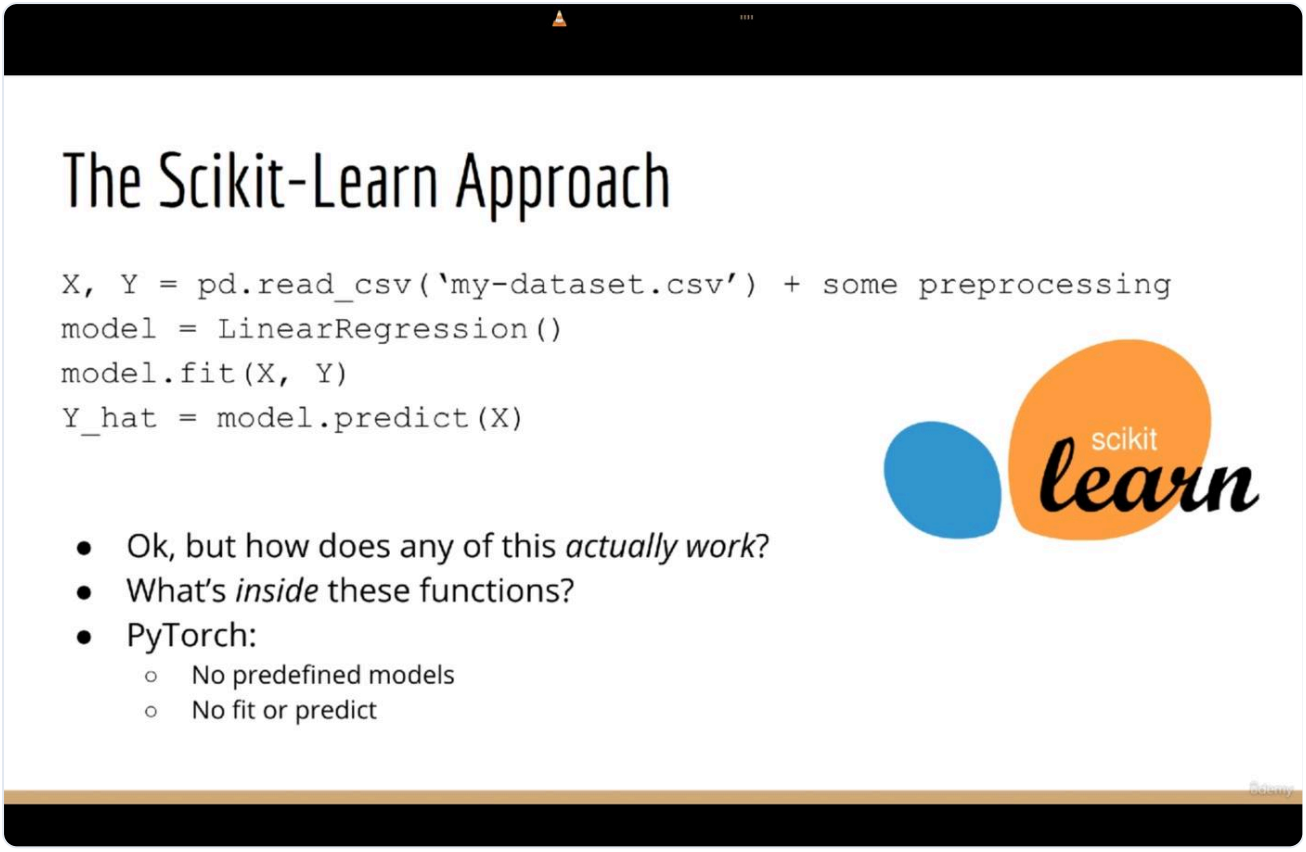


FIGURE 9.8

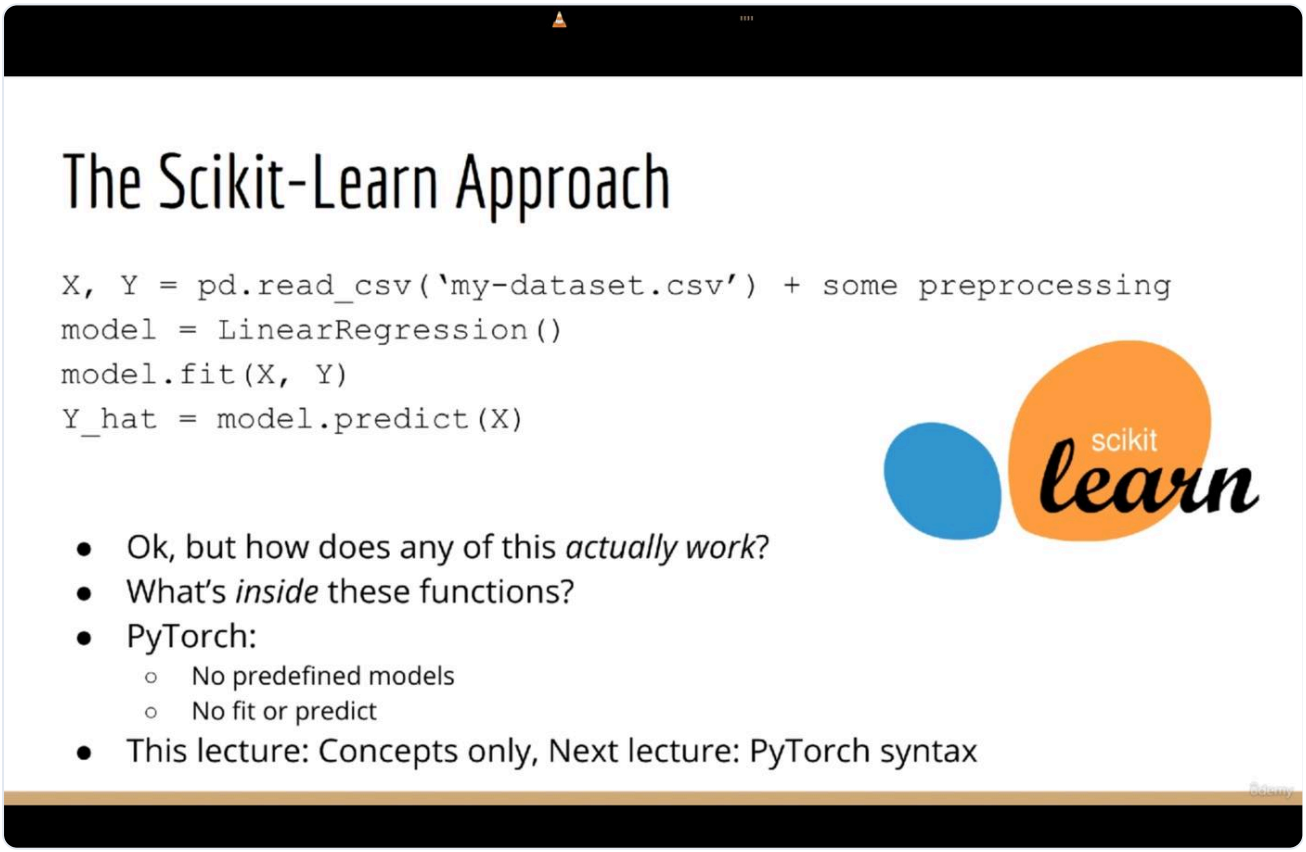


FIGURE 9.9

Next we can do multiple things with our fitted model such as making predictions with model dot predict the problem with this is this is just using an API but it doesn't tell us how any of this actually works what actually happens inside these functions and why is that important to know at this stage. What I want to highlight on this screen is what we should expect it to be different. When we look at PyTorch first model creation is going to be different. As I mentioned earlier PyTorch is going to allow us to build all kinds of different models using composable building blocks. So it's not going to be some predefined model as in scikit-learn next. I want to emphasize in PyTorch there is no such thing as a fit function or a predict function.

This is unlike TensorFlow and Keras which do provide you with those functions. So if you're looking for something simpler than TensorFlow or Keras maybe more your pace in PyTorch however that will not be the case. Therefore we're going to have to find out how to do all these steps in PyTorch using our knowledge of the concepts. Importantly however this lecture will focus only on the concepts and not on any PyTorch in the next lecture. We'll look at how to actually implement these concepts in PyTorch syntax so let's first look at what these concepts are. Before we discuss the details I'm going to go out of order in some sense but you'll see why that has to be the case.

The three main concepts we want to discuss are number one what is the model architecture and how do we actually build a model. If it's not defined for us like it is in scikit-learn. Number two how do we make predictions with the model. In fact this is closely tied to number one and number three.

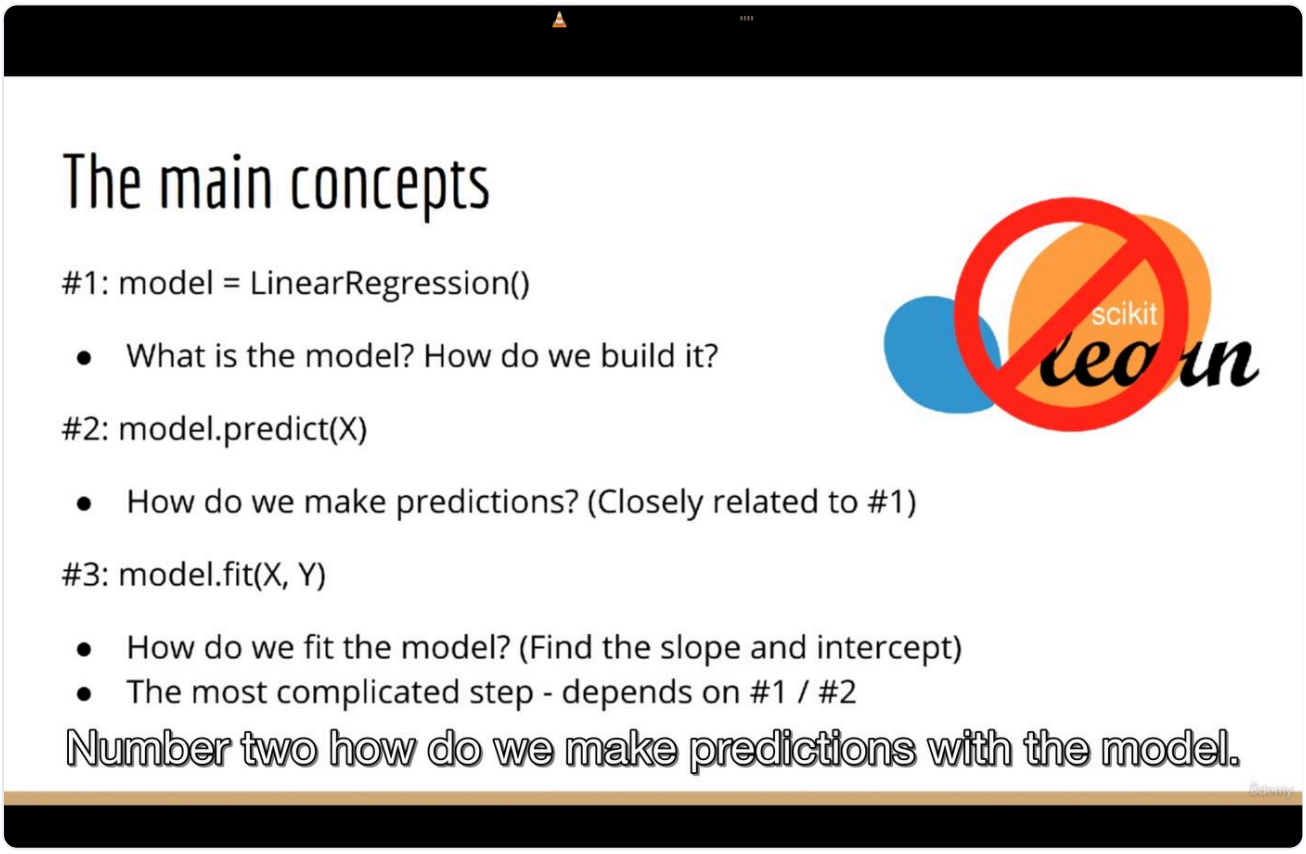


FIGURE 9.10 | The Main Concepts

How do we fit the model that is how do we find the slope and intercept parameters as you'll see. This is the most complicated step. It also depends on number one and number two which is why we have to discuss this last so number one. What is the model architecture. Well you already know this. It's why hat equals M X plus B. Notice our use of the hat symbol here. Typically we'll use Y for the true value also known as the target which is given in the dataset and y hat for the corresponding prediction. Fortunately for us this actually solves both of the first two problems for us.

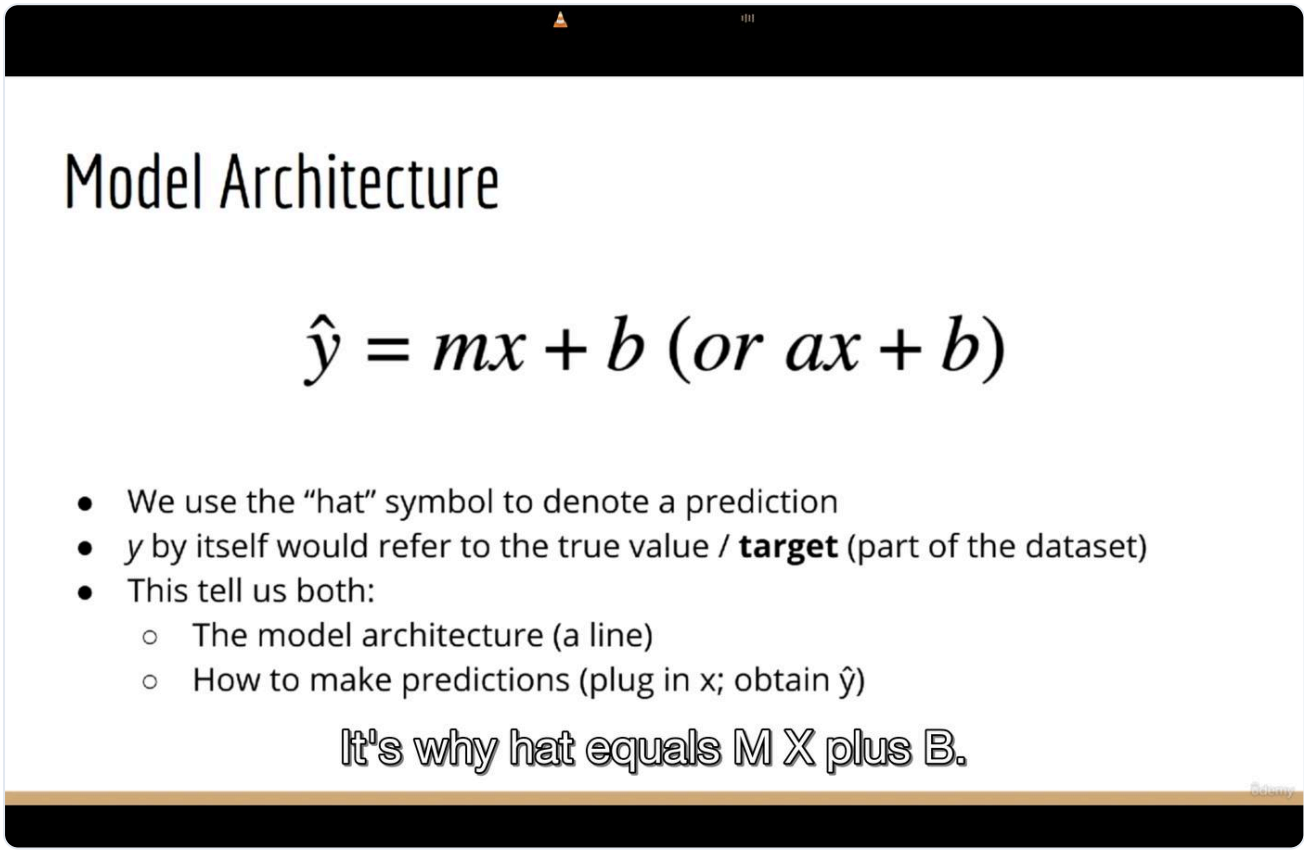


FIGURE 9.11

This not only defines the model architecture which is a line but it also tells us how to make predictions given some input x. I can see exactly how to calculate y hat. Pretty simple. So this is our model y hat equals M X plus b the hard part is number three. How do we actually find good values of the slope M and the intercept B.

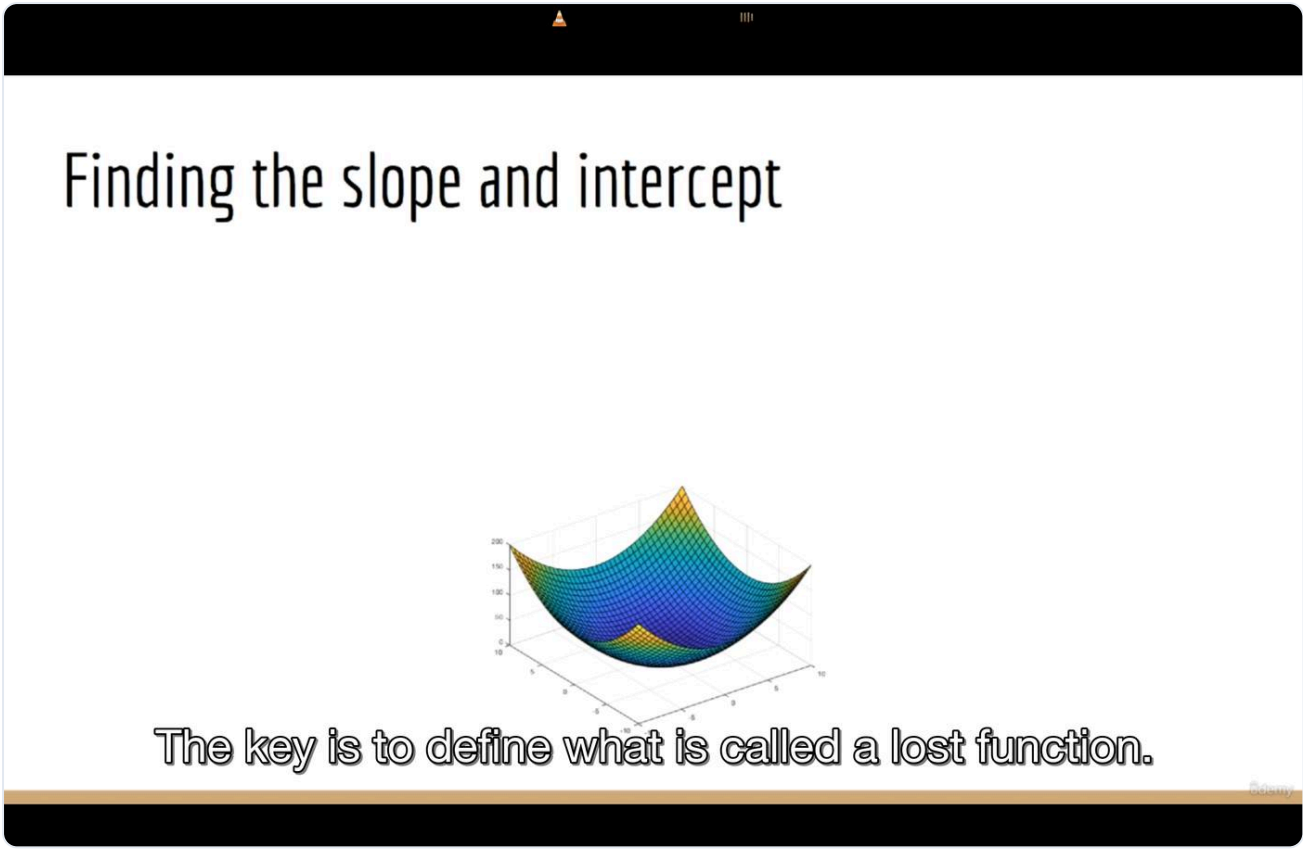


FIGURE 9.12 | Finding the Slope and Intercept

The key is to define what is called a loss function. I warn you that there are many synonyms for the loss function and you'll see them all. So don't be afraid if you see these different terms and think one of them might be wrong. That's not the case.

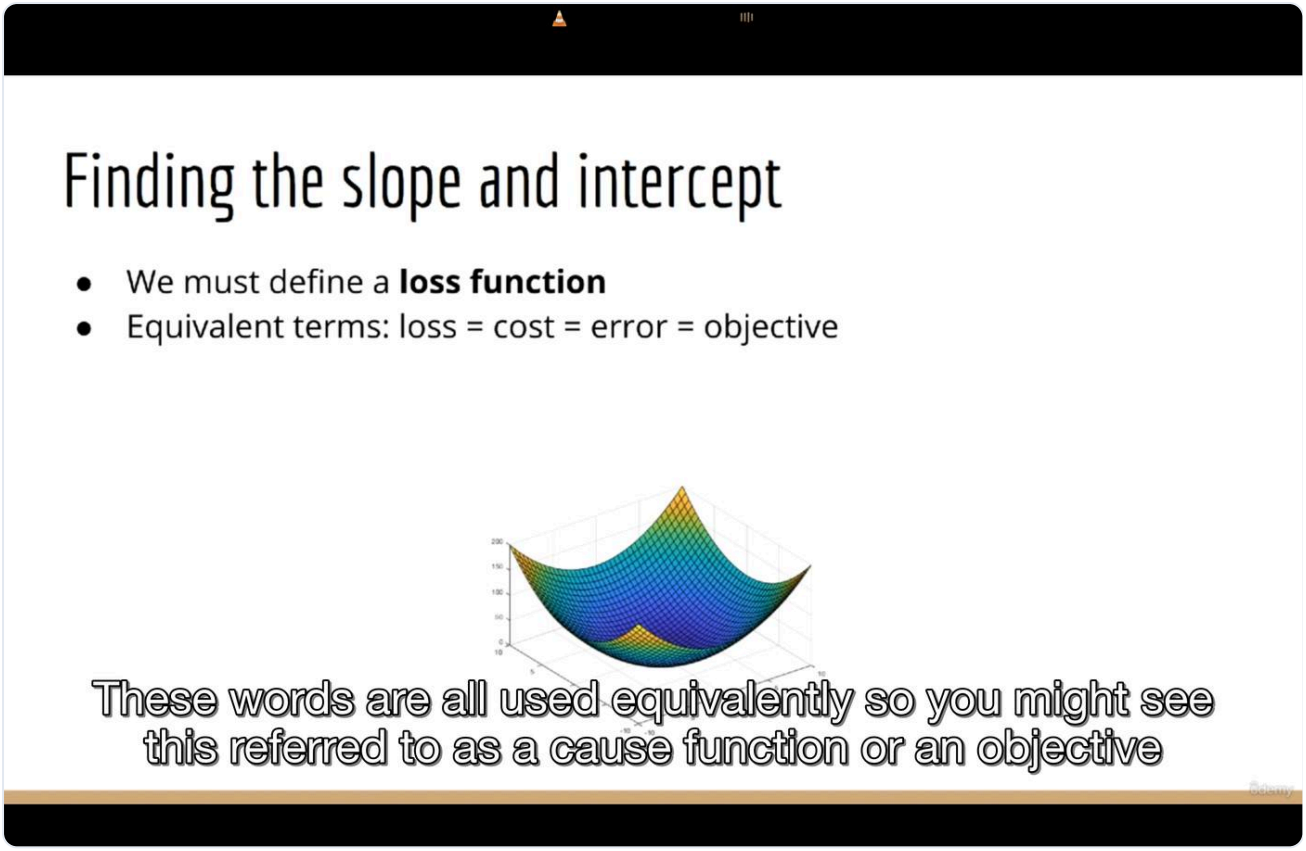


FIGURE 9.13 | Finding the Slope and Intercept

These words are all used equivalently so you might see this referred to as a cost function or an objective function or an error function. These all mean the same thing.

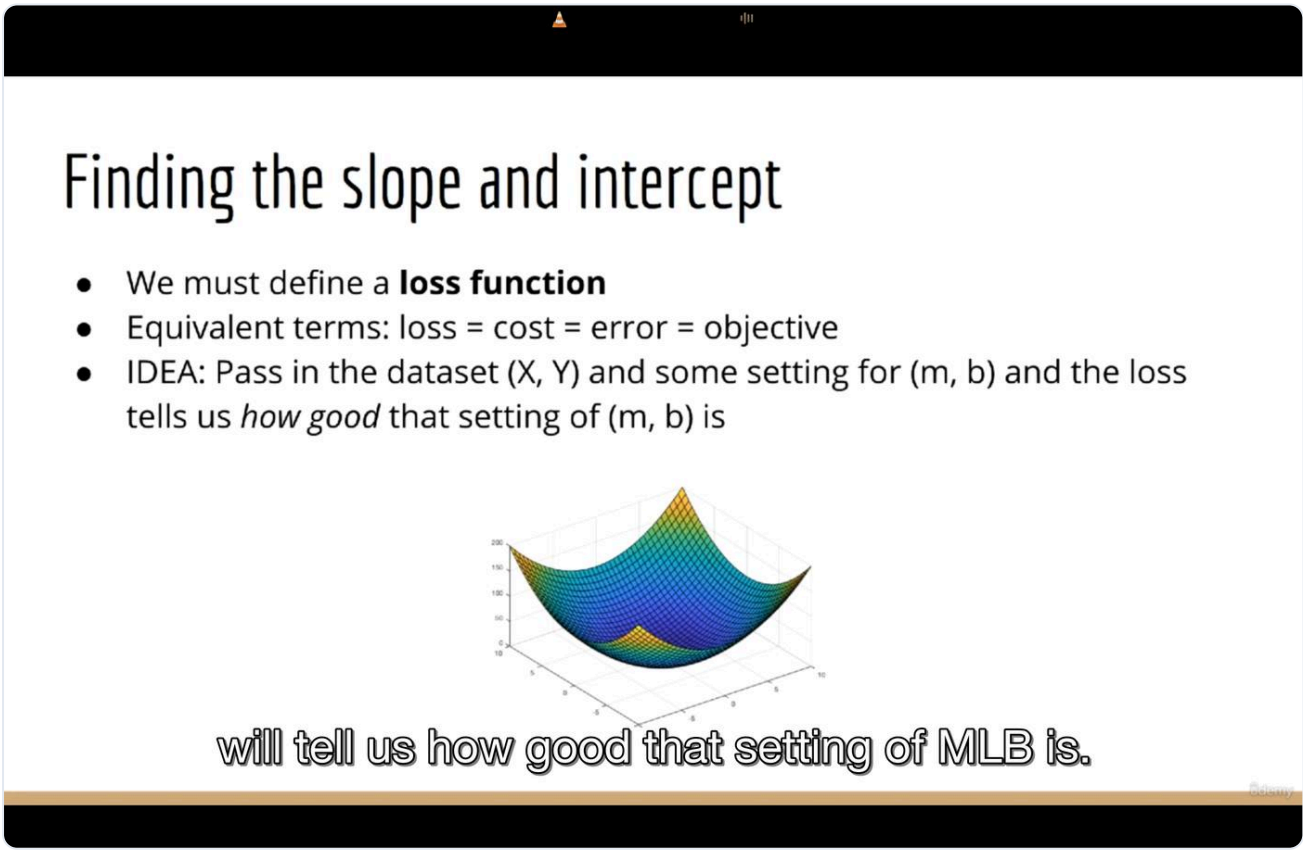


FIGURE 9.14 | Finding the Slope and Intercept

The basic idea is this if we pass in our data set X and Y along with some setting of M and B the loss function will tell us how good that setting of M and B is. Let's see how. First let's assume that we're given a set of pairs of data points X1 y 1 x 2 y 2 all the way up to X and Y N make a note to yourself now that it will be convention for us and many others to use the capital letter N to represent the number of samples in your dataset. This is not always the case but it's at the very least the convention I use and the convention and many others use. So if you see something different in the future or you've seen something different in the past don't freak out believe it or not.

This happens sometimes anyway.

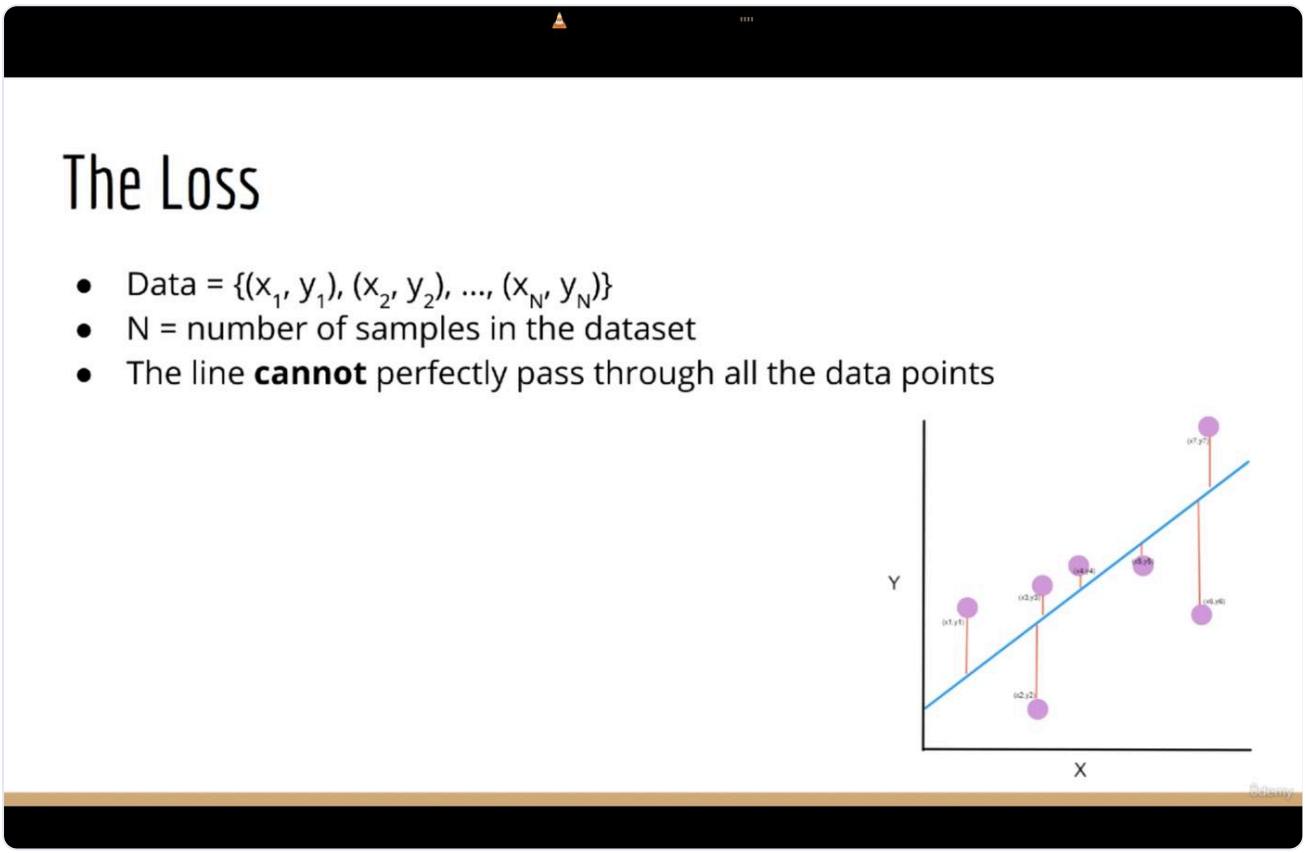


FIGURE 9.15 | The Loss

The idea behind linear regression is we will not find a line that perfectly passes through all the points that's impossible since as you can see there is no such line that can pass to all the points. All we can do is try to find a line that is a good fit. So how do we define a good fit. As I mentioned this is called a loss function. I'm going to start by telling you what it is and then describing to you why it makes sense for some of you it'll make sense immediately and for others it might not.

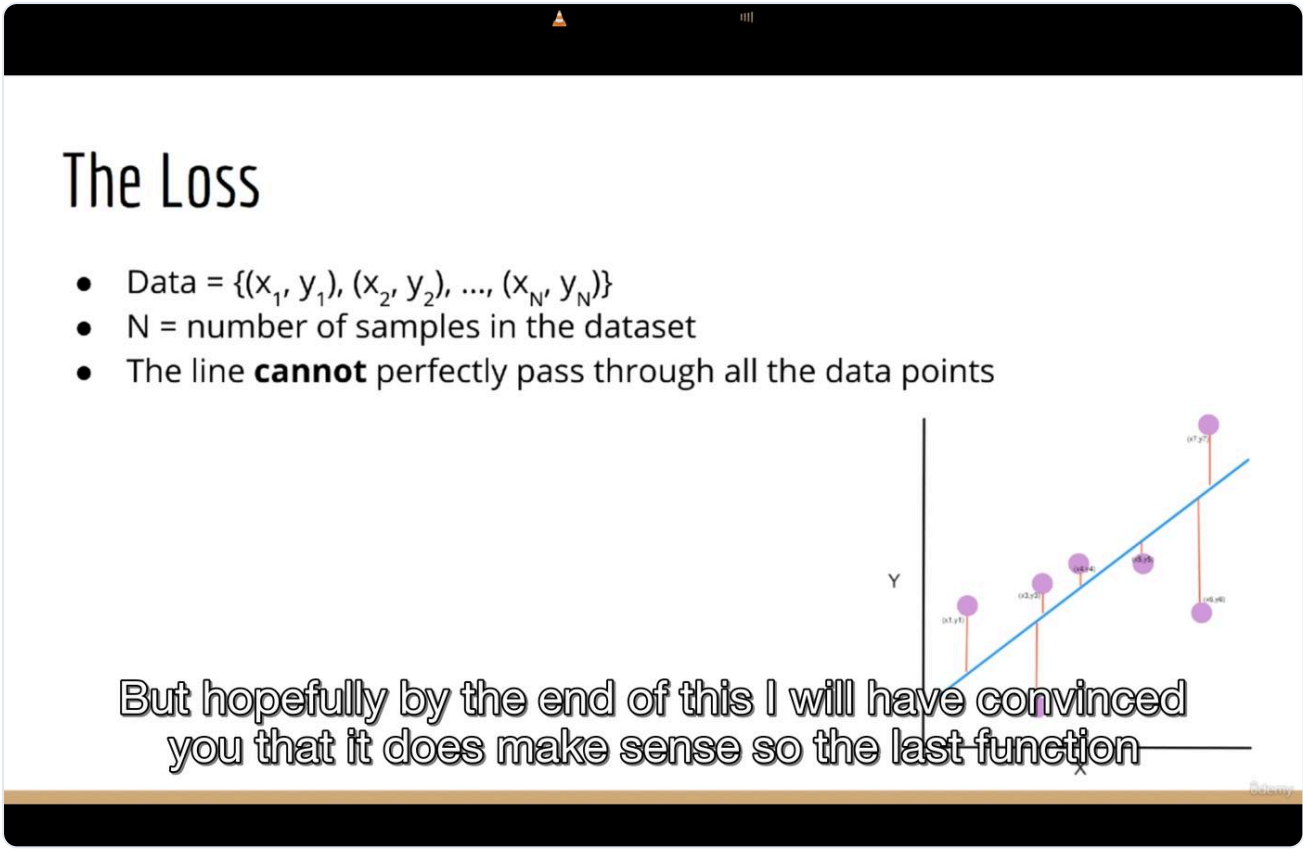


FIGURE 9.16 | The Loss

But hopefully by the end of this I will have convinced you that it does make sense so the loss function that we use in regression is called the mean squared error or MSE for short. Basically you take all the y I's and you find the squared difference between them and the line itself.

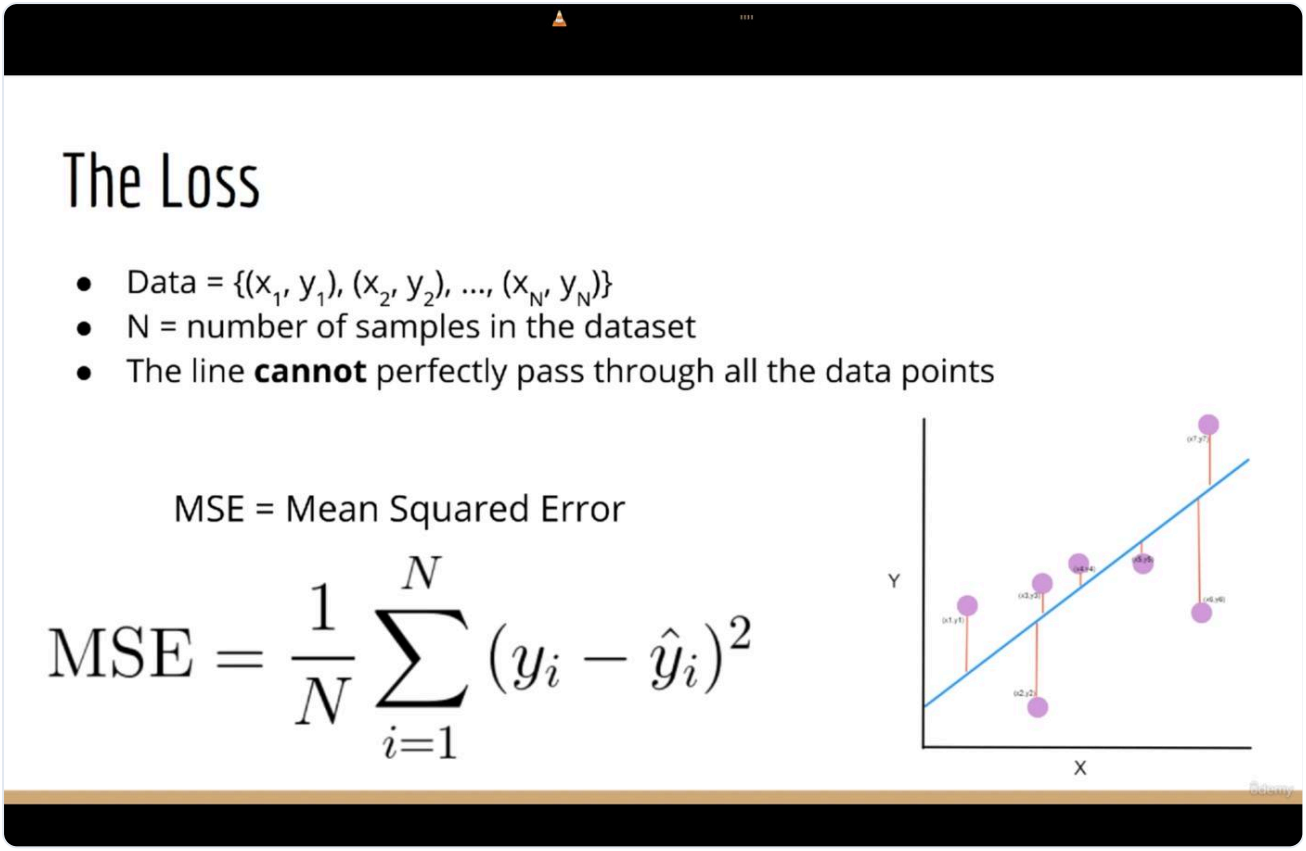


FIGURE 9.17 | The Loss

The y hat i's you can think of this as the squared vertical distance from each y i to the line each corresponding point on the line is a y hat i a prediction that given x y. Once we have all the squared differences we take their average you will know how to find averages so you know that is just the sum of the squared differences divided by the total number of points and the question is why does this make sense. I think this is a great exercise to try for yourself if you haven't done so before.

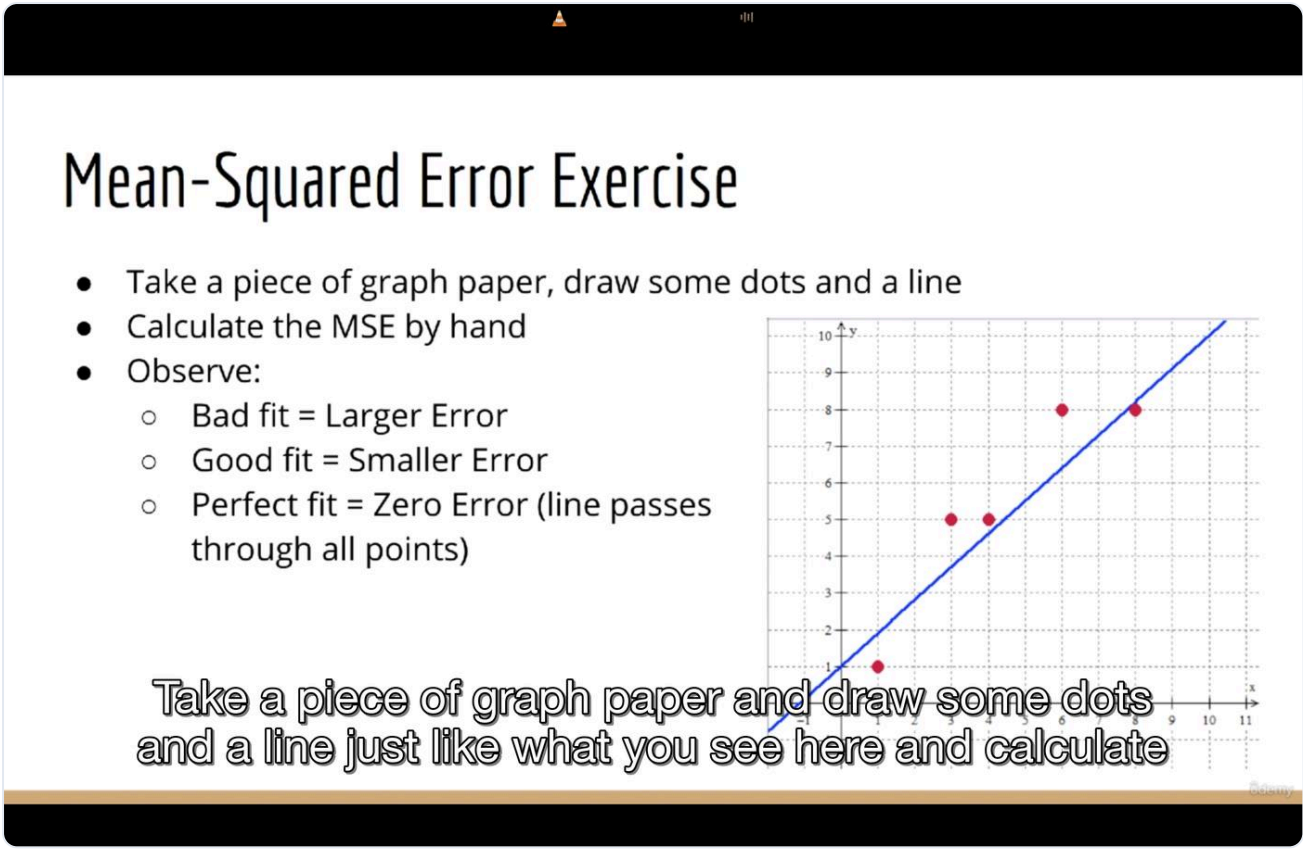


FIGURE 9.18

Take a piece of graph paper and draw some dots and a line just like what you see here and calculate the squared error by hand. What you should discover is that if the line is not a good fit then the error will be larger. If the line is a good fit then the error will be smaller. I think that's pretty intuitive. In addition you should recognize that if the data points perfectly lie on the line then the mean squared error will be 0.

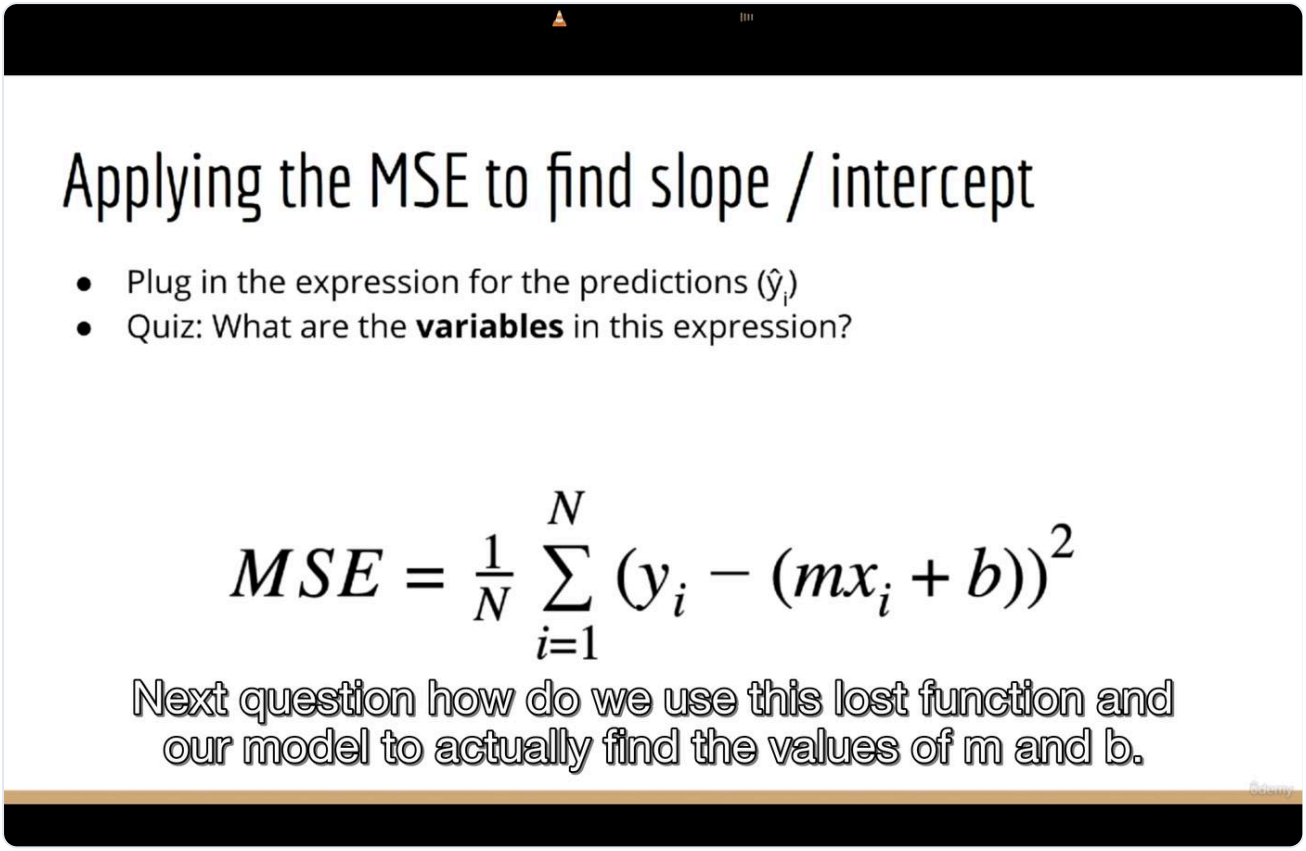


FIGURE 9.19 | Applying the MSE to Find Slope / Intercept

Next question how do we use this loss function and our model to actually find the values of m and b. Let's see what happens if we plug in our expression for y hat into the loss. This is clearly the equation we would get. Now I have a quiz question for you. What are the variables in this equation. I'll give you a second to think about it. So please take a moment to come up with the answer before moving on to the next slide. Ok so I hope you thought about the answer to what the variables are in this equation. If you said x and y. This is not correct.

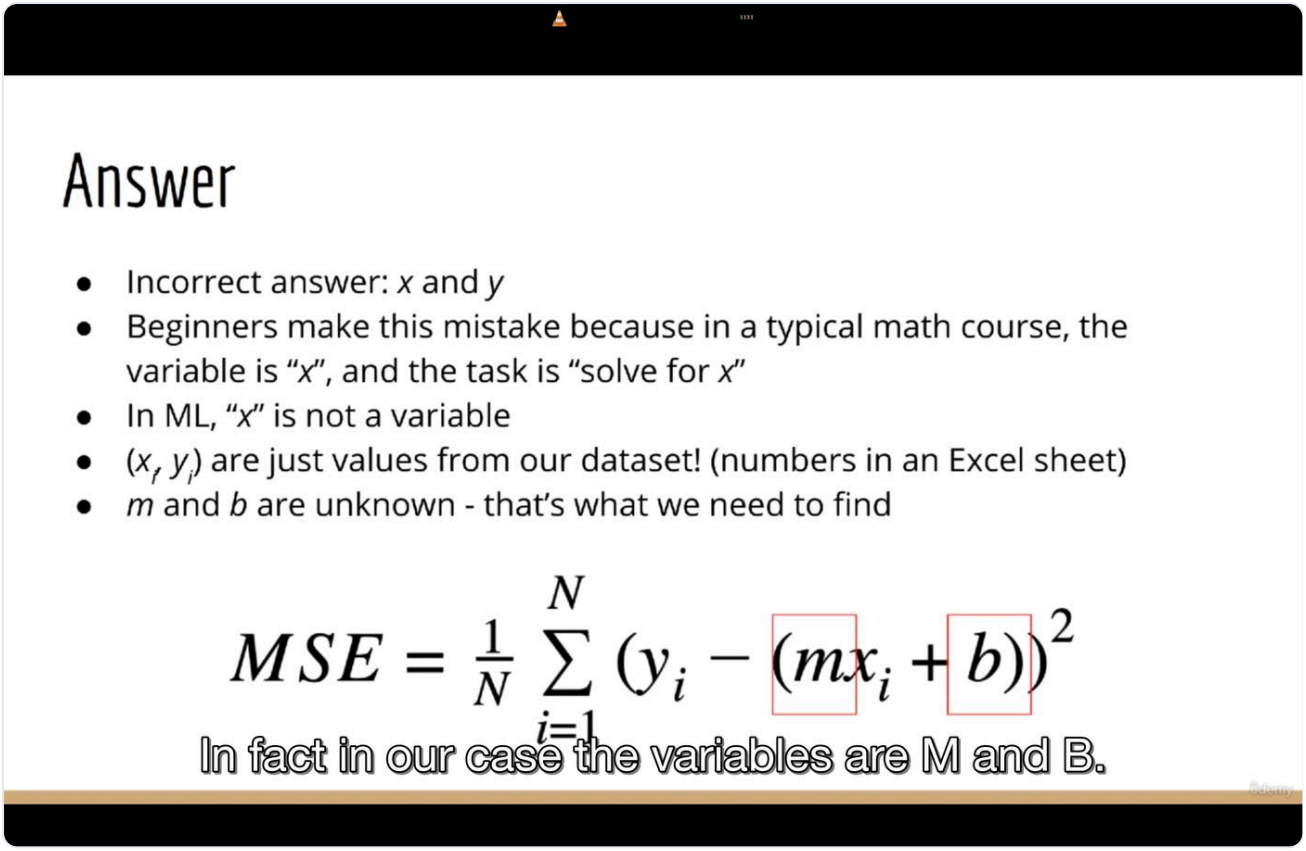


FIGURE 9.20

Many beginners make this mistake because in a typical math course the variable is X and you're always trying to do something like solve for x. However it's important to recognize that in machine learning X is normally not a variable X Y and Y are just values from my dataset. You can think of them as numbers from your Excel spreadsheet. In fact in our case the variables are M and B. Remember that these are the values that we are currently trying to find. They are unknown and therefore they are variables at this point.

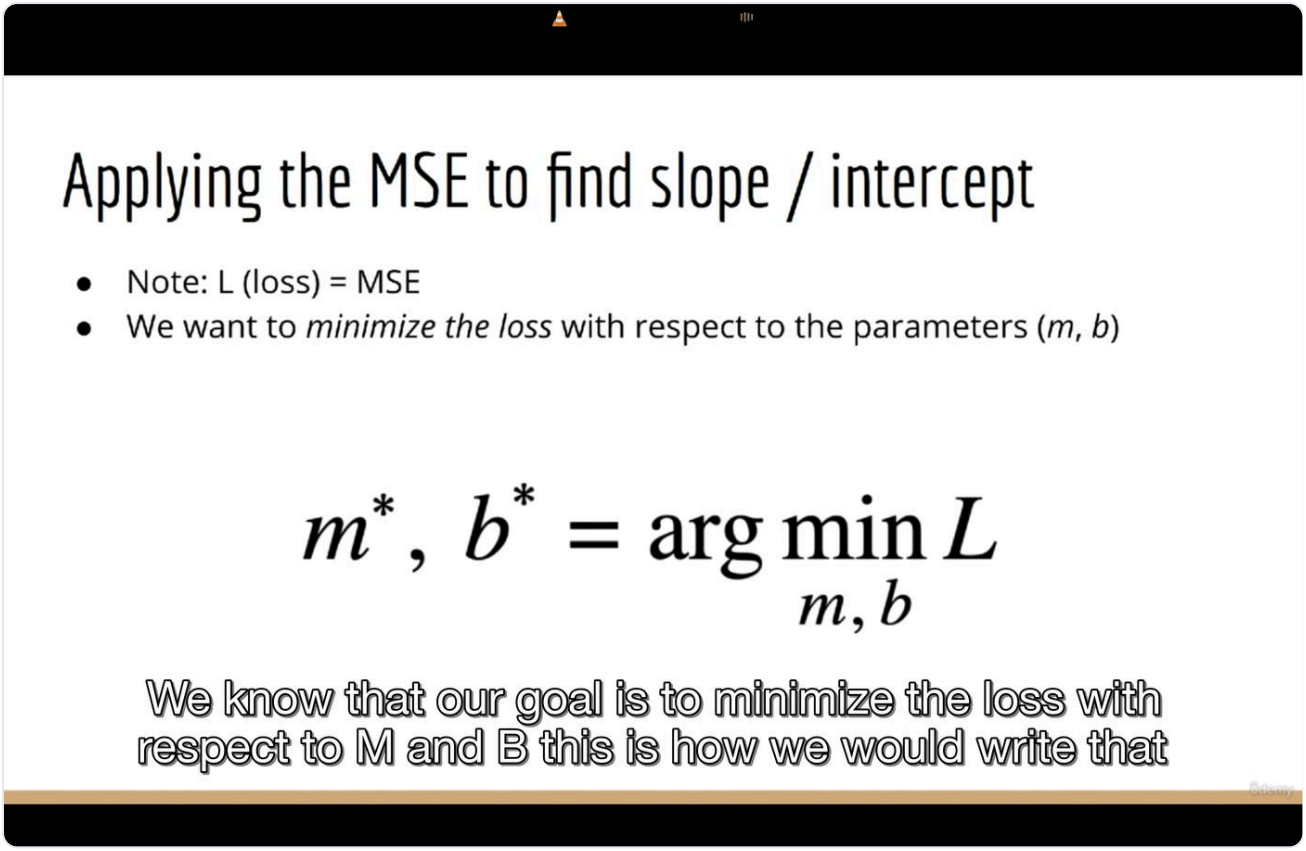


FIGURE 9.21 | Applying the MSE to Find Slope / Intercept

We know that our goal is to minimize the loss with respect to M and B this is how we would write that mathematically. If you're afraid of math don't worry about this equation too much because it doesn't give us anything new. It's just a way of saying what we already said but using math we would like to find the best values of m and be called M star and B star where these values of m and b are the ones that minimize the loss l out of all possible values of m and b how do we solve such a problem. It might help to consider a similar but simply problem say we have a function f of x equals X squared. A quadratic. We all know this from high school. You might say well I can just look at the graph and see that the minimum value is zero when x is zero.

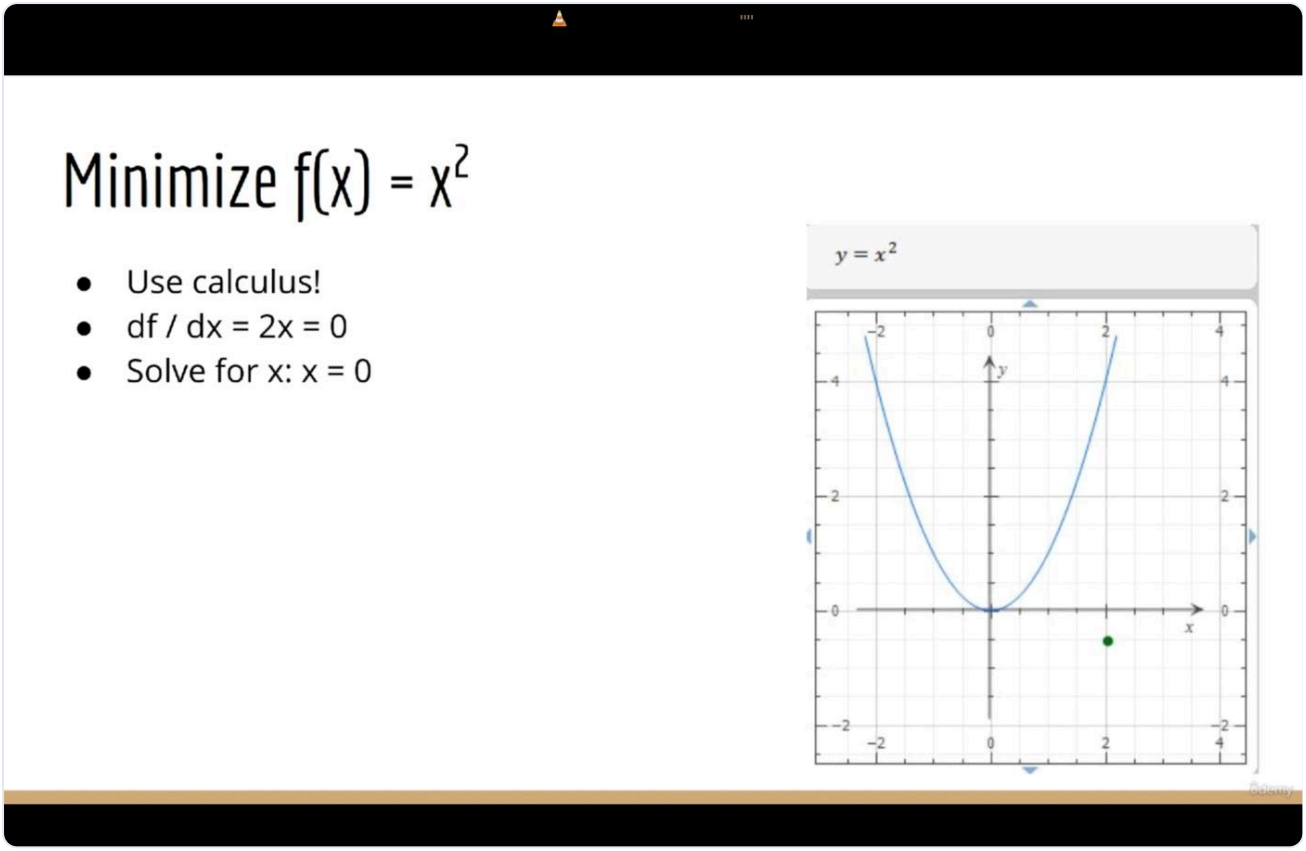


FIGURE 9.22

But remember we don't do that in machine learning because in machine learning we work in multiple dimensions. You can't just look at it at this point we have to introduce our old friend calculus. If you recall the way that we solve this is you find the derivative df by dx set it equal to zero and solve for x. This is because as you can see the derivative at the minimum is equal to zero so if you draw a tangent line at the minimum point it should be a horizontal line. There are of course a lot more details that I'm not including here but you should at least remember this from your high school math studies. So how do we find the solution in our case since we have two variables m and b we have to take partial derivatives but the idea is the same find the partial derivative of L with respect to M and set that to zero.

Find the derivative of L with respect to B and set that to zero.

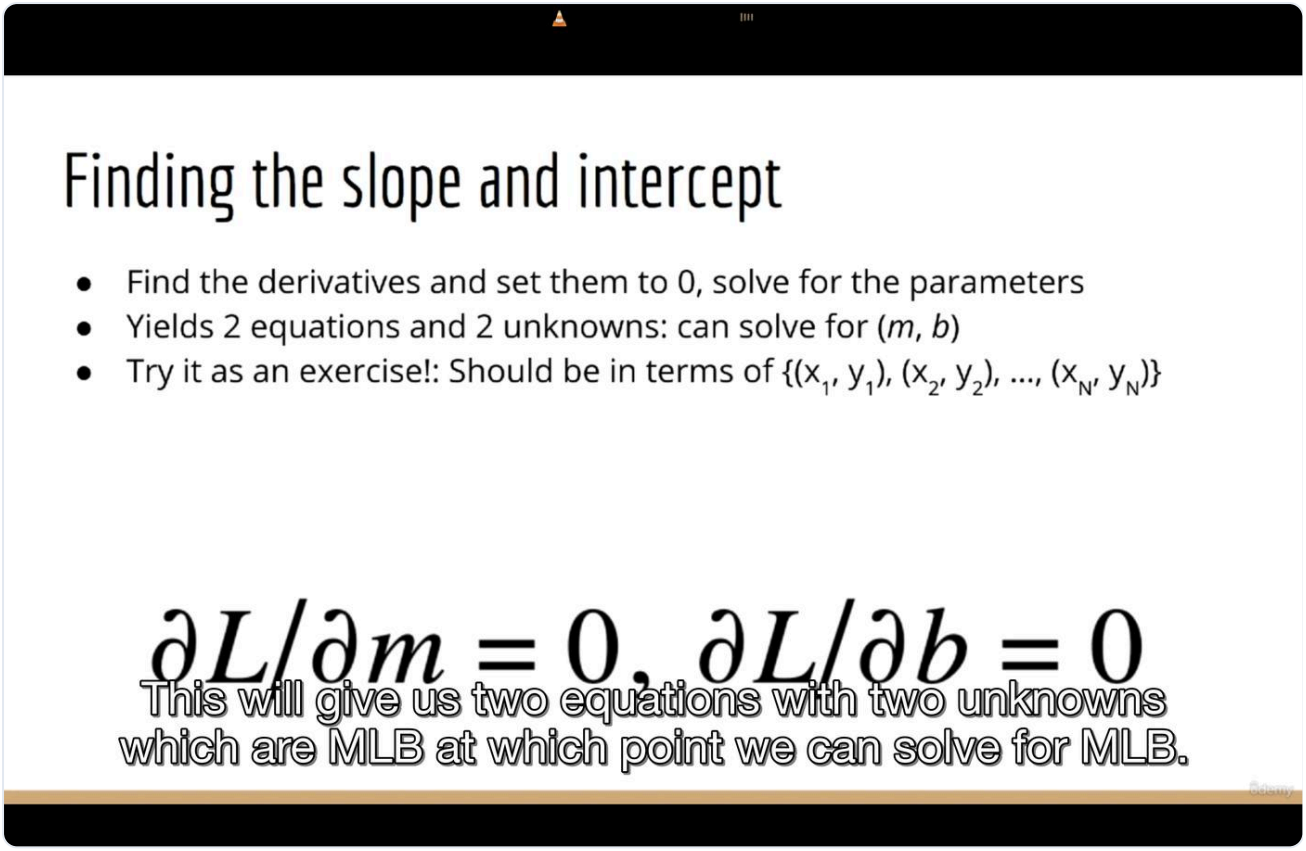


FIGURE 9.23 | Finding the Slope and Intercept

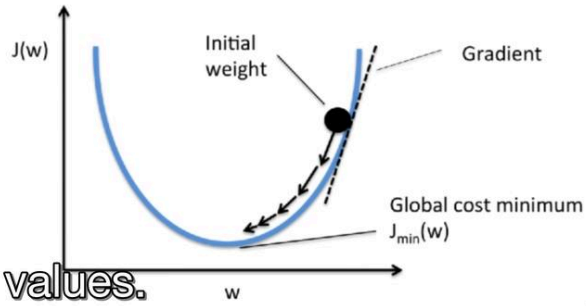
This will give us two equations with two unknowns which are M and B at which point we can solve for M and B. In fact if you've never done this before I would encourage you to do this calculation and solve for M and B as an exercise. This can be done for any arbitrary data set X1 y one up to X and Y N So your solution should be in terms of those values. There is a problem with this approach however unfortunately finding the derivatives and setting them to zero only works for linear regression.

Problem!

- The approach (find derivative, set it to 0) works **only** for linear regression, but not any other model we will learn in this course (e.g. linear classification)

```
# gradient descent
# pseudocode

θ = (m, b) = random()
for i in range(n_epochs):
    θ = θ - η∇_θ L
```



of those values.

FIGURE 9.24

Problem!

- The approach (find derivative, set it to 0) works **only** for linear regression, but not any other model we will learn in this course (e.g. linear classification)

```
# gradient descent
# pseudocode

θ = (m, b) = random()
for i in range(n_epochs):
    θ = θ - η∇_θ L
```

J(w)
Initial weight
Gradient
Global cost minimum
$J_{min}(w)$
w

FIGURE 9.25

It does not work for any other model we will discuss in this course. In fact the very next model we discuss for linear classification cannot be solved in this way. Instead we need to use a method called gradient descent. Very briefly the gradient is the multi-dimensional analogue of the derivative. So basically when you're working in multiple dimensions you have gradients in place of scalar derivatives the idea behind gradient descent can be understood with this simple picture. It's an iterative algorithm. So we're going to go in a loop on each iteration of the loop. We're going to take a small step in the direction of the gradient. It's been mathematically proven that if we keep doing this we will eventually end up at the minimum point in this lecture.

I don't want to get into detail about gradient descent. However you are encouraged to check out the lectures in the in-depth section of this course. If you are interested. Our goal is to get to how we do this in PyTorch as quickly as possible. Gradient descent is a really cool algorithm because it works in pretty much all cases. That's why libraries that have automatic differentiation such as Theano TensorFlow and PyTorch have been so popular for machine learning aside from deep learning. You can use the exact same techniques for things like Hidden Markov models.

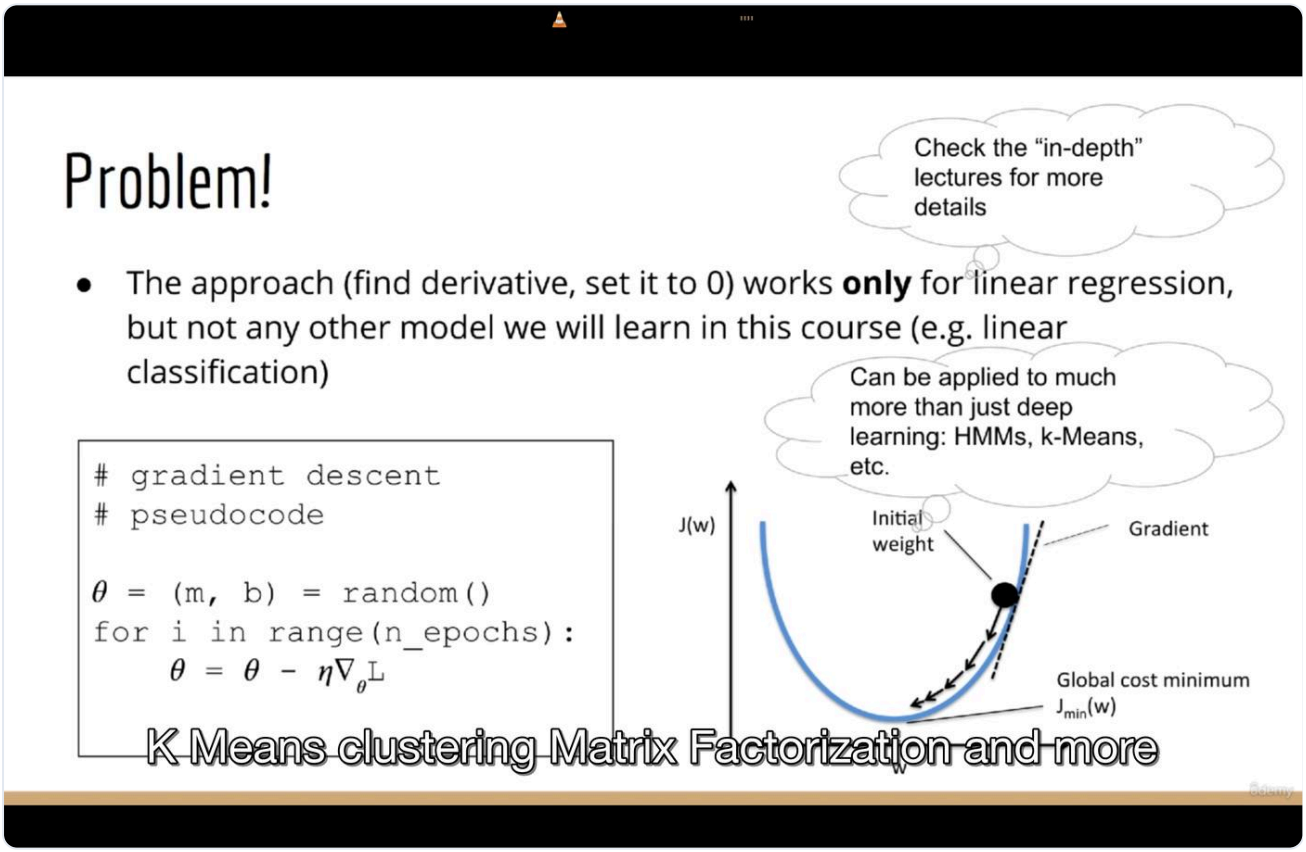


FIGURE 9.26

K Means clustering Matrix Factorization and more all right. So since this has been a pretty long lecture let's recap the steps.

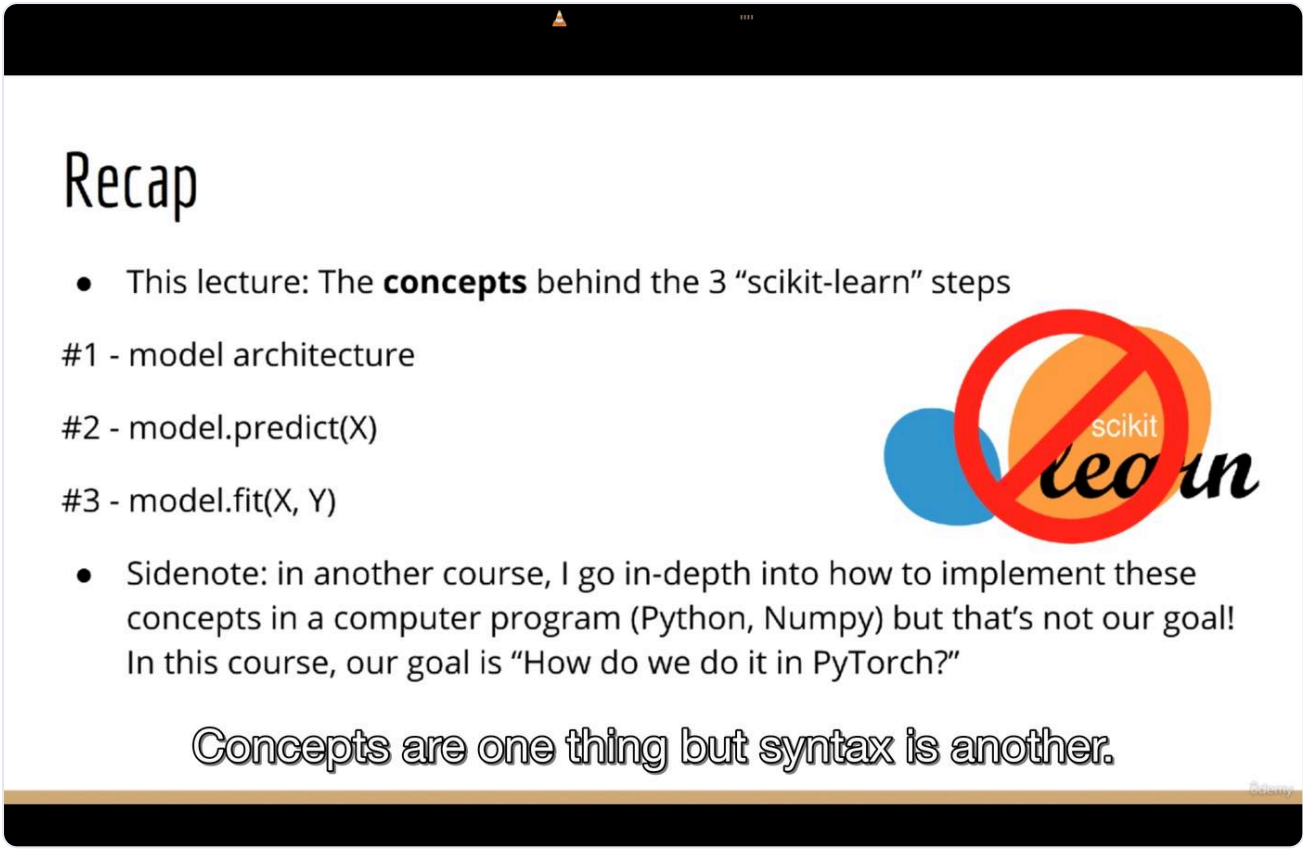


FIGURE 9.27

Our goal in this lecture was to take you from the three steps of the scikit-learn API to what actually happens conceptually as you recall the three steps are creating the model fitting the model and making predictions with the model. After this lecture you now understand at a high level what actually happens during these three function calls. However the concepts don't provide the full picture because we still need to implement this in code. In fact I have a course that goes very in-depth into all these concepts and how you can take them and put them into a computer program using basic libraries like NumPy in the Python programming language. But remember this course is about PyTorch.

So the next question we have to ask is how is this done in PyTorch. Concepts are one thing but syntax is another. For example the syntax for PyTorch will be different from the syntax for TensorFlow. Even if we are doing the exact same thing.

LECTURE 10

Train the Model (Gradient Descent Loop)

In this lecture we are going to do what I call code preparation.

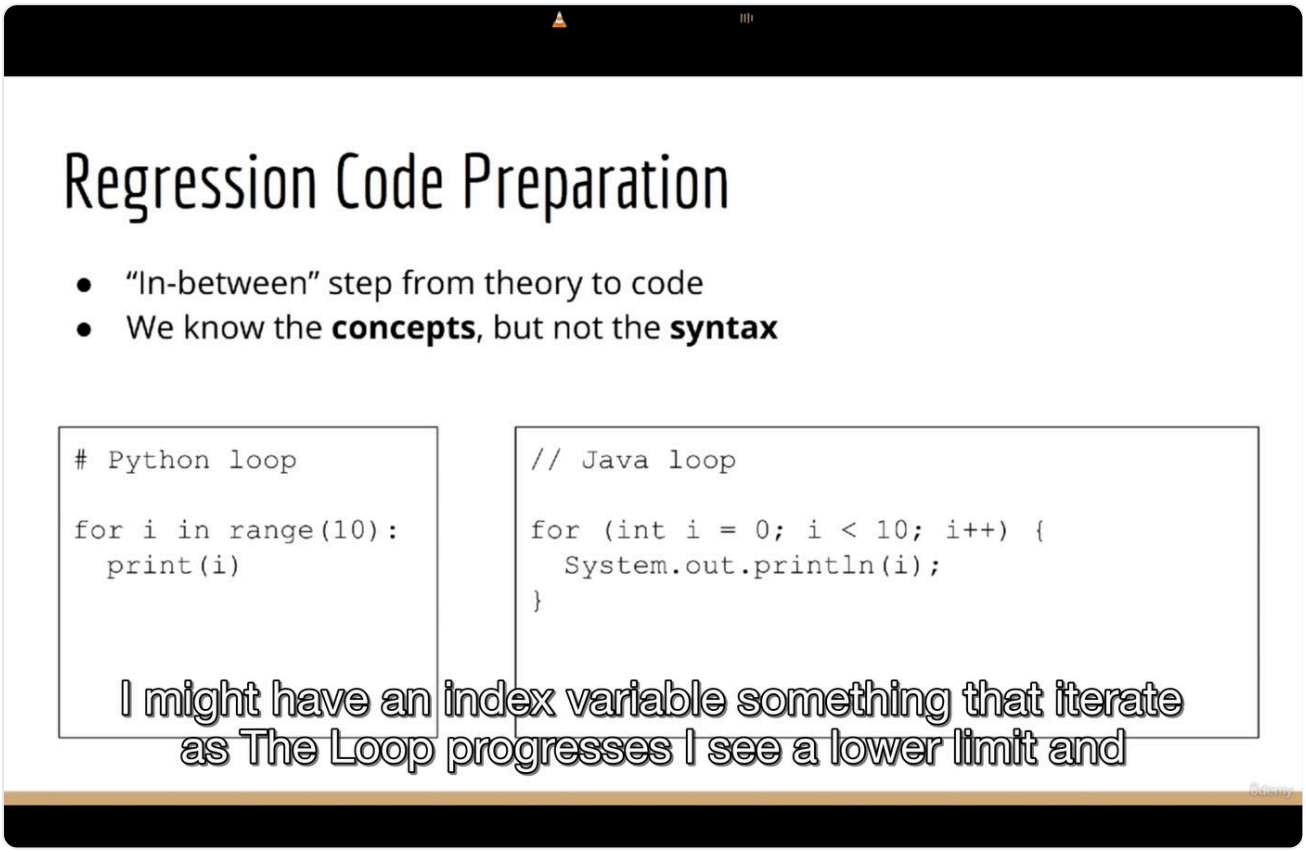


FIGURE 10.1 | Regression Code Preparation

It's like the in between step that will help us get from theory to code so we know what each of the steps are and we know conceptually how they work. The next question is what's the right syntax in PyTorch to actually make it work. I like to think of this like how I think of learning programming languages. I hope everyone in this class knows Python since that's the language we'll be using. So let's say what I want to do is print the numbers from zero to nine inclusive. I know how to do that in Python. It's just for i in range 10 ten print I but let's say that now I want to learn how to do this in Java. If you know Java. Just pretend that you don't for a moment.

What's the situation now. Well I understand the concept. The only challenges. What is the Java syntax for it. I know the concept but I don't know the syntax. So let's look at the syntax looking at this all the components should feel very familiar to you. Even if you've never seen it before. I can see the keyword for that's the same in both languages. That tells me I'm doing a for loop. Same concept different syntax. I see the index variable I again it's the same concept in both languages in a for loop. I might have an index variable something that iterate as The Loop progresses I see a lower limit and an upper limit the lower limit is zero and the upper limit is 10 which are inclusive and exclusive respectively.

Same thing in Python except that the lower limit is implicitly zero in Python so you don't have to specify it unless you want something other than zero. Finally I see that inside the loop we again have the same concept printing but again just with a different syntax. So I hope you get the analogy here which is that no one concepts and syntax are separate things. You might know the concepts but in different libraries the syntax will be different.

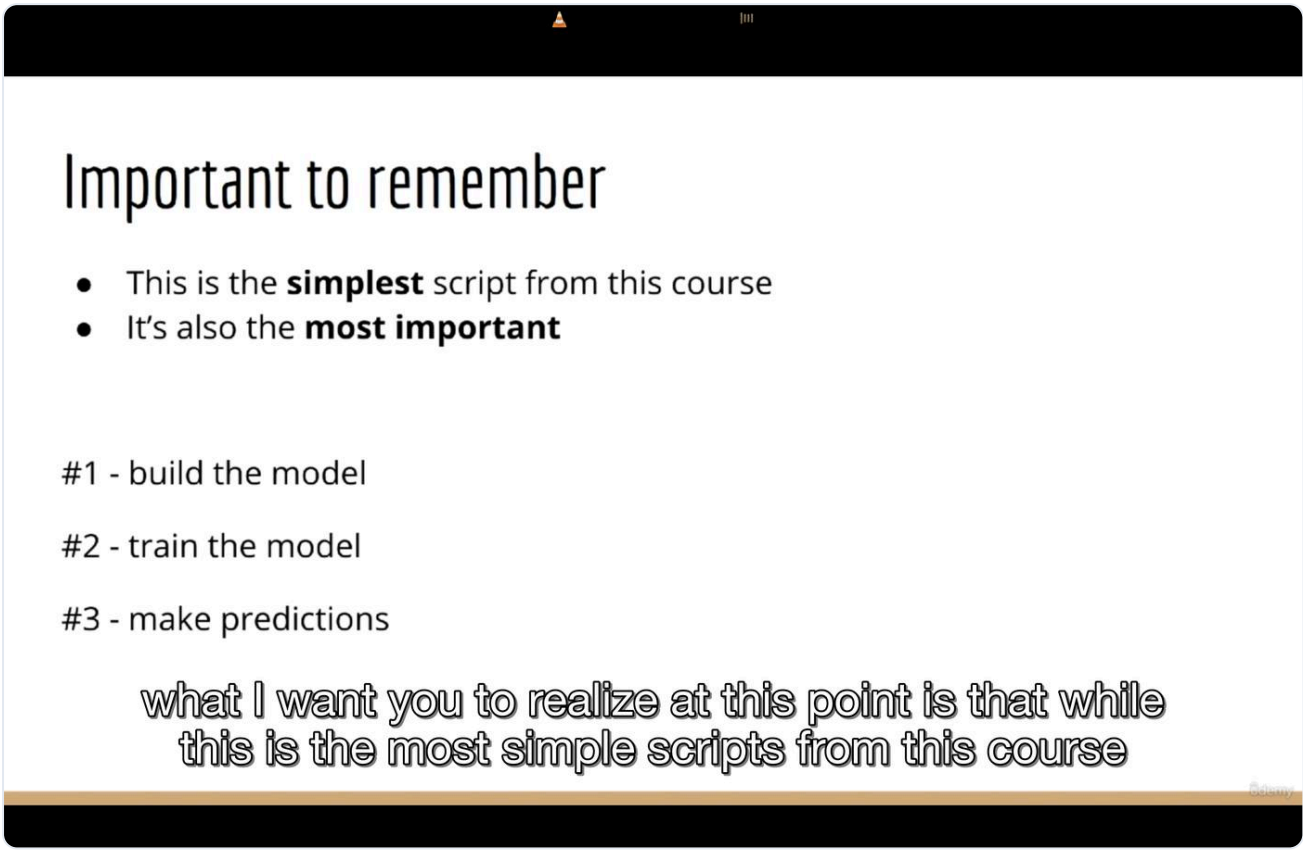


FIGURE 10.2

Number two since we already know the concepts our next task is to ask what is the syntax what I want you to realize at this point is that while this is the most simple scripts from this course it is also the most important you're going to notice a pattern in this course which is that no matter how complex what we do is these steps will essentially remain the same. Remember that what we want to focus on right now is number one. How do we build the model. Number two. How do we train the model. And number three how do we make predictions using the model let's start with number one. How do we build the model. At this stage it's going to be very easy. Just one line of code model equals nn dot Linear one one.

What this means is that we want a linear model with one input and one output.

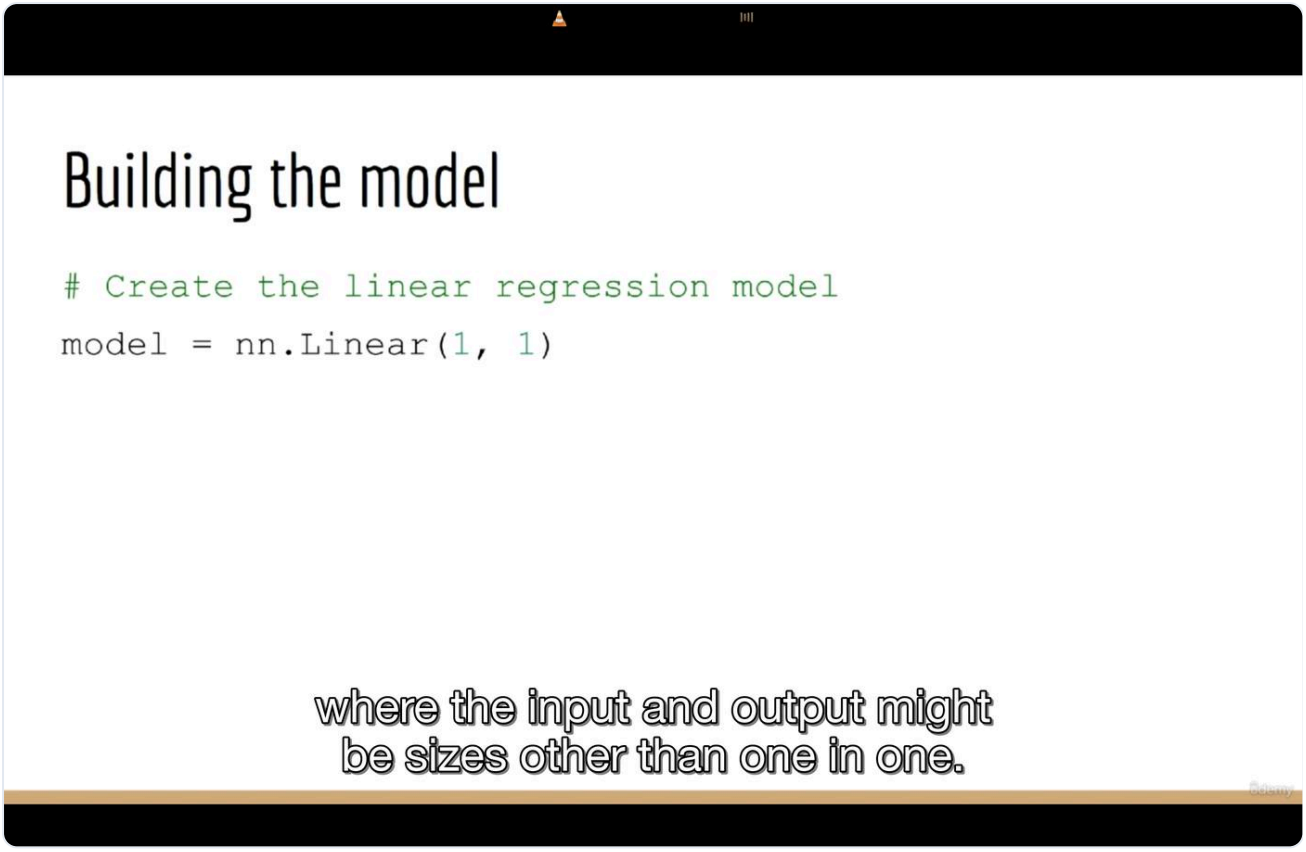


FIGURE 10.3

The first argument is for the input size and the second argument is for the output size there are cases where the input and output might be sizes other than one in one. But we'll see those later. Right now we're interested in doing simple linear regression where there is one x axis and one Y axis next step. Number two how do we train the model. If you've ever used other libraries such as Keras and scikit-learn you might assume this would be easy in PyTorch. But unfortunately it doesn't encapsulate all the steps as you would expect. Luckily thanks to the concepts we just learn that will help us in understanding the syntax.

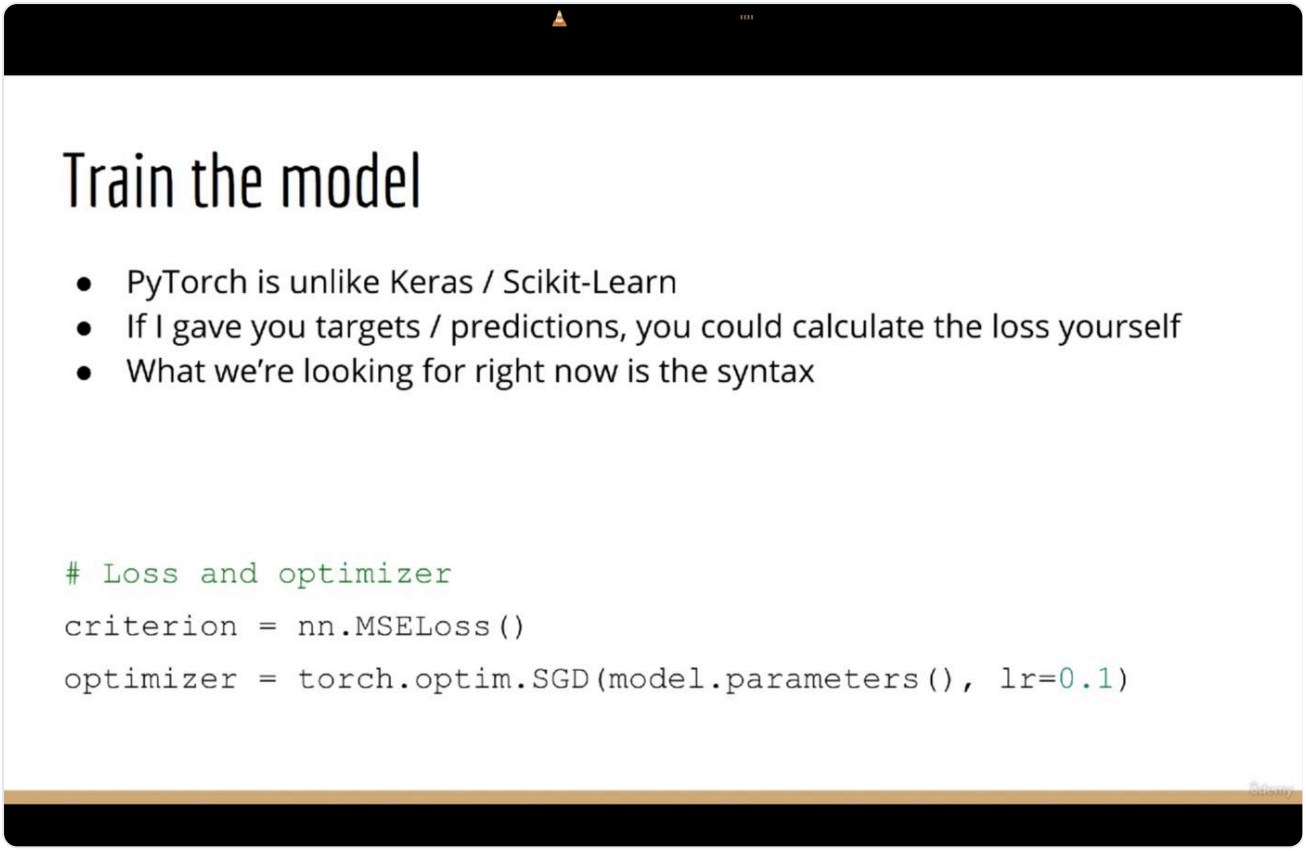


FIGURE 10.4 | Train the Model

The first step in training the model is to define a loss and an optimizer as we discussed the loss that we need for linear regression is the mean squared error and there just so happens to be this object called MSE loss in PyTorch. Typically we call this object a criterion. There's one interesting thing to notice here which is that this is a bit of an exercise in what I call API hunting. If you think only about the concepts you know the equation for the MSE loss. If I gave you the y's and the y hats you could calculate this loss yourself but in PyTorch it's not a matter of knowing the formula. It's a matter of knowing the syntax.

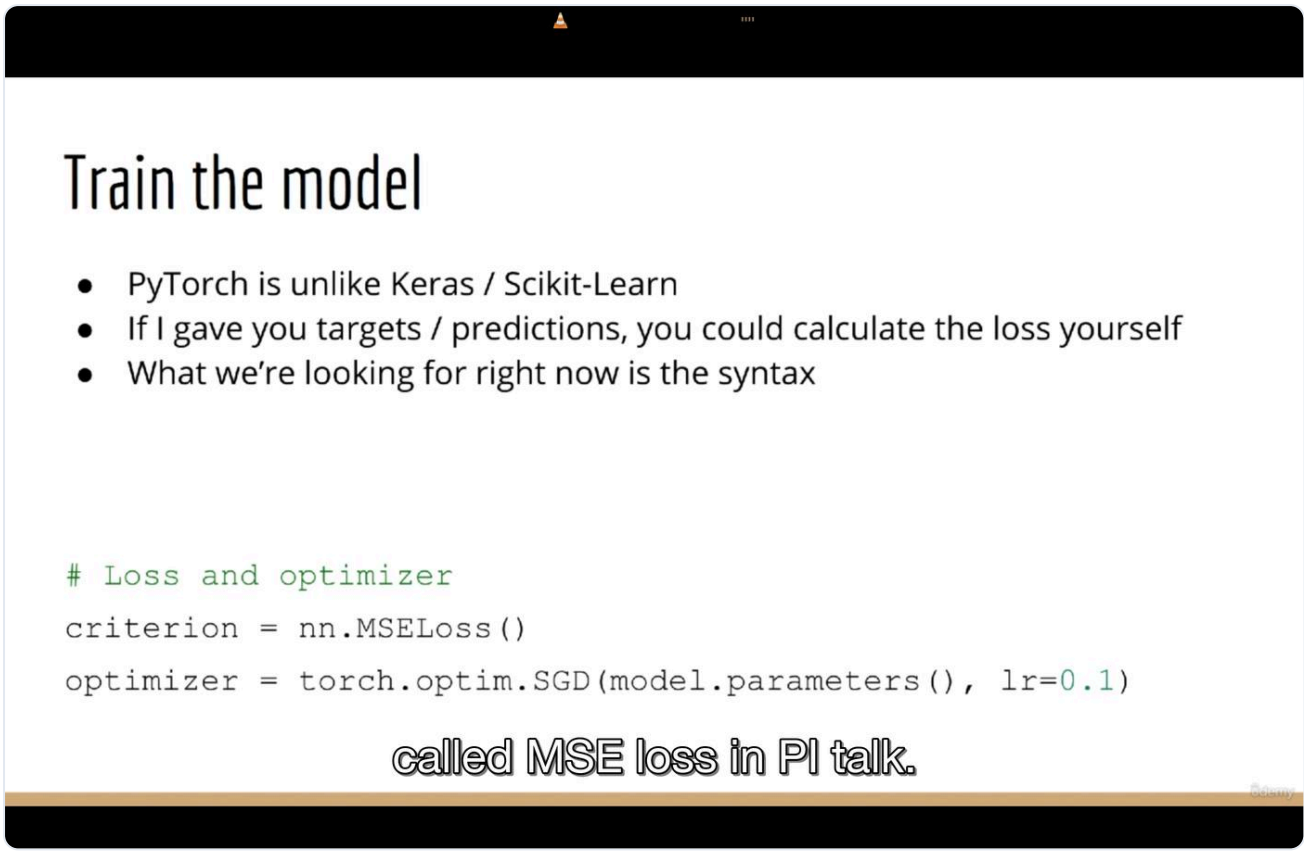


FIGURE 10.5 | Train the Model

So you need to find the right object that implements the loss you were looking for and then do the right things with that object at the same time. It's nice to know a little bit about the concepts so that you actually have some sense of what's going on. Next we have the optimizer. There are many such optimizers in PyTorch but the simplest one is called SGD. When we create the SGD object we pass in two arguments the models parameters which we get by calling the convenient function model dot parameters. And the learning rate as a side note SGD stands for stochastic gradient descent again in these lectures I'm not so concerned about what's going on mathematically but rather the syntax.

If you'd like to understand what's going on mathematically I would encourage you to check out the in-depth sections of this course All right.

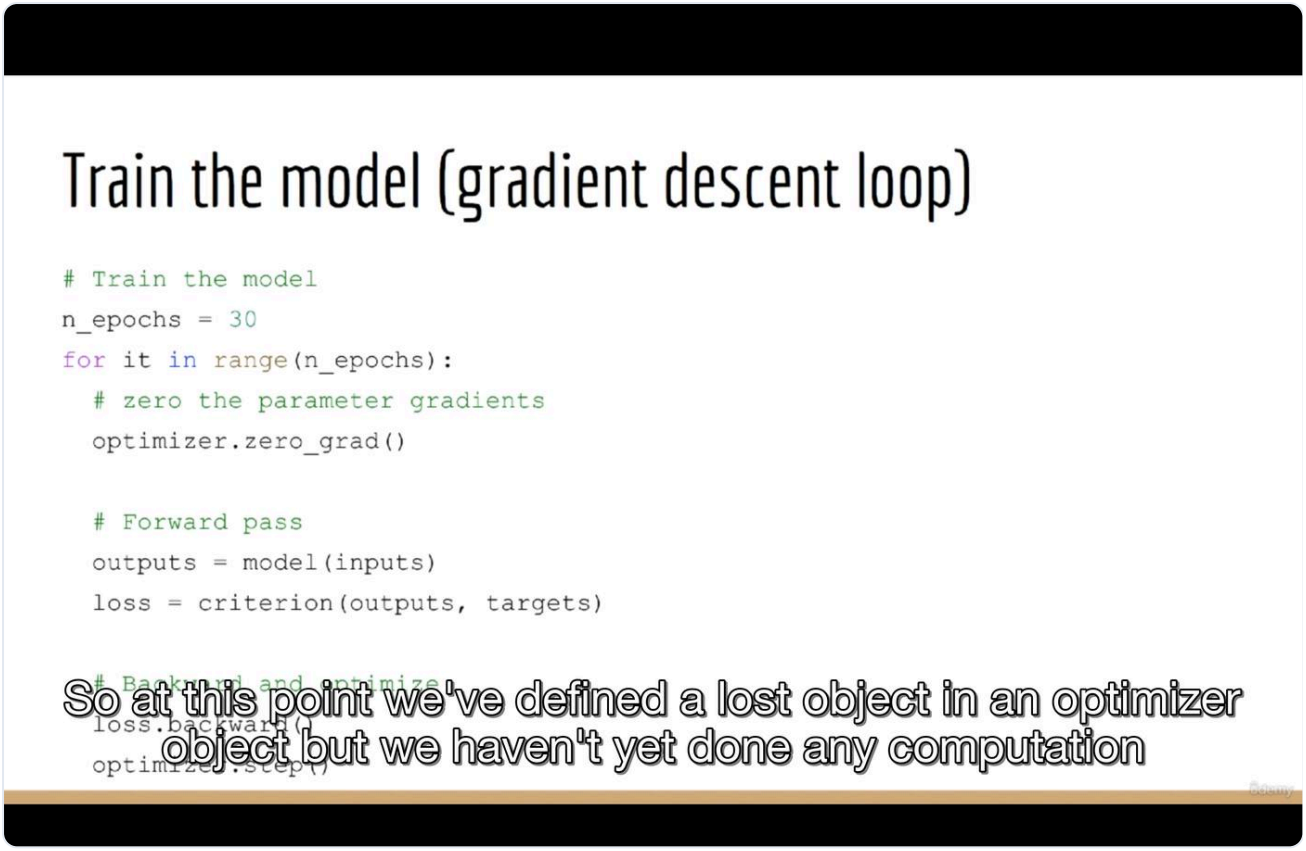


FIGURE 10.6 | Train the Model (Gradient Descent Loop)

So at this point we've defined a loss object and an optimizer object but we haven't yet done any computation with them. As you recall gradient descent involves doing a loop in inside the loop. You repeatedly apply the gradient descent equation so that's essentially what we want to do right now. But in PyTorch. So obviously we need a loop.

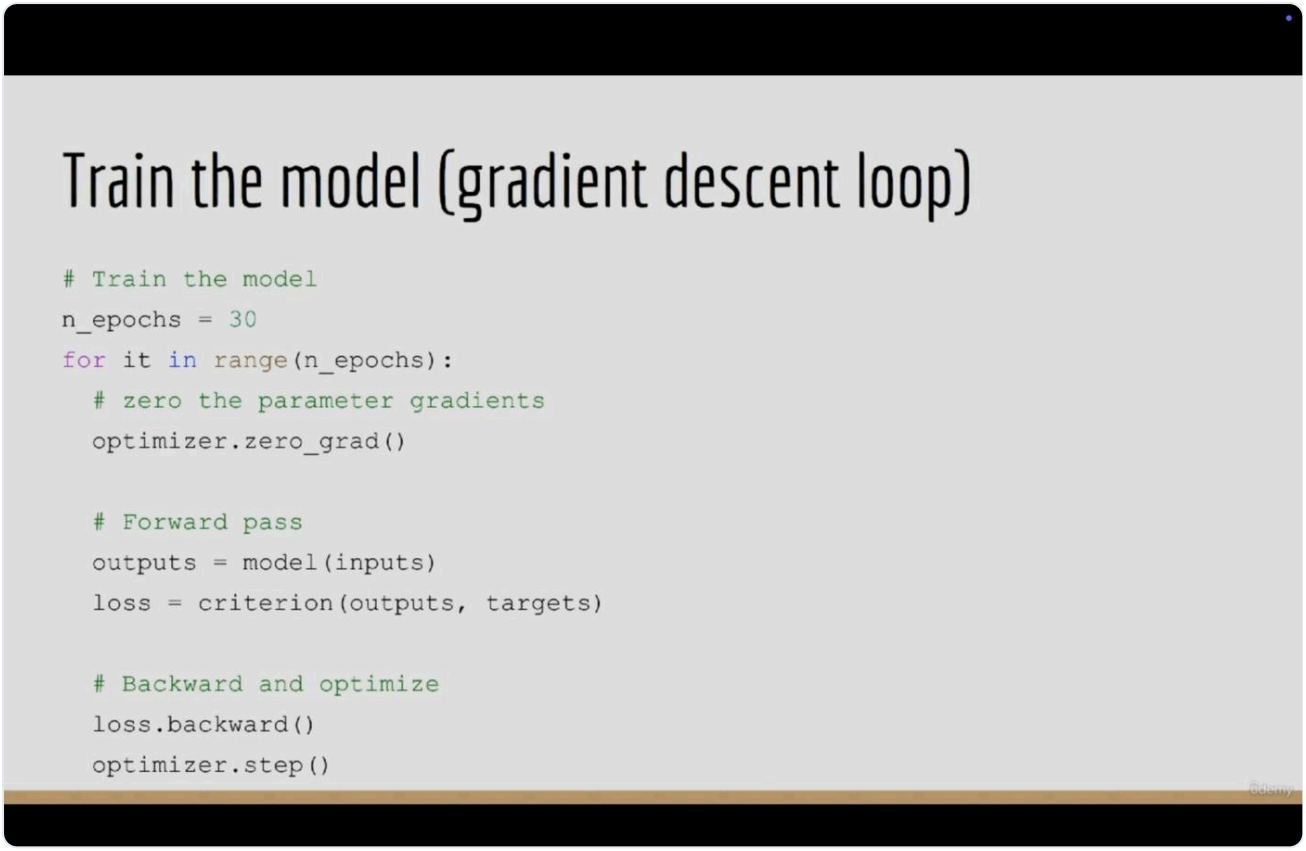


FIGURE 10.7 | Train the Model (Gradient Descent Loop)

Usually we call each iteration of the loop an epoch. Unfortunately it's not so easy as simply translating this one line of mathematical expression into one line of code. There are multiple steps in the code that have to be done. The weirdest part just happens to come first. We have to call optimizer dot zero grad at this stage.

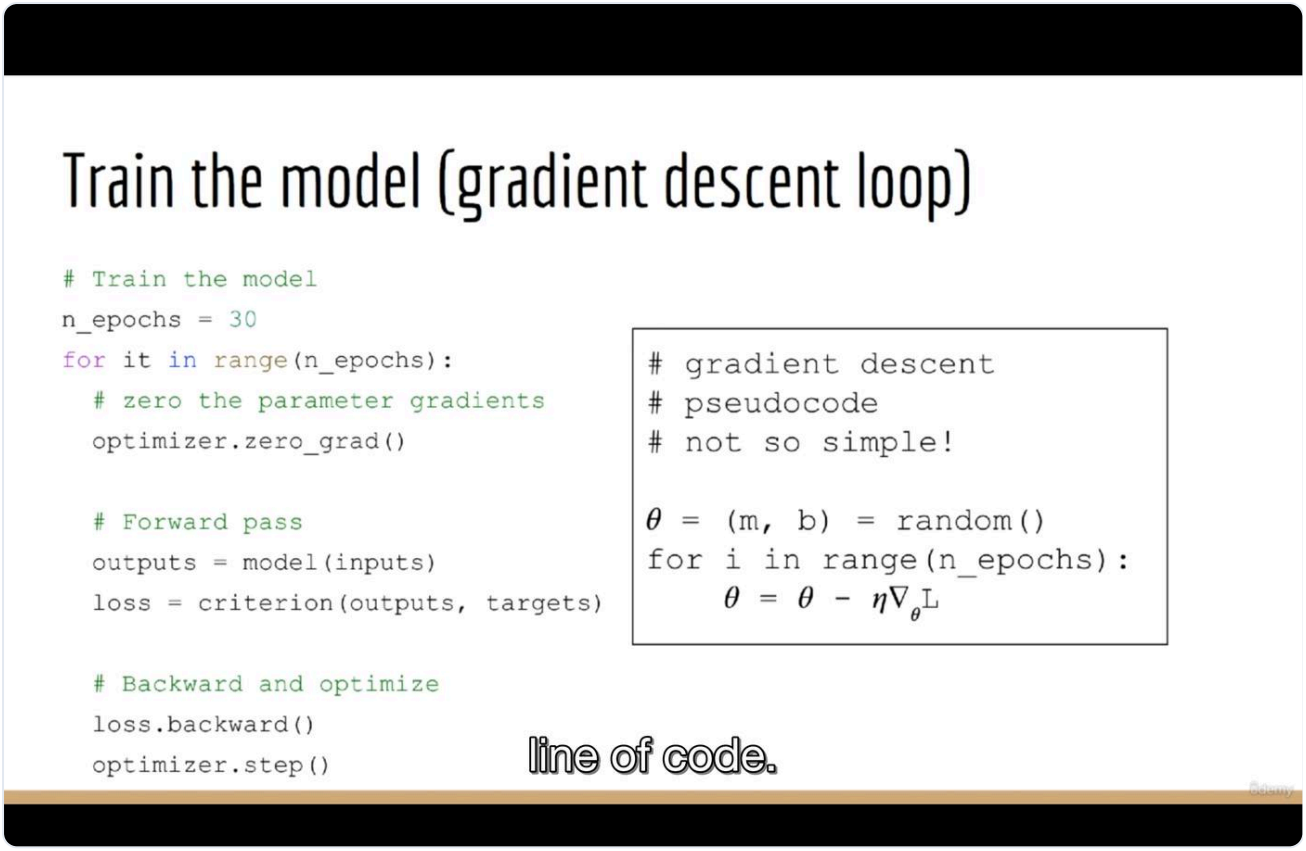


FIGURE 10.8 | Train the Model (Gradient Descent Loop)

I like to think of it just as some boilerplate code something that needs to be done. So just put it there and forget about it. If you want to know what it does then what we can say is that internally PyTorch accumulates gradients whenever you call loss dot backward which is a line that shows up later in this loop. Personally I would not worry about that because ninety nine percent of the time we'll be doing exactly what you see here.

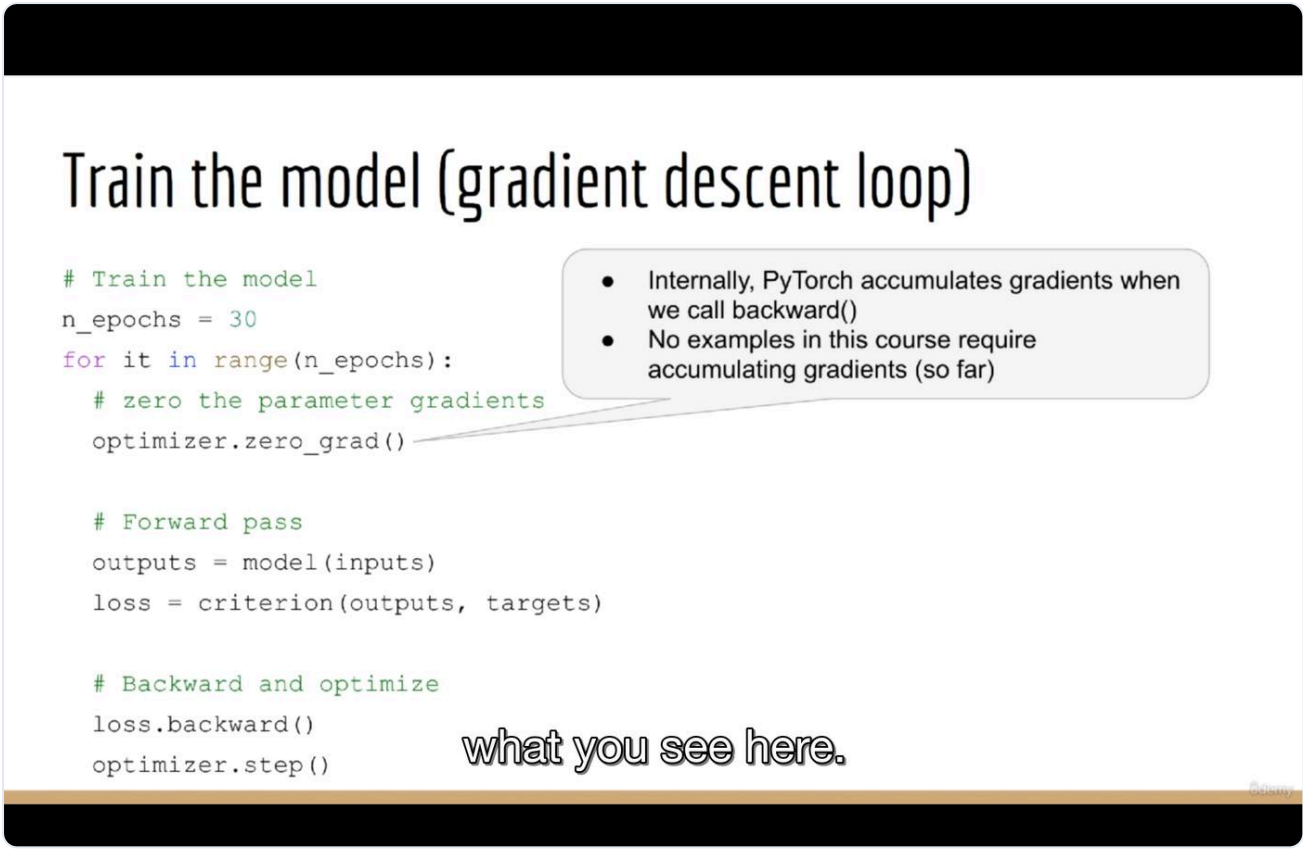


FIGURE 10.9 | Train the Model (Gradient Descent Loop)

And so far none of the examples in this course require you to actually accumulate gradients. The rest of the steps are relatively straightforward as you know in order to calculate the loss. We need the predictions. The y hats in order to get those we pass them into our model. This allows us to see the functional aspect of PyTorch.

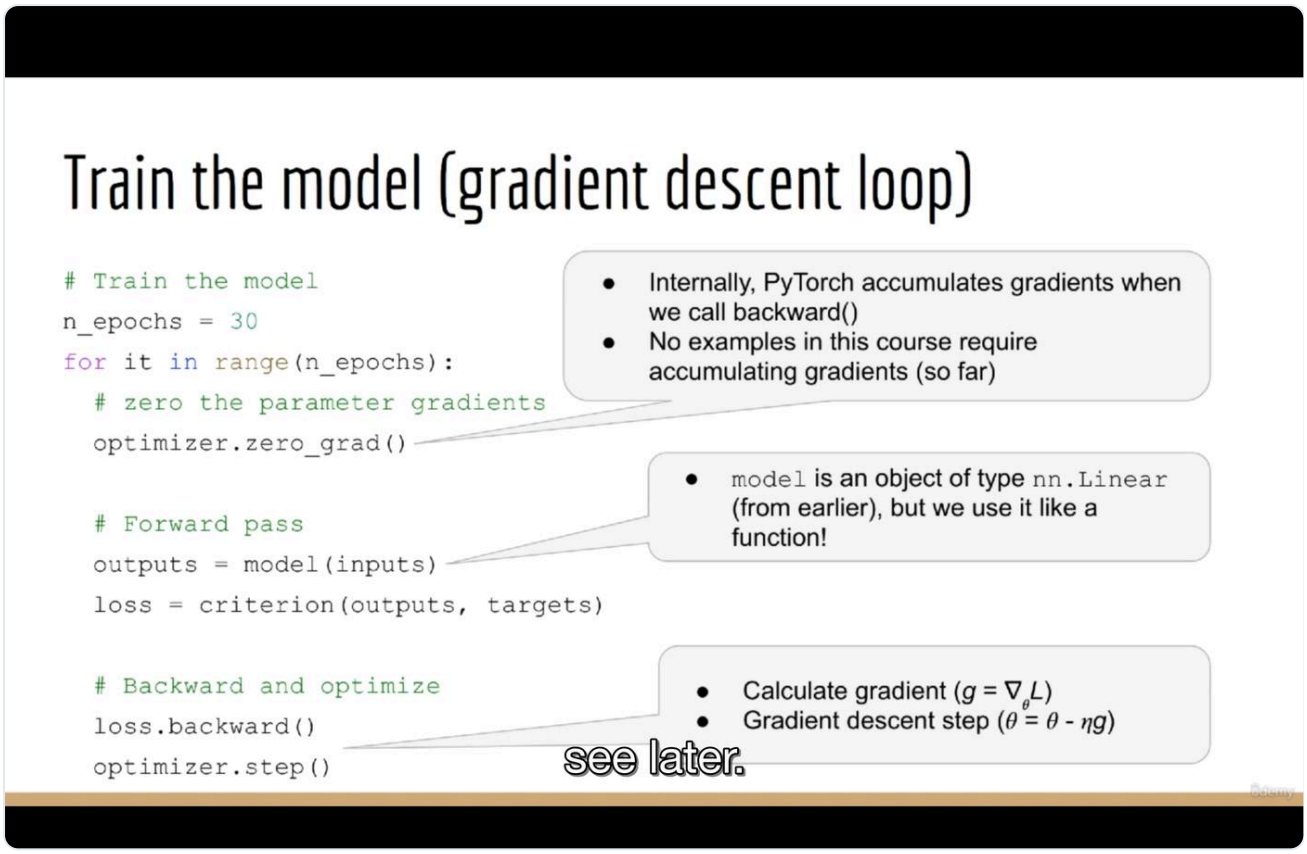


FIGURE 10.10 | Train the Model (Gradient Descent Loop)

Even though the variable model is an object of type nn dot Linear as you recall that's what we created earlier. It can be used as if it were a function. So when we pass in some inputs into the model and call it like a function we get the model outputs. The next step is to pass the arguments and the outputs into our criterion the mean squared error. Again this is an object but we're using it like a function. The next step is to call loss dot backward. Conceptually you can think of this as calculating the gradients. For example the partial derivative of L with respect to M and the partial derivative of L with respect to B the final step is to actually apply the gradients using the gradient descent equation as you'll see later.

There are a lot of variations to the basic gradient descent. So once you have the gradients there are a few different things you can do with them. Aside from just this and you may have noticed something strange in the previous coach that been previously I call the data X and Y but in the previous code snippet I use the variable names inputs and targets. Is this just a matter of some trivial renaming. Or did I really mean to say that they are different things. In fact they are different things. When I load in the data I usually call the inputs X and the targets y and usually they are numpy arrays but one important detail that we haven't looked at yet is that PyTorch does not work with regular numpy arrays instead PyTorch works with torch tensors.

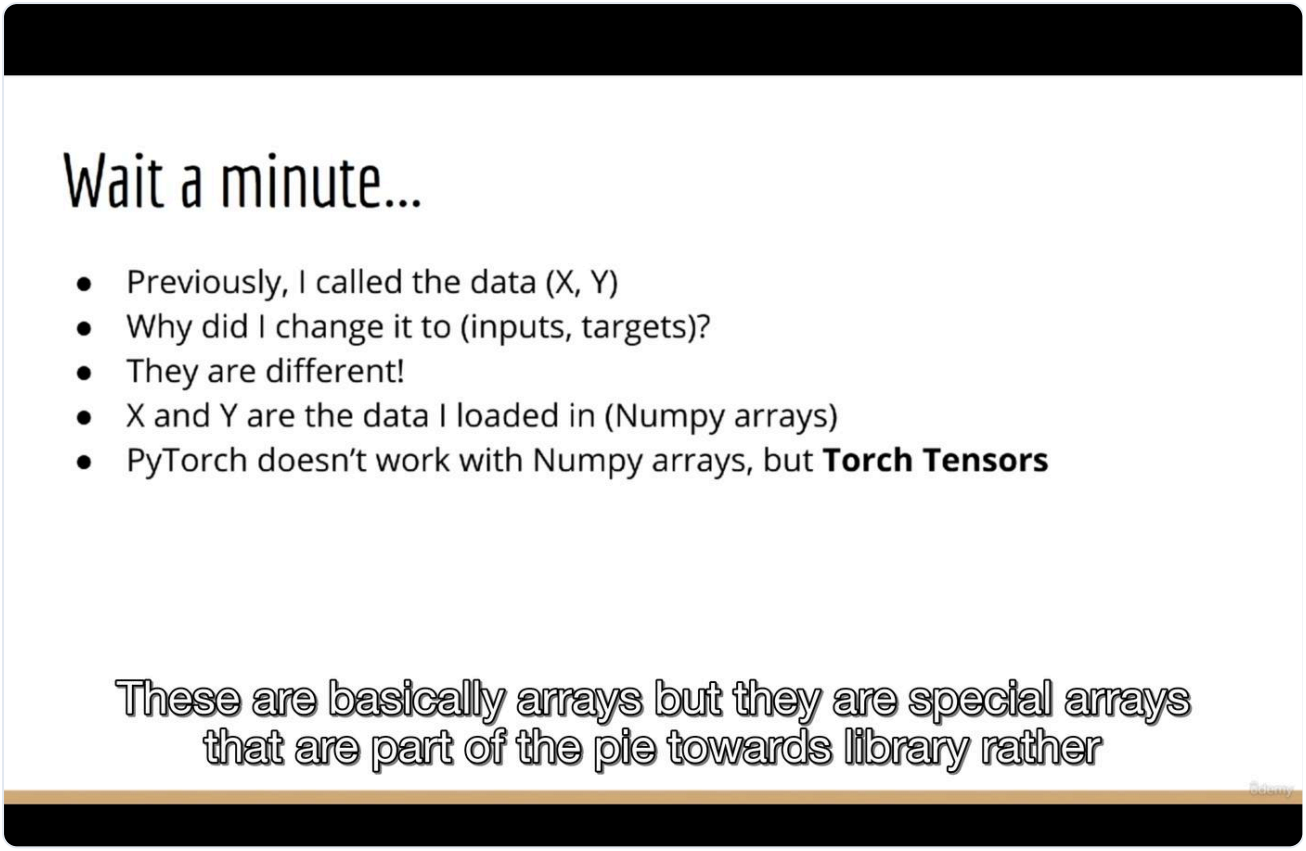


FIGURE 10.11

These are basically arrays but they are special arrays that are part of the PyTorch library rather than the NumPy library. So they have some special functionality so some conversion has to take place before you pass the data into your model in the following example. For simple linear regression the steps are what you see here. First we're going to reshape x and y to be N by 1 instead of just N in machine learning.

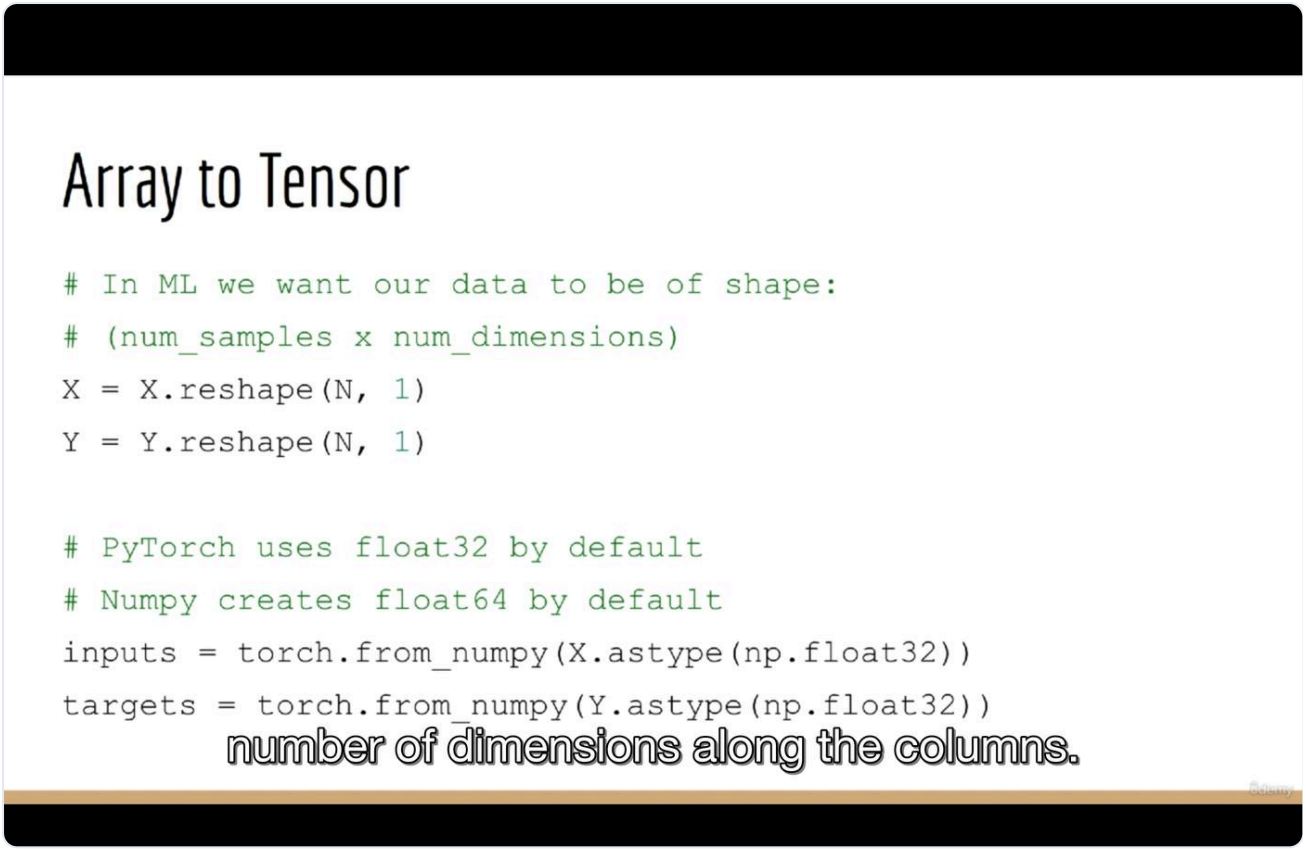


FIGURE 10.12 | Array to Tensor

We typically assume that our data is two dimensional but the number of samples along the rows and the number of dimensions along the columns. This applies even for the targets which is not the case in libraries such as Keras or scikit-learn so be aware of that second PyTorch is really picky about data types. It won't allow you to multiply a float 32 by a float 64 which is also known as a double. So you have to be mindful of this also by default. If you don't specify anything PyTorch will create all your variables as float 32 as in NumPy. If you don't specify anything your arrays will be float 64s. So before we create torch tensors we're going to cast our numpy arrays into float 32 the final step is to call the function torch dot from numpy.

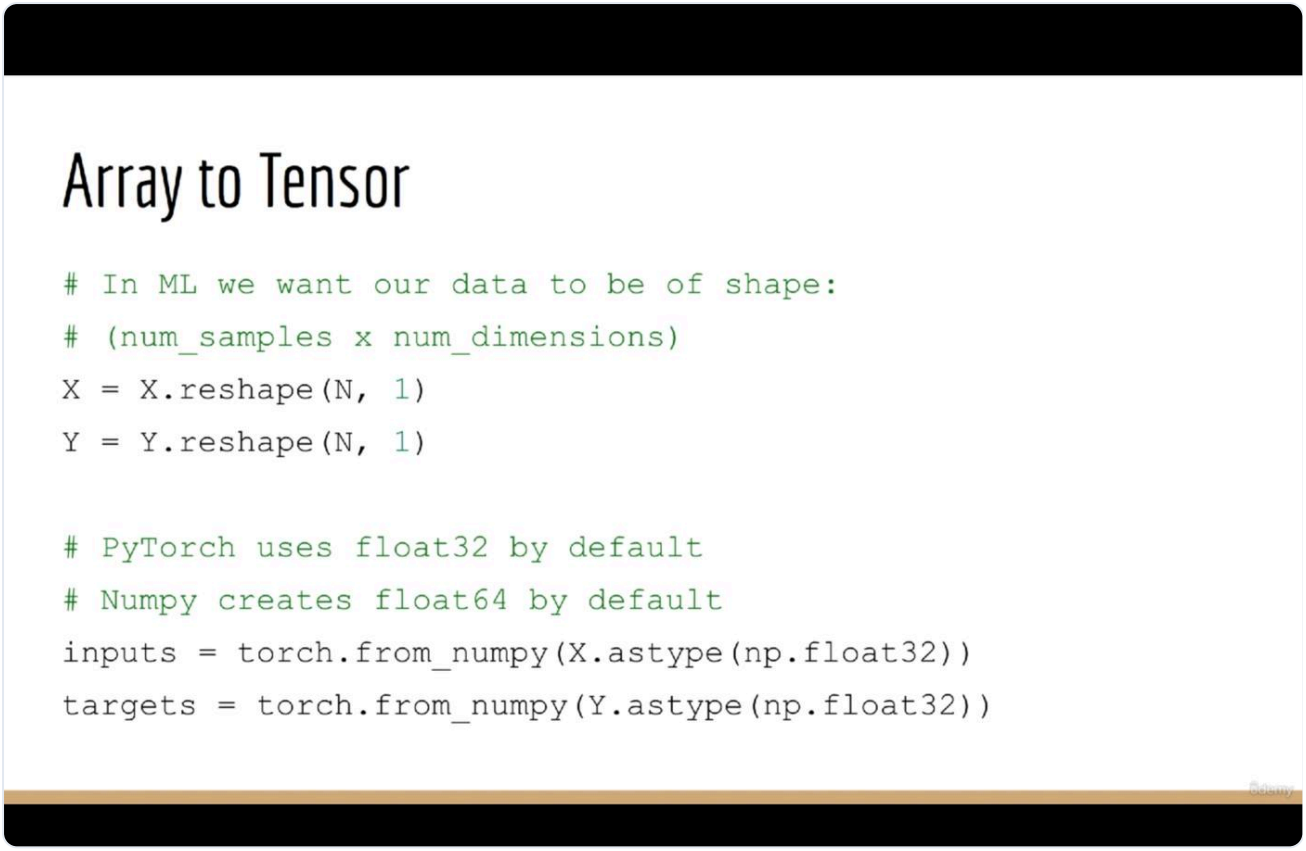


FIGURE 10.13 | Array to Tensor

This takes as input our numpy arrays and returns corresponding torch tensors. So these are the inputs and targets we were working with in the previous training loop the final step is to make predictions. You might think we already did that in the training loop when we called model of inputs but the important thing to remember is that PyTorch only works with torch tensors and the inputs variable is a torch tensor. It's reasonable to assume then that the output is also a torch tensor. Of course if you want to do any later processing such as plotting the array with matplotlib you might want to have a numpy array rather than a torch tensor. Therefore in addition to actually calculating the output you'll also want to convert it back into a numpy array.

Normally you can do this just using the numpy function.

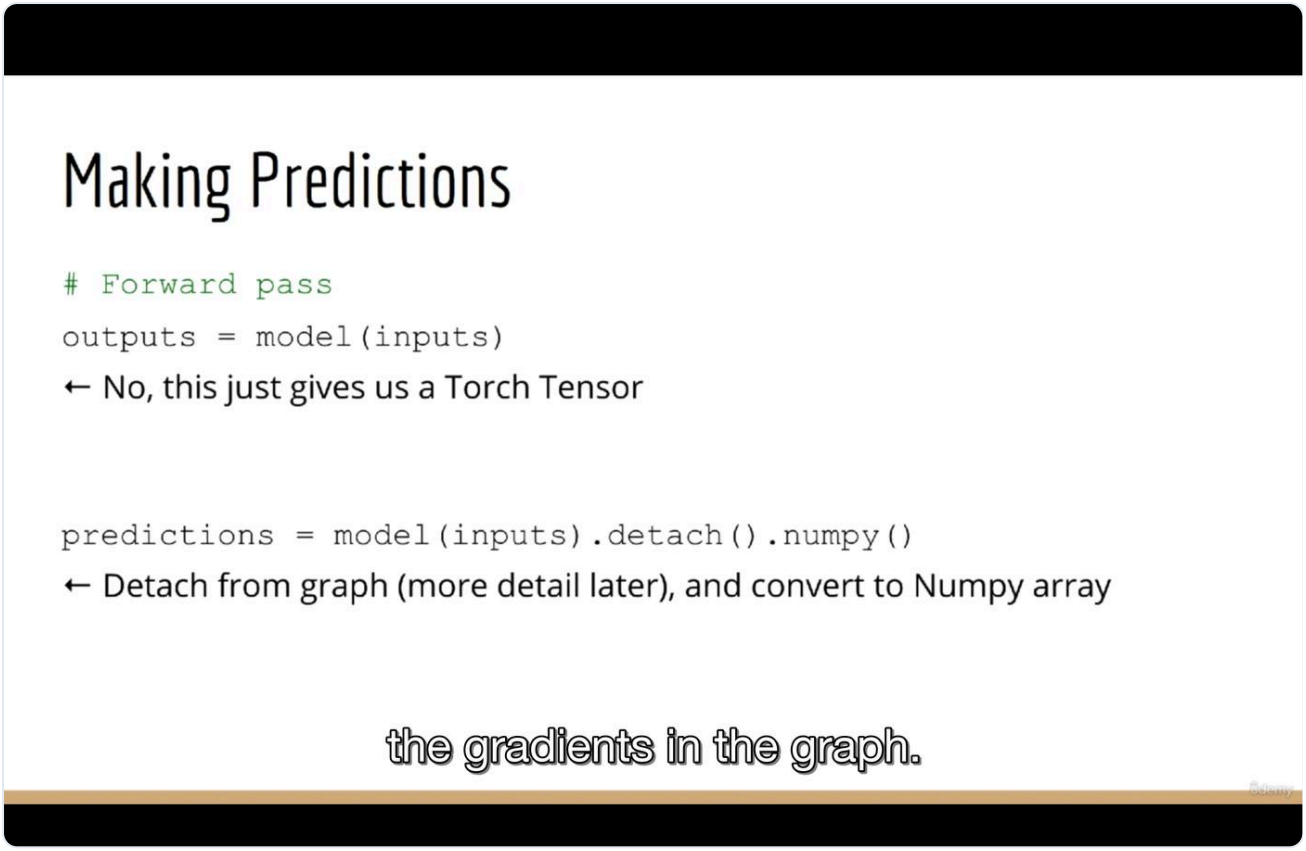


FIGURE 10.14

However PyTorch also does some stuff behind the scenes where it creates a graph and keeps track of the gradients in the graph. Basically you can just call the detach function before you call the numpy function. In order to detach that tensor from the graph all right. So that was our code preparation for simple linear regression.

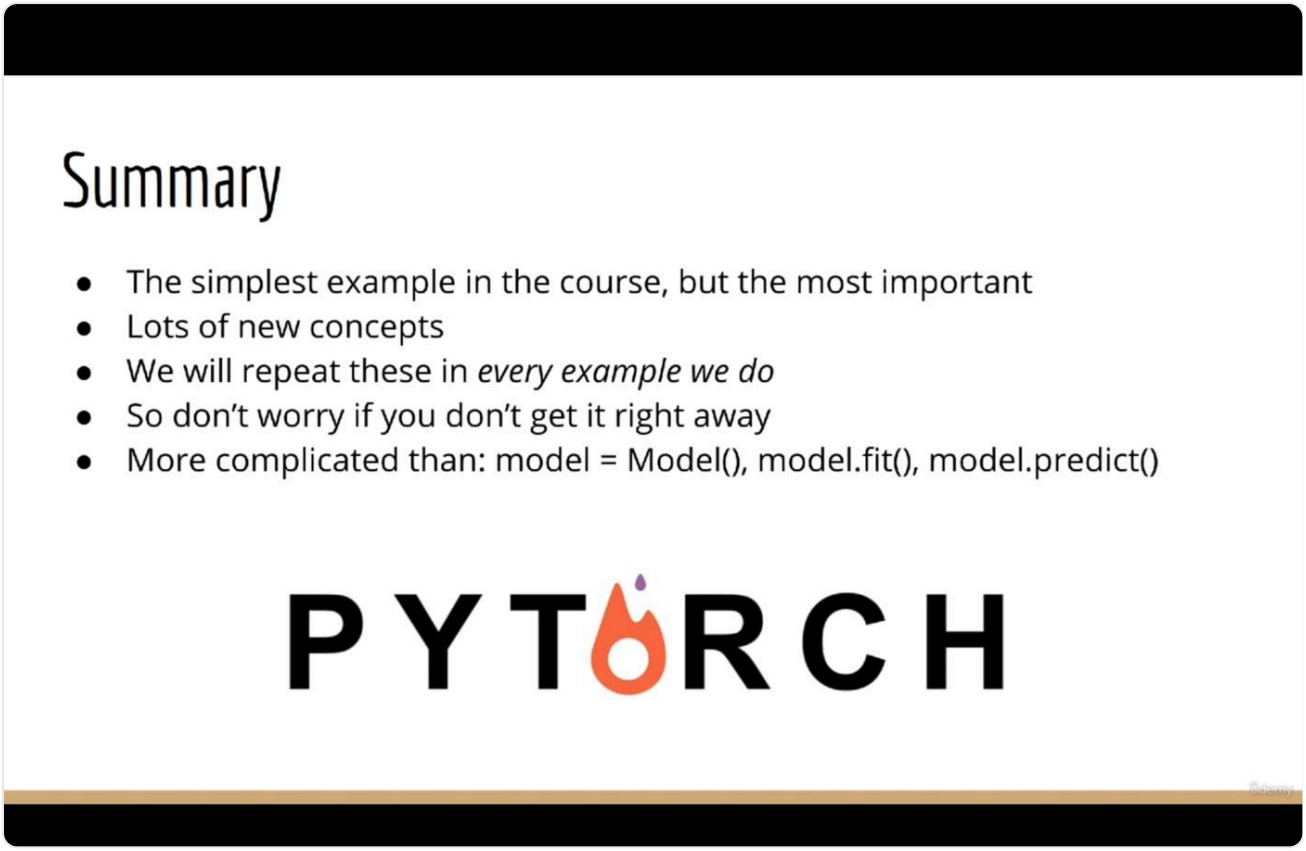


FIGURE 10.15 | Lots of New Concepts

I said this before but it's worth repeating while this is the simplest example in the course it's also the most important because it introduces a lot of new concepts mostly about how to use PyTorch in future examples. We'll just be repeating these same concepts over and over again. So you'll get lots of practice. You've seen that PyTorch is quite a bit more complicated than simply instantiating a model calling fit and then calling predict.

LECTURE 11

Synthetic Data and Gaussian Noise

Check `1.PyTorch-Linear-Regression.ipynb` for the code.

In this lecture we are going to do an example of simple linear regression in code. Using PyTorch in other words we are going to learn how to find the line of best fit. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at. To start we're going to do a few imports you'll see throughout the course that these imports are what will need pretty much every time. So we have torch and torch dot nn which is a module that contains a lot of useful stuff in PyTorch.

Next we have NumPy and matplotlib NumPy is for doing linear algebra and matplotlib is for doing plotting. Next we're going to generate some data to apply our model to. Now you might ask why aren't we using real data. Well don't get too excited. That's what we're going to do next. Using synthetic data is actually very important in machine learning but more on that later. So to generate this data I'm going to set N equal to 20 data points. Remember that our convention is to use the capital letter N to represent the total number of samples and the capital letter D to represent the total number of features as we move on to more complex datasets later in this course.

There will be some additional letters in our case D equals 1. So we don't really need to specify anything. Next we're going to generate some random points on the x axis. They're going to be uniformly distributed between minus five and plus five. There are of course several ways you could do this. So if you don't like the way I've done it you can feel free to implement your own. And this applies to anything in this course. So what we've done here is generated 20 random data points using the random function which outputs random numbers between 0 and 1. Next we multiply those numbers by 10 so that the range between 0 and 10. Finally we subtract 5. So the range becomes minus five to plus five.

Next we're going to generate the data points on the y axis because we're doing linear regression. We would like these y data points to be linearly correlated with the X data points. So to do that I'm going to say y equals zero point five times x minus one plus some Gaussian noise. Pay close attention to these numbers. This means that our true slope is zero point five and are true y intercept is minus one. These are the values that our model is going to try and find. Now you might ask why have I added Gaussian noise with mean zero and variance 1 rather than some other kind of noise. In fact when we have Gaussian noise the mean squared error becomes the correct loss function to use and thus since we're using the mean squared error as our loss function we would also like to use Gaussian noise centered at zero.

You're not expected to understand this during this course but when I mentioned these things people inevitably end up asking me why this is the case. So if you are interested in learning the answer I would recommend you look at the in-depth series of courses where I've already covered

all this material. Next we're going to do a plot of the data. As you can see it's sort of roughly forms a line it's a little bit hard to tell since that noise variance is quite high compared to the variance of the data itself. But I think it's pretty clear that the data is trending upward so that's everything for our dataset. Next we have the PyTorch stuff first we're going to define our model as promised is just one line nn dot Linear one one.

This means that we're creating a linear model with one input and one output next we're going to begin the training process as you know it starts with defining a loss function object and an optimizer object. Luckily you've seen this before. So there are no surprises. We're using the MSE loss and SGD the optimizer with a learning rate of zero point one. Next we're going to transform X and Y into data types which are appropriate for PyTorch. So we start by reshaping X and Y into n by 1 matrices next we cast both X and Y into float 32s and convert them into torch tensors in the next block. I printed out the type of the inputs variable so you can see that it is indeed a torch tensor.

Next we have our main training loop to start we set n epochs the number of iterations of the loop to 30. Now you might ask why did I choose 30 and not some other number like 10 or five or 1000. Well the trick is I've ran the script before and tried different values already. As a student you don't really see all the work that goes into choosing hyperparameters. You only see the final result so you might assume that you can just pick any number and everything will work out great. This is often surprising for beginners. In any case I discuss a lot about hyperparameters elsewhere so if you're interested in that please ask me about it on the Q and A something else you didn't see in the code preparation lecture is that I have a list of losses that I'm going to plot after we are done in each iteration of the loop.

We were going to store the loss in this list and at the end we're going to plot the loss per iteration. This will let us understand whether or not we chose good hyperparameters and whether or not the training process converge and so forth. So it's quite useful. Oh and just a hint for future examples. We are pretty much always going to plot this after training so I don't want anyone asking me what is this plot mean because it's going to be the exact same thing every time. So don't forget it. All right. So next we have our training loop which we've seen before. As I mentioned before the first step is this weird step we 0 the gradients. And this is because behind the scenes PyTorch is actually accumulating the gradient each time you call backward.

So this will zero the gradient to prevent them from accumulating and give us the correct answer. Next we do a forward pass to get the model predictions. Since you've now seen how to make predictions and get the output as a numpy array you know this is similar to that except we don't need to take the additional step of converting this into a numpy array. This is all in PyTorch land so we get out a PyTorch tensor that we can apply in the next step the next step is to calculate the loss using the criterion we defined earlier. Next we save the loss to our list of losses. In this case we do want to take the loss from PyTorch land into Python land. So we call the function item.

You might ask why do we call the function item and not the function numpy like we do when we make predictions. And that's because the loss is a single Python number and not a numpy array. So when your tensor is a single number and you want to bring that number back to Python land then you should use the item function. Next we calculate the gradients PyTorch encapsulates this in a function called backward to be called from the loss tensor. Finally we call optimizer dot step

to do one step of gradient descent. As always I like to print out useful information while the loop runs. So here you can see I'm printing the iteration number along with the corresponding loss at each iteration of the loop.

So if you look at these numbers they give you some sense of how training is progressing but the real value comes from plotting the loss per iteration which is what we have next. This is the kind of curve we like to see in deep learning a steady decrease downward fast the beginning and slow at the end. This plot gives us more confidence that training has completed successfully. It's not a 100 percent guarantee but this is at least one positive sign. Next we're going to use our trained model to make predictions and plot the result. As you recall a task for this model is to find a line of best fit. It would be quite silly if we didn't naturally plot that line.

The next step is therefore to pass our inputs into our model which gives us the predictions in terms of a torch tensor since we would prefer to work with numpy arrays. We first detach the tensor and then call the NumPy function. Now you might be wondering what happens if I do not call detach. We'll look at that in a moment. The next step is to plot the data points using a scatter plot and then to plot the predicted line using a line plot. As you can see from the result our line is in fact the line of best fit. Next we look at what will happen if you do not call the detach function as you can see. We get an error it says can't call numpy on variable that requires grad use var dot detach dot numpy instead.

Which is exactly what we did however this error message gives us a hint for an alternative way to make predictions. We can see that it has something to do with gradients in the next block of code. We can instruct PyTorch not to compute gradients. We use the context manager with torch dot no grad inside the block we make our prediction. But if you notice there is no call to the detach function as you can see the output is printed correctly without any error. The last thing we would like to do in this script is to inspect to the parameters of the model to check if they're close to the true values. As you recall these are the slope and intercept of the line in machine learning.

We call this the weight and bias in order to get these values we call model dot weight dot data dot numpy and model dot bias dot data dot numpy intuitively you can expect that model dot weight and model dot bias store the weight and bias as some special PyTorch tensor variable. However we would like to bring these back to numpy land so in order to do that we call data dot numpy. You can imagine that the reason we treat these differently from the model's predictions is because they are model parameters rather than data inputs or outputs. If we look at the values we can glean a few key pieces of information. First the obvious thing we want to check are they close to the true values we get zero point four six in minus one point two.

So it appears that they are one interesting thing about these is that w seems to be a two dimensional array whereas B seems to be a one dimensional array. You might ask why do we need a 2D array and a 1D array to store single numbers. In fact we don't but we need our model to be flexible enough to handle different scenarios. These scenarios are what you'll learn about in the later lectures the final thing I want to talk about in this lecture is what is the point of using synthetic data instead of real data. This is going to come up once or twice in this course. So it's good to discuss this early and often. One of the most important questions we can ask is Does our model actually do what we think it does.

As a beginner taking a course your automatic reflex is to make a guess or to think of the problem philosophically but this is not a philosophy course. It is a computer science course. We don't discuss maybes in this course it's binary yes or no it works or doesn't. So synthetic data is very useful in that regard. I can create synthetic data such that if my model does what I say it does then it will find the pattern in the synthetic data that I set it up to find if it can. Then my assertion is wrong doing things this way allows us to test the strengths and weaknesses of our models and to help us understand what they can and cannot do.

LECTURE 12

Exponential Growth of Computing

In the next couple of lectures we are going to see how easy it is to apply what we just learned to a real world data set. In fact this real world data set is one of the most significant datasets of all time. This lecture is about Moore's Law.

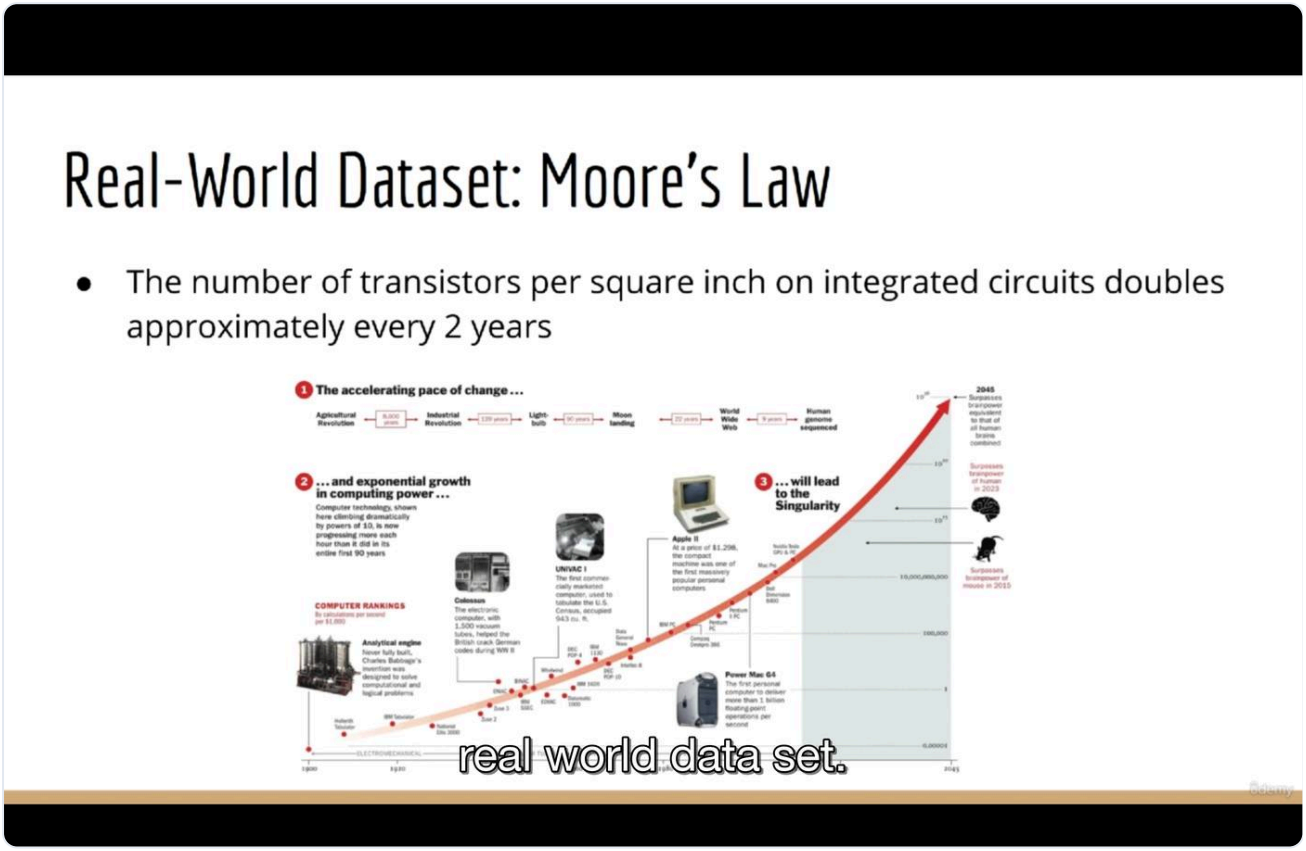


FIGURE 12.1

As you recall Moore's Law is the observation that the number of transistors per square inch on integrated circuits doubles approximately every two years. In other words roughly speaking computing power grows exponentially.

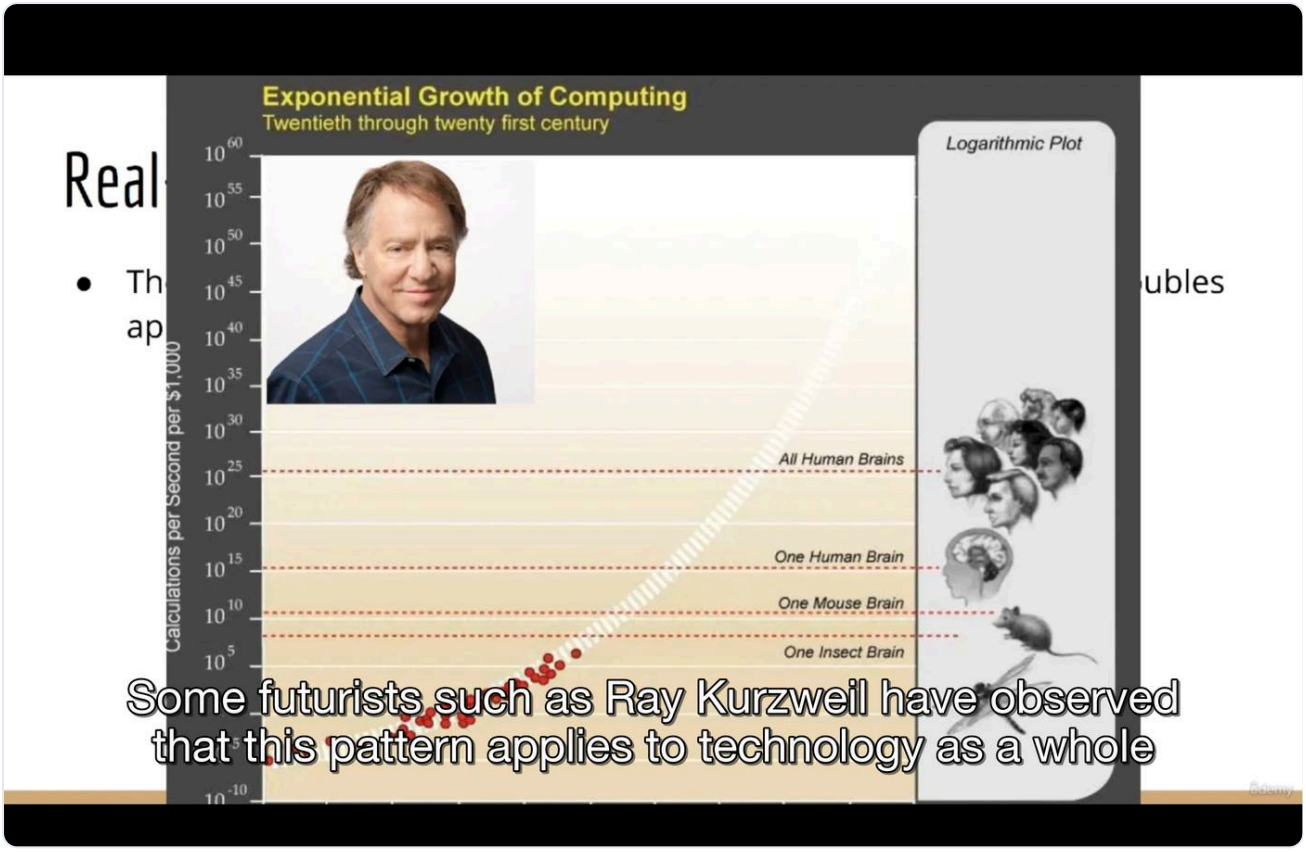


FIGURE 12.2 | Exponential Growth of Computing

Some futurists such as Ray Kurzweil have observed that this pattern applies to technology as a whole so that even when Moore's law runs out for transistors perhaps some new invention will appear to take its place. Maybe that will be quantum computing A.I.

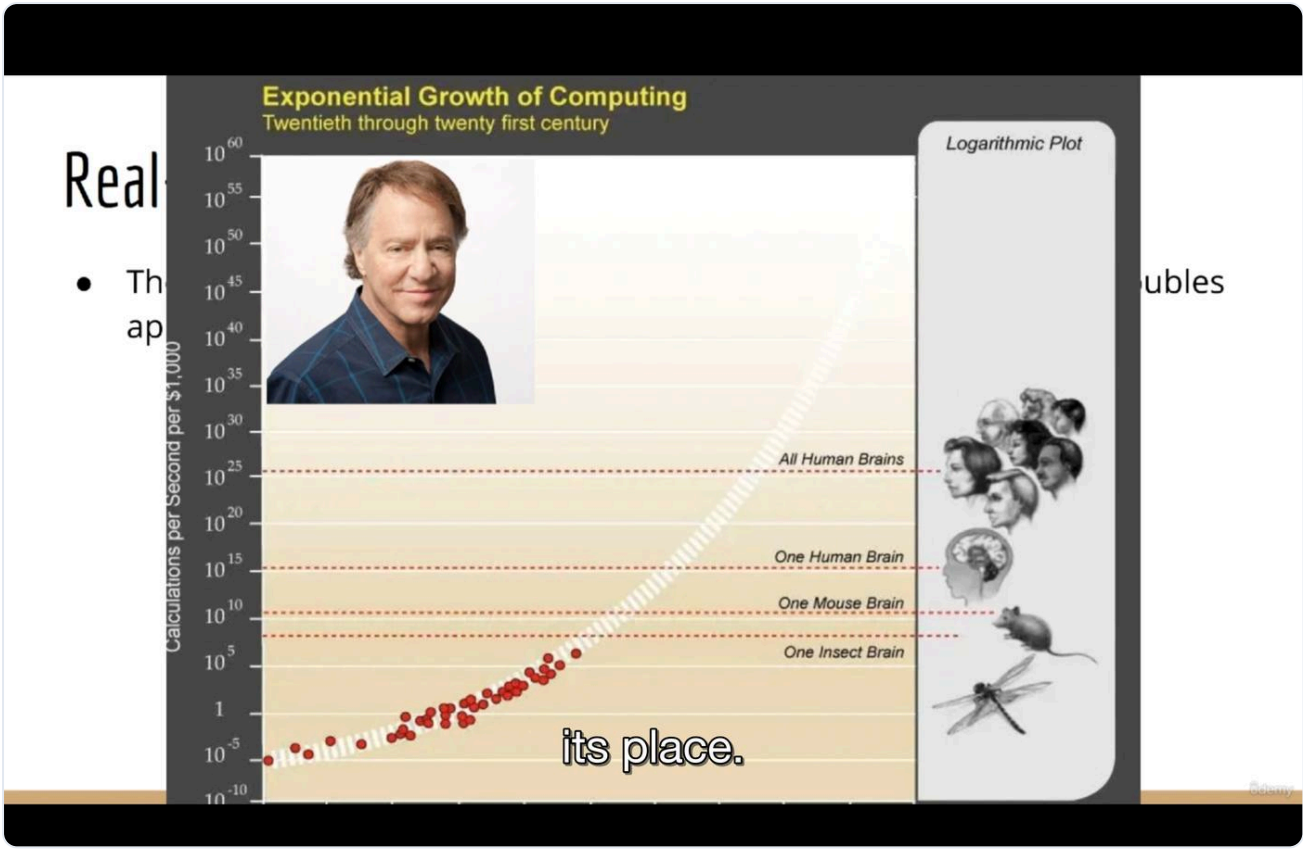


FIGURE 12.3 | Exponential Growth of Computing

Or something we can't yet imagine. Now you might be thinking wait a minute this isn't linear at all. If something grows by a constant factor over a given period of time that's actually exponential growth.

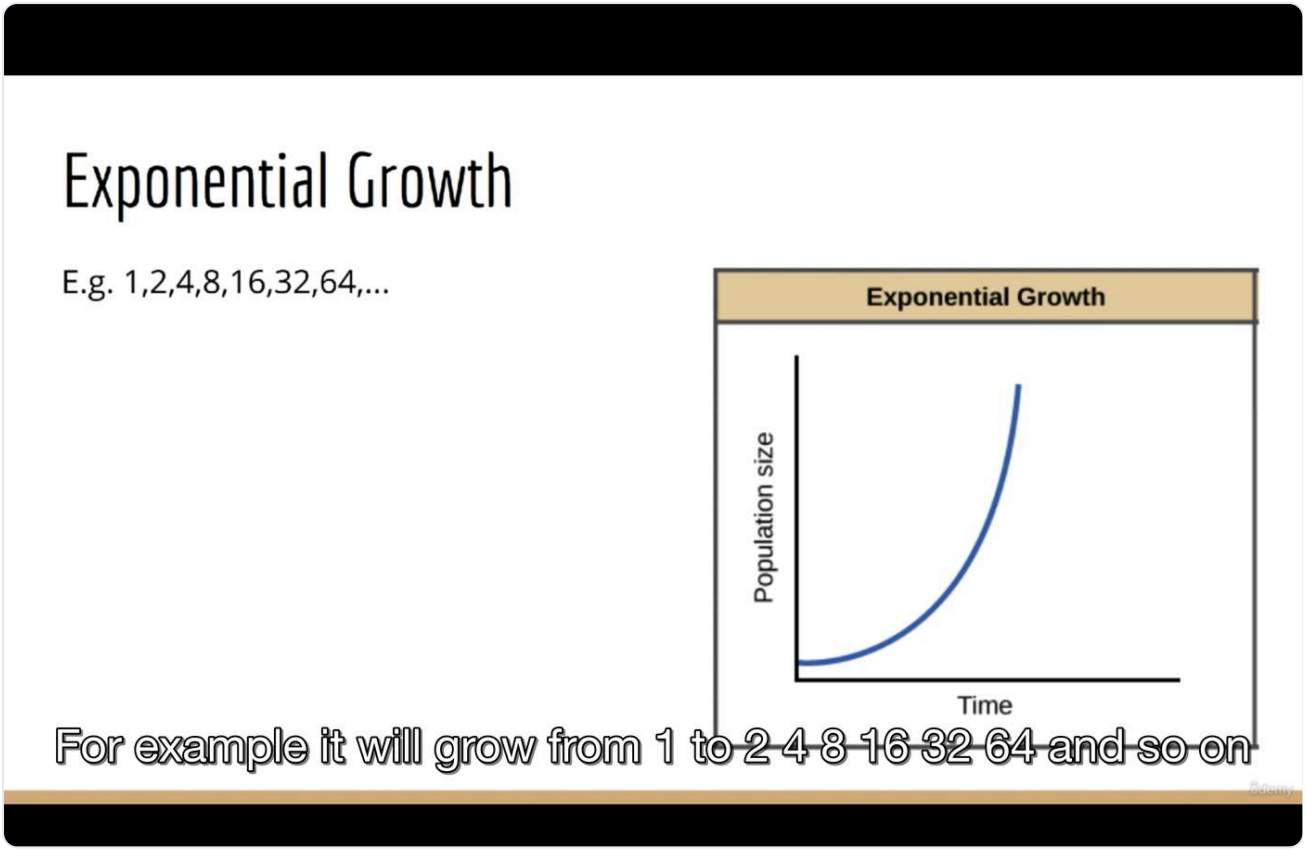


FIGURE 12.4 | Exponential Growth

For example it will grow from 1 to 2 4 8 16 32 64 and so on we can write the generic equation for exponential growth as C equals C naught times R to the power t here. C is the output variable and that's the thing we're counting C naught is the initial value. When t equals zero. T is the input variable which in our case represents time and finally R is the rate of growth.

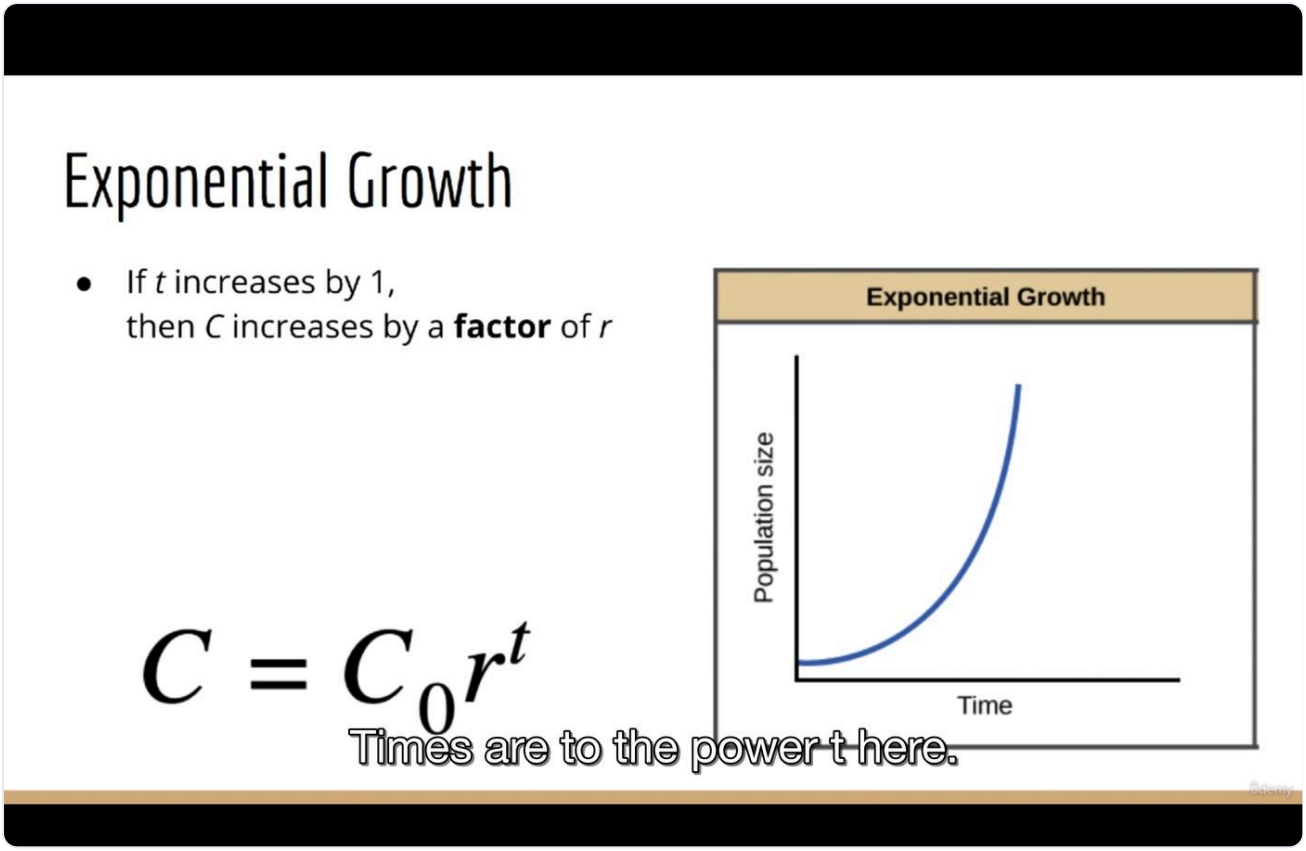


FIGURE 12.5 | Exponential Growth

You can see that if t increases by 1 then C increases by a factor of R. This holds true for all time periods. So if I met T equals 1 going to t equals 2 I can find the next value simply by multiplying by R.

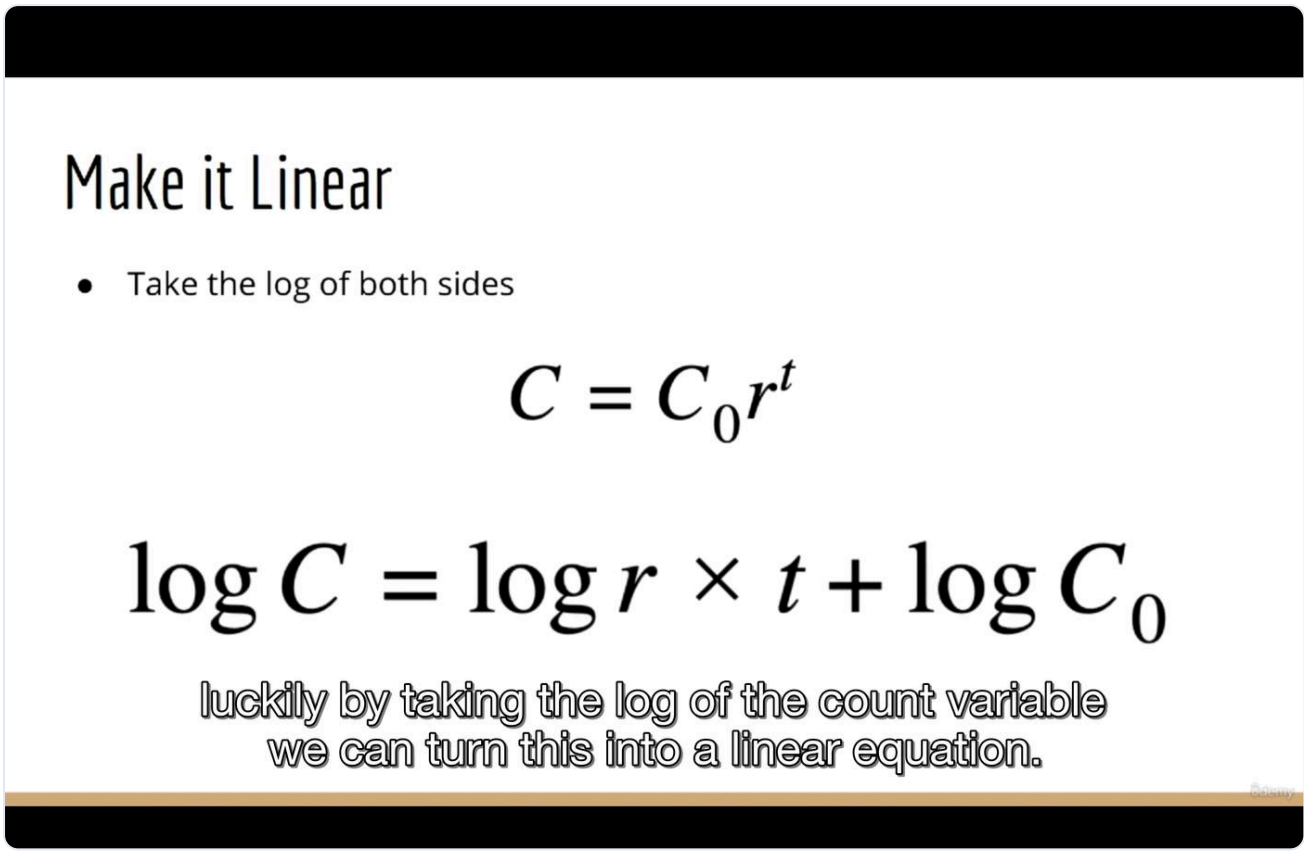


FIGURE 12.6 | Make It Linear

And similarly for going from T equals to the T equals 3 luckily by taking the log of the count variable we can turn this into a linear equation. Basically we take the log of both sides and we show that the log of C is a linear equation with respect to the time t. This is exactly what we want. So if we use a linear regression on the log of transistor counts with respect to time we should find that the rate at which transistor counts double is approximately two years if you can't see how this works right away. It might help to substitute the variables in the previous equation with some more familiar variables so let's replace log C with y. This will be the output variable.

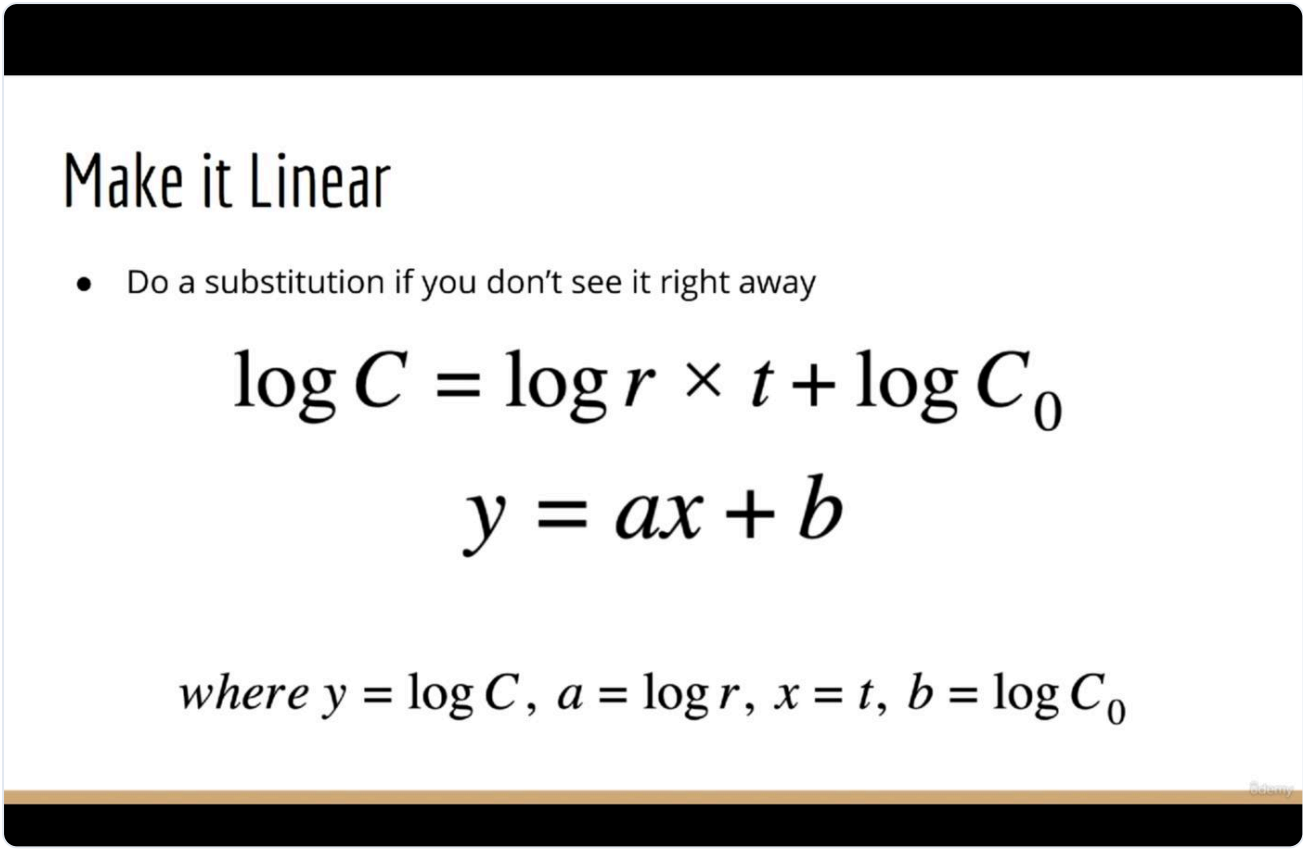


FIGURE 12.7 | Make It Linear

Let's replace log R with a slope. Let's replace T with X. This will be the input variable. And finally let's replace log of C naught with B the intercept. Now we just have Y equals A X plus B which is our linear equation from earlier. Note that I'm using the letter A instead of M for the slope which is perfectly okay at this point.

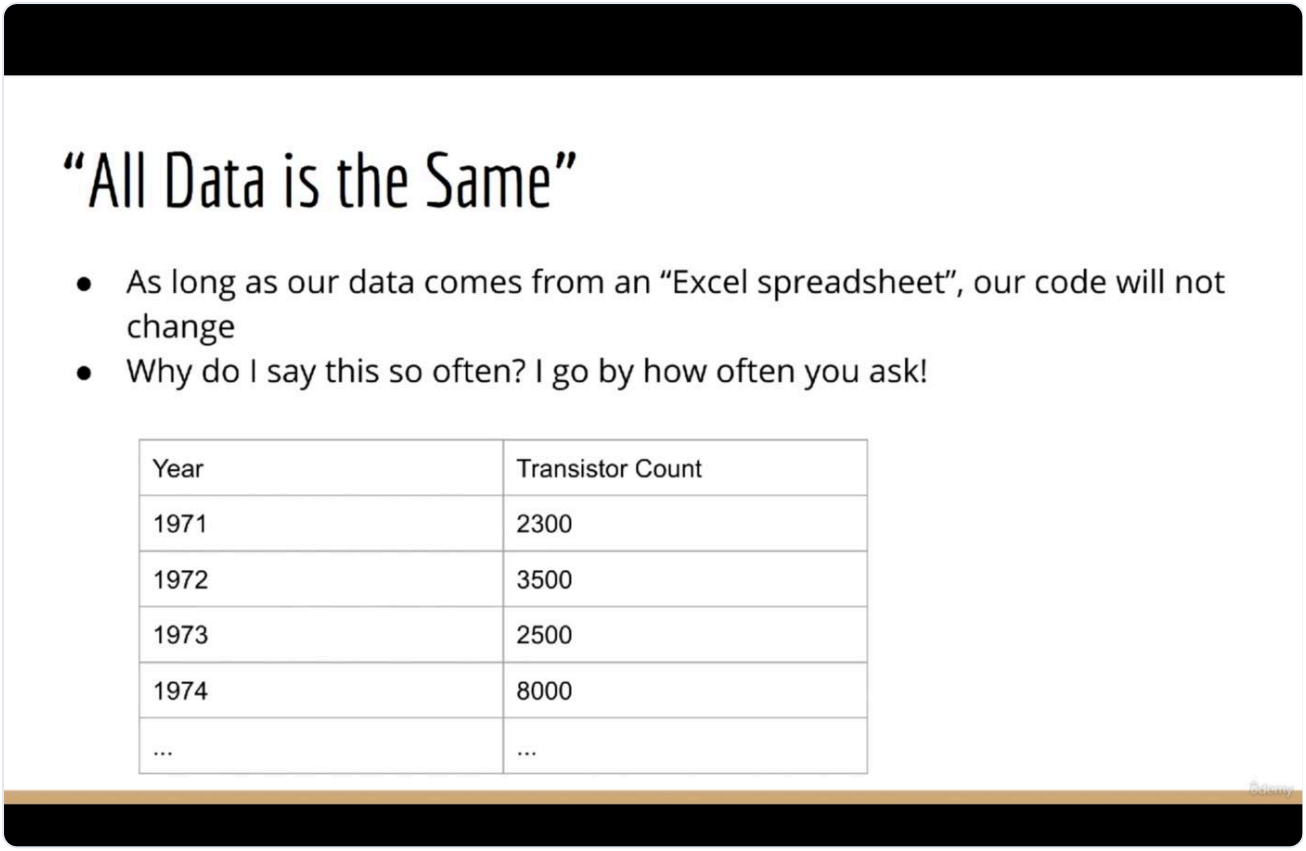


FIGURE 12.8

The problem becomes exactly the same as our previous problem. As long as we can transform our data set into a quote unquote Excel spreadsheet which has the log of transistor counts in the y column and the year in the X column then our code for building training and predicting what the PyTorch model should be exactly the same as it was before. This is what we mean by all data is the same as you will see. There is no difference in the model between this example with transistor counts and the last example with synthetic data and you might wonder why do I say this so often. And really I just go by how often I get asked. So it's really you. The students who guide what I say.

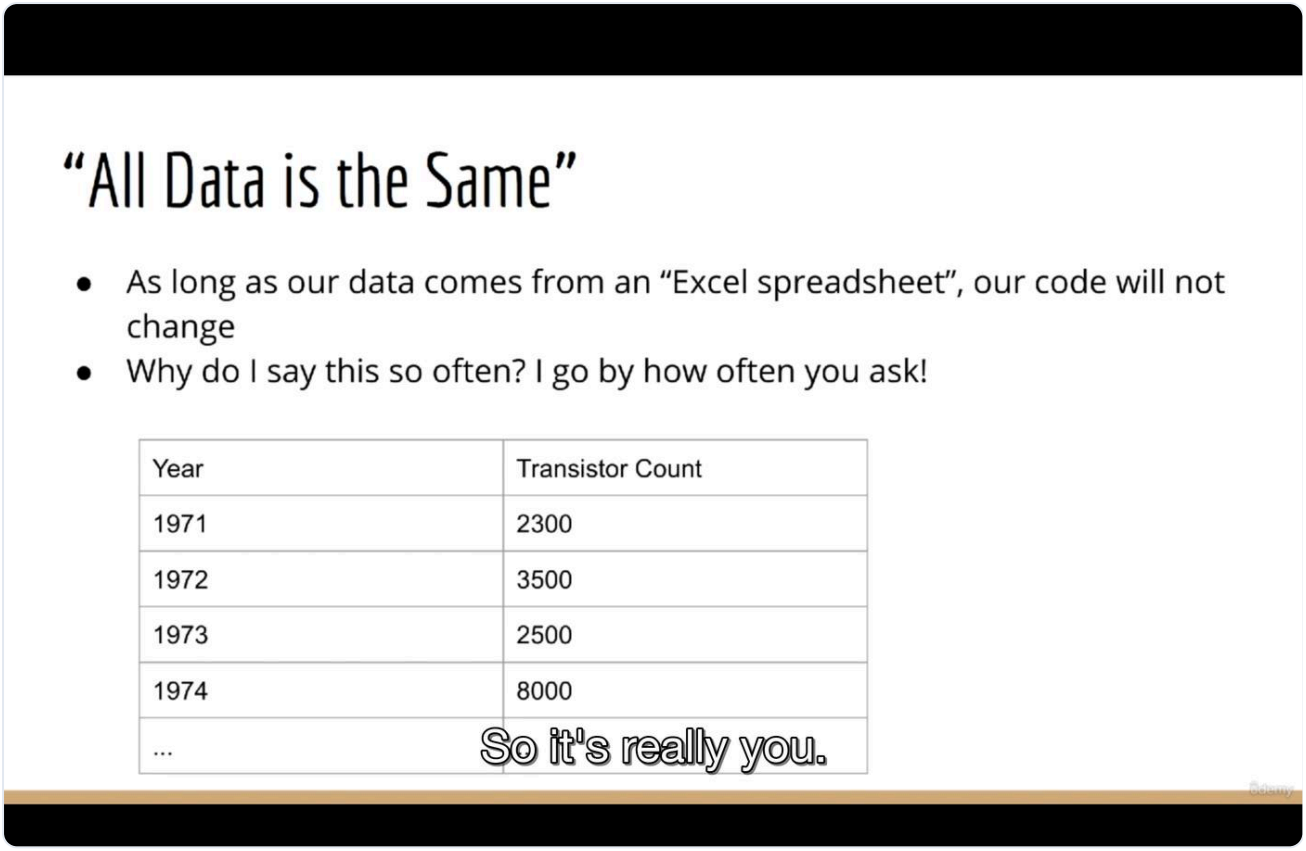


FIGURE 12.9

If I find that a lot of students have a problem with something then I will emphasize that point more often. So despite the fact that I repeat this constantly students will still sometimes ask me can you do an example on such and such dataset. And my response is as always the code is exactly the same. No changes are needed although because this message is pretty effective I think it definitely has helped a lot of students understand the idea a lot better. There is one caveat to this which you will see in the following lecture. In order to use linear regression or any other kind of model in this course effectively you should make sure your data is either normalized or in a small range of values and centered around zero.

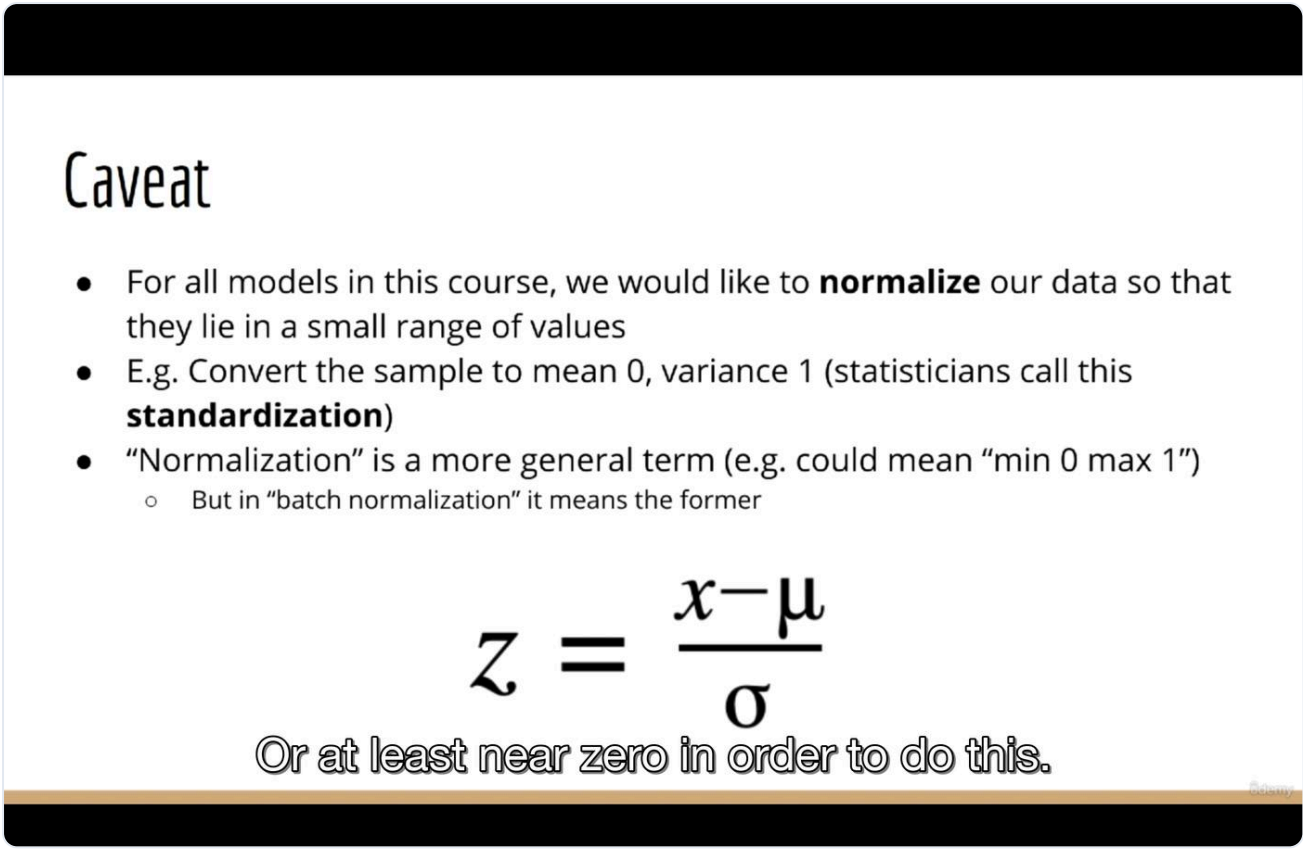


FIGURE 12.10

Or at least near zero in order to do this. We're going to subtract the mean and divide by the standard deviation. This makes it so that the sample has mean zero and variance one please note that in this course we will use the terms normalize and standardize pretty much interchangeably standardized is a term that is used more in statistics and it refers specifically to the case where you take a sample and transform it so that it has mean zero and a variance one. This is opposed to the term normalized which could refer to standardization but as a more general term overall for example it could mean transforming your data so that it stays in the range 0 to 1.

This is not the same as having mean zero and variance one on the other hand. In cases like batch normalization which you'll learn about later in this course it does refer to the standardization operation. The real question is Why do I mentioned this as a caveat. Here's the problem we're doing two transformations on our data. This is going to make it more difficult to prove that transistor counts double every two years. So here's a quick question for you given that the equation for exponential growth is C equals C naught times R to the power t. What does r have to be in order for the transistor count to double every two years. If T is in units of years I would strongly recommend you try to figure this out for yourself before we move on to the next lecture.

The next question you want to consider is if we take the log of both sides of this equation.

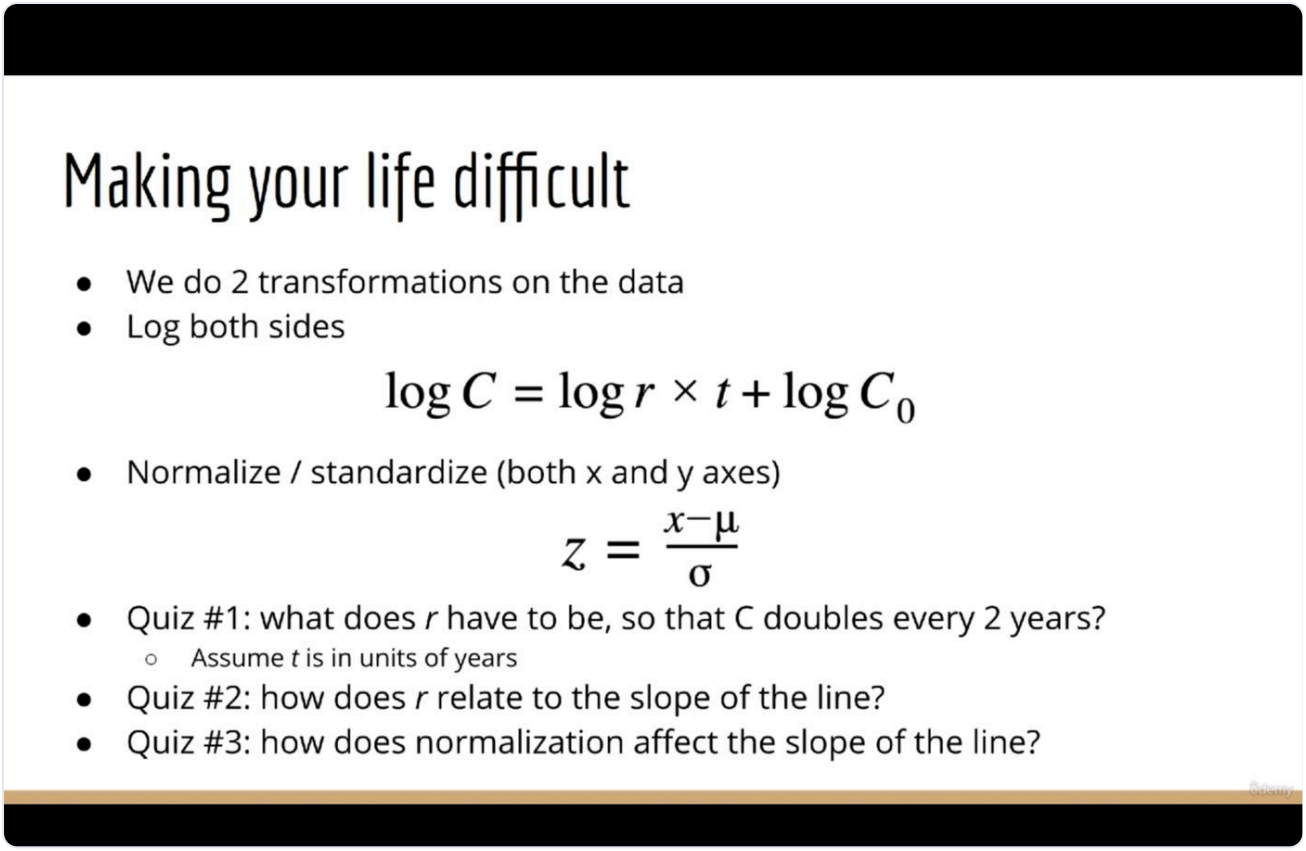


FIGURE 12.11

How does that value of r relate to the slope of the line. And finally if we normalize the data in both the x and y axes how does that affect the slope and intercept of the line. We'll go through these calculations in the next lecture in detail but the main point is we are not going to get the answer directly just by fitting a line to the data. For example just because the transistor count doubles every two years this does not mean the slope of the line is going to be 2. It's going to be related to the number two somehow but to find out exactly how you have to do the math.

LECTURE 13

Log and Rate of Growth

Refer `2.PyTorch-Regression-Moore's-Law.ipynb` for the code.

In this lecture we are going to look at the Colab notebook that performs linear regression in order to prove Moore's Law is true that the transistor count doubles approximately every two years. This lecture is going to walk you through a prepared Colab notebook although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible. As usual you can look at the title of the notebook to determine what notebook we are currently looking at. To start we're going to have all the same inputs as before with one additional import which is pandas. Pandas is useful for working with CSV files which is the format our data happens to be in to obtain the data we are going to use the w get method to get the CSV file hosted on my github next.

We are going to load in the data. The data doesn't contain any headers and it only contains two columns. I would recommend verifying that this is the case. The next thing we want to do is reshape x so that it's a two dimensional array of size N by 1 rather than a 1D array of length. Then we also need to do the same thing for y in PyTorch. Although this generally doesn't extend to other libraries. Remember that the convention is X is a 2D array of size N by D. And Y is a 2D array of size N by K where K is the number of outputs. Next we are going to do a scatter plot of this data. Notice that it shows exponential growth which we already know so next we're going to take the log of Y which is what we discussed previously.

We see that we get back what looks like a good candidate for linear regression. Next we are going to do a little bit of preprocessing on the data. Specifically we're going to standardize or normalize X and Y will want to keep these means and variances around for later use. So let's call them m x for the mean of x s x for the standard deviation of X. And similarly for y. What we have to keep in mind is that by doing this the units of X and Y no longer make sense. Previously X had units of years but this is no longer the case and Y had units of log counts which is no longer the case. We'll have to reverse the transformation later on if we want to get back to the original units in the next plot.

We do a scatter plot again which on first glance looks exactly the same as it did before but if you look closely you'll see that both the x and y axes are centered around zero and the range of values is now small which is exactly what we want. Next we're going to do a little more preprocessing to convert our data into float 32 next. It's time to create our linear regression in PyTorch model. Thanks to our rule all data is the same. This code will be exactly the same as before. Once we have our model which is just one line of code we're going to create our loss and optimizer as you can see there's one argument here that might be new to you momentum.

This is one of those variations on gradient descent that I was talking about earlier. If you want to know more about these you're encouraged to check out the in-depth section of the course. Next we're going to convert our numpy data arrays into torch tensors using the from numpy function.

Next we're going to run our main training loop as promised. This is the exact same thing as what we had earlier just with a different number of epochs. I just want to mention again because a lot of people ask me how do I know what number of epochs are needed. Just remember that there is no way around it trial and error. There was no magic formula. You just have to keep trying until your loss per iteration.

Looks right. You'll be seeing the same loop over and over again throughout the course. So each time I'm going to spend less and less time explaining it. All right. So from the loss values is printing out. It looks like the loss has converged. All right so next we're going to plot the loss per iteration as usual. And from this we can confirm that our loss has converged. Next we're going to plot our line of best fit. Again this is the same as before so there's no need to explain this code again. As you can see our line is indeed a line of best fit next. Let's get the trained weights of the model. You already know how to do this. It's model dot weight dot data dot numpy as expected.

It's a two dimensional array that holds a single number. The question is what does this number even mean. To answer that question we need a little theory let's start by recalling our model of exponential growth. We have C equals C naught times R to the power t as you recall we convert this into a linear equation by taking the log of both sides. So now we have log C equals log of C naught plus log R times t. If we rename the variables we can make this look a little more familiar so let's call the input x the output Y and the slope a good thing to remember is that the slope of this line is the log of R not R itself. We can leave the intercept term untouched because it's not related to the rate of growth.

It's just the initial value of the count. Next we have to take into account that our data is normalized. As you recall we normalize the data using the equations we see here. Let's call the transformed data x prime and y prime. So why prime is equal to Y minus m y divided by s y and x prime is equal to x minus m x divided by s x. Now we can begin to understand our own model our own model is also just a line just with different units and therefore a different slope and intercept so let's call the parameters of this line w and B so the line that we actually fitted would be y prime equals w times X prime plus b what we can do next is recover our original model by substituting X prime and Y prime with their expressions in terms of the original variables x and y.

So now we have Y minus m y divided by S Y is equal to w times x minus m x divided by s x plus B you should be able to see intuitively that this is still a linear equation. After doing some algebraic manipulation we can isolate y in terms of all the other variables. If you don't understand how we got to this step I would strongly recommend trying this on paper by yourself. Well we can see is that there is one term involving X and three other terms not involving x. Clearly the three terms not involving X are equal to the intercept log of C not remember. We don't care about that because that doesn't involve the rate of growth. Instead what we care about is the slope.

Luckily it has a simple expression it's just w times s y divided by s x we know that our original line has the form a x plus log of C naught and therefore we can conclude that if we had fitted a line to our original data the slope would be a equals w times s y divided by s x all right. So the next question we want to ask is what is the value of it. Luckily we know w we know s y and we

know s x so we can just plug them in to find a remember that W is a two dimensional array holding a single value. So in order to retrieve his value we have to index w twice at rows 0 in column 0. We see that we get another sort of strange number zero point three four.

It's not clear how this would lead to a rate of growth. That means the transistor count doubles every two years. Of course that's because there is still more work to do so let's go through a quick calculation to review once again our model for exponential growth is the count c is equal to C naught times R to the power t C naught is the count when t equals zero and r is the rate of growth. If we take the log of both sides we get an equation that's a linear in t we can express this in terms of our usual linear regression variables Y equals A X plus intercept. This shows us that Y is equal to log C A is equal to log R x is just the time t and the intercept is log of C naught because a is equal to log R we can just take the exponential of both sides to get that R is equal to e to the power a.

From here we can plug in the value for the slope that we found to get that R is equal to one point four zero seven although as you'll see we won't need to use this value directly next. Let's consider the doubling time or in other words how long it takes for C to double. We can do this by thinking of the original equation as the original time and the original count. This origin is completely arbitrary and as you'll see very shortly its values don't actually matter. So in order to double this count the left hand side would become 2 C and on the right hand side the time would just be t prime. We call it t prime because we don't know what it is. Yeah that's what we're trying to solve for so if we take these two equations and divide one by the other we see that the C disappears the C not also cancels out and as I promise the origin point doesn't matter we get 2 is equal to R to the power t prime minus t t prime minus t is actually the time duration that we care about.

That's how long it takes for the transistor count to double how far from T is t prime we can rearrange this to get that T prime minus t is equal to log 2 divided by log R which is equal to log 2 over a since we found a. This is just a number. And more importantly it's a constant number one important note is that the defining characteristic of exponential growth is that the growth rate does not change over time. In other words the value of t itself doesn't matter the same equation holds no matter what the value of t is. This is like saying the duration from t to t prime is always constant and as you can see here it's always log 2 divided by a so in the next block of code we can print out this value log 2 divided by a is approximately equal to 2 just as Moore's Law says it should be.

Therefore we've confirmed that Moore's Law is true in the final block for this notebook. I have a little exercise for you in this lecture. One of the most difficult parts was transforming the data using standardization and then reversing that transformation and you might ask yourself Is that really necessary. So the exercise for you is to find out what happens if you don't try to normalize the data. You might assume that since the data falls on a line it should be pretty easy to apply linear regression as we've done multiple times so far. So give that a try and I'll see you in the next lecture.

LECTURE 14

Difference: New Loss

In this lecture, we are going to discuss a tiny bit of theory related to linear classification. Previously, we discussed that linear regression. As you recall, this is the task where our goal is to find a line or curve of best fit. You want your model to output predictions, which are close to the data points in classification. This is not our task in the classification, but we do not want the line to be close to the data points. Rather, in classification, our goal is to separate data points into distinct classes. Although we can consider the case where we have more than two classes, we'll start with two since that is the most intuitive. Because this is linear classification analogous to linear regression, our model is still a line.

It's just that the job that we want this line to do is different from regression. Unlike regression, we don't care if this line is close to the data points. Instead, we want the line to separate data points of different classes. In other words, if one class consists of the red dots and one class consists of the blue dots, then we want all the red dots to be on one side of the line. And we want all the blue dots to be on the other side of the line. In this way, we can say that the line discriminates between the two classes. As you can see, this is not related at all to having the line be close to the data points. Also, I hope I haven't lost you in terms of what the colors of the dots mean.

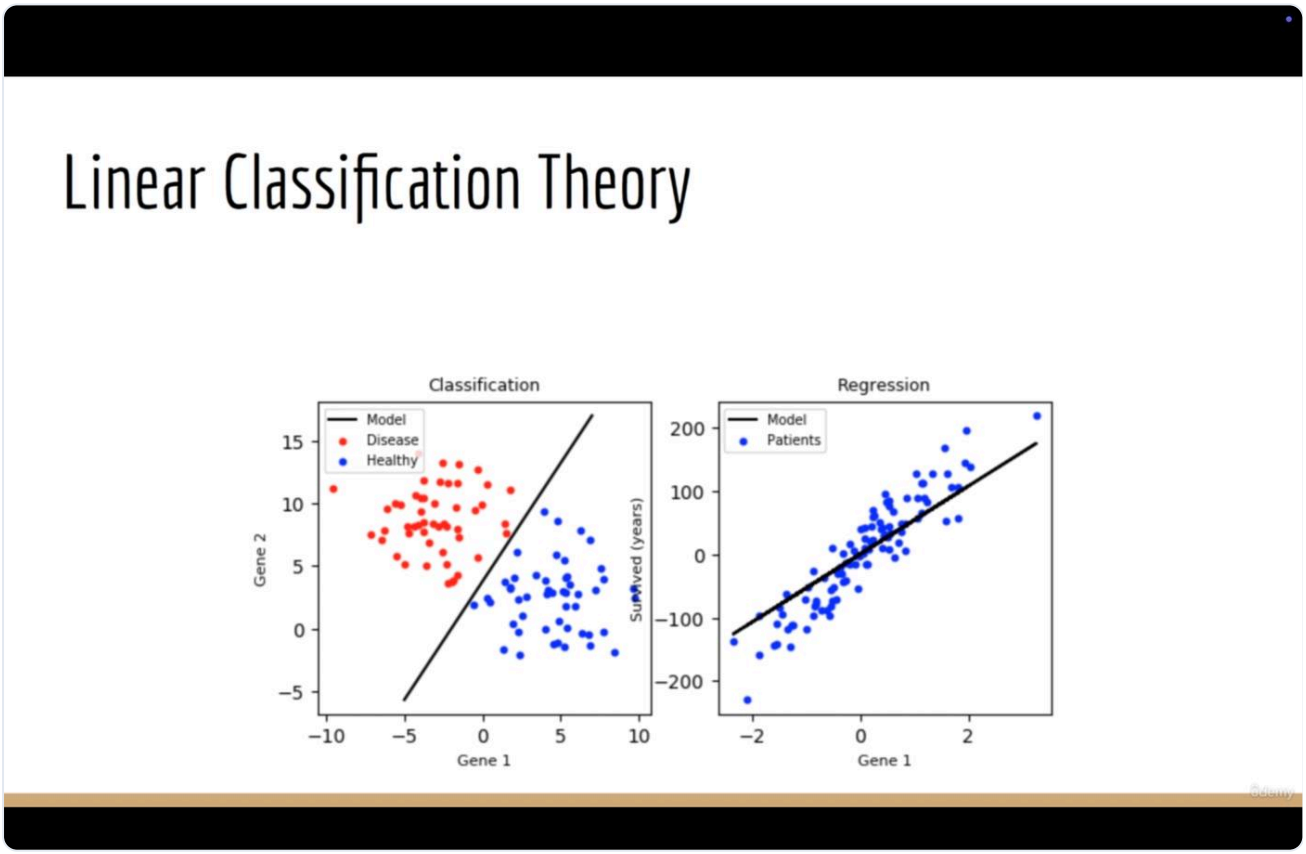


FIGURE 14.1 | Linear Classification Theory

Remember the rule? All data is the same. So for one particular data set, the color might represent whether a credit card transaction is fraudulent or not. For another data set, the color might represent whether an email is spam or not. For another data set, the color might represent whether a student will graduate or not. And as you can see in this example, it represents disease versus healthy. As with the previous lectures, we are going to start from a very basic perspective. How would this work in scikit-learn? We're going to learn what each of these steps mean in terms of concepts, and we'll also learn how each of these steps are done in PyTorch. Remember that in PyTorch there is no model constructor or fit function or predictor function, so we have to do all that ourselves.

To recap. Here are the steps we do in a typical machine learning script with scikit-learn. First we are going to load in some data we call that X and Y. Next, we're going to instantiate a model. Next we are going to train or fit the model using model dot fit x y. Next, we are going to make predictions with the model using model dot predict X. This can be the same X or a different x, for example, x train or x test.

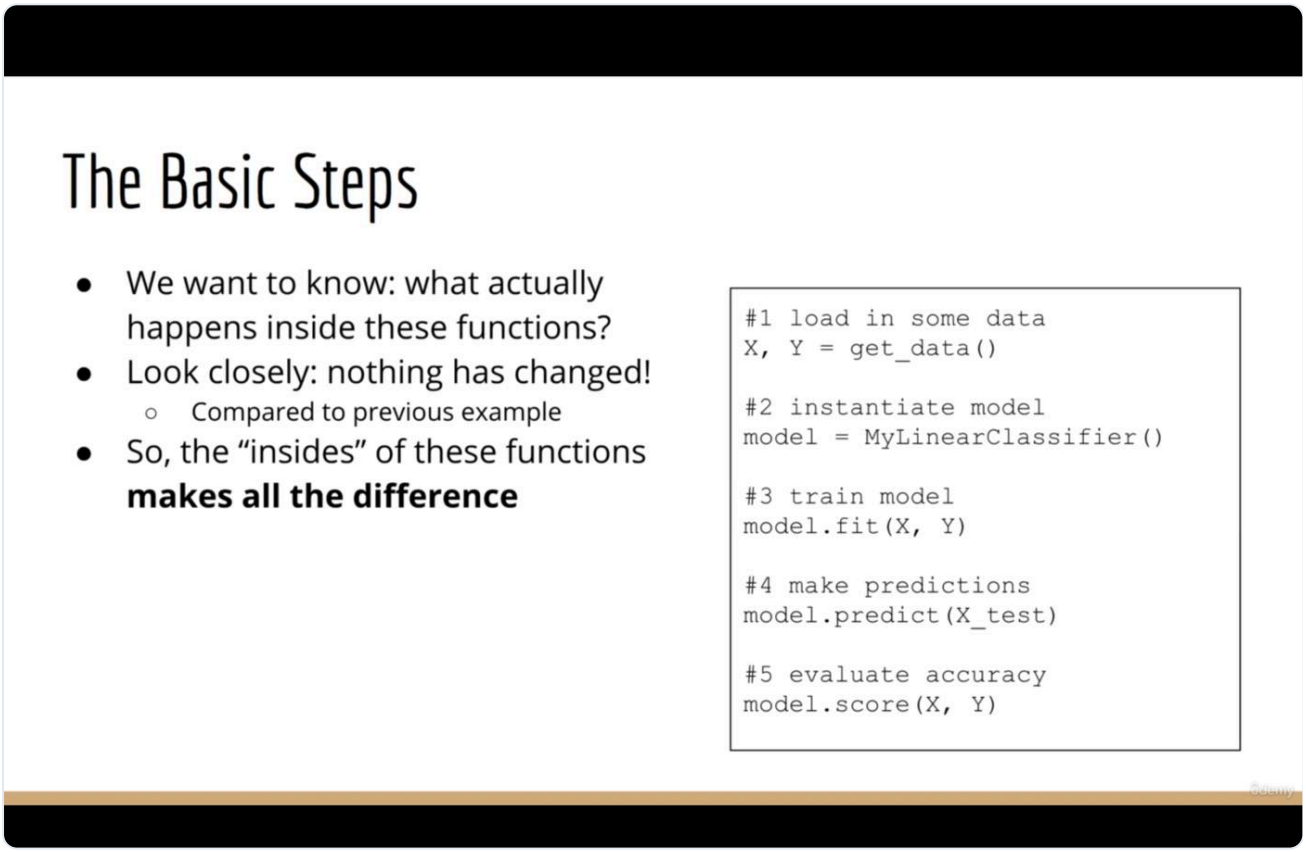


FIGURE 14.2 | The Basic Steps

Finally, we can evaluate our model using model dot score x y. For classification, the score returns the classification accuracy. And just to reiterate, this lecture is all about answering the question what actually happens inside these functions? This question is really important to answer because you'll notice that if you compare this to linear regression, literally nothing has changed except for the name of the class. So really what happens inside these functions makes all the difference. It's what really matters if you want to know how this stuff actually works. As a side note, if you're a beginner and you don't yet know what classification accuracy means, here's a quick definition.

As you know, our model is a binary classifier. It's predicting spam or not spam, for example.

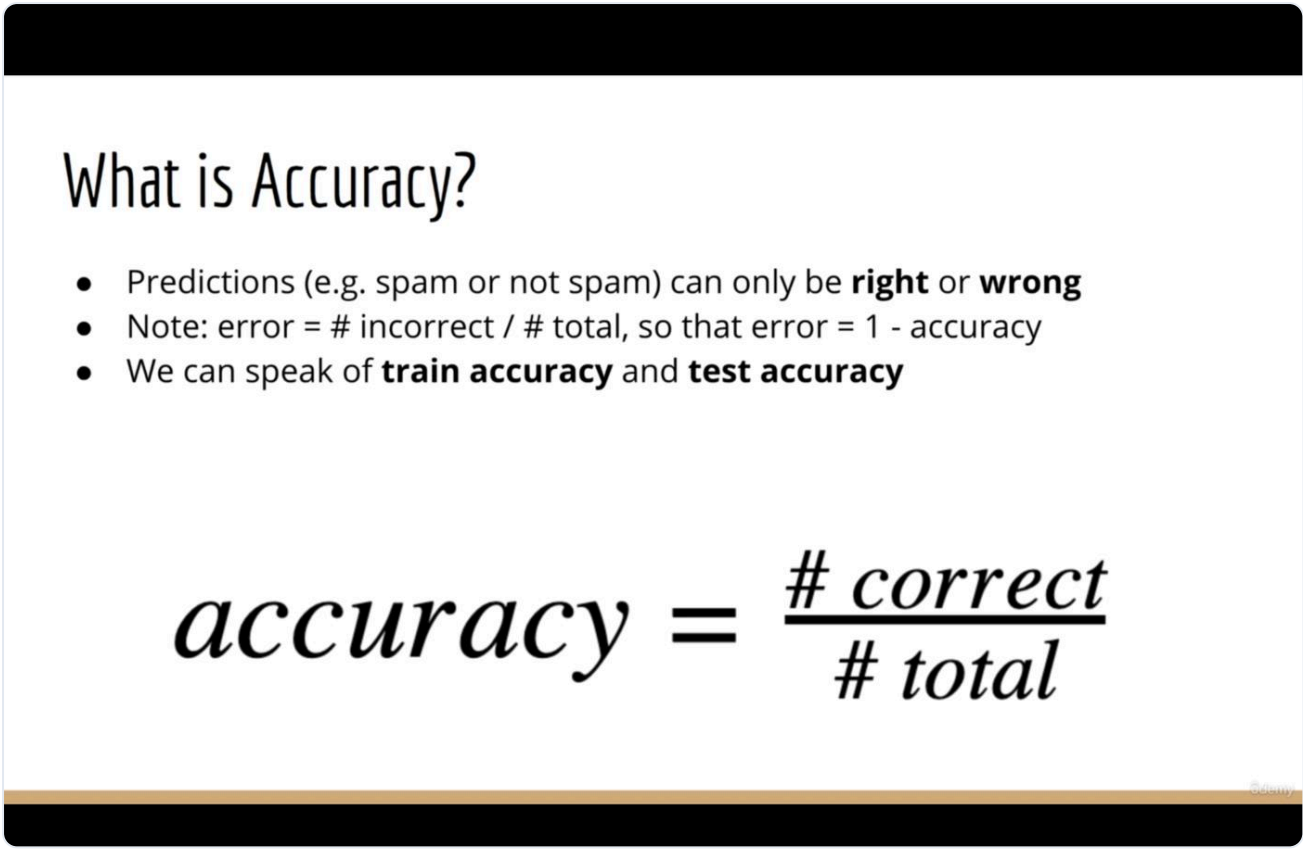


FIGURE 14.3 | What Is Accuracy?

Therefore its predictions can only be right or wrong. There are no other possibilities. Either it's right or it's wrong. The classification accuracy, then, is simply the number of predictions I get right. Divided by the number of total predictions. The classification error is the number of predictions I get wrong. Divided by the total number of predictions. It should be easy to verify that the classification error is equal to one minus the classification accuracy. Also note that you can calculate the accuracy and error on both the train and test sets so you can speak of the train accuracy and the test accuracy which would be evaluated on the train set and the test set respectively.

So the first concept we want to consider is what is the model? More accurately, we want to ask, what is the model architecture? As usual, it helps to look at a picture. So here is a typical picture of a linear classification problem. We have some data points and we want to separate the data points of different colors with a line as a sign. So why do we call this linear classification?

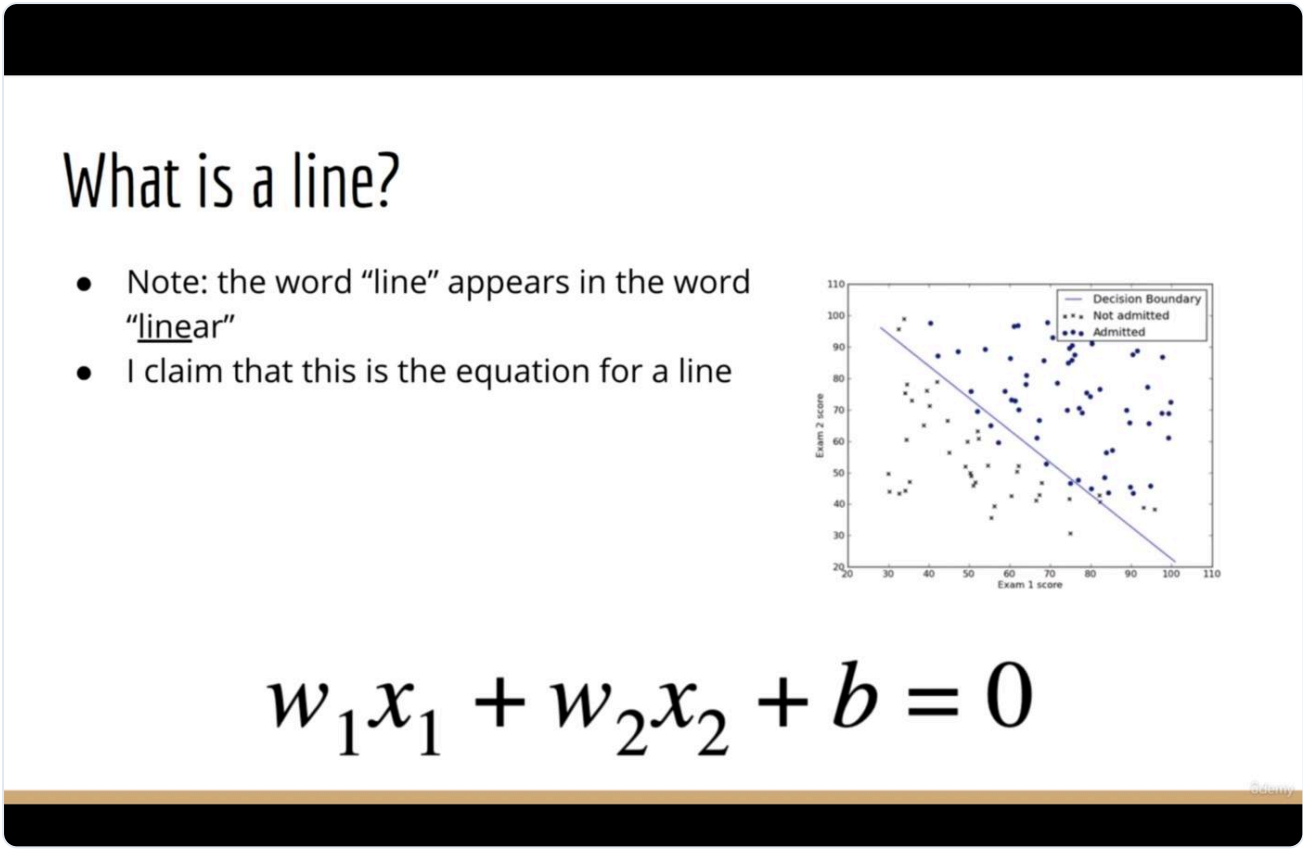


FIGURE 14.4 | What Is a Line?

Well, you will notice that the word line actually appears in the word linear. This surprises many people. So don't worry if you haven't noticed this before. All right. So what's the equation for a line? I claim that a line can be expressed using the equation w1x1 plus w2x2 plus b equals zero. Now at this point, you might say to yourself and say, Gosh, why does lazy programmer have to make everything so complicated? I already know that the equation for a line is Y equals m, x plus B, so why do we have to use this more complicated looking equation with WS and x one and x two and so on?

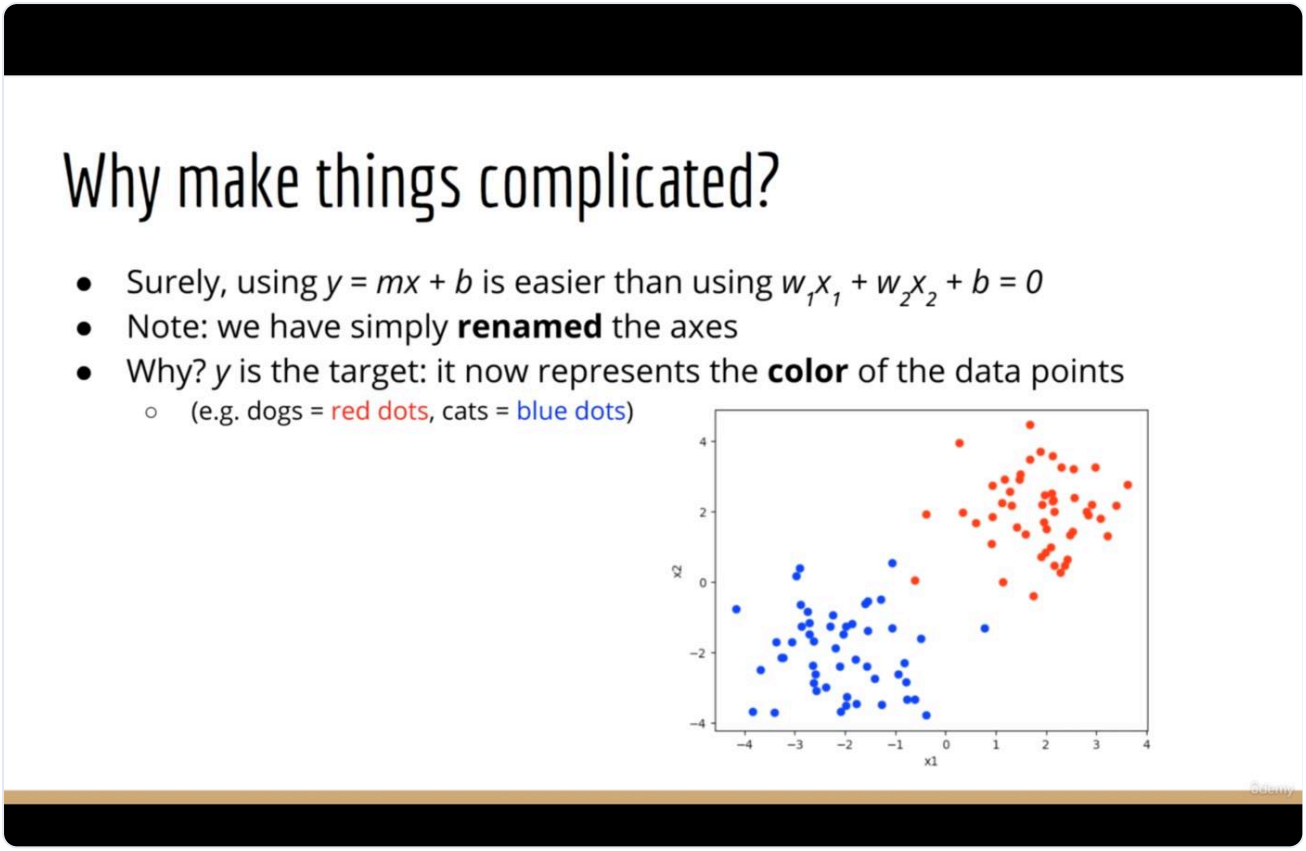


FIGURE 14.5 | Why Make Things Complicated?

In fact, we must. You'll see why this form of the line is useful very shortly. If this is new to you, I would recommend the following exercise. First, notice that all we've done is rename the axes from X and Y to x, one and two. This is important because these axes refer to the input data. If you recall, y is the target. For example, if we're going to classify images of cats versus images of dogs, then we might say dogs are the red dots and cats are the blue dots, but they are not an axis on this graph. So x one represents the horizontal axis and x two represents the vertical axis.

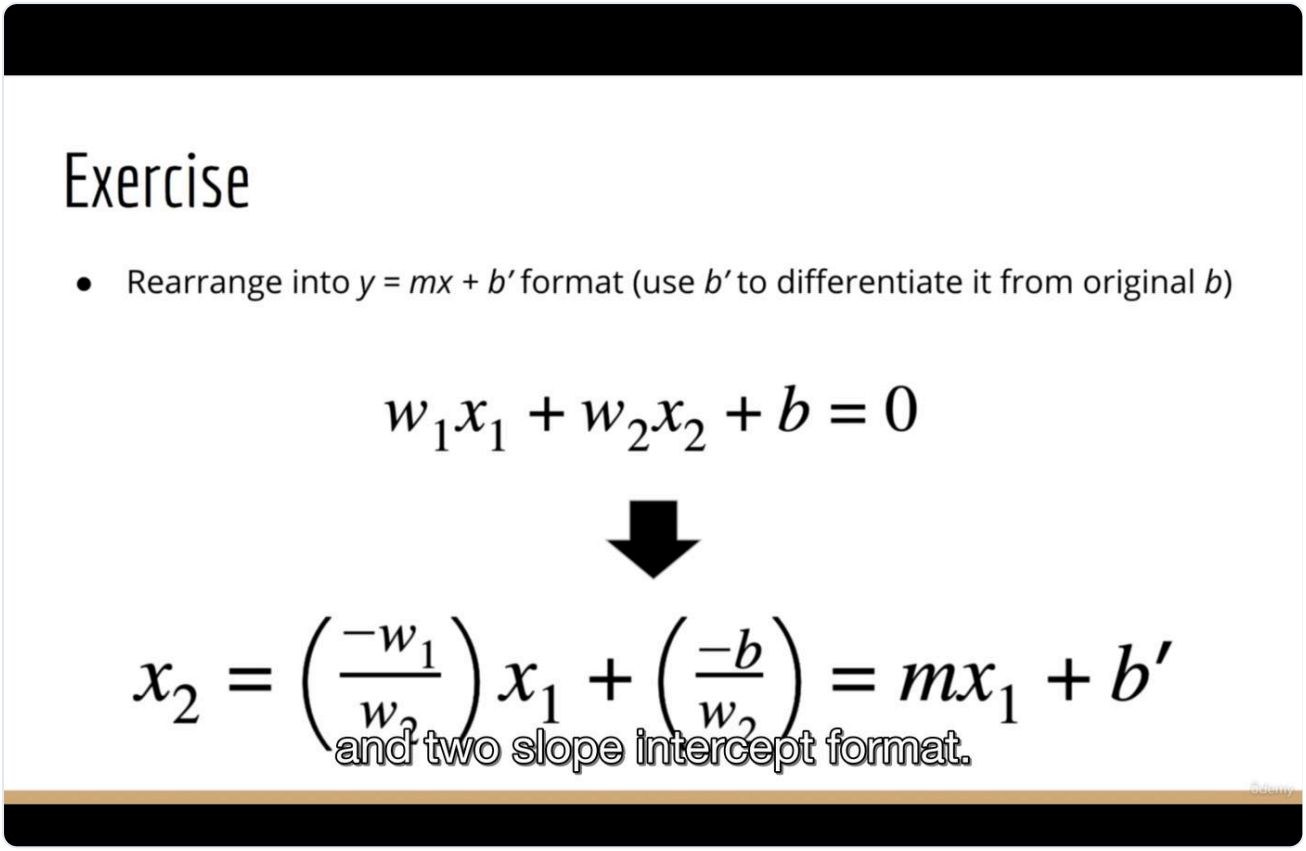


FIGURE 14.6

Y is not an axis. This is different from linear regression where the target y was an axis on the graph. Now that you know this, you should be able to rearrange the equation w1x1 plus w2x2 plus b equals zero and two slope intercept format. What I mean by that is have x two on the left side by itself and then you'll have some slope times x one plus some intercepts. Now it has the same format as y equals m x plus B, except that we use x one and x two instead of x and y. Also note that the B that appears in Y equals m x plus b is not the same as the B that appears in the other equation. So in this slide, I've called the intercept B prime to differentiate it from the original b.

As always, if you can't immediately see how I got this, you should try it by yourself on paper so you know how to do it.

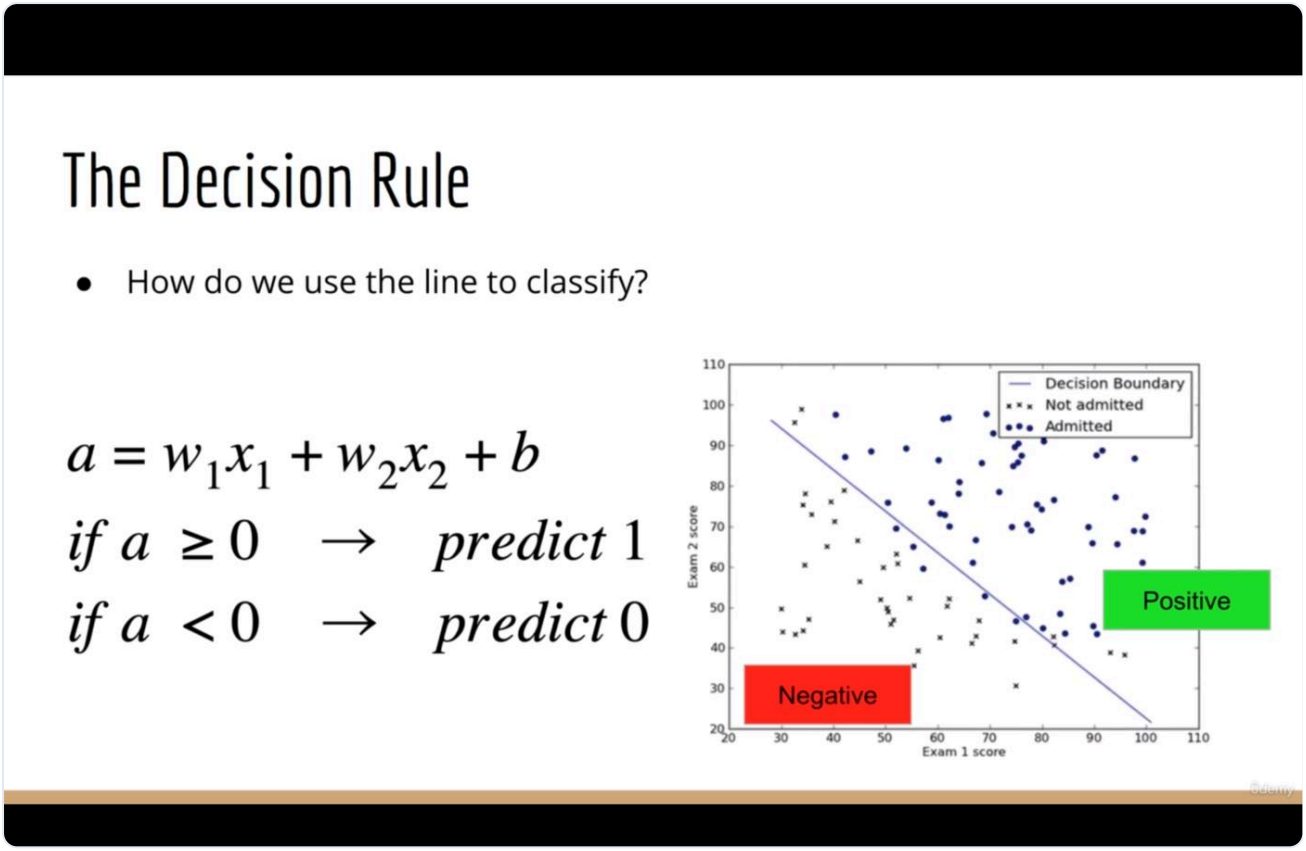


FIGURE 14.7 | The Decision Rule

It should only require elementary school algebra. The next question you might have is how does this line help us make predictions? Luckily due to the rules of geometry, if we plug in a data point X which is not on the line, then either we will get a number bigger than zero or less than zero. In fact, any data point on one side of the line will always give us a number bigger than zero. Any data point on the other side of the line will always give us a number less than zero. Using this, it's very easy to turn this into a prediction model. All we have to do is take in any data point x, which is a vector containing the elements x, one and next to pass it into the expression for our line w one, x one plus w two, x two plus b and then check its sign.

If the sign is positive, we predict one.

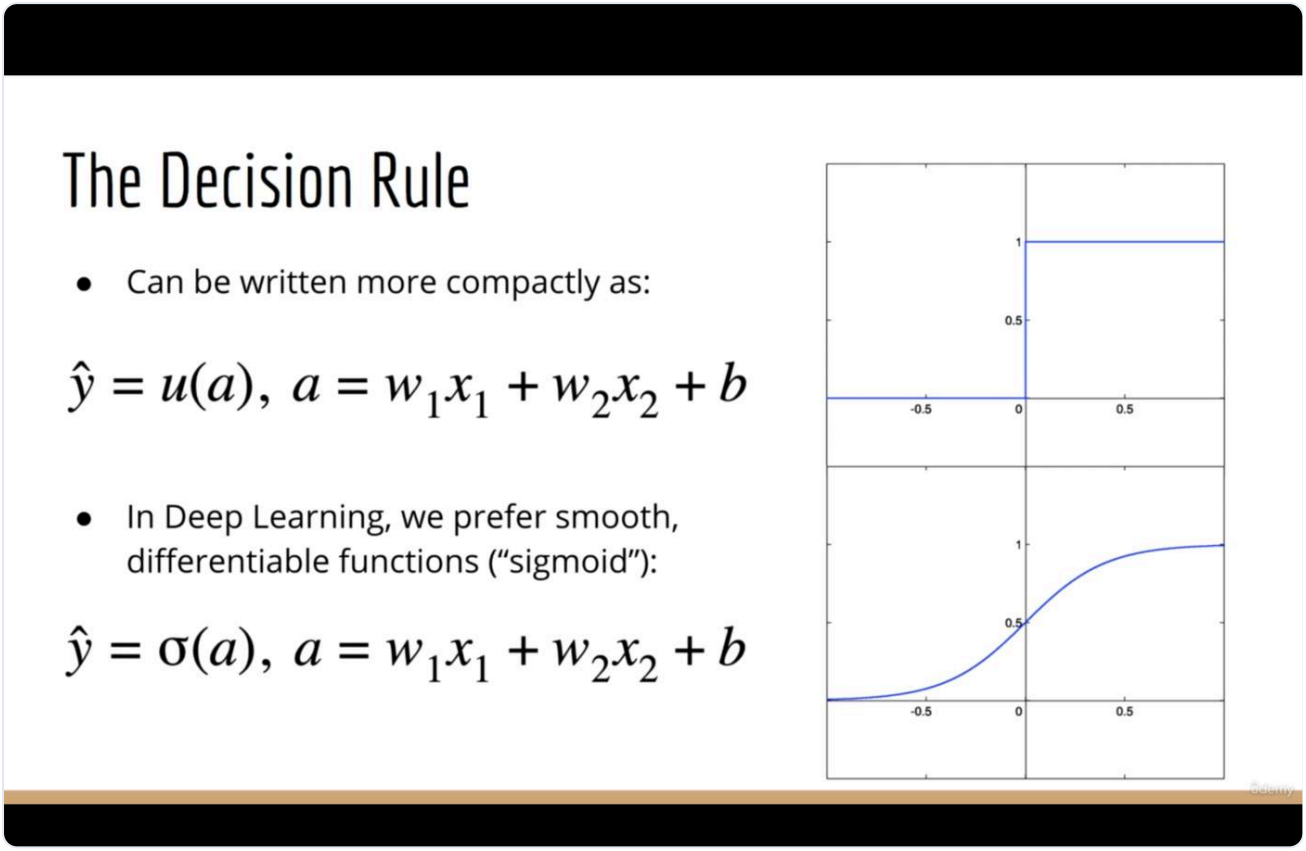


FIGURE 14.8 | The Decision Rule

If it's negative, we predict zero. Mathematically, you can encapsulate this decision rule using the step function, or if you want to be less formal, you can think of it as the picture that you see here. We call the activation. And if the activation is greater than zero, we predict one. Otherwise we predict zero. Now, as you may already know, in deep learning, we really like differentiable smooth functions. So rather than the step function, which is not smooth, what we do is take a smooth version of this called the Sigmoid. The Sigmoid is an S-shaped curve, and it maps the activation to a number between a zero and one.

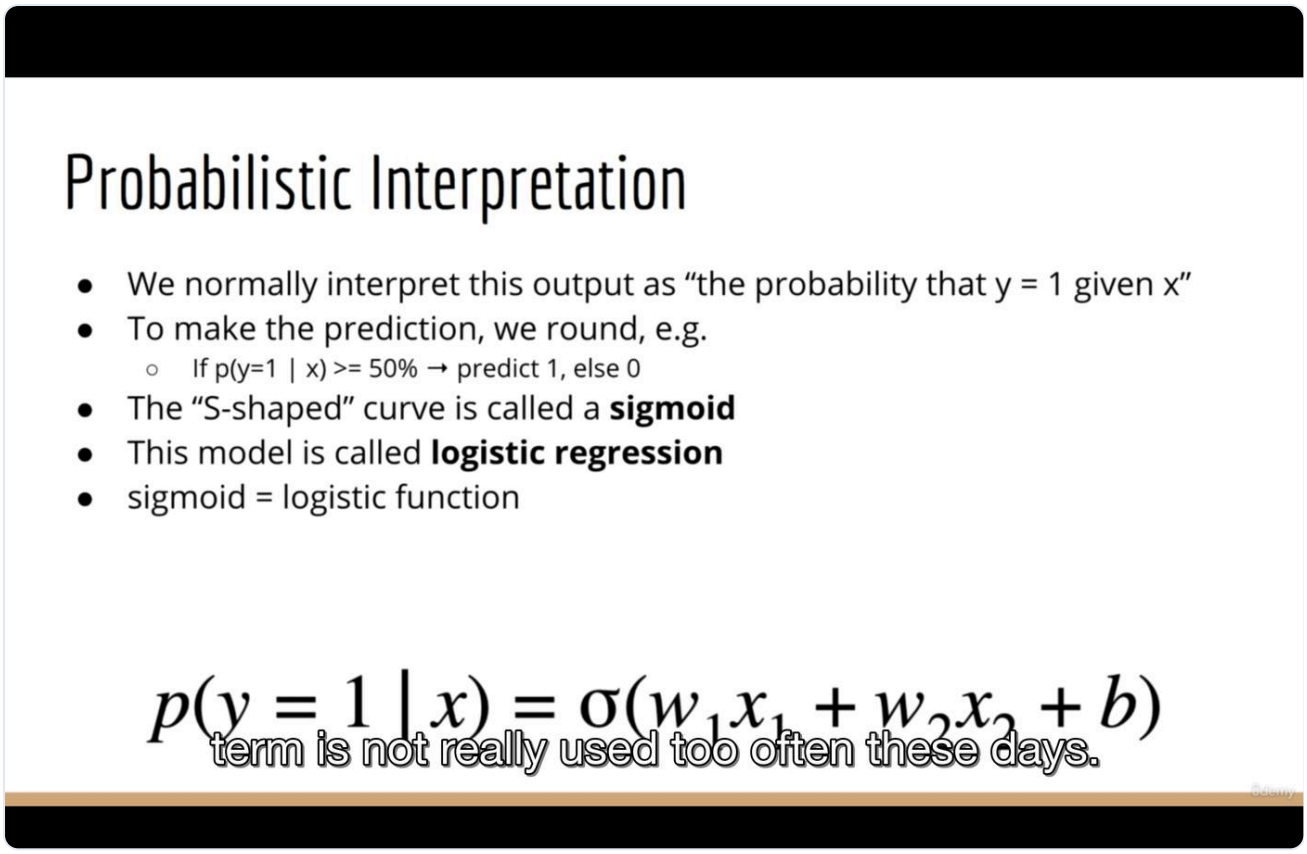


FIGURE 14.9

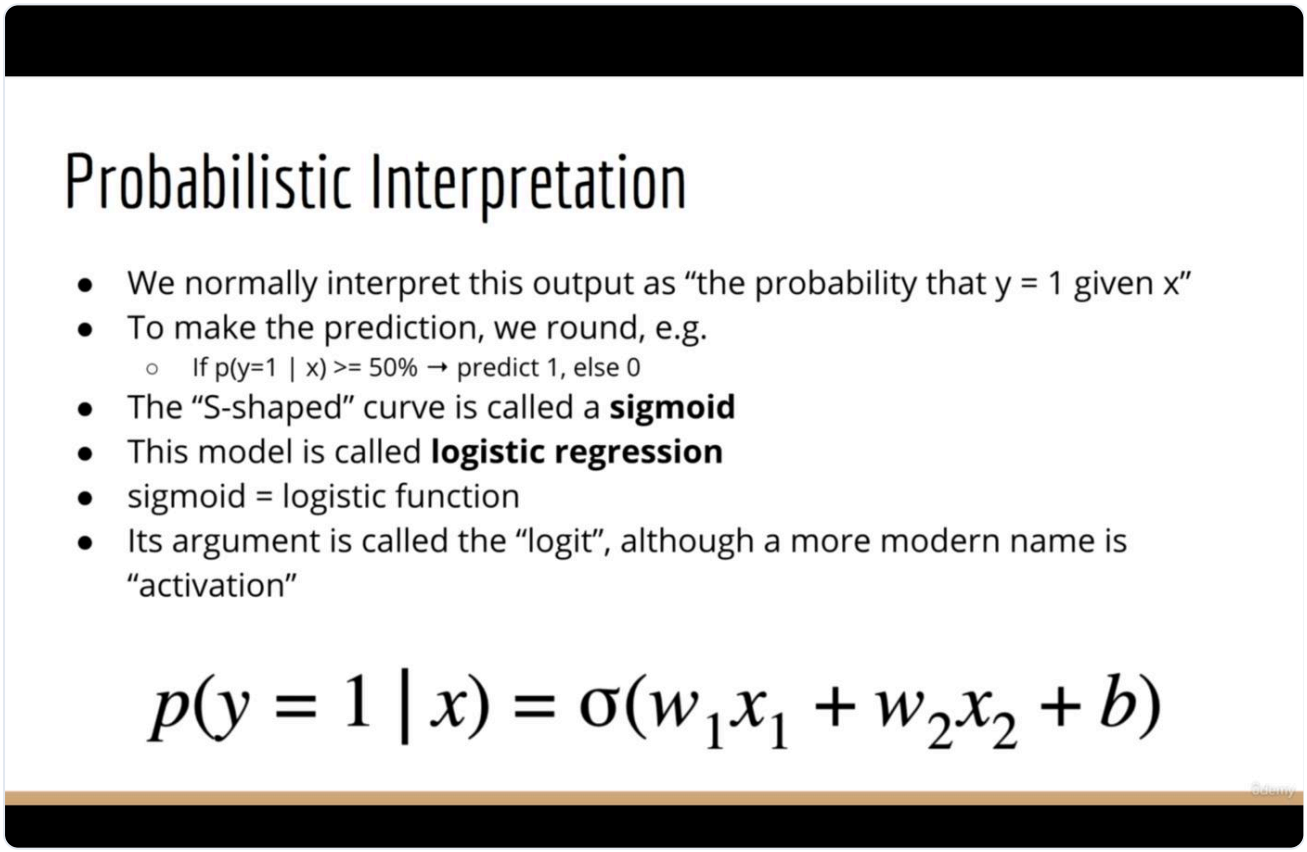


FIGURE 14.10 | To Make the Prediction, We Round, e.g

We usually interpret this as the probability that Y equals one given X. Then when we want to make our prediction, we just round this probability. So if the probability is greater than 50%, we predict one. Otherwise, we predict zero. Again, this is called the sigmoid function, which is important to know for our implementation. As a side note, when we apply the sigmoid function on top of a linear function, we call this model logistic regression. This is because we also sometimes call the sigmoid function in the logistic function, although this term is not really used too often these days. In addition, we also refer to the argument into the logistic function as the logit but a more current and generic term for this is activation.

You might realize that using what we know so far, there is a little bit of a notational challenge.

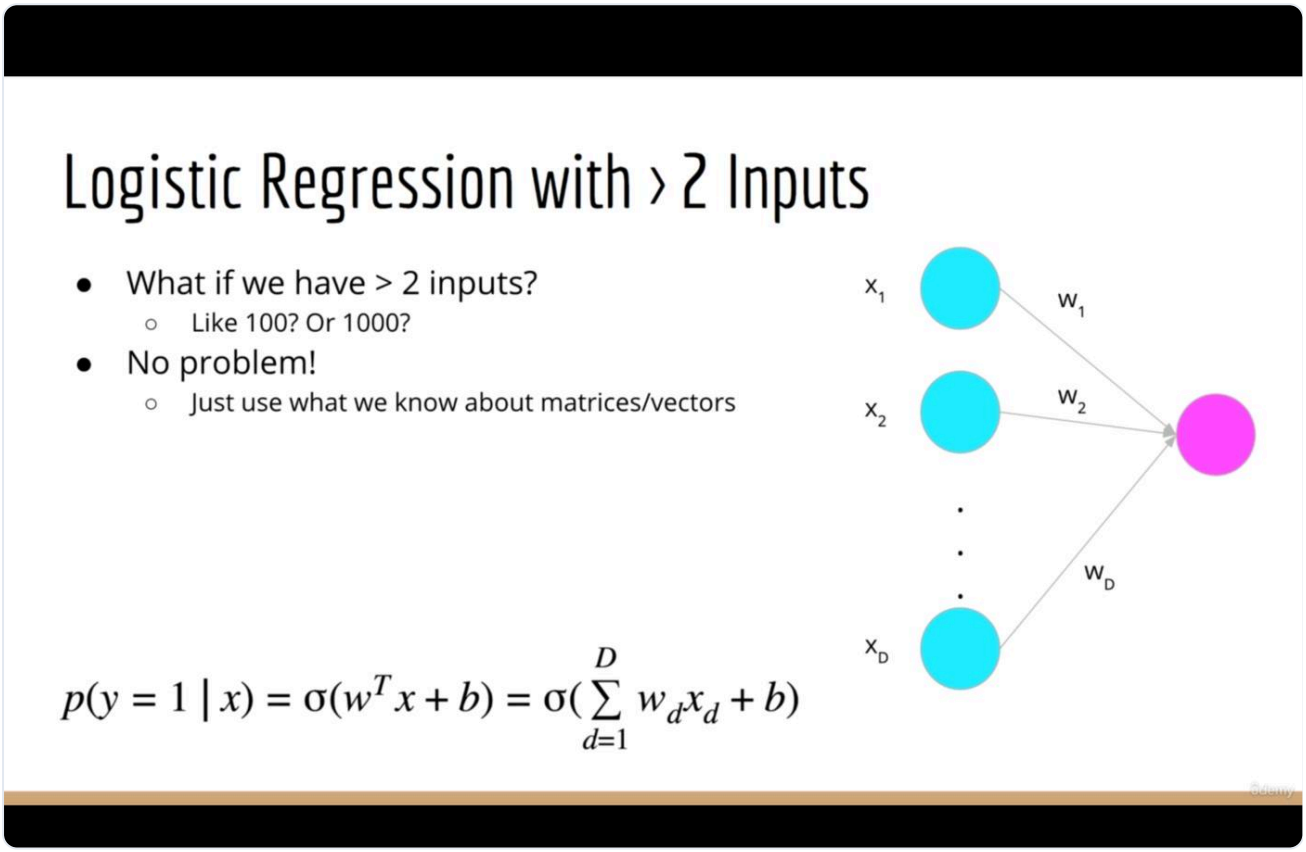


FIGURE 14.11

If we keep writing each component of X separately, we're going to run out of space. It's easy to write w1x1 plus w2x2 when there are only two components of x. But what if there are 100 or 1000? Luckily, we have a mathematical way of representing this. If we consider X to be a feature vector containing each component of X and W to be a weight vector containing each component of W, then our expression just becomes the dot product between w and x. So we can write this in a much more compact way by saying probability of y equals one given x is equal to the sigmoid of W transpose x plus b. Okay. So at this point, let's recap the three concepts we need to cover.

Number one, model architecture. Number two, using the model to make predictions.

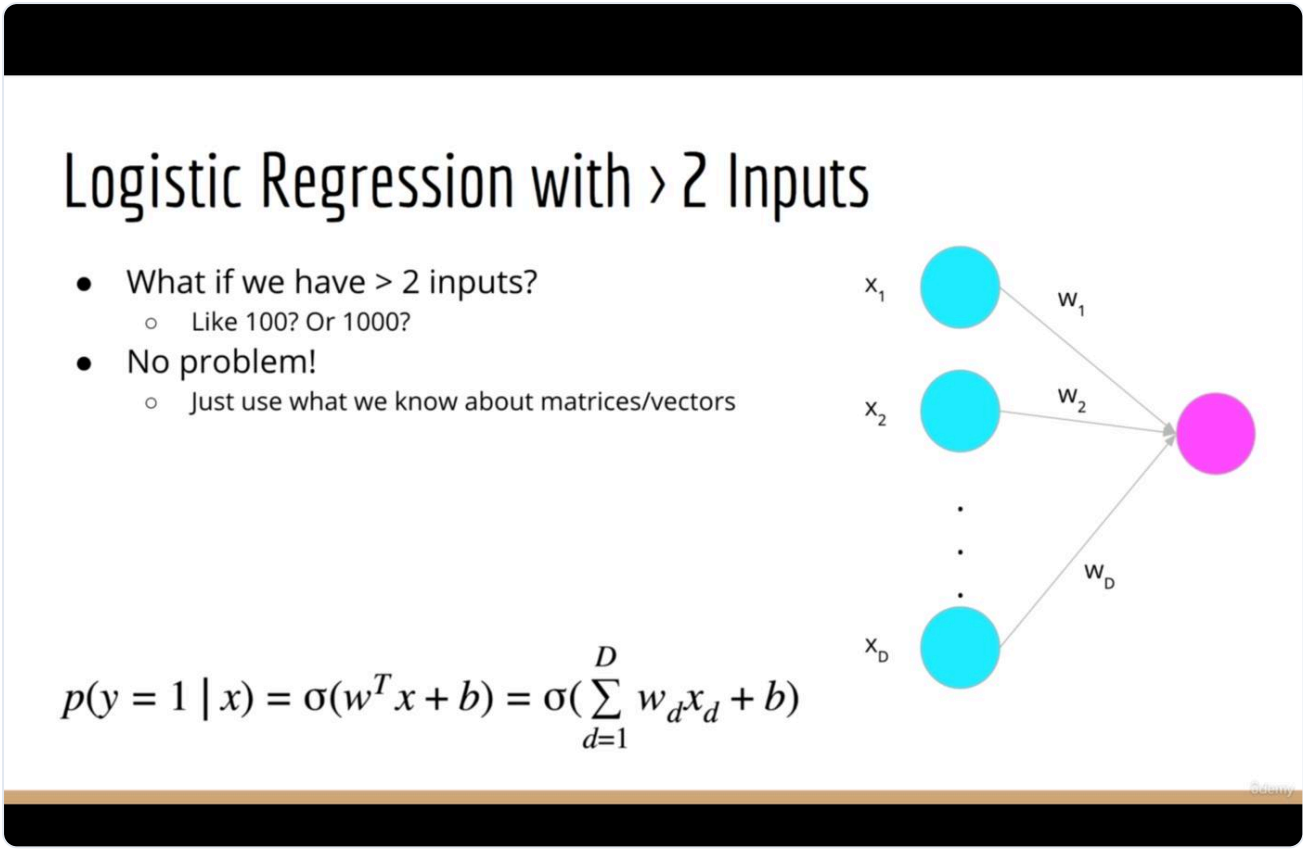


FIGURE 14.12

And number three, model training we just covered. Number one. And number two. You know what the model architecture is and you know how to use it to make predictions. Basically, given some input X, you know how to plug it into the previous formula to get some output prediction y hat. The final step is training. As I promised earlier, we are going to follow the exact same general steps as we did when we learned about linear regression. It may surprise you, but these are the same steps that we are going to follow for every subsequent example in this course. That's why I said it's the simplest example.

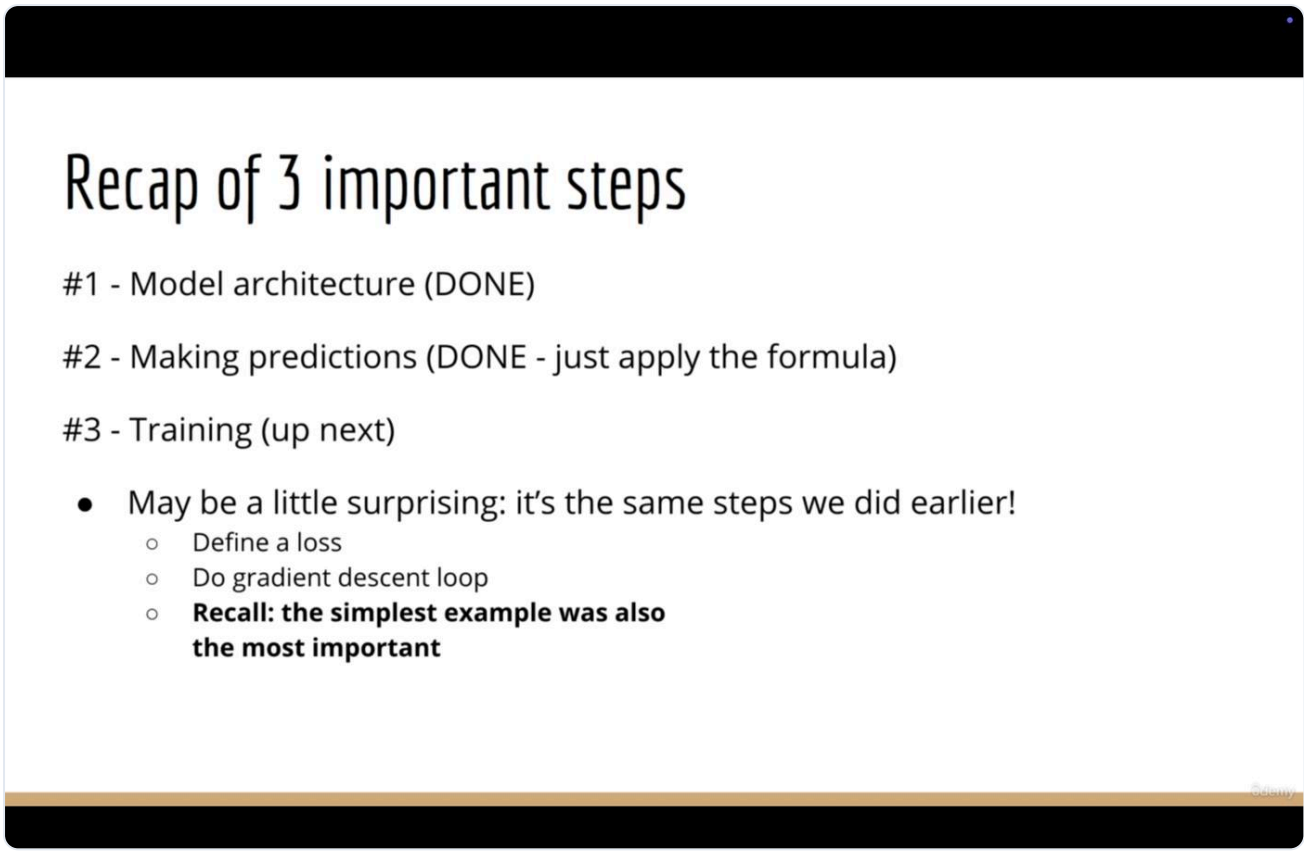


FIGURE 14.13 | Recap of 3 Important Steps

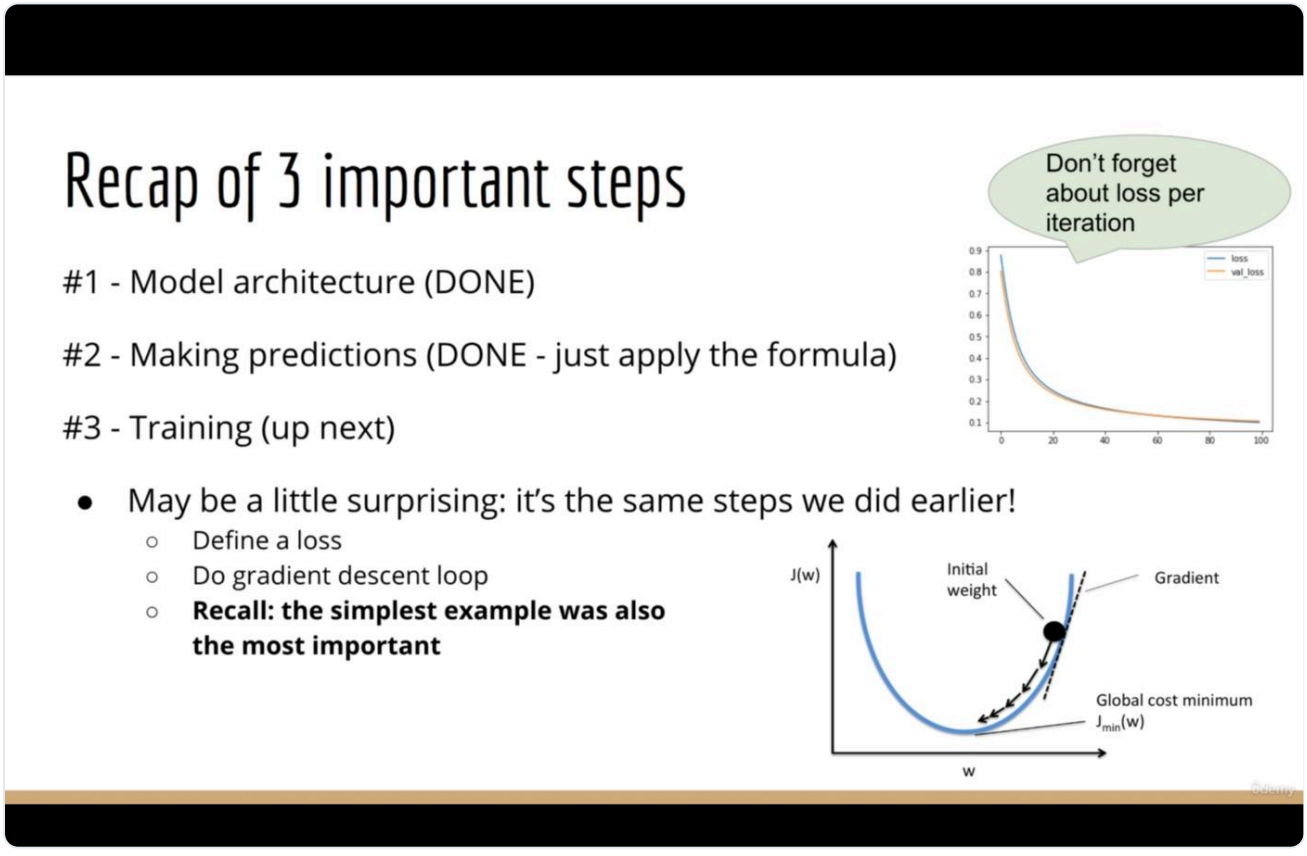


FIGURE 14.14 | Recap of 5 Important Steps About Se per

But it was also the most important. Now you see why. So as you recall, we need to define a loss function. Then after we've done that, all we need to do is apply the gradient descent procedure to update the parameters of the model in order to minimize that loss. Once we've completed training, we can plot the loss per iteration to ensure that the loss converges successfully, and we can test our model by checking its accuracy on the train and test sets. The main difference in the training process between regression and classification is that we have a different loss function. As you recall, in regression our target is a real number and our prediction is also a real number.

FIGURE 14.15 | Difference: New Loss

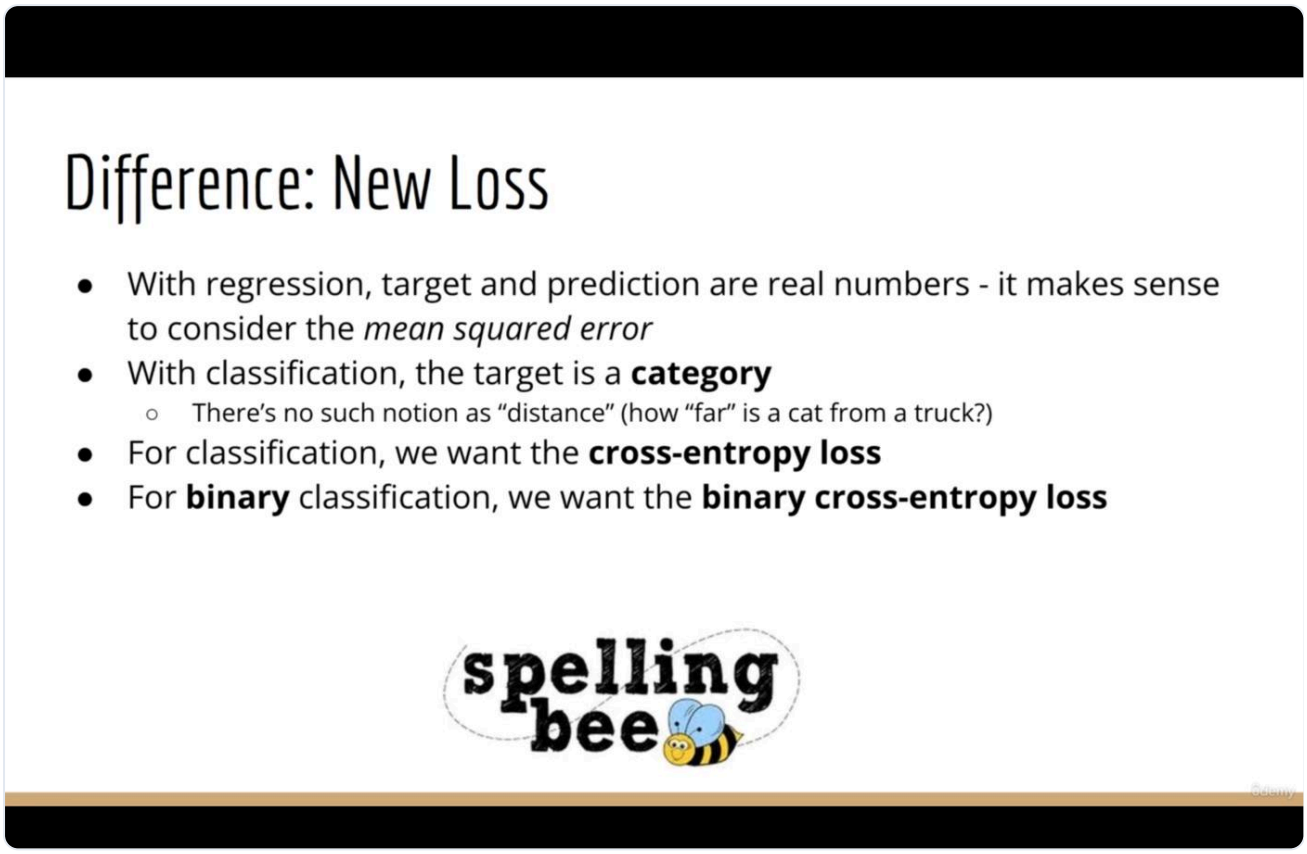


FIGURE 14.16 | Difference: New Loss

FIGURE 14.17 | Difference: New Loss

In this scenario, it makes sense to use a loss like the mean squared error or MSE. But in classification our target is a category. Our model output is the probability that the input belongs to each of the categories. So the mean squared error doesn't make any sense in this scenario. In fact, for classification, what we want is the cross entropy loss. There's one little wrinkle here, which is that we're doing binary classification in which there are only two possible classes, and the corresponding loss is called the binary cross entropy loss. In general, if your model can handle any number of classes, then you'll use the regular cross entropy loss. But we'll discuss that later in this course.

The reasoning behind the cross entropy loss is quite a bit more complicated than the mean squared error, so we won't discuss that theory in this lecture. If you want to learn more, then you should check out the in-depth sections of this course. For now, all you really need to know is the PyTorch API. Since as you recall, this course is not about mathematical theory but rather how we do things in PyTorch.

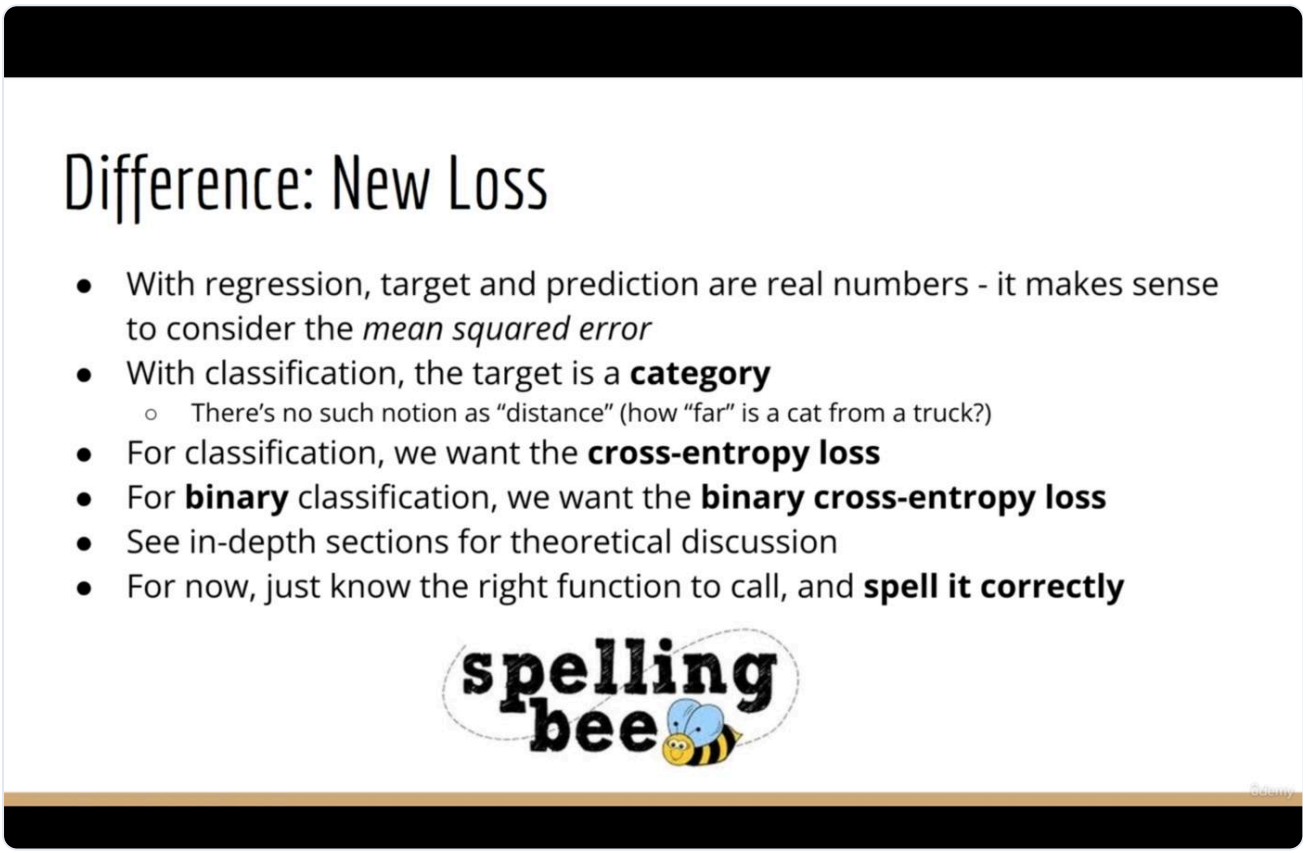


FIGURE 14.18 | Difference: New Loss

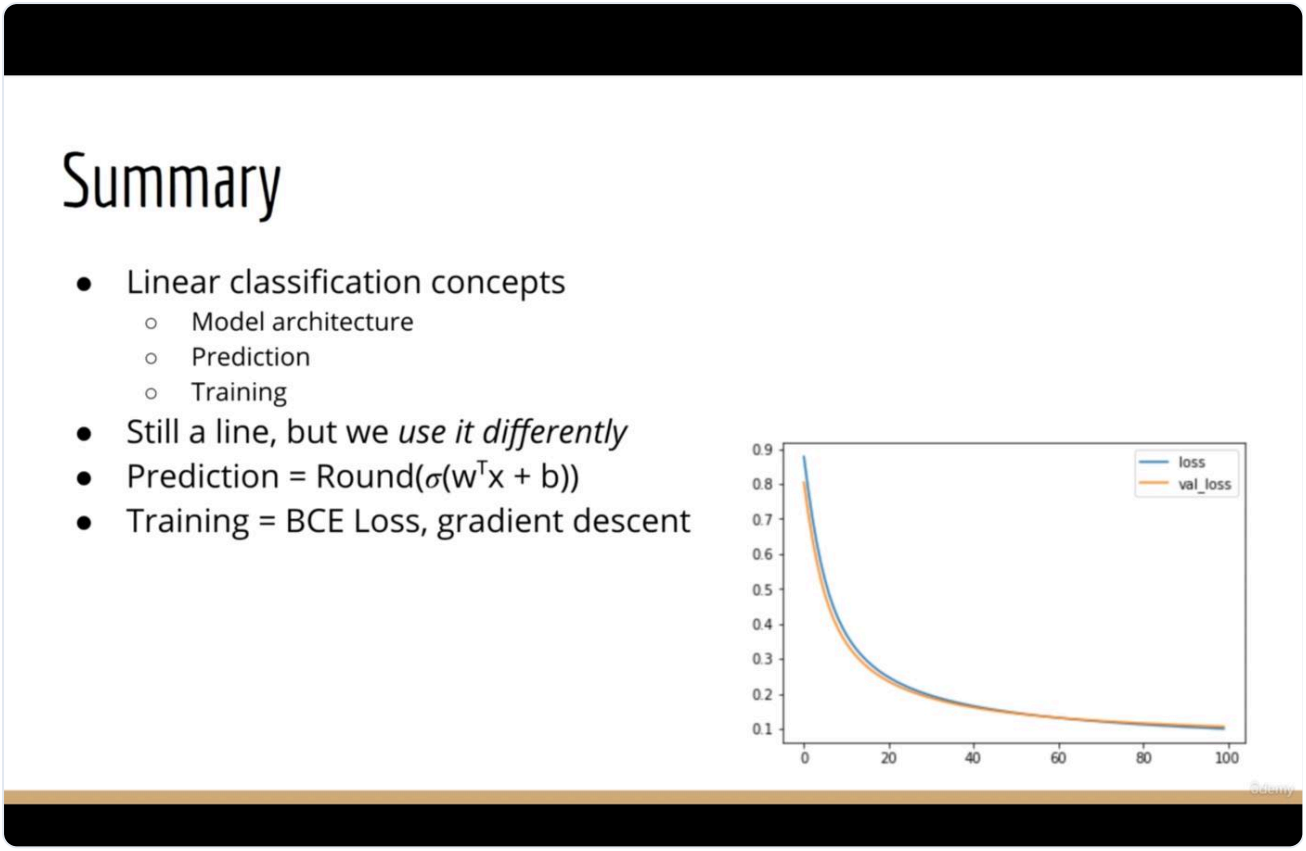


FIGURE 14.19

Therefore your job really is knowing the right function to call and spelling it correctly. Indeed, in this computer science course, what matters isn't your math skill but your spelling ability. To summarize this lecture, we went over the concepts behind linear classification. Our goal was to consider these three things. Number one, what is the model architecture? Number two, how do we make predictions? And number three, how do we train the model? As you saw, it was very similar to linear regression in the sense that our model is still a line or a hyperplane, but it's how we use this line that's different in regression. Our goal is to get the line close to the data points, but in classification, our goal is to separate the data points.

In regression, we just pass our data into W times X plus B and that is our prediction. But in classification we still pass our data through W, times X plus B, we just have the additional steps of applying the sigmoid function and rounding the output probability for number three. Again, it was very similar to our previous example where we set up a loss function and then use gradient descent to find the weights that minimize it. But the difference in classification is that instead of using the mean squared error, we use the cross entropy. And in our special case of binary classification, we use the binary cross entropy.

LECTURE 15

Build the Model

In this lecture we you're going to do code preparation for our linear classification script this example will look at a new classification dataset. So this lecture will go overloading the data as well. To recap here are the general tasks that we need to complete.

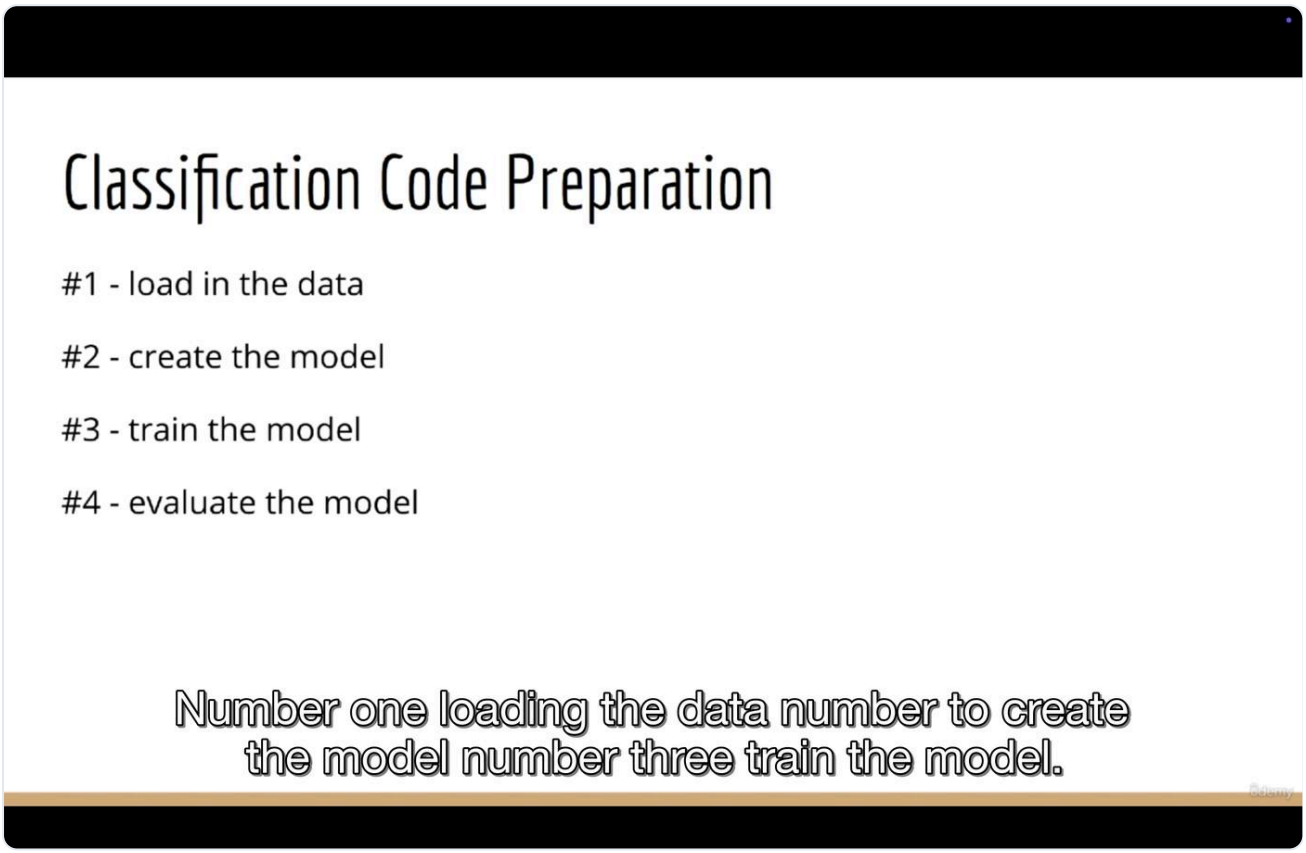


FIGURE 15.1 | Classification Code Preparation

Number one loading the data number to create the model number three train the model. And number four evaluate the model let's start with loading in the data. For this example we'll be looking at the famous breast cancer dataset. This data set happens to be included as part of the scikit-learn API. You'll notice that in this course we'll be looking at quite a few pretty famous datasets.

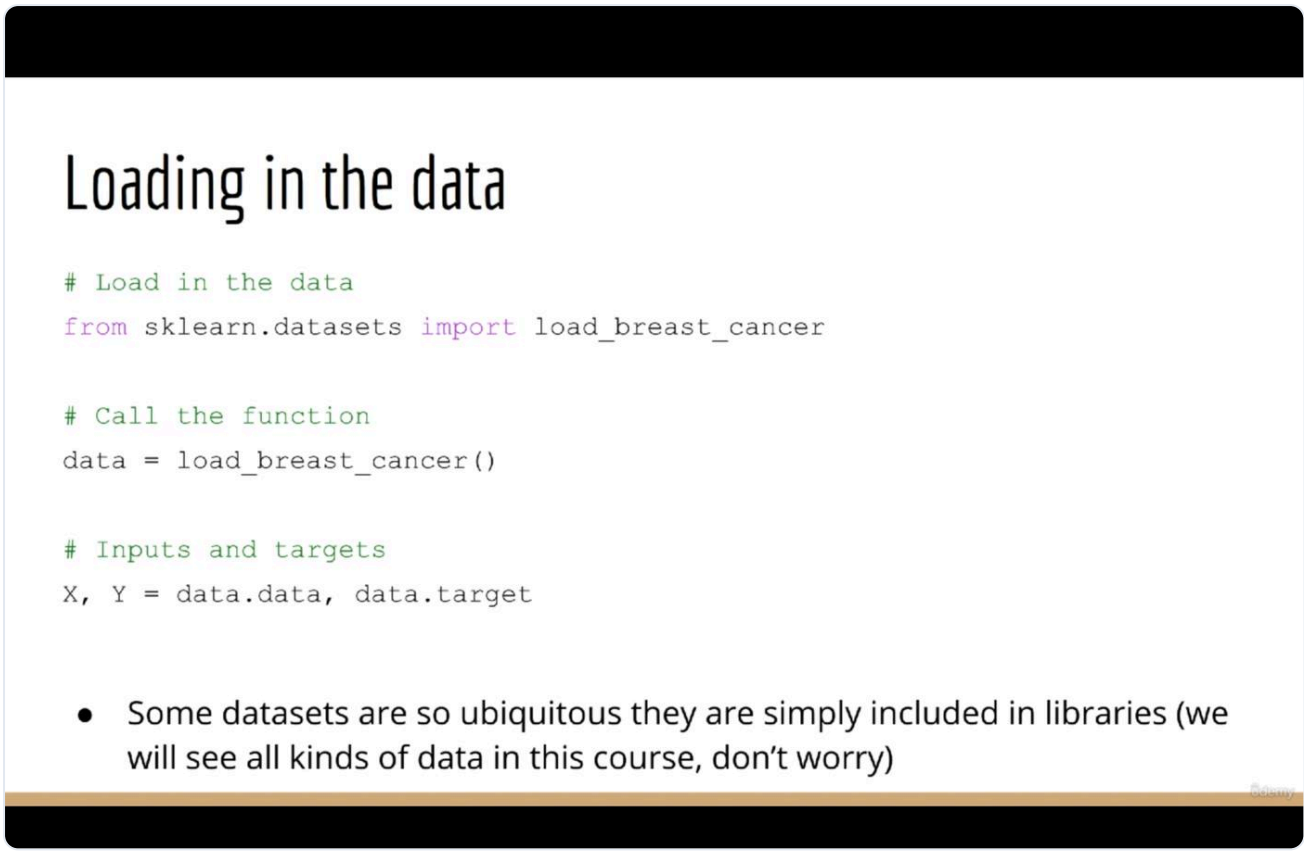


FIGURE 15.2 | Loading in the Data

These datasets are so famous that they are often included in the various machine learning libraries. So there's no need to download the dataset as we did in our previous example. Of course there will be plenty of examples in this course where the data do not come from libraries. So you'll get experience with each loading in data from scikit-learn is super easy. We just call the function load breast cancer which returns a data object. This data contains the X's and Y's so we'll need to access them using the objects attributes in particular the inputs can be accessed using the data attribute and the targets can be accessed using the targets attribute.

There are two ways we'll want to pre process the data before using it. First the data isn't normalized. You already learned from our earlier example why this is a good idea. If you did the exercise I suggested therefore we will use the scikit-learn module StandardScaler to normalize it. Second we want to split the data into train and test sets. Intuitively this is because we want to get a good idea of how the model will perform on data.

FIGURE 15.3

It hasn't seen before. Not on data it has already seen for the data we've already seen. We already know the answer so machine learning isn't necessary. The data we really care about is the data we have not seen. For example if you build a fraud detector you want to be able to ask your model whether a new transaction is fraudulent. This is important because your model might do very well on data. It's already seen but poorly on data it hasn't seen. We'll discuss this more in a later lecture. In any case this code shows you how to do both of these steps. Notice how we already use this idea of train test splits with the StandardScaler we fit the StandardScaler on the training data only and we apply the standardization on the test data using the fitted mean and variance of the training data.

Once we've prepared our data it's time to build a model. As you know the linear classifier is almost the same as linear regression just with one extra step.

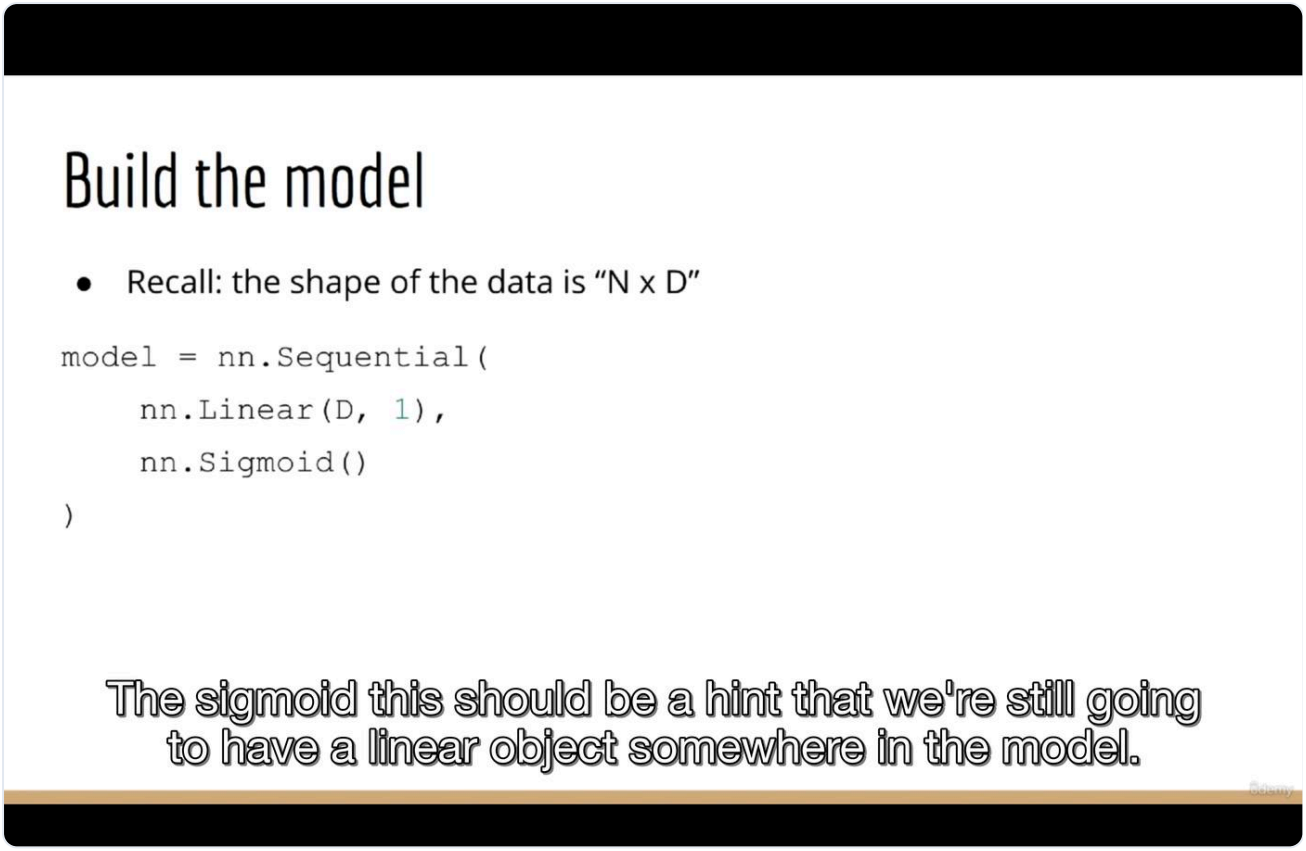


FIGURE 15.4 | Build the Model

The sigmoid this should be a hint that we're still going to have a linear object somewhere in the model. Well it just so happens that the sigmoid is also represented in PyTorch with an object the sigmoid object it should make complete sense then if we combine the linear object with the sigmoid object in sequence and in fact that's exactly what we've done. PyTorch allows you to easily stack these computations steps in sort of a wrapper objects called Sequential what you're telling PyTorch is I want my model to apply these functions in this order and the functions we want to apply in this case are the linear model than the sigmoid there's one detail here that I want you to notice and this is that when we create the linear layer the input sizes D while the output size is 1.

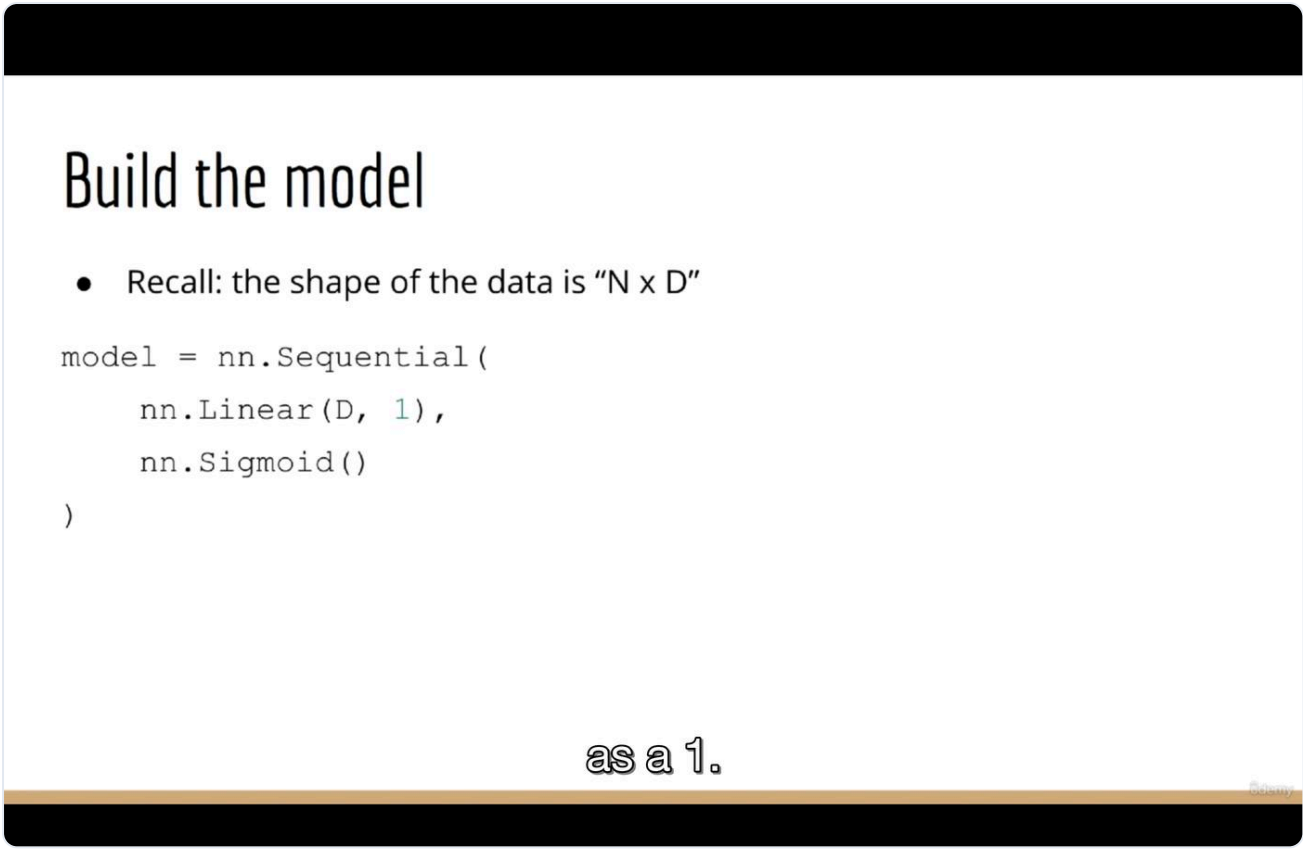


FIGURE 15.5 | Build the Model

As you recall our data matrix is of shape n by D where n is the number of samples and d is the number of features. So we'll have one input for each feature column in the dataset. In addition we have one output which represents the probability that the output should be classified as a 1.

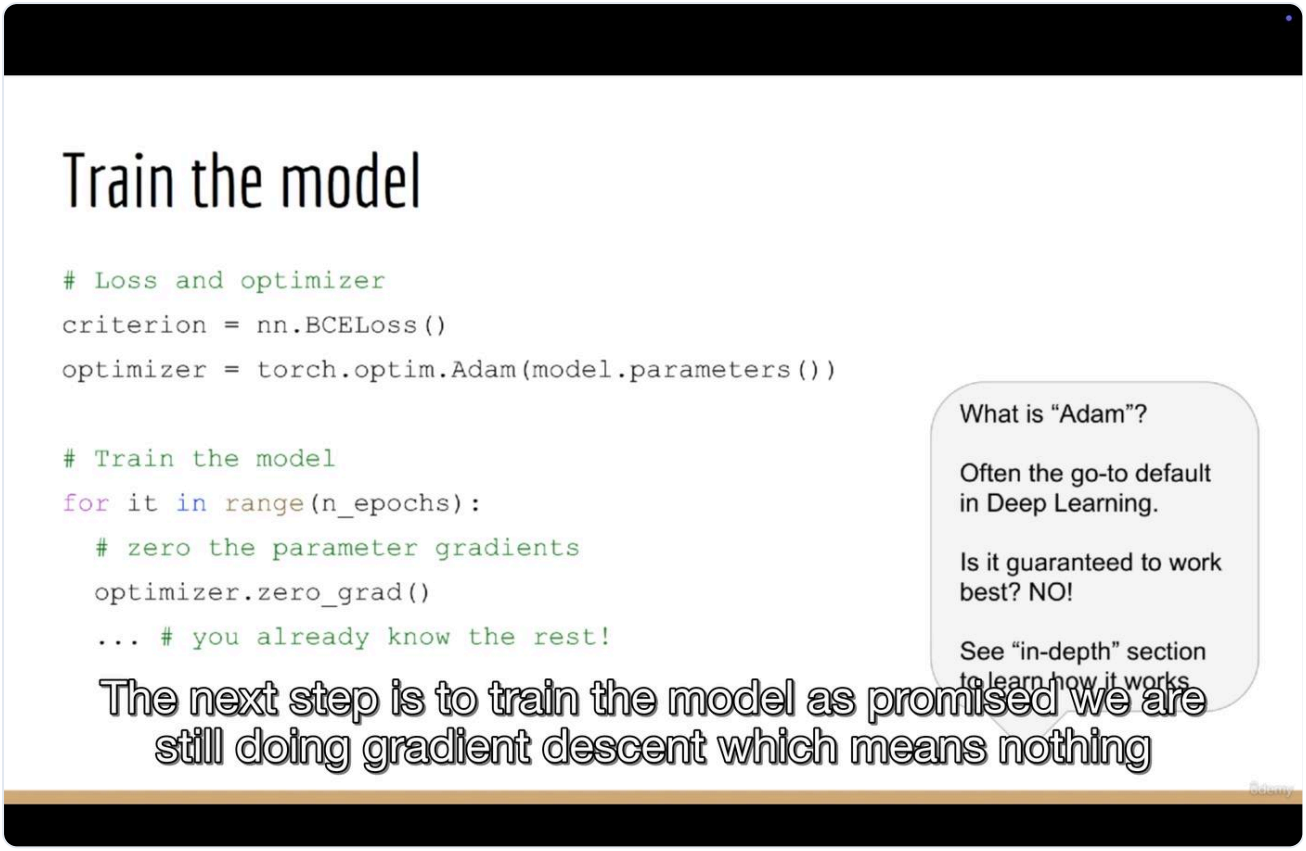


FIGURE 15.6

The next step is to train the model as promised we are still doing gradient descent which means nothing about our gradient descent loop is going to change. We still do zero grad get the output calculate the loss call backward and do one step of a gradient update. What's different is that we'll be using a different cost function but also a different optimizer as mentioned previously. Our loss function for binary classification is the binary cross entropy which is performed in the object BCE loss. In addition we'll be using the Adam optimizer which has become the go to default in deep learning in recent years. Normally I would say that the gradient optimizer you choose is like a hyperparameter so you should always experiment to see what works best in practice.

Many people simply choose Adam by default. This does not mean that Adam is actually guaranteed to work best. So you should still try other methods and observe the results yourself.

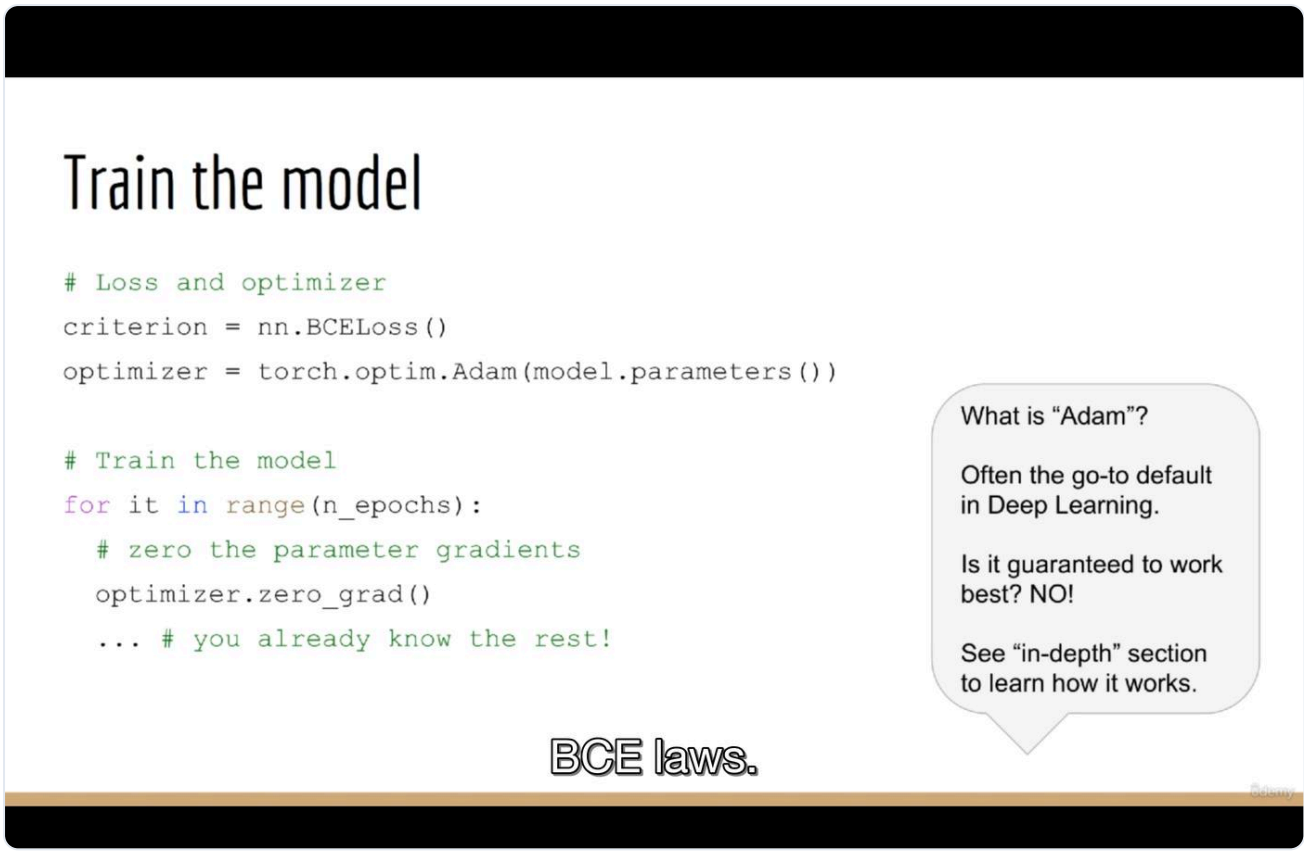


FIGURE 15.7 | Train the Model

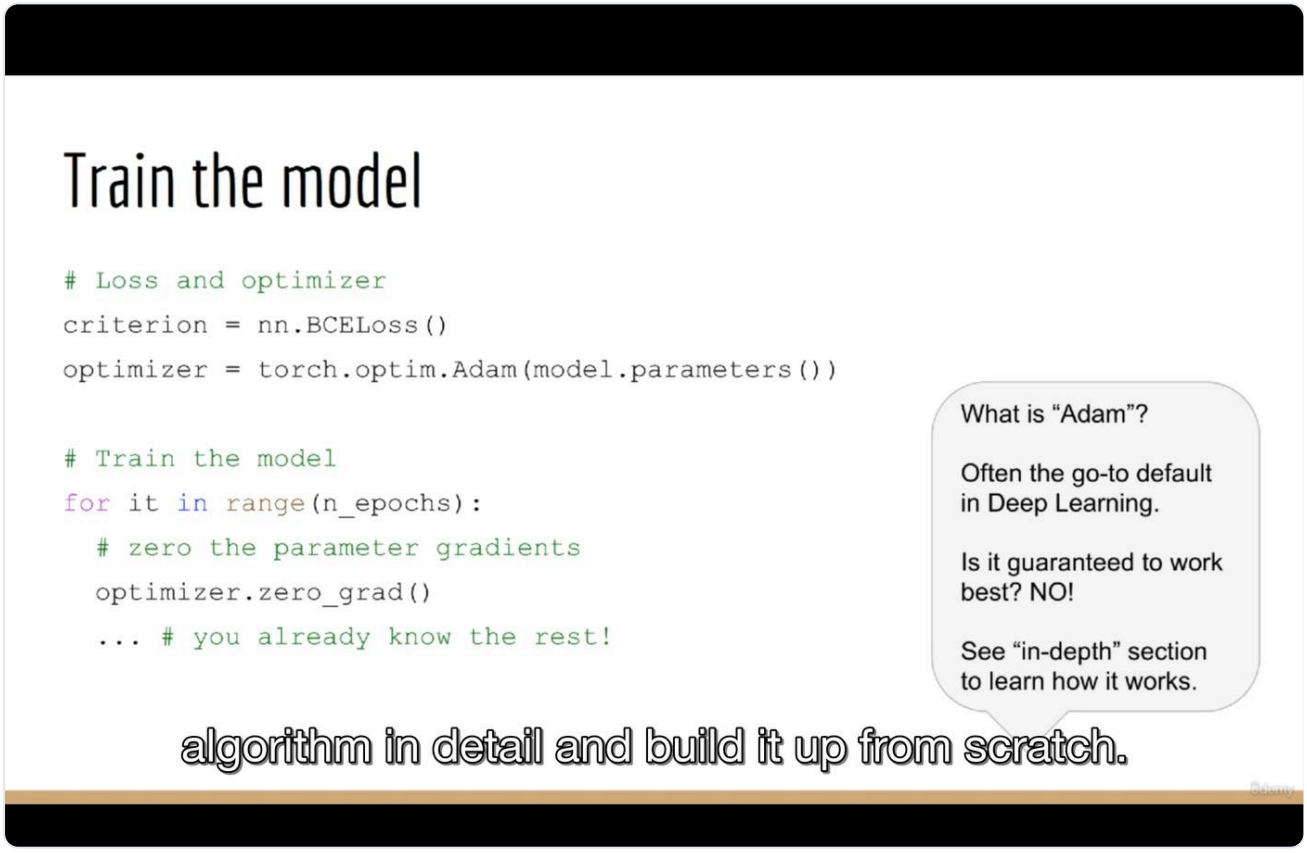


FIGURE 15.8 | Train the Model

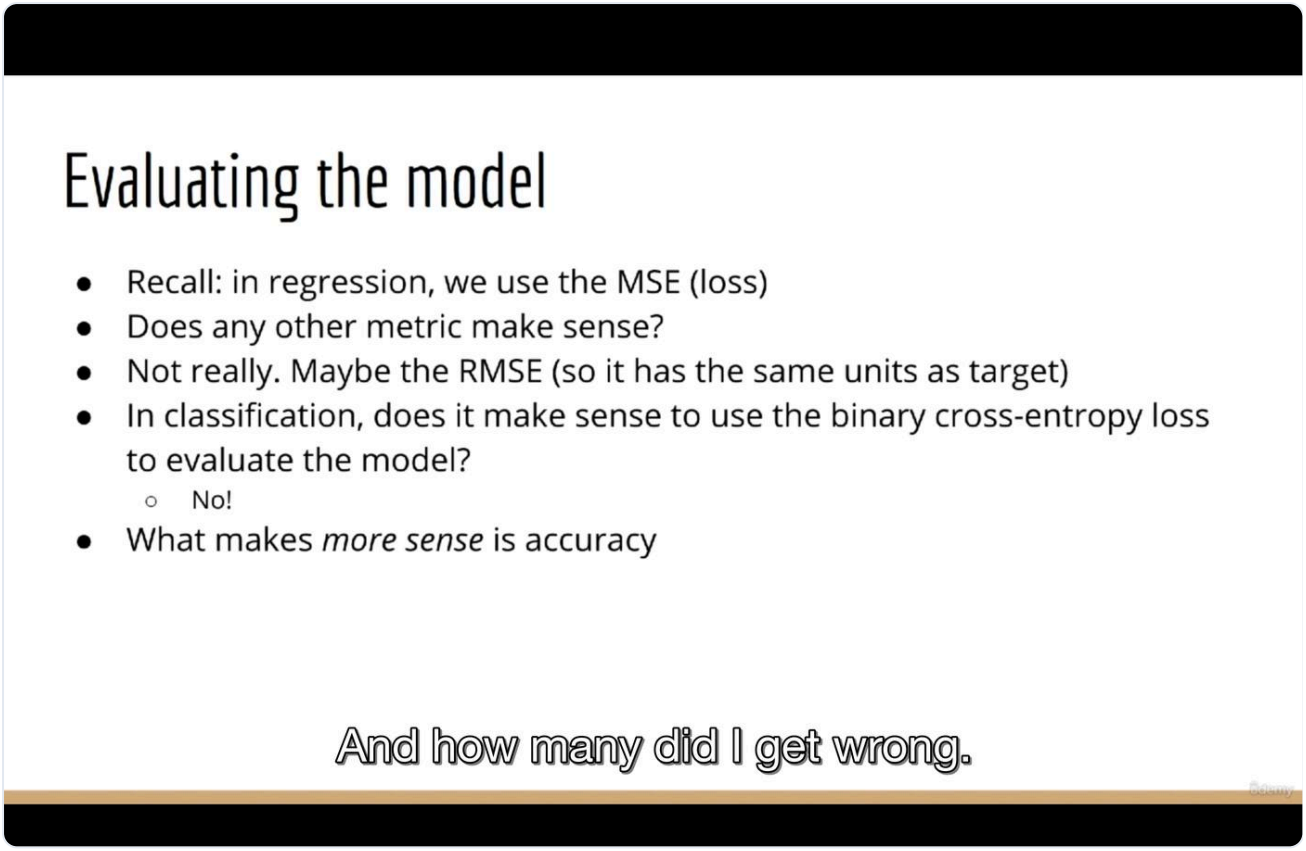


FIGURE 15.9 | Recall

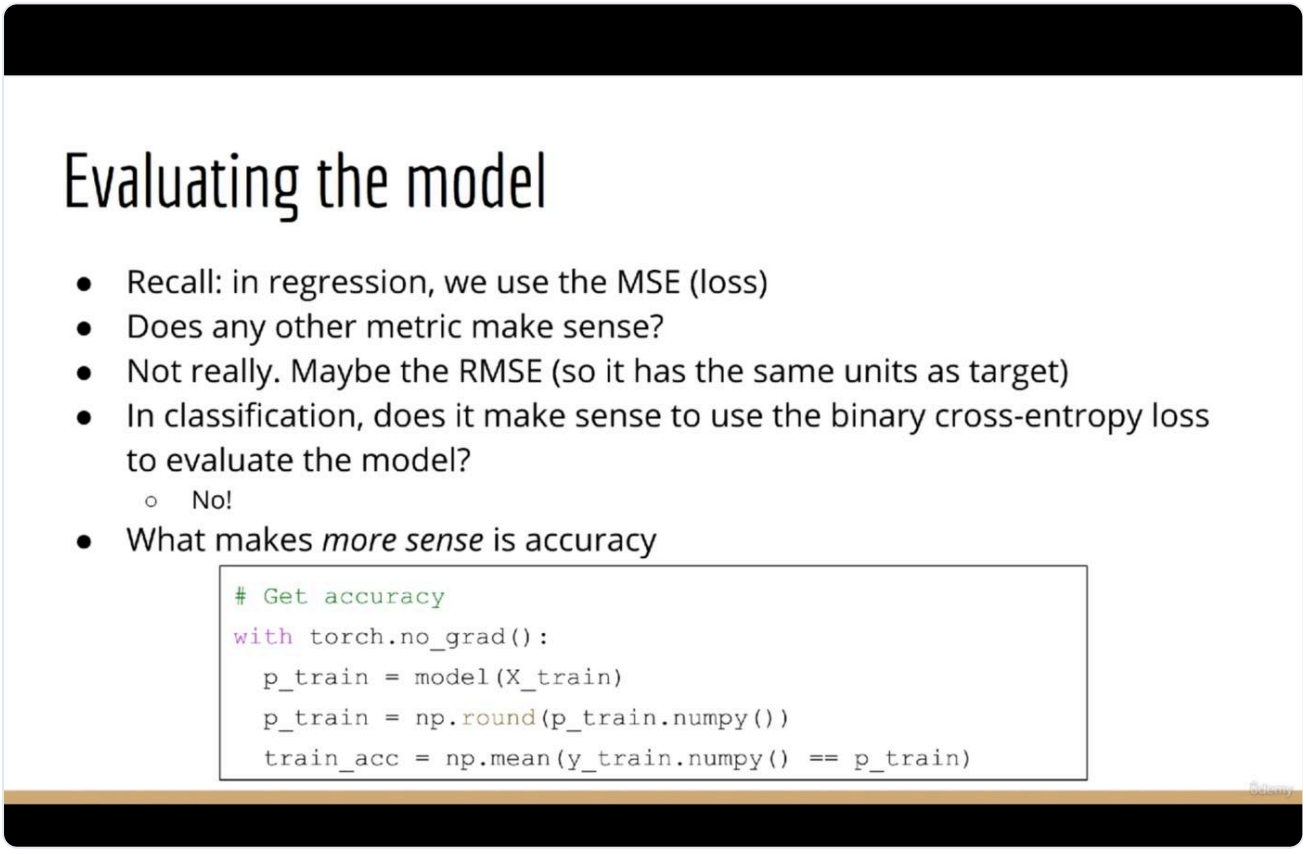


FIGURE 15.10 | Recall

If you want to learn more about Adam you're encouraged to check the in depth section of this course. And if that's still not enough then you'll want to go through the in-depth course where I discuss this algorithm in detail and build it up from scratch. The last thing we're going to do in our script is evaluate the model. This is different in classification compared to regression and regression. We use the mean squared error as our loss but we also use it as our evaluation metric. It kind of makes sense since I don't think any other metric would be significantly more advantageous. The mean squared error is kind of a natural way to look at the regression error alternatively you could look at the root mean squared error which is just the square root of the MSE so that it's in the same units as the target but still taking the square root is kind of a trivial transformation with classification.

It's a different story. We have the cross entropy as a loss which is not a natural or intuitive measurement. What we really care about is accuracy. Out of all my predictions how many did I get right. And how many did I get wrong. So in order to evaluate our model we're going to calculate both the train and test accuracy. This necessarily also involves making predictions with the train model as well so we'll learn how to do that too. Of course it's not so different from how we make predictions with our regression model. We just need to round our prediction to 0 or 1 since the targets are encoded as 0 or 1.

LECTURE 16

Test Loss and Train

Check the `3.PyTorch-Linear-Classification.ipynb` for the code.

In this lecture we are going to do an example of classification and code using PyTorch. Our problem for this lecture will be to predict whether a patient's diagnosis of breast tissue is malignant or benign. This is a classic application of machine learning and deep learning especially has made great progress in this field. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at.

To start we're going to have all our usual imports torch torch dot nn NumPy and matplotlib. Next we are going to load in the data. This dataset is part of scikit-learn so we're going to import the function load breast cancer from the module sklearn dot datasets. Next we're going to call this function and assign it to a variable called data. Next we're going to check the type of the data variable since it's not clear what it returns. We can see that it returns an object of type Bunch luckily I've already inspected this object. So I know that it acts kind of like a dictionary in that way we can call the function keys and it's going to show us all the keys that we can access from this object so we see data Target Target names description feature names and file name so the first two keys data and target refer to the input data X and the targets y respectively.

So I'm going to call data dot data dot shape to check the shape of X okay. So we see that it's five sixty nine by 30. So that means we have five hundred and sixty nine samples in 30 input features next. I'm going to print out the targets here so as you can see it returns a one dimensional array of zeros and ones as expected now luckily even though the targets are just zeros and ones we can still determine what they mean by printing out data. Target names as promised. They stand for malignant and benign. Next we can confirm that the length of y is equal to the length of X 569 next. Note that we can also print out the input feature names as well so we can see things like mean radius mean texture mean perimeter and so forth.

Of course you're free to look up this dataset online if you want to learn more about it but remember that we would just like to treat this as a geometry problem. Next we're going to split the data into train and test. You'll notice that I'm importing things as we go along since that helps tell the story. But in general you'll find that it's more conventional to put all the imports at the top. So here we're going to import the function train test split from the module sklearn dot model selection then we're going to call this function and make the test set 33 percent of the original data set. This returns X train x test y train and Y tests then we're going to assign the shape of X train to the variables N and D which may or may not be useful later next.

And this is important in deep learning. We want to scale our data the basic idea behind this is that because the output is a linear combination of the input you don't want one inputs have a very large range say one million and another input to have a very small range like say zero point

zero zero one. If this happens then the weights will be too sensitive when the input has a large range and not sensitive enough when the input has a small range. So the typical way to deal with this is a normalization or standardization. Basically this just means subtracting the mean and dividing by the standard deviation luckily scikit-learn already comes with a class for this called StandardScaler so we fit transform on X train and then we transform on X test.

Remember that it's important not to expose any of our test data to the training pipeline next. It's time to do our PyTorch work as you saw earlier. We first need to build a model object which is now a sequence of modules it takes in a list of two modules linear and sigmoid the linear layer has an input size D and an output size 1. As mentioned we want to apply the sigmoid activation so that the outputs are in the range 0 and 1. Next we are going to prepare to train our model. So we'll need a loss and an optimizer the loss we're going to use is the binary cross entropy loss or BCE loss which is the appropriate loss for binary classification. The optimizer will use is called Adam which is a common default chosen by deep learning practitioners next we're going to convert our data into torch tensors before we enter our training loop as you've seen before.

This involves calling the function torch dot from numpy. In addition we still need to convert all the data to float 32 one detail you may have missed. If you're not accustomed to PyTorch is that we want to reshape the targets to be a 2D array of size n by 1 instead of just a 1D array of length N in other libraries such as scikit-learn Keras and TensorFlow a 1D array is appropriate but that's not the case in PyTorch. Next we have our main training loop as promised we're not going to spend much time explaining this because it's pretty much exactly the same as before. There is one small additional detail here but it's one that doesn't require any new knowledge.

Now that we have both a train and test set we want to be clear about which data set we use where. So when we do the training portion meaning a step of gradient descent will want to use the train set X train and Y train. This involves doing a forward pass calculating the loss calling loss dot backward and then calling the optimizer step so you can see all these here. So we pass X train into model to get outputs we pass y train and outputs to get the loss we call loss dot backward and then optimizer dot step but on top of that we would still like to know the test loss per iteration. This will tell us whether or not our model is over fitting. For example if we see the test loss increasing during training then we know that we are over fitting our model to calculate the Test loss.

We do some of the same steps but with the test data instead of the train data. So we do the forward pass but we pass in x test instead of X train. Then we calculate the criterion with the test outputs against y test. Finally we save both the train loss and the test loss in our arrays of losses. Next we check our loss per iteration everything appears to look good and the train loss is not too different from the test loss which means our model is not over fitting next. We check the accuracy of our model. Remember that unlike with regression the metric we care about is very different from the loss function we minimized in order to train our network. Accuracy is simply the number we got right divided by the total number of predictions.

In order to get the train predictions we first pass X train into the model which gives us P train. We want to convert the output which is a torch tensor into a numpy array by calling the numpy function. These outputs are still probabilities so we want to call np dot round in order to convert

them all to zeros and ones. Once we have our predictions we can calculate the accuracy by doing a point wise comparison with y train and taking the mean. Now this calculation often confuses beginners. How does this calculate the accuracy. I would encourage you to do a few simple examples in the console on your own. If you don't get it right away remember that equals equals returns True or false.

You'll get true if two items are equal and false if not this operation happens element wise. So if y train and P train are arrays of length n then doing equals equals will give us an array of length N that contains all trues and falses so what happens when you take the mean of an array containing trues and falses. What does that even mean. Usually it only makes sense to calculate the mean for numbers and clearly true and false are not numbers. However in Python true is treated like one and false is treated like zero. So if you try one equals equals true in the console that returns true. And if you try 0 equals equals false in the console that also returns true.

In other words this is really like an array of zeros and ones. We have a 1 whenever you got the correct answer and 0 whenever we did not. You should convince yourself that calculating the accuracy is the same as just adding up all the ones and dividing by n which is also the same as taking the mean. If you don't get it right away write it down on paper next. We do the same thing for the test set to get the test accuracy. The result is that we get about 98 percent accuracy on both the train and test set which is very good. As an exercise. Here is a nice thing to try in TensorFlow when you do a training loop. You will actually get two arrays of data back.

One is the loss per iteration for both the train and test set and one is the accuracy per iteration for both the train and test set. You'll notice that we did not plot the accuracy per iteration here. So for the exercise you should try to modify this code so that during the training loop we not only collect the loss at each iteration but also the accuracy at each iteration when the loop is done. You should be able to plot the accuracy per iteration you should see that as the loss goes down the accuracy goes up. This is usually the case you might want to also think about what are the downsides of calculating the accuracy at each iteration of the loop.

LECTURE 17

State Dict

Check the `3.PyTorch-Linear-Classification.ipynb` for the code.

In this lecture we are going to answer one of the most common questions I get. That is once we've trained the model how do we save it for later use. For example after creating a model in Google Colab I might want to copy that model into my production server so that other services in my company can make use of that model that I just trained. This lecture will show you how to save a model to a file and then load it back at a later time. In addition we'll see how to download the saved model from Google Colab first. Note that this is a continuation of a notebook we were previously looking at. As usual you can determine what notebook we are currently looking at using the title of the notebook.

So if you left this notebook previously you may want to reopen it. At this point so you can follow along. Let's start with the central objects that we need for this process. The state dict as you can tell by its name. It's a dictionary. If you look at what we get when we call model dot state dict we can see that it's not just any dictionary but rather an ordered dictionary. That kind of makes sense since if you're iterating through the dictionary to populate the weights of a neural network you would want to do that in the order that your model expects as you can see it's nothing but the parameters of the model since we just have one layer in our model.

The first element is the weight matrix and the second element is the bias vector. The next step is to save the model we can accomplish that in just one line of code torch dot save for the first argument we pass in the model's state dict and for the second argument we pass in a file path I've just passed in my model dot pt so the file my model dot pt will be saved to the current directory you can see that after saving the model when I do an ls the file I just saved appears as expected the next step is to load and evaluate the model to ensure that we saved it correctly in order to do this we first have to recreate the model the same way it was created originally. That might seem strange since we still have to replicate the code we had earlier.

This will probably make more sense to you. Once you've seen more complex models which will involve defining our own classes in those cases you would just instantiate an object of that class which is one line of code which would be more compact than this. In any case passing code around is very lightweight because code files are just text anyway. We'll call this model too so as not to interfere with our original model. Next we call model 2 dot load state dict and pass in the state dict that we just saved we get that by calling torch dot load and passing in the file path where our model file is located. You can see that if your load is successful it will say all keys match successfully at this point.

We would like to know is our reloaded model the same as the model we saved. Luckily this is an easy check and we already know how to write this code. We just take our code from earlier to compute the accuracy of the model on the train set and test sets. But this time we'll use a model 2 instead of the original model. As you can see we get precisely the same accuracy as we did ear-

lier. At this point you might be wondering. So I just saved my model. It's in my local directory but it's not my local directory it's my Google drives local directory. What if my teammate asked me for this model. How can I share it. And that is the next question we are going to discuss.

In fact this is quite easy and it's another reason I really like Colab. So over in left here. You may have noticed this sidebar with some icons. If you click the folder icon you'll see all the files in your current directory including my model dot pt the model file we just saved so let's right click that and now you can see that there's this option to download and of course that's how you can download this file to your local harddrive another option is to use Python code to download the file. Now I've heard that this only works for Chrome but of course that may change in the future. We can use the same module we used earlier for uploading files so we import files from the google dot colab module.

Then we say files dot download and pass in the file path when we run the cell. The file is downloaded by our browser as usual all right. So that works as expected.

LECTURE 18

Linear Regression

In this lecture we are going to look at the models we just used more in depth. Specifically if you know a little bit about machine learning you may recognize that the models we just used are called linear regression and logistic regression.

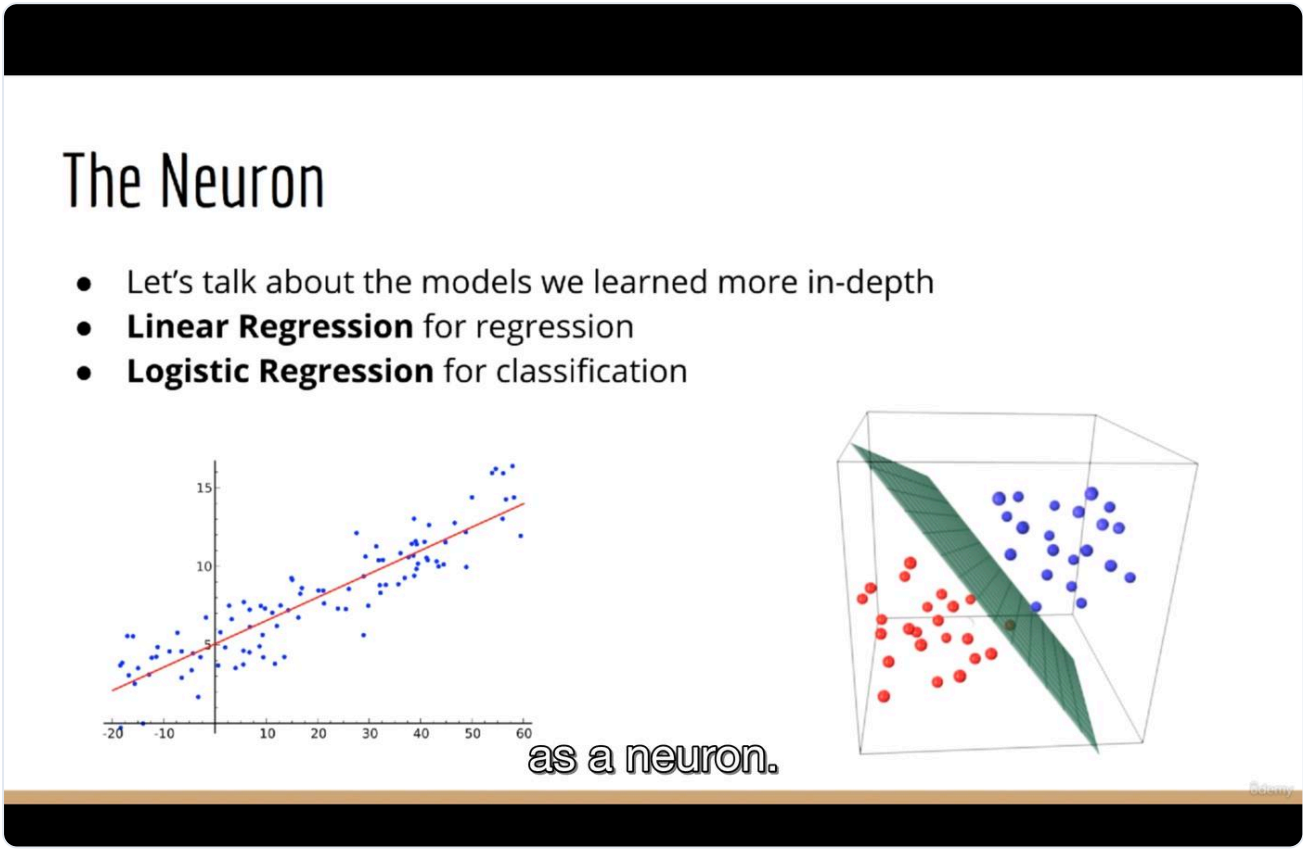


FIGURE 18.1 | The Neuron

We use logistic regression for classification and we use linear regression for regression. So this lecture will look at how these models work and also explain why we refer to logistic regression as a neuron. Just to remind you of the high level picture here. Deep Learning is the study of neural networks and neural networks are networks of neurons. So you can think of these as the basic fundamental unit of computation in deep learning let's start with linear regression in its most basic form linear regression just means line of best fit.

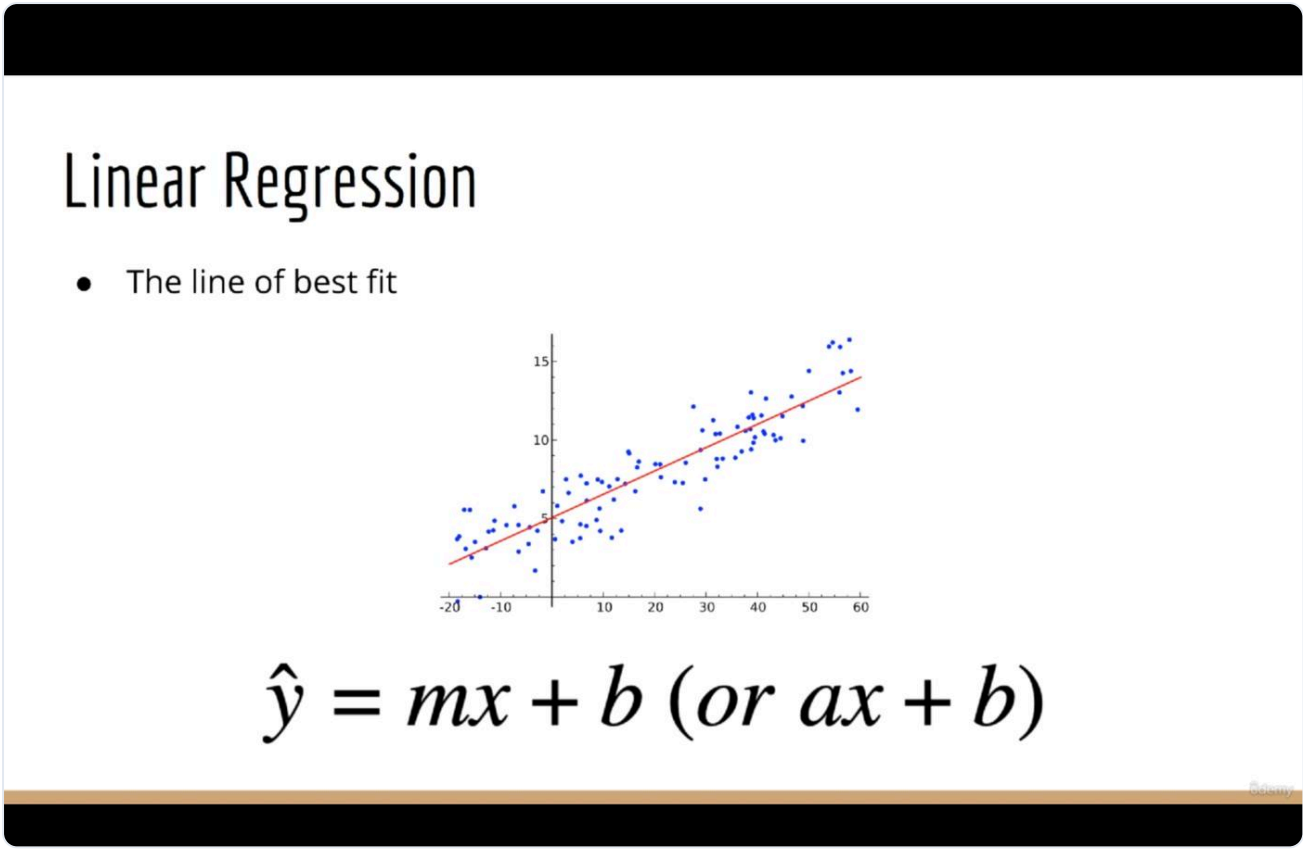


FIGURE 18.2 | Linear Regression

As you may recall from your high school math studies a line has the equation y hat equals M X plus b here X is the input variable M is the slope and B is the y intercept. When we put these together we get the equation for a line our job of course is to find a line that best fits our input data to give you a simple example of how we would use linear regression. Let's suppose we are trying to predict your salary based on how many years of experience you have. So in this case x would be the number of years of experience and y would be your salary. Now you might be wondering what is the interpretation of the slope in y intercept.

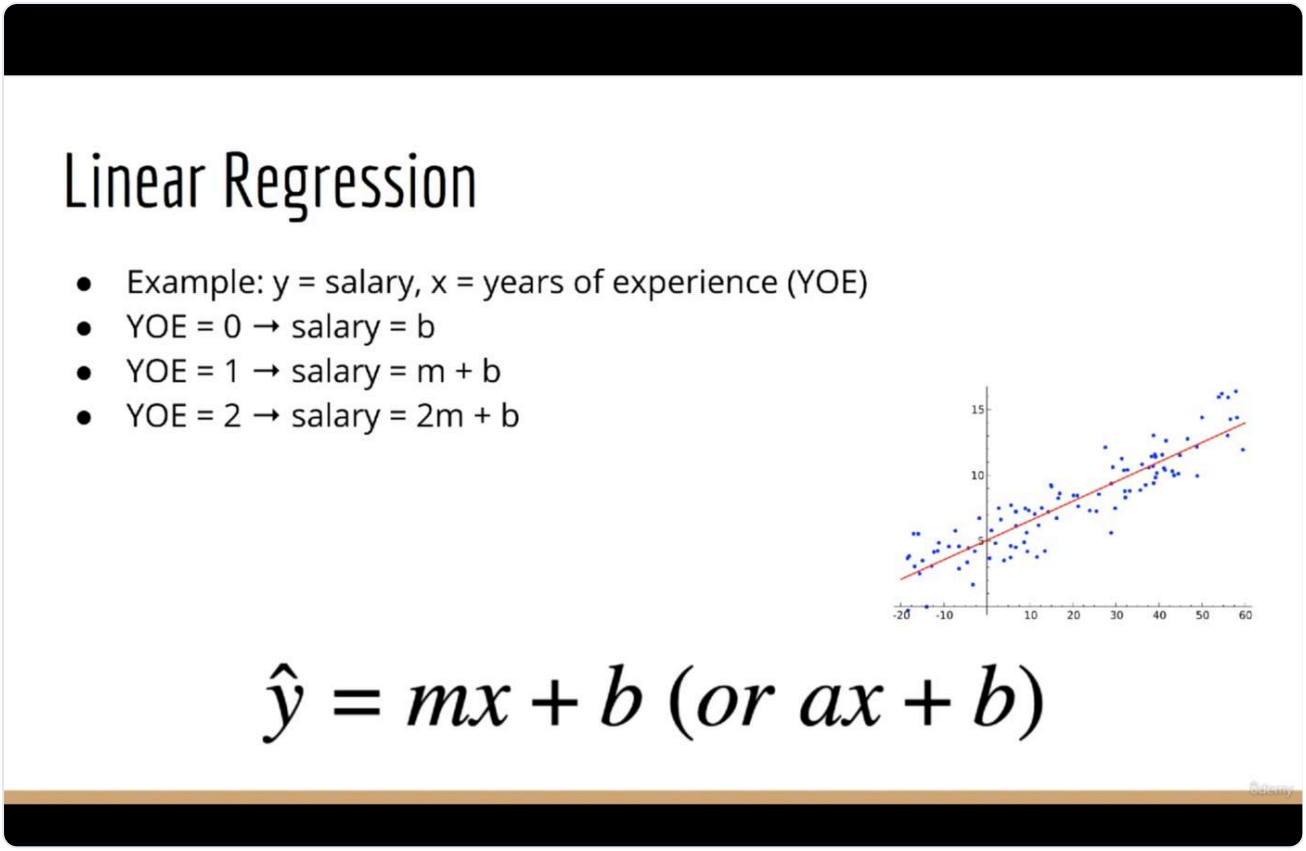


FIGURE 18.3 | Linear Regression

In this scenario. Well let's try to play around with this equation a little bit and see what happens. We know that b the y intercept is the value of y half when x equals zero. So in other words if you have zero years of experience this would be your salary. B If X equals 1 that means you have one year of experience and then your salary would be M Plus B if x equals two.

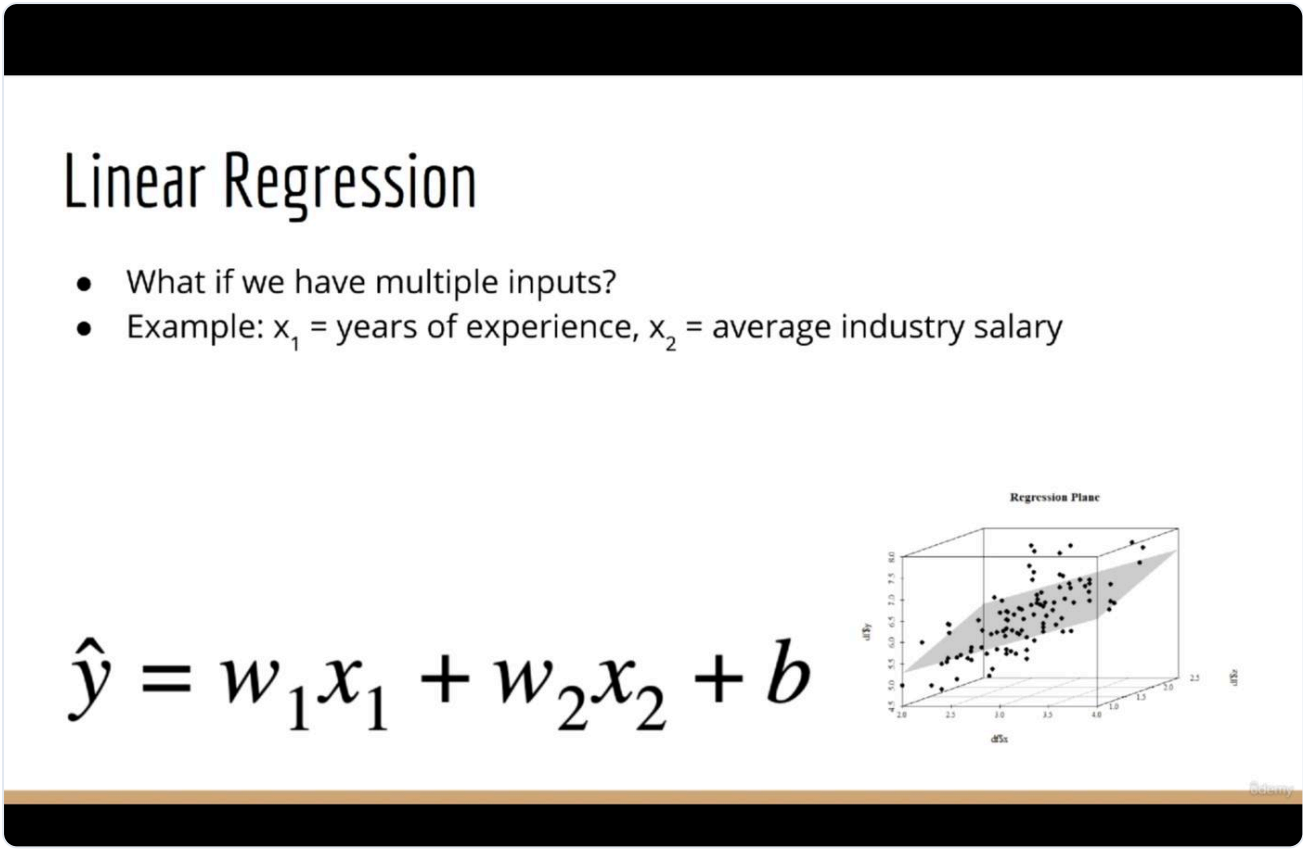


FIGURE 18.4 | Linear Regression

That means you have two years of experience and then your salary would be two M plus B. In other words M is the increase in salary that you get for each additional year of experience. Of course in the real world we might want to make a prediction about your salary based on multiple factors instead of just years of experience. We might have another input let's say average industry salary. So now your salary can also depend on what industry you are working in.

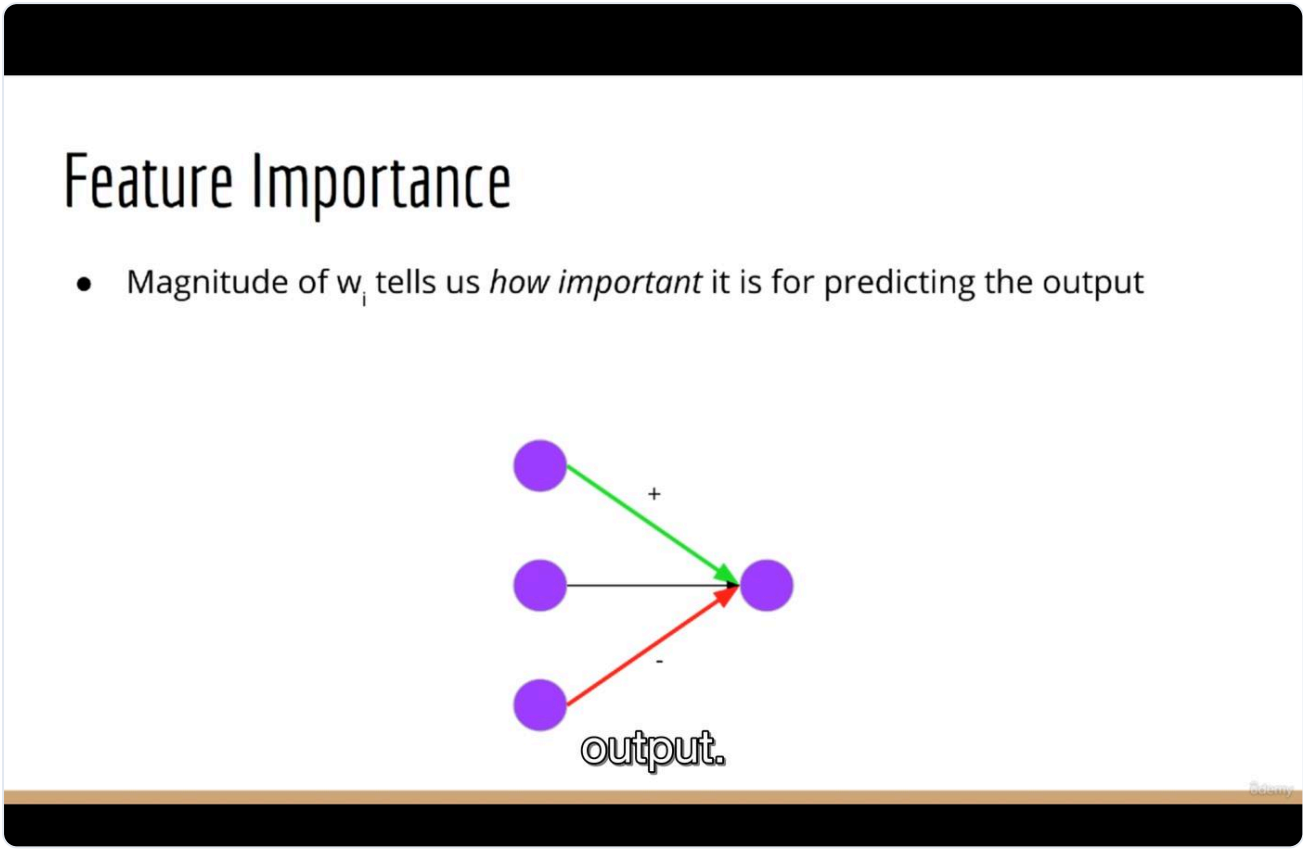


FIGURE 18.5

So we'll call it years of experience x 1 and we'll call average industry salary x 2 in this scenario. We would write out our model as Y equals w 1 x 1 plus W2 x 2 plus B in this equation w 1 and W2 are called the weights of the model and they are essentially the slope for each of the individual inputs X1 and X2 luckily the interpretation is the same as what we saw previously. W 1 is the increase in salary when X1 increases by 1 W2 is the increase in salary when x 2 increases by 1. B is the salary when both X1 and x 2 or 0 another way to think of the weights is that they tell us how important each input is to predicting the output. Imagine the extreme scenario where we have some w sub i equals zero.

In this case it doesn't matter how much we increase or decrease x sub i the output will not change at all.

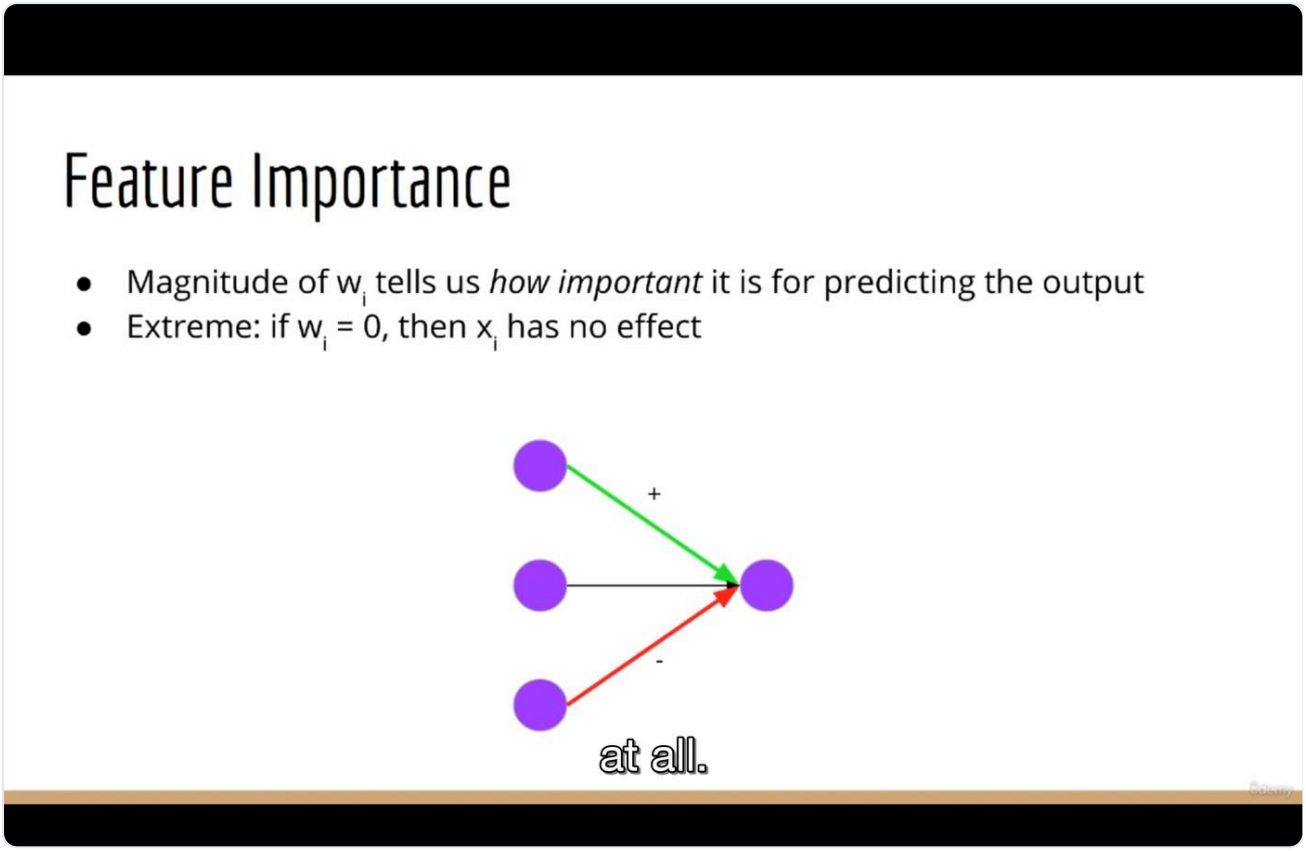


FIGURE 18.6

So in other words x sub i has no influence on the output. That's another way of saying x sub i is irrelevant. Now imagine that w sub i is very large in magnitude. Note that it can be either very positive or very negative. This means that x sub i has a large influence on the output.

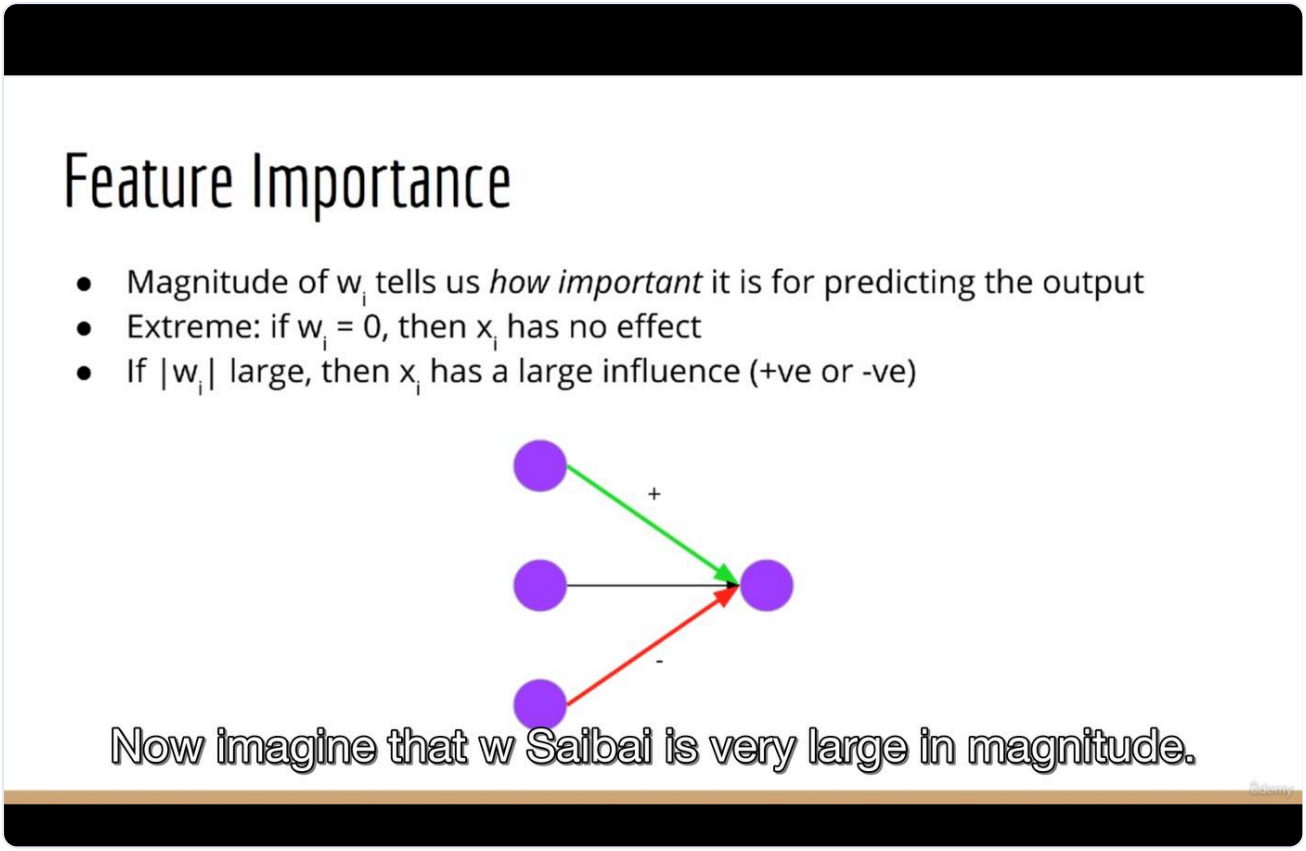


FIGURE 18.7

Because a small increase in x sub i leads to a large increase in output that's just another way of saying that this input feature is very important. Also the sign of w sub i controls the direction of the influence. If w sub i is very positive then an increase in x sub i will lead to a large increase in the output. If w sub i is very negative then an increase in x sub i will lead to a large decrease in the output now in what way is any of this related to neurons. Well to understand this you have to understand a little bit about biology.

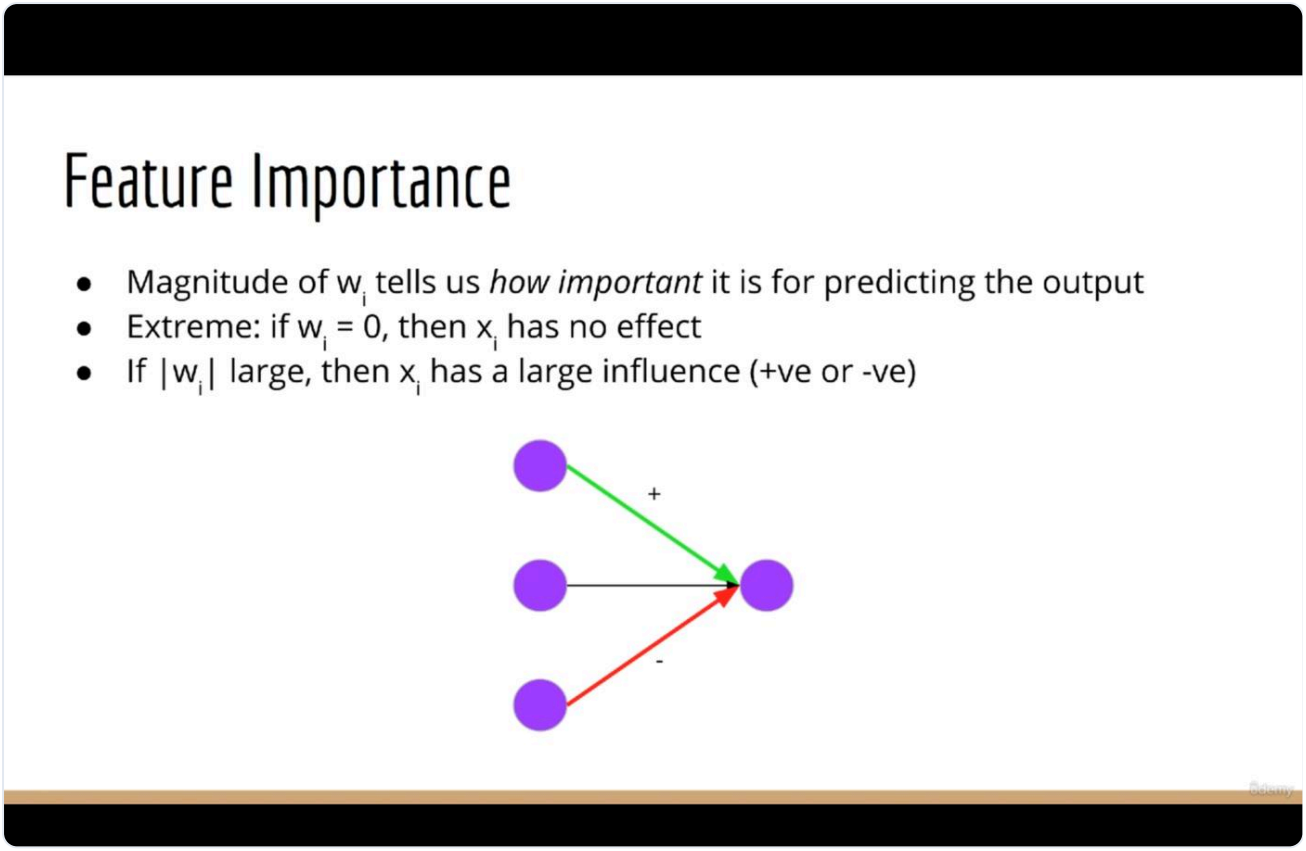


FIGURE 18.8

A neuron is a cell in your brain. You can think of it as we said before as the fundamental unit of computation neurons can talk to other neurons using electrical and chemical signals. So if you picture a neuron you can imagine that there are many other incoming neurons attached to it. These input terminals are called dendrites. As you can see in this image those incoming neurons might come from say your eyes or your ears or your nose or your hands.

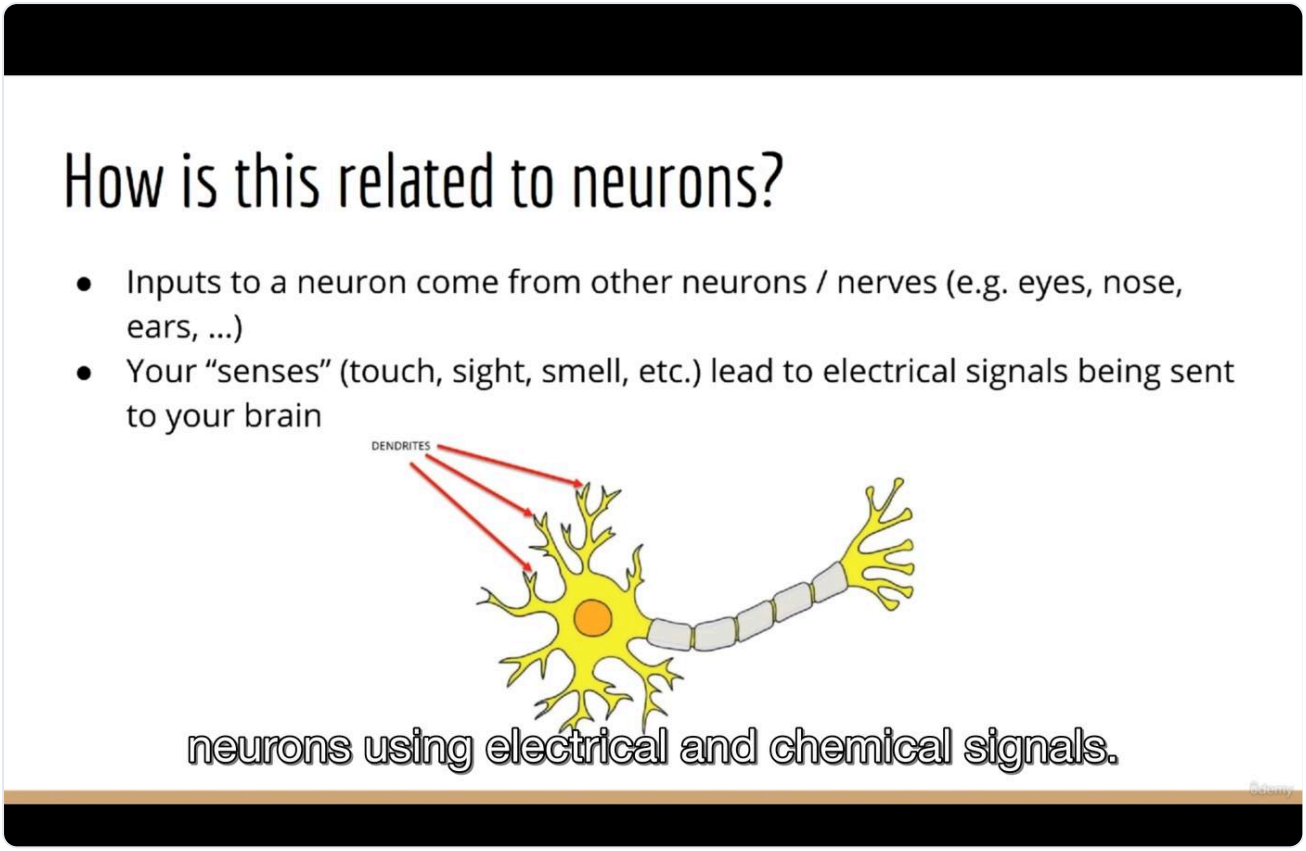


FIGURE 18.9 | How Is This Related to Neurons?

So if your eyes see something an electrical signal goes along the nerves in your eyes and travels to your brain the same thing happens when you hear something or you smell something or you touch something. Now remember we're picturing a single neuron it's taking in inputs from a lot of different places. Now this neuron it has to decide if it's going to pass this signal on to outgoing neurons because remember this neurons kind of in the middle there are some neurons going into it and it's going out to some other neurons. Now how does it do that. Well it does that in a very similar way to linear and logistic regression.

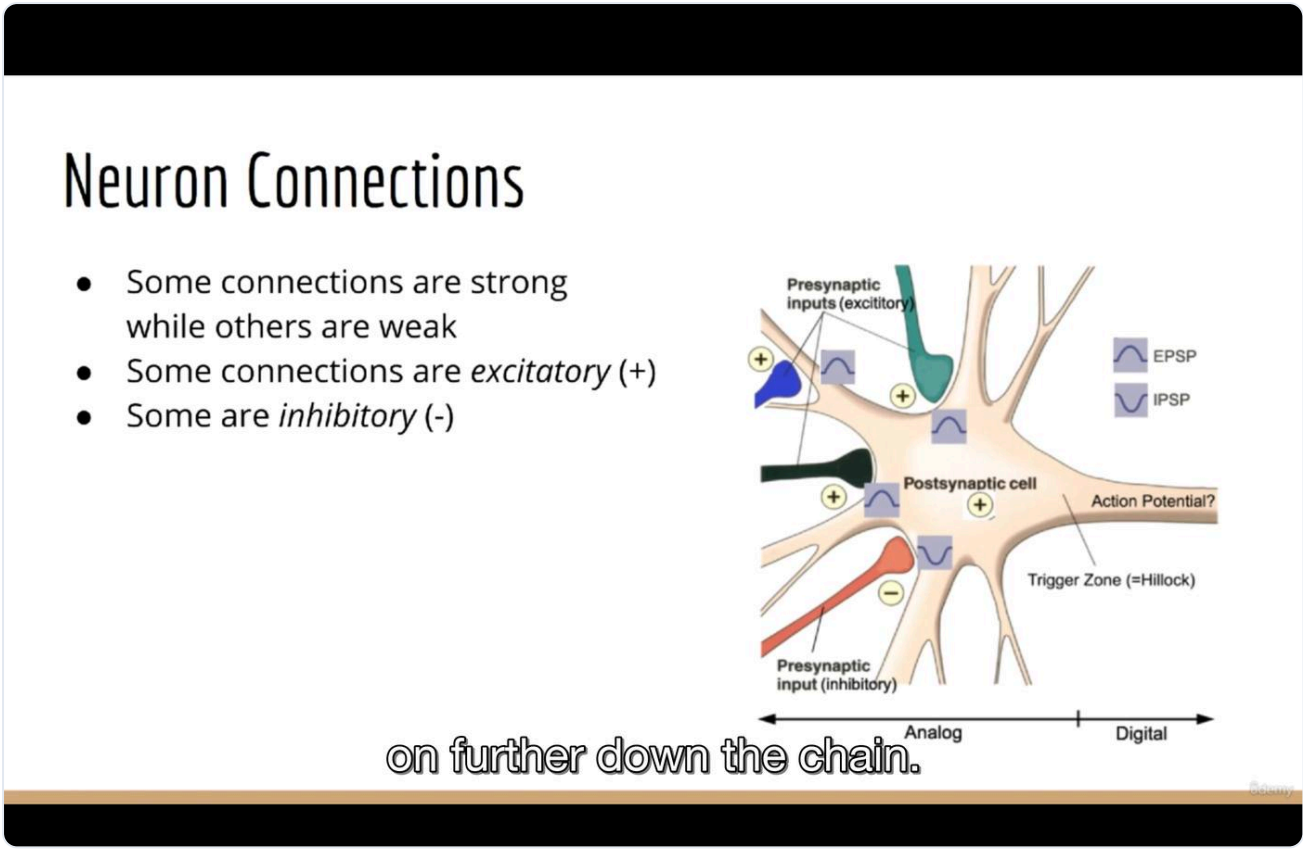


FIGURE 18.10 | Neuron Connections

It sums up all the incoming signals and then this summation becomes the outgoing signal that gets passed on further down the chain. The next thing you have to know is that not all neuron connections are created equal.

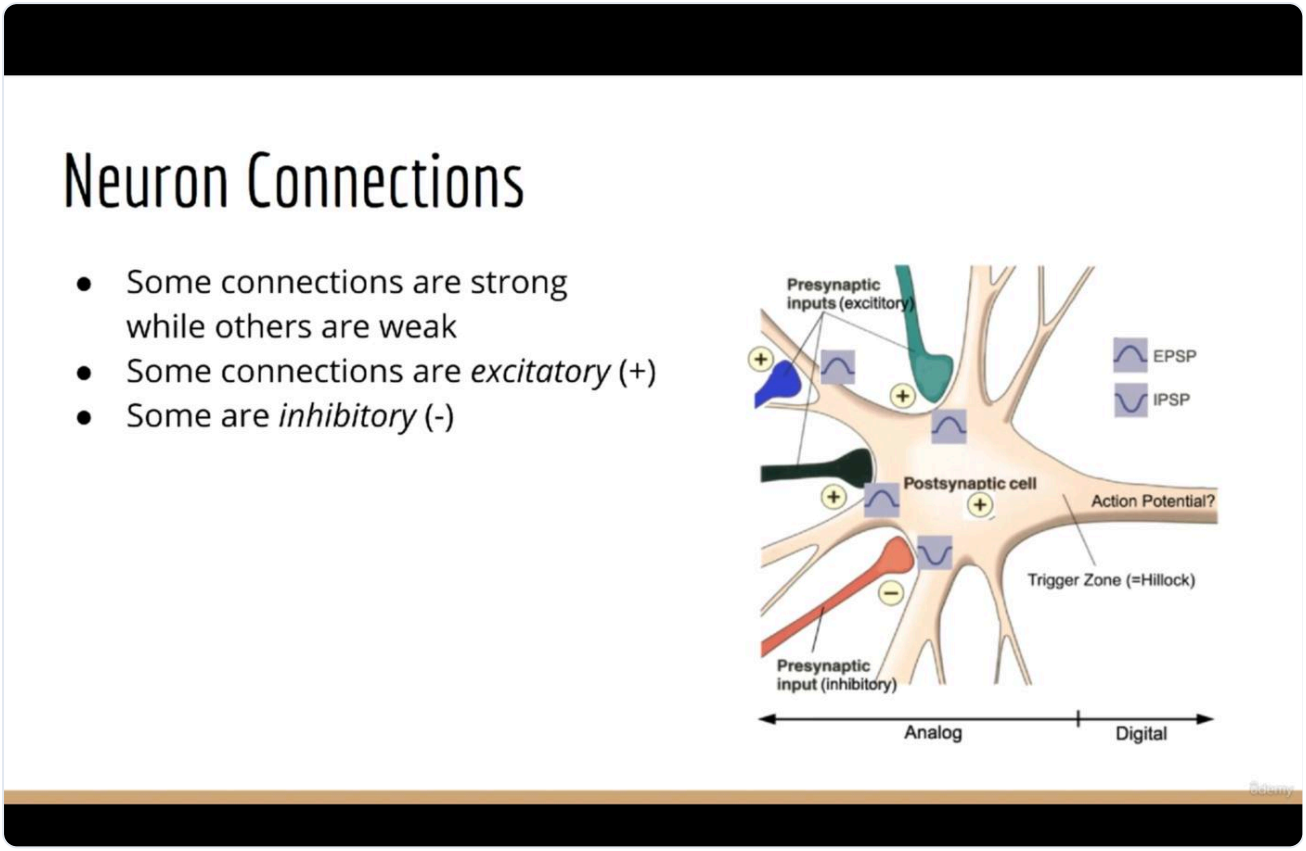


FIGURE 18.11 | Neuron Connections

Some connections are strong while others are weak. Some connections excite the receiving neuron while other connections inhibit the receiving neuron. This is just like the weights of a regression model. A large weight either positive or negative has a strong influence on the output. That's a strong connection. A small or nearly zero weight has only a weak influence on the output so that's a weak connection. A positive weight influences the output in the positive direction. So that's like an excitatory neuron and a negative weight influences the output in the negative direction. So that's like an inhibitory neuron the signal that gets passed along neurons has a special name.

It's called an action potential.

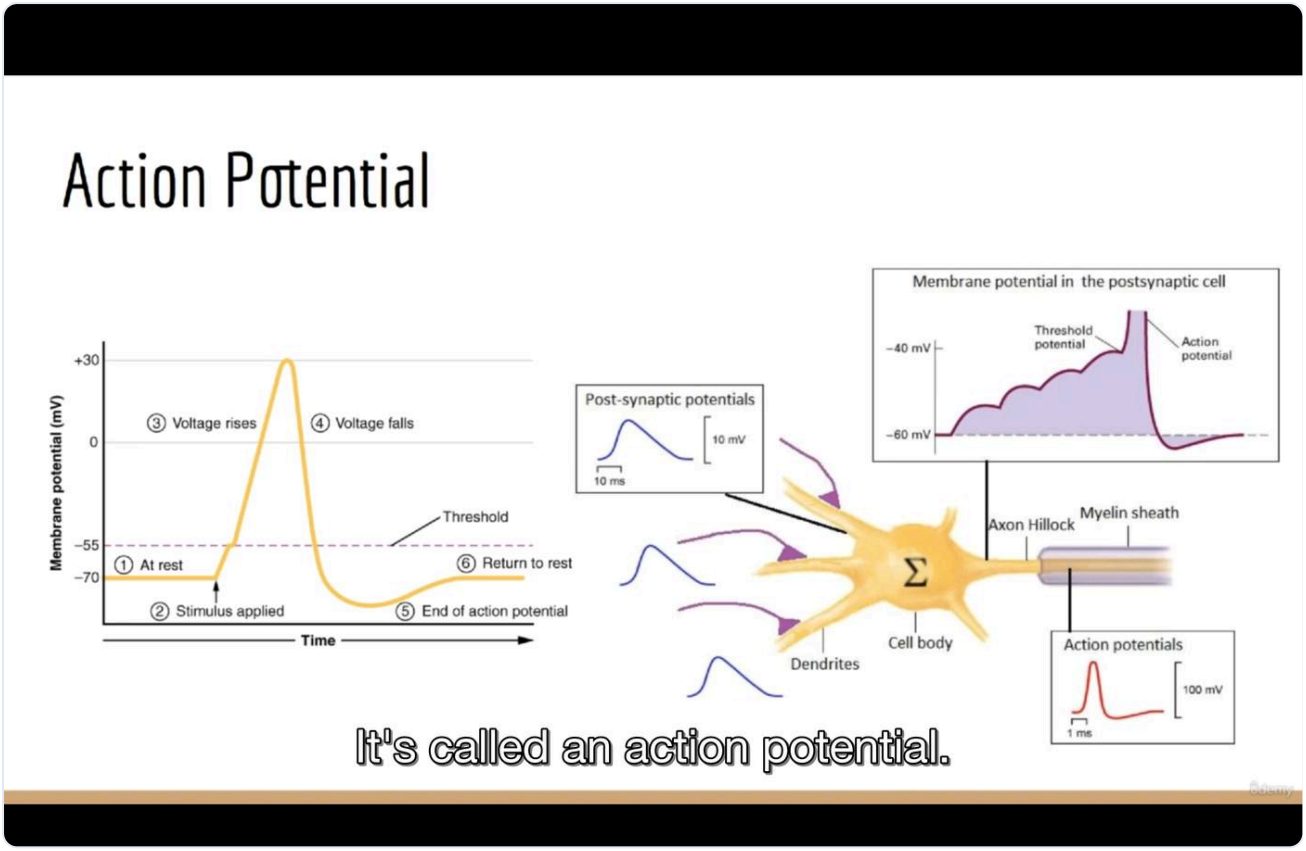


FIGURE 18.12

Basically it's a spike in electrical potential. So if you measure the electrical potential over time at a particular point in the neuron you would see a signal like this. Now action potentials don't behave in a very intuitive way. In particular you can think of them as binary outcomes just like logistic regression. In other words the neuron is more like logistic regression than linear regression although the linear regression equation is the main calculation that takes place in both cases. So here's how the action potential works. Basically we're going to sum up all the influences from all the incoming neurons. If the electrical potential of this sum is greater than some threshold then an action potential will propagate through the receiving neuron if the electrical potential is less than this threshold then nothing happens at all.

This is very much like binary classification where we make predictions which are either 0 or 1 in biology.

All-or-nothing principle

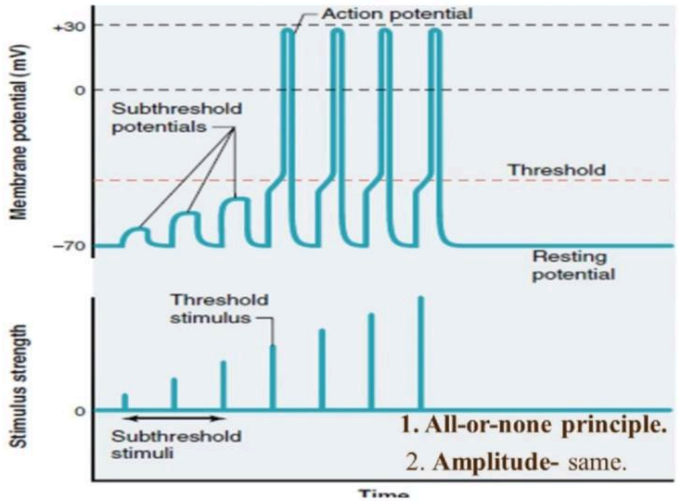
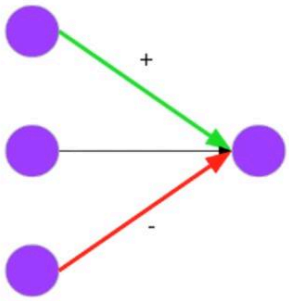


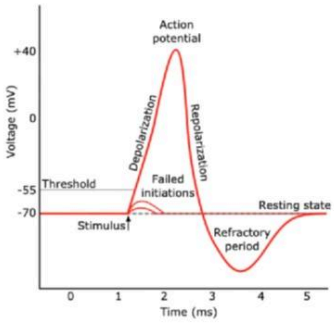
FIGURE 18.13

The Neuron

The "threshold" in this case is -b (negative)

As long as $w^T x$ is > -b, then $w^T x + b > 0$, and the prediction is 1





$$p(y = 1 \mid x) = \sigma(w^T x + b) = \sigma(\sum_{d=1}^{D} w_d x_d + b)$$

FIGURE 18.14 | The Neuron

We call this the all or nothing principle either an action potential fires or doesn't. As you can see here when the stimulus is too weak there's only a little bump in the response. So those are not action potentials but once the stimulus reaches a certain threshold a full action potential spike occurs. So there's no such thing as a spectrum of action potentials like an action potential of one volt two volts three volts and so forth. Instead it's just a binary decision. You get a spike or you don't. The incoming signals summed together are strong enough to generate the next action potential or they will simply be ignored so that's how you can think of logistic regression as a neuron.

Each x sub i is an input from some incoming neuron. Each w sub i is a weight that tells us how strong the connection with x sub i is and the sign of w sub i tells us whether that connection is excitatory meaning it positively influences an action potential or inhibitory meaning it negatively influences an action potential. The weighted sum of each x sub i which is then added to the bias term or threshold is then passed to the sigmoid function. Once the sigmoid function is rounded we get either 1 or 0 telling us whether or not an action potential should occur.

LECTURE 19

Gradient Descent

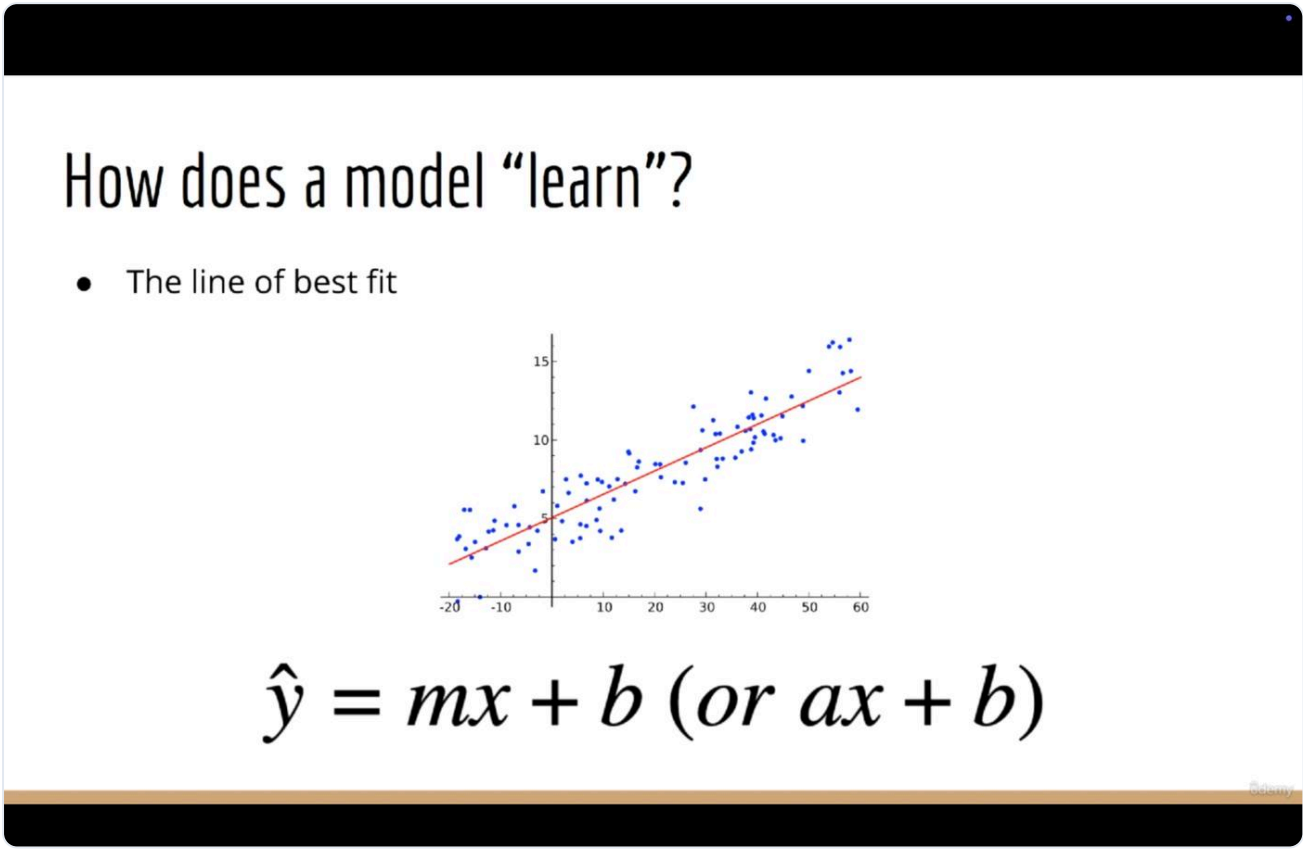


FIGURE 19.1

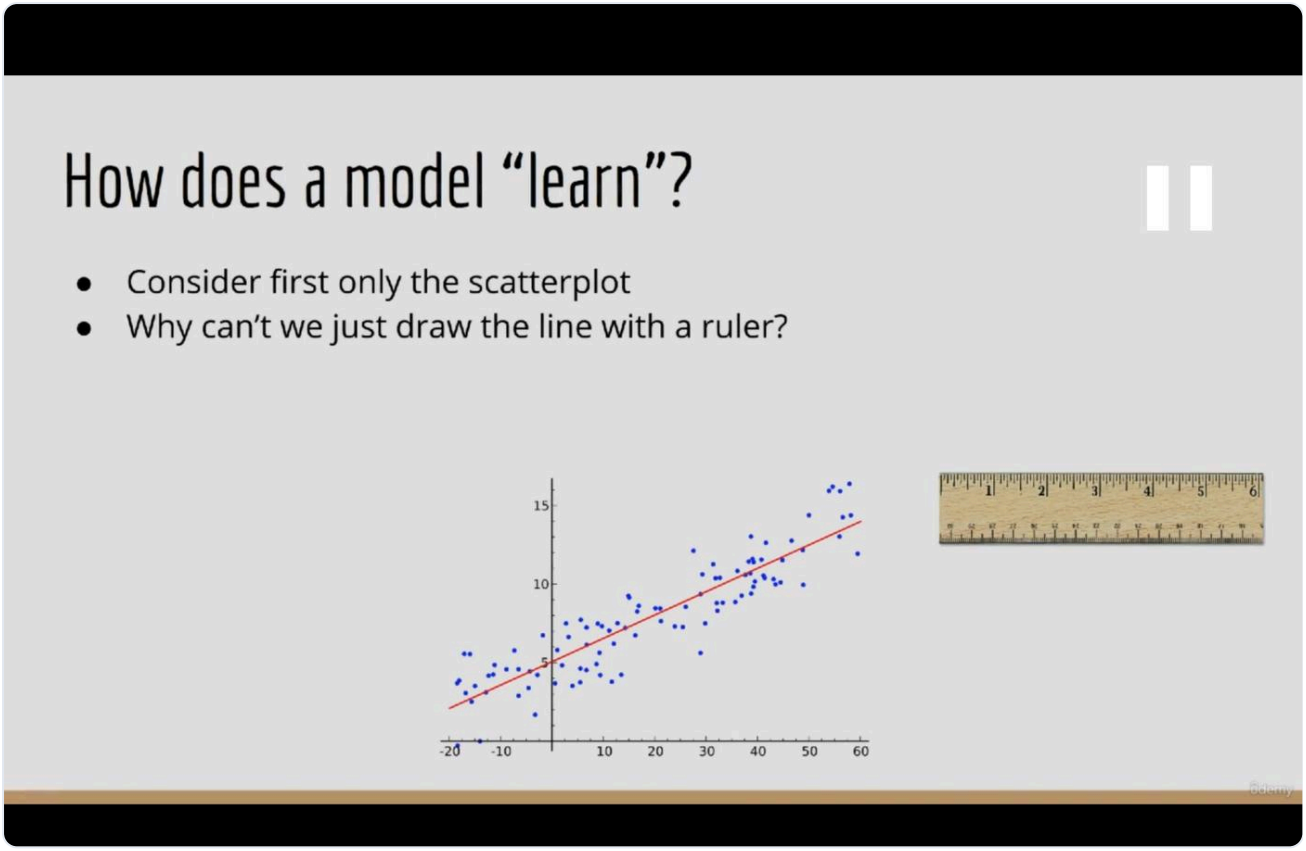


FIGURE 19.2

In this lecture we are going to answer the question how does a model learn. Let's start with linear regression and its most basic form linear regression just means line of best fit. As you may recall from your high school math studies a line has the equation y equals M X plus b here X is the input variable M is the slope and B is the y intercept. When we put these together we get the equation for a line and our job of course is to find a line that best fits our input data so you can imagine that our input data is a bunch of data points that approximately create the shape of a line. This plot is called a scatter plot. So for each point x and y in our dataset we're going to draw a dot on this chart.

Our goal is to find the line that best passes through all these data points. In practice this means finding the slope and y intercept one method. You may have used in high school is to take a ruler and to try and draw the best line on paper using visual inspection. Of course since this is data science we no longer use such methods. We have to be more systematic. The line that you draw might not be the same as the line that I draw. And furthermore if we are in multiple dimensions this won't be a line at all. So it's clear that we need something better and more quantitative. The idea is we're going to define an error function for each data point which is a pair made up of the input x i and the target y.

We're going to calculate a prediction y hat i equals m x i plus B then we are going to take the squared difference between y i and y hat i we're going to do this for each of our data points for I equals one up to n once we've done this we can add them all up and divide by n.

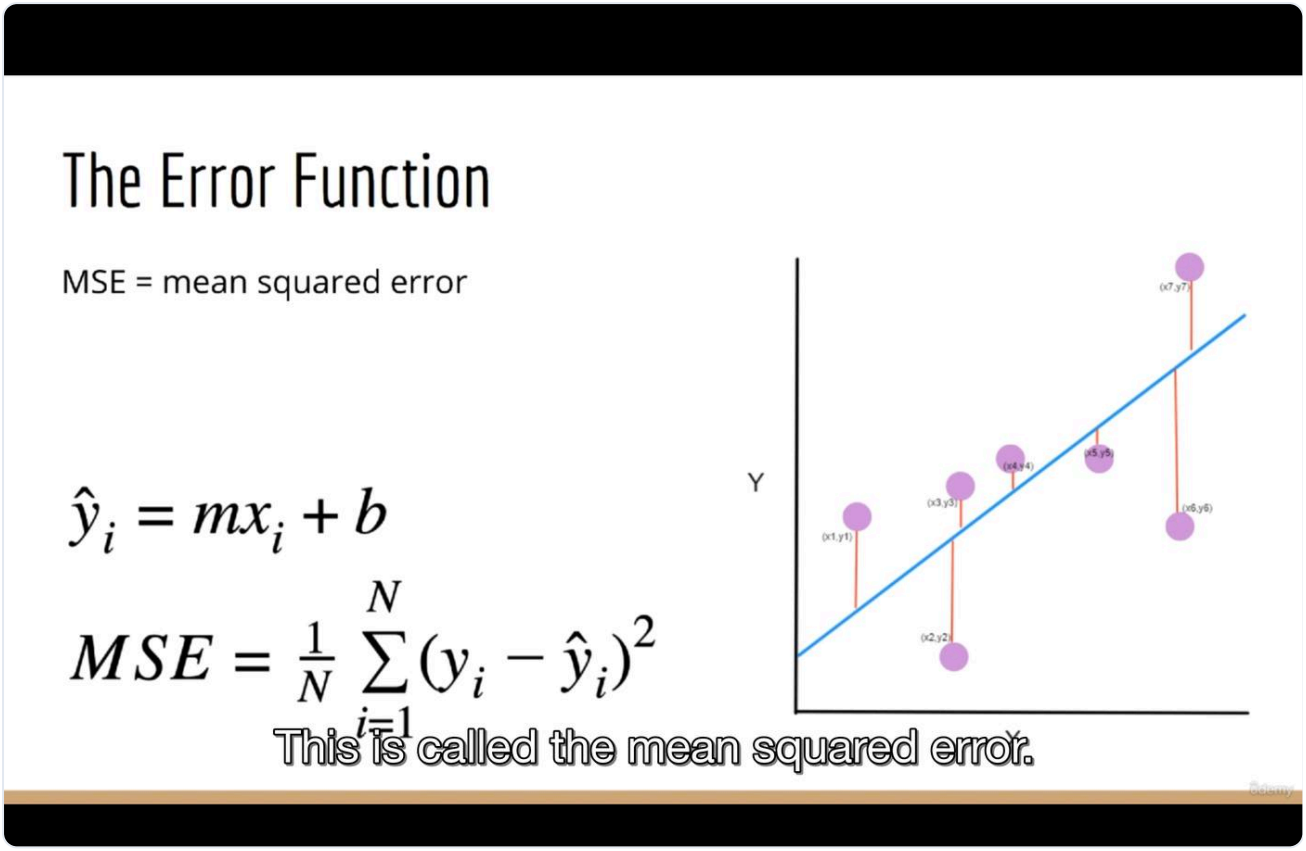


FIGURE 19.3 | The Error Function

This is called the mean squared error. It's the average squared deviation between our predictions and our targets. Basically this is going to tell us how accurate our model is. If our predictions are equal to the targets then this error will be zero because y i would be equal to y hat i. Then the more wrong we are the larger this error becomes. So that's generally how an error function should work as a side note.

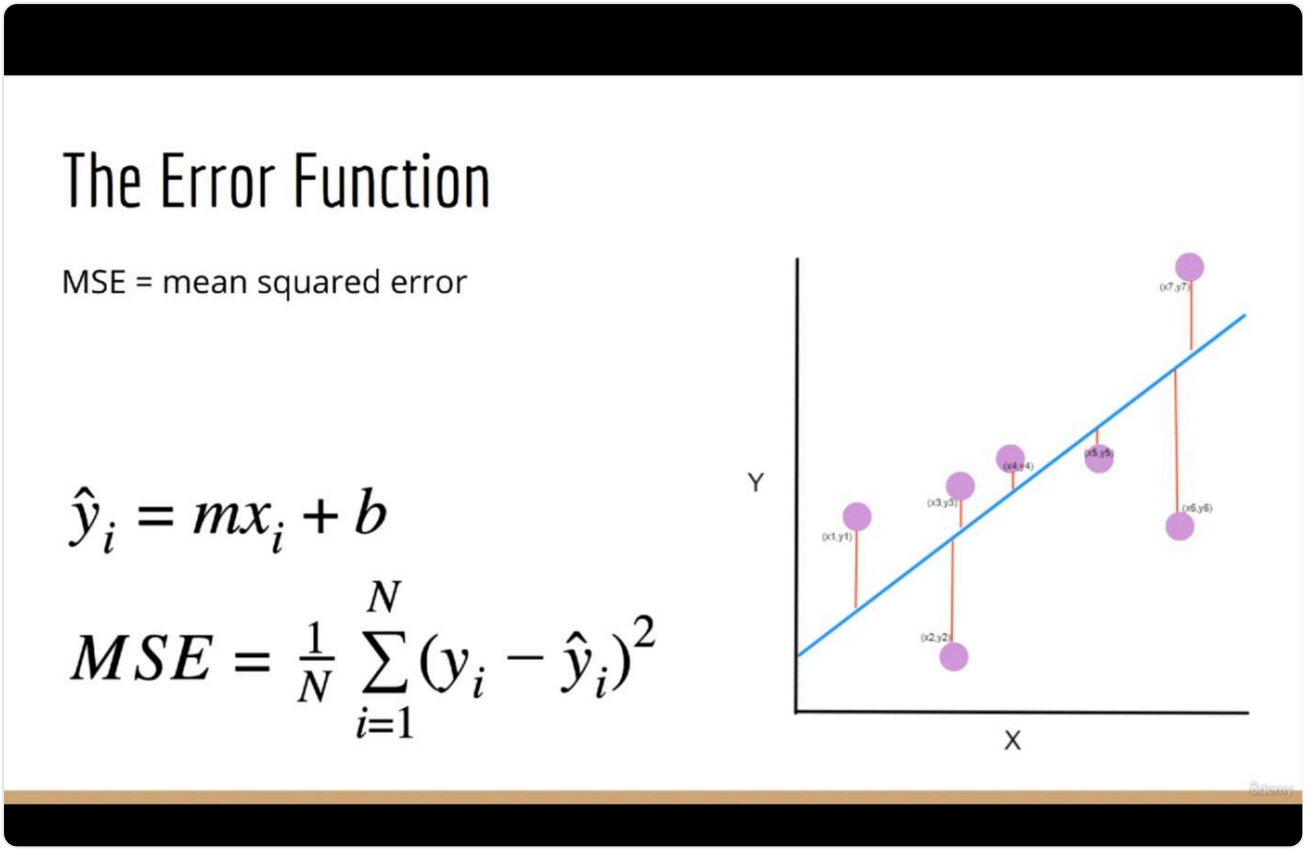


FIGURE 19.4 | The Error Function

Since some students get confused by this at first we have multiple names for this error and we usually use them interchangeably. Sometimes we call it a loss. Sometimes we call it an objective or sometimes we call it a cost. However these all mean the same thing. I like the term cost because it makes intuitive sense for business people as any good businessmen would do.

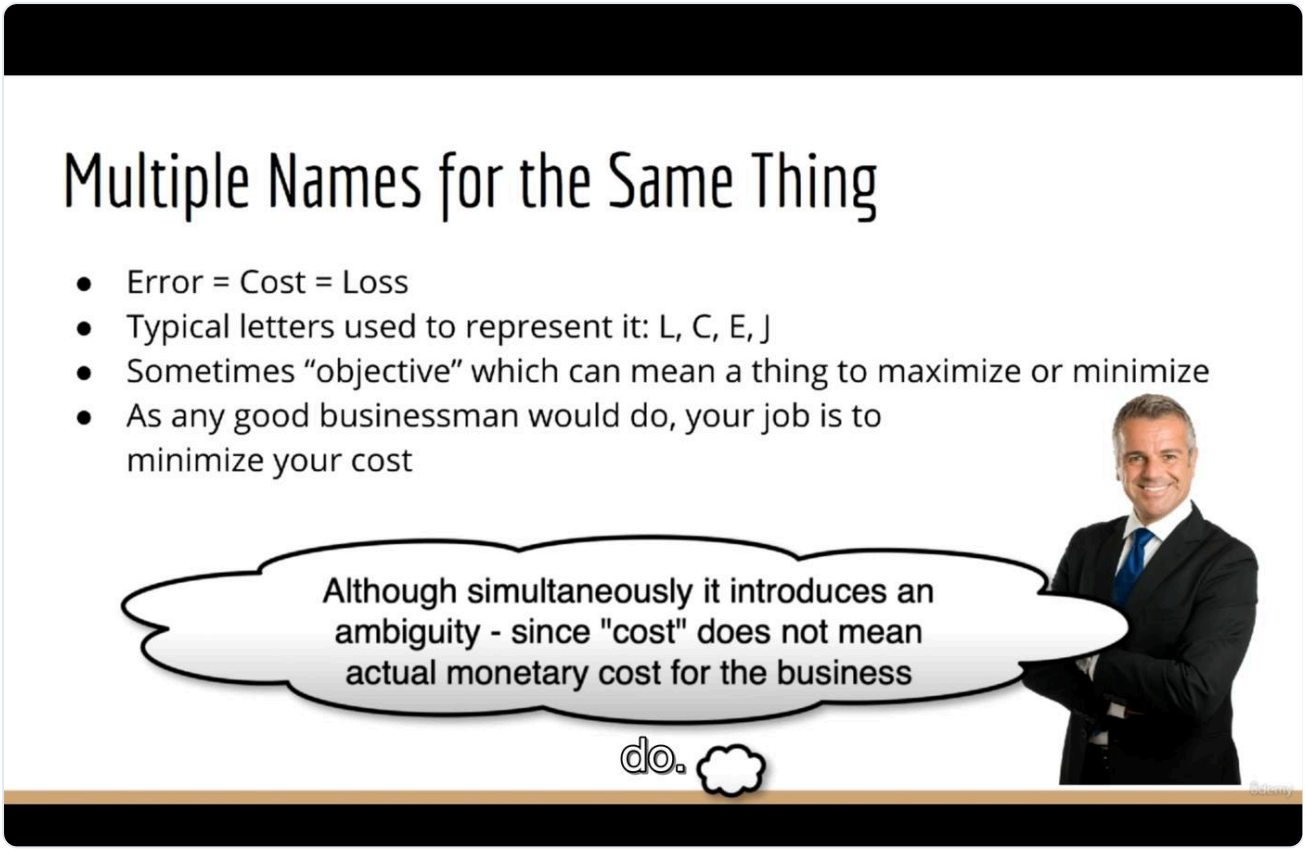


FIGURE 19.5 | Multiple Names for the Same Thing

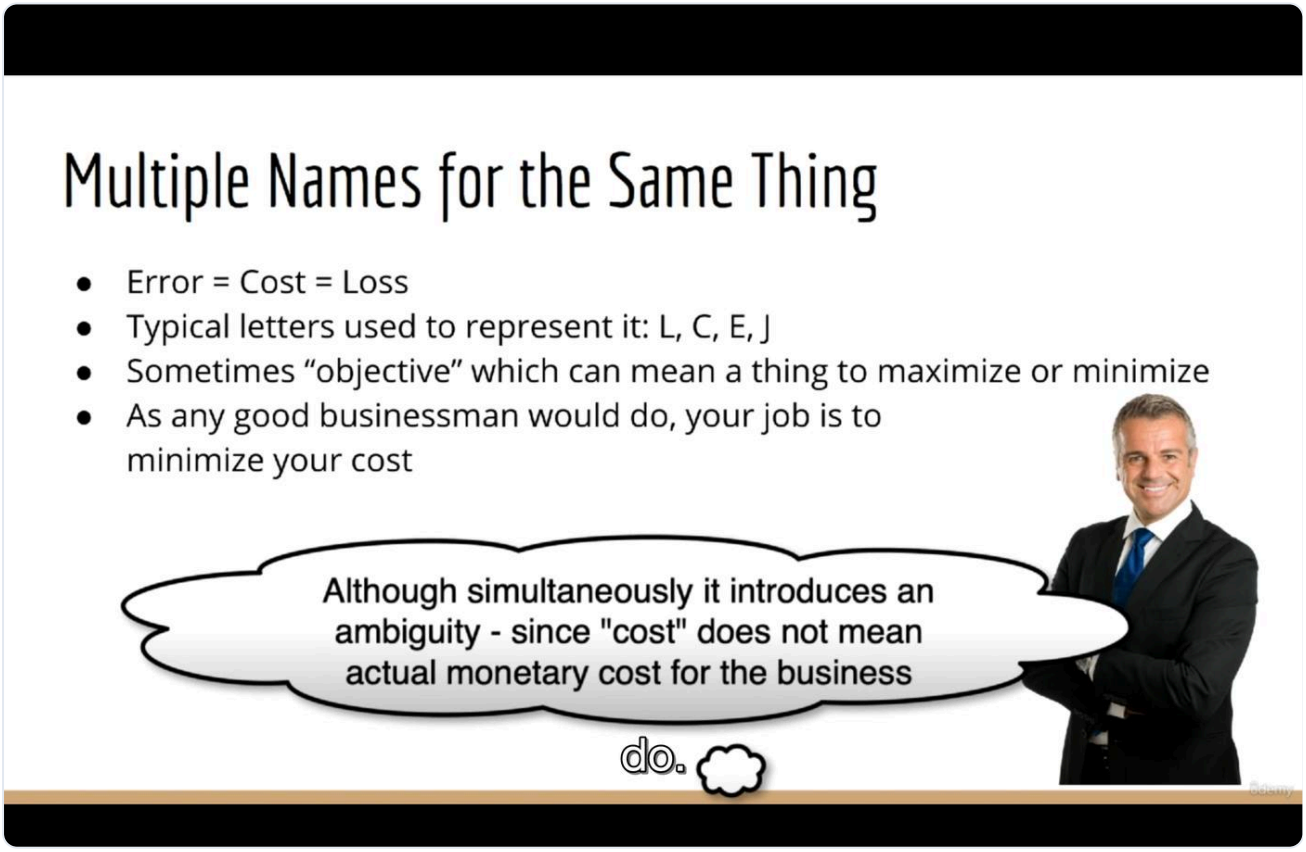


FIGURE 19.6 | Multiple Names for the Same Thing

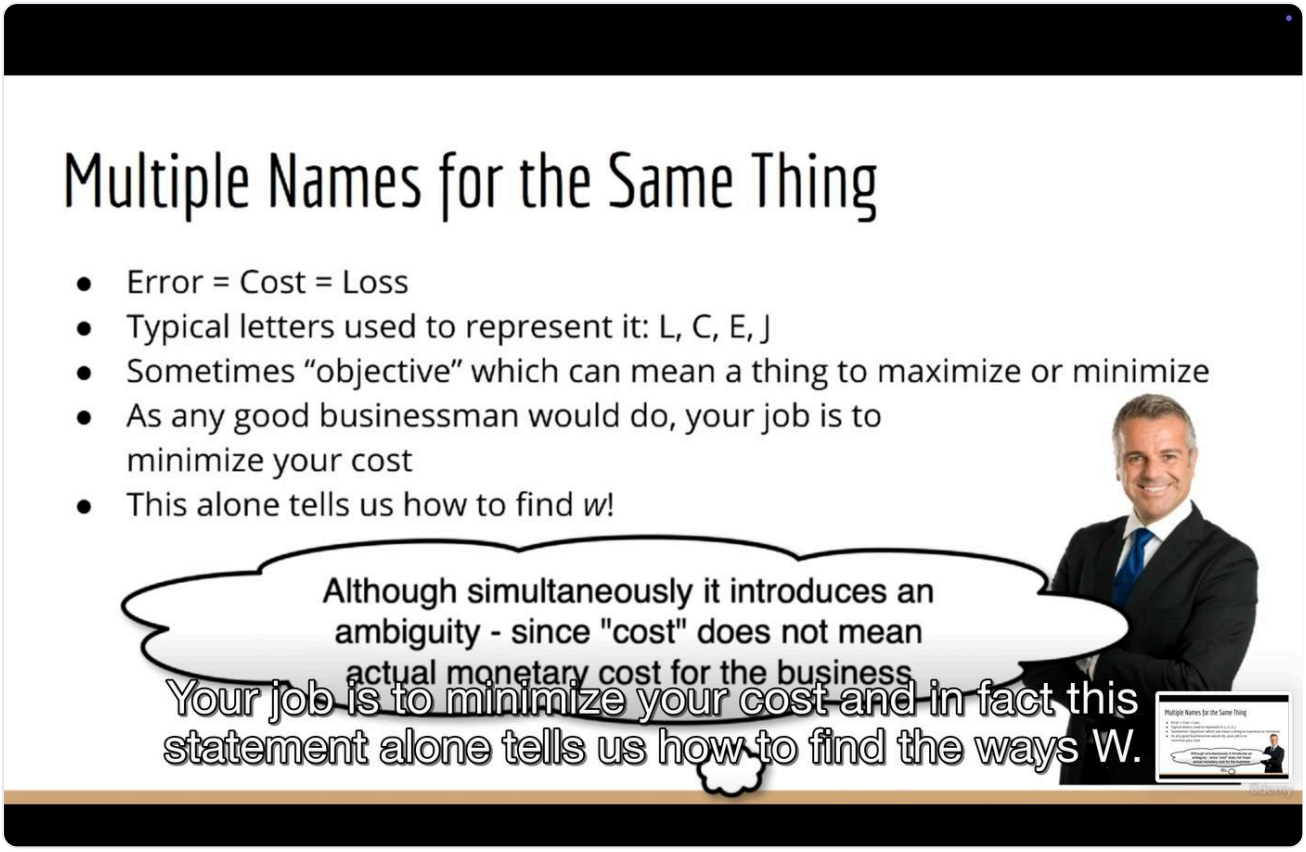


FIGURE 19.7 | Multiple Names for the Same Thing

Your job is to minimize your cost and in fact this statement alone tells us how to find the weights W. So how do we minimize this cost or in other words make it as small as possible. Well it's time to turn to our old friend calculus if you recall we can find the minimum or a maximum of a function by finding where its derivative is equal to zero. If you're not convinced by this try to draw a curve which has a minimum or maximum and draw the slope at the minimum or maximum point. If you drew your line correctly it should be a horizontal line meaning that the slope at this point is zero. And remember the slope at each point is just the derivative. So that's why we want to find the derivative set it to zero and then solve for the parameter in question now usually we have more than one parameter even in one dimensional linear regression.

This is the case because we have both the slope and the y intercept.

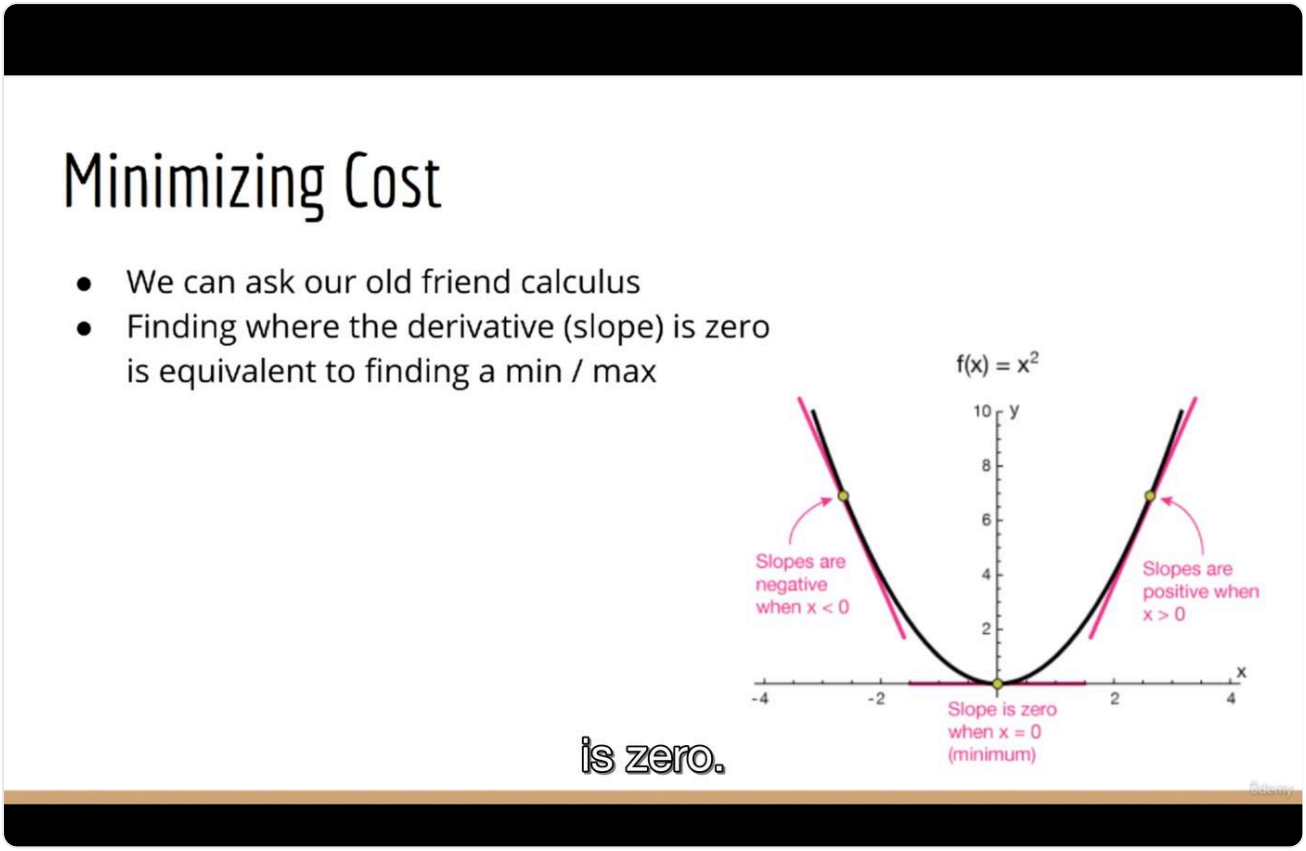


FIGURE 19.8

So what happens when we have a function of more than one variable in this case. The derivative is actually called the gradient. If you recall from your calculus studies the gradient is just a vector or tensor of partial derivatives for each of the variables we're taking the gradient with respect to so for example if you take the square function in one dimension that's called a parabola but if you have two input dimensions it's called a paraboloid. Beyond that we can't picture it but the basic idea is still the same find the gradient set it to 0 solve for the parameters by the way the term parameter is just another name for the weights. So that's the W.

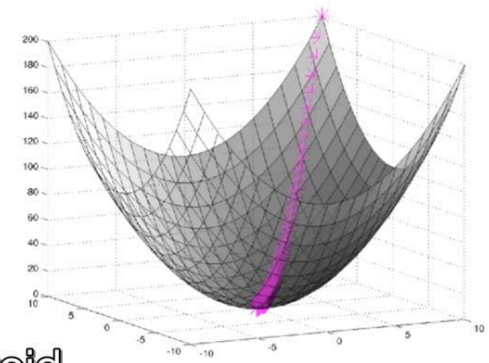
And b we've been referring to throughout this section luckily in this course we won't be calculating any derivatives manually since I have many other courses which do that a lot. And by a lot I really mean a lot. So in this course we're kind of excused from doing that because PyTorch already does that for us using a process called automatic differentiation luckily you don't have to know how that works either because as its name suggests it's automatic.

Minimizing Cost

- What if there's more than one variable?
- The multidimensional equivalent of *derivative* is the **gradient**
  - To solve for model parameters (weights), we find the gradient and set it to 0

$$\nabla_w J = \left(\frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \ldots, \frac{\partial J}{\partial w_D} \right)$$

$$Set: \nabla_w J = 0, \; solve \; for \; w$$



a parabola void.

FIGURE 19.9 | To Solve for Model Parameters (Weights)

Minimizing Cost

- What if there's more than one variable?
- The multidimensional equivalent of *derivative* is the **gradient**
  - To solve for model parameters (weights), we find the gradient and set it to 0

$$\nabla_w J = \left(\frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \ldots, \frac{\partial J}{\partial w_D} \right)$$

$$Set: \nabla_w J = 0, \; solve \; for \; w$$

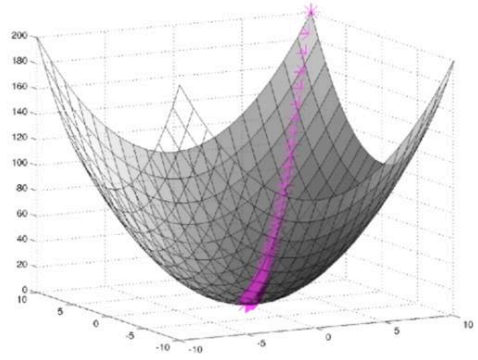


FIGURE 19.10 | To Solve for Model Parameters (Weights)

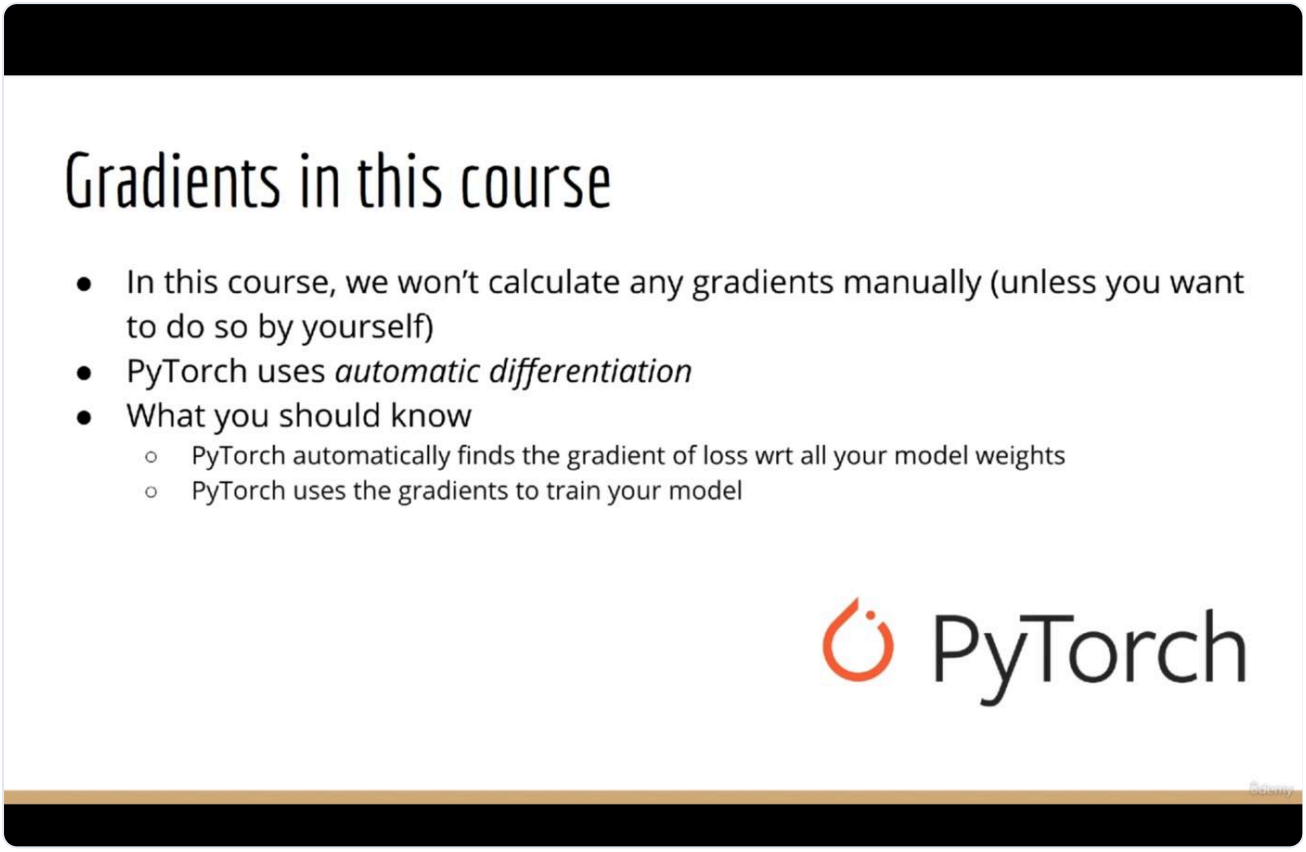


FIGURE 19.11 | Gradients in This Course

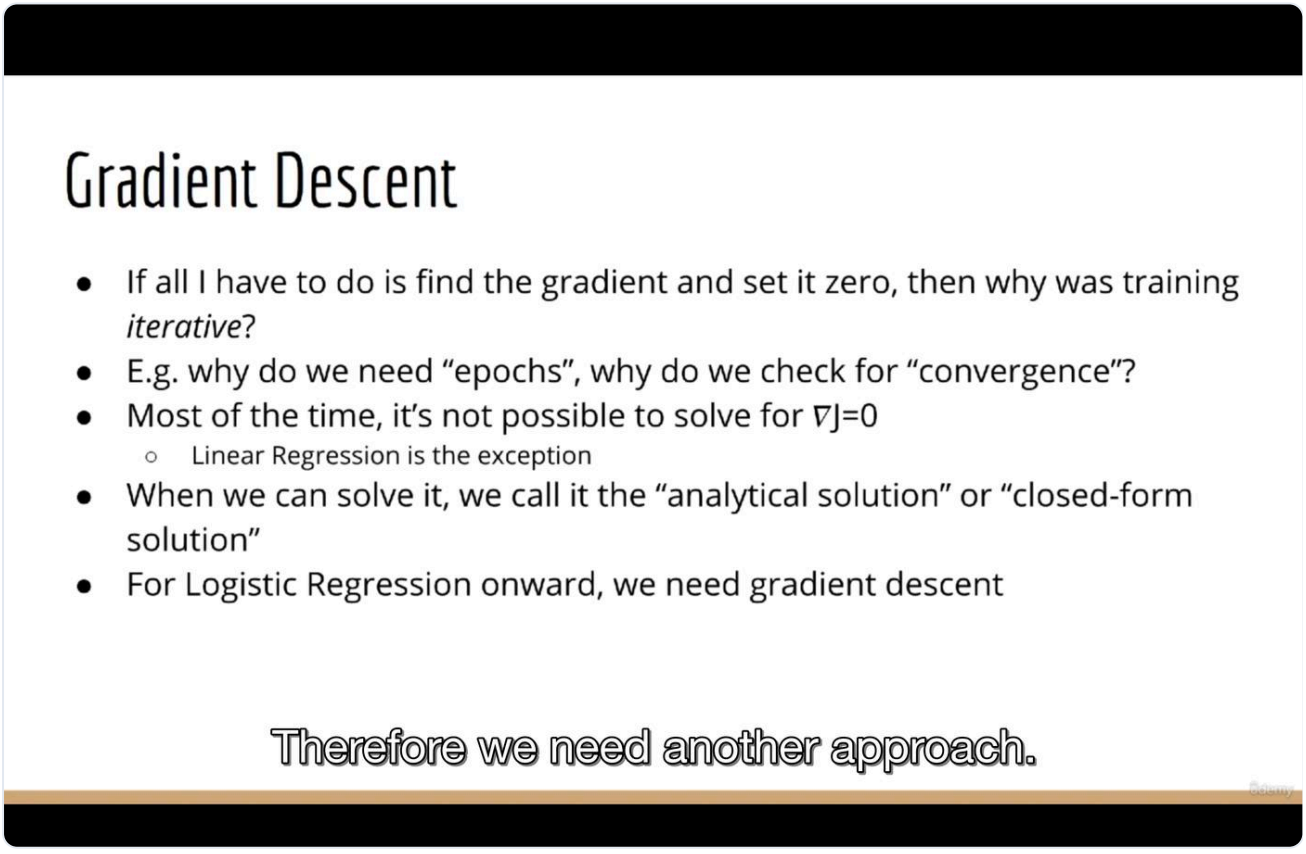


FIGURE 19.12 | Gradient Descent

So the only thing you really do have to know is that PyTorch is going to automatically find the gradient of all your weights and PyTorch uses these gradients to train your model at this point you might be wondering if all I have to do is find the gradient and set it to zero. Then why in our previous code did it involve an iterative training process. Recall that we had to specify the number of epochs to train for and this resulted in a plot of loss per iteration which we could check to confirm that the training algorithm converged nicely. Well in actuality it's not possible to actually solve for the equation that you get when you set the gradient to zero.

Most of the time the one exception to this in this course at least is linear regression where we can solve it we call the solution an analytical solution or a closed form solution. Basically this means that we can express the optimal value of W and B using an equation for logistic regression and the rest of the models we'll be discussing. It's not possible to do this. Therefore we need another approach. That approach is called gradient descent. Now there's a lot more to gradient descent than what's in this lecture. And you can check out the in-depth section of this course for that if you're interested. For now let's just talk about the basics. Essentially what we do is we start at a randomly initialized point for both W and B.

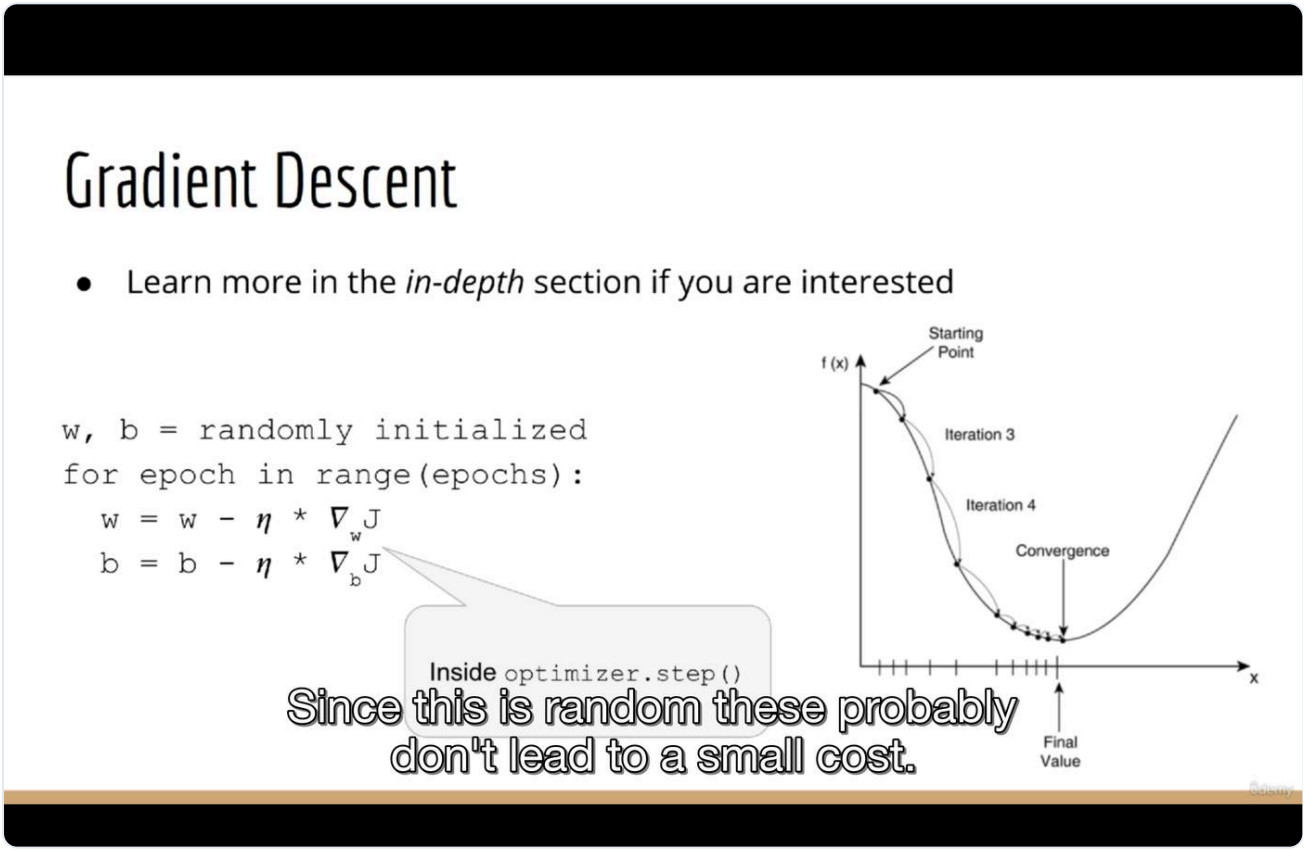


FIGURE 19.13 | Gradient Descent

Since this is random these probably don't lead to a small cost. So then we find the gradient of our loss with respect to W and b we then take small steps in this direction to update W and B on each iteration. Remember these iterations are called epochs mathematically. You can prove that this leads to a decrease in cost although we won't discuss that in this lecture. You can imagine that this is exactly what goes on inside the optimizer dot step function.

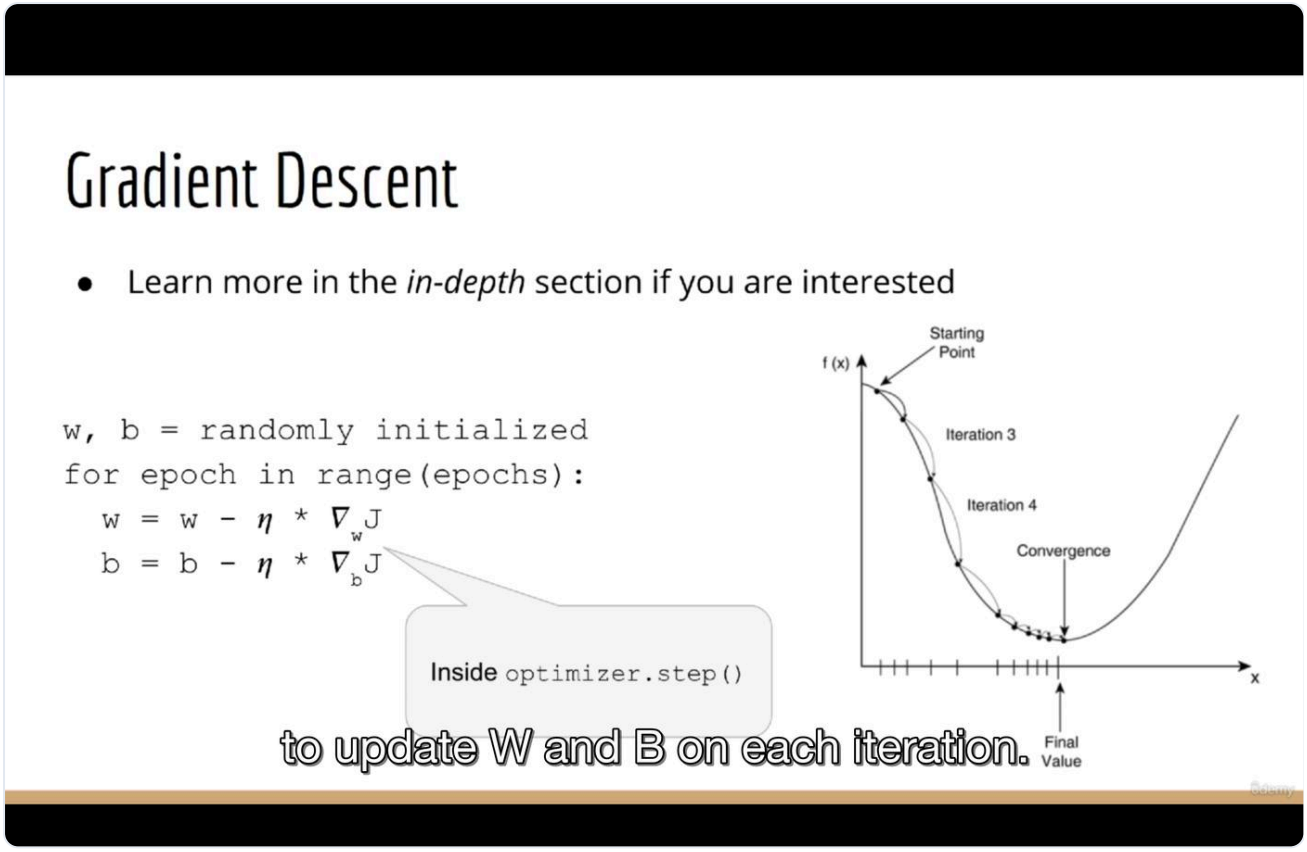


FIGURE 19.14 | Gradient Descent

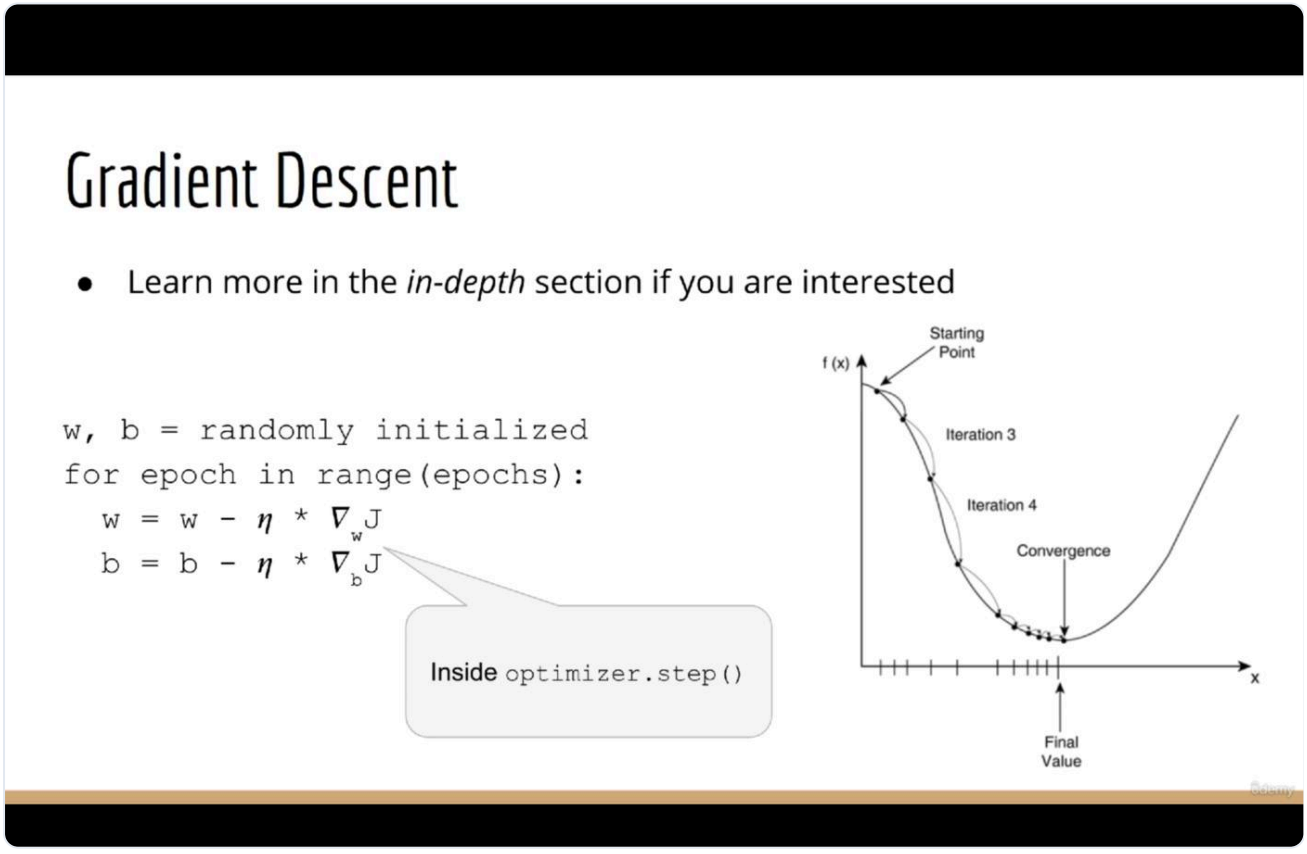


FIGURE 19.15 | Gradient Descent

This is really all that's happening for epoch in range epochs and then set w equal to W minus eta times the gradient of J. With respect to W and set B to B minus eta times the gradient of J with respect to B. So again all we're doing is taking small steps in the direction of the gradient with respect to W and B and one obvious question that arises from this is how small should these small steps actually be. Well the step size is specified by this Greek letter eta which we call the learning rate that specifies how fast or slow we want to train our model. It's important to set this value right because if you don't then your model won't get good results even if your model is actually a good model unfortunately there is no direct method of choosing a good learning rate.

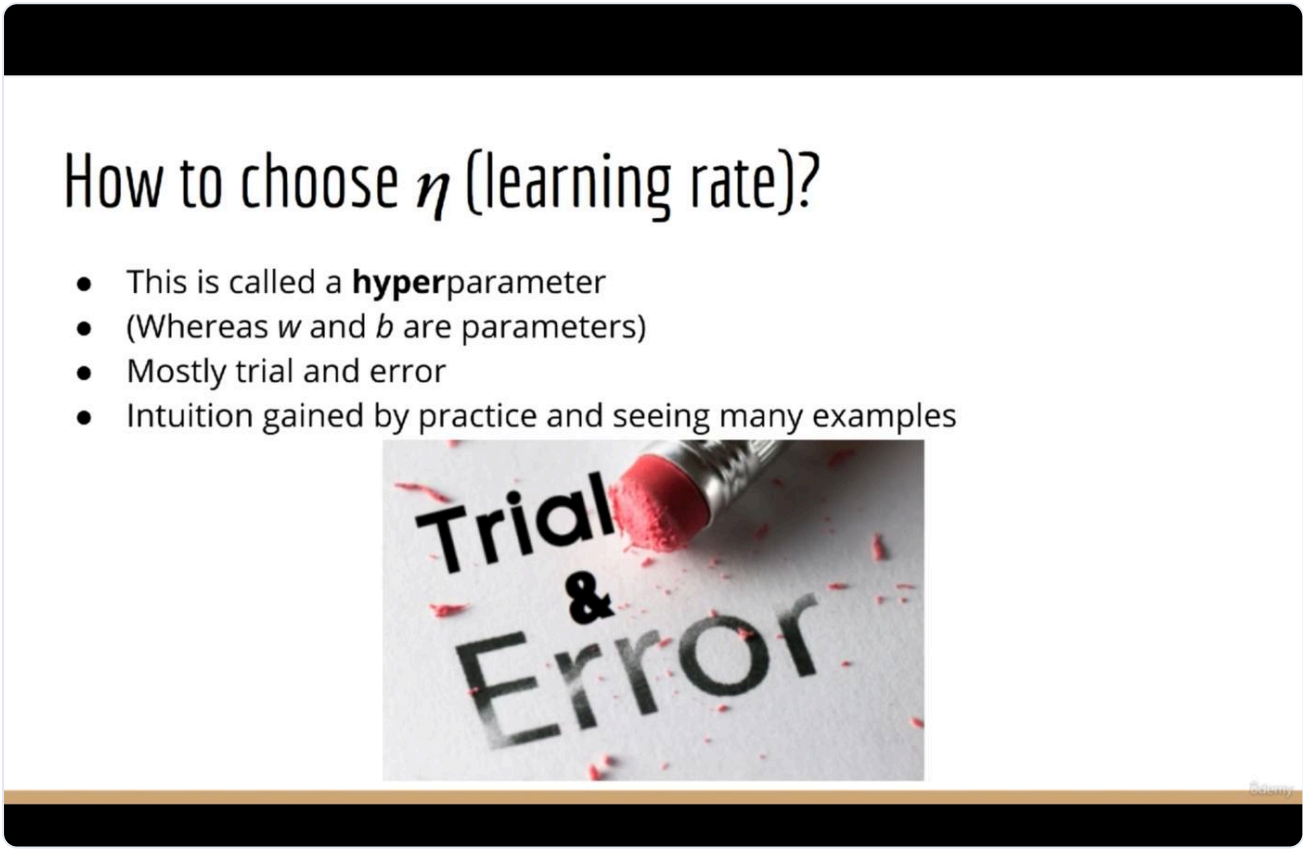


FIGURE 19.16 | How to Choose V (Learning Rate)?

Generally speaking the learning rate is something we call a hyperparameter. This is to differentiate it from W and B which are just regular old parameters. It's called a hyperparameter because it's still a parameter but it's not a parameter of the model itself. It's sort of like a meta parameter and in fact no hyperparameters are really chosen directly. It's more of a process of trial and error along with intuition that you gain from practicing a lot and seeing a lot of different examples as mentioned before.

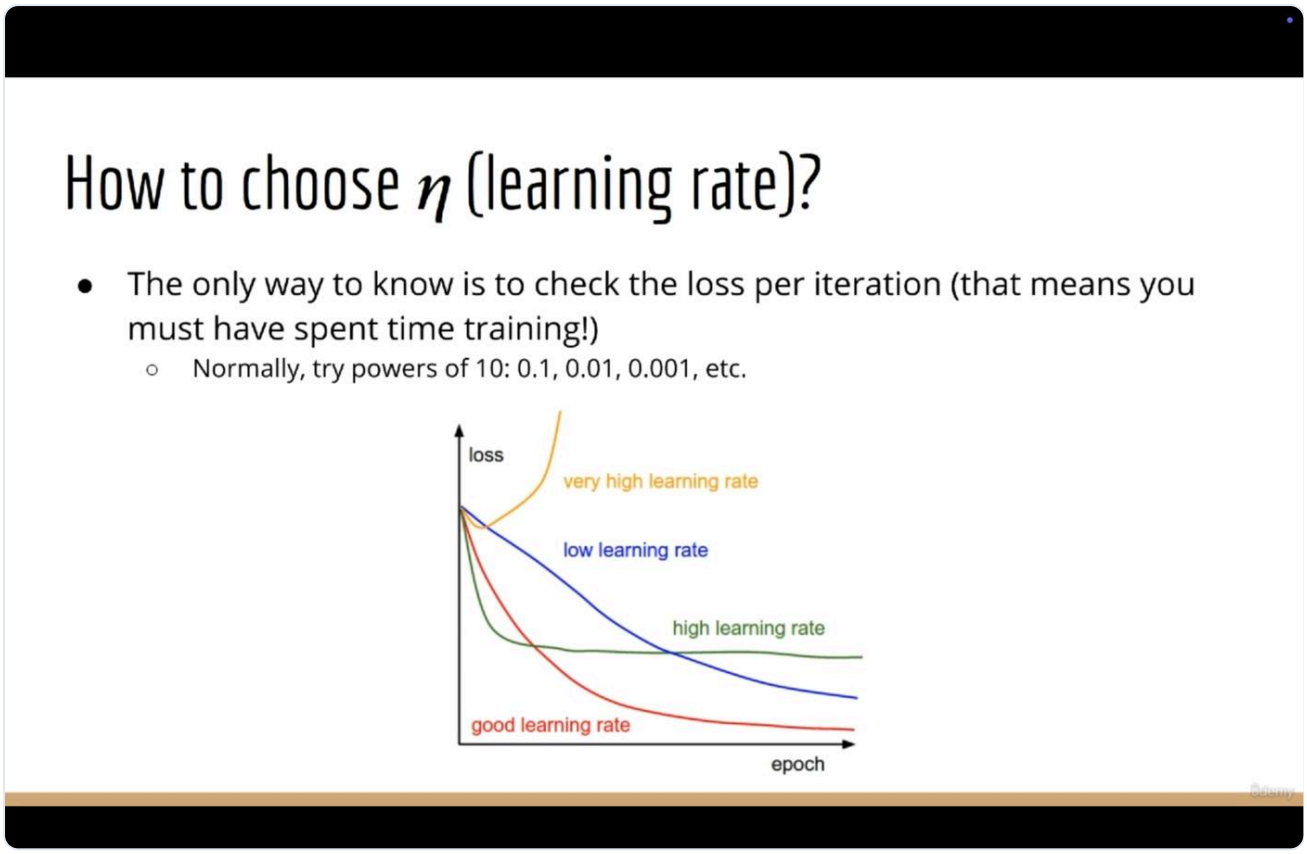


FIGURE 19.17 | How to Choose Y (Learning Rate)?

One way to know if your learning rate is too high or too low is to check the loss per iteration after training. This is somewhat unfortunate because training can take a long time and you only know the result once it's done. If you use a bad learning rate and got bad results while you still had to wait for them I really like this visualization because it encapsulates how to choose the learning rate quite well. If you're learning rate is too high then your loss will just shoot off to infinity.

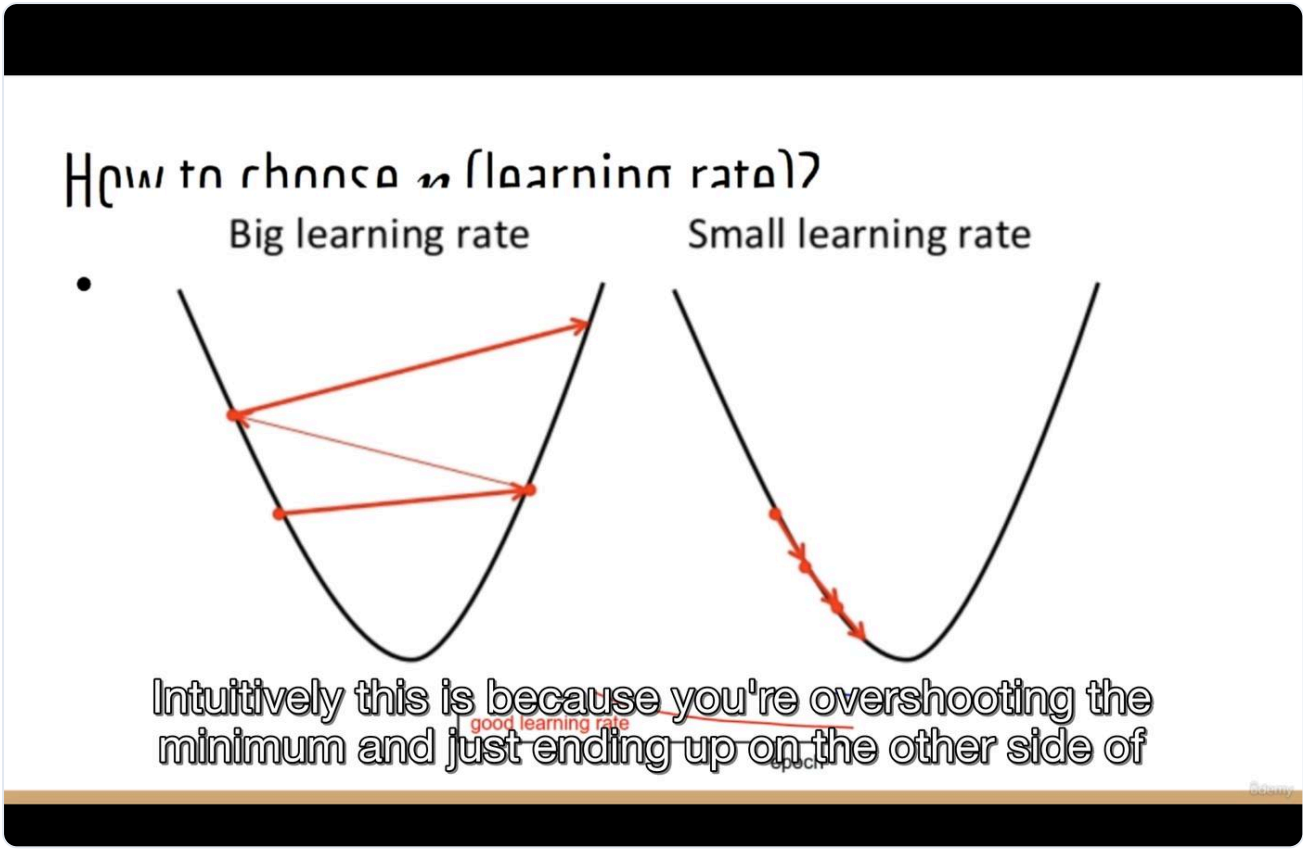


FIGURE 19.18 | Big Learning Rate Small Learning Rate

Intuitively this is because you're overshooting the minimum and just ending up on the other side of the canyon that means your steps are too large and you need to make them smaller. Sometimes your learning rate might be a bit too high so you'll see some convergence but it'll converge to a sub optimal value if your learning rate is too low then you'll see a really shallow curve. This is also not good because as I mentioned in training takes time. So if your learning rate is too low you'll have to wait longer. But not only that sometimes you can just get stuck at a suboptimal point. That's also not good. A good learning rate is in between these two extremes and so what you'll probably end up doing is finding the limits of your particular dataset what's too high what's too low and then trying different numbers in between until you find something good.

Normally we try numbers that are powers of 10 for example zero point one zero point zero one zero point zero zero one and so forth.

LECTURE 20

Binary Cross Entropy and Cross Entropy Loss

Check the notebook `3.PyTorch-Linear-Classification-with-Logits.ipynb` for the code.

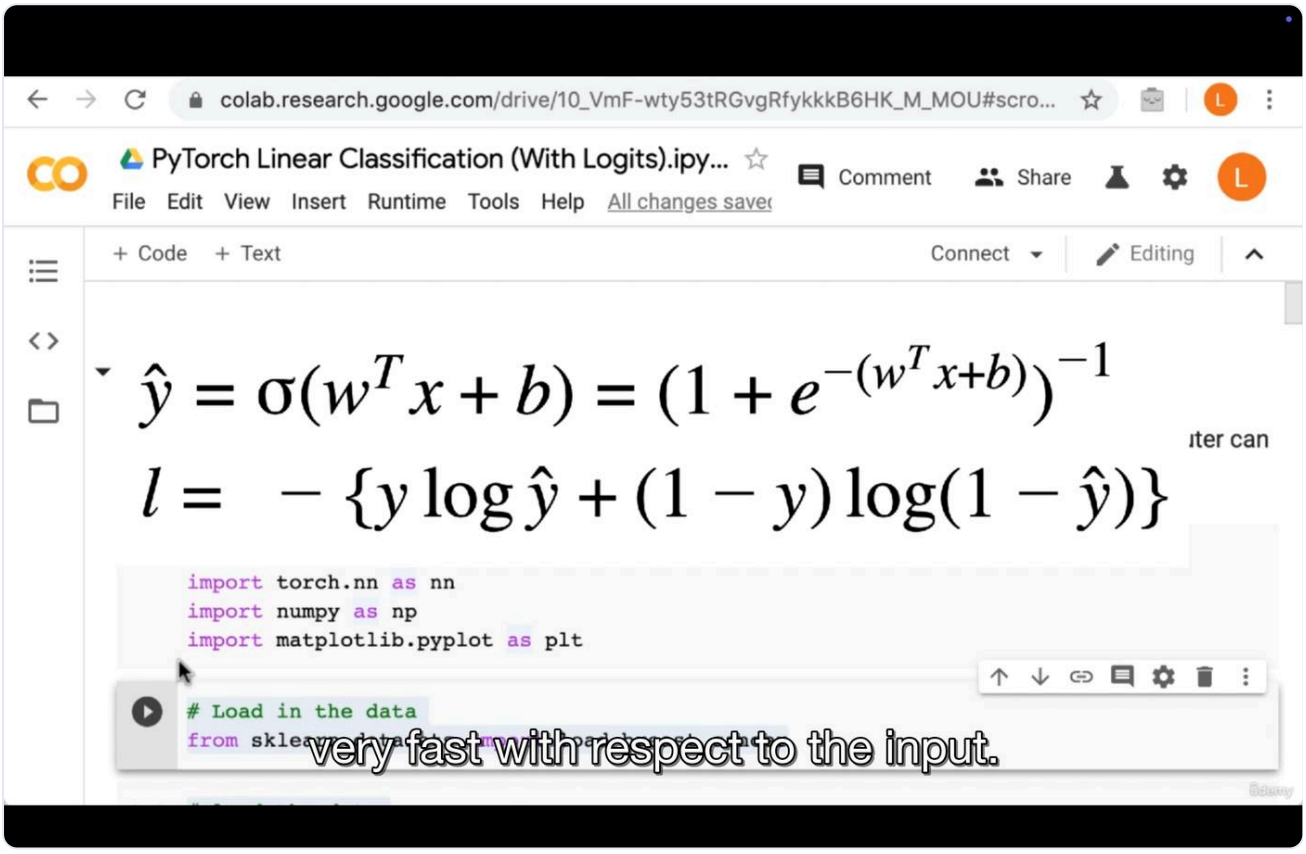


FIGURE 20.1

In this lecture we are going to look at an alternative and better way of building a logistic regression model in PyTorch as usual. You can look at the title of the notebook to determine what notebook we are currently looking at the motivation behind doing this is that when we perform the sigmoid we have to do an exponential operation. The problem with that is that exponentials are numerically unstable due to the fact that they grow very fast with respect to the input. In addition the log term in the binary cross entropy is also unstable for similar reasons. Well it turns out that because the binary cross entropy loss has a log in it it almost cancels out the exponential in some sense although we won't go through any derivations in this course.

The end result is that if you know you have to do both the sigmoid and then the binary cross entropy loss right after there is a way to combine them into a single function so that you avoid numerical instability in particular. You can express the loss only in terms of the activation which is also called the logit. In this case if you've ever studied statistics then you may have heard of this term before. Otherwise don't worry. Basically the logit is just the input into the logistic function and the logistic function is just another way of saying the sigmoid function. So in this script pretty much everything is exactly the same except in two places the first place is when we create the model so before we use the sequential and combined the linear layer with a sigmoid layer.

But in this script we no longer need the sigmoid since that's included in the cost calculation and therefore our model goes back to being just a linear model. The output of this model is the logit. Then when we create the loss function we use BCE with logits loss instead of BCE loss like we were earlier. As its name suggests this function calculates the binary cross entropy loss directly from the logits as you can see everything after this is the same as well. So our training loop is the same and we get the same results. One more difference we have to pay attention to is when we make predictions. If you recall the sigmoid always outputs a number between 0 and 1 and we interpret that as a probability.

So when we want to make a prediction we simply round those probabilities. Anything greater than zero point five is a prediction of one. And anything less than zero point five is a prediction of zero. But what do our predictions look like now since the output can again be any number. In this case we just go back to our geometrical picture. You can also think of what the sigmoid actually does. And if you follow that logic this should make sense. If you recall anything on one side of the hyperplane defined by our model is positive and anything on the other side is negative. The sigmoid simply maps all the positive numbers to probabilities greater than zero point five.

And similarly it maps all the negative numbers to probabilities less than zero point five. Thus it's very easy to make predictions. We simply check if the model output is greater than zero or less than zero. Anything greater than zero is considered a one prediction and anything less than zero is a zero prediction. Note that we can use the greater than sign directly even though the greater than sign returns trues and falses as we discussed earlier in python a true is 1 and False is 0. Therefore we don't need to do any further processing. Once we have our predictions we can check the model accuracy and again we can verify that the result is the same.

LECTURE 21

Multiple Validation Sets

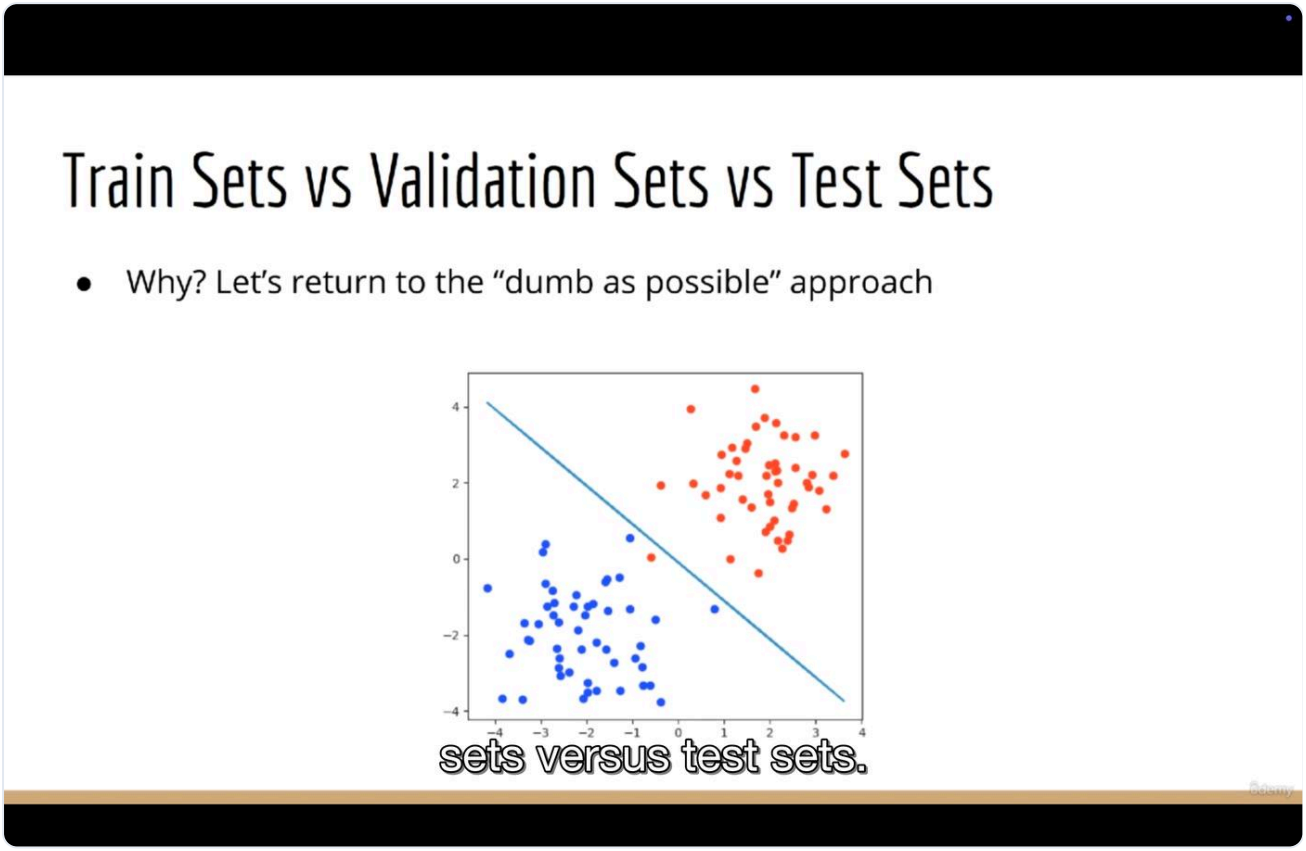


FIGURE 21.1 | Train Sets vs Validation Sets vs Test Sets

In this lecture we are going to talk a little bit more in-depth about training sets versus validation sets versus test sets. First let's think about why we would want to split our data up in the first place. As usual I'm going to return to my dumb as possible picture which is that machine learning is nothing but a geometry problem. Imagine that all the data I collected in the past looks like this. So I find a line that can discriminate between the two classes of data and I can conclude that my classifier has pretty good accuracy. Great. But remember what's the point of building a classifier. It's to use it on unseen data that I want to classify in the future so what happens when tomorrow's data looks like this.

Now all of a sudden the classifier is no good.

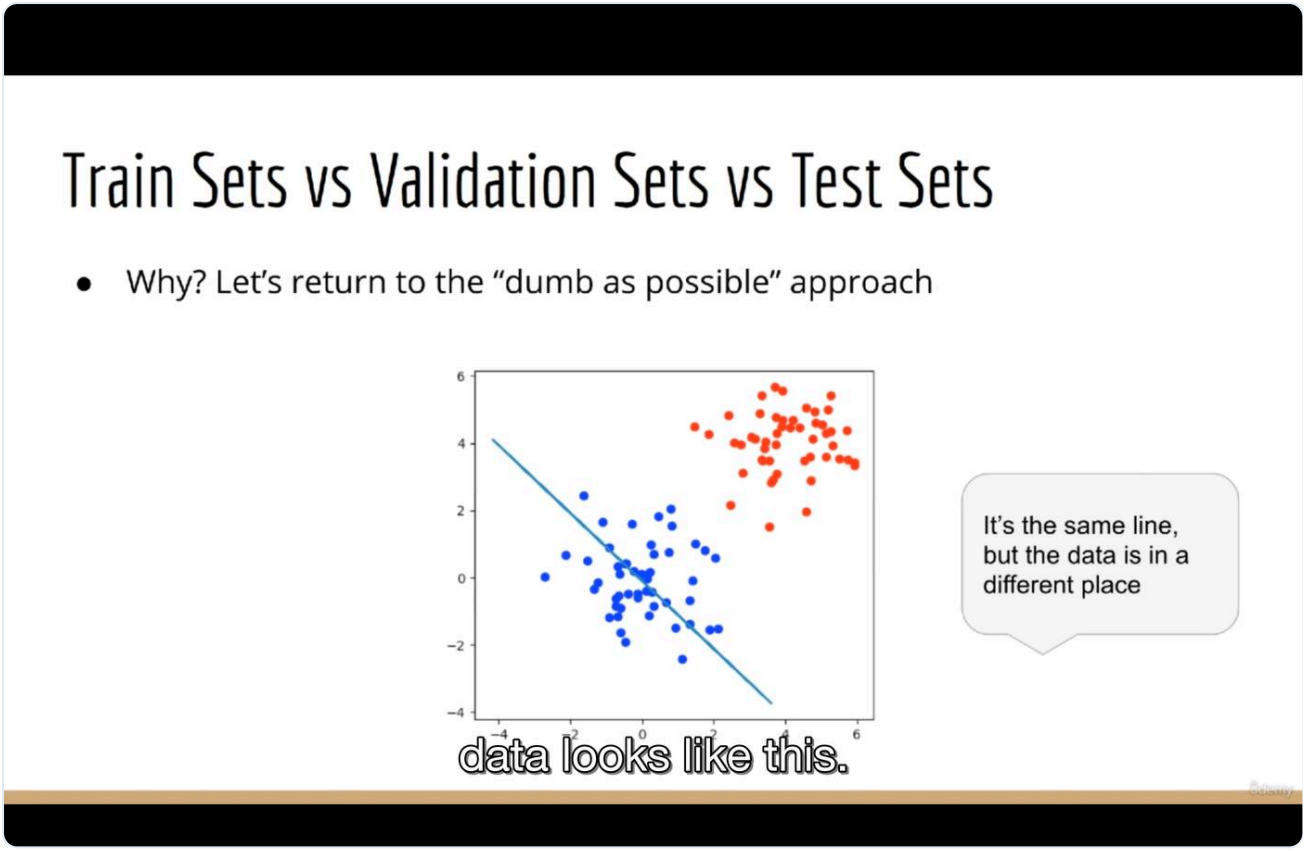


FIGURE 21.2 | Train Sets vs Validation Sets vs Test Sets

In fact it's terrible Well that sucks because this is actually the data I really care about classifying. I don't care about classifying past data. Why. Because I already know what the answer is. So there's no need to classify those data points anymore. Now you might think this example is a little extreme. Clearly the data we have seen so far in this course did not behave this way when we get good training accuracy. We tend to get good test accuracy but in fact we'll see some data that behaves exactly like this later in the course here's another image that should help you understand why we need train and test sets. The problem is over fitting if you have a complex enough model you will get very good accuracy on almost any data set but you can see here that although we fit very well to the training data we have a poor fit on the test data which in this case is the true function that we are trying to model.

This is called over fitting and it's the opposite of what we want which is good generalization there is a tradeoff that must happen here should you make your model more complex so that you're training accuracy is improved or should you make your model less complex so that there's a better fit to the test data.

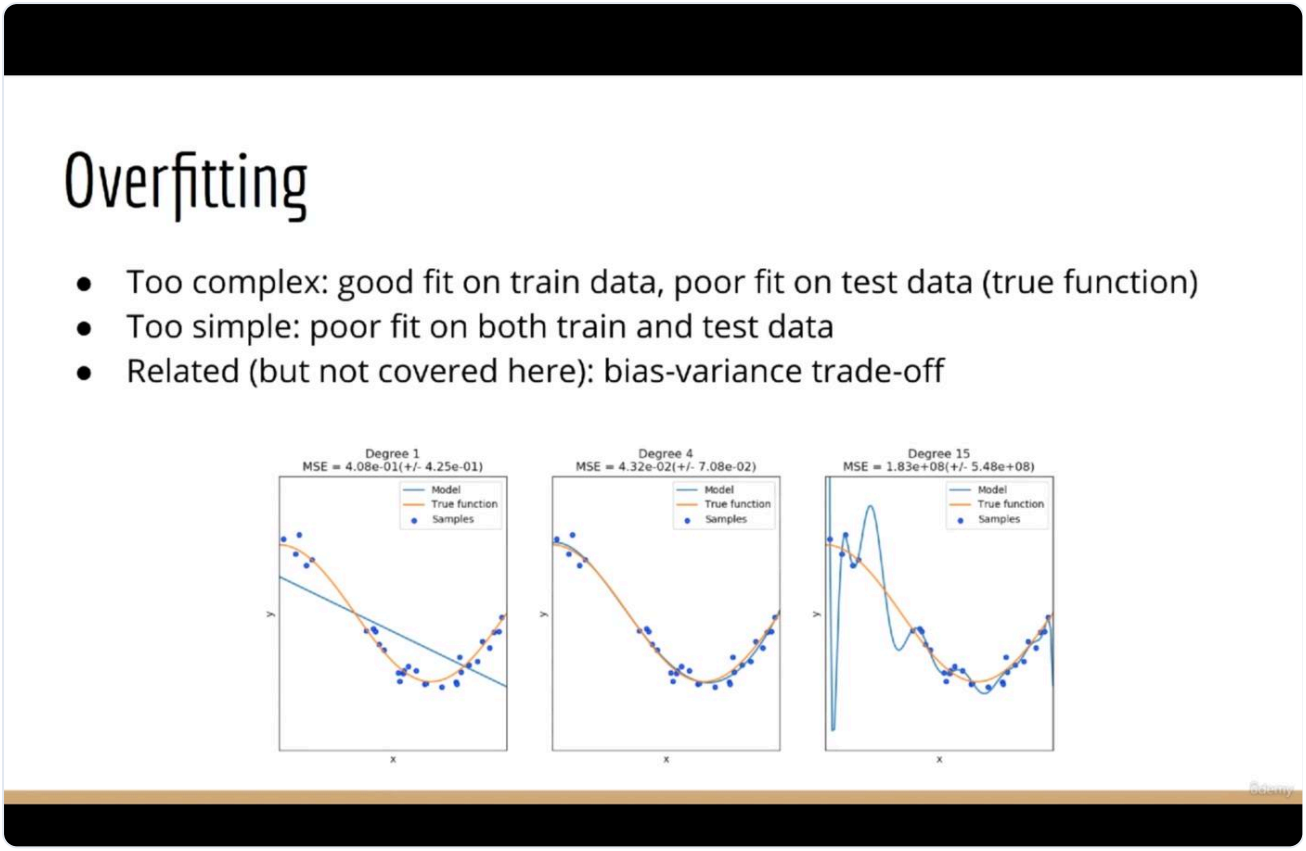


FIGURE 21.3

The problem is if your model is not complex enough then it will perform poorly on both the train and test data. So your job is to find the optimal amount of complexity such that it performs well on the test data. It can't be too complex but it also must be complex enough. You may also know this as the bias variance tradeoff. Although this concept is outside the scope of this course you're encouraged to check out the in-depth series if you're interested in learning more. So I hope that satisfies your intuition about why we need to have separate data for both training and testing. Now you may have heard that sometimes people split their data into three separate data sets the train set the validation set and the test set.

What's the difference. Why is it that sometimes we just have train and test and sometimes we have train validation and test.

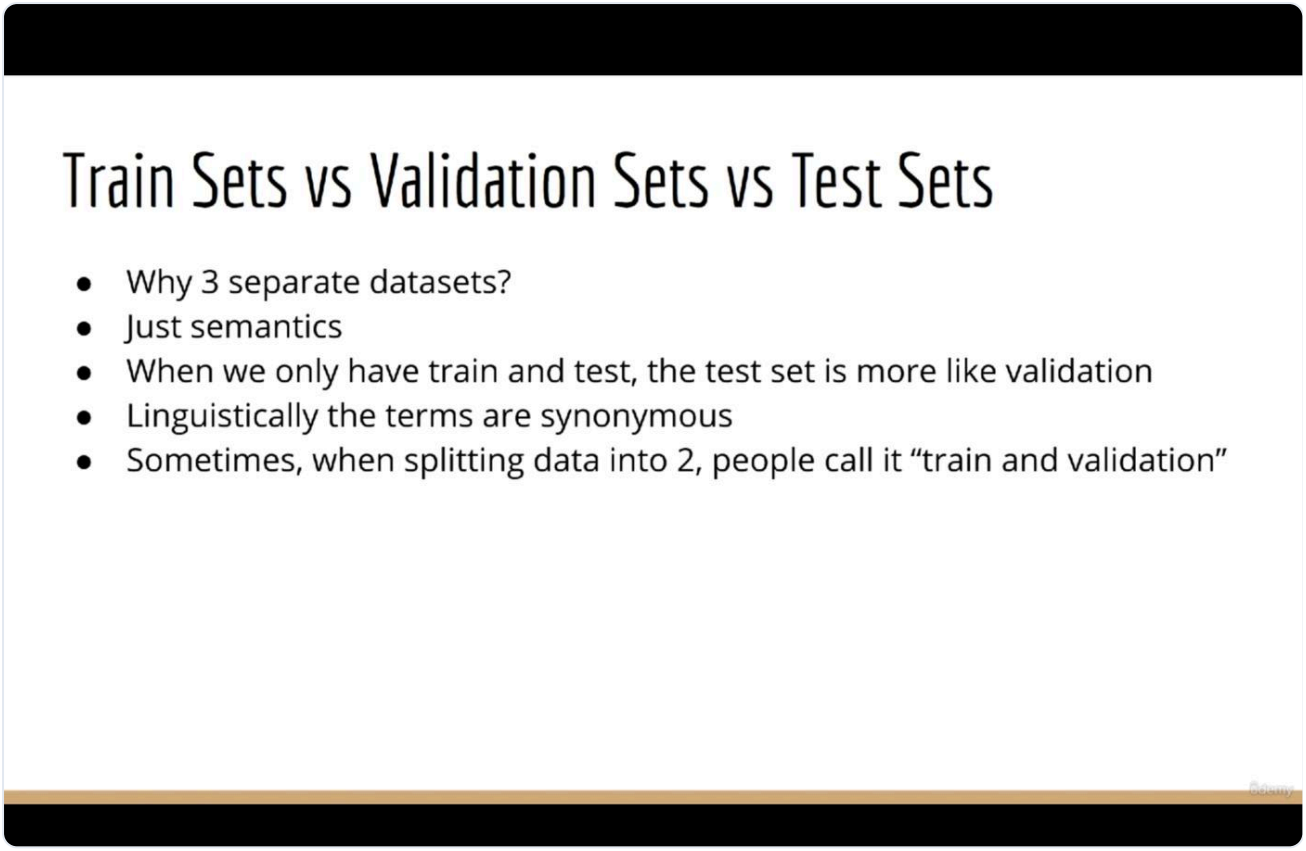


FIGURE 21.4 | Train Sets vs Validation Sets vs Test Sets

It's really just semantics when you have a train and test set only your test set is more like a validation set. In fact the words test and validation are pretty much synonymous. They mean the same thing linguistically. What you are doing when you split your data into train and test is you are actually using your test set to validate that your model works on unseen data. So in fact sometimes when people split their data into two parts they'll call them train and validation sets rather than train and test sets. In this context the meaning of both is exactly the same. I hope you're not confused so in what case well we want to distinguish between train validation and test for this.

I like to think of Kaggle contests although it's well known that Kaggle contests are not a good representation of how you will do machine learning in the real world. In fact there have been multiple instances of cheating on the platform. In any case the way that Kaggle works is this you will have some data that Kaggle gives you to train your model on. These are typically labeled as train and test. Importantly the test set does not come with any labels so you can't compute your models performance on the test set by yourself. This is unlike when we split the data into train and tests in our class all you can do with your test set is make predictions and submit them to Kaggle servers.

Then Kaggle will show you your test score along with the test scores from other participants.

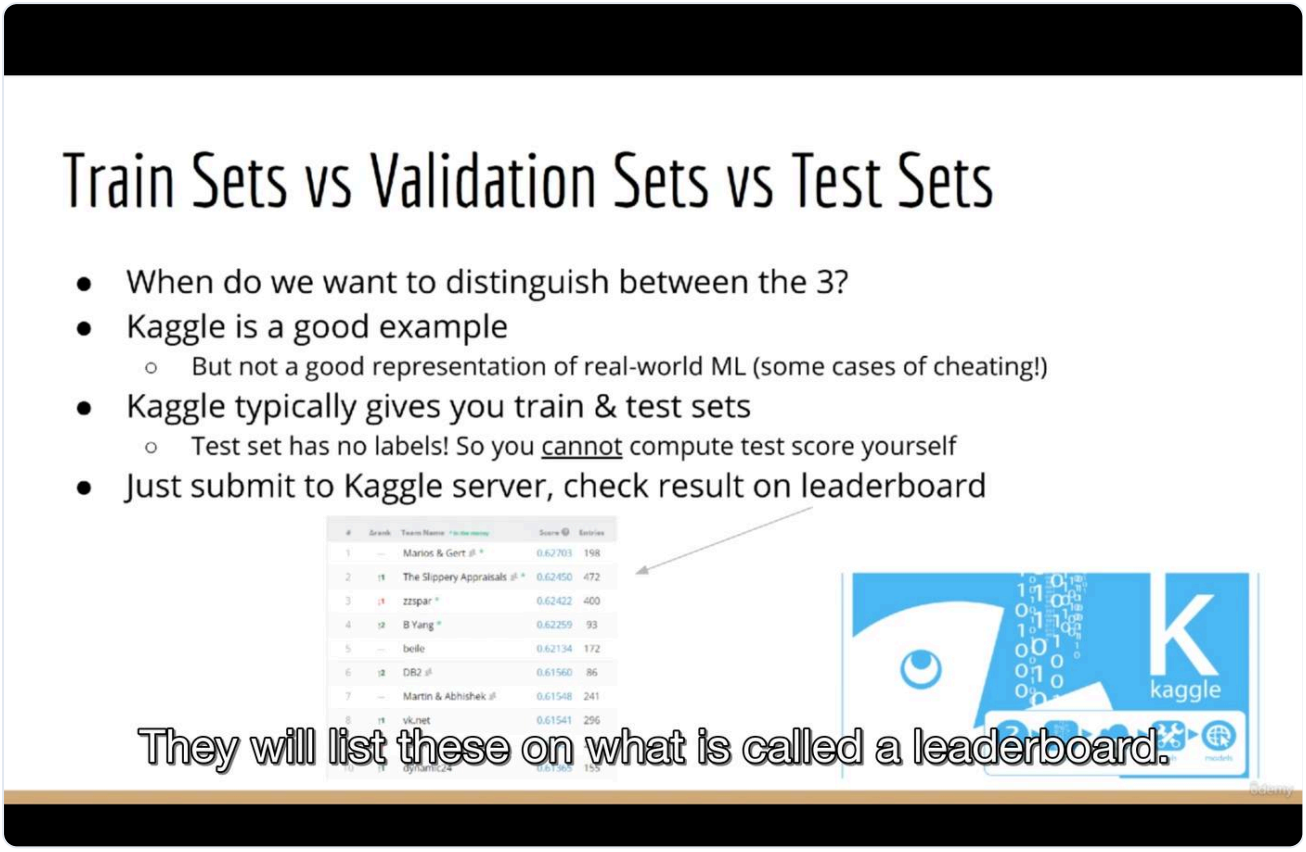


FIGURE 21.5 | Train Sets vs Validation Sets vs Test Sets

They will list these on what is called a leaderboard. This is the true test set because it's how Kaggle is testing your model. But if that's how Kaggle tests your model then how will you test your model.

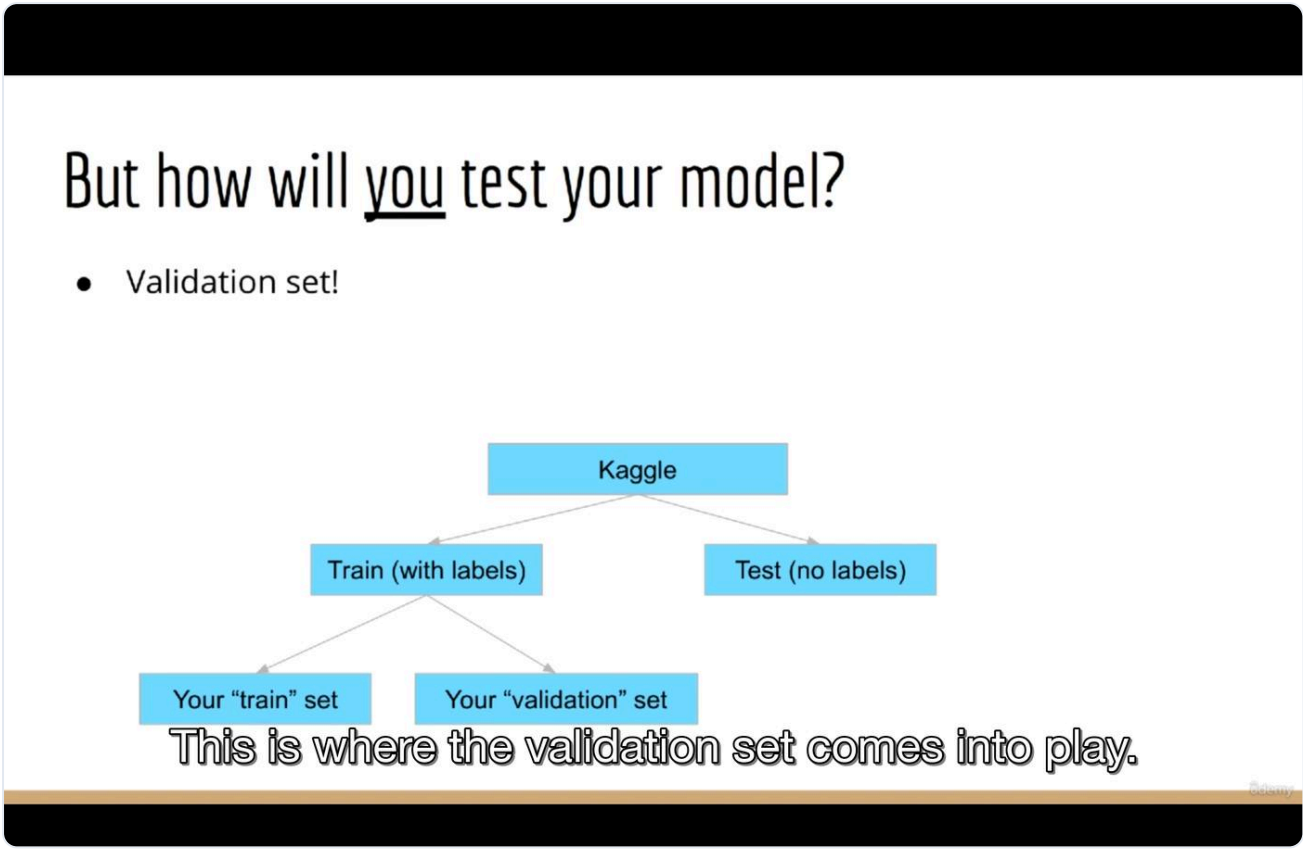


FIGURE 21.6 | But How Will You Test Your Model?

This is where the validation set comes into play. Remember it Kaggle.

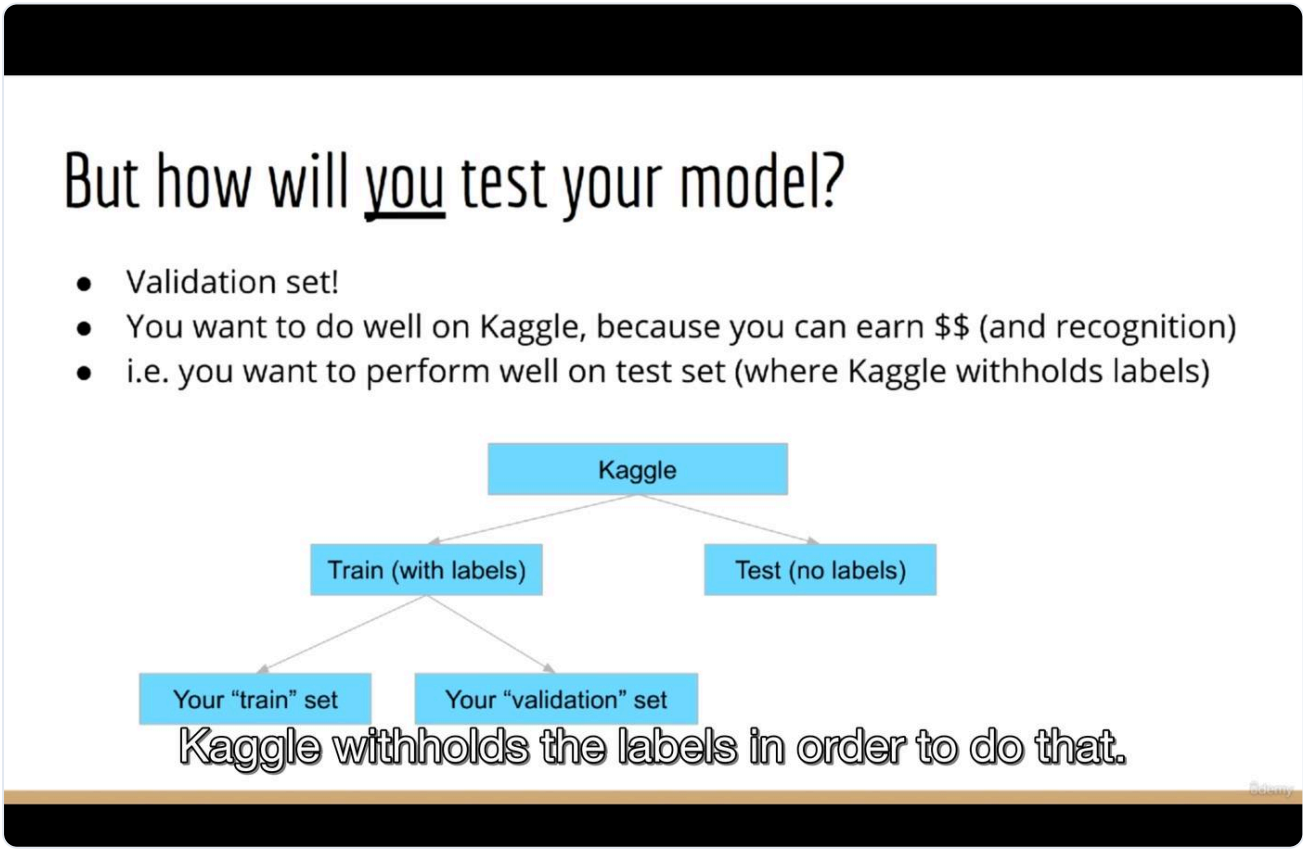


FIGURE 21.7 | But How Will You Test Your Model?

Generally speaking although this is not always the case we'll give you just that train and test set you want to perform well on the test set because if you win then you'll make some money. So you have to make sure that your model performs well on unseen data namely this test set for which Kaggle withholds the labels in order to do that.

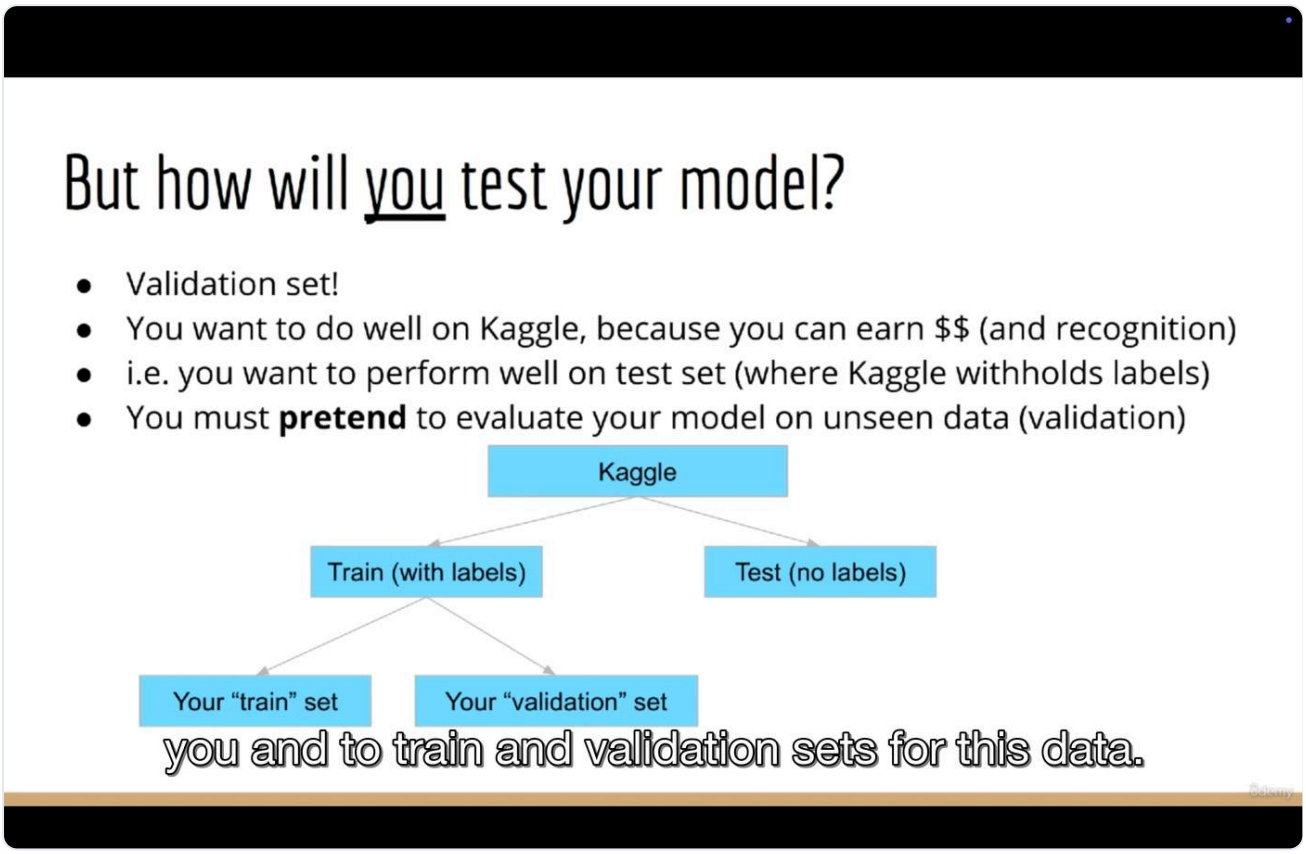


FIGURE 21.8 | But How Will You Test Your Model?

You have to pretend to evaluate your model on unseen data by splitting the train set that Kaggle gives you into train and validation sets for this data. You do have all the labels and so you can calculate both the train and validation accuracy sometimes. Although this isn't usually done in deep learning you will split your data randomly into train and validation sets multiple times. We call this cross validation.

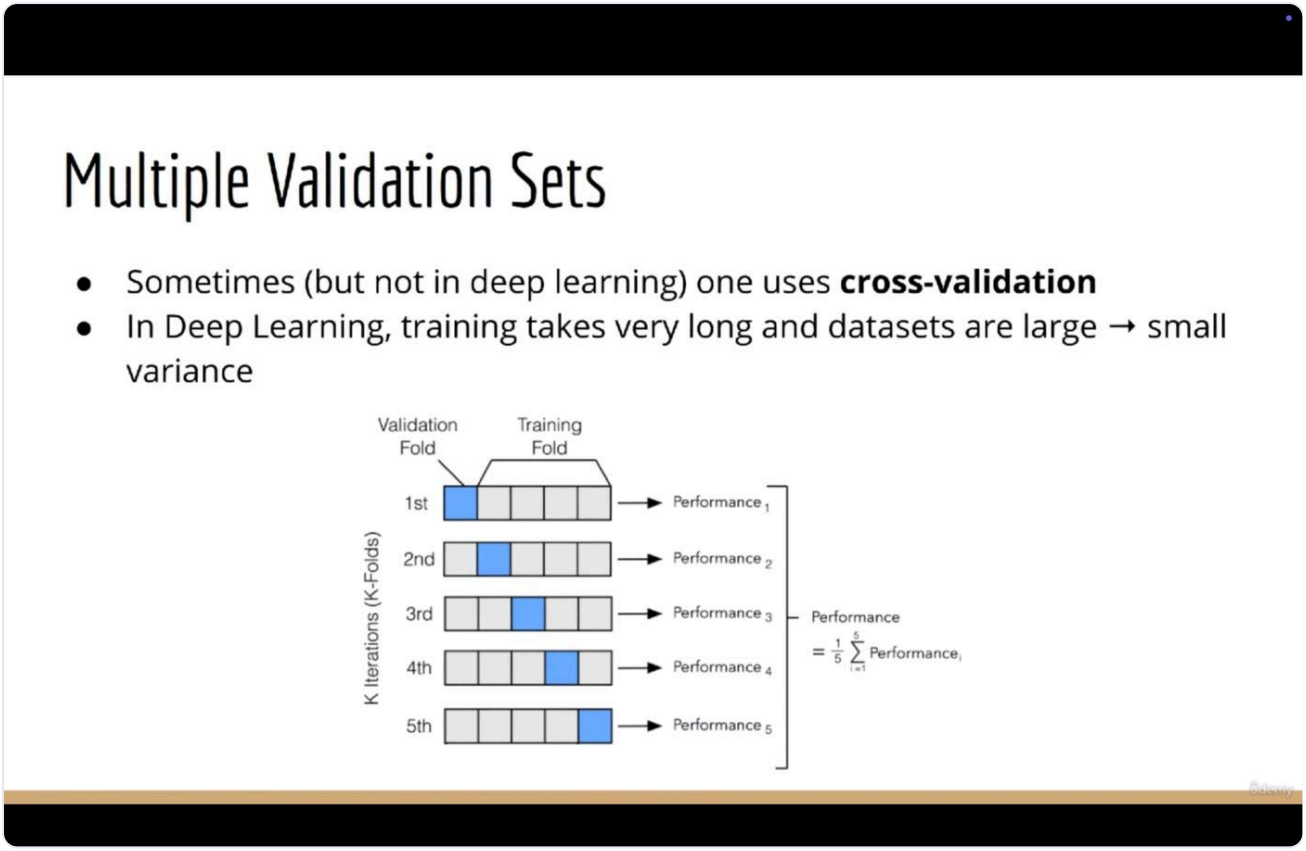


FIGURE 21.9 | Multiple Validation Sets

Here's the basic idea let's suppose you split your training data into five parts. Then you'll do a loop which runs five times on each iteration of the loop you'll consider four of the parts to be your train set and the remaining part to be your validation set on each iteration. You will train a new model on the current train set and evaluate the model on the current validation set. When you're done you'll have five different validation scores.

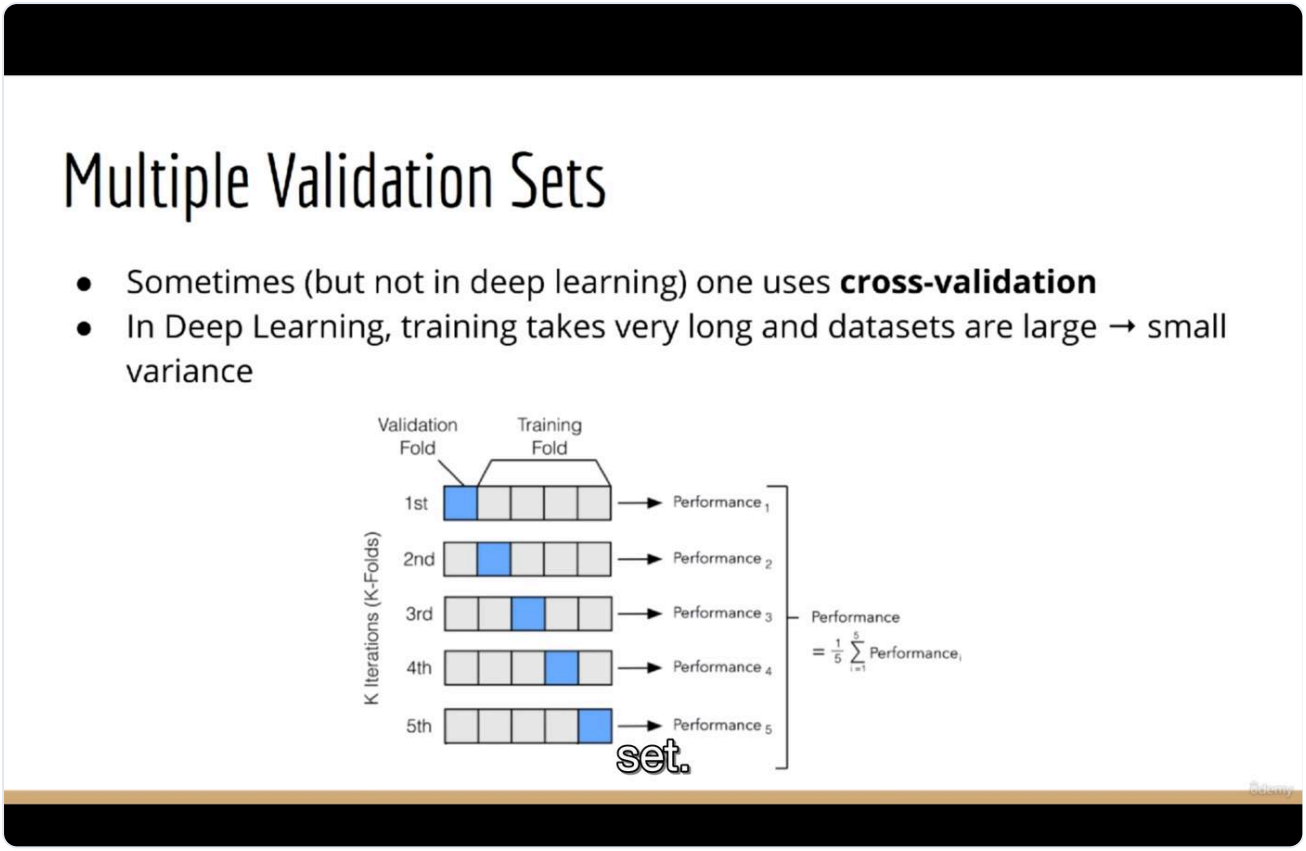


FIGURE 21.10 | Multiple Validation Sets

And from this you can calculate the mean variance and so forth to get some statistical measure of how well your model might perform on unseen data. The reason this is in use for deep learning is that the data sets are usually so large and training takes so long that the variance is pretty small and just one validation set should give you a pretty good idea of your model's performance. There is a small caveat to the leaderboard. In fact there are two leaderboards.

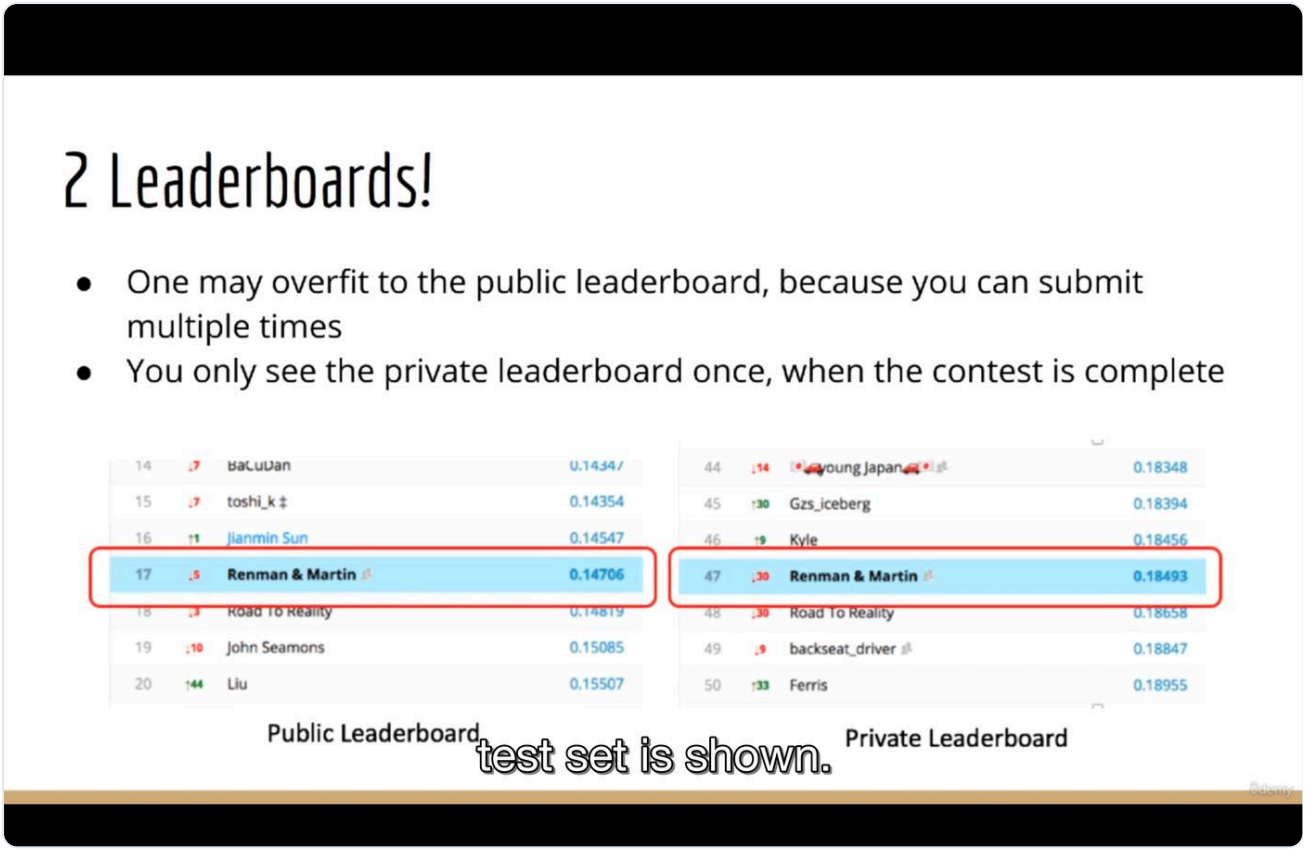


FIGURE 21.11

The public leaderboard and the private leaderboard the public leaderboard is where your score on the test set is shown. The private leaderboard uses yet another dataset and this is the one on which the winner of the contest is selected. This is of course to prevent over fitting to the public leaderboard test set the public leaderboard allows you to submit a solution multiple times so you have some chance of overfitting to it. On the other hand you only know your private leaderboard placement once at the end of the contest.

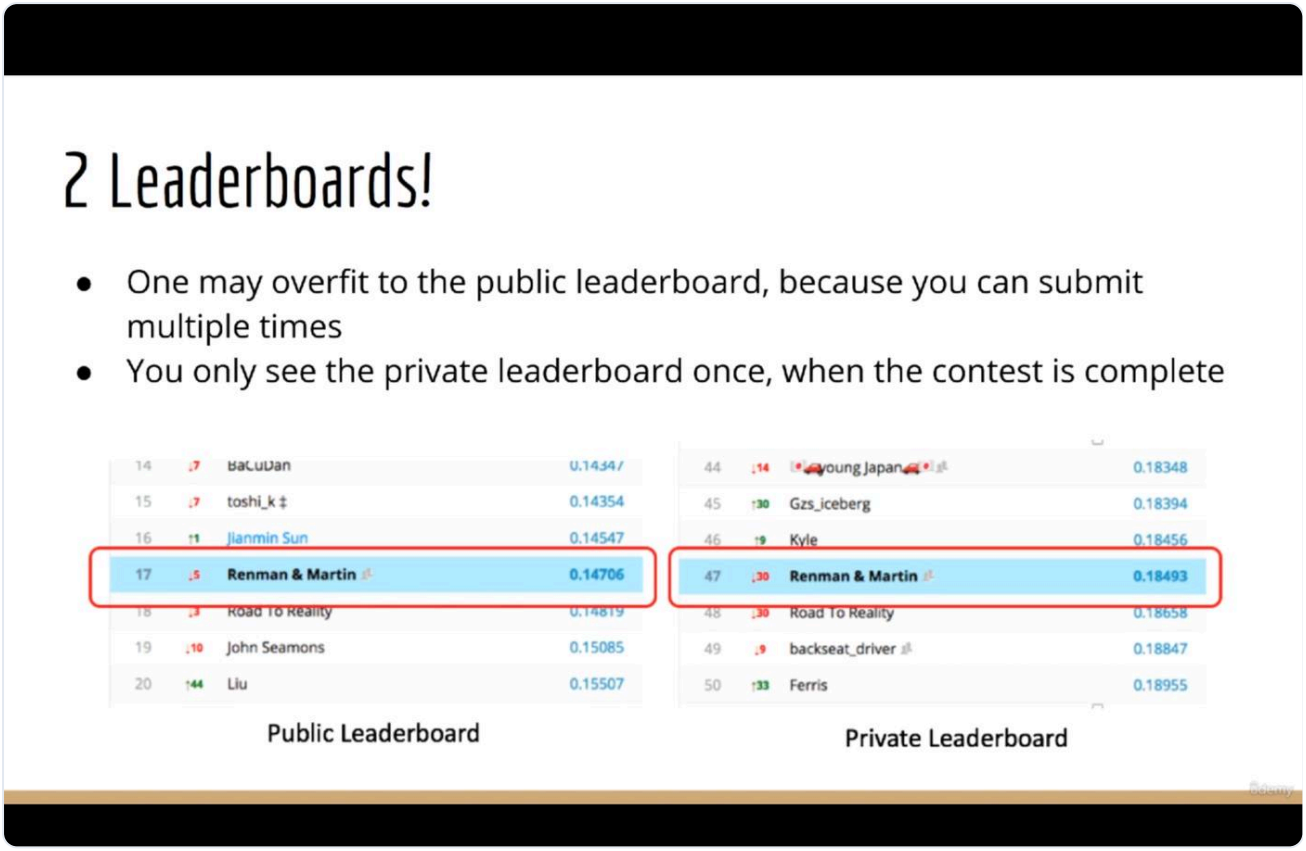


FIGURE 21.12

So in fact to confuse you even more now there is not just the train set. The validation set and the test set. But there is yet another test set of course Kaggle is not the real world's in the real world. Your test set is the set of data for which you really want the answer.

FIGURE 21.13

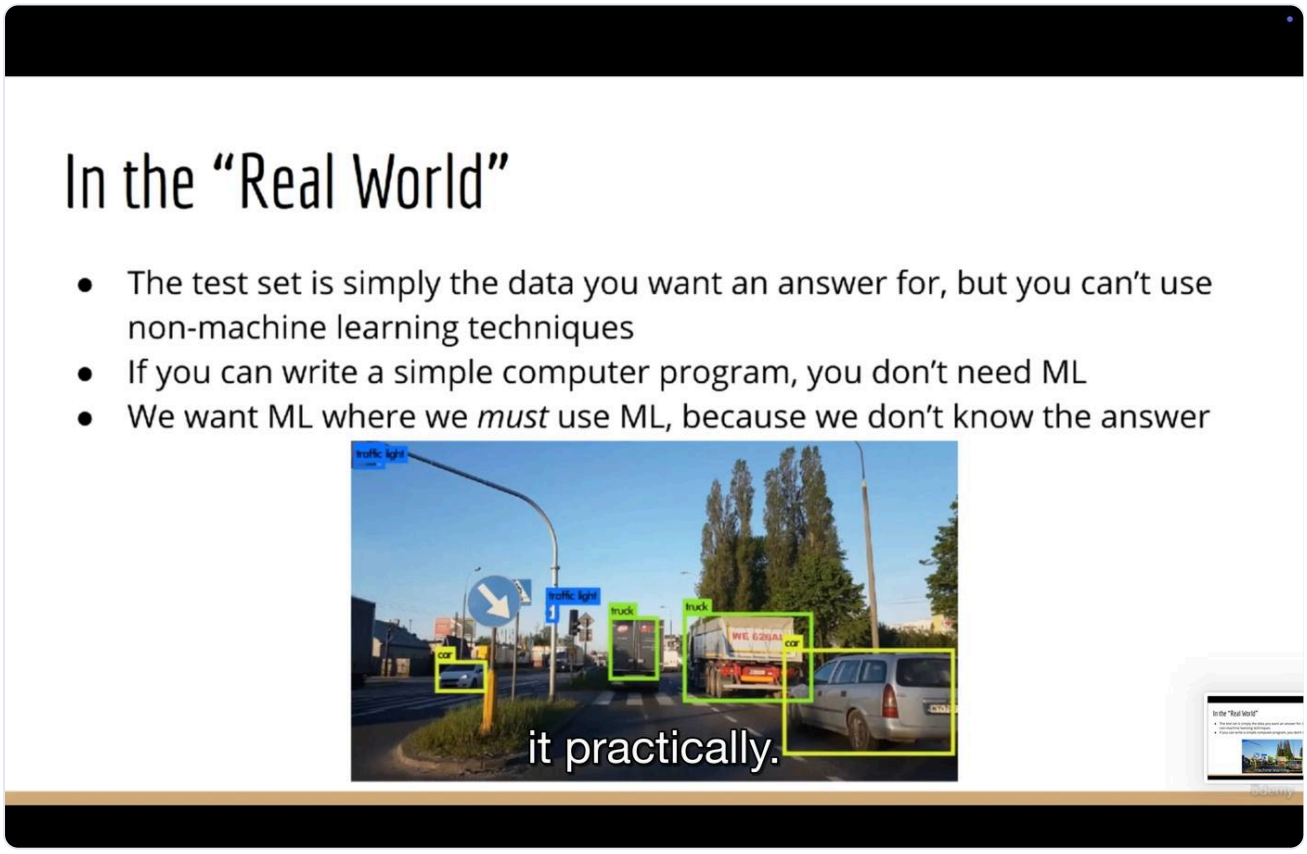


FIGURE 21.14

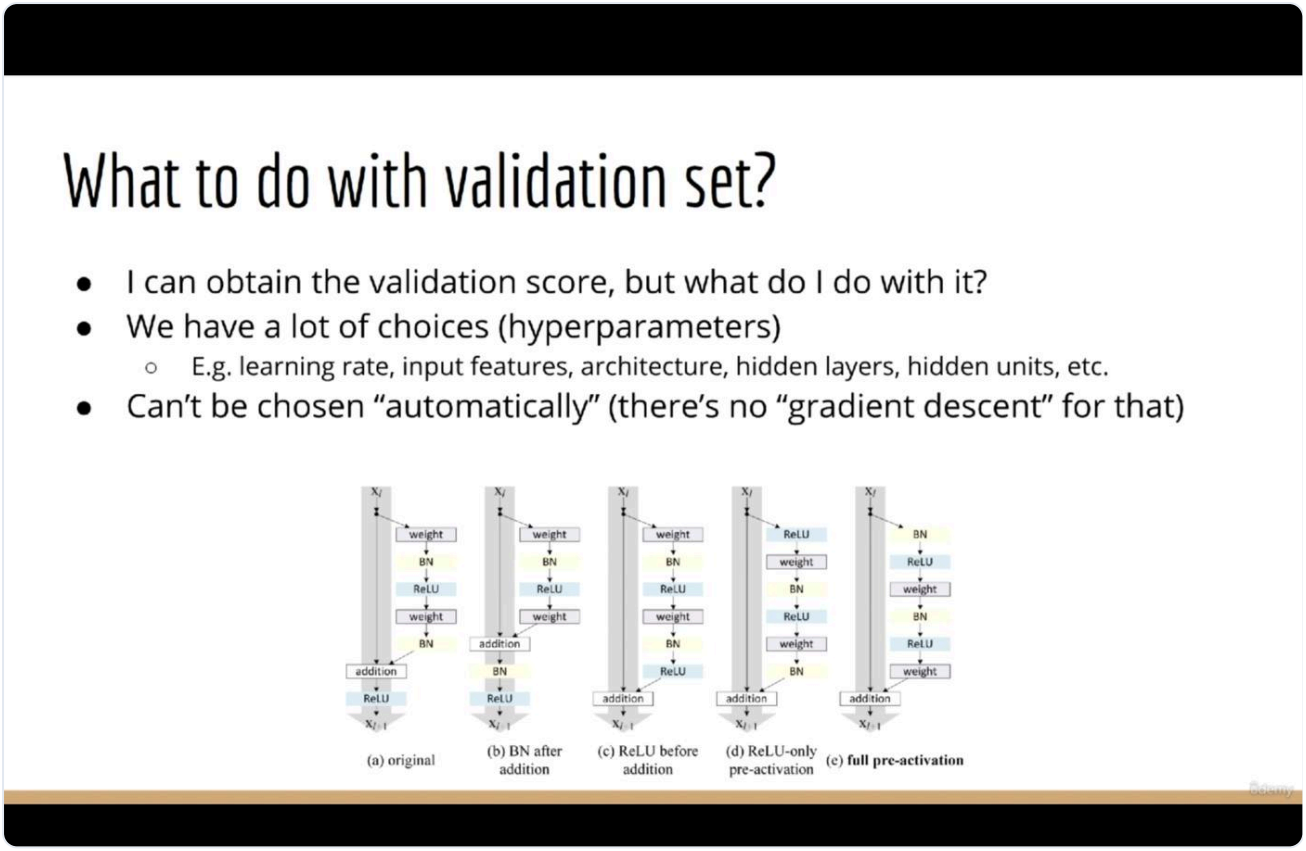


FIGURE 21.15 | What to Do with Validation Set?

But you can't produce using non machine learning techniques. If you could just write a basic computer program or implement a simple set of rules then you don't need machine learning. So when we are using machine learning it's because we really do not know the answer or we can't compute it practically. Some basic examples of this are fraud detection spam filtering or object detection we really want these to work on data that we haven't seen before. But it's clear that we can't do that work manually. The next question I want to answer is what is the validation set even for you might be thinking. Sure I can evaluate the model to get some idea of how I will perform on unseen data but then what.

What do I do with that result. How is it actionable recall that in machine learning. We have a lot of choices. We call these choices hyperparameters. For example in gradient descent we have to choose the learning rate. We can't learn the learning rate as you learn more about different variants of gradient descent. You will have to choose between those variants and then among those variants there are yet more choices of hyperparameters. If you have a lot of input features you might want to choose just a subset of the features so that the model can learn from the important features rather than just the noise. You also have to choose between different model architectures.

After you learn about basic linear models you'll learn about ANNs CNNs RNNs and so forth. You'll have to choose the number of layers in a network and the number of units in a layer. You'll also have to choose activation functions. There so many choices you will feel overwhelmed. Unfortunately it's not possible to choose these hyperparameters automatically. You can't just do gradient descent on model architectures. At this stage your approach amounts to just trial and error. And so how do you evaluate each of these different choices. The answer is by using the vali-

dation set typically you will pick the model that gives you the highest validation score and that will be the model that you use on your test data whether that's a Kaggle contest or pushing your model to production.

LECTURE 22

So in this short lecture, I want to...

So in this short lecture, I want to tell you about my suggestion box. The idea behind this is analogous to the kind of suggestion box you would find at a restaurant. You can write down your thoughts and leave them for the owner to look at to give them feedback so that they can improve your dining experience in order to share your thoughts with me and my suggestion box. Please go to lazy programmers on slash suggestions. Here you will find a simple form where you can tell me anything you like. I have a few suggested fields, but none of them are required. However, they would be helpful for me to know. For example, tell me your background. I think someone coming from a background in marketing is going to have very different feedback compared to someone coming from a Ph.D.

In physics. Tell me what course you are taking, so I have some idea of what you are talking about. Obviously, the more specific you are, the better I can help you. Tell me how difficult you thought the course was. Was it too easy? Was it too hard? Tell me if there was something I neglected to explain. For example, was there some word that I use that you have never heard of? Was there Python code that you haven't seen before? Tell me what you thought was missing from the course, even if it was just something small, like you missed explaining this line of code? Were you looking for an algorithm that wasn't included? This is helpful for me, but it's also helpful for you because many times students are looking for an algorithm or an explanation in the wrong course.

So I can say, actually, I teach that algorithm or I explain that thing in such and such course, and this isn't the right course. So have a look over there. Are there any topics you want to request that were not included in this course? For example, maybe you signed up for a deep learning course and you were looking for CNN's, but the course didn't have CNN's in it. Let me know. Finally, let me know your suggestions for future courses. If there's a particular topic you want me to teach that I don't yet have, of course, about, such as gradient boosting or transformers or even quantum mechanics. Let me know. Finally, there is a big text box for any comment you want to write.

You can write anything in this box and make it as long as you want. In fact, the longer the better. If you think I talked to slower, I talked too fast. Let me know if you think my prerequisite instructions aren't fair. Let me know. I want you to be as specific and as detailed as possible. A lot of the time people give me comments that I can't really act on, which is unfortunate. And this is often because they are too generic and not specific enough. Tell me exactly what lecture and exactly what time stamp you're referring to. If you want to refer to a question on the Q&A, include a link. Be very specific and give me concrete examples so that I know exactly how to act on the information you've given me.

Thanks for listening, and I'll see you in the next lecture.

LECTURE 23

Artificial Neural Networks (ANNs)

In this section of the course we are going to introduce the idea of artificial neural networks or ANNs for short.

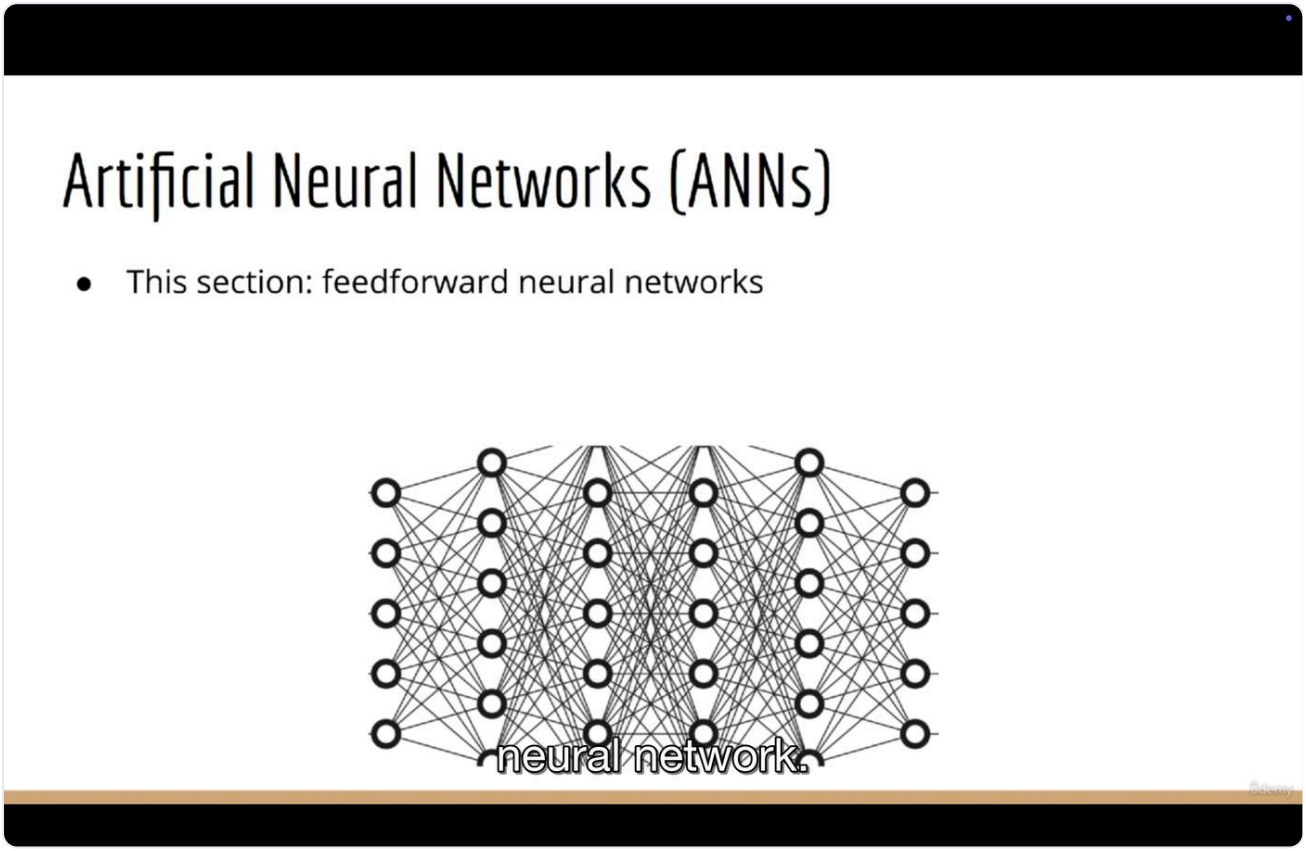


FIGURE 23.1 | Artificial Neural Networks (ANNs)

Specifically we're going to talk about a certain kind of artificial neural network called a feed forward neural network.

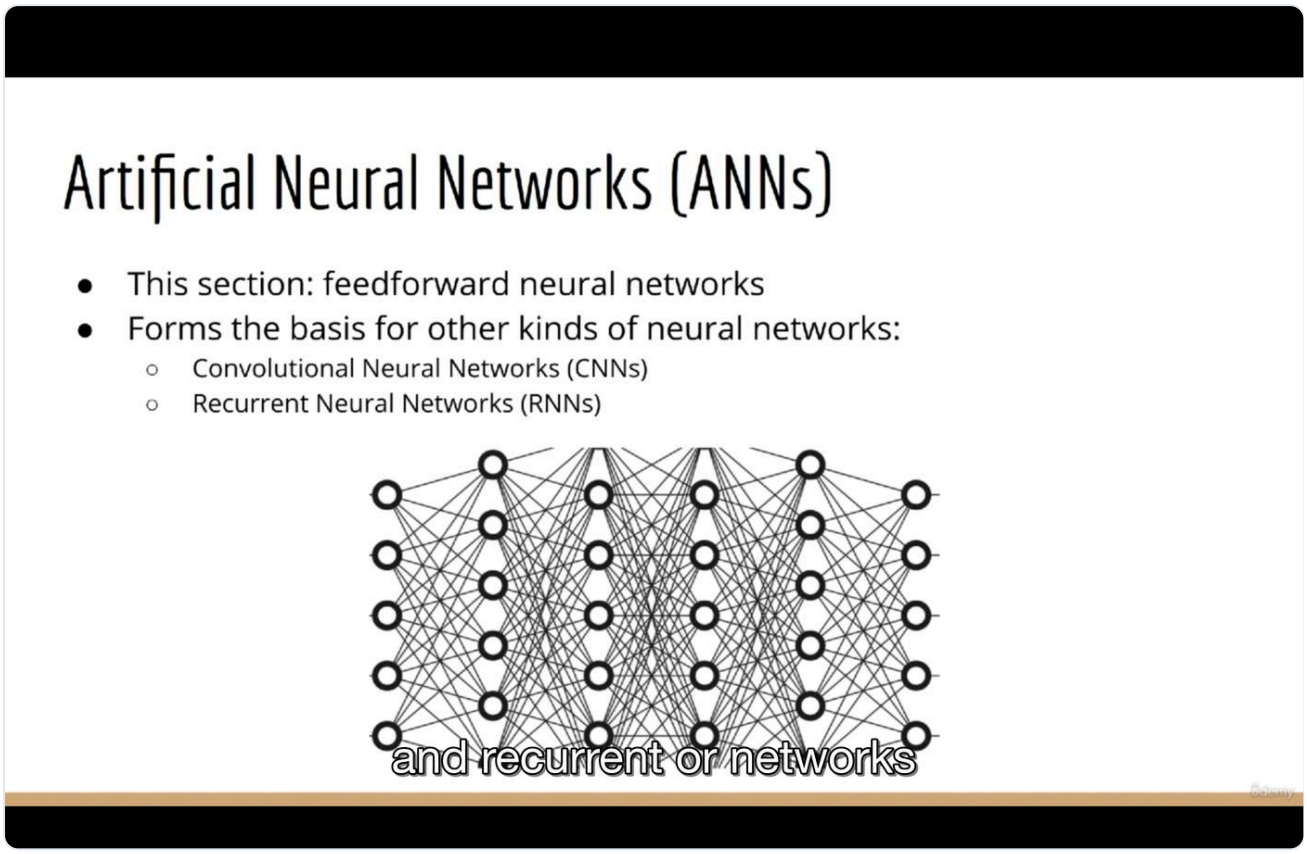


FIGURE 23.2 | Artificial Neural Networks (ANNs)

This is the most basic kind of neural network but as you'll see the concepts involved go quite deep and they also form the basis for other kinds of neural networks such as convolutional neural networks and recurrent neural networks to begin this discussion. Let's start with why and how neural networks came to be in the first place. This is something that a lot of teachers skip over but it's something that I find quite interesting and inspirational.

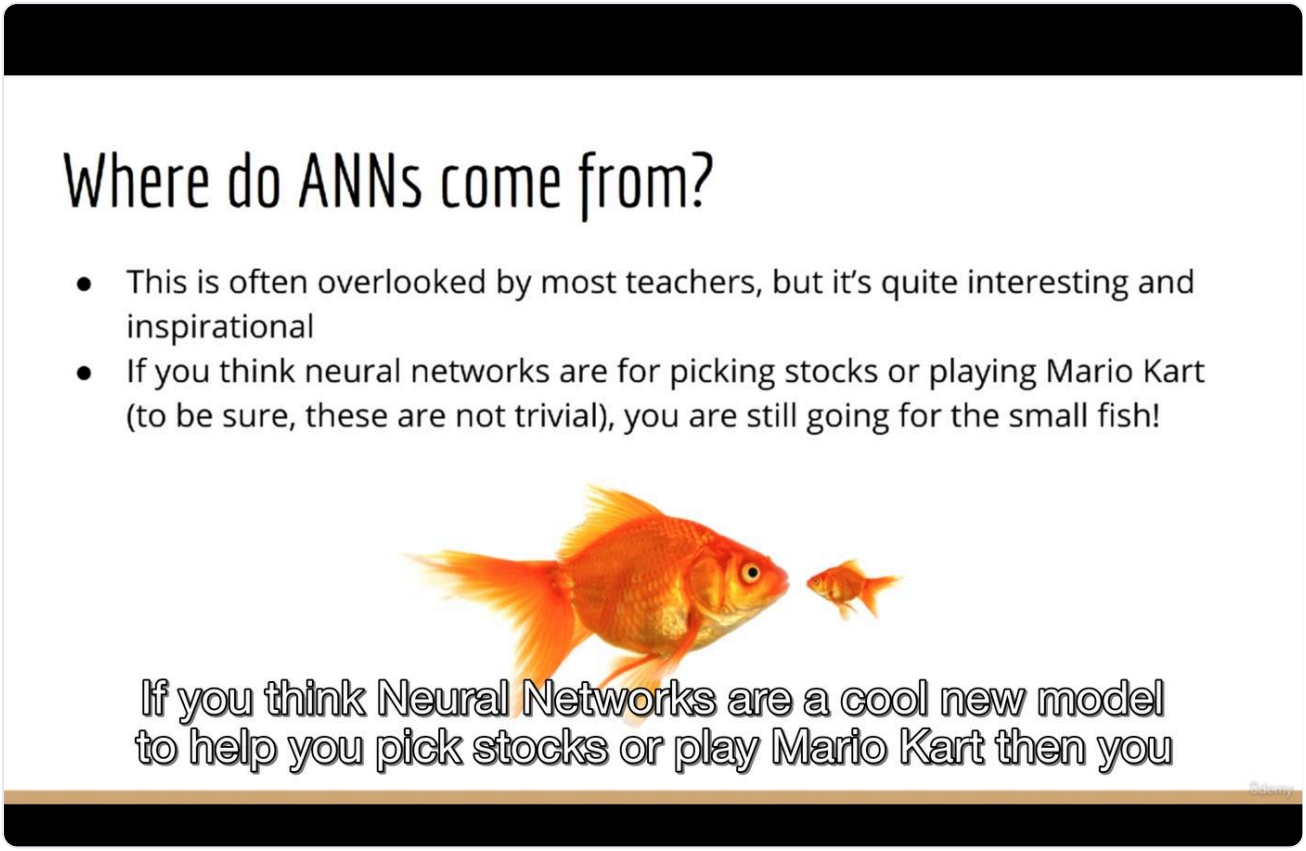


FIGURE 23.3 | Where Do ANNs Come from?

If you think Neural Networks are a cool new model to help you pick stocks or play Mario Kart then you are still looking for the small fish. In fact neural networks are way more interesting than that as you may have realized that the name artificial neural network means that we are trying to artificially create a neural network in a computer.

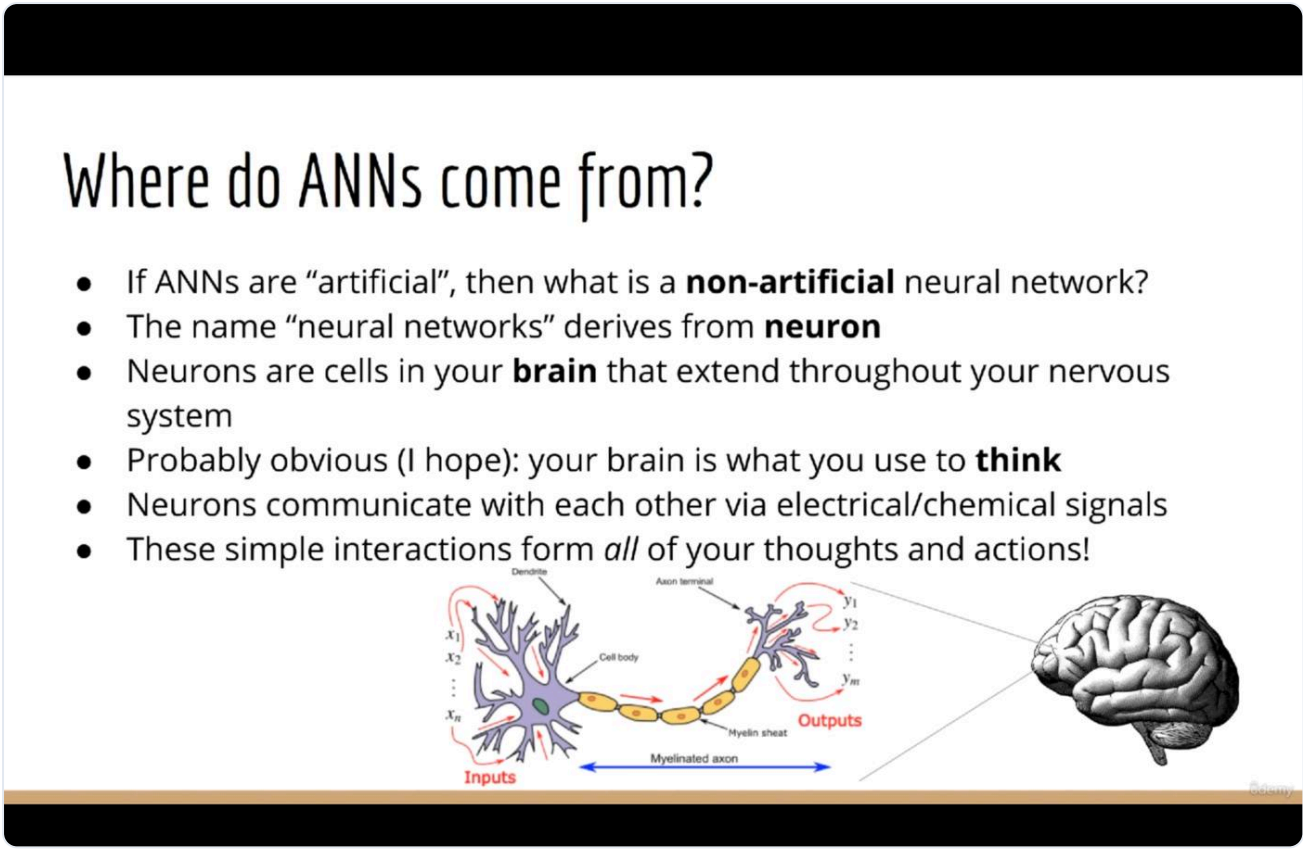


FIGURE 23.4 | Where Do ANNs Come from?

OK so then what's a real neural network as in a non artificial neural network the name neural network derives from neurons which are the cells in your brain and extend throughout your nervous system. Now this is probably too obvious for most of you but let's state it anyway just in case your brain is what you used to think. Neurons in your brain are connected to each other and can communicate with each other via electrical and chemical signals. Amazingly this simple physical and chemical system is what makes you you. All of your thoughts and aspirations your emotions and every action you take throughout the day. It's all driven by your neurons which in turn are just sending electrical and chemical signals around amongst themselves once scientists realize what the brain was doing and what it was responsible for.

The next question was almost obvious in hindsight can we build a brain. I mean if the brain is just a network of neurons and we can simulate neurons in a computer.

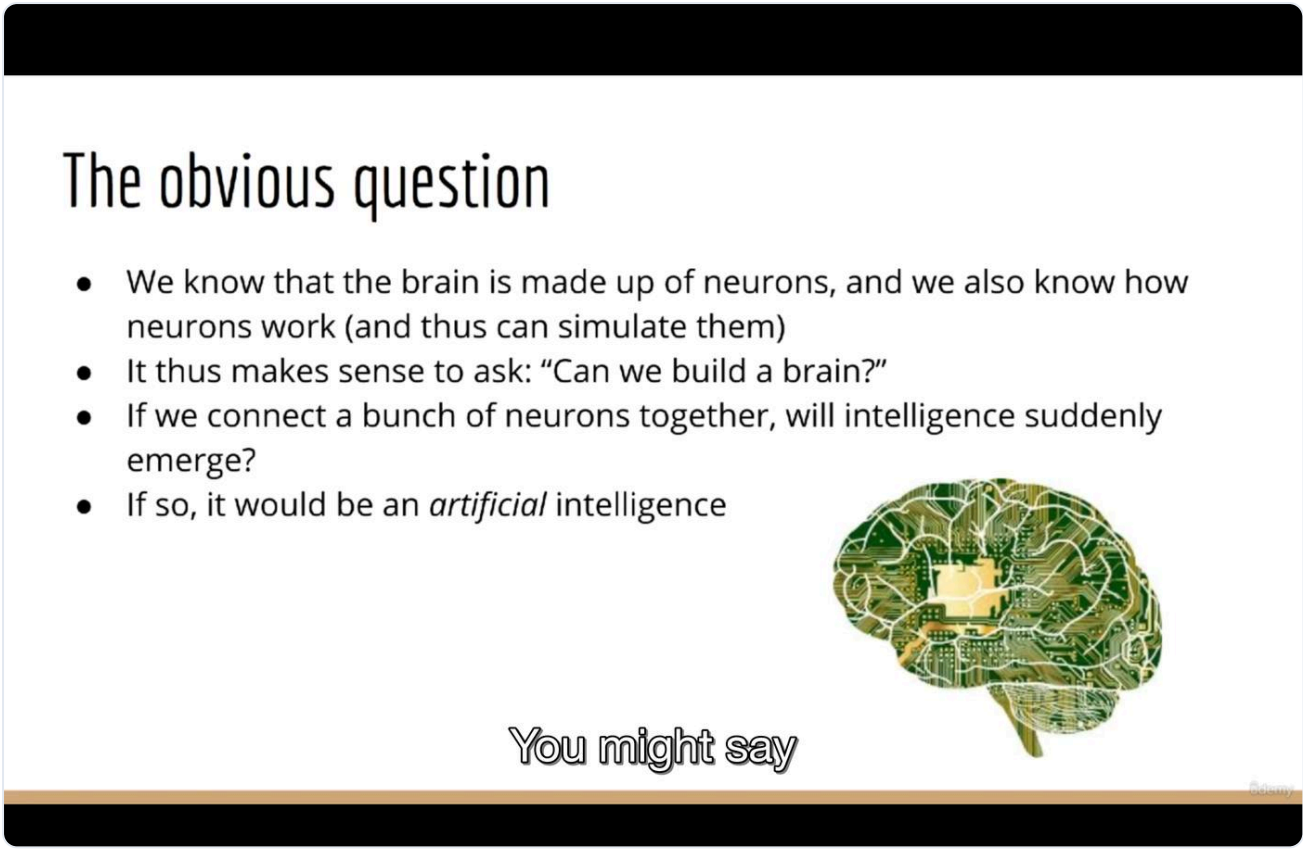


FIGURE 23.5 | The Obvious Question

If we connect a bunch of neurons through a computer simulation it seems that it might be possible to create some form of intelligence and artificial intelligence. You might say so let's take our model of a single neuron logistic regression. Now let's imagine that we have multiple neurons all taking in the same inputs but computing something different now. Let's do it again. Now we have multiple logistic regressions. Now let's imagine that all these neurons are connected to more neurons. So we just repeat the process pretending that the new layer of neurons are actually inputs to more neurons. That's basically a very tiny model of the brain neurons connected to neurons.

Of course this is necessarily simplistic. One side is the input and one side is the output. Of course the actual brain is much more complex. There are many inputs and many outputs in the middle. Wires can crisscross if we have a later neuron connecting back to an earlier neuron. We call that a recurrent connection. The neural networks we are about to discuss in this section contained no such complexities.

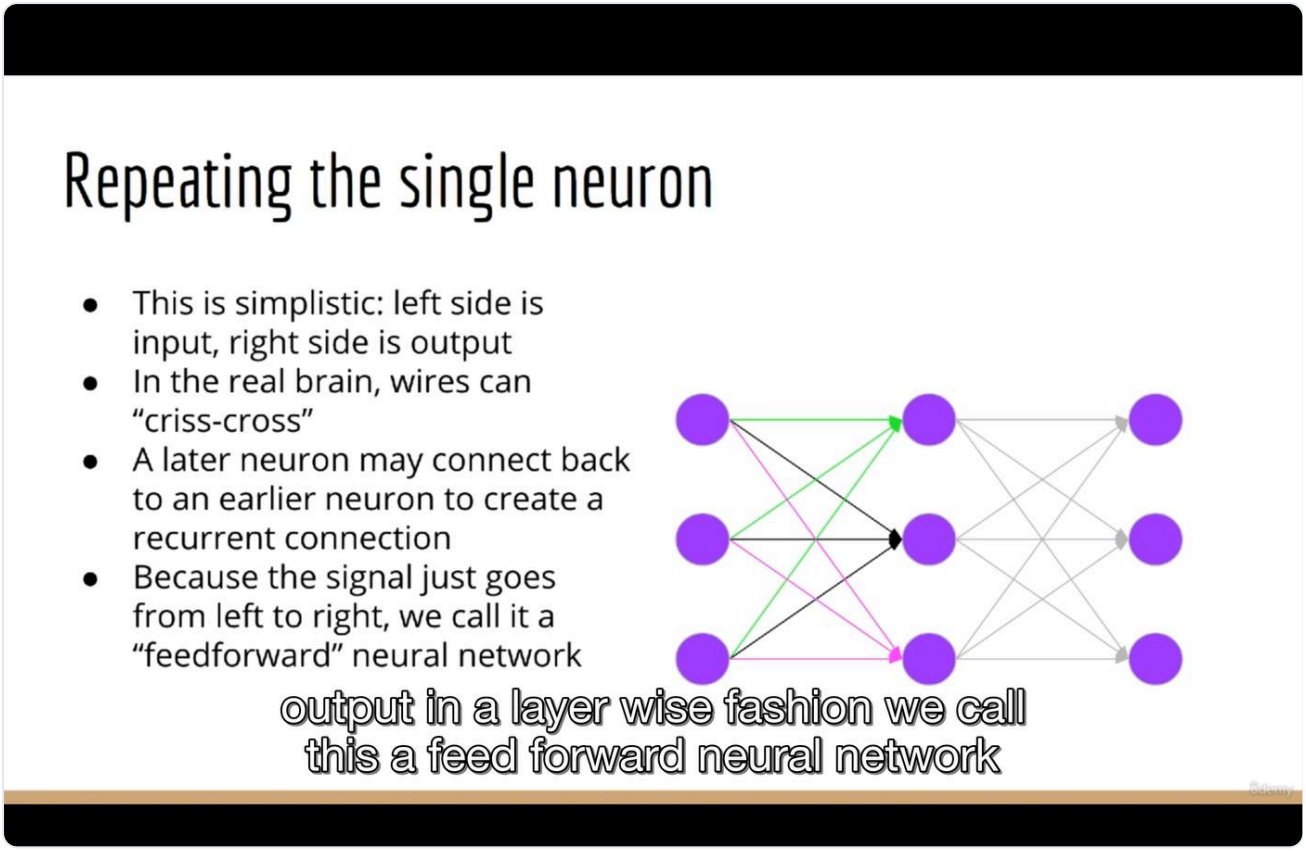


FIGURE 23.6 | Repeating the Single Neuron

Instead because the input is on one side and the output is on the other side and we go from input to output in a layer wise fashion we call this a feed forward neural network in the rest of this lecture. We are going to outline what we will discuss in this section of the course. First we are going to start out again by discussing the model architecture. As you know the model we'll be discussing in this section is the feed forward neural network the next step after discussing the model architecture will be to go back to the geometric picture. If you recall my motto goes machine learning is nothing but a geometry problem. So how do neural networks extend the capabilities of a basic linear model in terms of solving this geometry problem.

Next we're going to go more in-depth and discuss activation functions activation functions are very important in neural networks. They are what make a neural networks more expressive than the simple linear models you saw in the previous section. After that we're going to discuss how to do multi class classification using neural networks. If you recall in the previous section we only discussed the binary classification. This works if we only have two classes dog or cat fraud or no fraud purchase or leave the store and so on. But what if we have more classes for example we might be working on a self-driving car that needs to be able to recognize multiple kinds of objects on the road.

In this case a binary classification is not good enough and we need multi class classification.

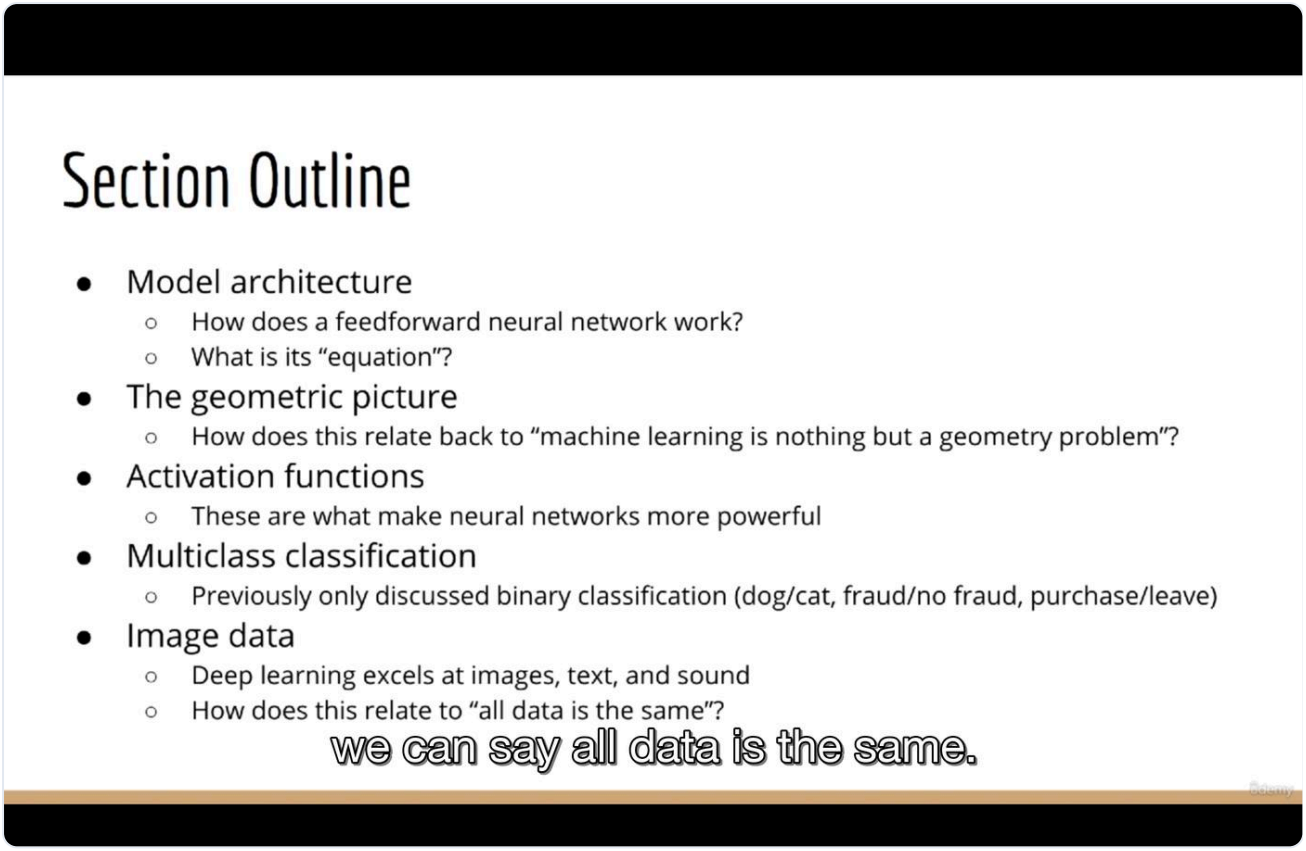


FIGURE 23.7 | Section Outline

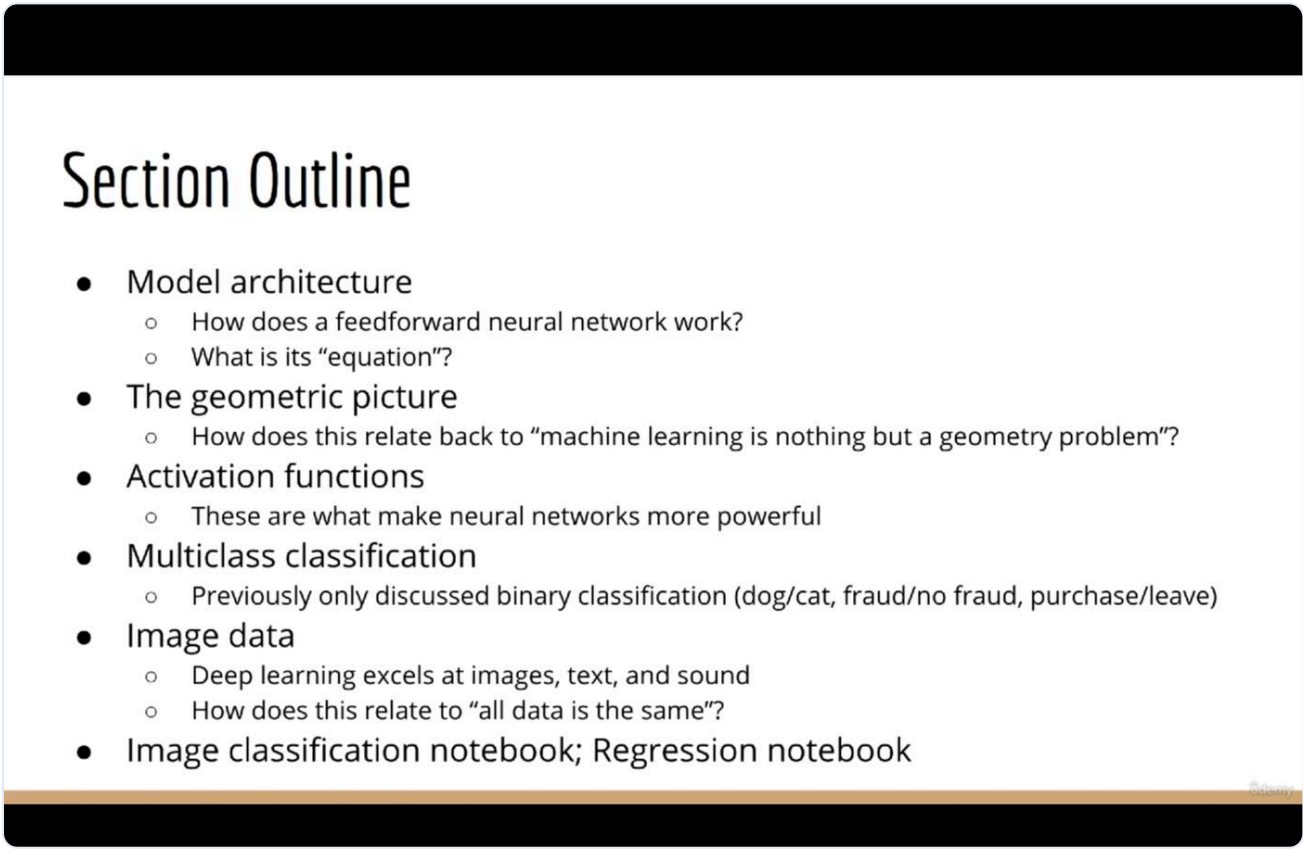


FIGURE 23.8 | Section Outline

Next we'll switch gears from talking about the neural network itself to a new kind of data images Deep Learning excels on unstructured data such as images text and sound. We'll look at how images are represented in a computer so that we can get back to the situation where we can say all data is the same. Finally we'll look at some notebooks that demonstrate how to do image classification using a neural network for multiple classes. We'll also look at neural networks for regression and see how they can be much more expressive than a simple linear model.

LECTURE 24

Repeating the Single Neuron

In this lecture we are going to discuss forward propagation as it relates to neural networks.

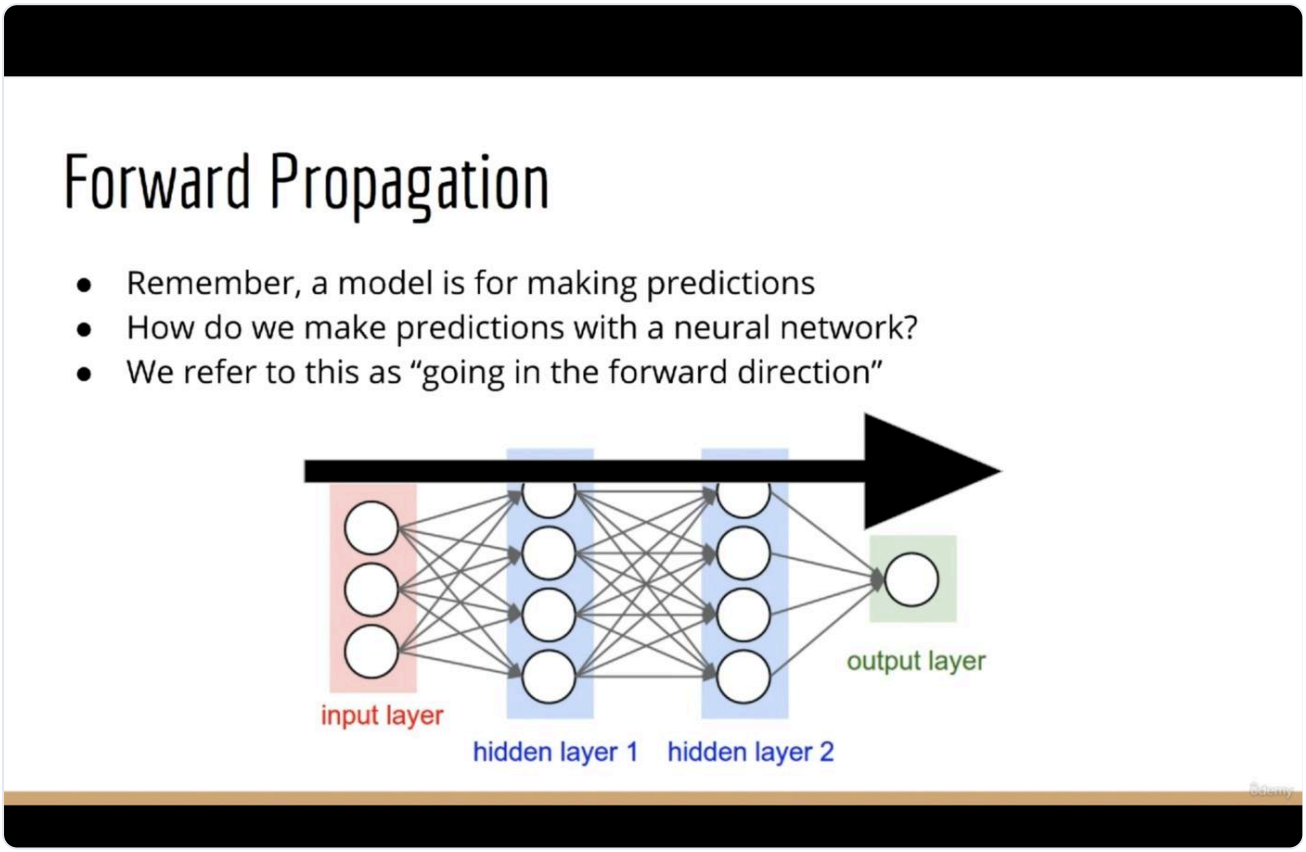


FIGURE 24.1 | Forward Propagation

Neural networks are all about making predictions and so making a prediction in a neural network is what we refer to as going in the forward direction. Previously we looked at logistic regression and how this might be seen as a computer simulation of a neuron. Well let's consider an interesting scenario. Here we have one neuron but let's say these same inputs propagate to another output. Now we have two neurons each of them could be calculating something different given the same inputs.

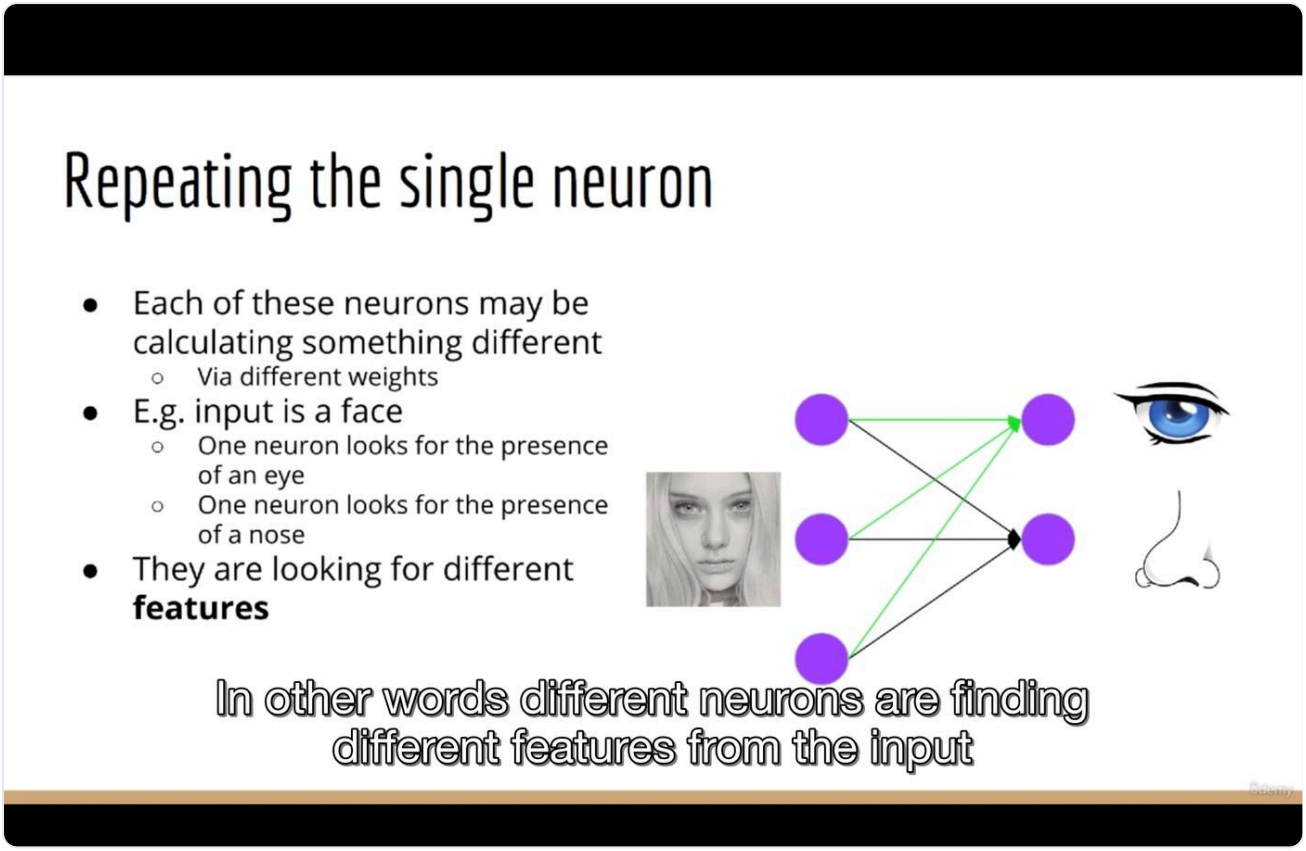


FIGURE 24.2 | Repeating the Single Neuron

If you want a sort of real world analogy of that suppose we're looking at a face one neuron might be looking for the presence of an eye. And another might be looking for the presence of a nose. In other words different neurons are finding different features from the input Well how about we add another neuron.

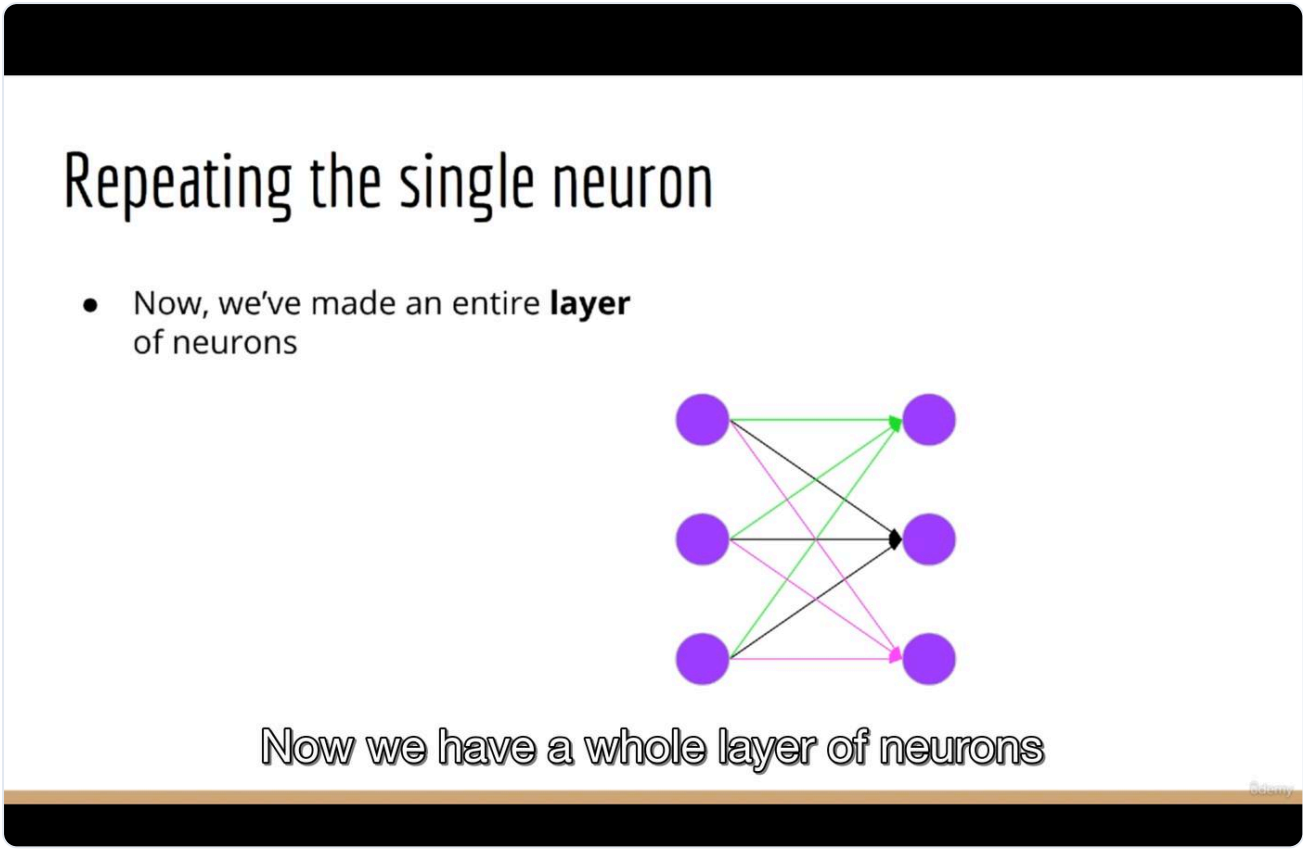


FIGURE 24.3 | Repeating the Single Neuron

Now we have a whole layer of neurons the key insight in deep learning made decades ago by researchers such as Geoffrey Hinton was that we could stack more neurons on top of the existing neurons. Remember that's kind of like how our brain is. We don't just have one neuron in our brain we have many. So one neuron might be connected to another neuron which might then be further connected to another neuron and so forth. So you end up with this long chain of neurons. The key idea is that we can think of neurons as uniform although these days we know a lot about the brain. And so this is not physically the case. Some parts of the brain are different than others but for our simple computational model we can assume the brain is uniform.

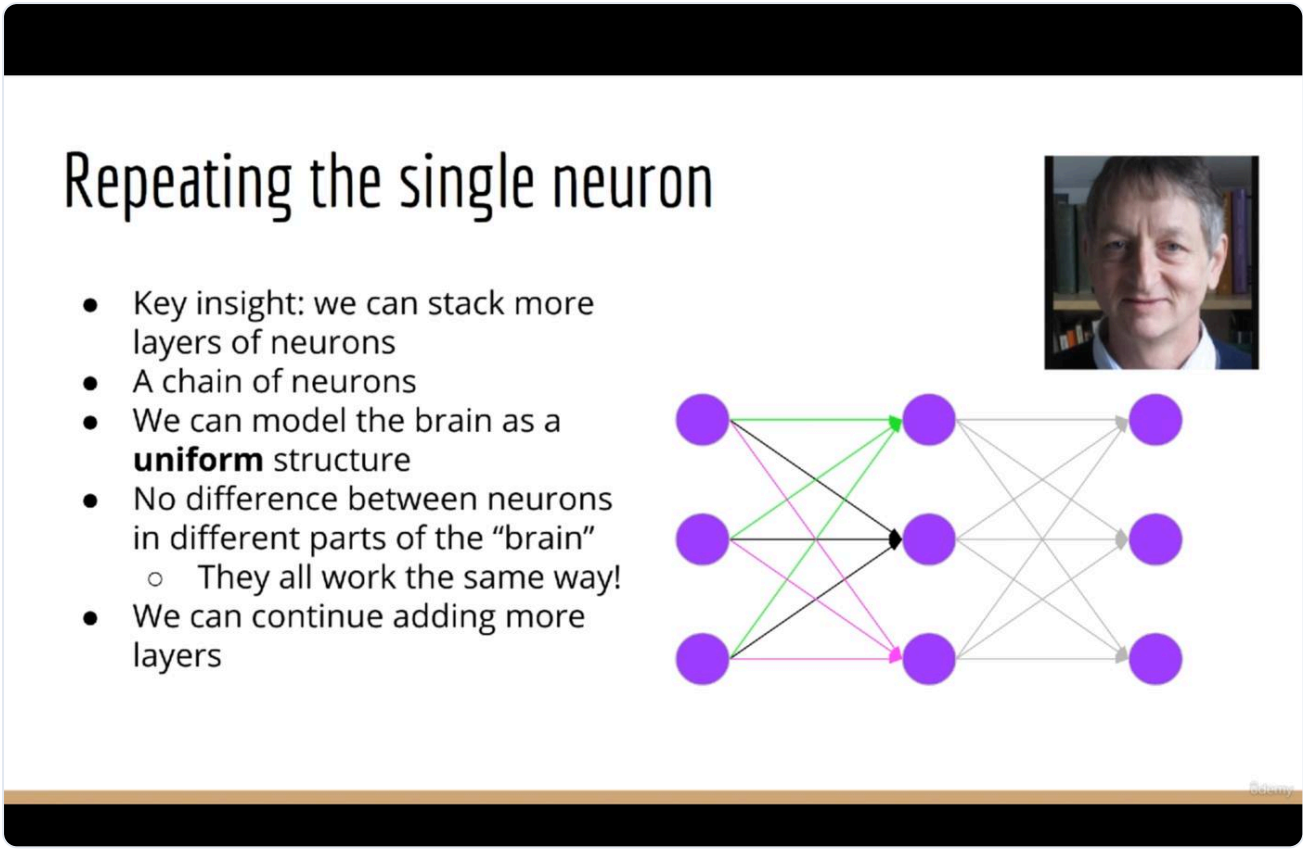


FIGURE 24.4 | Repeating the Single Neuron

There is no difference between a neuron in one part of the brain compared to another part as you might expect. We can just keep doing this. So we have one layer going to another layer going to another layer and so on so what have we done. Well we expanded our concept of neurons in two important ways. First we said that the same inputs could be attached to multiple different neurons each calculating something different. This means more neurons per layer. Second we said that neurons in one layer could act as inputs to another layer of neurons. So the first concept is to make the neural network more wide. The second concept is to make the neural network more deep and believe it or not.

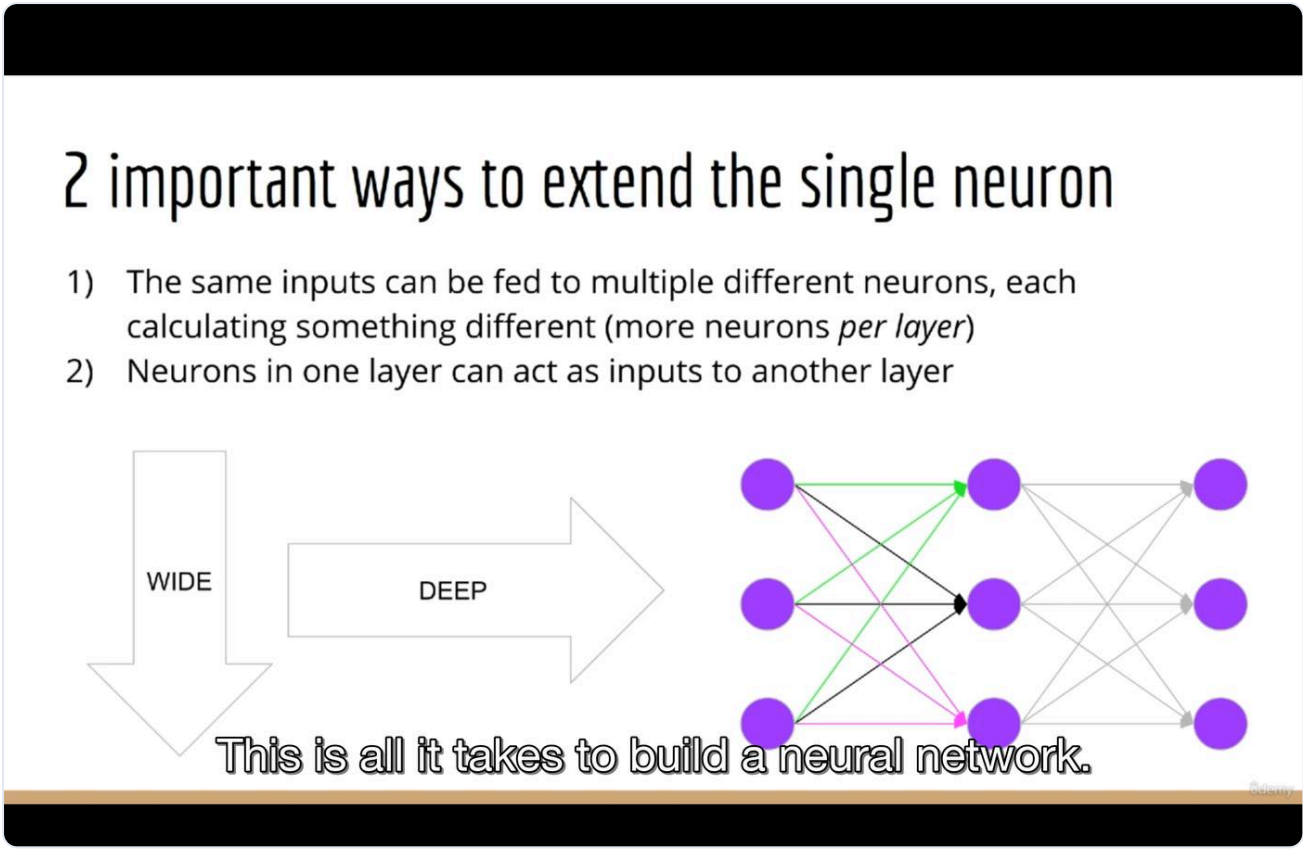


FIGURE 24.5 | 2 Important Ways to Extend the Single Neuron

This is all it takes to build a neural network. Take a neuron add more neurons across the same layer to make it wide and then add more layers to make it deep previously. We looked at the equation for a line and that became our model. So if you recall that was a x plus b we then said in order to make this into a neuron we're going to have multiple inputs and multiple weights.

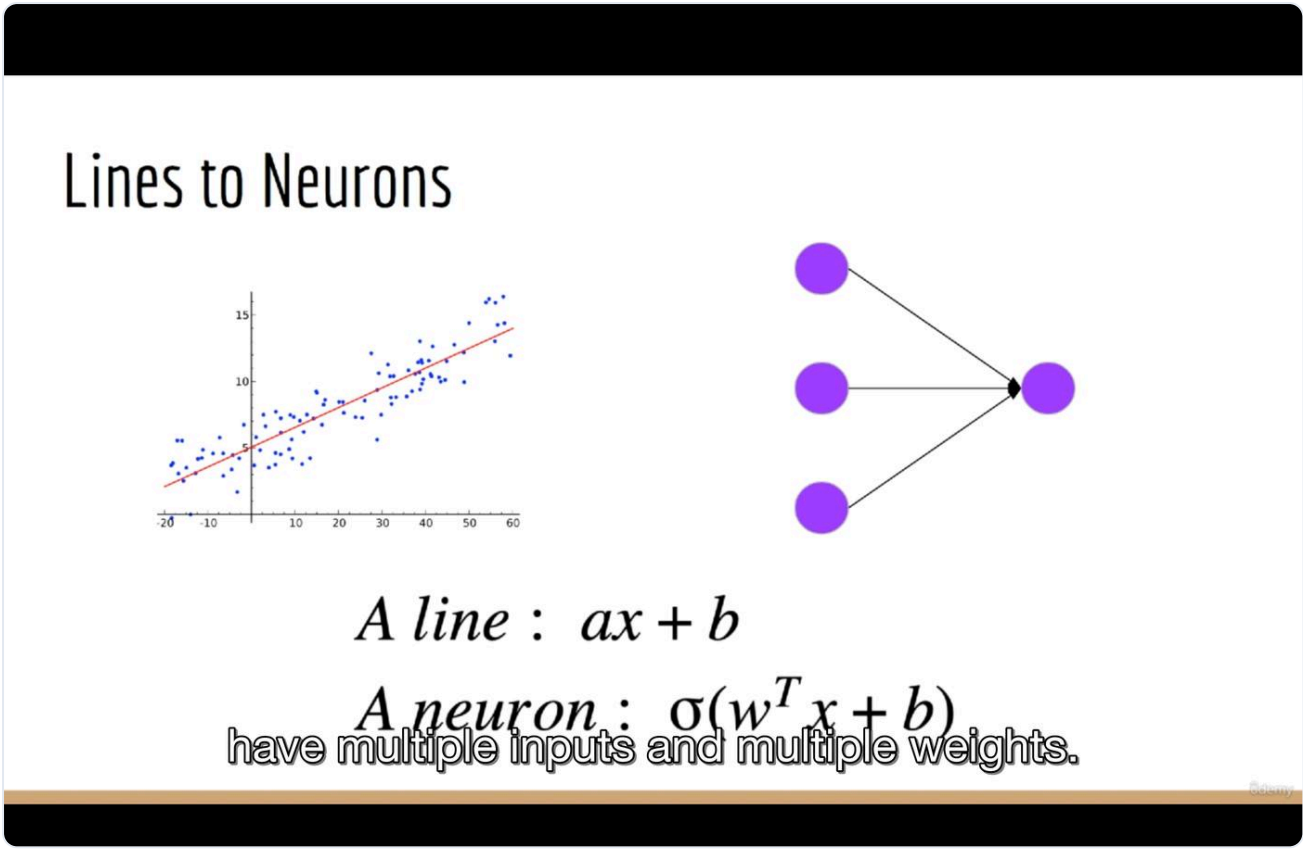


FIGURE 24.6 | Lines to Neurons

We're also going to attach a sigmoid at the end in order to map the output to a value between 0 and 1 representing something like the probability of an action potential. That became the sigmoid of W transpose x plus B the next question to consider is what should we do if we have multiple neurons per layer Well suppose I have some output at the next layer let's call that z sub j to represent the jth neuron in that layer. Then z sub j would just be the sigmoid of w sub j transpose x plus b sub j. Let's say that J goes from 1 to M so that we have m neurons in this layer.

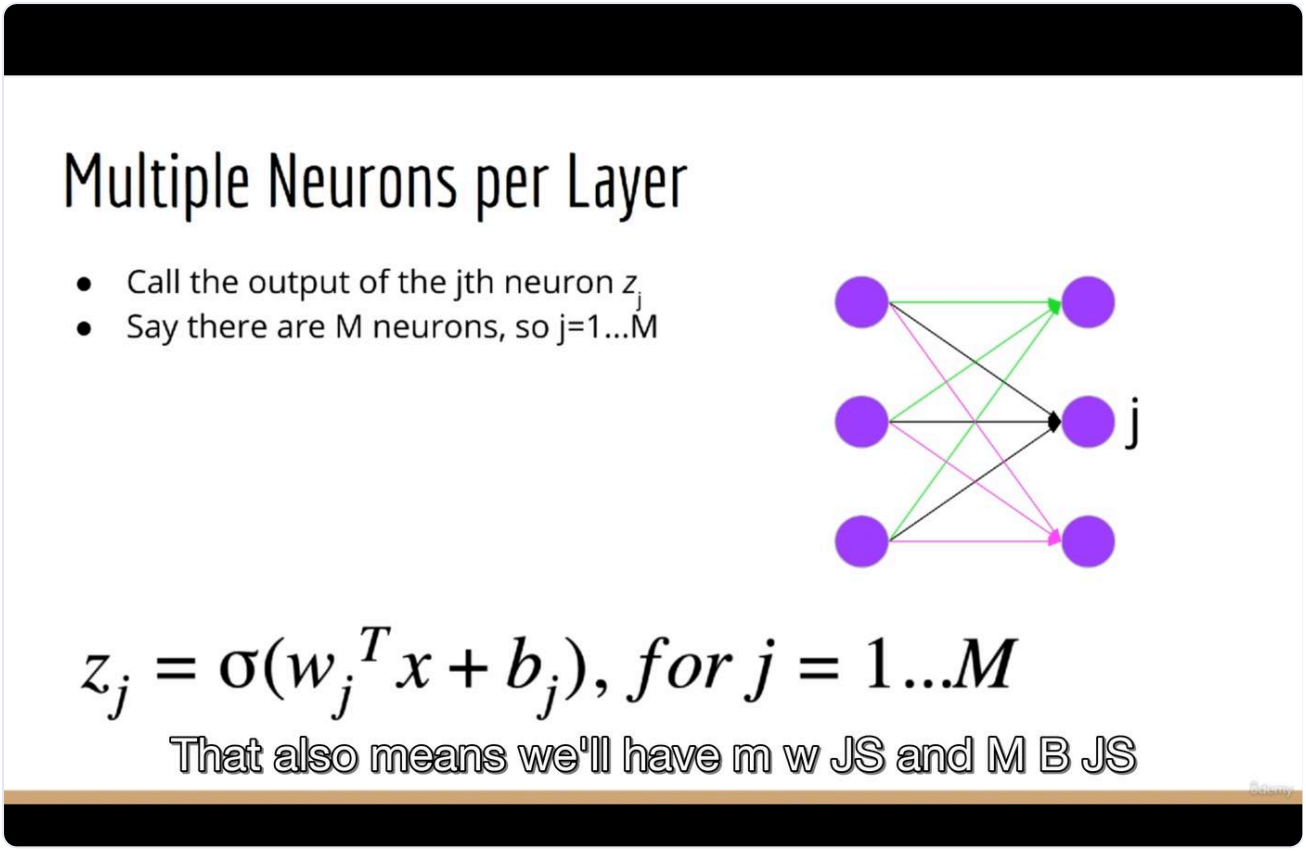


FIGURE 24.7 | Multiple Neurons per Layer

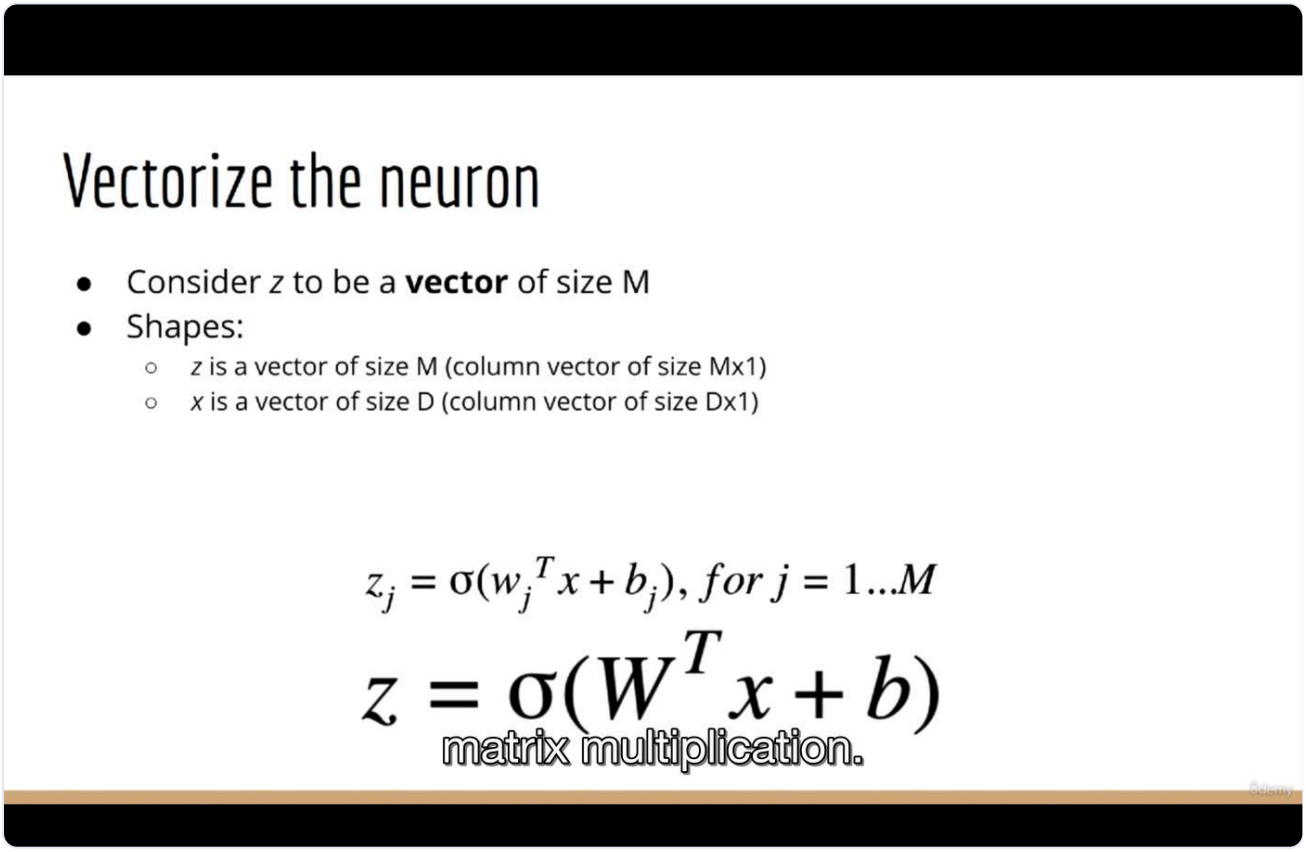


FIGURE 24.8

That also means we'll have m w j's and m b j's It turns out that we can make this calculation more efficient if we consider Z to be a vector of neuron values instead of just a scalar then we can write this more compactly. Now you might find this a little confusing if you're not used to working with matrices and functions of matrices. So we have to clarify a few things. First Z is a vector of size m x is as before a vector of size D or equivalently you can think of Z as a column vector of size m by 1 and x to be a column vector of size D by 1 in order to make this a valid matrix multiplication. We have the following. W is a matrix of size D by m. So when you transpose it you get m by D and then when you multiply that by X which is of size d you get out a vector of size m in order to add the bias term B.

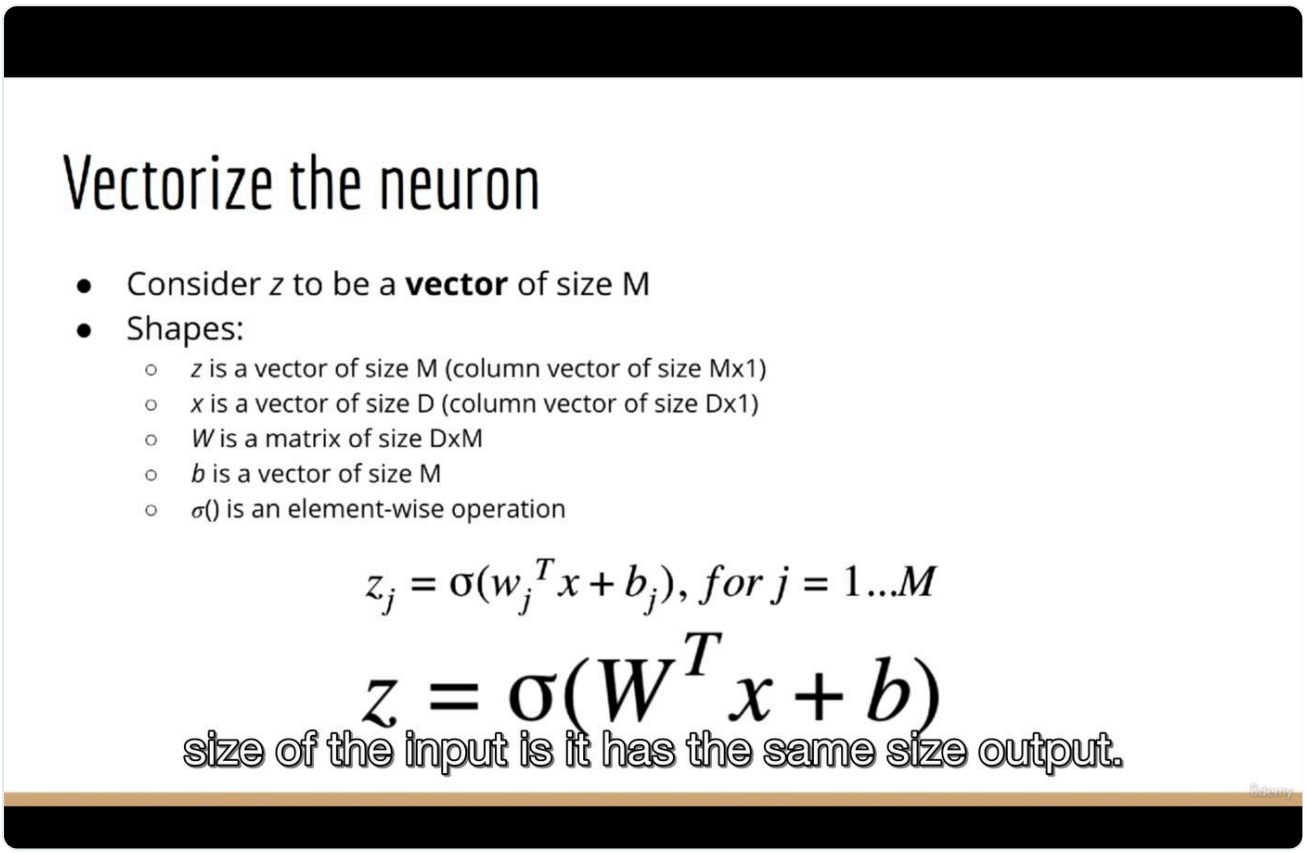


FIGURE 24.9

It is also a vector of size m the sigmoid here is meant to be an element wise operation so whatever the size of the input is it has the same size output. In other words it would be as if you apply the sigmoid to each component of its argument individually and then organize them into a vector or a matrix of the same shape using these concepts let's think about how we might mathematically Express going from the input to the output of a neural network. First let's consider that we are going to have multiple layers. So it's not enough to call our weights just w and B instead of we're going to give them indices. So let's say w superscript 1 and b superscript 1 are the weight and bias of the first layer w superscript 2 and b superscript 2 other weight and bias of the second layer and so forth. z superscript 1 will be the output of the first layer z superscript 2 will be the output of the second layer and so forth.

Now there are two things to notice about the final layer here. First you'll notice that I've used the capital letter L to denote the total number of layers the letter L is common and deep learning.

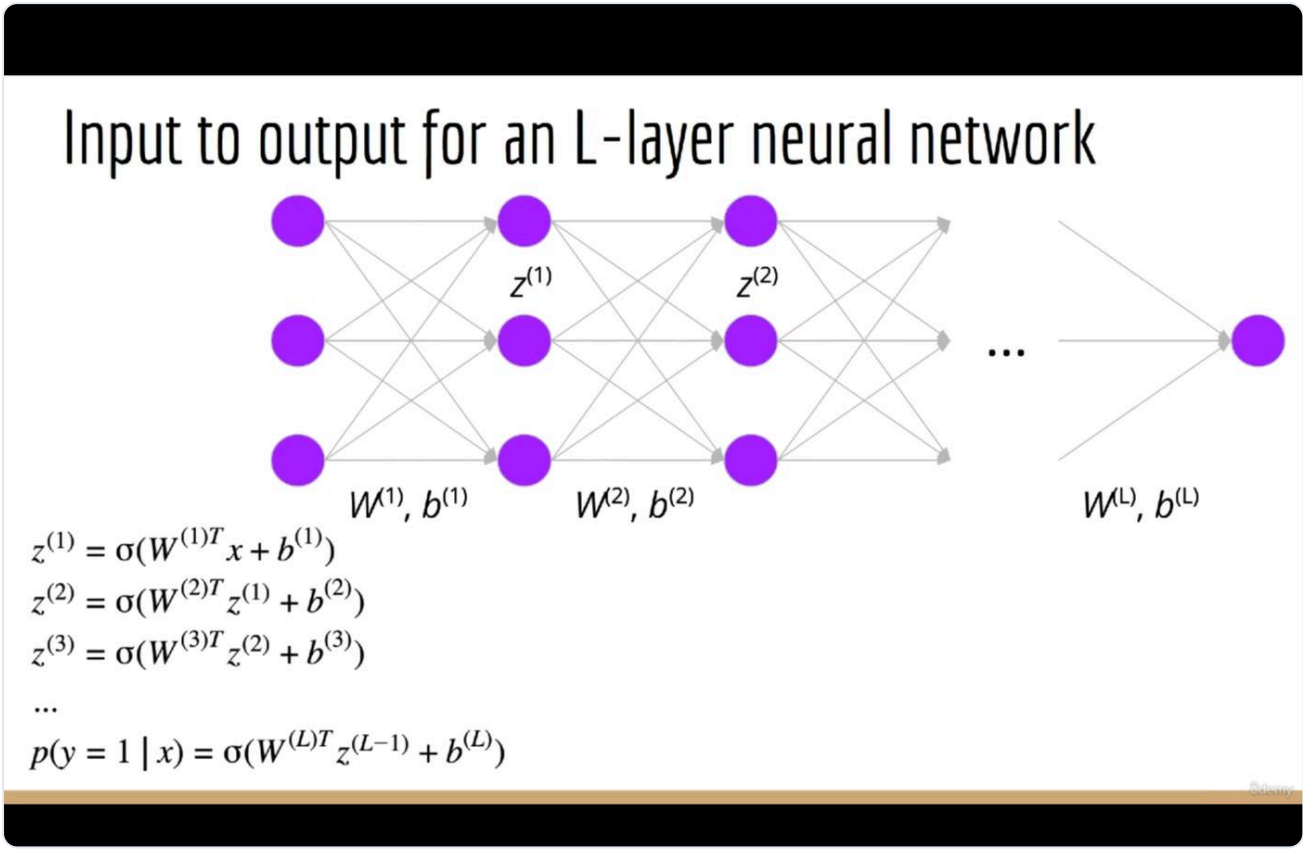


FIGURE 24.10 | Input to Output for an L-layer Neural Network

So if you see the letter L elsewhere don't be surprised. Just pay attention to the context in this context we mean the number of layers and of course the input to the last layer will be the output of the second last layer. So we have this nice repeating pattern at each layer each layer is just the same equation over and over again. So that's what we mean when we say that a neural network is uniform. The second thing to notice is that for now we'll assume that we are still doing binary classification. Therefore the final sigmoid maps the output of the neural network to be a single number between 0 and 1. You can imagine that if we are doing regression we would not want to have the sigmoid there so of course that this brings us to the next question.

What does a neural network for regression look like.

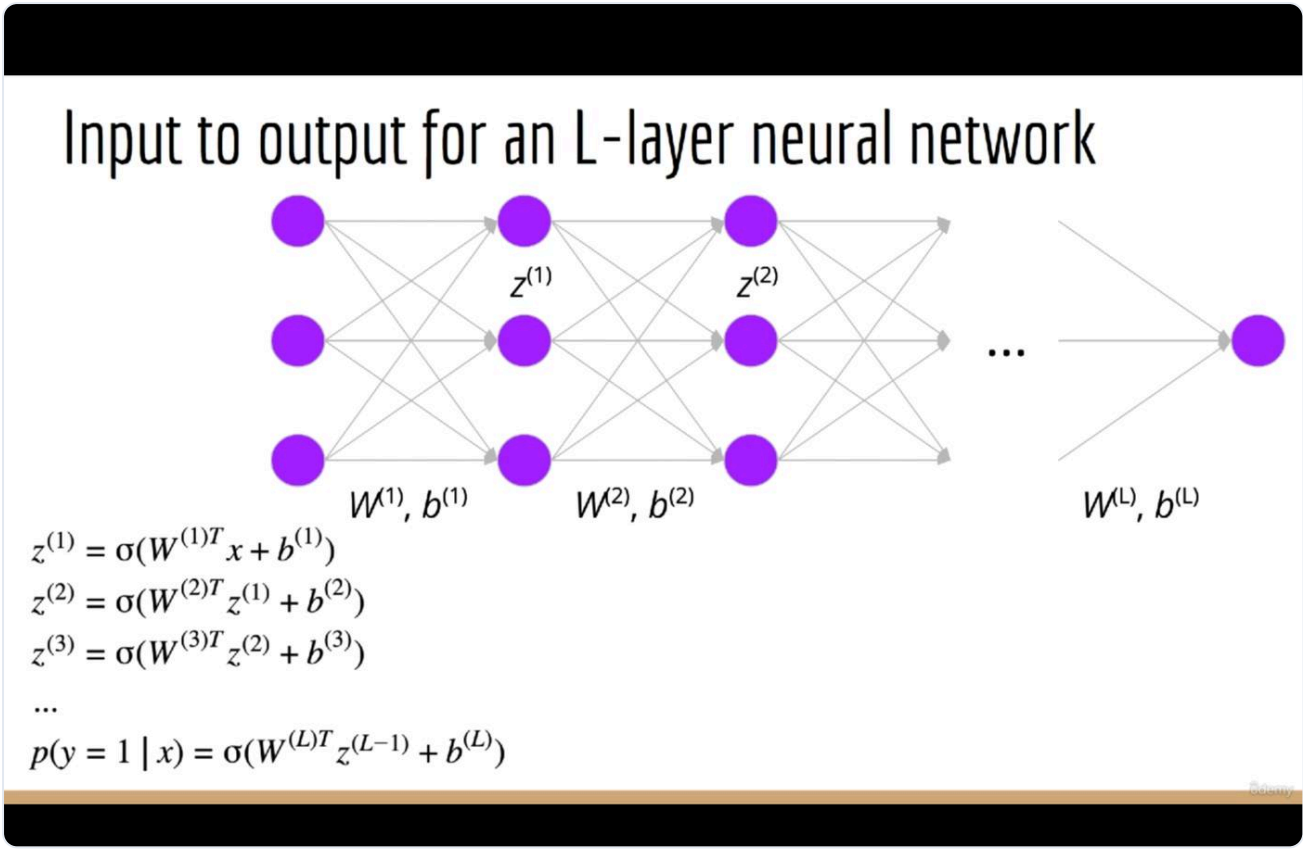


FIGURE 24.11 | Input to Output for an L-layer Neural Network

Surely we are not limited to just binary classification and luckily it's very similar. All we do is simply remove the final sigmoid. One interesting thing we can see from this if we look at only the final layer here is that it appears just to be regular old linear regression. And that brings us to our next major point. This is a very helpful perspective when you're thinking about neural networks. So I think it's important to learn about early on. If you think about what we just looked at neural networks for binary classification and neural networks for regression they are not that different from what we looked at previously.

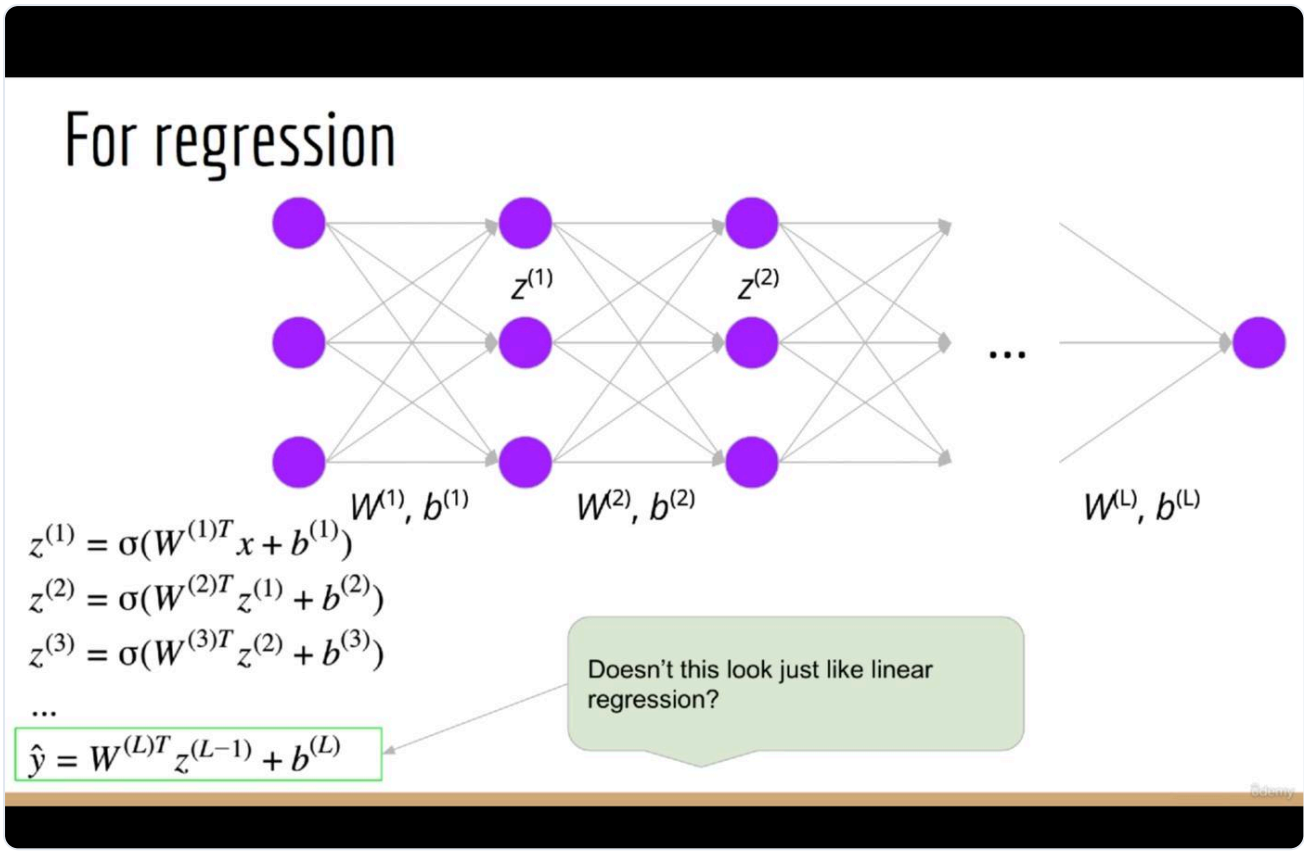


FIGURE 24.12 | For Regression

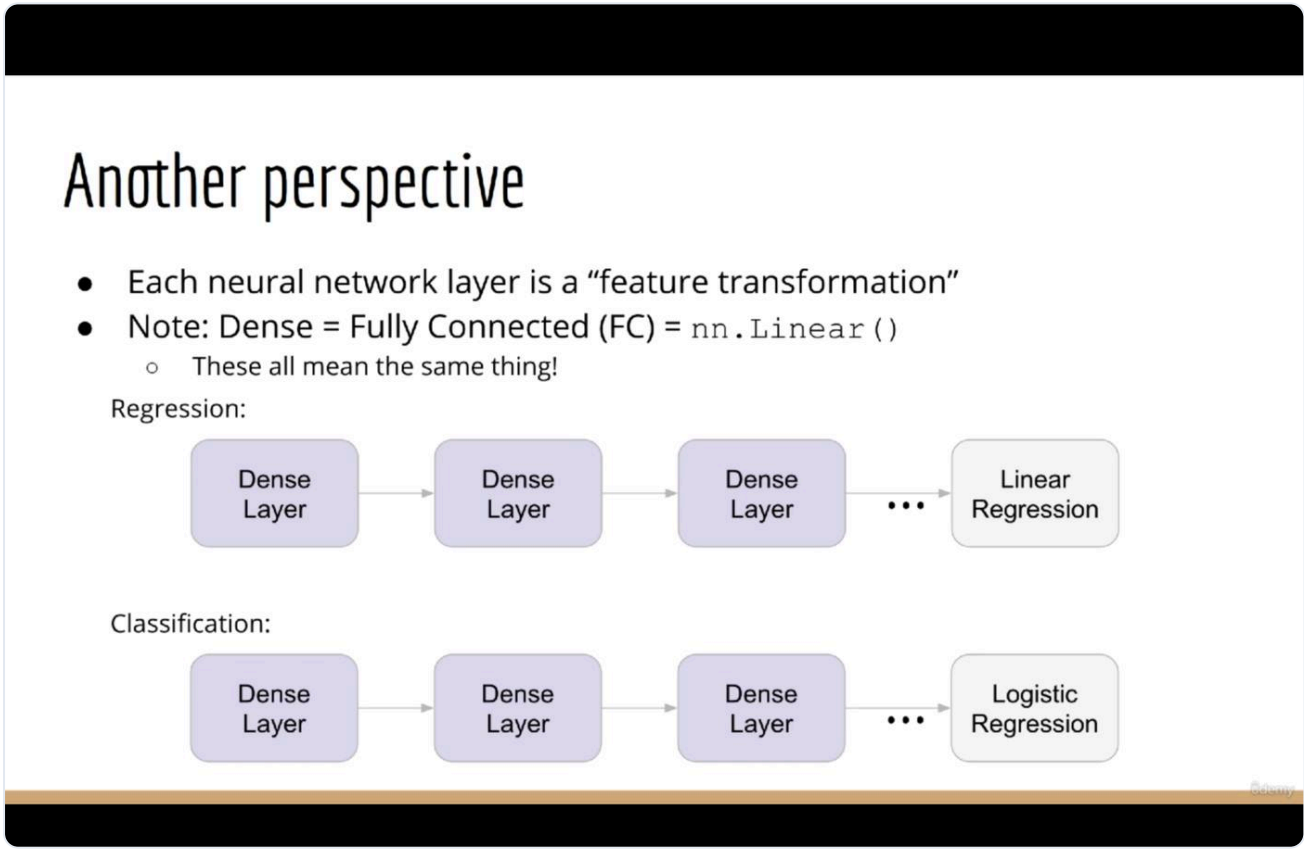


FIGURE 24.13 | Another Perspective

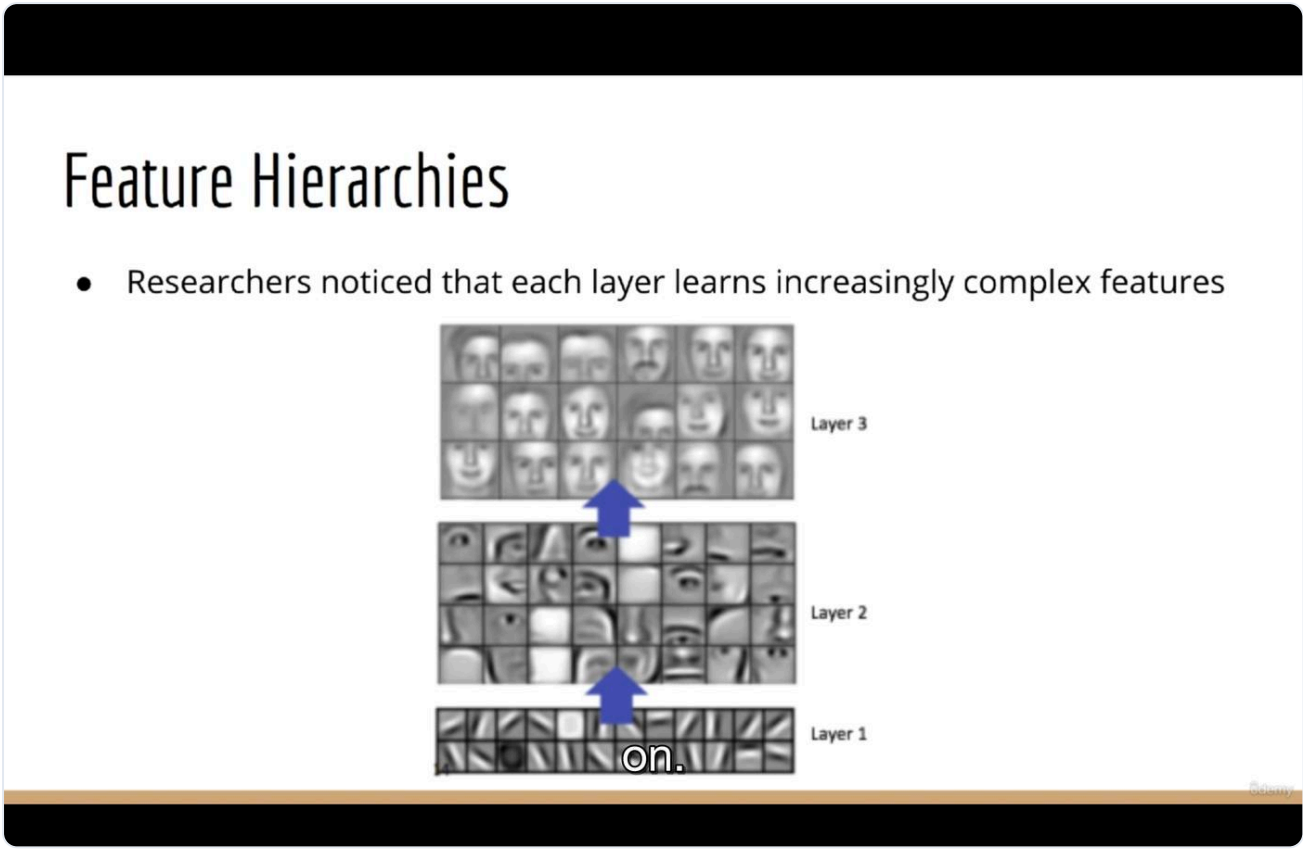


FIGURE 24.14 | Feature Hierarchies

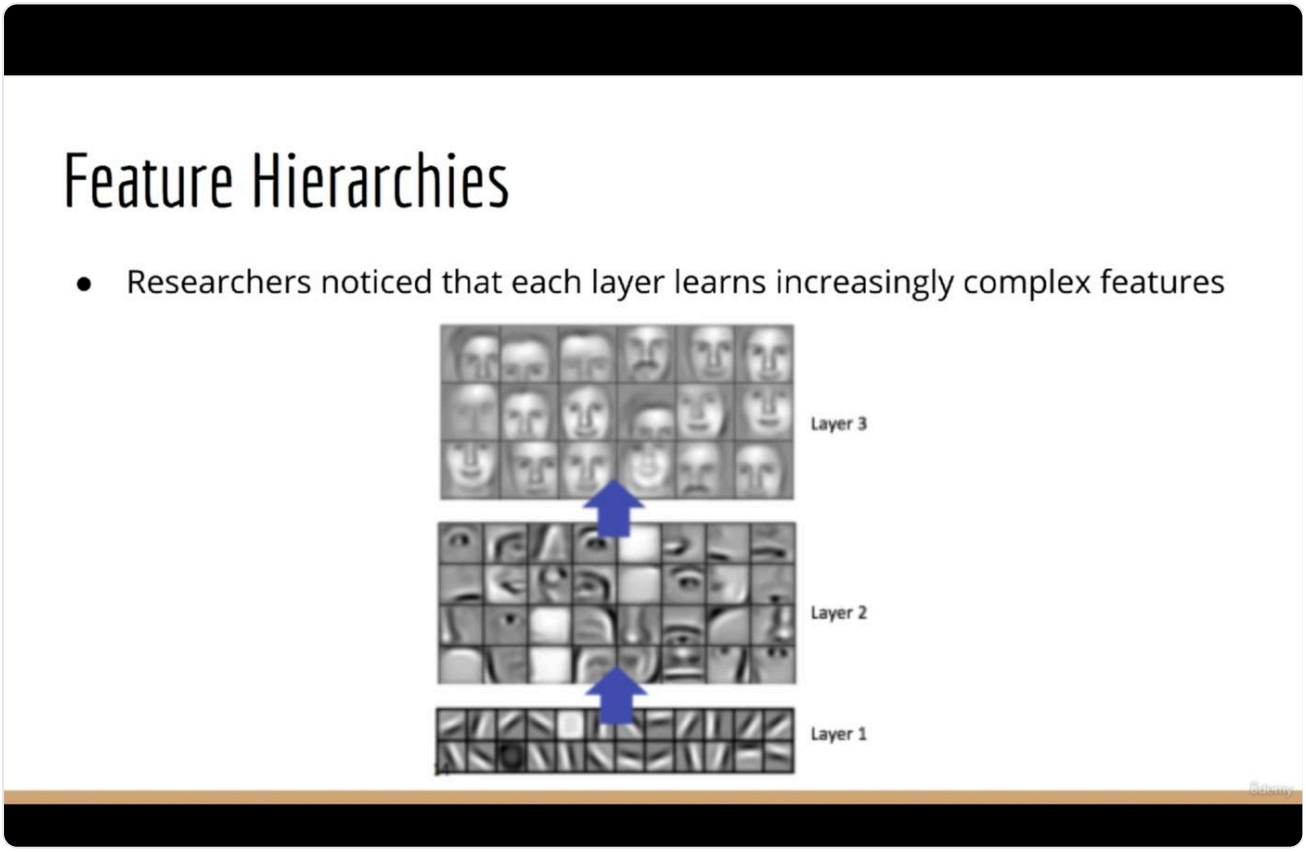


FIGURE 24.15 | Feature Hierarchies

If we look at just the final layers they look just like regular old linear regression and logistic regression. That's what we studied in the previous section. So here's a new way to think about neural networks instead of merely a network of layers of neurons. Really we're just doing a series of feature transformations so we go from feature to feature to feature. And then finally we just have a plain linear regression for regression or logistic regression for classification using these uniform layers. Researchers noticed that neural networks were able to learn hierarchies of features each layer seemed to be learning something more complex than the last.

And this was the motivation for making neural networks deep. And that's why this field is called deep learning one of my favorite examples of this is with facial recognition networks by visualizing what each layer has learned. Researchers found that the initial layers just learned some basic lines and strokes and in the middle the neural network started to learn about specific features of faces such as eyes nose mouth and so on. Finally at the final layers the neural networks started to see entire faces. In other words neural networks are able to break down a problem into smaller sub problems or in other words a hierarchy.

LECTURE 25

Making the Line More Complicated

https://playground.tensorflow.org/

At this point you're probably sick of me saying machine learning is nothing but a geometry problem. But here is where it gets really important. The question we want to answer right now is why are neural networks so important why can't we just use a single neuron. I mean it seems to be a pretty good model.

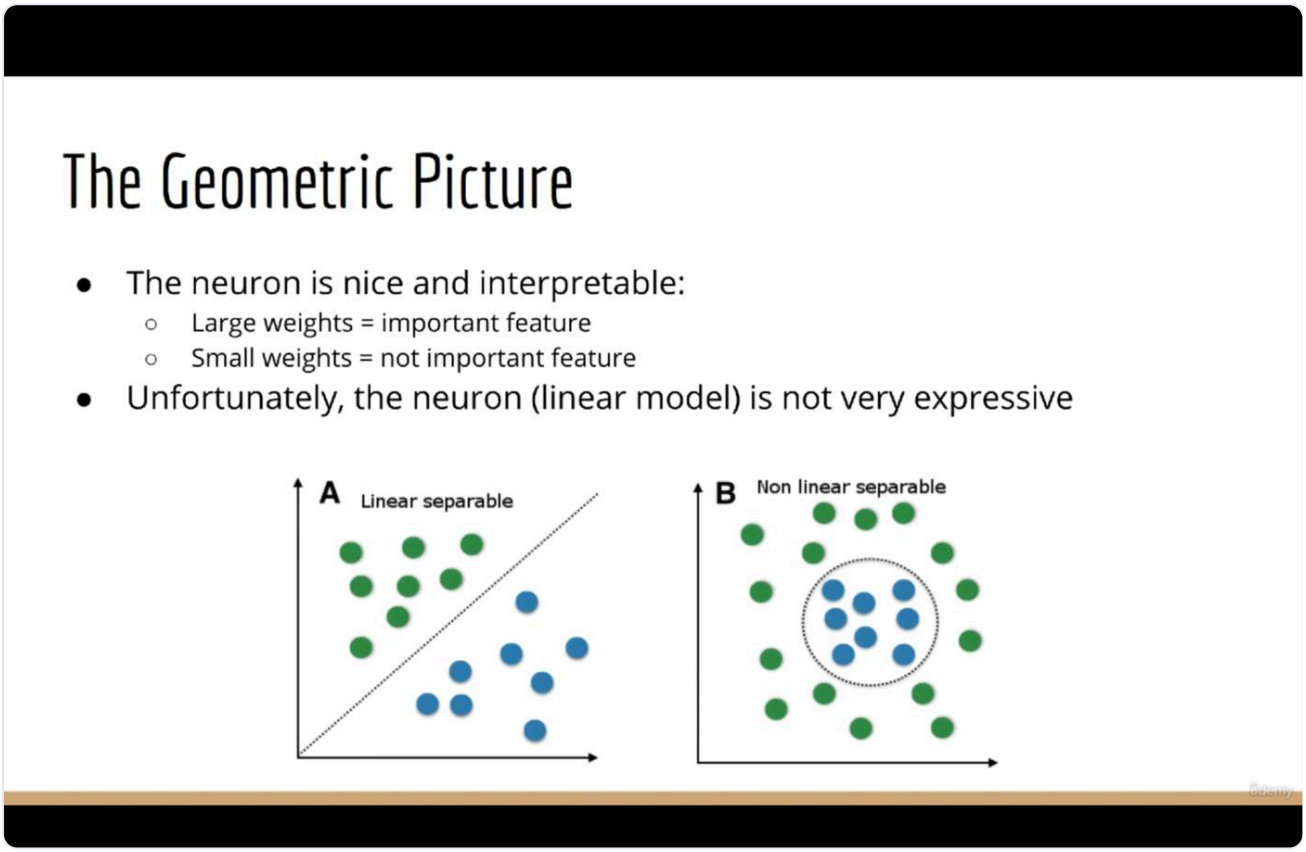


FIGURE 25.1

It's nice and interpretable. Large weights means the corresponding input is important and a very small or zero weights mean the input is not important. Unfortunately this type of model only gets us so far you'll recall that when we discuss linear regression and logistic regression I said there are two different ways we can make things more complicated. The first way we can make things more complicated is to add multiple inputs. That's something we encountered already.

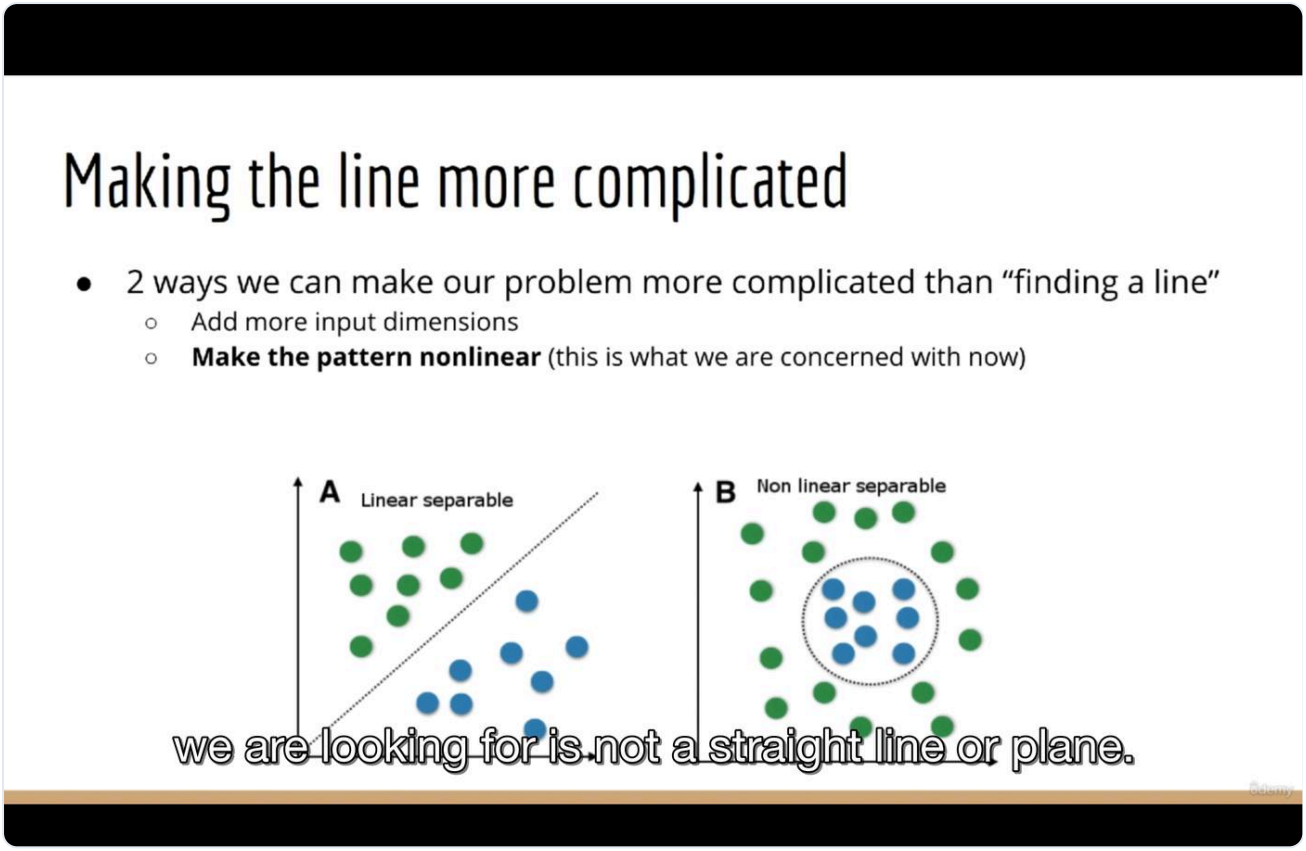


FIGURE 25.2 | Making the Line More Complicated

This is still pretty simplistic because you can always picture a plane in your head a straight non curving surface that separates data between classes. The second way we can make things more complicated is that the decision boundary or regression function we are looking for is not a straight line or plane. This is what we are interested in when we discuss neural networks. The reason the equation w transpose x plus B equals zero gives us a hyperplane is because this is the actual definition of a hyperplane.

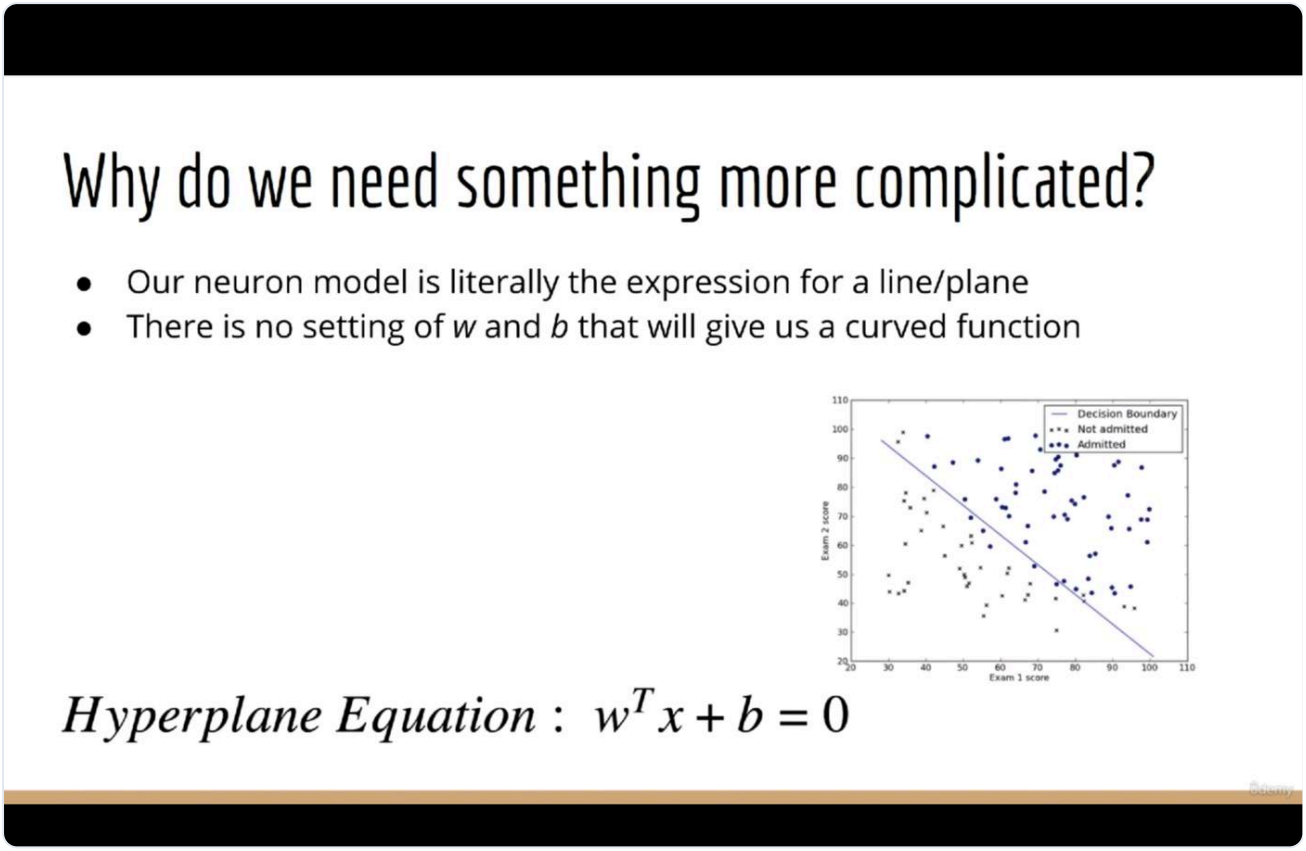


FIGURE 25.3 | Why Do We Need Something More Complicated?

There is no way around this. There's no possible setting of W and B that could give you a curved surface. So how can we get a curved surface so that we can solve more complicated problems one method you might have thought of is to use feature engineering. For example let's take linear regression. Suppose that for whatever reason salary is proportional to the square of the years of experience.

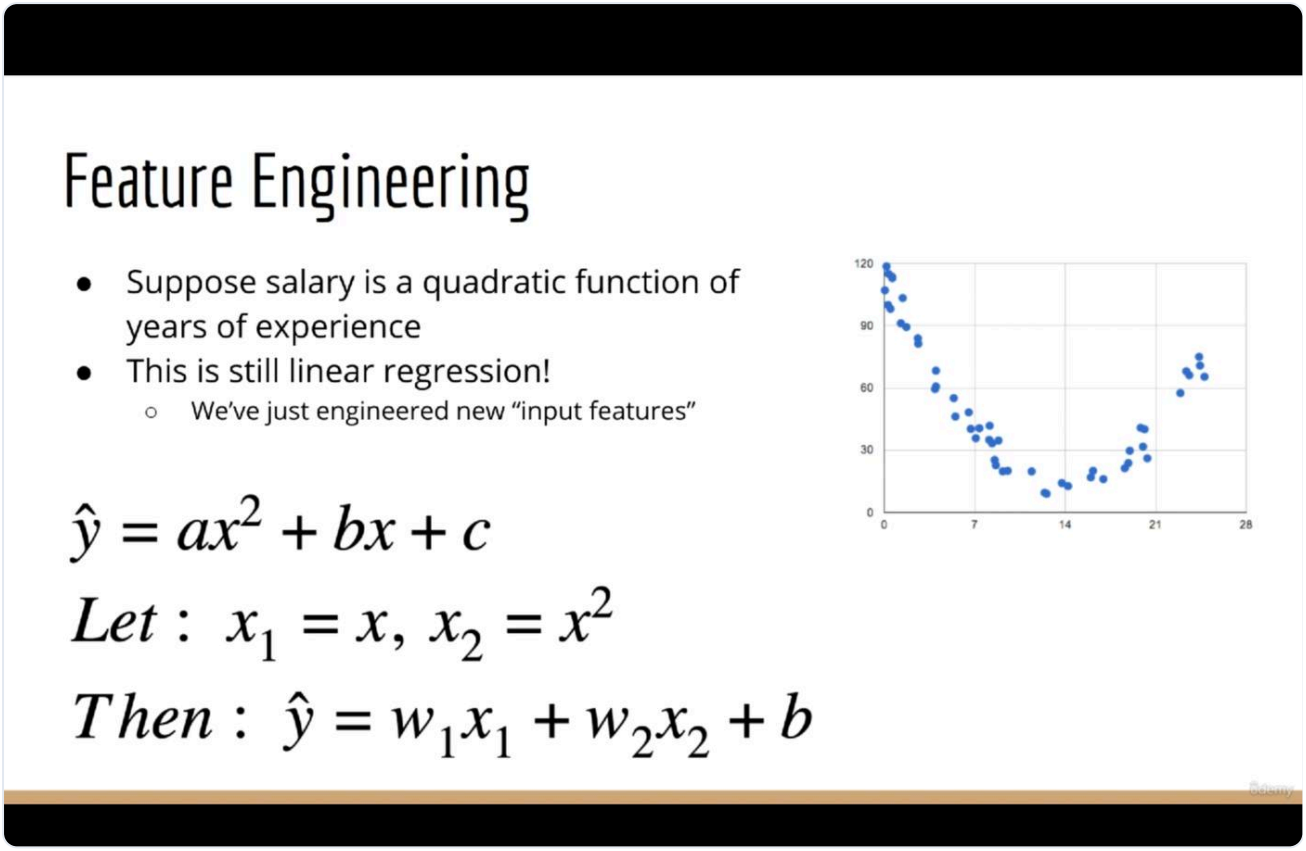


FIGURE 25.4 | Feature Engineering

Or put another way that salary is a quadratic function of years of experience. Then we could say that y hat equals X squared plus b x plus C Believe it or not. This is still just linear regression in disguise. It's easy to see this by doing the following. Take X the years of experience and call that the input feature x 1 then take X squared. The years of experience squared and call that the input feature x 2. Then my y hat becomes an equation of the form y hat equals w 1 x 1 plus W2 x 2 plus B. Of course we've already learned that this is just a linear regression the problem with feature engineering is that there are many different possible features squaring the inputs is common but so is combining the inputs.

So if I have the inputs X1 and X2 than I could make one of the features x1 times x2 We call these interaction terms but what happens if I have 3 input features.

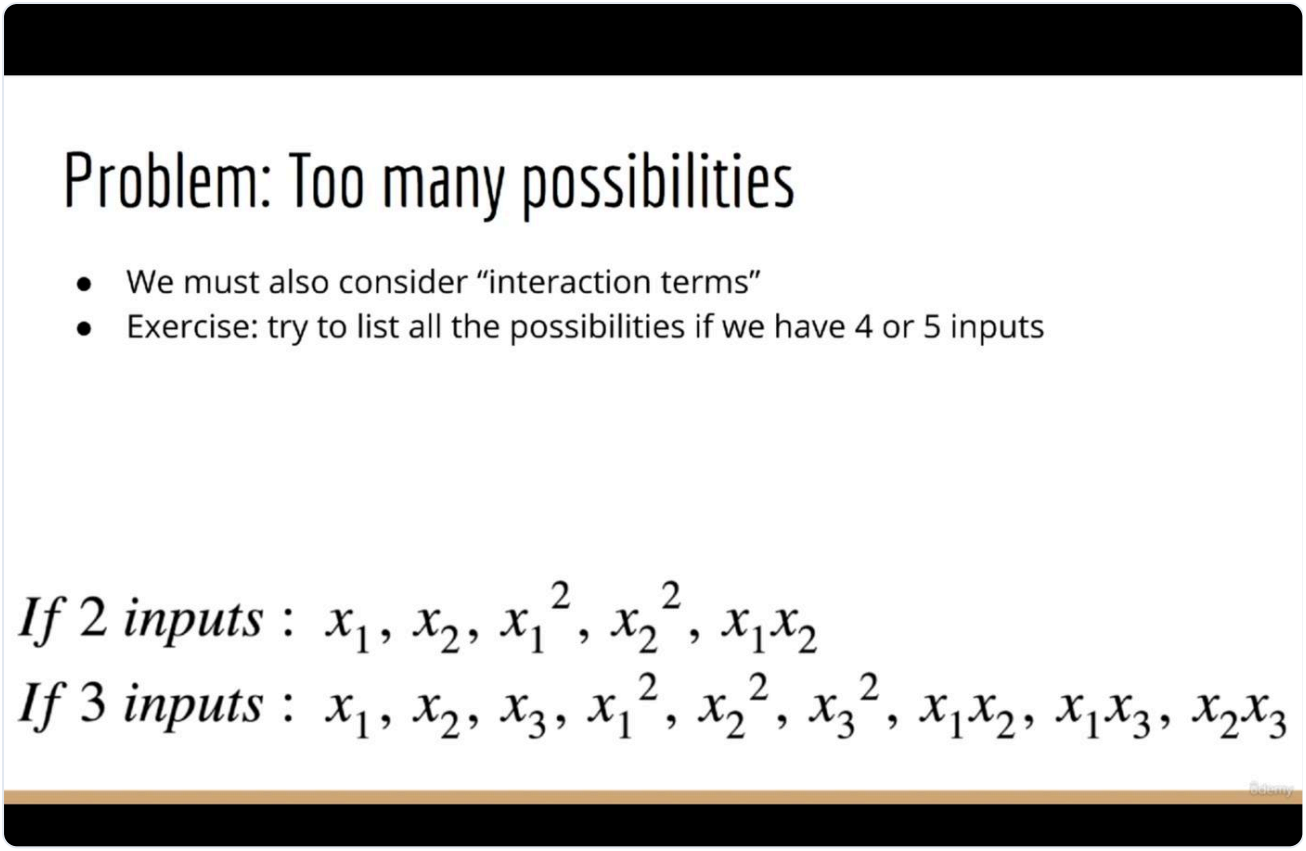


FIGURE 25.5

Now I have to consider X1 squared X 2 squared and x 3 squared. But we also have to consider x1 x 2 x1 x 3 and x 2 x 3 in total. There are 6 quadratic terms as an exercise you might want to list out all the possible combinations we could have if we had 4 inputs or 5 inputs. You should discover that the number of terms we have to consider grows quite fast and thus feature engineering. Even in this simple form seems to be a somewhat clumsy solution. But now let's remember our neuron. If we consider just the first layer of a neural network we can see that there are multiple neurons. Importantly they are all multiple different neurons. Each neuron is a different non-linear feature derived from all the inputs because we've applied the sigmoid activation function.

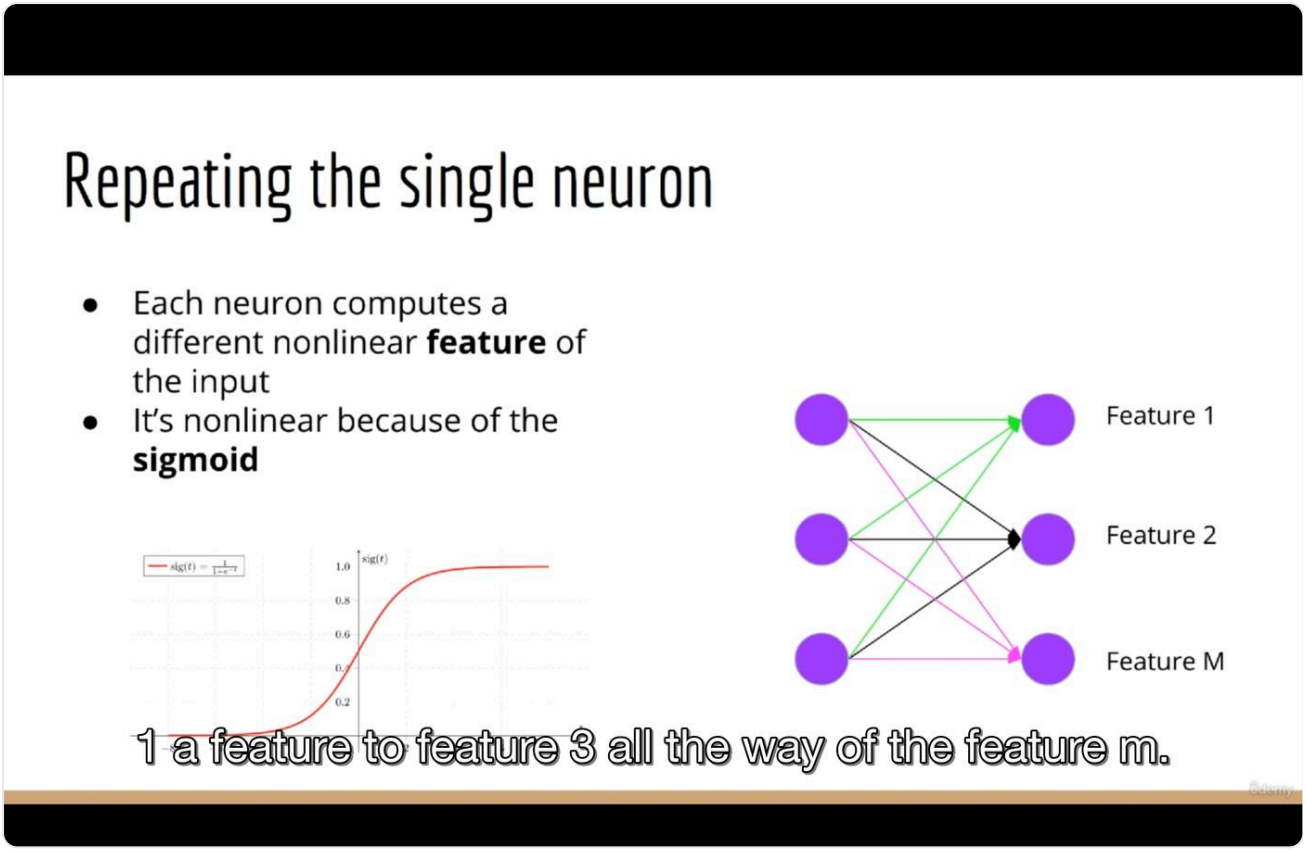


FIGURE 25.6 | Repeating the Single Neuron

The feature is not just a simple linear combination of input features so you can think of these as feature 1 a feature to feature 3 all the way of the feature m. Now a lot of people wonder how does this make a neural network nonlinear. Remember that a linear function always takes the form w transpose x plus B a neural network takes the form of the equation you see here for a two layer neural network this is what we get if we drop the Z term and just make the whole thing into one equation W2 transpose sigmoid of w 1 transpose x plus B1 and then plus B2. The important thing to notice about this is you can not simplify this into a linear function.

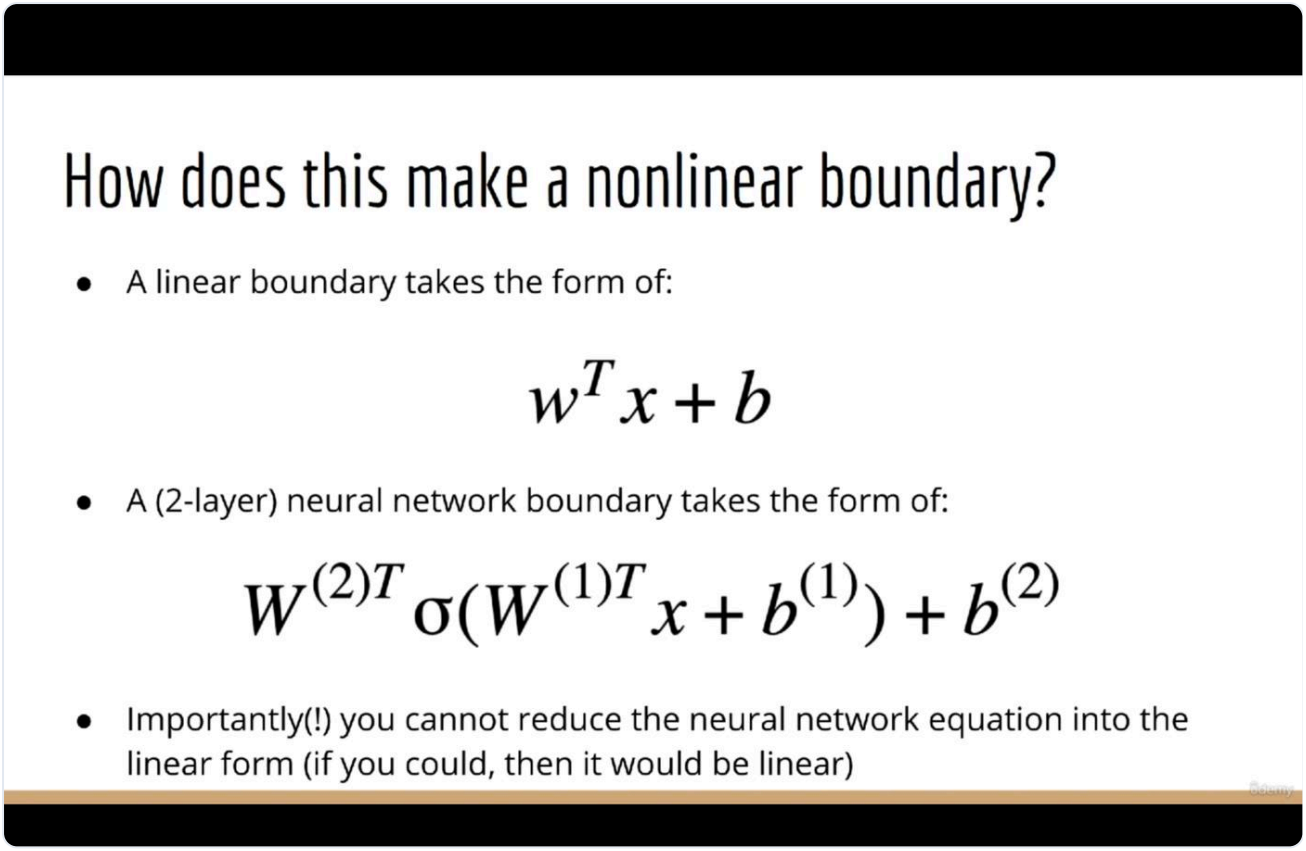


FIGURE 25.7

If you could then we would have a linear decision boundary so let's see what would happen if we had no sigmoid at all. Let's suppose we have a regression network with no sigmoid then it's just W2 transpose times w 1 transpose x plus B1 plus B2 of course using basic arithmetic. We can expand the product due to the fact that there is no sigmoid. It's just straightforward multiplication. Then we just get w prime transpose x plus b prime where I've substituted w prime equals W2 transpose times w 1 transpose and b prime equals w 2 transpose times B1 plus B2. And so you can see that if we have no sigmoid this just reduces to a linear equation. There is an added bonus to using neurons for our features.

If you recall all the weights and biases are randomly initialized and found using an iterative algorithm called gradient descent.

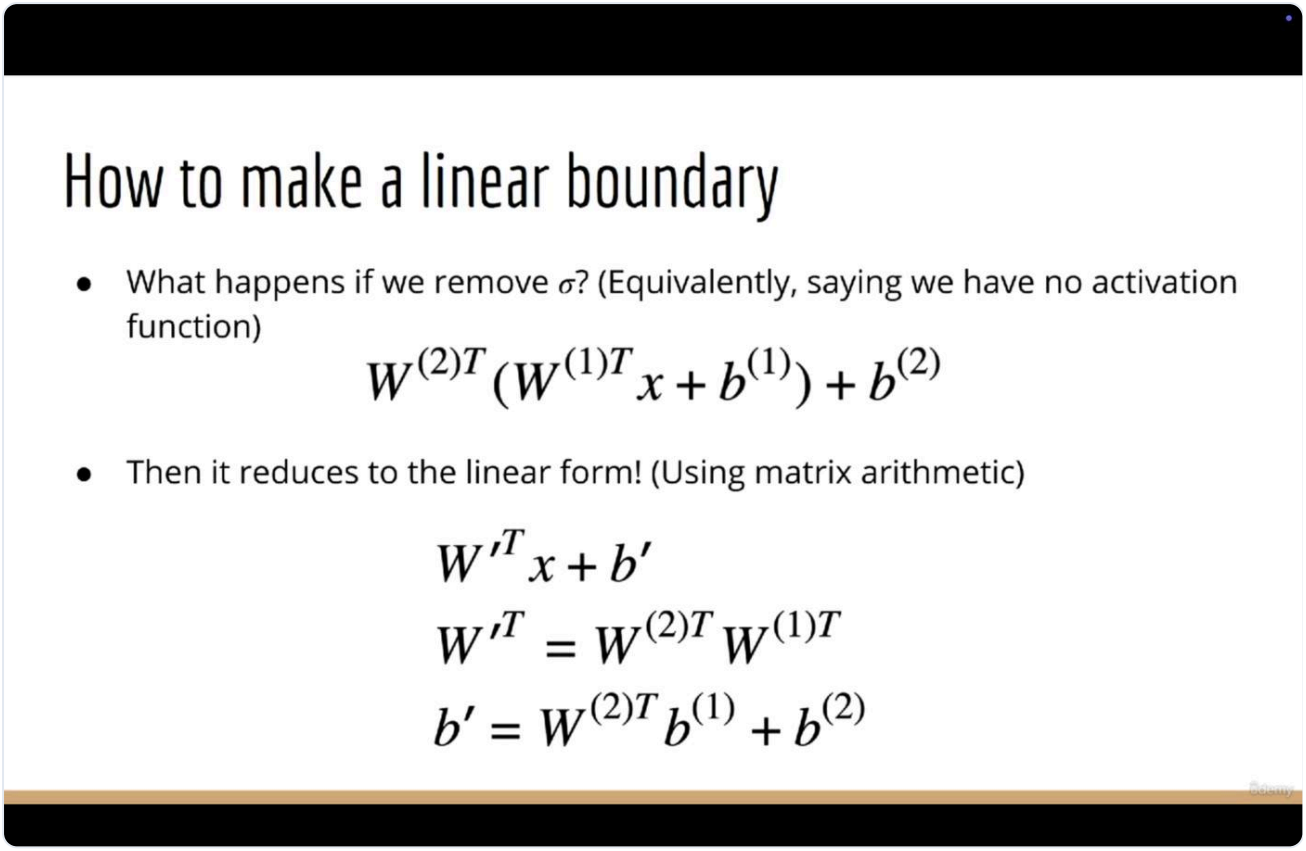


FIGURE 25.8 | How to Make a Linear Boundary

When we call the model dot fit function. What this means is that instead of having to do manual feature engineering to try and guess what features might be good a neural network will do this automatically. In the olden days of machine learning feature engineering at times used to be the only method of really applying machine learning successfully and usually this would require domain knowledge. So if you wanted to build a good image classifier for say lung cancer detection you would have to be very knowledgeable about different computer vision techniques but not only that you would also have to be very good at looking at x rays of lung cancer. By the way.

Note that I'm not a doctor so I don't know if they actually look at x rays but you get the idea. If you wanted to build a good stock predictor you would have to be very knowledgeable about finance if you wanted to build a good fraud detector you would have to be very knowledgeable about the insurance industry and as you may have heard deep learning is so popular because it allows people who are not domain experts to build very good machine learning models.

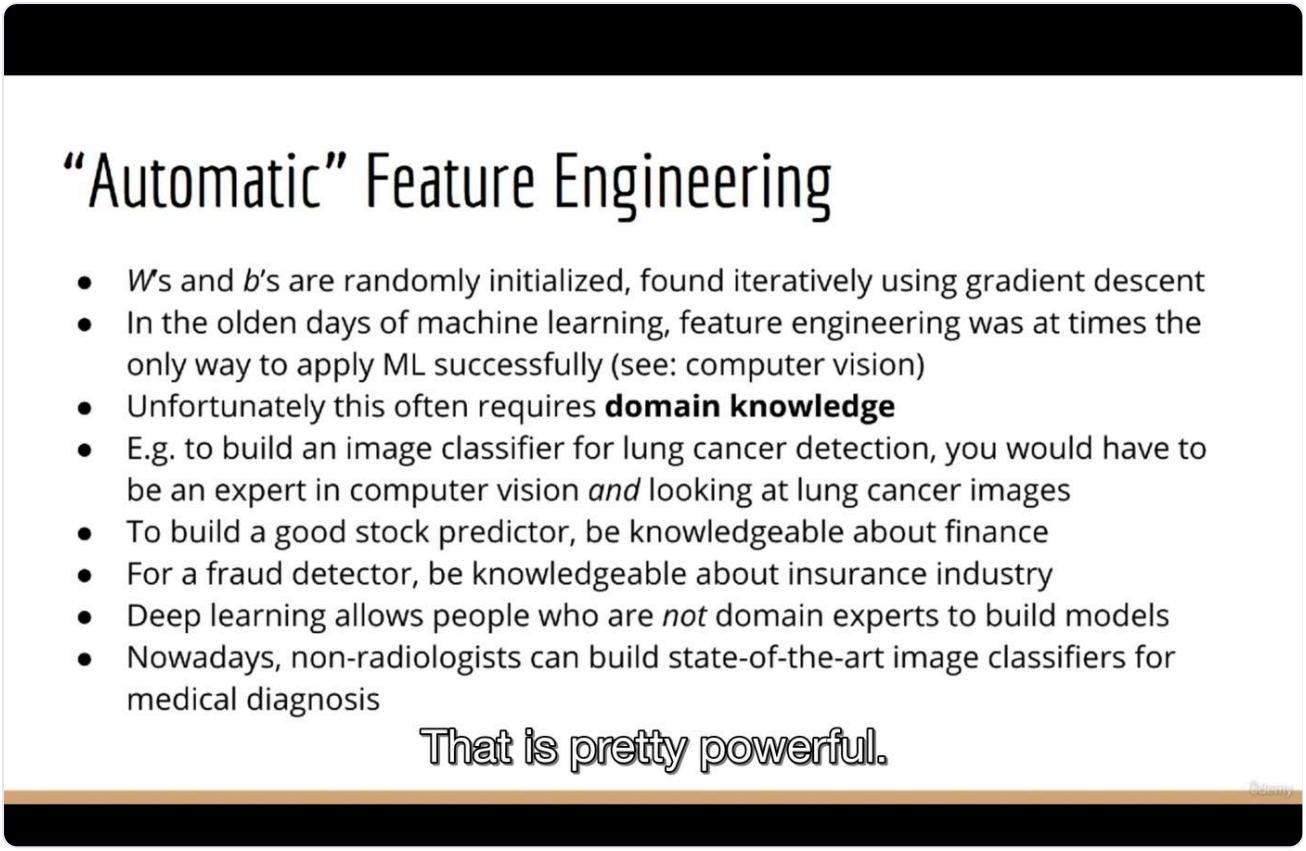


FIGURE 25.9

Nowadays we have people who are not expert radiologists building state of the art image classifiers for medical diagnosis all they are experts in is machine learning itself. That is pretty powerful. One tool I really like is the TensorFlow playground.

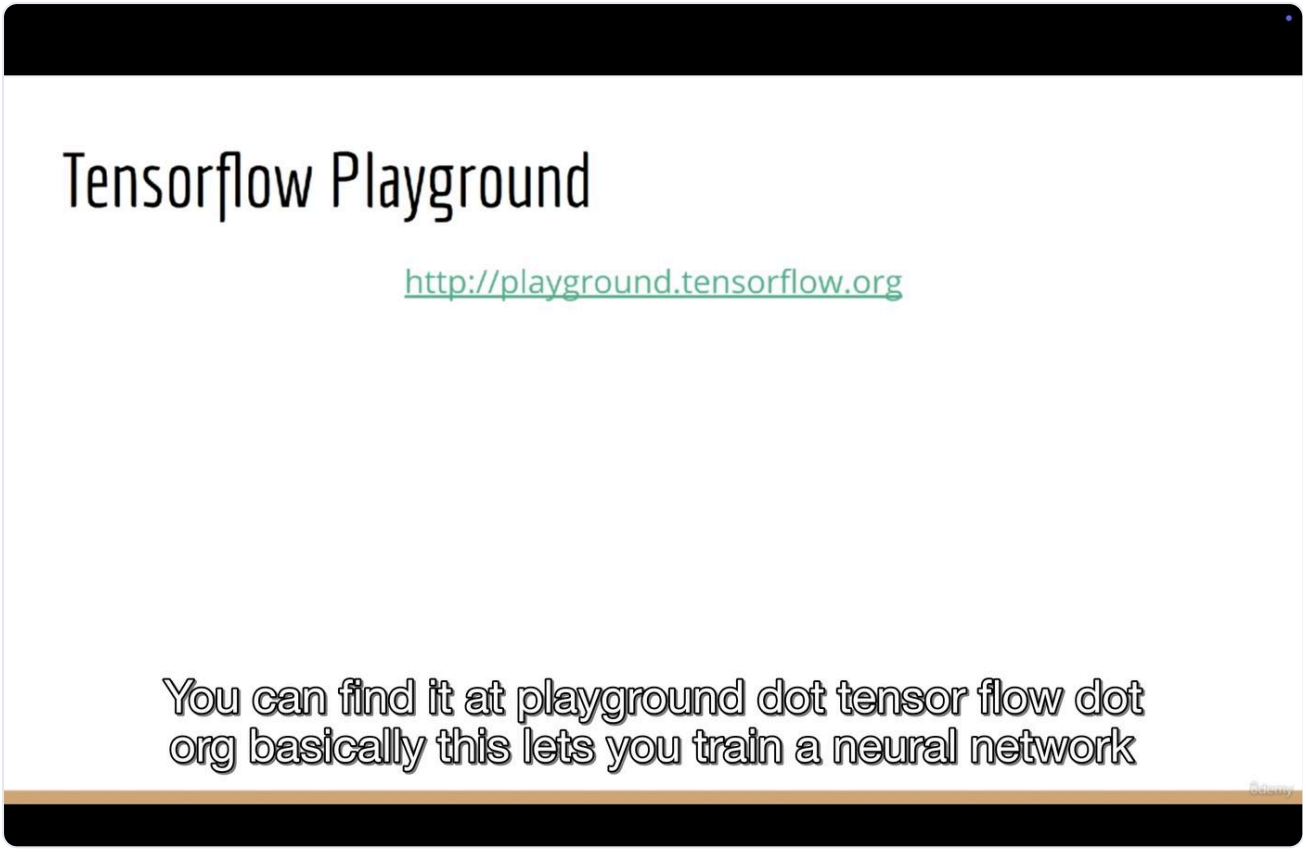


FIGURE 25.10 | Tensorflow Playground

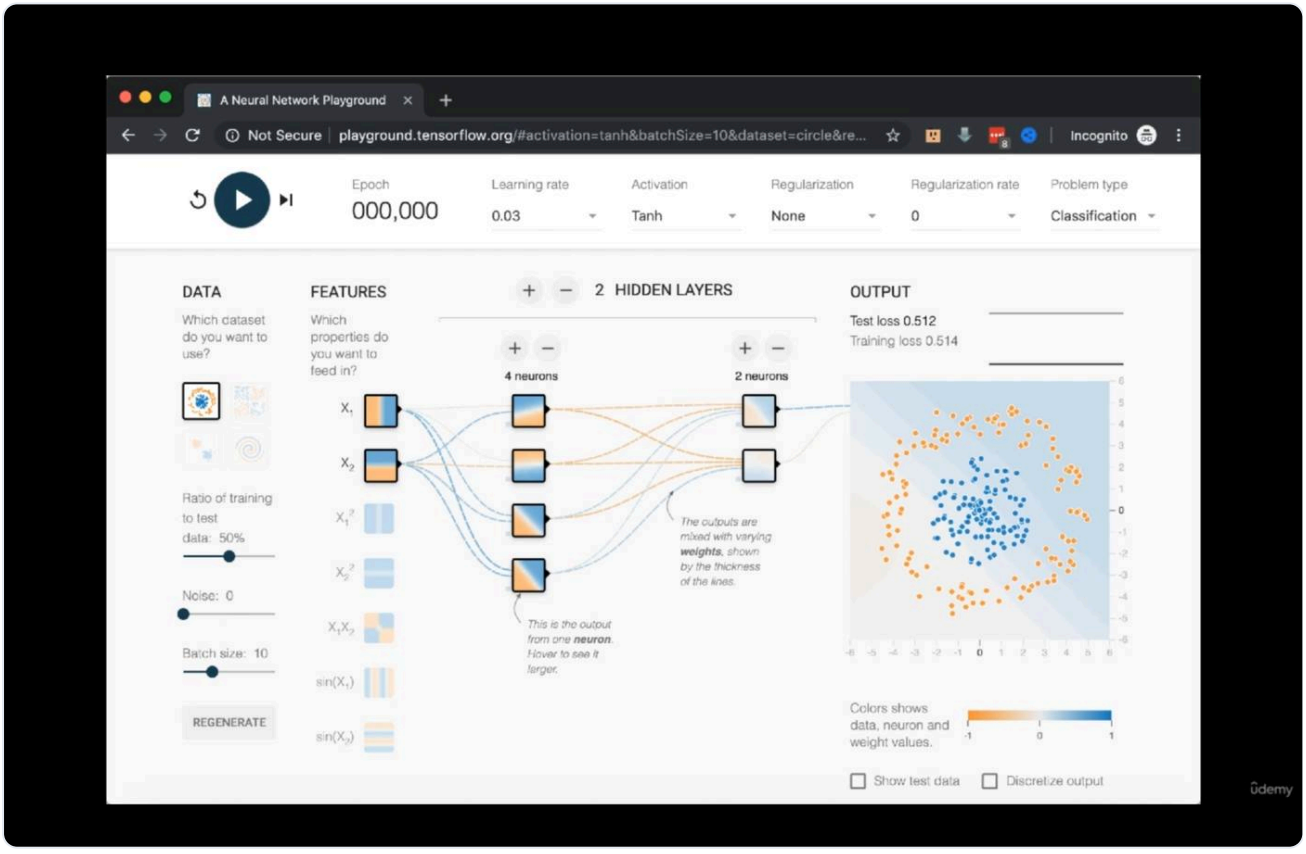


FIGURE 25.11

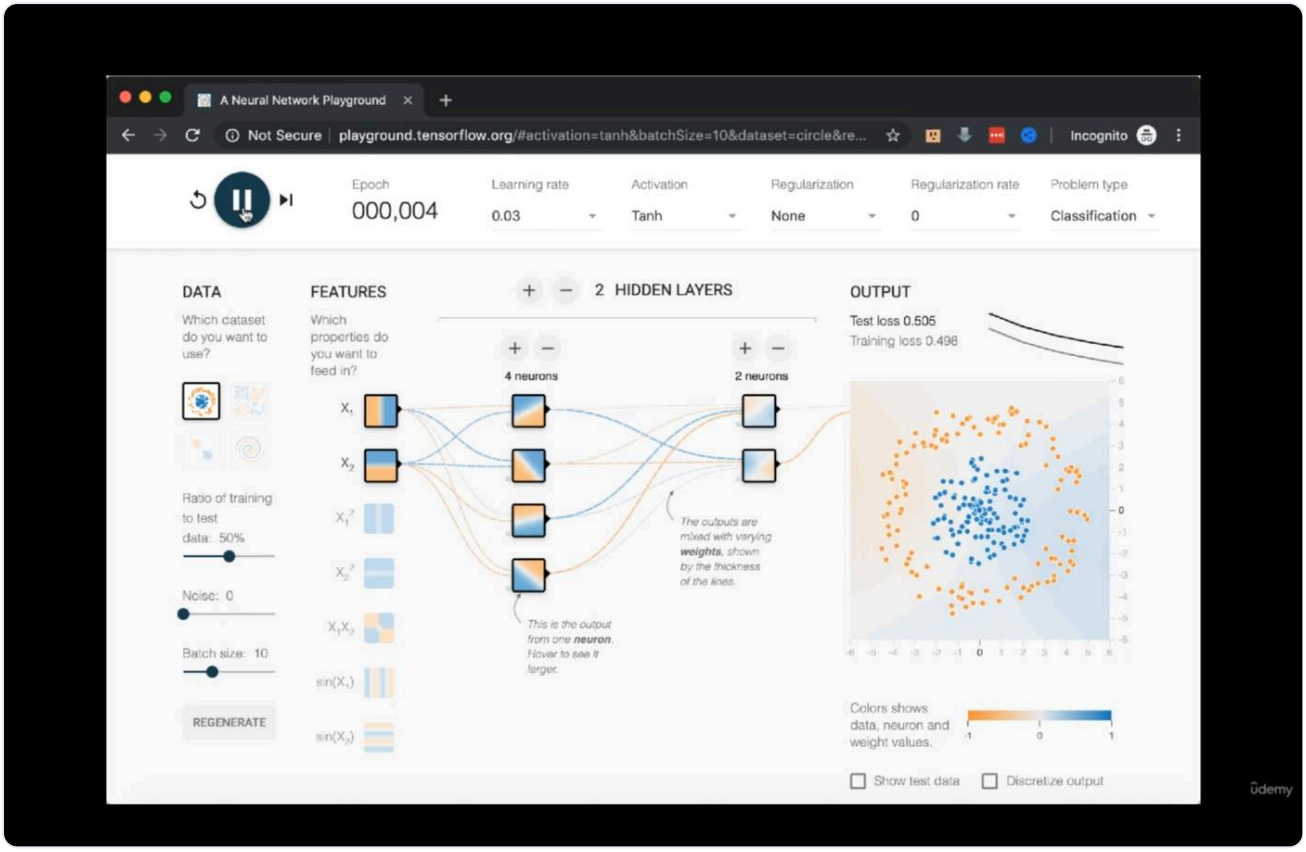


FIGURE 25.12

You can find it at playground dot TensorFlow dot org basically this lets you train a neural network on some synthetic data right in your browser. It lets you see for yourself that the neural network is learning a nonlinear decision boundary so if you go there it looks like the default is the doughnut problem where you have one circle inside another circle. And of course the decision boundary should be a circle. Your inputs are only the raw data x1 and x2 but you could also use feature engineering if you so choose. So you could select X1 squared X two squared x1 x 2 and even the sine of x1 x2 but we're not going to do that because we want to show that neural networks are able to learn non-linear features using only the original inputs.

So you never have to manually create things like this yourself and so by default we have a two hidden layer neural network with four hidden units so there's one two three four so let's just run this and see what happens all right. So as you can see it learns the nonlinear decision boundary quite efficiently without the need for any manual feature engineering you're encouraged to play around with this yourself. So try switching datasets so you can pick let's say this one and then run it again also try different numbers of hidden layers and different numbers of units per hidden layer. This should help give you a stronger intuition for how neural networks work and lets you observe for yourself that they are finding nonlinear decision boundaries.

LECTURE 26

The Vanishing Gradient Problem

In this lecture we are going to discuss activation functions. Previously we learned that the sigmoid function allows us to build neural networks and it does several important things for us.

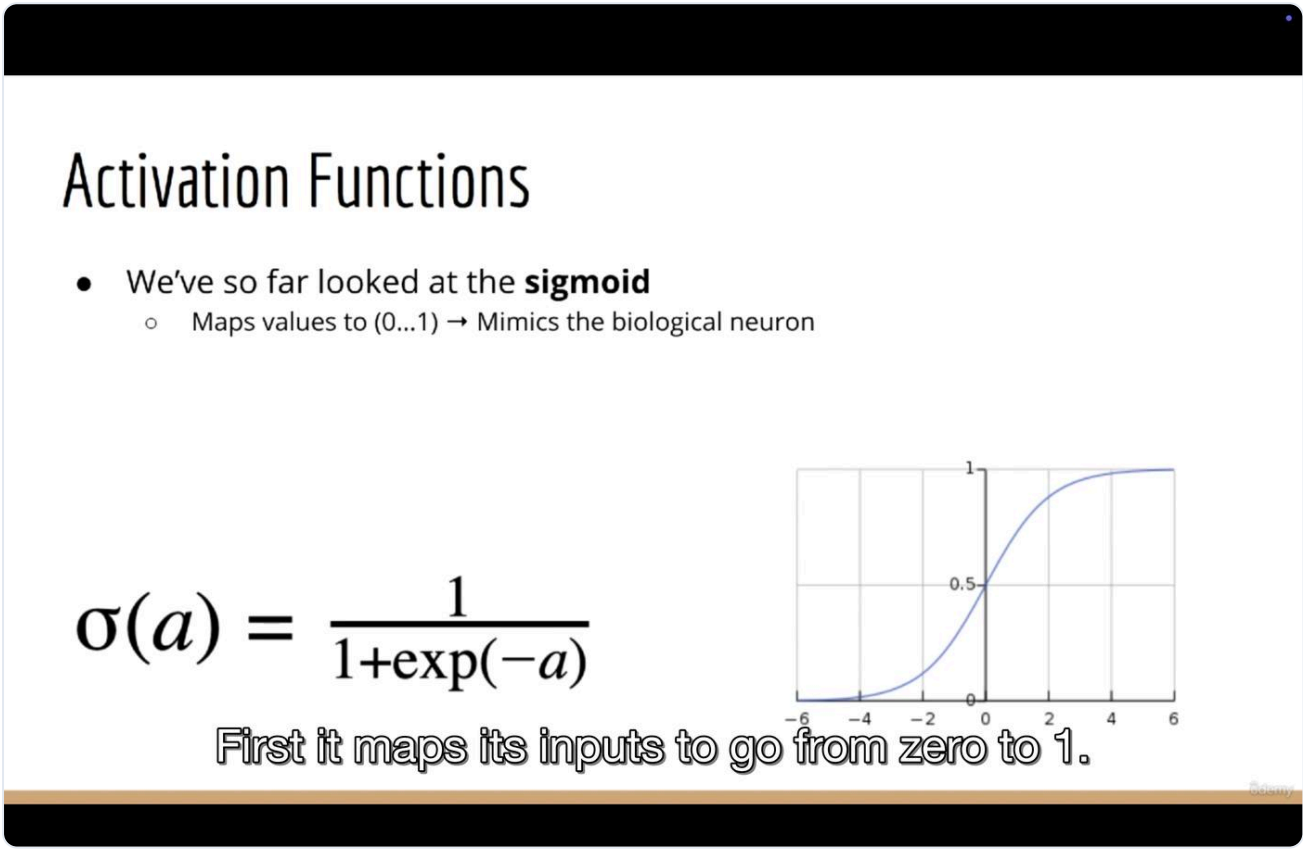


FIGURE 26.1 | Activation Functions

First it maps its inputs to go from zero to 1. This is nice because it mimics the biological neuron. So our neural network is literally a network of artificial neurons. Second and this is more practical. It makes our neural network nonlinear. The function itself is non-linear and it also makes it so that we can't reduce the neural network into a simple linear equation. At the same time in modern Deep Learning we've discovered that there are some problems with the sigmoid. And so it is no longer used as often except in some specific cases if you recall something we discussed earlier was the importance of standardization.

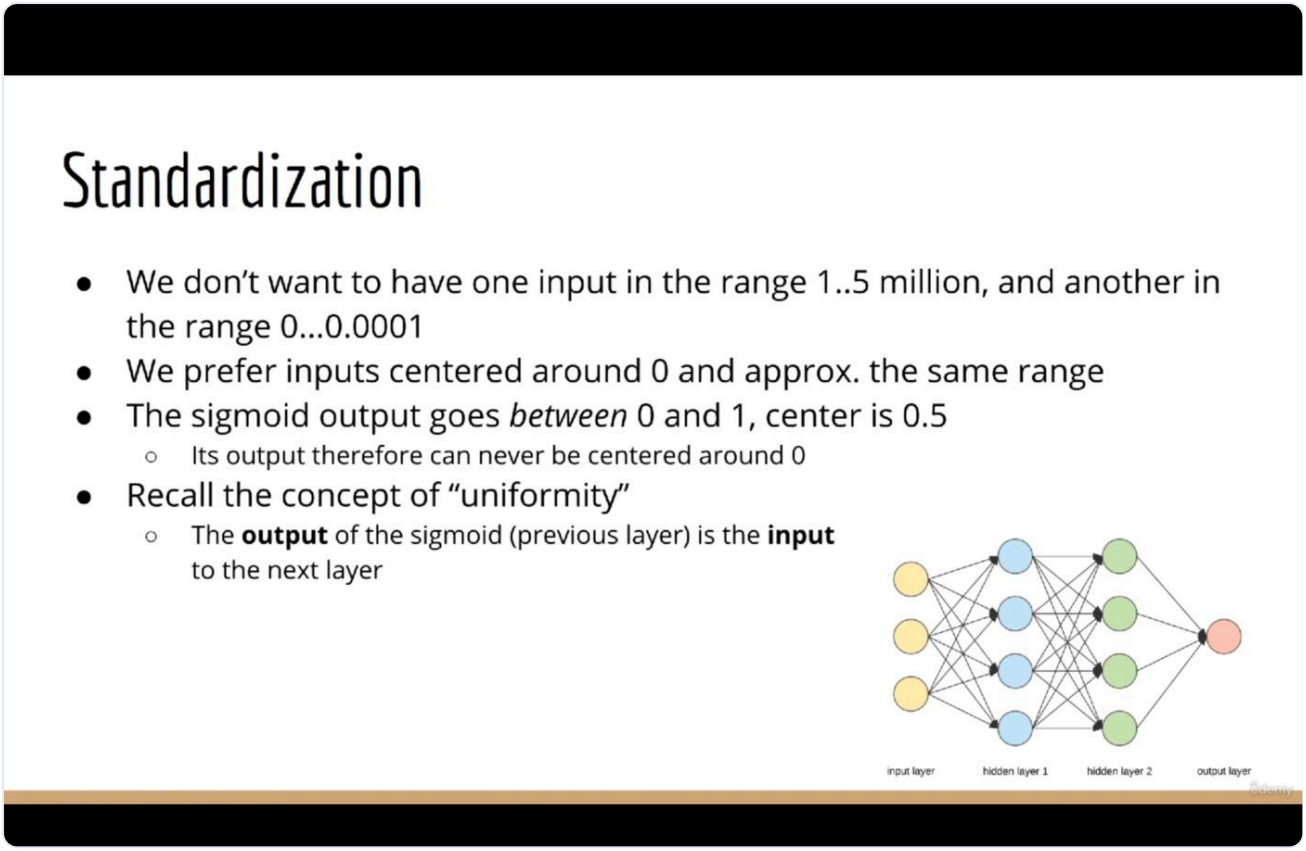


FIGURE 26.2 | The Sigmoid Output Goes Between 0 and 1, Center Is 0.5

We don't want to have one input in the range one million to 5 million and then another input going from zero to zero point 0 0 0 1 instead we would like to have all of our data centered around zero and in approximately the same range. Now the sigmoid is problematic in this regard. If you recall the output of a sigmoid is always between 0 and 1 and its middle value is therefore zero point five. Therefore the output of a sigmoid can never be centered around zero. This goes back to our idea of the uniformity of a neural network one layer of a neural network takes as input the output of the previous layer. So if the previous layer is outputting numbers centered around zero point five that's not quite right.

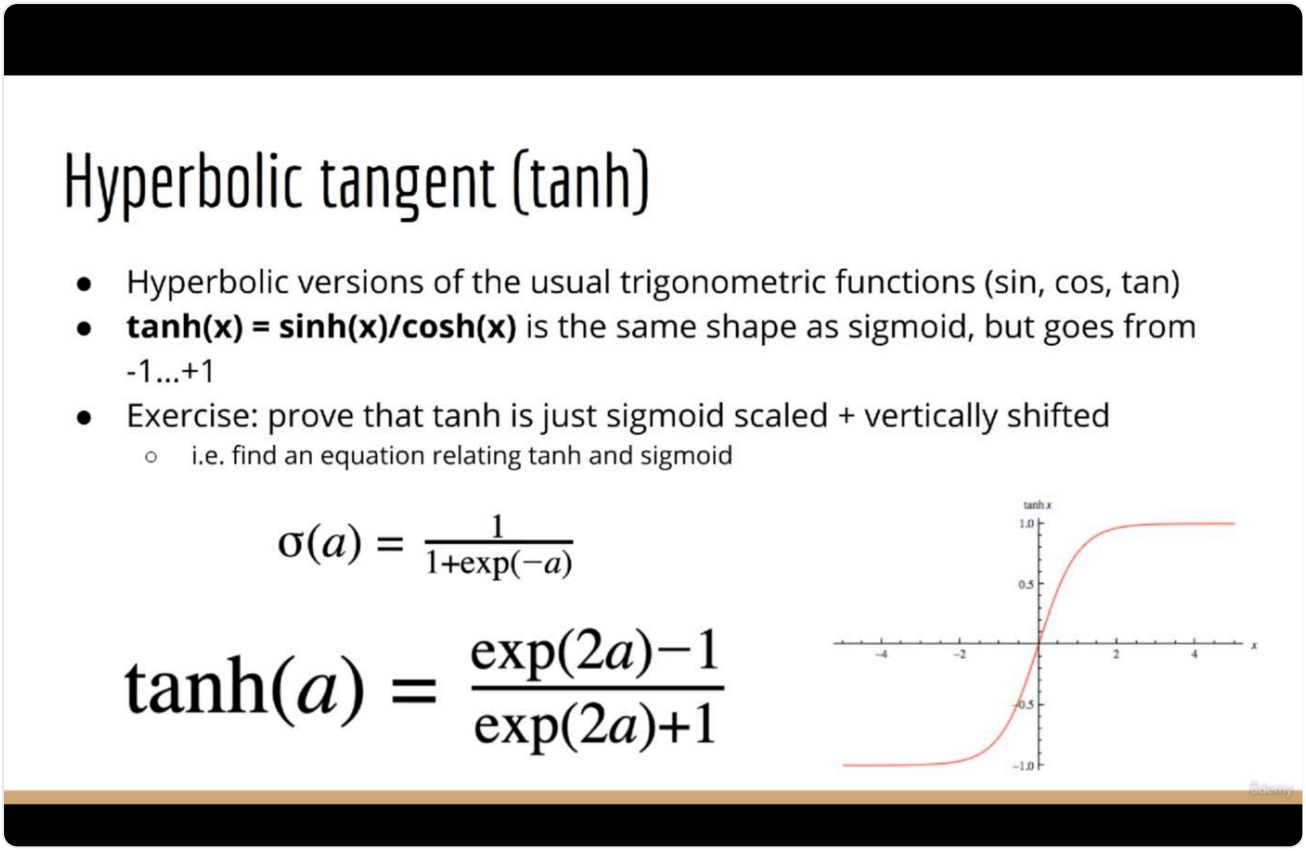


FIGURE 26.3 | Hyperbolic Tangent (Tanh)

This is because this output is going to become the input to the next layer and the next layer wants to also see inputs which are standardized. Therefore we want both the inputs and the output of each layer to be standardized luckily there is a solution to this although it takes away from this idea that a neuron network is made up of this idealized neuron simulation instead of using the sigmoid we'll use a function that has the exact same shape as the sigmoid just centered around zero. This function is called the tanh which is short for hyperbolic tangent if you recall there are hyperbolic versions of all the trigonometric functions. So there is a hyperbolic sine hyperbolic cosine and so forth.

Well it turns out that the hyperbolic tangent analogously to the trigonometric tangent is just the hyperbolic sine over the hyperbolic cosine and it has the equation you see here the sigmoid has a similar equation one divided by one plus the exponent of minus X. The major difference between the sigmoid and the tanh is that the sigmoid goes between 0 and 1 while the tanh goes between minus 1 and plus 1 as an exercise.

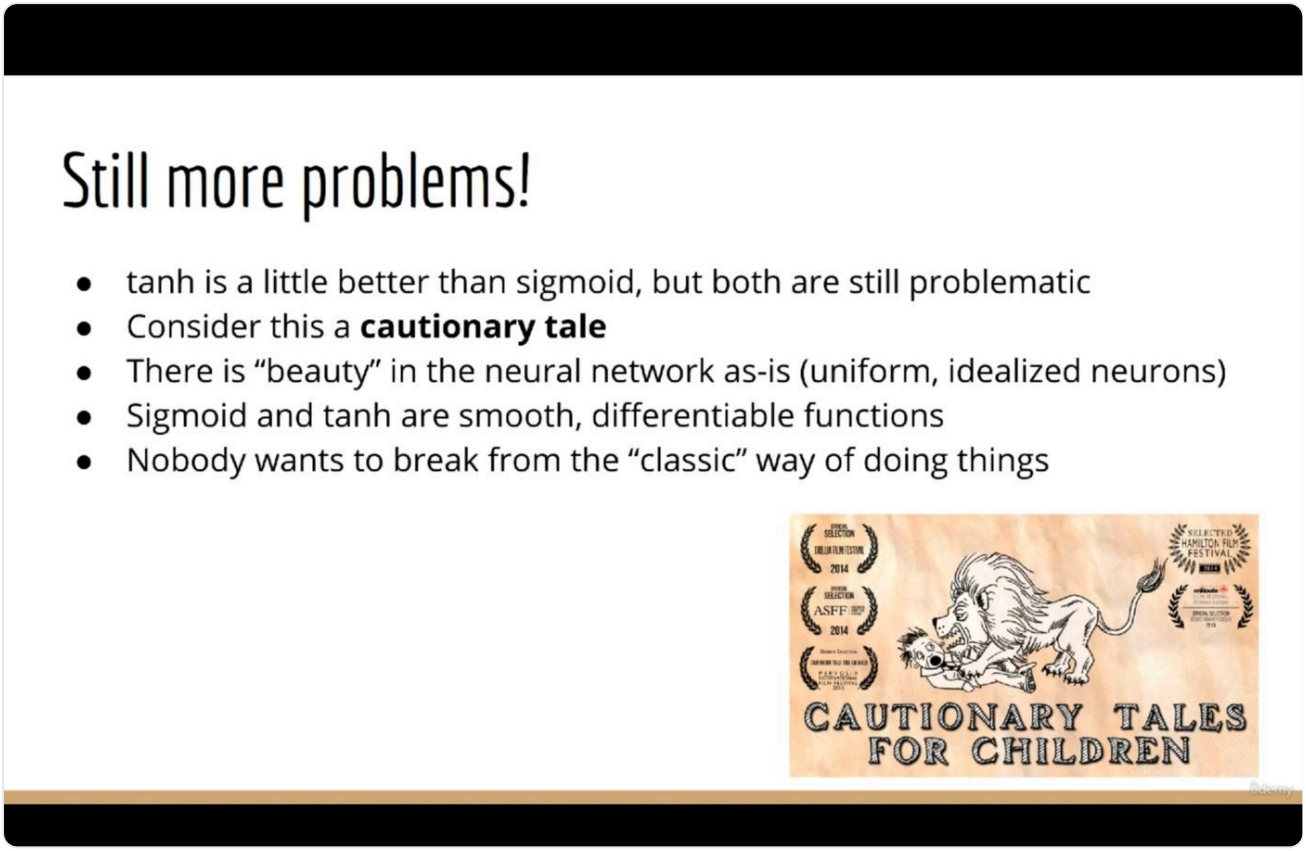


FIGURE 26.4 | Consider This a Cautionary Tale

You may want to try and prove the relationship between the sigmoid and the tanh in particular that the tanh is just a scaled and vertically shifted version of the sigmoid now although the tanh is a little better than the sigmoid the story is not over in modern deep learning. Researchers have figured out that there is actually a problem with both of these activation functions. I like to tell this story as a bit of a cautionary tale for a long time. Researchers were very attached to the beauty of neural networks as is they liked this idea of a uniform neural network made up of idealized neurons. They like this idea that the sigmoid and hence also the tanh were smooth differentiable functions.

It is mathematically convenient and beautiful. The problem is they don't work that well but nobody wanted to break away from this classic way of doing things the major problem with sigmoid and tanh is called the vanishing gradient problem. Now this part requires a little bit of calculus knowledge. So if you're not comfortable with that then feel free to skip ahead. Remember that our method of training a neural network is gradient descent. And obviously this involves finding the gradient of the cost with respect to the parameters. The problem is when you have a very deep neural network your gradient has to propagate backwards throughout the neural network starting from the end.

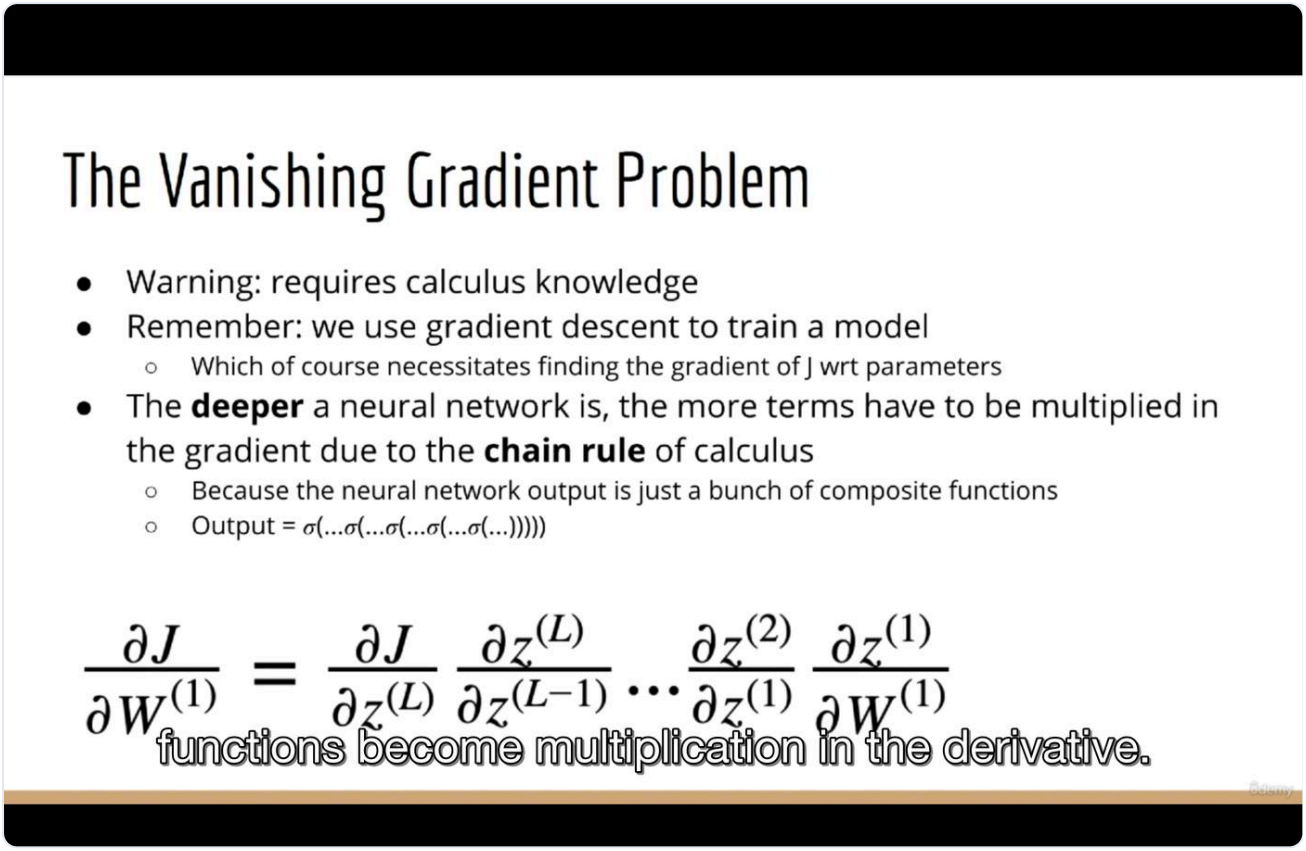


FIGURE 26.5 | The Vanishing Gradient Problem

What happens is your output is made up of a bunch of composite functions basically a sigmoid of a sigmoid of a sigmoid and so on when you take the gradient of composite functions you get the chain rule so composite functions become multiplication in the derivative. So what does this end up looking like. Well the further you go back in the neural network the more terms you have in the chain rule to multiply.

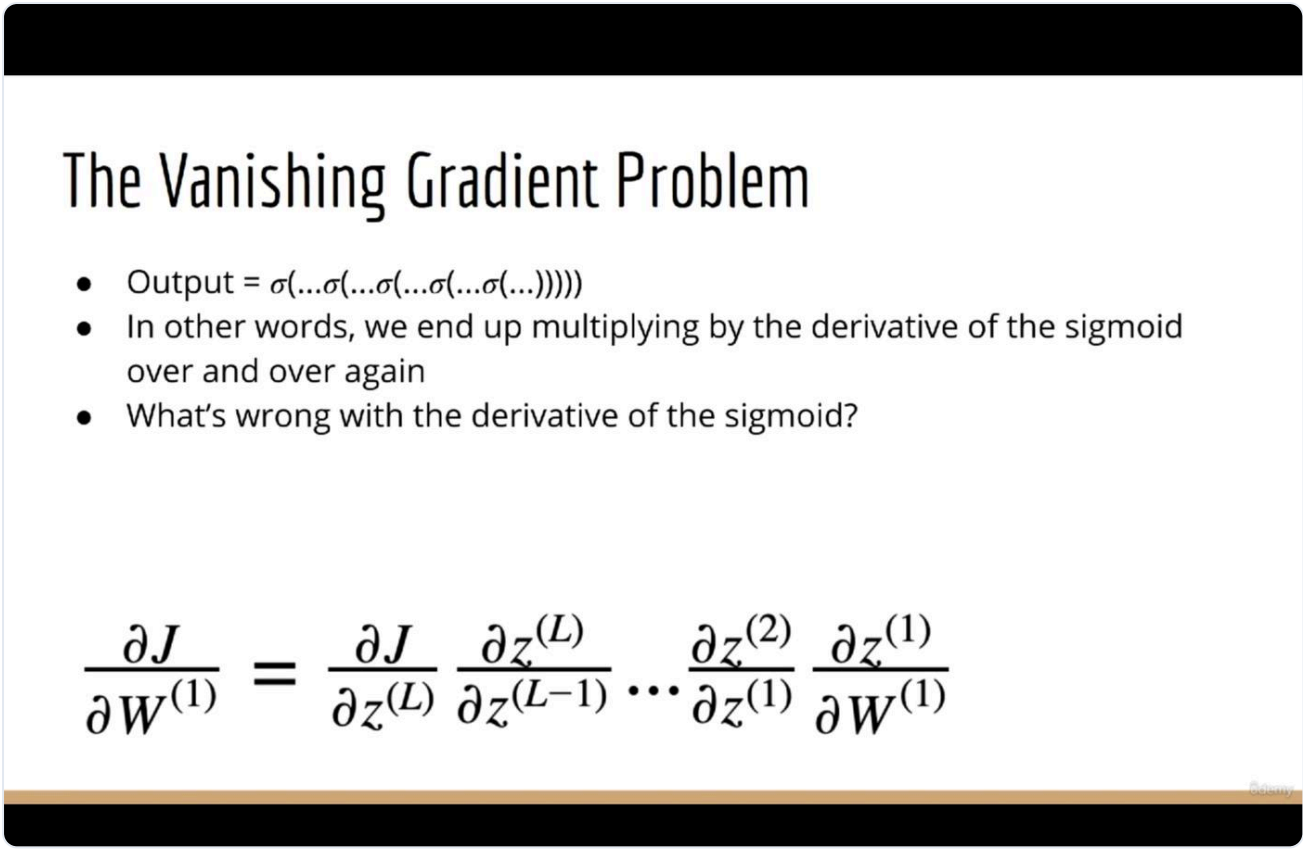


FIGURE 26.6 | The Vanishing Gradient Problem

So you're multiplying by the derivative of the sigmoid. Over and over again what happens when you do that. Why is the derivative of the sigmoid a problem well consider what the sigmoid actually looks like most of the sigmoid is flat. In other words the derivative of the sigmoid is very nearly zero at most points only in the center is it non-zero. It also turns out that the maximum value of the derivative is only zero point two five. This is the highest possible value of the derivative of the sigmoid.

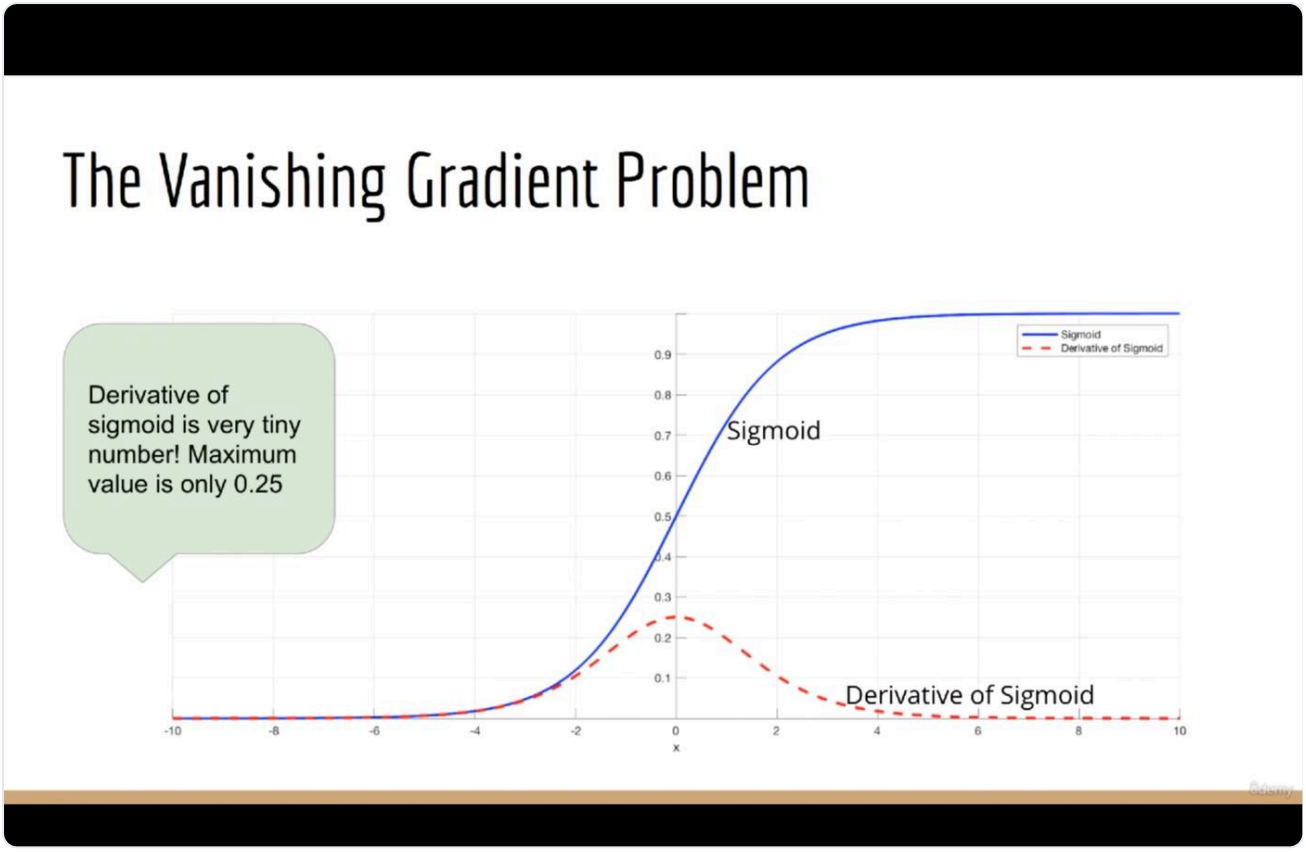


FIGURE 26.7 | The Vanishing Gradient Problem

Okay so why is that a problem. Well what happens when you multiply numbers that are very small. The answer is you get an even smaller number. Let's say you multiply zero point two five. The maximum possible value five times the result is zero point two five to the power five which is approximately zero point zero zero one. What happens if more realistically you have a value like zero point one multiplied by itself five times. That's zero point one to the power five which is zero point zero zero zero zero one.

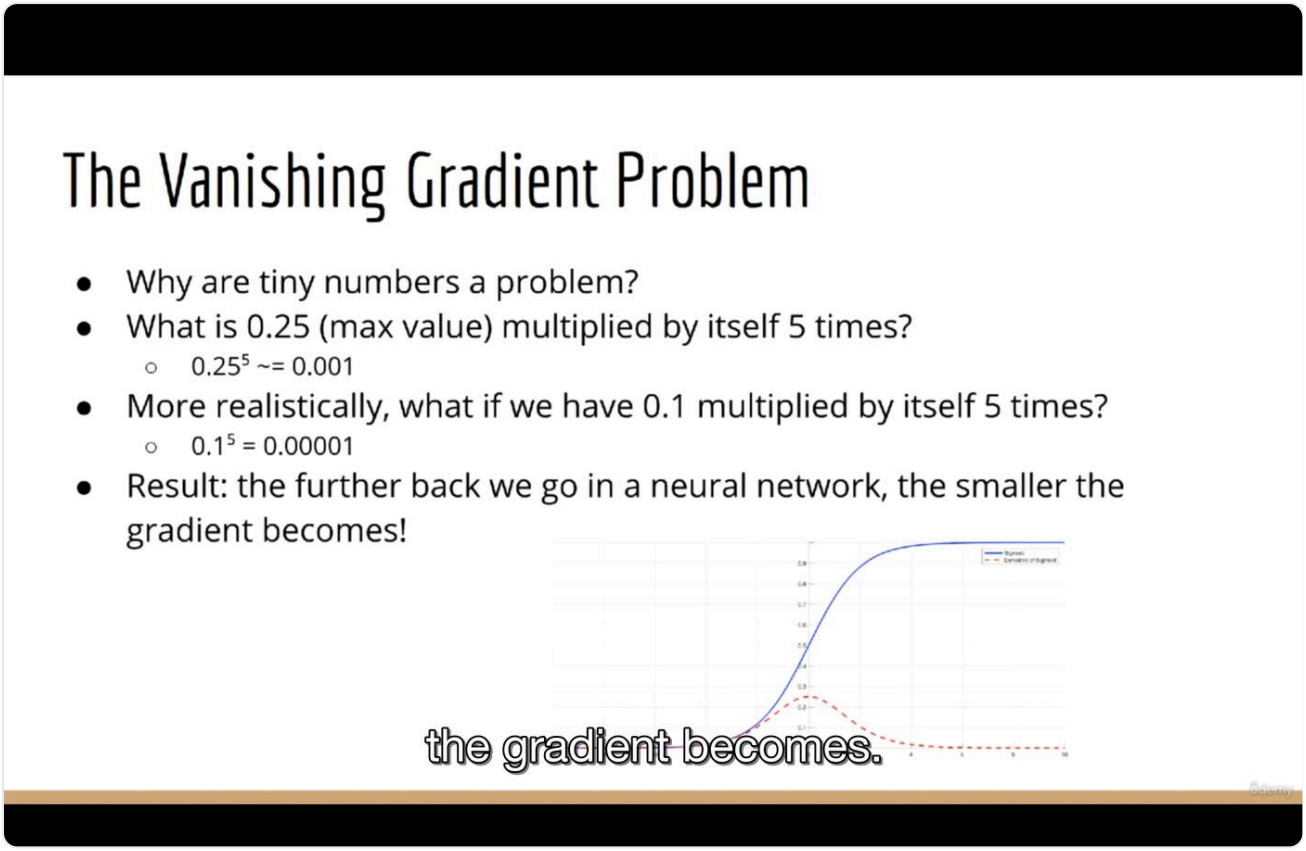


FIGURE 26.8 | The Vanishing Gradient Problem

In other words this leads the result that the further you go back in the neural network the smaller the gradient becomes. We call this the vanishing gradient problem. So how does this problem manifest.

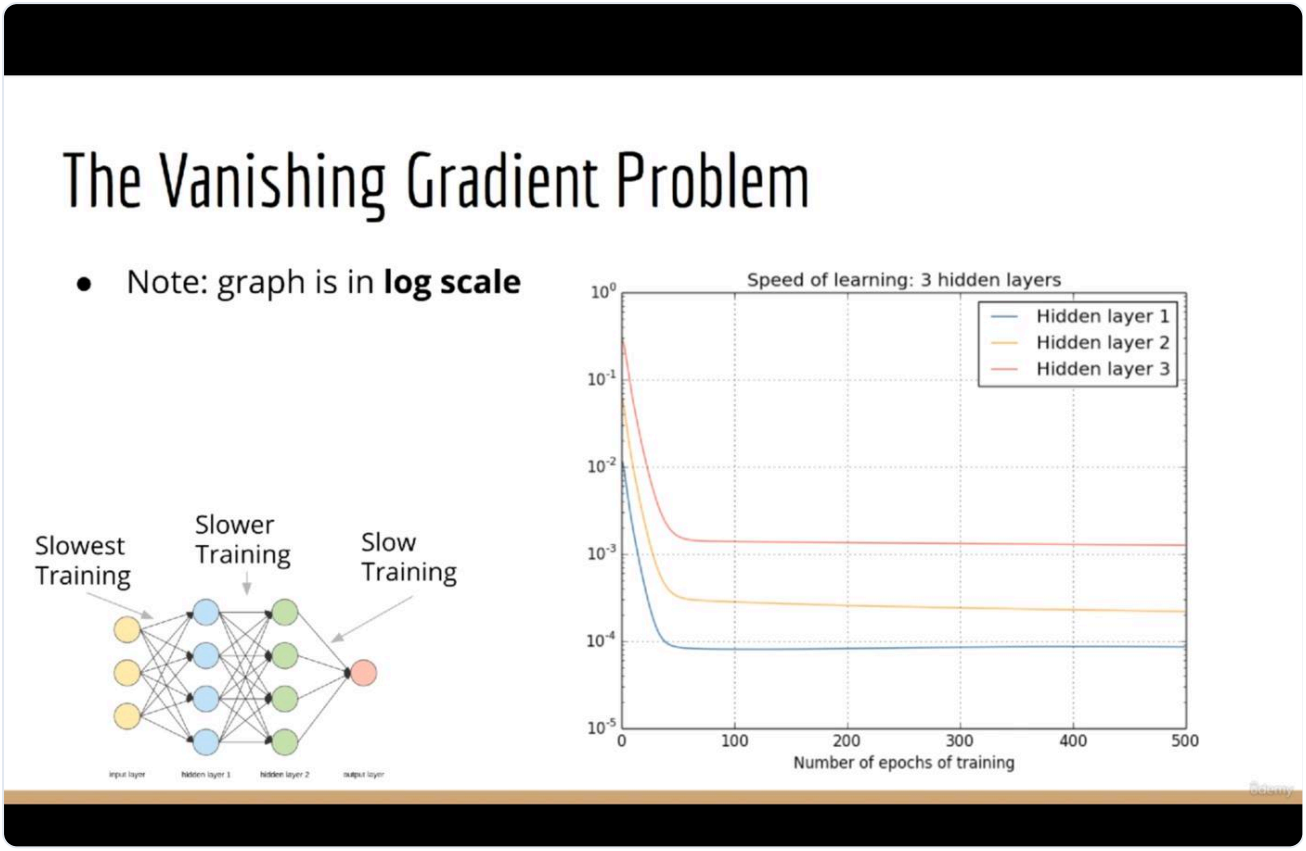


FIGURE 26.9 | The Vanishing Gradient Problem

We'll take a look at this graph which shows the magnitude of the gradient at each layer of the neural network as it is trained. You'll notice that the further you go back the smaller the gradient gets. Remember that the training algorithm is to take small steps in the direction of the gradient.

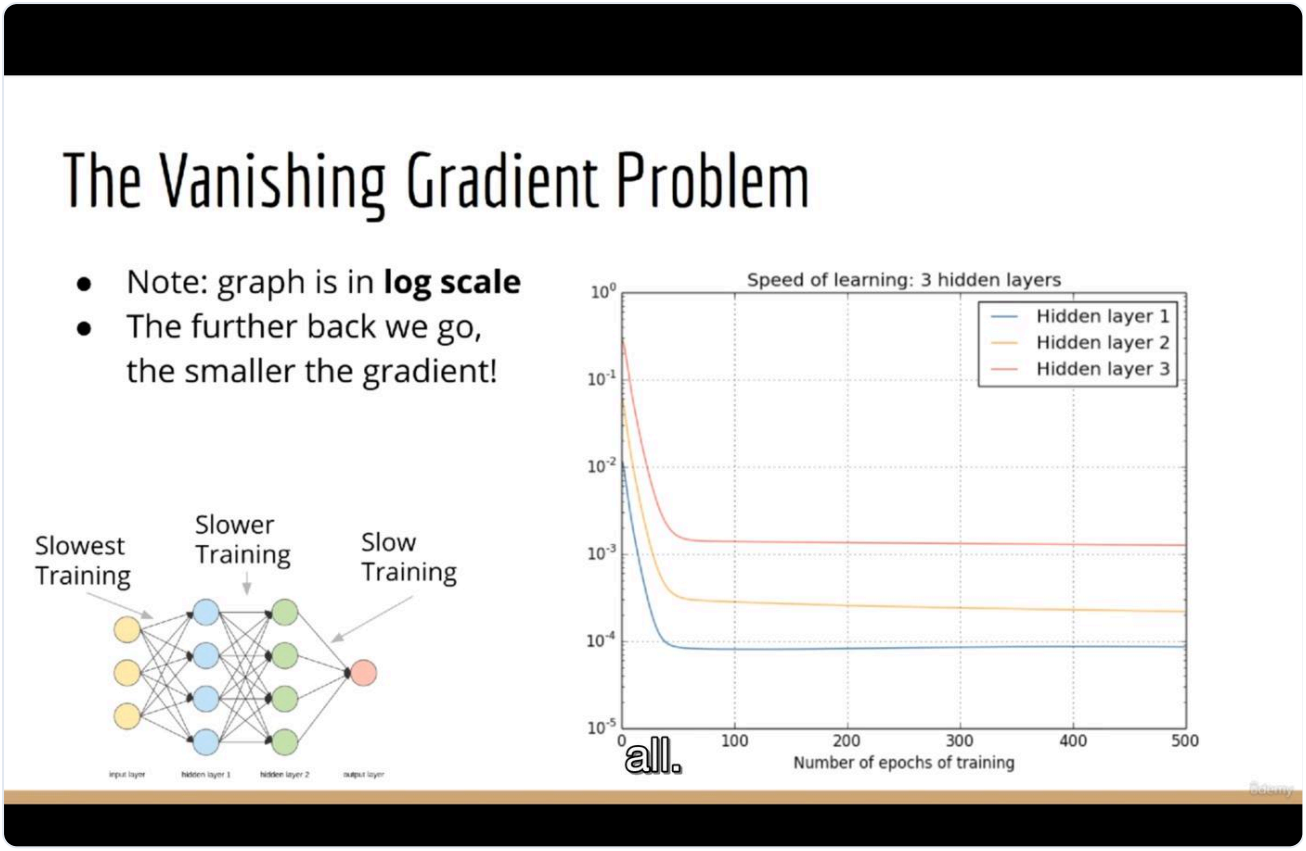


FIGURE 26.10 | The Vanishing Gradient Problem

Well if the gradient is nearly zero that means the update to the weights is also nearly zero. The end result is that weights close to the input of the neural network are almost not trained at all. This was a problem in the olden days which prevented us from building very deep neural networks like we have today which can have hundreds of layers back in the day.

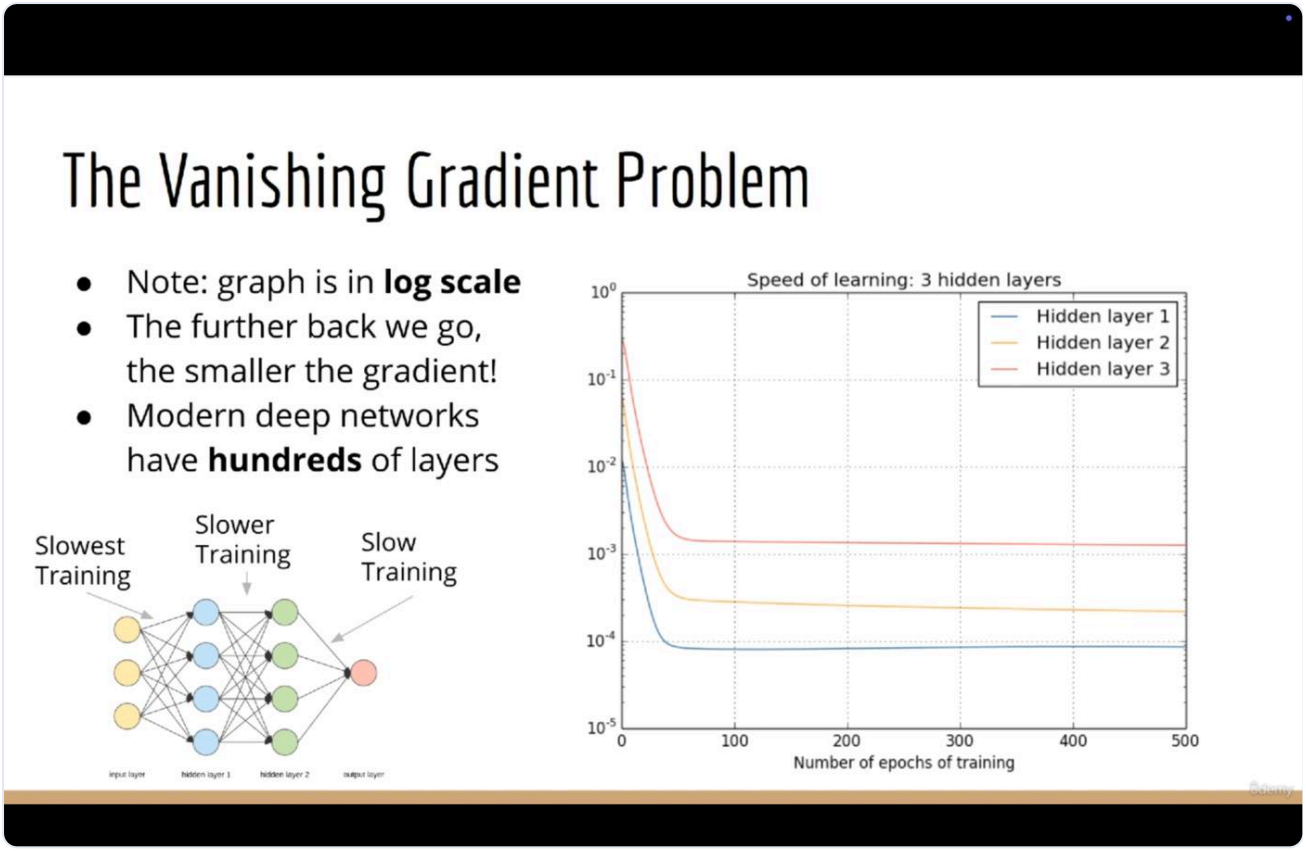


FIGURE 26.11 | The Vanishing Gradient Problem

There was lots of theory around how to solve this problem. One method was called a greedy layer wise pre training which was invented by Geoffrey Hinton and his students. Since all the layers of the neural network could not be trained all at once the idea was you could train each layer one at a time so what you would do is train the first layer using some alternative loss function.

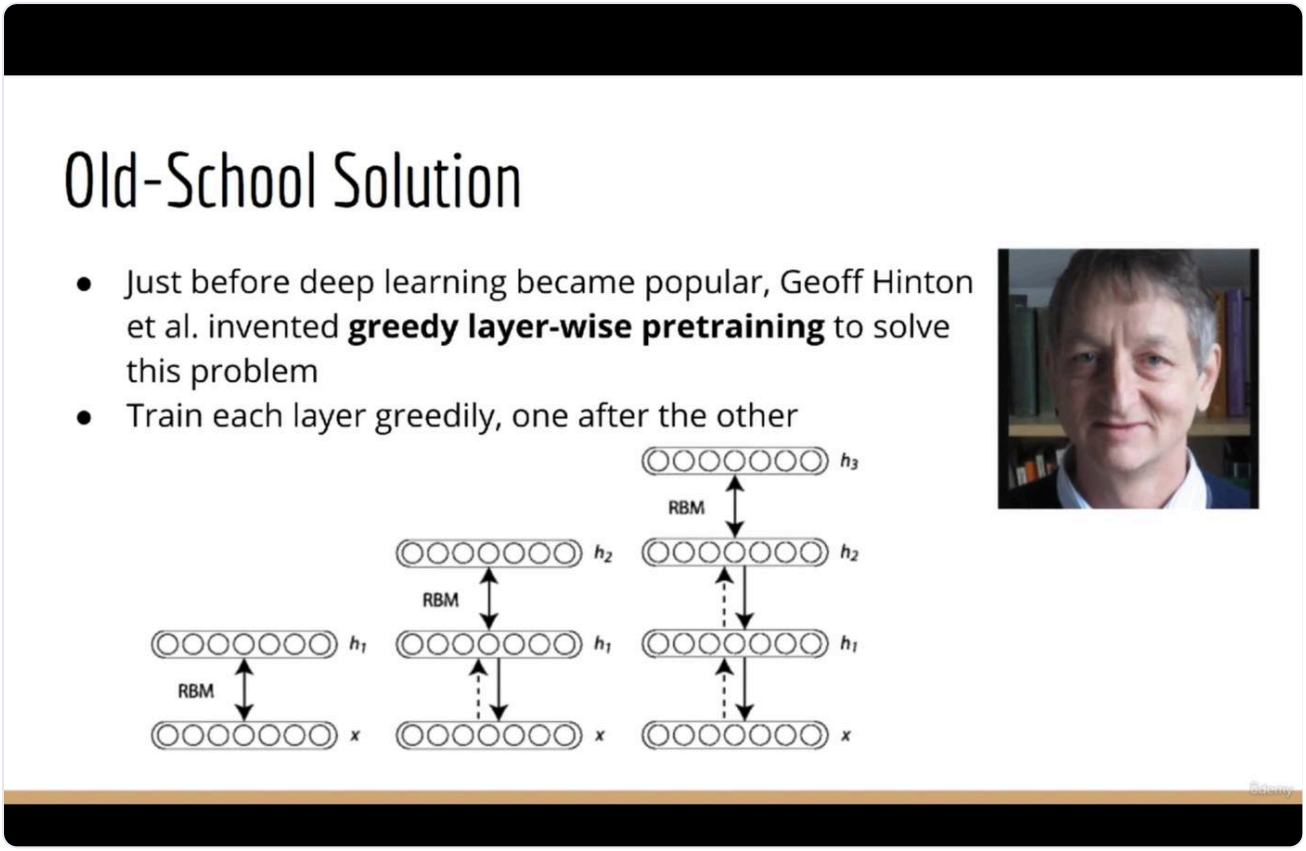


FIGURE 26.12

By the way if you want to know what that loss function is it's basically an autoencoder but hold that thought for now as it's not an important detail then once you were done training the first layer you would add on a second layer and train that layer by itself not touching the first layer anymore. Then you would add a third layer and train only the third layer leaving the first two layers alone. Once you got to the last layer all of the previous layers would already be trained to some extent.

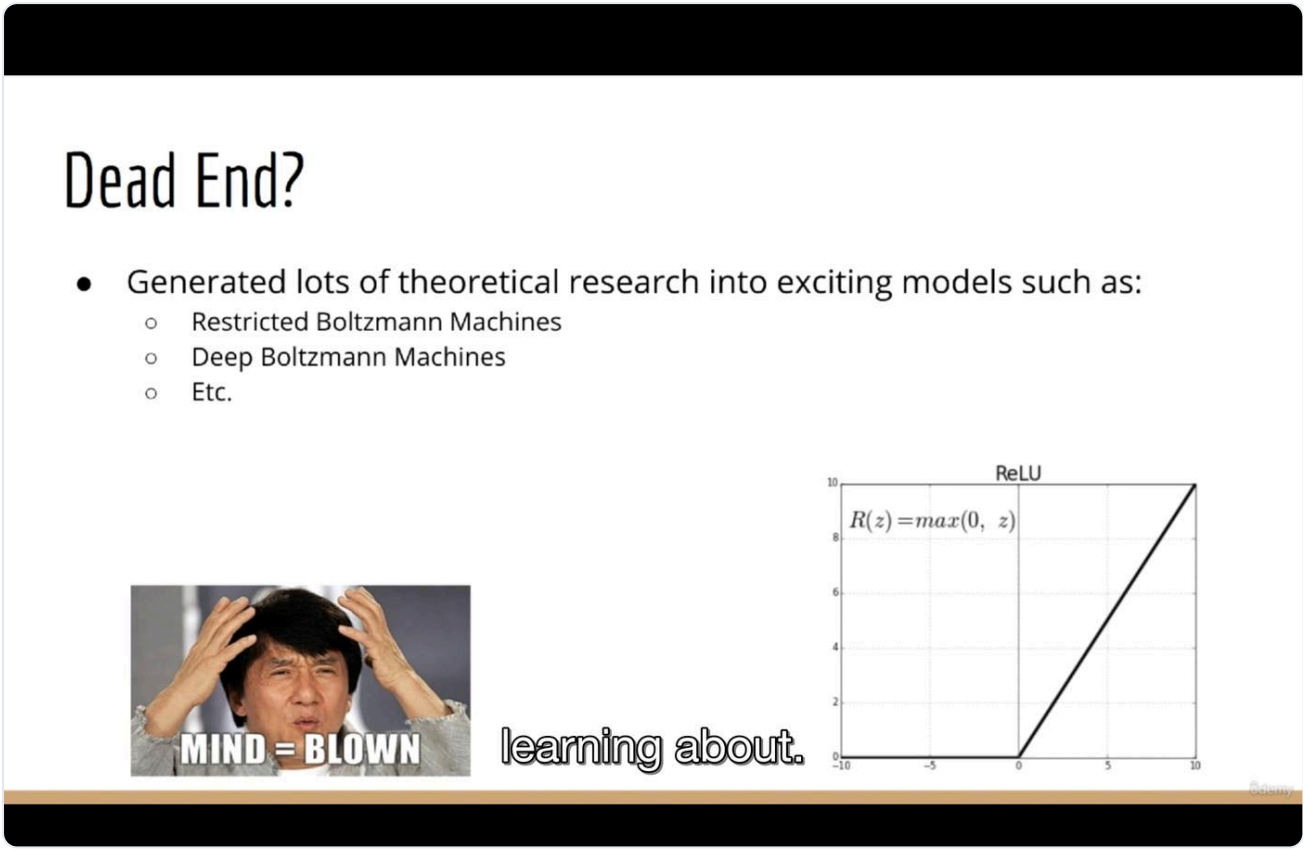


FIGURE 26.13 | Dead End?

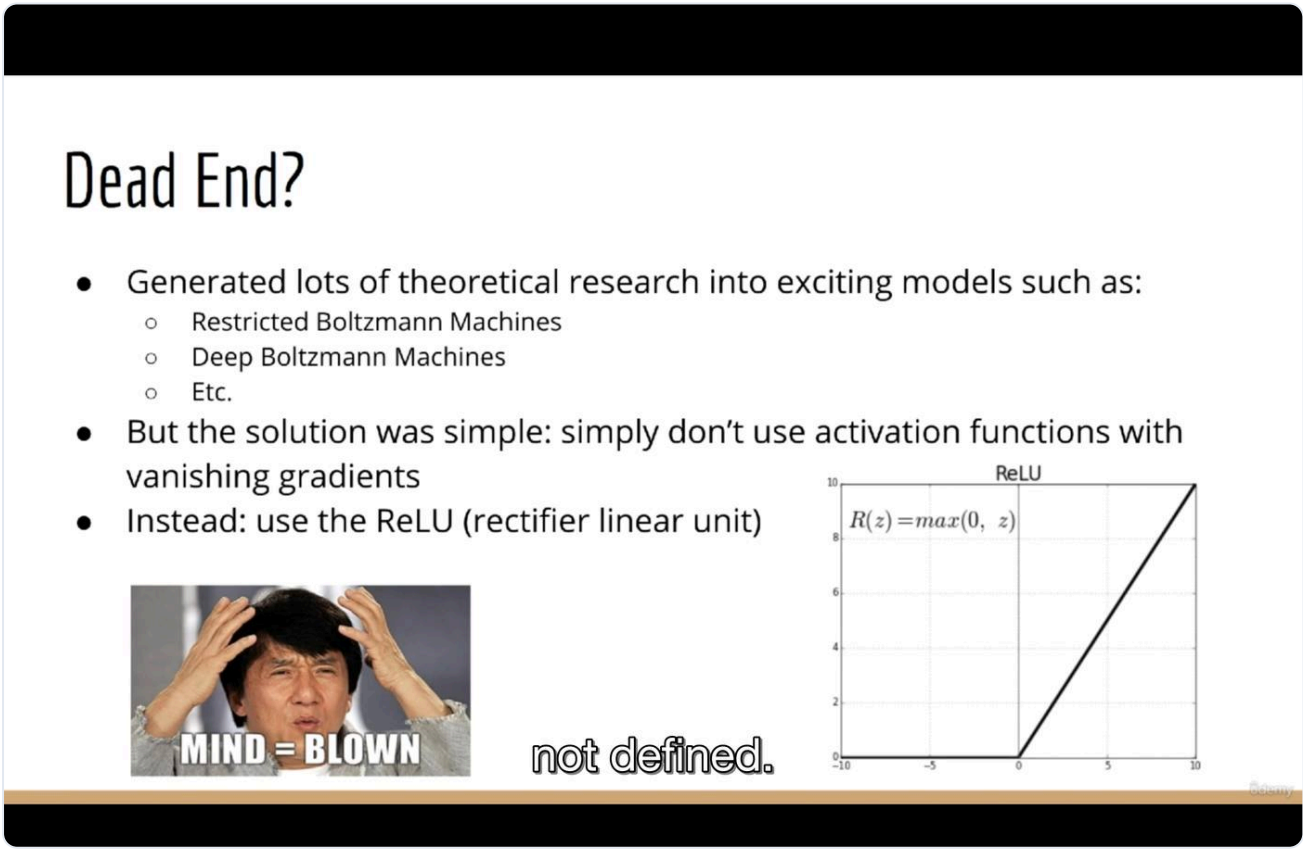


FIGURE 26.14 | Dead End?

Now despite all this theory which generated lots of theoretical research into new models such as restricted boltzmann machines deep Boltzmann machines and so forth today we no longer require such complicated models in order to train very deep neural networks although that's not to say they're not worth learning about. In fact the solution was simple Just don't use activation functions that have vanishing gradients. So throw away these beautiful smooth differential functions like the tanh and the sigmoid instead. Let's use this ugly looking not completely differentiable function called the ReLU the ReLU is short for rectified linear unit. Basically it looks like a hockey stick and it has a corner at zero where the derivative is technically not defined.

The fact that values greater than zero never have a zero gradient makes training neural networks a lot more efficient.

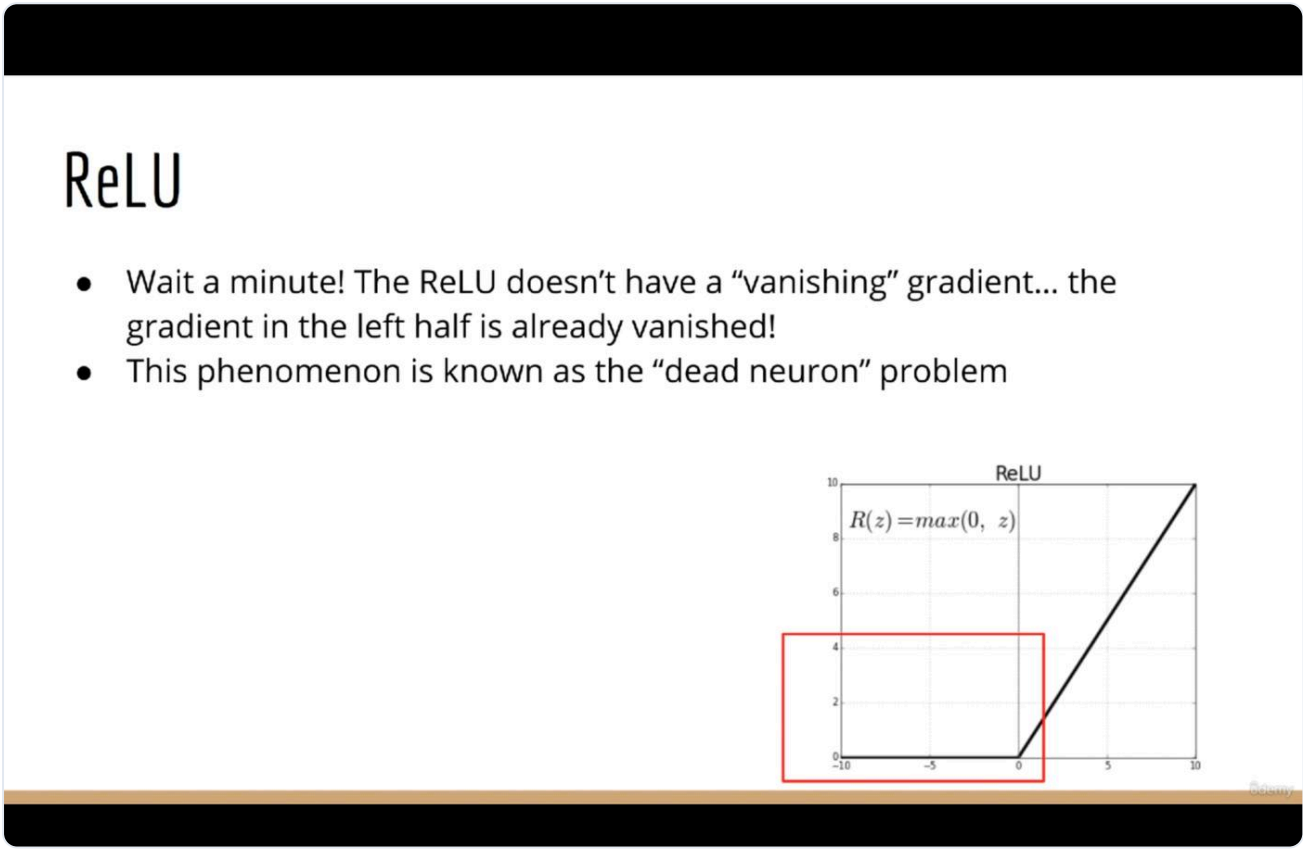


FIGURE 26.15

Now you might be wondering wait a minute. If a zero gradients are so bad then why does the ReLU work at all. It appears that half the function any input less than zero has a derivative that is exactly zero. Never mind vanishing the gradient is already vanished. Indeed this is somewhat of a problem as it leads to a phenomenon called Dead neurons. The neurons are neurons that always output zero because the weighted sum of its inputs are always less than or equal to zero. However and this is important in deep learning what we care about is experimental results.

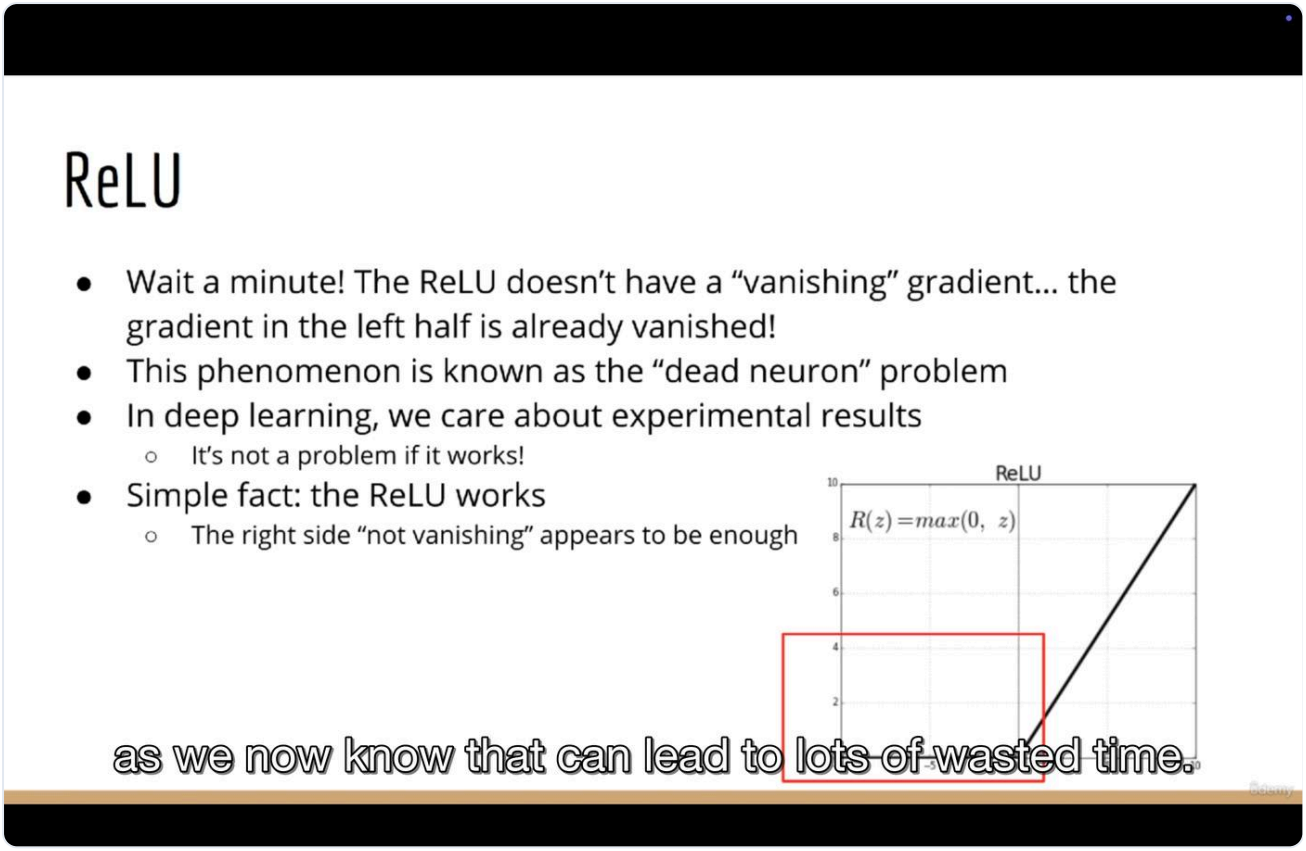


FIGURE 26.16

In other words the most important thing is that it works not how theoretically satisfying is because as we now know that can lead to lots of wasted time. The fact that just the right side doesn't vanish seems to be good enough according to what we've observed.

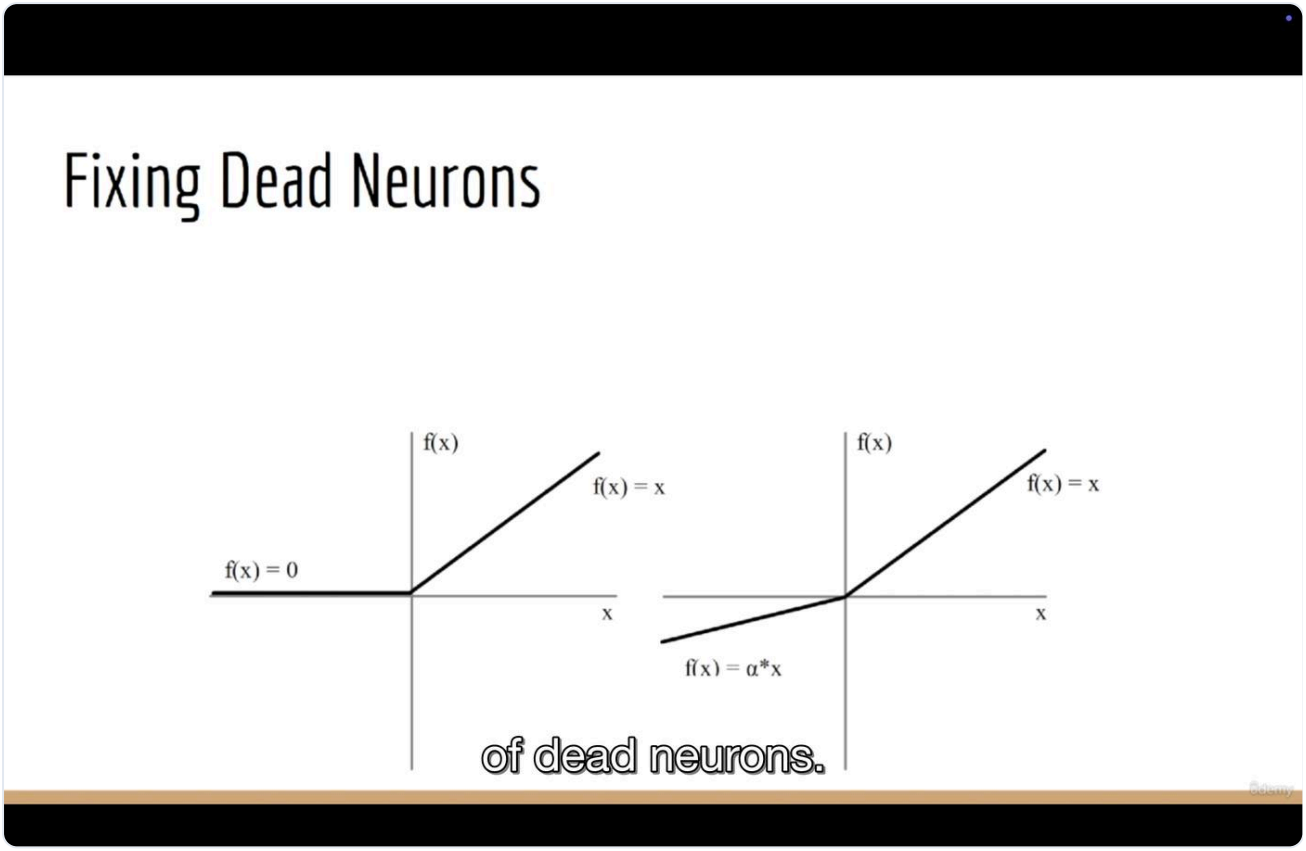


FIGURE 26.17

Of course some researchers have tried to make modifications to the simple ReLU to solve this problem of dead neurons. One alternative is the leaky ReLU which has a slope of less than one like zero point one for values less than zero.

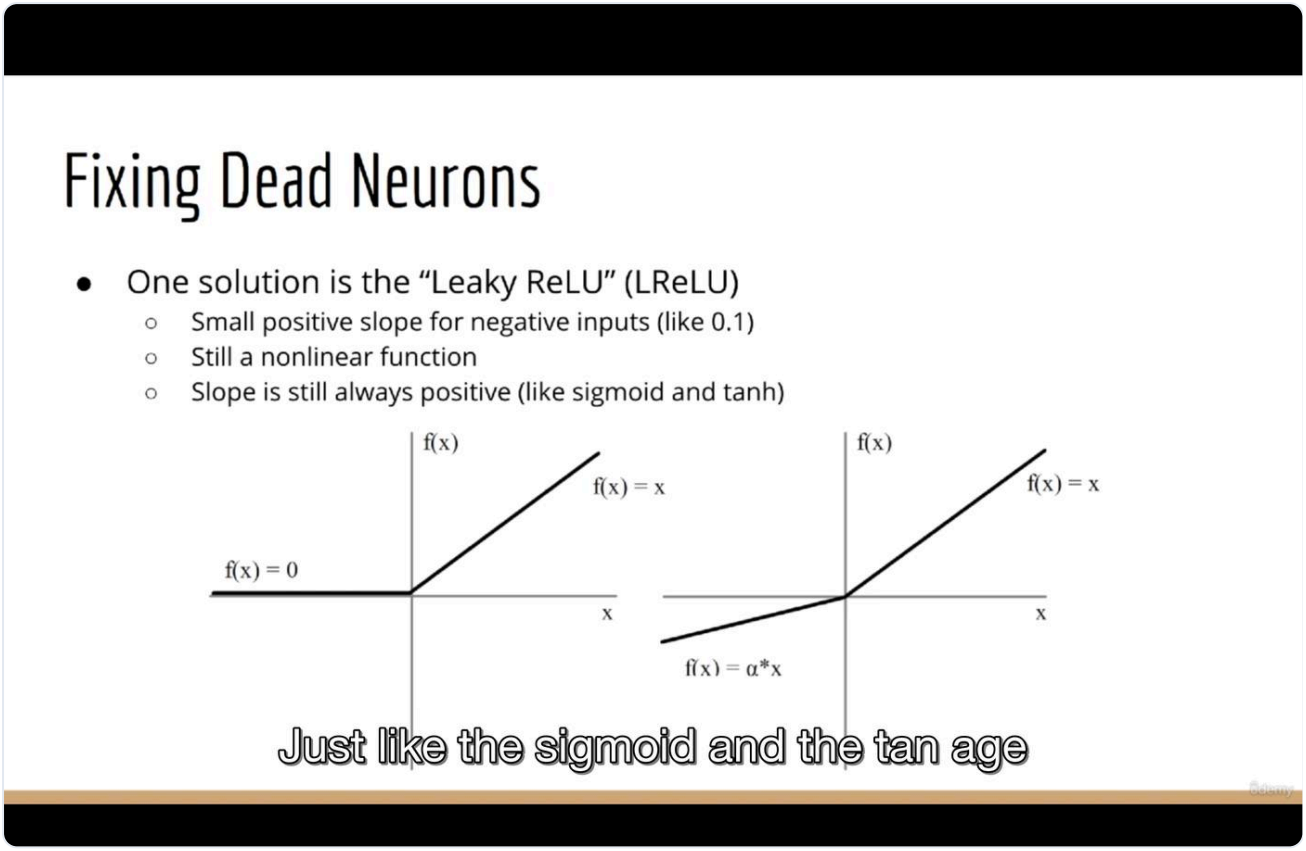


FIGURE 26.18 | Small Positive Slope for Negative Inputs (Like 0.1)

Importantly this is still a nonlinear function so we're able to learn non-linear geometrical patterns using the leaky ReLU your derivatives will always be positive. Just like the sigmoid and the tanh there are other options as well such as the ELU or exponential linear unit which has a more steadily decreasing value on the left side.

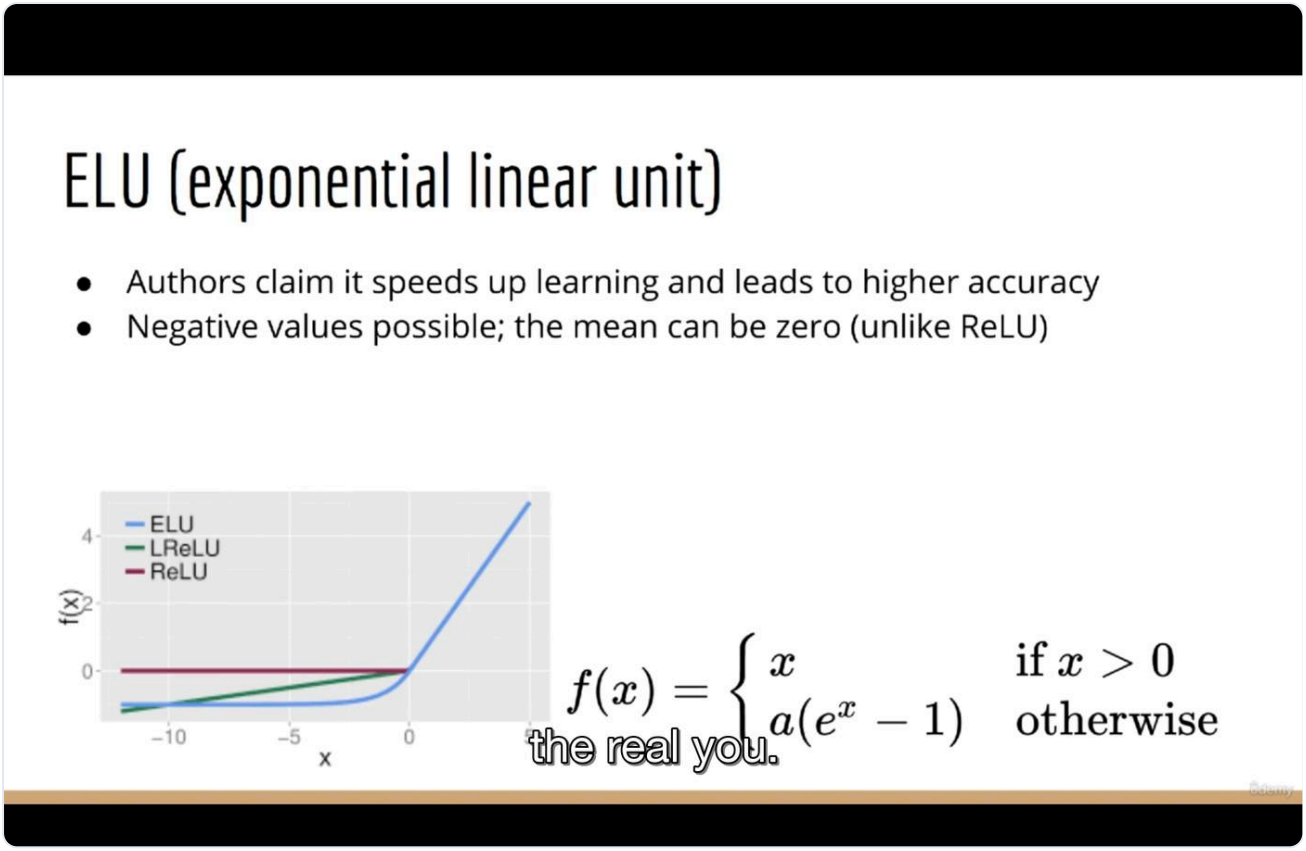


FIGURE 26.19 | ELU (Exponential Linear Unit)

The authors claim that this activation function speeds up learning and leads to higher accuracy than the ReLU. One interesting aspect of the ELU is that it allows its outputs to be negative which goes back to the idea that we like the mean of the values to be close to zero another option which is very similar is the softplus activation which because you're taking the log of the exponent looks very linear when the input is reasonably large now for both of these previous activation functions. There is the vanishing gradient on the left side but we've established that it's not so much of a problem since we know that the ReLU already works and it has gradients that are equal to zero also.

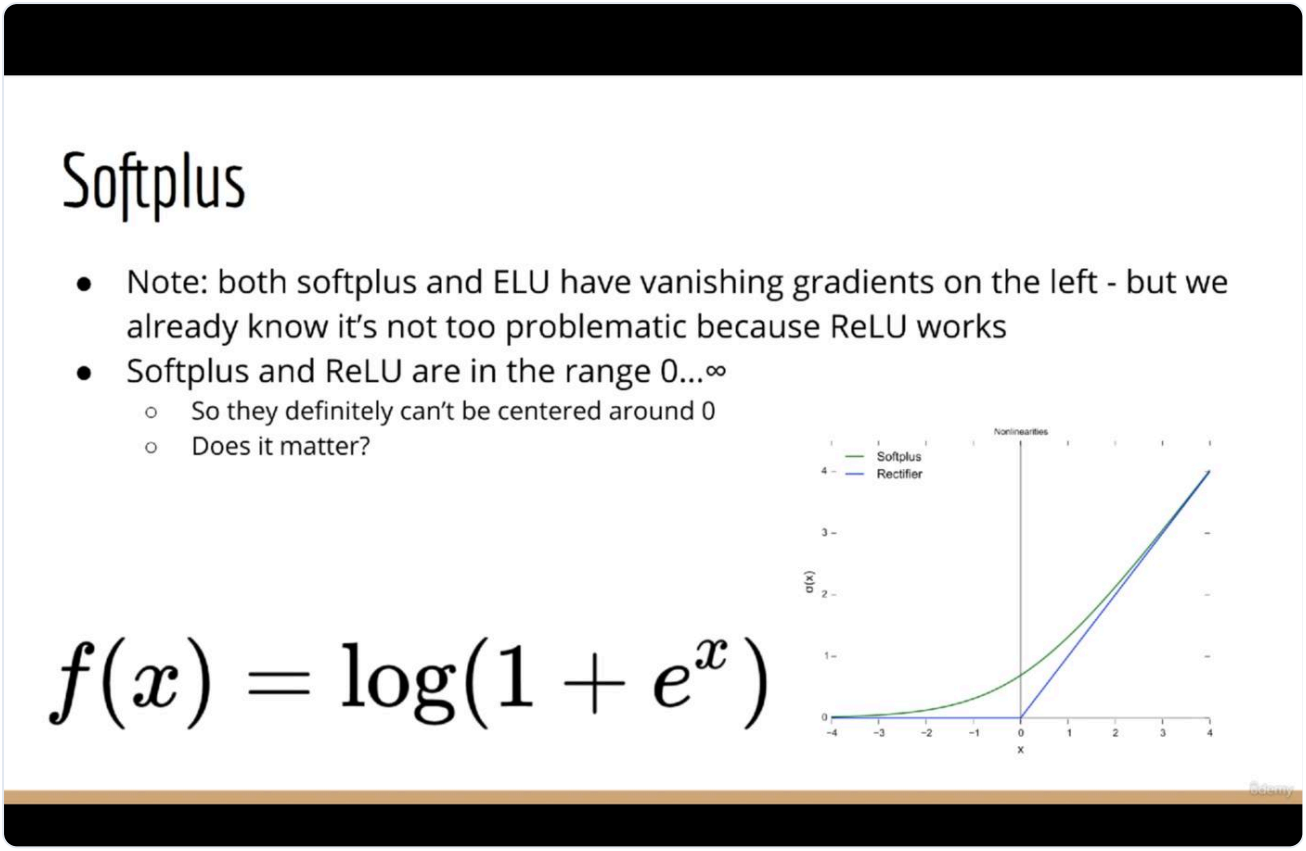


FIGURE 26.20

Although we initially stated that we would like the inputs at each layer to be centered around zero we can see that the ReLU and softplus do not accomplish this for the softplus and the ReLU the minimum value is zero while the maximum value is infinity. This definitely means they won't be centered around zero. So is the ReLU not a good choice in the end.

Default: ReLU

- Most people still use ReLU as a reasonable default
- Sometimes, you'll find that LReLU and ELU offer no benefit
- You just have to try it yourself
- "Machine learning is experimentation, not philosophy"
- Never use your *mind* to guess the output of a computer program
- Instead, use a computer!

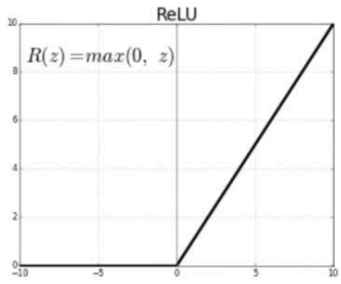


FIGURE 26.21 | Default: ReLU

Biological Plausibility

- ReLU may be even more biologically plausible than sigmoid, but requires a different perspective
- Action potentials by themselves are not distinct wrt stimuli
- Instead, it is the **frequency** of action potentials that changes
  - More stimulation → higher frequency

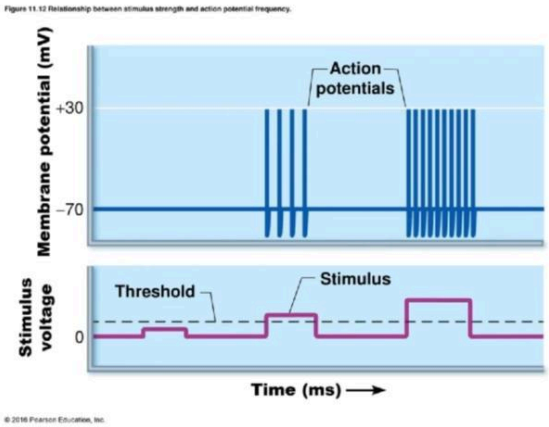
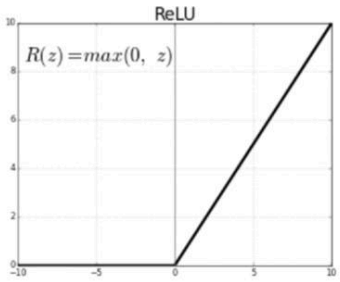


FIGURE 26.22 | Biological Plausibility

Biological Plausibility

- Can think of the ReLU as modeling the action potential **frequency**
- The minimum is zero
  (no action potentials)



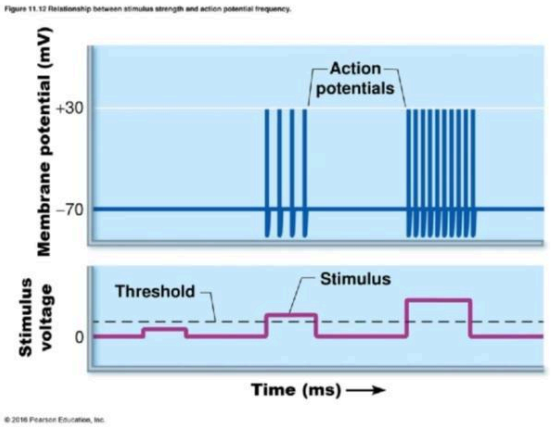


FIGURE 26.23 | Biological Plausibility

Optional: More Biological Plausibility

- Relationship between action potential frequency and stimulus intensity is nonlinear
- Can be modeled with the log() or root functions (e.g. square root)
- Sound / decibel scale provides good intuition

dB	Relative intensity	Example
90	1	Inside a bus
100	10	Electric saw
110	100	Loud orchestra
120	1000	Jet engine
130	10,000	Artillery fire (pain)

FIGURE 26.24 | Optional

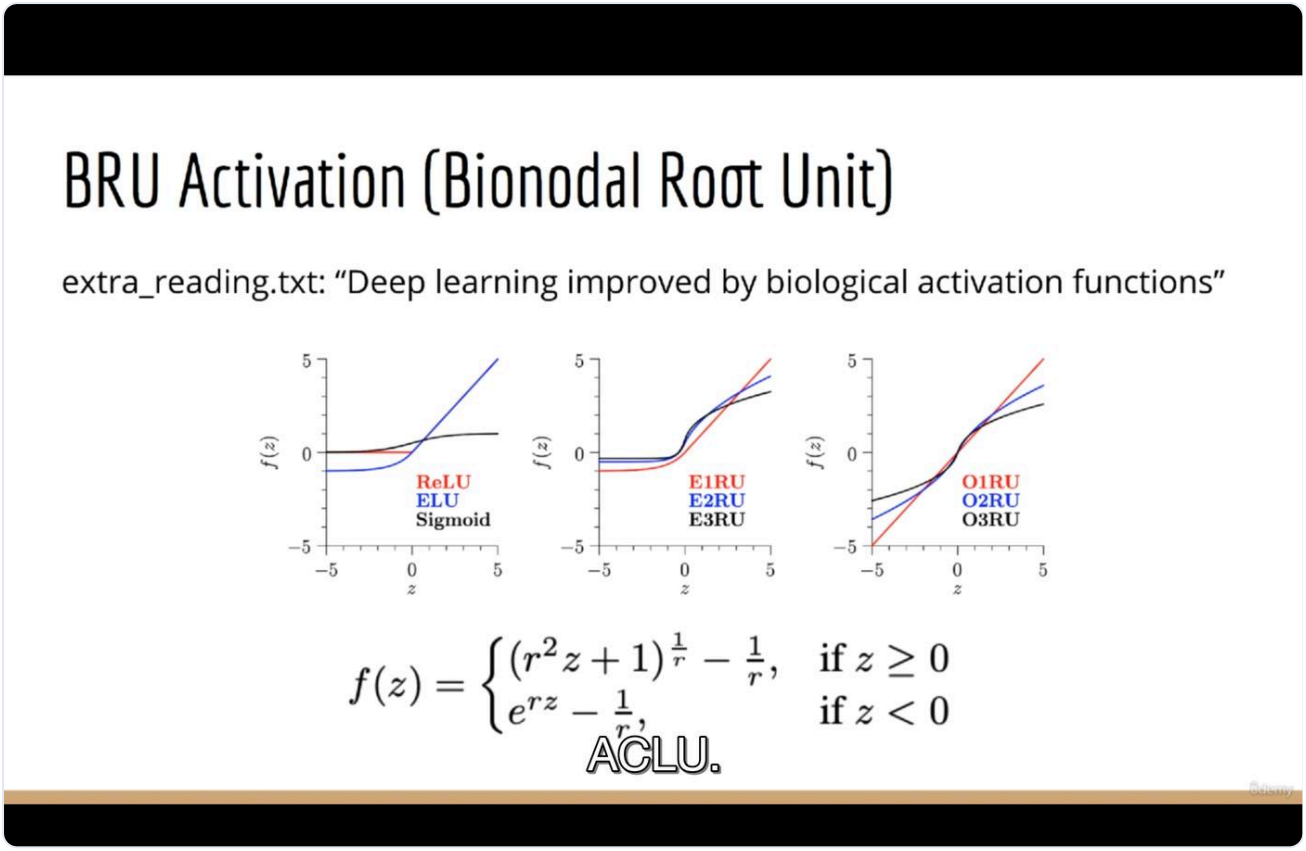


FIGURE 26.25 | BRU Activation (Bionodal Root Unit)

Now despite all this work to find alternatives to the ReLU activation these days most people still use the ReLU as a reasonable default choice. It works well and sometimes you'll find that using other alternatives such as the leaky ReLU or the ELU offer no benefit. Sometimes they do. Which is why you always have to experiment for yourself. My motto which a lot of my students are tired of hearing by this point is that machine learning is experimentation and not philosophy. Never use your mind to try and predict the outcome of a computer program. If you have a computer that is always the suboptimal course of action why not simply run the computer program with a computer.

Your mind is not suitable for running computer programs but computers are therefore follow the rule. Don't use philosophy use experimentation interestingly some researchers have talked about the biological plausibility of the ReLU activation function. In fact it may even be more biologically plausible than the sigmoid to understand this you have to understand a little more about how action potentials encode information. What is the difference between an action potential. When you hear a quiet sound versus an action potential. When you hear a loud sound. The answer is there is no difference. An action potential is just an action potential. The key really is in the frequency of action potentials.

When you hear a very quiet sound your neurons are only activated a little bit so you'll get some action potentials but they won't be very frequent. If you hear a very loud sound like say you're at a concert or a party then your neurons are getting lots of stimulation. The action potentials are going to be very frequent. In other words closer together in time. So what we're saying is the more intense a stimulus is the higher the frequency of the action potentials. We call this fre-

quency coding what does this mean in terms of the ReLU. Well you can think of it as the ReLU is just encoding the actual potential frequency itself. Of course the minimum frequency is zero and it's not possible to have a negative frequency.

That's why the minimum value of the ReLU is zero. Then as the inputs into the neuron get larger because they are being stimulated more the receiving neuron also gets stimulated more increasing its frequency now. One thing to note is that we can go even deeper than this although this is not part of mainstream deep learning quite yet. As a side note if you don't want to listen to this part it's not required. It's something that I think is interesting to those of us who are interested in modelling brains. But if you are a hardcore statistician then perhaps you may think differently what we know based on neuroscience is that unlike the ReLU activation action potential frequency is not linearly related to its input stimuli.

Instead we have a nonlinear relationship. It can be modeled using the log function or a root function like the square root a good way to understand this is with sound something that is twice as loud does not sound twice as loud. Using measurements of sound intensity this is the intuition behind the decibel scale. As you recall the decibel scale is a logarithmic scale. For example a 90 decibels sound would be like sitting inside a moving bus 100 decibels sound would be like standing by an electric saw at 110 decibels sound would be like a loud orchestra of 120 decibels sound would be like standing near a jet engine. A 130 decibels sound would be like being close to artillery fire and this value is the threshold of pain.

So when you hear a sound this loud it actually hurts physically. To put that in perspective 100 decibels is ten times more powerful than 90 decibels 110 decibels is 100 times more powerful 120 decibels is 1000 times more powerful and 130 decibels is ten thousand times more powerful some work has been done to experiment with activation functions that more accurately model the frequency characteristics of real neurons such as the bee are you. But as a whole they haven't yet caught on in the Deep Learning Community. I've attached the paper on the bee are you to extra reading dot txt in case you want to check it out. The author reports that the BYU activation function led to better results than the ReLU and the ELU.

So it may be something worth trying in your own project.

LECTURE 27

The Softmax Function

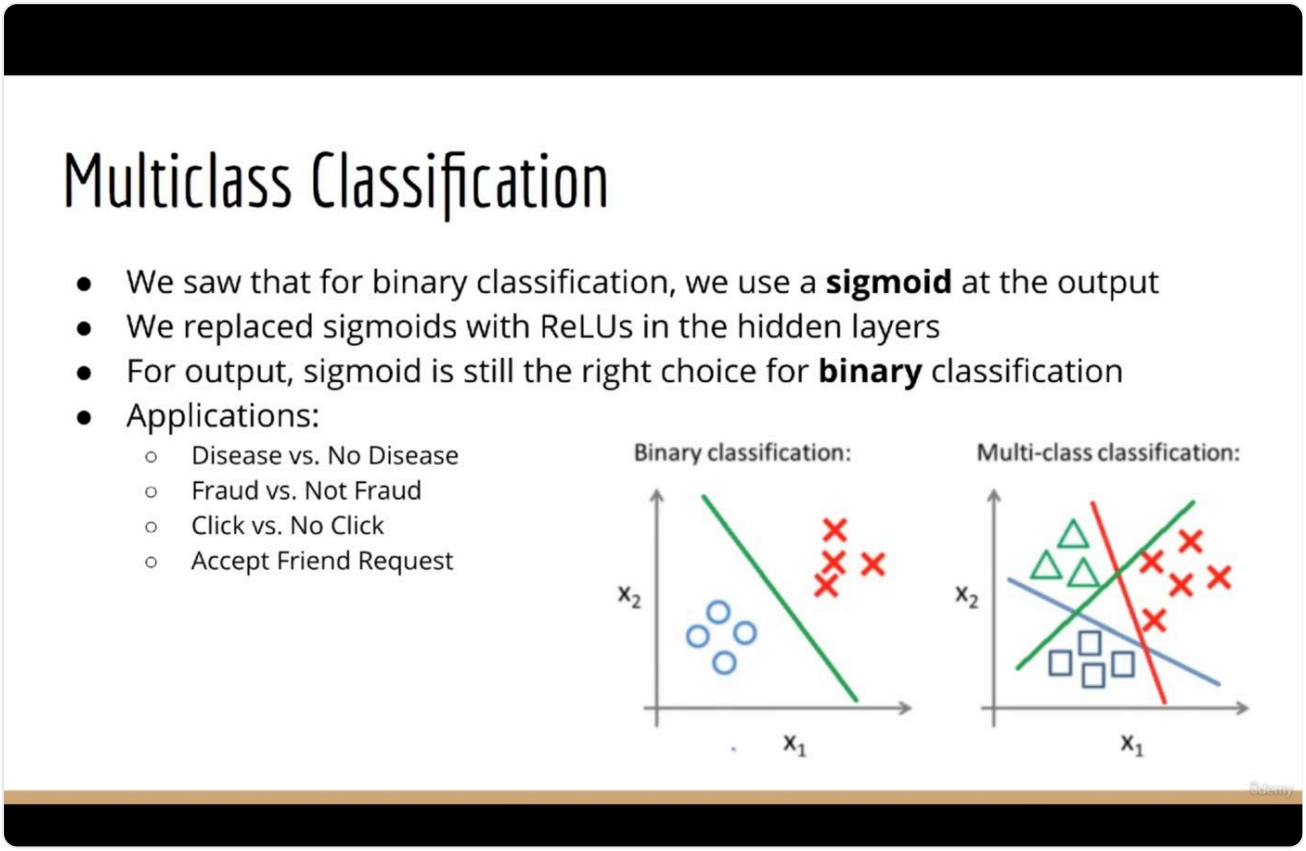


FIGURE 27.1

In this lecture we are going to discuss another very important topic multi class classification previously we discuss the binary classification which used a sigmoid at the output. We also had sigmoid for the hidden layer activation functions but we just got rid of that in favor of the ReLU which tends to work better for the output. However if we are doing binary classification the sigmoid is still the right choice binary classification has many applications. We can predict disease or notices is the user fraudulent or not. Well the user click my ad or not. Will the user accept a friend request or not. And so on and so forth. But there are situations that binary classification cannot handle such as when we might have multiple categorical outcomes let's think of some examples.

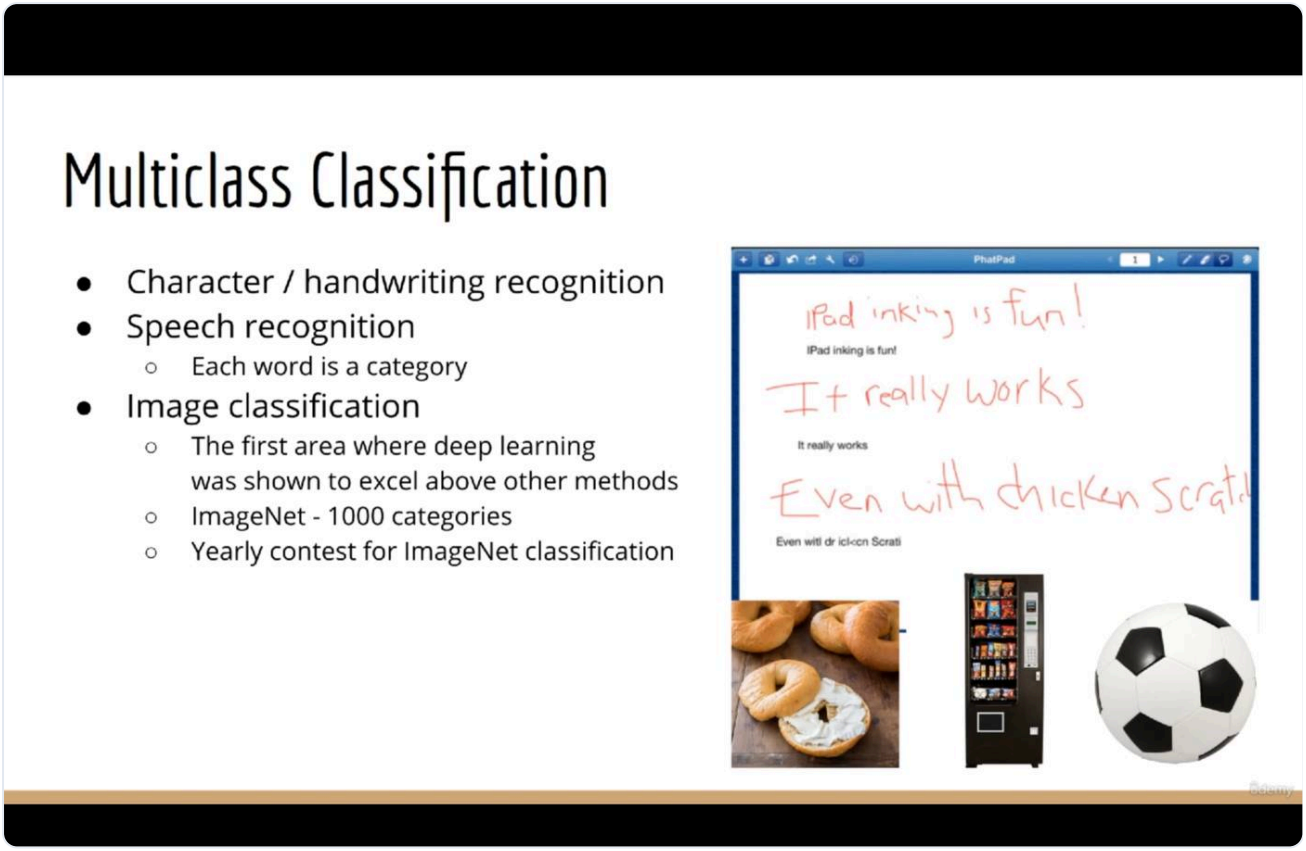


FIGURE 27.2

One example is optical character recognition or handwriting recognition. If we want to recognize alphanumeric characters then there are at least 36 possibilities 10 digits and 26 letters. What about speech recognition. If we want to recognize words then each word counts as a separate category image classification is another one and for a long time this was the standard benchmark which proved that deep learning works better than other machine learning models. You may have heard of a famous dataset called the ImageNet dataset which is made up of millions of images in the thousand categories such as bagel vending machine and soccer ball. Every year there is a contest to see who can build the best ImageNet classifier and of course these are all deep learning models.

So how does multi class classification work. Well suppose we have just calculated the activation at the final layer of our neural network but have not yet applied the activation function. So we have a superscript l equal to w superscript l transpose dot z superscript l minus 1.

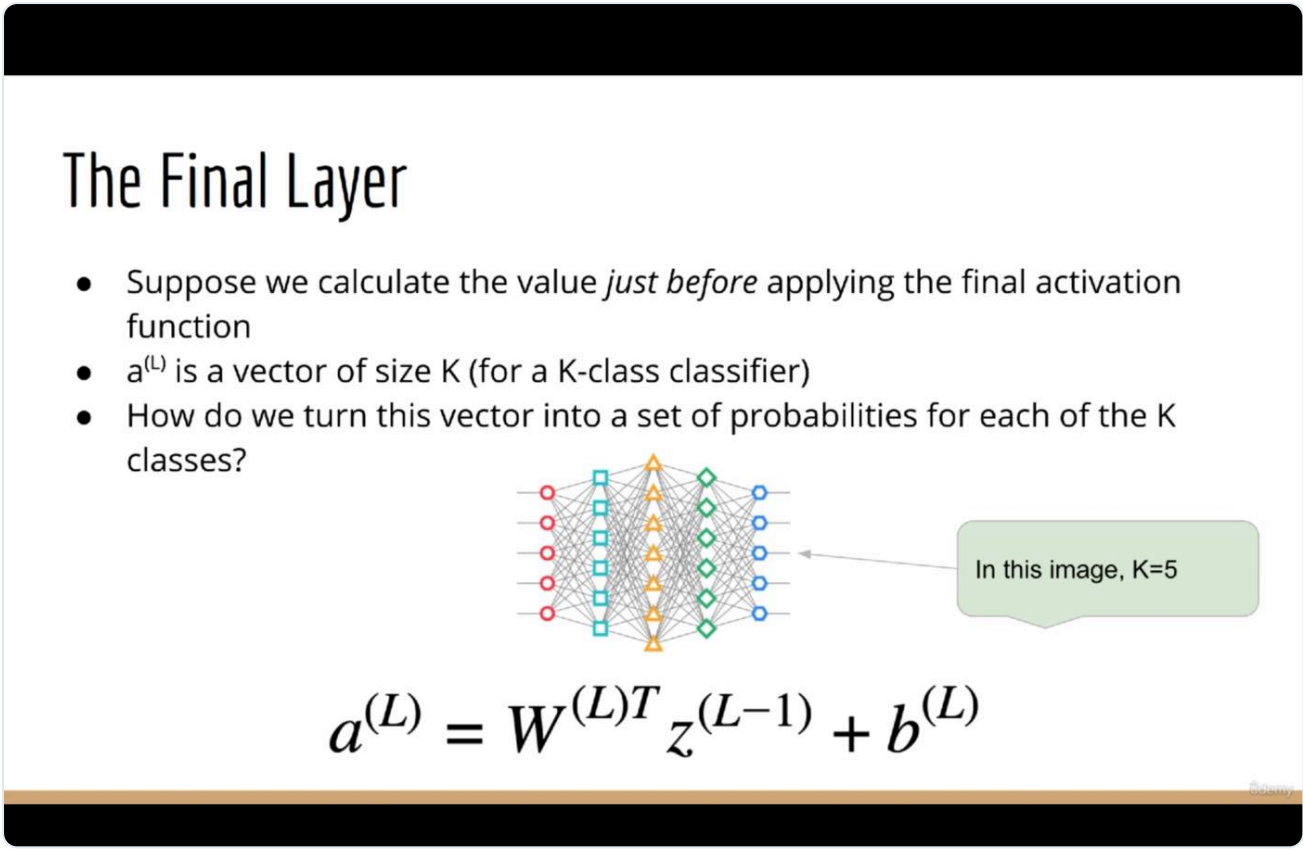


FIGURE 27.3 | The Final Layer

So that's the Z at the previous layer plus b superscript l it's important to remember that if the layer size is k then a superscript l is a vector of size k what we would like to do is map these values to probabilities for each of the K possible output categories. Okay so that's your first lesson here. If we have K possible outcomes then we should have K output nodes. The question now is what function can actually map the vector a superscript l 2 probability values let's think about what the requirements are for these probabilities.

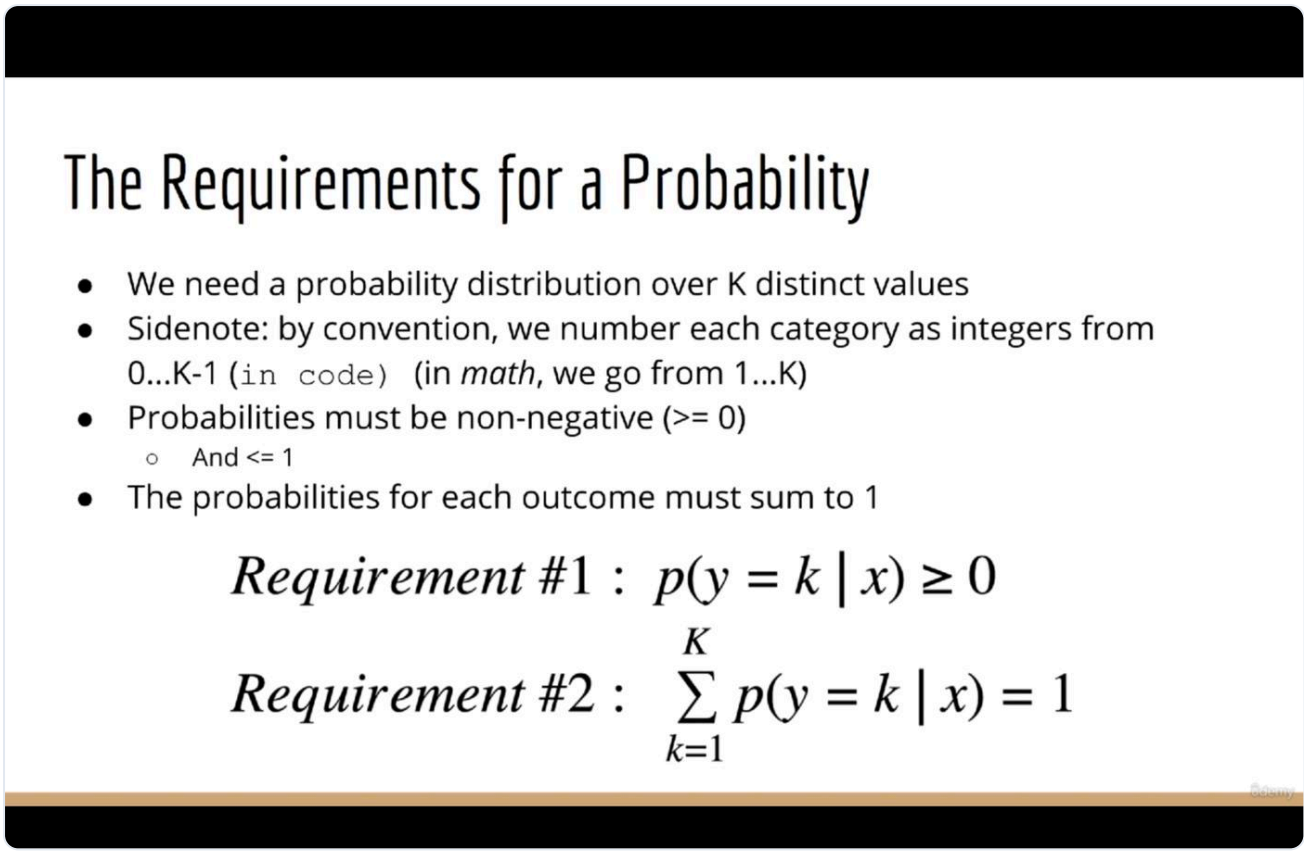


FIGURE 27.4 | The Requirements for a Probability

We need a probability distribution over a K distinct values. As a side note we assign these values the integers 0 to K minus 1 first since they are probabilities. They must be non negative. In other words they must be greater than or equal to zero. But there is also an upper limit. The maximum value of a probability is 1. But more importantly for this to be a proper probability distribution all the possible different values must sum to 1. So the second requirement is that all the probabilities must sum to 1 luckily there is a function that accomplishes exactly this. It is called the softmax function first let's drop the superscript l on the activation a for now so we can just say a is a vector of size k then the softmax of this vector A is the exponential of a divided by the sum of the exponential of a across each component.

As you can see this satisfies both of our requirements. The exponential of any number is always positive so therefore each probability must be non negative.

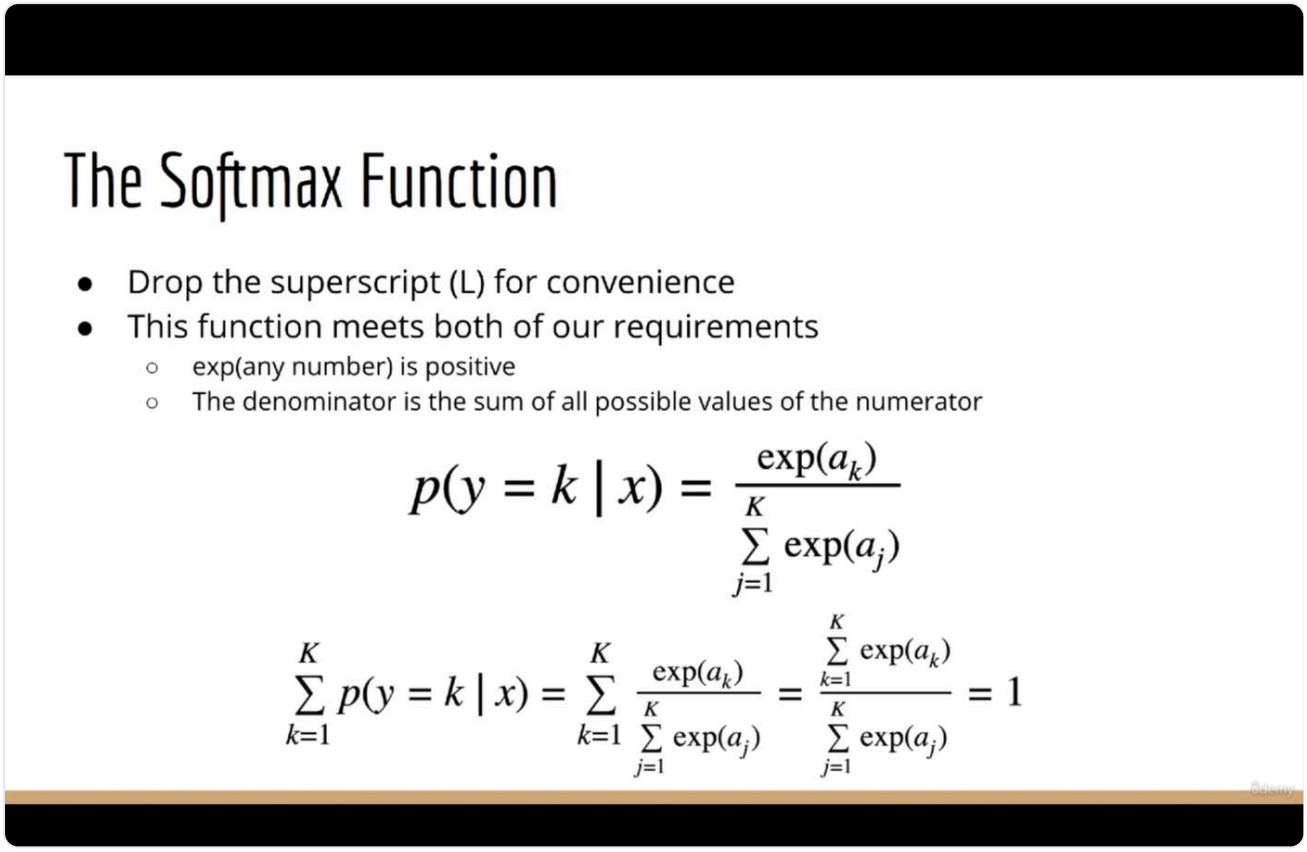


FIGURE 27.5 | The Softmax Function

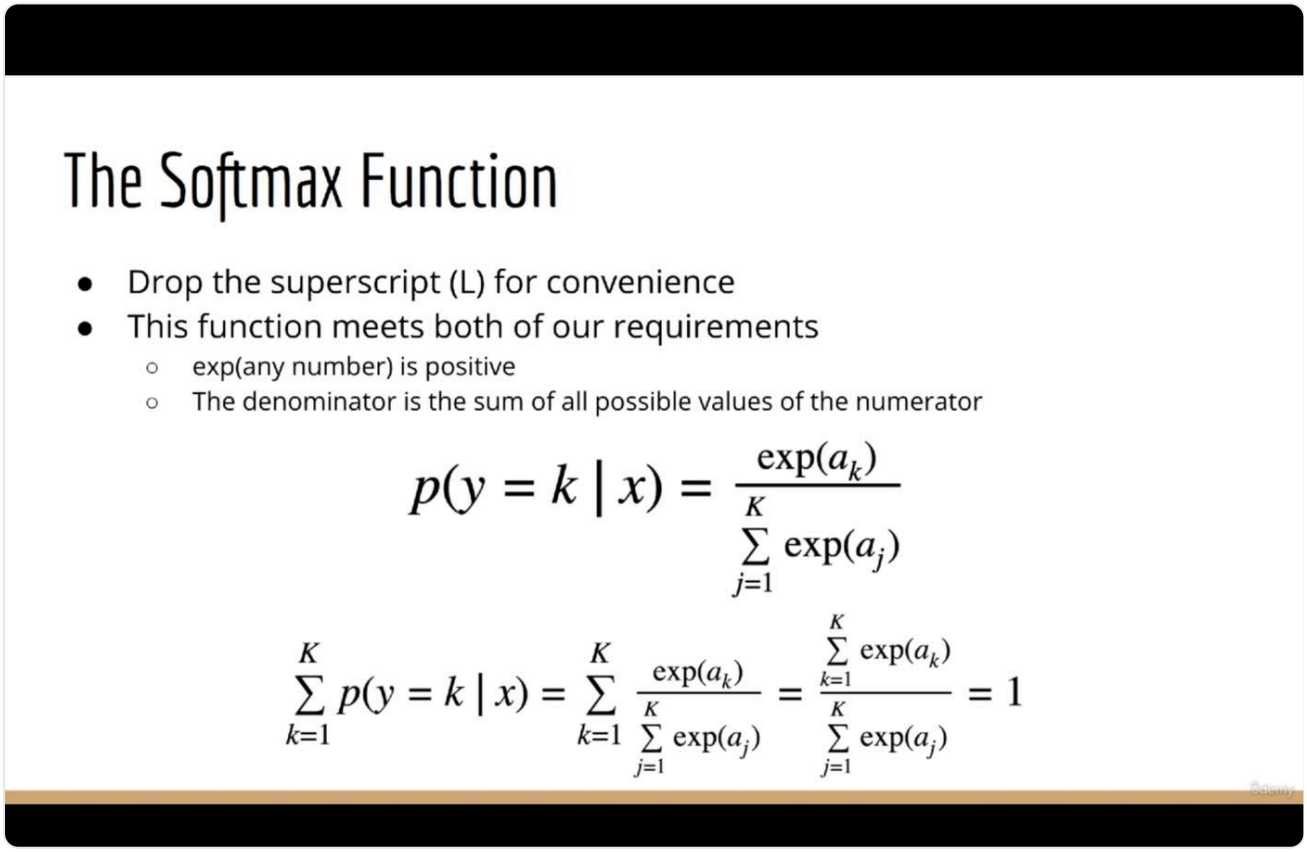


FIGURE 27.6 | The Softmax Function

Second because the denominator is just the sum of all the possible values of the numerator. Then if we sum all the possible values of the numerator we just get the same sum and therefore we get the same sum on the top and the bottom and they cancel out to give us 1 now that we know how the softmax works. Let's answer the most important question how do we use it in PyTorch. Well just like most other functions we've applied so far it's very simple. There is an object we instantiate called softmax.

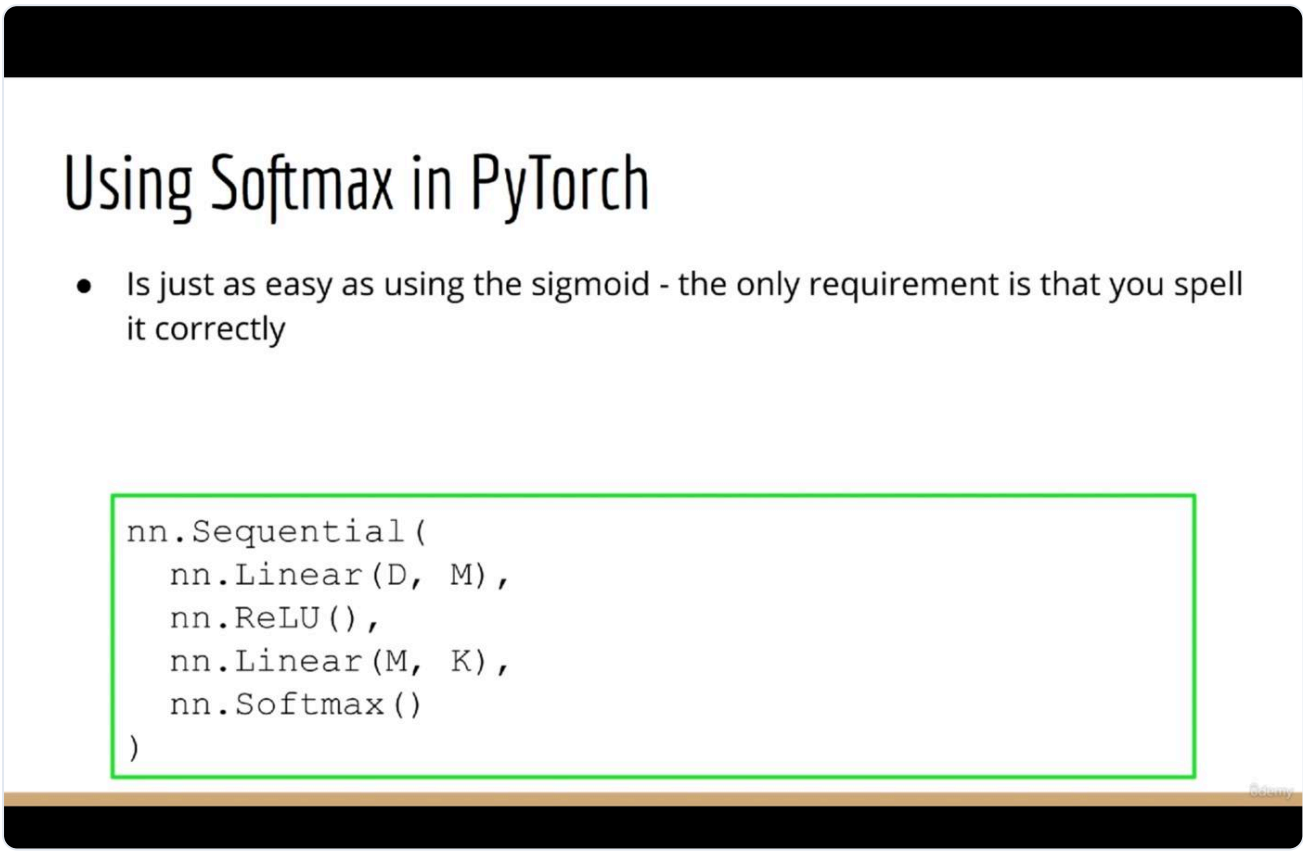


FIGURE 27.7 | Using Softmax in Pylorch

In other words the only requirement is that you spell it correctly. Of course you can always implement the softmax yourself but in PyTorch there's no need.

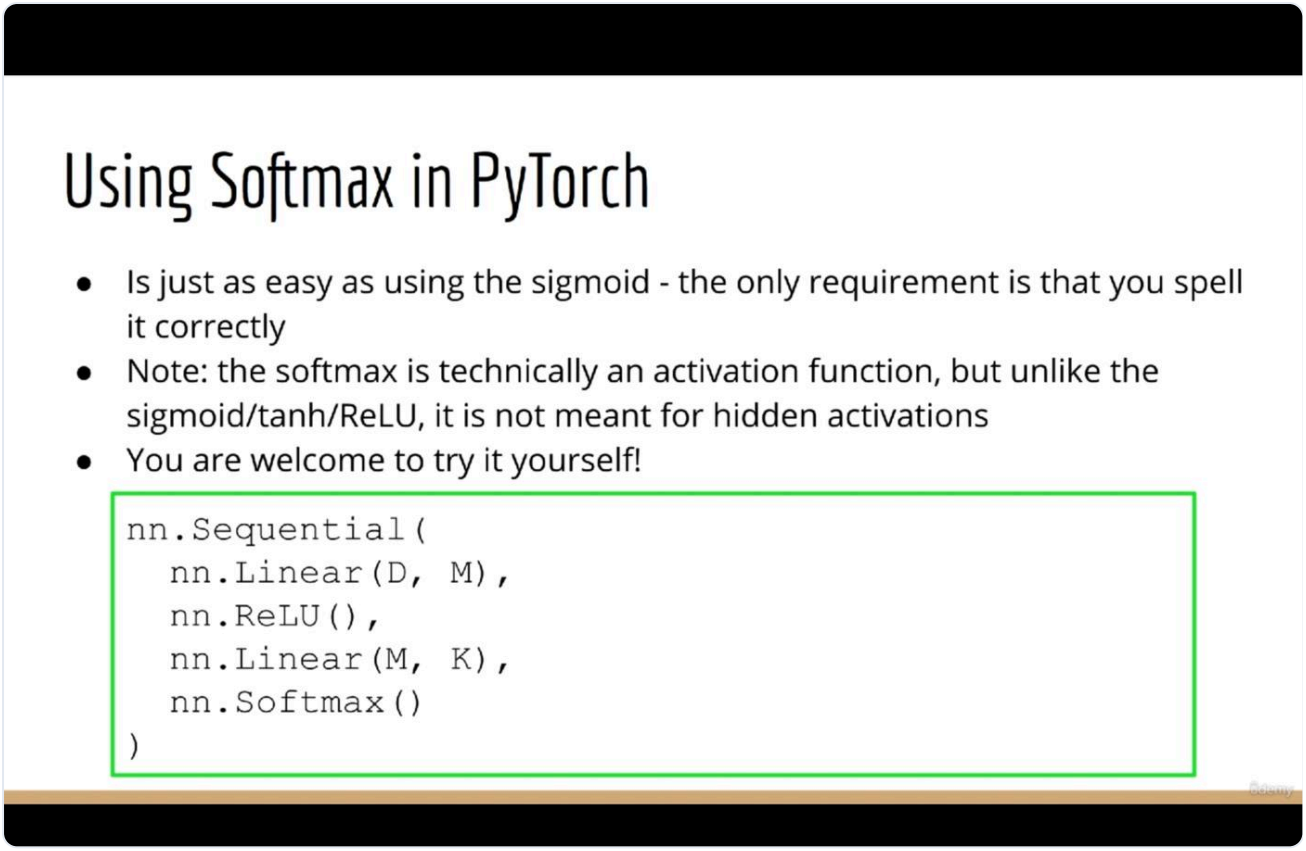


FIGURE 27.8 | Using Softmax in Pylorch

As a side note the softmax is considered an activation function but unlike the ReLU sigmoid and tanh it is not meant for a hidden layer activations if you want to try using the softmax as a hidden layer activation you are most welcome to but you'll generally find that it doesn't work that well. So the softmax is technically an activation function but we normally only use it when we're trying to get an output probability from a vector of activation values such as at the end of a neural network now that you know how to use the softmax in PyTorch. The next thing I'm going to tell you is to not use it just like we learned earlier with the sigmoid and the binary cross entropy PyTorch combines those two functions into a single loss for numerical stability.

So PyTorch is going to combine the softmax function with the cross entropy loss into this single loss which is simply called cross entropy loss PyTorch does in fact have a standalone categorical loss which is called NLL loss but that is outside the scope of this course. In the example below we've implemented logistic regression but instead of being a binary logistic regression it's now a multi class logistic regression that predicts k different categories. As you can see the model is still just a basic linear layer.

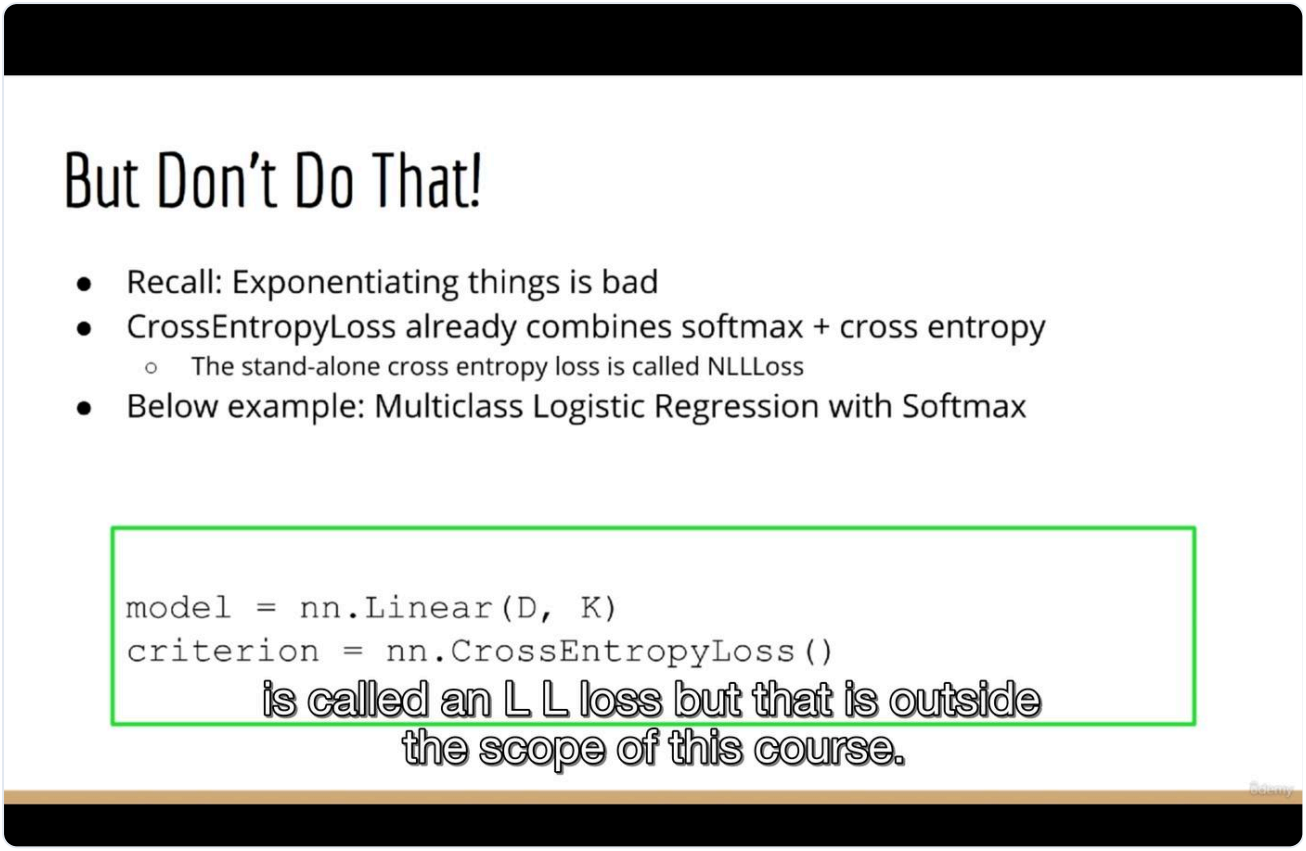


FIGURE 27.9 | The Stand-alone Cross Entropy Loss Is Called NLLLoss

All the magic really happens inside the cross entropy loss object. The last thing I want to do in this lecture is summarize the three types of tasks we've encountered so far that neural nets can perform and their corresponding activation functions two of them you already saw in the previous section of this course. And now we have a third. So first we have a regression where we use it no activation function at all. Sometimes we also call this the identity function which is just a fancy way of saying a function that returns its input. Second we have binary classification where we use the sigmoid activation function using the sigmoid. We only need one output node because since we know the probability of the output being one the probability of the output being 0 is just 1 minus that.

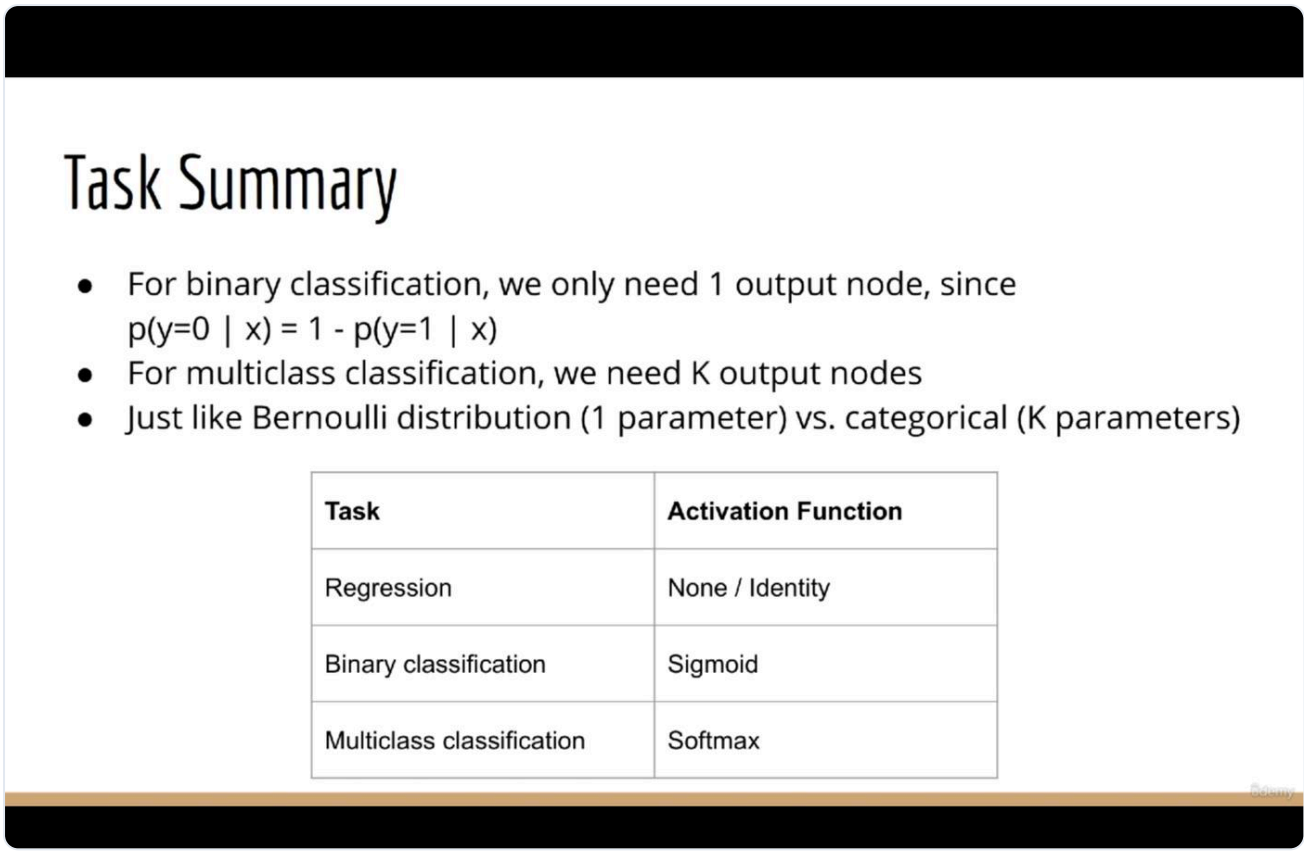


FIGURE 27.10

This is the same type of thing you see when you're looking at the Bernoulli distribution versus the categorical distribution the Bernoulli distribution is a categorical distribution where there can only be two outcomes. Thus it only has one parameter. The categorical distribution on the other hand if it has K outcomes it also has K parameters. And that brings us to our third scenario which is multi class classification. For this we use the softmax activation function one important thing to mention that sometimes confuses people is that these three activation functions apply whether you are using a feed forward neural network a linear model or even a more complicated network like a CNN or an RNN.

It doesn't matter what kind of model you have these three activation functions always correspond to these three tasks. So for example if I don't want to use a neural network but just a simple linear classifier for multiple classes then I would have just a single dense layer with a softmax activation.

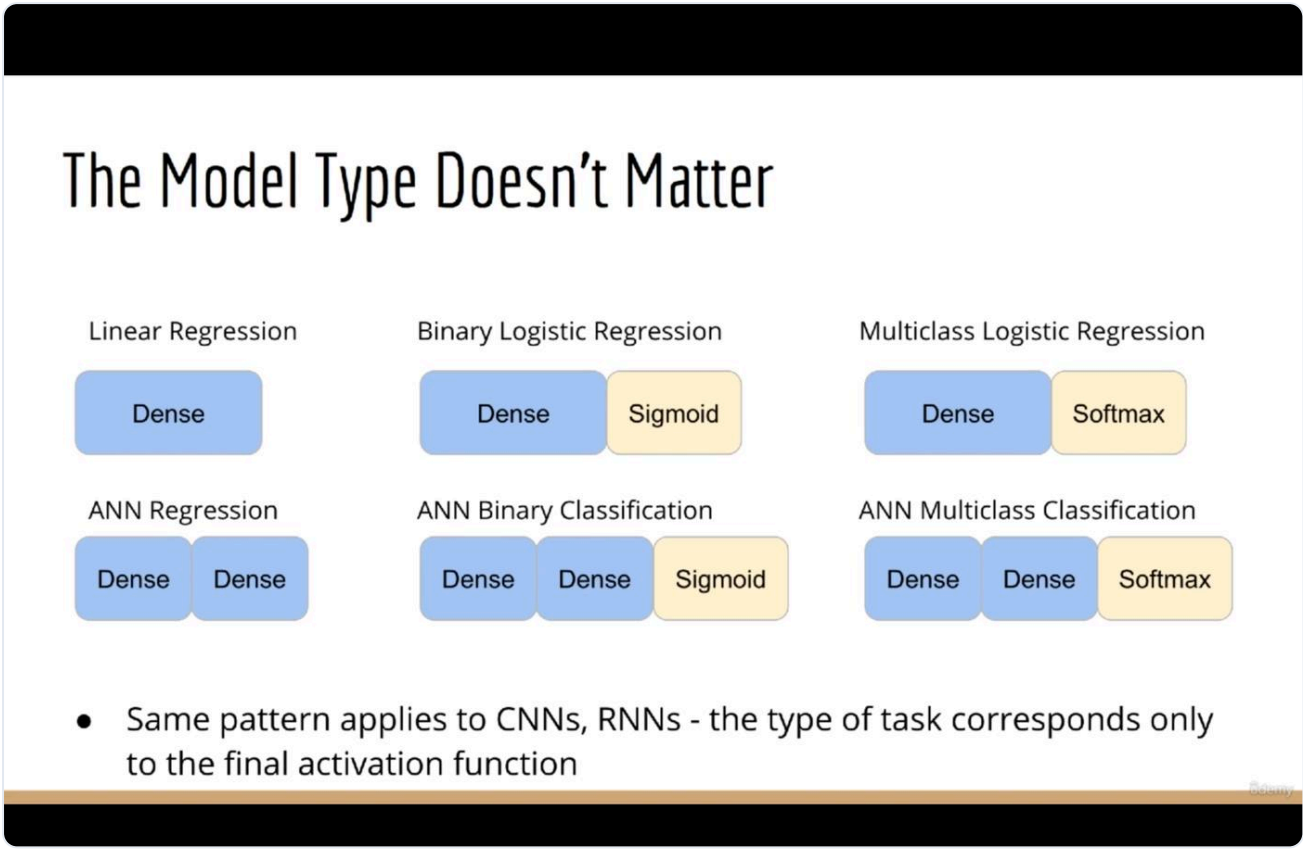


FIGURE 27.11 | The Model Type Doesn't Matter

This is still called logistic regression by the way. It's just not binary logistic regression. Instead it's called multi class logistic regression it's important to remember that what came before the final layer in the model is not related to the task. You can have a CNN or an RNN or any other kind of complicated neural network. It is still the case that the final layer of the network will be a dense layer with one of these three activation functions corresponding to the task being done. Another important thing to remember is that the softmax function is more general whereas the sigmoid can only handle binary classification.

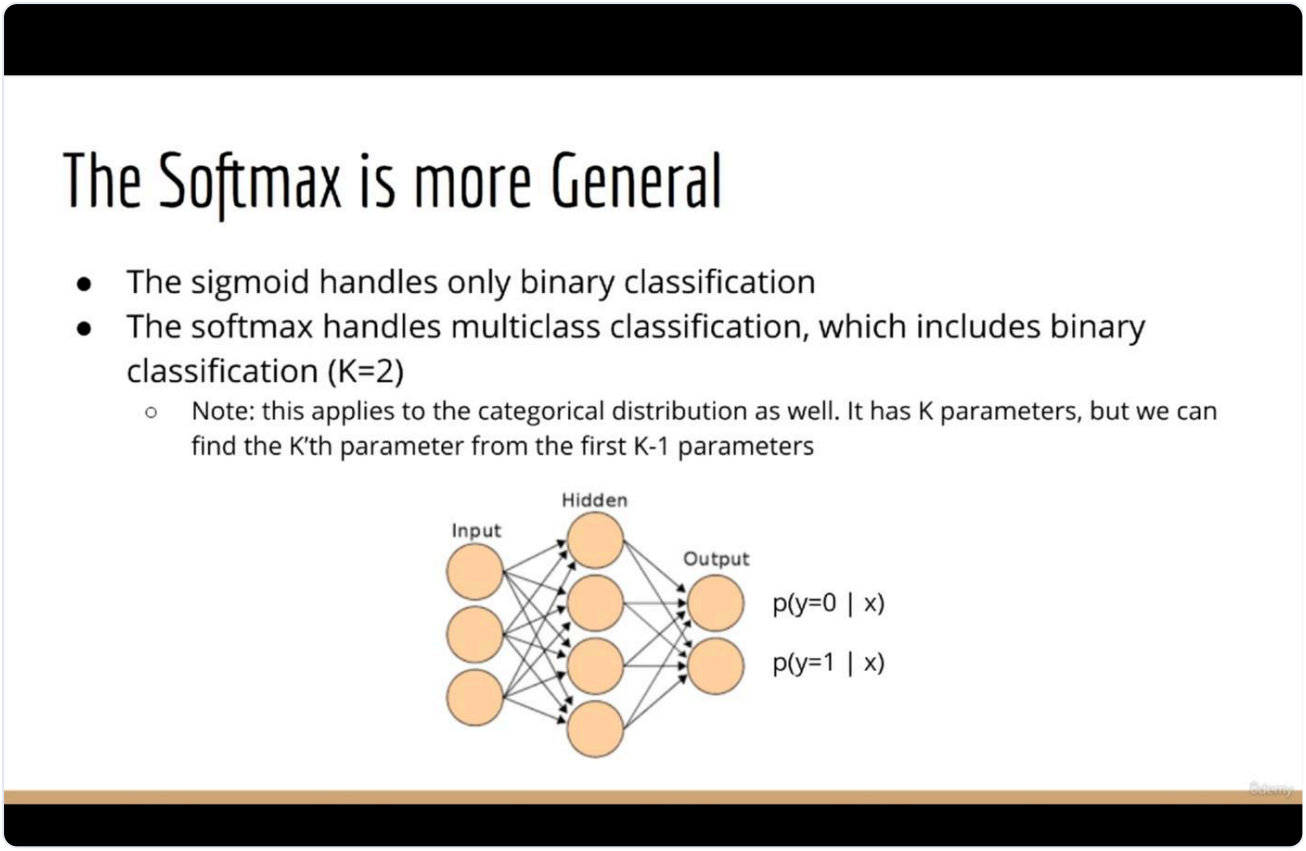


FIGURE 27.12 | The Softmax Is More General

The softmax function can handle multi class classification which includes binary classification. So it's possible if you want to just use the softmax for all types of classification. If you set k equals 2 then you're doing binary classification. It's just that in this scenario your outputs are a bit redundant because you already know that they both sum to one and hence only one of them is really needed.

LECTURE 28

Images as Input to Neural Networks

In this lecture we are going to switch gears a bit and discuss the kind of data we'll be working with next. Previously we discussed topics related to the model itself how a neural net will get structured and the different kinds of functions involved. That's the model. But just as important is the data where deep learning and neural networks really shine is on unstructured data such as images sound and text.

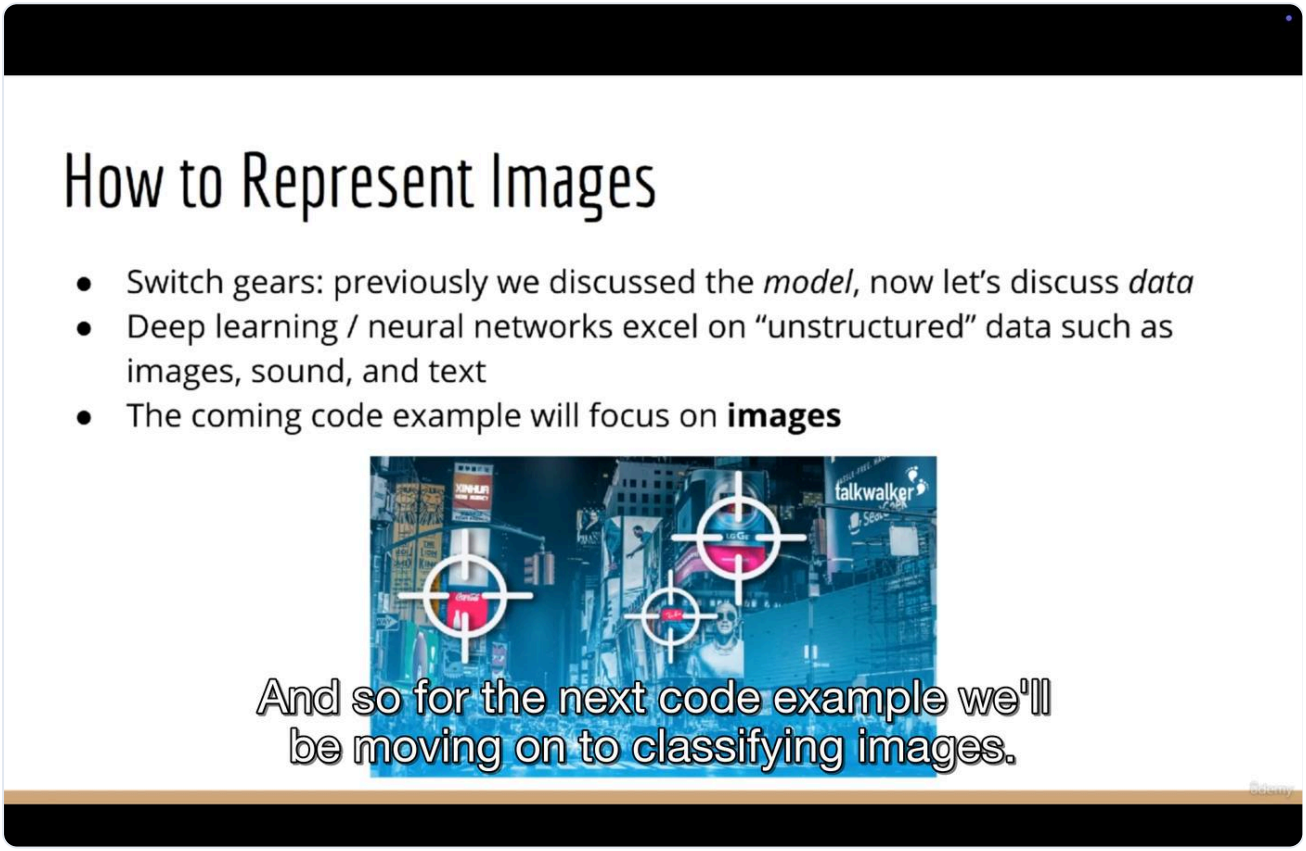


FIGURE 28.1 | How to Represent Images

And so for the next code example we'll be moving on to classifying images. Of course this presents a challenge. Previously you learn my rule.

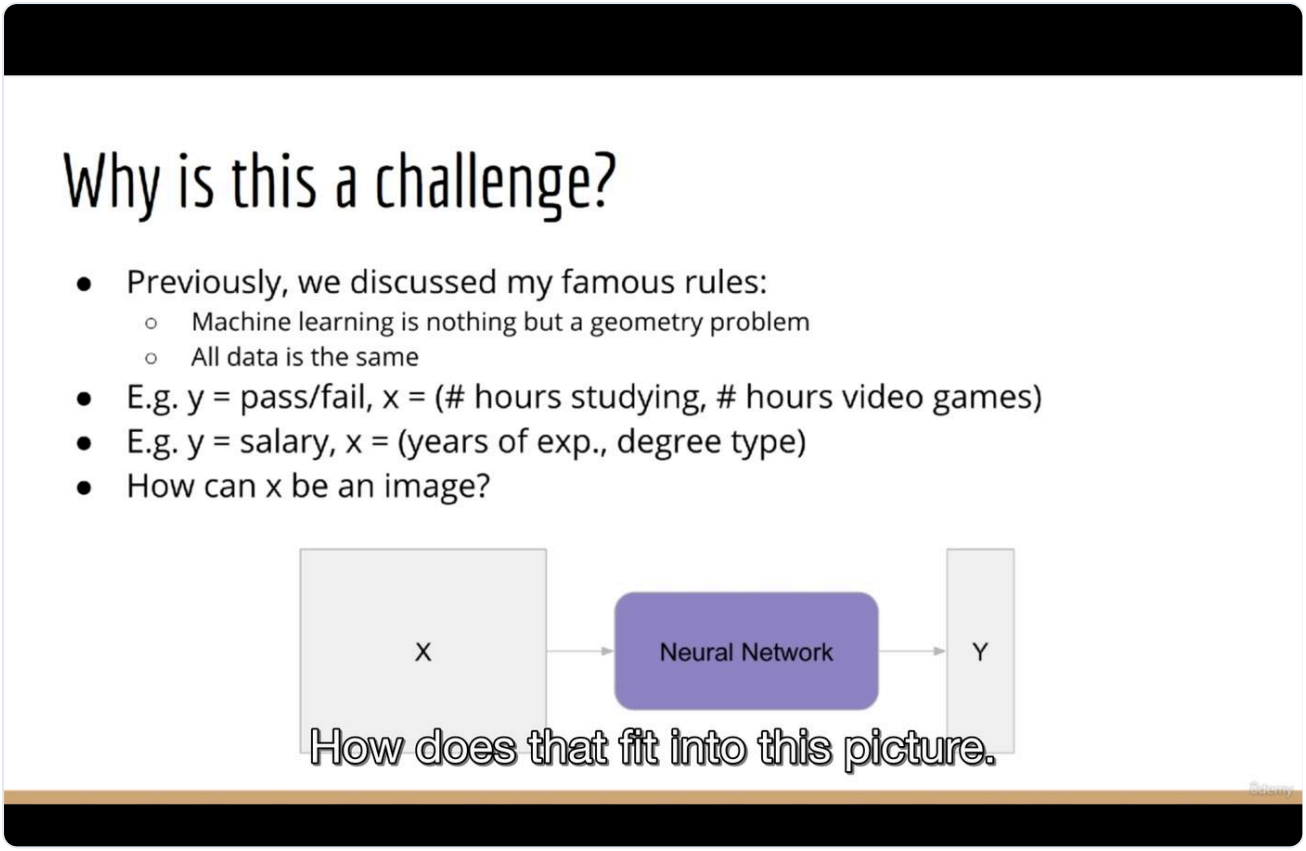


FIGURE 28.2 | Why Is This a Challenge?

Machine learning is nothing but a geometry problem and all data is the same. We looked at some pretty simple examples with tabular data such as predicting whether a student would pass or fail an exam based on how many hours they spent studying and how many hours they spent playing video games. Another example was predicting salary based on factors such as years of experience and degree type. The question now is if your input data X is an image. How does that fit into this picture. As usual we're going to start by doing the dumbest thing possible. But first we have to understand how images are represented in a computer in the first place. So if we just look at an image let's say the famous Lena image we can see that the image has two properties height and width.

Therefore your first thought might be let's store this in a matrix. The next question is what do we store in this matrix. Well suppose I have a matrix A and I'm looking at the position A i j. This refers to the ith row and the jth column of the image. You might think OK so A i j should store the color that appears at this exact position of the image at position i j.

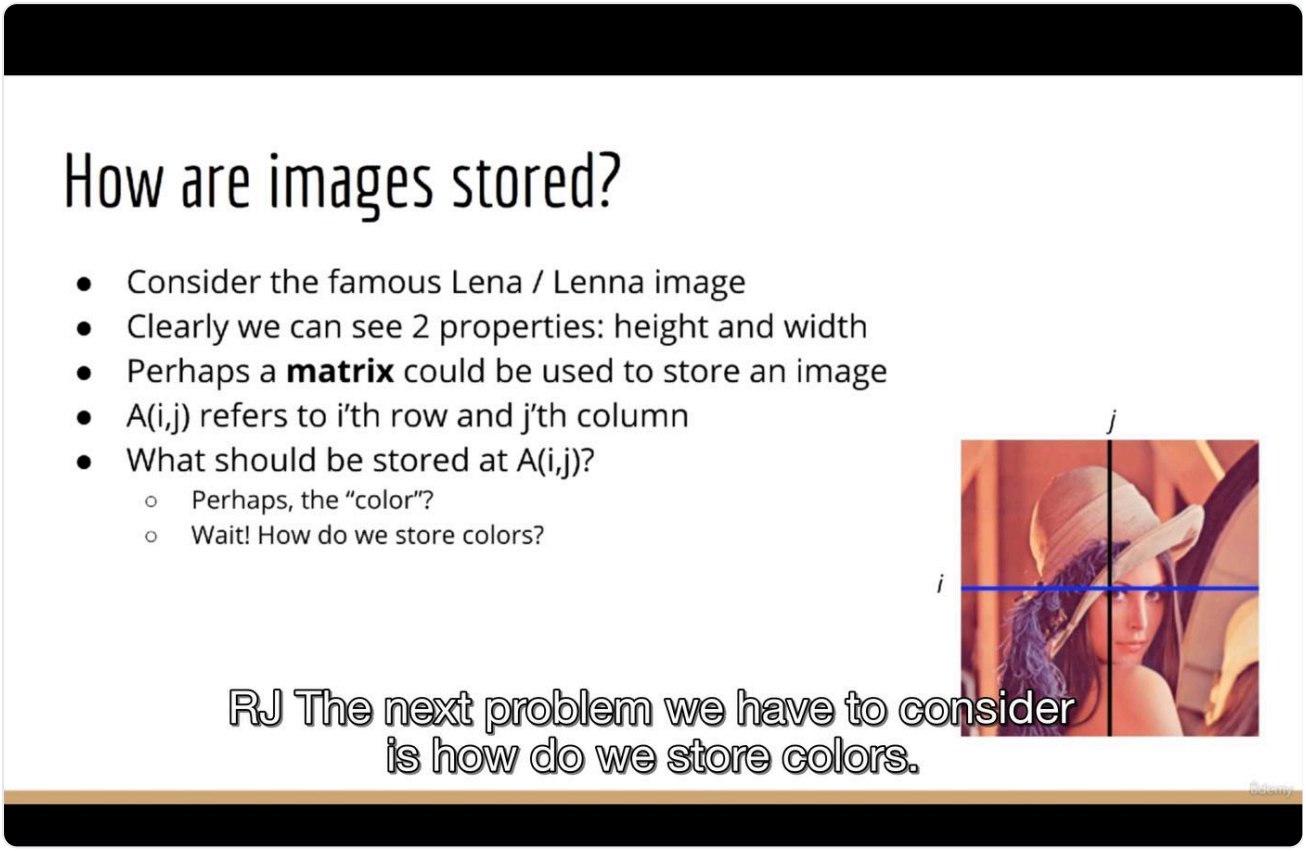


FIGURE 28.3 | How Are Images Stored?

The next problem we have to consider is how do we store colors. In fact there are quite a few ways to store colors and computers but in this course will only discuss the most common scheme called RGB which stands for red green blue. Now I want you to think back to kindergarten. Yes all the way back to kindergarten. I bet you didn't know that what you learned there would come in handy now if you forgot what you learned in kindergarten. Well I'm not sure if I can help you.

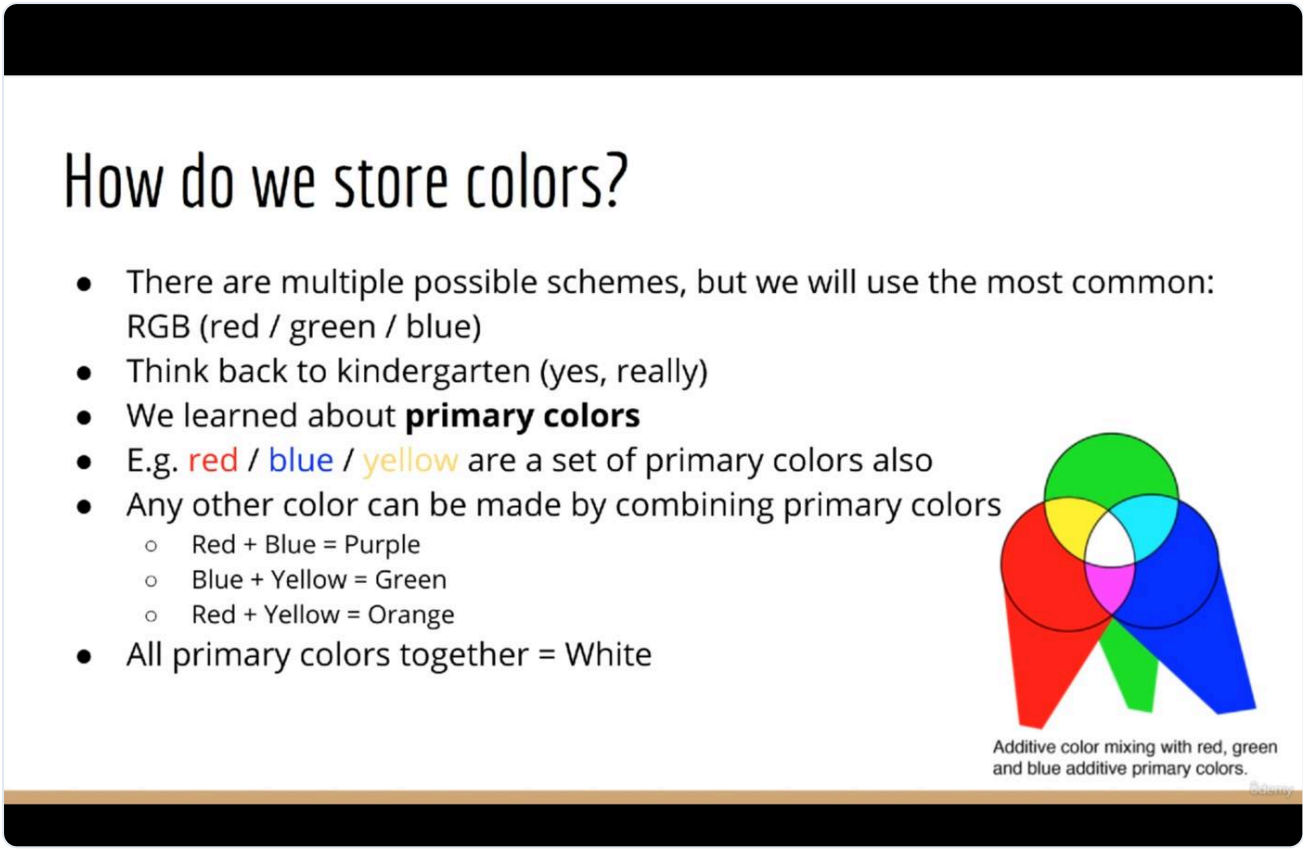


FIGURE 28.4 | How Do We Store Colors?

You recall that there's this idea of primary colors primary colors or a set of colors from which we can form other colors by combining them a common set of Primary Colors is red blue and yellow although red green and blue also makes a set of primary colors. So what do you get if you mix red and blue you get purple. What do you get if he makes a blue and yellow you get green. What happens if you mix red and yellow. You get orange. Now this one you might not know but if you mix all the primary colors together you get white in fact from the primary colors it's possible to combine them in different proportions to get any color. So how do we represent colors.

Well in fact it's not just a number. It's a set of three numbers.

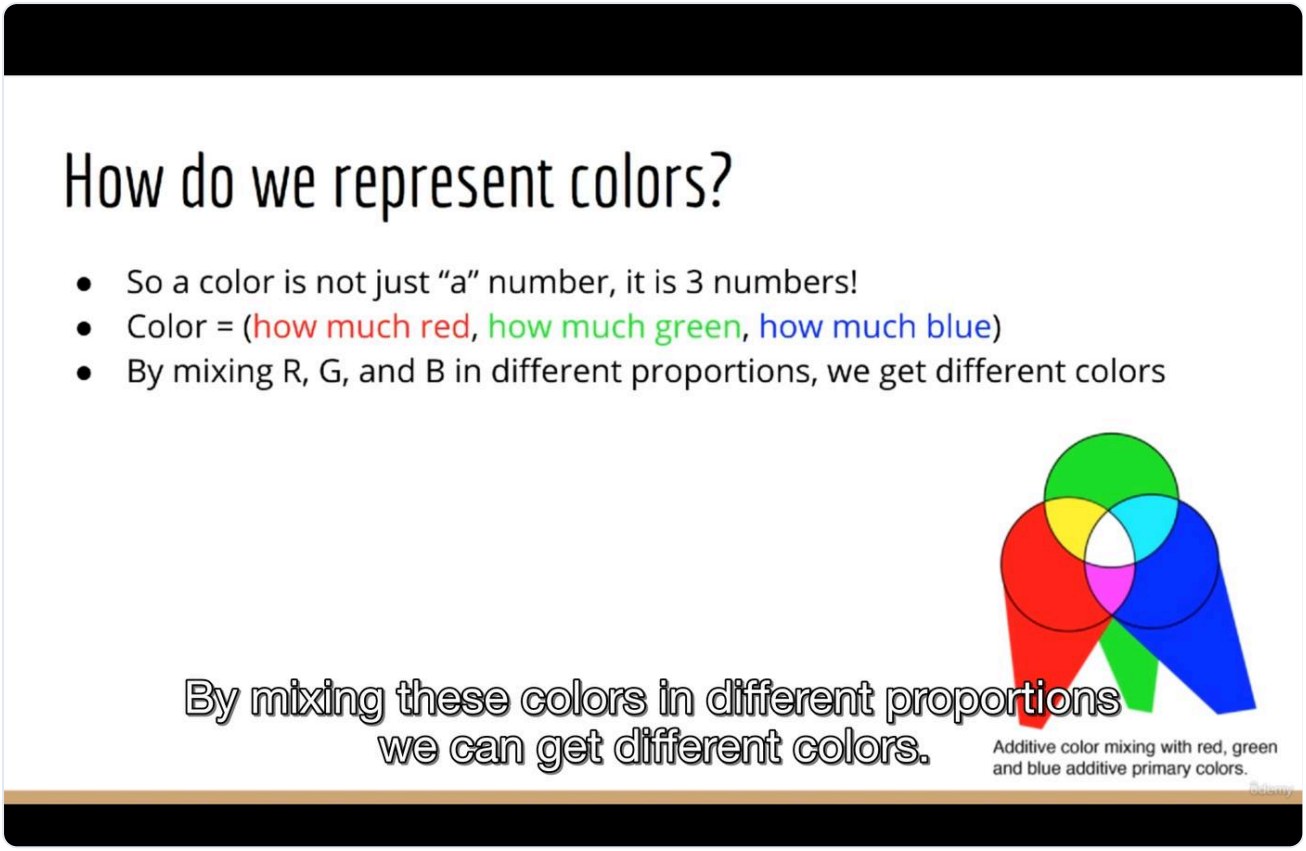


FIGURE 28.5 | How Do We Represent Colors?

This set of three numbers tells us how much red how much green and how much blue. By mixing these colors in different proportions we can get different colors. So what does this tell us about how to represent images. Well in addition to height and width we need another dimension always of size three which represents the three components of color. So our image will actually be a three dimensional tensor A i j k.

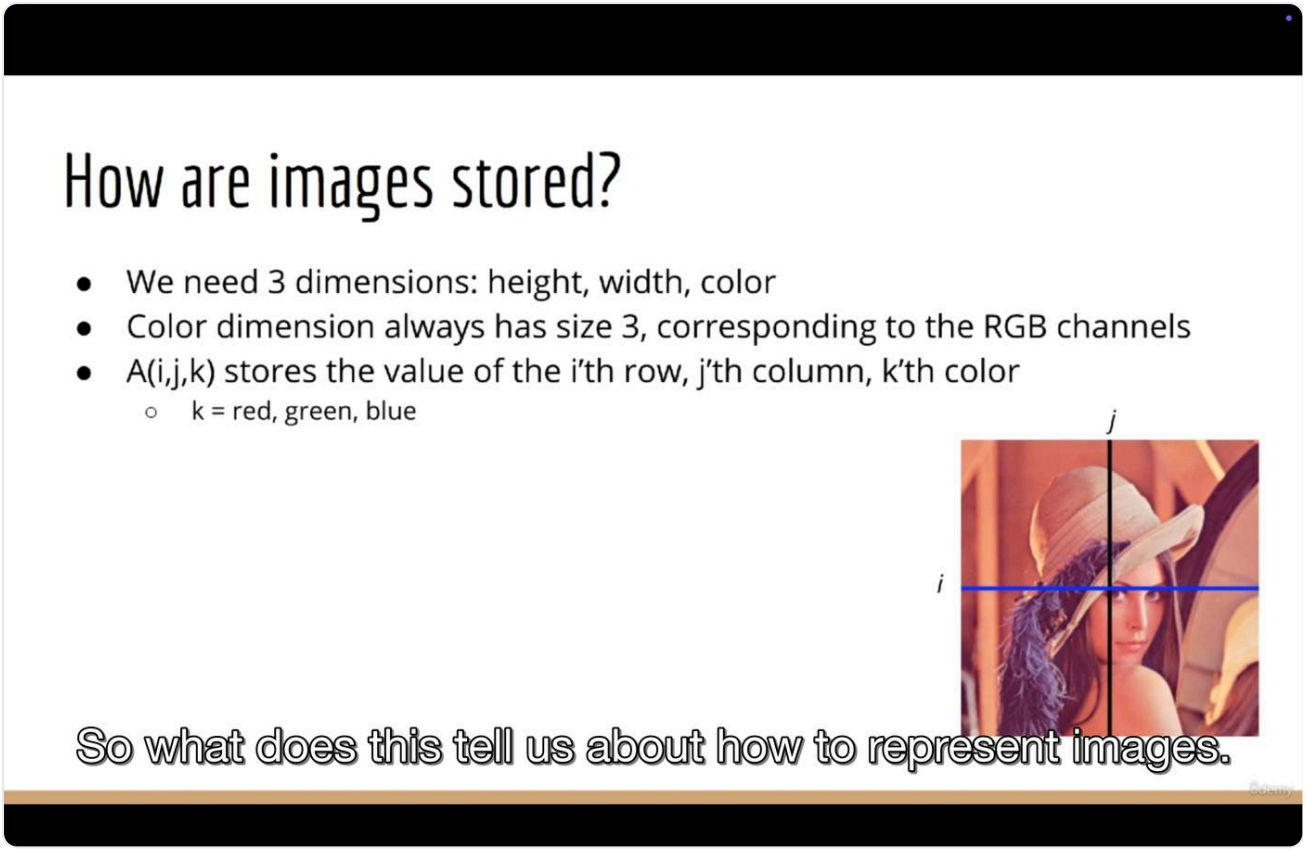


FIGURE 28.6 | How Are Images Stored?

Okay. This is the component of the image representing the ith row jth column and kth color the next important thing we have to discuss about how images are stored in computers has to do with quantization physically. We know that color is light and we measure it with light intensity. It's a physical value in the real world and therefore it's pretty much continuous if it's continuous. That means it has an infinite number of possible values at the same time.

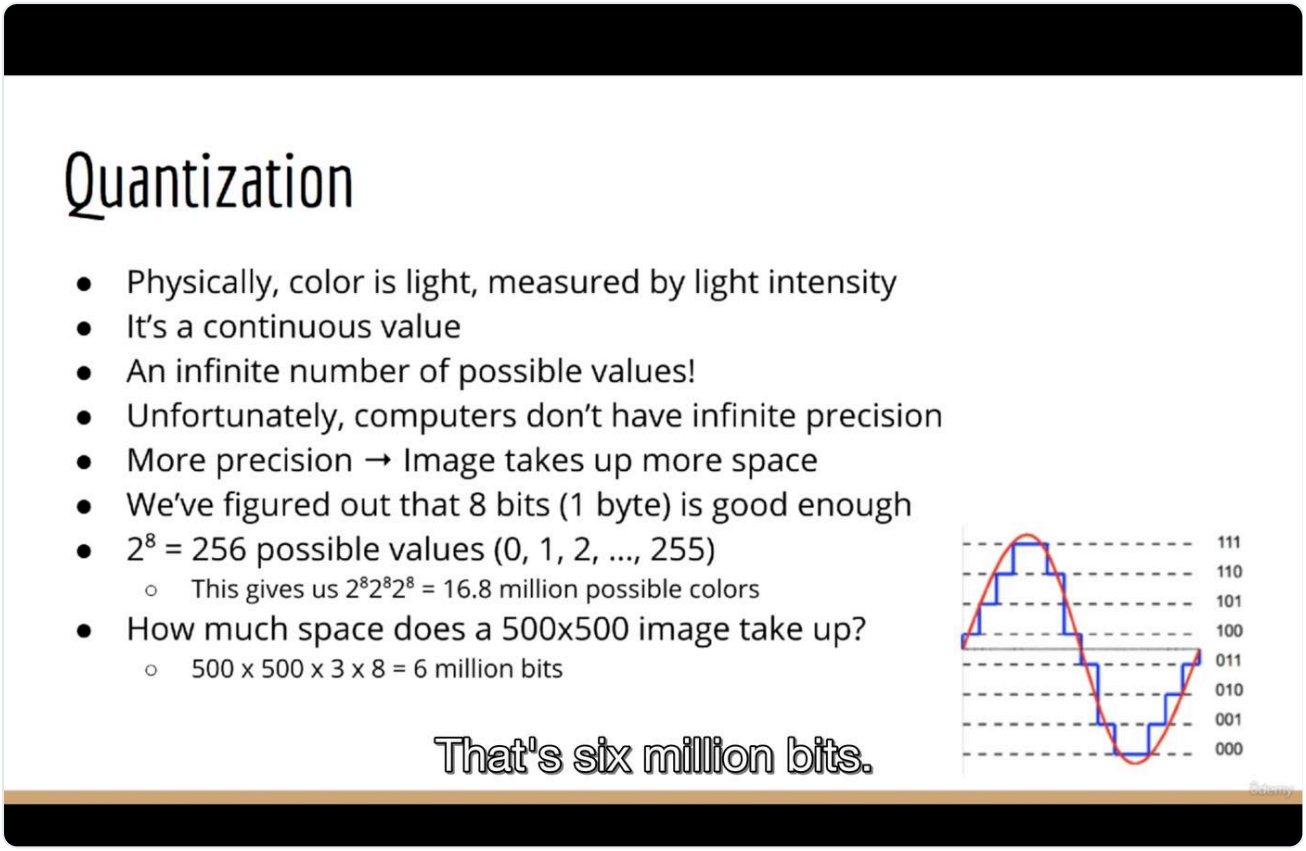


FIGURE 28.7 | Physically, Color Is Light, Measured by Light Intensity

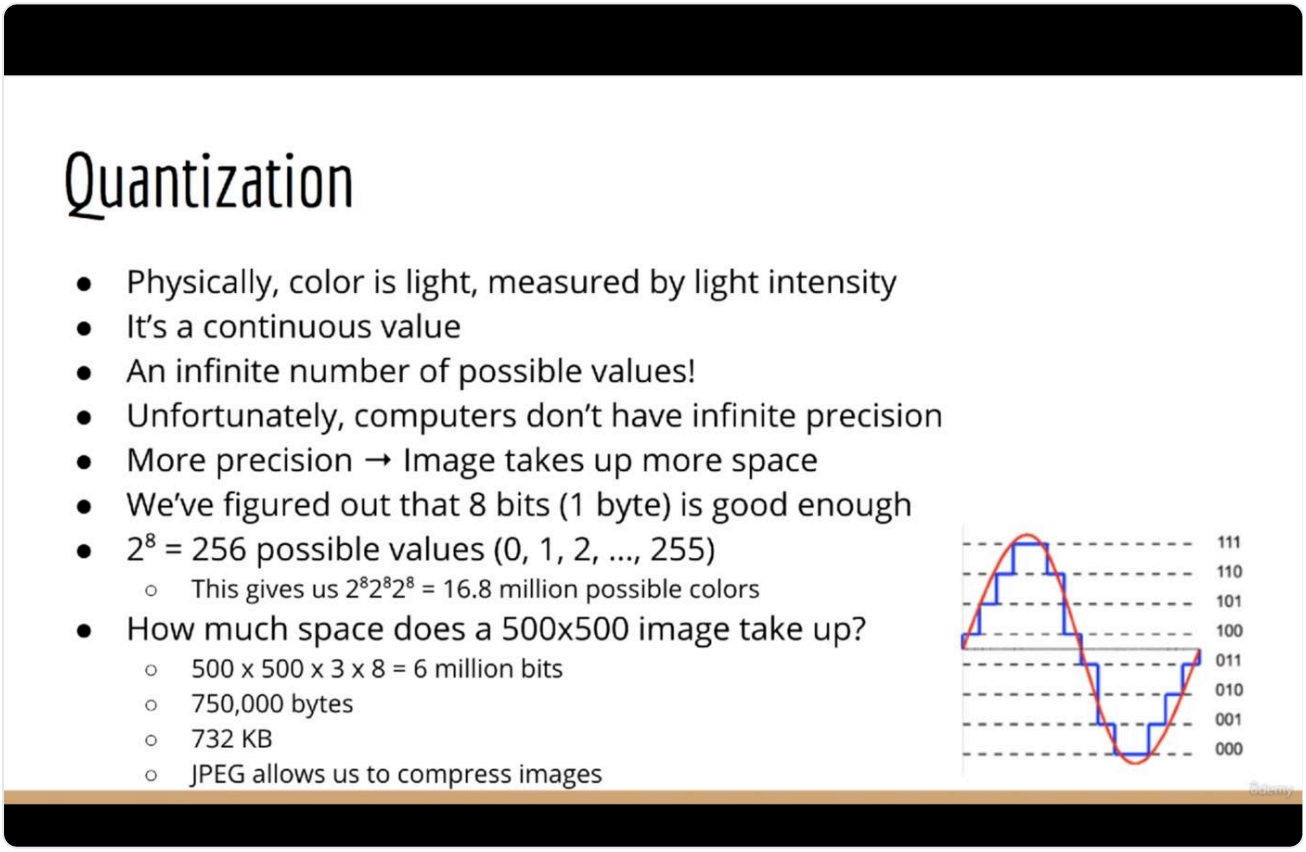


FIGURE 28.8 | Physically, Color Is Light, Measured by Light Intensity

We know that computers don't have infinite precision and even if they did we might not want to make use of it because that would make our image files take up lots of space. Well it just so happens that people have already figured out that eight bits is good enough eight bits gives us two to the eight possible values which is equal to 256 possible values. These are encoded with the numbers zero up to 255 so in total considering that we have the red green and blue channels this gives us two to the power eight times two to the power eight times two to the power eight possible colors which is equal to about sixteen point eight million now. Using this we can actually calculate how much space it would take to store a RAW image.

Suppose we have an image of size 500 by 500. Then the number of bits it requires would be five hundred times five hundred times three times eight. That's six million bits. If we divide that by eight to get bytes we get seven hundred fifty thousand bytes that's equal to about seven hundred thirty two kilobytes which is almost one megabyte. You might realize that this is quite large for a 500 by 500 image which is why we have image compression algorithms such as jpeg that help us store image files in a much more compact format.

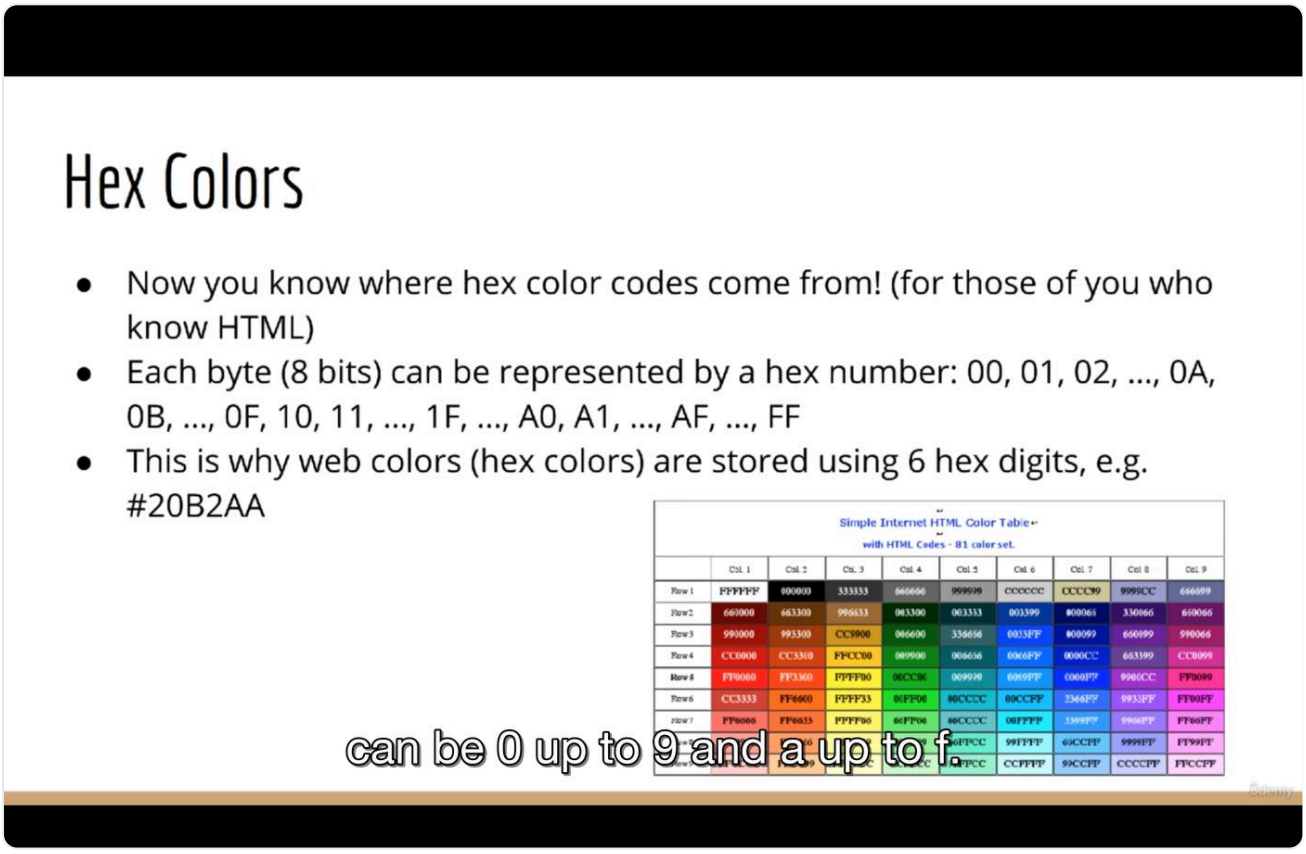


FIGURE 28.9 | Hex Colors

As a side note for those of you who are web developers this is where hex colors come from. These also happen to use the same color scheme and quantization. So we have three bytes with each byte representing the value for the red green and blue color channels. Importantly one byte can be represented with two hex digits hex digits go from 0 up to 9 and then a up to F. So it's a base 16 number system two base 16 numbers give us 256 possible values. So that's why we need two base 16 numbers or 2 hex numbers to represent one byte a color which is represented by 3 of these bytes would require 6 hex digits. So that's why when you're coding in HTML colors are represented by six digits where the digits can be 0 up to 9 and a up to f.

Now we can simplify this a little bit if we have an image that does not have color.

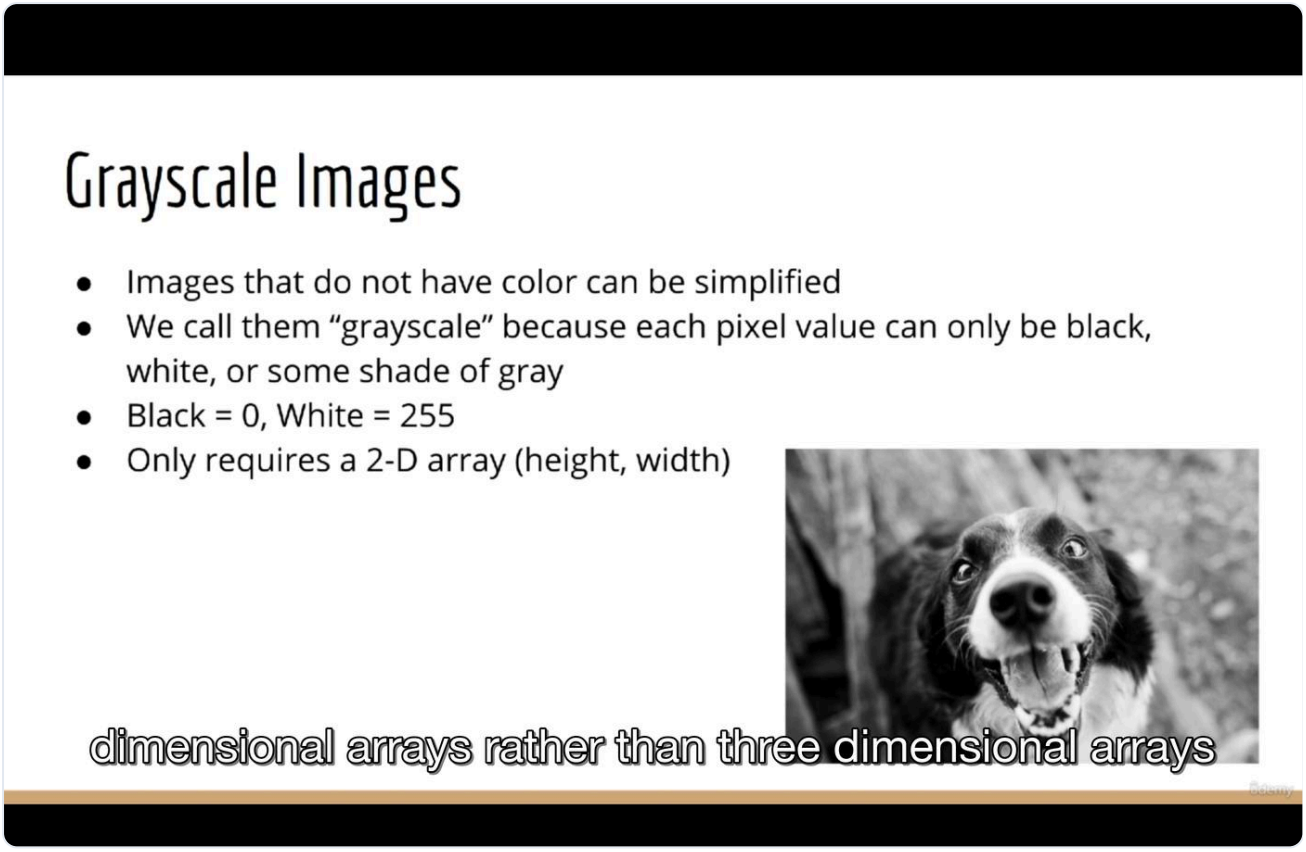


FIGURE 28.10 | Grayscale Images

We call these grayscale images because each pixel can only be black or white or something in between. Typically we assign black the value of zero in white the value of 255 any value in between is a different shades of grey and because we only need one value instead of three grayscale images are stored in two dimensional arrays rather than three dimensional arrays in matplotlib. If you tried to use the imshow function to plot a grayscale image it will end up assigning its own color scheme. But note that this is not the true color of the image. This is just a heat map where we assign one color such as red to the high values and then another color such as Blue to the low values.

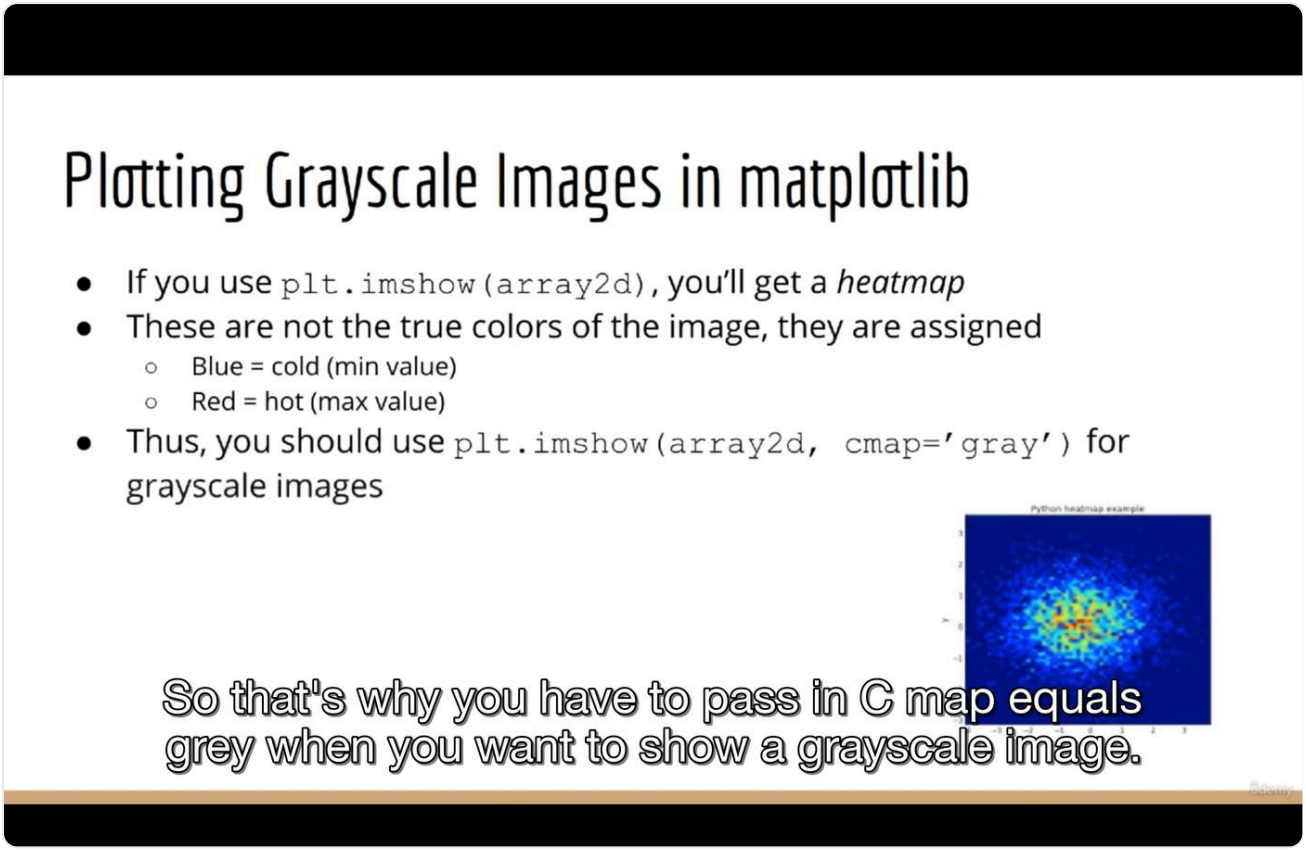


FIGURE 28.11 | Plotting Grayscale Images in Matplotlib

So that's why you have to pass in cmap equals gray when you want to show a grayscale image. The last thing I want to mention about how images are represented in computers is that sometimes we don't use 8 bit unsigned integers from 0 to 255.

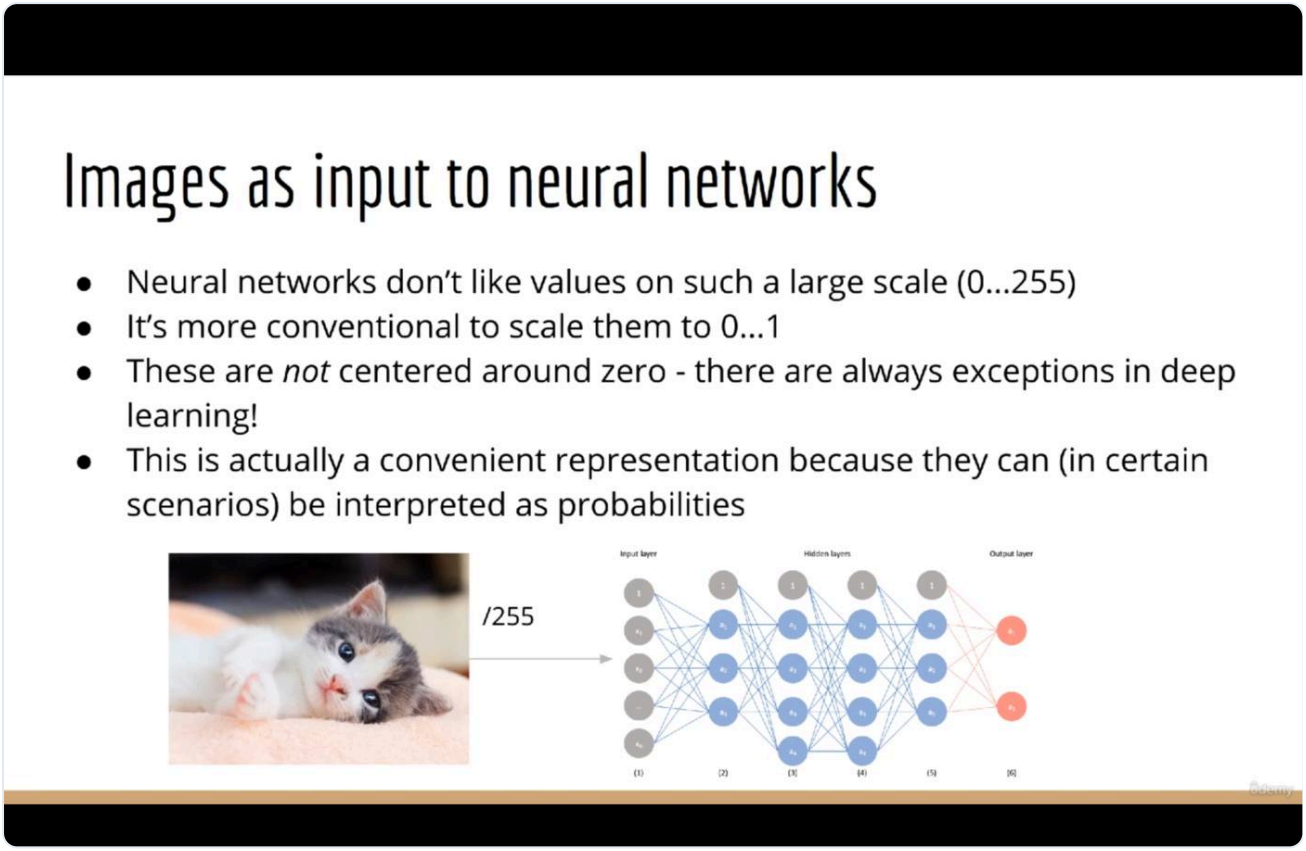


FIGURE 28.12 | Images as Input to Neural Networks

If you recall neural networks don't like values on such a large scale. So for neural networks it's more convenient to scale image values to be floating points between 0 and 1 0 represents no intensity and a 1 represents full intensity. And yes the astute among you might notice that if we represent images as values between 0 and 1 these are not standardized values because they are not centered around 0 there are always exceptions and deep learning for images. This representation is particularly convenient because it's possible to interpret these values as probabilities as mention. There are always exceptions and deep learning. So here is another one.

One of the most famous neural networks today is called the VGG which is used for many computer vision applications such as image classification transfer learning style transfer and object detection. VGG also previously won the ImageNet contest in multiple categories which as you recall is a competition held every year for research groups to present state of the art image recognition models. The interesting thing about VGG is that the original authors did not scale the input data but they did shift it by its mean across each color channel.

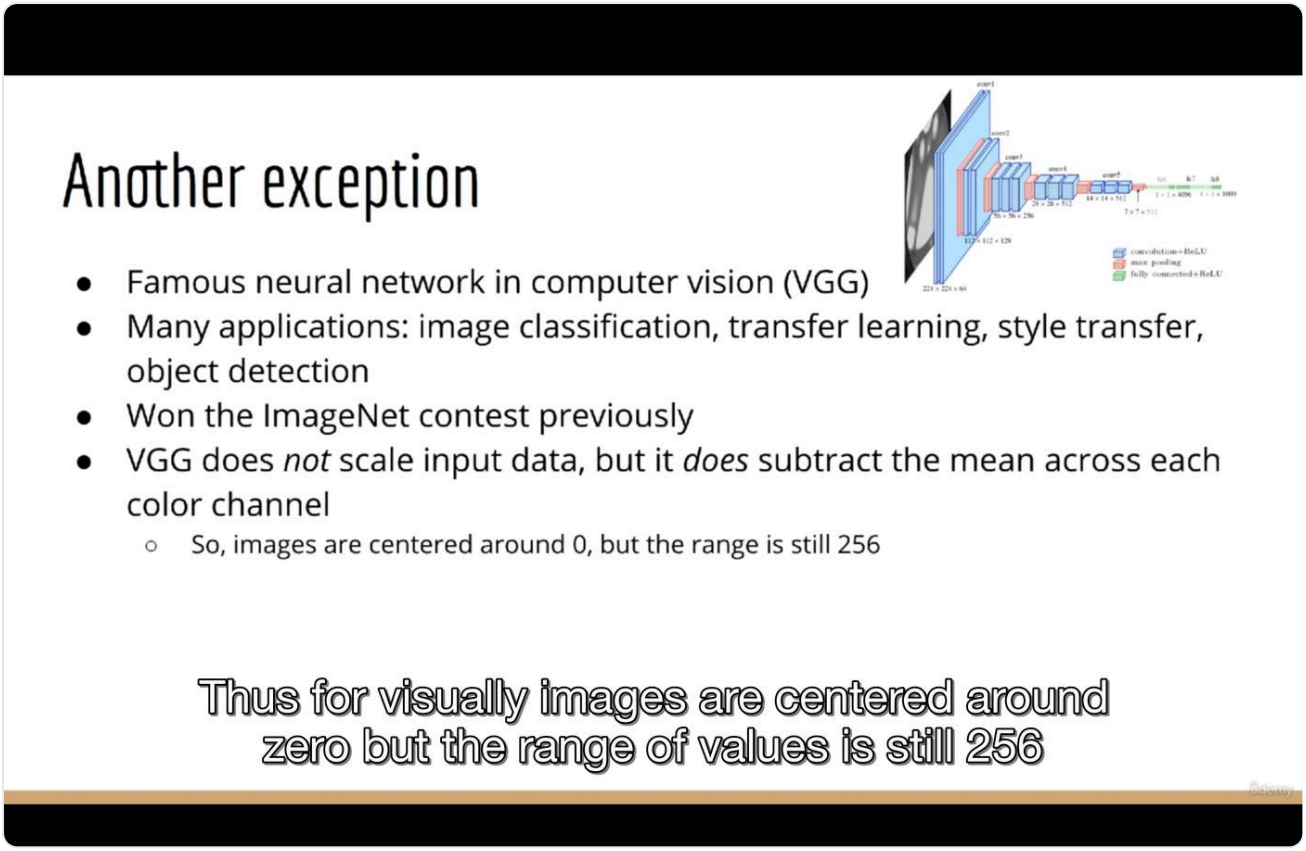


FIGURE 28.13

Thus for VGG images are centered around zero but the range of values is still 256 so far. We've discussed how images are represented at a computer. However this still begs the question how is an image represented as input into a neural network if you recall our input data is usually represented by a matrix X of size n by D where n is the number of samples and d is the number of features but this doesn't make sense in our case if we have an image sample. How can it have D features an image itself is a three dimensional object.

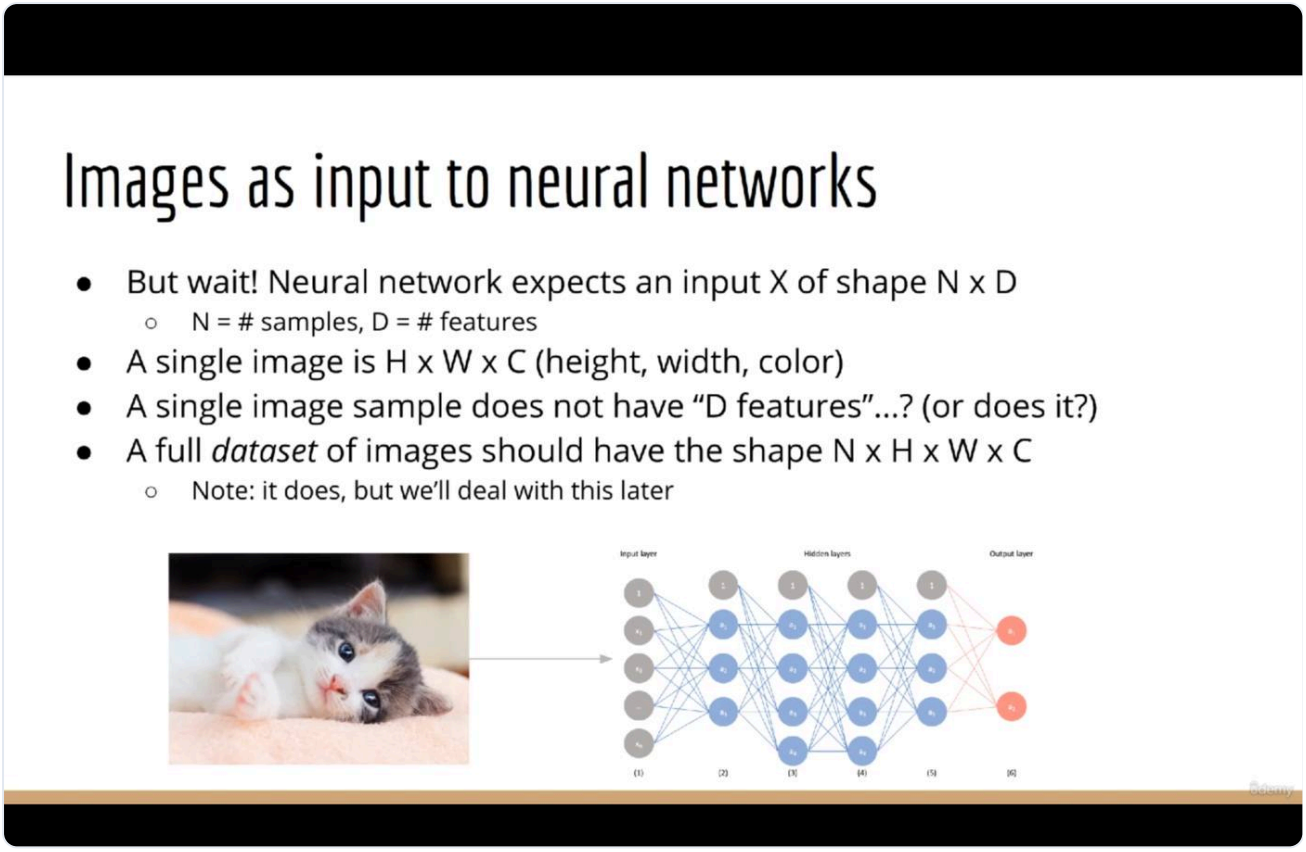


FIGURE 28.14 | Images as Input to Neural Networks

Therefore it would seem that in order to represent a data set of images we would need to have a four dimensional tensor of shape N by H by W by C where N is the number of samples. H is the image height W is the image width and C is the number of color channels which is always 3. Now this is a little bit of foreshadowing since this is what we'll do with later but for now we are going to do something simpler to keep things simple. Let's say we have a black and white image of size three by three a very tiny image. Well we can just number each pixel in a zigzag fashion going from left to right and top to bottom.

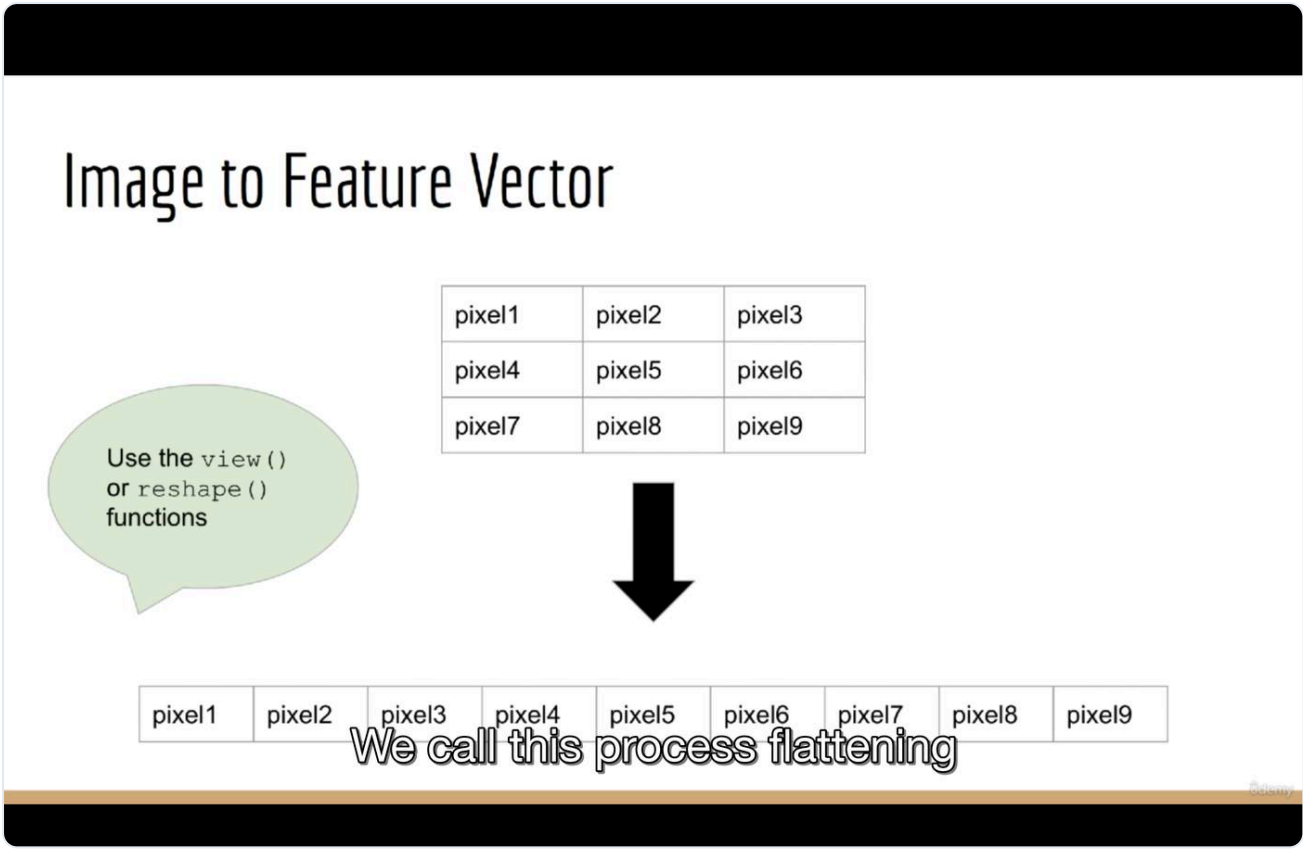


FIGURE 28.15 | Image to Feature Vector

So we have pixel one pixel two pixel three all the way to pixel nine in the bottom right corner. Well if we want to represent this as a vector why not just line them up in a row and call it a vector. So we have a vector of size 9 containing pixel 1 pixel 2 pixel 3 all the way up to pixel 9. We call this process flattening in this way.

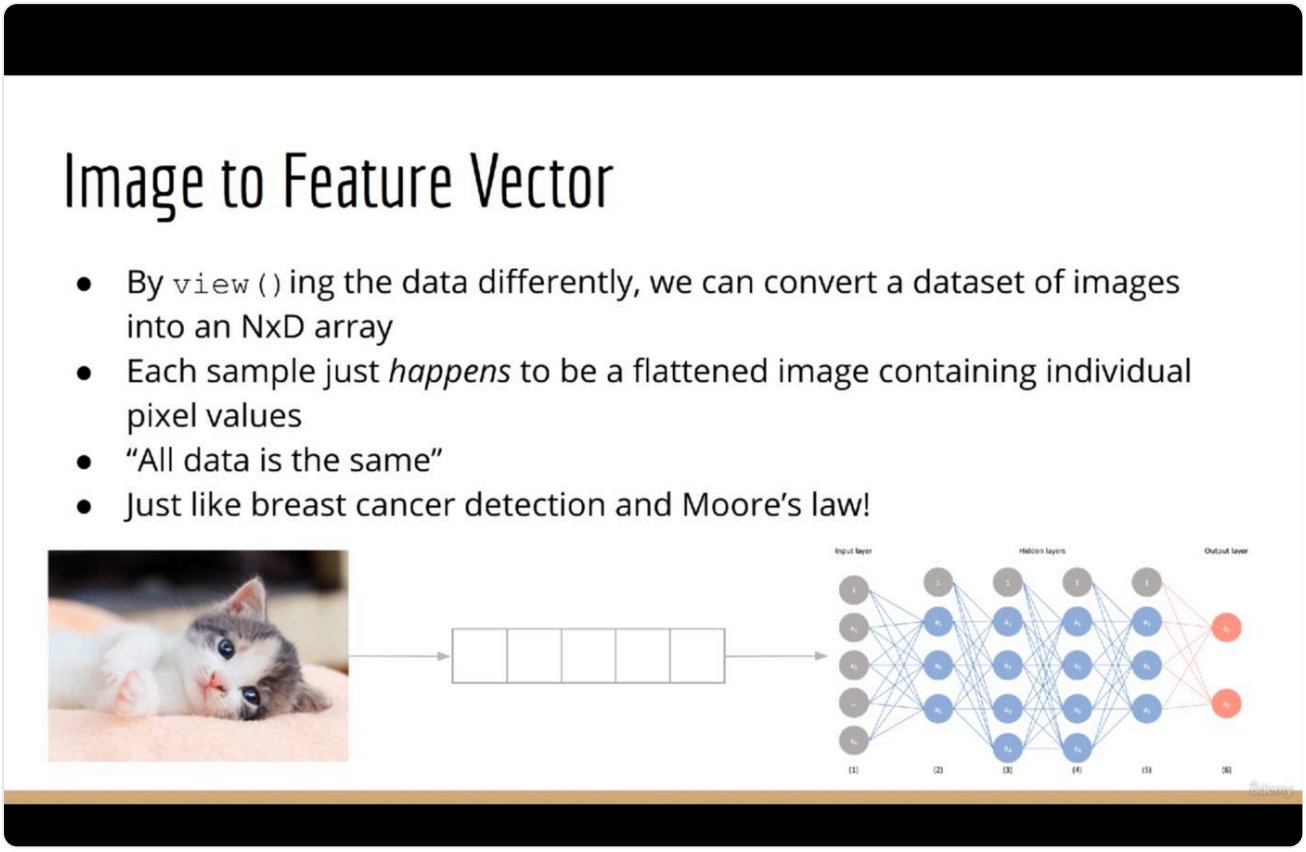


FIGURE 28.16 | Image to Feature Vector

We can still store all of our images in an N by D array here N is the number of samples and D is height times width times color each row of this matrix is still one sample and each sample just happens to be a flattened image. The features of the image just happened to be the individual pixel values. And so here we encounter again this idea that all data is the same. We saw previously how we can represent all sorts of different tabular data as an N by D array from breast cancer detection to Moore's Law and so on. Now we can represent images in this format as well so it is a truly powerful concept.

LECTURE 29

Why Does It Work?

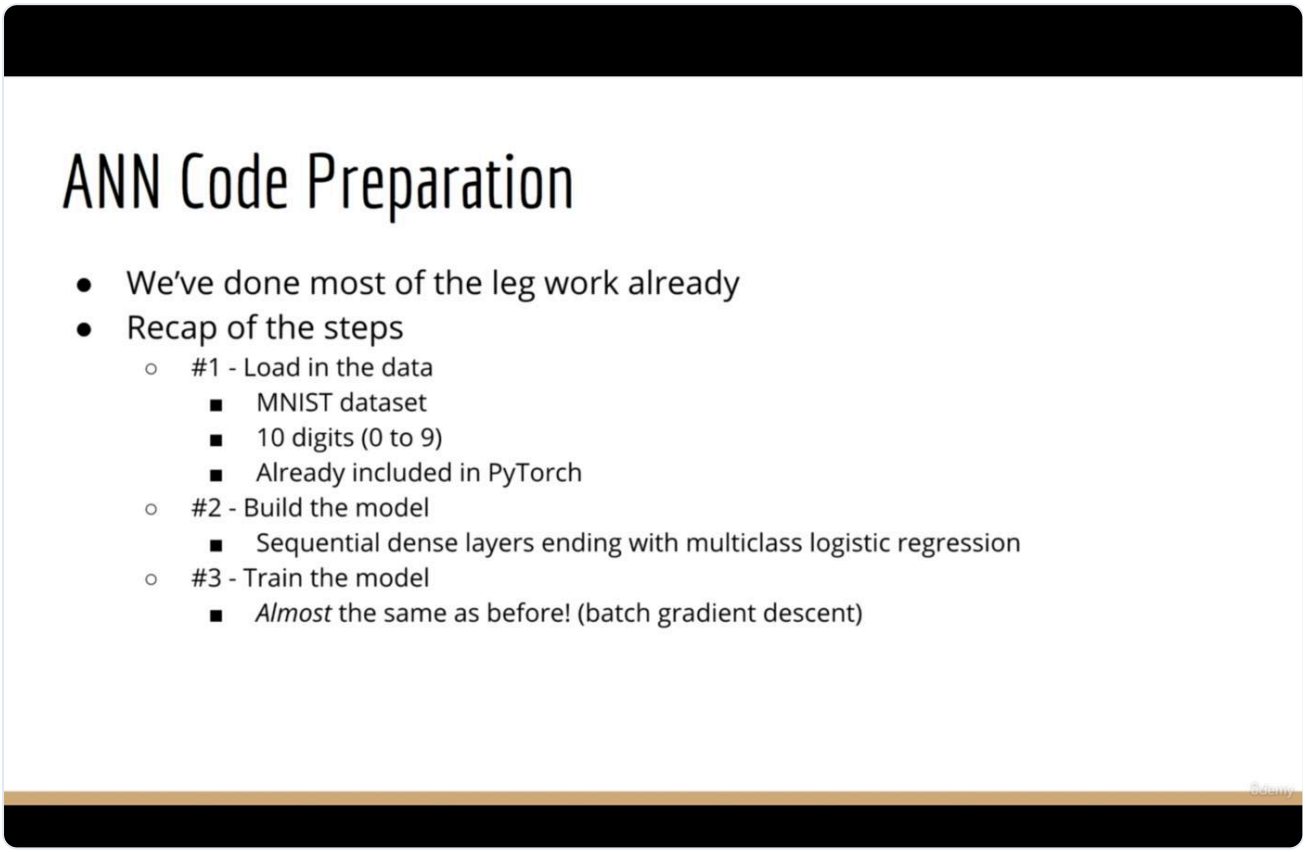


FIGURE 29.1

At this point, we've done most of the legwork to describe what we are about to implement in code. Now we just have to discuss a little more about what the code is going to look like and then we can look at the actual code. So let's again recap all the usual steps in the context of our data and our new model. So step number one is to load in the data for the coming example, will be looking at the classic MNIST data set, which is a data set containing handwritten digits. Our job is to accept as input an image of the digit and then classify what the digit is. Since there are ten possible digits, zero up to nine. This is a multiclass classification problem.

Also note that even though these data are technically images, they don't come from actual image files like JPEG or PNGs, instead the MNIST data set is actually already included in the torchvision library, which makes things a little bit more convenient for us. Also, they are greyscale images, so we don't have to worry about color right now. Later in the course, we'll look at how to load an actual image files. Step number two is to build our model as per this section of the course, the model is a feedforward neural network. Basically, it's going to be multiclass logistic regression with a few more layers before it all chained together sequentially.

Step number three is to train the model. Now, this is the beauty of deep learning and deep learning frameworks. There is zero additional work to do here. All we have to do is pretty much the same training loop we have already seen, although there is one small modification we are going

to make in order to improve the training process called batch gradient descent. We'll see what that is later on in this lecture. Step number four is to evaluate the model.

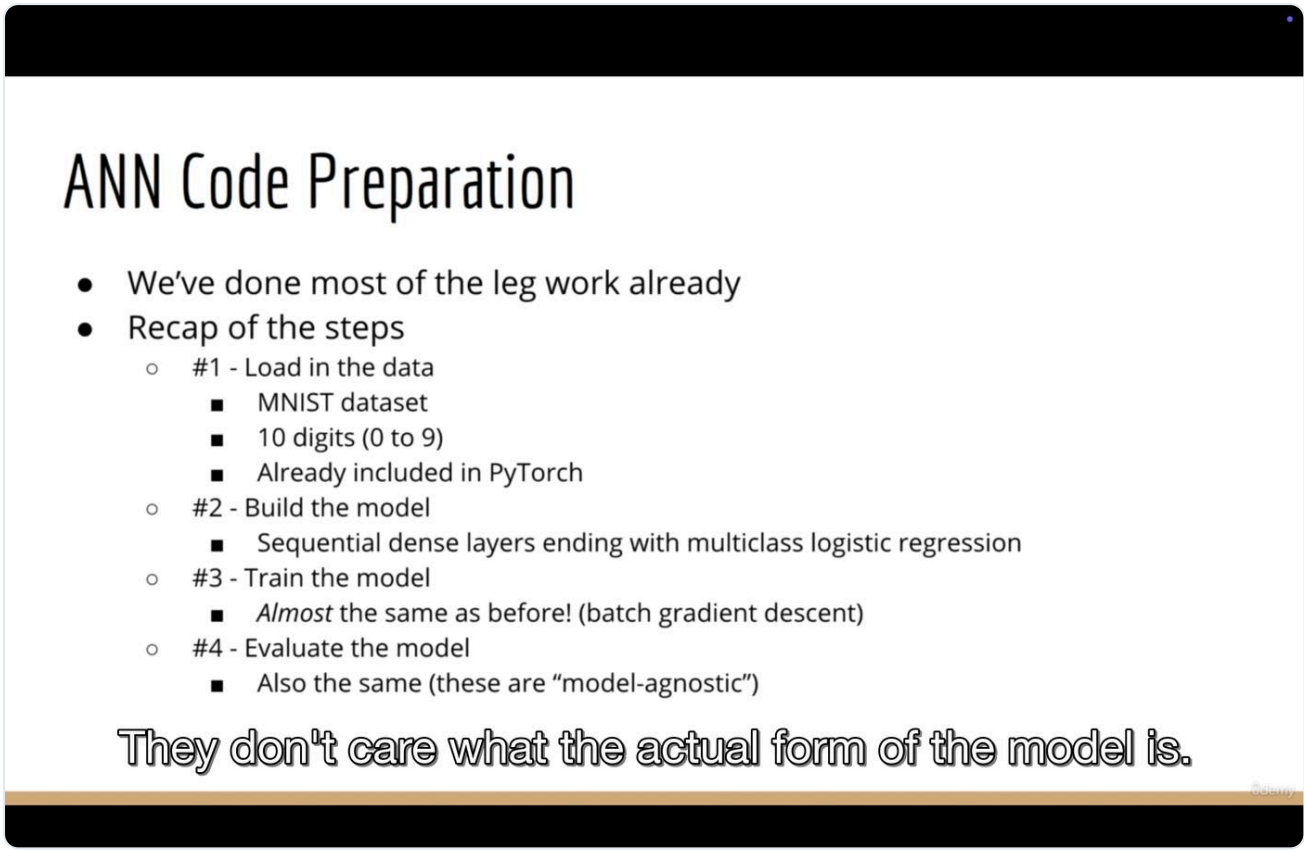


FIGURE 29.2

Again, this is exactly the same, you could say that these steps are model agnostic. They don't care what the actual form of the model is. Step number five is to use the model to make predictions.

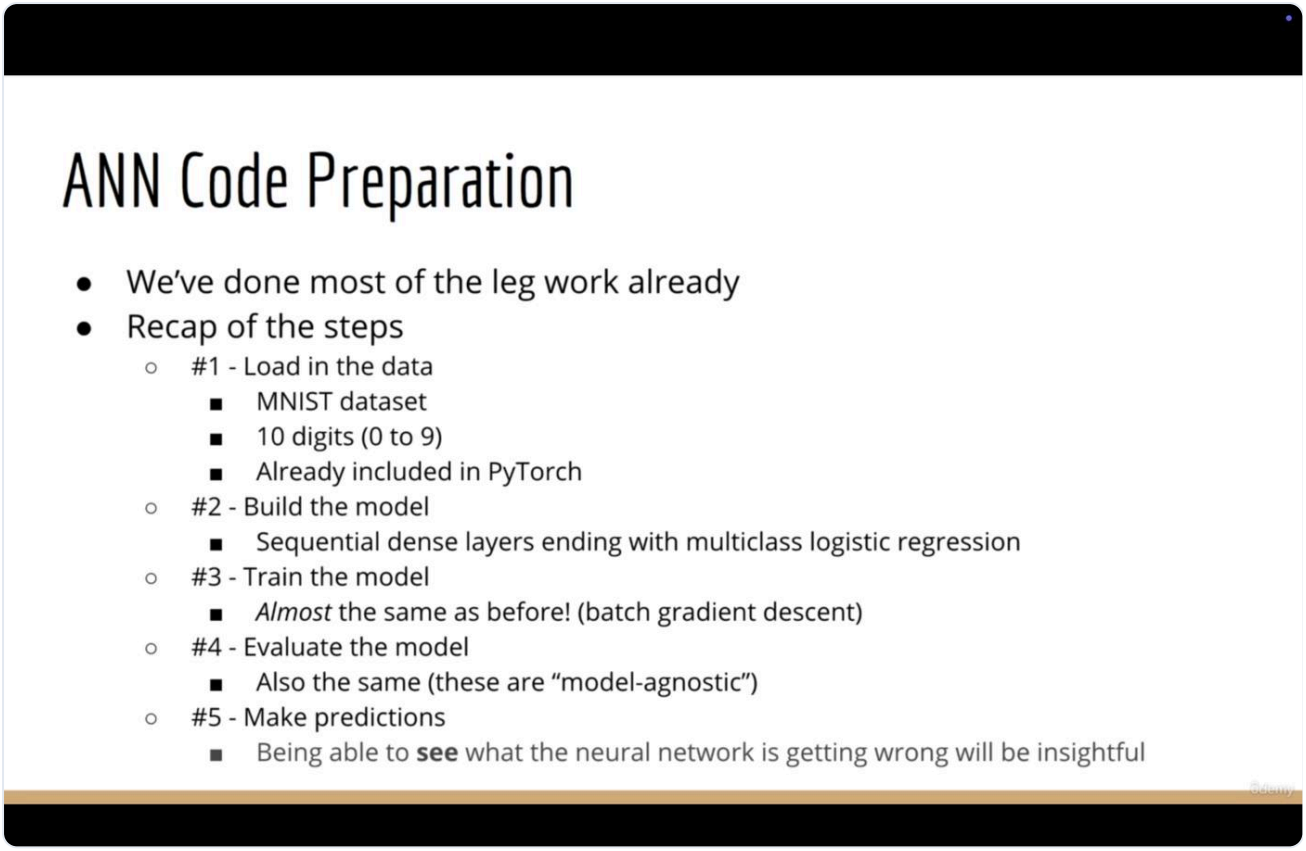


FIGURE 29.3

Well, do something interesting in this section, which will be to look at which predictions the neural network is getting wrong. This might give us some insight on why the neural network is doing what it's doing. So let's start by breaking down step number one, as mentioned, this data is going to come from the torch library itself, which will keep things simple for us, actually, to be more specific. It comes from a sister library called torchvision, which you normally have to install separately. But since everything is installed for you in Google Colab, we won't need to worry about that. torchvision specializes in providing functionality for, you guessed it, machine vision.

This includes both helper functions and data sets of which will see multiple in this course.

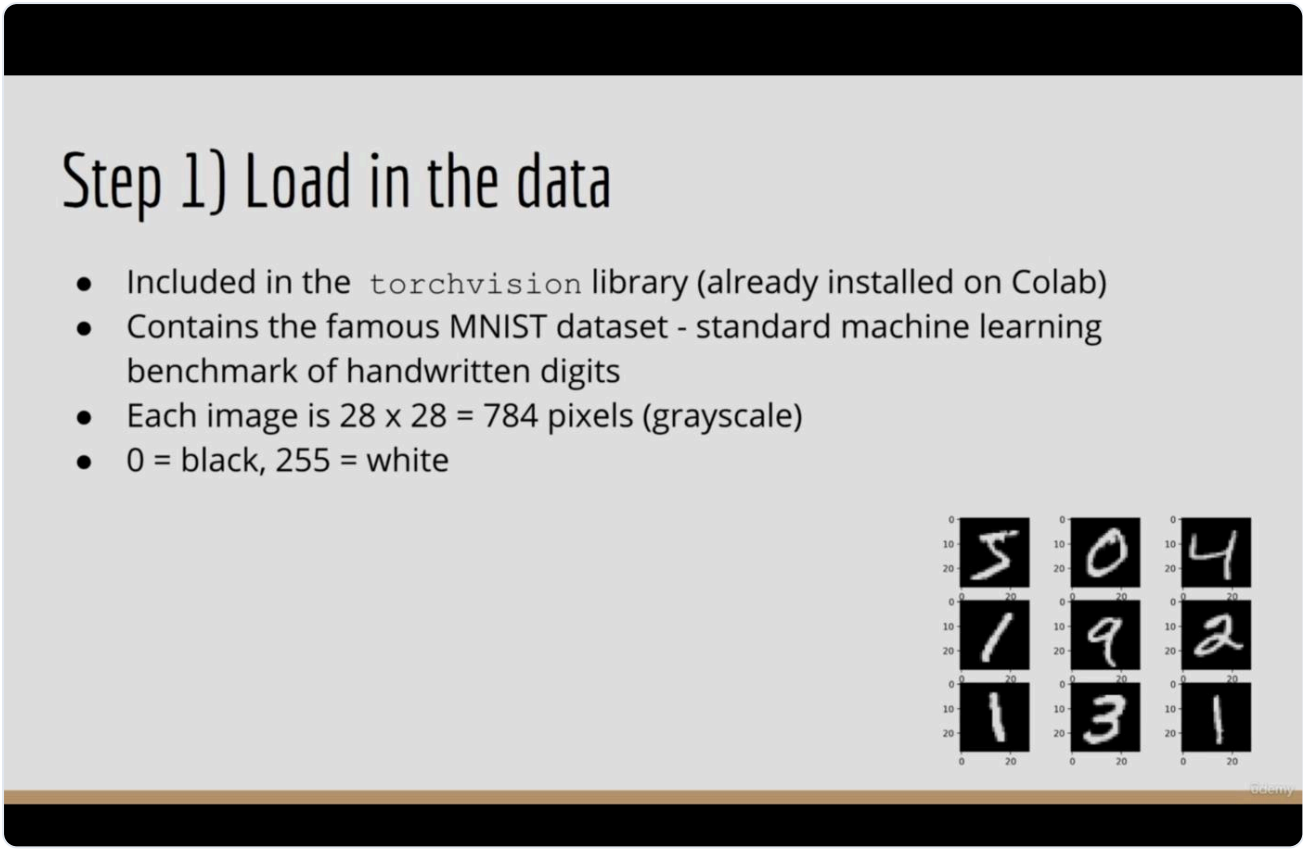


FIGURE 29.4

Of course, one of these data sets is the famous MNIST data set, which is a standard machine learning benchmark of handwritten digits. This is an image classification problem. So the data are images and the targets are what the image is of. Each image is the same size 28 by 28 pixels, which is equal to seven hundred eighty four pixels in total. The MNIST data set contains only grayscale images. So the images are just of size 28 by 28 and not twenty eight by twenty eight by three, which is what they would be if they had color.

FIGURE 29.5 | Load in the Data

A grayscale image is pretty basic. Just imagine it like a matrix of numbers, the number in each element of the matrix tells us how dark or how light that pixel should be. So if you have a very small number close to zero, you'll get a very dark color like black. If you have a very large number, close to 255, you'll get a very light color close to white. When we call the function torchvision datasets dot MNIST and pass in train equal to true, we get back a training data set object, which is similar to what we saw in scikit-learn. Specifically, if we call the data attribute, we get the input data, which are the images and if we call the target's attribute, we get the targets, which are the labels telling us what digit the corresponding image is of.

We can think of train data set data as X train and train data set targets as Y train.

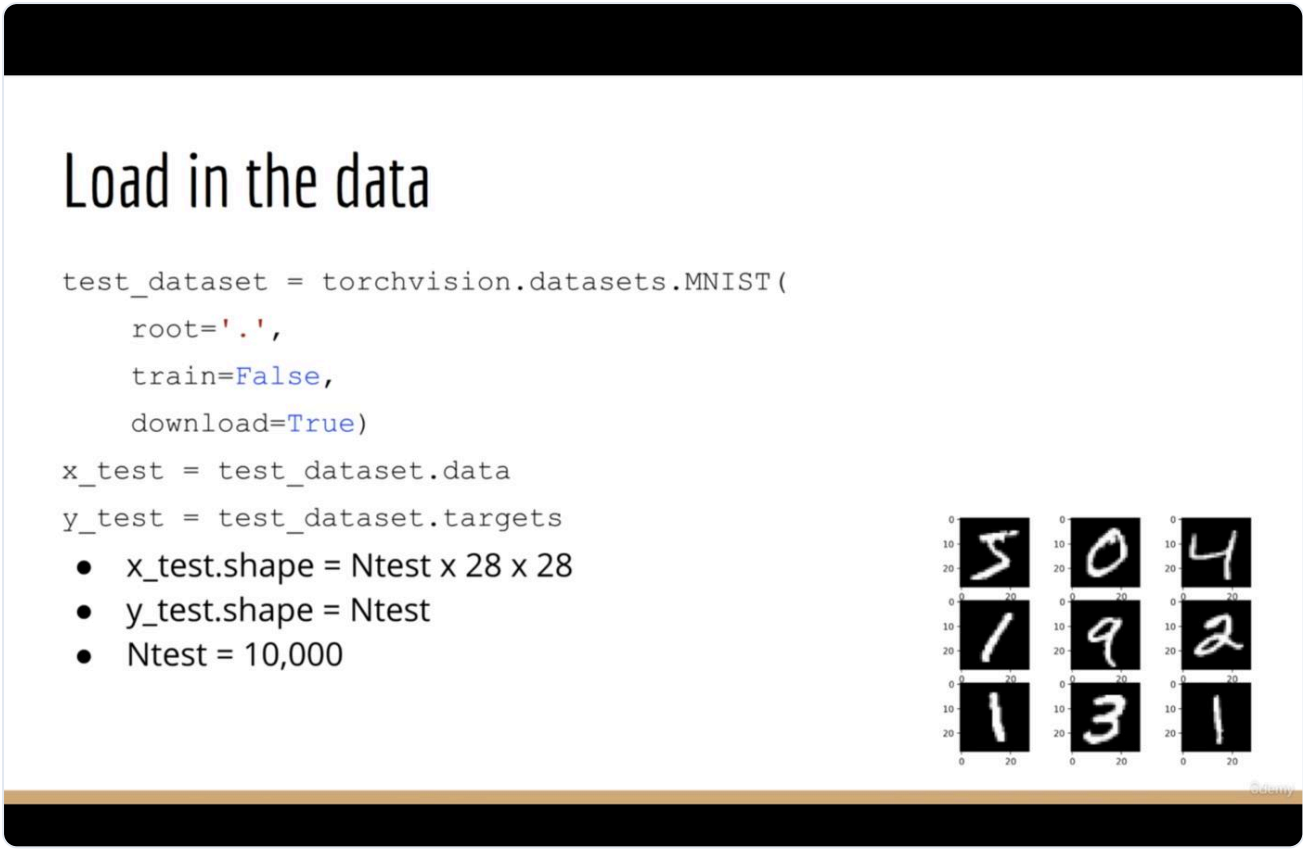


FIGURE 29.6 | Load in the Data

So X train is of shape N by twenty eight by twenty eight, whereas Y train is a 1D vector of length N in our case N is equal to sixty thousand. Correspondingly, if we call the same function, torchvision datasets dot MNIST, but we pass in a train equal to false, then we'll get back the test data set as before. We can call test data, set data to get X test and we can call test data, set targets to get Y tests. X test is of size N test by 28 by 28 and Y test is a 1D vector of length N test and in our case N test is 10000. As a reminder, the image pixels are stored as integers from zero to 255, so we'll have to scale them to go from zero to one. This is because we like the input data into a neural network to be in a small range, as you recall.

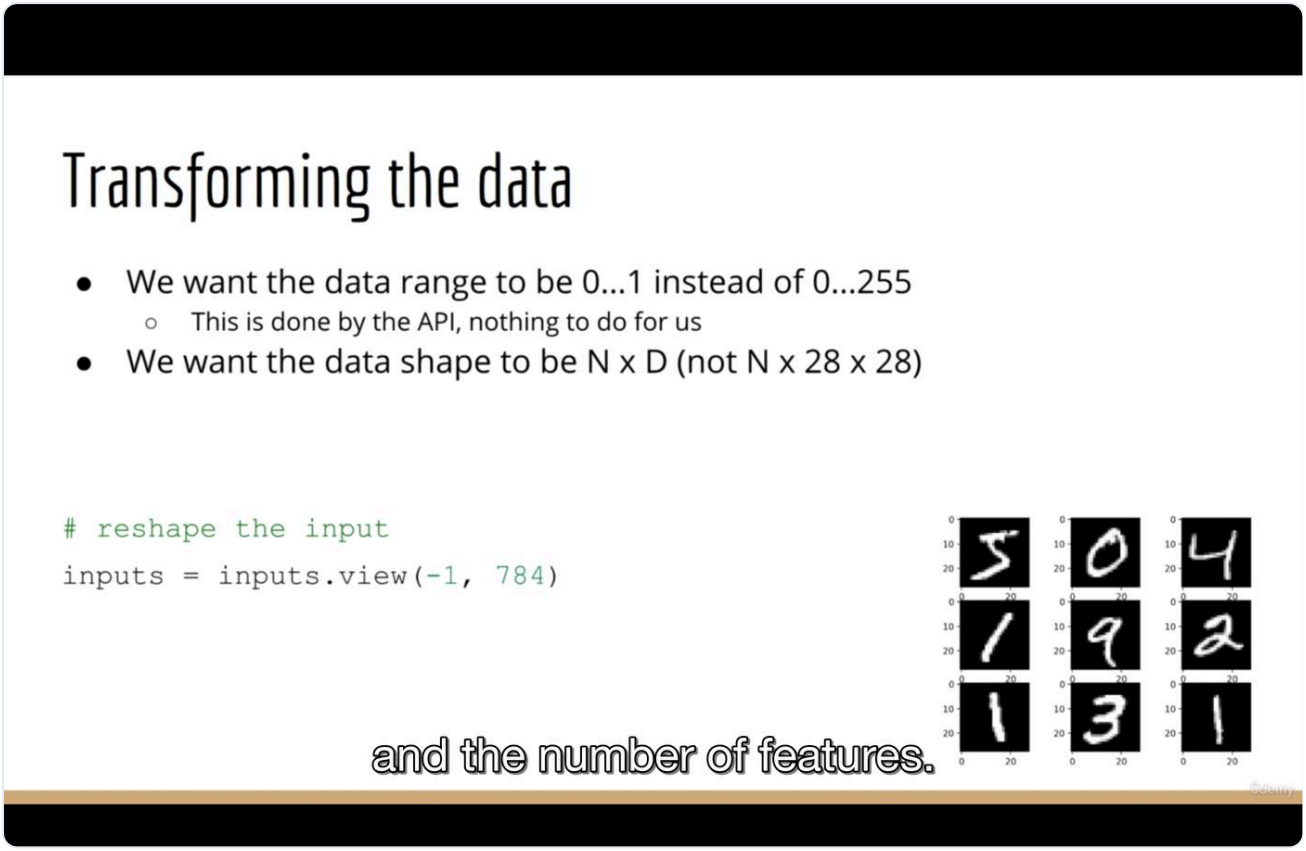


FIGURE 29.7

In addition, the neural networks and other machine learning models that we have seen so far expect their inputs to be of shape N by D, which is a two dimensional array and is the number of samples and the number of features. But as you just learned, our data sets come in the shape N by twenty eight by twenty eight, which is a three dimensional array. Therefore we'll have to use the special view function in PyTorch to flatten the data into a two dimensional array and treat each of the pixels as if they were an input feature. This is thanks to my rule. All data is the same. We don't care if this input represents a breast cancer feature vector or image pixels.

Neural networks will work the same way regardless. Step number two is to instantiate the model. So first, let's start with the equation that represents how we get from the input to the output in our neural network. If we write it out all in one line, you can see that it follows the pattern inside to outside.

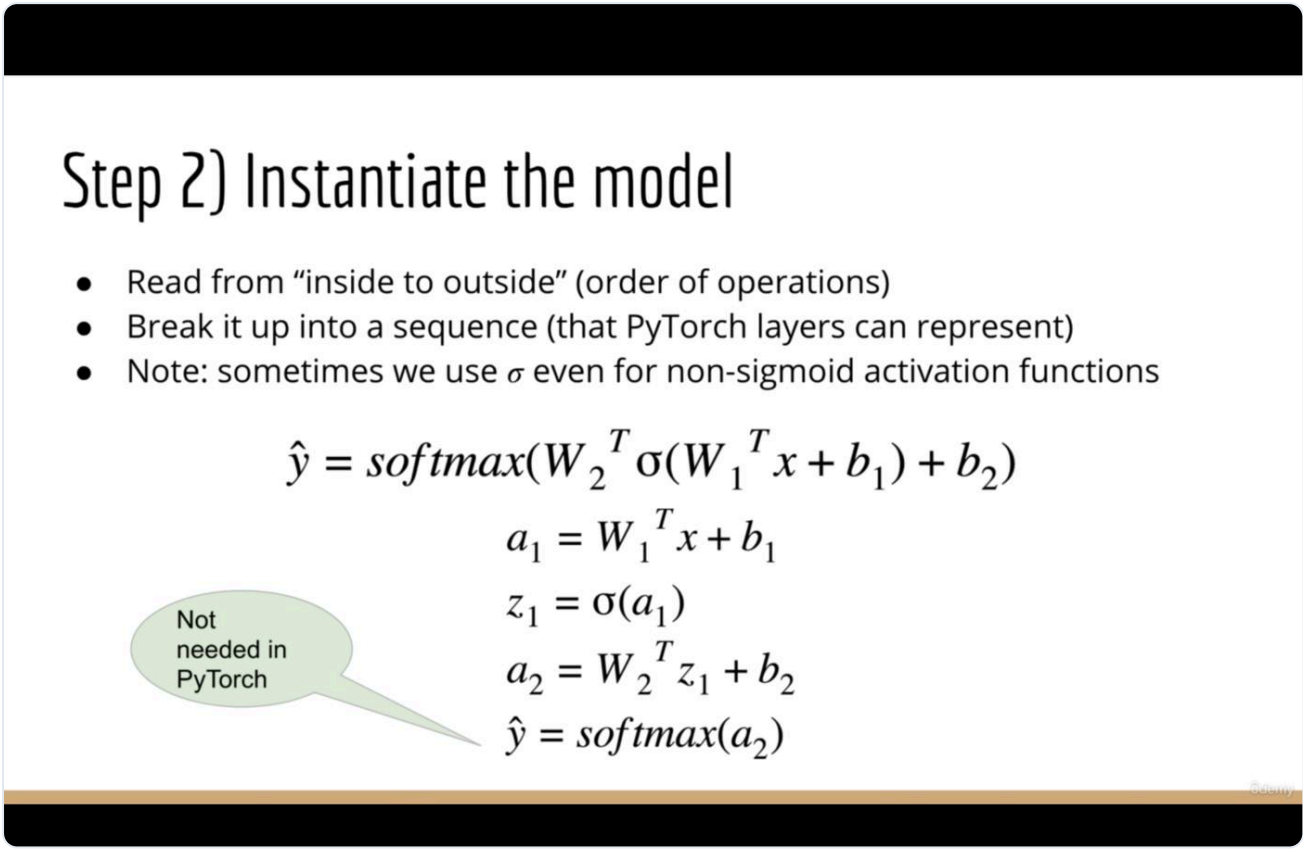


FIGURE 29.8

That's just your basic order of operations for arithmetic. But we can split it up into intermediate variables so that we can see each step one at a time. So A1 is equal to one at times X plus one. z1 is equal to sigma of a1. A2 is equal to two times one plus two and then technically Y is equal to softmax of a two. But remember that we omit this step in PyTorch because it's already included in the loss function. One important thing to note is that even when we use an activation function such as the ReLU, we often still write it as sigma when we write down the math. So that's just something to keep in mind for the future. So you don't get confused when you see it.

Basically, the important thing to remember is that each of these math equations is just an object in PyTorch.

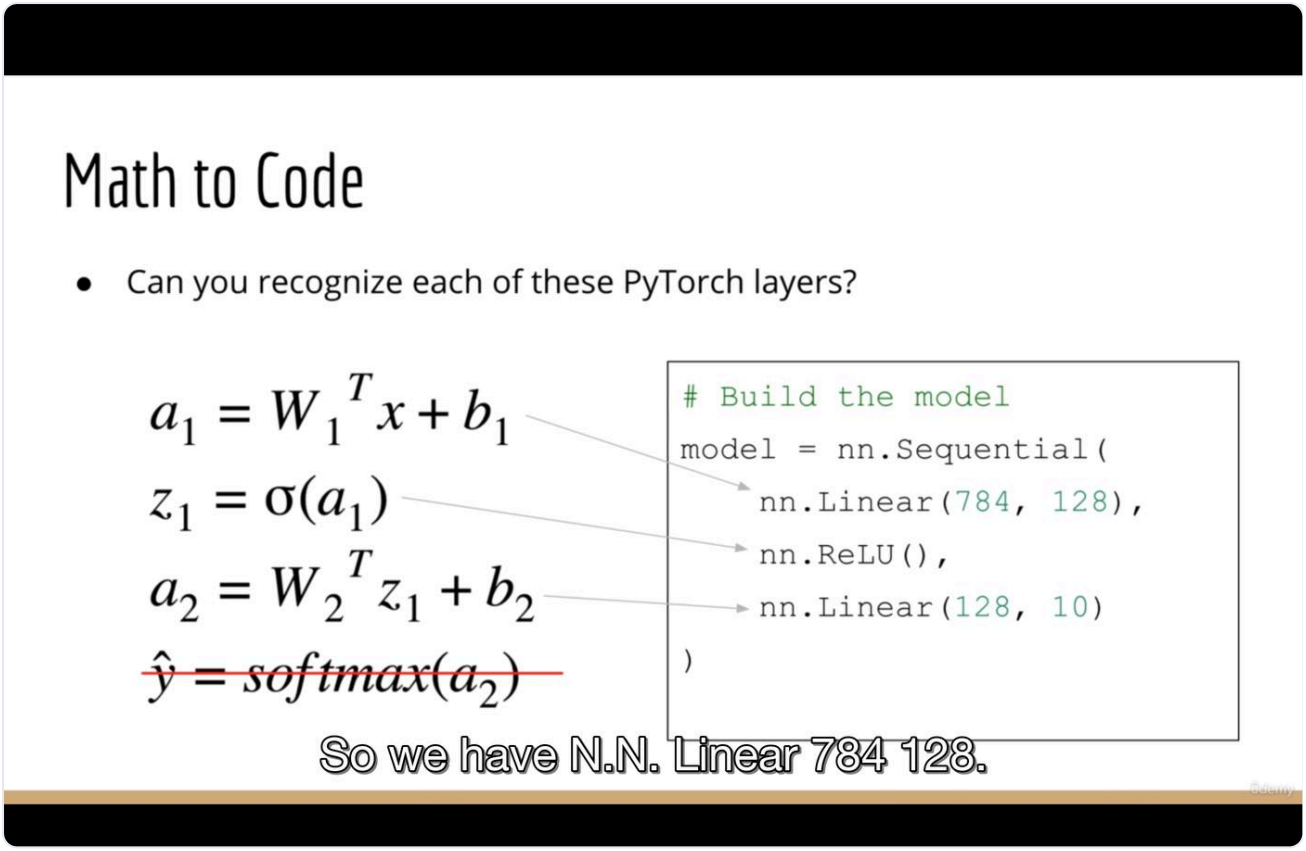


FIGURE 29.9 | Math to Code

So we have nn dot Linear 784 128. And then you add a final nn dot Linear 128 ten. As mentioned previously, ReLU is a fine default choice. But you might be wondering why did I choose the value 128. And of course, there is no special reason. This is just a matter of hyperparameter selection for which you should use trial and error as well as past experience to guide your decisions. You might also notice that oddly, I had to specify the number 128 twice. This makes sense because if the output of the first layer is one twenty eight and that's the input to the next layer, then the next layer must also have input.

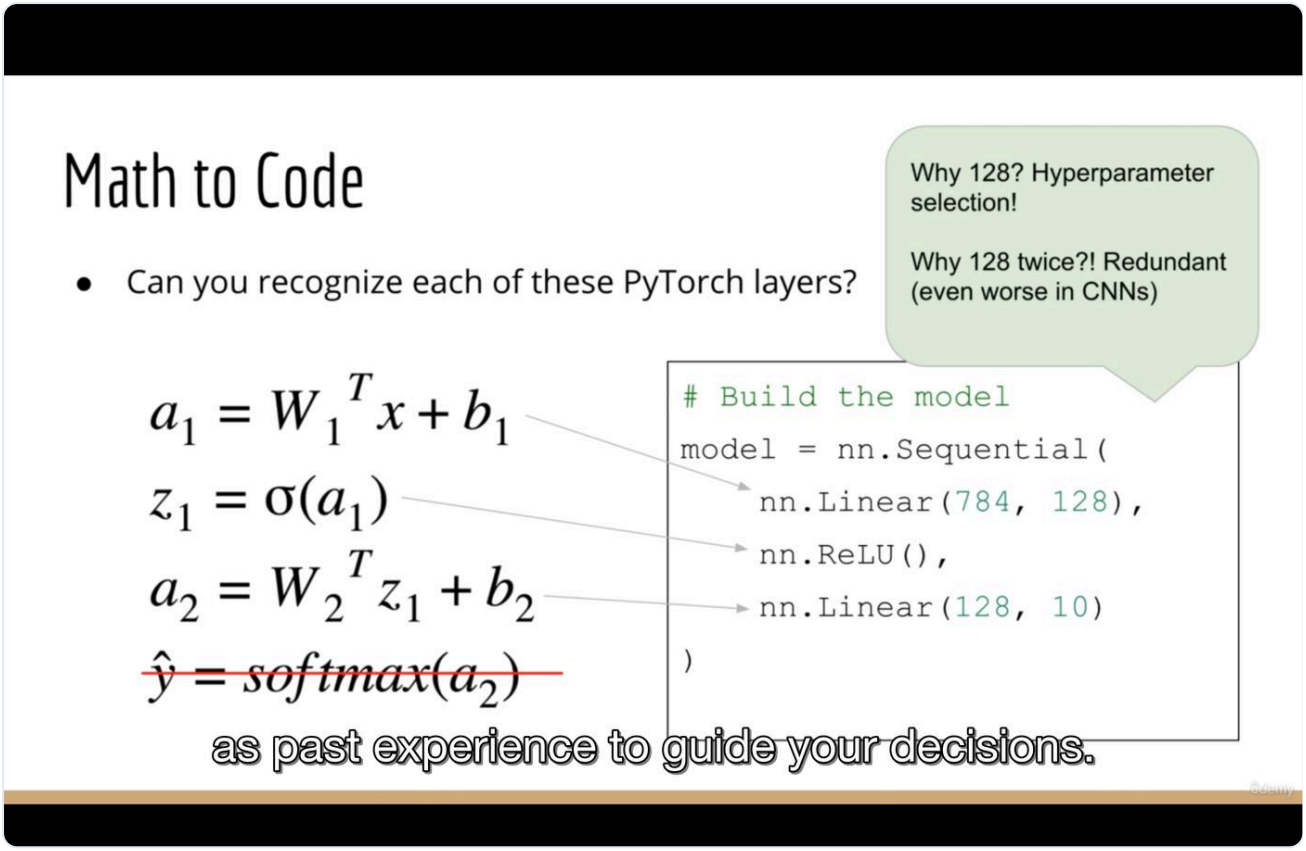


FIGURE 29.10

Size 128. But it also does not make sense in the sense that this is a totally redundant, for example, in TensorFlow 2 or Keras, you only have to specify this number once, and it also leads to more opportunities for error. If you specify different numbers, your program simply won't run. You'll notice that the same theme will apply to later sections of the course as well. And this gets especially complicated for convolutional neural networks. Finally, there are 10 output's because there are ten classes which correspond to the 10 digits, zero up to nine. Step number three is training the model, this will almost be the same loop as we had previously, but with one additional detail.

Previously on Each Epoch, we took the entire data set, made a prediction on it, and then calculated the loss and did gradient descent.

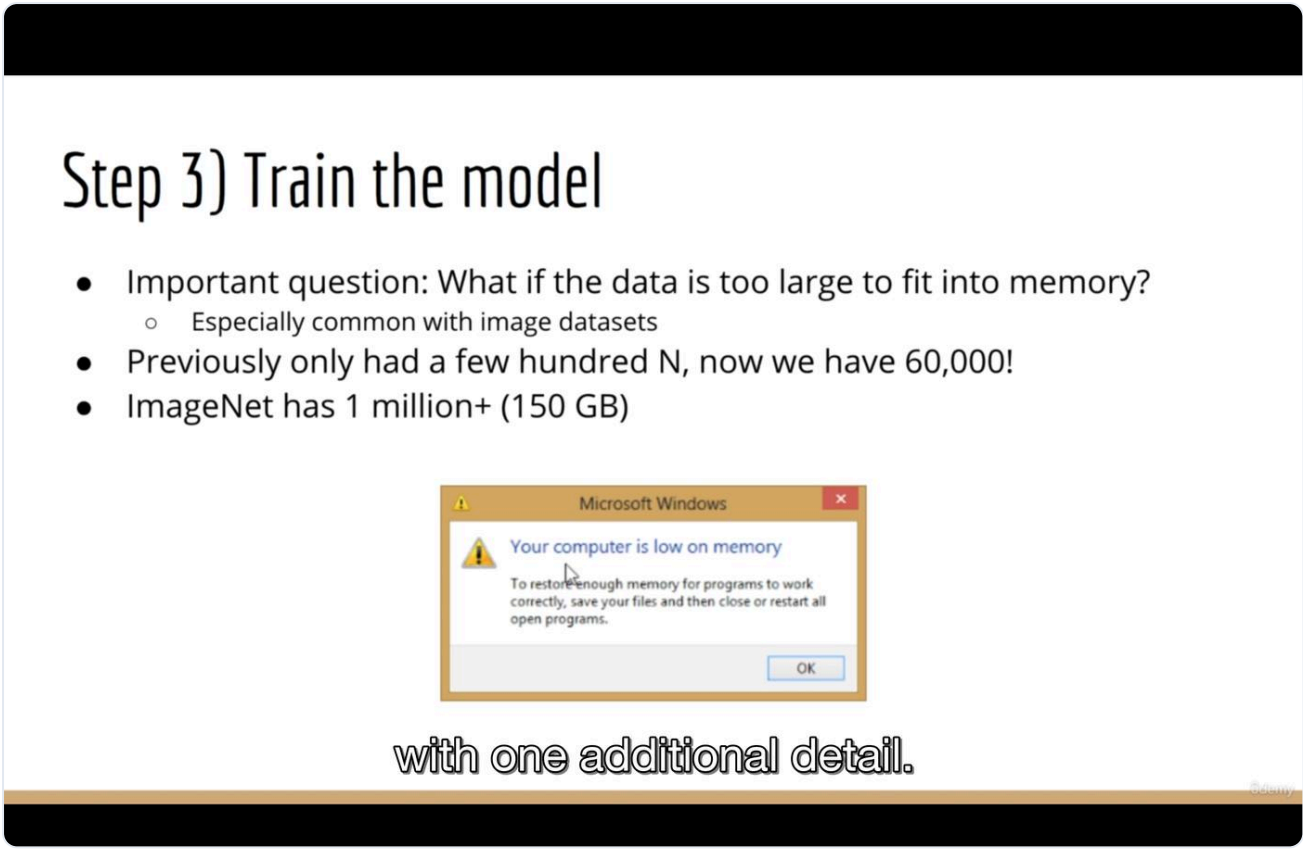


FIGURE 29.11

But ask yourself, what if your data set is actually too large to fit into memory? Previously we just had a few hundred samples. Now we have 60000. Today's data sets, such as the famous ImageNet data set have millions of images, ImageNet is approximately one hundred fifty gigabytes, which can't even fit on some laptop drives, never mind fit into memory. So clearly we need a better solution. The solution is batch gradient descent, and the idea is this let's suppose we split our data up into batches, then instead of having just one loop over each of the epochs, we'll have a nested loop over each batch. Then we'll do all our usual steps on that small batch rather than over the entire data set.

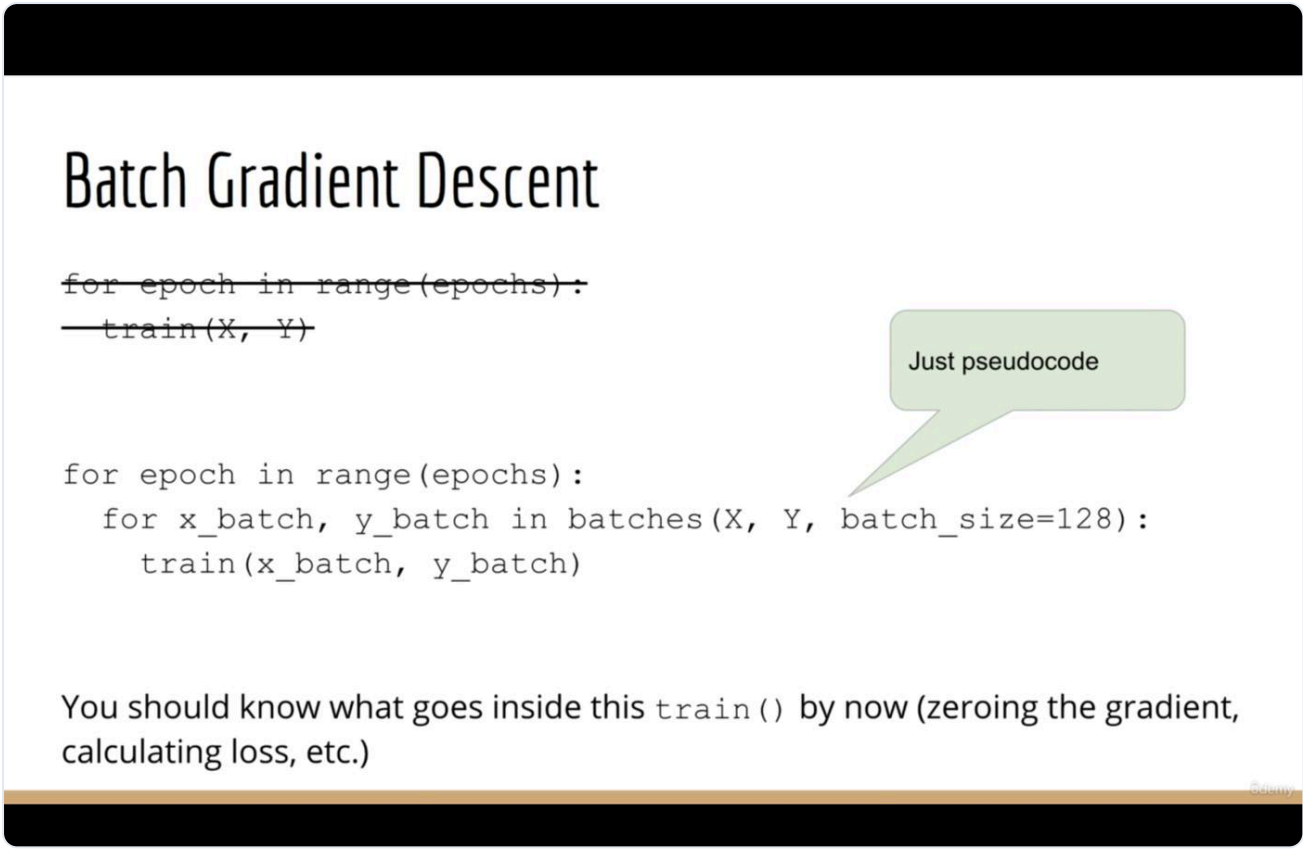


FIGURE 29.12 | Batch Gradient Descent

That includes zeroing the gradient, calculating the outputs for that batch, calculating the loss and doing one step of gradient descent. In fact, batch gradient descent is such a fundamental part of deep learning that the PyTorch library comes with a DataLoader class which comes with a batch size argument and acts as a generator over the data set. So although in my in-depth courses I will teach you how to write a batch gradient descent loop by yourself in PyTorch, we can just loop over a data loader object and it automatically yields batches of inputs and targets. The next question to consider is, why does this work, why is it OK to do gradient descent over batches instead of the entire data set?

Here's some intuition. Suppose we want to know the average height of all human beings on Earth. Think about why that would be an infeasible task, just like how storing all of ImageNet in memory is infeasible.

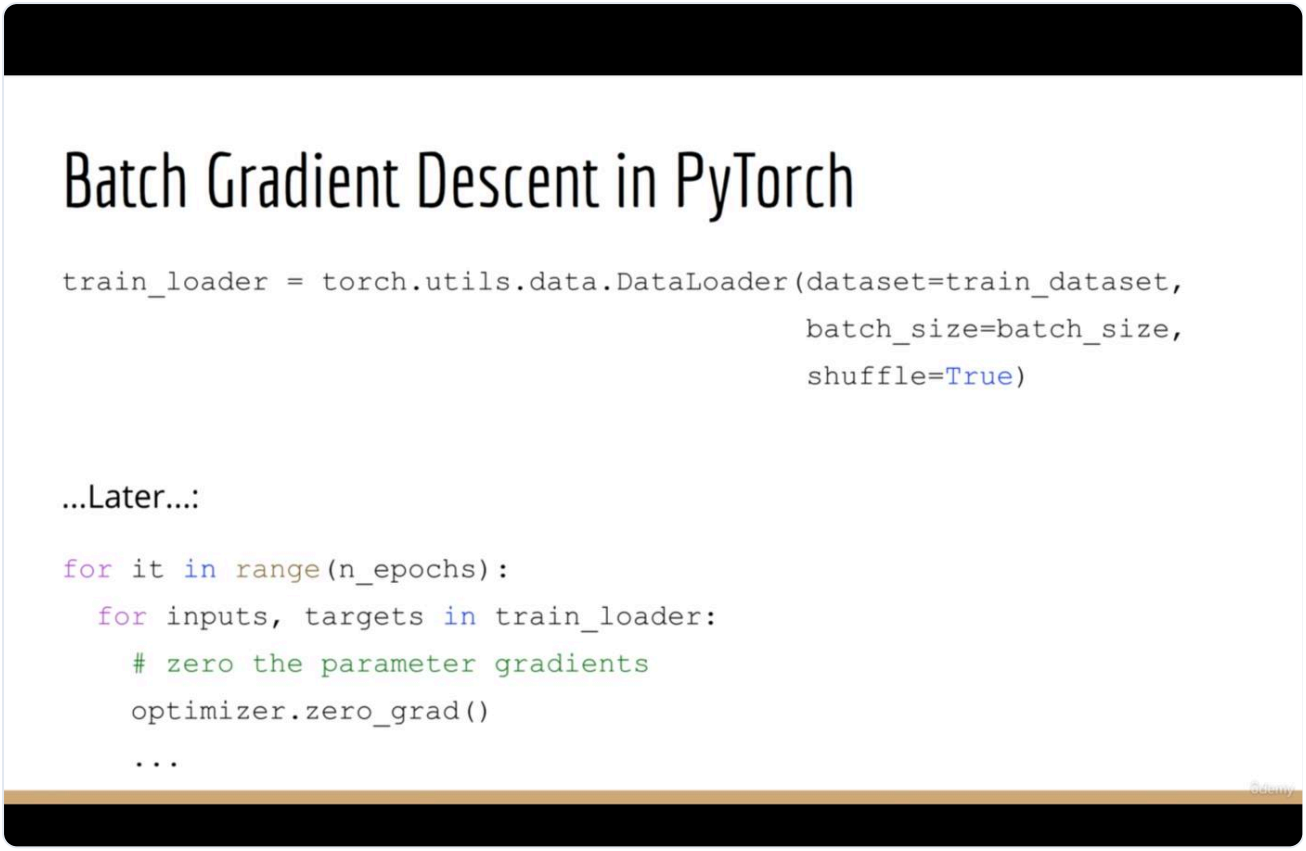


FIGURE 29.13 | Batch Gradient Descent in PyTorch

Well, there are seven billion people currently on Earth. You can't possibly ask them all how tall they are. Therefore, we have to do what all scientific experiments do. Take a random sample from the population. This is what scientists do when they want to test drugs or when they want to do a psychology experiment. They can't possibly ask everyone in the world to participate in their experiment. So they have to choose just a small sample of people and generate statistics based on this sample.

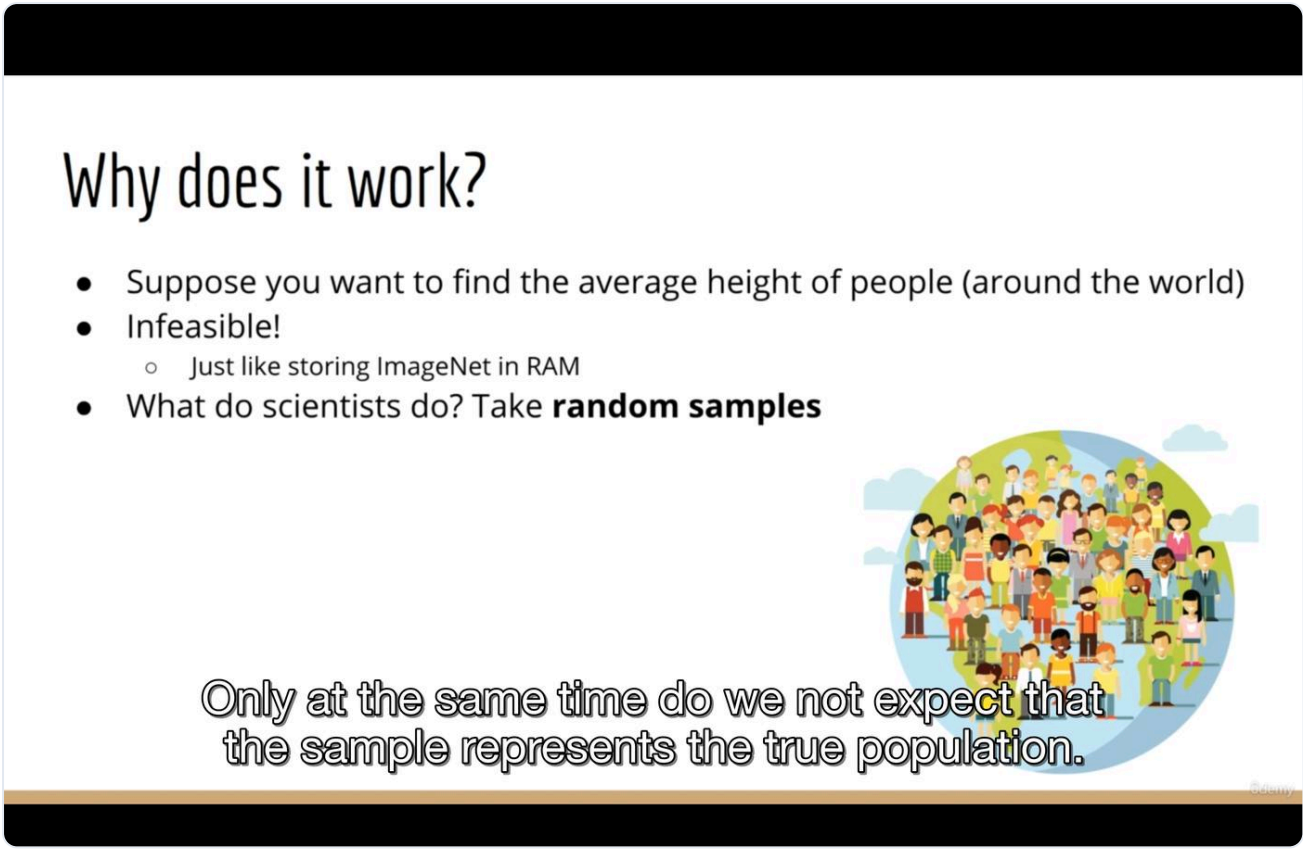


FIGURE 29.14 | Why Does It Work?

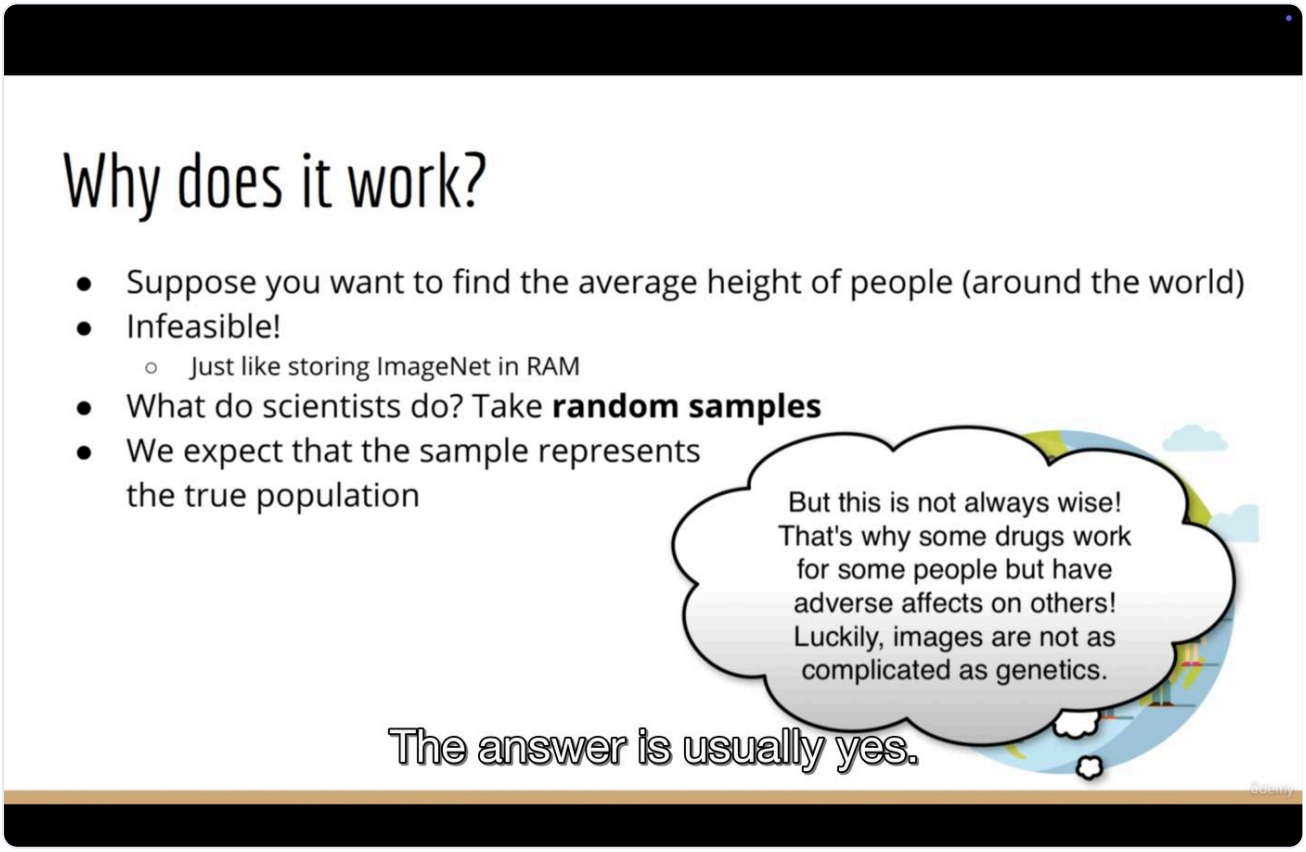


FIGURE 29.15 | Why Does It Work?

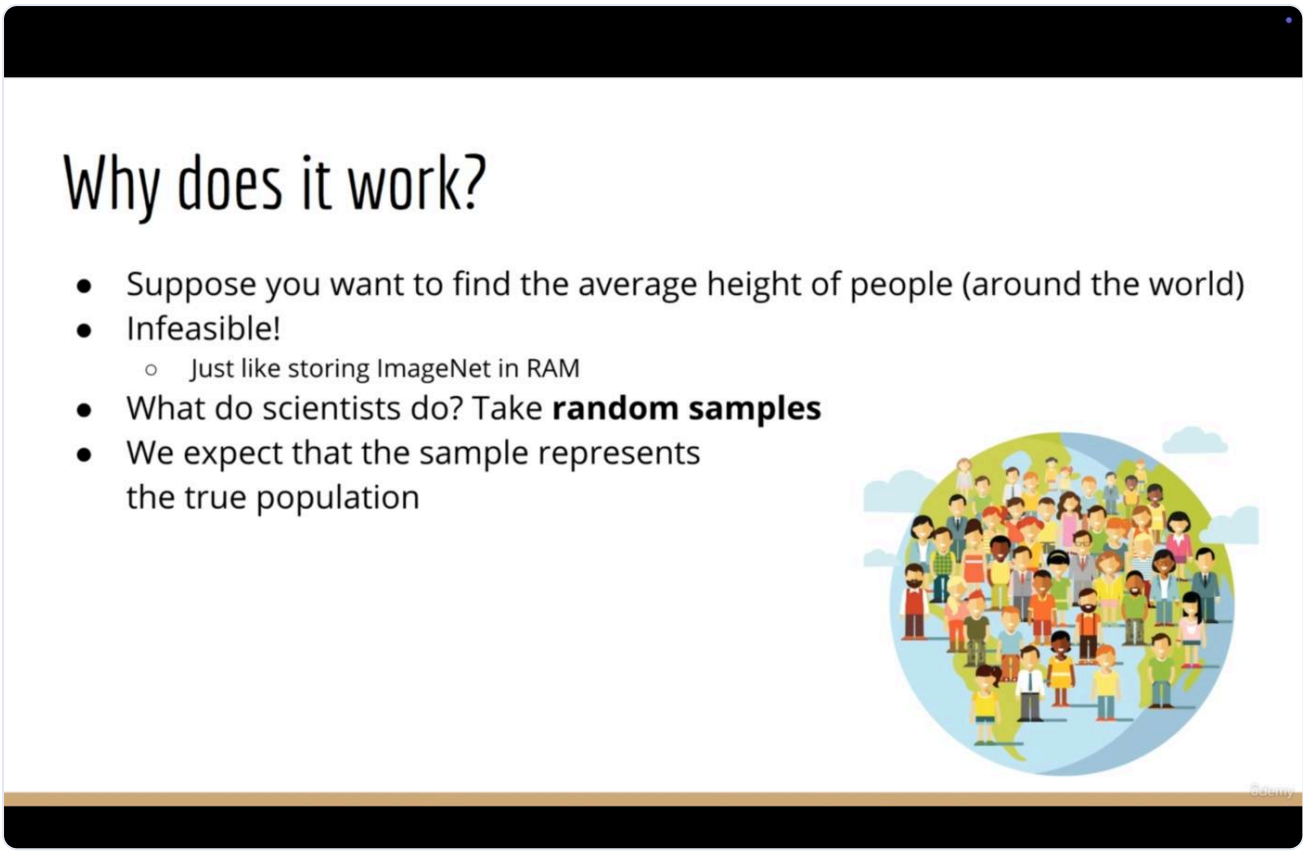


FIGURE 29.16 | Why Does It Work?

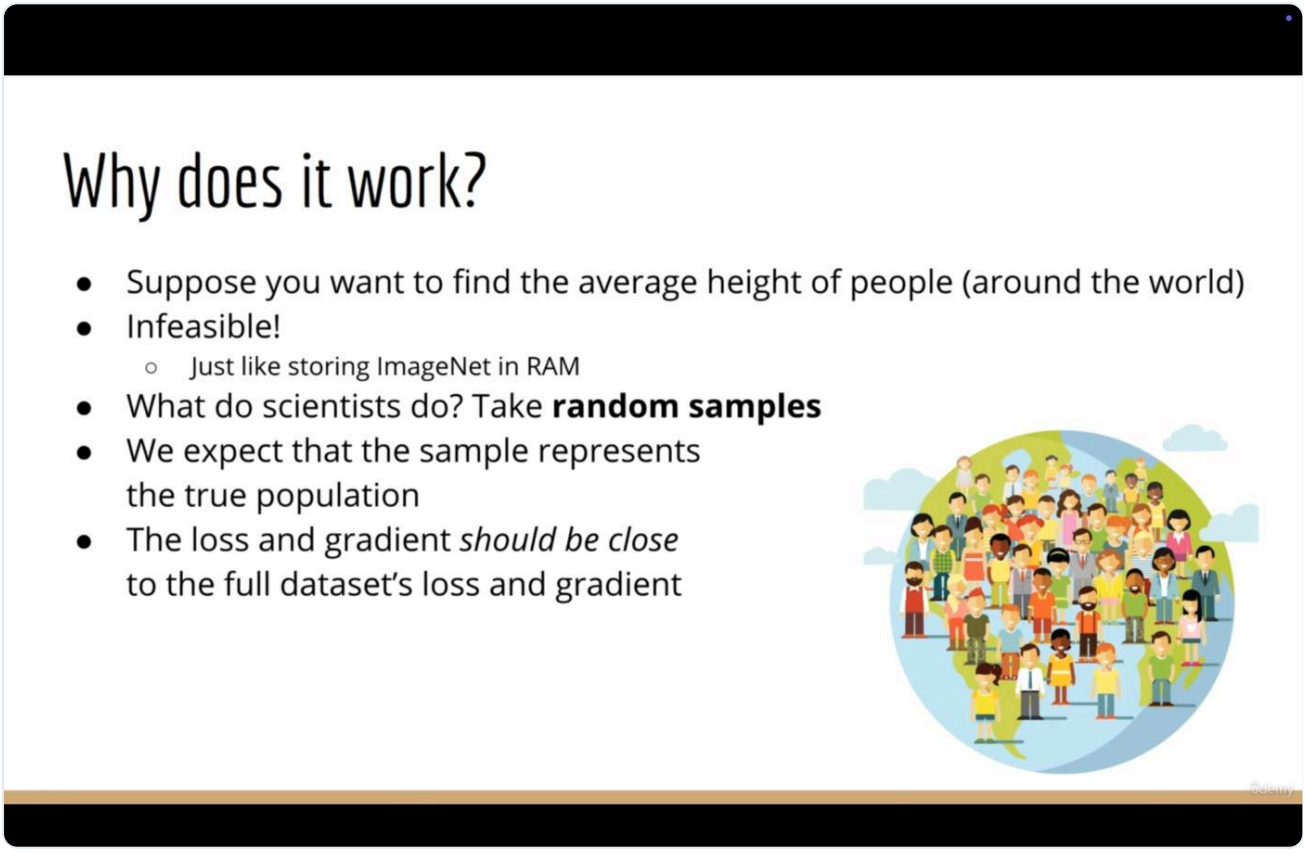


FIGURE 29.17 | Why Does It Work?

Step 4/5) Evaluating and Making Predictions

- Can't evaluate on the entire dataset all at once!
  - Can't model(inputs) where inputs = entire dataset
- Use the data loader again

```
n_correct = 0
n_total = 0
for inputs, targets in train_loader:
    # calculate output for this batch
    output = model(inputs)
    ...
acc = n_correct / n_total
```

FIGURE 29.18

Step 4/5) Evaluating and Making Predictions

- How do we make predictions for multiclass classification?
- Output is logits of size K (# classes), not probabilities (no softmax)
  - Can't check if > 0 as in binary classification
- Hint: Why is the softmax called "**soft**" + "**max**"?
- What does the max() function do?
- Confirm for yourself:
  - If A > B and A > C, then softmax([A, B, C]) = [a, b, c] where a > b and a > c

```
# Get prediction
# torch.max returns both max and argmax
_, predictions = torch.max(outputs, 1)
```

FIGURE 29.19

Only at the same time do we not expect that the sample represents the true population. The answer is usually yes. Do we expect that our measurements on the sample will be close to the measurements we could theoretically take on the entire population? The answer is yes. The same thing happens when we do batch gradient descent, even though we are only looking at a small sample of the data at any given time. We do expect that the loss and its gradient will be representative of the entire dataset, which itself is what we expect to be representative of the true distribution of where the data set comes from. Furthermore, on each epoch we still see every data point, so we don't miss anything unlike real world population samples.

Finally, step one, step five are to evaluate the model and make predictions using the model, as we just discussed, with large data sets, it may not be feasible to load in the entire data set into memory. Therefore, we can't just do model of inputs and pass in the entire data set all at once. And so this is yet another place where we have to make use of our data loader objects due to the fact that our predictions come in batches, we won't just have a single array of predictions to compare to the targets like we did before. One way we can get around this is to just apply the definition of accuracy directly and calculate it ourselves. For each batch we get the predictions and the targets.

We don't want the accuracy on this batch only, but what we can do is count up the number of predictions we got. Correct. If we tally up how many we got correct and how many samples we saw in total, then we can just divide the number correct by the number total once the loop is over. And that's our accuracy. Another complication is how we actually make predictions in the first place. Recall that we are no longer doing binary classification. So it's not a matter of just checking whether the output is greater than zero or less than zero instead the output is of size K where K is the number of classes. As you know, these K outputs will theoretically be passed through a softmax function to give us the probability that the input belongs to each of these classes.

But do we need these probabilities? Well, just like with binary classification, the answer is no. Why is the softmax function called softmax function in the first place? It's because it's kind of like the max function, except that instead of just telling us which element is the biggest, it just weights each of the elements relative to each other. In other words, a softer version of the max. In other words, the maximum value before we do the softmax will still be the maximum value after we do the softmax. It's just that the softmax happens to map those values to probabilities. Therefore, we just need to pick which of the K outputs has the maximum value and not worry about them being probabilities or not.

In PyTorch, this is accomplished with torch dot max, which returns both the maximum value and its index. And because we're doing classification, we're actually interested in the index and not the value itself.

LECTURE 30

Data

Check the code in the notebook `4.PyTorch-ANN-MNIST.ipynb` .

In this lecture we are going to look at the notebook which implements a feedforward neural network for image classification on the MNIST data set. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at. So as usual we're going to start by importing PyTorch NumPy and matplotlib. We'll also be using the torchvision library which includes the MNIST data set and utility functions for handling images.

We'll start by downloading the train data which happens when we call the torchvision dot datasets dot MNIST function for the first time. Let's go through each of these arguments one by one so you understand that. First we have the root argument where we specify the file path we want to download the data to. In this case we're going to download the data to the local directory. Next we set train equal to true which indicates that this function will return the train data set. Next we set transform to transforms dot ToTensor which is an operation from the torchvision library that does some useful preprocessing for us. This way everything we want to do to the data happens inside this function.

We'll see later how we can make even more sophisticated use of the transform argument. Finally we have the download argument which tells PyTorch to download the data from the printout we can see that in fact 4000 downloaded. If you read it carefully you can deduce that these are the train inputs. The train labels the test inputs and the test labels. You can also see that they've been downloaded to the folder MNIST slash raw. Next we're going to print out the data attribute of the train dataset which as we know represents the input data. Really what we want to do is make sure what we're seeing makes sense. So what we can see is that this appears to be a three dimensional array of a bunch of zeros.

It also has this type equal to torch dot uint8. Does this make sense. In fact it does. There are of course some non-zero values in the dataset but we can't see them because a large percentage of each image in the MNIST dataset is black. Only the digits themselves are nonblack and those take up a tiny part of the image so it's not surprising that if we print out all the values near the edges of the image that they would be all zero. We can verify this by checking the maximum value in the tensor which is as expected 255. Next we can check the shape of the tensor which is sixty thousand by twenty eight by twenty eight. As I mentioned earlier finally we can print the targets attribute which returns a one dimensional tensor of integers which appear to be between 0 and 9.

As expected next we're going to call the MNIST function again but this time we want the test data set. So we need to pass in a train equal to False. Notice how this time nothing is downloaded. That's because the four files we needed to download were already downloaded in the previous

step. When we check the shape of the input data we see that it's ten thousand by twenty eight by twenty eight as promised. So there are sixty thousand training samples and 10000 test samples now that we have our data. It's time to build our model. As you can see it's exactly as I've specified in the earlier lectures. We have a linear layer followed by a ReLU followed by another linear layer all wrapped in a Sequential and there is no need for a softmax as we know it's been combined with the cross entropy loss for numerical stability the next step is new since.

From this point forward we'll be looking at larger and larger data sets. There is a need to make use of the GPU we know that GPUs are useful for speeding up deep learning because while they were originally built for gaming a lot of the matrix algebra that happens in physics engines is the same as the matrix algebra that happens in deep learning so we can make use of gaming technology to speed up deep learning computations. So basically if you have a GPU available it will be called CUDA colon 0. So what we're doing here is checking if that string is in the list of available devices. If it is we'll set that to be the device and if it's not we'll set the device to be the string CPU next.

And this is important we call model dot to device. This transfers all the parameters of our model to the GPU you can picture your RAM and your GPU as being two different physical spaces which they are and in order to do any computation like multiplying or adding all the numbers we want to compute on have to be on the same device. So whenever we want to calculate the output of our model both of the models parameters and the input data have to be on either the GPU. Or on the main RAM but we can't have one set of numbers on one device in another set of numbers. On the other device next we're going to set our loss and optimizer which you've already seen.

Note that this is our new loss the cross entropy loss meant for multiple categories next there's another new thing we're going to create generators which will allow us to loop through each batch of data as we iterate through epoch. These are called data loaders in PyTorch for the input arguments we specify the data arrays which are the train data set and test data set variables we specify the batch size and we specify whether or not to shuffle the data you'll notice that we shuffle the training data but not the test data. This is because for the training data if we loop through each sample in the same order each time this will introduce unwanted correlations which will decrease performance.

Think of our example of measuring the average height of everyone in the world or more practically drug testing. We always want our sample to be random for the test data. There is no need to shuffle it because all we want to do is evaluate the loss and accuracy. Next we have a little snippet of code just to test how the data loader works. As mentioned it's a generator so we can do a full loop over it. For this data loader I'm going to set the batch size to 1 and I'm going to print out not just the shape of the data we get on each iteration but also the data itself. We can see something interesting. If we look at the data which is that the data now ranges from 0 to 1 instead of from 0 to 255.

This is kind of unintuitive at first because you might think why was it that when we printed the data earlier it was 0 to 255 but only when we use the data loader as it becomes 0 to 1. This seems strange because it seems like the data loader object should be generic. It can generate data from any kind of data set that you pass in. In fact the normalization comes from the ToTensor function

we passed in earlier to the MNIST function although we didn't see it at the time because it does not get called when you just call the data attribute. So in the next line we can see what happens if we call the transform function manually on the data. The max is 1 as expected and so this transform function is called internally by the data loader but it's not called when you just do train data set dot data next we're going to train our model.

You'll notice that we only need 10 epochs to train this model which is a lot less than before. Why might that be. If you think about it this goes back to the theory behind batch gradient descent as you recall a batch is a representative sample of the entire dataset. In other words doing batch gradient descent on a single batch is like doing gradient descent on the entire dataset but much faster since we're only looking at a small subset of the data and keep in mind that the total number of iterations is still very high because we have sixty thousand data points to loop over. In fact some data sets are so large that we only do a single epoch over the entire dataset.

So that's why the total number of epochs is small here. It looks small but we're actually still doing a pretty large number of gradient descent steps in total. Now I hope that you recognize most of the elements of this loop at this point. As mentioned we'll be looking at more or less the same thing for each example throughout the course. So what's different. First we have two nested loops one over the epochs and one over the train loader. Before we go into the inner loop we first need to create an empty list to store the train loss. We want the loss per epoch but what we actually get is the loss per batch. So let's just store the losses for each batch and then deal with them at the end.

Alternatively you could just plot the loss per batch instead of the loss per epoch. Next we enter the inner loop through the train loader. The first thing we do in this loop is to transfer the data to the GPU as you recall our model parameters are already on the GPU. And if we want to do any computation between the model and the data both have to be on the GPU next we're going to reshape the data to be N by D where D is 784 and N is the batch size. Note that for flexibility we can specify the first argument as minus one which tells PyTorch to just assign whatever value is appropriate for the data we're given. This works the same in NumPy. If you want to try it out next we have all the usual steps to perform one step of gradient descent.

At the end when we get a loss we append it to our list of losses for this particular epoch. Once we're outside the epoch we simply take a mean of this list to be the train loss for that epoch as noted. This is a little misleading. Technically it's not the loss for the epoch because we have been training the entire time we've been looping through this epoch so the losses we've collected represent a number of different states of our model. Nonetheless it would be inefficient to have to loop through the data twice just to calculate the loss so we don't really want to do that if we don't need to Next we calculate the Test loss which goes through a similar loop.

Remember that we can't calculate the test loss all at once because it may not fit into memory. If our dataset is too large this loop has all the same steps as the train loop except for the backward and optimizer steps. Finally at the end of the outer loop we store the losses we collected and print them out. As usual we would like to see the loss per iteration which is what we have next. The

next step is to calculate the accuracy again having a data loader instead of just a plain array makes things more difficult. So we have to loop through the data rather than just do a single computation. We start by initializing the counts for the number correct and the number total to zero.

Then we loop through the data loader inside the loop. We transfer the data to the GPU reshape the inputs and then get the outputs these outputs are logits and not probabilities but that means we can still take the max to get the prediction. We call torch dot max over axis 1 which means that we take the max over the columns and this returns both the maximum value in each row and the corresponding indices. We only want the indices since that corresponds to the classes so the first thing returns from this function would be the maximum value. And the second thing returned from this function would be the index. Next we check how many of the predictions are equal to the targets and assign that to N correct since this is all in PyTorch land.

We have to call dot item to bring it back to Python land in order to get the number of samples. That's just the shape of the targets at the 0th index. Finally when we're outside the loop the accuracy is just the number correct divided by the number total. As you've seen next we do the same thing for the test accuracy and when we're done we print the train accuracy and the test accuracy as you can see we do pretty well. Next we have some code to plot a confusion matrix. This code is outside the scope of this course. So you don't have to worry about understanding it just knowing what a confusion matrix is is what I want you to focus on. So the idea is we want to draw a table with the true labels on one axis and the predicted labels on the other axis so on this next block of code we have a loop similar to the above where we make predictions but now we just store them in numpy arrays instead of calculating the accuracy so you see here we use concatenate rather than summing and at the end we call our confusion matrix function so basically the confusion matrix tells us for each label how many predictions correspond to that label.

Hopefully we expected to find most of the entries along the diagonal where the label would be equal to the prediction. Of course since we don't have 100 percent accuracy there are a few entries not on the diagonal. Importantly however it's good to try and see if we can make sense of these results. They're very interpretable because we're working with images. And when you have images it's very easy to simply look at them. So where do we have the highest inaccuracy. It seems to be where the true label is for and the predicted label is 9 this actually makes sense. If you write down a 9 the connection points in the main lines all appear in approximately the same spot.

You can imagine that it would be easy for a human observer to make the same mistake. We can look for more instances of a large number of predictions being incorrect such as a three being confused with an 8 and a 2 being confused with an 8. You should be able to recognize why these mistakes were being made. The next thing we do is actually plot some of these misclassified samples to confirm our suspicions. Perhaps the misclassified forces really do look like science in order to find this out. We're going to select a randomly misclassified sample and plot the image to do this. First we have to gather the indices of all the misclassified samples. We can do this using the numpy where function the where function simply returns the index value where the input array is true so we can pass in the boolean array p test not equal to y test.

Note that the where function returns the results in a tuple but we only care about the first element. So that's why we use the square brackets 0. Next we're going to use np dot random dot choice to select one of these random misclassified indices and assign this variable to ie. So let's check out the results. So here's a 5 predicted as a 9. That kind of makes sense because this looks like a 9 use a for predicted as a 9. As you can see it kind of looks like a 9 here's a 2 predicted as an 8 here's a 3 predicted as a 9 here's a 5 predicted as a 3 Here's a 7 predicted as a 2. So this one actually looks like a 2 so I think overall it makes sense what we're seeing digits that confuse the computer are the same as digits that would confuse us.

LECTURE 31

Original Data Points and Synthetic Data

Check the code in the notebook `5.PyTorch-Regression.ipynb`.

In this lecture we are going to look at a notebook which demonstrates the use of a feedforward neural network for regression. This lecture is going to walk you through a prepared at Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at. The important thing about this notebook is that unlike the other examples we did this one is going to involve synthetic data. That means data that we are going to create rather than data we've collected from an experiment or a database.

Now you might ask why should we do this. Is it even practical. Is this just the lazy programmer of being a science geek. And of course if you think any of these things you are wrong using synthetic data is very important for understanding the behavior and quirks of any machine learning algorithm you want to be able to see where it succeeds and where it fails. The important keyword there is. We'll be creating a dataset which we can actually visualize and look at. So we can observe that the neuron network will create a prediction surface that corresponds to the true function that we generated the data from of course. Don't take my word for it that this is important.

One great example of this is in the field of clustering. So if I go over to scikit-learn's web page on different clustering algorithms you can see a ton of examples of this. In particular it shows a set of some pretty contrived two dimensional data and then it compares how each algorithm clusters this data. By doing this we can see where each algorithm is successful and where it is not. In fact this is the same thing you saw in the TensorFlow playground without being able to see the decision boundary and the corresponding data with your own eyes. You can imagine that this website would be much less engaging. Okay so now you know why visualizing all synthetic data is important onto the code as usual.

We're going to start by importing PyTorch, NumPy and matplotlib. We'll also need this 3D thing to help us make three dimensional plots. Next we're going to create the dataset. We'll start by creating the inputs X which will just be two dimensional data points uniformly distributed between minus three and plus three. As you know the random function returns data uniformly distributed between 0 and 1. So if we multiply by six and subtract three the scales the data to be between minus three and plus three. Next we calculate the targets Y which is just the cosine of two times the first feature plus the cosine of three times the second feature. Now you might ask Why am I using this function.

Is it some kind of special function. The answer is no. I just wanted a non linear function with a few bumps and curves and the cosine does just that. Next we're going to plot this data on a 3D scatter plot so that you can see what it looks like. So let's run this now one thing I don't like about notebook compared to running a real python script is that you can't play around with the plots.

So for stuff like this I like to run the actual python script on my own computer which shows you the plot in a separate window. Then you can zoom into the plot rotated and so forth. This makes it a lot easier to get a good feel for the data. So if you run this on your own machine you'll be able to rotate the plot and look at it from different angles.

So you can really see what the underlying function should look like. So next we're going to build the model since the data is in the form N by D where d equals 2. There's no real preprocessing to do so this is almost the same architecture as the previous example. We have one hidden layer with 128 hidden units and a ReLU activation. The difference is now we have two inputs and one output instead of seven hundred eighty four inputs and 10 outputs. And since this is regression there is no activation function and we're regressing on a single scalar value. So the output dimension is 1 Next we're going to create the loss and optimize optimizer since we're doing regression.

We'll use the MSE loss. You'll see that instead of using the default Adam I've set the learning rate to 0.01. Now you might be wondering where can I learn about all these different optimizer objects and what their arguments are. The answer is as always the PyTorch documentation. Your next question might be how do I know that a learning rate of 0.01 is good. And remember that this is because I actually sat down and tried it. This is the only way. Again if you're ever wondering how many epochs or what learning rate should I use. The answer is to try it and see next we have our training loop which I've put into a function. This may or may not be useful in the future for copying and pasting.

In any case I just wanted to show you an alternative way of writing this other than the fact that the loop is a function. Nothing else here is new. Next we're going to plot the loss per iteration to confirm that the training process converge nicely if it doesn't. That means you have to go back and modify your hyperparameters so this appears to look OK. Now we have the next step which is to make predictions using the data since this data is two dimensional and visualizing will we have a very nice situation we can actually plot the entire prediction surface as a function along with the original data points. This will confirm to us that the neural network even though it's just a bunch of linear equations with ReLUs can approximate this sum of cosines.

That's pretty amazing if you think about it. We can approximate a cosine equation without having any cosine. Now some of you might be wondering how we actually make this 3D surface plot so let's go through that the first thing we need to do is create a mesh grid we can start by choosing points along the x axis and the x 2 axis which I've said are just fifty evenly spaced points between minus three and plus three. So that's what linspace does. Next we call the mesh grid function which basically does the cross product between these two sets of points. So every X1 and every x 2 paired up with each other we assign these to the variables x x and y y which represent the first axis and the second axis respectively unfortunately this data is in the right format for our machine learning model which expects and an N by 2 array as input in order to convert this into an end by 2 array.

We're going to flatten x x and y y pass it into the V stack function which stacks these 2 arrays vertically and then transpose the result you can do some simpler examples on smaller arrays to confirm to yourself that this does the right thing. Next we use our model to make a prediction on X grid and then flatten the result since the result is N by 1. Finally we call the plot_surface function

which plots the 3D surface. The first three arguments are the X1 axis the x 2 axis and then the function value which is why the other arguments are not so important. But you can play around them if you want. So let's run this Okay so we can see that the neural network manages to approximate this function quite well.

As mentioned previously you get much more out of being able to manipulate the plot by rotating and zooming in so as an exercise you might want to try exporting this script as a python file. So you do that as file download dot py and then run this on your local machine so I'm not finished running this on my machine. So here is the loss here's the predictions along with the original data points so it really helps to be able to rotated and spin it around all right. And the last thing we're going to do here is try to test out if the neural network can extrapolate so we know that the cosine function is going to repeat periodically from minus infinity to plus infinity.

So you might wonder can the neural network figure that out. So what I've done here is I've changed linspace to go from minus five to plus five. So just to make it a little bigger. And if we run this we see that it doesn't quite work. So which is assumes that the pattern keeps going in the direction that was going at the edges and of course this is because the neural network doesn't use a periodic activation function. So we wouldn't expect it to be periodic with respect to the inputs.

LECTURE 32

Common Sense Things to Try

So in this lecture, we will be answering the question, how can I choose hyperparameters? Please note that while I've discussed this in the course already, sometimes people forget. So I'm making this lecture to explicitly remind you of this topic. Now, this is often a point of frustration amongst beginners. Beginners often feel like there must be some direct method of doing this. They think because this is science, that there must be an answer, that there must be some formula or some algorithm that will yield to the correct results. Unfortunately, this is not the case. And as a responsible and mature scientist, you would do well to get comfortable with this reality as a side note.

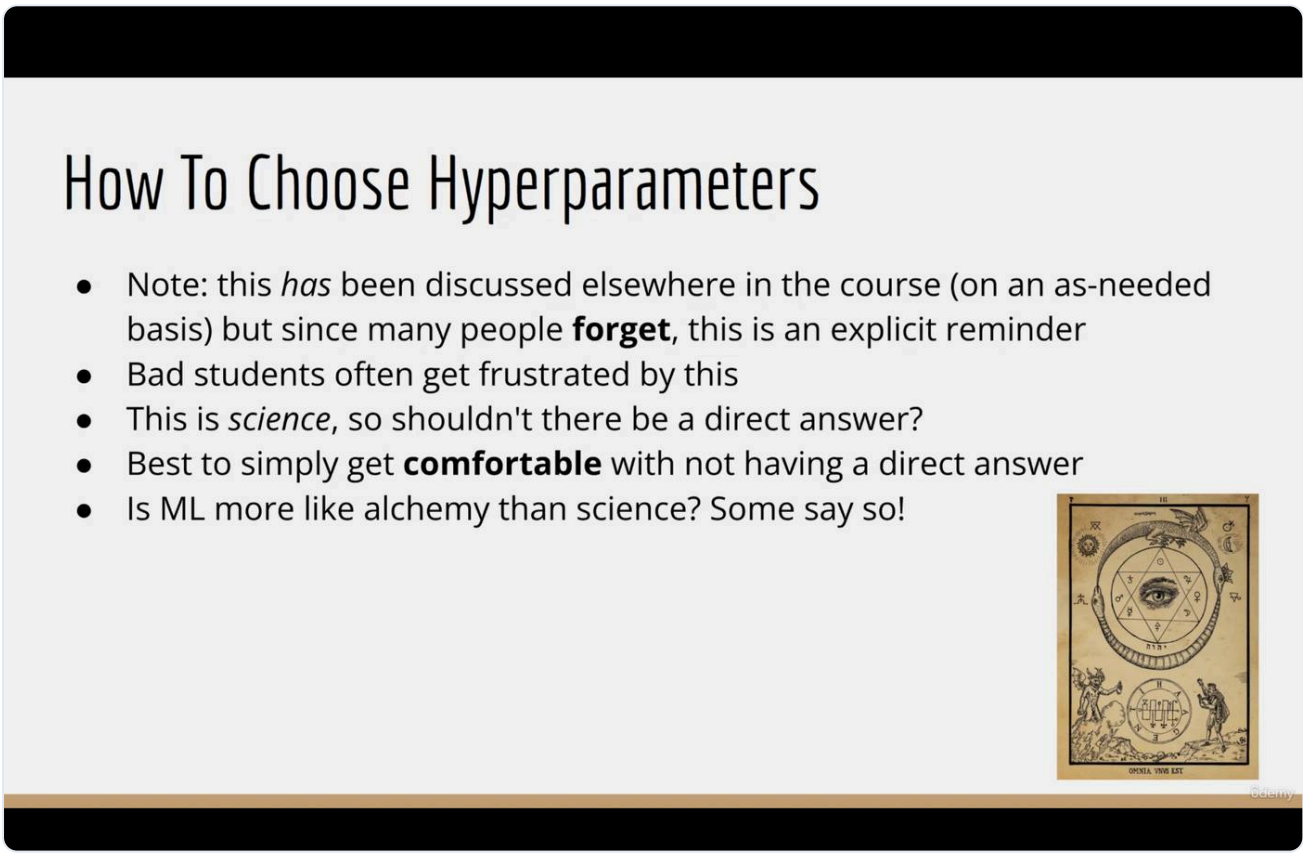


FIGURE 32.1 | How to Choose Hyperparameters

Some have said that machine learning is more like alchemy than science. I would tend to agree with this notion, and this is one of those areas that makes it apparent.

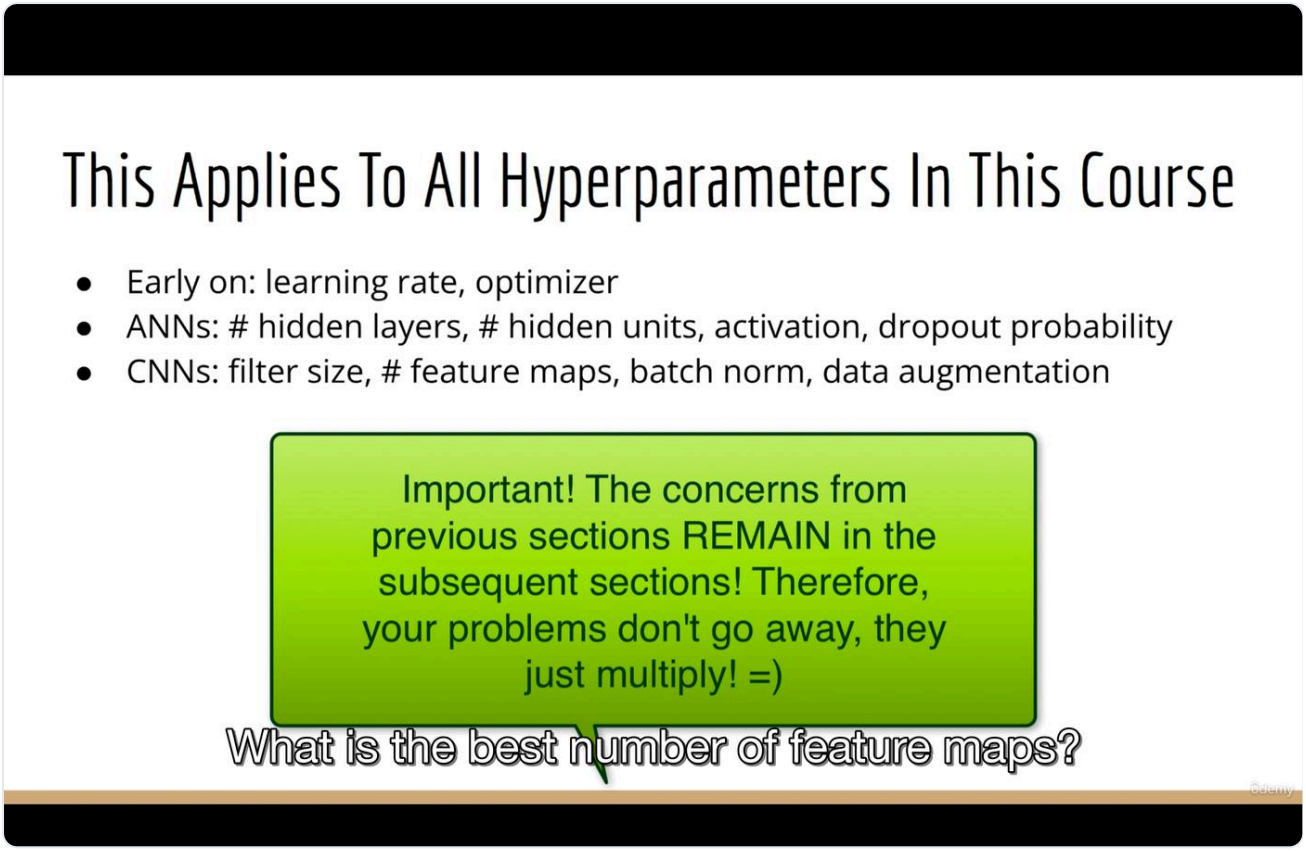


FIGURE 32.2 | This Applies to All Hyperparameters in This Course

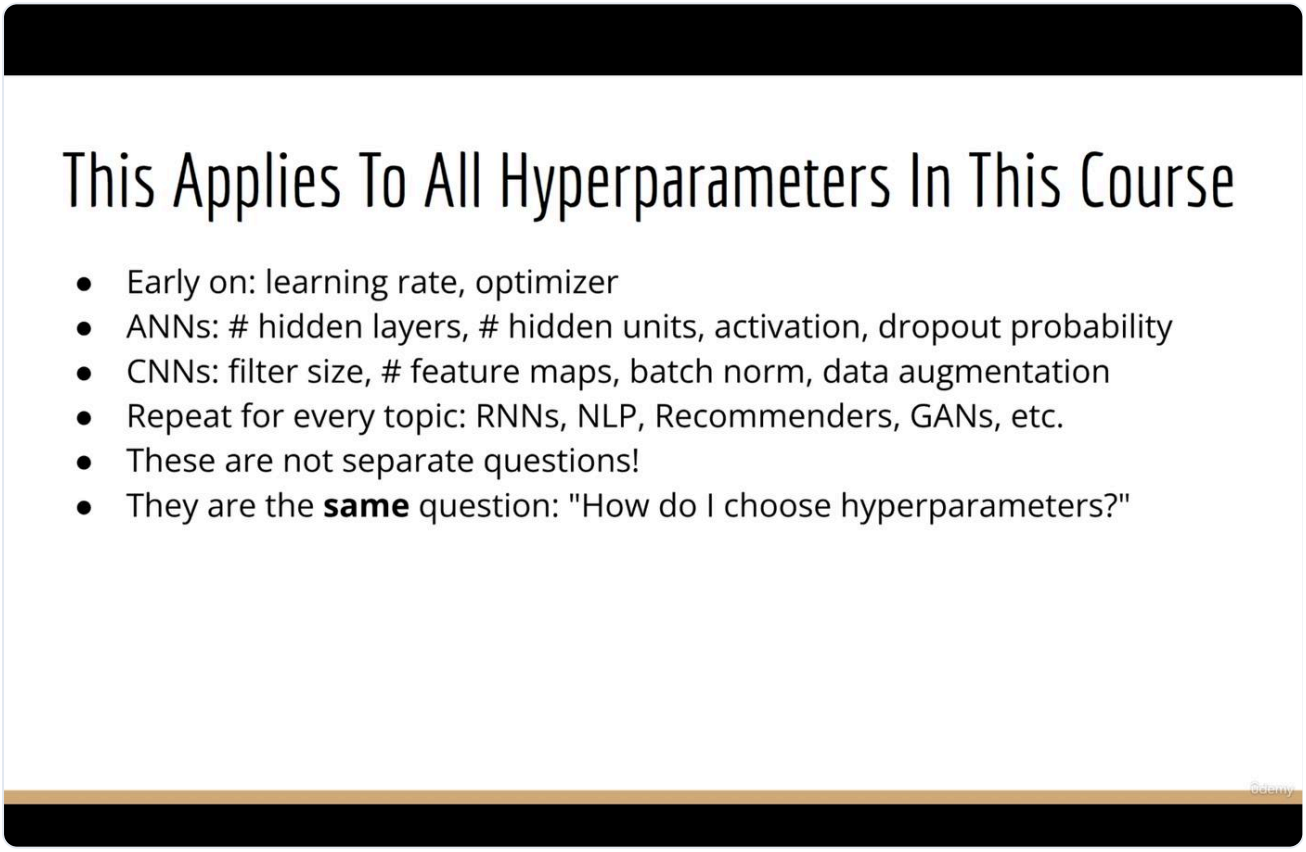


FIGURE 32.3 | This Applies to All Hyperparameters in This Course

So before we continue on, I want to mention that this applies to all the hyperparameters discussed in this course. For example, in the early parts of the course, our main concern is how do we choose the learning rate? What is the best optimizer in the ANN section? Our concerns may be how do we choose the number of hidden layers? How do we choose the number of hidden units? How do we choose the best activation? What dropout probability should I use in the CNN section? Our concerns may be what is the best filter size? What is the best number of feature maps? And these general themes repeat throughout as you learn about RNNs, NLP, recommenders, GANs and so forth.

So please keep in mind that this is not actually a separate question for each of these items. These are all actually the same question. Specifically, it's the question of how to choose hyperparameters. So the answer to this question, although many of you may find this disappointing, is my famous rule. Machine learning is experimentation, not philosophy. The problem is, many beginners start to look for a deep philosophical reasons for choosing this number of hidden layers or that number of hidden units. In fact, the correct answer is to simply try random values and see what result you get. Obviously, the final choice should be the one that gives you the best results.

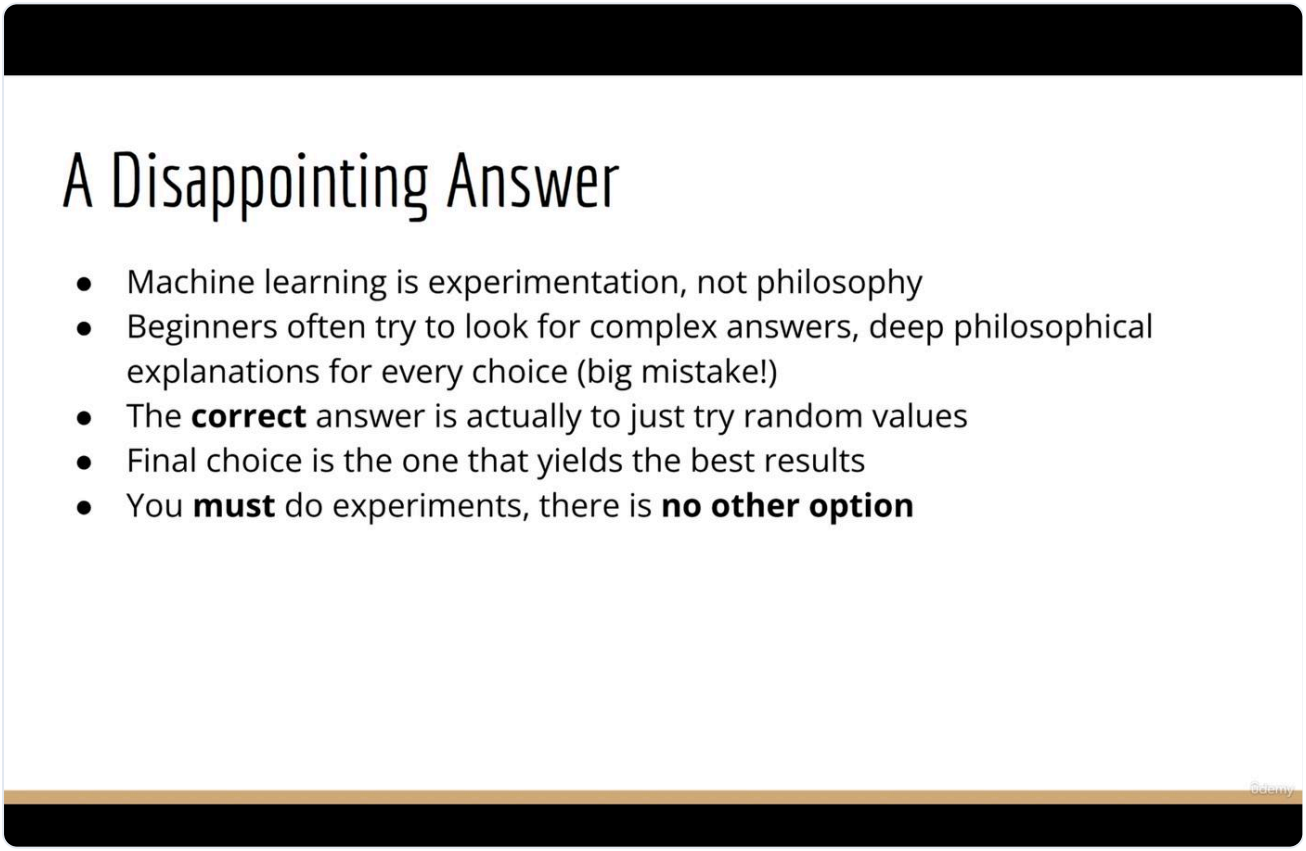


FIGURE 32.4 | A Disappointing Answer

The basic idea is that you must do experiments. There is simply no other option. One obvious thing you can do if you want to be sure you're at least in the right range is to look up the papers in your field to see what they used. This is assuming that your data sets are similar, so that should give you a reasonable starting point for your random search. But do keep in mind that this adds a bias and could potentially prevent you from searching closer to the optimal point. This will also

help you get a sense for the scales at which to try random values. For example, learning rates in many other parameters are usually chosen on a log scale. For example, zero point one zero point zero one zero point zero zero one and so forth.

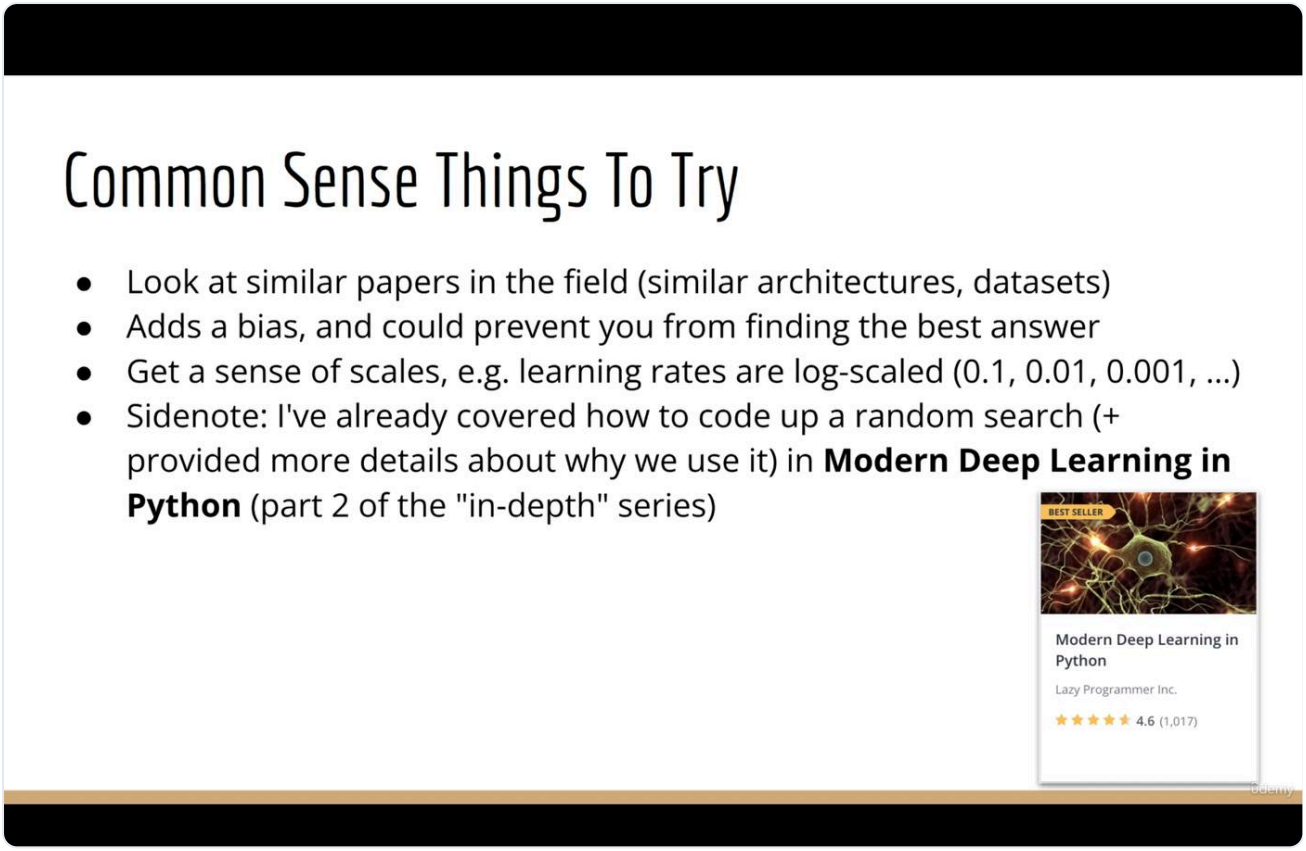


FIGURE 32.5 | Common Sense Things to Try

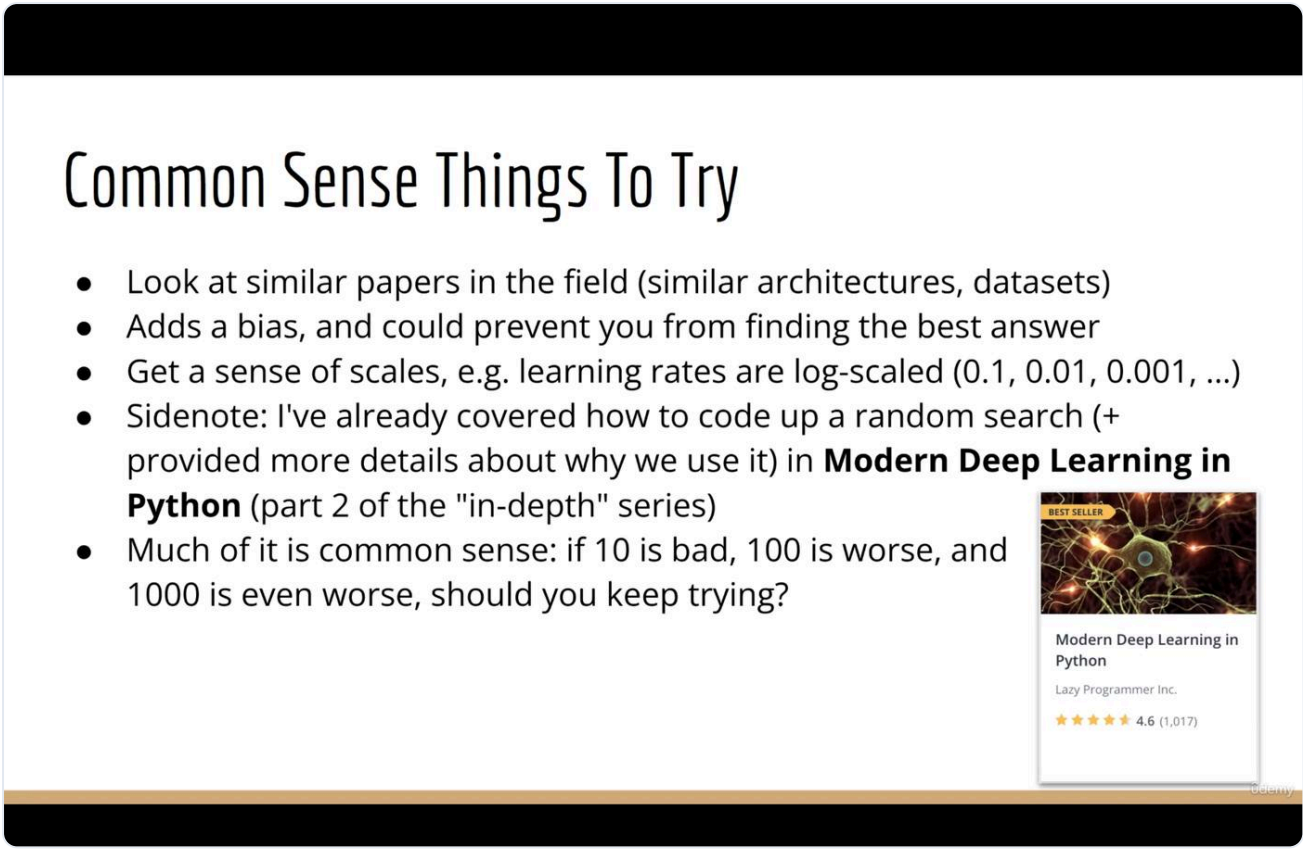


FIGURE 32.6 | Common Sense Things to Try

If you're interested in seeing more specifics, including actual code, to run a random search, I've already covered this in my in-depth series of deep learning courses, specifically modern deep learning in Python. Do note, however, that most of it is just common sense. For example, if a value of 10 is bad and 100 is worse and one thousand is even worse, you probably won't continue one in that direction.

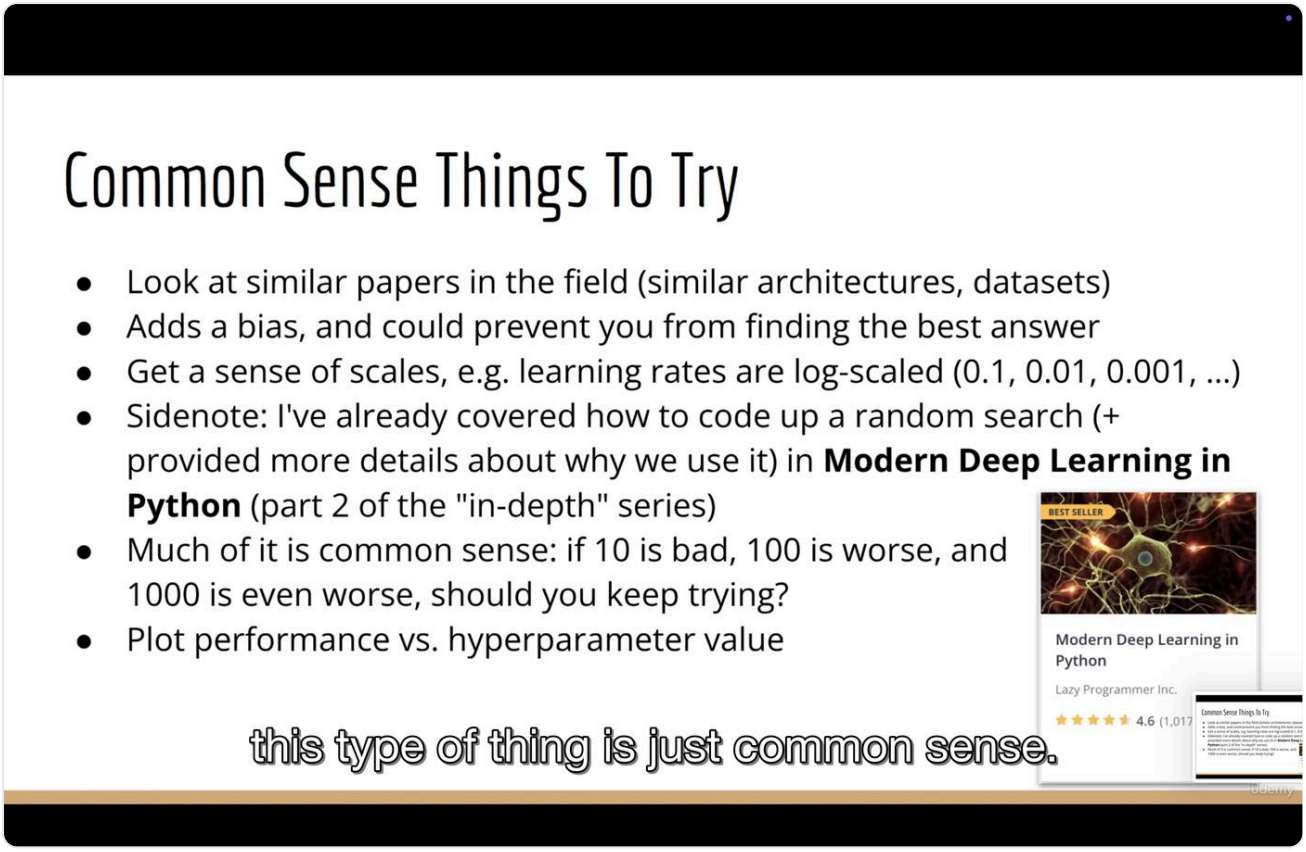


FIGURE 32.7 | Common Sense Things to Try

It may help if you made a plot of your model's performance versus the hyperparameter value, but again, this type of thing is just common sense. Now, many beginners still object to this, they say, but lazy programmer, there must be some rules of thumb.

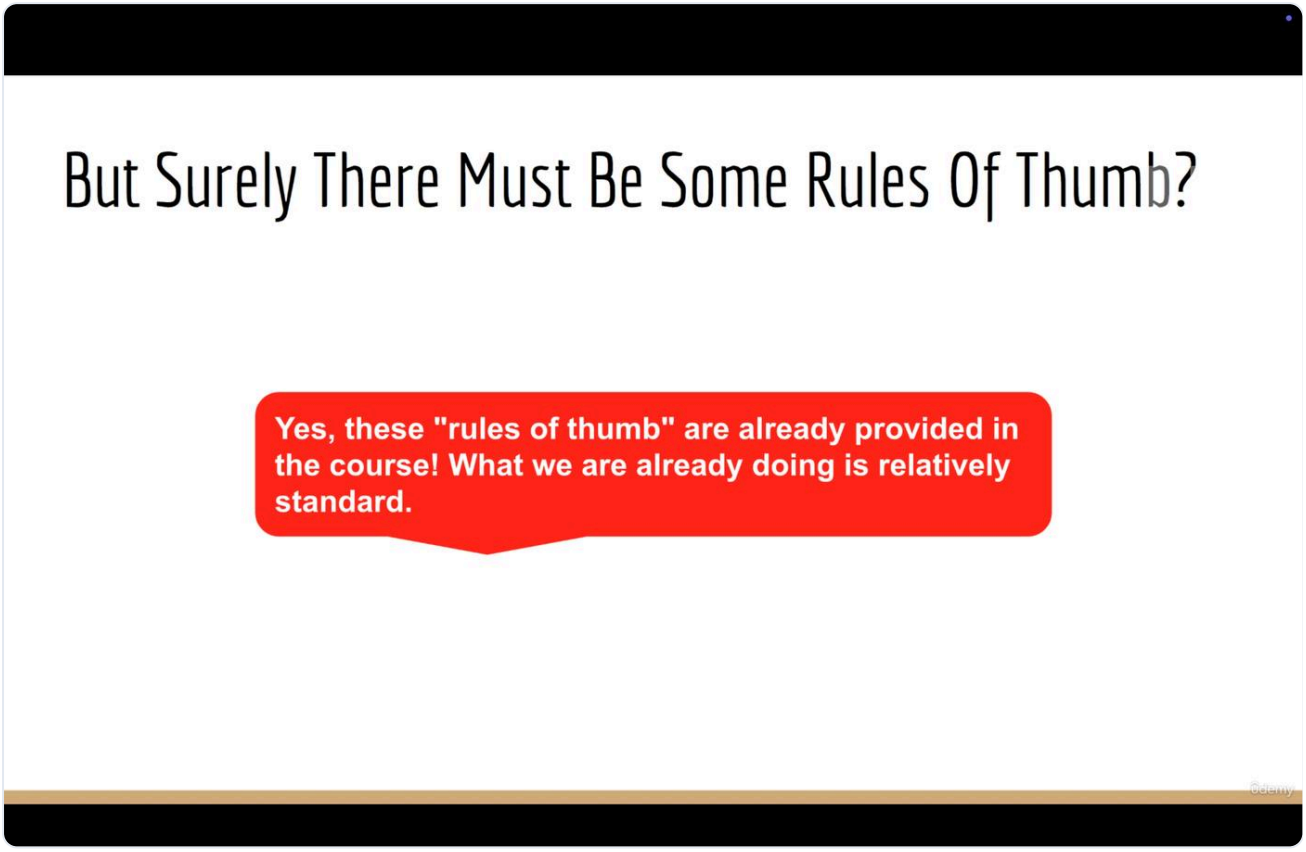


FIGURE 32.8 | But Surely There Must Be Some Rules of Thumb?

Why can't you just give me some simple rules of thumb? And the answer to that is this is simply a bad approach. I'll illustrate this with several examples.

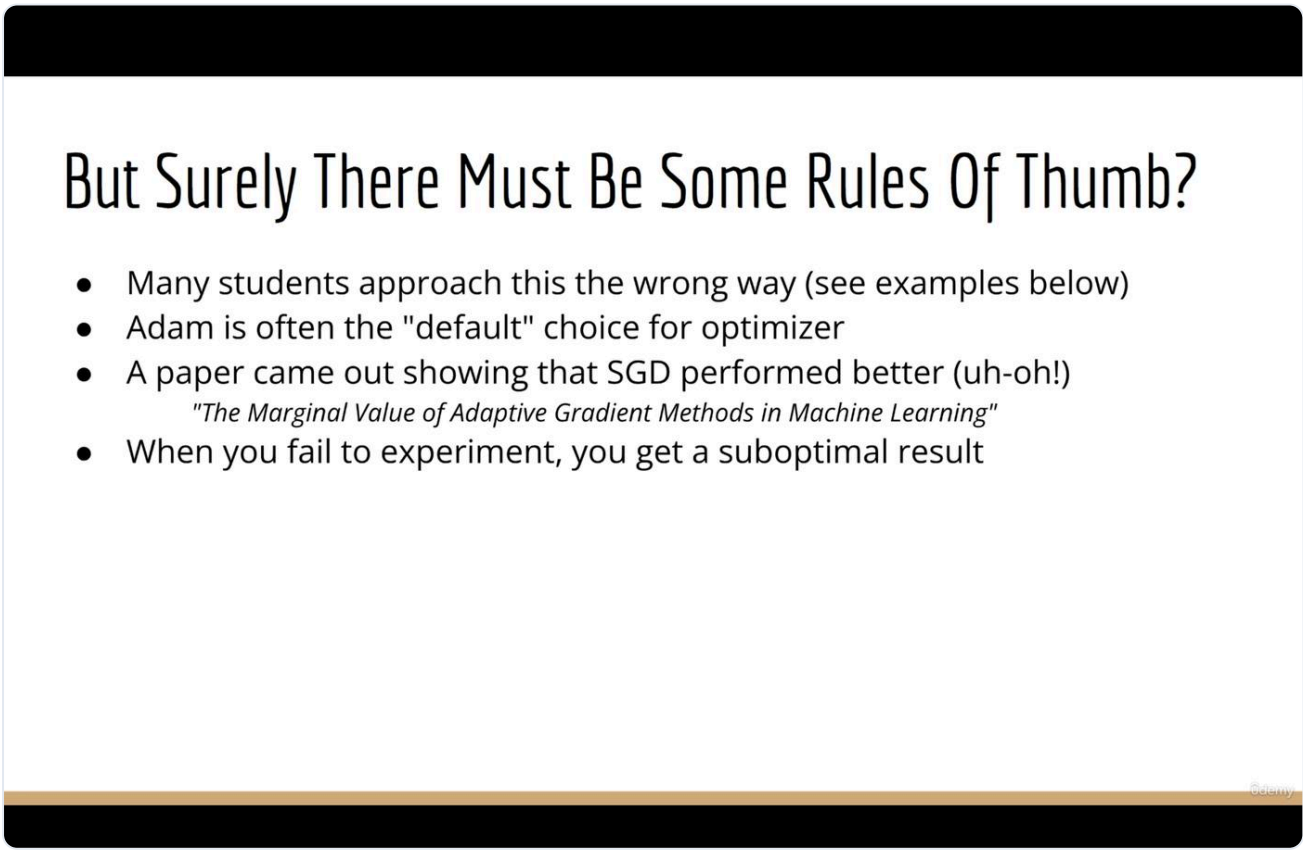


FIGURE 32.9 | But Surely There Must Be Some Rules of Thumb?

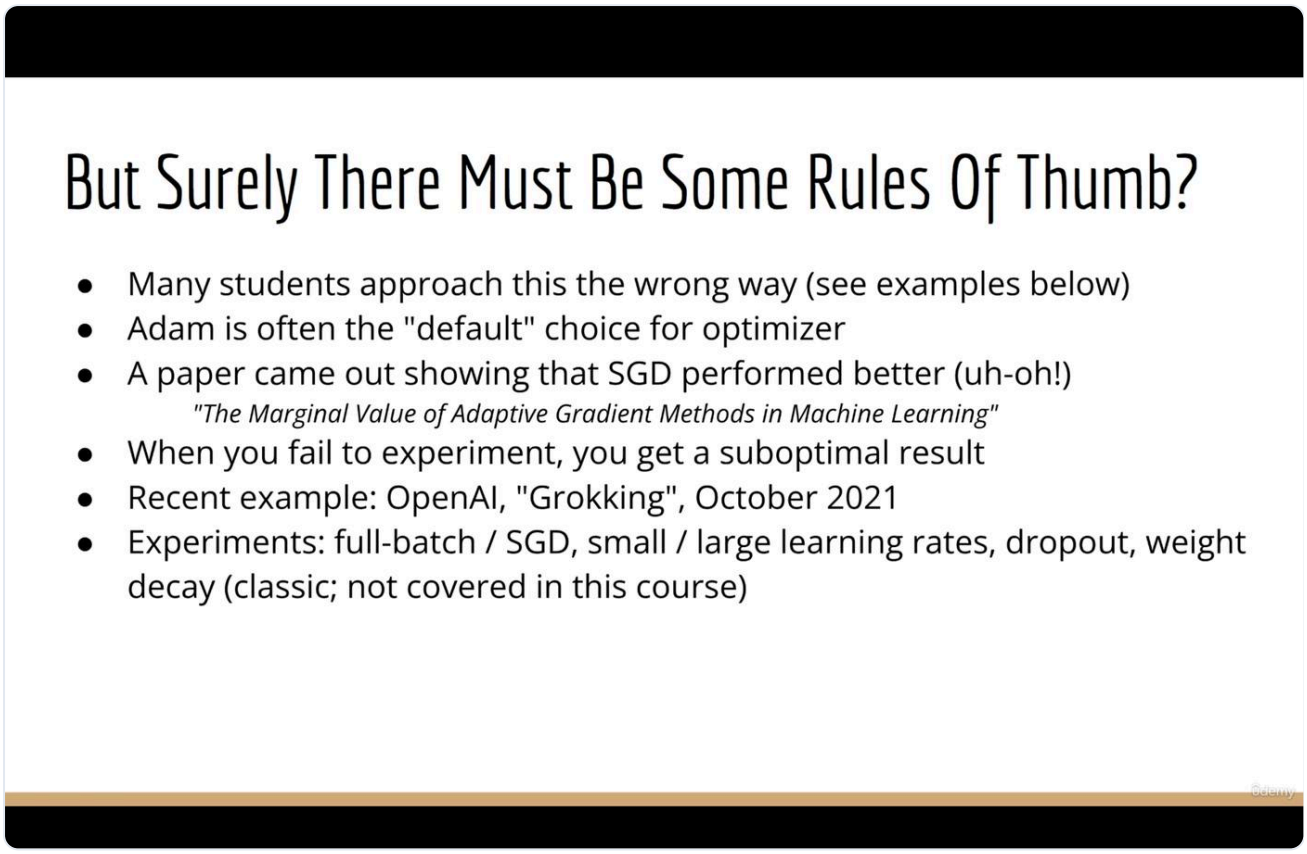


FIGURE 32.10 | But Surely There Must Be Some Rules of Thumb?

As you recall, we've noted that Adam is often the default choice for optimization method. However, there was a paper that came out showing that plain SGD performed better. Thus, if you're the type of student that tries to follow only rules of thumb without actually doing the requisite work, you may have landed at a suboptimal result. This is what happens when you fail to experiment. Here's another very recent example from OpenAI, released in October 2021. In this paper called Grokking, the authors explore an interesting phenomenon regarding the ability of neural networks to generalize. Now the idea behind this paper is not that relevant for this discussion, but what is relevant is what the authors tried.

Specifically, they tried different approaches, such as full versus stochastic gradient descent. They tried different learning rates. They tried dropout, and they even tried weight decay, which is a method that has somewhat fallen out of favor in the modern era of deep learning. So even researchers at one of the top AI companies in the world still must do these experiments and even on simple things like weight decay. Furthermore, I lied a little bit when I said the idea behind this paper is not relevant for this discussion.

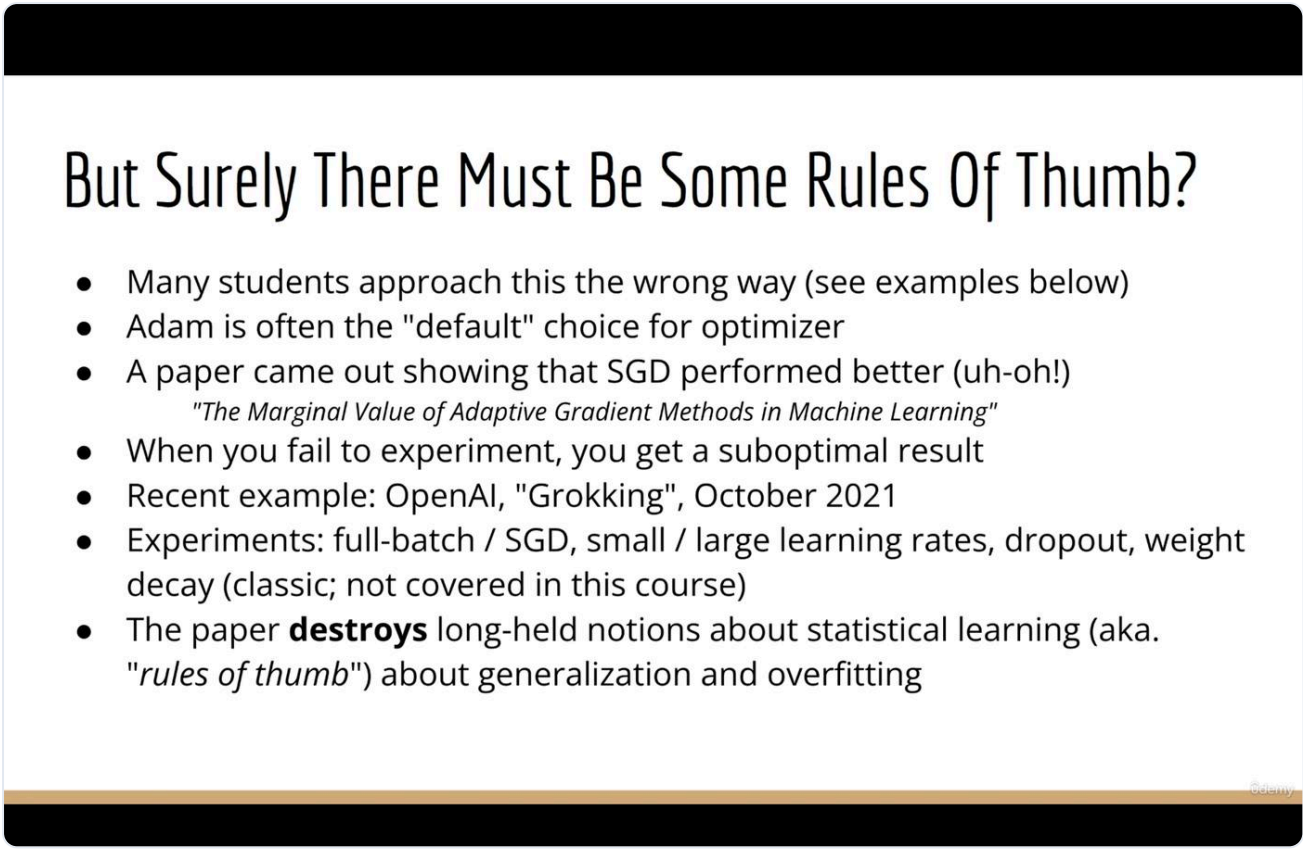


FIGURE 32.11 | But Surely There Must Be Some Rules of Thumb?

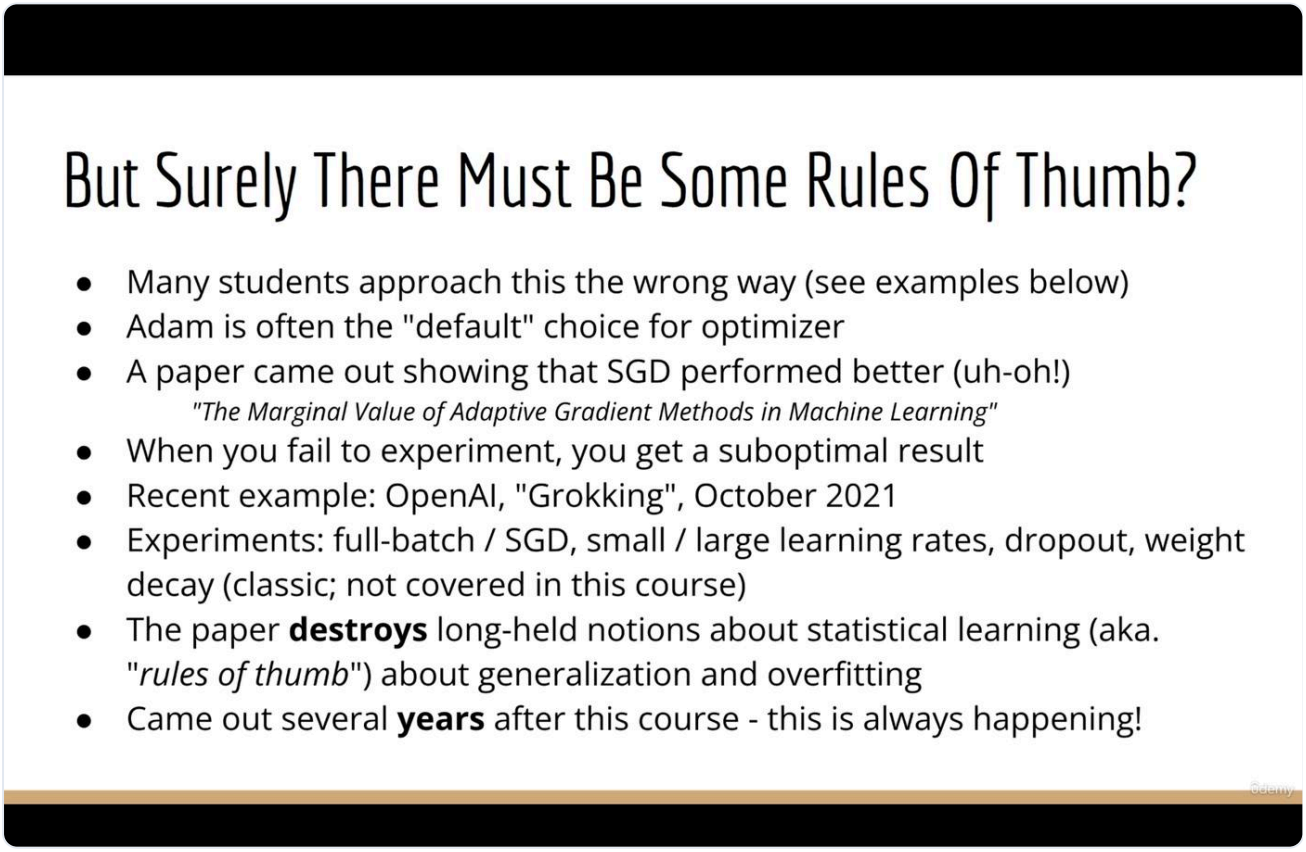


FIGURE 32.12 | But Surely There Must Be Some Rules of Thumb?

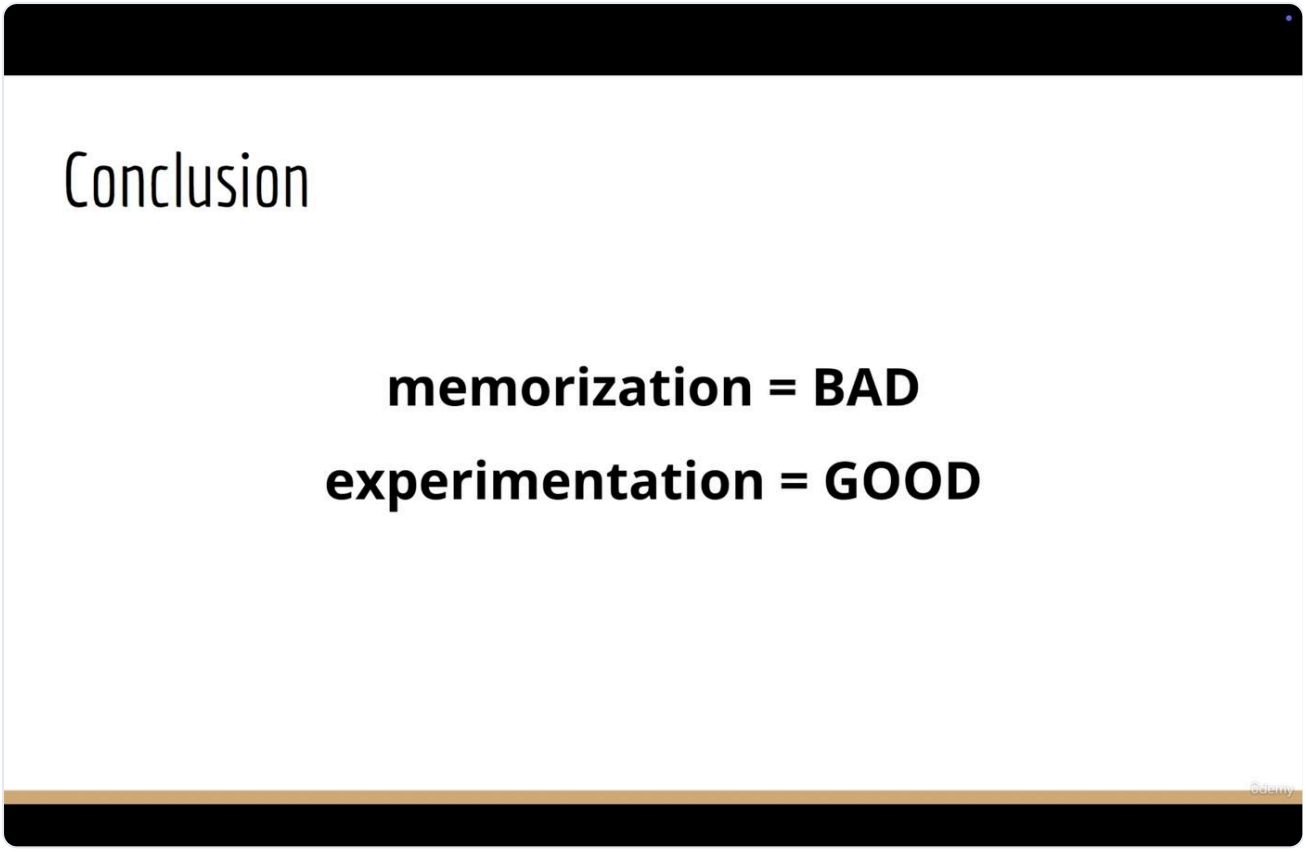


FIGURE 32.13

In fact, what the paper demonstrates is that reality disagrees with some long-held notions about statistical learning. Specifically, they show that neural networks can in fact generalize well. If you keep training beyond the point, it has over, fit it. So again, if you're the type of student that just wants to memorize some rules of thumb, you would again be proven wrong by those who did the actual experiments. Importantly, note that this paper came out several years after the release of this course. You should expect things to constantly change. You should expect any rules of thumb you've memorized to soon become irrelevant. Put more simply, memorization bad. Experimentation good.

LECTURE 33

The Mechanics of Convolution

In this lecture, we are going to discuss convolution in the preparation for building convolutional neural networks.

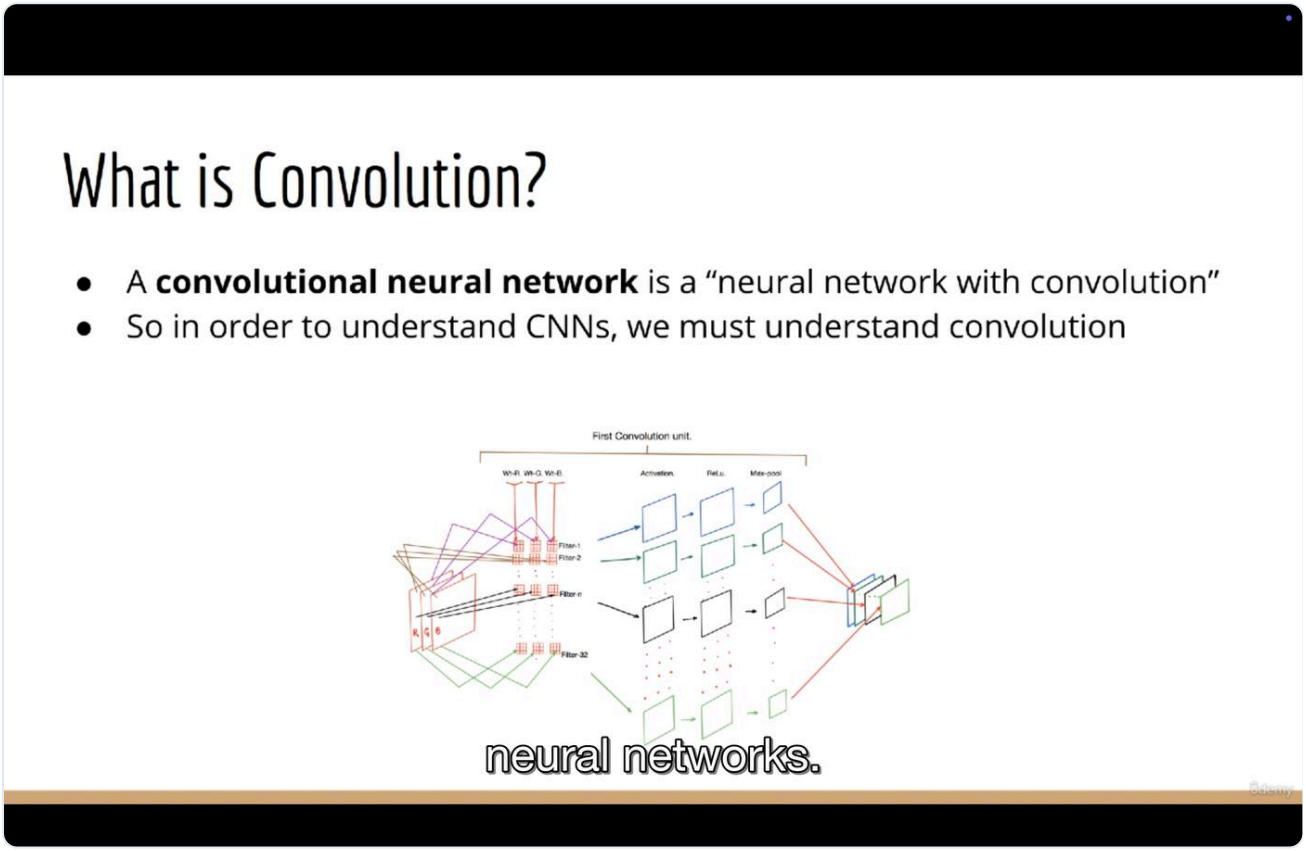


FIGURE 33.1 | What Is Convolution?

Of course, given the name, it's pretty obvious that a convolutional neural network is a neural network with convolution. And so in order to understand CNNs we must first understand convolution. First, I want to mention that people sometimes treat convolution as this mysterious thing. But as with our discussion of machine learning, we want to take the dumb as possible approach. Sure, convolution can be complex.

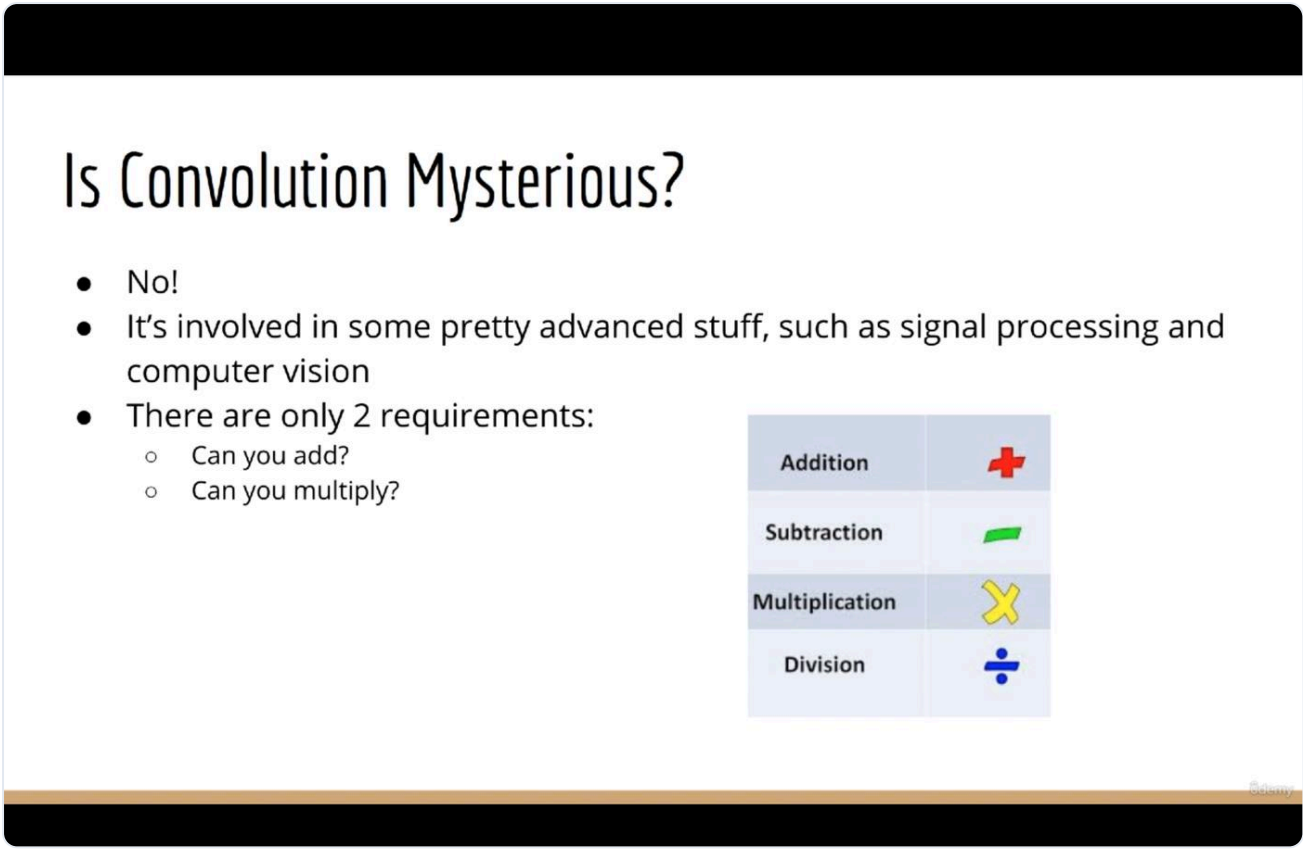


FIGURE 33.2

It's a central operation in the signal processing and computer vision. But in fact, it's quite simple. There are only two requirements. First, can you add. Second, can you multiply? If your answer to these two questions is yes, then you understand convolution. Believe it or not, convolution is just adding and multiplying. So let's start by not doing any math. Let's just look at convolution qualitatively with convolution. There are three objects you want to pay attention to. First is the input image.

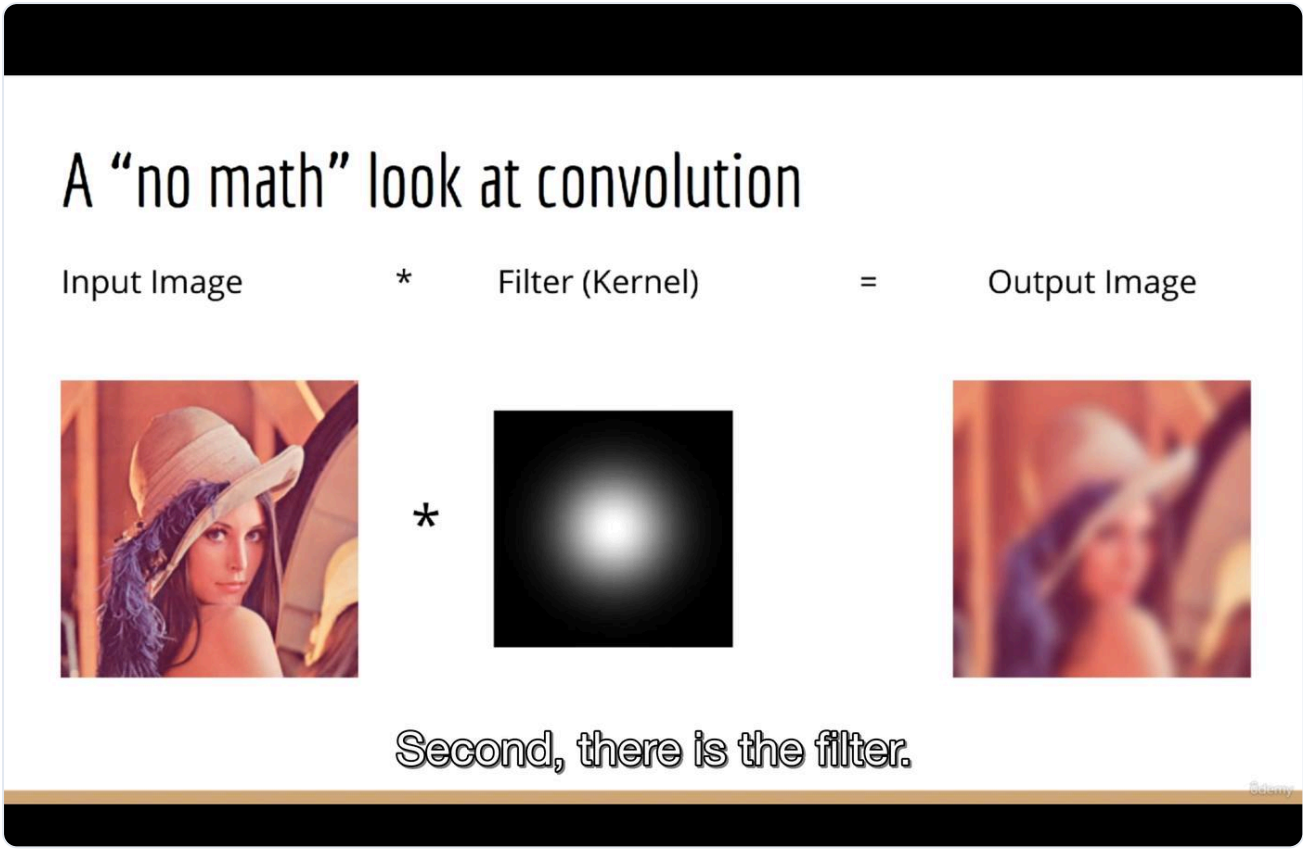


FIGURE 33.3

Second, there is the filter. Note that another name for the filter is the kernel. So we're going to use those two terms interchangeably. Finally, there's the output image. The output image is what you get when you convolve the input image with the filter. In other words, if I perform the convolution operation on the input, image and the filter, I get the output image. The mathematical symbol for convolution is a star or asterisk. Not to be confused with the asterisk that we use in computer programming for multiplication. So if you're running convolution in code, it will not be an asterisk because in code we already use the asterisk for multiplication.

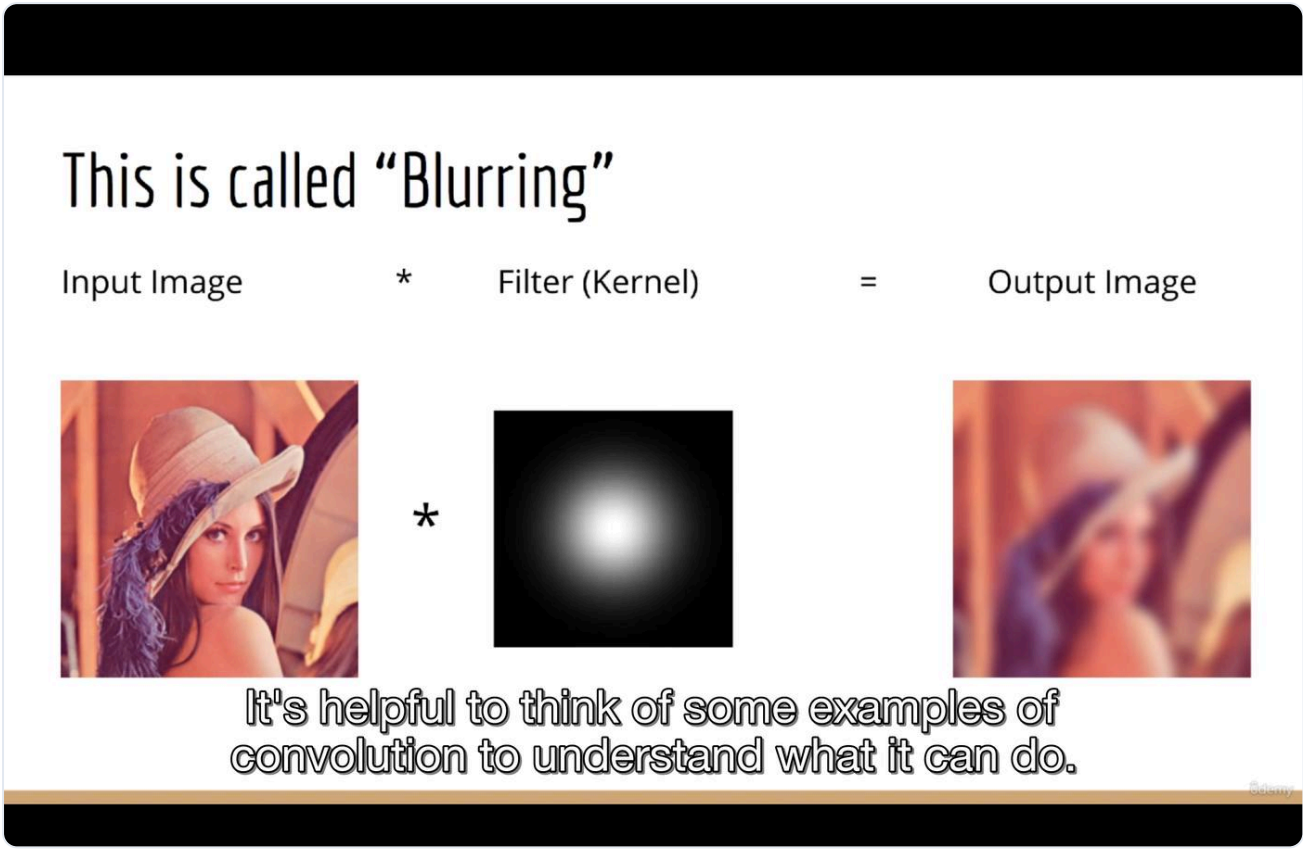


FIGURE 33.4

It's helpful to think of some examples of convolution to understand what it can do. There are two examples I really like. The first example is blurring. So the input image is just the original image and the output image is a blurred version of that image. You might recognize this operation from applications such as Photoshop. The second example I really like is edge detection as input.

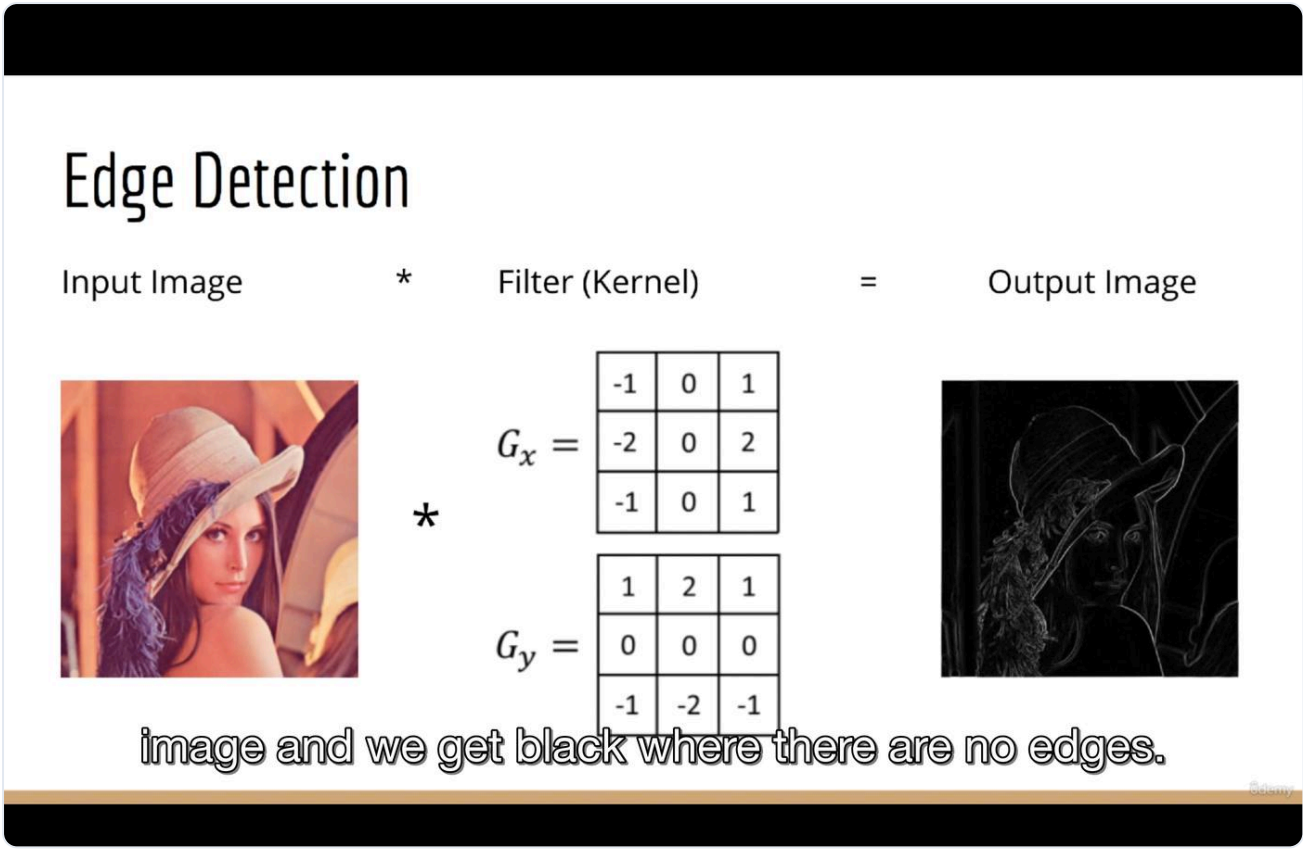


FIGURE 33.5 | Edge Detection

We have the original image and as output, we get white lines where there are edges in the original image and we get black where there are no edges. So the output image highlights all the edges in the original input image.

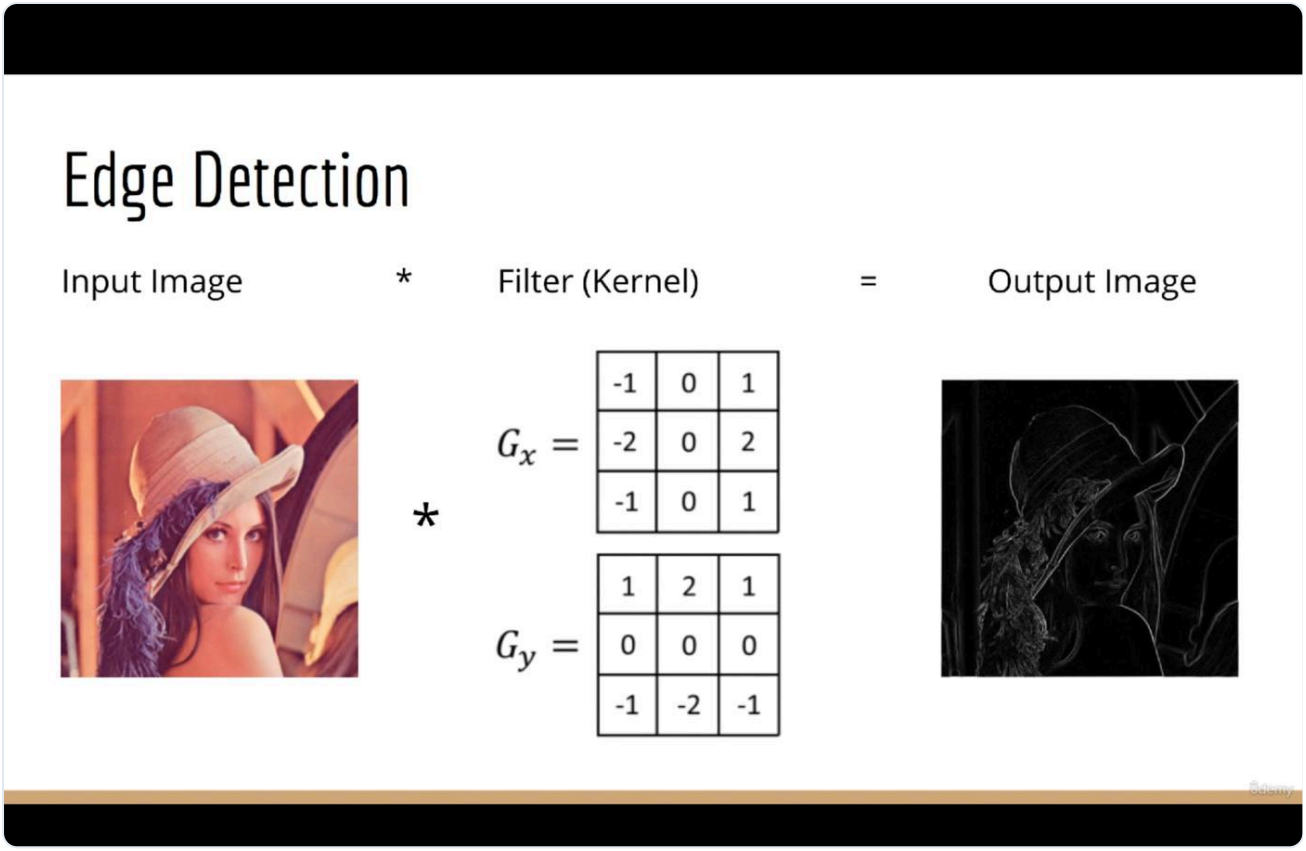


FIGURE 33.6 | Edge Detection

So that's your first lesson on how to view convolution, this perspective is that convolution is an image modifier. The input is the original image and the output is a modified or transformed version of that image.

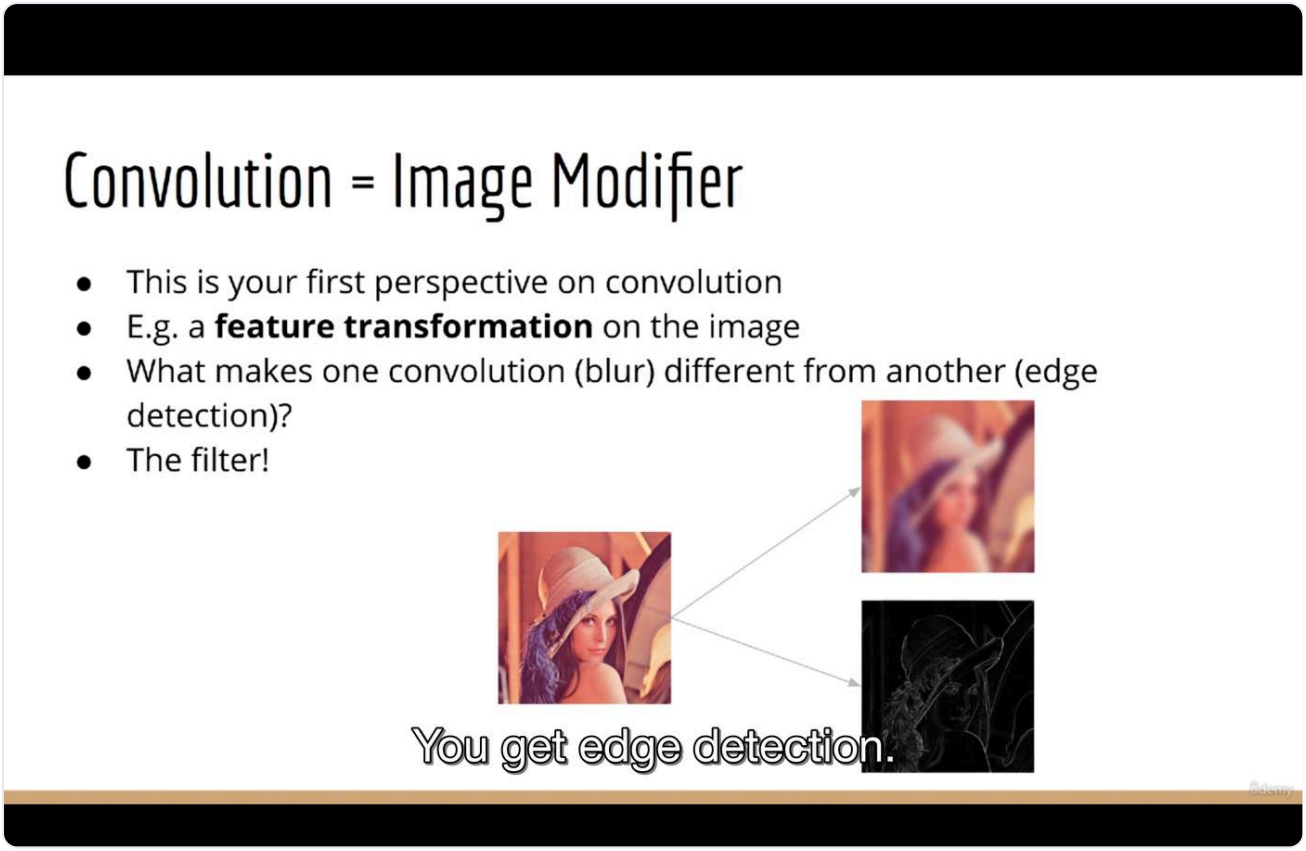


FIGURE 33.7

In other words, you might think of this as a feature transformation on the input image. You know, that sounds curiously like what a neural network does. In any case, what makes a blurring and edge detection actually work if they are both convolution? What makes one convolution different from another convolution? Well, the answer is the filter when you use a Gaussian filter. This blurs the image when you use an edge detection filter. You get edge detection. You can think of that as sharpening the image. Your next question might be how do we find or design these filters in the first place? That's something we'll discuss later in the section. But the answer, as you might have come to expect, is just another dumb as possible approach.

OK, so let's get into the nitty gritty of how convolution works. I promise you that all you need to know is how to add and multiply. Let's see if that's true. We're going to use it. Very tiny images for this example.

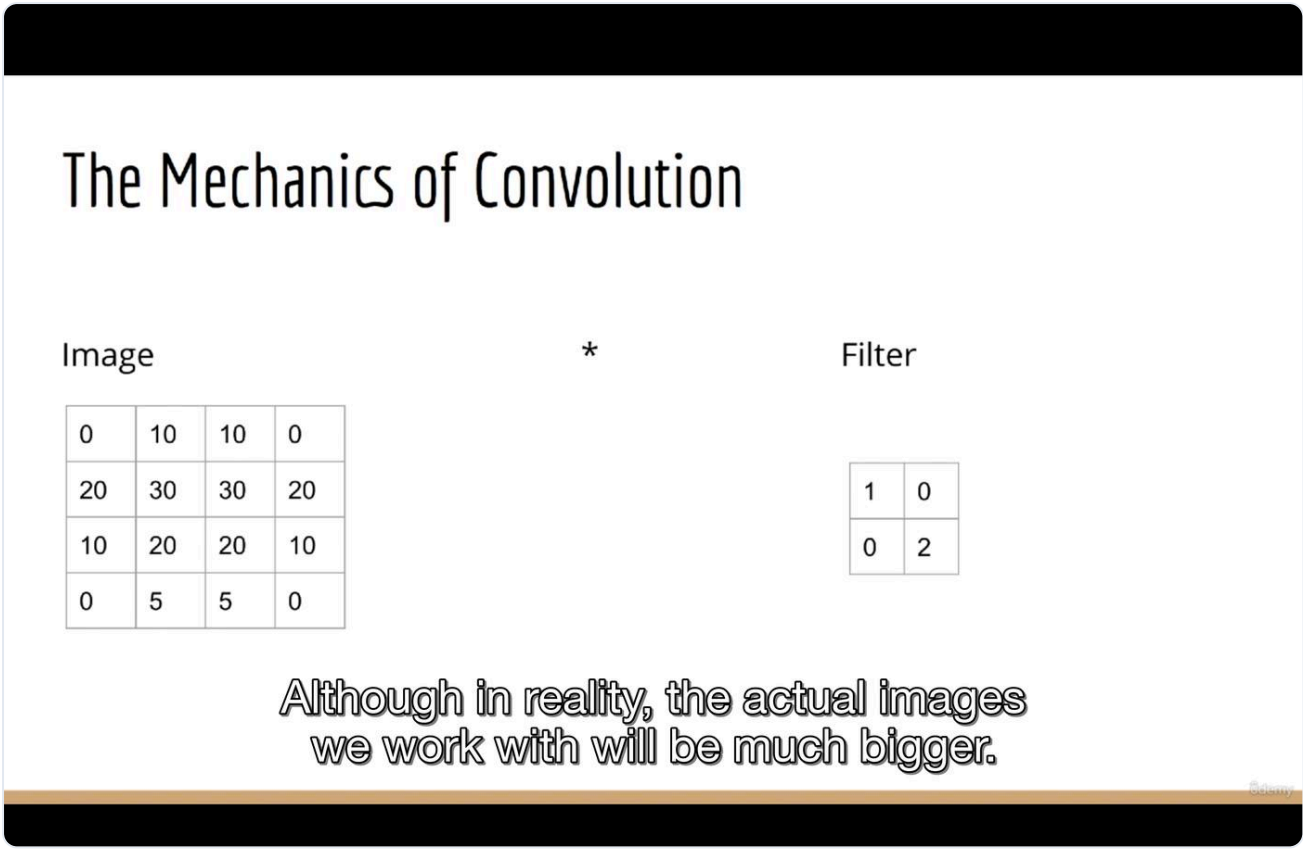


FIGURE 33.8 | The Mechanics of Convolution

Although in reality, the actual images we work with will be much bigger. This is just so we can feasibly do these calculations by hand. So let's start with an image zero 10, 10, zero, 20, 30, 30, 20, 10, 20, 2010 and zero five five zero. And we also have the filter one zero zero two.

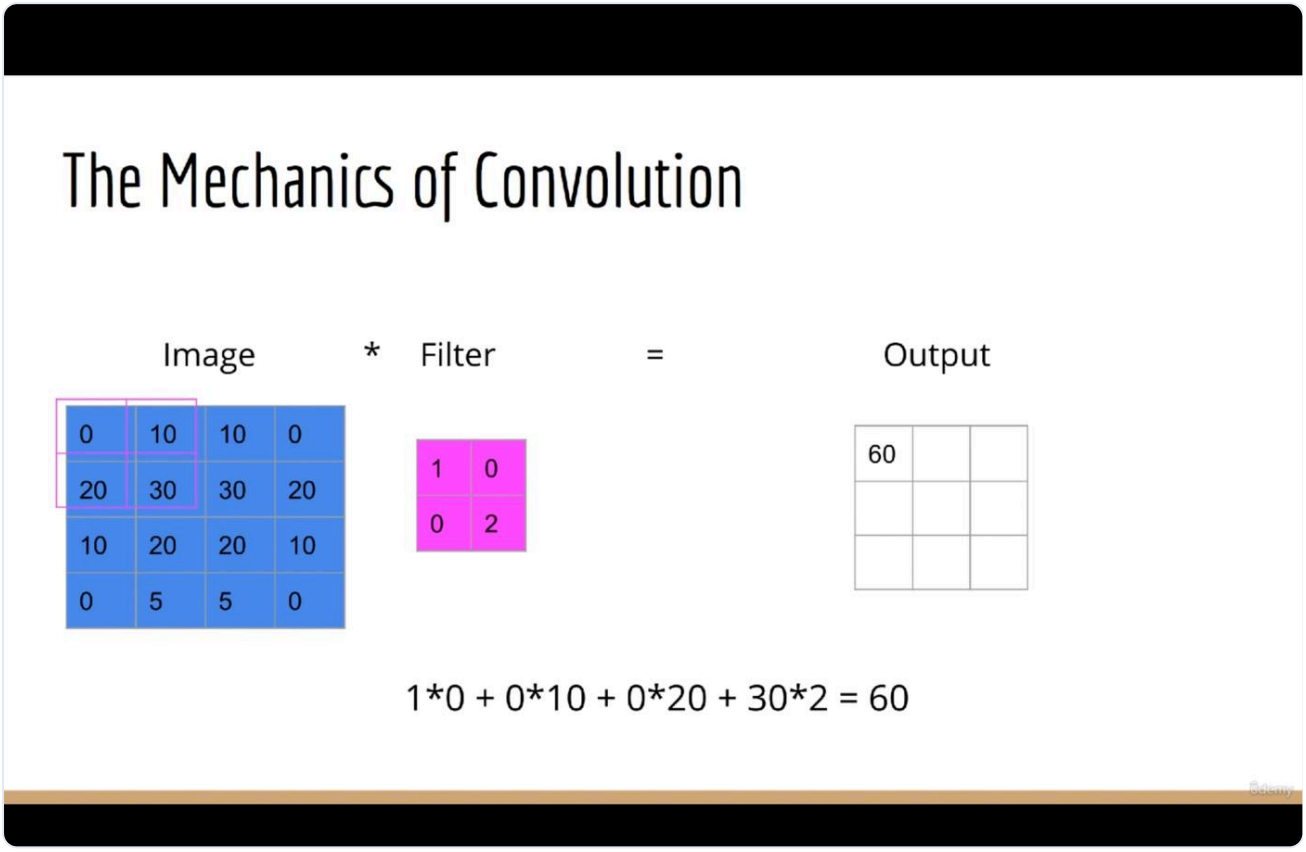


FIGURE 33.9 | The Mechanics of Convolution

So how do we convolve this image with this filter? Let's imagine overlaying the filter at the upper left corner of the image. Then all we need to do is element-wise multiplication and add up all the results. So, yeah, one time zero plus zero times 10 plus zero times 20 plus 30 times two.

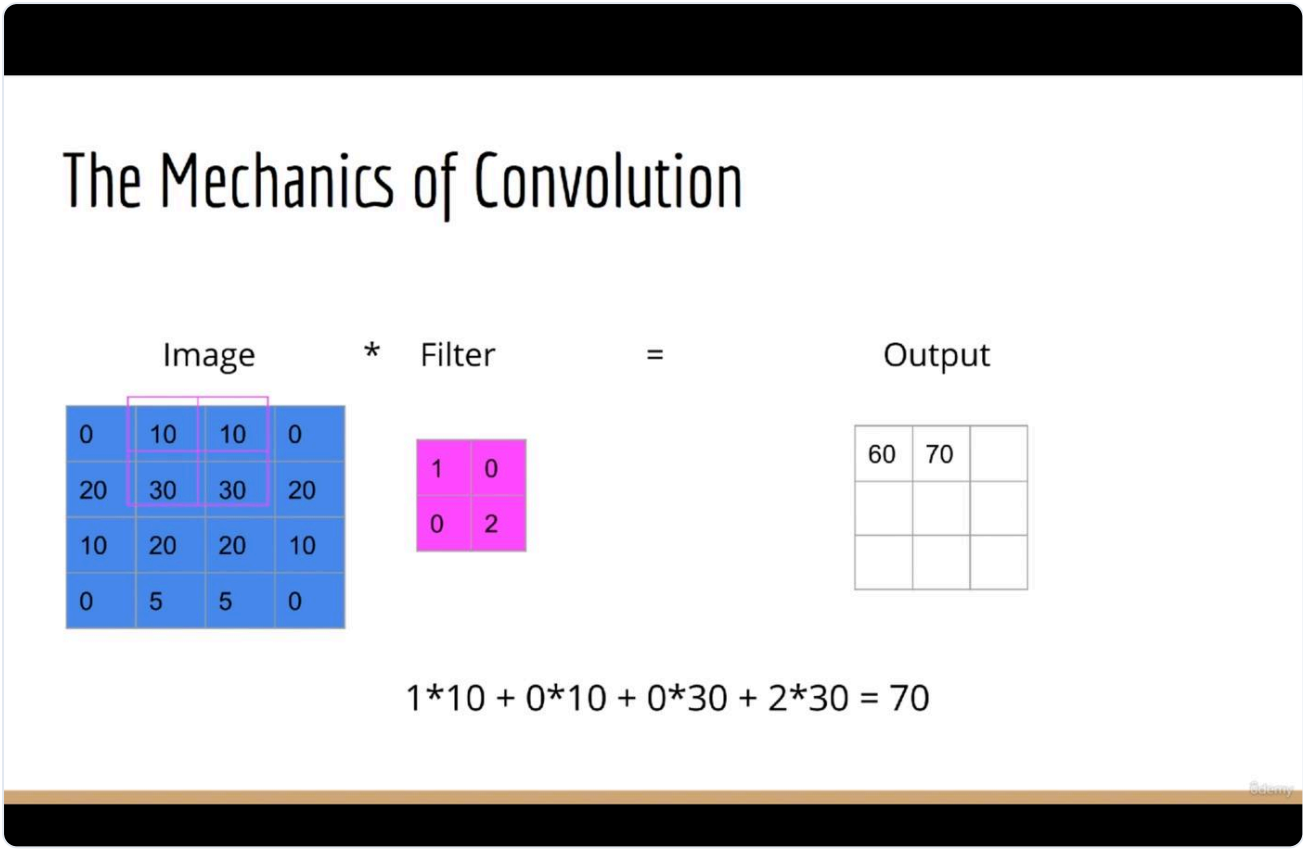


FIGURE 33.10 | The Mechanics of Convolution

That is equal to 60. This gives us our first output in the output image. And as promised, all you needed to do was multiply and add. Now, let's move our filter over one space to the right and then let's do our element-wise multiplication and add up the results again so we get one times ten plus zero times ten plus zero times thirty plus two times thirty.

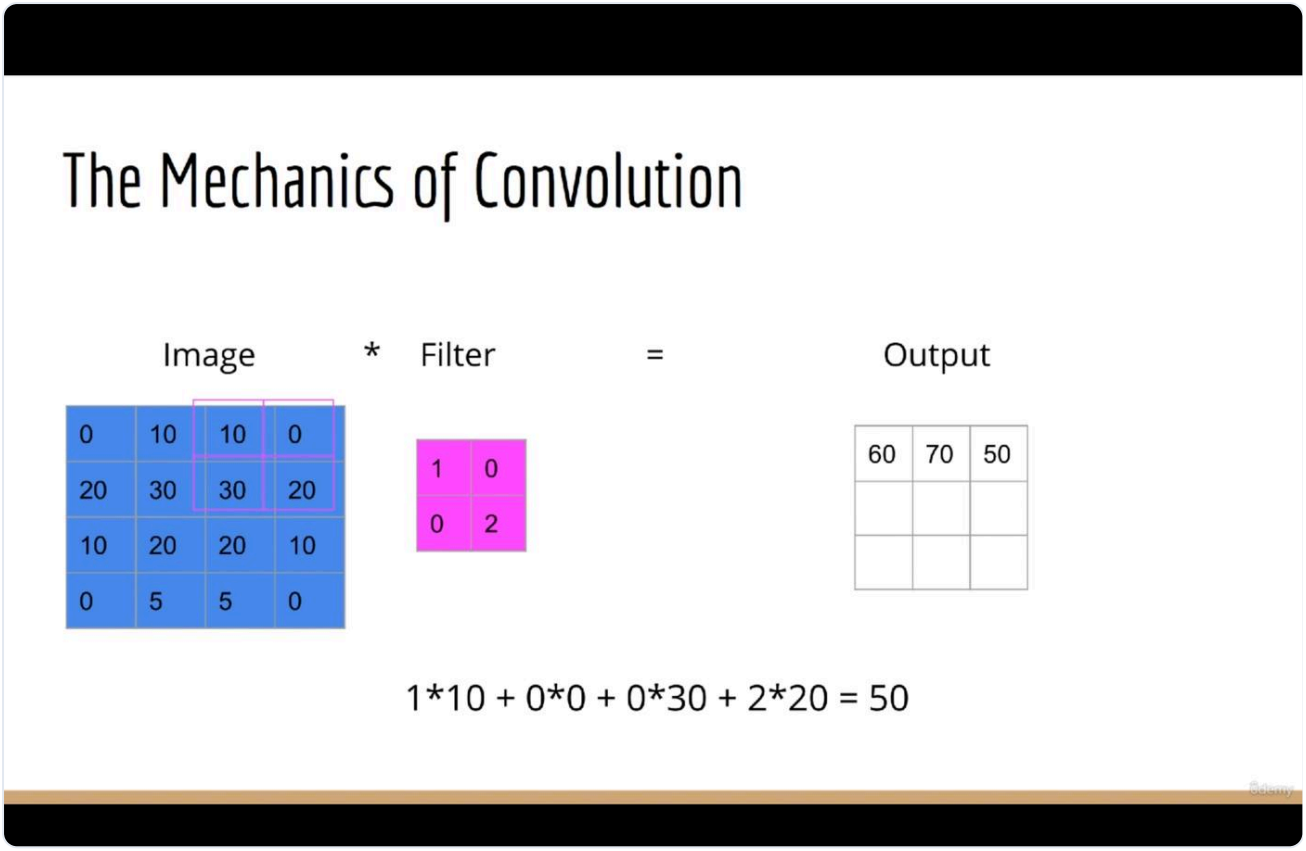


FIGURE 33.11 | The Mechanics of Convolution

That's 70. This gives us our second output, which goes to the right of the first output. Now, let's move the filter over one more space to the right and do the same operation again. We get one times ten plus zero times zero plus zero times 30 plus two times 20, and that's 50.

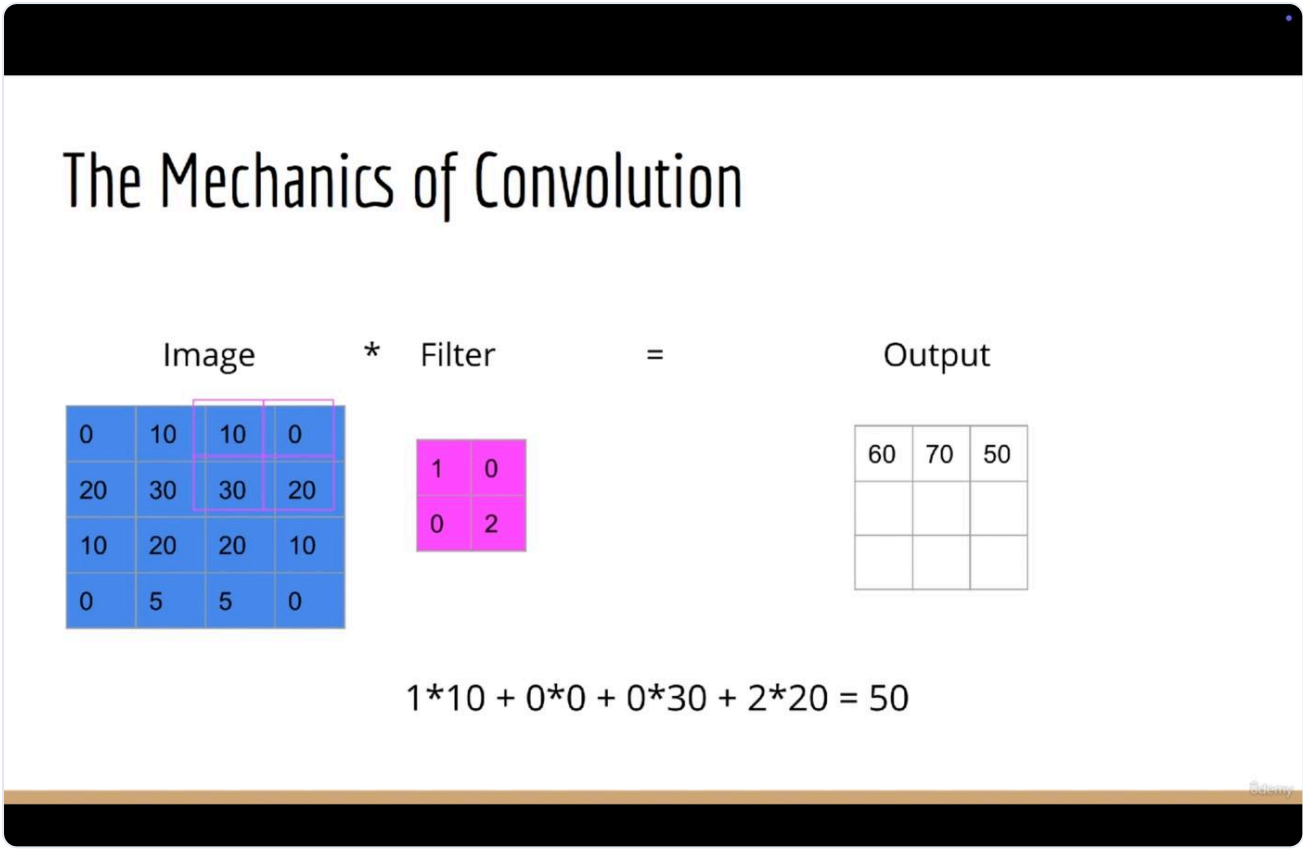


FIGURE 33.12 | The Mechanics of Convolution

This gives us a third output, which goes to the right of the first two outputs. Now you realize that there's no more space to move our filter to the right anymore. So let's instead zigzag back to the left, but go down one row and then let's repeat our calculation. We get one times 20 plus zero times 30 plus zero times ten plus two times 20.

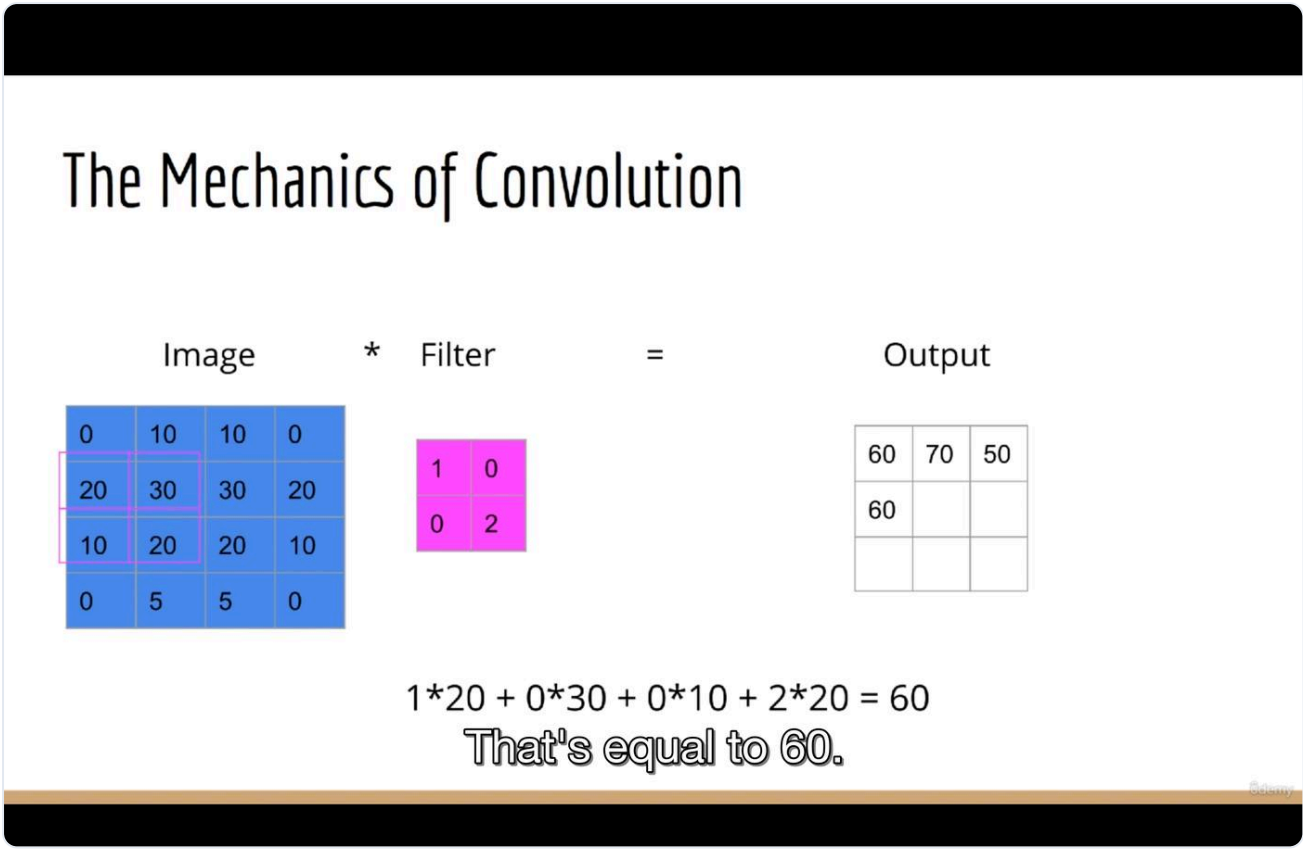


FIGURE 33.13 | The Mechanics of Convolution

That's equal to 60. This gives us our first output in the second row. So you can see that the output location corresponds to where we've placed the filter along the original image. So we're not going to go through the rest of the calculations, since that would be a bit redundant. But what I want you to do is if you don't understand this, do the rest of the calculations by hand to make sure that these results are correct. So we have 60, 70, 50, 60, 70, 50, and then 20, 30, 20. One helpful exercise to get a better idea of how Convolution works is to write pseudo code and even real code to implement the algorithm we just described. This will help us uncover a few hidden details.

You may have not considered just by starting the code, you realize one thing you need to take care of, which is what size. Should I initialize the output array? As if you recall in our example, the input image had length four while the kernel had length two the output had length three. So what's the pattern?

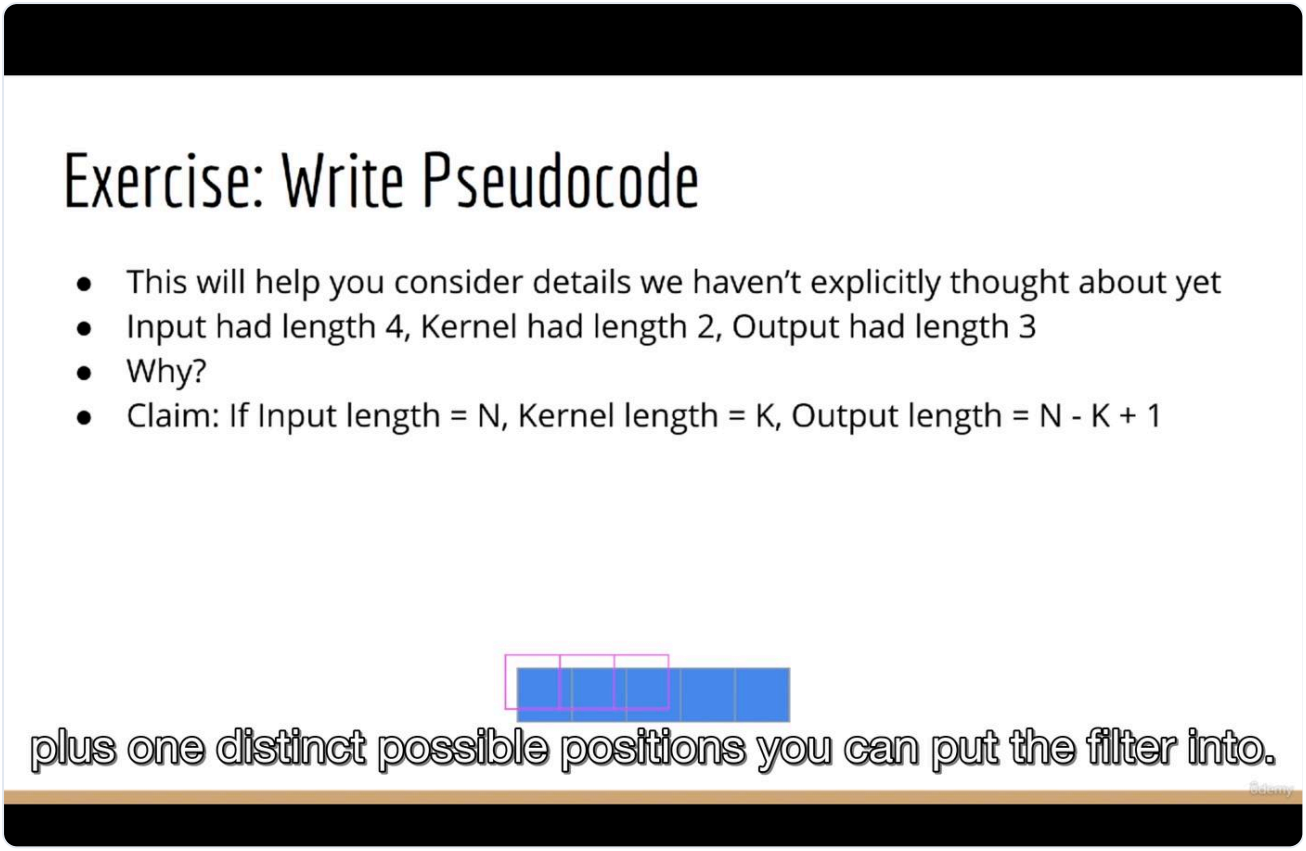


FIGURE 33.14 | Exercise: Write Pseudocode

I claim that if you have an array of length N and a filter of length K, then there are N minus K plus one distinct possible positions you can put the filter into. You might want to draw this on paper. If you don't see right away why this is true. So the first step in our pseudocode is to initialize the output array. But here's another detail. You may have not considered that the output height will be the input height minus kernel height plus one the output width will be the input width minus kernel width plus one. But in our example, both our image and kernel were square. So you might be wondering, is this always the case? What's the convention? And the answer is that four images, usually these are not square.

This just has to do with cameras and screens. Most screens like your computer screen and your TV screen are not square. And therefore, cameras do not take square pictures. Therefore, images we find in the wild that we want to use as data are typically also not square. Some neural networks do use square images for convenience. So when you build a data set, you make them square. And one example of this is MNIST, which we've already seen.

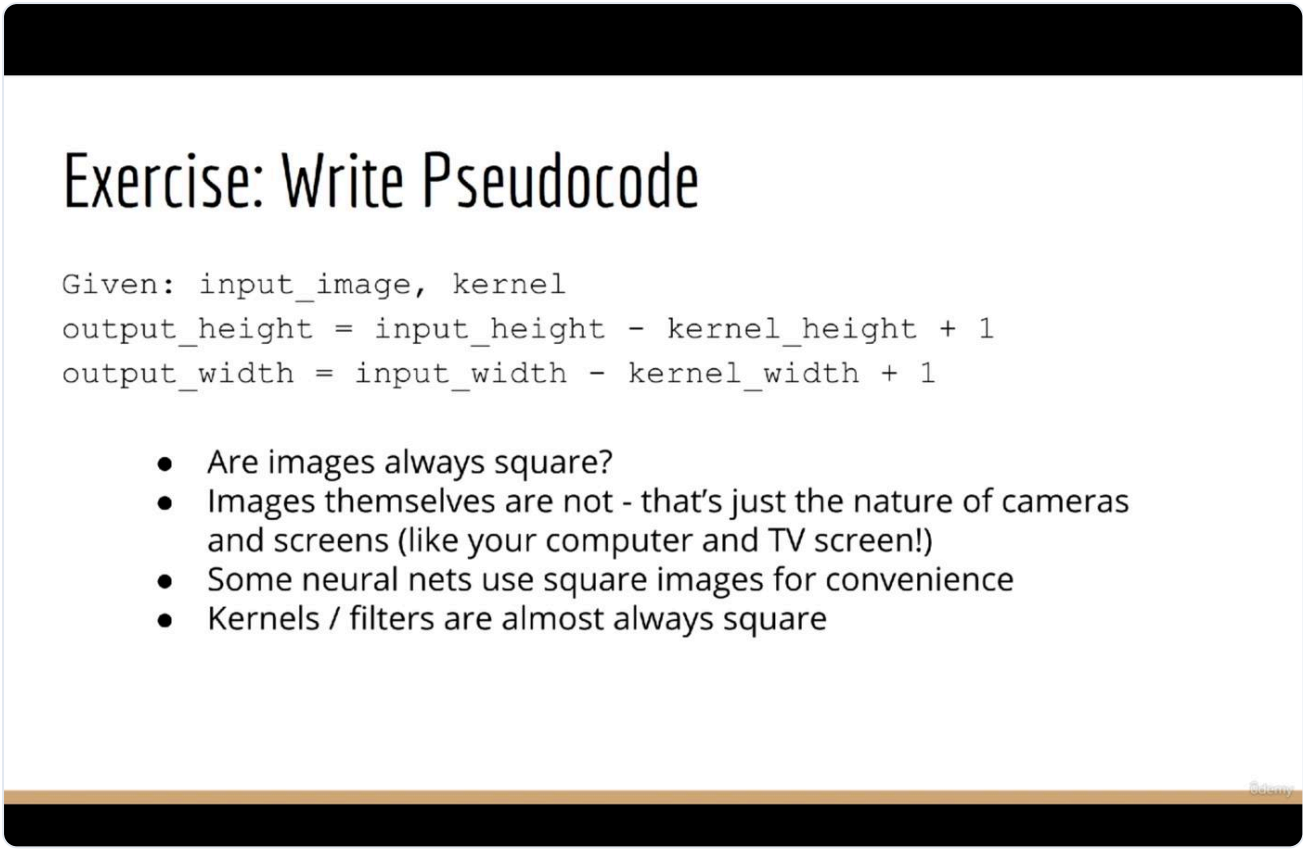


FIGURE 33.15 | Exercise: Write Pseudocode

On the other hand, kernels are almost always square by convention. Let's move on, the next step is just to fill in the output array by performing the convolution algorithm. So first we loop through zero up to output height with the index. I. Inside that, we loop through zero up to output width with the index J. So the pair I J will always index our output. Then we loop through each position of the kernel. So we have I going from zero up to kernel height.

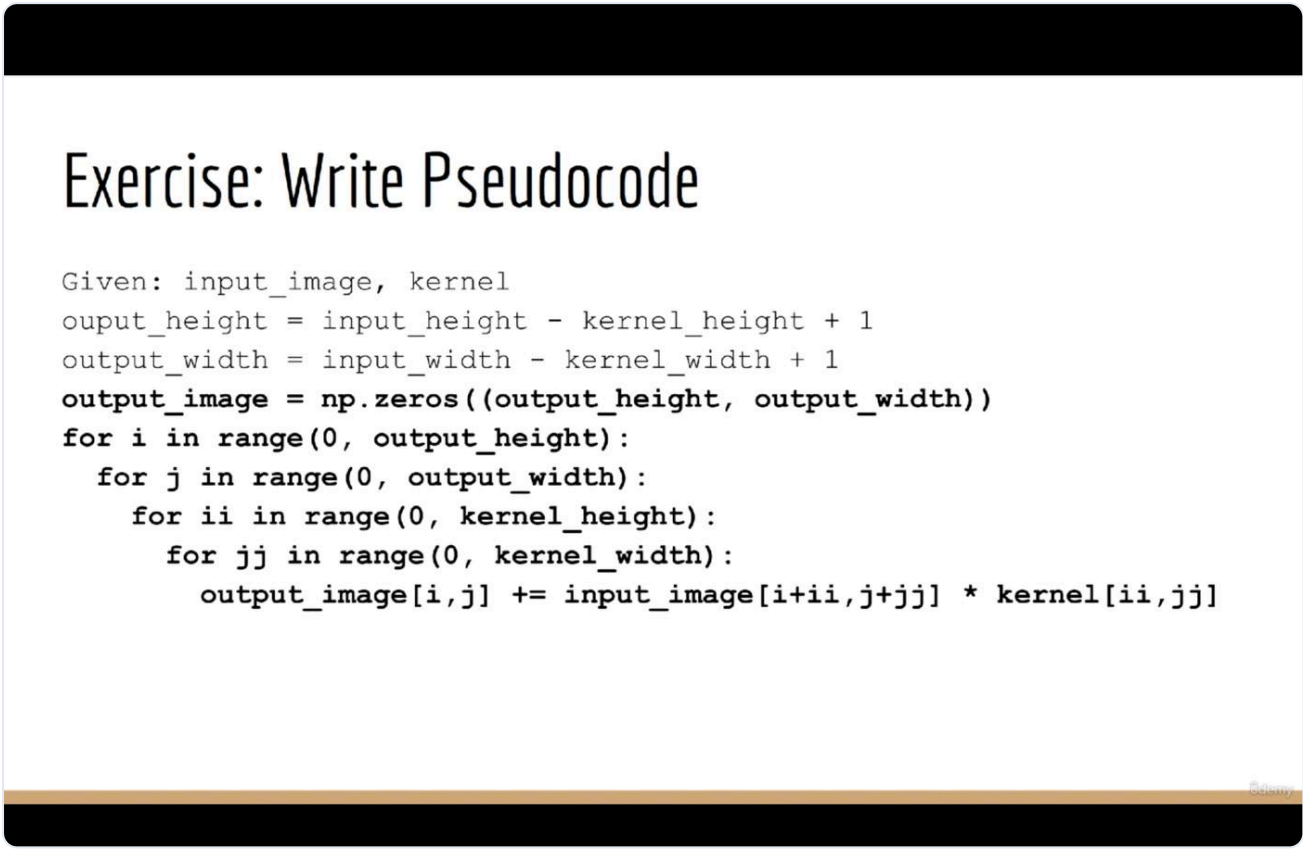


FIGURE 33.16 | Exercise: Write PSeudocode

And we have JJ going from zero up to kernel width. Finally, inside all these loops, we have our main calculation, which, as promised, is just multiplication. And addition, we multiply the input image at position. I plus I j plus j k by the kernel. Add position i j j. And we add this result to the output image at I J. Now it's OK to accumulate using plus equals because we initialize the output image to be all zeros. As an exercise, you might want to try and put this into code so that you can confirm that it works and returns the expected result. The inner part of the pseudocode is key because it helps us understand the equation that defines convolution.

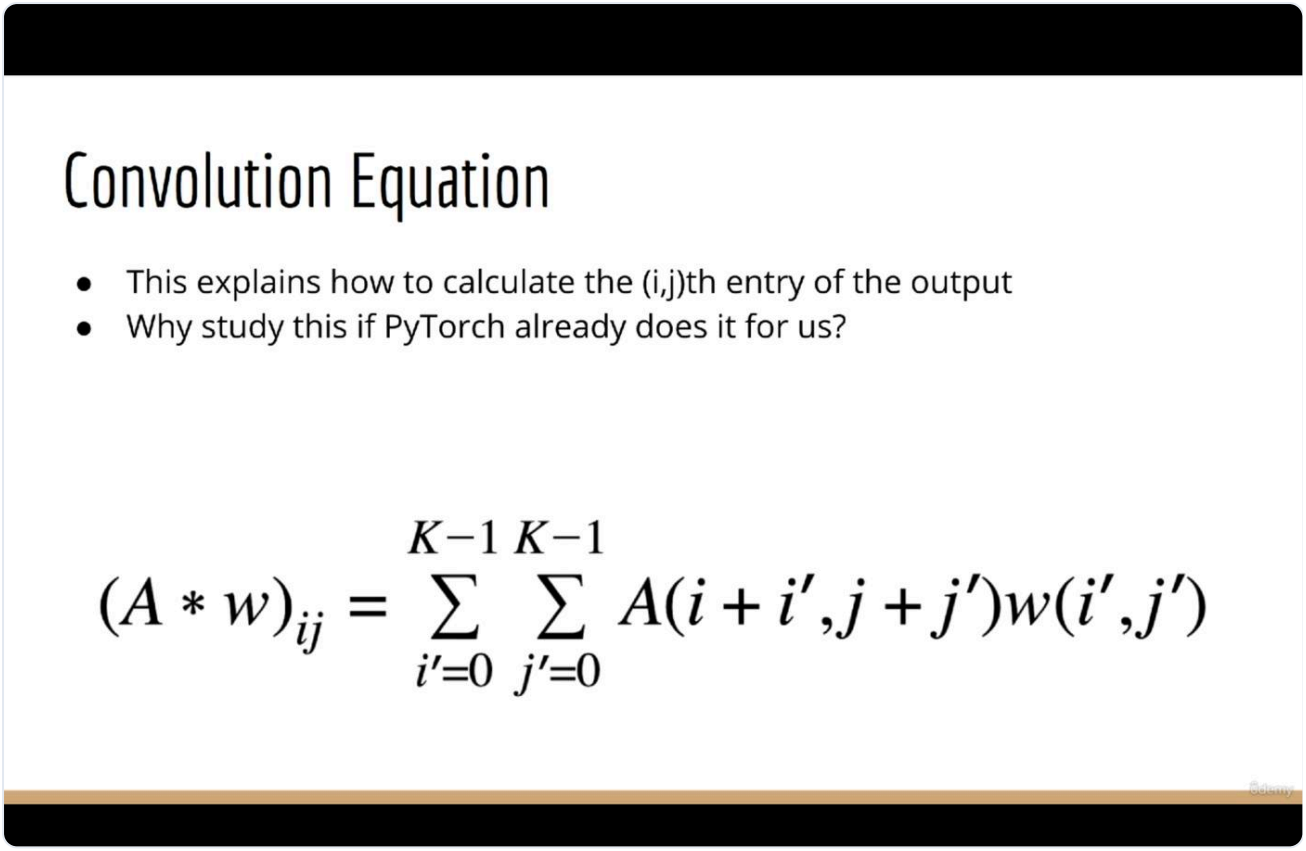


FIGURE 33.17 | Convolution Equation

So here we can see that the I J th entry of the output A convolved with W is the sum over I prime. And the sum over J prime of A at I plus I prime. J plus J prime times. W at I prime. J Prime. Now you might ask, why are we looking at these complicated equations. If PyTorch already does all this work for us. And the answer is that this will help you immensely in understanding the different perspectives on convolution that we are going to discuss later.

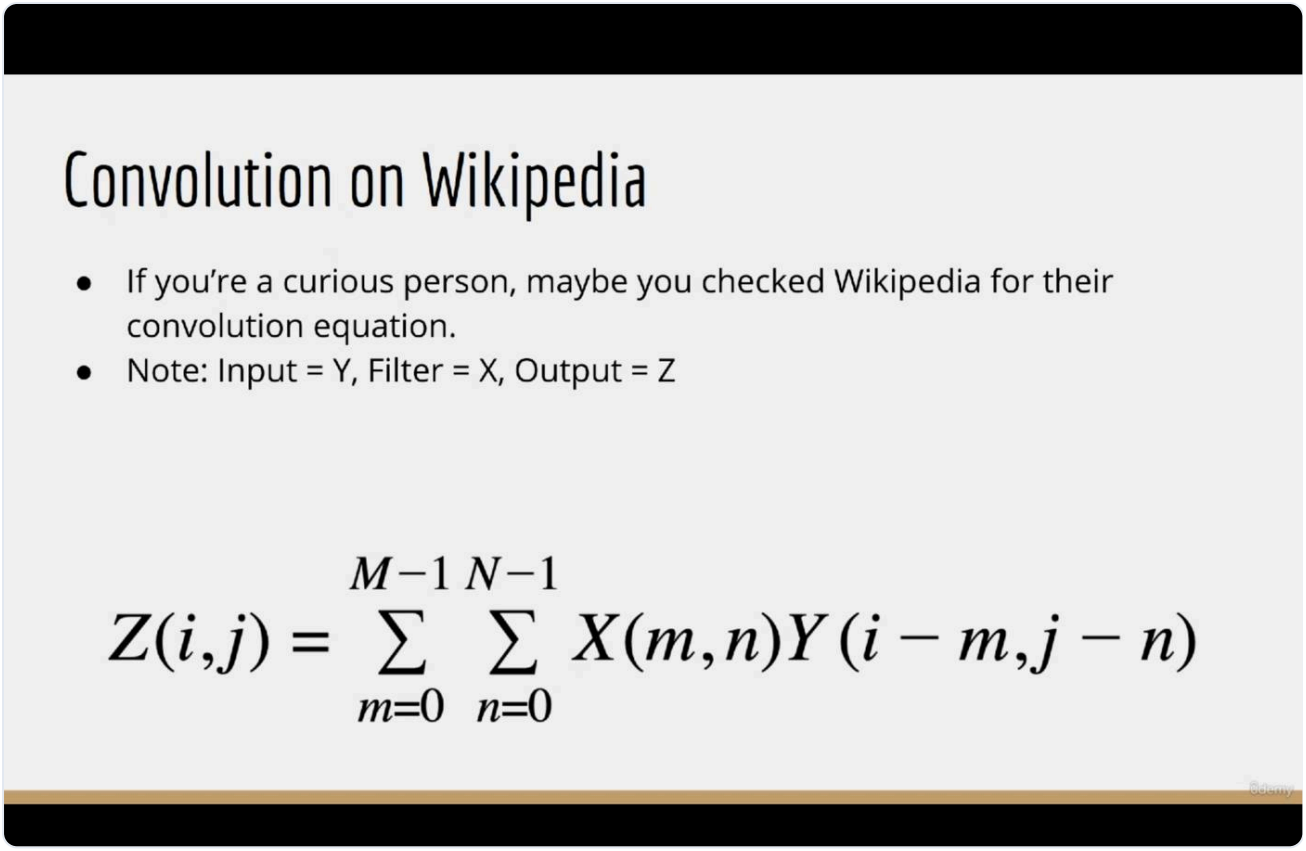


FIGURE 33.18 | Convolution on Wikipedia

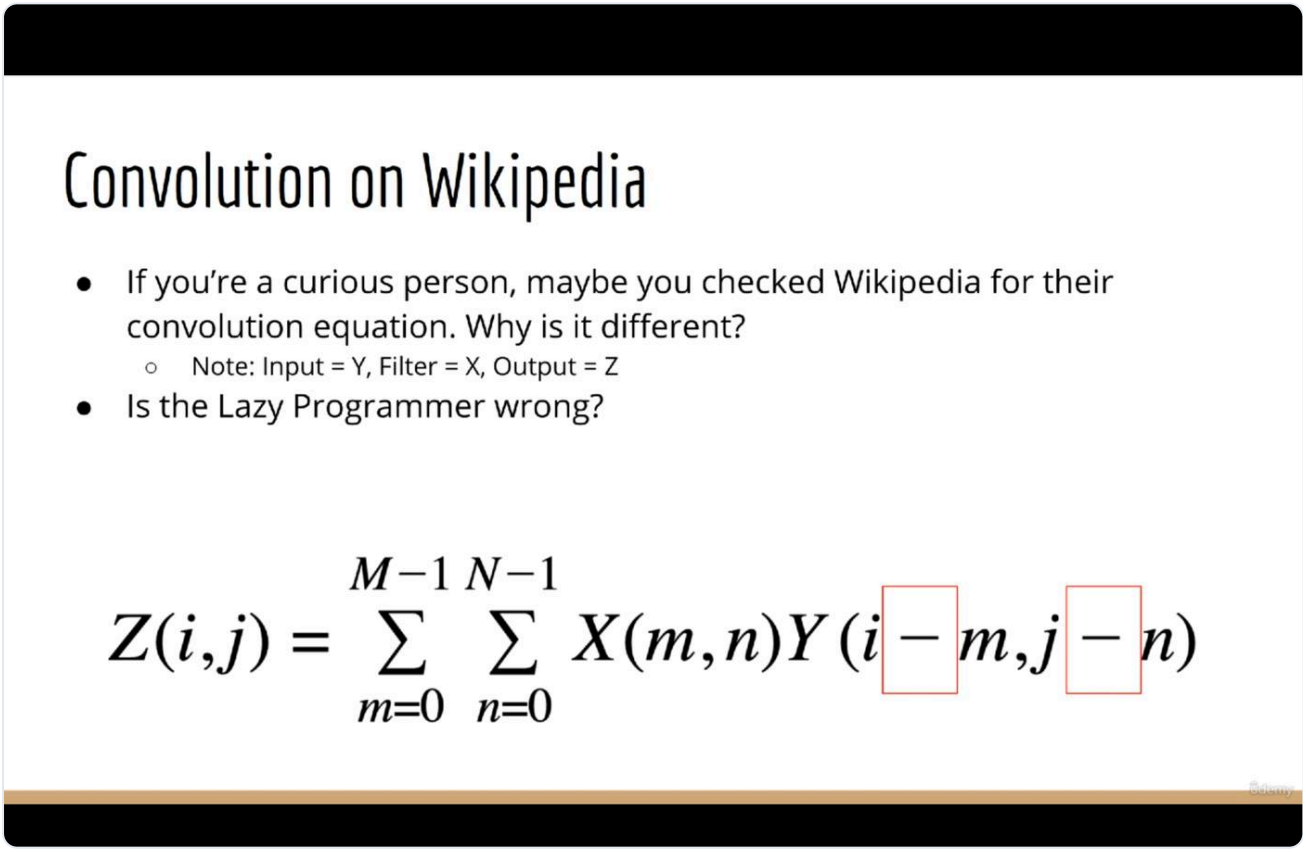


FIGURE 33.19 | Convolution on Wikipedia

Moreover, if you're a curious person and you go on Wikipedia to read about convolution, you'll see something very similar to this. In this example, you can think of X as the filter, Y as the input image, and Z is the output image. Now, you might notice something weird about this equation from Wikipedia, which is that instead of plus signs, we have minus signs. Why is that? Is the lazy programmer wrong? And the answer is, in fact, all of deep learning is wrong. But for better or worse, that's just the way we do things. In the end, it doesn't make a difference because the filters we use will be learned that using gradient descent, in other words, automatically.

So the process of finding the filter will be done automatically using gradient descent, which will find the best values that optimize our loss function. So it doesn't matter if the filter is reversed or not. In fact, if you use a library like SciPy, you'll notice that it already has a function called convolve2d. The problem is if you use this function as is, you'll get a totally different answer than we did. And as a side note, you might want to try it yourself to confirm that what I'm saying is true. Now, this is because convolve2d does a proper convolution and not the deep learning version of convolution with plus instead of minus. In order to make SciPy's convolution work the same way, we have to flip the filter both horizontally and vertically and set the mode argument equal to valid.

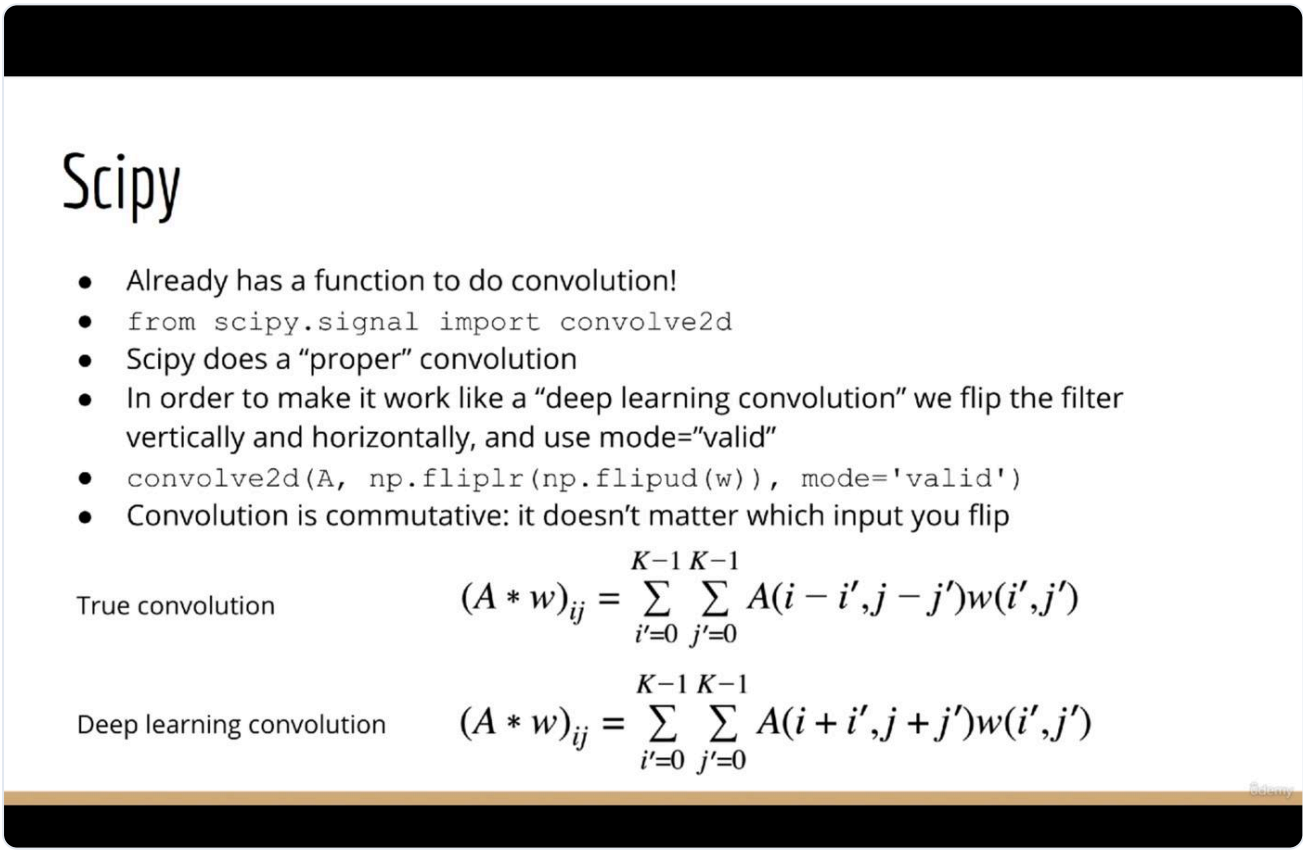


FIGURE 33.20

Now, as a side note, convolution is a commutative operation, a convolve with W is the same as W Convolve with a, therefore it doesn't matter which input we flip. In fact, what we are doing in deep learning is actually known as the cross correlation. I actually think this is a much more helpful and descriptive name compared to convolution convolution in and of itself. Probably doesn't mean much to you. But the word correlation does. You probably think of the word correlated as sameness.

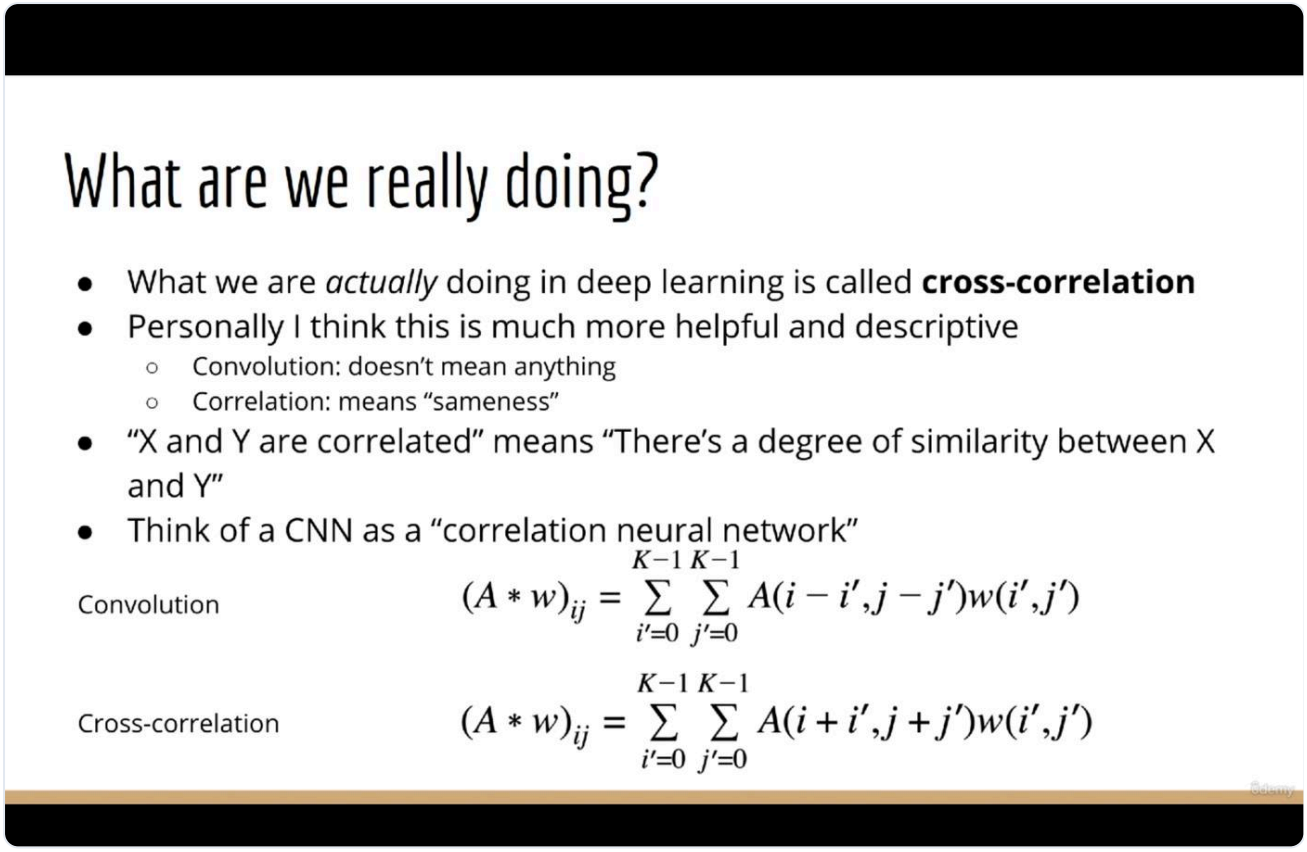


FIGURE 33.21 | What Are We Really Doing?

So if I say X and Y are correlated, that means to you that there is some degree of similarity between X and Y. Therefore, you might think of CNN as a correlational neural network rather than a convolutional neural network. The only difference between correlation and convolution is that convolution reverses the orientation of the filter, whereas correlation does not. The final topic I want to talk about in this lecture is this mode argument to understand this. It's helpful to look at these animations which kind of summarize how Convolution works. Basically, you're sliding the filter across every possible position in the input image.

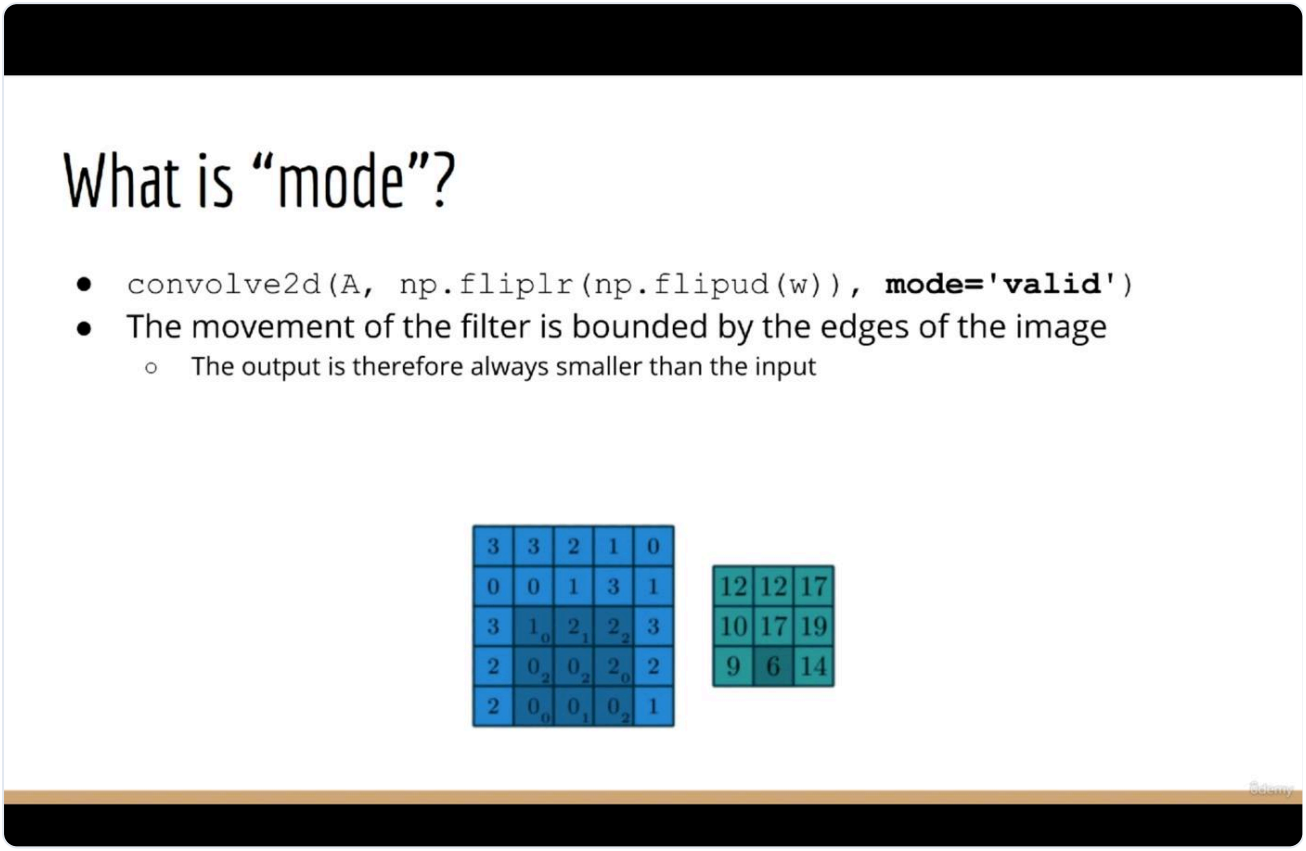


FIGURE 33.22

And this animation, the motion of the filter is bounded by the edges of the image. Because of this output, image is always smaller than the input image.

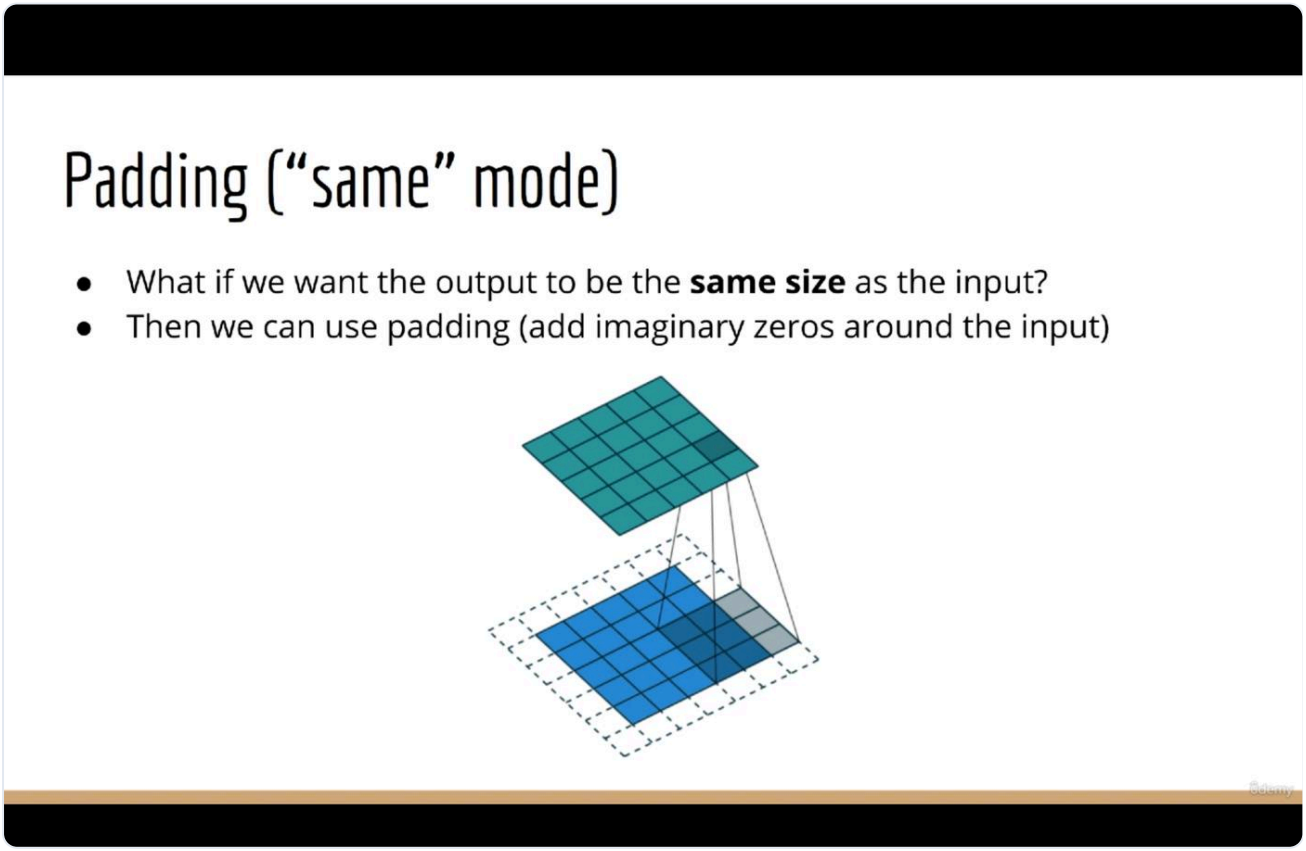


FIGURE 33.23

Even more padding! ("full" mode)

- We *could* extend the filter further and still get non-zero outputs
- Are we losing information if we don't?
- "Full" padding:
  - Input length = N
  - Kernel length = K
  - Output length = N + K - 1

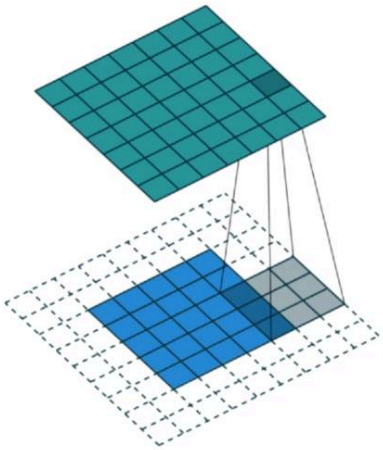


FIGURE 33.24

Summary of Modes

- Input length = N, Kernel length = K

Mode	Output Size	Usage
Valid	N - K + 1	Typical
Same	N	Typical
Full	N + K - 1	Atypical

size in.

FIGURE 33.25 | Mode Output Size Usage

But you might wonder, what if I want the output image to be the same size as the input image in this case, what you can do is add padding. This is equivalent to adding a virtual array of zeros around the input image so that the filter can extend out to those values. The reason I say it's virtual is because you wouldn't really want to allocate the space in code. There's no reason to, since you already know that anything multiplied by zero is still zero. So in other words, to quote unquote add padding, you can just pretend there are zeros surrounding the input image as many zeros as you need to ensure that the output size is equal to the input size.

You may have noticed, however, that even with padding, we still lose some information. What I mean by that is there are outputs we could calculate if we extended the padding even further. That would be non-zero outputs. So one other thing you can do, which is not as common these days, is to extend the padding further so you can catch all these non-zero outputs. This results in an output size of N plus K minus one. If your input image has length N and your filter has length K. Again, you should draw this out on paper. If you're not convinced this is true. To summarize the three modes of convolution, let's look at this table. The first one we discussed was called valid convolution.

This applies when the kernel can only touch the original input image. The output size is N minus K plus one. The second one we discussed was called same convolution. In this scenario, we add some padding just enough so that the output size is the same as the input size in. The third mode we discussed is called full convolution in this scenario. We extend the filter out as far as possible so that at least one point on the filter overlaps with one point on the input image. The output size is N plus K minus one. Normally in deep learning, we use valid or same convolutions.

LECTURE 34

Cosine Similarity

In this lecture we are going to continue our discussion of convolution by looking at a totally different perspective on how it works.

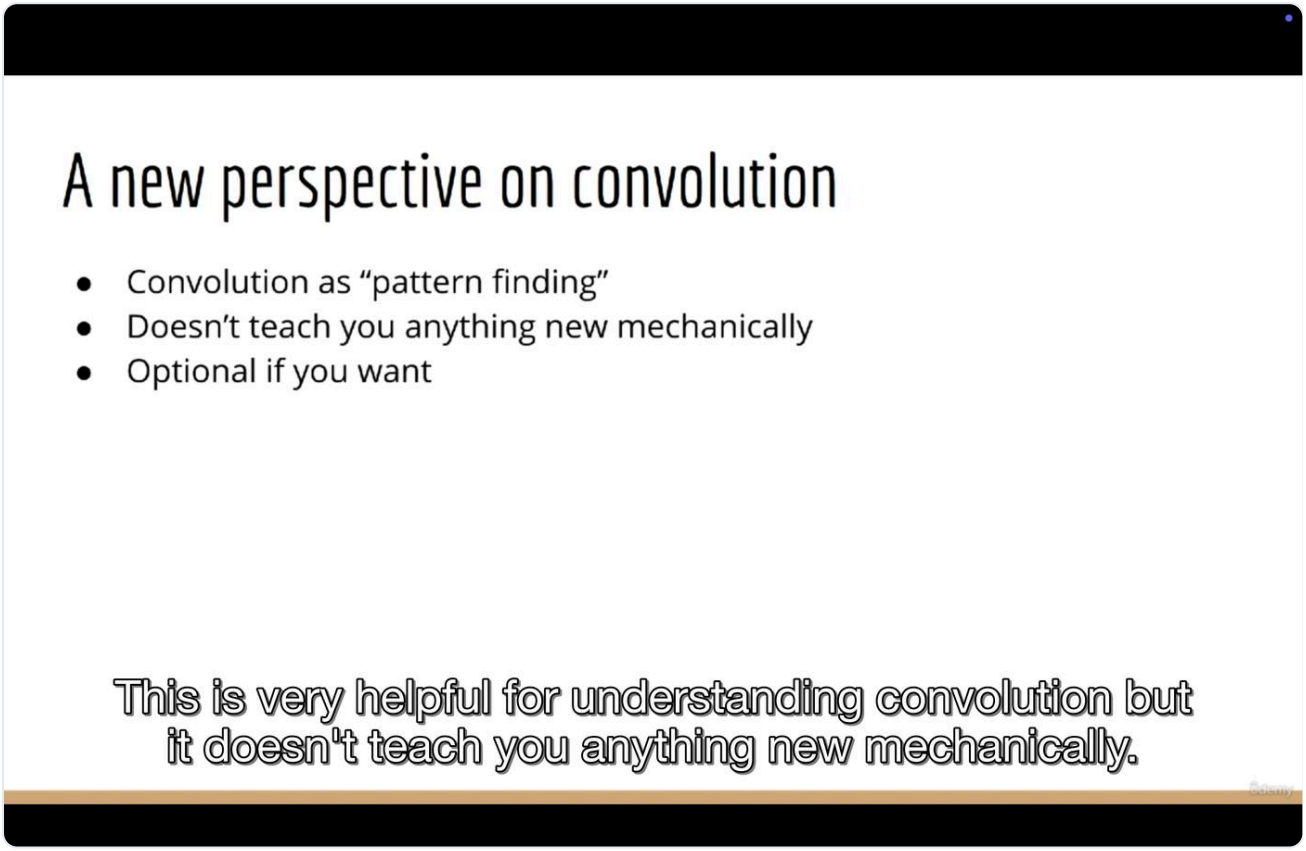


FIGURE 34.1 | A New Perspective on Convolution

This is very helpful for understanding convolution but it doesn't teach you anything new mechanically. So this lecture is optional. If you want to move on and just learn about how to make CNNs something I mentioned very often is vectorization.

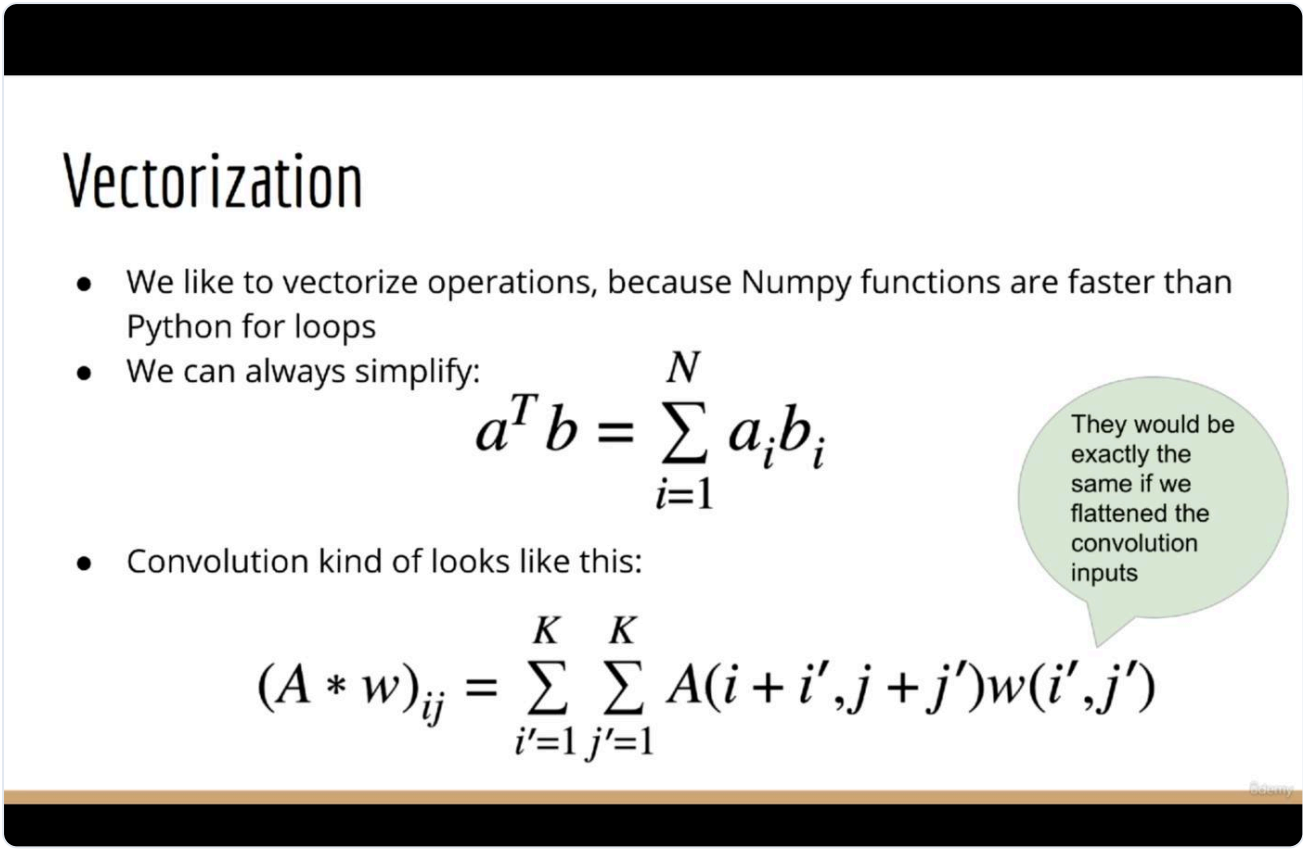


FIGURE 34.2 | Python for Loops

We like to vectorize operations in code because using NumPy functions is a lot more efficient than writing your own Python for loops. The pattern you're looking for when you want to vectorize an operation is usually of the form of a dot product a dot product is an element-wise multiplication and then the summation of those results. So whenever you see the sum over I of A times B I that's a dot product. This also applies to matrix multiplication. Since if a and b are matrices then the matrix multiplication of A and B is just A dot B you'll notice that for convolution We yet again have something similar. The only difference is that there are two summations.

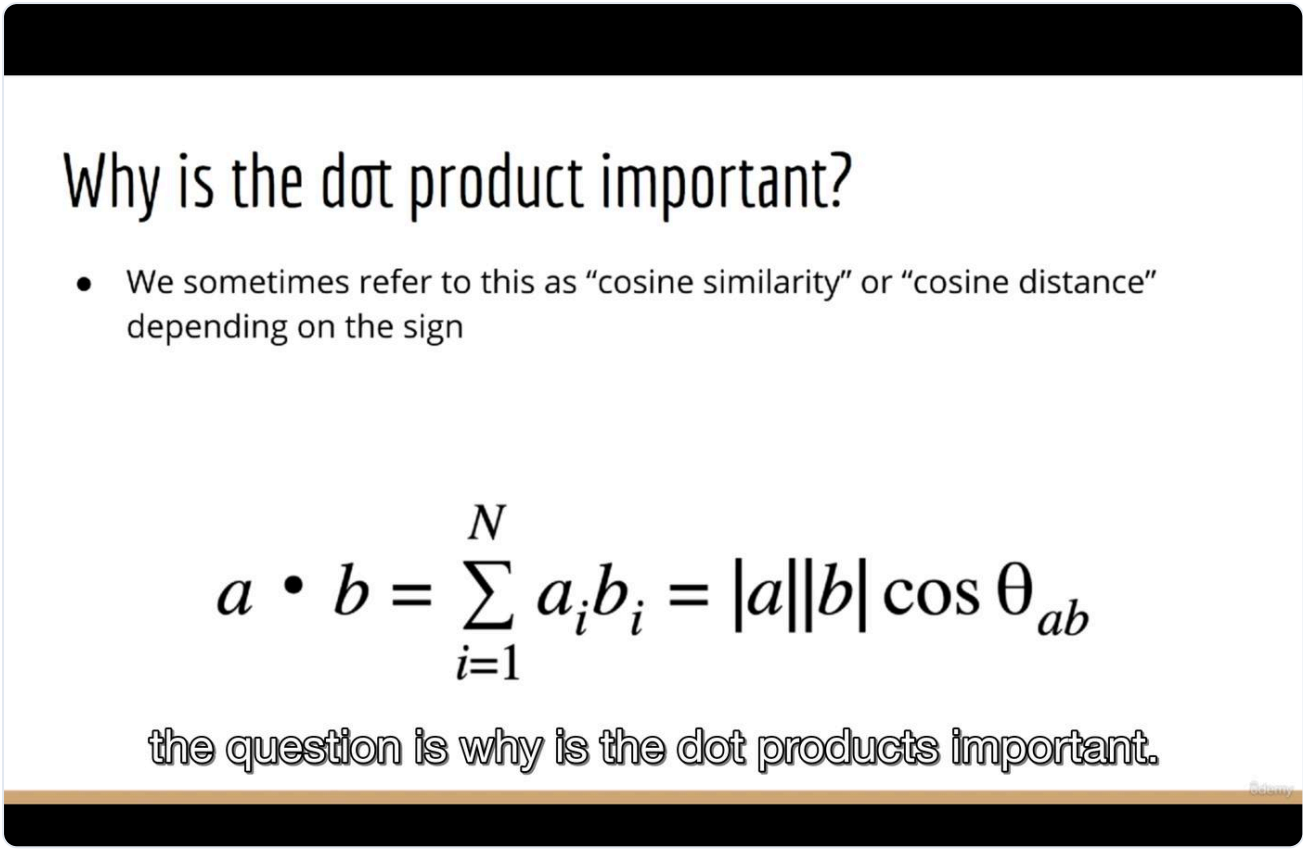


FIGURE 34.3

However this is not really a relevant detail since the outcome is still the same instead of summing over one axis we are summing over two axes but it's still an element-wise someone at the question is why is the dot product important. One definition of the dot product. Other than that element-wise some an ad is that it's the magnitude of a multiplied by the magnitude of B multiplied by the cosine of the angle between and B. We sometimes call this the cosine similarity or cosine distance depending on the sign that you use. So how does this work geometrically.

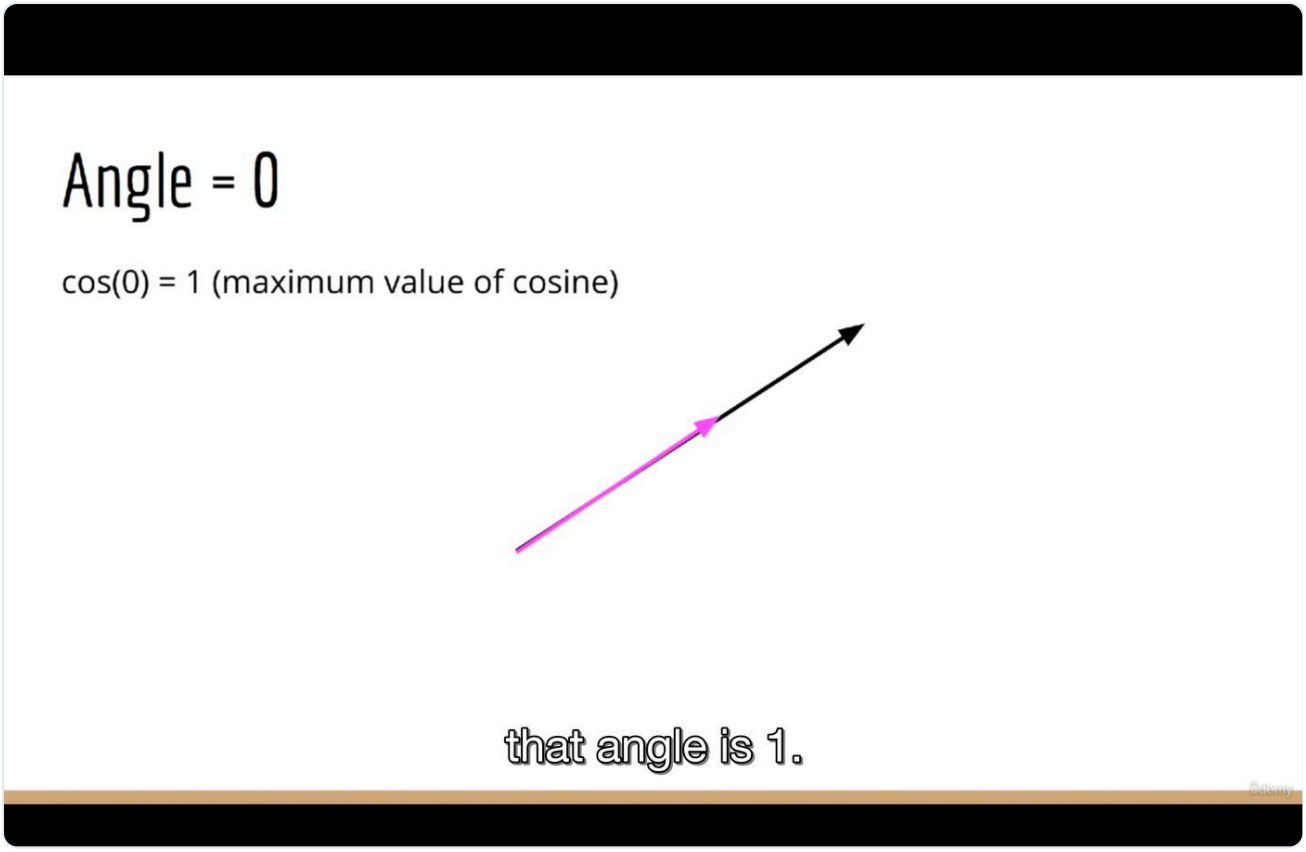


FIGURE 34.4

Consider just the angle for a moment if the angle between the two vectors is zero then the cosine of that angle is 1. That's the maximum value of the cosine.

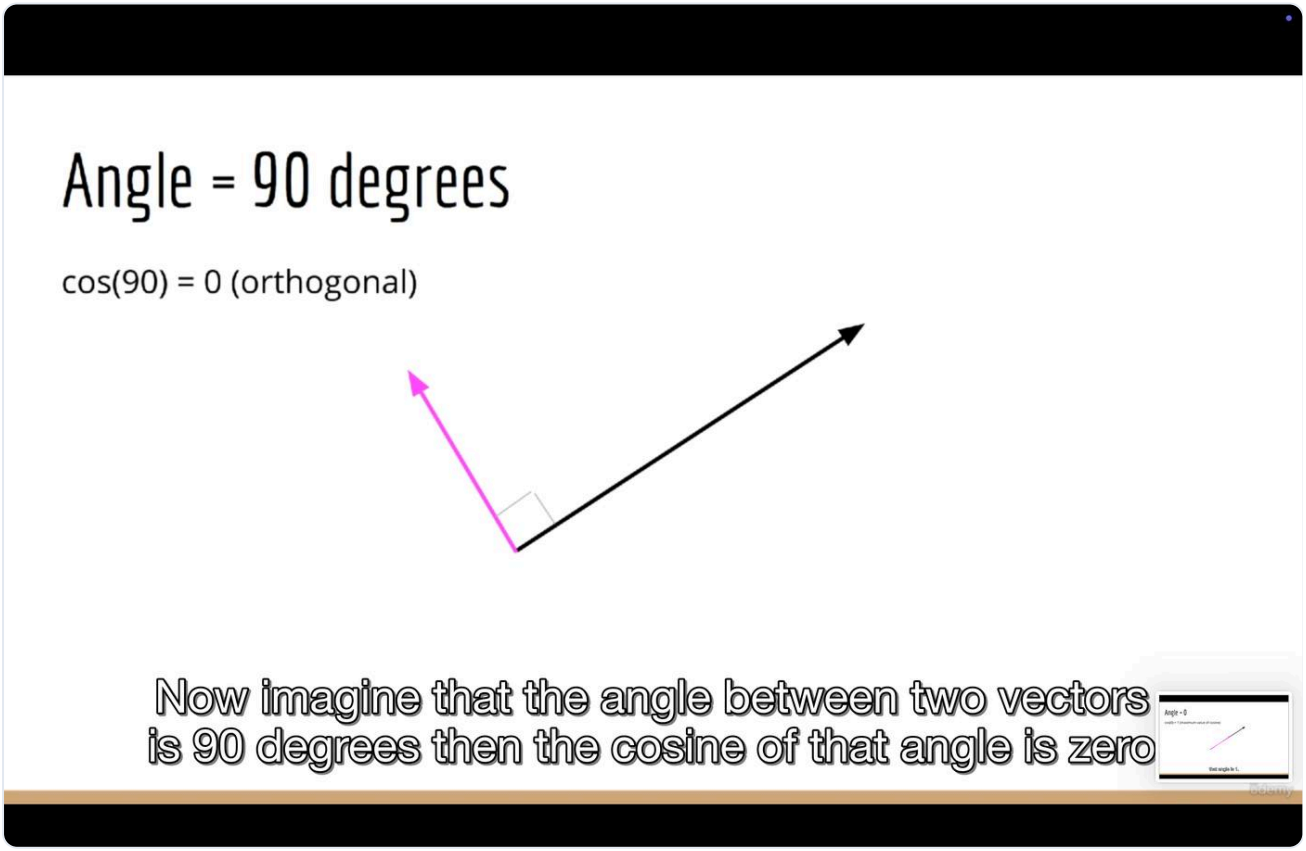


FIGURE 34.5 | Now Imagine That Tine Angle Between Two Vectors

Now imagine that the angle between two vectors is 90 degrees then the cosine of that angle is zero finally.

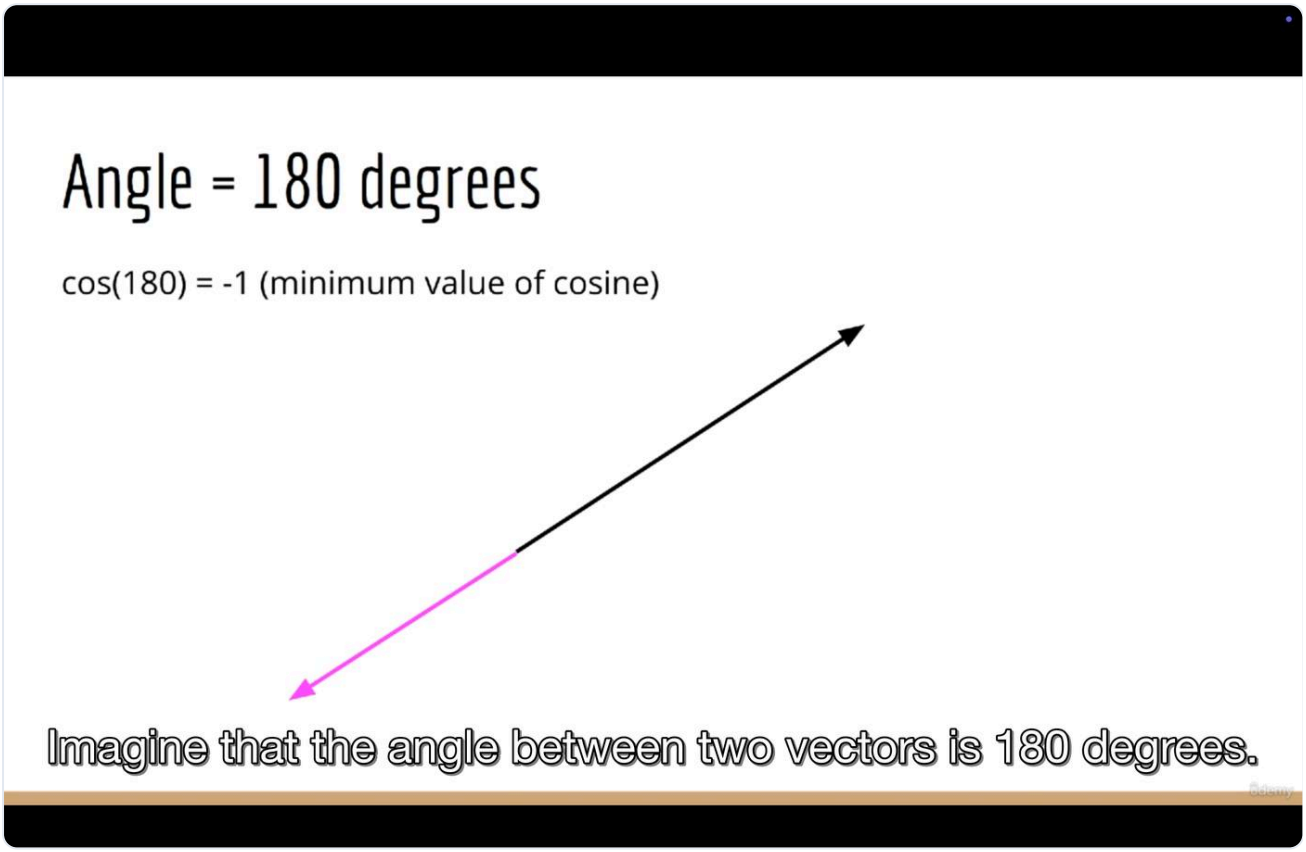


FIGURE 34.6

Imagine that the angle between two vectors is 180 degrees. This is basically as far apart as possible.

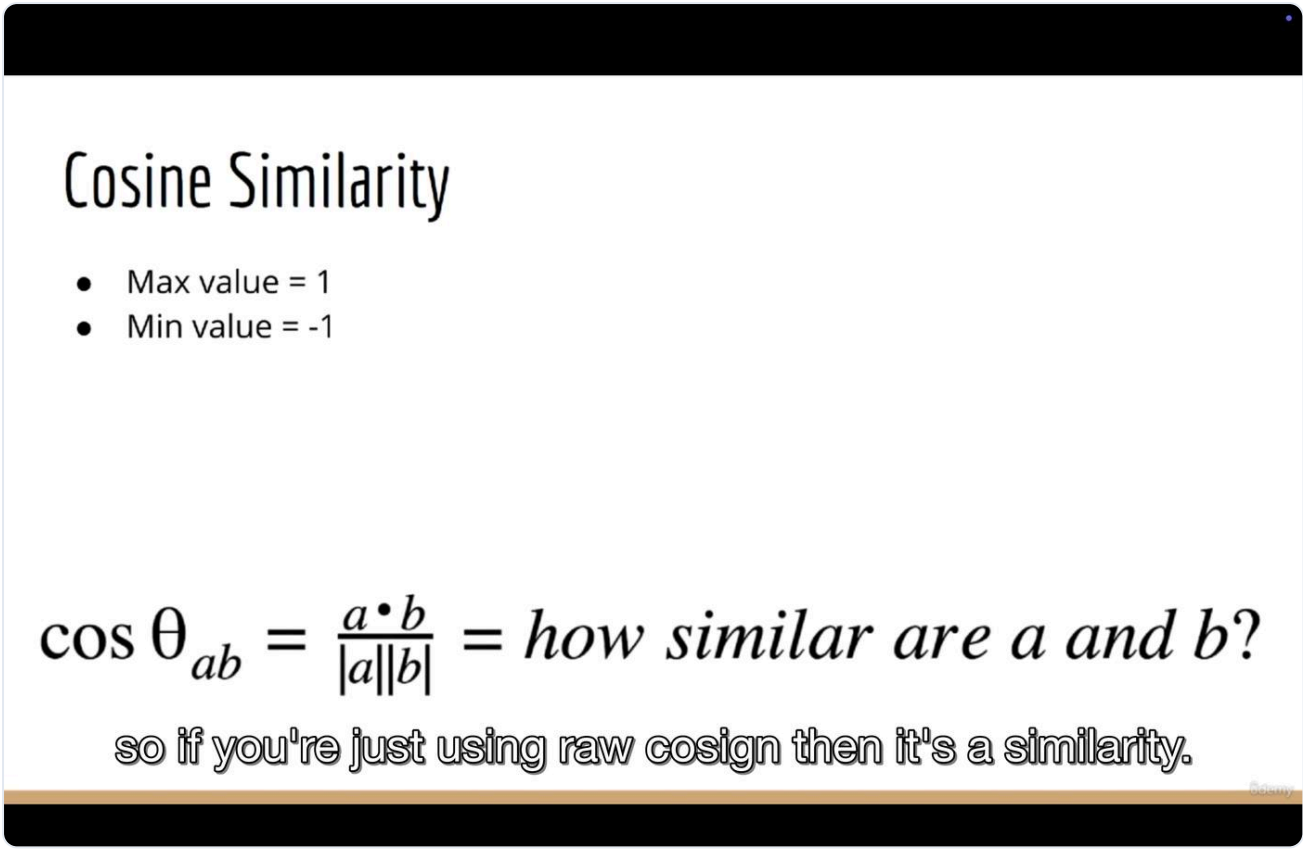


FIGURE 34.7 | Cosine Similarity

Then the cosine of that angle is minus 1 which is the minimum value of the cosine so if you're just using raw cosine then it's a similarity. The larger the number the closer the two vectors on the smaller the number the further away they are the maximum value. When A and B are parallel is one the minimum value when A and B are anti parallel is minus one. And if A and B are orthogonal then the cosine similarity is just zero OK. So why is that important.

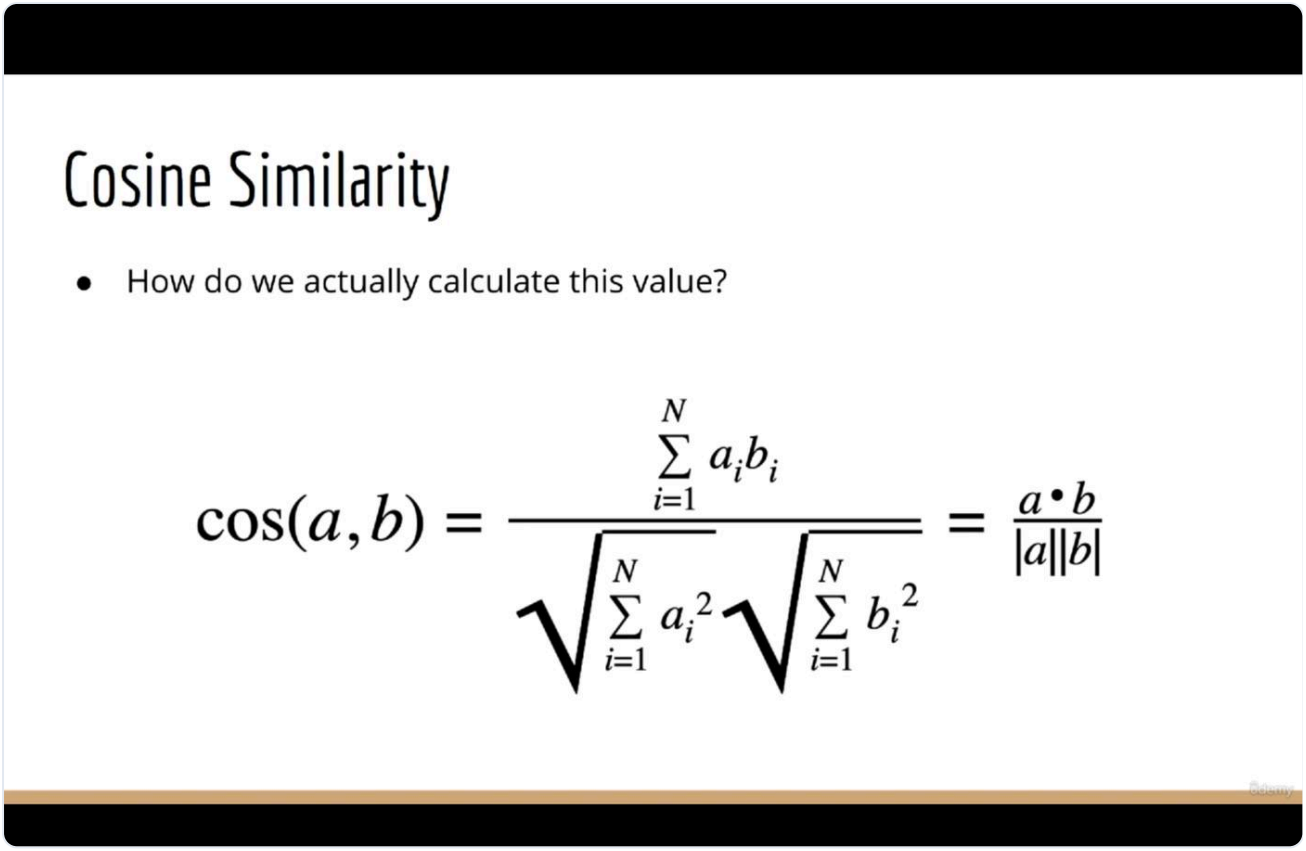


FIGURE 34.8 | Cosine Similarity

Consider now how you would find that the cosine of the angle between two vectors. That is just A dot B divided by the magnitude of A and the magnitude of B. So I just rearrange the equation that we had before now. Let's compare this to another popular measurement the Pearson correlation the Pearson correlation is defined as what you see here. But notice how similar this is to cosine similarity.

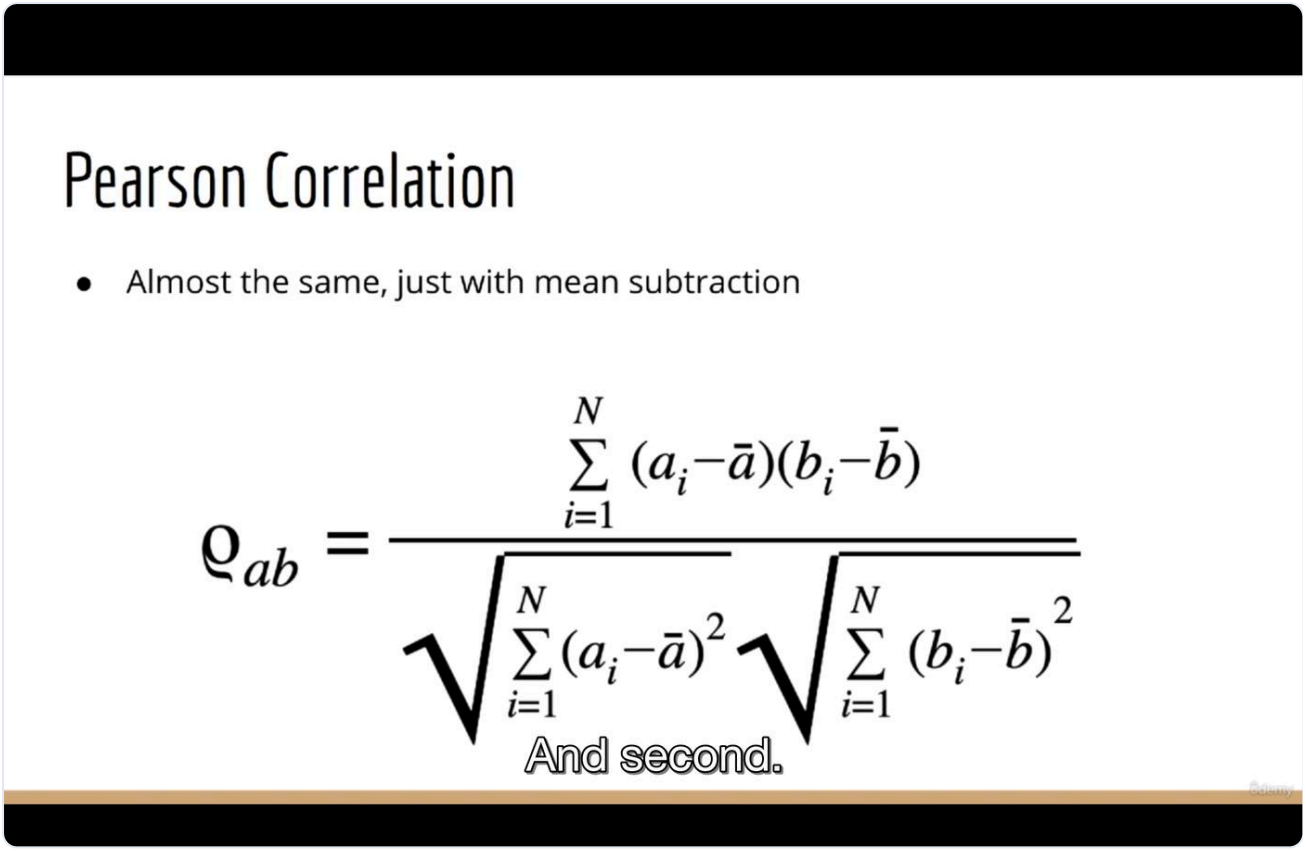


FIGURE 34.9 | Pearson Correlation

The only difference is that the Pearson correlation uses mean subtraction. And so now you have two hints that convolution is really correlation. The first one from before it was that we were actually doing what is called the cross correlation. And second. Now you have that the dot product is actually very similar to the Pearson correlation so I hope that by this point you are convinced that the dot product. While that seems like an abstract concept can be thought of as a correlation measure.

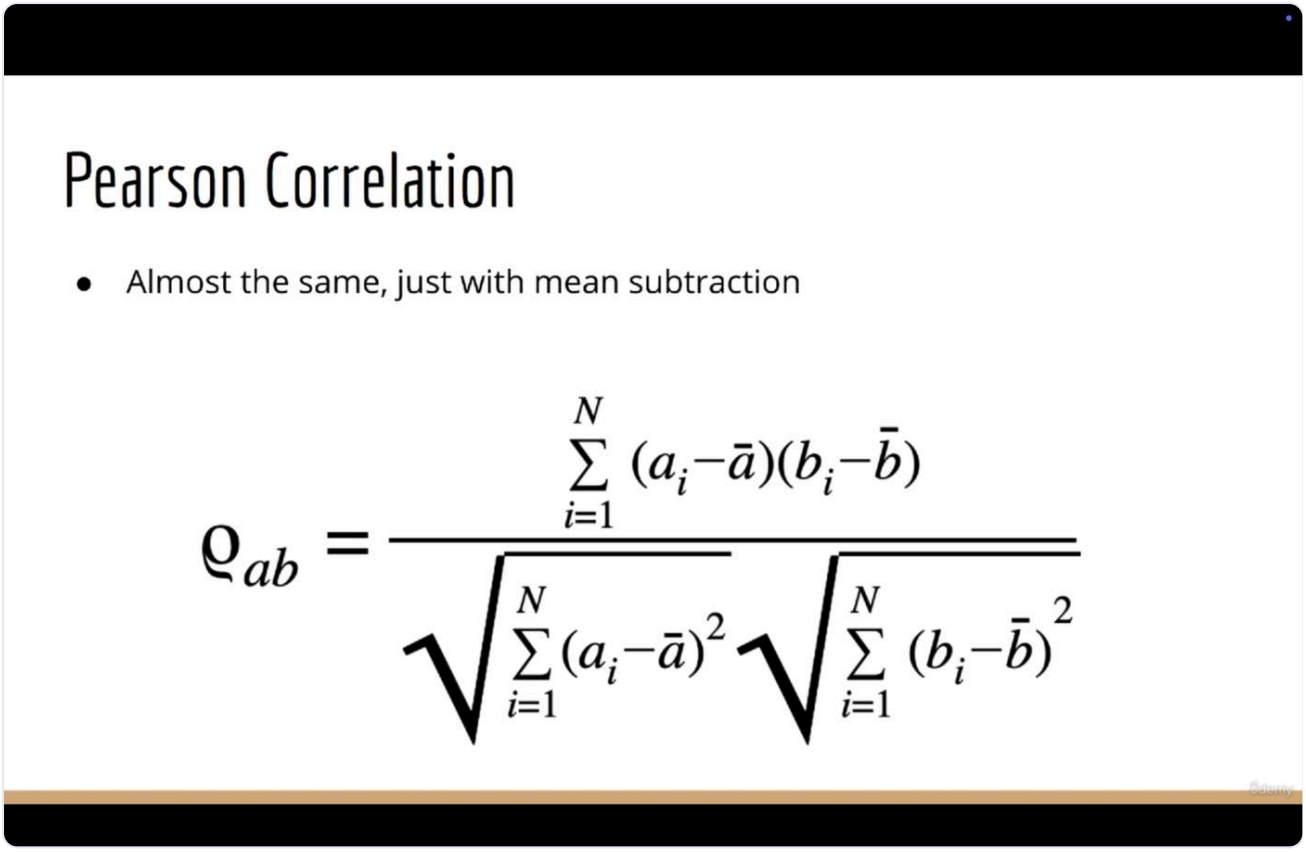


FIGURE 34.10 | Pearson Correlation

It tells me how correlated is the first thing with the second thing. If they are highly positively correlated then the dot product should be large and positive. That means two vectors pointing in nearly the same direction. If they are highly negatively correlated then the dot product should be large and negative. That means the two vectors are pointing in nearly opposite direction.

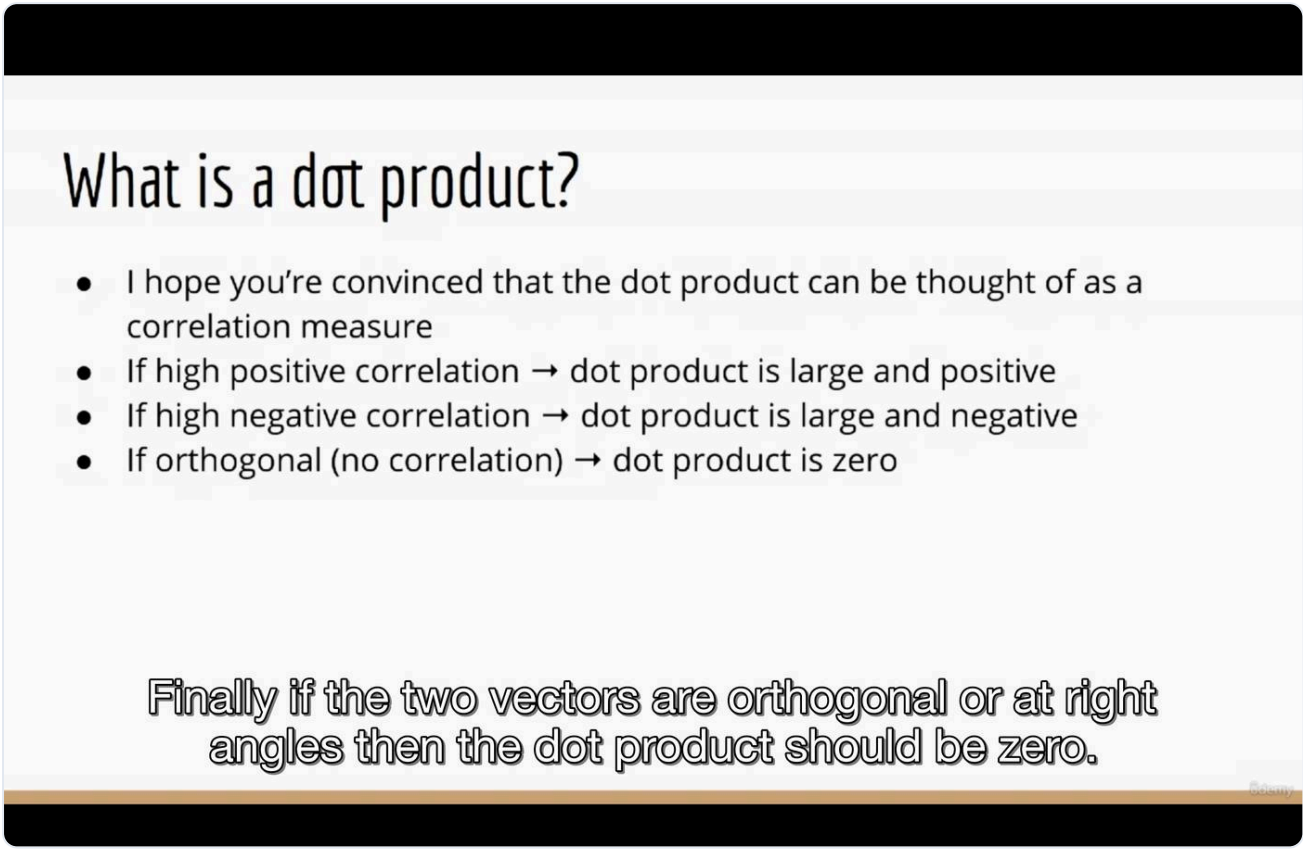


FIGURE 34.11 | What Is a Dot Product?

Finally if the two vectors are orthogonal or at right angles then the dot product should be zero. The reason why this is important is you don't have to think of a filter as an abstract concept. In fact it's just a pattern finder. This actually makes the term filter make a lot more sense.

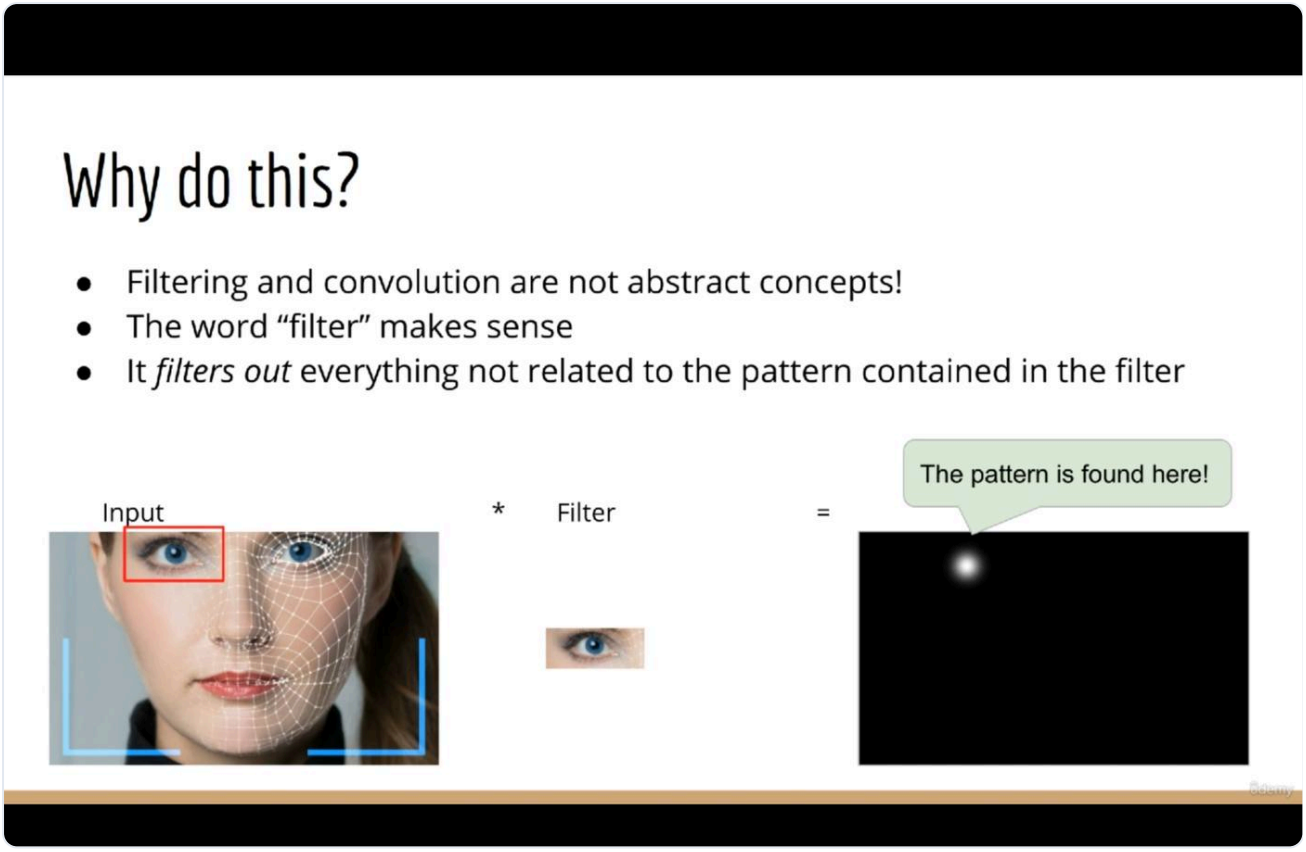


FIGURE 34.12

It filters out everything not related to the pattern contained in the filter by setting them to zero and keeps everything that is related to the pattern. So what convolution is doing is it's passing this filter along each point on the original input image and sliding it along at each point. It asks is the pattern here is the pattern here is the pattern here. And so forth. Then it gives us a high number in the positions where the pattern is found and then a small number where the pattern is not found and thus this is your first alternative perspective on convolution. It's just a sliding pattern finder that passes through an entire image. Looking for a particular pattern.

LECTURE 35

Alternative Perspectives on Convolution

In this lecture we are going to continue our discussion on alternative perspectives of convolution. Again this is very helpful for understanding convolution but it doesn't teach you anything new mechanically. So this lecture is optional.

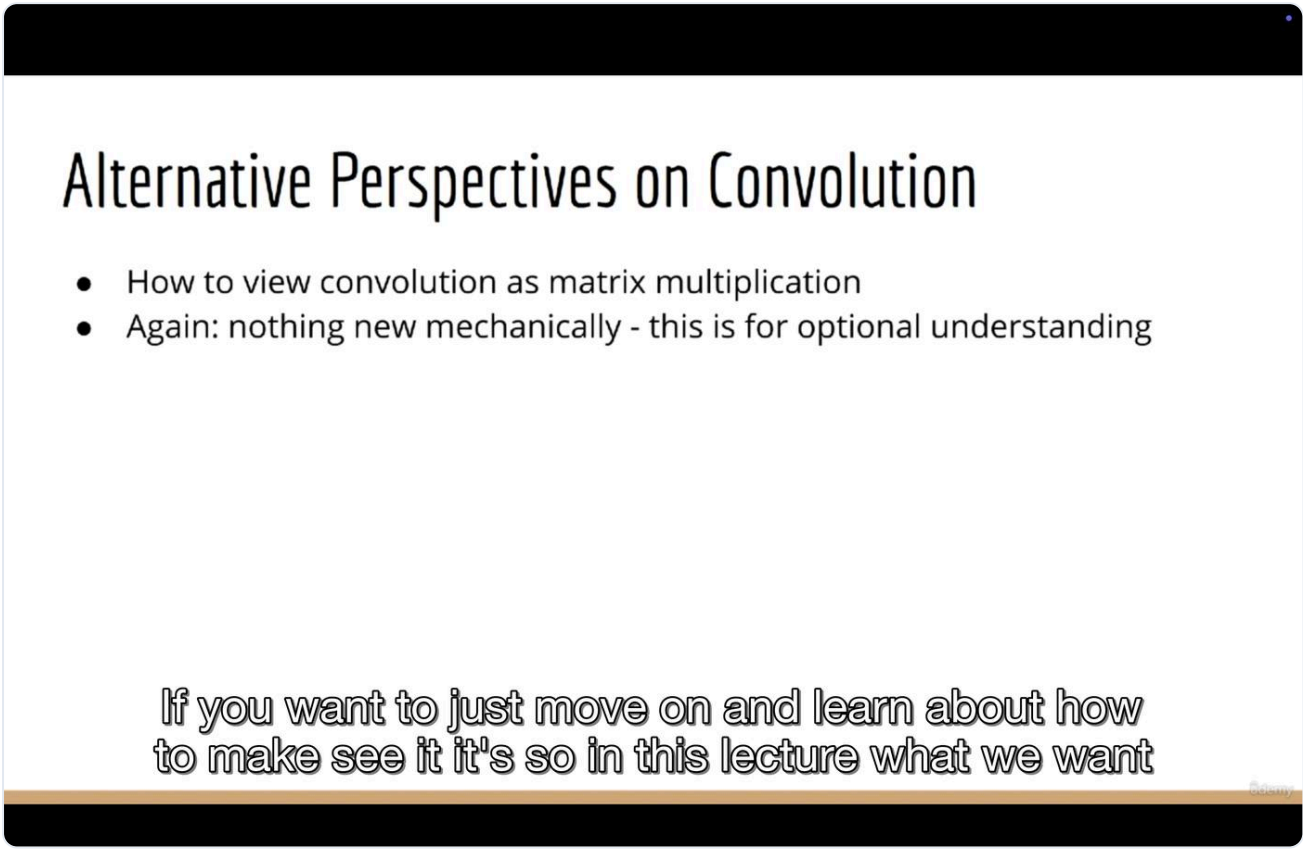


FIGURE 35.1 | Alternative Perspectives on Convolution

If you want to just move on and learn about how to make CNNs so in this lecture what we want to talk about is the equivalence of convolution and a matrix multiplication. In other words In what scenario are they actually the same thing let's do an example.

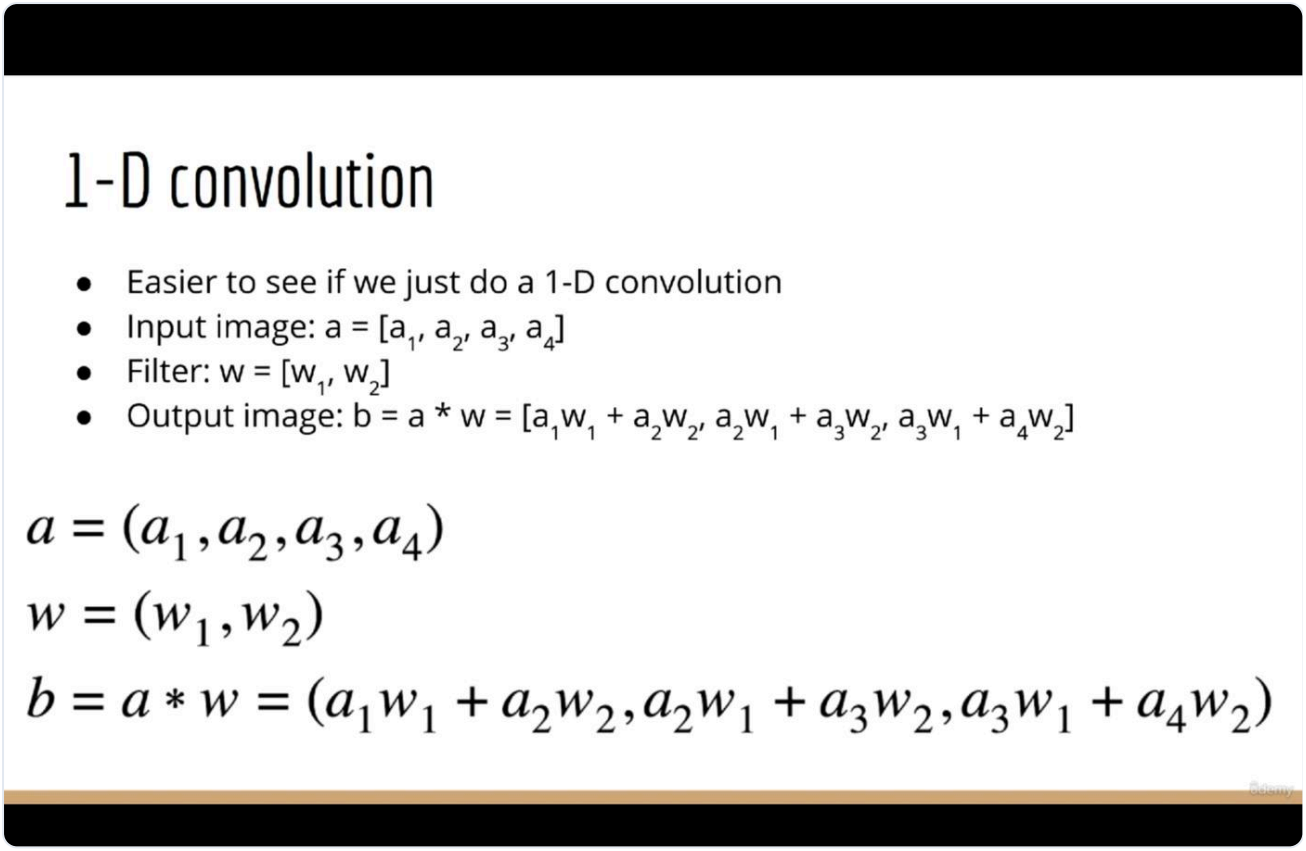


FIGURE 35.2

This is easier to see if we only do a one dimensional convolution. So let's first go over how that would work. This should be easy since two dimensional convolution is just a generalization of this. So let's start with a one dimensional image A equals a one a two a three a four.

1-D convolution

- We can summarize it as follows
- Same as 2-D convolution, just without 2nd index

$$b_i = \sum_{i'=1}^{K} a_{i+i'} w_{i'}$$

so in general we can write one dimensional convolution as the sum from i prime equals one up to k of

FIGURE 35.3

Matrix Multiplication

- We can do the same operation using matrix multiplication

$$\begin{pmatrix} a_1w_1 + a_2w_2 \\ a_2w_1 + a_3w_2 \\ a_3w_1 + a_4w_2 \end{pmatrix} = \begin{pmatrix} w_1 & w_2 & 0 & 0 \\ 0 & w_1 & w_2 & 0 \\ 0 & 0 & w_1 & w_2 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{pmatrix}$$

b w a

FIGURE 35.4 | Matrix Multiplication

We also have a filter W equals w one W2 then the convolution output will be B which is A convolved with W and the elements are A1 times w one plus a two times w two a two times w one plus a three times w two and a three times w one plus a four times W2 so in general we can write one dimensional convolution as the sum from i prime equals one up to k of a at I plus I prime times w of I prime you'll notice that this is the same equation we had for two dimensional convolution just with the second index dropped using this it's pretty easy to see how we would implement the same thing using matrix multiplication what we've done here is we created a matrix where we repeat the filter along each row but on each row we shift it to the right by one space.

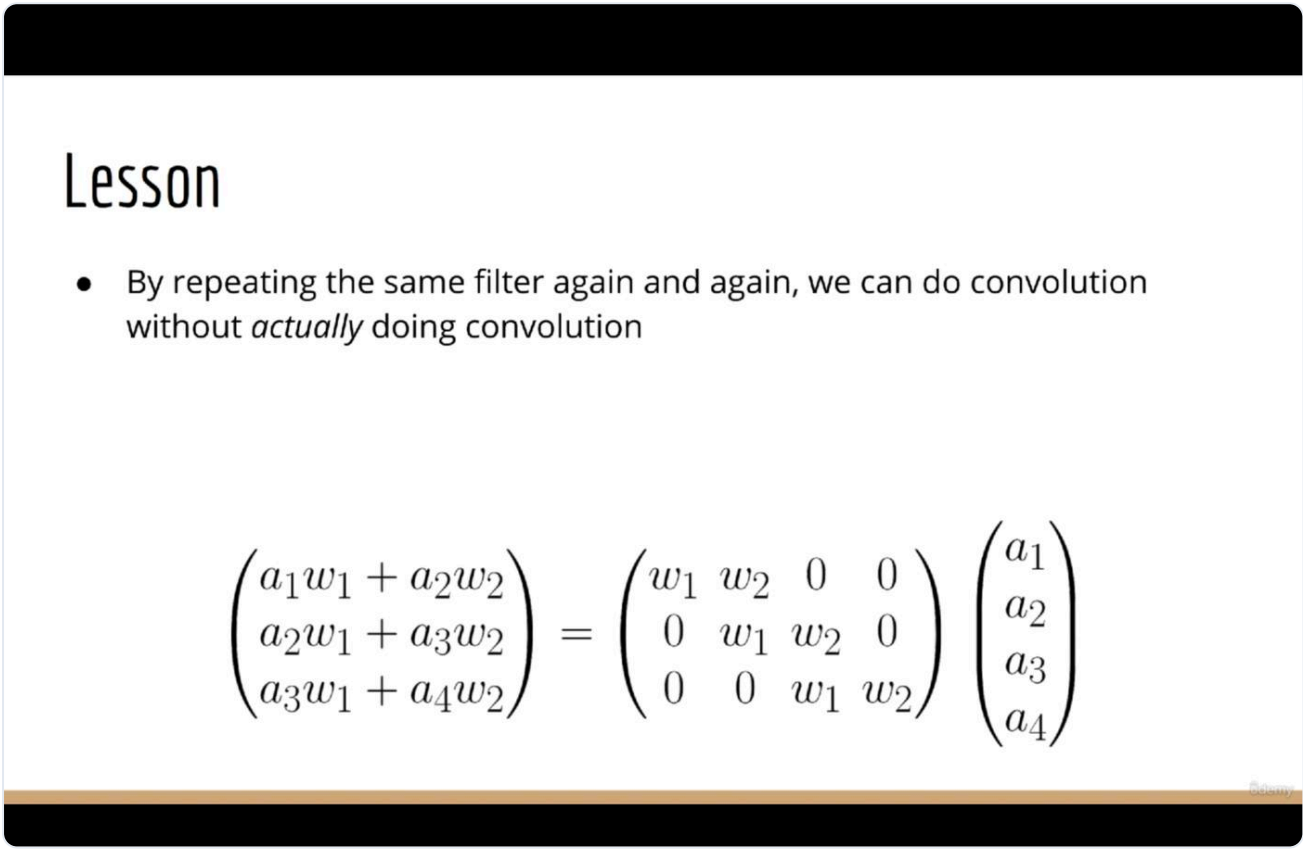


FIGURE 35.5

Now I recommend you do this by hand so you can see that it works if you multiply the matrix by the original input vector A you get the correct output of the convolution So what's the lesson here is that by repeating the same filter again and again inside a matrix we can implement convolution without actually doing convolution instead we can just use matrix multiplication but there is another problem with this particular method of doing matrix multiplication and this is that the equivalent matrix repeats the filter multiple times. So the result is that the matrix takes up a lot more space than the original filter which was just a two element array originally.

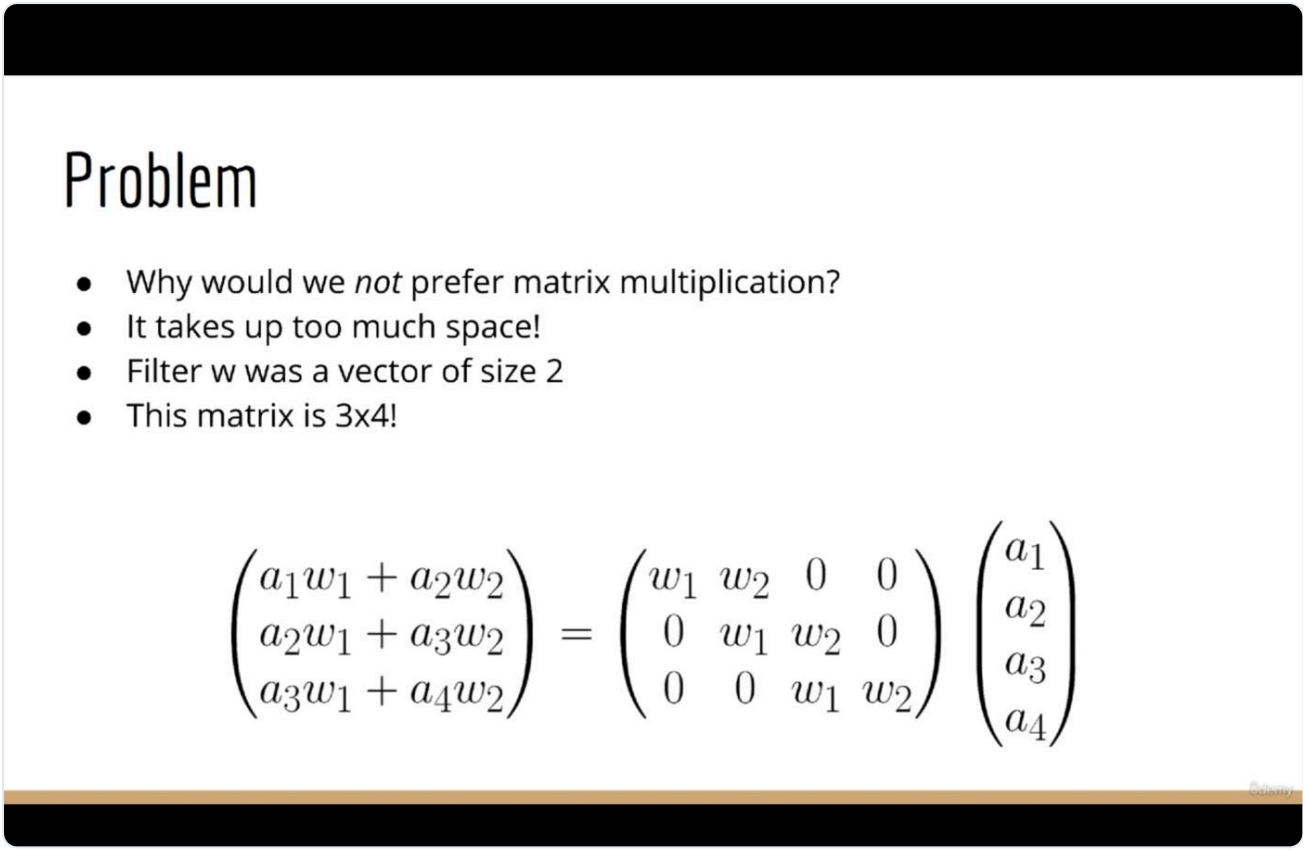


FIGURE 35.6 | Why Would We Not Prefer Matrix Multiplication?

In other words while this was a helpful perspective you don't necessarily want to implement convolution this way instead. It's helpful to look at this from the opposite direction.

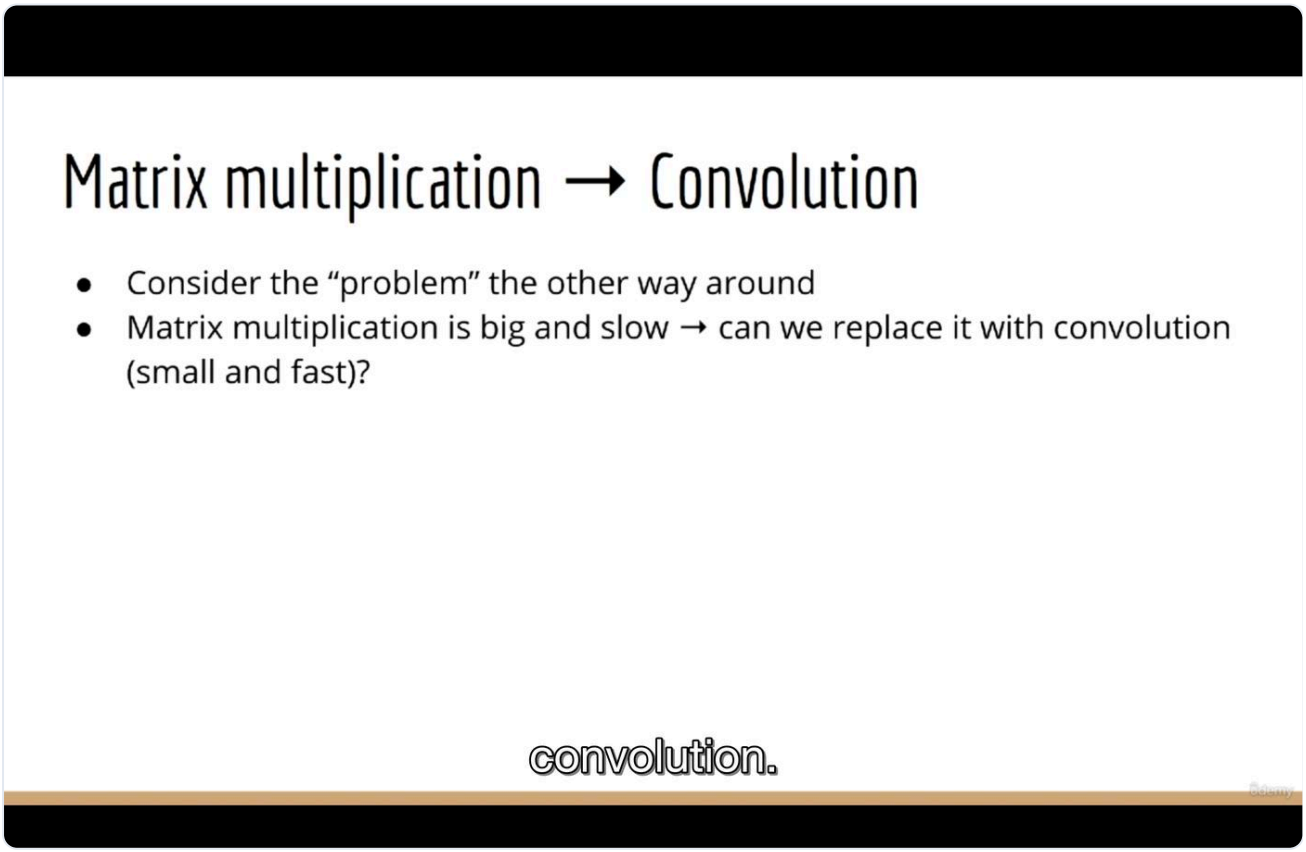


FIGURE 35.7

What if there are instances where instead of doing a full matrix multiplication we can replace it with convolution. This is the idea behind parameter sharing or weight sharing. Think of it this way inside a neural network we are constantly doing this operation a equals w transpose x.

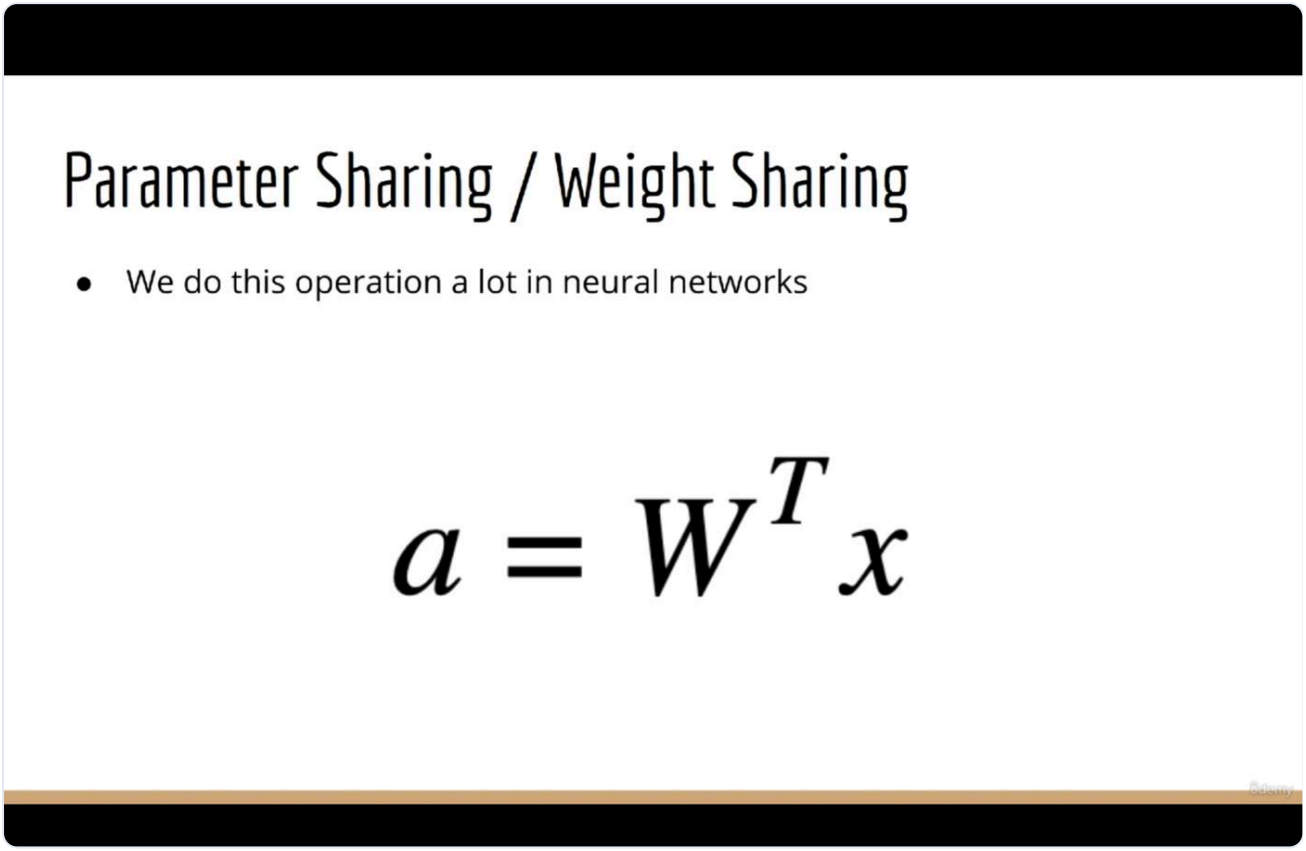


FIGURE 35.8 | Parameter Sharing / Weight Sharing

Here A is the output activation and X is the input feature vector. W is the weight matrix.

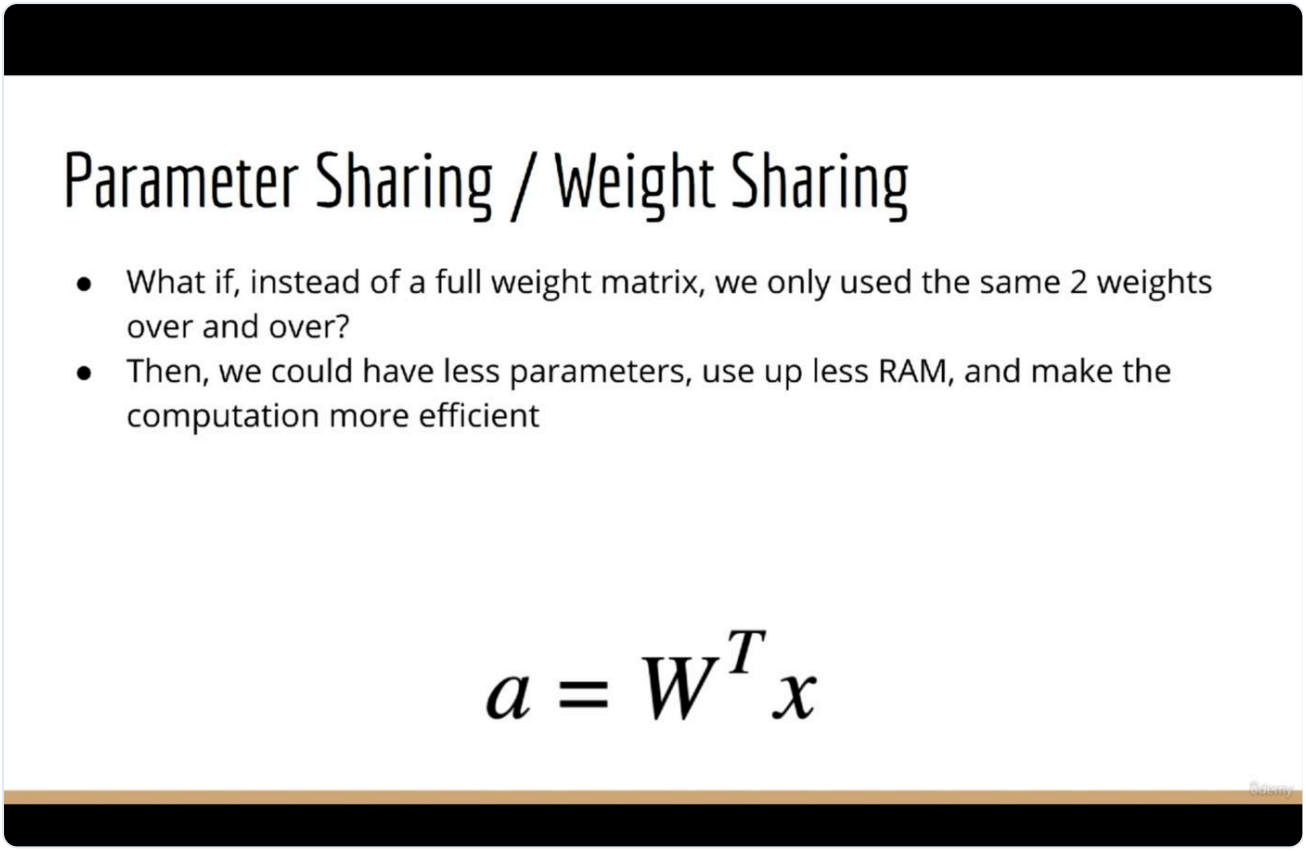


FIGURE 35.9 | Parameter Sharing / Weight Sharing

Well what if instead of a full weight matrix we just had the same two weights repeating over and over again. Then we have less parameters our neural network takes up less memory and ram and thus we can make the computation more efficient. In other words convolution saves both space and time by using less weights why might we want to do this. Consider what we did in the previous section where we had a fully connected neural network looking at a single image. Luckily those images were just grayscale images of size 28 by twenty eight which gives us seven hundred eighty four features. But what if we had a slightly larger image and add color. The CIFAR-10 dataset is one such example.

Those images are 32 by 32 by 3.

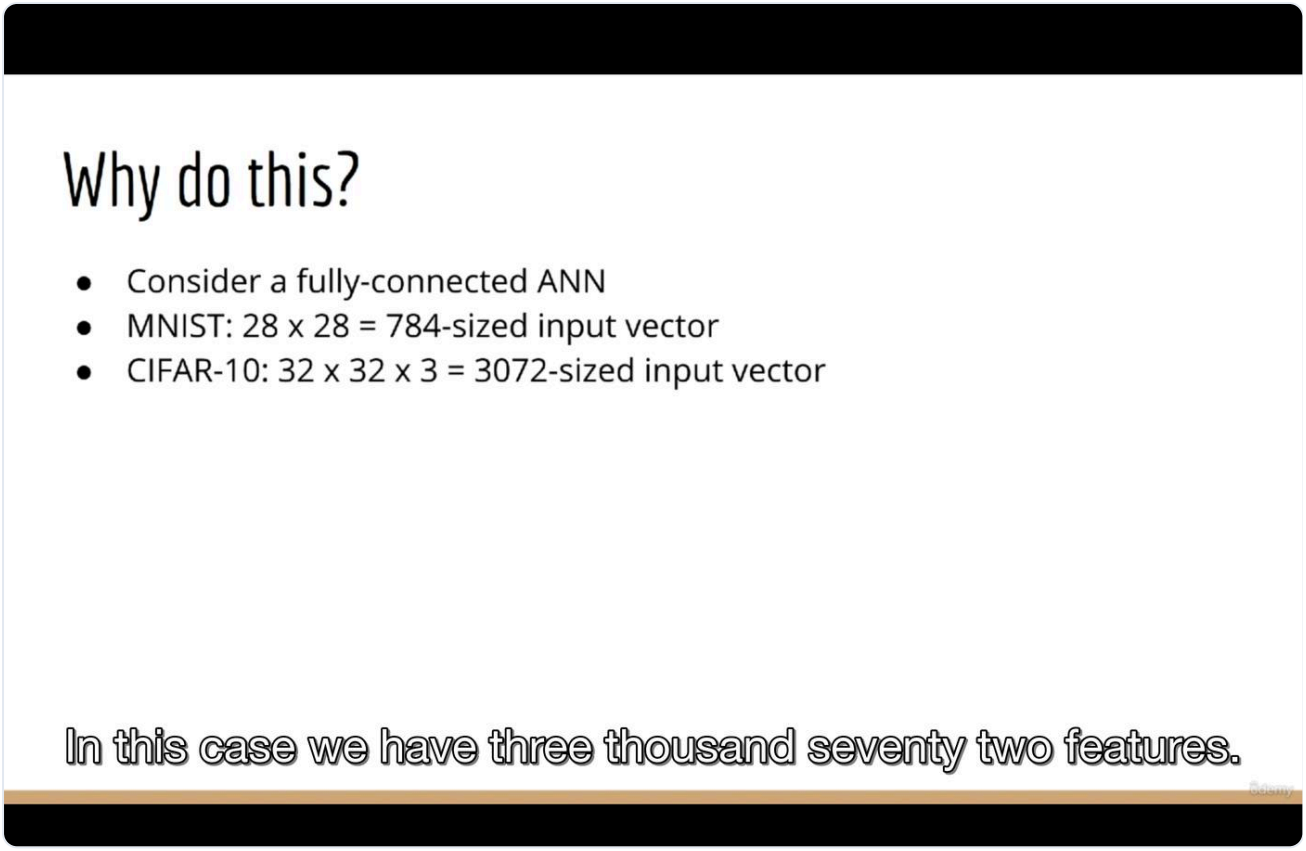


FIGURE 35.10 | Why Do This?

In this case we have three thousand seventy two features. That's quite a large increase from seven eighty four. And keep in mind a 32 by 32 image is quite modest in size to say the least.

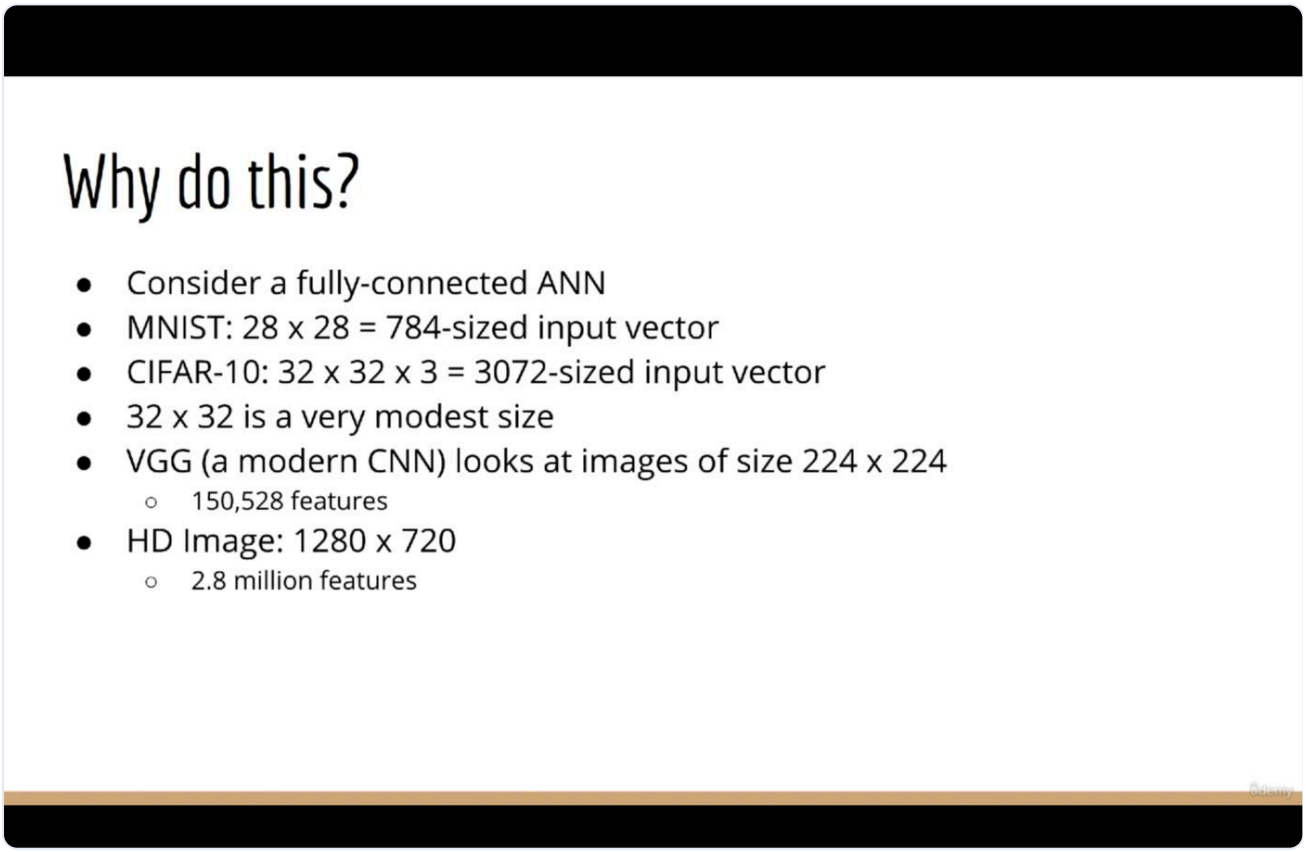


FIGURE 35.11 | Why Do This?

Modern CNNs such as VGG look at images of size 224 by 224. If you use a full weight matrix then you would have one hundred fifty thousand five hundred twenty eight features. Suppose you're looking at an image with modern HD resolution that is 1280 by 720. In this case you would have about 2.8 million features. As you can imagine this is much too large for a neural network but there is another good reason to use weight sharing which is that you don't need different weights for each part of the neural network. Remember that convolution is actually correlation and the filter is really a pattern finder.

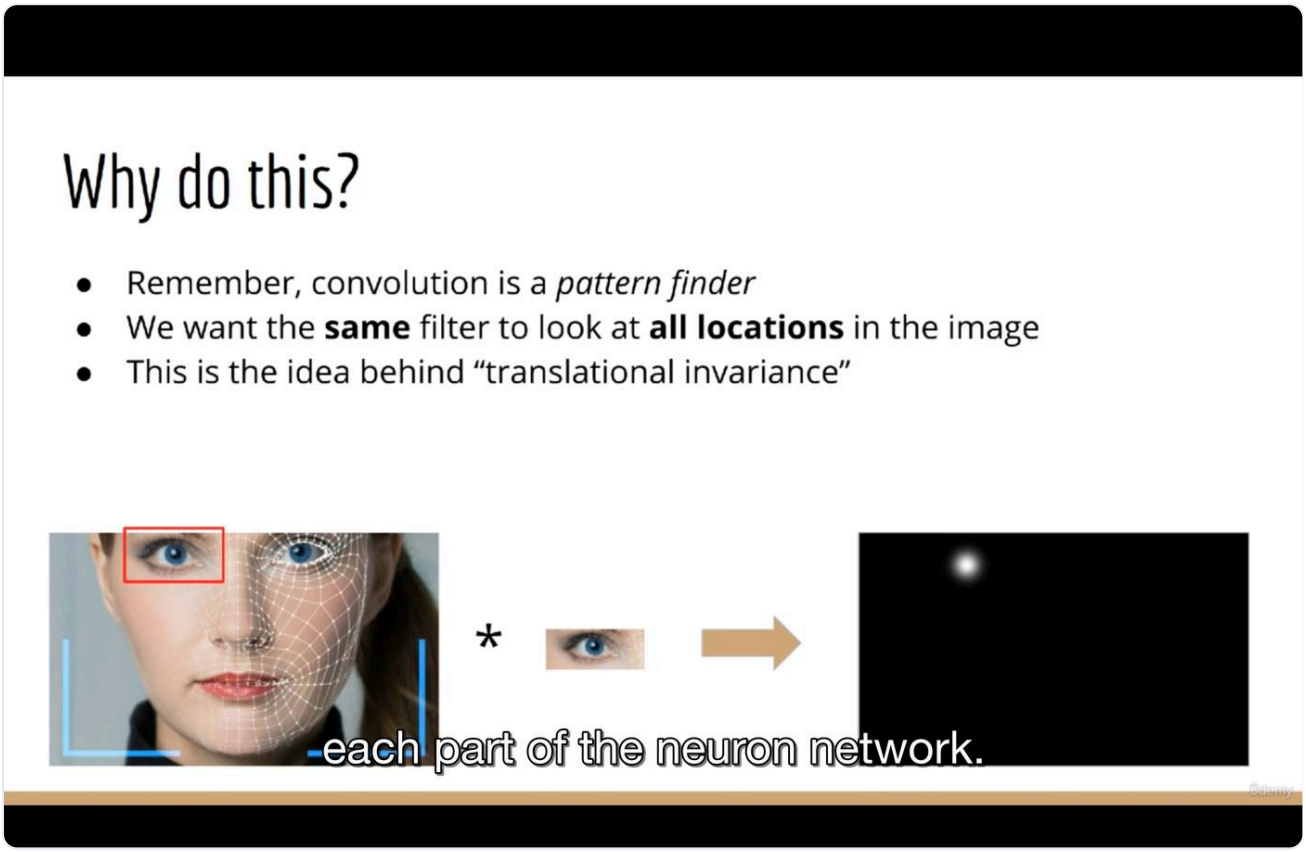


FIGURE 35.12 | Why Do This?

In this scenario we actually want the same filter to look at all locations on the image. This is the idea behind translational invariance suppose we are building a dog versus cat recognizer. Here is an image containing a cat. Here's another image containing a cat. But what's the difference between these two images. The only difference is that the cat is in a different position.

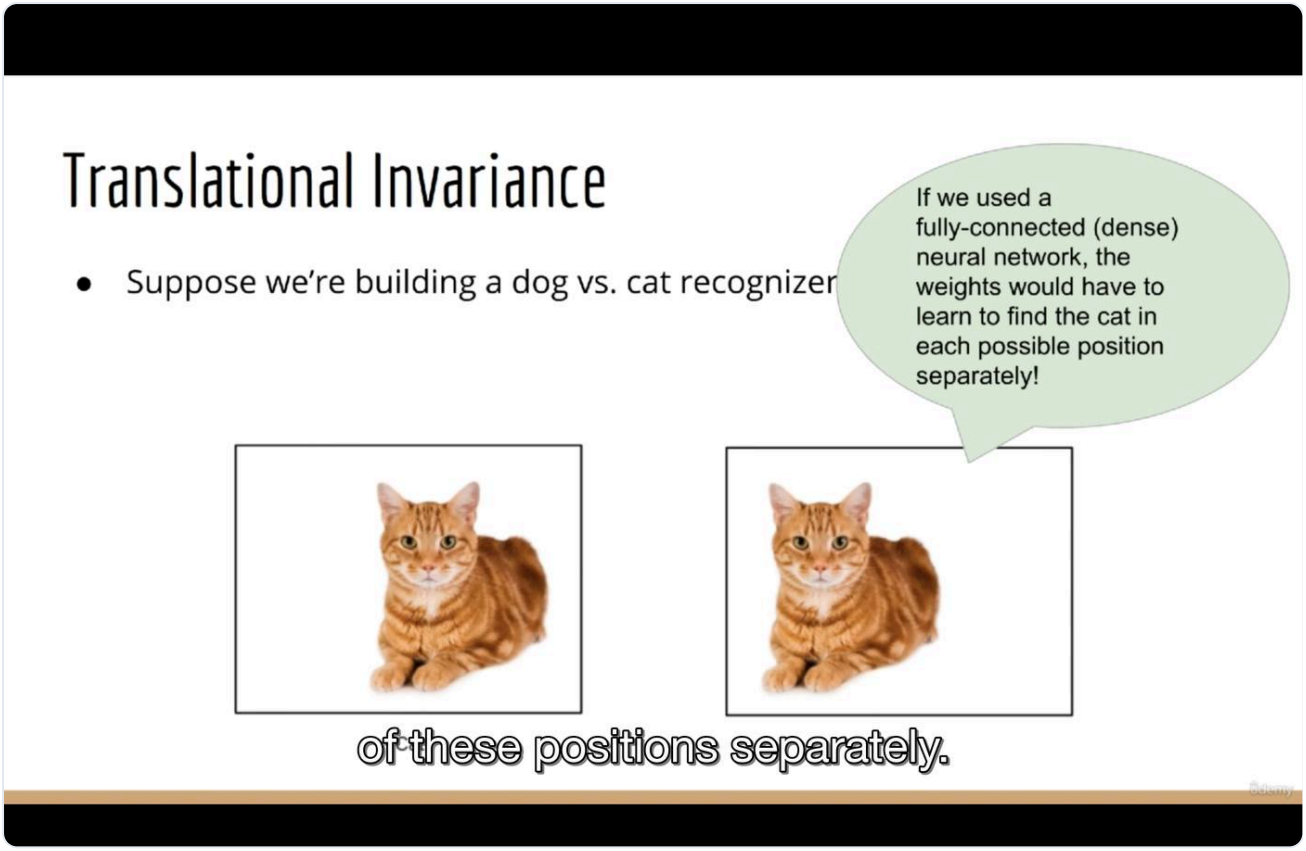


FIGURE 35.13

Now if we use the fully connected or dense neural network we would have to learn the weights for each of these positions separately. And on top of that this neural network won't generalize well because if we come across the same cat but in a new position the neural network would have failed to recognize it. So in fact it's better to have a shared pattern finder because this pattern finder looks at all locations on the image. You don't need to learn the weights to look at a cat at every single possible point which would actually be infeasible.

LECTURE 36

Convolution on Color Images

In this lecture we are going to extend our understanding of convolution so that it looks a little more like what goes on in a neural network. First remember that in general a neural network accepts as input color images. We know that these are three dimensional objects height by width by color.

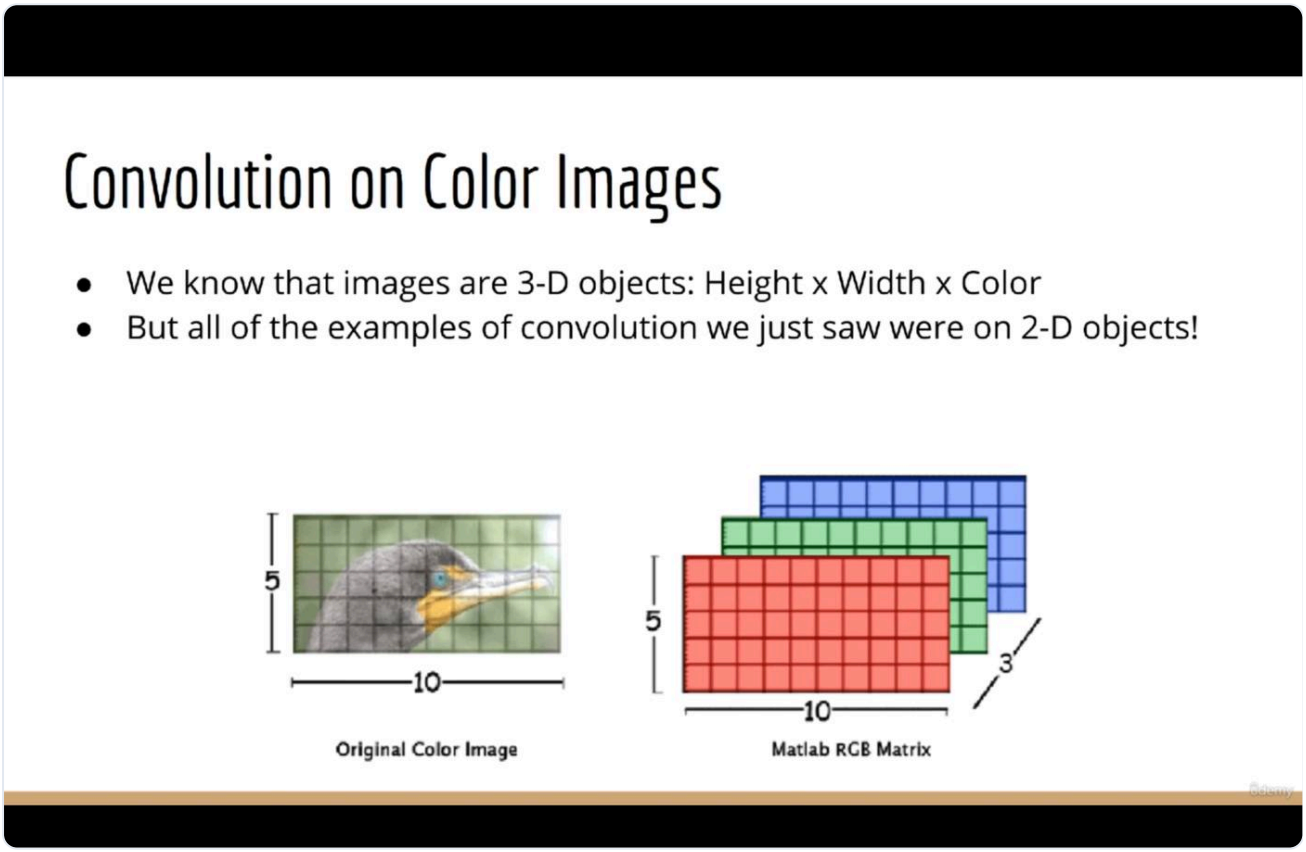


FIGURE 36.1 | Convolution on Color Images

But for all the examples of convolution We just looked at both the image and the filter where two dimensional. So how can we reconcile this discrepancy previously.

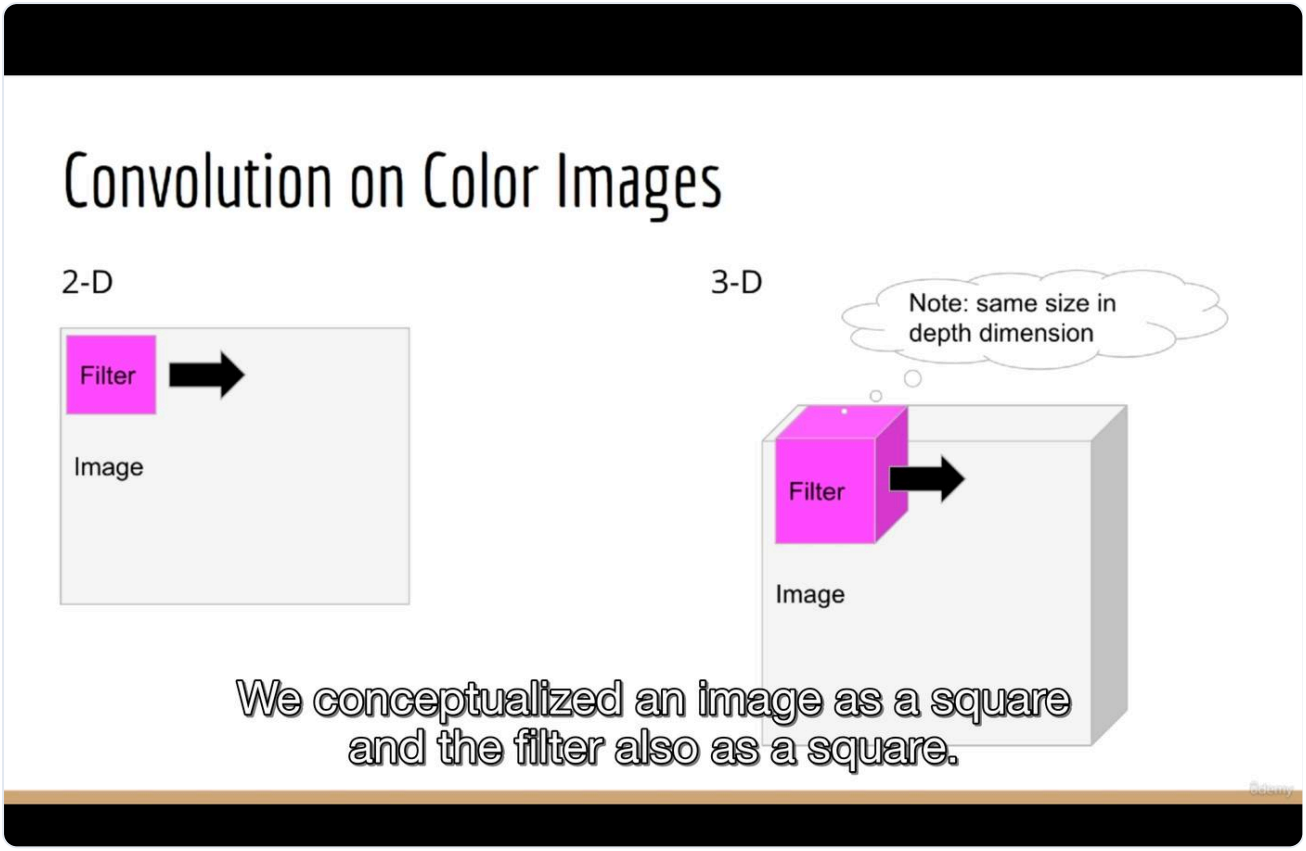


FIGURE 36.2 | Convolution on Color Images

We conceptualized an image as a square and the filter also as a square. Then we would slide this little square along the big square and do multiply add and that's convolution. So if our image has three dimensions we just go with the same analogy. Our color image is a big box and our filter is a little box. Then we slide this little box along the big box and at each position we do a multiply add and that's a convolution here the details at each position we used to sum over the height and width of the kernel because the kernel was 2D but now since the kernel is 3D we're going to sum over the color channel as well.

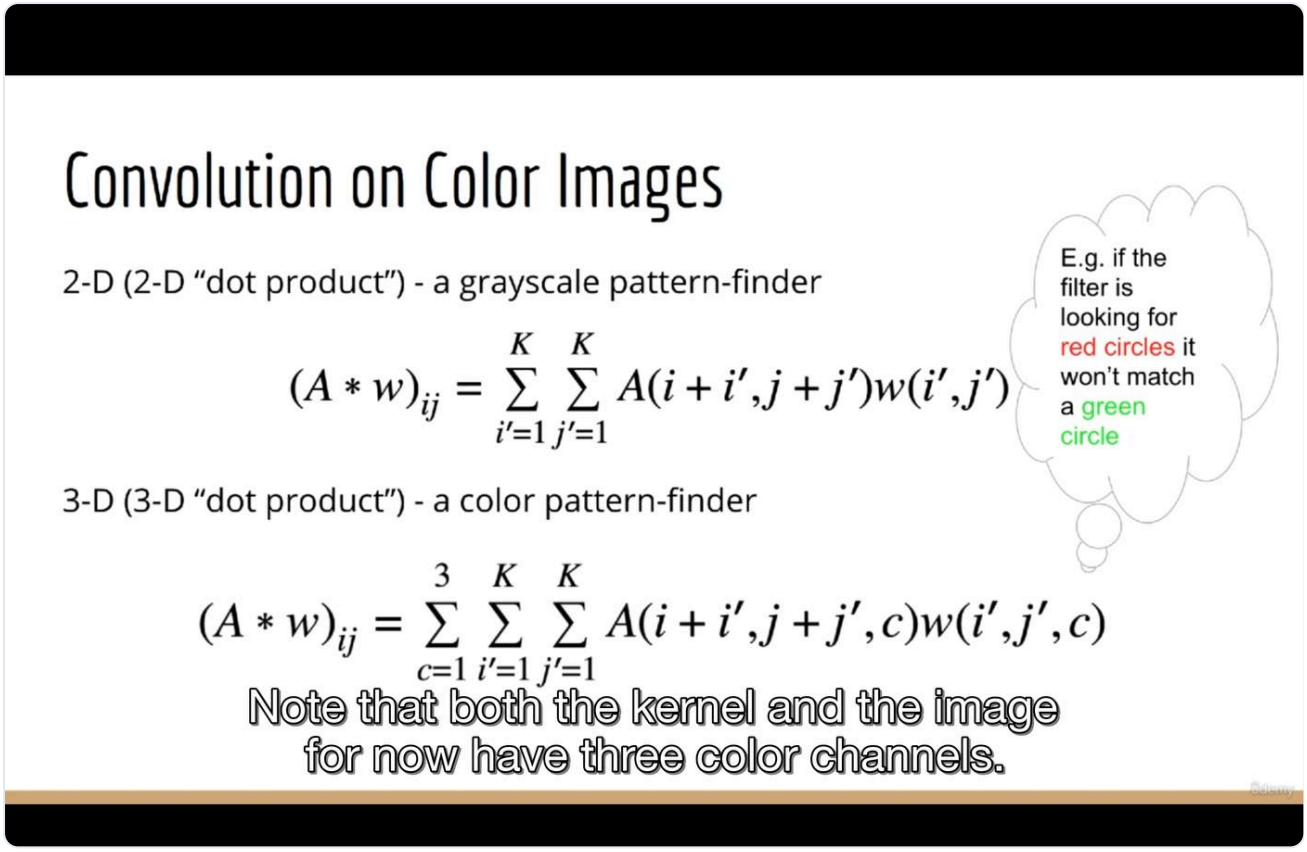


FIGURE 36.3 | Convolution on Color Images

Note that both the kernel and the image for now have three color channels.

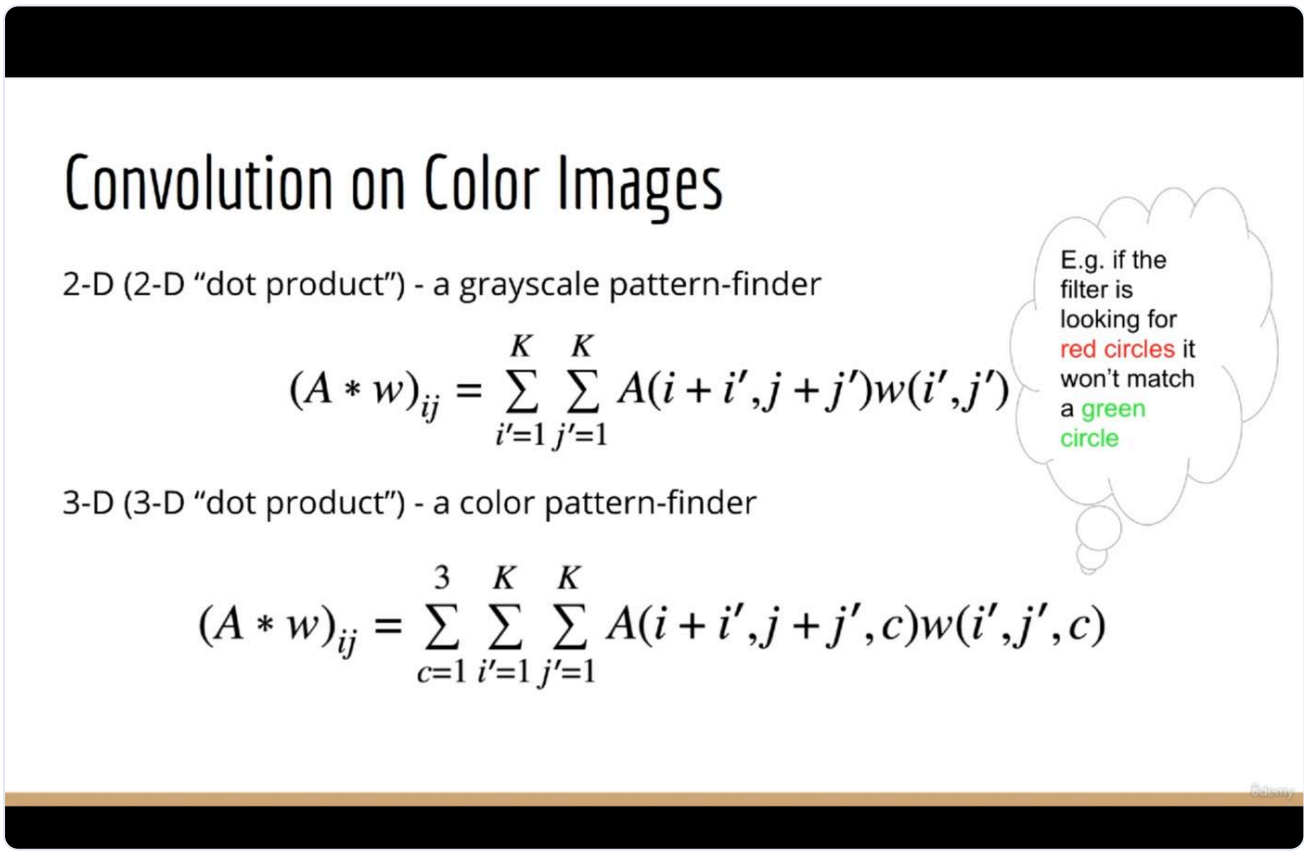


FIGURE 36.4 | Convolution on Color Images

If you recall that's red green blue the way that you can think of this is that instead of the filter being a grayscale pattern finder is now a color pattern finder. Suppose the filter is looking for a red circle so there is a circle in the red channel. Now it's only going to match red circles. It won't match blue circles or green circles. This is opposed to the black and white filter which has no color at all. If it saw a circle then the pattern would be found. There's no concept of color in a black and white image ultimately. This picture is still not complete. Using what we just learned this is what would happen. Our input image is a three dimensional object height by width by three our kernel is also a three dimensional object.

K by K by 3 the output of this convolution is only 2 dimensional height minus K plus 1 with minus K plus 1.

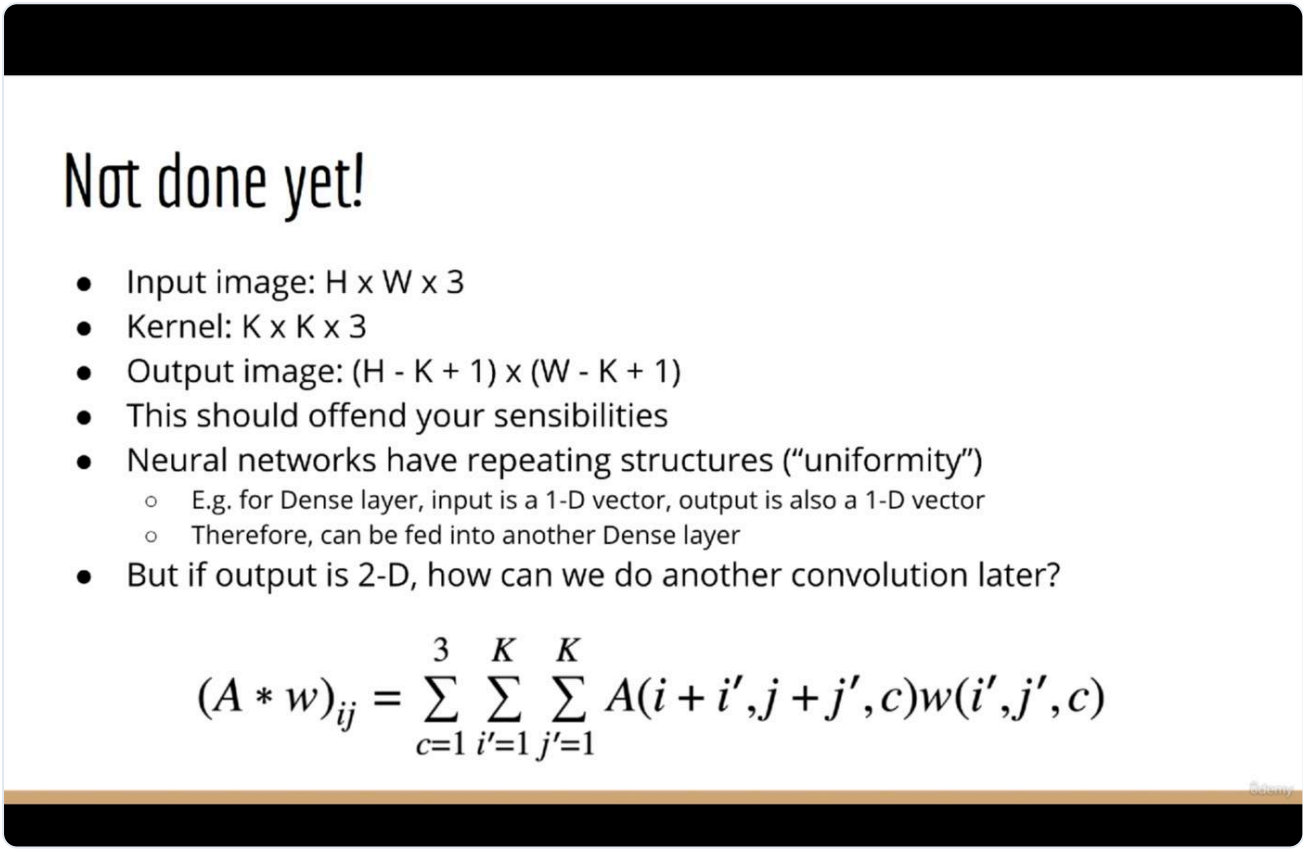


FIGURE 36.5 | Input Image: H X W x3

And this is because we're summing over all three dimensions and so the third dimension kind of gets dotted out now there's nothing necessarily wrong with the output being two dimensional although this should offend your sensibilities to a certain extent. Remember this idea of the uniformity of a neural network and how we have repeating structures the feedforward neural networks because you can add any number of layers one dense layer followed by another followed by another. This is okay because the input and output are of the same type. The input is a vector and the output is also a vector which can then be the input into the next layer. But for convolution so far this doesn't seem to work.

If the input is three dimensional but the output is only two dimensional then we do not have this uniformity. We can't plug in the same kind of convolution right after the first one since the input is not the same as the output but here's the question to consider. Remember that the filter can be

thought of as a pattern finder so it's looking for a particular pattern or in other words looking for a particular feature. But we know that in machine learning we have multiple features just like how each hidden unit in a neural network represents a different feature based on the inputs that kind of makes sense. If we're looking at an image we would want to identify multiple features in the image.

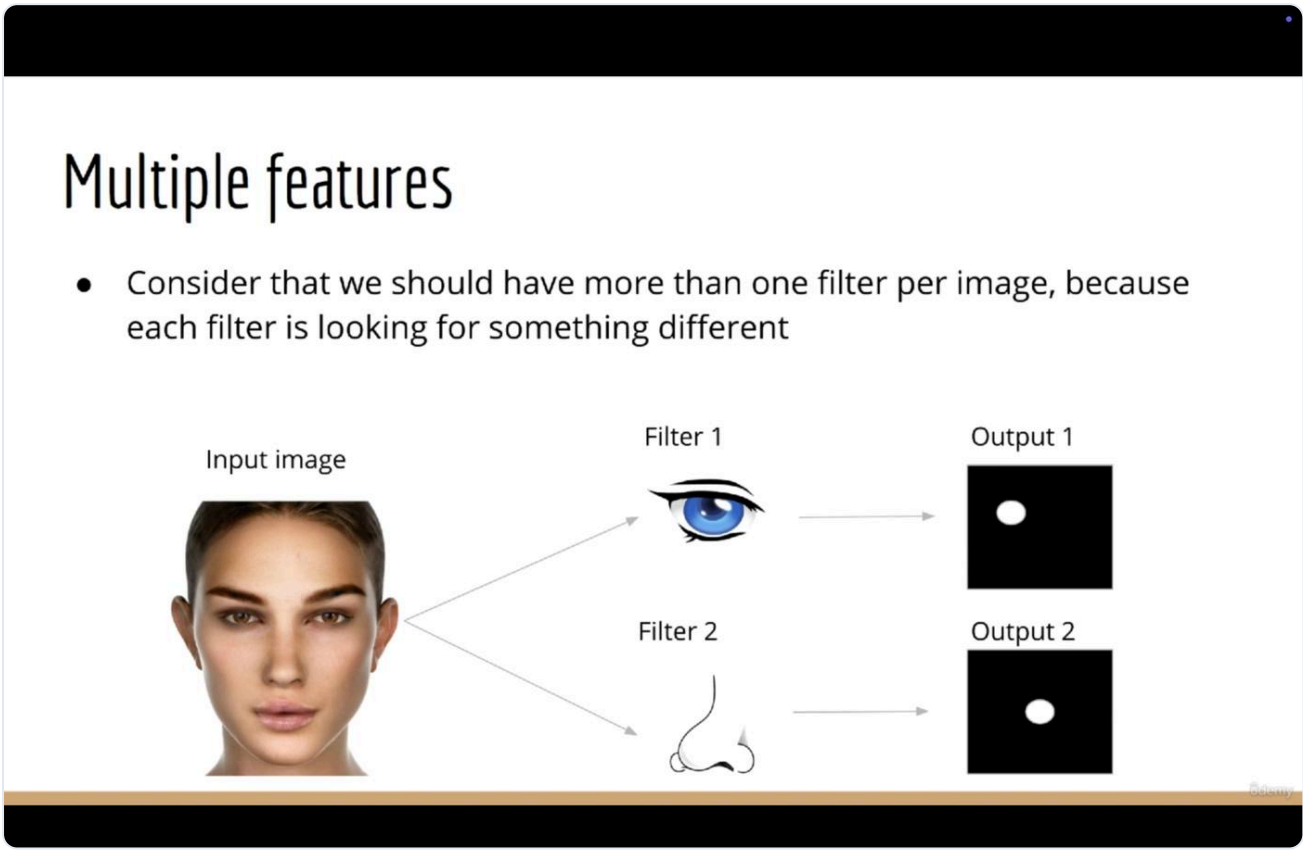


FIGURE 36.6 | Multiple Features

For example let's say you're building a facial recognition network when a filter might be looking for an eye and another filter might be looking for a nose. So it makes sense then that if we want to find multiple features we should have multiple filters to find them. So consider for a second what would happen if you had two filters acting on the same input image called the input image a call the first filter w one called the second filter W2 the output of A convolved with W one is B1 and the output of A convolved with W2 is B2. Let's say for simplicity's sake that we're using same mode convolution so that B1 and B2 have the same height and width as a.

So if A is H by W by 3 then B one is h by W and B two is also h by W. Now we have two outputs B one and B two but with different features found in each. What should we do with them. Well how about we simply stack them together. Now all of a sudden in our output is three dimensional. That's height by width by 2.

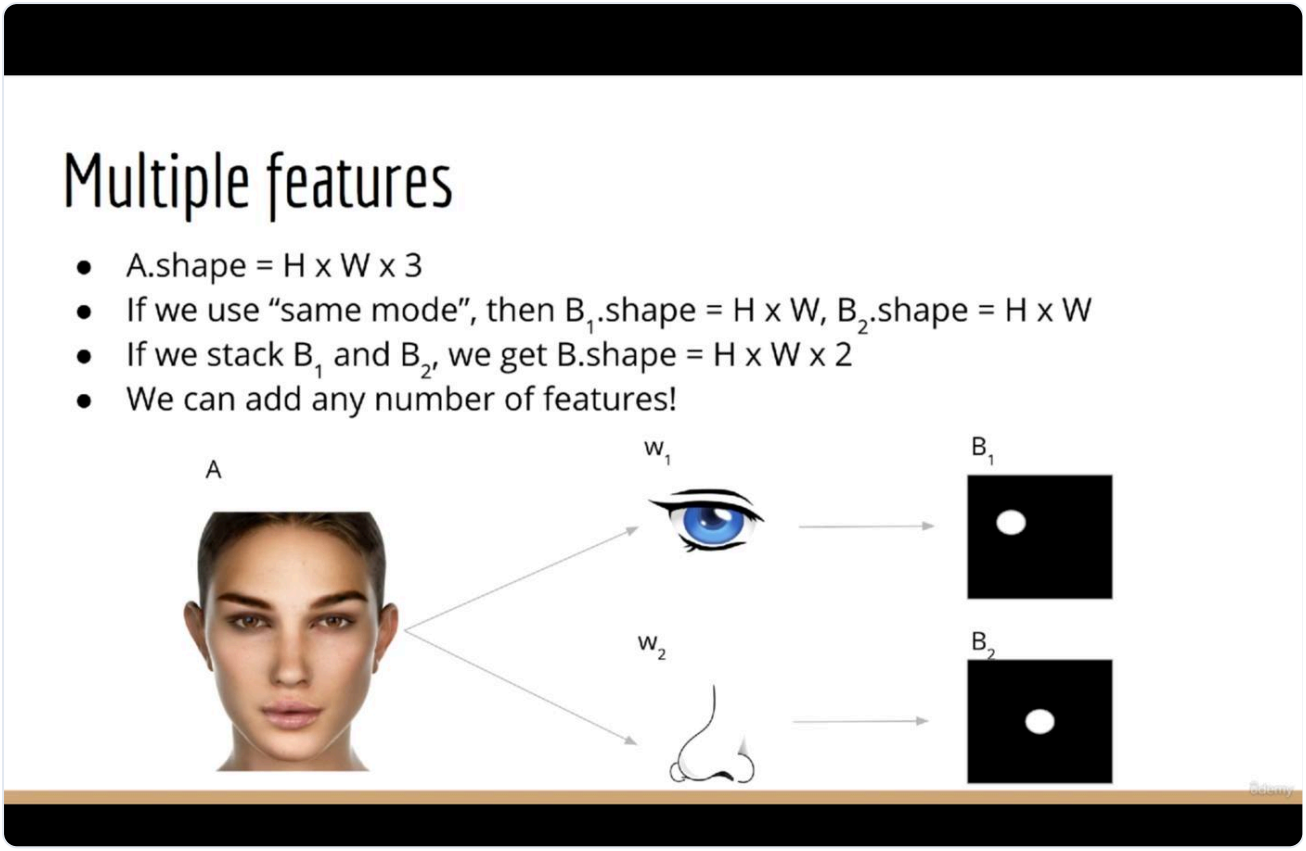


FIGURE 36.7 | Multiple Features

And in fact we can add any number of filters to find any number of features. So nose eyes lips eyebrows chin so on now our input looks like our output.

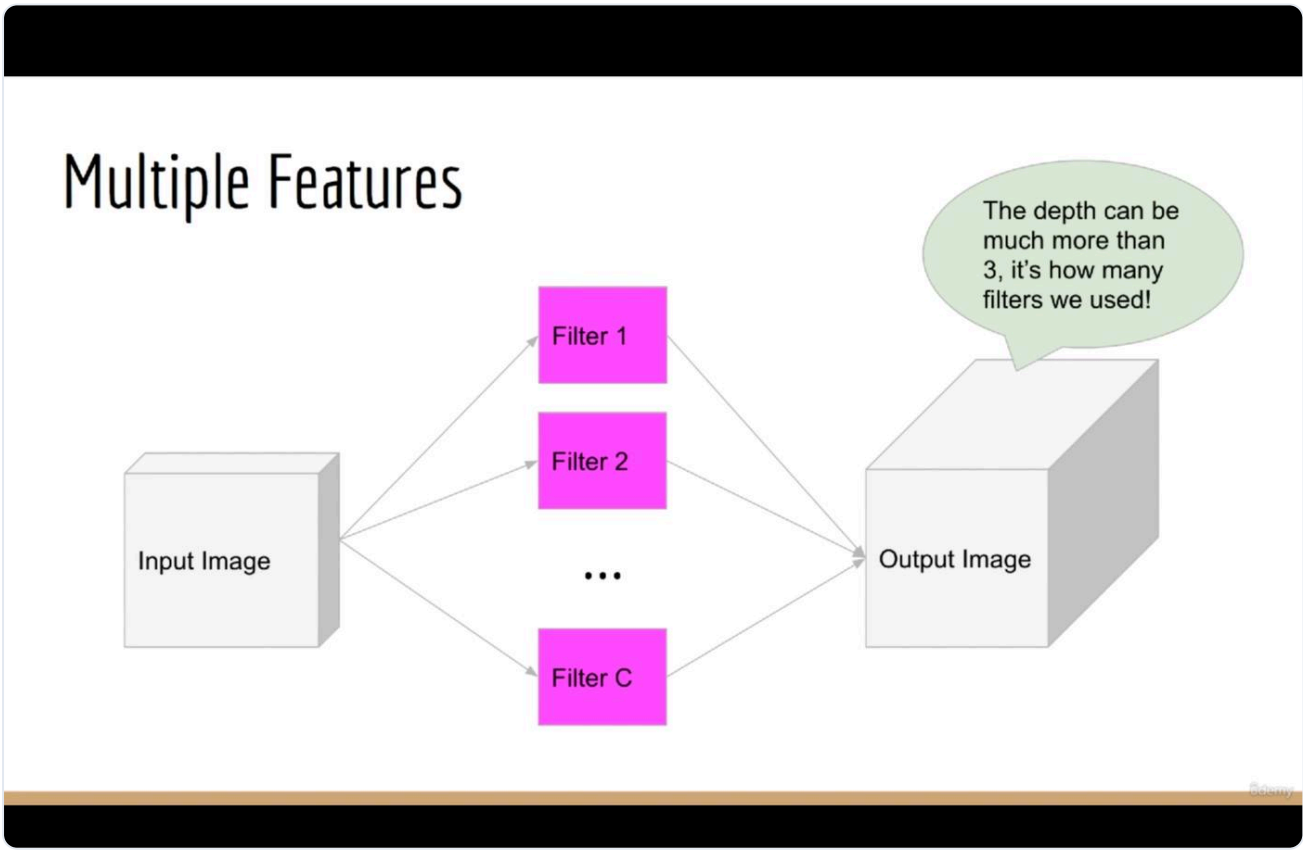


FIGURE 36.8 | Multiple Features

They are both three dimensional. This means we can have multiple layers of convolution one followed by the other in a repeating pattern just like we do with ANNs. But of course just like with the ANN and dense layers.

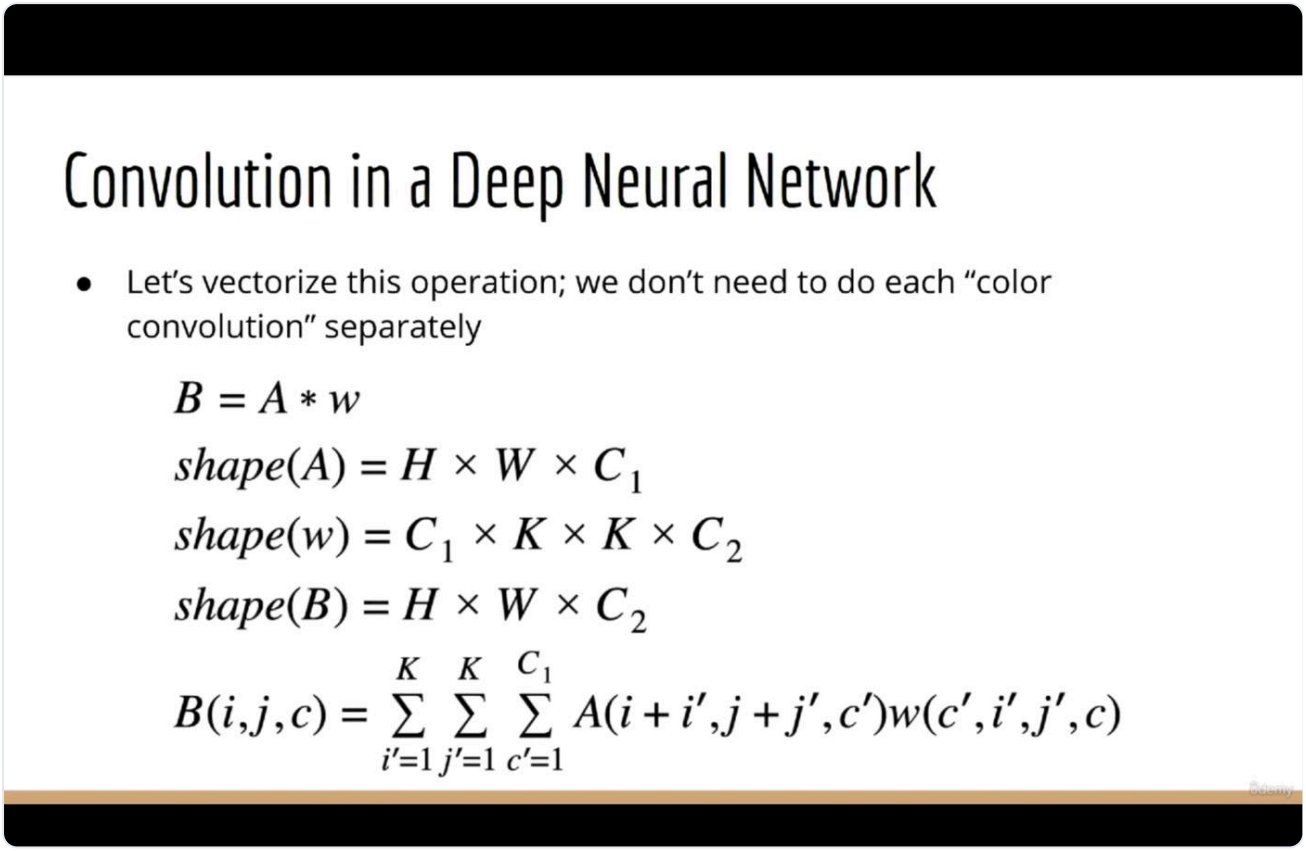


FIGURE 36.9 | Convolution in a Deep Neural Network

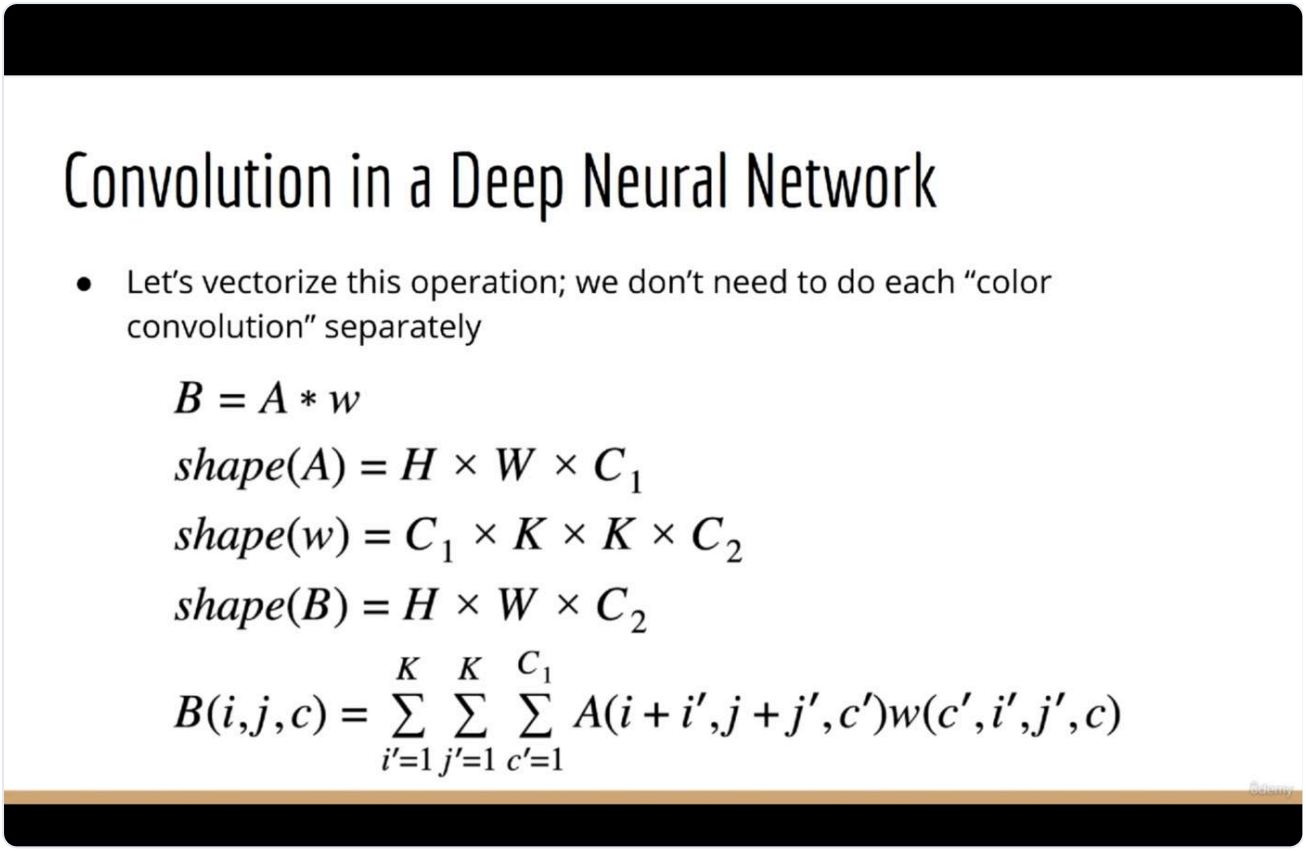


FIGURE 36.10 | Convolution in a Deep Neural Network

We want to vectorize this operation so that it can be done all in one go rather than doing say 10 different convolutions and then stacking them together so here is the formula that represents how to do a complete convolution in a deep neural network so that both the input and output can be three dimensional objects. Now this is a lot to look at all at once. So let's try to consider each part one step at a time. First let's consider the shape of everything. We know that a will be h by W by C 1. You can think of C 1 as the number of color channels. The image has then we have a filter w which has the shape c 1 by K by K by C to note that this is now a four dimensional tensor then we have the output image.

B assuming that we are doing same mode convolution the height and with dimensions will be the same h by W but the third dimension will be c 2. So when our equation for the convolution We can see that it's pretty much the same as before just with more indices. B I J C represents the image pixel value at a row ie column J and colour channel c. Note that I use the term colour loosely here because it doesn't really represent colour anymore we can have an arbitrary number of features so C can be 10 100 or 1000. Conceptually however we're still just doing what we talked about previously just individual convolutions which output two dimensional images and then stacking all those images together to get a three dimensional object so let's just summarise what we learned in this lectures since that was a bit long.

First we started with realizing that we only defined convolution for grayscale images and with corresponding two dimensional filters this is of course not flexible enough for CNNs which must accept color images as input. So then we said okay let's just extend that by doing a three dimensional dot product between a color image and a three dimensional filter with the same number of

color channels. So we're dotting over the color channel next. We realize that this breaks uniformity so we can't stack layers together to build a deep neural network. And this is deep learning after all. So we kind of need to be able to do that.

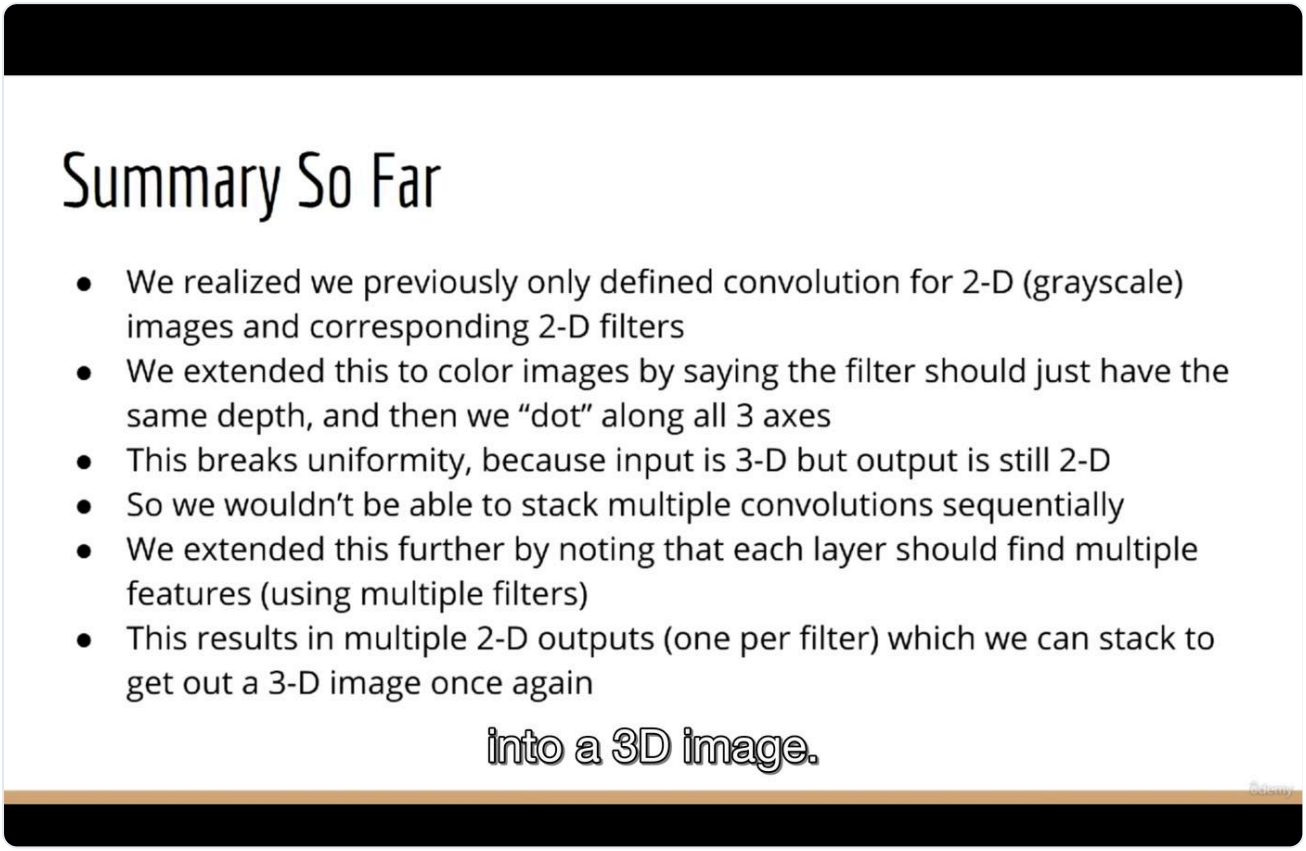


FIGURE 36.11

And this is because when you convolve a 3-D image with a 3-D filter you still get a 2D image as output. This is just like when you dot a one-D vector by another one-D vector you get a scalar. So we extended this by realizing that we don't just want one filter that's going to find a one single feature we need to find multiple features. Therefore we need multiple filters which will output multiple 2D images which of course can be stacked into a 3D image. Then we vectorize this operation so that both the input and output a three dimensional. Now you might be wondering is it correct to call the final dimension color that may be the case for the input into the neural network which is always height by width by three.

But this breaks down at every other layer. For example if you have height by width by 10 then the 10 dimensions don't really represent color. But as we discussed conceptually we're just finding different features in an image. So our terminology changes to cover this more general case we call these feature maps. This makes sense because you can think of each 2D image as a map as in a literal map you can look at which is the result of a convolution with some filter which is looking for some feature so this map is telling us where in the original image that this feature can be found. So just like a map the final output is just a stack of all these different maps one map for each feature.

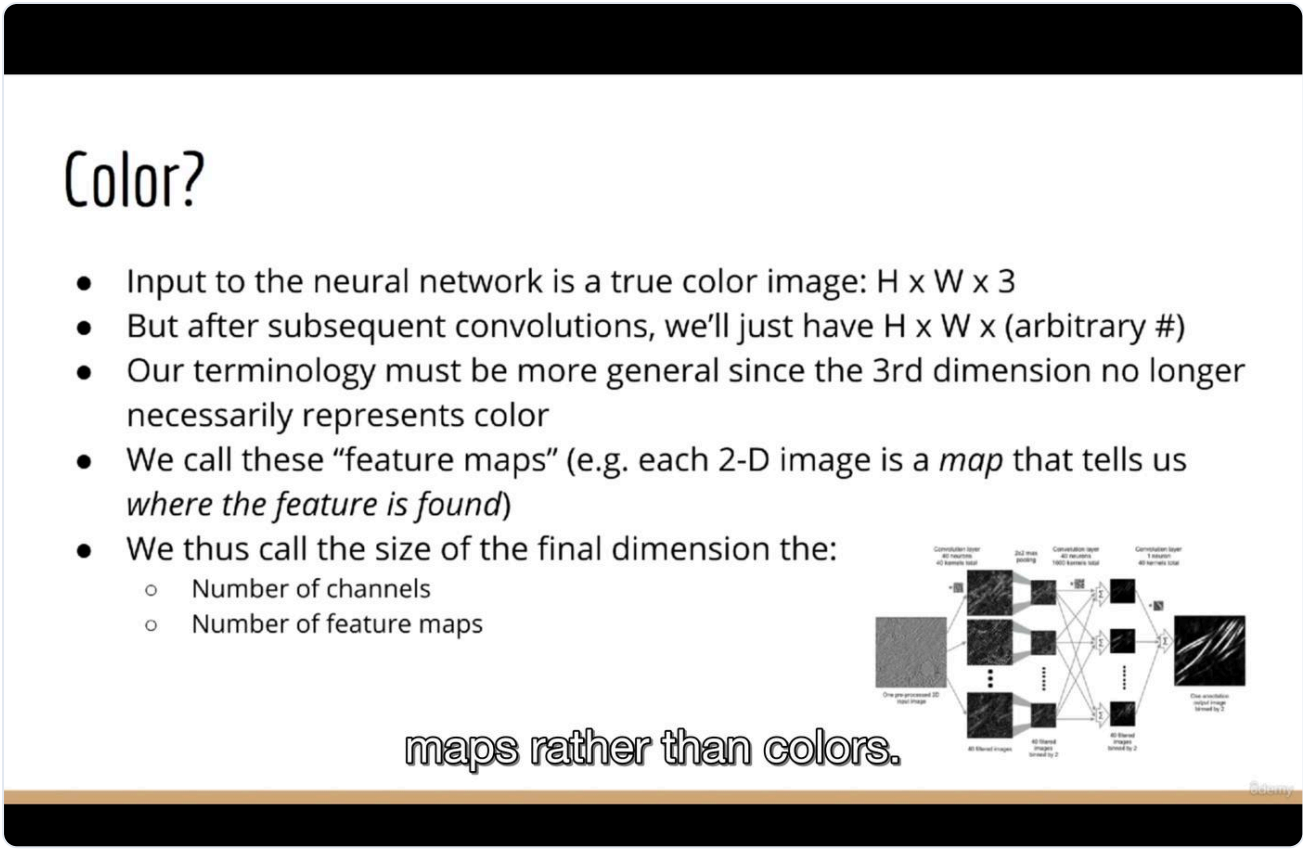


FIGURE 36.12

And so the size of the final dimension is referred to as the number of channels or the number of feature maps rather than colors. Finally I want to talk about how convolution is going to look as a neural network layer if you recall. You can think of convolution as just matrix multiplication but with shared weights and as you know in a feedforward ANN we do not just have matrix multiplication at each layer we do some other things as well.

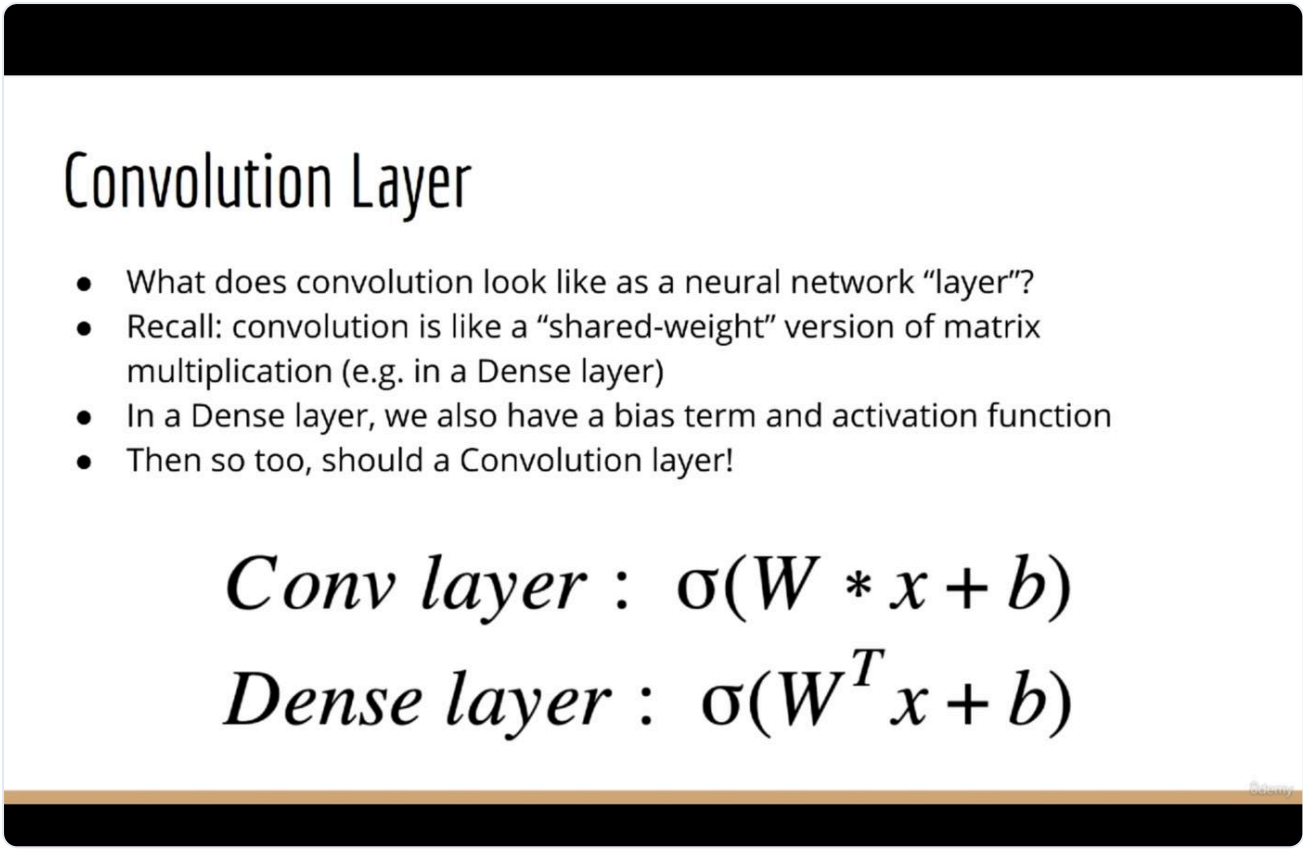


FIGURE 36.13 | Convolution Layer

In particular we can add a bias term and apply an activation function well a convolution layer works the same way we're going to add a bias term and apply an activation function because remember this allows us to find nonlinear features or nonlinear functions of the input which are more powerful than simple linear functions importantly we have to consider what the shape of the bias term should be given that the output of the convolution is a three dimensional image in a dense layer. If we have a vector of size m then our bias term is also a vector of size m.

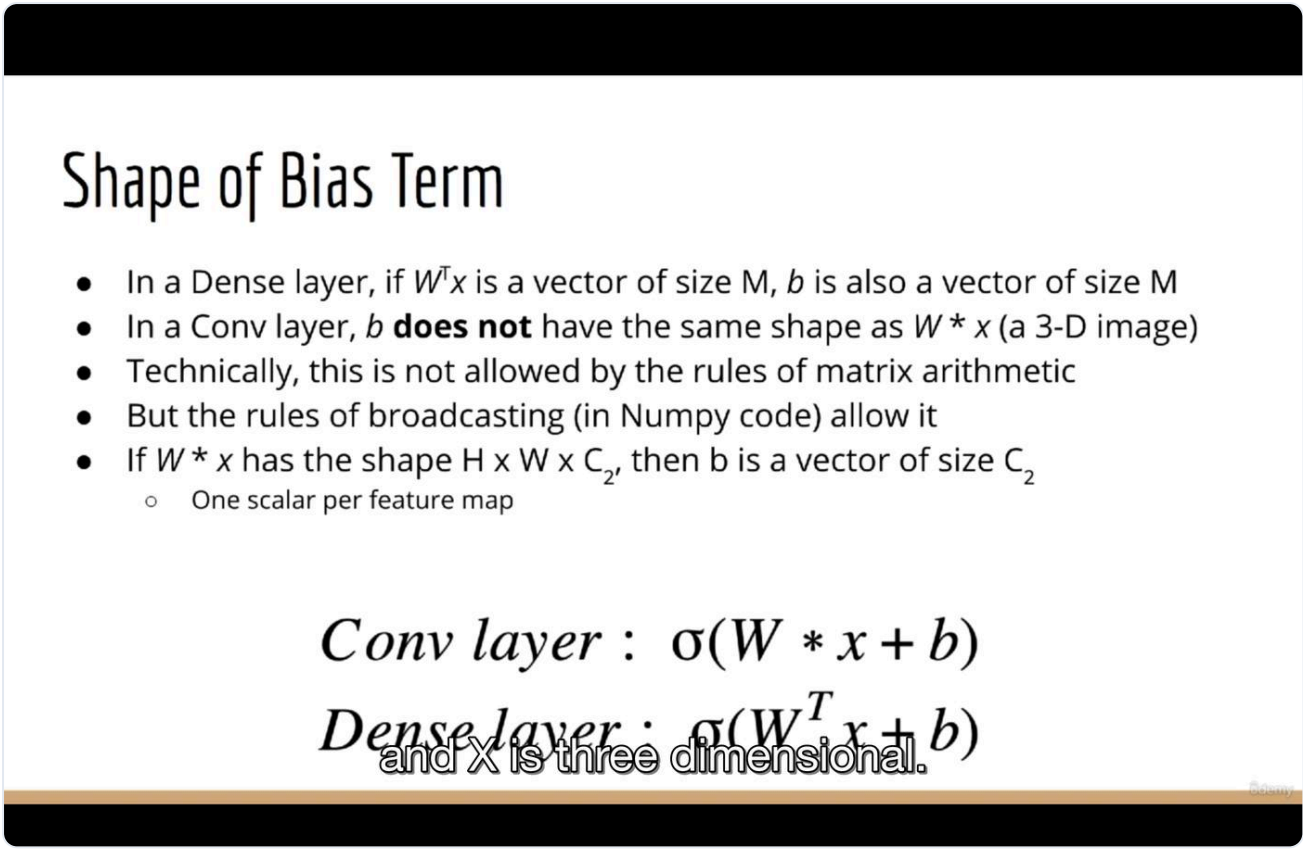


FIGURE 36.14 | Shape of Bias Term

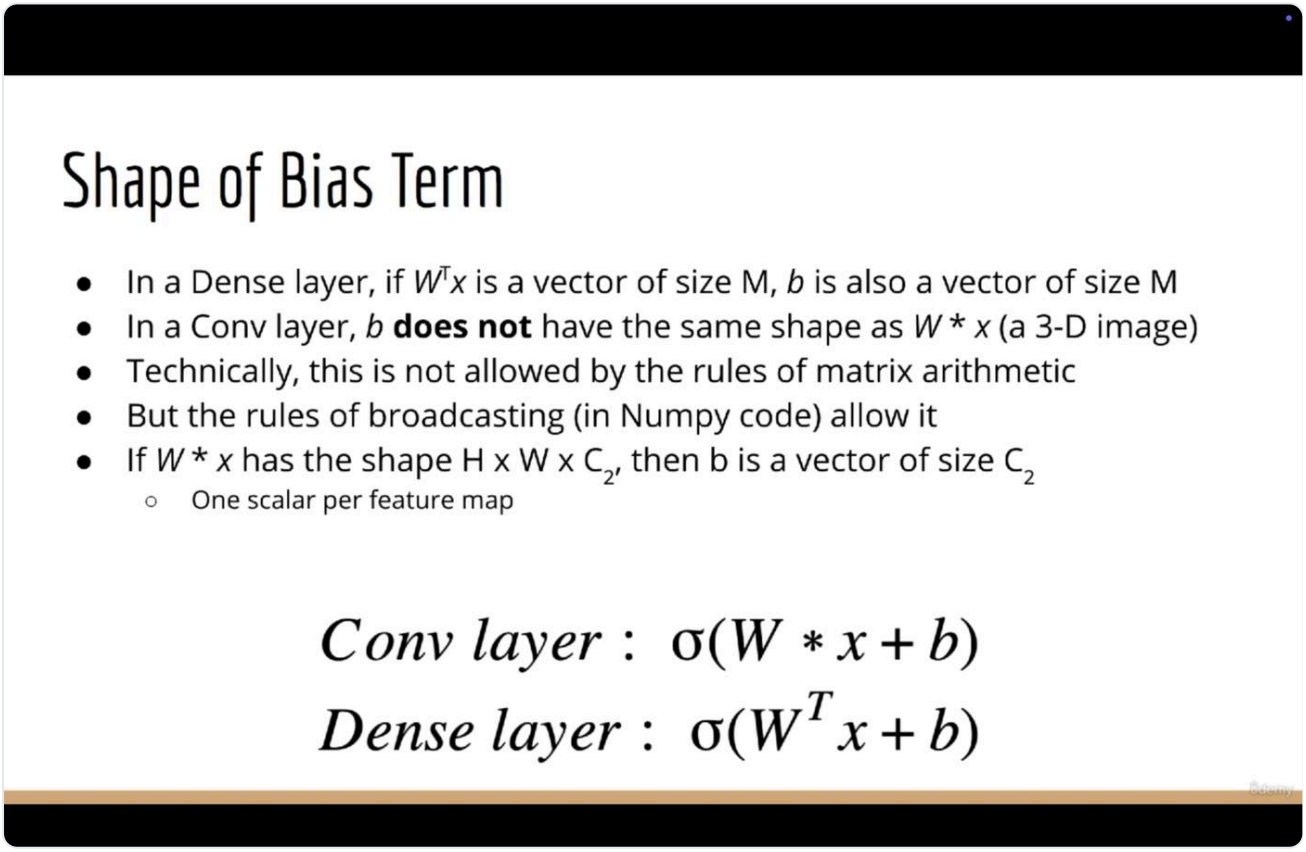


FIGURE 36.15 | Shape of Bias Term

In other words there is one scalar for each element in the result but for convolution the bias term will not have the same shape as W convolved with X which is a three dimensional image now you might think isn't this equation invalid if B does not have the same shape as W convolved with X. Technically this shouldn't be allowed by the rules of Matrix arithmetic. If we want to add two matrices together they should have the same shape. Technically this is true but this is conceptual only and this will actually work in code. Due to the rules of broadcasting so the bias vector B is only one dimensional and it has the size C too if W convolved with X has a shape H by W by C2.

In other words the same bias term applies to every pixel for each feature map. As a side note convolution is generally a commutative operation so it doesn't really matter if you write w star x or x star w although this is not entirely true in the Deep Learning case since W is four dimensional and X is three dimensional. Nonetheless this is just conceptual only. So let's pretend that w comes first so that it looks more like the dense layer equation all right. So now that you understand how convolution and a full convolution layer works in a neural network let us consider our savings. Remember that one way to look at convolution is that its parameter sharing sharing parameters means that we use the same weights in multiple places.

Reducing the number of total weights we need. So let's do a quick calculation to determine our savings. Suppose our input image is of size 32 by 32 by 3. A modest size image. Now let's suppose the size of the filter is three by five by five by sixty four. So there are sixty four output feature maps then our output image will have the shape twenty eight by twenty eight by sixty four. This is because 32 minus five plus one is twenty eight.

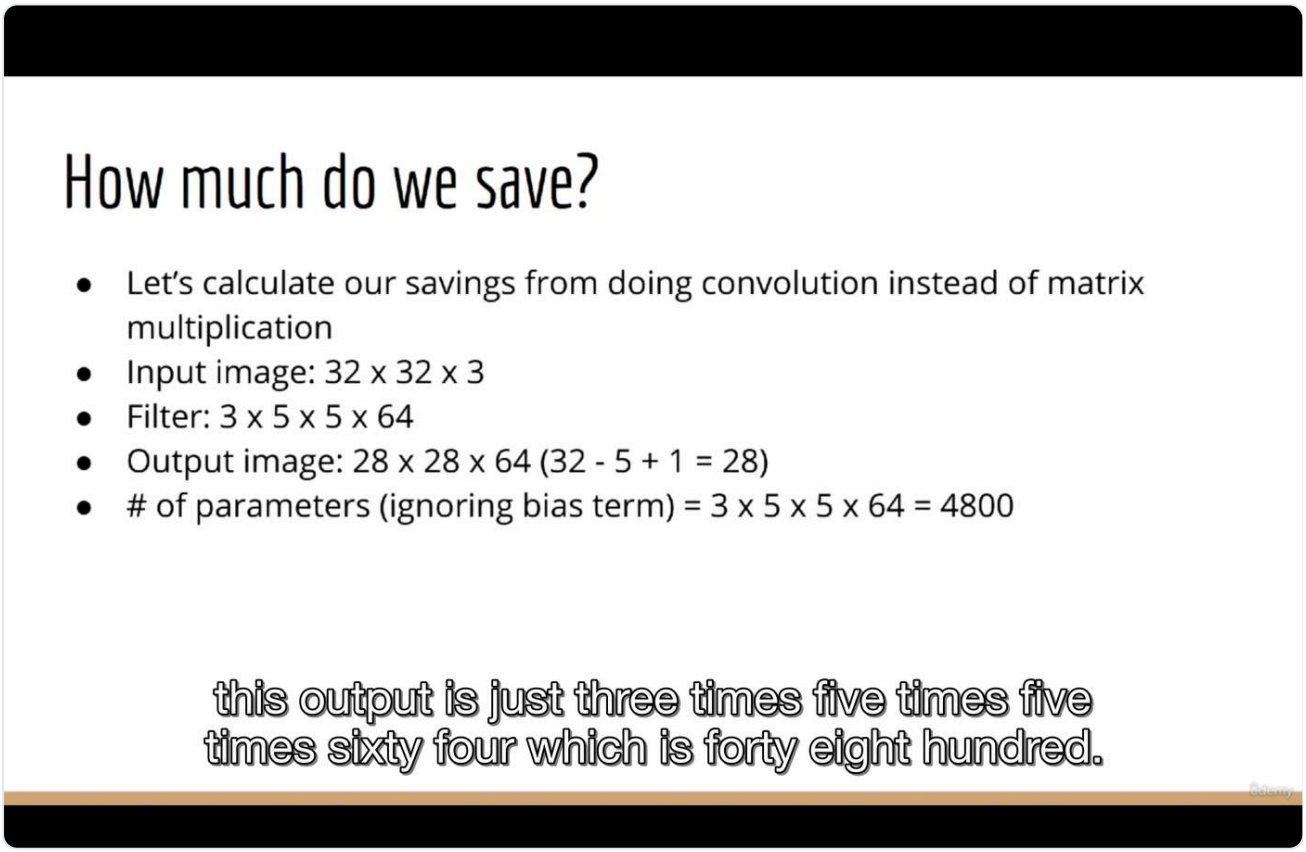


FIGURE 36.16 | How Much Do We Save?

And let's just assume we're using valid mode convolution the number of parameters we needed to compute this output is just three times five times five times sixty four which is forty eight hundred. Now let's compare that to the case where we would use a feedforward neural network with a flattened image like we did in the previous section then our input would be 32 times 32 times three which is three thousand seventy two our output again would be a size twenty eight by twenty eight by sixty four which is fifty thousand one hundred seventy six. So if we use the dense layer we would have a weight matrix of size three thousand seventy two times fifty thousand one hundred seventy six which is approximately one hundred fifty four million.

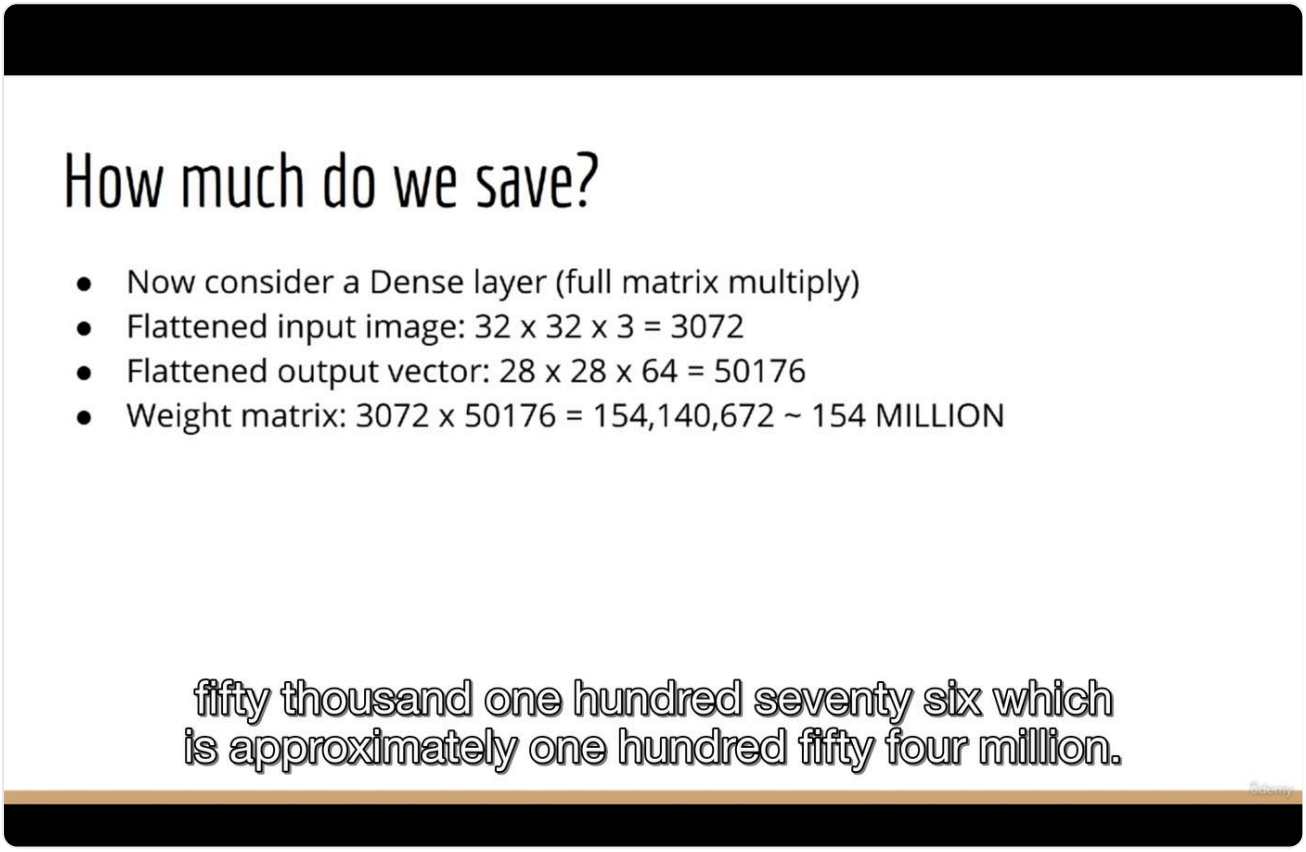


FIGURE 36.17 | How Much Do We Save?

Now that's a lot of parameters for just one layer of a neural network. In fact compared to using convolution We have about thirty two thousand times more parameters as you can imagine. This would require a huge amount of RAM and a huge amount of compute it would take longer and would perform suboptimally because as we stated earlier convolution is a pattern finder.

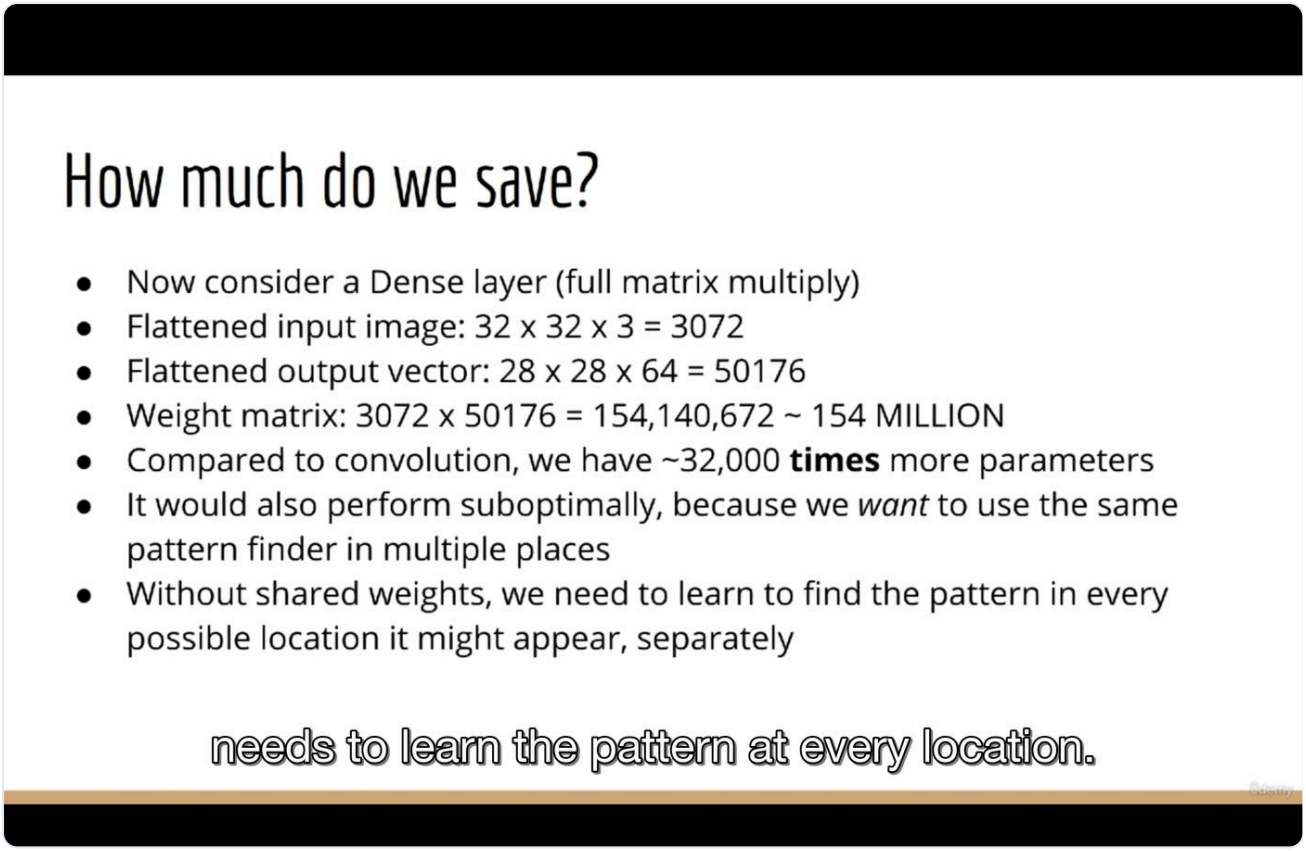


FIGURE 36.18 | How Much Do We Save?

We use the same filter to find the same pattern in multiple places without shared weights every weight needs to learn the pattern at every location. The pattern could appear which is suboptimal.

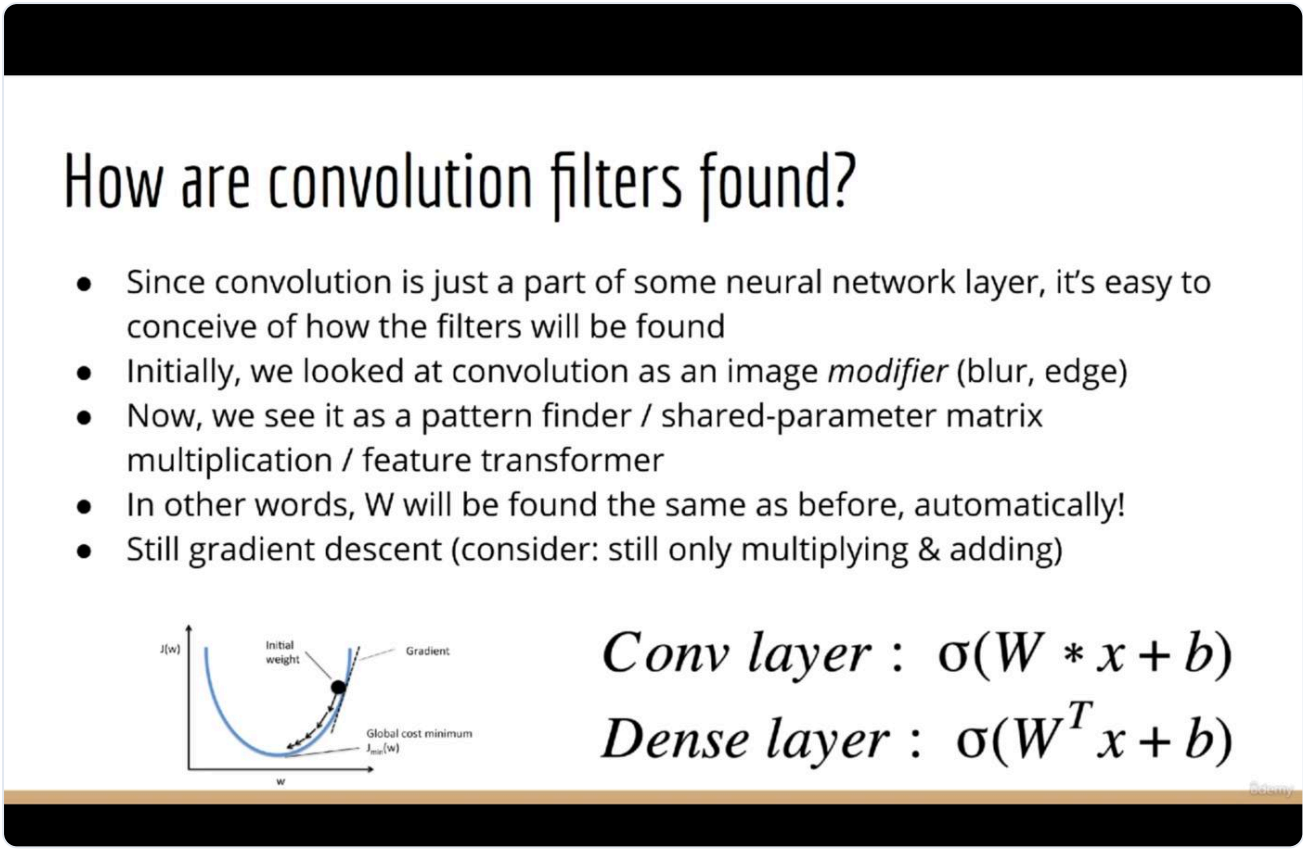


FIGURE 36.19 | How Are Convolution Filters Found?

This would be bad for a generalization using this concept of convolution just being a part of a neural network layer. It's easy to see how we are going to find these convolution filters through the process of training at this point. We've come quite a long way since our first introduction to convolution where we had the perspective that convolution is just an image modifier. The examples we use were blurring and edge detection. We took an image and then convolution modified it in some way but now we have a deeper understanding of convolution. It's in fact a pattern finder and a shared parameter matrix multiplication. It's just a feature transformer.

In this way it's just like the neural network layers we'd seen earlier in the course and so having this perspective it becomes pretty obvious how these filters or feature Transformers will be found during the training process. The answer is that it's all going to happen automatically when we do gradient descent. This will update both the filter and the bias term as usual and they just happen to be a different kind of operation than what we had in the linear layer. But conceptually you know that whether you have a linear layer or a convolution layer both of these are just multiplying and adding so there should be no problems applying gradient descent as usual.

LECTURE 37

Different Pool Sizes

Now that you understand how exactly a convolution layer works including the bias term and activation function we can now consider the architecture of a convolutional neural network and why it's that way. So as a little bit of a history lesson modern CNNs essentially all originated from the same model.

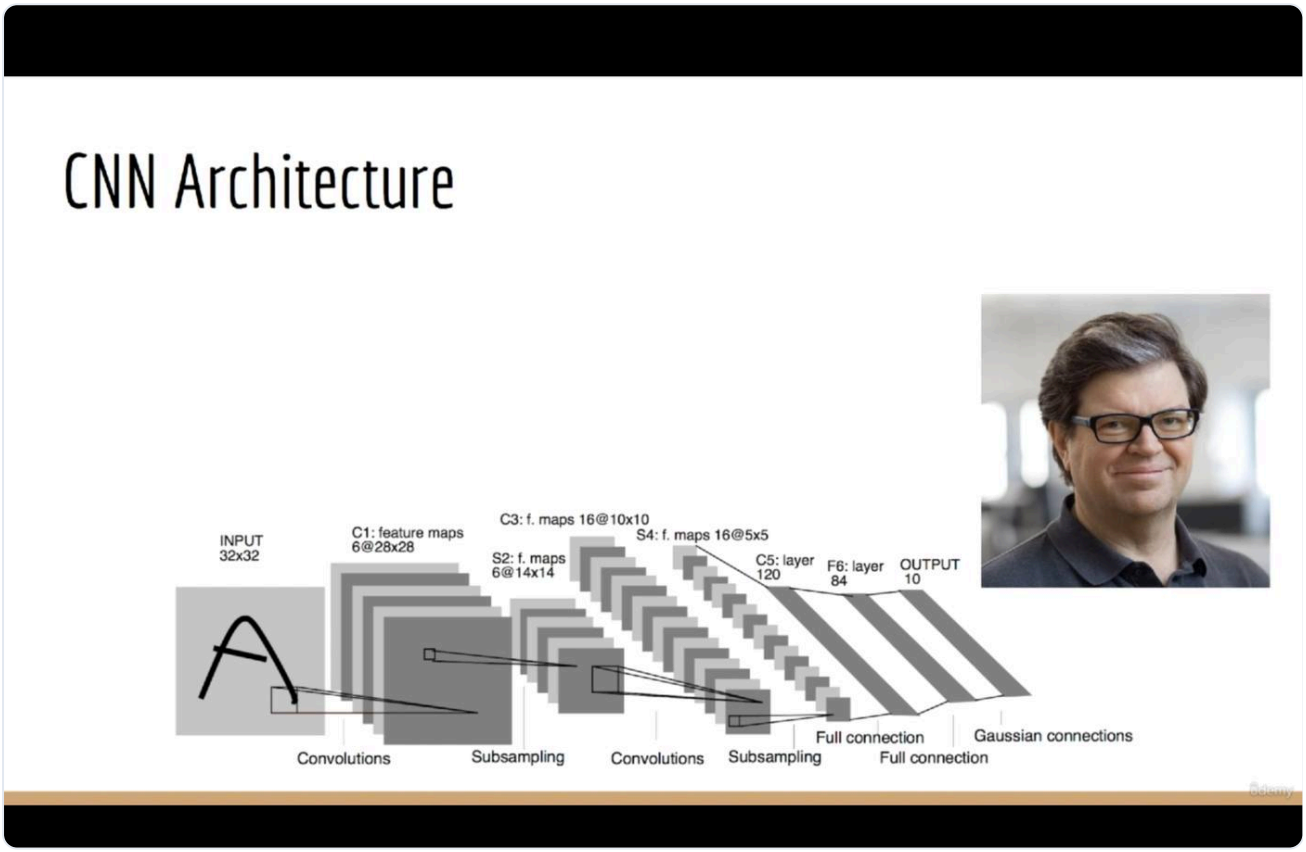


FIGURE 37.1

LeNet.

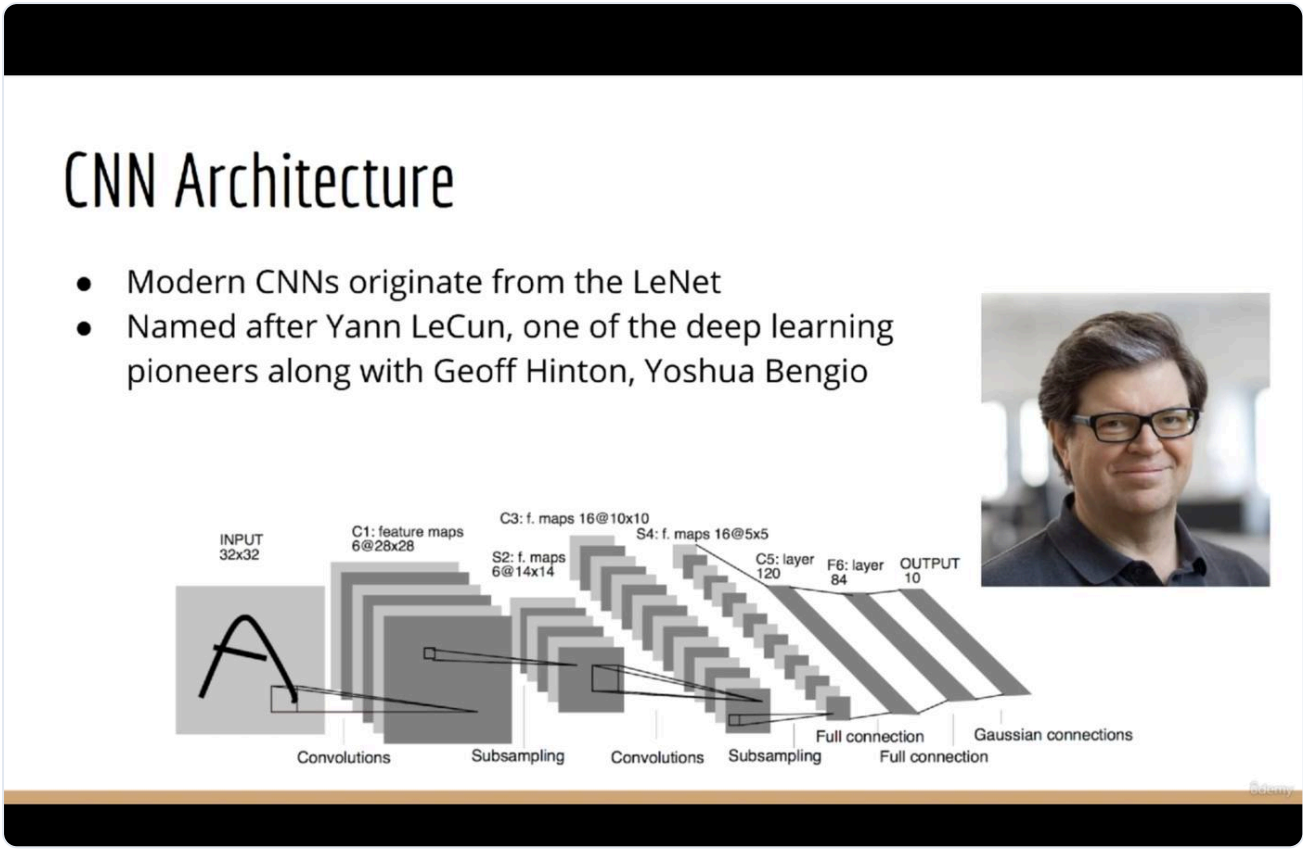


FIGURE 37.2 | Named After Yann LeCun, One of the Deep Learning

This is named after Yann LeCun one of the original Deep Learning pioneers along with Geoffrey Hinton and Yoshua Bengio. What we're going to do in this lecture is just start with what the architecture is and then go through it piece by piece so that you understand why it is the way it is. Okay so let's look at a typical CNN. A typical CNN has two stages. The first stage is a series of convolutional layers.

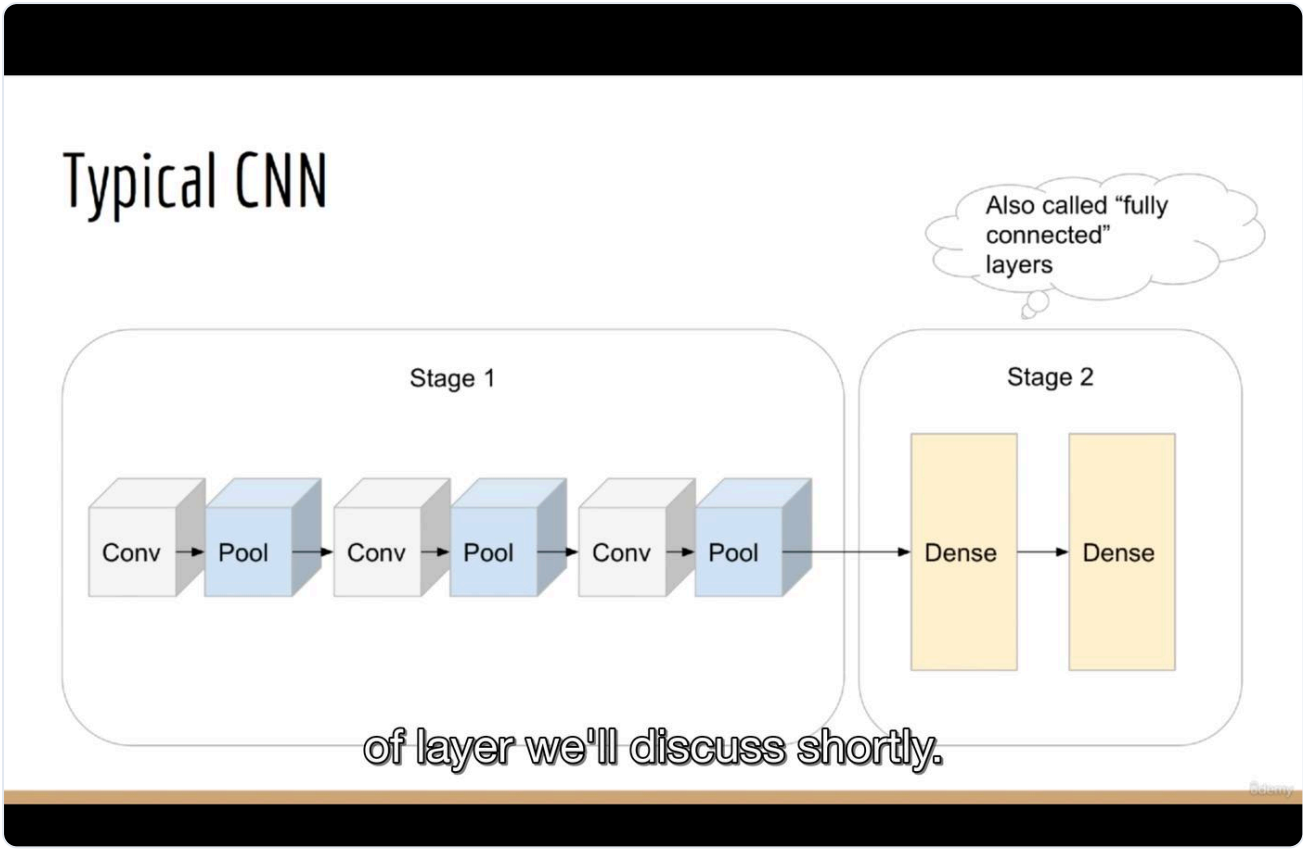


FIGURE 37.3

Importantly these convolutional layers are usually followed by pooling layers which is another type of layer we'll discuss shortly. The second stage is a series of dense layers also called a fully connected layers. These are just a regular feedforward neural network. As you saw in the previous section.

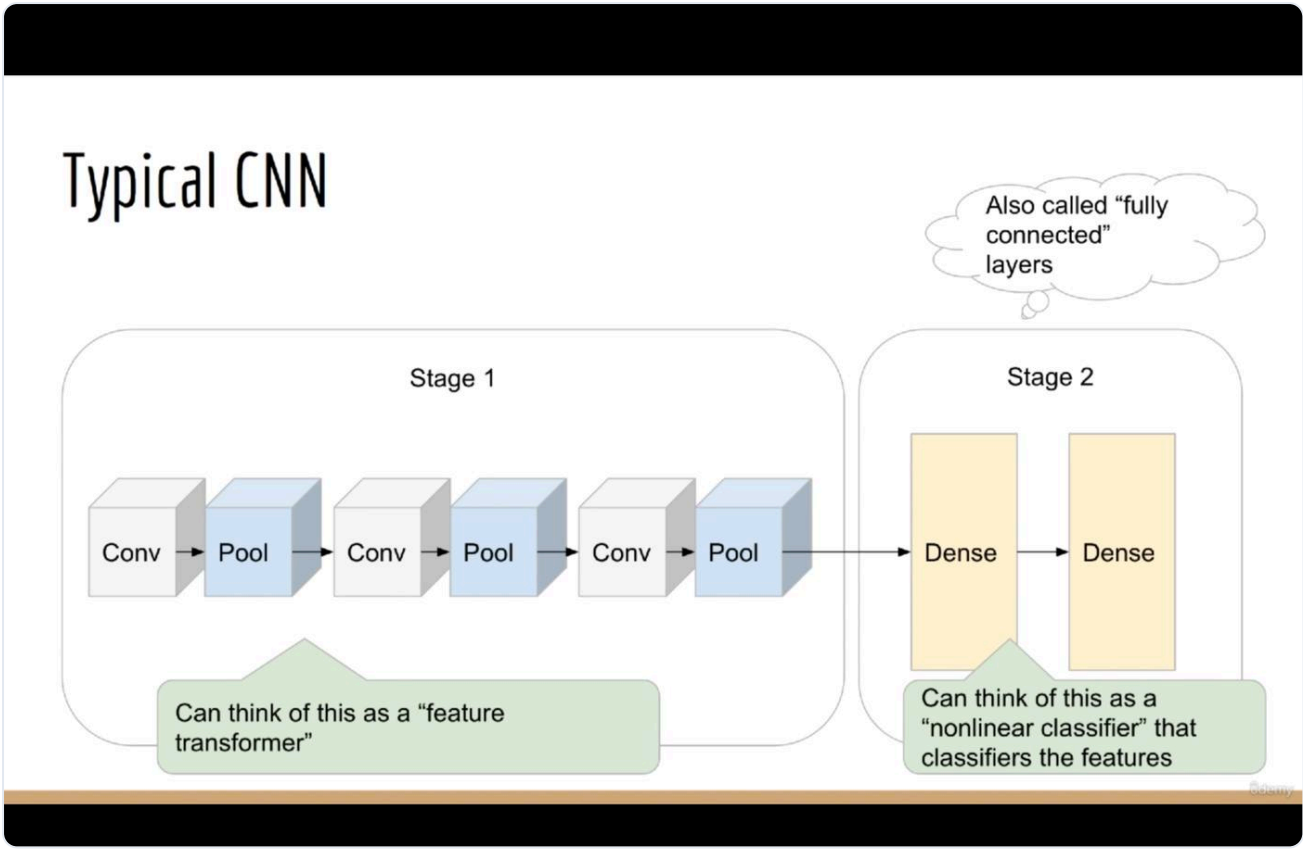


FIGURE 37.4

If you recall I said in the previous section that neural networks have this hierarchical structure. Each layer is always looking at features that were found in the previous layer so you can even think of the first stage as just another feature transformer. It's a feature transformer that specifically works on images and finds image features.

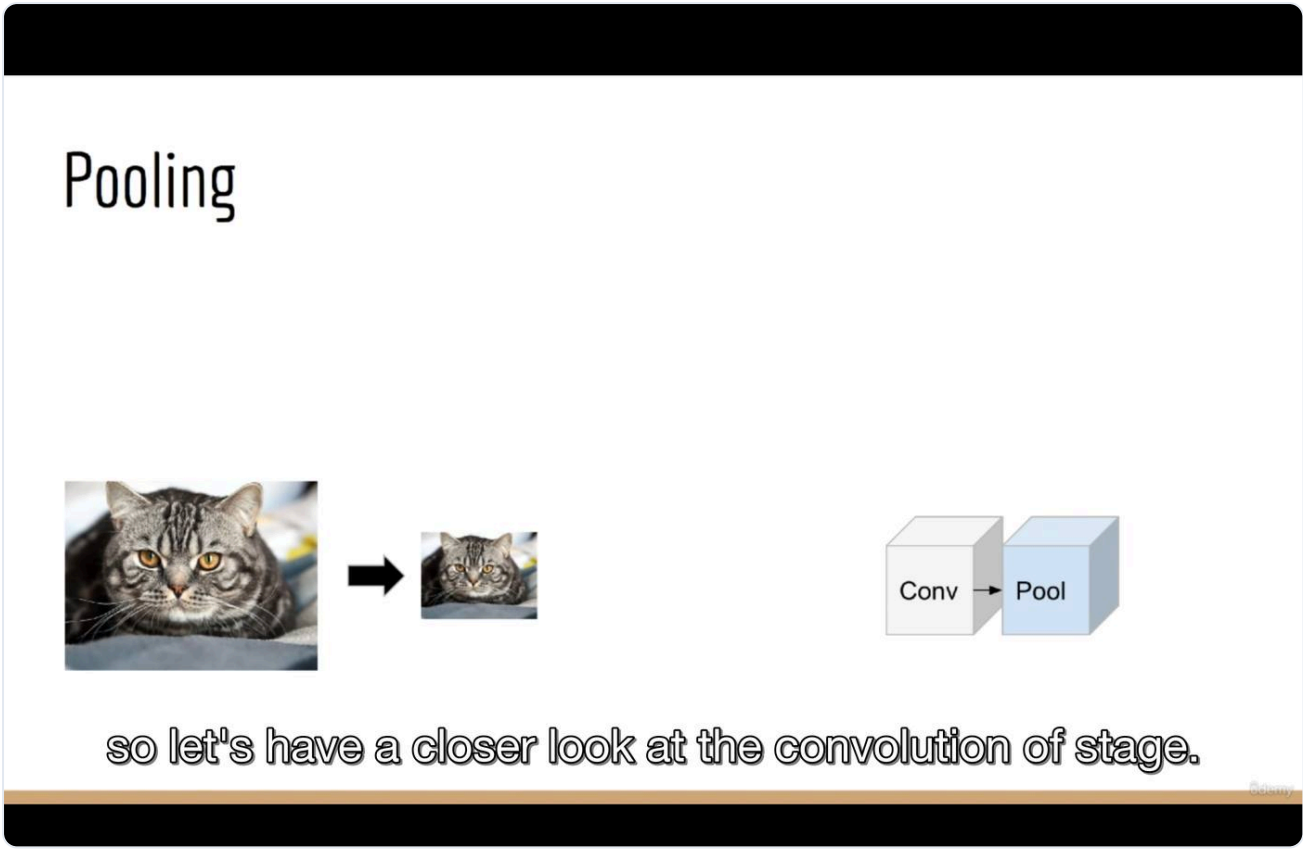


FIGURE 37.5

Then once it's found those image features it passes those into a neural network and then the neural network does what it does best nonlinear classification or regression so let's have a closer look at the convolutional stage. Now I just added a new layer without really explaining it. The pooling layer. Why would we want to do that at a high level. Pooling is a down sampling operation what is down sampling. That's when you make a smaller image out of a bigger image. So imagine your image is one hundred by one hundred. After pooling. If the pool size is two then you would get a 50 by 50. That's down sampling by 2.

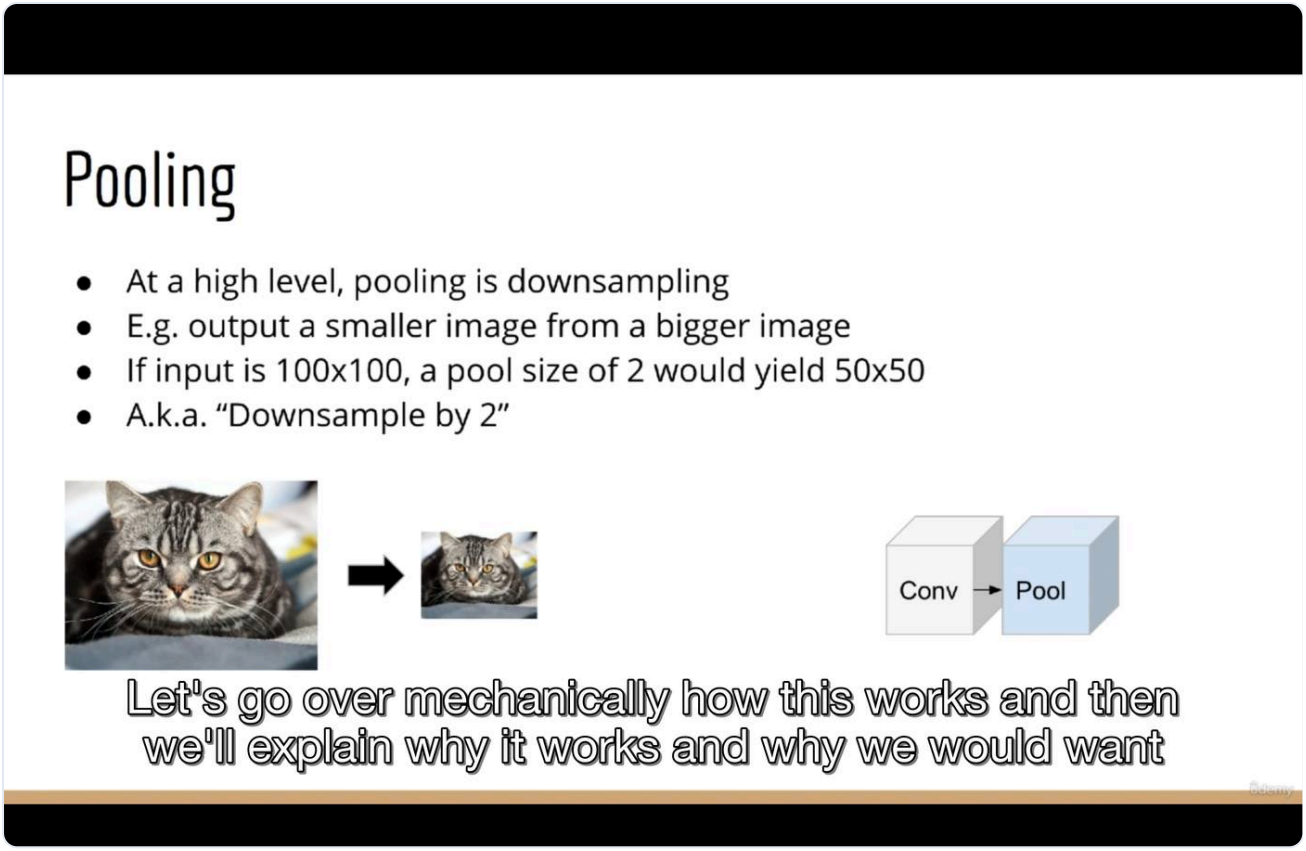


FIGURE 37.6 | At a High Level, Pooling Is Downsampling

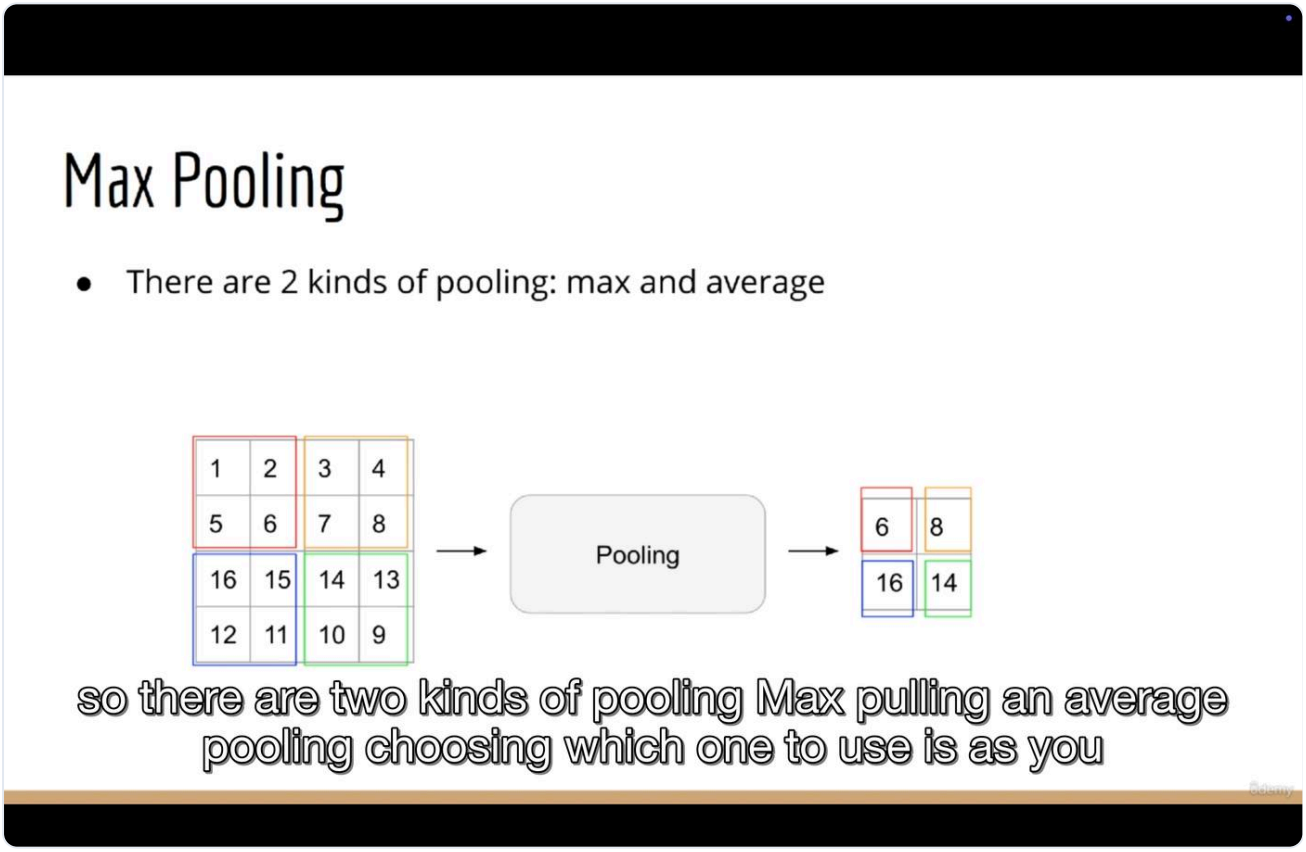


FIGURE 37.7 | Max Pooling

Let's go over mechanically how this works and then we'll explain why it works and why we would want to include it in our neural network so there are two kinds of pooling max pooling and average pooling choosing which one to use is as you probably guessed a hyperparameter choice. Let's talk about Max pooling first since I think this one's a bit more common. The best way to see how it works is by example.

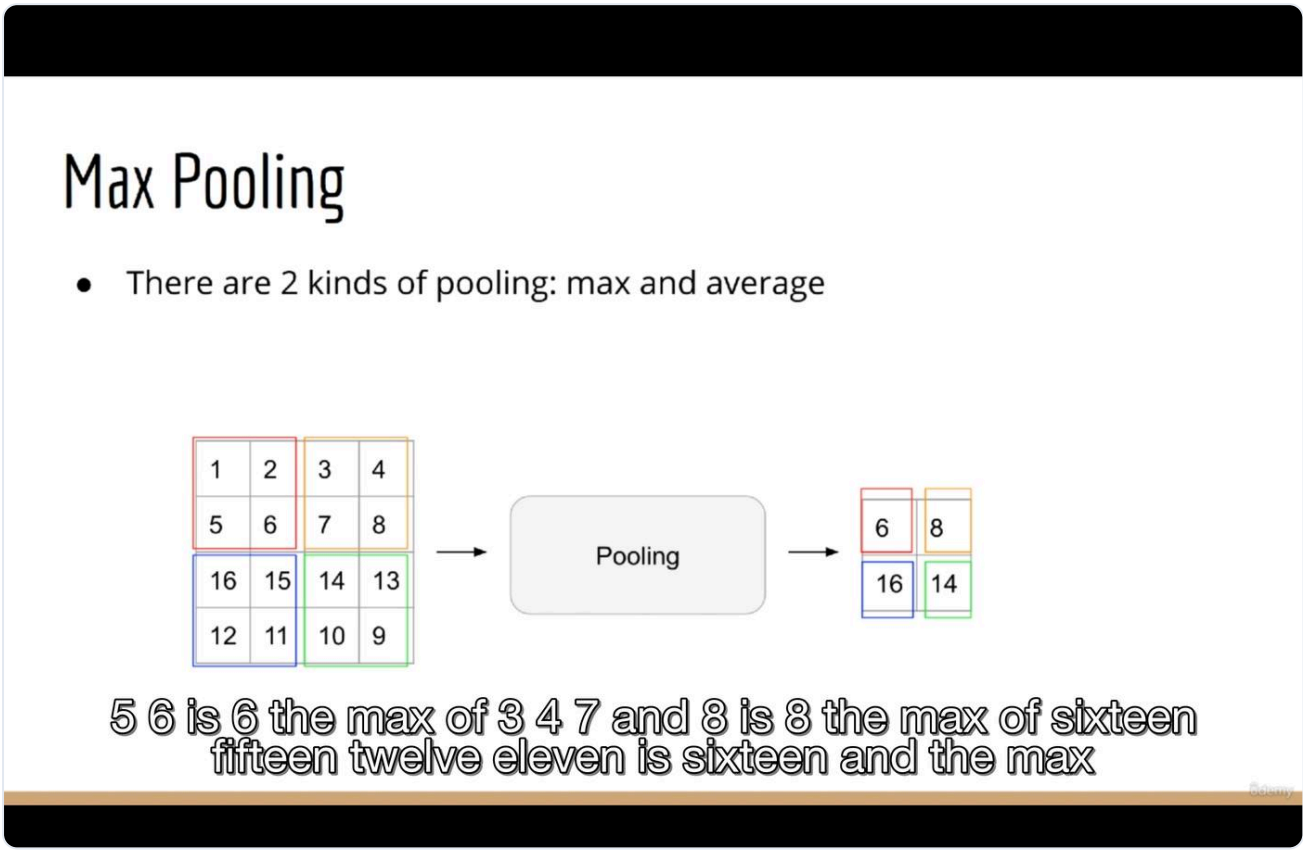


FIGURE 37.8 | Max Pooling

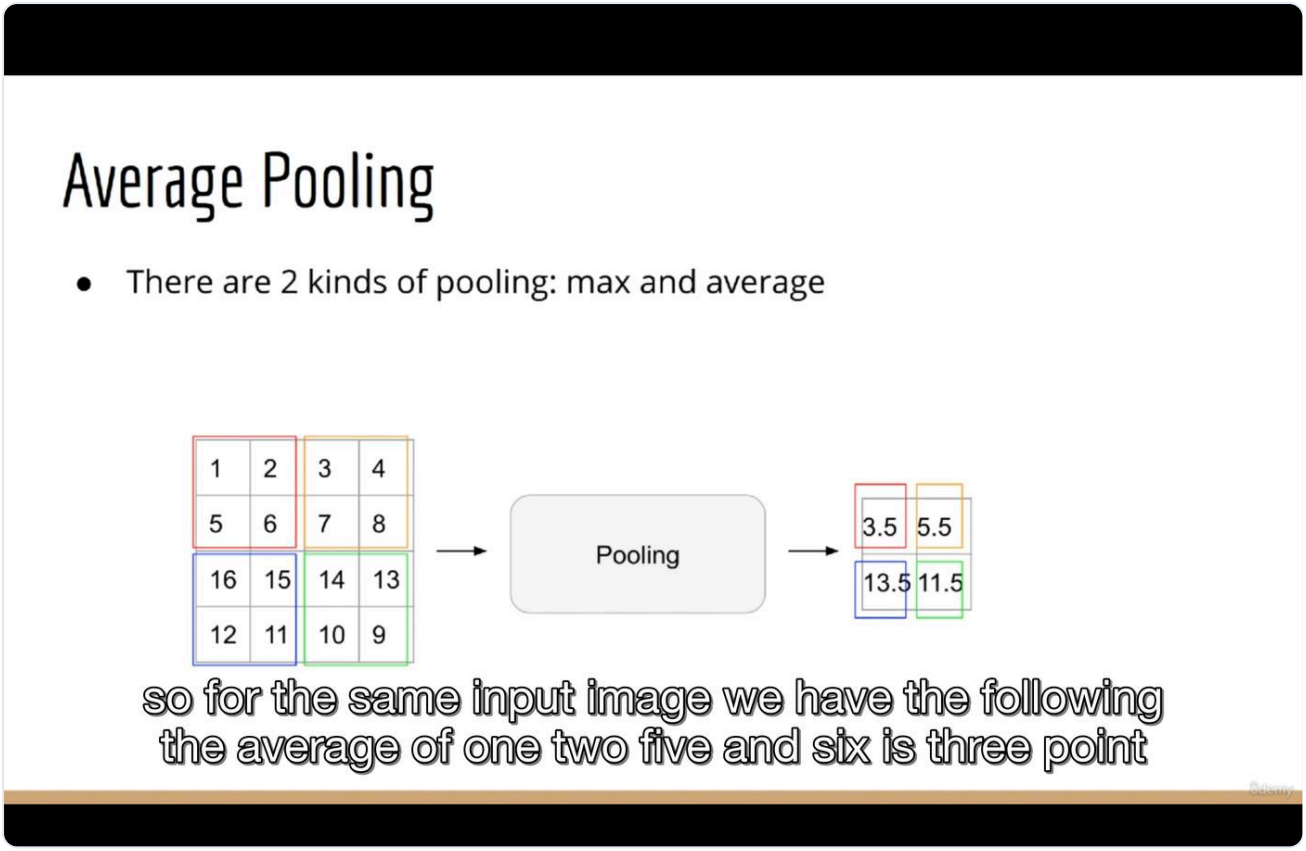


FIGURE 37.9 | Average Pooling

So if we start with an image let's say it's a four by four and contains these numbers one two three four five six seven eight sixteen fifteen fourteen thirteen twelve eleven ten nine so just the numbers one to sixteen but organized non sequentially what Max pooling will do is take each two by two square of which there are four and just return the maximum value in each of those squares so the max of 1 2 5 6 is 6 the max of 3 4 7 and 8 is 8 the max of sixteen fifteen twelve eleven is sixteen and the max of fourteen thirteen Ten Nine is fourteen so the output image is six eight sixteen fourteen as you might have guessed average pooling is similar but instead of taking the max we take the average so for the same input image we have the following the average of one two five and six is three point five the average of three four seven and eight is 5.5 the average of sixteen fifteen twelve eleven is thirteen point five and the average of thirteen fourteen Ten Nine is eleven point five. So the output image is three point five five point five thirteen point five eleven point five now that you know how pooling works. Let's talk about why we should use it the first advantage of pooling is practical.

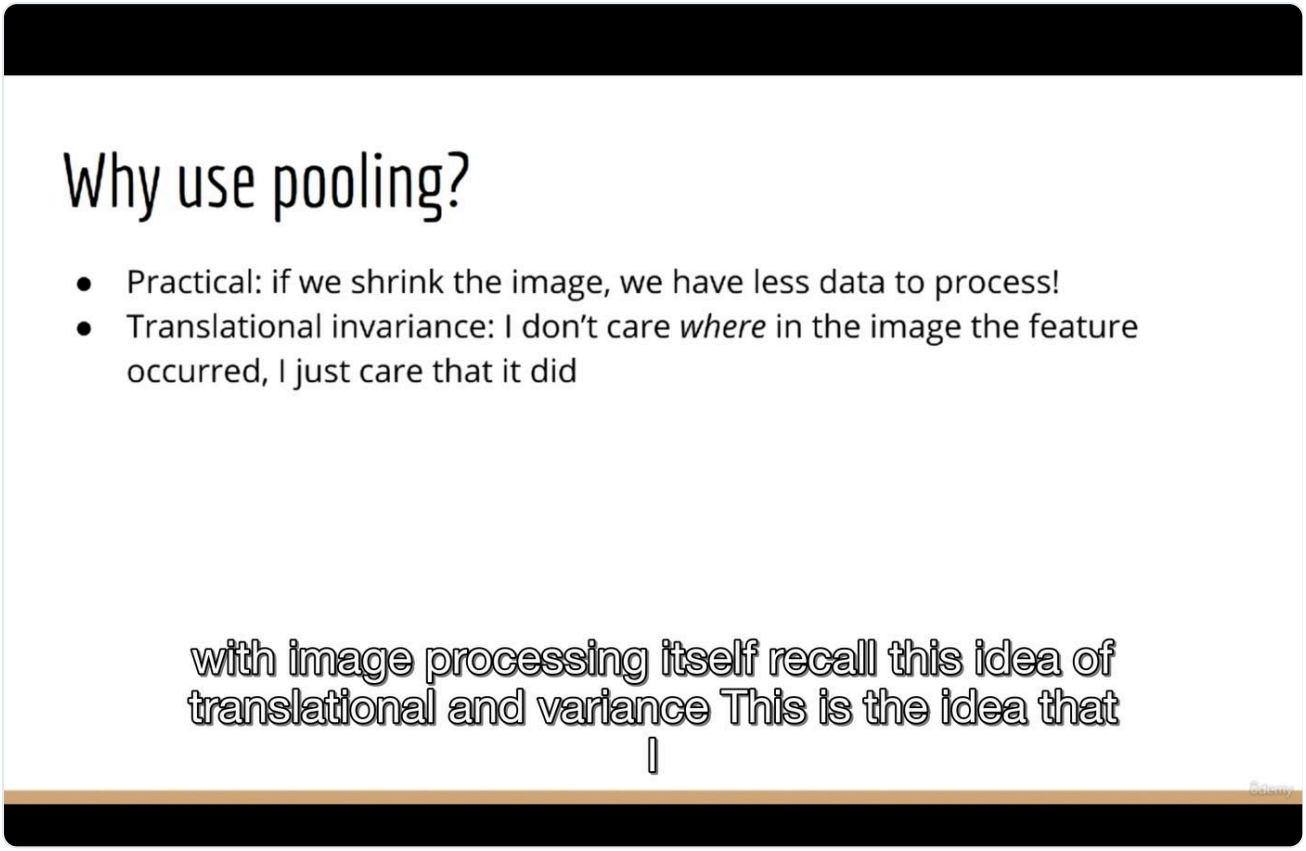


FIGURE 37.10 | Why Use Pooling?

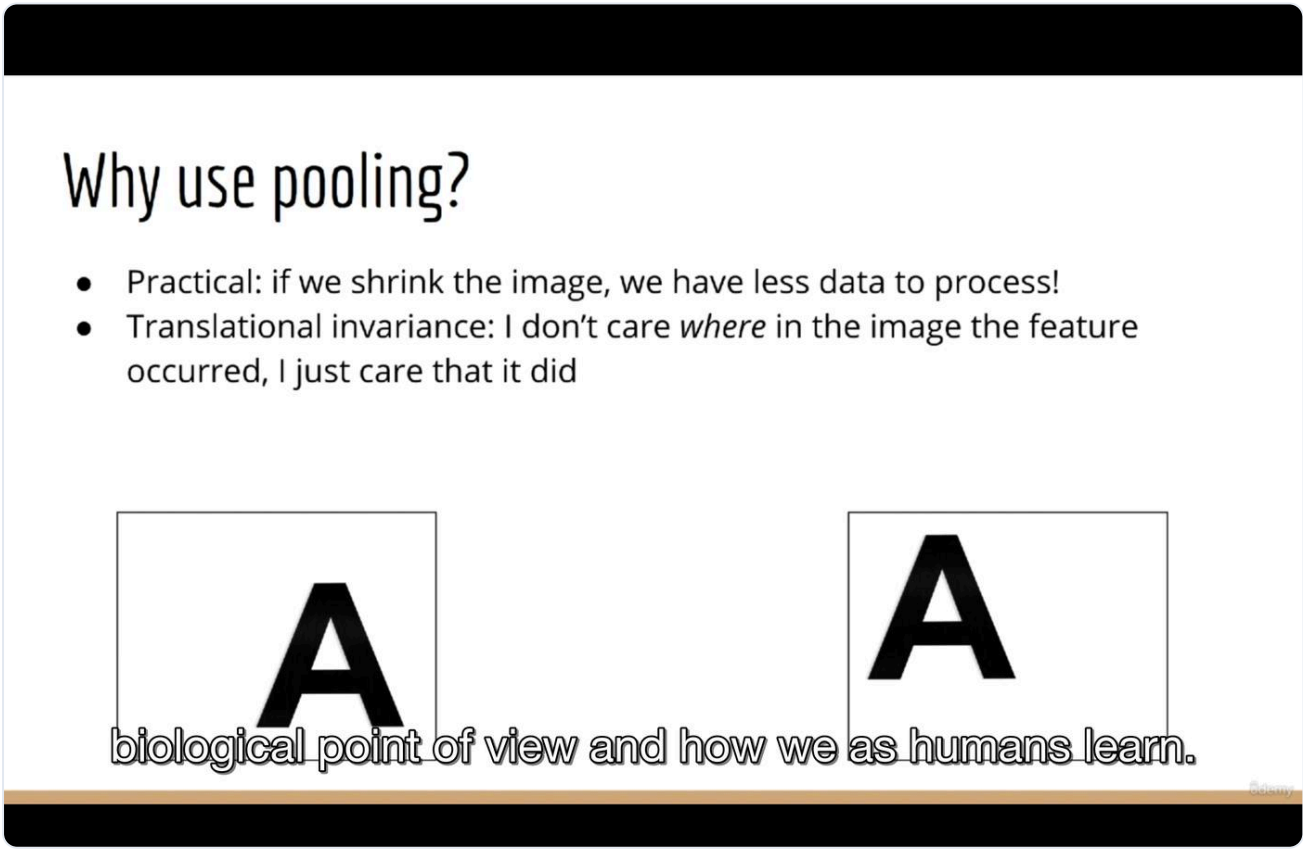


FIGURE 37.11 | Why Use Pooling?

If we down sample the image the image shrinks and therefore we have less data to process less multiplications to do and of course that will speed things up but the second and more important advantage has to do with image processing itself recall this idea of translational invariance This is the idea that I don't care where in an image the feature occurred. I just care that it occurred for example. Both you and I can recognize this as an A we also recognize this as an A. It doesn't matter where I put the A on the screen so translational invariance is important from a biological point of view and how we as humans learn. We don't have to learn how to recognize A's at each point in our field of vision.

Once we know what something looks like we can recognize it. No matter if it's up down left or right. So what is Max pooling doing.

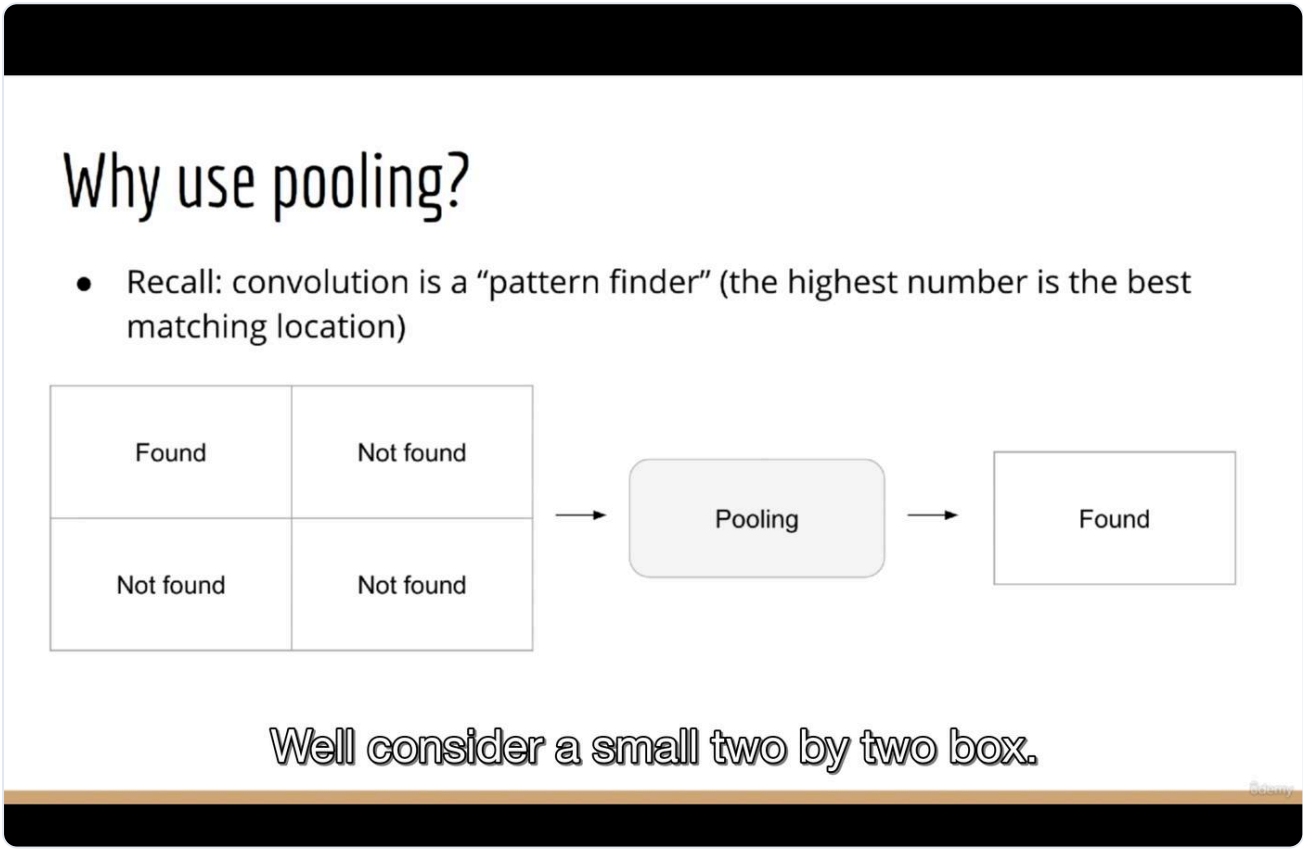


FIGURE 37.12 | Why Use Pooling?

Well consider a small two by two box. Remember that convolution it tells us whether or not a feature has been found. It's a pattern finder. So imagine that in this two by two box I have these flags pattern found not found not found and not found. So it's a high number and a small number and a small number and a small number. The pattern being found. Returns a higher number. So when I take the max it says the pattern has been found. That's good because it tells me that the pattern has been found without caring where it was found. Average pooling is the same idea but I think Max pooling is a little more intuitive in this regard. As a side note keep in mind that in this lecture I've assumed that we're using a pool size of two and that at each pooling layer we select boxes which are two spaces apart now pooling layers have some flexibility in this regard.

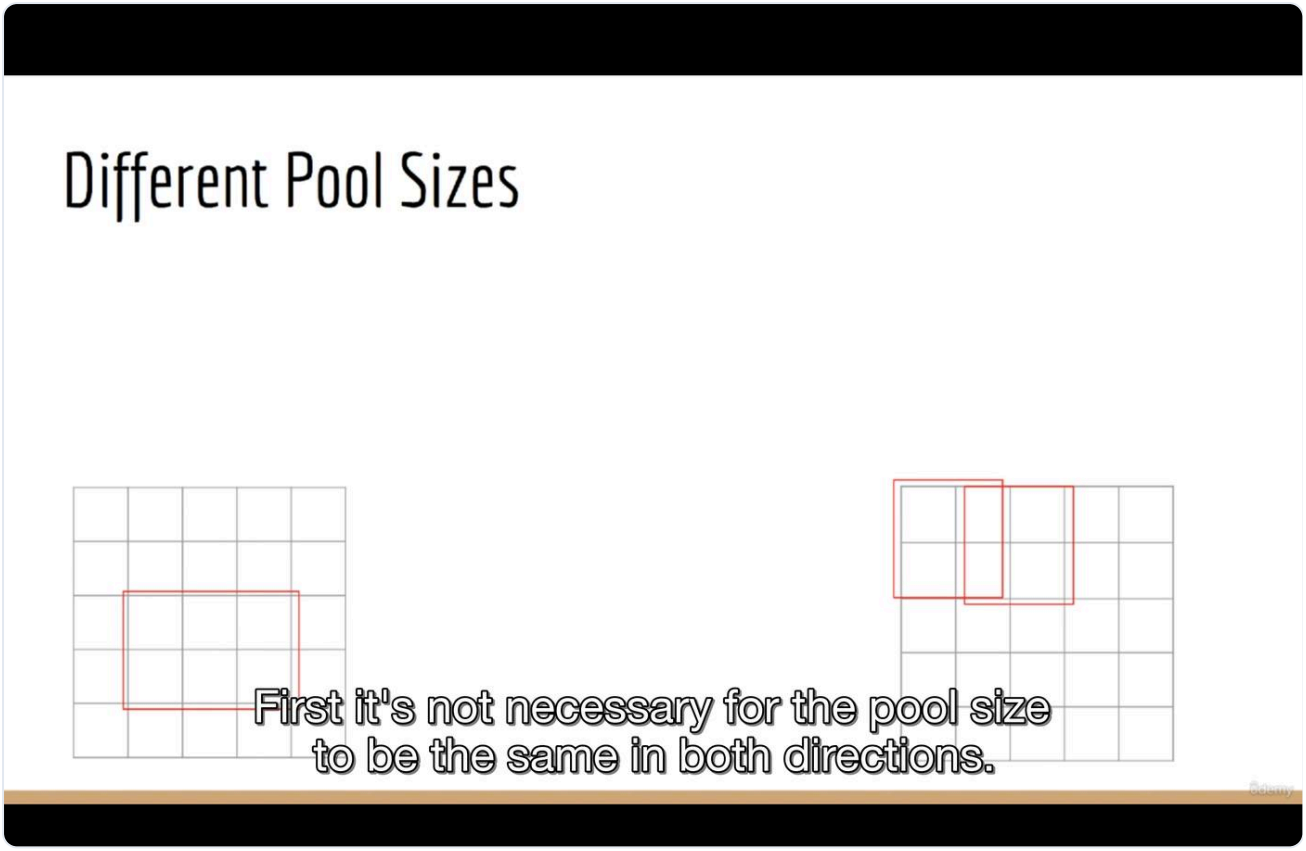


FIGURE 37.13 | Different Pool Sizes

First it's not necessary for the pool size to be the same in both directions. For example you could look at a two by three box or a three by two box although generally that's unconventional.

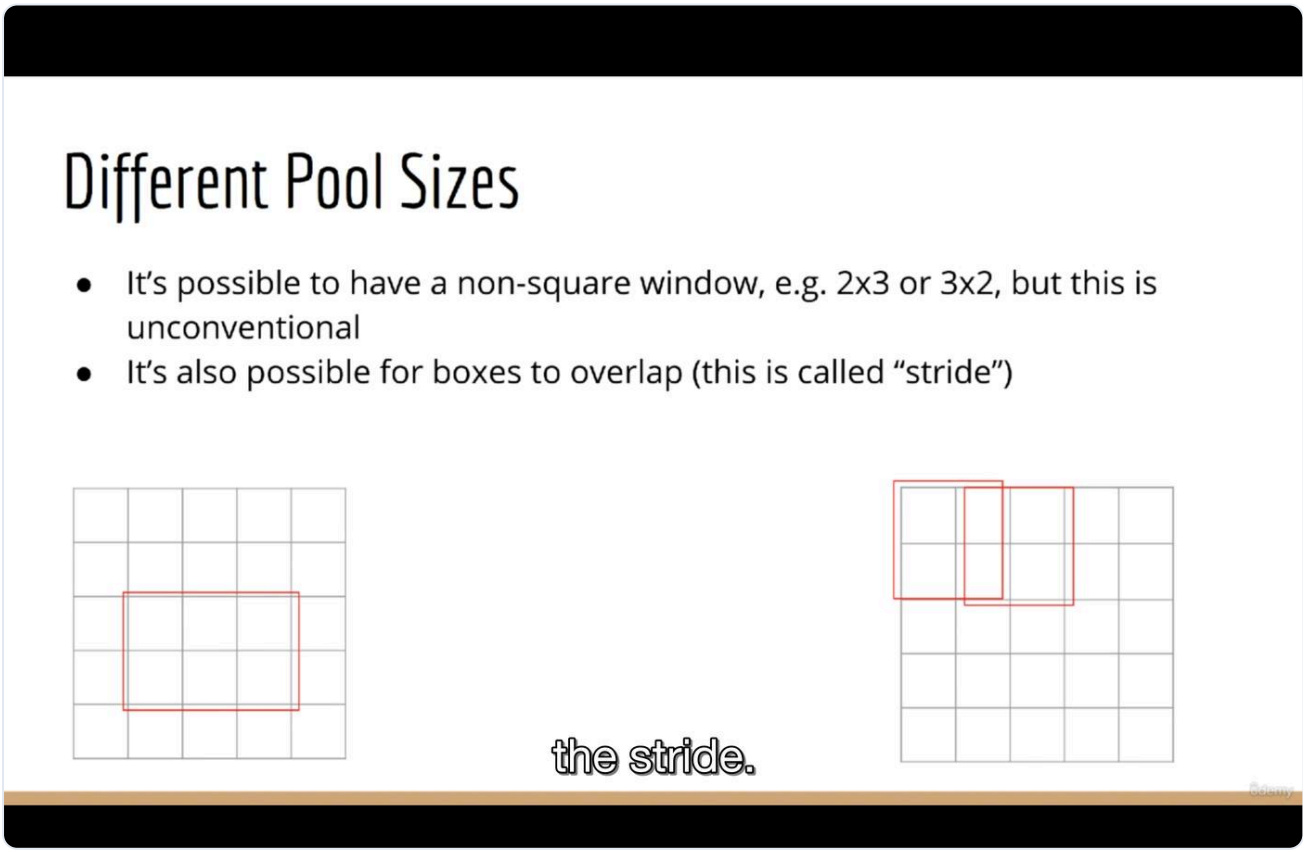


FIGURE 37.14 | Different Pool Sizes

Second is that it's possible for the boxes to overlap the hyperparameter that controls this is called the stride.

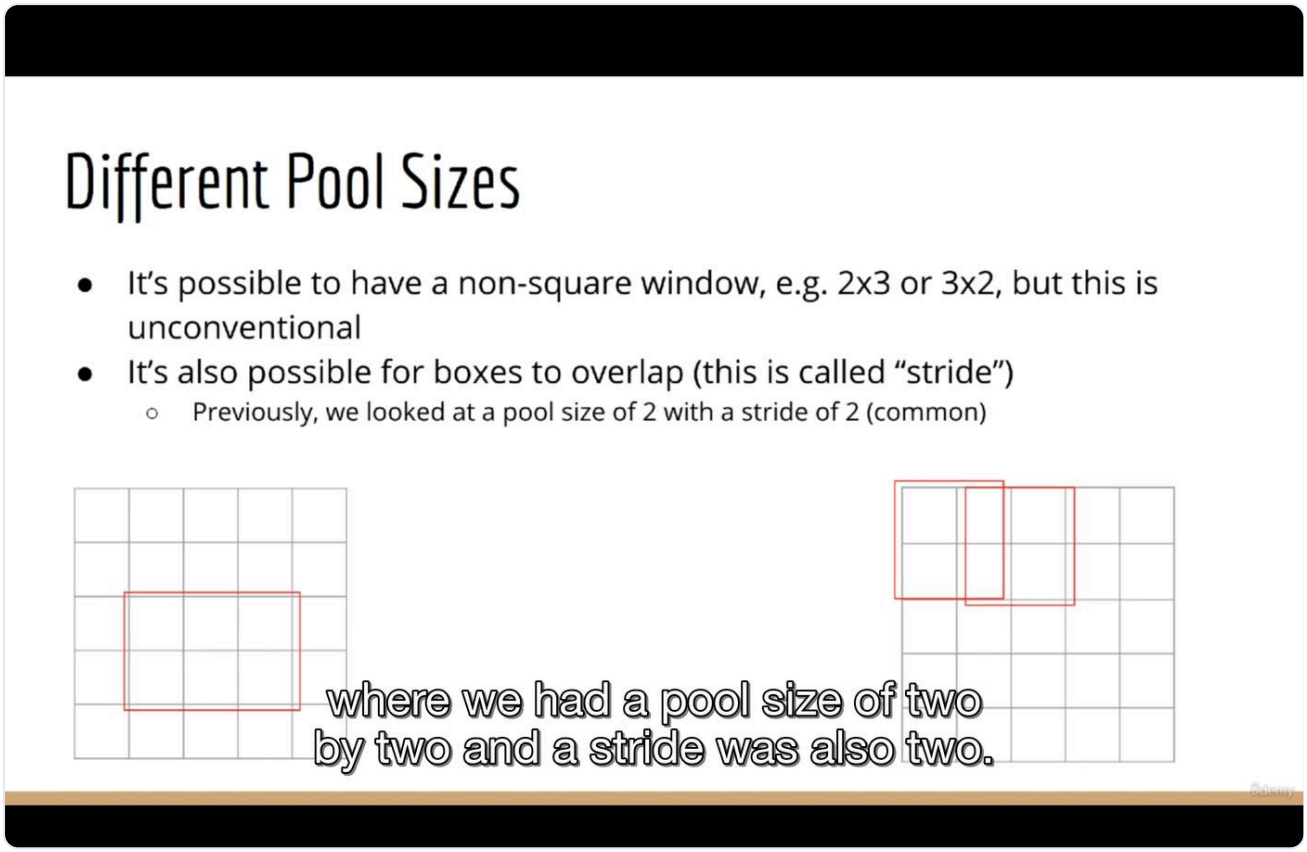


FIGURE 37.15 | Different Pool Sizes

It tells us how many spaces to move the pooling box for each output previously we looked at the situation where we had a pool size of two by two and a stride was also two. This is a pretty common scenario.

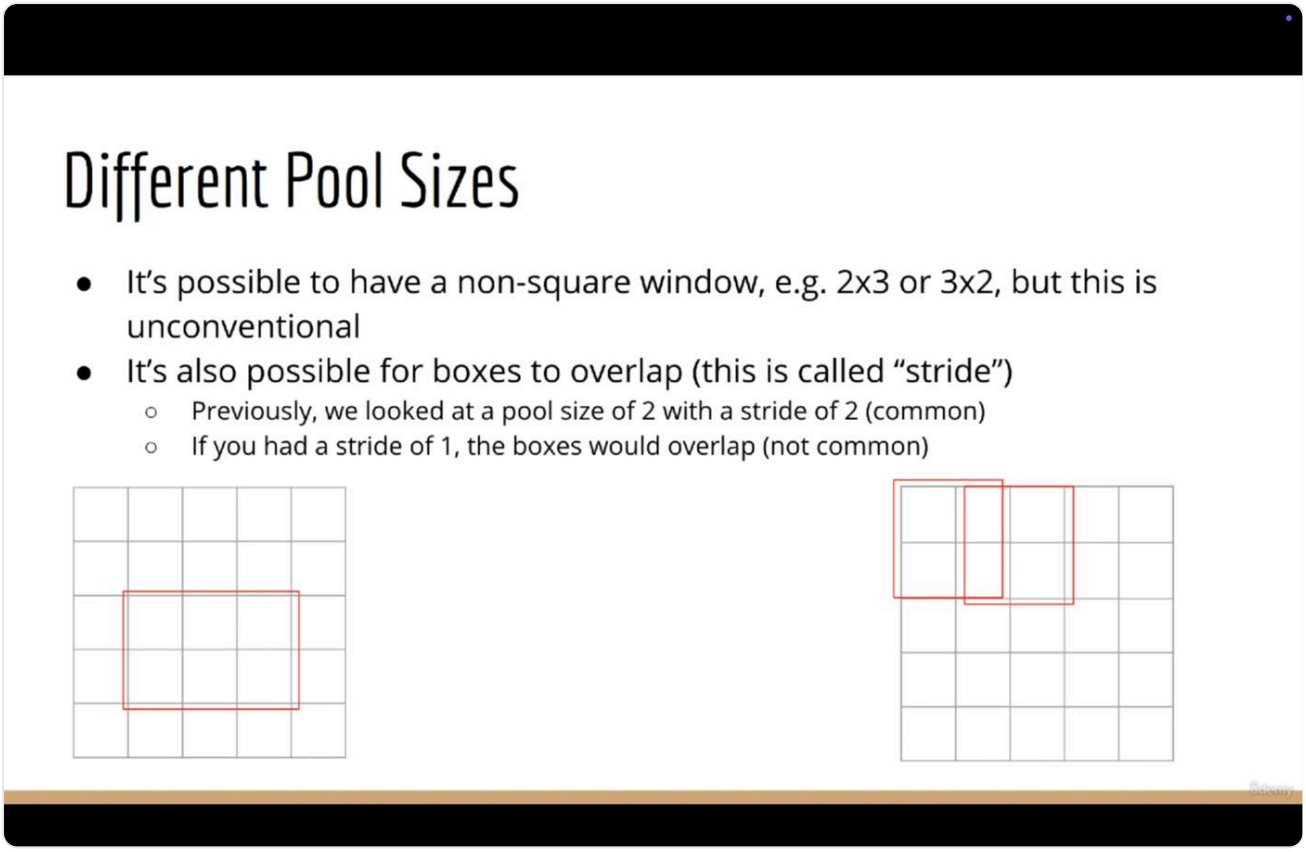


FIGURE 37.16 | Different Pool Sizes

If you had a stride of one then our two by two box would overlap. This is not really too common. So while you have these options which I want to make you aware of they are usually not changed too often. So what you see in this lecture is what we do most often we use the pool size of two in each direction and we have a stride of two so that we don't have any overlapping boxes.

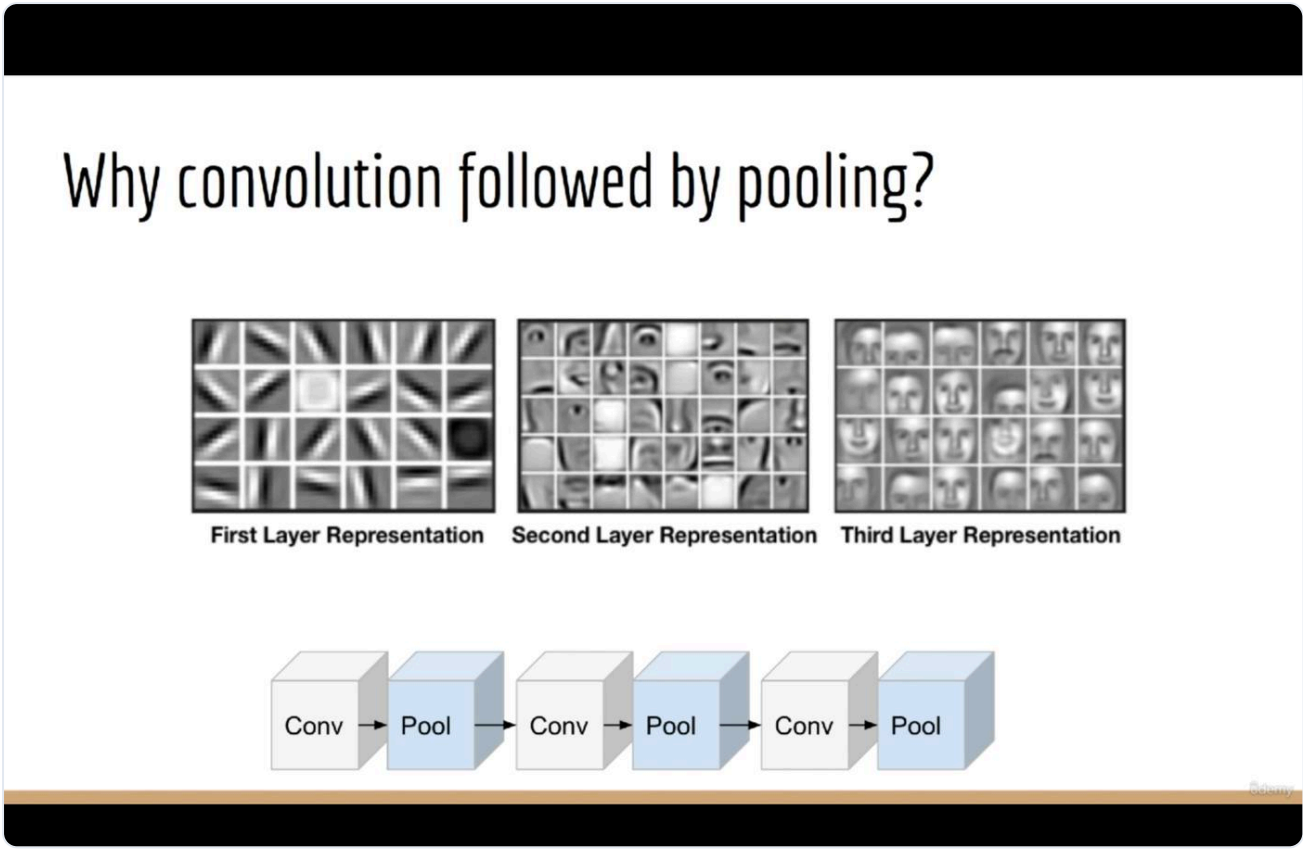


FIGURE 37.17 | Why Convolution Followed by Pooling?

So now that you know what pooling does and why it works let's discuss why the convolution part of the CNN is organized the way it is. Why should we have a sequential pattern consisting of a conv layer followed by a pooling layer followed by a conv layer followed by another pooling layer and so forth. If you recall I told you earlier that one cool thing researchers discovered is that CNNs are able to learn features hierarchically so the initial layers end up learning basic strokes. The next layer ends up learning individual facial features such as nose eyes and lips. The next layer ends up learning whole faces the key points to keep in mind is that after each pooling the image shrinks but the filter sizes generally stay around the same common filter sizes are 3 by 3 5 by 5 and 7 by 7.

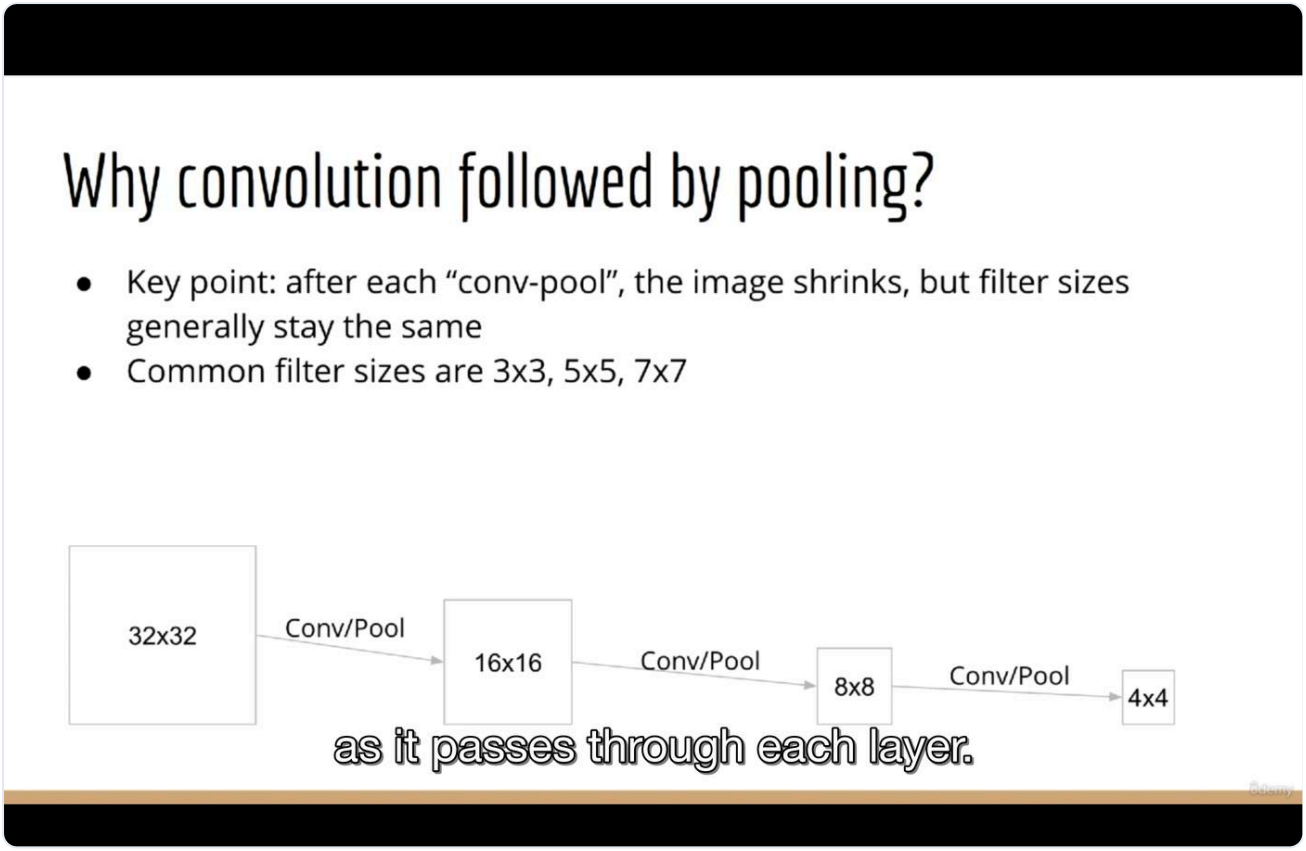


FIGURE 37.18 | Why Convolution Followed by Pooling?

Generally speaking they are much smaller than the input image but what happens as the image shrinks as it passes through each layer. Well let's suppose we start with a 32 by 32 image and we have four convolution and pooling layers. Let's assume we do convolution and same mode so it doesn't change the size of the image and let's assume all our convolution filters are size 3 by 3. Although this doesn't come into play just yet. In this case after the first convolution and pooling the image shrinks down to 16 by 16. After the second convolution and pull in the image shrinks down to eight by 8. After the third convolution and pooling the image shrinks down a four by four.

And finally after a fourth convolution and pooling the image shrinks down to two by two what happens after this is still a mystery.

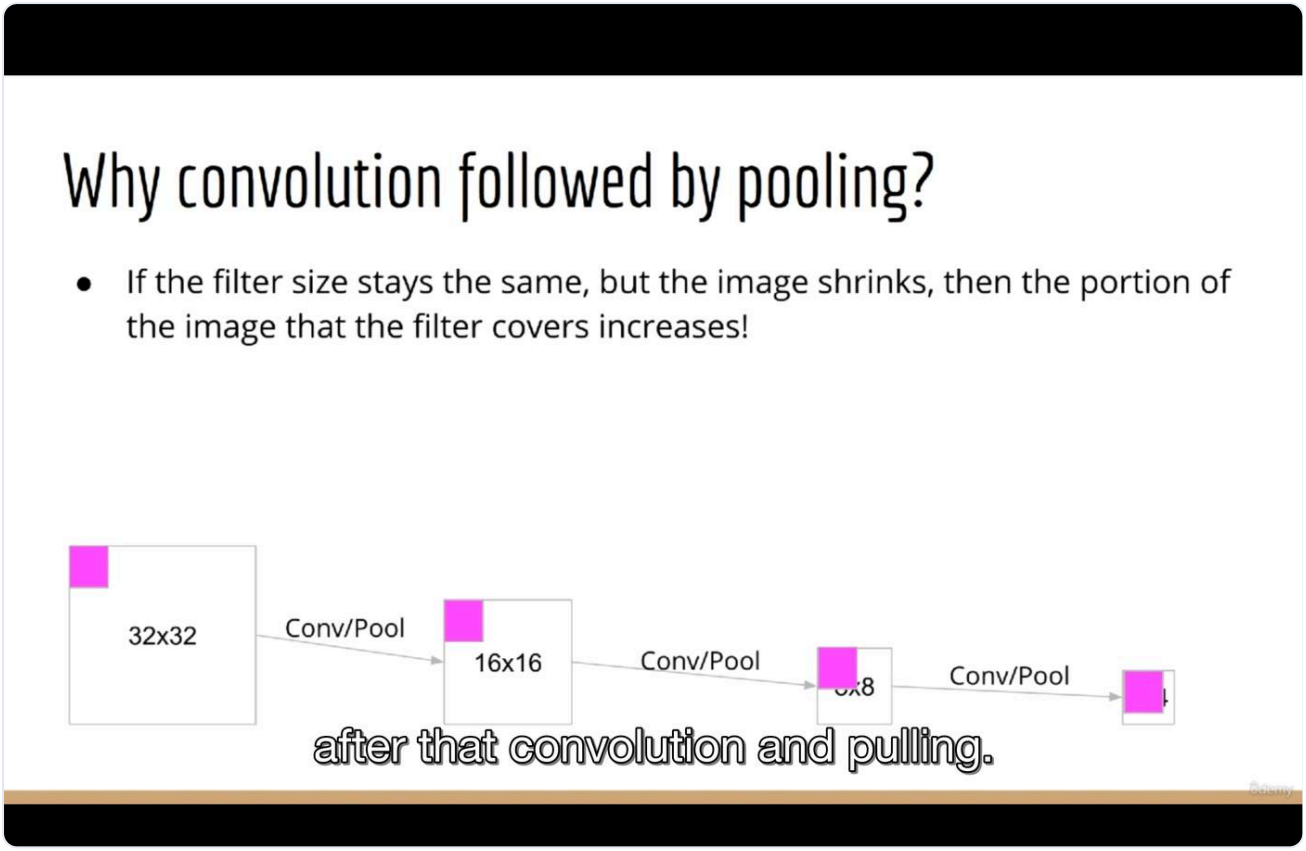


FIGURE 37.19 | Why Convolution Followed by Pooling?

So let's just leave that for now and focus on the image shrinking as you can see if you take a three by three filter and place it over a 32 by 32 image it occupies only a very small portion of the image. In other words is looking for very tiny patterns to match like for example simple edges and strokes. Then the image becomes size 16 by 16. Now all of a sudden in the three by three filter takes up four times as much space in the image it takes up twice as much space in each direction and two times two is four. In any case it occupies a much larger portion of the image relative to the first filter. So now it's looking for patterns which take up more space on the image patterns like nose eyes and lips.

Next we get to an eight by eight image a three by three filter occupies almost a quarter of the image after that convolution and pooling.

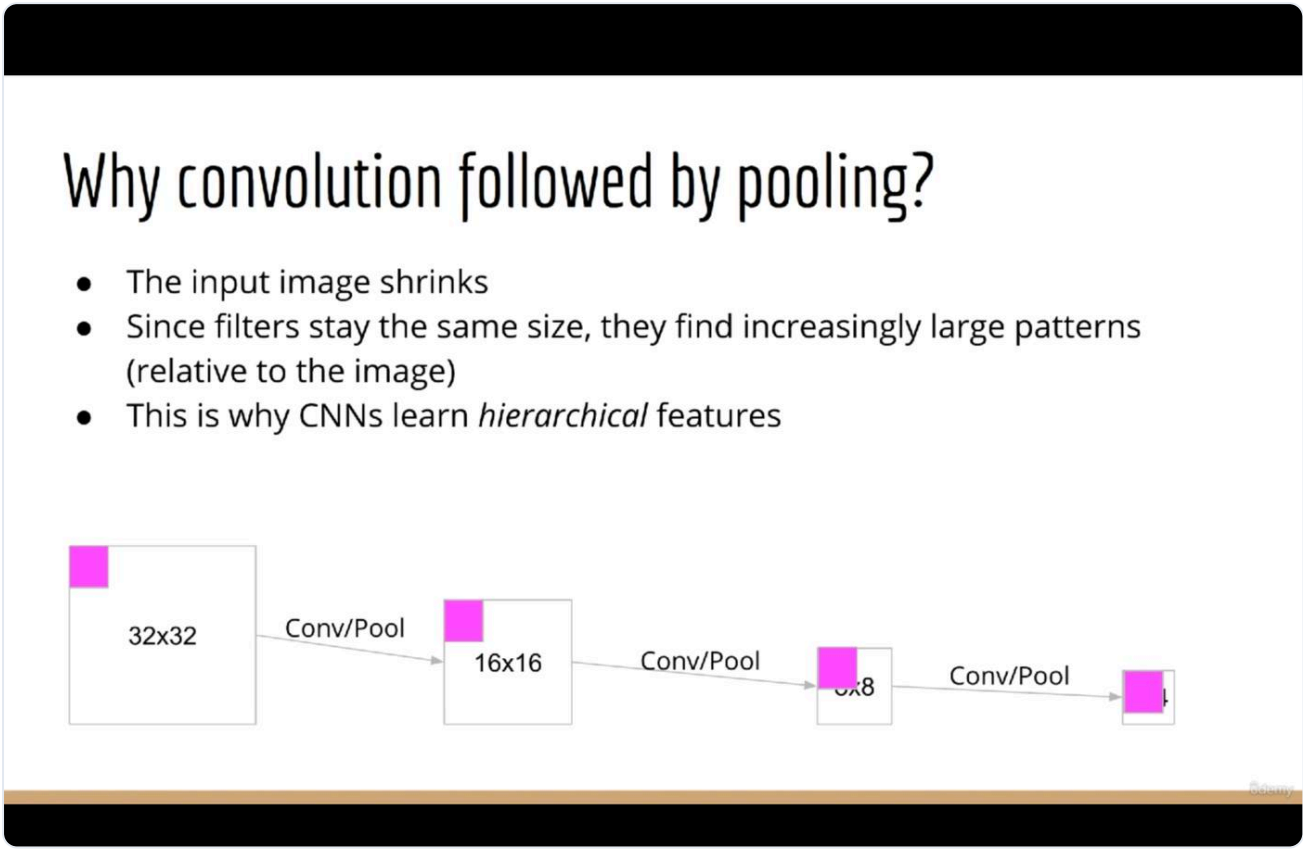


FIGURE 37.20 | Why Convolution Followed by Pooling?

We get a four by four image now a three by three filter takes up most of the image. So what have we learned. We've learned that in a convolutional neural network convolution and pooling results in two things which are corollaries of each other first that the image input shrinks. Second since filters generally stay the same size the filter looks for larger and larger patterns in image. This is what leads to the CNN learning hierarchical features of the input. It first learns things that are relatively small compared to the total size of the image. Then it learns to find bigger patterns and bigger patterns and so forth. The last concepts regarding the convolutional layers we have to think about is are we not losing information at every layer.

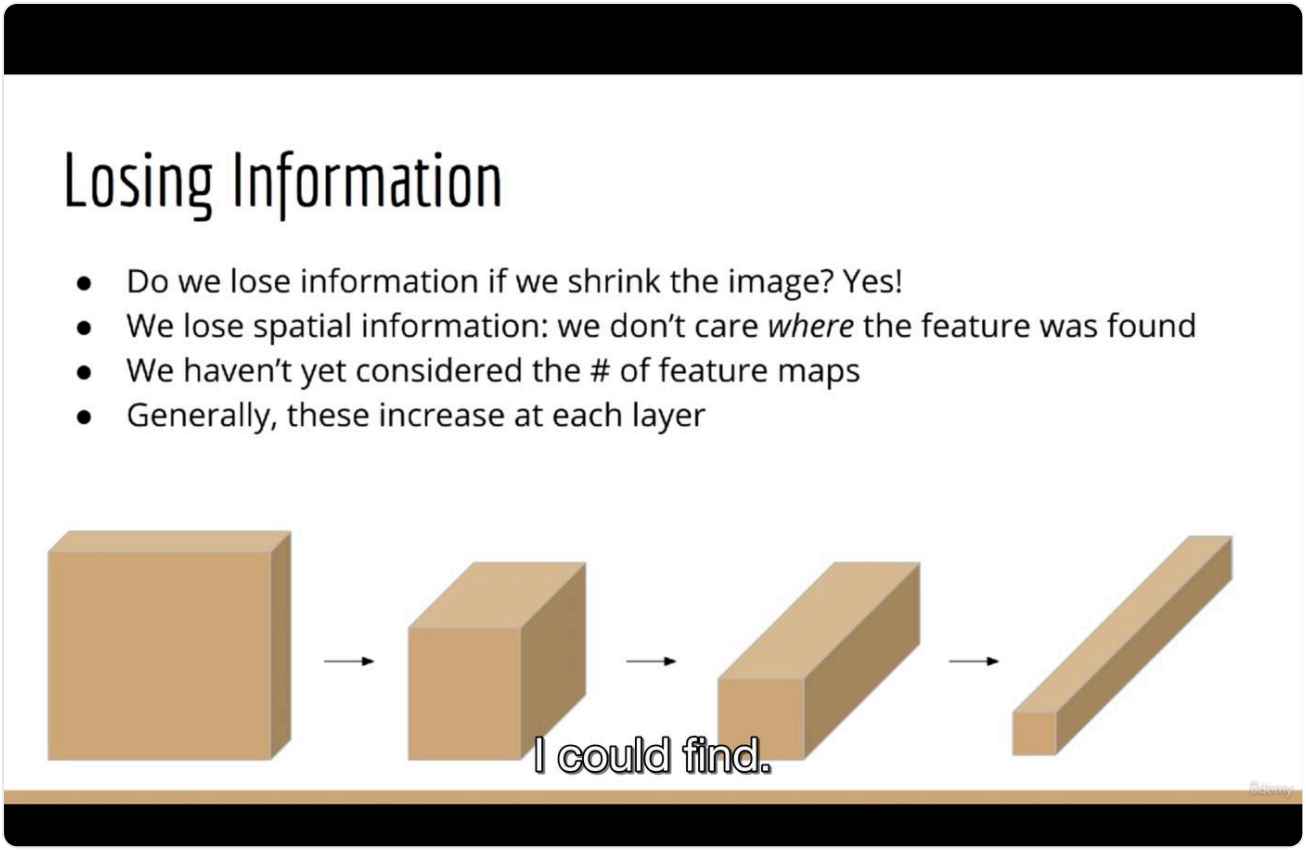


FIGURE 37.21 | Losing Information

If we shrink the image and the answer is yes that's true. We are losing spatial information because the image keeps shrinking. We're saying we don't care where in the image the feature was found just that it was found somewhere in the image. So we're losing information about where it was. But there's another component of the convolution process we haven't considered yet which is the number of feature maps. Again we have a general pattern to follow which is that while the size of the image generally decreases the number of feature maps generally increases. In other words I don't care where the feature is found. I just care that it was found but I do care about different possible features.

I could find.

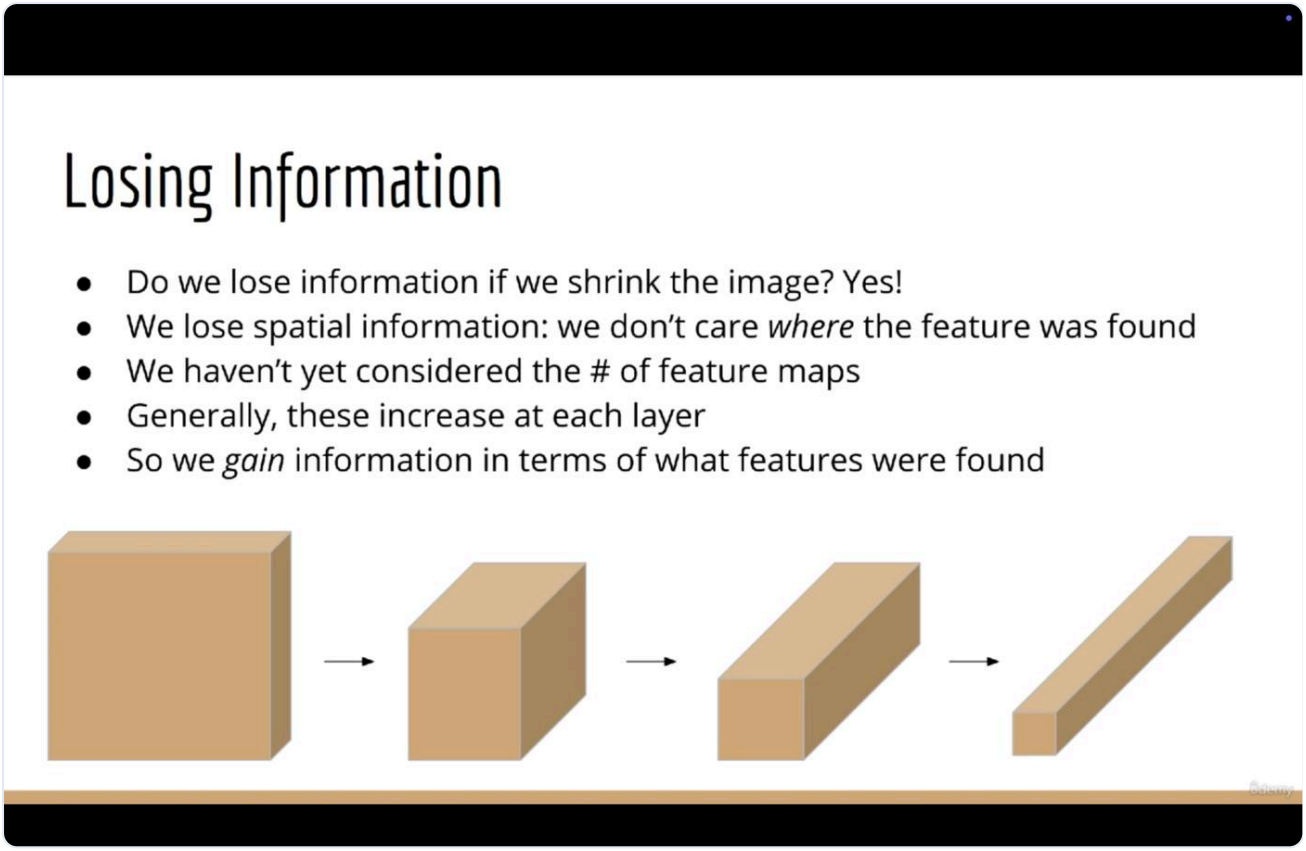


FIGURE 37.22 | Losing Information

So while you're losing spatial information you're gaining information in terms of what features were found in the image one last note on this convolution and pooling pattern. One of the hardest things to grasp for new students and deep learning is that we have so many choices. We call these hyperparameters previously for a minute. We looked at the learning rate the number of hidden layers and the number of hidden units. But now with CNN is we have a ton of more choices with convolution.

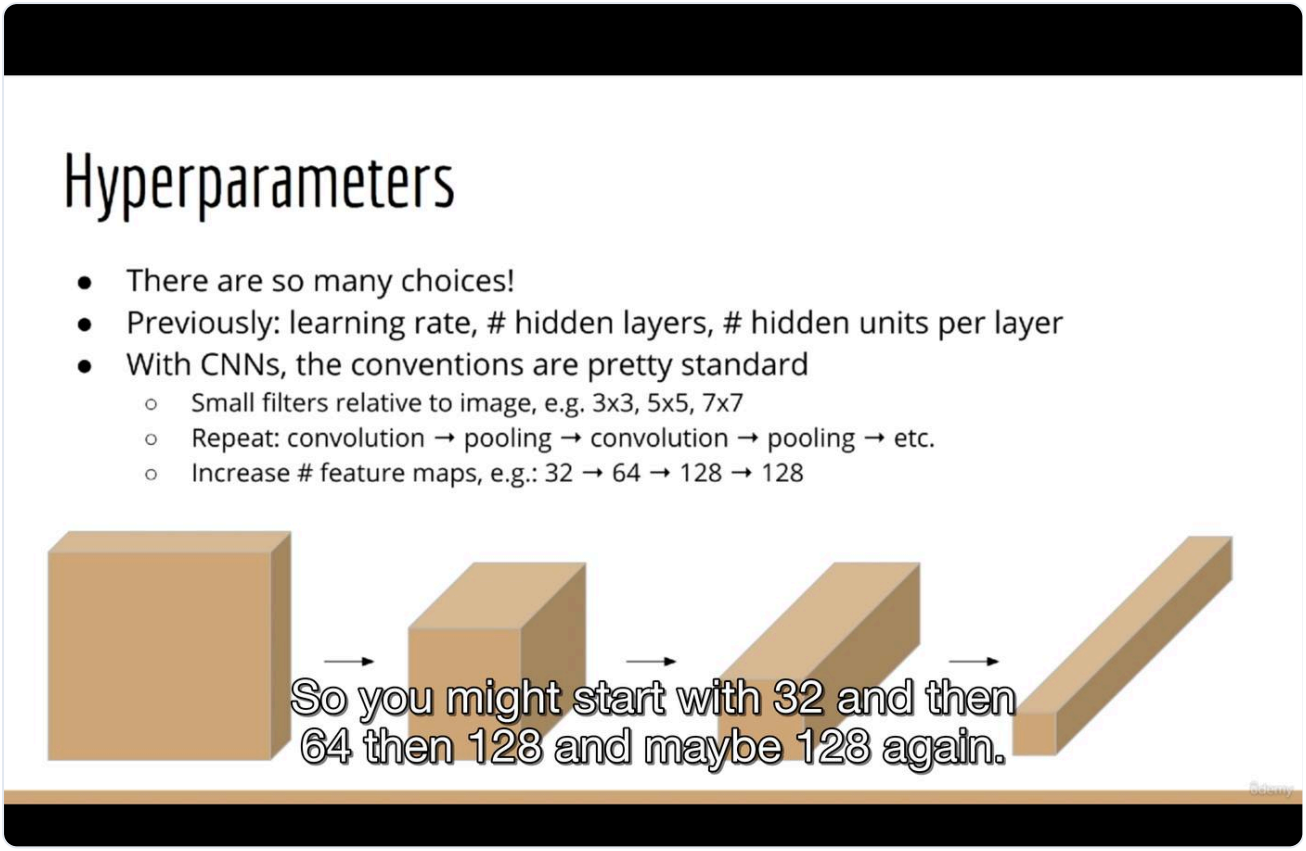


FIGURE 37.23

We have to choose the filter size. We have to choose the number of feature maps we have to choose the pool size and so forth. But luckily I think this is one instance where there is a general pattern that most people follow. So you're not going in completely blind so stick with these guidelines and you can be sure that you're at least doing things per accepted convention that is choosing small filters relative to the image like three by three five by five or seven by seven. Another guideline is to repeat the pattern convolution followed by pooling. Another guideline is to increase the number of feature maps at each convolution.

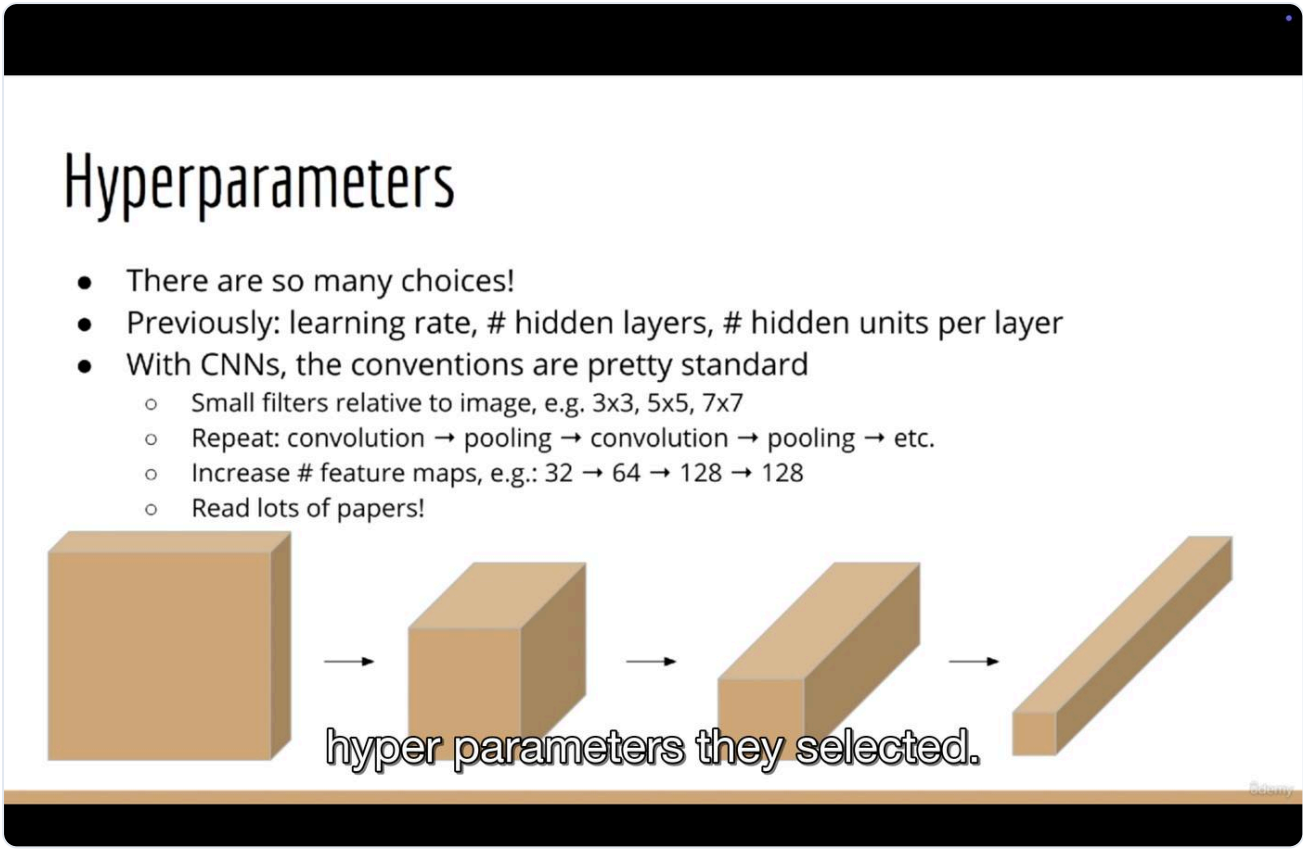


FIGURE 37.24

So you might start with 32 and then 64 then 128 and maybe 128 again. The best way to learn about how other people are doing it is to read lots of papers and check out which hyperparameters they selected. Generally you'll find that this is the pattern followed by most convolutional networks next. I want to mention something a little funny but in fact pooling is sometimes not what we are going to end up using in our CNN. Researchers have found that we can do something similar but which is more efficient and sometimes works just as well.

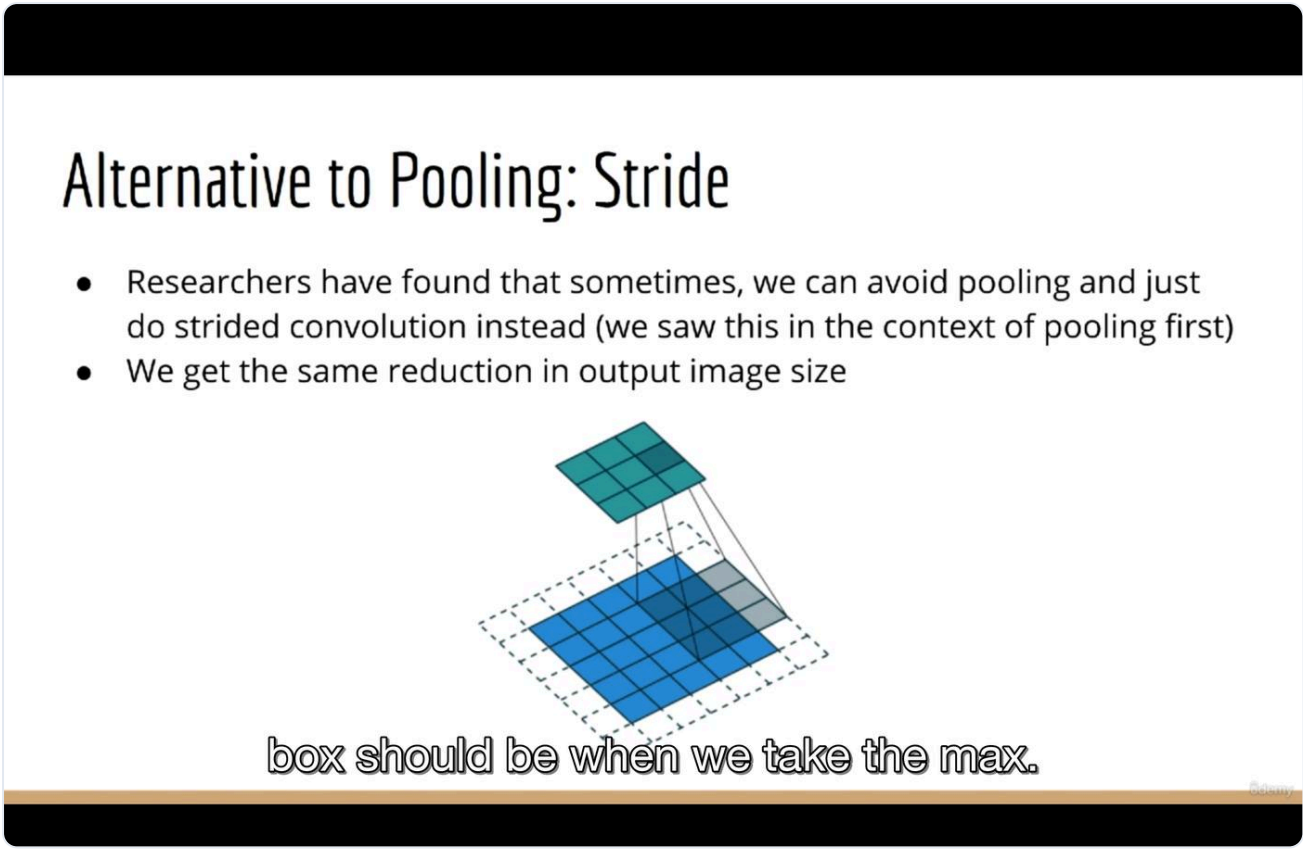


FIGURE 37.25 | Alternative to Pooling: Stride

In particular recall that for pooling we have this concept of stride that tells us how far apart each box should be when we take the max. In fact convolution has a stride option as well so in this animation you can see how convolution works with Stride. Remember convolution just means multiply and add with a sliding window when we have convolution with a stride parameter. The stride tells us how far apart each window should be. And of course our intuition tells us that if we use a stride of two then the output image length will be half of what the input was.

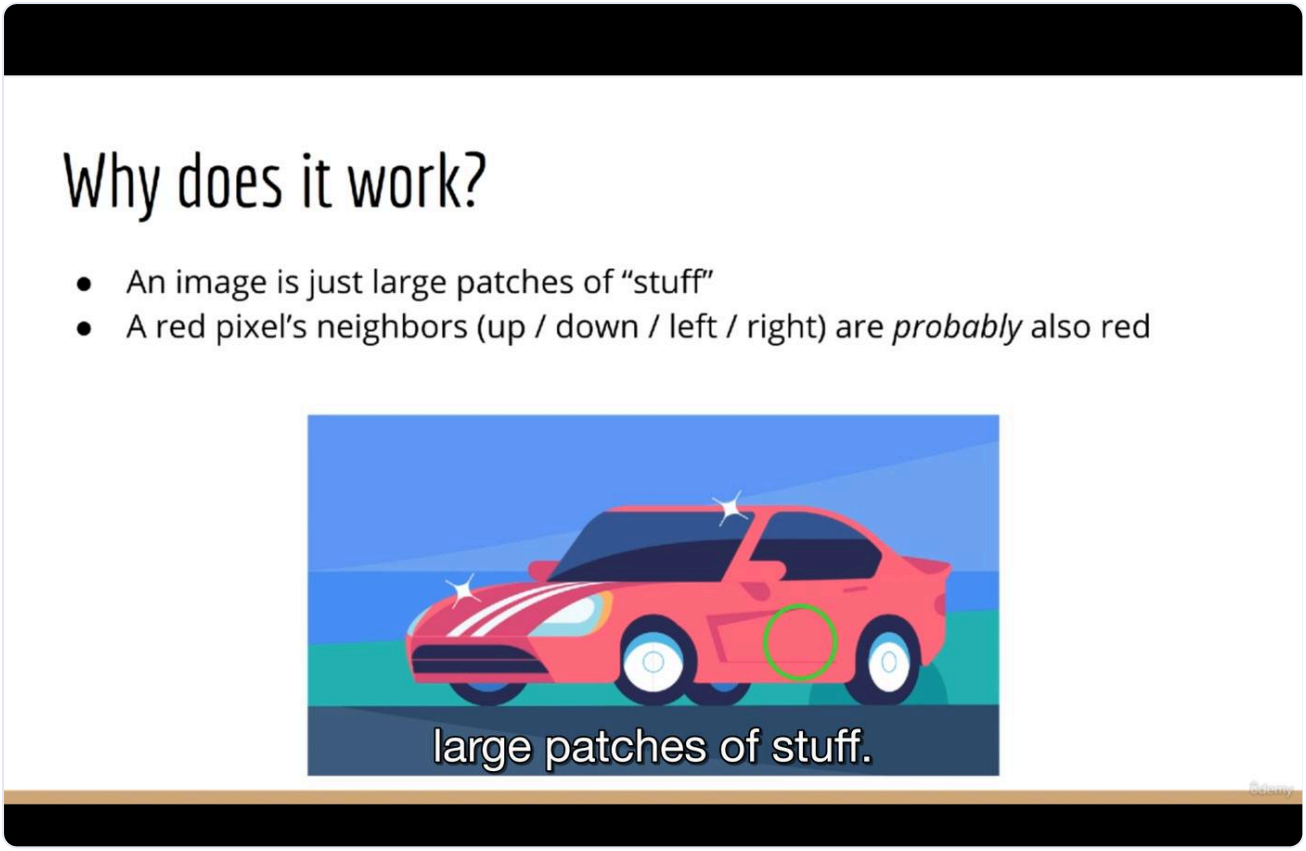


FIGURE 37.26 | Why Does It Work?

In other words we get the same reduction in size by using a strided convolution instead of convolution followed by pooling the intuition behind why this works is that if you consider what an image looks like an image is just large patches of stuff. What I mean by that is each pixel is likely to have a similar value to its neighboring pixels. For example suppose I'm looking at a red car. I find a red pixel. Now ask yourself is the pixel above that also red.

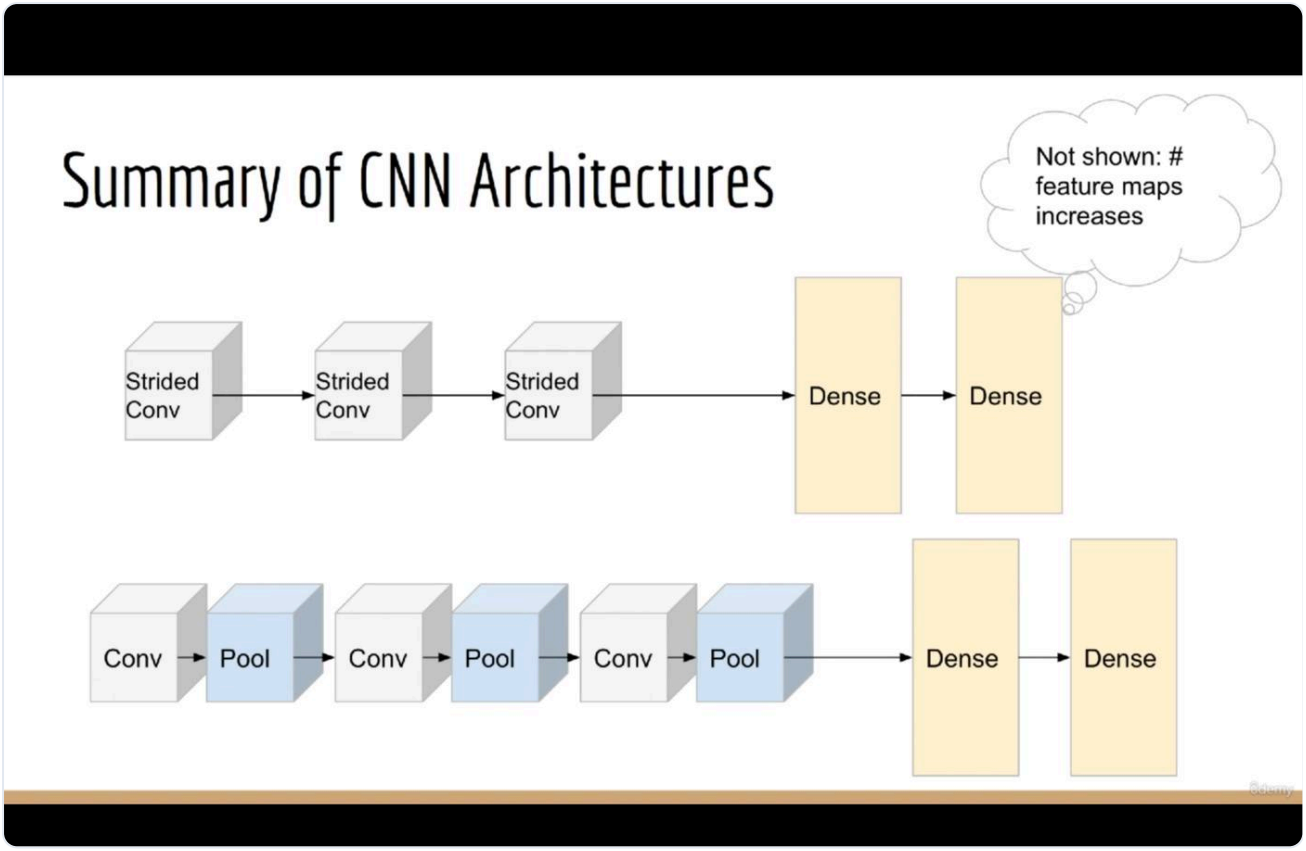


FIGURE 37.27

Yes. Is the pixel below that also probably red. Yes. The same can be said of the pixel to the left and to the right. That's just the nature of images pixels near each other are often very highly correlated. You won't have this random jumping around of pixel values because something like that wouldn't look like an image at all.

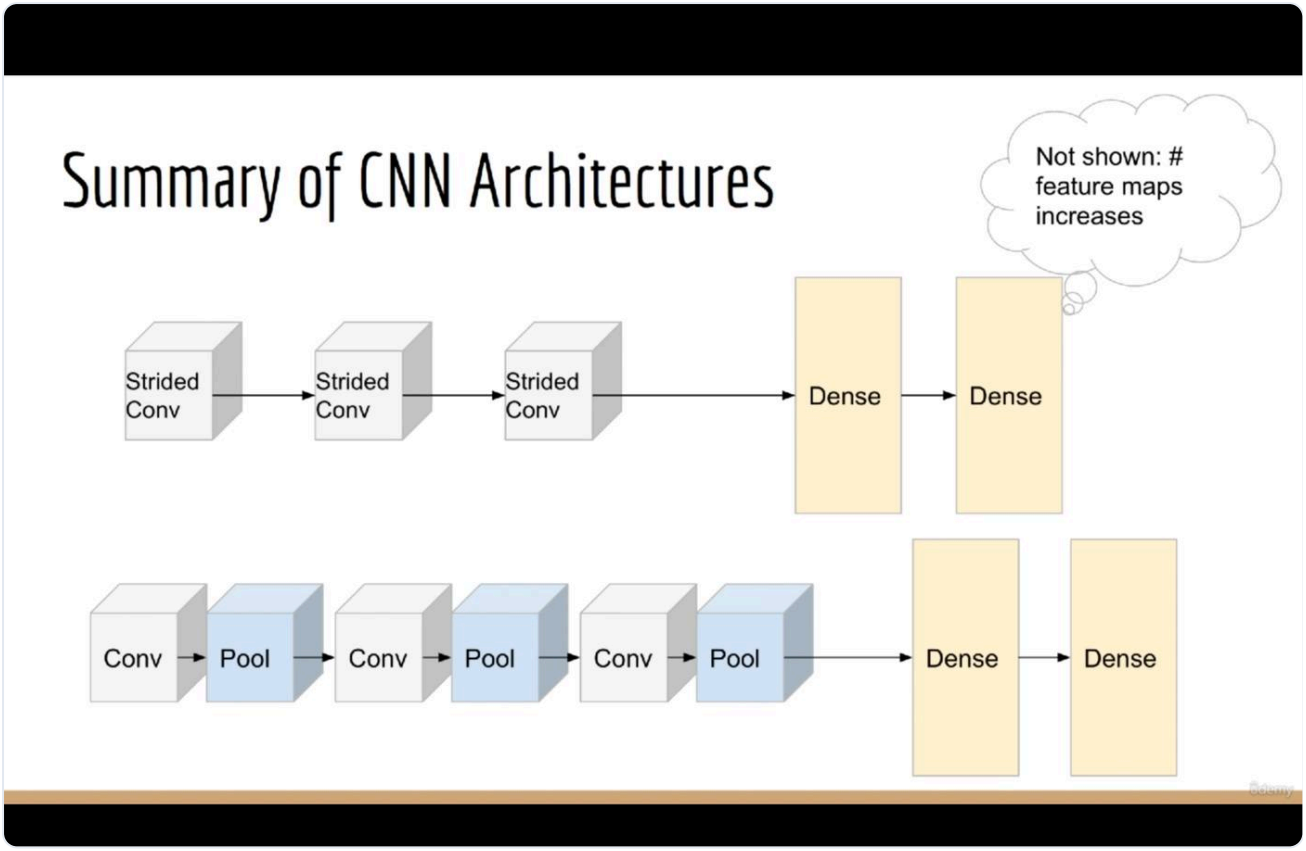


FIGURE 37.28

That would probably just be noise and so using a stride of two simply means having your filter skipping every other pixel. It's saying we don't care what those skipped pixels are because they probably are very close to the pixels we were already looking at. So to summarize the convolution part of our convolutional neural network now looks like this. We can have a series of strided convolutions all with approximately the same filter size with increasing feature maps at each layer. Or we can have a series of convolutions followed by pooling again all with approximately the same filter size and with increasing numbers of feature maps at each layer the last part of this lecture will focus on the second half of the CNN which is the feedforward neural network part.

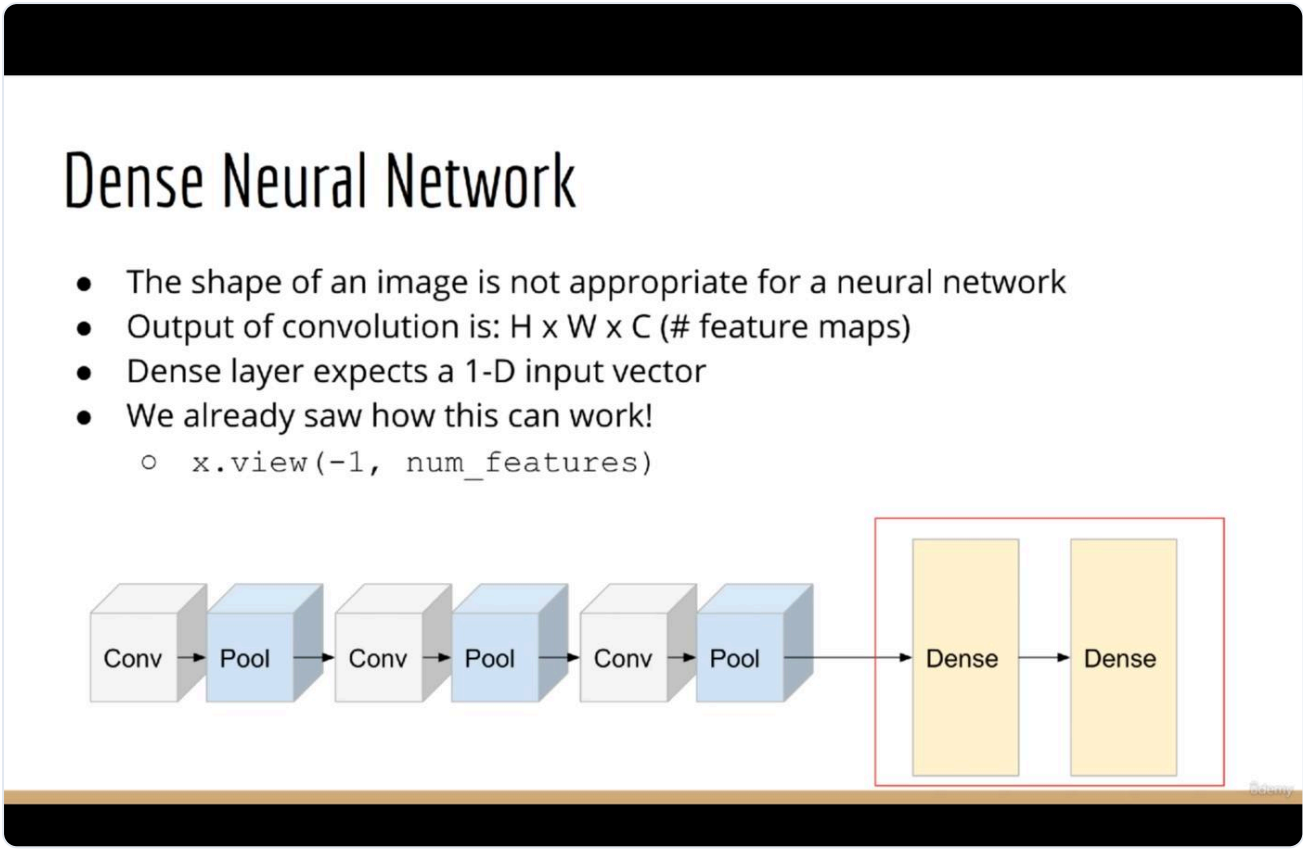


FIGURE 37.29 | Dense Neural Network

Now there's not much to discuss here except that we have to consider the shape of an image is not appropriate for a dense feedforward network. Remember that when an image comes out of a convolution it's going to be three dimensional height by width by number of feature maps but a feedforward neural network takes in a feature vector a one dimensional object. Luckily we've already discussed how this works. We can turn a 3D object into a one-D object using the view function in PyTorch one alternative to the flatten layer which I really like is the global Max pooling layer. This is a little different from the regular kind of pooling we discussed earlier consider the question.

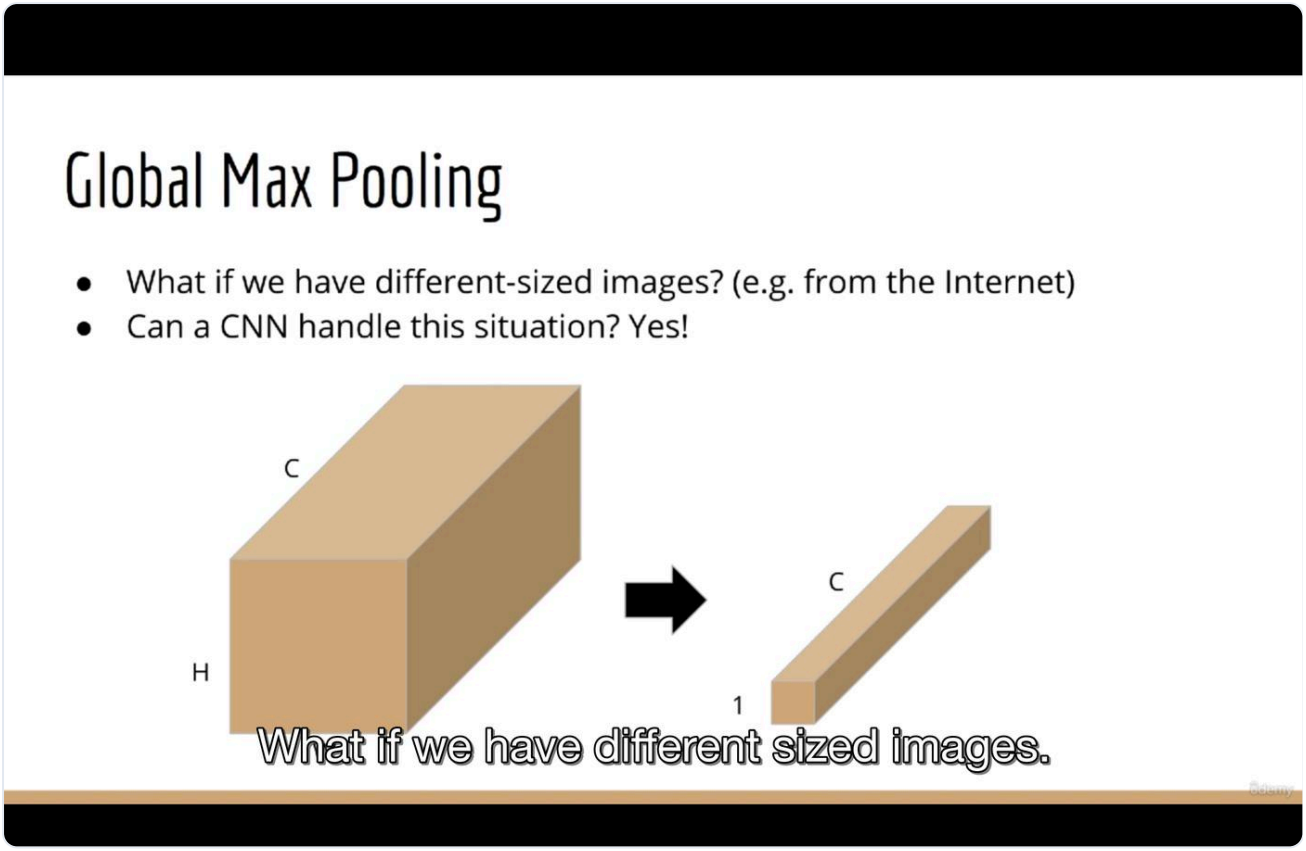


FIGURE 37.30 | Global Max Pooling

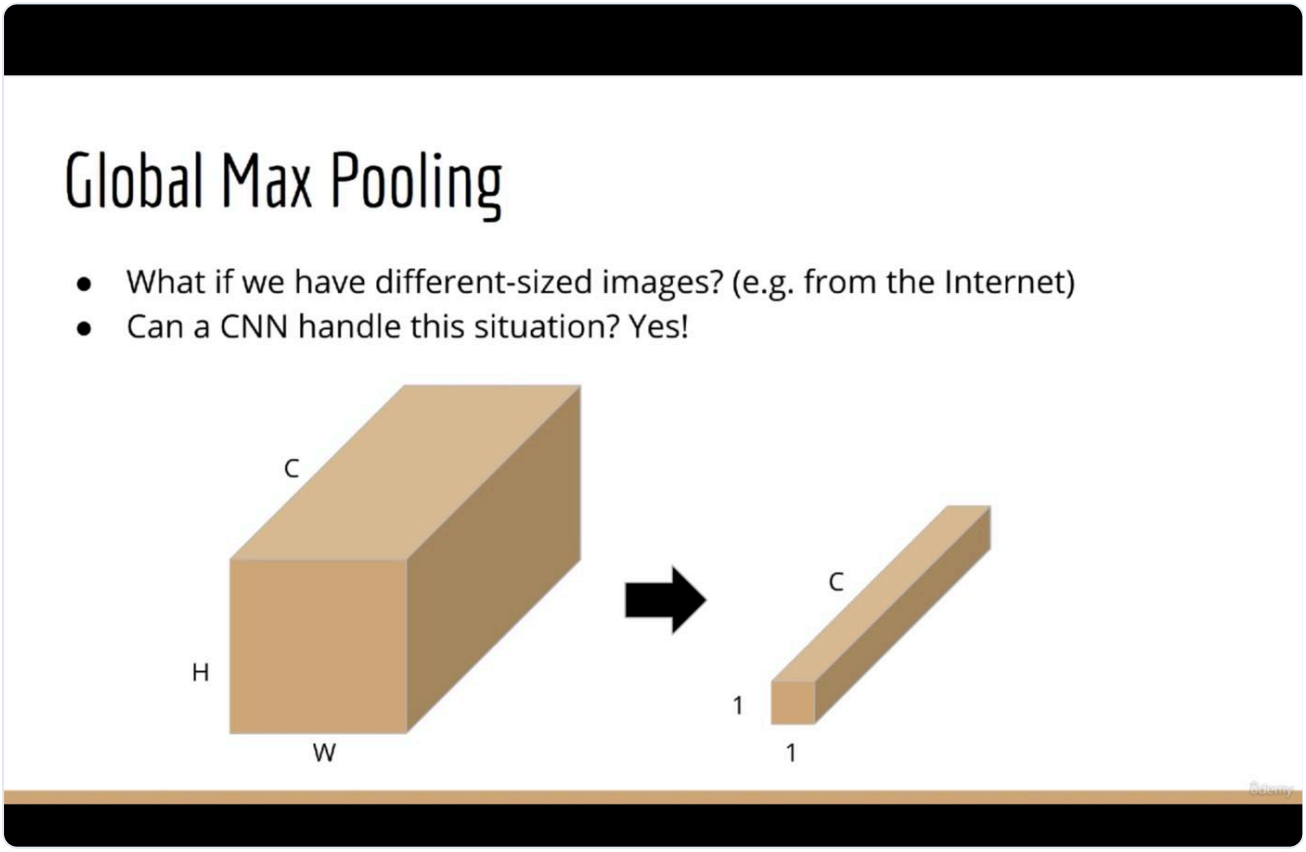


FIGURE 37.31 | Global Max Pooling

What if we have different sized images. The largest source of images is the internet. Generally speaking images we find on the Internet are not all the same size so it makes sense to wonder is it possible to build a convolutional neural network that is capable of handling different image sizes. And the answer is yes. Using a global Max pooling layer Let's consider first why a simple flatten layer would not work. Suppose again we have an input image of size 32 by 32 and four convolutions assume again we use convolution and same mode. And we have a stride of two. So after the first convolution We get a 16 by 16. After the second convolution We get an eight by eight.

After the third convolution We get a four by four. And after the fourth convolution We get a two by two. But now let's say we have an input image of size 64 by 64. In this case the output after four convolutions would be four by four. Let's say for simplicity's sake the number of output feature maps is one hundred then for the 32 by 32 image. Doing a flatten operation would yield an output of size two by two by one hundred two times two times 100 which is four hundred for a sixty four by sixty four image. Doing a flatten operation would yield an output size four times four times 100 which is sixteen hundred.

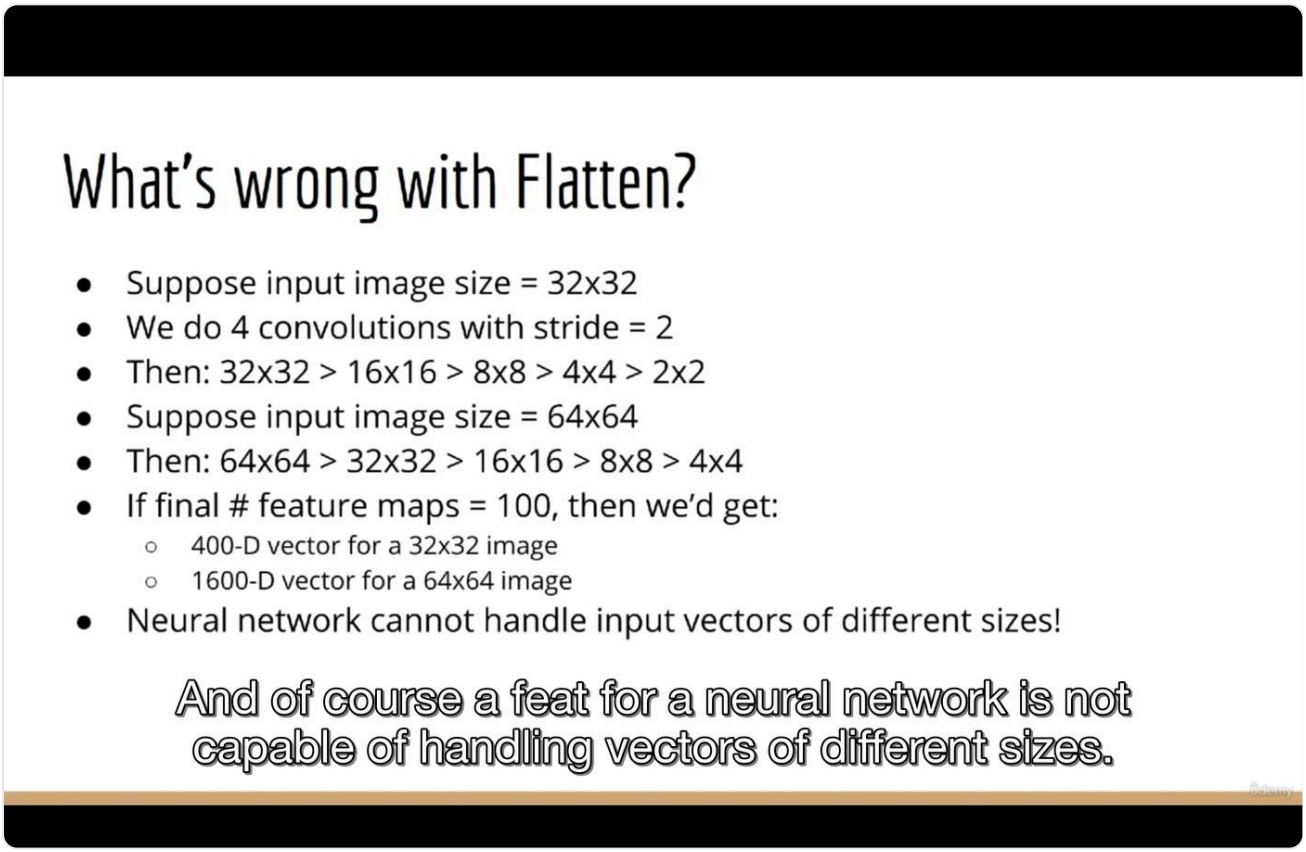


FIGURE 37.32

And of course a feedforward neural network is not capable of handling vectors of different sizes. Now let's consider how global max pooling works. The idea is simple if our input image is h by W. By C then the output is one by one by C or equivalently just a vector of size C.

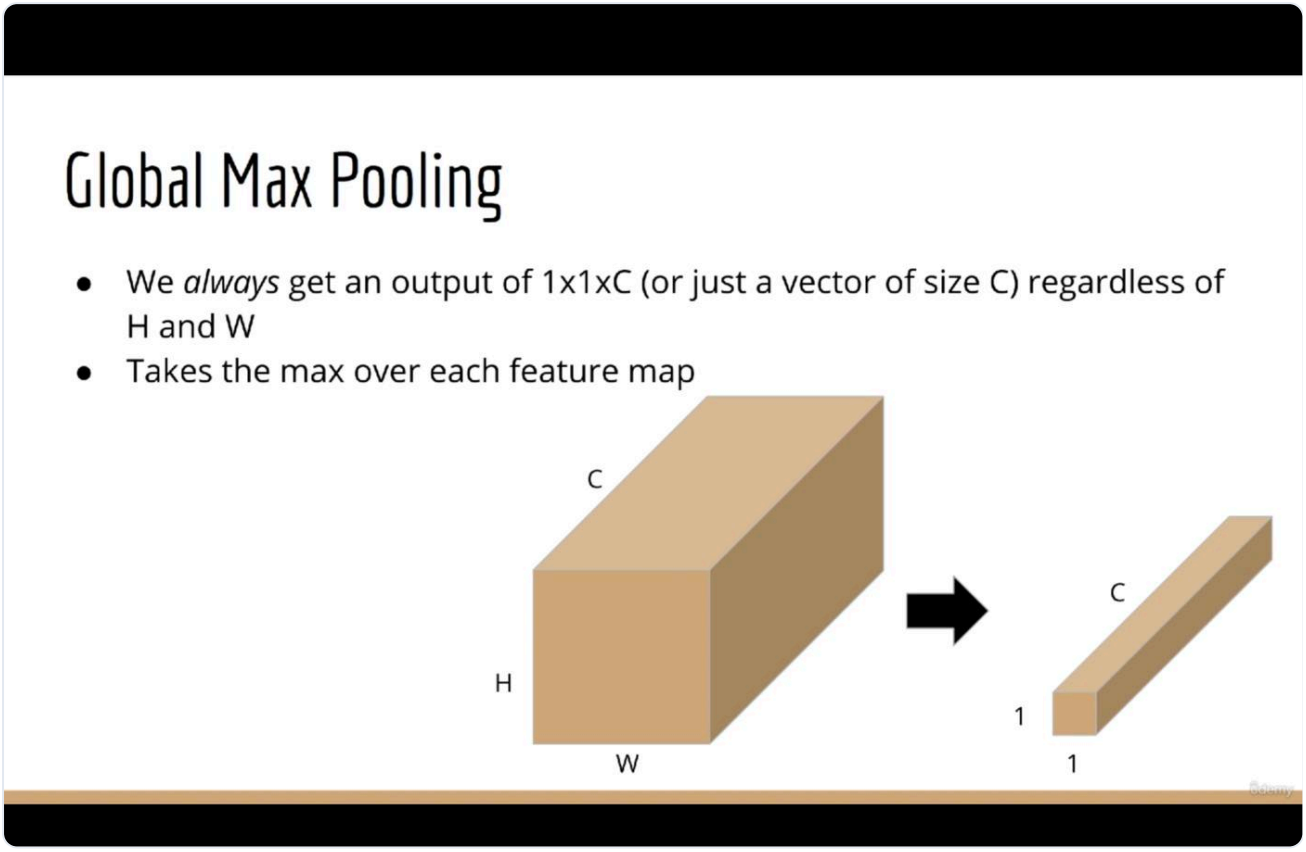


FIGURE 37.33 | Global Max Pooling

In other words it takes a max over the entire image along the spatial dimensions over each feature map. Since we have C feature maps we end up with what is essentially a one dimensional vector of size C since the 1 dimensions are redundant. This follows the same general idea of pooling where we're saying I don't care where in the image the feature was found just that it was found somewhere.

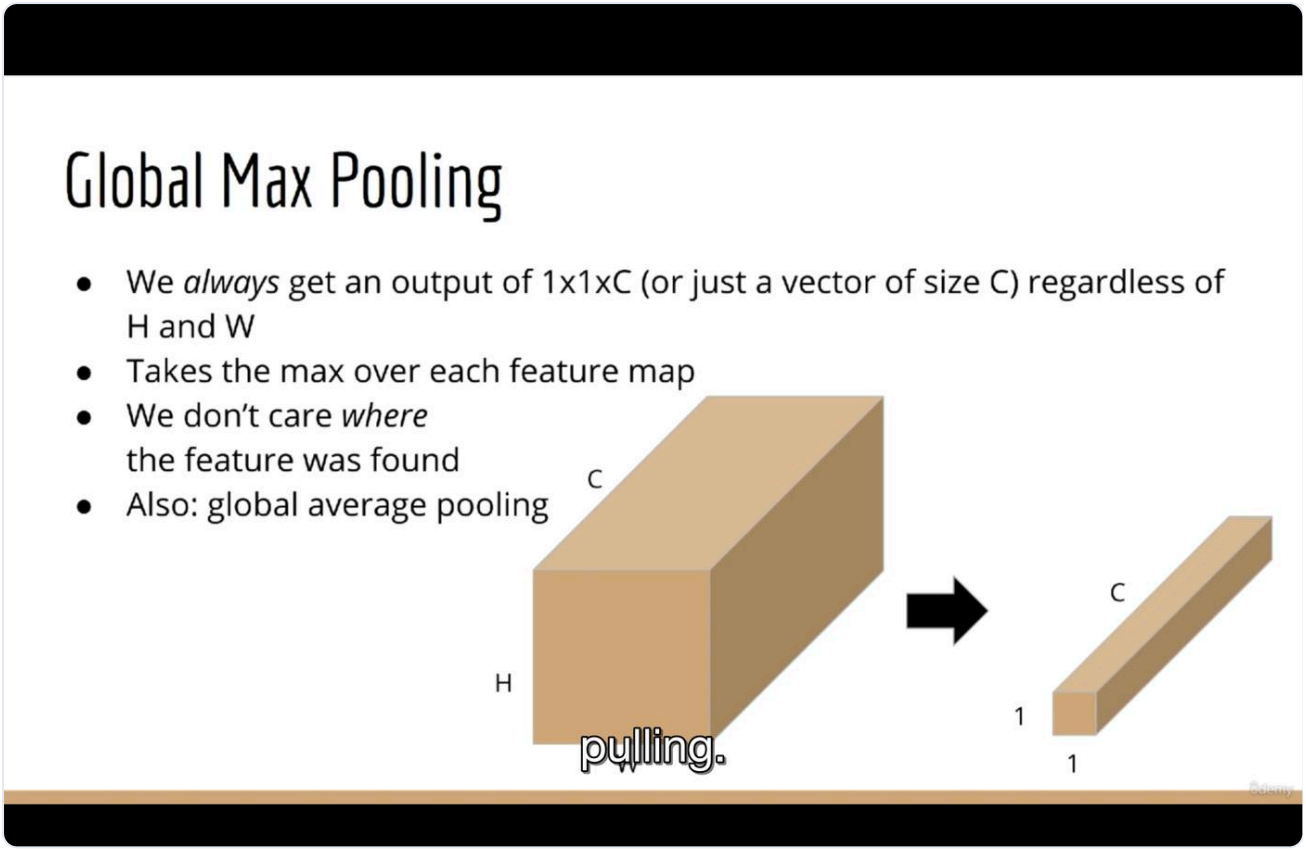


FIGURE 37.34 | Global Max Pooling

Global max pooling is the extreme of this saying that the feature can be found anywhere in the entire image across the entire height and width since the output is always a vector of size C it does not depend on the size of the image meaning that it allows the network to handle images of any size. As a side note we also have global average pooling which is just the global version of regular average pooling. The only exception to this is if your images are too small in which case you'll just get an error when you try to run your code. For example if your image starts out as a two by two then it's not possible to reduce the dimensions by half four times.

Exception

- If you pass in images that are too small, you will get an error
- E.g. if you start with a 2x2 image, you cannot halve it 4 times
- You'll encounter this if you try to add too many conv layers to your CNN

You'll notice this happens if you do something like add too many convolutions to your neural network.

FIGURE 37.35

Summary

Step #1:

- Conv > Pool > Conv > Pool > ...
- Strided Conv > Strided Conv > ...

Step #2:

- Flatten: x.view(-1, shape)
- Global max pool: torch.max(x, dims)

Step #3:

- Dense > Dense > ...

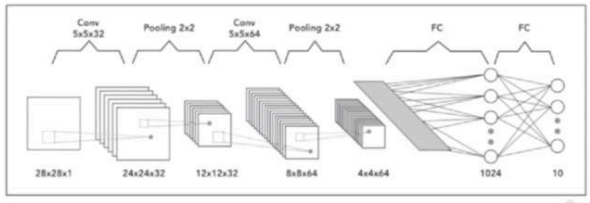


FIGURE 37.36

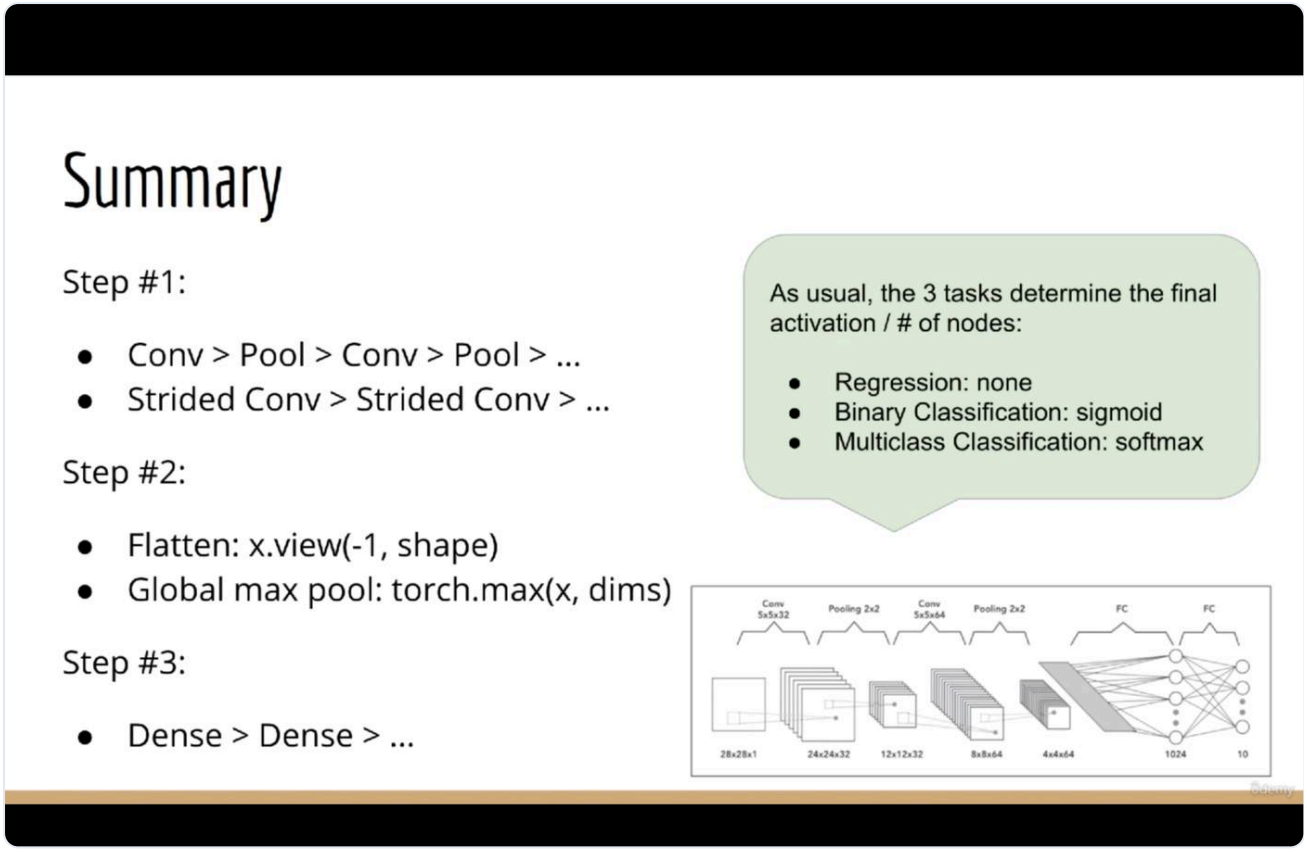


FIGURE 37.37

You'll notice this happens if you do something like add too many convolutions to your neural network. All right so let's summarize the basic architecture of a modern convolutional neural network where the first stage is a series of stride convolutions or optionally convolution followed by pooling and repeating that pattern in the interface between convolution layers and dense layers. We have a flatten. We also optionally have a global Max pooling that reduces the image to just the max for each feature map. Finally once we flattened our data into a one dimensional vector we have a regular feedforward neural network made up of a series of dense layers as usual and as you learned in the previous section the activation and number of nodes for the final layer depends on the type of task being done.

So if you're doing scalar regression then you'll only have one output node with no activation function. If you're doing binary classification you could have one output node with a sigmoid. However for k equals two or more classes you can have K output nodes with a softmax activation.

LECTURE 38

Why Use Custom Models?

In this lecture, we are going to begin discussing code preparation in preparation for our CNN notebook, this lecture will focus specifically on how we're going to build a model since as we move to more and more complicated architectures, we'll need more and more sophisticated ways of building models. There will be two major components of this lecture. First, as you know, the critical new piece of math that makes an ANN into a CNN is convolution. So we'll see how to use the convolution module in. Second, we'll need to integrate this convolution module into the neural network. I'm going to show you two ways of doing this, but the highlight will be the object oriented method of building PyTorch models.

So if you don't remember your object oriented programming from your first year programming class, now would be the time to crack open your old textbook and reacquaint yourself with those concepts.

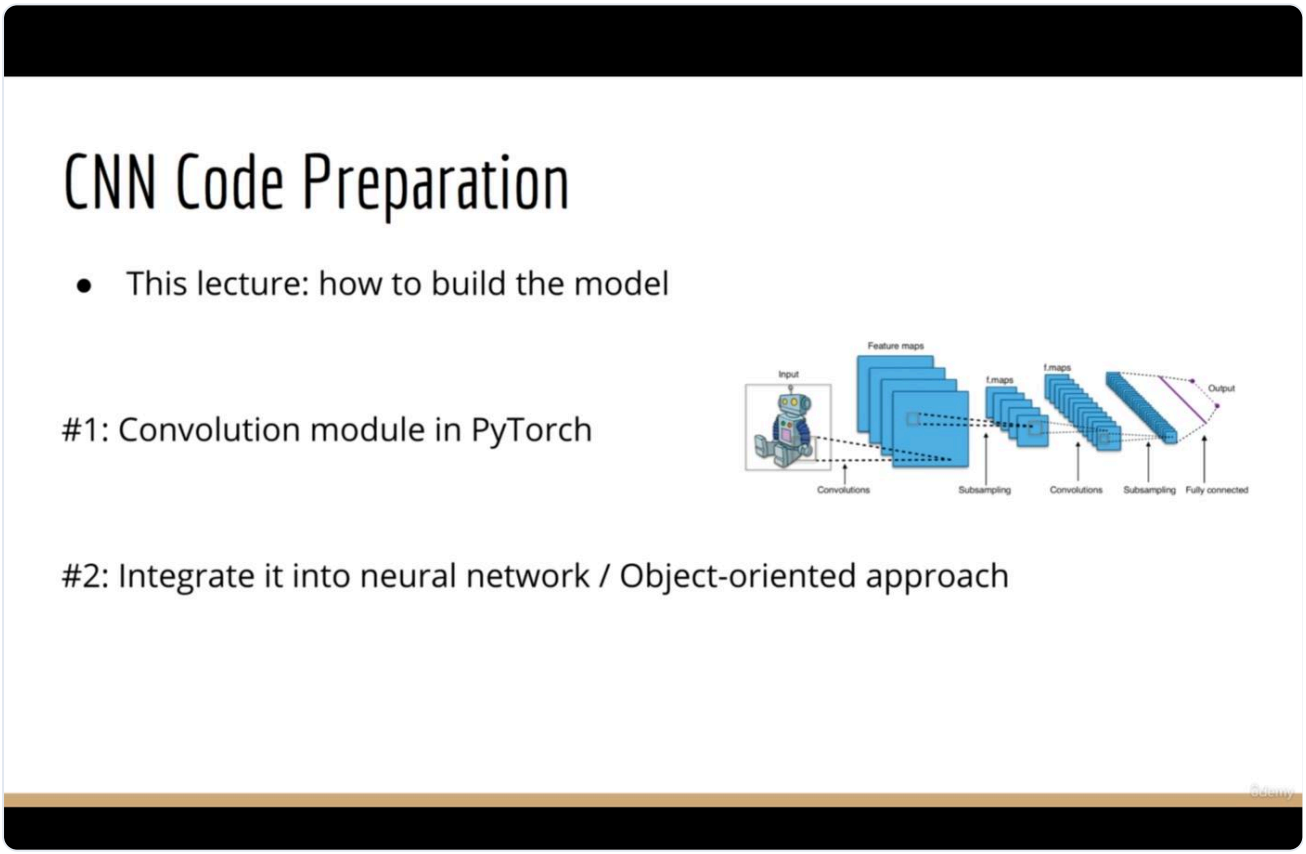


FIGURE 38.1 | CNN Code Preparation

So, number one, we want to look at the convolution module, the module we need is called Conv2d. And on this slide, what I want to do is highlight its arguments and relate them to the math that we already learned. As you know, the main parameter in convolution is the filter or the kernel, which is a four dimensional tensor composed of the number of input channels, the number of output channels and the kernel height and width. Recall that the number of channels is

also known as the number of feature maps. In case you're using different terminology. Because we usually have the kernel height equal to the kernel width we can just specify it as a single number, which we usually do, OK, when we talk about it in math.

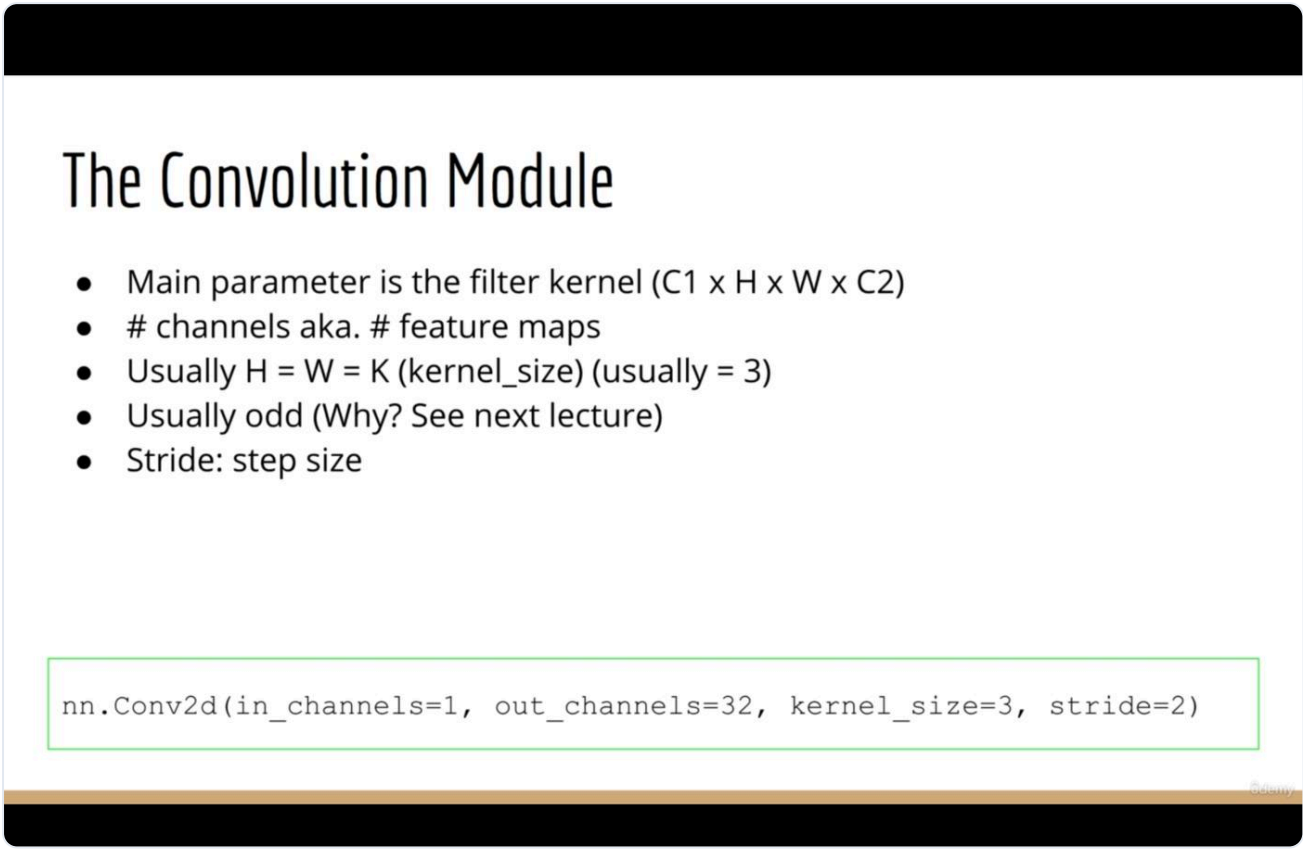


FIGURE 38.2 | The Convolution Module

If you wanted to have your kernel height different from your kernel width, you could also specify it as a tuple. In modern, deep learning, most of the time, you'll see kernels of size three, although in the old days you'd also see kernels of size seven and kernels of size five. Notice how the size is typically odd. You'll be able to deduce why that is the case after the next lecture. So that takes care of the first three arguments. The last relevant argument for us is the stride which controls the step size when you move the kernel across the input image. So why do we use Conv2d and not Conv1d or Conv3d, as you know, color images are three dimensional, height by width by color?

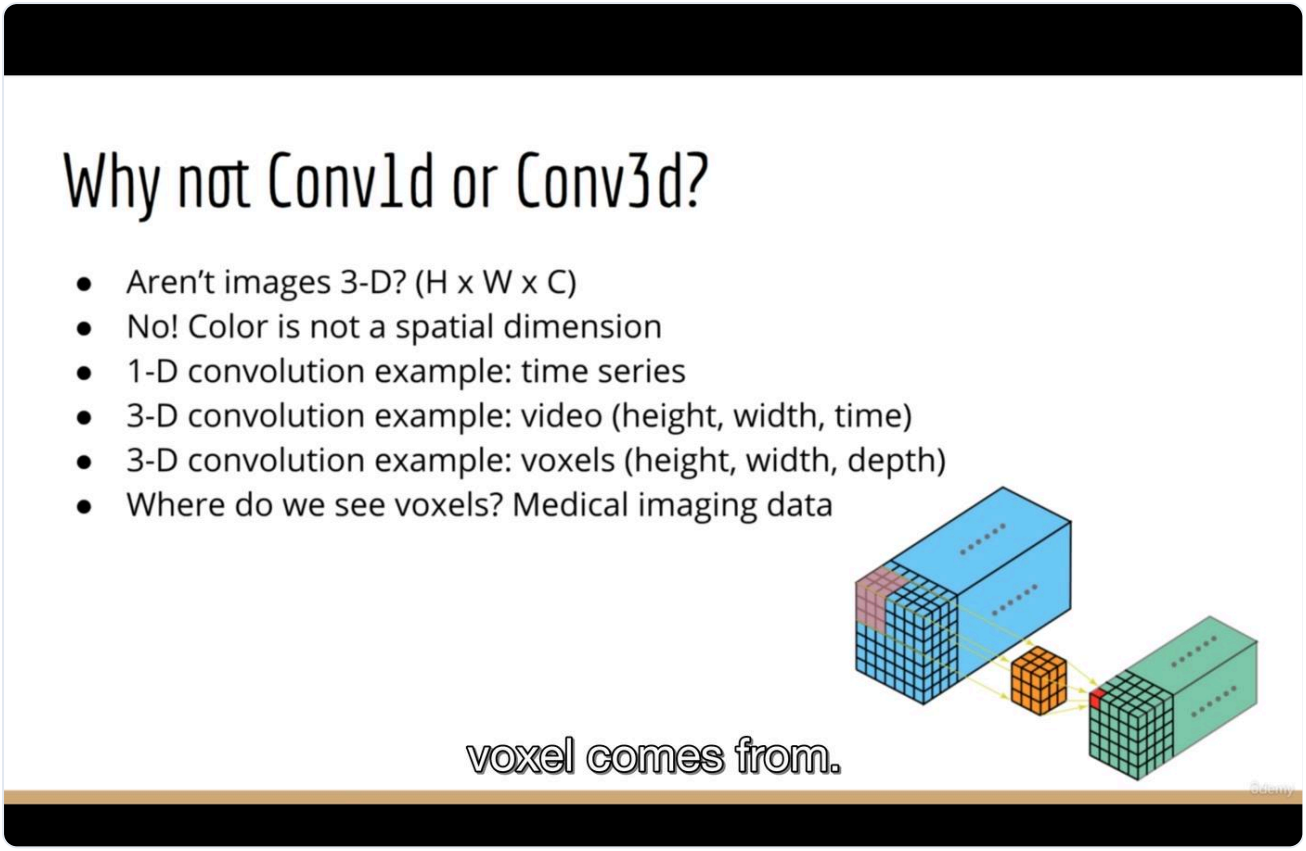


FIGURE 38.3

Well, it is 2D because of the color dimension of an image is not a spatial dimension. The signal doesn't flow in that direction. It's also possible to have 1D convolutions and 3D convolutions using the Conv1d and the Conv3d layers, respectively. So, for example, if you're looking at a signal that varies with time, that would be a 1D convolution. If you're looking at a video, for example, that would be a 3D convolution. That's because height and width are the first two spatial dimensions and time is a third dimension. It's also possible to have objects that have three spatial dimensions, height width and depth instead of pixels. We have voxels. You might encounter these.

If you look at something like medical imaging data, by the way, you might be wondering where the word voxel comes from. Well, it's pretty obvious that it's analogous to the word pixel by pixel is actually a short form of the phrase picture element, and thus voxel is the short form of the phrase volume element in any case.

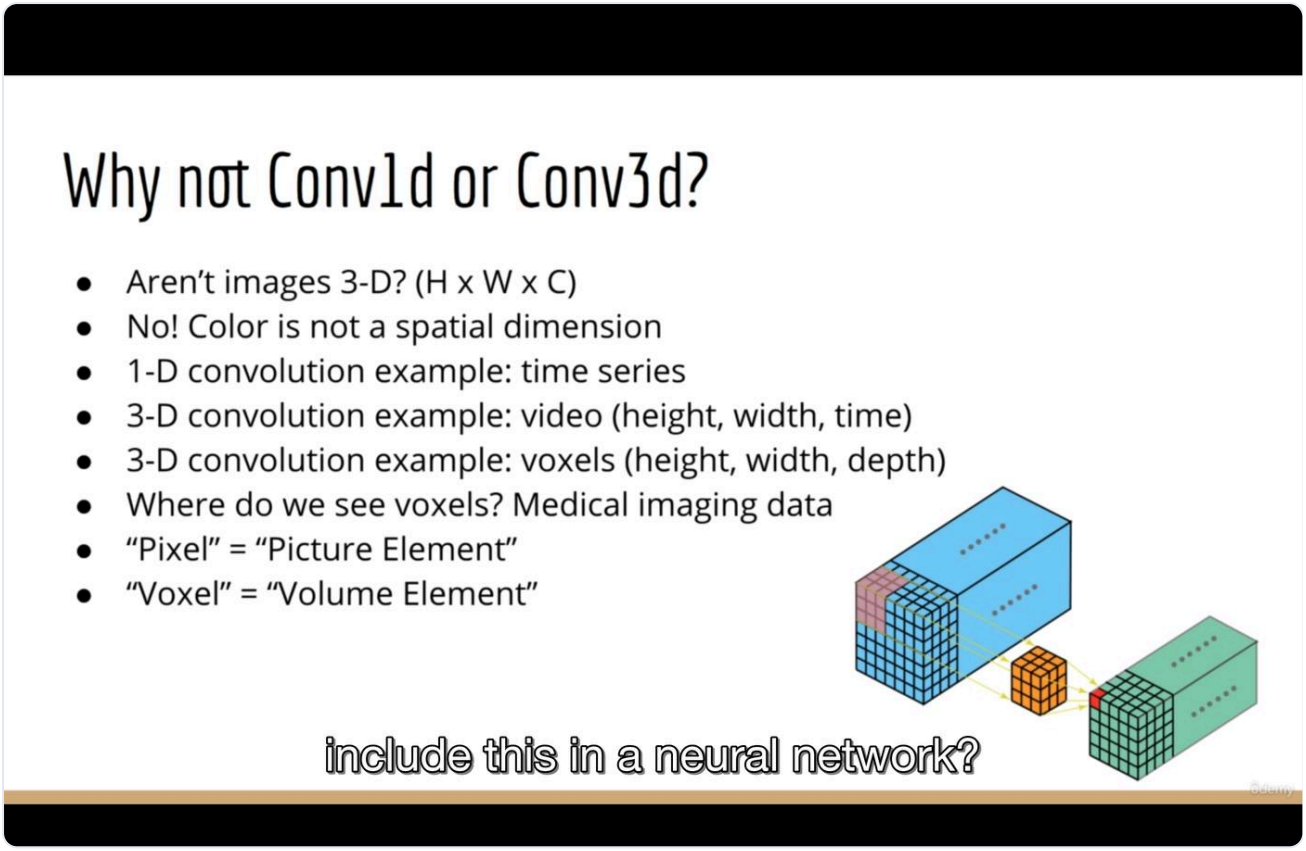


FIGURE 38.4

Now we can say that we fully understand the Conv2d layer so we can move on to asking how do we include this in a neural network? Let's now talk about the concept of inheritance. PyTorch models are all based on modules, and I've probably used that word a few times throughout the course without really explaining what they are.

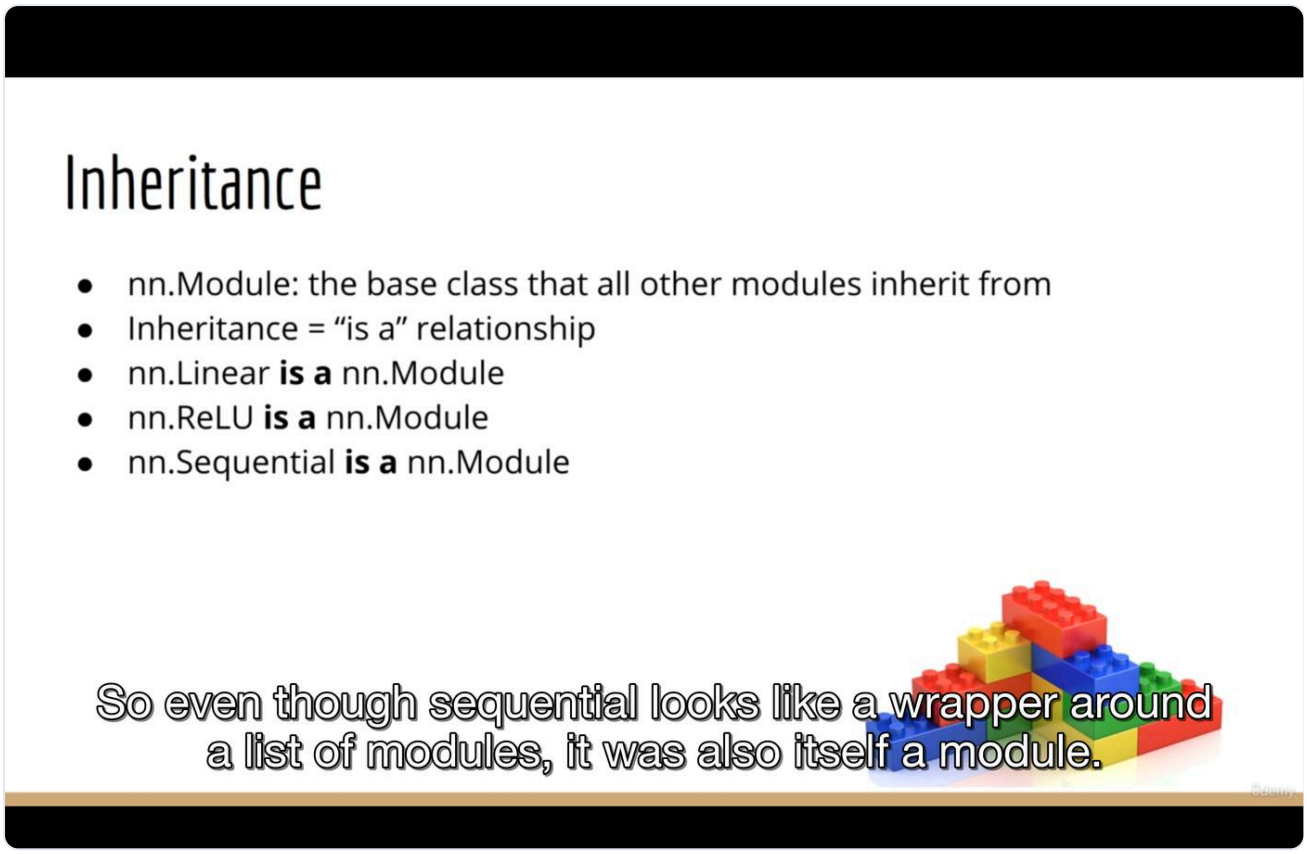


FIGURE 38.5

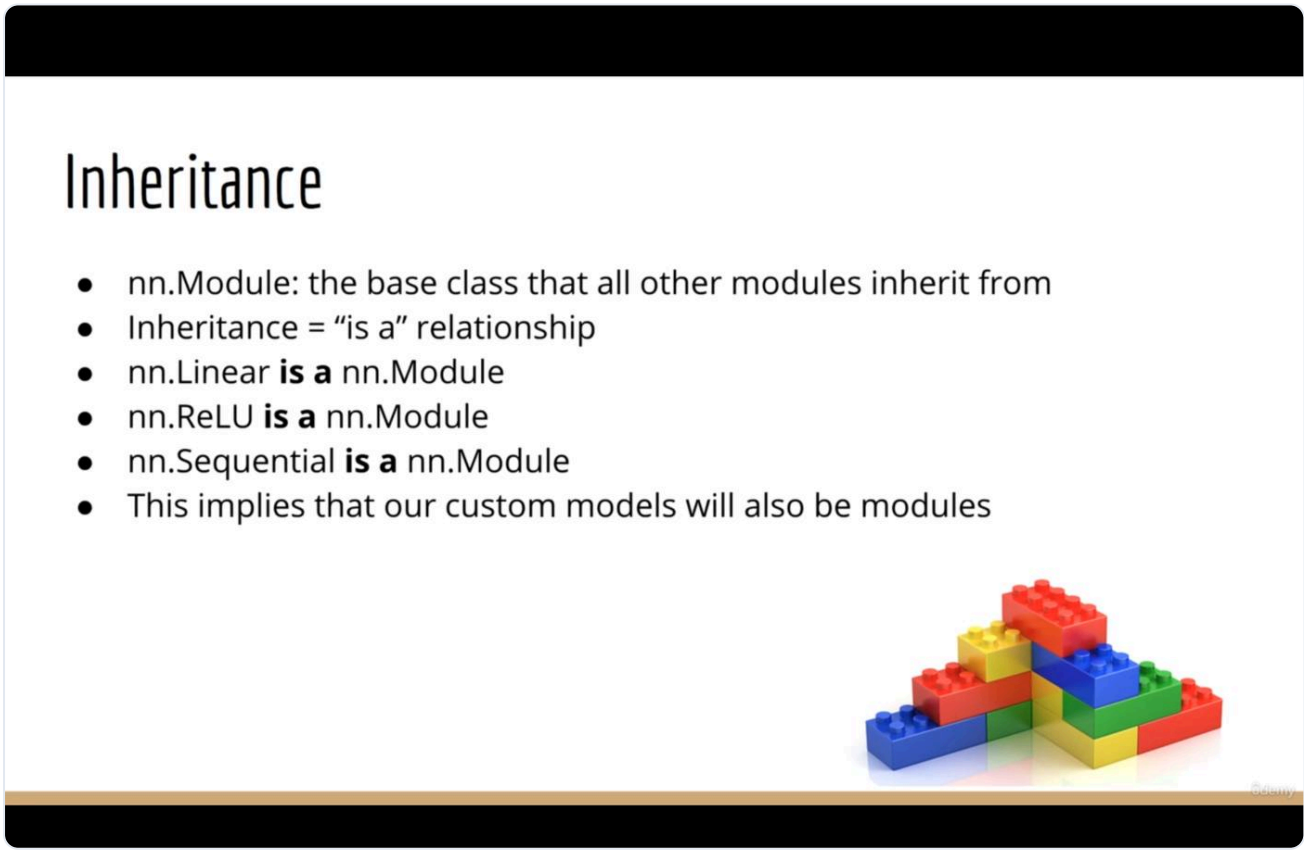


FIGURE 38.6

Well, a module is the base class that all other modules inherit from inheritance implies an is a relationship. For example, the linear module, which is the first one we saw in this course, inherits from the module class. That means it is a module. Similarly, the ReLU module is also a module. Similarly, the sequential module is also a module. This might strike you as surprising because these three things are kind of very different from each other. So even though sequential looks like a wrapper around a list of modules, it was also itself a module. So when you combine a bunch of modules to build a model, that model is itself a module. This implies that when we create our own custom models, they are also going to be modules and we are going to inherit from the module class.

So when I said earlier that PyTorch is all about composing models from building blocks, well, modules are those building blocks. One interesting consequence of everything being a module and models being modules is that there this hierarchical structure, the first model we saw in this course was just a linear module. Then we saw that we could stack linear layers together in a sequential object, which is a module of modules. That's also a model. Later, you'll see that we can repeat this pattern so that our model is a module of modules of modules. So what we've learned is that a module is a model, but a module of modules can also be a model, as can a module of modules of modules and so on.

By taking the perspective that everything is a module building complex, things becomes easier because you only have to look at your current level of abstraction.

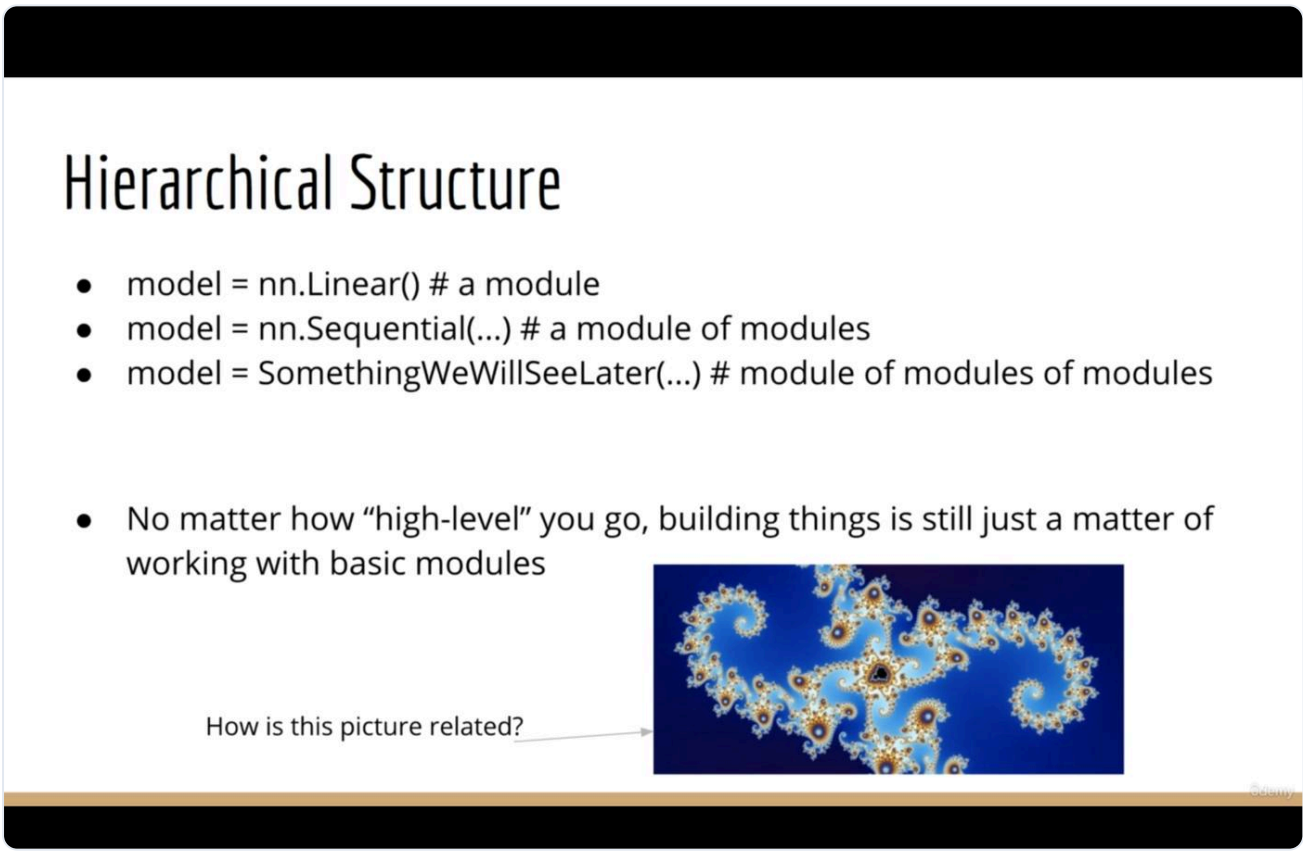


FIGURE 38.7 | Hierarchical Structure

And no matter how high level that may be, you're still just working with modules. So why might we want to build a custom model instead of just doing what we've been doing so far?

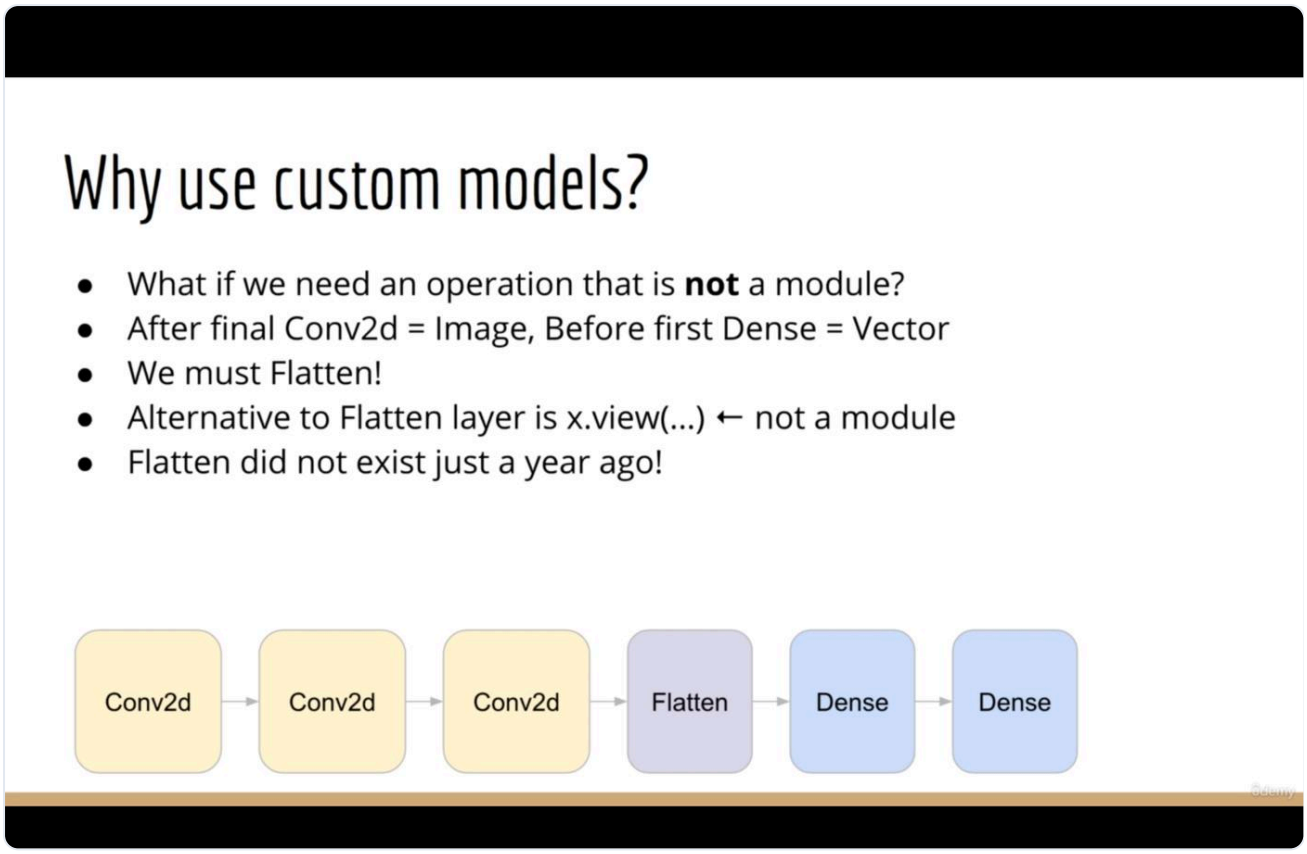


FIGURE 38.8 | Why Use Custom Models?

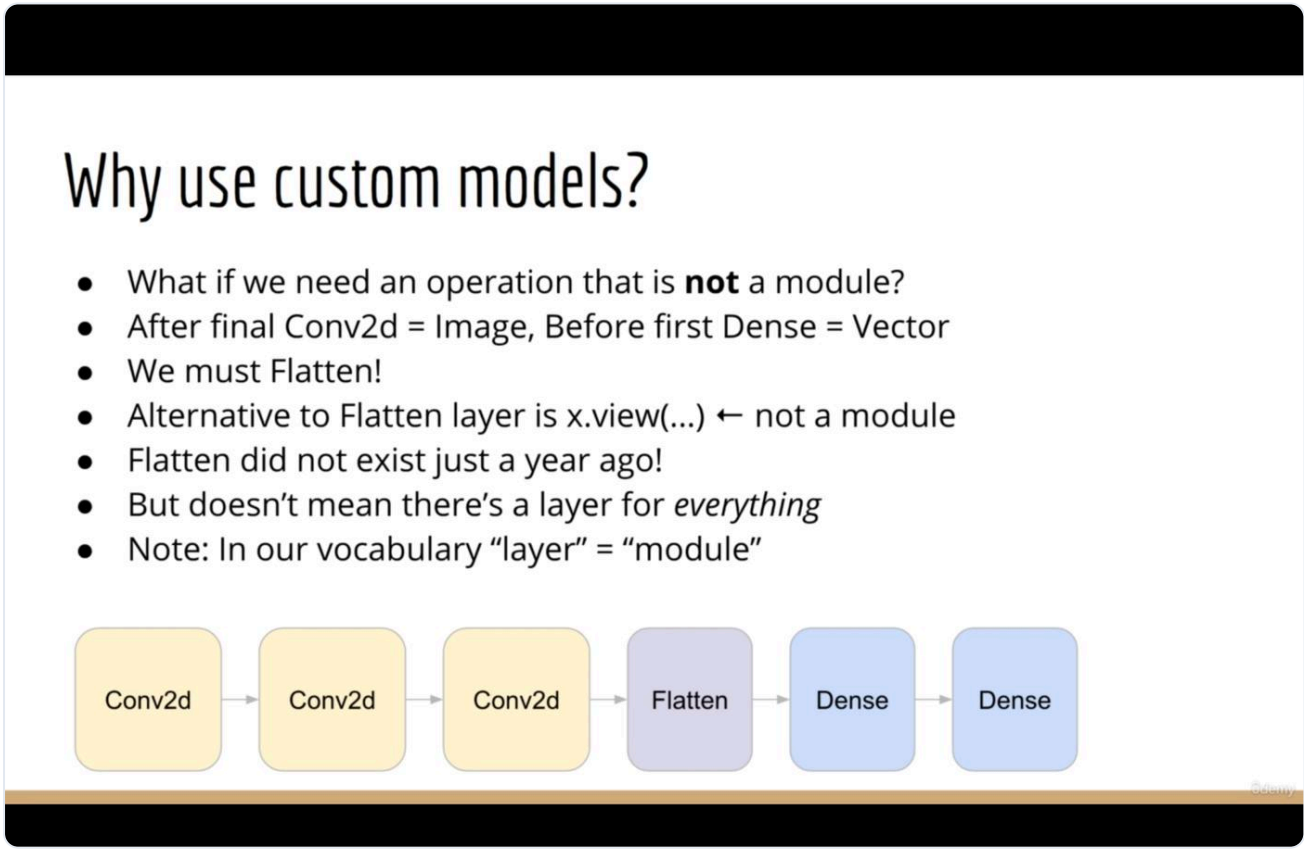


FIGURE 38.9 | Why Use Custom Models?

Well, sometimes we need to do an operation that is not a module. For example, you know that the interface between the convolution layers and the dense layers in a neural network is a flatten. So after all the convolutions and before all the dense layers, we must flatten the data so that it fits into those respective layers at the boundary between these two different kinds of layers. The output of a convolution is an image, while a dense layer expects an input vector. Now, although PyTorch does have a flatten layer, you might wonder what if it didn't exist or what if some other operation you wanted to do it didn't exist. In that case, you could build a custom model in the case of flattened.

We know that we can always just use the view function to reshape an image back into a batch of vectors. In fact, not even one year ago, counting from the time this course was being made, there was no flatten layer in PyTorch.

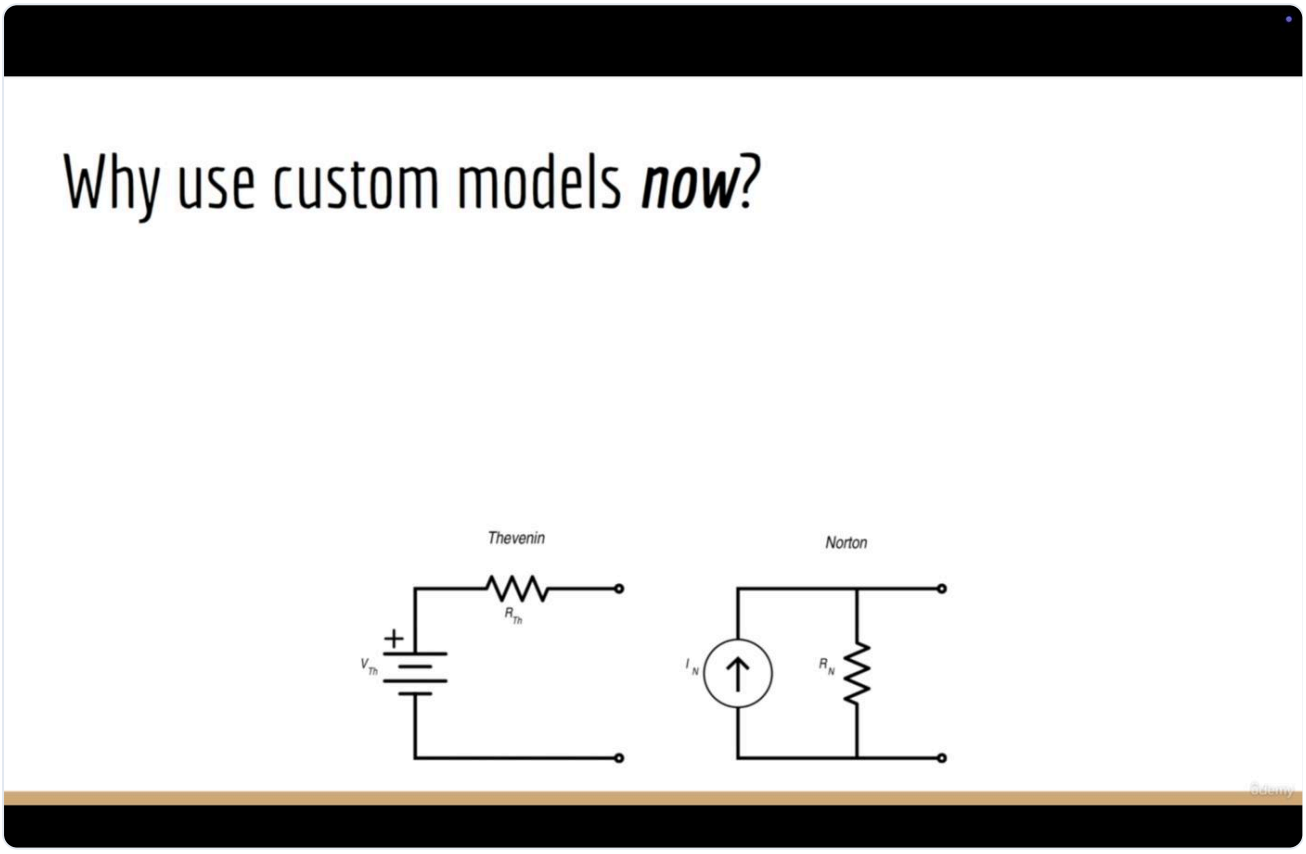


FIGURE 38.10 | Why Use Custom Models Now?

So you had to build a custom model. So you're lucky in that sense today, a flattened layer exists, which makes things easier for you, but at the same time there are still many things you might want to do, where there does not exist a prebuilt layer for you. As a side note, I commonly referred to these objects as layers, even though in PyTorch they are technically called modules. Layer is a better visual term in my opinion, since a neural network is made of layers. It's just easier to understand and see it in your mind. So please don't get confused when I use the words layer and module interchangeably in the context of PyTorch. In any case, I hope it's clear why we need custom models when we have all the layers or modules we need and they all occur sequentially in our model, which is not always the case, by the way, then we can wrap them up in a sequential, as we've been doing so far.

But if we want to do something more complex, then we need to build a custom model that inherits from the module class.

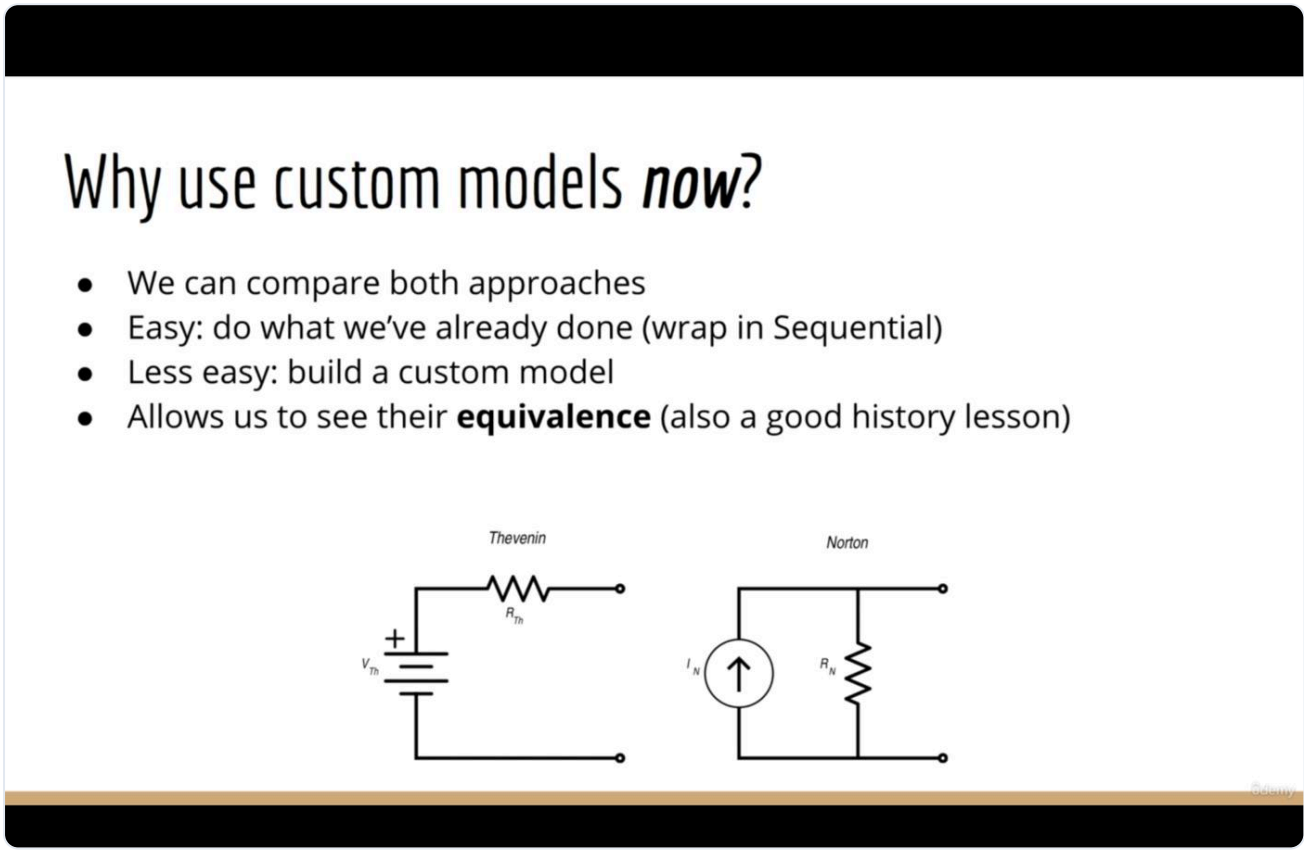


FIGURE 38.11 | Why Use Custom Models Now?

Introducing this idea early in the context of CNNs is nice because it allows you to compare both approaches, the easy approach is to do what we've already been doing, which is to build a sequential model that wraps up all the layers we want to apply in sequential order. The less easy approach is to build a custom model, but doing it both ways allows you to see the equivalence between the two and hopefully that makes building custom models less intimidating than it would be otherwise. In addition, it's also a nice history lesson because less than just one year ago, you wouldn't have had this luxury. OK, so let's start by looking at a simple model we've already seen, and again, just to recap the simplest CNN we can make looks like this.

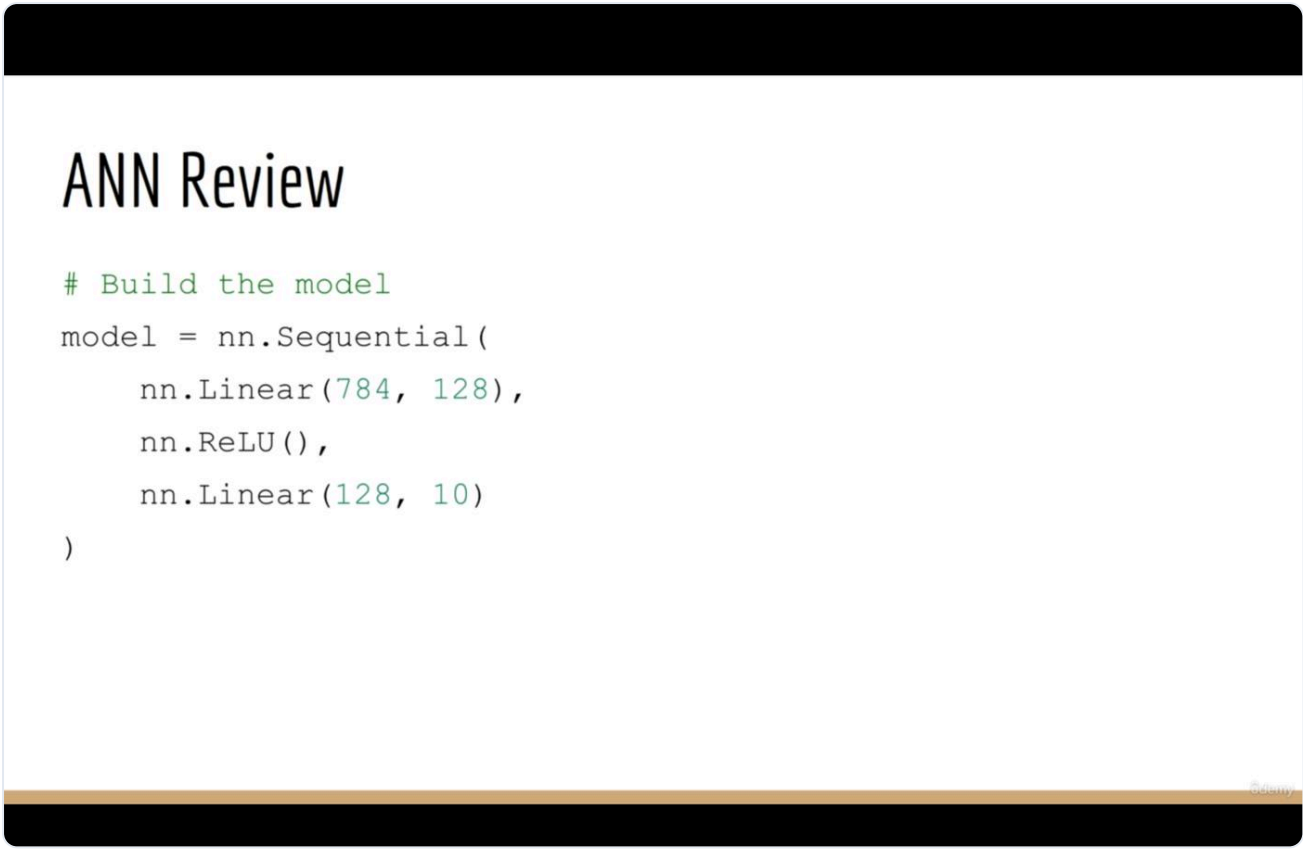


FIGURE 38.12

We have a linear layer followed by a ReLU, followed by another linear layer. No final activation function is needed, as you know. So what does this look like if we wanted to build a custom model? So first, we define a class that inherits from nn.Module inside the constructor, we define all of our sub modules or layers.

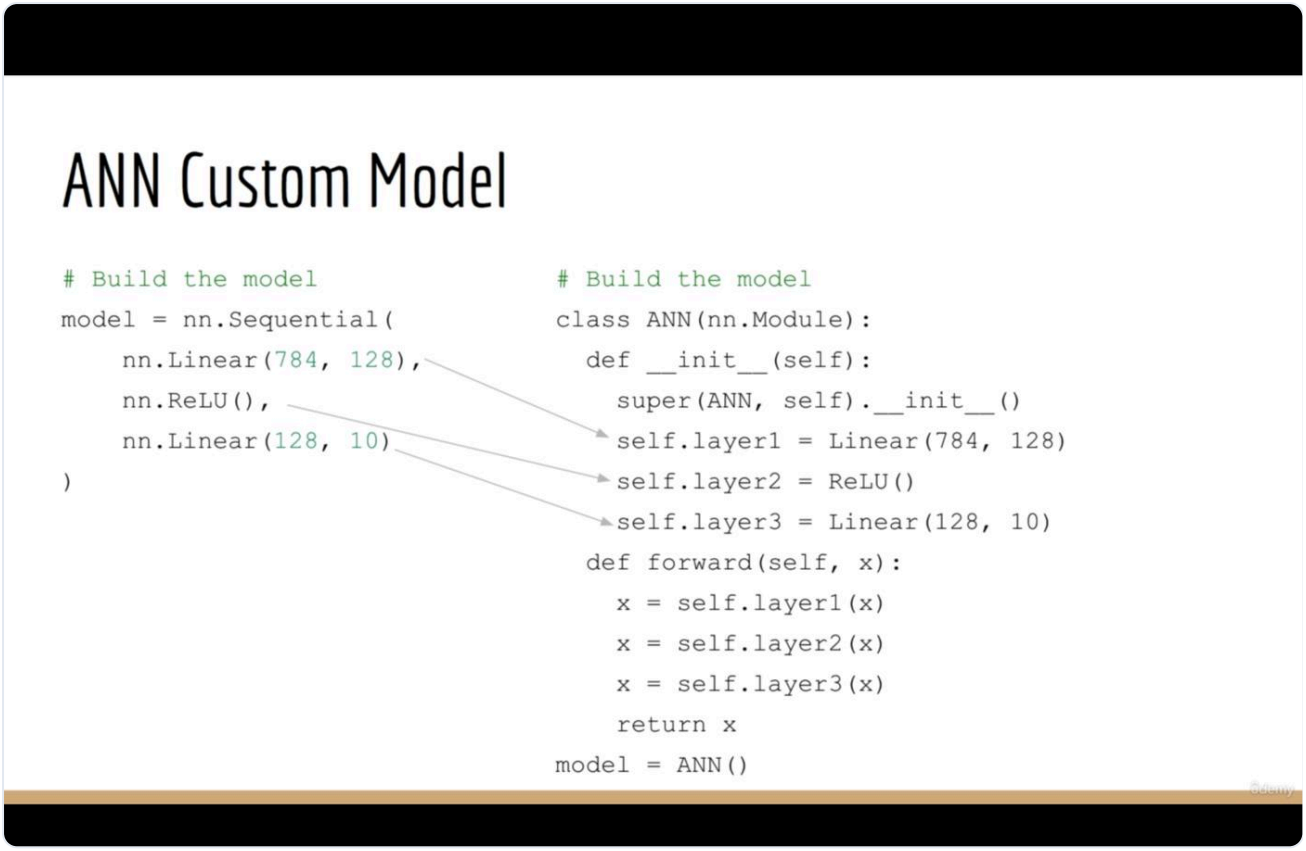


FIGURE 38.13

These are the two linear layers and the ReLU. You notice that all the layers in the sequential model are still defined in the constructor of the custom model. We assign these to instance variables so that they can be accessed from other methods, specifically the forward function, as you can see, the forward function is where we actually apply the operations defined by the layers we created. This is just like the simple linear regression model, when we create the linear regression model, we say variable equals nn.Linear.

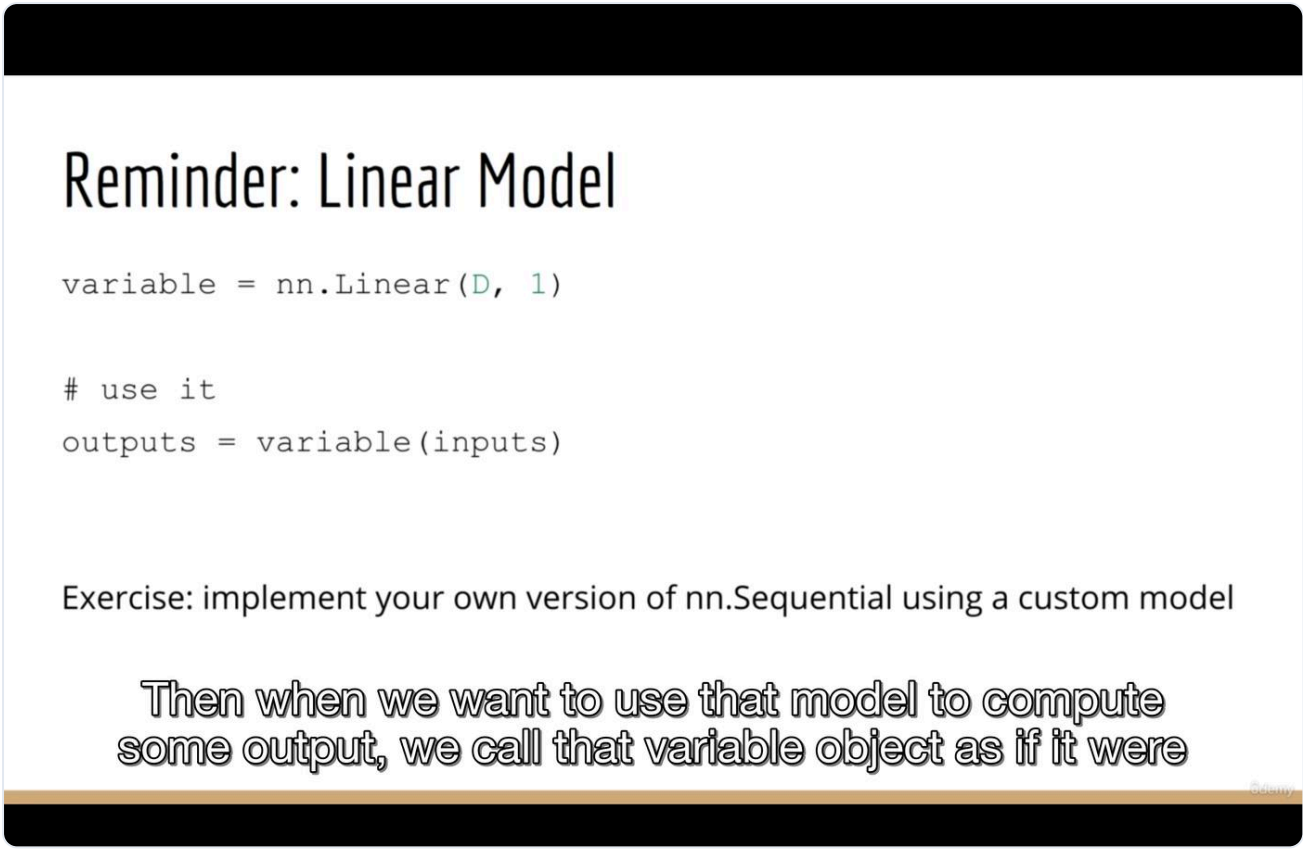


FIGURE 38.14

Then when we want to use that model to compute some output, we call that variable object as if it were a function. Similarly, inside the forward function, when we want to compute the output of each of those layers. We simply call the module objects we created in the constructor. As an exercise, I would recommend trying to implement your own version of the sequential object using what you just learned, this would be a great exercise to test your understanding of module inheritance and building custom models.

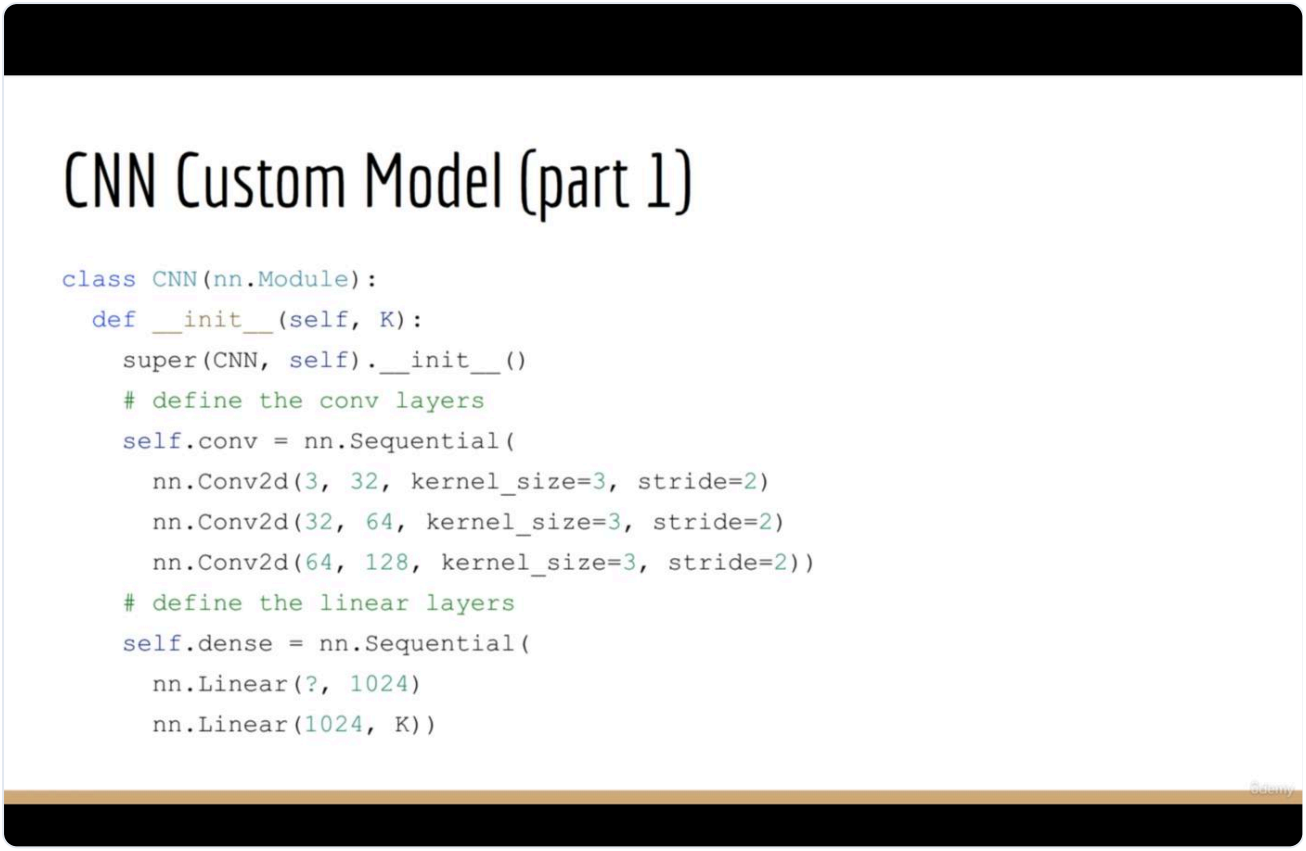


FIGURE 38.15 | CNN Custom Model (Part 1)

So please give it a try. So at this point, I hope it's easy to see the correspondence between a sequential model and a custom model that inherits from the module class. The next thing we're going to do is apply what we learned to build a more complex model, specifically a convolutional neural network.

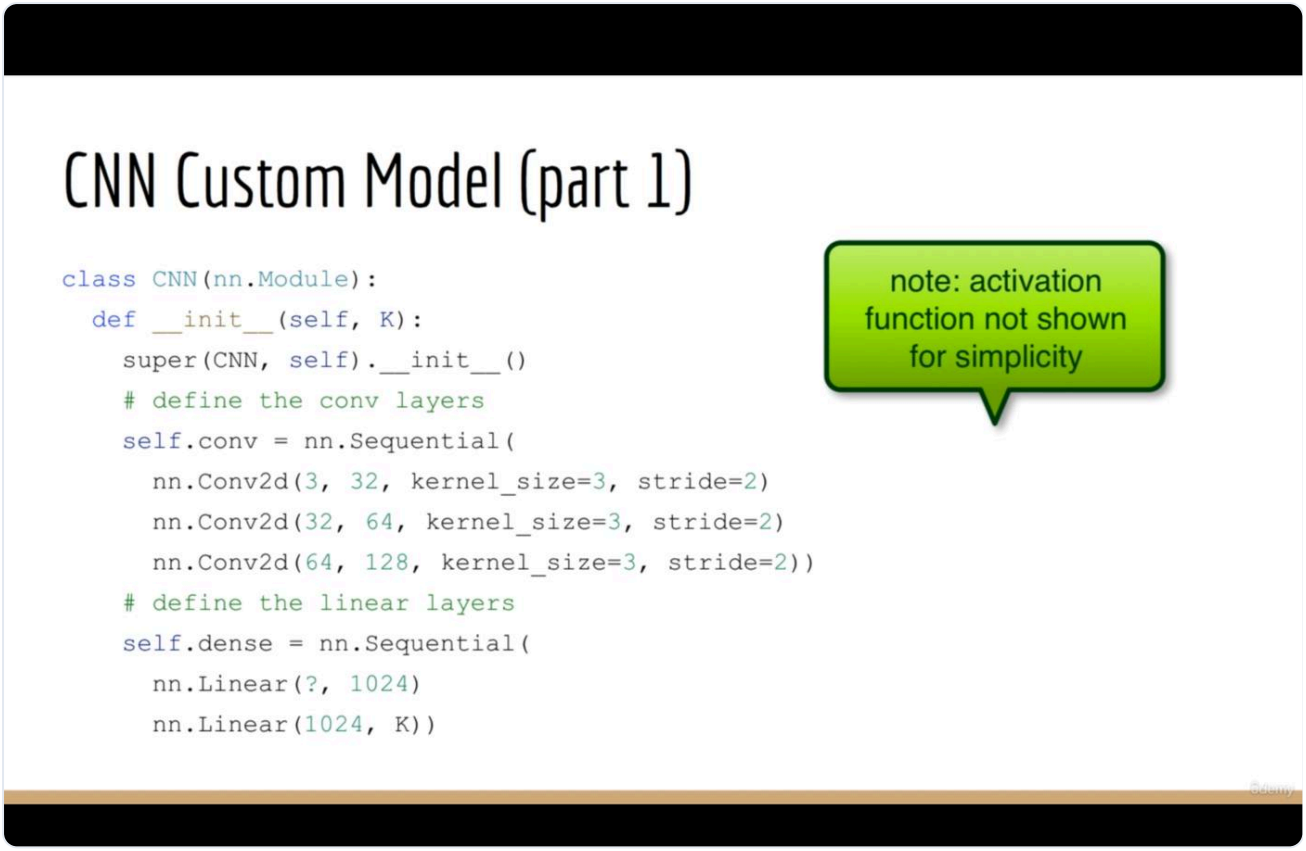


FIGURE 38.16 | CNN Custom Model (Part 1)

As you can see, it's pretty much the same thing just with more layers and different kinds of layers, as we discussed in the three lectures. What we have is a sequence of convolution layers, followed by a sequence of dense layers, conveniently because everything in PyTorch is a module. We can wrap all the convolution layers in a single sequential object and all the dense layers in a second sequential object. That will make it easier to call each of the layers from inside the forward function instead of having to call each of the layers one at a time, since there might be hundreds of layers, we can just make a few calls to the sequential objects in addition and note the view function in between the two sequentials.

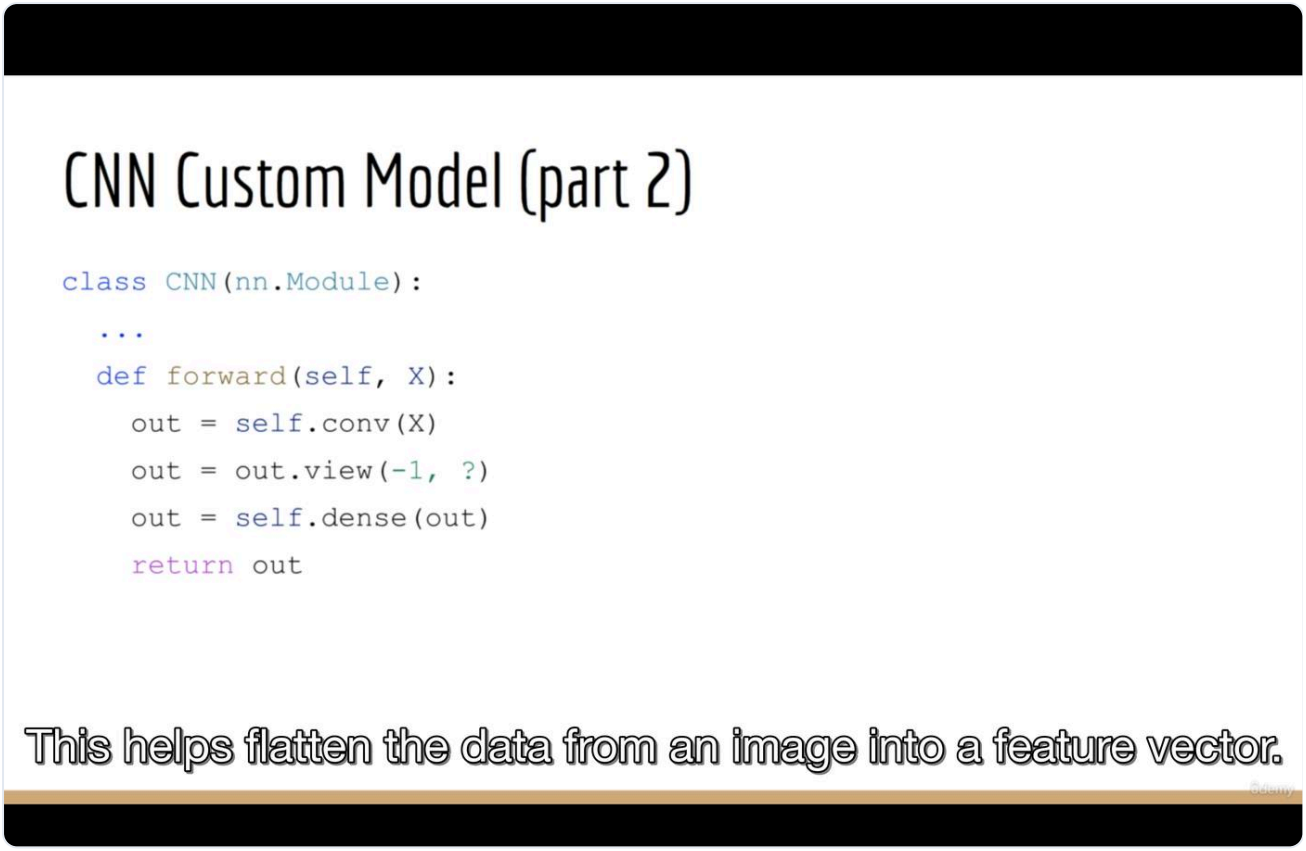


FIGURE 38.17 | CNN Custom Model (Part 2)

This helps flatten the data from an image into a feature vector. You realize something strange about the previous code, which is that I didn't specify the input size for the first linear layer, to be clear, this is not correct PyTorch syntax. I'm using this as a placeholder until I explain these concepts in a future lecture. Remember that in PyTorch there is a level of redundancy when specifying input, size and output size of each layer, the input size of one layer must match the output size of the incoming layer. But in the case of convolution, how do we calculate the size of the first linear layer's input? The answer is we'll have to manually calculate the size of the image as it passes through each convolution.

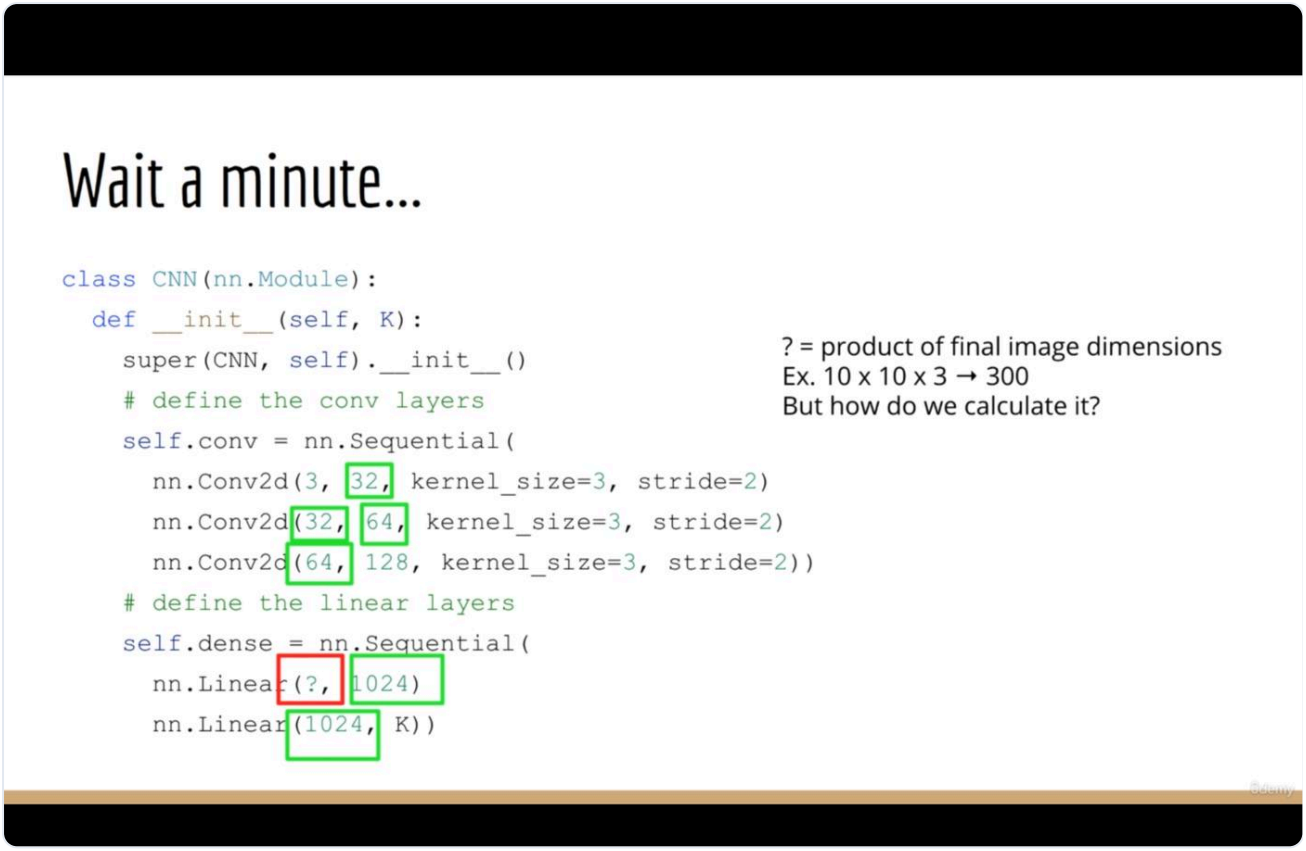


FIGURE 38.18 | But How Do We Calculate It?

Then when we flatten the final image, the feature vector dimensionality will simply be the product of each of the dimensions of the image. For example, ten by ten by three would become 300. It's a contrast with our custom model. I also want to show you how we would build a CNN using the newly created flatten layer, which allows us to make use of the sequential module. It's pretty easy to see that all the same layers appear. And the only thing that's different is that instead of explicitly calling the view function, we can use a flatten layer instead. There is, however, still this small issue of how we calculate the initial input size for the first linear layer.

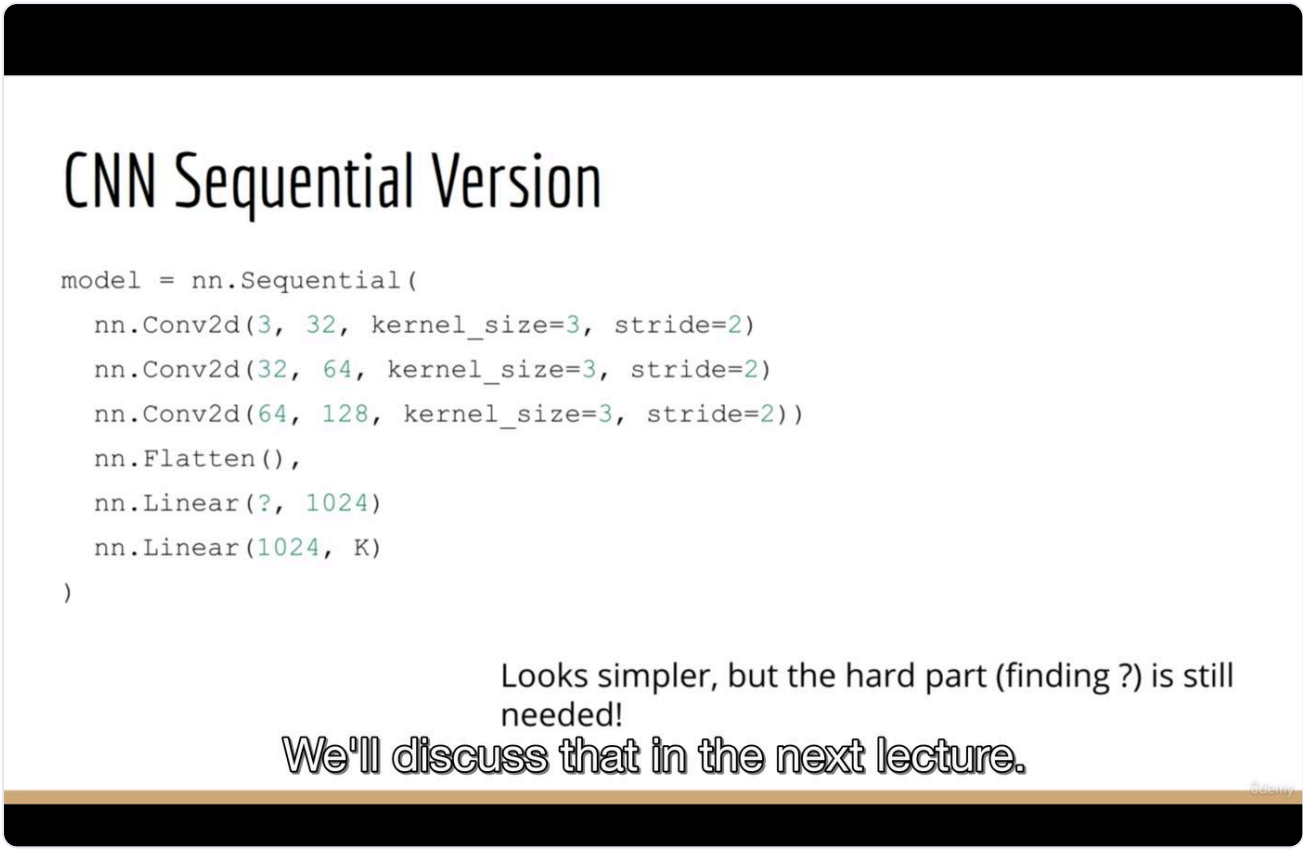


FIGURE 38.19 | CNN Sequential Version

We'll discuss that in the next lecture. So just hold that thought for now. So what you may have assumed earlier is that building custom models is hard, so maybe it's just better to use the sequential method for CNN, but in fact, they are nearly equal. In fact, the hardest part is figuring out this question mark. What is the initial input size of the first linear layer? And you can see that either way, whether you build a custom model or just use sequential, you still have to do this work. I want to mention one small addition we sometimes make use of in convolutional neural networks, which is dropout regularisation.

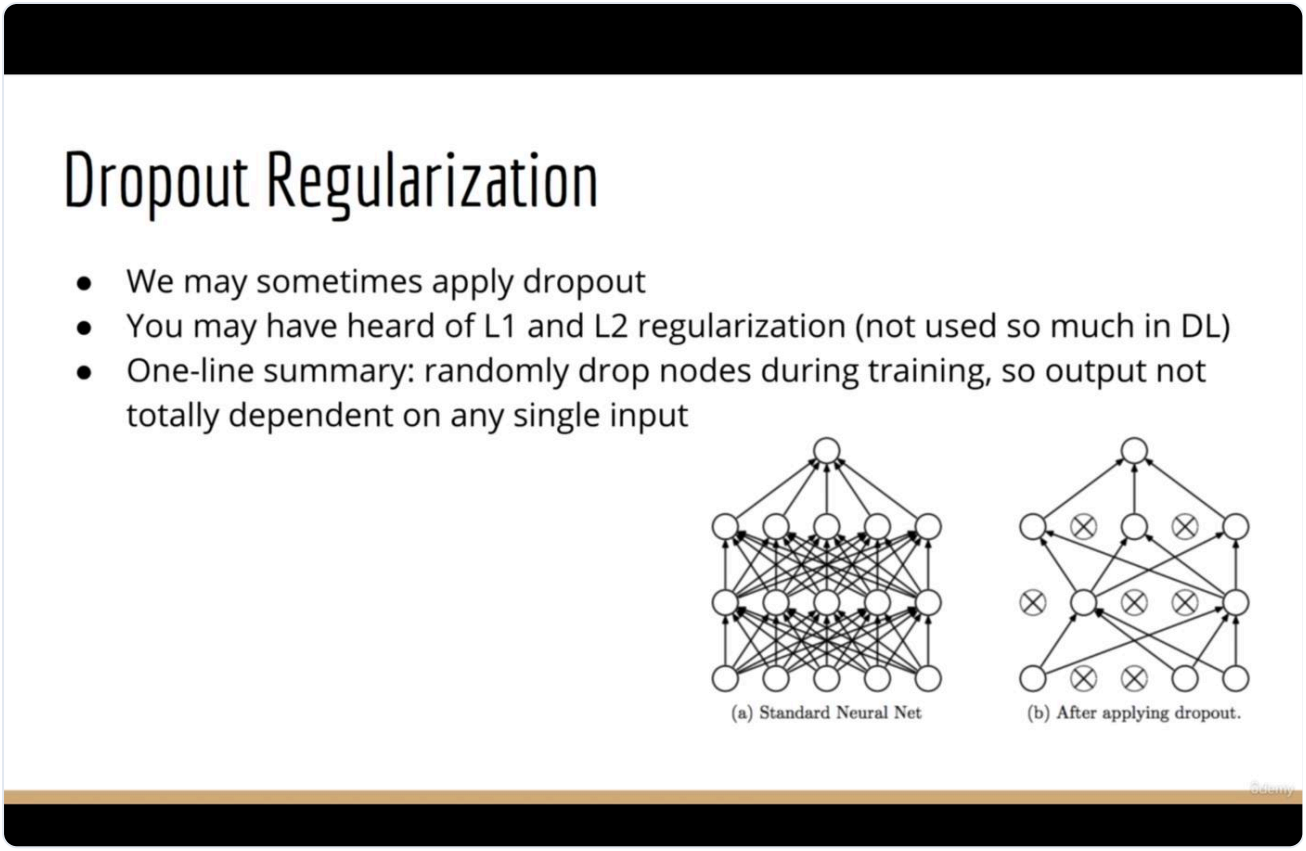


FIGURE 38.20 | We May Sometimes Apply Dropout

If you've studied machine learning before, you may have heard of concepts such as L2 and L1 Regularisation, which aren't really used that often in deep learning anymore. Instead, dropout regularisation is used more commonly. My goal isn't to get in-depth about dropout in this course because I already have a course for that. That describes it in detail. Instead, I'll give you a one line summary, which is that by dropping random nodes in a neural network during training, we make the neural network less dependent on any particular input feature, which helps to spread out the influence of each input. This is similar to what L1 and L2 regularisation try to do.

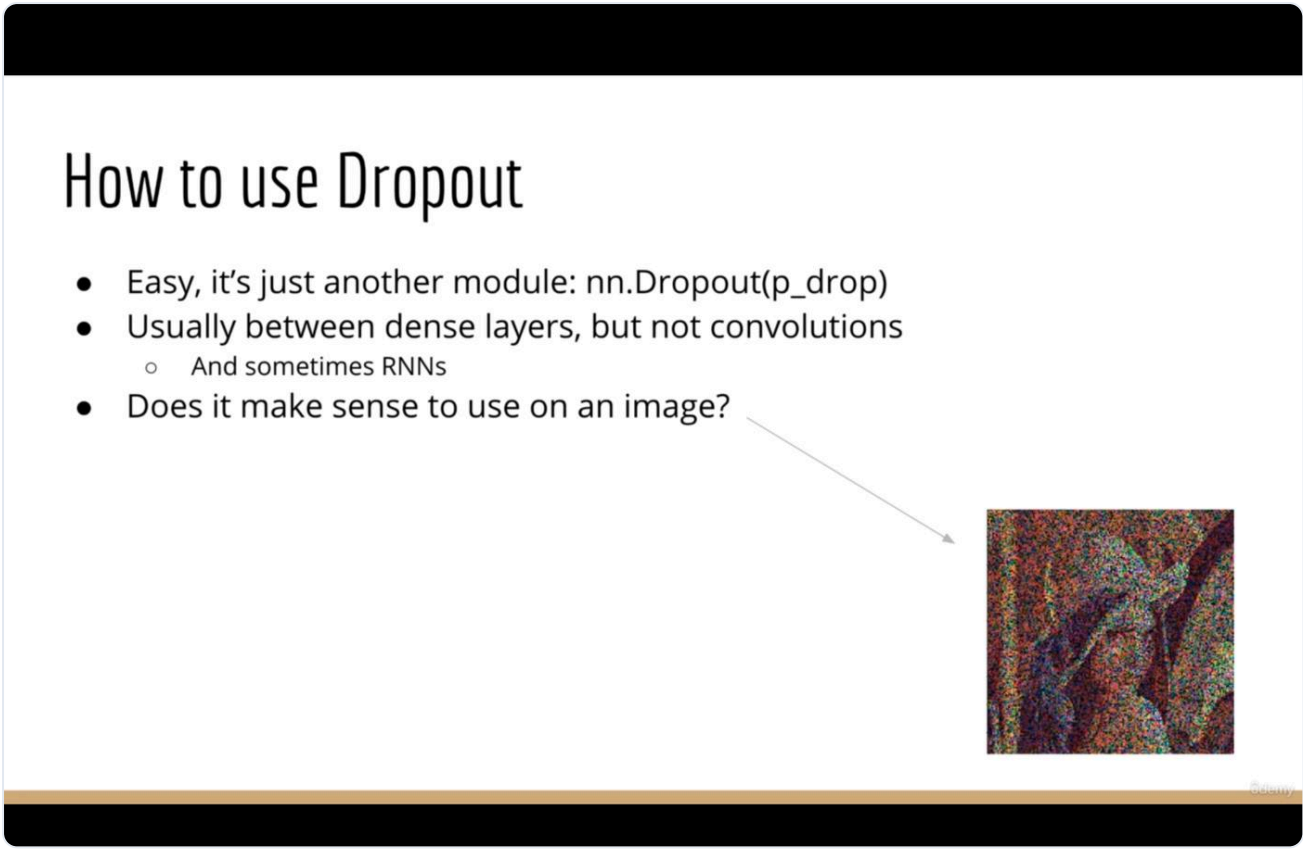


FIGURE 38.21 | How to Use Dropout

In PyTorch, using dropout is as simple as including a dropout layer and specifying the probability of dropping a node during training, usually dropout is used between dense layers, but not with convolutions. Sometimes you'll see them used in RNNs as well, which you'll learn about in another section. One interesting discussion I've seen pop up here and there is whether or not you should use dropout regularisation in the convolutional layers. While the PyTorch API lets you do this, it might not necessarily be a good idea, remember that convolution is pattern finding it's looking for a specific stroke or edge. If you suddenly remove half the pixels from an image, is the stroke still visible?

Can it still be considered the stroke that we are looking for? Personally, I lean more towards no, but I've seen a few people use dropout in their convolutional layers. Even if it doesn't hurt, if it also doesn't help, then there's no point in using it. As always, I recommend experimentation over guessing. So if you're interested in trying it, I would strongly suggest doing so. OK, so one small note about how to use dropout. Now the details are outside the scope of this course, but if you're interested in learning more about how dropout works, please see extra reading in the course repo. One thing you'll just have to accept on faith for now is that it works differently when you are training your neural network compared to when you were using it for predictions.

So because of this PyTorch has two modes for your model train mode and eval mode.

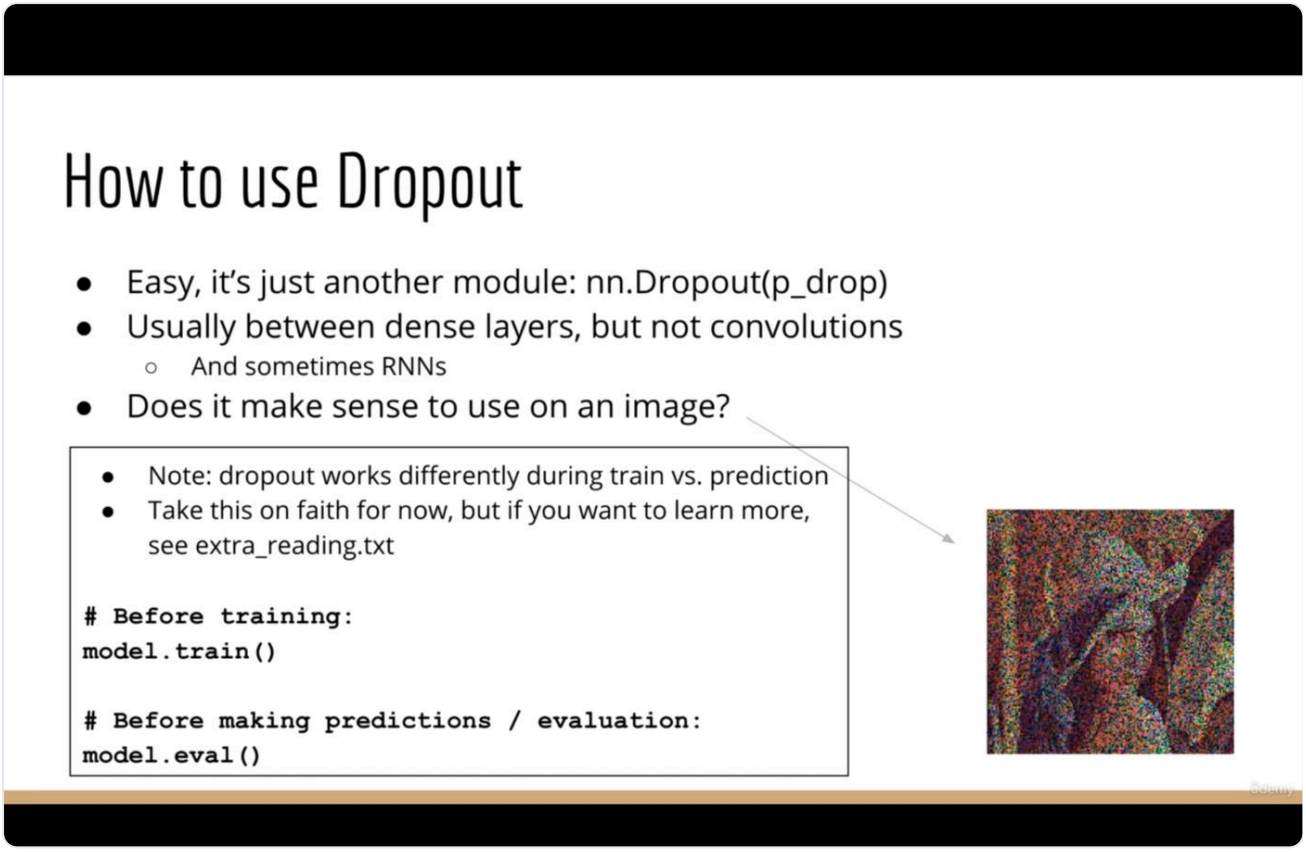


FIGURE 38.22 | How to Use Dropout

Basically, before you do any training, you should make sure your model is in train mode by calling model train.

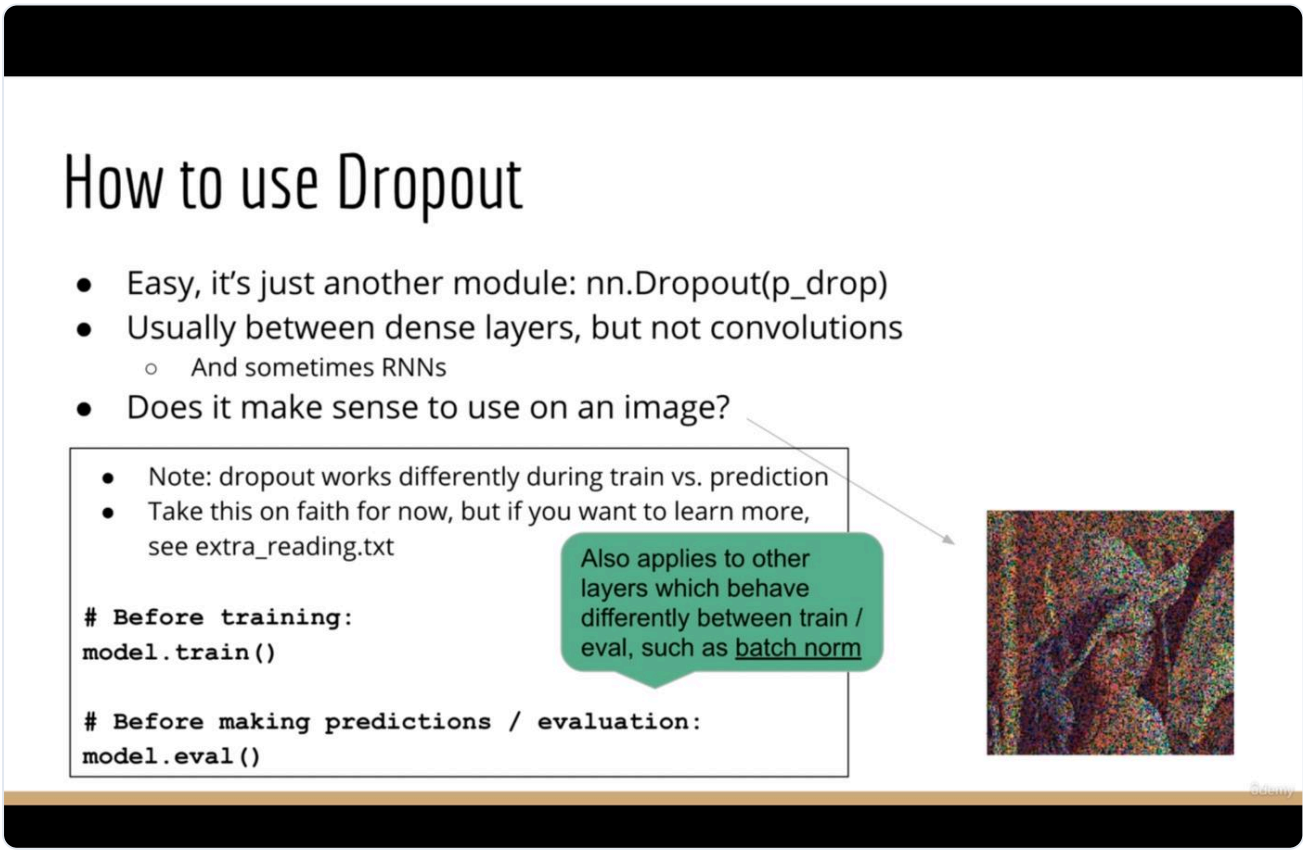


FIGURE 38.23 | How to Use Dropout

Before you do predictions, you can switch to eval mode by calling model eval and note that this also applies to other layers which behave differently during training, such as batch norm, which will be discussed later. As a final note for this lecture, I want to give you some foresight into why learning about how to build custom models is so useful. We went over one major reason previously, which is that you might need to build a model that contains operations which are not PyTorch modules.

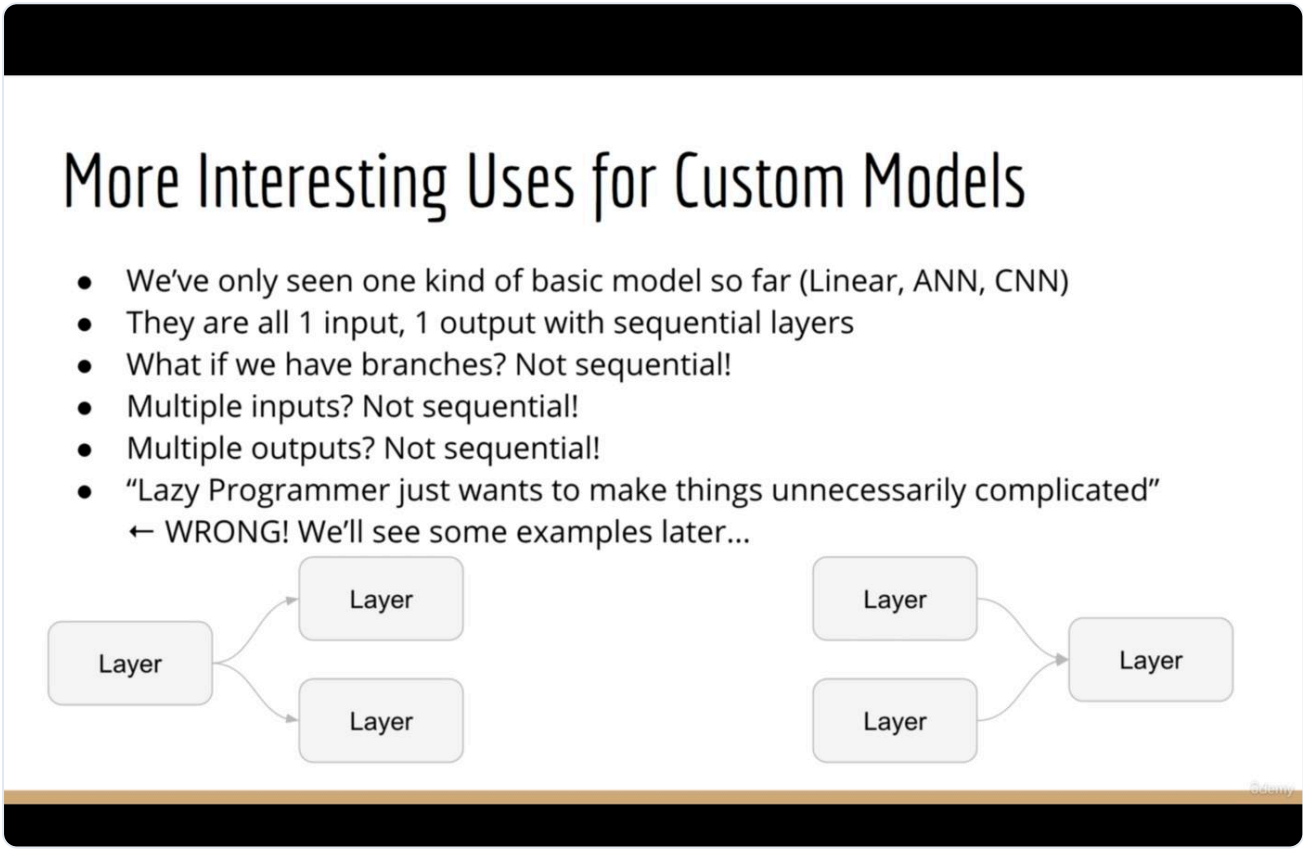


FIGURE 38.24 | More Interesting Uses for Custom Models

But so far we've only seen one basic kind of neural network in this course, one that encompasses linear models ANNs. And CNN's that is these models always have one input and one output, and each layer is stacked on top of the last. Have you ever considered what if I want my model to have branches? This by definition is not sequential. So you cannot use nn.Sequential. What if I want my model to have multiple inputs? Same story, cannot use nn.Sequential because this is not sequential. What if I want my model to have multiple outputs? Same story cannot use nn.Sequential. Now, you might scoff and say, I will never need these types of models, lazy programmers, just showing off that he knows all these complicated things, but in fact, you are wrong.

We will be looking at such models later in this course.